STUDY GUIDE

PYTHON & SOFTWARE ENGINEERING: FROM ZERO TO PROFESSIONAL

A practical, hands-on textbook for a working professional moving into data roles and becoming a data, ML and AI engineer.

Taught the IIT Professor Way — Real Numbers. Real Problems. Real Understanding.

CHAPTER 1 Setup and Thinking in Python

CHAPTER 2 Control Flow, Functions, Scope and Recursion

CHAPTER 3 Strings, Text Processing and Regular Expressions

CHAPTER 4 Collections, Comprehensions and the Standard-Library Toolbox

CHAPTER 5 Files, Paths, JSON, CSV, Dates and Time Zones

CHAPTER 6 Errors, Exceptions, Logging and Debugging

CHAPTER 7 Iterators, Generators, Decorators and Context Managers

CHAPTER 8 Object-Oriented Programming

CHAPTER 9 Modules, Packages, Environments, Type Hints and Validation

CHAPTER 10 Testing and Code Quality

CHAPTER 11 Concurrency and Performance

CHAPTER 12 APIs, the Web, Scraping and Command-Line Tools

CHAPTER 13 Professional Practice: Clean Code, CI/CD, Secrets, Security and Packaging

CHAPTER 14 Capstone Projects and the Python Interview Bank

CHAPTER 15 Project Lab: Four Portfolio Projects, Built and Tested Step by Step

APPENDIX Cheat Sheet

A Note From Your Professor

Dear learner,

You may already be good at spotting patterns, asking careful questions and checking whether a number makes sense. That is a real strength, and Python is the tool that will let you turn that strength into data, machine learning and AI work.

This document teaches Python the way a good engineer uses it: not just syntax, but how to think, test, debug, secure and ship code. Every chapter starts with something that works, shows you the real output (all code was run on Python 3.14, using only the standard library), and then explains why it works, line by line. Where a tool needs the internet or a paid service (pytest, FastAPI, pydantic and similar), the text says "not run here" and shows the standard usage, and you should run it yourself.

I will be honest: this is a large document, and some chapters are long. That is on purpose. You do not need to finish it in a week. You need to understand each part completely.

The one rule: **never move on until the current topic feels obvious.** If a section feels foggy, type the code yourself, change it, break it on purpose, and read the error message. Confusion that you fix today saves you hours in an interview and weeks on the job. Study about one hour a day. Slow and clear beats fast and fuzzy.

What Every Topic Contains

1. An **INTUITION** callout: the problem the idea solves, in everyday language.
2. A **formal definition** or "how it works", in plain words, with every new term defined.
3. **Worked examples** with real, runnable code and the exact output (programming sections) or numbered problems with steps (maths-style sections).
4. **The picture**: an ASCII diagram, table or image, explained completely.
5. A **real-world mapping** table, and the **connection to the real job**.
6. **Applications**, grouped by role.
7. **COMMON MISTAKE** callouts: what beginners get wrong and how to catch it.
8. **Check yourself**: three short questions with answers.

Every chapter ends with a **Chapter recap** (with a small cheat-sheet table), a graded **Practice set** with answers, and a **Prompt pack** of five prompts you can paste into any AI assistant.

Contents

1. **Setup and Thinking in Python.** The REPL, scripts, how Python runs code, basic types, mutability, references and copies.
2. **Control Flow, Functions, Scope and Recursion.** Decisions, loops, `match`, arguments, closures, LEGB, recursion.
3. **Strings, Text Processing and Regular Expressions.** Slicing, methods, f-strings, UTF-8, regex.
4. **Collections, Comprehensions and the Standard-Library Toolbox.** List, tuple, set, dict, `collections`, `itertools`, `functools`, `statistics`.
5. **Files, Paths, JSON, CSV, Dates and Time Zones.** Reading and writing data safely, configuration.
6. **Errors, Exceptions, Logging and Debugging.** Tracebacks, `try/except`, custom exceptions, `logging`, `pdb`.
7. **Iterators, Generators, Decorators and Context Managers.** Lazy data pipelines and reusable wrappers.
8. **Object-Oriented Programming.** Classes, dunder methods, dataclasses, inheritance, SOLID, design patterns.
9. **Modules, Packages, Environments, Type Hints and Validation.** Imports, venv, `pyproject.toml`, mypy, pydantic, notebooks.
10. **Testing and Code Quality.** Test pyramid, pytest ideas, mocks, TDD, ruff, black, pre-commit.
11. **Concurrency and Performance.** GIL, threads, processes, asyncio, profiling, vectorization, Little's law.
12. **APIs, the Web, Scraping and Command-Line Tools.** HTTP, REST, FastAPI basics, auth, responsible scraping, argparse.
13. **Professional Practice.** Clean code, architecture, CI/CD, secrets, SQL injection, hashing, packaging.
14. **Capstone Projects and the Python Interview Bank.** Four projects and 45 graded questions.
15. Project Lab: Four Portfolio Projects, Built and Tested Step by Step

Appendix: **Cheat Sheet** (`99_cheatsheet.md`) with an honest closing note.

How To Study This Document

The learning loop for every topic:

1. **Build first.** Type the code from the section yourself. Do not copy and paste. Predict the output before you run it.
2. **Ask.** When something is unclear, use a prompt from the chapter's Prompt pack and ask an AI to explain it another way.
3. **Just-in-time theory.** Read the explanation after you have seen the code work. It will make more sense.
4. **Rebuild.** Close the document and write the same program from memory, with a small change (a new field, a new rule).
5. **Teach.** Explain the idea aloud, as if to a colleague, in simple words. If you cannot, you do not own it yet.

Suggested pace. About one chapter per week at one hour a day. Chapters 1 to 4 are the foundation; spend extra time there. Do the Practice set at the end of the chapter, then use the Prompt pack for a mock interview. Chapters 5 to 7 build real-world scripting skills. Chapters 8 to 10 make you a software engineer. Chapters 11 to 13 make you employable. Chapter 14 is where you prove it.

Using the prompt packs. Each pack has five prompts: teach me, give me graded exercises, quiz me, review my work, mock interview. Paste one into an AI assistant. When a prompt says `[PASTE HERE]`, paste your own code. Always run the code the AI gives you. Check it against the real output in this document.

Prerequisites

- Basic computer skills: files, folders, a terminal (Command Prompt, Terminal or PowerShell).
- Comfort with school mathematics. You do not need advanced maths for this document.
- No prior programming or data experience is required. Many examples use everyday data such as shop orders, delivery times and logs.

Tools you need

All free.

- **Python 3.10 or newer** (examples were run on 3.14), from python.org.
- A text editor: **VS Code** (free) with its Python extension.
- A terminal and **Git** (see the Linux and Git material in the skill map).
- Later chapters mention `pytest`, `ruff`, `black`, `mypy`, `pydantic`, `httpx` / `requests` and `FastAPI`. Install them with `pip` inside a virtual environment (Chapter 9) when you reach them. Everything runnable in the text uses only the standard library.

CHAPTER 1

SETUP AND THINKING IN PYTHON

Run code, meet the basic types, and understand what a variable really is.

Imagine your manager gives you a 2 GB file of every order an online shop took this year and says, "Tell me the average order value for each city by tomorrow." You cannot do that by hand, and Excel will not even open the file. With Python you can do it in 20 lines. But before those 20 lines make sense, you must understand the ground floor: how you run Python, what a variable really is, and why a change in one place can silently change data in another place. This chapter builds that ground floor. By the end you will run code in two ways, know the seven basic types, predict what a copy does and does not copy, and avoid the bugs that trip up most beginners.

Running example for this chapter. We use one line from an online-shop orders file: `A1042,3,480,true`. It means: order `A1042`, `3` items, amount `480` rupees, paid `true`. Python reads it as plain text, and turning that text into useful numbers is our first real job.

All code in this document was run with Python 3.14 on a Mac. If your Python is older, small details (like the bytecode listing in Section 1.1) may look different. Everything else works on any Python 3.10 or newer.

How to use the code blocks. For each example, first cover the output with your hand and **predict** what will print. Then compare. Then read the explanation. Being wrong is useful: it shows exactly where your mental model needs repair.

1.1 Running Python: REPL, Scripts and How Python Runs Your Code

INTUITION

In a kitchen you can taste one spoonful of sauce at once (that is a quick experiment), or you can write the full recipe down and cook it again later (that is a repeatable run). Python gives you the same two modes. The **REPL** is the quick taste. A **script** is the written recipe.

Formal definition

- **Python** is a programming language. The program that reads your code and runs it is called the **interpreter**. On your machine the command is usually `python3`.
- **REPL** means Read-Eval-Print Loop. You type one line, Python reads it, evaluates it, prints the result, and waits for the next line. Start it by typing `python3` in a terminal. Leave it with `exit()` or Ctrl+D.
- A **script** is a text file ending in `.py`. You run it with `python3 file.py`.
- A **module** is any `.py` file that other code imports. A script and a module are the same kind of file. The only difference is how you use it.
- One-line mode: `python3 -c "print('hi')"` runs a string of code.

How Python runs code

Python does not turn your code directly into machine instructions the way C does. It works in three steps:

1. **Compile** your source text into **bytecode**. Bytecode is a list of small, simple instructions for a "virtual machine".
2. **Run** the bytecode on the **Python Virtual Machine (PVM)**, a loop that executes one instruction at a time.
3. **Cache** the bytecode in a `.pyc` file inside the `__pycache__` folder, so imports are faster next time.

```
your code      compile         bytecode        PVM executes
add.py  ----->  (automatic) --> LOAD, ADD, ... --> result
                     |
                     +--> __pycache__/add.cpython-314.pyc  (cache)
```

The standard Python from python.org is called **CPython** because it is written in C. There are others (PyPy, for example), but when people say "Python" they nearly always mean CPython.

Worked example 1.1.1: use the REPL

Predict: Before you look, what do you think each line prints? Which lines print nothing?

I piped three lines into the REPL to show you what a session looks like. The `>>>` is the prompt.

```
printf '2+3\nx=10\nx*x\n"ab"*3\nimport math\nmath.sqrt(16)\n' | python3 -q -i
```

```
>>> 5
>>> >>> 100
>>> 'abababab'
>>> >>> 4.0
>>>
```

The lines `x=10` and `import math` print nothing, because they are statements, not expressions. An **expression** has a value (`2+3` is `5`). A **statement** does something (`x=10` stores a value). The REPL prints the value of every expression automatically. A script does not. In a script you must call `print()` .

Worked example 1.1.2: a script and its bytecode

Save this as `a.py` :

```
import sys, dis
def add(a, b):
    return a + b
dis.dis(add)
print(sys.version_info[:2] >= (3, 10))
print(__name__)
```

Run `python3 a.py` . Real output:

```
  2           RESUME                   0

  3           LOAD_FAST_BORROW_LOAD_FAST_BORROW 1 (a, b)
              BINARY_OP                0 (+)
              RETURN_VALUE
True
__main__
```

Read it like this:

- `dis.dis` shows the bytecode of the function `add` . Load `a` and `b` , add them, return the result. Exact names of instructions change between Python versions. That is normal.
- `__name__` is a built-in variable. When you run a file directly, its value is `"__main__"` . When another file imports it, the value is the module's name. We use this in Chapter 9 to write `if __name__ == "__main__":` .

Real-world mapping

Idea	Example	Where it appears in the job
REPL	`python3` , then `len(open("r.csv").readlines())`	Checking a file before you write a pipeline
Script	`python3 summarise.py results.csv`	A nightly report job
Module	`import utils`	Shared helper code in a team repo
Bytecode cache	`__pycache__/`	A folder you add to `.gitignore`

The picture

```
REPL                          Script
____                          ______
type -> result now            write file -> run whole file
good for: trying ideas,       good for: anything you want to
checking a function           repeat, share, test, schedule
state lost when you exit      state lost when script ends
```

Connection to the real job

- Data analysts and scientists live in the REPL and notebooks to explore data. Engineers save the final logic as scripts and modules that run on a schedule (for example, an Airflow task or a cron job).
- `python3 -c` is handy in shell pipelines and CI jobs for tiny checks.
- Knowing about `.pyc` and `__pycache__` stops you from committing them by mistake. They belong in `.gitignore`.

Applications

- Quick data checks in a terminal.
- Automation scripts (rename files, call APIs, parse logs).
- Entry points of services and batch jobs.
- Learning a new library by trying it in the REPL.

COMMON MISTAKE

Naming your own file after a standard library module, for example `json.py`, `random.py` or `email.py`. When you write `import json`, Python finds *your* file first and your program breaks in strange ways. Never use a standard library name for your own files.

PROFESSOR'S NOTE

Use the REPL for 5 minutes before writing any new function. Type small inputs and look at the output. This habit alone makes you faster than people who write 40 lines and then debug.

Check yourself

1. What is the difference between an expression and a statement?

Answer: An expression has a value (`2+3`). A statement performs an action (`x = 10`). The REPL auto-prints expression values.

1. What does `__name__` equal when you run `python3 a.py`?

Answer: The string `"__main__"`.

1. Why does Python create `__pycache__` folders?

Answer: To store compiled bytecode (`.pyc` files) so later imports start faster.

1.2 Variables and the Basic Types

INTUITION

A cardboard box with a label is a good picture for other languages. For Python, a better picture is a **sticky note**. The value lives somewhere in memory. The variable name is a sticky note that you stick on it. You can move the note to another value any time, and you can put two notes on the same value.

Formal definition

A **variable** is a name bound to an object. Assignment `x = 10` means: create the object `10` (or reuse it), then bind the name `x` to it. Every object has a **type**. Python is **dynamically typed**: a name has no fixed type, but each object does. Python is also **strongly typed**: it will not silently mix `"5" + 5`.

The basic types:

Type	Meaning	Example
`int`	whole number, any size	`42`, `10**30`
`float`	decimal number (64-bit, IEEE 754)	`3.14`
`complex`	real + imaginary part	`2+3j`
`bool`	`True` or `False` (a subtype of `int`)	`True`
`str`	text (Unicode characters)	`"hello"`
`bytes`	raw bytes (numbers 0 to 255)	`b"hi"`
`NoneType`	the single value `None`, meaning "nothing here"	`None`

Worked example 1.2.1: numbers

Predict: Will `0.1 + 0.2 == 0.3` print `True`? What is `-7 // 2`?

```
print(type(42), type(3.14), type(2+3j), type(True), type("hi"), type(b"hi"), type(None))
print(7/2, 7//2, 7%2, -7//2, -7%2, 2**10, divmod(17,5))
print(0.1+0.2, 0.1+0.2==0.3)
import math
print(math.isclose(0.1+0.2,0.3))
from decimal import Decimal
print(Decimal("0.1")+Decimal("0.2"))
print(10**30, (2+3j)*(1-1j), (2+3j).real, (2+3j).imag, abs(3+4j))
```

Real output:

```
<class 'int'> <class 'float'> <class 'complex'> <class 'bool'> <class 'str'> <class 'bytes'> <class 'NoneType'>
3.5 3 1 -4 1 1024 (3, 2)
0.30000000000000004 False
True
0.3
1000000000000000000000000000000 (5+1j) 2.0 3.0 5.0
```

What to learn from this:

- `/` always gives a float. `//` is floor division (rounds *down*, so `-7//2` is `-4`, not `-3`). `%` is the remainder and always has the sign of the divisor, so `-7 % 2` is `1`.
- Integers have no size limit. `10**30` is exact.
- Floats are stored in binary. `0.1` has no exact binary form, just like `1/3` has no exact decimal form. So `0.1 + 0.2` is `0.30000000000000004`. Never compare floats with `==`. Use `math.isclose`. For money, use `decimal.Decimal` (made from strings) or integer paise.
- `(2+3j)*(1-1j) = 2 - 2j + 3j - 3j² = 2 + j + 3 = 5+1j`. Check by hand: it matches.

Worked example 1.2.2: truthiness, bool and conversion

```
print(True+True, isinstance(True,int), bool(0), bool(""), bool([]), bool("0"), bool(None))
print(int("42")+1, float("2.5"), str(99)+"!", int(3.9), round(2.5), round(3.5), round(2.675,2))
```

Real output:

```
2 True False False False True False
43 2.5 99! 3 2 4 2.67
```

- `True + True` is `2` because `bool` is a kind of `int`.
- **Truthiness**: in an `if`, these count as false: `0`, `0.0`, `""`, `[]`, `{}`, `()`, `set()`, `None`, `False`. Everything else is true. The string `"0"` is true because it is not empty.
- `int(3.9)` cuts off the decimals (gives `3`). It does not round.
- `round(2.5)` gives `2` and `round(3.5)` gives `4`. Python rounds exact halves to the nearest *even* number ("banker's rounding"). And `round(2.675, 2)` gives `2.67`, not `2.68`, because the float `2.675` is really a tiny bit less than 2.675.

Worked example 1.2.3: text, bytes and UTF-8

```
s = "héllo ₹"
print(len(s), len(s.encode("utf-8")), s.encode("utf-8"))
print(b"abc"[0], bytes([72,105]), b"Hi".decode())
```

Real output:

```
7 10 b'h\xc3\xa9llo \xe2\x82\xb9'
97 b'Hi' Hi
```

- The string has 7 **characters** but 10 **bytes** in UTF-8. `é` takes 2 bytes. `₹` takes 3 bytes. The rest take 1 each: 5 + 2 + 3 = 10.
- A `str` is text. `bytes` is raw data. `.encode()` turns text into bytes. `.decode()` turns bytes into text. You must name the encoding; always use `"utf-8"`.
- Indexing `bytes` gives an integer (`97` is the code of `a`).

Real-world mapping

Type	Example in our running row `A1042,3,480,true`	Where it appears in the job
`str`	`"A1042"`, and also `"480"` before conversion	Raw CSV and log fields
`int`	`3`, `480` after `int("480")`	Item counts, amounts, minutes
`float`	`411.25` (an average)	Means, percentages, model scores
`bool`	`ok == "true"` gives `True`	Flags such as `is_error`
`bytes`	`b"\xff"`	Files, network data, hashes
`None`	a missing delivery date	Missing values and "no result"

Worked example 1.2.4: our running row, with the typical mistakes

Predict: `amount` came from `split`. What is its type? What does `amount + amount` give?

```
row = "A1042,3,480,true"
order_id, items, amount, paid = row.split(",")
print(type(amount), amount + amount)
amount_rs = int(amount)
print(amount_rs + 20, paid == "true")
try:
    print("amount: " + amount_rs)
except TypeError as e:
    print("TypeError:", e)
print("amount: " + str(amount_rs))
try:
    print(undefined_name)
except NameError as e:
    print("NameError:", e)
try:
    b"\xff".decode("utf-8")
except UnicodeDecodeError as e:
    print("UnicodeDecodeError:", e)
```

Real output:

```
<class 'str'> 480480
500 True
TypeError: can only concatenate str (not "int") to str
amount: 480
NameError: name 'undefined_name' is not defined
UnicodeDecodeError: 'utf-8' codec can't decode byte 0xff in position 0: invalid start byte
```

Line by line:

- `split(",")` cuts the text at each comma and returns a list of four strings. We unpack them into four names. Note that `amount` is the *text* `"480"`, so `+` joins text and gives `480480`. This is the single most common beginner surprise with CSV data.
- `int(amount)` makes the number `480`. Now `+ 20` is arithmetic: `500`. Python has no `true` keyword, so we compare the text with `"true"` to get a real `bool`.
- Mixing `str` and `int` with `+` is an error: Python is strongly typed and refuses to guess. Fix: convert with `str(...)` (or use an f-string, Chapter 3).
- A `NameError` means you used a name that was never assigned. Read the name in the message: usually a typo.
- A `UnicodeDecodeError` means the bytes are not valid in the encoding you asked for. Byte `0xff` can never start a UTF-8 character.

The picture

```
name         object (in memory)
----         ------------------
x   ------>  [ int: 10 ]
y   ------>  [ str: "hi" ]
z   ------>  [ None ]

x = "now text"   moves the sticky note x to a new object.
The old int 10 is untouched (and freed if nothing else points to it).
```

Connection to the real job

- Float rounding explains why two data pipelines can give sums that differ in the 12th decimal. In tests, use `math.isclose` or `pytest.approx`.
- `None` is how Python represents missing values. Pandas uses `NaN` for the same idea in numeric columns (see the Data Analyst document).
- UTF-8 bugs (`UnicodeDecodeError`) are one of the most common failures when reading CSV files from different systems.
- Amounts and dates read from a CSV file of orders arrive as strings. You convert them with `int()` and `float()` before computing totals and averages.

Applications

- Money and billing (`Decimal`).
- Text cleaning and NLP (`str` , encoding).
- Binary files, network packets, hashes (`bytes`).
- Optional values in APIs (`None`).
- Signal processing and physics (`complex`).

COMMON MISTAKE

Using `==` to compare floats, and using `float` for money. Another very common one: reading a file without saying `encoding="utf-8"` , which works on your laptop and fails on the server.

Check yourself

1. What is `-9 // 4` and `-9 % 4` ?

Answer: `-3` and `3` . Floor of -2.25 is -3, and `-9 - (4 * -3) = 3` .

1. Why is `bool("False")` equal to `True` ?

Answer: It is a non-empty string, and every non-empty string is truthy.

1. How many bytes is `"₹"` in UTF-8?

Answer: 3 bytes.

1.3 Mutability, Identity, References and Memory

INTUITION

Two friends share one Google Doc. If one of them edits it, the other sees the change, because there is only one document. But if you each have your own printed copy, edits do not travel. A **mutable** object is the shared Google Doc. An **immutable** object is a printed page: you cannot edit it, you can only print a new one.

Formal definition

- **Mutable** objects can change after creation: `list` , `dict` , `set` , `bytearray` , and most classes you write.
- **Immutable** objects can never change: `int` , `float` , `complex` , `bool` , `str` , `bytes` , `tuple` , `frozenset` , `None` . Doing `n += 1` on an int creates a new int and moves the name.
- **Identity** is "same object in memory". `a is b` tests identity. `id(a)` gives a number for that identity.
- **Equality** is "same value". `a == b` tests equality (it calls the object's `__eq__`).
- Use `is` only for singletons: `x is None` , `x is True` . Use `==` for everything else.
- **Reference counting**: CPython counts how many names point to each object. When the count hits zero, memory is freed at once. A **garbage collector** (module `gc`) also finds and frees groups of objects that point to each other (cycles) when no outside name points to them.
- **Shallow copy** (`copy.copy` , `list[:]` , `list.copy()`) copies the outer container but shares the inner objects. **Deep copy** (`copy.deepcopy`) copies everything inside, recursively.

Worked example 1.3.1: aliasing

Predict: After `b.append(4)` , does `a` change?

```
import copy
a=[1,2,3]; b=a; c=a[:]
b.append(4)
print(a,b,c, a is b, a is c, a==c, id(a)==id(b))
```

Real output:

```
[1, 2, 3, 4] [1, 2, 3, 4] [1, 2, 3] True False False True
```

- `b = a` does not copy. It puts a second note on the same list. So `b.append(4)` changes what `a` sees.
- `c = a[:]` is a new list made earlier, so it still holds `[1, 2, 3]`. After the append, `a == c` is `False` because the values differ now, and `a is c` is `False` because they are different objects.

Worked example 1.3.2: shallow vs deep copy, and tuples with lists inside

Predict: A tuple is immutable. Can its inner list still grow?

```
import copy
nested=[[1,2],[3,4]]
sh=copy.copy(nested); dp=copy.deepcopy(nested)
nested[0].append(99)
print(sh, dp)
t=(1,[2,3]); t[1].append(9); print(t)
try:
    t[0]=5
except TypeError as e: print("TypeError:",e)
```

Real output:

```
[[1, 2, 99], [3, 4]] [[1, 2], [3, 4]]
(1, [2, 3, 9])
TypeError: 'tuple' object does not support item assignment
```

- The shallow copy `sh` has its own outer list but shares the inner lists, so it saw the `99`. The deep copy `dp` did not.
- A tuple is immutable, but it only promises that *which objects it holds* never changes. If one of those objects is a list, the list can still change. The tuple became `(1, [2, 3, 9])`.

Worked example 1.3.3: passing to functions

```
def f(lst): lst.append("x")
def g(n): n+=1; return n
L=[]; f(L); print(L); n=1; g(n); print(n)
```

Real output:

```
['x']
1
```

Python passes **the reference** (call by object reference, sometimes called "call by sharing"). Function `f` changes the same list the caller owns, so the caller sees `['x']`. Function `g` cannot change the caller's `n`, because `n += 1` makes a new int and re-points only the local name.

Worked example 1.3.4: the grid bug, memory size, and garbage collection

Predict: In `[[0]*3]*2`, if you set the first cell of row 0 to 1, how many rows change?

```
import sys, gc
grid=[[0]*3]*2; grid[0][0]=1; print(grid)
grid=[[0]*3 for _ in range(2)]; grid[0][0]=1; print(grid)
print(sys.getsizeof(0), sys.getsizeof([]), sys.getsizeof([1,2,3]), sys.getsizeof("a"), sys.getsizeof({}))
class Node:
    def __init__(s): s.other=None
p=Node(); q=Node(); p.other=q; q.other=p
del p,q
print("collected:", gc.collect()>0)
import weakref
class K: pass
k=K(); r=weakref.ref(k); print(r() is k); del k; print(r())
```

Real output:

```
[[1, 0, 0], [1, 0, 0]]
[[1, 0, 0], [0, 0, 0]]
28 56 88 42 64
collected: True
True
None
```

- `[[0]*3]*2` repeats the *same* inner list twice. Changing one row changes both. The list comprehension builds two separate rows. This is a classic bug.
- `sys.getsizeof` shows the size of the object itself in bytes, **not** the objects it points to. An empty list is 56 bytes here and an `int` is 28 bytes. These numbers depend on Python version and platform. For a full size of a nested structure you have to add up inner objects yourself, or use `tracemalloc` (Chapter 11).
- Objects `p` and `q` point to each other. After `del p, q` reference counts do not reach zero, so only the cycle collector can free them. `gc.collect()` returned a positive count, meaning it found and freed objects.
- A **weak reference** (`weakref.ref`) does not keep an object alive. After `del k` the object is gone and `r()` returns `None`.
- `__slots__` is a way to reduce memory per instance. We cover it in Chapter 8.

Worked example 1.3.5: when `is` surprises you

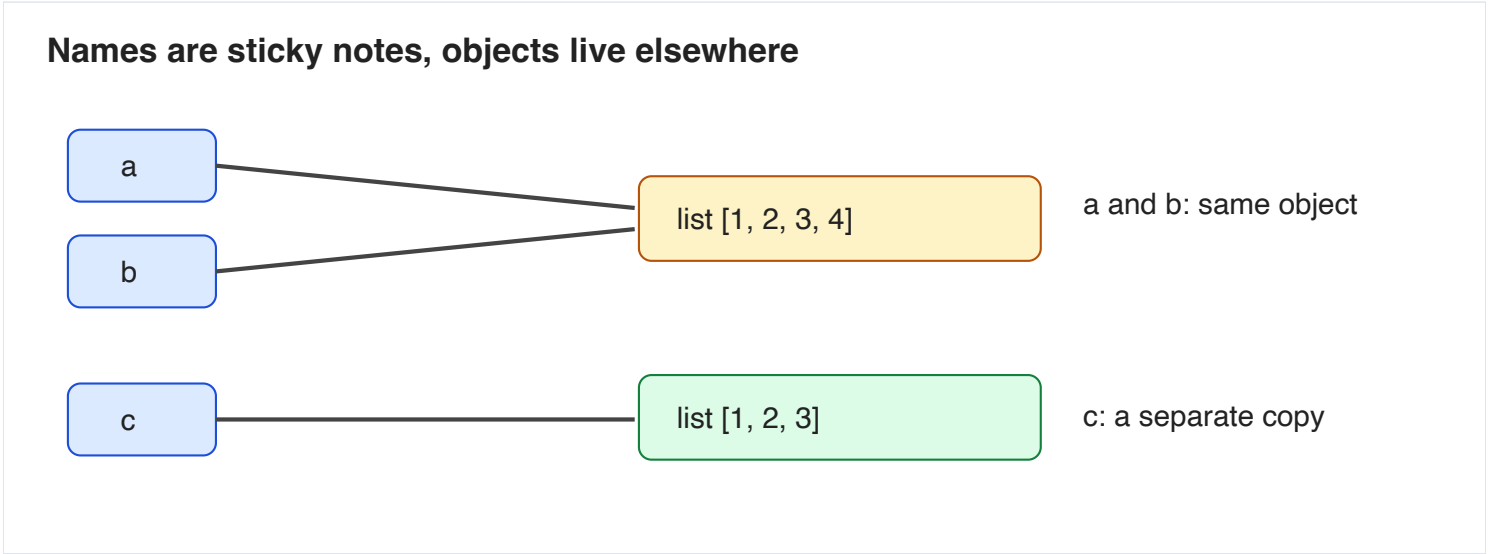
```
a = 1000; b = 1000
print(a == b, a is b)
x = [1,2]; y = [1,2]
print(x == y, x is y)
v = None
print(v is None, v == None)
```

Real output:

```
True True
True False
True True
```

`a is b` printed `True` here only because Python reused one constant object when compiling this small file. Inside a function call or from user input, two equal big ints are often different objects. **The result of `is` on numbers and strings is an implementation detail. Never rely on it.** Lists made separately are never the same object.

The picture



Two names on one list, a third name on a copy

Read the diagram left to right. The blue boxes are names. The arrows go from a name to the object it points at. Names `a` and `b` have arrows to the same box, so a change through either name shows up through the other. Name `c` points to a different box that holds a copy. The text version below shows the same idea and the grid bug.

```
a = [1,2,3]
b = a            a ----+
                       +--> [1,2,3]
                 b ----+

c = a[:]         c ------> [1,2,3]   (new object)

grid = [[0]*3]*2
grid --> [ row , row ]       both slots point to ONE row
              \   /
               [0,0,0]
```

Connection to the real job

- The most common "ghost bug" in data code is a function that changes the list or DataFrame it was given. The caller's data changes and nobody knows why. Copy inputs at the start (`df = df.copy()`) or do not change them at all.
- Understanding references explains `SettingWithCopyWarning` in pandas.
- Memory awareness matters when loading millions of rows. A list of Python ints costs far more than the same numbers in a NumPy array.
- Long-running programs: if a script that processes millions of orders slowly eats memory, reference cycles and global lists are the first suspects.

Applications

- Safe function design (do not change arguments).
- Caches and shared state (dict as a cache).
- Deep copies of configuration before modifying for an experiment.
- Memory tuning for large data jobs.

COMMON MISTAKE

Thinking `b = a` makes a copy. Also: using `is` instead of `==` to compare values. A good rule: if you write `is` and the right side is not `None` , `True` or `False` , stop and ask why.

KEY INSIGHT	In Python, assignment never copies data. It only attaches a name to an object. Copies happen only when you ask (`.copy()` , slicing, `deepcopy`) or when an operation builds a new object (like `+` on strings).

Check yourself

1. After `a = [1,2]; b = a; b += [3]` , what is `a` ?

Answer: `[1, 2, 3]` . `+=` on a list changes the list in place, and `a` and `b` are the same object.

1. Is `t = (1, 2)` followed by `t[0] = 9` allowed?

Answer: No. Tuples are immutable, so you get a `TypeError` .

1. What is the difference between `copy.copy` and `copy.deepcopy` ?

Answer: `copy` copies only the outer container and shares inner objects. `deepcopy` copies inner objects recursively.

1.4 Unpacking, Naming Rules and First Good Habits

INTUITION	Good names are the cheapest documentation. A teammate (or you, six months later) should read `average_order_value` and know what it means without a comment. Unpacking is like handing out four parcels to four people in one move, instead of one by one.

Formal definition

- Names use letters, digits and `_` , and cannot start with a digit. Names are case sensitive. You cannot use keywords (`if` , `class` , `for` , ...).
- Style (**PEP 8**, the official Python style guide; PEP means Python Enhancement Proposal): `snake_case` for variables and functions, `PascalCase` for classes, `UPPER_CASE` for constants.
- **Unpacking** assigns several names at once: `a, b = 1, 2` . A starred name collects the rest: `first, *rest = items` .
- **Swap** with `a, b = b, a` , no temporary variable needed.
- You can write big numbers with underscores: `1_250_000` . Python ignores them.

Worked example 1.4.1: unpacking

Predict: What do `a` and `b` hold after the swap? What is `rest` ?

```
a, b = 1, 2
a, b = b, a
print(a, b)
amounts = [480, 360, 510, 295]
first, *rest = amounts
print(first, rest, sum(amounts)/len(amounts))
n = 0
n += 1; print(n)
a = [1,2]; b = a; b += [3]; print(a)
total_sales = 1_250_000
print(total_sales, f"{total_sales:,}")
```

Real output:

```
2 1
480 [360, 510, 295] 411.25
1
[1, 2, 3]
1250000 1,250,000
```

- The right side `b, a` is built first as the pair `(2, 1)`, and only then assigned. That is why no temporary variable is needed.
- `first` gets the first amount. `*rest` collects everything else into a new list.
- `n += 1` on an int makes a new int. `b += [3]` on a list changes the list in place, so `a` changes too. This is the one place where `+=` behaves differently for mutable and immutable types.
- `f"{total_sales:,}"` is an f-string with a thousands separator. We study f-strings in Chapter 3.

Problem 1.4.1 Using the four order amounts above (480, 360, 510, 295 rupees), find the mean and how much cheaper the cheapest order is than the dearest, in percent.

Formula	mean = sum / count. saving % = (dearest − cheapest) / dearest × 100.
Plug in	sum = 480 + 360 + 510 + 295. count = 4. dearest = 510. cheapest = 295.
Compute	sum = 1645, so mean = 1645 / 4 = 411.25. saving = (510 − 295) / 510 × 100 = 215 / 510 × 100 = 42.157...
Answer	The mean is 411.25 rupees. The cheapest order is about 42.2% cheaper than the dearest.
Check	In code, `sum(amounts)/len(amounts)` printed `411.25` above. And `510 * (1 - 0.42157)` is about `294.999`, which rounds to 295, so the percentage is right.
Why it matters	In real work you compute many statistics like this from a list of numbers. The mean alone hides the spread, which is why later chapters and the Statistics document use percentiles.

Real-world mapping

Habit	Bad	Good
Name a number	`x = 480`	`amount_rs = 480`
Name a constant	`limit = 2000`	`FREE_DELIVERY_LIMIT = 2000`
Unpack a record	`r[0]`, `r[2]`	`order_id, items, amount, paid = row.split(",")`

The picture

```
order_id, items, amount, paid = "A1042,3,480,true".split(",")

 "A1042,3,480,true" --split(",")--> ["A1042", "3", "480", "true"]
                                      |       |     |       |
                                  order_id  items amount  paid
```

The split produces one list. Unpacking hands one list item to each name, in order. If the counts do not match you get `ValueError: too many values to unpack` or `not enough values to unpack`. That error is useful: it tells you a data row has the wrong number of columns.

Connection to the real job

Every log parser, CSV reader and API client starts by splitting a line and naming the parts. Clear names let another engineer review your pipeline in minutes. Code reviews at most companies reject names like `x`, `tmp2` and `data1`.

Applications

- **Data Analytics:** unpack columns from a row; name measures clearly in a report script.
- **Data Engineering:** parse log lines into named fields before loading them.
- **Machine Learning:** `X_train, X_test, y_train, y_test = split(...)` is unpacking in nearly every ML tutorial.
- **Retail operations:** `order_id, items, amount, paid = ...` for each line of an orders file.

COMMON MISTAKE	Using a built-in name as your own variable, such as `list = [1, 2]`, `str = "x"` or `max = 5`. Later `list(...)` stops working with `TypeError: 'list' object is not callable`. Use `items`, `text`, `max_value` instead.

Check yourself

1. What does `first, *rest = [7]` give for `rest`?

Answer: `[]`, an empty list.

1. Is `2nd_try = 5` a legal name?

Answer: No. A name cannot start with a digit.

1. What style does PEP 8 recommend for a function name?

Answer: `snake_case`, such as `parse_row`.

1.5 Which Python? Identity Traps and Measuring Memory

Three questions come up as soon as you leave toy scripts. Which Python version does this feature need? Why does `is` sometimes say `True` for equal numbers and sometimes `False`? How big is this object really? This section answers all three with code you can run.

Minimum versions in one table

Every chapter of this book says the code was run on Python 3.14. Most of it works on older versions, but some features do not. Use this table when a teammate says "it fails on my machine". Python 3.9 reached end of life in October 2025 and 3.10 reaches it in October 2026, so for new projects choose 3.12 or newer.

Feature	First version	Chapter
Walrus operator `:=`	3.8	2
`match` statement, `X | Y` in type hints, `dataclass(slots=True)`	3.10	2, 8, 9
`ExceptionGroup` and `except*`, `tomllib`, `asyncio.TaskGroup`, `asyncio.timeout`, `enum.StrEnum`, `typing.Self`	3.11	5, 6, 8, 11
`def f[T](...)` generics, `itertools.batched`, `typing.override`, quotes reused inside f-strings	3.12	3, 4, 9
`datetime.utcnow()` deprecated (use `datetime.now(timezone.utc)`)	3.12	5
Optional free-threaded build (experimental in 3.13, officially supported in 3.14)	3.13	11

Identity versus equality, three cases

Recall from 1.3 that `==` asks "do these have the same value?" and `is` asks "are these the very same object?". CPython (the standard implementation) keeps a cache of the small integers from -5 to 256, so two names for such a number point at one object. Larger integers built at run time are new objects each time. Strings can be **interned**, which means the interpreter keeps one shared copy of the text.

Problem 1.5.1 Predict `is` and `==` for a small int, a big int built at run time, and a string built by joining pieces.

Case 1	`int("256")` twice: 256 is inside the small-int cache, so both names point to one object. `is` gives `True`.
Case 2	`int("1000")` twice: 1000 is outside the cache, so two objects. `==` is `True`, `is` is `False`.
Case 3	`"".join(["he","llo"])` builds a new string at run time. It equals `"hello"` but is a different object. `sys.intern` returns the one shared copy for both.

```
import sys
a = int("256"); b = int("256")
print(a is b)
x = int("1000"); y = int("1000")
print(x == y, x is y)
u = "hello"
t = "".join(["he", "llo"])
print(t == u, t is u, sys.intern(t) is sys.intern(u))
```

```
True
True False
True False True
```

COMMON MISTAKE

Writing `if x is 1000:` or `if name is "admin":`. It may work in a quick test and fail in production, because it depends on caching that the language does not promise. Python 3.8 and later print a `SyntaxWarning` for this. Use `==` for values. Use `is` only for `None`, `True` and `False`.

How big is it really?

`sys.getsizeof(obj)` returns the size in bytes of the object itself, **not** of the things it points to. A list of one tiny string and a list of one 100,000-character string have the same `getsizeof`, because each list holds only one pointer. To measure everything an object keeps alive, walk its contents and count each object once. Exact byte counts depend on your Python version and platform, so treat the numbers below as "what this machine printed".

Problem 1.5.2 Show that `getsizeof` is shallow, then write `deep_size` that follows dicts, lists, tuples and sets and counts shared objects once.

Idea	Keep a `seen` set of `id()` values so an object reachable twice is counted once.
Step 1	Add `getsizeof(obj)` for the object itself.
Step 2	If it is a container, add the deep size of every item (and every key and value for a dict).
Check	`[shared, shared]` must be smaller than two copies of `[shared]`, because the inner list is counted once.

```
import sys
small = ["a"]
big = ["a" * 100_000]
print(sys.getsizeof(small) == sys.getsizeof(big))
print(sys.getsizeof("a" * 100_000))
def deep_size(obj, seen=None):
    seen = set() if seen is None else seen
    if id(obj) in seen:
        return 0
    seen.add(id(obj))
    total = sys.getsizeof(obj)
    if isinstance(obj, dict):
        total += sum(deep_size(k, seen) + deep_size(v, seen) for k, v in obj.items())
    elif isinstance(obj, (list, tuple, set)):
        total += sum(deep_size(i, seen) for i in obj)
    return total
row = {"id": 1, "tags": ["a", "b"]}
print(sys.getsizeof(row), deep_size(row))
shared = ["x" * 1000]
print(deep_size([shared, shared]) < 2 * deep_size([shared]))
print(sys.getrefcount(shared) >= 2)
```

```
True
100041
184 456
True
True
```

The dictionary alone is 184 bytes here, but it keeps 456 bytes alive once its keys, the nested list and the strings are counted. The last line shows `sys.getrefcount`, which reports how many references point at an object. CPython frees an object the moment that count reaches zero. Chapter 11 (section 11.7) continues with garbage collection.

Going deeper

- `id(obj)` is the memory address of the object in CPython, but that is an implementation detail. Never use it as a stable key across runs.
- Small-int caching and interning are optimisations. Other implementations (PyPy, for example) can behave differently, which is one more reason to avoid `is` for values.
- A `str` of one-byte characters costs about 1 byte per character plus a header of roughly 41 bytes here (100,041 bytes for 100,000 characters). A string with any character above U+00FF costs 2 or 4 bytes per character, which matters when you load large text.

Chapter recap

- Python reads your source, compiles it to bytecode, and the virtual machine runs it. `.pyc` files in `__pycache__` are only a cache.
- Use the REPL to try ideas fast and scripts to make them repeatable. Never name your file like a standard library module.
- A variable is a name stuck to an object. Assignment never copies data.
- Types: `int` (unlimited size), `float` (binary, so never compare with `==`), `complex`, `bool`, `str` (text), `bytes` (raw data), `None` (nothing).
- Always say `encoding="utf-8"`. Text and bytes are different things.
- Immutable types (`int`, `str`, `tuple`, ...) never change. Mutable types (`list`, `dict`, `set`) change in place, and every name pointing at them sees the change.
- Use `==` for values and `is` only for `None`, `True`, `False`.
- Copy with `.copy()` or slicing (shallow) or `copy.deepcopy` (deep). Functions receive references, so avoid changing arguments.
- Use clear `snake_case` names and unpacking.

Pattern	Example
Run a script	`python3 file.py`
Run one line	`python3 -c "print(1)"`
Float compare	`math.isclose(a, b)`
Text to number	`int("480")`, `float("2.5")`
Text to bytes and back	`s.encode("utf-8")`, `b.decode("utf-8")`
Shallow / deep copy	`x.copy()`, `copy.deepcopy(x)`
Swap	`a, b = b, a`
Collect rest	`first, *rest = items`

Practice set

Problem 1.P.1 (easy) What is the output of `print(type(5/1), 5//2, 5%2)` ?

Answer: `<class 'float'> 2 1`. Division `/` always gives a float. Floor division gives 2, remainder 1.

Problem 1.P.2 (easy) What does `bool([0])` return?

Answer: `True`. The list has one element, so it is not empty. The element being `0` does not matter.

Problem 1.P.3 (easy) How many characters and how many UTF-8 bytes in `"₹5"` ?

Answer: 2 characters. `₹` is 3 bytes and `5` is 1 byte, so 4 bytes.

Problem 1.P.4 (what is the output?)

```
a = [1, 2]
b = a
b = b + [3]
print(a, b)
```

Answer: `[1, 2] [1, 2, 3]`. Here `b = b + [3]` builds a *new* list and re-points `b`. Compare with `b += [3]` from Section 1.4, which changes the list in place. Same-looking code, different result.

Problem 1.P.5 (what is the output?) What does `print(round(0.5), round(1.5), round(2.5))` print?

Answer: `0 2 2`. Halves round to the nearest even number.

Problem 1.P.6 (find the bug)

```
amount = "480"
total = amount + 20
```

Answer: `TypeError: can only concatenate str (not "int") to str`. Convert first: `total = int(amount) + 20`, which is 500.

Problem 1.P.7 (find the bug) A function should return the 3 smallest items but the caller's list also ends up sorted:

```
def top(items):
    items.sort()
    return items[:3]
```

Answer: `items.sort()` changes the caller's list in place. Use `sorted(items)`, which returns a new list: `return sorted(items)[:3]`.

Problem 1.P.8 (computation) Make a 3x3 grid of zeros where changing `grid[0][0]` does not change other rows. Write the line and say why it works.

Answer: `grid = [[0]*3 for _ in range(3)]`. The comprehension runs the inner expression three times and creates three separate lists.

Problem 1.P.9 (scenario) Your function receives a config dict and must test a different timeout without changing the original. What do you do?

Answer: `import copy; test_cfg = copy.deepcopy(cfg); test_cfg["timeout"] = 5`. Use `deepcopy` if the dict contains nested dicts or lists, because a shallow `cfg.copy()` would share them.

Problem 1.P.10 (interview) "Is Python pass-by-value or pass-by-reference?" Answer in two sentences.

Answer: Neither exactly. Python passes object references by value ("call by object reference"): the function gets a name bound to the same object, so changes to a mutable object are visible to the caller, but re-assigning the parameter name inside the function does not affect the caller's name.

Problem 1.P.11 (interview) "Why does `0.1 + 0.2 != 0.3` in Python, and how do you compare floats safely?"

Answer: Floats are stored in binary and `0.1` and `0.2` cannot be represented exactly, so their sum is `0.30000000000000004`. Use `math.isclose(a, b)` (or `abs(a-b) < tolerance`), or use `decimal.Decimal` when exact decimal arithmetic matters, for example money.

More practice: build, stretch and review

Problem 1.P.12 (build) Write `swap_ends(items)` that returns a new list with the first and last items swapped and leaves the input unchanged. It must work for lists of length 0 and 1. Test it on `[1, 2, 3, 4]`, `[9]` and `[]`.

```
def swap_ends(items):
    result = list(items)
    if len(result) >= 2:
        result[0], result[-1] = result[-1], result[0]
    return result
data = [1, 2, 3, 4]
print(swap_ends(data), data, swap_ends([9]), swap_ends([]))
```

```
[4, 2, 3, 1] [1, 2, 3, 4] [9] []
```

Answer: as shown. `list(items)` makes a copy, so the original is untouched (section 1.3). Tuple unpacking swaps the two items without a temporary variable. Self-check: did you test the empty list and the one-item list? Those are where beginners get `IndexError`.

Problem 1.P.13 (what is the output?) What prints?

```
a = [1, 2]
b = a
b += [3]
c = a
c = c + [4]
print(a, c)
```

```
[1, 2, 3] [1, 2, 3, 4]
```

Answer: `b += [3]` on a list changes the list in place, and `a` points at the same list, so `a` becomes `[1, 2, 3]`. `c = c + [4]` builds a new list and rebinds `c`, so `a` is not affected again.

Problem 1.P.14 (stretch) A tuple holds a list: `t = ([1], "x")`. You run `t[0] += [2]`. Does it raise an error? What is `t` afterwards?

```
t = ([1], "x")
try:
    t[0] += [2]
except TypeError as e:
    print(e)
print(t)
```

```
'tuple' object does not support item assignment
([1, 2], 'x')
```

Answer: It raises `TypeError` *and* the list still changes. `+=` first extends the inner list in place (that works, because the list is mutable), then tries to store the result back into the tuple slot (that fails, because tuples cannot be assigned to). Lesson: an immutable container does not make its contents immutable.

Problem 1.P.15 (interview) Why can `a is b` be `True` for `256` and `False` for `1000` in the same program, and what should you write instead?

Answer: CPython caches the small integers -5 to 256, so equal small ints are one object. Larger ints built at run time are separate objects. This is an optimisation, not a promise. Compare values with `==`.

Problem 1.P.16 (project-sized) Write a script `temps.py` that reads lines like `mon 21.5` from a text file, stores them in a dict of day to float, and prints the hottest day and the average. Handle a blank line and a bad number without crashing. Rubric (score yourself, 1 point each): (1) uses a function, not top-level code only; (2) handles blank lines; (3) bad numbers are reported and skipped with `try/except ValueError`; (4) average is computed from the dict values, not hard-coded; (5) you ran it on a file with at least one bad line.

Answer: No single answer. Full marks is 5/5. A common error is dividing by `len(lines)` instead of `len(temps)`, which counts skipped lines. You will use file reading properly in Chapter 5.

Prompt pack

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional at an Indian company moving into data and AI roles, and English is my second language, so use simple words and short sentences. Teach me Python Chapter 1: how Python runs code (REPL, scripts, bytecode), the basic types (int, float, bool, str, bytes, None), and why float comparison is tricky. Explain step by step with small runnable examples, then ask me three questions one at a time and check my answers before moving on.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional at an Indian company moving into data and AI roles, with English as a second language. Topic: Python mutability, identity (`is` vs `==`), references, shallow vs deep copy. Give me 8 graded exercises from easy to hard, each asking me to predict the output of a short snippet. Do not give solutions yet. After I answer, I will paste my answers and you will correct them and explain any wrong one with a picture made of text.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data and AI roles, and English is my second language. Quiz me on Python basics: types, truthiness, integer vs float division, string vs bytes, UTF-8, unpacking, and naming. Ask exactly one question at a time, wait for my answer, say whether it is right, explain the reason in simple words, then ask the next question. Make the questions harder when I get three right in a row.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data roles, with English as a second language. Review my Python code for beginner problems: wrong types, confusing names, mutable-argument bugs, unsafe copies, float comparisons, and missing encoding. Explain each problem in simple words, show the fixed code, and tell me which rule I broke. Here is my code: [PASTE HERE]
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data and AI roles, with English as a second language. Run a mock Python interview for a junior data role, focused on the basics from Chapter 1: mutable vs immutable, `is` vs `==`, copy vs deepcopy, call by object reference, float precision, str vs bytes. Ask one question at a time, give me a score from 1 to 5 for each answer, and show a model answer after each. At the end give me a list of my weak points.
```

CHAPTER 2

CONTROL FLOW, FUNCTIONS, SCOPE AND RECURSION

Make your program decide, repeat, and reuse its own logic.

A delivery app does not just handle one order. It loops over many orders, checks conditions, branches when a payment fails, and reuses the same "pack the parcel" step again and again. Python programs work the same way. In this chapter you learn the building blocks that do this: **decisions** (`if` , `match`), **repetition** (`for` , `while`), and **reusable steps** (functions). You also learn the rules for where a name is visible (**scope**) and how a function can call itself (**recursion**). By the end you will write small, correct functions that process a list of results, and you will know the five classic traps: the mutable default, the late-binding closure, `UnboundLocalError` , modifying a list while looping, and runaway recursion.

Running example. A list of five online-shop orders. Each is a tuple `(item, status_code, delivery_minutes)` , where the status code is what the shop's order system reported:

```
samples = [("book",200,480),("lamp",200,1250),("phone",500,3100),("book",200,310),("phone",503,60)]
```

A **tuple** is a fixed group of values in round brackets (Chapter 4 explains it fully). Here, think of each one as a row of an orders file.

2.1 Control Flow: Decisions and Loops

INTUITION

Think of a traffic police officer at a crossing. For each car he asks a question ("is it an ambulance?") and sends it left or right. That is `if` . A roundabout where cars go around again and again until they find their exit is a loop. **Control flow** means "the order in which lines run, and the questions that change that order".

Words you need

Word	Meaning
Condition	an expression that is true or false, such as `code < 300`
Block	the indented lines under a `:` . Python uses indentation (4 spaces) to group code
Iterable	something you can loop over: a list, a string, a file, `range(...)`
Iteration	one trip around a loop

How it works

```
if condition:        # runs if true
    ...
elif other:          # tried only if the first was false
    ...
else:                # runs if all were false
    ...
for item in iterable:    # one trip per item
    ...
while condition:         # repeat while true
    ...
```

- `break` leaves the loop at once. `continue` skips to the next iteration.
- A loop can have an `else:` block. It runs only if the loop **ended without** `break` . The name is confusing; read it as "no break happened".
- `range(start, stop, step)` gives integers from `start` up to but **not including** `stop` .

- `enumerate(items, start=1)` gives `(index, item)` pairs. `zip(a, b)` walks two iterables together and stops at the shorter one.
- `match` (Python 3.10 and newer) is **structural pattern matching**: it looks at the *shape* of a value, not just equality.
- The **walrus operator** `:=` assigns and returns a value inside an expression. Use it sparingly.
- A **ternary expression** `A if cond else B` picks one of two values.

Worked example 2.1.1: classify and loop

Predict: The status codes 500 and 503 are server errors in the order system. Which rows will print `server error`?

```
def classify(code):
    if code < 300: return "ok"
    elif code < 500: return "client error"
    else: return "server error"
for item, code, mins in samples:
    print(f"{item:<7}{code} {mins:>5} min  {classify(code)}")
for i in range(0, 10, 3): print(i, end=" ")
print()
for i, (item, _, _) in enumerate(samples, start=1): print(i, item, end="; ")
print()
print(list(zip([1,2,3],"abc")), list(zip([1,2,3],"ab")))
```

Real output:

```
book   200   480 min  ok
lamp   200  1250 min  ok
phone  500  3100 min  server error
book   200   310 min  ok
phone  503    60 min  server error
0 3 6 9
1 book; 2 lamp; 3 phone; 4 book; 5 phone;
[(1, 'a'), (2, 'b'), (3, 'c')] [(1, 'a'), (2, 'b')]
```

Line by line:

- `for item, code, mins in samples` unpacks each tuple (Chapter 1, Section 1.4) into three names.
- The f-string `{item:<7}` means "left-align in 7 characters", and `{mins:>5}` means "right-align in 5". Chapter 3 explains f-strings.
- `range(0, 10, 3)` gives 0, 3, 6, 9. It stops before 10.
- `_` is the usual name for a value you do not care about.
- The last `zip` call stopped after two pairs because `"ab"` has only two letters.

Worked example 2.1.2: `break`, loop `else`, `while`, `continue`

Predict: Our courier has a budget of 3000 delivery minutes in total. At which order does the running total pass it?

```
budget = 3000; spent = 0
for item, code, mins in samples:
    spent += mins
    if spent > budget:
        print("over budget at", item); break
else:
    print("within budget")
attempts = 0
while True:
    attempts += 1
    if attempts % 2: continue
    if attempts >= 6: break
print("attempts", attempts)
```

Real output:

```
over budget at phone
attempts 6
```

- Running total: 480, then 1730 (480+1250), then 4830 (1730+3100). 4830 is above 3000 at `phone`, so `break` fires and the `else` block is skipped.
- In the `while True` loop odd numbers hit `continue` and go back to the top. Even numbers reach the `break` test. The loop ends at 6.
- `while True` with a `break` inside is a common pattern for "retry until success".

Worked example 2.1.3: `match`, walrus, ternary

Predict: What does `route(5)` return?

```
def route(req):
    match req:
        case {"method": "GET", "path": "/health"}: return "health"
        case {"method": "POST", "path": p} if p.startswith("/pay"): return f"pay:{p}"
        case {"method": m}: return f"other {m}"
        case _: return "bad"
print(route({"method":"GET","path":"/health"}), route({"method":"POST","path":"/pay/1"}), route({"method":"PUT"}),
route(5))
def kind(x):
    match x:
        case 0: return "zero"
        case int() | float(): return "number"
        case [a, b]: return f"pair {a},{b}"
        case [first, *rest]: return f"list starting {first} (+{len(rest)})"
        case str(): return "text"
        case _: return "unknown"
print([kind(v) for v in (0, 2.5, [1,2], [1,2,3,4], "hi", None)])
data = [480, 1250, 3100]
if (n := len(data)) > 2: print("many samples:", n)
import re
line = "ERROR code=503"
if m := re.search(r"code=(\d+)", line): print(int(m.group(1)))
x = 500
print("slow" if x > 400 else "fast", 200 <= x < 600)
```

Real output:

```
health pay:/pay/1 other PUT bad
['zero', 'number', 'pair 1,2', 'list starting 1 (+3)', 'text', 'unknown']
many samples: 3
503
slow True
```

- `match` tries each `case` from the top. A `case` can match a dictionary shape (`{"method": m}` captures the value into `m`), a type (`int()`), a list shape (`[a, b]`, `[first, *rest]`), and can have an `if` guard. `case _` is the "anything else" default.
- Order matters: `case 0` is before `case int()`, so `0` is "zero", not "number". `route(5)` is `"bad"` because `5` is not a dict.
- `(n := len(data)) > 2` stores the length in `n` and tests it in one line. The regex example (Chapter 3) uses `:=` to test "did the search find something?" and name the result at once.
- Python allows **chained comparison**: `200 <= x < 600` means `200 <= x and x < 600`.

The picture

Each diamond is an `if/elif`. Only the first yes-branch runs. Lines of code inside a branch are indented under it.

Real-world mapping

Concept	Example	Where it appears in the job
`if/elif/else`	classify a status code	Data validation, alert rules
`for`	walk every sample	Processing rows of a file
`while` + `break`	retry an API until it answers	Robust pipelines
`for ... else`	"search; if not found, do X"	Lookup logic
`match`	route by request shape	Parsers, event handlers

Applications

- **Data Analytics:** label each row (`"fast"` , `"slow"`) before grouping.
- **Data Engineering:** loop over files in a folder; retry loop for flaky network calls.
- **Machine Learning:** training loops (`for epoch in range(10)`), stop early with `break` when loss stops improving.
- **GenAI:** route a chatbot message by shape (`match` on a parsed JSON event).

COMMON MISTAKE

Changing a list while looping over it. Removing items shifts the later ones and some are skipped. Loop over a copy (`for it in items[:]`) or build a new list with a comprehension (Chapter 4). Another one: `=` (assignment) instead of `==` (comparison) in a condition. Python reports a `SyntaxError` , which is a kindness.

PROFESSOR'S NOTE

Names like `i` leak out of a loop. After `for i in range(3): pass` , the name `i` still exists and equals 2. This is legal, but do not depend on it.

Check yourself

1. What does `list(range(2, 8, 2))` give?

Answer: `[2, 4, 6]` . It stops before 8.

1. When does a loop's `else` block run?

Answer: Only when the loop finished without hitting `break` .

1. Why must `case 0` be placed before `case int()` ?

Answer: `match` takes the first case that fits. `0` is also an `int` , so a later `case 0` would never be reached.

2.2 Functions: Arguments, Defaults and Lambdas

INTUITION

A function is a recipe card. You write the steps once, give it a name, and then you can say "make this recipe with these ingredients" as many times as you like. A restaurant kitchen does the same: one recipe for tomato sauce is reused in the pizza, the pasta and the soup.

Formal definition

A **function** is a named, reusable block of code. You define it with `def`, and you **call** it with brackets.

- **Parameter**: the name in the definition (`values`). **Argument**: the real value you pass in the call (`[1,2,3]`).
- **Positional arguments** are matched by order. **Keyword arguments** are matched by name (`ndigits=1`).
- **Default value**: `def mean(values, ndigits=2)`. The caller may leave it out.
- `*args` collects extra positional arguments into a tuple. `**kwargs` collects extra keyword arguments into a dict. At a call site `*list` and `**dict` do the reverse: they spread values out.
- A `/` in the parameter list means "everything before me is positional-only". A `*` means "everything after me is keyword-only".
- A function without `return` returns `None`.
- A **docstring** (a string as the first line) documents the function. **Type hints** (`old: float`) are labels for humans and tools. Python does not enforce them at run time (Chapter 9).
- **Functions are objects.** You can pass them to other functions. A **lambda** is a tiny unnamed function with one expression: `lambda p: p[1]`.

Worked example 2.2.1: arguments in every style

Predict: What happens with `mean(ndigits=1, values=[1,2])`? And with `f(1,2,3,4)`?

```
def mean(values, ndigits=2):
    return round(sum(values)/len(values), ndigits)
print(mean([480,1250,3100]), mean([480,1250,3100], 0), mean(ndigits=1, values=[1,2]))
def f(a, b, /, c, *, d): return (a,b,c,d)
print(f(1,2,3,d=4), f(1,2,c=3,d=4))
for bad in ("f(1,2,3,4)", "f(a=1,b=2,c=3,d=4)", "mean()"):
    try: eval(bad)
    except TypeError as e: print("TypeError:", e)
def stats(*args, **kwargs):
    return args, kwargs
print(stats(1, 2, unit="ms", n=3))
nums = [3, 1, 2]; opts = {"ndigits": 1}
print(mean(nums, **opts), max(*nums))
```

Real output:

```
1610.0 1610.0 1.5
(1, 2, 3, 4) (1, 2, 3, 4)
TypeError: f() takes 3 positional arguments but 4 were given
TypeError: f() got some positional-only arguments passed as keyword arguments: 'a, b'
TypeError: mean() missing 1 required positional argument: 'values'
((1, 2), {'unit': 'ms', 'n': 3})
2.0 3
```

- The mean of 480, 1250, 3100 is 4830 / 3 = 1610. With `ndigits=0`, `round` still returns a float (`1610.0`).
- In `f`, `a` and `b` can only be given by position, and `d` can only be given by name. The error messages tell you exactly which rule you broke.
- `mean(nums, **opts)` spreads the dict into `ndigits=1`. `max(*nums)` spreads the list into `max(3, 1, 2)`.

Worked example 2.2.2: the mutable default trap

Predict: What do three calls to `add_sample` print?

```
def add_sample(x, bucket=[]):
    bucket.append(x); return bucket
print(add_sample(1), add_sample(2), add_sample(3))
def add_sample2(x, bucket=None):
    if bucket is None: bucket = []
    bucket.append(x); return bucket
print(add_sample2(1), add_sample2(2))
print(add_sample.__defaults__)
```

Real output:

```
[1, 2, 3] [1, 2, 3] [1, 2, 3]
[1] [2]
([1, 2, 3],)
```

Why: the default value is built **once**, when `def` runs, not each time you call. So all calls share one list (Chapter 1: mutable objects shared by reference). All three prints show the same list, which ended as `[1, 2, 3]`. The fix is to use `None` as the default and build a fresh list inside. The last line proves the list lives in `add_sample.__defaults__`.

Worked example 2.2.3: functions as values, lambda, sorting

```
pairs = [("book", 480), ("phone", 3100), ("lamp", 1250)]
print(sorted(pairs, key=lambda p: p[1], reverse=True))
print(list(map(lambda p: p[1]/1000, pairs)), list(filter(lambda p: p[1] > 1000, pairs)))
def minmax(v): return min(v), max(v)
lo, hi = minmax([5,2,9]); print(lo, hi)
def nothing(): pass
print(nothing())
def pct(old: float, new: float) -> float:
    """Percent improvement from old to new."""
    return (old - new) / old * 100
print(pct(480, 360), pct.__doc__)
print(pct.__annotations__)
```

Real output:

```
[('phone', 3100), ('lamp', 1250), ('book', 480)]
[0.48, 3.1, 1.25] [('phone', 3100), ('lamp', 1250)]
2 9
None
25.0 Percent improvement from old to new.
{'old': <class 'float'>, 'new': <class 'float'>, 'return': <class 'float'>}
```

- `key=` tells `sorted` what to compare. `lambda p: p[1]` means "take the second item of each pair" (the minutes).
- `map` applies a function to each item, `filter` keeps items for which the function is true. Both return lazy iterators, so we wrap them in `list(...)` (Chapter 7).
- A function can "return two values": it really returns one tuple, which the caller unpacks.
- `pct(480, 360)` is `(480-360)/480*100 = 25.0`, the same improvement formula we used in the style guide. Check: `480 * 0.75 = 360`.

Problem 2.2.1 Write a function `slow_share(samples, limit)` that returns the percentage of samples whose delivery time is above `limit`. Use our five samples and limit 1000.

Step 1	Count samples above the limit: 1250 and 3100 are above 1000. The others (480, 310, 60) are not. Count = 2.
Step 2	Divide by the total number of samples: 2 / 5 = 0.4.
Step 3	Multiply by 100: 40.0.
Answer	40% of the samples were slower than 1000 minutes (a late delivery in this example).
Check	Code below prints 40.0.
Why it matters	"Share of orders delivered later than the promised time" is a standard service KPI for any shop.

```
def slow_share(samples, limit):
    slow = [mins for _, _, mins in samples if mins > limit]
    return len(slow) / len(samples) * 100
print(slow_share(samples, 1000))
```

```
40.0
```

The picture

```
def mean(values, ndigits=2):        call: mean([1,2], ndigits=1)
        ^        ^                          ^           ^
   parameter  parameter+default       argument       argument by name

 caller ----- arguments ----> [ function body ] ----- return value ----> caller
```

Real-world mapping

Concept	Example	Where it appears in the job
Default argument	`read_csv(path, sep=",")`	Library functions you will call daily
`**kwargs`	pass options through to another function	Wrapper functions, decorators (Chapter 7)
`lambda` as `key=`	sort rows by one column	Quick data ordering
Keyword-only	`train(model, *, epochs=10)`	APIs that must stay readable

Applications

- **Data Analytics:** helper functions like `clean_amount(text)` used on every row.
- **Data Engineering:** one function per pipeline step so each can be tested.
- **Machine Learning:** `train(model, data, *, lr=0.01, epochs=10)` with keyword-only hyper-parameters.
- **Operations:** a `late_share(orders, limit)` helper reused across every weekly delivery report.

COMMON MISTAKE

Mutable default arguments (`def f(x, bucket=[])`). Never use `[]`, `{}` or `set()` as a default. Use `None` and create the object inside. Another: forgetting `return` and then wondering why you got `None`.

Check yourself

1. What are `args` and `kwargs` inside `def g(*args, **kwargs)` when called as `g(1, a=2)`?

Answer: `args = (1,)` and `kwargs = {"a": 2}`.

1. What does a function return if it has no `return`?

Answer: `None`.

1. What is wrong with `def f(items=[]):`?

Answer: The default list is created once and shared by all calls, so data leaks between calls.

2.3 Scope, Closures and `global` / `nonlocal`

> **INTUITION**
>
> Think of rooms in a house. You look for your keys in your own room first, then in the house, then in the street. You never look in your neighbour's house. **Scope** is the set of "rooms" where Python looks for a name, and it always looks inner room first.

The LEGB rule

When Python sees a name it searches four places in order:

1. **L**ocal: inside the current function.
2. **E**nclosing: inside any function that wraps this one.
3. **G**lobal: at the top level of the file (module).
4. **B**uilt-in: names Python gives you, like `len` and `print`.

The first match wins. **Assigning** to a name inside a function makes it local for the *whole* function. To change an outer name you must declare `global name` (module level) or `nonlocal name` (enclosing function).

A **closure** is an inner function that remembers variables from the function that created it, even after the outer function has finished. Think of it as a function that carries a small backpack.

Worked example 2.3.1: local, global, UnboundLocalError

Predict: `broken()` reads `counter` and then assigns it. Does it print the global value?

```
x = "global"
def show():
    x = "local"
    return x
print(show(), x)
def broken():
    try:
        print(counter)
        counter = 1
    except UnboundLocalError as e: print("UnboundLocalError:", e)
broken()
counter = 0
def inc():
    global counter
    counter += 1
inc(); inc(); print(counter)
```

Real output:

```
local global
UnboundLocalError: cannot access local variable 'counter' where it is not associated with a value
2
```

- `show` creates its own local `x`; the global `x` is untouched.
- In `broken`, the later line `counter = 1` makes `counter` local for the entire function. The `print(counter)` above it then reads a local name that has no value yet. That is the error. Python decides locality when it compiles the function, not line by line.
- `inc` says `global counter`, so `counter += 1` changes the module-level variable. Use `global` rarely. A function that returns a value is easier to test.

Worked example 2.3.2: closures with `nonlocal`, and the LEGB order

```
def make_counter():
    count = 0
    def step():
        nonlocal count
        count += 1
        return count
    return step
c1 = make_counter(); c2 = make_counter()
print(c1(), c1(), c1(), c2())
print(c1.__closure__[0].cell_contents)
v = "G"
def outer():
    v = "E"
    def inner():
        v = "L"
        return v
    return inner(), v
print(outer(), v)
def make_limit(limit):
    def check(mins): return mins <= limit
    return check
within = make_limit(2000)
print(within(1250), within(3100))
```

Real output:

```
1 2 3 1
3
('L', 'E') G
True False
```

- `c1` and `c2` each have their own `count`. Calling `c1` three times gives 1, 2, 3. `c2` starts from 1. Each closure owns a separate backpack.
- `c1.__closure__[0].cell_contents` shows the backpack: the current count, 3.
- `outer()` returned `('L', 'E')` and the global stayed `'G'`: each level has its own `v`.
- `make_limit(2000)` builds a checker that remembers `limit = 2000`. This is the same pattern decorators use in Chapter 7.

Worked example 2.3.3: the late-binding trap and `partial`

Predict: What will `[f() for f in fns]` print for `lambda: i` created in a loop?

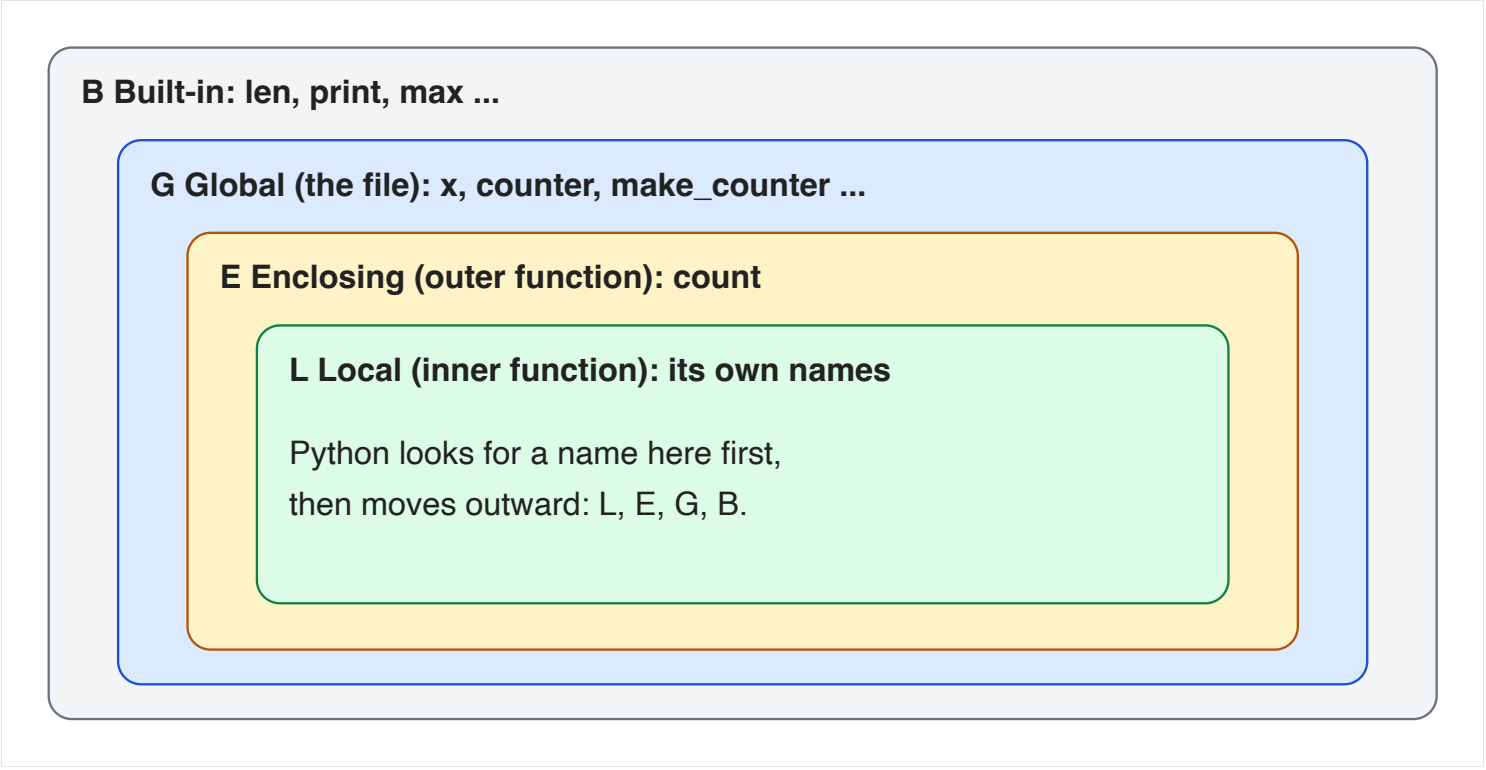
```
fns = [lambda: i for i in range(3)]
print([f() for f in fns])
fns = [lambda i=i: i for i in range(3)]
print([f() for f in fns])
from functools import partial
def power(base, exp): return base**exp
square = partial(power, exp=2)
print(square(7))
```

Real output:

```
[2, 2, 2]
[0, 1, 2]
49
```

A closure looks up `i` **when it is called**, not when it was created. By then the loop is over and `i` is 2, so all three give 2. Fix: bind the current value as a default (`i=i`), or use `functools.partial` , which stores the argument right now. `partial(power, exp=2)` is a new function with `exp` already filled in.

The picture



Nested rooms: Local inside Enclosing inside Global inside Built-in

Read the picture from the inside out. The smallest green room is the function you are in. Python searches that room first, then the next larger room, and so on out to the grey room of built-in names. The first room that contains the name wins. The text version shows the same order.

```
built-in   [ len, print, ... ]
global     [ x, counter, show ... ]
enclosing      [ count ]          <- make_counter's local, kept alive
local              [ step's own names ]

lookup order:  L -> E -> G -> B   (inner room first)
```

Real-world mapping

Concept	Example	Where it appears in the job
Closure	`make_limit(2000)`	Configurable checks, decorators with arguments
`global`	a module-level `counter`	Quick scripts (avoid in real code)
`partial`	`partial(parse, sep=";")`	Passing a pre-configured function to `map` or a thread pool
LEGB	a local `max` hides the built-in	Debugging "object is not callable"

Applications

- **Data Engineering:** factory functions that return a parser configured for one file format.
- **Machine Learning:** `partial(train_step, lr=0.01)` passed to a runner.
- **Web/API work:** callbacks and decorators remember their settings through closures.

COMMON MISTAKE	Creating lambdas in a loop and expecting each to remember its own `i` (see 2.3.3). Also: naming a variable `list`, `max` or `sum` at module level, which hides the built-in everywhere below it.

Check yourself

1. What does LEGB stand for?

Answer: Local, Enclosing, Global, Built-in, the order Python searches names.

1. Which keyword lets an inner function change a variable of its outer function?

Answer: `nonlocal`.

1. Why does reading a variable before assigning it in the same function give `UnboundLocalError`?

Answer: The assignment makes the name local for the whole function, so the early read finds a local with no value.

2.4 Recursion

INTUITION	Russian nesting dolls. To count the dolls, you open the biggest one and say "1 plus the count of the dolls inside". You keep opening until you reach the smallest doll that holds nothing. A **recursive** function solves a problem by calling itself on a smaller version of the same problem.

Formal definition

A recursive function needs two parts:

1. **Base case**: the smallest problem, answered directly, with no further call.
2. **Recursive case**: reduce the problem, call yourself, and combine the result.

Without a base case, or if the problem does not get smaller, Python stops with `RecursionError` after about 1000 nested calls (`sys.getrecursionlimit()`). Each call uses memory on the **call stack**, the list of calls still waiting for an answer.

Worked example 2.4.1: factorial and its call trace

Predict: `fact(3)` is `3 * 2 * 1`. In what order do the "->" lines print?

```
def factorial(n):
    if n <= 1: return 1       # base case
    return n * factorial(n-1) # recursive case
print(factorial(5), factorial(20))
def trace(n, depth=0):
    print("  "*depth + f"fact({n})")
    if n <= 1:
        print("  "*depth + "-> 1"); return 1
    r = n * trace(n-1, depth+1)
    print("  "*depth + f"-> {r}"); return r
trace(3)
```

Real output:

```
120 2432902008176640000
fact(3)
  fact(2)
    fact(1)
    -> 1
  -> 2
-> 6
```

The calls go **down** (3, 2, 1), then the answers come **up** (1, 2, 6). Hand check: `5! = 5*4*3*2*1 = 120`. The result for 3 is `3 * 2 = 6`: it waited for `fact(2)`, which waited for `fact(1)`.

Worked example 2.4.2: runaway recursion, and fibonacci three ways

Predict: How many function calls does `fib_counted(20)` make? (Guess before you look.)

```
import sys
def bad(n): return n * bad(n-1)
try: bad(5)
except RecursionError as e: print("RecursionError:", str(e)[:45])
print(sys.getrecursionlimit())
calls = 0
def fib_counted(n):
    global calls; calls += 1
    return n if n < 2 else fib_counted(n-1) + fib_counted(n-2)
print(fib_counted(20), calls)
from functools import lru_cache
calls2 = 0
@lru_cache(maxsize=None)
def fib_memo(n):
    global calls2; calls2 += 1
    return n if n < 2 else fib_memo(n-1) + fib_memo(n-2)
print(fib_memo(20), calls2, fib_memo(90))
def fib_iter(n):
    a, b = 0, 1
    for _ in range(n): a, b = b, a+b
    return a
print(fib_iter(20), fib_iter(90))
```

Real output:

```
RecursionError: maximum recursion depth exceeded
1000
6765 21891
6765 21 2880067194370816120
6765 2880067194370816120
```

- `bad` never stops (no base case), so Python raises `RecursionError`.
- Plain recursive Fibonacci recomputes the same values again and again: **21,891 calls** just for `fib(20)`. The cost roughly doubles for each extra step.
- `@lru_cache` (a decorator, Chapter 7) remembers earlier answers. **Memoization** means "store a result the first time and reuse it". Only 21 calls were needed (for n = 0 to 20). So `fib_memo(90)` is instant.
- A loop (`fib_iter`) gives the same answers with no recursion and no stack. When a loop is easy to write, prefer the loop.

Worked example 2.4.3: recursion on nested data

```
def flatten(x):
    out = []
    for item in x:
        if isinstance(item, list): out.extend(flatten(item))
        else: out.append(item)
    return out
print(flatten([1,[2,[3,4]],5,[[6]]]))
def bsearch(a, t, lo=0, hi=None):
    if hi is None: hi = len(a)-1
    if lo > hi: return -1
    mid = (lo+hi)//2
    if a[mid] == t: return mid
    return bsearch(a, t, mid+1, hi) if a[mid] < t else bsearch(a, t, lo, mid-1)
arr = [2,5,8,12,16,23,38,56]
print(bsearch(arr, 23), bsearch(arr, 7))
def depth_sum(tree):
    return tree["mins"] + sum(depth_sum(c) for c in tree.get("steps", []))
recipe_tree = {"mins":50,"steps":[{"mins":120,"steps":[{"mins":30}]},{"mins":200}]}
print(depth_sum(recipe_tree))
```

Real output:

```
[1, 2, 3, 4, 5, 6]
5 -1
400
```

- `flatten` is natural as recursion because the data is nested to an unknown depth.
- `bsearch` (binary search) halves the search range each call. It found `23` at index 5, and `7` is not there (`-1`).
- `depth_sum` adds up a recipe tree. A **recipe tree** is a dish made of steps, where some steps are themselves made of smaller steps, each with a preparation time. 50 + 120 + 30 + 200 = 400 minutes. Trees and folders are the classic use of recursion.

Problem 2.4.1 Count how many calls `factorial(5)` makes, and the deepest call depth.

Step 1	`factorial(5)` calls `factorial(4)`, which calls 3, then 2, then 1.
Step 2	That is calls for n = 5, 4, 3, 2, 1: five calls. The base case stops at n = 1.
Step 3	The call stack holds all five at the moment `factorial(1)` runs, so depth is 5.
Answer	5 calls, depth 5. The work grows in a straight line with n.
Check	Compare to `fib(20)` above: 21,891 calls for n = 20. Straight-line recursion is cheap; branching recursion with repeated work is not.
Why it matters	Count the calls before you trust a recursive solution on large input.

The picture

```
fib(4)
 +-- fib(3)
 |    +-- fib(2)
 |    |    +-- fib(1)
 |    |    +-- fib(0)
 |    +-- fib(1)
 +-- fib(2)         <- fib(2) is computed AGAIN (wasted work)
      +-- fib(1)
      +-- fib(0)
```

Each node is one call. The repeated `fib(2)` is why memoization helps: it remembers `fib(2)` after the first time.

Real-world mapping

Concept	Example	Where it appears in the job
Recursion on trees	folder walk, JSON of unknown depth	Parsing nested API data
Memoization	`lru_cache` on a slow lookup	Caching repeated calls
Divide and conquer	binary search	Interview problems, sorted data
Recursion limit	`RecursionError` on deep data	Choosing a loop or an explicit stack

Applications

- **Data Engineering:** walking nested JSON and directory trees.
- **Machine Learning:** decision trees are built and evaluated recursively.
- **Data Science:** dynamic programming (cached recursion) for optimisation problems.
- **Interview prep:** trees, graphs, backtracking.

COMMON MISTAKE

No base case, or a base case that is never reached (`factorial(-1)` with `if n == 0` loops forever). Also, recursing on huge input: Python does not optimise tail calls, so deep recursion fails around 1000 levels. Convert to a loop.

Check yourself

1. What are the two parts of every recursive function?

Answer: A base case and a recursive case that makes the problem smaller.

1. Why is plain recursive `fib(40)` slow?

Answer: It recomputes the same sub-problems an enormous number of times.

1. What does `@lru_cache` do?

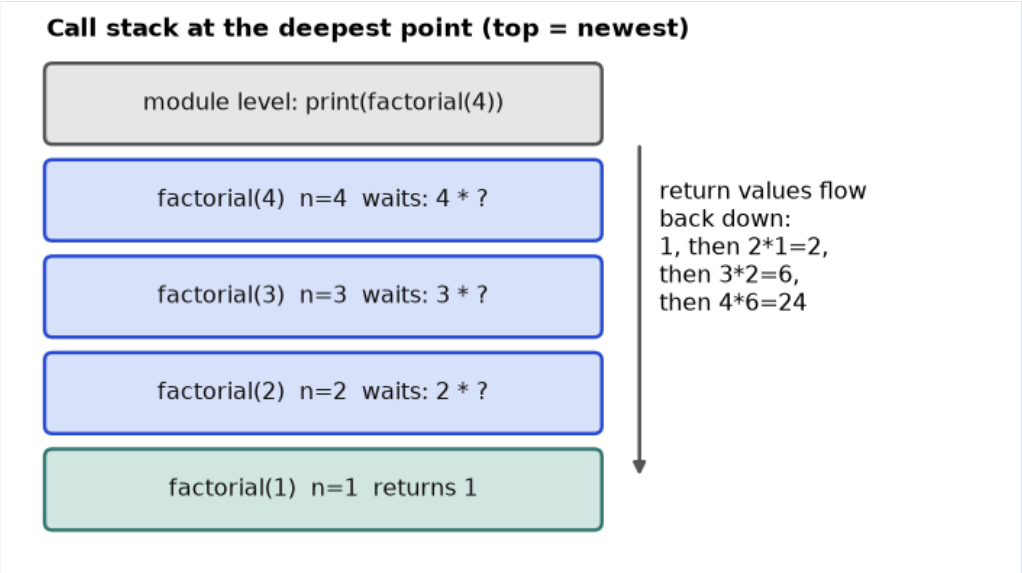
Answer: It stores results of earlier calls (memoization) so repeated calls return instantly.

2.5 The Call Stack, Recursion Limits and Closure Traps

Section 2.4 showed recursion working. This section shows what Python does while it works, how to survive when the input is too deep, and one closure trap that catches almost every learner once.

The call stack

Each time you call a function, Python creates a **frame**: a small record holding that call's local variables and the line it is on. Frames sit on a **call stack**, newest on top. When a function returns, its frame is removed and the caller continues. A recursive function stacks one frame per call, so `factorial(4)` has four frames open at once at the deepest point.



The call stack for factorial(4) at its deepest point. Each box is one frame with its own n.

How to read it: start at the bottom, the module that called `factorial(4)`. Each frame above it is a waiting call. Only the top frame (n=1) can finish. When it returns 1, the frame below gets 1 and computes 2 * 1, and so on down to 24.

Problem 2.5.1 Trace `factorial(4)` by printing when each call starts and ends, with indentation equal to the depth.

Idea	Pass a `depth` argument that grows by one per call, and print `"  " * depth` before each message.
Going down	Four "enter" lines, each indented more than the last, until the base case n = 1.
Coming back	Returns print in reverse order: 2, then 6, then 24.

```
def factorial(n, depth=0):
    pad = "  " * depth
    print(f"{pad}enter factorial({n})")
    if n == 1:
        print(f"{pad}base case, return 1")
        return 1
    result = n * factorial(n - 1, depth + 1)
    print(f"{pad}return {result}")
    return result
factorial(4)
```

```
enter factorial(4)
  enter factorial(3)
    enter factorial(2)
      enter factorial(1)
      base case, return 1
    return 2
  return 6
return 24
```

When the stack is too deep

Python limits the stack to protect your computer. `sys.getrecursionlimit()` is 1000 by default. Going past it raises `RecursionError`. For deep data, replace the call stack with your own list used as a stack. The loop below sums a nested list without recursion, and it handles a list nested 5,000 levels deep that would crash a recursive version.

Problem 2.5.2 Measure the limit, then sum a list nested 5,000 deep without recursion.

Step 1	Recurse until `RecursionError` and return the depth reached. It is a little below the limit because some frames are already in use.
Step 2	Keep `stack = [nested]`. Pop an item. If it is a list, push its children. Otherwise add it to the total.
Check	`[1, [2, [3, [4]]], 5]` sums to 15. The deeply nested list holds only the number 1.

```
import sys
def depth(n=0):
    try:
        return depth(n + 1)
    except RecursionError:
        return n
print(sys.getrecursionlimit())
d = depth()
print(900 < d <= sys.getrecursionlimit())
def total(nested):
    stack, s = [nested], 0
    while stack:
        item = stack.pop()
        if isinstance(item, list):
            stack.extend(item)
        else:
            s += item
    return s
deep = 1
for _ in range(5000):
    deep = [deep]
print(total(deep), total([1, [2, [3, [4]]], 5]))
```

```
1000
True
1 15
```

COMMON MISTAKE

Raising the limit with `sys.setrecursionlimit(10**6)` to "fix" a `RecursionError`. If the recursion really is too deep, the program can now crash the whole interpreter instead of raising a clean error. Prefer a loop with an explicit stack, or memoisation (section 2.4) so the depth stays small.

The late-binding closure trap

A closure (section 2.3) remembers a *variable*, not its value at the time the function was made. That matters in loops. All three lambdas below read the same `i`, and by the time you call them the loop has finished with `i = 2`.

Problem 2.5.3 Build three functions that return 0, 1 and 2. Show the broken version, then two fixes. Also show the parameter kinds `/`, `*args`, keyword-only and `**kw` in one signature.

Broken	`[lambda: i for i in range(3)]`: every lambda looks up `i` when called, and finds 2.
Fix 1	A default argument is evaluated once, at definition: `lambda i=i: i`.
Fix 2	A factory function gives each closure its own `i`.
Signature	Before `/` the argument is positional-only. After `*args`, `sep` is keyword-only. Extra keywords land in `kw`.

```
funcs = [lambda: i for i in range(3)]
print([f() for f in funcs])
fixed = [lambda i=i: i for i in range(3)]
print([f() for f in fixed])
def make(i):
    return lambda: i
print([make(i)() for i in range(3)])
def report(first, /, *args, sep="-", **kw):
    return first, args, sep, kw
print(report(1, 2, 3, sep="+", unit="kg"))
try:
    report(first=1)
except TypeError as e:
    print(e)
```

```
[2, 2, 2]
[0, 1, 2]
[0, 1, 2]
(1, (2, 3), '+', {'unit': 'kg'})
report() missing 1 required positional argument: 'first'
```

KEY IDEA

A function body runs when the function is **called**. Default values are evaluated when the function is **defined**. Closure variables are looked up when the function is **called**. Keeping those three moments apart explains most "why did it do that?" surprises.

Chapter recap

- `if/elif/else` decide, `for` and `while` repeat, `break` / `continue` change the loop, and a loop `else` runs only when no `break` happened.
- `match` checks the shape of data. Put specific cases before general ones, and end with `case _`.
- Functions take parameters (definition) and arguments (call). Use keyword arguments for clarity, `*args` and `**kwargs` to pass many.
- Never use a mutable default value. Use `None` and create inside.
- Python looks up names in the order Local, Enclosing, Global, Built-in.
- Assigning in a function makes the name local. Use `nonlocal` or `global` only when you must.
- A closure remembers variables from where it was created. In loops, bind values early (`i=i`) or use `partial`.
- Recursion needs a base case and shrinking input. Use `lru_cache` for repeated sub-problems, or a loop when it is simple.

Pattern	Example
Loop with index	`for i, x in enumerate(items, start=1):`
Walk two lists	`for a, b in zip(xs, ys):`
Loop `else`	`for ...: ... break` / `else: not found`
Pattern match	`match v: case [a, b]: ...`
Safe default	`def f(x, bucket=None): bucket = bucket or []` (use `is None` when `[]` is valid input)
Sort by field	`sorted(rows, key=lambda r: r[1])`
Fix an argument	`partial(power, exp=2)`
Cache a function	`@lru_cache(maxsize=None)`

Practice set

Problem 2.P.1 (easy) What does this print?

```
for i in range(3):
    if i == 1: continue
    print(i)
```

Answer: `0` then `2`. When `i` is 1, `continue` skips the print.

Problem 2.P.2 (easy) What is `list(zip("ab", [10, 20, 30]))`?

Answer: `[('a', 10), ('b', 20)]`. `zip` stops at the shortest input.

Problem 2.P.3 (what is the output?)

```
def f(x, y=[]):
    y.append(x)
    return len(y)
print(f(1), f(2))
```

Answer: `1 2`. The default list is shared, so the second call sees two items.

Problem 2.P.4 (what is the output?)

```
x = 5
def g():
    x = x + 1
    return x
print(g())
```

Answer: It raises `UnboundLocalError`. The assignment makes `x` local, so `x + 1` reads a local `x` that has no value. Fix: pass `x` as a parameter, or declare `global x` (not preferred).

Problem 2.P.5 (computation) Use `match` to return `"empty"` for `[]`, `"one"` for a one-item list and `"many"` otherwise.

Answer:

```
def size(x):
    match x:
        case []: return "empty"
        case [_]: return "one"
        case _: return "many"
```

`size([])` is `"empty"`, `size([7])` is `"one"`, `size([1,2])` is `"many"`.

Problem 2.P.6 (computation) How many times is the base case reached when computing `fib(4)` recursively? (Base cases are n = 0 and n = 1.)

Answer: 5. The leaves of the tree above are `fib(1)`, `fib(0)`, `fib(1)`, `fib(1)`, `fib(0)`. That is three `fib(1)` and two `fib(0)`, so 5 leaves.

Problem 2.P.7 (find the bug)

```
fns = []
for n in [10, 20, 30]:
    fns.append(lambda: n * 2)
print([f() for f in fns])
```

Answer: It prints `[60, 60, 60]` because every lambda reads `n` when called, after the loop ended with `n = 30`. Fix: `lambda n=n: n * 2` gives `[20, 40, 60]`.

Problem 2.P.8 (find the bug)

```
def countdown(n):
    print(n)
    countdown(n - 1)
```

Answer: There is no base case, so it recurses until `RecursionError`. Add `if n < 0: return` at the top.

Problem 2.P.9 (scenario) You receive 10,000 delivery times (in minutes) and must report how many are above 2000. A teammate writes a recursive function that processes one item per call. What is wrong and what do you do?

Answer: Recursion depth would reach 10,000, far above the default limit of 1000, so it raises `RecursionError`. Use a loop or a comprehension: `sum(1 for mins in times if mins > 2000)`.

Problem 2.P.10 (interview) "What is the difference between `*args` and `**kwargs`?"

Answer: `*args` collects extra positional arguments into a tuple. `**kwargs` collects extra keyword arguments into a dict. They let a function accept any number of inputs, and are used to forward arguments to another function.

Problem 2.P.11 (interview) "Explain Python's scoping rule and when you need `nonlocal`."

Answer: Python uses LEGB: Local, Enclosing, Global, Built-in. Assignment inside a function creates a local name. If an inner function must rebind a variable owned by an enclosing function, it must declare it `nonlocal`, otherwise Python creates a new local variable instead.

More practice: build, stretch and review

Problem 2.P.12 (build) Write `power(base, exp)` recursively using the idea that `base**exp` equals `half * half` when `exp` is even, where `half = power(base, exp // 2)`, and `half * half * base` when it is odd. Test `power(2, 10)` and `power(3, 5)`.

```
def power(base, exp):
    if exp == 0:
        return 1
    half = power(base, exp // 2)
    return half * half if exp % 2 == 0 else half * half * base
print(power(2, 10), power(3, 5))
```

```
1024 243
```

Answer: as shown. The base case is `exp == 0`. Each call halves `exp`, so the depth is about $\log_2(\text{exp})$ (for `exp = 10`: 10, 5, 2, 1, 0, five calls). That is far shallower than multiplying `exp` times. Check: $2^{10} = 1024$ and $3^5 = 243$.

Problem 2.P.13 (build) The Collatz rule: if `n` is 1, stop; if even, go to `n // 2`; if odd, go to `3n + 1`. Write `collatz_steps(n)` that counts the steps to reach 1. What do 6 and 27 give?

```
def collatz_steps(n, steps=0):
    if n == 1:
        return steps
    return collatz_steps(n // 2 if n % 2 == 0 else 3 * n + 1, steps + 1)
print(collatz_steps(6), collatz_steps(27))
```

```
8 111
```

Answer: 8 and 111. By hand for 6: 6, 3, 10, 5, 16, 8, 4, 2, 1, which is 8 steps. The recursion depth for 27 is 111, well under the limit of 1000, so this version is safe for small inputs.

Problem 2.P.14 (stretch) Why does a recursive function with a mutable default such as `def f(x, acc=[])` that appends to `acc` give wrong answers on the second top-level call, and what is the fix?

Answer: The default list is created once, when `def` runs, and is shared by every call. The second top-level call starts with the first call's items. Fix: use `acc=None` and write `if acc is None: acc = []` inside, or put the list in an inner helper function.

Problem 2.P.15 (review, mixes chapters 1 and 2) Without running it, say what prints for `fs = [lambda x=k: x * 2 for k in (1, 2, 3)]` followed by `print([f() for f in fs])`, and then say what it prints if you remove `=k` and write `lambda: k * 2`.

Answer: With `x=k` the default captures each value: `[2, 4, 6]`. Without it, every lambda reads the final `k`, which is 3: `[6, 6, 6]` (see Problem 2.5.3).

Prompt pack

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional at an Indian company moving into data and AI roles, and English is my second language, so use simple words. Teach me Python control flow: if/elif/else, for, while, break, continue, loop else, enumerate, zip, and match statements. Use one running example of online-shop delivery orders. Explain step by step with short runnable code, ask me to predict each output first, and check my understanding with three questions at the end.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data roles, with English as a second language. Topic: Python function arguments (positional, keyword, defaults, *args, **kwargs, keyword-only, positional-only) and the mutable default argument bug. Give me 8 graded exercises from easy to hard with no solutions. When I paste my answers, mark each one and explain every mistake in simple words with a small diagram.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data and AI roles, and English is my second language. Explain Python scope (LEGB), closures, global and nonlocal with an analogy and runnable examples. Then quiz me one question at a time, including the late-binding lambda trap and UnboundLocalError. Wait for my answer before giving the next question, and explain the reason for each correct answer.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data and AI roles, with English as a second language. Teach recursion from zero: base case, recursive case, call stack, RecursionError, memoization with lru_cache, and when to use a loop instead. Walk through fib(5) by drawing the call tree in text. Then give me 5 recursion exercises on nested lists, trees, and binary search, with hints but no full solutions. Here is my attempt: [PASTE HERE]
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data and AI roles, and English is my second language. Run a mock Python interview on Chapter 2 topics: default arguments, *args and **kwargs, closures, scope, decorators-lite using closures, recursion limits, and loop else. Ask one question at a time, give a score from 1 to 5, show a model answer, and finish with a short list of topics I should revise.
```

CHAPTER 3

STRINGS, TEXT PROCESSING AND REGULAR EXPRESSIONS

Turn messy text into clean, usable fields.

Most real data starts as text: log lines, CSV rows, JSON from an API, user comments, file names. An online shop, for example, keeps an access log with one line for every visit to its website. In this chapter you learn how Python stores text, the 20 string methods that solve 90% of cleaning jobs, the f-string mini-language for printing neat reports, and **regular expressions** (regex), a compact language for describing text patterns. By the end you will be able to take one raw web-server log line and pull out the method, path, status code and processing time as proper typed values.

Running example. One line from a web-server access log. It is one visit to a shop website. It tells us who asked for what and how long the server took to answer:

```
10.0.0.5 - - [07/Nov/2026:10:15:32 +0530] "GET /api/orders/42 HTTP/1.1" 200 512 0.480
```

Reading left to right: client IP, two unused dashes, timestamp, the request in quotes (method, path, protocol), status code `200`, bytes sent `512`, and time taken `0.480` seconds.

3.1 Strings: Slicing and the Everyday Methods

INTUITION

A string is a necklace of beads, where each bead is one character and each bead has a number: 0 for the first, 1 for the second, and so on. Slicing means "give me the beads from position A to position B". You cannot change a bead in place. To change the necklace you make a new one.

Formal definition

- A `str` is an immutable sequence of Unicode characters (Chapter 1). Indexing starts at 0. Negative indexes count from the end: `-1` is the last character.
- **Slice** `s[start:stop:step]` takes characters from `start` up to but not including `stop`. Any part may be left out. `s[::-1]` reverses.
- Methods never change the original string. They return a new one.
- **Raw string** `r"..."`: backslashes are kept as they are (`r"\t"` is two characters, not a tab). You need these for regex and Windows paths.
- **Escape sequences**: `\n` new line, `\t` tab, `\\` backslash, `\"` quote.

The methods you will use every day

Task	Method	Example result
remove spaces at both ends	`s.strip()` (also `lstrip`, `rstrip`)	`" hi "` becomes `"hi"`
cut into a list	`s.split(sep, maxsplit)`	`"a,b".split(",")` gives `['a','b']`
glue a list	`sep.join(list)`	`"-".join(["a","b"])` gives `"a-b"`
replace	`s.replace(old, new)`	all matches replaced
find position	`s.find(x)` returns `-1` if missing, `s.index(x)` raises `ValueError`	
test start / end	`s.startswith(x)`, `s.endswith(x)`	`True` / `False`
test inside	`x in s`	`True` / `False`

Task	Method	Example result
count	`s.count(x)`	number of non-overlapping matches
case	`upper`, `lower`, `title`, `capitalize`, `swapcase`, `casefold`	
test type	`isdigit`, `isalpha`, `isalnum`, `isspace`, `isupper`	
pad	`center`, `ljust`, `rjust`, `zfill`	`"7".zfill(3)` gives `"007"`
cut a prefix or suffix	`removeprefix`, `removesuffix`	
split in three	`partition(sep)`	`('a', '=', 'b=c')`

Worked example 3.1.1: slicing and the core methods

Predict: `t` is the request text `GET /api/orders/42?status=paid HTTP/1.1`. What does `t.split()` give? What does `t.find("zzz")` give, and what does `t.index("zzz")` do?

```
s = "  GET /api/orders/42?status=paid HTTP/1.1  "
print(repr(s.strip()), len(s))
t = s.strip()
print(t[0:3], t[4:], t[-8:], t[::-1][:5], t[::2][:6])
print(t.split(), t.split("/", 2), t.rsplit("/", 1))
print("-".join(["a","b","c"]), " ".join("hello world".split()))
print(t.replace("GET","POST"), t.find("api"), t.find("zzz"), t.index("api"))
try: t.index("zzz")
except ValueError as e: print("ValueError:", e)
print(t.startswith("GET"), t.endswith("1.1"), "api" in t, t.count("/"))
```

Real output:

```
'GET /api/orders/42?status=paid HTTP/1.1' 43
GET /api/orders/42?status=paid HTTP/1.1 HTTP/1.1 1.1/P GT/p/r
['GET', '/api/orders/42?status=paid', 'HTTP/1.1'] ['GET ', 'api', 'orders/42?status=paid HTTP/1.1'] ['GET /api/orders/42?
status=paid HTTP', '1.1']
a-b-c hello world
POST /api/orders/42?status=paid HTTP/1.1 5 -1 5
ValueError: substring not found
True True True 4
```

Line by line:

- `repr(...)` shows the quotes, so you can see that the two spaces on each side are gone. The original had length 43 (39 characters plus 4 spaces).
- `t[0:3]` is `GET`. `t[4:]` is everything from index 4. `t[-8:]` is the last 8 characters, `HTTP/1.1`. `t[::-1]` reverses the text, so the first 5 characters of it are `1.1/P`. `t[::2]` takes every second character: `GT/p/r`.
- `split()` with no argument cuts on any run of whitespace. `split("/", 2)` cuts at most 2 times, leaving the rest in the last item. `rsplit` cuts from the right.
- `" ".join("hello world".split())` is the standard trick to squeeze many spaces into one.
- `find` returns `-1` when there is no match, `index` raises an error. Prefer `in` when you only need yes/no, because `-1` is a valid-looking index and causes silent bugs.

Worked example 3.1.2: more methods, raw strings, Unicode

Predict: What is `"a,b,,c".split(",")`? And `"a b c".split(" ")` versus `"a b c".split()`?

```
print("a,b,,c".split(","), "a b  c".split(" "), "a b  c".split())
print("line1\nline2\r\nline3".splitlines(), "x".center(7,"*"), "7".zfill(3), "ab".ljust(5,".")+"|", "ab".rjust(5)+"|")
print("www.site.com".removeprefix("www."), "file.csv".removesuffix(".csv"))
print("a\tb", r"a\tb", len("a\tb"), len(r"a\tb"))
print("abc".partition("b"), "a=b=c".partition("="), "a=b=c".rpartition("="))
print("Straße".upper(), "ß".casefold(), "STRASSE".casefold() == "straße".casefold())
print(ord("A"), chr(97), chr(8377), "\N{INDIAN RUPEE SIGN}")
s2 = "abc"
try: s2[0] = "z"
except TypeError as e: print("TypeError:", e)
print("abc" < "abd", "B" < "a", sorted(["b","A","a","B"]), sorted(["b","A","a","B"], key=str.lower))
```

Real output:

```
['a', 'b', '', 'c'] ['a', 'b', '', 'c'] ['a', 'b', 'c']
['line1', 'line2', 'line3'] ***x*** 007 ab...|    ab|
site.com file
a       b a\tb 3 4
('a', 'b', 'c') ('a', '=', 'b=c') ('a=b', '=', 'c')
STRASSE ss True
65 a ₹ ₹
TypeError: 'str' object does not support item assignment
True True ['A', 'B', 'a', 'b'] ['A', 'a', 'b', 'B']
```

What to learn:

- `split(",")` keeps empty pieces between two commas. `split(" ")` also does, but `split()` with no argument skips empty pieces. For messy spaces use `split()`.
- `splitlines()` handles `\n` and `\r\n` (Windows line endings), a very common surprise in files from other systems.
- `r"a\tb"` has 4 characters (the backslash and the `t` are kept), while `"a\tb"` has 3 (a real tab).
- `casefold()` is the strong, Unicode-aware version of `lower()`. Use it to compare text without caring about case: German `ß` becomes `ss`.
- `ord` gives the number of a character, `chr` does the reverse. `₹` is code 8377.
- Strings compare by character code, so `"B" < "a"` is true (capitals come first). To sort ignoring case, pass `key=str.lower`.
- Strings are immutable, so `s2[0] = "z"` fails.

The picture

```
 t =  G  E  T     /  a  p  i ...
 +ve  0  1  2  3  4  5  6  7
 -ve -n ...                 -2 -1

 t[0:3]  -> "GET"           (index 0,1,2; stop 3 not included)
 t[4:]   -> from index 4 to the end
 t[-8:]  -> last 8 characters
 t[::-1] -> step -1 = backwards
```

Each character has two addresses: a positive one counting from the left and a negative one counting from the right. The `stop` number is a boundary, not a character, which is why `t[0:3]` has exactly 3 characters (`stop - start`).

Real-world mapping

Method	Example	Where it appears in the job
`strip` , `lower`	clean a column of city names	Data cleaning before joins
`split` , unpack	parse `key=value` pairs	Config and log parsing

Method	Example	Where it appears in the job
`join`	build a CSV line or SQL list	Report generation
`startswith`	filter file names	ETL file pickers
`casefold`	compare user input	Deduplicating names and emails

Applications

- **Data Analytics:** clean text columns (`str.strip().str.lower()` in pandas uses the same method names).
- **Data Engineering:** split log lines, build file paths and S3 keys.
- **NLP / GenAI:** tokenising, cleaning documents, building prompts.
- **Retail / Operations Analytics:** read order-system and website access-log lines.

COMMON MISTAKE

Building a long string with `+=` in a big loop (slow, because each step makes a new string). Collect parts in a list and use `"".join(parts)` once. Another: forgetting that methods return new strings, so `s.strip()` alone does nothing; you need `s = s.strip()` .

Check yourself

1. What is `"hello"[1:4]` ?

Answer: `"ell"` (indexes 1, 2, 3).

1. Why use `casefold()` instead of `lower()` for comparison?

Answer: It handles Unicode special cases (like `ß`) so text with different case compares equal.

1. What is the difference between `find` and `index` when the text is missing?

Answer: `find` returns `-1` ; `index` raises `ValueError` .

3.2 Formatting Text: f-strings and Friends

INTUITION

A mail-merge letter: one template, many names. Formatting lets you write the template once ("{name} took {mins} mins") and Python fills in the holes, aligned and rounded the way you ask.

Formal definition

An **f-string** is a string with `f` before the quote. Inside `{ }` you put any expression and, after a colon, a **format spec**: `{value:fill align width , .precision type}` . Common pieces:

Spec	Meaning	Example
`.1f`	1 decimal place	`480.4567` gives `480.5`
`>10` `<10` `^10`	right, left, center in 10 characters	
`,` or `_`	thousands separator	`1,250,000`
`.1%`	multiply by 100, 1 decimal, add `%`	`0.25` gives `25.0%`
`03d`	integer, zero padded to width 3	`5` gives `005`
`x` , `b`	hexadecimal, binary	`255` gives `ff`

Spec	Meaning	Example
`=` after name	print name and value (debugging)	`{mins=}`
`!r`	use `repr()`	shows quotes

Older styles still appear in old code: `"{} {}".format(a, b)` and `"%s %d" % (a, b)`. Learn to read them, write f-strings.

Worked example 3.2.1: format specs

Predict: What does `f"{pct:.1%}"` give for `pct = 0.25`?

```
name, mins, pct = "apple", 480.4567, 0.25
print(f"{name} took {mins:.1f} mins")
print(f"{name:>10}|{name:<10}|{name:^10}|")
print(f"{1250000:,} {1250000:_} {pct:.1%} {255:x} {255:#b} {5:03d} {mins:10.2f}|")
print(f"{mins=}", f"{name!r}", f"{2+3*4}", f"{{literal}}")
w = 8; print(f"{name:>{w}}|")
print("{} took {} mins".format(name, 480), "{0}-{0}-{1}".format("a","b"), "{n}:{v:.2f}".format(n="x", v=1.5))
print("%s took %d mins (%.1f%%)" % (name, 480, 25.0))
```

Real output:

```
apple took 480.5 mins
     apple|apple     |  apple   |
1,250,000 1_250_000 25.0% ff 0b11111111 005     480.46|
mins=480.4567 'apple' 14 {literal}
   apple|
apple took 480 mins a-a-b x:1.50
apple took 480 mins (25.0%)
```

- `480.4567` rounded to one decimal is `480.5`, and to two decimals `480.46`.
- `{mins:10.2f}` means width 10, 2 decimals, so the number is padded with spaces on the left.
- `{{` and `}}` print literal braces. `{2+3*4}` runs the expression and shows 14. The width itself can be a variable: `{name:>{w}}`.

Worked example 3.2.2: a neat report table, and a wrong format code

Predict: What happens with `f"{'abc':d}"`?

```
rows = [("book", 480, 0.0), ("shoes", 1250, 0.5), ("lamp", 3100, 12.25)]
print(f"{'item':<8}{'mins':>7}{'late%':>8}")
for l, m, e in rows: print(f"{l:<8}{m:>7,}{e:>8.2f}")
try: print(f"{'abc':d}")
except ValueError as e: print("ValueError:", e)
```

Real output:

```
item       mins   late%
book        480    0.00
shoes     1,250    0.50
lamp      3,100   12.25
ValueError: Unknown format code 'd' for object of type 'str'
```

Text columns are left-aligned (`<8`) and numbers right-aligned (`>7`), so digits line up. Asking for `d` (integer) on a string is an error, and the message names both the code and the type. Fix by using `s` for strings or no code at all.

The picture

```
f"{mins:>10,.2f}"
    |  | | | |
    |  | | | +-- type: f = fixed decimals
    |  | | +---- precision: 2 digits after the point
    |  | +------ thousands separator
    |  +-------- width: 10 characters, align right (>)
    +----------- the value (any expression)
```

Read the spec from left to right: align, width, separator, precision, type. Most of the time you only need two or three of them.

Real-world mapping

Spec	Example	Where it appears in the job
`:.2f`	`f"{rmse:.2f}"`	Model metrics in reports
`:,`	`f"{rows:,}"`	Row counts in logs
`:.1%`	`f"{err:.1%}"`	Error rates, accuracy
`{x=}`	`f"{n_rows=}"`	Quick debugging output

Applications

- **Data Analytics:** report tables and e-mail summaries.
- **Machine Learning:** training logs like `epoch 3 loss 0.2154`.
- **GenAI:** building prompts from templates with `f"""..."""`.
- **Data Engineering:** building file names (`f"{date:%Y/%m/%d}/part-{i:04d}.csv"`).

COMMON MISTAKE

Putting user-supplied text into a SQL string with an f-string. That is an injection risk (Chapter 13). Format for **display** only; use parameters for databases. Another: using the same quote type inside an f-string on old Python versions (before 3.12 you must switch quote style).

Check yourself

1. How do you print `3.14159` with 2 decimals?

Answer: `f"{3.14159:.2f}"` gives `3.14`.

1. What does `f"{0.07:.0%}"` print?

Answer: `7%`.

1. How do you print literal curly braces in an f-string?

Answer: Double them: `{{` and `}}`.

3.3 Encoding: Text, Bytes and UTF-8 in Practice

INTUITION

A computer can only store numbers. An **encoding** is a code book that says "number 97 means the letter a". Everyone must use the same code book, or text turns into garbage (the famous `Ã©` instead of `é`).

Formal definition

- **Unicode** is a list that gives every character in the world a number (a **code point**). `₹` is code point 8377.
- **UTF-8** is the most common way to turn those numbers into bytes. English letters take 1 byte, `é` takes 2, `₹` takes 3, most emoji take 4. It is the standard for the web and for new files.
- `str.encode("utf-8")` gives `bytes`. `bytes.decode("utf-8")` gives `str`. If the code book is wrong, you get `UnicodeDecodeError`, or silent garbage ("mojibake").
- `open(path, encoding="utf-8")` always names the encoding explicitly. Do not trust the default, it differs between systems.
- The decode error policy: `errors="replace"` puts `�` in place of bad bytes, `errors="ignore"` drops them. Both hide problems, so use them only when you accept data loss.

Worked example 3.3.1: mojibake and error policies

Predict: If we encode `"café ₹"` as UTF-8 and then wrongly decode it with Latin-1 (an older code book), do we get an error or garbage?

```
s = "café ₹"
b = s.encode("utf-8")
print(b, len(s), len(b))
print(b.decode("latin-1"))
print(b"caf\xe9".decode("latin-1"))
try: b"caf\xe9".decode("utf-8")
except UnicodeDecodeError as e: print("UnicodeDecodeError:", e)
print(b"caf\xe9".decode("utf-8", errors="replace"), b"caf\xe9".decode("utf-8", errors="ignore"))
```

Real output:

```
b'caf\xc3\xa9 \xe2\x82\xb9' 6 9
cafÃ© â¹
café
UnicodeDecodeError: 'utf-8' codec can't decode byte 0xe9 in position 3: unexpected end of data
caf caf
```

- 6 characters become 9 bytes (`é` is 2 bytes and `₹` is 3).
- Decoding those bytes with the wrong code book (Latin-1) does **not** fail. It silently gives `cafÃ© â¹`. That is **mojibake**, and it is worse than an error because nobody notices. If you see `Ã©` in your data, someone read UTF-8 as Latin-1.
- The byte `\xe9` is the letter `é` in Latin-1, but in UTF-8 it is the start of a longer character, so UTF-8 raises an error.
- `errors="replace"` gave `caf` and `errors="ignore"` gave `caf`. Both lose data.

Worked example 3.3.2: files and look-alike characters

Predict: Two strings both look like `é`. Are they equal?

```
import tempfile, os, unicodedata
with tempfile.TemporaryDirectory() as d:
    p = os.path.join(d, "x.txt")
    with open(p, "w", encoding="utf-8") as f: f.write("café ₹\n")
    print(open(p, "rb").read())
    print(open(p, encoding="utf-8").read().strip())
    print(open(p, encoding="latin-1").read().strip())
a = "e\u0301"   # letter e + combining accent
b = "\u00e9"    # one single character
print(a, b, a == b, len(a), len(b), unicodedata.normalize("NFC", a) == b)
```

Real output:

```
b'caf\xc3\xa9 \xe2\x82\xb9\n'
café ₹
cafÃ© â¹
é é False 2 1 True
```

- The file on disk holds UTF-8 bytes. Reading it with `encoding="utf-8"` gives the right text. Reading it as Latin-1 gives mojibake again.
- `a` and `b` print identically but are **different strings**: one is two code points, the other one. `==` is `False`. `unicodedata.normalize("NFC", ...)` converts both to one standard form so they compare equal. Normalise text before you compare or deduplicate it.

The picture

```
 "é"  --encode("utf-8")-->  b'\xc3\xa9'   --decode("utf-8")-->  "é"   (correct)
                              |
                              +--decode("latin-1")-->  "Ã©"      (mojibake)
```

The same two bytes can mean different text, depending on which code book you use to read them. So the reader must know the encoding. UTF-8 is the safe default.

Real-world mapping

Situation	Symptom	Fix
CSV exported from Excel on Windows	`UnicodeDecodeError`	try `encoding="utf-8-sig"` or `"cp1252"`
Names in two systems	`José` fails to match `José`	`unicodedata.normalize("NFC", s)`
API returns bytes	you see `b'...'`	`.decode("utf-8")`
Text shows `Ã©`	wrong decoding upstream	find where UTF-8 was read as Latin-1

Applications

- **Data Engineering:** ingesting files from many source systems with different encodings.
- **Data Science / NLP:** cleaning text before tokenising; deduplicating names.
- **GenAI:** token counts depend on bytes and characters, and prompts contain non-English text.

COMMON MISTAKE

Using `errors="ignore"` to make an error go away. You hide data loss. Find the true encoding first (ask the source team, or check the file's byte-order mark).

Check yourself

1. Why does reading UTF-8 text as Latin-1 not raise an error?

Answer: Latin-1 assigns a character to every byte value, so any bytes decode. The text is simply wrong.

1. What is the safest way to open a text file?

Answer: `open(path, encoding="utf-8")`, naming the encoding every time.

1. Why can `"é" == "é"` be `False`?

Answer: One may be a single code point and the other a letter plus a combining accent. Normalise with `unicodedata.normalize`.

3.4 Regular Expressions

INTUITION	`split` and `find` work when the text has a fixed shape ("cut at the comma"). But what if you must find "any 3 digits that stand alone" or "an e-mail address"? A **regular expression** (regex) is a mini language to describe the *shape* of text, like a search form: "digits, a dot, digits". You write the shape once, and the engine finds every piece of text with that shape.

Words you need

Word	Meaning
Pattern	the regex string, e.g. `r"\d+"`
Match	a piece of text that fits the pattern
Group	a part of the pattern in `( )` whose matched text you want back
Greedy	takes as much text as it can (default). **Lazy** takes as little as it can (add `?`)
Anchor	a position, not a character: `^` start, `$` end, `\b` word edge

The pattern language (the 20% that does 80%)

Piece	Meaning
`.`	any one character (except new line)
`\d` `\w` `\s`	a digit, a word character (letter, digit, `_`), a whitespace
`\D` `\W` `\S`	the opposite of the above
`[abc]` `[a-z]` `[^abc]`	one of those characters / a range / none of those
`*` `+` `?`	0 or more, 1 or more, 0 or 1
`{3}` `{2,4}`	exactly 3, between 2 and 4
`a|b`	
`( )` and `(?P<name>)`	a group, and a named group
`^` `$` `\b`	start of text, end of text, word edge
`(?<=x)` `(?=x)`	look-behind and look-ahead: "preceded by x" / "followed by x", without taking x
`\1`	the same text as group 1 again (back-reference)

Always write patterns as **raw strings** (`r"..."`) so Python does not eat the backslashes.

The functions in module `re`:

Function	What it does
`re.search(p, s)`	first match anywhere, or `None`
`re.match(p, s)`	match only at the start
`re.fullmatch(p, s)`	the whole string must match

Function	What it does
`re.findall(p, s)`	list of all matches (or of group contents)
`re.finditer(p, s)`	iterator of match objects
`re.sub(p, repl, s)`	replace matches; `repl` may be text or a function
`re.split(p, s)`	split at matches
`re.compile(p, flags)`	prepare a pattern once and reuse it

Flags: `re.I` ignore case, `re.M` `^` and `$` per line, `re.S` dot matches new line, `re.X` verbose (allows spaces and comments in the pattern).

Worked example 3.4.1: find things in the log line

Predict: `re.findall(r"\b\d{3}\b", log)` looks for stand-alone 3-digit numbers. Which values will it return?

```
import re
log = '10.0.0.5 - - [07/Nov/2026:10:15:32 +0530] "GET /api/orders/42 HTTP/1.1" 200 512 0.480'
print(re.search(r"\d+", log).group(), re.match(r"\d+", log).group(), re.match(r"GET", log))
print(re.findall(r"\d+\.\d+\.\d+\.\d+", log))
print(re.findall(r"\b\d{3}\b", log))
```

Real output:

```
10 10 None
['10.0.0.5']
['200', '512', '480']
```

- `\d+` means "one or more digits". The first match is `10` at the very start. `re.match` also gives `10` because the log starts with digits. `re.match(r"GET", log)` is `None` because `GET` is not at the start. This is the main difference between `match` and `search`.
- `\d+\.\d+\.\d+\.\d+` is "digits dot digits dot digits dot digits": the IP address. The `\.` is a literal dot, because a bare `.` would mean "any character".
- `\b\d{3}\b` finds three digits with a word edge on both sides. It found `200` (status), `512` (bytes) and `480` (the digits after the dot in `0.480`). It did not return `2026` (4 digits), `+0530` (4 digits), or `42` (2 digits). Notice `480` was a **false positive**: the pattern knows nothing about meaning. That is why real parsing uses a full pattern with named groups, as below.

Worked example 3.4.2: named groups parse a log line

Predict: What will `m.groupdict()` contain for `status`: the number 200 or the text `'200'`?

```
m = re.search(r'"(?P<method>[A-Z]+) (?P<path>\S+) HTTP/[\d.]+" (?P<status>\d{3}) (?P<bytes>\d+) (?P<secs>[\d.]+)', log)
print(m.groupdict())
print(m.group("path"), m.start("path"), m.end("path"), m.span())
d = m.groupdict(); print(int(d["status"]) + 1, float(d["secs"])*1000)
```

Real output:

```
{'method': 'GET', 'path': '/api/orders/42', 'status': '200', 'bytes': '512', 'secs': '0.480'}
/api/orders/42 47 61 (42, 85)
201 480.0
```

- Regex always returns **text**. `status` is `'200'`, not the number 200. You convert it yourself (`int(...)`). This is the same lesson as Chapter 1.
- `(?P<name>...)` gives a group a name, so you read `m.group("path")` instead of counting brackets.
- `m.span()` is `(42, 85)`: the matched text starts at index 42 and ends before index 85. `m.start("path")` is 47.

- `float("0.480") * 1000 = 480.0` ms.

Problem 3.4.1 Parse the full log line into a record with typed values: IP, timestamp, method, path, status (int), bytes (int) and processing time in whole milliseconds. Then try three lines, one of them garbage, and count the parsed lines.

Step 1	Describe the line piece by piece: IP `\S+`, two ignored words `\S+ \S+`, timestamp inside `[ ]` as `[^\]]+` (any characters except `]`), request in quotes, status `\d{3}`, bytes `\d+`, seconds `[\d.]+`.
Step 2	Anchor with `^` and `$` so the whole line must match, and name every group.
Step 3	Convert: `int(status)`, `int(bytes)`, `round(float(secs) * 1000)`.
Step 4	Loop over lines and use the walrus `:=` (Chapter 2) to keep only lines that match.
Answer	The code below parses two lines, skips the garbage line, and produces `ms = 480` for the first.
Check	`float("0.480") * 1000` is 480.0, which rounds to 480. The two parsed status codes are `'200'` and `'500'`, and the sum of seconds is 0.480 + 3.100 = 3.58, matching the output.
Why it matters	This is the first step of every log-analysis pipeline. Skipping bad lines (instead of crashing) is what makes a parser usable on real files.

```
pat = re.compile(r'^(?P<ip>\S+) \S+ \S+ \[(?P<ts>[^\]]+)\] "(?P<method>[A-Z]+) (?P<path>\S+) [^"]+" (?P<status>\d{3}) (?
P<bytes>\d+) (?P<secs>[\d.]+)$')
rec = pat.match(log).groupdict()
rec["status"] = int(rec["status"]); rec["bytes"] = int(rec["bytes"]); rec["ms"] = round(float(rec.pop("secs"))*1000)
print(rec)
lines = [log, '10.0.0.7 - - [07/Nov/2026:10:15:40 +0530] "POST /api/pay HTTP/1.1" 500 87 3.100', 'garbage line']
parsed = [m.groupdict() for l in lines if (m := pat.match(l))]
print(len(parsed), [p["status"] for p in parsed], sum(float(p["secs"]) for p in parsed))
```

```
{'ip': '10.0.0.5', 'ts': '07/Nov/2026:10:15:32 +0530', 'method': 'GET', 'path': '/api/orders/42', 'status': 200, 'bytes':
512, 'ms': 480}
2 ['200', '500'] 3.58
```

Worked example 3.4.3: sub, split, greedy vs lazy, lookaround, flags

Predict: For `<b>bold</b> and <i>it</i>`, does `<.+>` find 4 tags or 1 piece?

```
print(re.sub(r"\d+\.\d+\.\d+\.\d+", "x.x.x.x", log)[:20])
print(re.sub(r"(\w+)@(\w+)\.com", r"\2:\1", "asha@acme.com, ravi@zeta.com"))
print(re.sub(r"\d+", lambda m: str(int(m.group())*2), "a1 b22 c333"))
print(re.split(r"[,;]\s*", "a, b;c,   d"), re.split(r"(,)", "a,b"))
text = "<b>bold</b> and <i>it</i>"
print(re.findall(r"<.+>", text), re.findall(r"<.+?>", text))
print(re.findall(r"^\w+", "ab cd\nef gh", re.M), re.findall(r"HELLO", "hello Hello", re.I))
print(re.findall(r"(?<=\$)\d+", "cost $45 and $120"), re.findall(r"\d+(?=%)", "5% of 20 and 30%"))
print(re.findall(r"(\w)\1", "aabbcd hello"), re.sub(r"\s+", " ", "a   b \n c"))
```

Real output:

```
x.x.x.x - - [07/Nov/
acme:asha, zeta:ravi
a2 b44 c666
['a', 'b', 'c', 'd'] ['a', ',', 'b']
['<b>bold</b> and <i>it</i>'] ['<b>', '</b>', '<i>', '</i>']
['ab', 'ef'] ['hello', 'Hello']
['45', '120'] ['5', '30']
['a', 'b', 'l'] a b c
```

- `sub` can use group numbers in the replacement (`\2:\1` swaps the parts) and can take a **function** that receives each match and returns the new text (`a1 b22 c333` becomes `a2 b44 c666`). A function is the tool for "replace using a calculation". Use `sub` to mask PII (personal data): replace IP addresses or phone numbers before sharing logs.
- Wrapping the separator in `( )` in `split` keeps it in the result.
- **Greedy** `<.+>` took as much as possible, from the first `<` to the very last `>`, giving one big piece. **Lazy** `<.+?>` stops at the first `>`, giving four tags.
- `re.M` makes `^` match at the start of every line (`ab` and `ef`). `re.I` ignores case.
- Look-behind `(?<=\$)\d+` finds numbers preceded by `$` without including the `$`. Look-ahead `\d+(?=%)` finds numbers followed by `%`: `5` and `30`, but not `20`.
- `(\w)\1` finds a character followed by itself: `a`, `b`, `l` (from `aa`, `bb`, `ll` in hello).

Worked example 3.4.4: validate, escape, verbose, and the slow-regex danger

Predict: Is `a.b+c@x.co.in` a valid e-mail for our simple pattern?

```
print(re.fullmatch(r"\d{10}", "9876543210") is not None, re.fullmatch(r"\d{10}", "98765432100") is not None)
email = re.compile(r"^[\w.+-]+@[\w-]+(\.[\w-]+)+$")
print([bool(email.match(x)) for x in ("a@b.com","a.b+c@x.co.in","bad@","@x.com")])
print(re.escape("a.b*c"), re.findall(re.escape("1.5"), "1.5 and 125"))
verbose = re.compile(r"""
   (?P<h>\d{2}):   # hours
   (?P<m>\d{2})    # minutes
""", re.X)
print(verbose.search("at 14:05 sharp").groupdict())
import time
t=time.perf_counter(); re.search(r"(a+)+b", "a"*24+"c"); bad_seconds = time.perf_counter()-t
t=time.perf_counter(); re.search(r"a+b", "a"*24+"c"); good_seconds = time.perf_counter()-t
print(bad_seconds > 0.05, good_seconds < 0.01)
```

Real output:

```
True False
[True, True, False, False]
a\.b\*c ['1.5']
{'h': '14', 'm': '05'}
True True
```

- `fullmatch` demands the whole string match. A 10-digit phone number passes, an 11-digit one does not.
- The e-mail pattern accepts the first two and rejects `bad@` and `@x.com`. Real e-mail rules are much more complicated, so this is a **sanity check**, not a full validator. The only real proof is to send an e-mail.
- `re.escape` adds backslashes so special characters are read literally. Use it when the search text comes from a user or a file, otherwise `1.5` would also match `125`.
- `re.X` lets you lay out a long pattern over several lines with comments. Do this for any pattern longer than a line.
- The last line is a warning. `(a+)+b` on `aaaa...c` makes the engine try an exponential number of ways to split the `a`s before giving up (**catastrophic backtracking**). Both checks printed `True`: the bad pattern took more than 0.05 seconds on just 25 characters (the exact time depends on your machine, and it doubles with every extra `a`), while the safe pattern took under 0.01 seconds. Avoid nested repeats like `(x+)+`. On web servers, a bad regex plus user input can freeze a service (a "ReDoS" attack).

The picture

```
Pattern:  \b \d{3} \b          Text:  ... "HTTP/1.1" 200 512 0.480
           |   |    |                               ^^^ ^^^     ^^^
        edge 3 digits edge                           |   |       |
                                                   match match  match (false positive!)

Groups:   "(?P<method>[A-Z]+) (?P<path>\S+)"   on   "GET /api/orders/42 ..."
                 |                 |
              'GET'           '/api/orders/42'
```

Think of the engine as a cursor that slides over the text. At each position it asks, "can the pattern start here?" It reports a match when yes, and moves on when no.

Real-world mapping

Regex idea	Example	Where it appears in the job
Named groups	parse an access log	Log analysis, data ingestion
`sub`	mask phone numbers or IPs	Privacy engineering (PII masking)
`fullmatch`	validate an ID format	Data validation rules
`findall`	pull all hashtags or numbers from text	NLP preprocessing, feature extraction
lazy `.+?`	extract text between HTML tags	Quick scrapers (but prefer a real parser for HTML)

Applications

- **Data Analytics:** extract numbers, IDs and dates from free-text columns.
- **Data Engineering:** parse semi-structured logs into tables; validate file names.
- **Data Science / NLP:** cleaning text, finding patterns before modelling.
- **MLOps / Security:** scan code or logs for leaked secrets with patterns (Chapter 13).
- **Retail / Operations Analytics:** pull `order_id` and `delivery_minutes` out of application logs to join with the orders table.

COMMON MISTAKE

Parsing HTML or JSON with regex. They have nested structure that regex cannot handle reliably. Use `html.parser` or BeautifulSoup for HTML, and `json` for JSON. Another mistake: forgetting the raw string (`"\d"` instead of `r"\d"`), which gives a warning in recent Python versions. And forgetting that `.` needs `\.` when you mean a literal dot.

PROFESSOR'S NOTE

Build a regex in small steps. Start with one piece, test it in the REPL, add the next piece, test again. Never write a 100-character regex in one go. Try `re.X` and comments to keep it readable.

Check yourself

1. What is the difference between `re.match` and `re.search`?

Answer: `match` only tries at the start of the string. `search` looks anywhere.

1. What does `.+?` do differently from `.+`?

Answer: `.+` is greedy (takes as much as possible). `.+?` is lazy (as little as possible).

1. Why should you wrap user-supplied search text in `re.escape`?

Answer: So characters like `.` and `*` are read as plain characters and not as regex operators.

3.5 Regex in Depth, Unicode Forms and Text Tools

Section 3.4 gave you the core of regular expressions. This section adds the tools that make them readable and safe (named groups, lookarounds, verbose mode), the one regex failure that can take a server down (catastrophic backtracking), and three standard-library helpers for text: `unicodedata`, `textwrap` and `string.Template`.

Named groups, lookarounds and greedy versus lazy

A **named group** `(?P<name>...)` lets you read a match by name instead of by number, so the pattern survives edits. A **lookahead** `(?=...)` asks "is this text coming next?" without consuming it. A **lookbehind** `(?<=...)` asks "was this text just before?". A **negative lookahead** `(?!...)` asks "is this NOT coming next?". Quantifiers are **greedy** by default (take as much as possible); adding `?` makes them **lazy** (take as little as possible).

Problem 3.5.1 From `"2026-03-05 ERROR disk=/dev/sda1 used=93%"` extract the date, level and percent by name, then rewrite the date as day/month/year.

Pattern	`(?P<date>\d{4}-\d{2}-\d{2}) (?P<level>[A-Z]+) .* used=(?P<pct>\d+)%`
Read	`m.group("date")`, or `m["pct"]`, or `m.groupdict()` for all.
Rewrite	`re.sub` with `\g<d>/\g<m>/\g<y>` in the replacement reorders named groups.

```
import re
log = "2026-03-05 ERROR disk=/dev/sda1 used=93%"
m = re.search(r"(?P<date>\d{4}-\d{2}-\d{2}) (?P<level>[A-Z]+) .* used=(?P<pct>\d+)%", log)
print(m.group("date"), m.group("level"), int(m["pct"]))
print(m.groupdict())
print(re.sub(r"(?P<y>\d{4})-(?P<m>\d{2})-(?P<d>\d{2})", r"\g<d>/\g<m>/\g<y>", log[:10]))
```

```
2026-03-05 ERROR 93
{'date': '2026-03-05', 'level': 'ERROR', 'pct': '93'}
05/03/2026
```

Problem 3.5.2 In `"cost: $40, tax: $3, fee: 5 EUR, 7 USD"` find (a) numbers followed by " USD", (b) numbers right after a dollar sign, (c) words before a colon except "tax". Then show greedy versus lazy on `"<b>one</b> and <b>two</b>"`.

(a)	`\d+(?= USD)`: digits, then a lookahead for " USD" that is not part of the match.
(b)	`(?<=\$)\d+`: digits that have `$` immediately before them. The `$` is not in the match.
(c)	`\b(?!tax)\w+(?=:)`: at a word start, refuse if "tax" follows, take the word, require a colon after it.
Greedy	`<b>.*</b>` runs to the LAST closing tag, so it returns one long match. Lazy `.*?` stops at the first.

```
import re
prices = "cost: $40, tax: $3, fee: 5 EUR, 7 USD"
print(re.findall(r"\d+(?= USD)", prices))
print(re.findall(r"(?<=\$)\d+", prices))
print(re.findall(r"\b(?!tax)\w+(?=:)", prices))
html = "<b>one</b> and <b>two</b>"
print(re.findall(r"<b>.*</b>", html))
print(re.findall(r"<b>.*?</b>", html))
```

```
['7']
['40', '3']
['cost', 'fee']
['<b>one</b> and <b>two</b>']
['<b>one</b>', '<b>two</b>']
```

Problem 3.5.3 Make a long pattern readable with `re.VERBOSE`, which ignores whitespace and allows `#` comments.

```
import re
pat = re.compile(r"""
    (?P<user>[\w.]+)   # name part
    @
    (?P<host>[\w.-]+)  # domain
    """, re.VERBOSE)
print(pat.fullmatch("ada.l@example.org").groupdict())
```

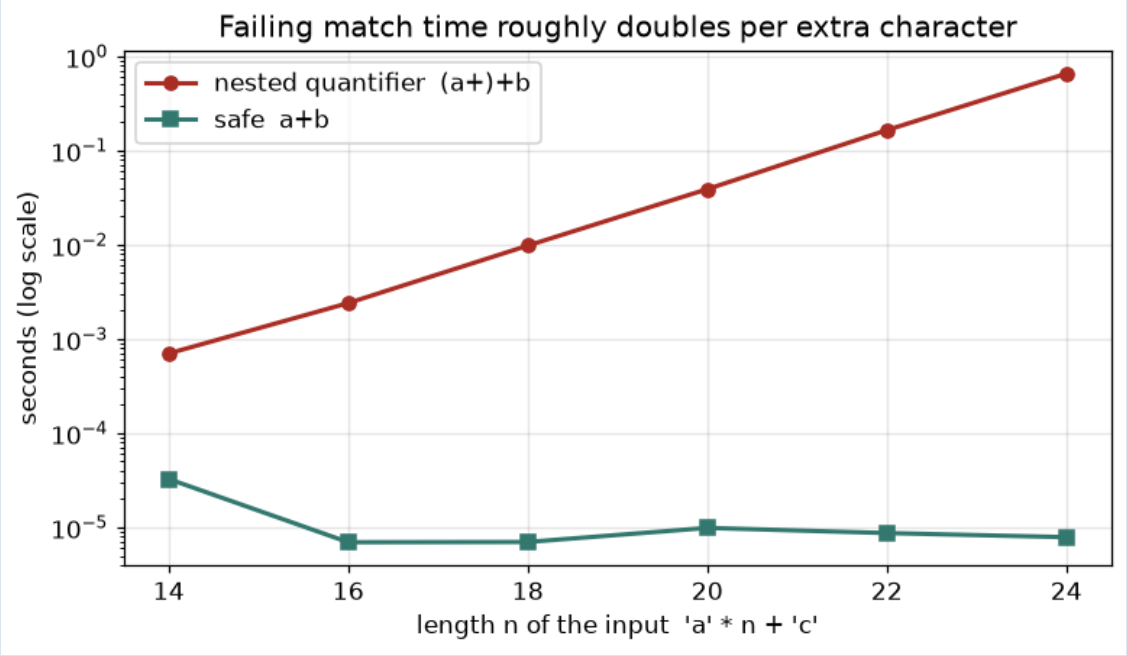
```
{'user': 'ada.l', 'host': 'example.org'}
```

COMMON MISTAKE

Parsing HTML or nested structures with regex, and using a greedy `.*` across a whole line. The greedy example above swallowed two tags into one match. For HTML use a parser; for simple single-line fields use lazy quantifiers or a negated class such as `[^<]*`.

Catastrophic backtracking (ReDoS)

Python's `re` tries alternatives one by one and goes back (**backtracks**) when one fails. A pattern with a quantifier inside a quantifier, such as `(a+)+b`, can split the same run of letters in exponentially many ways. On input that almost matches (many `a` then a `c`) the engine tries every split before giving up. A **ReDoS** (regular expression denial of service) is an attack that sends such input to a server. The safe pattern `a+b` fails instantly on the same text.



*Time to fail a match on 'a' * n + 'c'. Measured on the machine that built this book; your absolute times will differ, the shape will not.*

How to read it: the vertical axis is logarithmic, so a straight rising line means exponential growth: each extra character multiplies the time by a roughly constant factor (about 2 here). The flat green line is the safe pattern. Going from 14 to 24 characters costs the red pattern hundreds of times more, and if the trend continued, 40 characters would take many hours.

```
import re, time
def steps(pattern, n):
    s = "a" * n + "c"
    t = time.perf_counter()
    re.match(pattern, s)
    return time.perf_counter() - t
slow = [steps(r"(a+)+b", n) for n in (16, 20, 22)]
print(slow[2] / slow[0] > 20)
print(steps(r"a+b", 5000) < 0.01)
```

```
True
True
```

The first line says that going from 16 to 22 characters made the nested pattern more than 20 times slower. The second says the safe pattern handled 5,000 characters in under 10 milliseconds. Defences: avoid nested quantifiers, limit input length before matching, and for untrusted input consider a linear-time engine such as the third-party `re2` bindings (not run here).

Unicode forms, wrapping and templates

The same visible text can be stored in different ways. "é" can be one code point (`U+00E9`) or an "e" followed by a combining accent (`U+0301`). They look identical but compare as different. `unicodedata.normalize` converts to a standard form: **NFC** composes (one code point where possible) and **NFD** decomposes. Always normalise before comparing or deduplicating user text. `str.casefold()` is the aggressive lowercase for comparisons: it turns `"ß"` into `"ss"` , which `lower()` does not.

Problem 3.5.4 Compare two spellings of "café", normalise, strip accents, then wrap and template some text.

Step 1	`"caf\u00e9"` has 4 code points, `"cafe\u0301"` has 5 (both print as "café"), and `==` says `False` .
Step 2	After NFC both have 4 and compare equal.
Step 3	NFD splits "é" into "e" plus a combining mark; dropping marks (`unicodedata.combining(c)` is non-zero) leaves "cafe".
Step 4	`textwrap.fill` breaks lines at a width; `shorten` cuts to a width with a placeholder. `Template` uses `$name` and is safe for user-supplied templates because it cannot run code.

```
import unicodedata, textwrap
from string import Template
a = "caf\u00e9"      # one code point for e-acute
b = "cafe\u0301"     # e plus a combining accent
print(a == b, len(a), len(b))
na, nb = (unicodedata.normalize("NFC", s) for s in (a, b))
print(na == nb, len(na), len(nb))
d = unicodedata.normalize("NFD", a)
print(len(d), [unicodedata.name(c) for c in d[-2:]])
print("".join(c for c in d if not unicodedata.combining(c)))
print("ss" == "ß".casefold(), "ß".lower() == "ss")
text = "Python makes it easy to wrap long lines of text for terminals and emails."
print(textwrap.fill(text, width=30))
print(textwrap.shorten(text, width=30, placeholder="..."))
t = Template("Dear $name, you owe $$amount.")
print(t.substitute(name="Ada", amount=40))
print(t.safe_substitute(name="Ada"))
```

```
False 4 5
True 4 4
5 ['LATIN SMALL LETTER E', 'COMBINING ACUTE ACCENT']
cafe
True False
Python makes it easy to wrap
long lines of text for
terminals and emails.
Python makes it easy to...
Dear Ada, you owe $40.
Dear Ada, you owe $$amount.
```

`$$` in a template is a literal dollar sign. `safe_substitute` leaves unknown names untouched instead of raising `KeyError`, which is why the last line still shows `$amount`.

KEY IDEA

Text equality is about code points, not looks. Normalise (NFC), then casefold, then compare. Do this for usernames, e-mail addresses and search keys, or two "identical" users will be stored separately.

Chapter recap

- A string is an immutable sequence of characters. Index from 0, slice with `s[start:stop:step]`, and remember `stop` is not included.
- String methods return new strings. Learn `strip`, `split`, `join`, `replace`, `find`, `startswith`, `lower`, `casefold` first.
- Build big strings with `"".join(list)`, not with `+=` in a loop.
- Use f-strings for output. The format spec `{value:>10,.2f}` controls alignment, separator, decimals and type.
- Text is `str`, data is `bytes`. Always name the encoding, and use UTF-8.
- If you see `Ã©`, UTF-8 was read as Latin-1. If two identical-looking strings differ, normalise with `unicodedata`.
- Regex describes the shape of text. Use raw strings, named groups, `fullmatch` to validate, `sub` with a function to transform.
- Regex returns text. Convert to numbers yourself. Do not use regex for HTML or JSON.
- Avoid nested repeats such as `(a+)+`: they can freeze a program.

Pattern	Example
Slice	`s[2:5]`, `s[-3:]`, `s[::-1]`
Clean and split	`s.strip().split(",")`
Join	`",".join(parts)`
Format	`f"{x:,.2f}"`, `f"{p:.1%}"`, `f"{n:05d}"`
Encode / decode	`s.encode("utf-8")`, `b.decode("utf-8")`
Find one	`re.search(r"(?P<n>\d+)", s)`
Find all	`re.findall(r"\d+", s)`
Validate	`re.fullmatch(r"\d{10}", s)`
Replace	`re.sub(r"\s+", " ", s)`
Compile once	`pat = re.compile(r"...", re.X)`

Practice set

Problem 3.P.1 (easy) What is `"performance"[2:6]` ?

Answer: `"rfor"`. Indexes 2, 3, 4, 5 are `r`, `f`, `o`, `r`.

Problem 3.P.2 (easy) What is `"a-b-c".split("-", 1)` ?

Answer: `['a', 'b-c']`. The second argument is the maximum number of cuts.

Problem 3.P.3 (easy) Write an f-string that prints `0.4567` as a percentage with one decimal.

Answer: `f"{0.4567:.1%}"` gives `45.7%`.

Problem 3.P.4 (what is the output?)

```
s = "hello"
s.upper()
print(s)
```

Answer: `hello`. `upper()` returns a new string and we threw it away. Use `s = s.upper()`.

Problem 3.P.5 (what is the output?) `import re; print(re.findall(r"\d+", "a12b345c6"))`

Answer: `['12', '345', '6']`. Each run of digits is one match.

Problem 3.P.6 (computation) How many bytes does `"₹100"` take in UTF-8?

Answer: 6 bytes. `₹` is 3 bytes, and each of the digits `1`, `0`, `0` is 1 byte: 3 + 3 = 6.

Problem 3.P.7 (find the bug)

```
import re
print(re.findall("\d+\.\d+", "version 3.14 and 2.7"))
```

Answer: The pattern is not a raw string. Recent Python versions warn about the invalid escape `\d`. Use `r"\d+\.\d+"`. The result is `['3.14', '2.7']`.

Problem 3.P.8 (find the bug) A regex to get the text inside the first bracket pair from `"[a] and [b]"` returns the wrong text:

```
re.search(r"\[(.+)\]", "[a] and [b]").group(1)
```

Answer: It returns `a] and [b` because `.+` is greedy. Use the lazy form `r"\[(.+?)\]"`, which returns `a`.

Problem 3.P.9 (scenario) You read a CSV from a partner and names like `Zoë` show as `ZoÃ«`. What happened and what do you do?

Answer: The file is UTF-8 but was read as Latin-1 (or similar). Re-read with `open(path, encoding="utf-8")` (or `pandas.read_csv(..., encoding="utf-8")`). Do not "fix" it with `errors="ignore"`.

Problem 3.P.10 (scenario) Write one line that extracts all status codes (3-digit numbers after a closing quote and a space) from a log string `text`.

Answer: `re.findall(r'" (\d{3}) ', text)`. For our sample line it returns `['200']`. The group in `( )` makes `findall` return only the digits.

Problem 3.P.11 (interview) "What is the difference between `str` and `bytes` in Python 3?"

Answer: `str` is text, a sequence of Unicode characters. `bytes` is raw binary data, a sequence of numbers 0-255. You convert with `encode` (str to bytes) and `decode` (bytes to str), and must name an encoding such as UTF-8.

Problem 3.P.12 (interview) "Why can a regex be dangerous in production, and how would you reduce the risk?"

Answer: A pattern with nested repeats, like `(a+)+b`, can take exponential time on some inputs (catastrophic backtracking), which can freeze a service. Avoid nested quantifiers, keep patterns simple, limit input length, use `fullmatch` with anchors, and test with long non-matching input.

More practice: build, stretch and review

Problem 3.P.12 (build) Write `slugify(title)`: remove accents, lowercase, replace every run of characters that are not letters or digits with a single `-`, and strip `-` from both ends. Test `"Crème Brûlée & Café!"`.

```
import re, unicodedata
def slugify(title):
    d = unicodedata.normalize("NFKD", title)
    ascii_text = "".join(c for c in d if not unicodedata.combining(c))
    return re.sub(r"[^a-z0-9]+", "-", ascii_text.lower()).strip("-")
print(slugify("Crème Brûlée & Café!"))
```

```
creme-brulee-cafe
```

Answer: as shown. NFKD splits accented letters into base plus mark, the generator drops the marks, and the regex turns "space, ampersand, space" into one hyphen. The trailing "!" becomes a final hyphen that `strip("-")` removes. Self-check: does an input of only punctuation give an empty string?

Problem 3.P.13 (find the bug) A teammate finds decimals with `re.findall(r"\d+.\d+", text)`. On `"1.5 and 2x7"` it returns two matches. Why, and what is the fix?

```
import re
print(re.findall(r"\d+.\d+", "1.5 and 2x7"), re.findall(r"\d+\.\d+", "1.5 and 2x7"))
```

```
['1.5', '2x7'] ['1.5']
```

Answer: An unescaped `.` means "any character", so `2x7` matches. Escape it as `\.` to mean a literal dot.

Problem 3.P.14 (build) Turn `"user=ada level=3 ok=yes"` into a dictionary using one `findall`.

```
import re
line = "user=ada level=3 ok=yes"
print(dict(re.findall(r"(\w+)=(\w+)", line)))
```

```
{'user': 'ada', 'level': '3', 'ok': 'yes'}
```

Answer: With two groups, `findall` returns a list of pairs, and `dict` turns pairs into keys and values. Values stay strings, so convert `level` with `int` if you need a number.

Problem 3.P.15 (interview) Your login form validates the username with `^(\w+\s?)+$`. A security reviewer says it is a ReDoS risk. Explain, and give two fixes.

Answer: `(\w+\s?)+` has a quantifier inside a quantifier, and the optional `\s?` means a run of letters can be split into groups in exponentially many ways. A long name ending in an illegal character (for example `"aaaa...a!"`) forces the engine to try all splits before failing. Fixes: (1) rewrite so each position has only one way to match, such as `^\w+(\s\w+)*$`; (2) check `len(name) <= 64` before running the regex.

Problem 3.P.16 (stretch, rubric) Write a function `parse_log_line(line)` that uses one verbose, named-group regex to parse `"2026-03-05 ERROR disk=/dev/sda1 used=93%"` into a dict with int `pct`, and returns `None` for a line that does not match. Rubric (1 point each): (1) `re.VERBOSE` with

comments; (2) named groups; (3) `fullmatch` or anchors so junk is rejected; (4) returns `None`, never raises, on bad lines; (5) you tested three good lines and two bad ones.

Answer: No single answer; 5/5 is full marks. Use Problem 3.5.1 as the starting pattern and add the disk part as another named group.

Prompt pack

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data roles, and English is my second language, so use simple words and short sentences. Teach me Python string slicing and the 15 most useful string methods for cleaning text. Use a web-server log line as the running example. For each method show a small example, ask me to predict the output first, and explain with a picture of index positions. End with three questions to check me.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data roles, and English is my second language. Teach me Python f-string formatting: alignment, width, decimals, thousands separator, percent, zero padding and the {x=} debug form. Give me 8 graded exercises, from easy to hard, where I must write the format string for a given output. Do not give solutions until I answer. Then review my answers and explain mistakes in simple words.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data roles, with English as a second language. Explain text encoding in Python: Unicode, UTF-8, bytes vs str, mojibake, UnicodeDecodeError, errors="replace", and Unicode normalisation. Use real examples such as a CSV with Indian names. Then quiz me one question at a time, wait for my answer, and explain why each answer is right or wrong.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data roles, and English is my second language. Teach me regular expressions in Python from zero: character classes, quantifiers, groups, named groups, greedy vs lazy, anchors, look-ahead, re.sub with a function, and catastrophic backtracking. Build one pattern step by step to parse an access-log line. Then give me 6 graded regex exercises with no solutions. Here is a pattern I am stuck on: [PASTE HERE]
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data roles, with English as a second language. Run a mock interview on Python text processing: strings, immutability, f-strings, encoding, and regular expressions. Ask one question at a time, include two small coding questions that I answer by typing code, give a score from 1 to 5 for each answer, show a model answer, and finish with a list of my weak points and what to practise next.
```

CHAPTER 4

COLLECTIONS, COMPREHENSIONS AND THE STANDARD-LIBRARY TOOLBOX

Choose the right container, and let `collections`, `itertools` and `functools` do the heavy lifting.

An online shop records thousands of delivery times. To answer "which product is delivered slowest?" you must group numbers by item, count them, sort them, remove duplicates and find the top few. In Python every one of those jobs has a ready-made container or tool, and picking the right one can make a program 100 times faster or ten times shorter. In this chapter you learn the four core containers (list, tuple, set, dict), the one-line loops called comprehensions, and three standard-library modules that professionals use every day. By the end you will group, count, rank and deduplicate data without writing clumsy loops.

Running example. The same shop delivery records as before, as `(item, delivery_mins)` pairs, where the number is minutes from order to doorstep. Keep this table in your head:

item	delivery_mins
book	480
lamp	3100
book	310
lamp	60
shoes	1250
book	290

4.1 Lists and Tuples: Ordered Sequences

INTUITION

A list is a shopping list on paper: you can add items, cross them out, reorder them. A tuple is a printed receipt: it records exactly what was bought, and you do not edit it. Use a list for "a pile of similar things that may change", and a tuple for "one record with fixed parts".

Formal definition

- **list** `[a, b, c]`: ordered, **mutable**, can hold anything (even mixed types). Indexing and slicing work exactly like strings (Chapter 3).
- **tuple** `(a, b)`: ordered, **immutable**. A one-item tuple needs a trailing comma: `(5,)`. Without it, `(5)` is just the number 5.
- **namedtuple**: a tuple whose fields also have names (`s.item`).
- Both support `len`, `in`, `+`, `*`, `min`, `max`, `sum` (numbers), `count`, `index`.

Main list methods:

Method	What it does	Returns
`append(x)`	add one item at the end	`None`
`extend(iterable)`	add many items at the end	`None`
`insert(i, x)`	insert before position `i`	`None`
`pop()` / `pop(i)`	remove and return the last / the i-th item	the item

Method	What it does	Returns
`remove(x)`	remove the first item equal to `x`	`None`
`sort(key=, reverse=)`	sort **in place**	`None`
`sorted(list)`	a **new** sorted list	new list
`reverse()`	reverse in place	`None`

Notice that methods that change the list in place return `None`. This is a deliberate signal: "I changed the thing, there is no new thing."

Worked example 4.1.1: a list of delivery times

Predict: After `mins.sort(reverse=True)`, which index does the value `480` have?

```
mins = [480, 1250, 3100, 310, 60]
mins.append(95)
mins.extend([700, 220])
mins.insert(0, 5)
print(mins, len(mins))
print(mins[0], mins[-1], mins[1:4])
last = mins.pop()
first = mins.pop(0)
mins.remove(95)
print(last, first, mins)
copy_sorted = sorted(mins)
print(copy_sorted, mins)
mins.sort(reverse=True)
print(mins, mins.index(480), mins.count(60), min(mins), max(mins), sum(mins))
try:
    mins.remove(9999)
except ValueError as e:
    print("ValueError:", e)
try:
    mins[100]
except IndexError as e:
    print("IndexError:", e)
```

Real output:

```
[5, 480, 1250, 3100, 310, 60, 95, 700, 220] 9
5 220 [480, 1250, 3100]
220 5 [480, 1250, 3100, 310, 60, 700]
[60, 310, 480, 700, 1250, 3100] [480, 1250, 3100, 310, 60, 700]
[3100, 1250, 700, 480, 310, 60] 3 1 60 3100 5900
ValueError: list.remove(x): x not in list
IndexError: list index out of range
```

Step by step:

- After four changes the list has 9 items. `insert(0, 5)` put the `5` in front.
- `pop()` removed `220` (last), `pop(0)` removed `5` (first), `remove(95)` deleted the value 95. Six items remain.
- `sorted(mins)` made a **new** list and left `mins` unchanged. `mins.sort(reverse=True)` changed `mins` itself. After sorting, `480` is at index 3 (after 3100, 1250, 700).
- Check the sum: 3100 + 1250 + 700 + 480 + 310 + 60 = 5900. Matches.
- Asking for a missing value or a too-big index raises `ValueError` or `IndexError`. Read the message: it tells you what was missing.

Worked example 4.1.2: tuples and named tuples

```
pt = (480, "book")
one = (5,)
print(type((5)), type(one), pt[0])
item_mins, name = pt
from collections import namedtuple
S = namedtuple("S", "item mins")
s = S("book", 480)
print(s, s.item, s[1], s._asdict())
```

Real output:

```
<class 'int'> <class 'tuple'> 480
S(item='book', mins=480) book 480 {'item': 'book', 'mins': 480}
```

- `(5)` is an `int`; `(5,)` is a tuple. The comma makes the tuple, not the brackets.
- The `namedtuple` lets you read `s.item` instead of `s[0]`. Code with names is much easier to read. In Chapter 8 you will meet `dataclass`, which is the more modern choice for records.
- Tuples can be **dictionary keys** and **set members** (lists cannot), because they never change.

The picture

```
 list   [ 480 | 1250 | 3100 | 310 | 60 ]     can grow, shrink, reorder
 index     0      1      2      3     4
 neg      -5     -4     -3     -2    -1

 tuple  ( "book", 480 )                      fixed shape, one record
```

A list is a row of numbered boxes. Python stores in each box not the value itself but a reference to it (Chapter 1), so a list of 1,000 items holds 1,000 arrows.

Real-world mapping

Container	Example	Where it appears in the job
list	all delivery times of one day	Column of data before it becomes a DataFrame
tuple	`(item, mins)` one row	Rows returned by a database cursor
namedtuple	`S("book", 480)`	Light records with readable fields
list as stack	`append` / `pop`	Undo, parsing nested brackets

Applications

- **Data Analytics:** rows read from a file are a list of tuples.
- **Data Engineering:** batches of records are lists; database rows are tuples.
- **Machine Learning:** a list of feature names; a tuple for an array shape such as `(1000, 20)`.
- **Operations Analytics:** the list of delivery times is the input of every percentile.

COMMON MISTAKE

Writing `sorted_list = mins.sort()`. `sort()` returns `None`, so `sorted_list` becomes `None`. Use `sorted(mins)` when you want a new list. Another: `list.pop(0)` and `list.insert(0, x)` on a huge list are slow (every item shifts). Use `deque` (Section 4.4).

Check yourself

1. What is `type((7))` and `type((7,))`?

Answer: `int` and `tuple`.

1. What does `[1, 2, 3].append(4)` return?

Answer: `None`. It changes the list in place.

1. Why can a tuple be a dictionary key but a list cannot?

Answer: A tuple is immutable (hashable); a list can change, so Python refuses to hash it.

4.2 Sets: Uniqueness and Fast Membership

INTUITION	A set is a guest list at the door. Each name is on it at most once, and the guard can check "is this person on the list?" instantly, without reading the whole list. The order of names does not matter.

Formal definition

- **set** `{a, b}`: unordered, no duplicates, mutable. An empty set is `set()`, because `{}` makes an empty **dict**.
- **frozenset**: an immutable set. It can be a dict key or a member of another set.
- Members must be **hashable**: Python can compute a fixed number (a **hash**) from them. Numbers, strings and tuples of those are hashable. Lists, sets and dicts are not.
- Set maths: `a | b` union, `a & b` intersection, `a - b` difference, `a ^ b` symmetric difference (in exactly one), `a <= b` subset.
- Why fast? A set stores items in a **hash table**: the hash tells Python which "drawer" to open, so a lookup takes about the same time for 10 items or 10 million (called O(1), "constant time").

Worked example 4.2.1: set operations

Predict: `a` has three product names and `b` has two. How many items are in `a ^ b`?

```
a = {"book", "shoes", "lamp"}
b = {"lamp", "pen"}
print(sorted(a | b), sorted(a & b), sorted(a - b), sorted(a ^ b))
print("lamp" in a, len(a), {1, 2} <= {1, 2, 3}, a.isdisjoint({"x"}))
print(set([1, 1, 2, 2, 3]), sorted(set("hello")))
print({frozenset([1, 2]): "ok"})
print(type({}), type(set()))
try:
    {[1, 2]}
except TypeError as e:
    print("TypeError:", e)
```

Real output:

```
['book', 'lamp', 'pen', 'shoes'] ['lamp'] ['book', 'shoes'] ['book', 'pen', 'shoes']
True 3 True True
{1, 2, 3} ['e', 'h', 'l', 'o']
{frozenset({1, 2}): 'ok'}
<class 'dict'> <class 'set'>
TypeError: cannot use 'list' as a set element (unhashable type: 'list')
```

- The union has 4 names. Only `lamp` is in both. `a ^ b` has 3 names: `book`, `shoes`, `pen` (the ones in exactly one set).

- I used `sorted(...)` when printing because the printed order of a set is not guaranteed and can change between runs for text values. Never depend on set order.
- `set("hello")` has only 4 letters, because the second `l` is dropped. `{}` is a `dict`, `set()` is the empty set.

Worked example 4.2.2: why sets are fast

Problem 4.2.1 You have 200,000 IDs and must ask "is 199,999 in the collection?" 200 times. Compare a list and a set.

Step 1	Build both: `big = list(range(200_000))` and `bigset = set(big)`.
Step 2	Time 200 lookups of the last item in each. In a list, Python compares the item with each element from the left: up to 200,000 comparisons per lookup. In a set it hashes the item once.
Step 3	Compare the two times as a ratio.
Answer	In my run the list was more than 100 times slower (the script printed `True` for "ratio above 100"). The exact ratio depends on your machine; I measured only that it exceeds 100.
Check	The code is below. Run it yourself and print `tl/ts` to see your own number.
Why it matters	A join or a filter such as "keep rows whose id is in this other list" can take minutes with a list and under a second with a set. This is the single most useful speed-up for a beginner.

```
import time
big = list(range(200_000)); bigset = set(big)
t=time.perf_counter(); [199_999 in big for _ in range(200)]; tl=time.perf_counter()-t
t=time.perf_counter(); [199_999 in bigset for _ in range(200)]; ts=time.perf_counter()-t
print("list slower than set by more than 100x:", tl/ts > 100)
```

```
list slower than set by more than 100x: True
```

The picture

```
 list lookup:   [ . . . . . . . . . X ]     walk from the left: many steps
 set  lookup:   hash("X") -> drawer 7 -> open drawer 7 -> found
```

Real-world mapping

Set idea	Example	Where it appears in the job
deduplicate	`set(user_ids)`	Unique visitors, distinct customers
membership test	`if id in blocked_ids`	Filtering rows quickly
`a - b`	products on the price list but never sold	Data reconciliation between two systems
`a & b`	customers in both files	Join checks

Applications

- **Data Analytics:** count distinct users; find users who bought in both months.
- **Data Engineering:** compare yesterday's and today's key sets to find new and deleted rows.
- **Machine Learning:** build a vocabulary of unique words; train/test leakage checks (`train_ids & test_ids` must be empty).
- **Security:** blocklists and allowlists.

COMMON MISTAKE	Using a list for repeated membership tests (`if x in big_list` inside a loop). Convert once to a set. Another: expecting a set to keep order, or writing `{}` to create an empty set.

Check yourself

1. How do you create an empty set?

Answer: `set()`. The literal `{}` is an empty dict.

1. What is `{1, 2, 3} & {2, 3, 4}`?

Answer: `{2, 3}`.

1. Why is `x in my_set` fast?

Answer: Python hashes `x` and jumps straight to the right slot of a hash table instead of scanning.

4.3 Dictionaries: Key-Value Lookup

INTUITION	A dictionary is a phone book. You do not read every page: you look up the name (the **key**) and get the number (the **value**). Because the lookup uses a hash table (the same trick as a set), it is instant even for millions of entries.

Formal definition

- **dict** `{key: value}`: maps keys to values. Keys must be hashable and unique. Values can be anything.
- Since Python 3.7 a dict **remembers insertion order**.
- `d[k]` raises `KeyError` if missing. `d.get(k)` returns `None`; `d.get(k, default)` returns your default.
- `d.setdefault(k, default)` returns `d[k]`, first inserting the default if the key is missing.
- `keys()`, `values()`, `items()` give **views** (live windows onto the dict). `items()` gives `(key, value)` pairs.
- Merge: `a | b` (right side wins on clashes), or `{**a, **b}`. `d.update(...)` changes in place.
- `pop(k)`, `del d[k]`, `d.fromkeys(keys, value)`, `dict(zip(keys, values))`.

Worked example 4.3.1: the basics

Predict: What does `stat.get("zzz")` return, and what does `stat["zzz"]` do?

```
stat = {"book": 480, "lamp": 3100}
stat["shoes"] = 1250
print(stat, stat["lamp"], stat.get("zzz"), stat.get("zzz", 0), len(stat))
try:
    stat["zzz"]
except KeyError as e:
    print("KeyError:", e)
print(list(stat), list(stat.values()), list(stat.items()))
print(stat.setdefault("pen", 0), stat)
print(stat.pop("pen"), stat.pop("nope", "none"))
print({"a": 1, "b": 2} | {"b": 3, "c": 4}, {**{"a": 1}, **{"a": 9}})
stat.update(lamp=2900)
del stat["book"]
print(stat, "lamp" in stat)
print(dict(zip("abc", [1, 2, 3])), dict.fromkeys(["a", "b"], 0))
print(sorted(stat.items(), key=lambda kv: kv[1]))
```

Real output:

```
{'book': 480, 'lamp': 3100, 'shoes': 1250} 3100 None 0 3
KeyError: 'zzz'
['book', 'lamp', 'shoes'] [480, 3100, 1250] [('book', 480), ('lamp', 3100), ('shoes', 1250)]
0 {'book': 480, 'lamp': 3100, 'shoes': 1250, 'pen': 0}
0 none
{'a': 1, 'b': 3, 'c': 4} {'a': 9}
{'lamp': 2900, 'shoes': 1250} True
{'a': 1, 'b': 2, 'c': 3} {'a': 0, 'b': 0}
[('shoes', 1250), ('lamp', 2900)]
```

Explanation:

- `d[k]` with a missing key gives `KeyError: 'zzz'`. `get` is the safe choice when "missing" is normal.
- `list(stat)` lists the keys (looping over a dict gives keys). `items()` gives pairs; use `for k, v in stat.items()`.
- In the merge, `b` appears in both dicts and the **right** one wins: `3`.
- `sorted(stat.items(), key=lambda kv: kv[1])` sorts pairs by value: 1250 before 2900. A dict itself has no sort; you sort its items.
- `"lamp" in stat` tests the **keys**, and is fast.

Worked example 4.3.2: grouping, the most common dictionary task

Problem 4.3.1 From the running table, group the elapsed times by item and compute the average per item.

Step 1	Start with an empty dict. For each `(item, mins)`, make sure the item has a list, then append `mins`. `setdefault(item, [])` does both in one line.
Step 2	After the loop, each item maps to its list: book → [480, 310, 290], lamp → [3100, 60], shoes → [1250].
Step 3	Average = sum / count. book: (480 + 310 + 290) / 3 = 1080 / 3 = 360. lamp: (3100 + 60) / 2 = 3160 / 2 = 1580. shoes: 1250 / 1 = 1250.
Answer	book averages 360 mins, lamp 1580 mins, shoes 1250 mins.
Check	The code prints exactly these values (`360.0`, `1580.0`, `1250.0`).
Why it matters	Interpretation: `lamp` has the worst average, but with only two samples (3100 and 60) the average hides a huge spread. Always look at the spread and the count, not just the average.

The code in 4.4 does the same with `defaultdict`.

The picture

```
key       value
"book"  ->  480
"lamp"    ->  3100          hash("lamp") picks the slot, so lookup is O(1)
"shoes" ->  1250
```

A dict is a two-column table where you can jump from a key to its row without scanning.

Real-world mapping

Dict idea	Example	Where it appears in the job
lookup table	`{"US": "United States"}`	Mapping codes to names
record	`{"id": 7, "name": "Asha"}`	One JSON object from an API
group-by	item → list of times	Aggregations before pandas

Dict idea	Example	Where it appears in the job
config	`{"timeout": 5}`	Settings passed into functions

Applications

- **Data Engineering:** every JSON document you read becomes dicts and lists.
- **Data Analytics:** `df.to_dict()`, and mapping values with `Series.map(dict)`.
- **Machine Learning:** hyper-parameter dictionaries; word-to-index vocabularies.
- **GenAI:** chat messages are lists of dicts (`{"role": ..., "content": ...}`).

COMMON MISTAKE

Changing the size of a dict while looping over it. Python stops you with `RuntimeError: dictionary changed size during iteration`. Loop over `list(d)` (a copy of the keys) or build a new dict. Another: using `d[k]` where `d.get(k)` is the right tool, and getting `KeyError` in production.

Check yourself

1. How do you read a key that may be missing, with a default of 0?

Answer: `d.get(key, 0)`.

1. In `{"a": 1} | {"a": 2}`, what is the result?

Answer: `{"a": 2}`. The right dict wins.

1. Does a dict keep insertion order?

Answer: Yes, since Python 3.7.

4.4 The `collections` Module and Friends

INTUITION

The built-in containers are hand tools. The `collections` module is the power-tool shelf: a dict that makes missing lists by itself (`defaultdict`), a tally counter (`Counter`), and a queue that is fast at both ends (`deque`).

Formal definition

Tool	What it is	Use it for
`defaultdict(factory)`	dict that creates a default value for a missing key by calling `factory`	grouping, counting
`Counter(iterable)`	dict subclass that counts items; `most_common(n)`	frequency tables
`deque(maxlen=n)`	double-ended queue; O(1) at both ends; optional fixed size	queues, sliding windows
`OrderedDict`	dict with extra order tools like `move_to_end`	LRU-style tricks (a normal dict already keeps order)
`heapq`	keeps the smallest item first (a **heap**)	top-N, schedulers
`bisect`	binary search in a sorted list	insert while keeping sorted order

Worked example 4.4.1: defaultdict, Counter, deque, heapq, bisect

Predict: After appending 0 to 4 into a `deque(maxlen=3)`, which numbers remain?

```
from collections import defaultdict, Counter, deque, OrderedDict
rows = [("book",480),("lamp",3100),("book",310),("lamp",60),("shoes",1250),("book",290)]
dd = defaultdict(list)
for l, v in rows:
    dd[l].append(v)
print(dict(dd))
print({k: sum(v)/len(v) for k, v in dd.items()})
c = Counter(l for l, _ in rows)
print(c, c.most_common(2), c["book"], c["zzz"])
print(Counter("abracadabra").most_common(3), Counter(a=3) + Counter(a=1, b=2))
q = deque(maxlen=3)
for x in range(5):
    q.append(x)
print(q)
print(q.popleft())
print(q)
q.appendleft(9)
print(q)
q.rotate(1)
print(q)
import heapq, bisect
h = [5, 1, 8, 3]
heapq.heapify(h)
print(heapq.heappop(h), heapq.nsmallest(2, [5, 1, 8, 3]), heapq.nlargest(2, [5, 1, 8, 3]))
srt = [10, 20, 30, 40]
print(bisect.bisect_left(srt, 25), bisect.bisect_right(srt, 30))
bisect.insort(srt, 25)
print(srt)
```

Real output:

```
{'book': [480, 310, 290], 'lamp': [3100, 60], 'shoes': [1250]}
{'book': 360.0, 'lamp': 1580.0, 'shoes': 1250.0}
Counter({'book': 3, 'lamp': 2, 'shoes': 1}) [('book', 3), ('lamp', 2)] 3 0
[('a', 5), ('b', 2), ('r', 2)] Counter({'a': 4, 'b': 2})
deque([2, 3, 4], maxlen=3)
2
deque([3, 4], maxlen=3)
deque([9, 3, 4], maxlen=3)
deque([4, 9, 3], maxlen=3)
1 [1, 3] [8, 5]
2 3
[10, 20, 25, 30, 40]
```

- `defaultdict(list)` creates an empty list the first time a key appears, so `dd[l].append(v)` needs no `setdefault`. The result is the same grouping as 4.3.1.
- `Counter` counts: `book` 3 times, `lamp` twice, `shoes` once. A missing key gives `0` (not an error). `most_common(2)` gives the top two. Counters can be added. In `"abracadabra"` the letter `a` appears 5 times (a-b-r-**a**-c-**a**-d-**a**-b-r-**a**: 5), `b` 2 times, `r` 2 times.
- A `deque` with `maxlen=3` keeps only the last 3 items: 2, 3, 4. This is a **sliding window**, handy for "average of the last 3 samples". `rotate(1)` moves items one place to the right.
- `heapq.heappop` returns the smallest item (1). `nsmallest(2, ...)` is `[1, 3]`, `nlargest(2, ...)` is `[8, 5]`.
- `bisect_left(srt, 25)` gives index 2: the place where 25 would go in the sorted list. `insort` inserts it there and keeps the list sorted.

The picture

```
 deque(maxlen=3)        append(3) ->  [1,2,3]
 oldest falls out  <--  append(4) ->  [2,3,4]   1 dropped from the left
```

Real-world mapping

Tool	Example	Where it appears in the job
`Counter`	count status codes per test run	Quick value counts, error summaries
`defaultdict`	group lines by session id	ETL grouping
`deque(maxlen)`	rolling window of last 100 delivery times	Moving averages, rate limits
`heapq.nlargest`	10 slowest deliveries	Top-N reports

Applications

- **Data Analytics:** `Counter` for quick frequency checks before using pandas.
- **Data Engineering:** `deque` as an in-memory buffer; `heapq` for merging sorted streams.
- **Machine Learning:** `Counter` for word counts and class balance.
- **Operations Analytics:** rolling windows of delivery times to detect a slow-down.

COMMON MISTAKE

Reading `defaultdict` entries just to test them (`if d[k]:`) creates the key as a side effect. Use `k in d` or `d.get(k)` for checks. Another: using `sort()` after every insert; use `heapq` or `bisect.insort` .

Check yourself

1. What does `Counter("aab")["z"]` return?

Answer: `0` . A missing key counts as zero.

1. What is the oldest item of `deque([1,2,3], maxlen=3)` after `append(4)` ?

Answer: `1` is dropped, so the deque is `[2, 3, 4]` .

1. When is `defaultdict(list)` better than `dict.setdefault` ?

Answer: When you group many items: it avoids repeating `setdefault` and reads more clearly.

4.5 Comprehensions and Generator Expressions

INTUITION

A comprehension is a factory line in one sentence: "from this input, take each item, keep only those that pass the test, transform them, and put them in a new container". It replaces a four-line loop with one line that reads like English.

Formal definition

- **List comprehension:** `[expression for item in iterable if condition]` .
- **Set comprehension:** `{expression for ...}` . **Dict comprehension:** `{key: value for ...}` .
- **Generator expression:** `(expression for ...)` . It does not build a container. It produces items **one at a time on demand** (Chapter 7), using almost no memory.
- A **nested comprehension** has two `for` parts. Read it left to right, like nested loops, with the outer loop first.
- The loop variable of a comprehension stays inside it (it does not leak, unlike a normal `for` loop).

Worked example 4.5.1: the common shapes

Predict: In `{l: m for l, m in zip(items, mins)}` , the item `book` appears twice. Which value does the dict keep?

```
mins = [480, 1250, 3100, 310, 60]
print([x / 1000 for x in mins])
print([x for x in mins if x > 400])
print(["slow" if x > 1000 else "ok" for x in mins])
items = ["book", "shoes", "lamp", "book", "lamp"]
print({l: m for l, m in zip(items, mins)})
print(sorted({l.upper() for l in items}))
print([(i, j) for i in range(3) for j in range(i)])
matrix = [[1, 2, 3], [4, 5, 6]]
print([x for row in matrix for x in row], [[r[i] for r in matrix] for i in range(3)])
x = 10
res = [x for x in range(3)]
print(x)
print([x for x in mins if x > 100 and x < 2000], [y for x in mins if (y := x // 100) > 10])
```

Real output:

```
[0.48, 1.25, 3.1, 0.31, 0.06]
[480, 1250, 3100]
['ok', 'slow', 'slow', 'ok', 'ok']
{'book': 310, 'shoes': 1250, 'lamp': 60}
['BOOK', 'LAMP', 'SHOES']
[(1, 0), (2, 0), (2, 1)]
[1, 2, 3, 4, 5, 6] [[1, 4], [2, 5], [3, 6]]
10
[480, 1250, 310] [12, 31]
```

- The condition goes at the **end** for filtering (`if x > 400`). A `if ... else ...` that **chooses a value** goes at the **front** (`"slow" if x > 1000 else "ok"`).
- Later keys overwrite earlier ones: `book` was first 480 and then 310; the dict keeps `310` . Duplicated keys silently lose data, so check for duplicates before building a dict from pairs.
- The nested comprehension `[(i, j) for i in range(3) for j in range(i)]` is the same as two loops: for each `i` , `j` goes from 0 up to `i - 1` . That gives `(1,0)` , `(2,0)` , `(2,1)` .
- Flatten a matrix with `[x for row in matrix for x in row]` (outer loop first), and transpose with `[[r[i] for r in matrix] for i in range(3)]` .
- `x` is still `10` after the comprehension that uses `x` : the comprehension has its own scope.
- The last line uses the walrus `:=` (Chapter 2) to compute `x // 100` once and also filter on it.

Worked example 4.5.2: generator expressions save memory

Predict: If I run `sum(gen)` twice on the same generator, will both give 30?

```
gen = (x * x for x in range(5))
print(type(gen).__name__, sum(gen), sum(gen))
import sys
print(sys.getsizeof([x for x in range(100000)]), sys.getsizeof(x for x in range(100000)))
print(max(mins, default=None), max([], default=None), any(x > 3000 for x in mins), all(x > 50 for x in mins))
```

Real output:

```
generator 30 0
800984 200
3100 None True True
```

- A generator can be used **once**. After `sum(gen)` consumed it, the second `sum(gen)` finds nothing and gives `0` . This is a classic surprise. 0 + 1 + 4 + 9 + 16 = 30 for the first call, which checks out.

- The list of 100,000 numbers takes about 800 KB for the list object (`800984` bytes; the exact figure depends on your Python version). The generator object is about 200 bytes, whatever the size of the range. The generator makes numbers only when asked.
- `any(...)` and `all(...)` stop as soon as the answer is known. With a generator expression they do not build a list first. `max(..., default=None)` avoids a `ValueError` on empty input.

The picture

```
[ f(x) for x in data if test(x) ]
  |          |          |
what goes   where it   which items
in (output) comes from  pass (filter)

equals:   out = []
          for x in data:
              if test(x): out.append(f(x))
```

Real-world mapping

Form	Example	Where it appears in the job
list comp	`[float(x) for x in row]`	Converting a column of text to numbers
dict comp	`{name: i for i, name in enumerate(vocab)}`	Building lookup tables
set comp	`{u.lower() for u in users}`	Deduplicating with cleaning
generator	`sum(len(l) for l in lines)`	Streaming huge files

Applications

- **Data Analytics:** clean and convert lists before creating a DataFrame.
- **Data Engineering:** stream lines of a huge file lazily with a generator expression.
- **Machine Learning:** build index maps, one-hot dictionaries, feature lists.
- **Interviews:** most Python coding tasks are shorter with comprehensions.

COMMON MISTAKE

Writing a comprehension so long or nested that nobody can read it (three `for` s and two `if` s). If it does not fit in one readable line, use a normal loop. Also: using a comprehension just for its side effects (`[print(x) for x in data]`), which builds a useless list.

Check yourself

1. Where does the filter `if` go in a list comprehension?

Answer: At the end: `[x for x in data if x > 0]` .

1. Why does the second `sum(gen)` print 0?

Answer: A generator is used up after one pass.

1. How do you flatten `[[1,2],[3]]` with a comprehension?

Answer: `[x for row in data for x in row]` .

4.6 `itertools` and `functools`: Ready-Made Loop Tools

INTUITION	Before you write a loop, check whether the toolbox already has a tool. `itertools` has tools for walking and combining sequences. `functools` has tools for treating functions as values (caching, fixing arguments, folding a list into one value).

Formal definition

`itertools` tool	What it does
`chain(a, b)`	walk `a` then `b` as one sequence
`islice(it, n)`	take the first `n` items of any iterator
`count(start)`, `cycle(x)`	endless counters / repeaters (use with `islice`)
`product(a, b)`	every combination, one from each (like nested loops)
`combinations(x, r)`, `permutations(x)`	choose `r` items, order ignored / order matters
`accumulate(x, func)`	running totals (or running max)
`pairwise(x)`	neighbouring pairs (Python 3.10 and newer)
`batched(x, n)`	chunks of size `n` (Python 3.12 and newer)
`zip_longest(a, b, fillvalue=)`	zip that does not stop at the shorter input
`groupby(x, key)`	group **adjacent** equal keys

`functools` tool	What it does
`lru_cache` / `cache`	remember results of a function by its arguments
`partial(f, ...)`	a new function with some arguments pre-filled
`reduce(f, x, start)`	fold a sequence into one value by applying `f` repeatedly

Worked example 4.6.1: itertools

Predict: `groupby` on an **unsorted** list of items `book, lamp, book`. How many groups come out: 2 or 3?

```
from itertools import chain, groupby, product, combinations, permutations, islice, accumulate, count, cycle, zip_longest,
pairwise, batched
print(list(chain([1,2],[3],"ab")), list(islice(count(10), 3)), list(islice(cycle("ab"), 5)))
print(list(product("ab", [1,2])), list(combinations("abc", 2)), len(list(permutations("abc"))))
print(list(accumulate([1,2,3,4])), list(accumulate([3,1,4], max)), list(pairwise([1,4,9,16])))
print(list(zip_longest([1,2,3], "ab", fillvalue="-")), list(batched(range(7), 3)))
rows = [("book",480),("book",310),("lamp",3100),("lamp",60),("shoes",1250)]
for key, grp in groupby(rows, key=lambda r: r[0]):
    print(key, [v for _, v in grp])
unsorted = [("book",480),("lamp",3100),("book",310)]
print([(k, len(list(g))) for k, g in groupby(unsorted, key=lambda r: r[0])])
```

Real output:

```
[1, 2, 3, 'a', 'b'] [10, 11, 12] ['a', 'b', 'a', 'b', 'a']
[('a', 1), ('a', 2), ('b', 1), ('b', 2)] [('a', 'b'), ('a', 'c'), ('b', 'c')] 6
[1, 3, 6, 10] [3, 3, 4] [(1, 4), (4, 9), (9, 16)]
[(1, 'a'), (2, 'b'), (3, '-')] [(0, 1, 2), (3, 4, 5), (6,)]
book [480, 310]
lamp [3100, 60]
shoes [1250]
[('book', 1), ('lamp', 1), ('book', 1)]
```

- `accumulate([1,2,3,4])` gives running totals `1, 3, 6, 10`. With `max` it gives a running maximum: `3, 3, 4`.
- `pairwise` gives neighbours. Differences between neighbours (`b - a`) are easy to get from it, for example the change between consecutive measurements.
- `product("ab", [1,2])` has 2 x 2 = 4 pairs. `combinations("abc", 2)` has 3 (`ab`, `ac`, `bc`). `permutations("abc")` has 3! = 6.
- `groupby` only groups **neighbours**. With sorted data it gives 3 groups. With the unsorted list it gives **three** groups, and `book` appears twice. **Always sort by the key first**, or use `defaultdict`. This is the number-one `groupby` bug.

Worked example 4.6.2: functools

```
from functools import reduce, cache, partial
print(reduce(lambda a, b: a + b, [1,2,3,4]), reduce(max, [3,9,2]), reduce(lambda a, b: a*b, range(1,6), 1))
@cache
def sq(n):
    print("computing", n); return n*n
print(sq(4), sq(4))
print(sq.cache_info())
hexint = partial(int, base=16)
print(hexint("ff"), hexint.func.__name__, hexint.keywords)
```

Real output:

```
10 9 120
computing 4
16 16
CacheInfo(hits=1, misses=1, maxsize=None, currsize=1)
255 int {'base': 16}
```

- `reduce` folds: `1+2+3+4 = 10`, the biggest of `3, 9, 2` is 9, and `1*2*3*4*5 = 120` (that is 5!). The start value `1` in the last call is the "nothing yet" value.
- `@cache` ran the function body once ("computing 4" printed once) and served the second call from memory. `cache_info()` shows 1 hit and 1 miss.
- `partial(int, base=16)` is a new function that reads hexadecimal: `ff` is 255 (15 x 16 + 15). `lru_cache(maxsize=N)` is the same as `cache` but forgets old entries, so memory stays bounded.

The picture

```
groupby on sorted rows:   book book | lamp lamp | shoes      -> 3 groups
groupby on UNsorted rows: book | lamp | book                 -> 3 groups (book twice!)
```

Real-world mapping

Tool	Example	Where it appears in the job
`product`	all combinations of 3 learning rates and 4 batch sizes	Grid search for models

Tool	Example	Where it appears in the job
`batched` / `islice`	send 1000 rows at a time to an API	Batch loading
`accumulate`	running total of revenue	Cumulative metrics
`cache`	cache an expensive lookup	Speeding up repeated calls
`chain`	read several files as one stream	ETL

Applications

- **Machine Learning:** `product` to generate hyper-parameter grids; `combinations` for feature pairs.
- **Data Engineering:** `batched` for chunked inserts; `chain.from_iterable` to join file streams.
- **Data Analytics:** `accumulate` for running totals; `pairwise` for period-over-period changes.
- **Backend / APIs:** `lru_cache` for config and lookup caching.

COMMON MISTAKE

Using `groupby` on unsorted data (see above). Also: using `@cache` on a function with a changing result (like "current time" or a database read). It will return the old answer forever. Cache only pure functions: same inputs, same output.

Check yourself

1. How many items does `list(combinations("abcd", 2))` have?

Answer: 6 (`4 x 3 / 2`).

1. What must you do before `groupby` ?

Answer: Sort the data by the same key.

1. What does `partial(int, base=2)("101")` return?

Answer: `5` .

4.7 A Tour of Other Standard-Library Modules

INTUITION

Python ships with "batteries included": hundreds of ready modules that need no installation. A professional first asks "is there a standard module for this?" before writing code or installing a package. This section is a map, not a manual. You will use the modules marked in later chapters.

The map

Module	What it gives you	Chapter where you go deeper
`math`	`sqrt` , `ceil` , `floor` , `log10` , `pi` , `gcd` , `isclose`	this one
`statistics`	`mean` , `median` , `stdev` , `quantiles`	this one
`random`	random numbers; `seed` makes them repeatable	this one
`time`	`perf_counter` (timing), `sleep` , `time` (seconds since 1970)	11

Module	What it gives you	Chapter where you go deeper
`datetime`, `zoneinfo`	dates, times, time zones	5
`os`, `sys`, `pathlib`, `shutil`, `glob`, `tempfile`	files, paths, the system	5
`json`, `csv`, `sqlite3`	data formats and a small database	5, 12
`re`, `string`	text patterns	3
`collections`, `itertools`, `functools`, `heapq`, `bisect`, `copy`	data tools	1, 4
`logging`, `unittest`, `argparse`, `subprocess`	professional tooling	6, 10, 12
`threading`, `multiprocessing`, `concurrent.futures`, `asyncio`	doing many things at once	11
`urllib`, `http`	web access and a tiny web server	12
`hashlib`, `uuid`	fingerprints (hashes) and unique ids	13

Worked example 4.7.1: a mini delivery report with `statistics`

Problem 4.7.1 Ten delivery times (mins): `[480, 1250, 3100, 310, 60, 290, 700, 220, 95, 540]`. Find the mean, the median and the 95th percentile.

Formula	mean = sum / count. The median is the middle value of the sorted list (average of the two middle ones when the count is even). A **percentile** is the value below which that percent of the data falls. The `inclusive` method puts the p-th percentile at position `(n − 1) × p` in the sorted list and blends the two neighbours.
Plug in	The sum is 480 + 1250 + 3100 + 310 + 60 + 290 + 700 + 220 + 95 + 540 = 7045 and n = 10. Sorted list: `[60, 95, 220, 290, 310, 480, 540, 700, 1250, 3100]`.
Compute	mean = 7045 / 10 = 704.5. Median = (310 + 480) / 2 = 395. For p95: position = (10 − 1) × 0.95 = 8.55 (counting from 0). The values at positions 8 and 9 are 1250 and 3100. So p95 = 1250 + 0.55 × (3100 − 1250) = 1250 + 1017.5 = 2267.5.
Answer	Mean 704.5 mins, median 395 mins, p95 2267.5 mins.
Check	The code below prints the same three numbers, and `statistics.stdev` gives 910.6.
Why it matters	The mean (704.5) is nearly twice the median (395) because one very late delivery (3100) pulls it up. The p95 shows the tail that customers feel. This is why service reports use percentiles, such as "95% of orders arrive within X minutes".

```
import math, statistics
mins = [480, 1250, 3100, 310, 60, 290, 700, 220, 95, 540]
print(math.sqrt(144), math.ceil(2.1), math.floor(-2.1), math.log10(1000), math.gcd(12, 18))
print(statistics.mean(mins), statistics.median(mins), round(statistics.stdev(mins), 1))
print(statistics.quantiles(mins, n=20, method="inclusive")[18])
```

Real output:

```
12.0 3 -3 3.0 6
704.5 395.0 910.6
2267.5
```

`quantiles(mins, n=20)` cuts the data into 20 equal parts and returns the 19 cut points, so index 18 is the 95th percentile. Different tools use slightly different percentile methods (NumPy, pandas and spreadsheets each have their own default), so the numbers can differ in small ways. Always state your method.

Worked example 4.7.2: random, time, uuid, hashlib

Predict: If you call `random.seed(42)` before two `randint` calls, will they give the same number?

```
import random, time, uuid, hashlib
random.seed(42); a = random.randint(1, 100)
random.seed(42); b = random.randint(1, 100)
print(a == b, len(random.sample(range(100), 5)))
t0 = time.perf_counter(); time.sleep(0.01); el = time.perf_counter() - t0
print(el >= 0.01)
u = uuid.uuid4(); print(len(str(u)), u.version)
print(hashlib.sha256(b"hello").hexdigest()[:16], hashlib.md5(b"hello").hexdigest())
```

Real output:

```
True 5
True
36 4
2cf24dba5fb0a30e 5d41402abc4b2a76b9719d911017c592
```

- Same seed, same "random" numbers. That is how you make an experiment or a test **repeatable**. `random` is not safe for passwords or tokens (use `secrets`, Chapter 13).
- `time.perf_counter()` is the right clock for timing code, and `sleep(0.01)` really waited at least 10 ms.
- A `uuid4` is a random 36-character id (`version` 4). The chance of two being equal is negligible.
- `sha256(b"hello")` always gives the same fingerprint, starting `2cf24dba5fb0a30e`. The MD5 value of `hello` is `5d41402abc4b2a76b9719d911017c592` (a well-known value). MD5 is broken for security, but still fine for a quick checksum.

Real-world mapping

Module	Example	Where it appears in the job
`statistics`	mean, median, p95 of delivery times	Quick reports without installing anything
`random.seed`	reproducible train/test split	Machine learning experiments
`uuid`	give each request a trace id	Logging and distributed tracing
`hashlib`	fingerprint a file or a row	Data deduplication and change detection

Applications

- **Data Analytics:** check a number by hand with `statistics` before trusting a library.
- **Data Engineering:** row hashes to detect changed records; uuids as keys.
- **Machine Learning:** seeds for reproducibility.
- **Service Operations:** `perf_counter` for timing code, percentiles for service-level promises.

COMMON MISTAKE

Reporting only the average delivery time. Averages hide tails. Always report the median and a high percentile (p90, p95, p99) together with the sample count. Another: using `random` for secrets, or `time.time()` to measure short durations (use `perf_counter`).

Check yourself

1. Why is the mean larger than the median in our ten values?

Answer: One very large value (3100) pulls the mean up, but the median ignores how large the extremes are.

1. What does `random.seed(1)` do?

Answer: It sets the starting point of the random sequence, so the same numbers repeat on every run.

1. Which clock should you use to time a piece of code?

Answer: `time.perf_counter()`.

4.8 Exact Numbers, Stable Sorting, Copying and Hashing

This section digs into four things that every collection user eventually trips over: floats that are not exact, sorting by more than one rule, copies that are not independent, and what "hashable" really means. Each has three worked cases.

Exact numbers: float, Decimal and Fraction

Floats store numbers in binary, and most decimal fractions such as 0.1 have no exact binary form, so tiny errors appear. For money use `decimal.Decimal`, which does decimal arithmetic exactly. For ratios use `fractions.Fraction`, which stores a numerator and denominator.

Problem 4.8.1 Show the float error, the exact alternatives, the rounding trap in `round()`, and banker's rounding.

Float	`0.1 + 0.2` is `0.30000000000000004`, so `== 0.3` is `False`.
Decimal	Build from **strings**: `Decimal("0.1") + Decimal("0.2")` is exactly `0.3`. `Decimal(0.1)` keeps the float's error.
Rounding	`round(2.675, 2)` gives 2.67 because the float is really 2.67499999... With `Decimal("2.675")` and `ROUND_HALF_UP` you get 2.68. Python's own `round` uses **round-half-to-even**: 0.5 goes to 0, 1.5 to 2, 2.5 to 2.
Fraction	1/3 + 1/6 is exactly 1/2. `Fraction(0.1).limit_denominator(10)` recovers 1/10.

```
from decimal import Decimal, ROUND_HALF_UP
from fractions import Fraction
print(0.1 + 0.2, 0.1 + 0.2 == 0.3)
print(Decimal("0.1") + Decimal("0.2"), Decimal("0.1") + Decimal("0.2") == Decimal("0.3"))
print(Decimal(0.1))
print(round(2.675, 2), Decimal("2.675").quantize(Decimal("0.01"), rounding=ROUND_HALF_UP))
print(round(0.5), round(1.5), round(2.5))
print(Fraction(1, 3) + Fraction(1, 6), Fraction("0.75"), Fraction(0.1).limit_denominator(10))
x = 0.0
for _ in range(10):
    x += 0.1
print(x, x == 1.0)
```

```
0.30000000000000004 False
0.3 True
0.1000000000000000055511151231257827021181583404541015625
2.67 2.68
0 2 2
1/2 3/4 1/10
0.9999999999999999 False
```

COMMON MISTAKE Creating a Decimal from a float: `Decimal(0.1)` faithfully copies the float's hidden error (see the long line above). Pass a string, `Decimal("0.1")`. For comparing floats use `math.isclose(a, b)`, never `==`.

Sorting: stability, multiple keys and cmp_to_key

Python's `sorted` and `list.sort` are **stable**: items with equal keys keep their original order. That lets you sort by several rules by sorting in passes, least important key first. For two keys it is simpler to return a tuple from `key`, and to flip one numeric key use a minus sign. For an old-style comparison function (returns negative, zero or positive) wrap it with `functools.cmp_to_key`.

Problem 4.8.2 Sort `rows = [("bo", 3), ("al", 2), ("cy", 3), ("di", 2)]` by score descending, ties by name ascending, in four ways.

Stability	Sorting by score alone keeps equal scores in input order: al before di, bo before cy.
Two passes	First sort by name, then by `-score`. Stability keeps the name order within each score.
Tuple key	`key=lambda r: (-r[1], r[0])`
cmp_to_key	A function that compares scores in reverse and then names.

```
from functools import cmp_to_key
rows = [("bo", 3), ("al", 2), ("cy", 3), ("di", 2)]
print(sorted(rows, key=lambda r: r[1]))
print(sorted(sorted(rows), key=lambda r: -r[1]))
print(sorted(rows, key=lambda r: (-r[1], r[0])))
def cmp(a, b):
    return (b[1] > a[1]) - (b[1] < a[1]) or (a[0] > b[0]) - (a[0] < b[0])
print(sorted(rows, key=cmp_to_key(cmp)))
print(sorted(["b10", "a2", "a10"], key=lambda s: (s[0], int(s[1:]))))
```

```
[('al', 2), ('di', 2), ('bo', 3), ('cy', 3)]
[('bo', 3), ('cy', 3), ('al', 2), ('di', 2)]
[('bo', 3), ('cy', 3), ('al', 2), ('di', 2)]
[('bo', 3), ('cy', 3), ('al', 2), ('di', 2)]
['a2', 'a10', 'b10']
```

The last line shows a "natural" sort: the key turns `"10"` into the integer 10, so `a2` comes before `a10`. A plain string sort would put `a10` first.

Copying: shallow, deep and sharing

A **shallow copy** makes a new outer container but reuses the same inner objects. A **deep copy** (`copy.deepcopy`) copies everything inside too, and it copes with cycles. Three cases show when each matters.

Problem 4.8.3 Predict the output of each case before reading the answer.

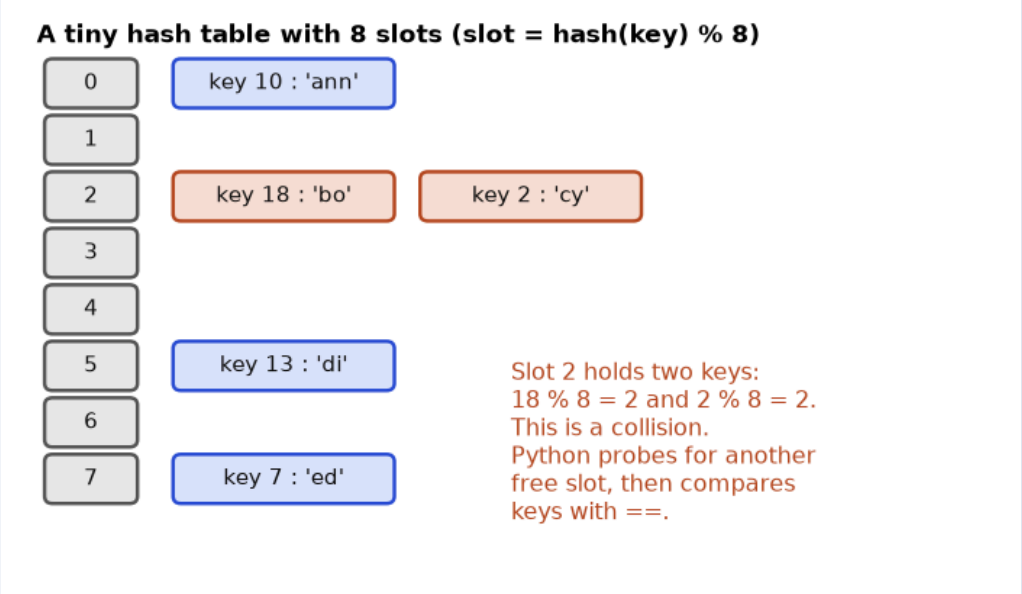
Case 1	A list of lists. `a.copy()` is shallow, so appending to `a[0]` also changes `shallow[0]` but not `deep[0]`.
Case 2	A dict copied with `dict(cfg)`. Changing the top-level key `n` in the copy is isolated. Appending to the shared `paths` list changes the original too.
Case 3	A list that contains itself. `deepcopy` handles the cycle: the clone points at itself, not at the original. A `copy.copy` of an object copies only its attribute references, so two objects share one list.

```
import copy
a = [[1, 2], [3, 4]]
shallow = a.copy()
deep = copy.deepcopy(a)
a[0].append(99)
print(shallow[0], deep[0])
cfg = {"paths": ["/tmp"], "n": 1}
view = dict(cfg)
view["n"] = 2
view["paths"].append("/var")
print(cfg)
ring = [1]
ring.append(ring)
clone = copy.deepcopy(ring)
print(clone[1] is clone, clone is ring)
class Box:
    def __init__(self, items):
        self.items = items
b1 = Box([1])
b2 = copy.copy(b1)
b2.items.append(2)
print(b1.items)
```

```
[1, 2, 99] [1, 2]
{'paths': ['/tmp', '/var'], 'n': 1}
True False
[1, 2]
```

Hashing: why keys must be hashable

A dict or set finds an item by computing `hash(key)`, using it to choose a slot, and then comparing keys with `==` if the slot is busy. The rule that makes this work: **if** `a == b` **then** `hash(a) == hash(b)`. Mutable containers (list, dict, set) are unhashable because their contents, and so their equality, can change after they are stored.



A toy hash table with 8 slots. Real tables are larger and resize as they fill, but the idea is the same.

How to read it: each key is reduced to a slot number with `hash % 8`. Keys 18 and 2 both land in slot 2, a collision, so one waits in another place and lookups compare with `==` to tell them apart. Because the slot number is computed rather than searched, lookup time stays about the same however many keys exist.

Problem 4.8.4 Show three hashing facts: equal numbers collapse to one key, mutable keys fail, and defining `__eq__` removes the default hash.

Case 1	`1`, `1.0` and `True` are equal and have equal hashes, so as dict keys they are one key. The first key object stays; the last value wins.
Case 2	A list as a key raises `TypeError`.
Case 3	A class that defines `__eq__` but not `__hash__` becomes unhashable. Adding `__hash__` based on the same data makes two equal objects collapse in a set.

```
print(hash(1) == hash(1.0), hash("a") == hash("a"))
print({1: "int", 1.0: "float", True: "bool"})
try:
    {[1, 2]: "x"}
except TypeError as e:
    print(e)
class P:
    def __init__(self, x): self.x = x
    def __eq__(self, o): return self.x == o.x
try:
    {P(1)}
except TypeError as e:
    print(e)
class Q(P):
    __hash__ = lambda self: hash(self.x)
print(len({Q(1), Q(1), Q(2)}))
```

```
True True
{1: 'bool'}
cannot use 'list' as a dict key (unhashable type: 'list')
cannot use 'P' as a set element (unhashable type: 'P')
2
```

Random numbers you can repeat

Python's `random` module produces a repeatable sequence from a **seed**. Seeding makes tests and experiments reproducible. A separate `random.Random(seed)` object gives you your own generator, so other code cannot disturb it. Never use `random` for passwords or tokens; use `secrets` (Chapter 13).

```
import random
random.seed(7)
r1 = [random.randint(1, 6) for _ in range(5)]
random.seed(7)
r2 = [random.randint(1, 6) for _ in range(5)]
print(r1 == r2)
print(random.Random(42).sample(range(100), 3) == random.Random(42).sample(range(100), 3))
```

```
True
True
```

KEY IDEA

Four questions to ask about any collection code: Is this number exact enough (Decimal)? Does order among ties matter (stable sort)? Who else holds a reference to the inner objects (copy depth)? Is the key stable and hashable? Most data bugs in this chapter's area come from skipping one of them.

Chapter recap

- A **list** is an ordered, changeable sequence. A **tuple** is fixed. `sort()` changes in place and returns `None`; `sorted()` returns a new list.
- A **set** keeps unique items and tests membership fast (hash table). Use `set()` for an empty one and never rely on its order.
- A **dict** maps keys to values, keeps insertion order, and looks up by key in constant time. Use `get` for safe reads and `|` to merge.
- `defaultdict` for grouping, `Counter` for counting, `deque` for queues and windows, `heapq` for top-N, `bisect` for sorted inserts.
- Comprehensions build lists, sets and dicts in one readable line. The filter `if` goes at the end; a choosing `if/else` goes at the front.
- Generator expressions produce items lazily, use almost no memory, and can be used only once.
- `itertools` and `functools` replace many hand-written loops. `groupby` needs sorted input.
- Pick the container by the question you ask most often: order, uniqueness, or lookup by key.
- The standard library already has `math`, `statistics`, `random`, `uuid` and `hashlib`. Report median and percentiles, not only the mean, and seed random numbers when you need repeatability.

Need	Tool
Ordered, changeable	`list`
One fixed record	`tuple` / `namedtuple`
Unique + fast `in`	`set`
Lookup by key	`dict`, `d.get(k, default)`
Group by key	`defaultdict(list)`
Count items	`Counter(items).most_common(n)`
Sliding window / queue	`deque(maxlen=n)`
Smallest/largest n	`heapq.nsmallest(n, x)` / `nlargest`
Filter and transform	`[f(x) for x in data if test(x)]`
Running totals	`itertools.accumulate`
Mean, median, p95	`statistics.mean`, `median`, `quantiles(x, n=20)[18]`
Cache a function	`@functools.cache`

Practice set

Problem 4.P.1 (easy) What is `[1, 2, 3][::-1]`?

Answer: `[3, 2, 1]`. The step `-1` walks backwards.

Problem 4.P.2 (easy) What does `len({1, 2, 2, 3, 3, 3})` return?

Answer: `3`. A set keeps each value once.

Problem 4.P.3 (easy) Write a dict comprehension that maps each word in `["a", "bb", "ccc"]` to its length.

Answer: `{w: len(w) for w in ["a", "bb", "ccc"]}` gives `{'a': 1, 'bb': 2, 'ccc': 3}`.

Problem 4.P.4 (what is the output?)

```
a = [3, 1, 2]
b = a.sort()
print(a, b)
```

Answer: `[1, 2, 3] None`. `sort()` changes `a` and returns `None`.

Problem 4.P.5 (what is the output?)

```
d = {"x": 1}
d2 = d
d2["y"] = 2
print(d, len(d))
```

Answer: `{'x': 1, 'y': 2} 2`. `d2` is the same dict (Chapter 1), not a copy. Use `d.copy()` for a copy.

Problem 4.P.6 (computation) Use `Counter` to find the most common word in `"to be or not to be"`.

Answer: `Counter("to be or not to be".split()).most_common(1)` gives `[('to', 2)]`. In this text `to` and `be` both appear twice, and `most_common` lists equal counts in the order first seen, so `to` is first.

Problem 4.P.7 (computation) Given `mins = [480, 1250, 3100, 310, 60]`, write one line that gives the share of values above 400 as a percentage.

Answer: `sum(1 for x in mins if x > 400) / len(mins) * 100`. Three values (480, 1250, 3100) are above 400, so 3 / 5 * 100 = 60.0.

Problem 4.P.8 (find the bug)

```
users = ["a", "b", "a"]
for u in users:
    if users.count(u) > 1:
        users.remove(u)
```

Answer: The loop changes the list while looping (Chapter 2 mistake), so items can be skipped. Deduplicate with `list(dict.fromkeys(users))`, which keeps the first occurrence and the order, giving `['a', 'b']`.

Problem 4.P.9 (find the bug) Why is this slow on a million rows?

```
seen = []
for row_id in ids:
    if row_id not in seen:
        seen.append(row_id)
```

Answer: `row_id not in seen` scans the whole list every time, so the work grows with the square of the data size. Use `seen = set()` and `seen.add(row_id)`, where each test is constant time.

Problem 4.P.10 (scenario) You have 5 million log lines and need the 10 most frequent IP addresses. Which tools do you use and why?

Answer: Stream the lines with a generator and count with `Counter(ip for ip in ips_generator)`, then `.most_common(10)`. A generator keeps memory small; `Counter` counts in one pass.

Problem 4.P.11 (interview) "What is the difference between a list and a tuple, and when would you choose each?"

Answer: A list is mutable, a tuple is immutable (and hashable if its items are). Choose a list for a collection that grows or changes, and a tuple for a fixed record, a dict key, or a value that must not change by mistake. Tuples are also slightly smaller and faster to create.

Problem 4.P.12 (interview) "How does a dict find a key so quickly, and what is required of the keys?"

Answer: It uses a hash table: it computes the hash of the key to find the slot directly, so lookup takes constant time on average. Keys must be hashable (immutable, with a stable hash), and equal keys must have equal hashes. That is why lists cannot be keys.

More practice: build, stretch and review

Problem 4.P.13 (build) An invoice has item prices `"19.99"`, `"5.01"` and `"3.50"`. Compute the subtotal, 8.25% tax rounded half-up to cents, and the total, all exactly.

```
from decimal import Decimal, ROUND_HALF_UP
prices = ["19.99", "5.01", "3.50"]
subtotal = sum(Decimal(p) for p in prices)
tax = (subtotal * Decimal("0.0825")).quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)
print(subtotal, tax, subtotal + tax)
```

```
28.50 2.35 30.85
```

Answer: By hand: 19.99 + 5.01 + 3.50 = 28.50. Tax is 28.50 x 0.0825 = 2.35125, which rounds to 2.35. Total 30.85. Prices come in as strings so no float error enters.

Problem 4.P.14 (build) Write `group_by_len(words)` that returns a dict from word length to the sorted list of words with that length, with keys in increasing order. Test on `["pear", "fig", "plum", "kiwi", "yam", "apple"]`.

```
from collections import defaultdict
def group_by_len(words):
    groups = defaultdict(list)
    for w in words:
        groups[len(w)].append(w)
    return {k: sorted(v) for k, v in sorted(groups.items())}
print(group_by_len(["pear", "fig", "plum", "kiwi", "yam", "apple"]))
```

```
{3: ['fig', 'yam'], 4: ['kiwi', 'pear', 'plum'], 5: ['apple']}
```

Answer: as shown. `defaultdict(list)` creates the empty list the first time a length appears. Self-check: try an empty input; you should get `{}`.

Problem 4.P.15 (what is the output?) With `rows = [("bo", 3), ("al", 2), ("cy", 3), ("di", 2)]`, what does `sorted(rows, key=lambda r: r[1], reverse=True)` return? Does `reverse=True` break stability?

```
rows = [("bo", 3), ("al", 2), ("cy", 3), ("di", 2)]
print(sorted(rows, key=lambda r: r[1], reverse=True))
```

```
[('bo', 3), ('cy', 3), ('al', 2), ('di', 2)]
```

Answer: As shown. Python keeps equal items in their original order even with `reverse=True` (bo before cy, al before di). It is not the same as sorting ascending and reversing the list, which would flip the ties.

Problem 4.P.16 (find the bug) A teammate builds a 2 by 3 grid with `grid = [[0] * 3] * 2` and sets `grid[0][0] = 1`. What does the grid look like, and what is the fix?

```
grid = [[0] * 3] * 2
grid[0][0] = 1
print(grid)
good = [[0] * 3 for _ in range(2)]
good[0][0] = 1
print(good)
```

```
[[1, 0, 0], [1, 0, 0]]
[[1, 0, 0], [0, 0, 0]]
```

Answer: `[row] * 2` repeats a reference to the same inner list, so both rows are one list. A comprehension builds a fresh list per row.

Problem 4.P.17 (interview) Why can a tuple be a dict key but a list cannot? Is `(1, [2])` a valid key?

Answer: A key must be hashable, and equal keys must have equal hashes forever. A tuple of hashable items is hashable; a list can change, so its hash could change while it sits in the table. `(1, [2])` is a tuple, but hashing it hashes the inner list, so it raises `TypeError: unhashable type: 'list'`.

Prompt pack

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data roles, and English is my second language, so use simple words and short sentences. Teach me the four Python containers: list, tuple, set, dict. For each, explain with an everyday analogy, show when to choose it, and give a small runnable example using delivery-time data. Ask me to predict outputs before showing them, then give me three questions to check my understanding.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data roles, with English as a second language. Topic: Python dictionaries and sets and why lookups are fast (hash tables). Explain hashing in simple words, then give me 8 graded exercises about grouping, counting, deduplicating and merging data using dict and set. Do not give solutions at first. When I paste my answers, correct them and explain each mistake step by step.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data roles, and English is my second language. Teach me list, set and dict comprehensions and generator expressions in Python. Show how to turn a normal loop into a comprehension step by step, explain when NOT to use one, and explain why a generator can be used only once. Then quiz me with 6 short "what is the output?" questions, one at a time, waiting for my answer each time.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data roles, with English as a second language. Teach me the Python collections module (defaultdict, Counter, deque, OrderedDict), heapq and bisect, plus itertools (groupby, product, combinations, accumulate, batched) and functools (cache, partial, reduce). For each tool give one realistic data-engineering use case and a runnable example. Then review this code of mine and suggest better tools:
[PASTE HERE]
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data roles, and English is my second language. Run a mock Python interview on collections: list vs tuple, mutability, set operations, dict internals,
```

```
comprehensions, the groupby-needs-sorted-input trap, and time complexity of list vs set lookups. Ask one question at a time, score each answer from 1 to 5, give a model answer, and end with my top three weak areas.
```

CHAPTER 5

FILES, PATHS, JSON, CSV, DATES AND TIME ZONES

Real programs read data from disk, talk in JSON and CSV, and live in a world of clocks and time zones. This chapter makes you safe in all four.

Imagine your manager hands you a folder of 400 daily log files, a partner's CSV with quotes inside the cells, an API reply in JSON, and a question: "How many orders did we get yesterday in New York time?" Before you can use any analysis tool, you must get the data in, get the text apart correctly, and get the clock right. This chapter gives you those four skills: files and paths, JSON, CSV, and dates with time zones. We finish with how to keep settings and secrets out of your code. By the end you can read and write files safely, convert between JSON and Python objects, parse a CSV with tricky cells, and do date maths without the classic daylight saving traps.

Running example for this chapter: a small online shop. It has an `orders.csv` file (order id, city, amount), JSON logs with delivery times in minutes, and servers in India and New York.

Every example in this chapter was run with `python3` (Python 3.14). The output shown is the real output. Folders are created with `tempfile` (a module that makes folders that delete themselves), so nothing is left on your computer. For each example, try the **Predict** step first: cover the output, guess it, then compare.

5.1 Files and Paths with `pathlib`

INTUITION

A file path is an address. If you write the address by hand as a string, you must remember that Windows uses `\` and Mac/Linux use `/`. `pathlib` is a small address book that does this for you. Think of a courier given a delivery address: if the address is wrong, the whole delivery fails. In Python it is the same, so we handle paths with care.

How it works

- `Path("data") / "raw" / "x.csv"` builds a path. The `/` operator joins parts with the right separator for your computer.
- `p.name` is the file name, `p.stem` is the name without extension, `p.suffix` is the extension, `p.parent` is the folder.
- `p.exists()`, `p.is_file()`, `p.is_dir()` ask questions about the disk.
- `p.read_text(encoding="utf-8")` and `p.write_text(...)` read or write a whole small file in one call.
- `p.read_bytes()` and `p.write_bytes(...)` do the same for binary data (images, zip files).
- `open(p, mode, encoding=...)` gives a file object. Modes: `"r"` read (default), `"w"` write (erases old content!), `"a"` append, `"b"` binary (add to the others, like `"rb"`).
- Always use `with open(...) as f:` so the file is closed even if an error happens (more in Chapter 7).
- Always pass `encoding="utf-8"` for text. Without it, Python uses the computer's default, and your code may behave differently on another machine.
- `p.rglob("*.log")` finds files in all sub-folders. `p.glob("*.log")` looks in one folder only. The older `glob.glob(pattern)` returns strings.
- `os` is the older module (`os.path.join`, `os.listdir`, `os.environ`). `shutil` copies and deletes whole trees (`shutil.copy`, `shutil.rmtree`, `shutil.move`). `tempfile` makes temporary files and folders that clean up after themselves.
- `sys.argv` is the list of command-line words. `sys.argv[0]` is the script name.

Words you need

Word	Plain meaning
path	The address of a file or folder on disk
encoding	The rule that turns letters into bytes (UTF-8 is the standard that handles every language)
byte	One number from 0 to 255; files are made of bytes

Word	Plain meaning
mode	The word that tells `open` what you want: read, write, append

Item	Example value	Where it appears in the job
Path	`data/raw/orders_2026-11-06.csv`	Daily batch jobs name files by date
Suffix	`.parquet` , `.csv` , `.log`	Pipelines pick a reader by the suffix
Mode `"a"`	append to `app.log`	Log files that grow all day
Temp folder	`/tmp/tmpab12`	Tests that must not touch real data

Worked examples

Problem 5.1.1 Create a temporary folder with a `logs` sub-folder, write two log files, then inspect and read them.

Step 1	Use `tempfile.TemporaryDirectory()` so the folder vanishes at the end.
Step 2	Build paths with `/` , write text, and ask for name, stem, suffix, size.
Step 3	Find files with `rglob` and `glob.glob` , and read line by line with `enumerate` .

Predict: Before you run it, how many bytes do you think `app.log` has? (Hint: count letters and newline characters.)

```
from pathlib import Path
import tempfile, shutil, glob, os

with tempfile.TemporaryDirectory() as d:
    root = Path(d)
    (root / "logs").mkdir()
    (root / "logs" / "app.log").write_text("line1\nline2\nline3\n", encoding="utf-8")
    (root / "logs" / "old.log").write_text("x\n", encoding="utf-8")
    (root / "notes.txt").write_text("hello", encoding="utf-8")
    p = root / "logs" / "app.log"
    print(p.name, p.stem, p.suffix, p.parent.name)
    print(p.exists(), p.is_file(), p.stat().st_size)
    print(sorted(x.name for x in root.rglob("*.log")))
    print(sorted(os.path.basename(x) for x in glob.glob(str(root / "logs" / "*.log"))))
    with open(p, encoding="utf-8") as f:
        for n, line in enumerate(f, start=1):
            print(n, line.rstrip())
```

```
app.log app .log logs
True True 18
['app.log', 'old.log']
['app.log', 'old.log']
1 line1
2 line2
3 line3
```

- **Explain** Line by line: `mkdir()` makes the folder; `write_text` makes and fills a file; `stat().st_size` is the size in bytes; `rglob("*.log")` searches the folder and everything under it for names ending in `.log` ; `enumerate(f, start=1)` numbers the lines from 1; `rstrip()` removes the newline at the end so `print` does not add a blank line.
- **Answer** The file is 18 bytes: three lines of 5 letters plus a newline each is 3 × 6 = 18. Reading with `for line in f` gives one line at a time, so even a 10 GB file would not fill memory.
- **Check** `"line1\nline2\nline3\n"` has 5+1+5+1+5+1 = 18 characters, and all are ASCII, so bytes equal characters. The output 18 matches.

- **Why it matters** The same loop works on a 3-line file and a 30 GB log, so your code does not need to change when data grows.

The typical wrong version. You open a file that is not there:

```
open("/no/such/dir/data.csv", encoding="utf-8")
```

```
FileNotFoundError: [Errno 2] No such file or directory: '/no/such/dir/data.csv'
```

The fix is to check the path first (`Path(x).exists()`), or to catch `FileNotFoundError` and give a clear message (Chapter 6).

Problem 5.1.2 Copy a file, delete a folder, append twice to a file, and write raw bytes.

Step 1	`shutil.copy(src, dst)` copies one file. `shutil.rmtree(folder)` deletes a folder and everything inside it. There is no recycle bin, so be careful.
Step 2	Open with mode `"a"` twice. The second open does not erase the first write.
Step 3	Write `bytes([72, 105, 255])` . 72 is `H` , 105 is `i` , and 255 is not a printable ASCII letter.

Predict: What does `a.txt` contain after two appends? Will the first `"one"` survive?

```
from pathlib import Path
import tempfile, shutil, os

with tempfile.TemporaryDirectory() as d:
    root = Path(d)
    (root / "logs").mkdir()
    (root / "logs" / "app.log").write_text("line1\n", encoding="utf-8")
    (root / "notes.txt").write_text("hello", encoding="utf-8")
    shutil.copy(root / "logs" / "app.log", root / "backup.log")
    print(sorted(x.name for x in root.iterdir()))
    shutil.rmtree(root / "logs")
    print(sorted(x.name for x in root.iterdir()))
    with open(root / "a.txt", "a", encoding="utf-8") as f:
        f.write("one\n")
    with open(root / "a.txt", "a", encoding="utf-8") as f:
        f.write("two\n")
    print(repr((root / "a.txt").read_text()))
    (root / "b.bin").write_bytes(bytes([72, 105, 255]))
    print((root / "b.bin").read_bytes())
print(Path("data") / "raw" / "x.csv")
print(os.sep, os.path.join("data", "raw", "x.csv"))
```

```
['backup.log', 'logs', 'notes.txt']
['backup.log', 'notes.txt']
'one\ntwo\n'
b'Hi\xff'
data/raw/x.csv
/ data/raw/x.csv
```

- **Explain** `shutil.copy` made `backup.log` . After `rmtree` only `backup.log` and `notes.txt` remain. `repr(...)` shows the hidden `\n` characters so you can see them.
- **Answer** After `rmtree` , the `logs` folder is gone. Both appends are kept (`'one\ntwo\n'`). Binary data prints as `b'Hi\xff'` , where `\xff` means the single byte 255. On Windows, the last two lines would show `data\raw\x.csv` and `\` .
- **Check** Two writes of 4 characters each should give 8 characters: `one\n` + `two\n` . The repr shows exactly that.
- **Why it matters** Mode `"w"` here would have left only `'two\n'` . One wrong letter in a log writer can erase a day of evidence.

The picture

```
Path("data") / "raw" / "x.csv"
       |          |        |
     folder     folder     file
 p.parent -> data/raw     p.name -> x.csv
 p.stem   -> x            p.suffix -> .csv
```

Task	pathlib	older style
join	`Path("a") / "b"`	`os.path.join("a","b")`
exists	`p.exists()`	`os.path.exists(p)`
list folder	`p.iterdir()`	`os.listdir(p)`
find by pattern	`p.rglob("*.csv")`	`glob.glob("**/*.csv", recursive=True)`
read all	`p.read_text()`	`open(p).read()`

Connection to the real job

Data engineers read thousands of files from folders: raw logs, daily exports, Parquet files. Pipelines list files with `rglob`, process them one at a time, and move finished ones with `shutil.move`. Think of a bank clerk working through a long stack of paper slips one at a time; the same loop-over-lines pattern applies, and `for line in f` is the safe way for huge files.

Applications

- **Data Analytics:** open the monthly export a business user emails you.
- **Data Engineering:** a nightly job lists `raw/2026-11-06/*.csv`, processes each file, and moves it to `done/`.
- **Machine Learning:** load thousands of image files from class folders (`cats/`, `dogs/`).
- **MLOps / Testing:** a unit test builds a temporary folder, runs the code, and deletes everything.

COMMON MISTAKE Opening a file with mode `"w"` when you meant `"a"`. `"w"` erases the file at once, before you write anything. If you are unsure, write to a new file name first.

COMMON MISTAKE Forgetting `encoding="utf-8"`. The script works on your laptop and breaks on the server when a name like "café" appears.

COMMON MISTAKE Reading a huge file with `f.read()` or `readlines()`. That loads everything into memory. Loop over the file object instead.

Check yourself

1. What is `Path("reports/q1.final.csv").stem`? Answer: `q1.final` (only the last suffix `.csv` is removed).
2. Which mode adds to the end of a file without erasing it? Answer: `"a"`.
3. Why use `with open(...)`? Answer: it closes the file automatically, even when an error happens inside the block.

5.2 JSON

INTUITION	Two programs, maybe written in different languages, need to exchange data. They need a common written language. JSON is that language: plain text, easy for humans, and every language can read it. When a shop's website asks a courier service for a delivery status, the reply is almost always JSON.

Formal definition

JSON stands for JavaScript Object Notation, but it is not tied to JavaScript. It has only a few types. Here is how they map to Python:

JSON	Python
object `{}`	`dict`
array `[]`	`list`
string	`str`
number	`int` or `float`
`true` / `false`	`True` / `False`
`null`	`None`

Four functions matter:

- `json.dumps(obj)` turns a Python object into a JSON **string**. (s = string)
- `json.loads(text)` turns a JSON string into Python objects.
- `json.dump(obj, file)` and `json.load(file)` do the same with a file object.

Why "dumps" and "loads"? The `s` stands for *string*. "Dump" means "pour out into text", "load" means "read back in". Think of packing a suitcase (`dumps`) and unpacking it at the other end (`loads`).

Item	Example value	Where it appears in the job
JSON object	`{"id": 7, "delivery_minutes": 20}`	One API reply or one log event
JSON array	`[{"id":1},{"id":2}]`	A page of results from an API
`null`	`"manager": null`	A missing value in an event
JSON Lines	one object per line	Event logs in a data lake

Useful options: `indent=2` (pretty print), `sort_keys=True` , `ensure_ascii=False` (keep non-English letters readable), `default=function` (tell it how to handle types it does not know).

Worked examples

Problem 5.2.1 Convert a dictionary to JSON and back. Check what is preserved.

Step 1	Build a dict with a list, a bool, `None` , and a float.
Step 2	Call `dumps` , then `loads` , and compare with `==` .
Step 3	Notice that JSON uses lowercase `true` / `null` , and that dict keys become strings.

Predict: Will `back == d` be `True` ? What happens to the key `1` in `{1: "a"}` ?

```
import json
d = {"name": "Asha", "age": 29, "skills": ["python", "sql"],
     "active": True, "manager": None, "score": 91.5}
s = json.dumps(d)
print(s)
print(json.dumps(d, indent=2, sort_keys=True))
back = json.loads(s)
print(back == d, type(back["active"]), back["manager"])
print(json.loads('{"1": "a"}'), json.dumps({1: "a"}))
print(json.dumps((1, 2)), json.loads(json.dumps((1, 2))))
```

```
{"name": "Asha", "age": 29, "skills": ["python", "sql"], "active": true, "manager": null, "score": 91.5}
{
  "active": true,
  "age": 29,
  "manager": null,
  "name": "Asha",
  "score": 91.5,
  "skills": [
    "python",
    "sql"
  ]
}
True <class 'bool'> None
{'1': 'a'} {"1": "a"}
[1, 2] [1, 2]
```

- **Explain** `dumps` writes `True` as `true` and `None` as `null`, because those are the JSON spellings. `sort_keys=True` orders the keys A to Z, which makes files easy to compare. `indent=2` adds line breaks and two-space indents.
- **Answer** Round trip kept equality. But two things changed on the way: the integer key `1` became the string `"1"`, and the tuple `(1, 2)` became a list `[1, 2]`. JSON has no tuple and no integer keys.
- **Check** `back == d` printed `True`, so for plain types nothing is lost. The tuple and key examples show the exceptions.
- **Why it matters** If you save `{1: "a"}` and read it back, `data[1]` raises `KeyError` because the key is now `"1"`. Surprises like this cause real bugs in pipelines.

Problem 5.2.2 Handle things JSON does not know: a `datetime`, a bad JSON string, non-English text.

Step 1	Dump a `datetime` and watch the `TypeError`.
Step 2	Fix it with `default=`, which is called for unknown objects.
Step 3	Load a string with single quotes, which is not valid JSON, and read the error.
Step 4	Compare `ensure_ascii` settings.

Predict: What error will the first `dumps` raise? Which Python type is it complaining about?

```
import json
from datetime import datetime
try:
    json.dumps({"when": datetime(2026, 1, 5)})
except TypeError as e:
    print("TypeError:", e)
print(json.dumps({"when": datetime(2026, 1, 5)}, default=lambda o: o.isoformat()))
try:
    json.loads("{'a': 1}")
except json.JSONDecodeError as e:
    print("JSONDecodeError:", e)
print(json.dumps({"city": "Chennai", "word": "café"}))
print(json.dumps({"word": "café"}, ensure_ascii=False))
```

```
TypeError: Object of type datetime is not JSON serializable
{"when": "2026-01-05T00:00:00"}
JSONDecodeError: Expecting property name enclosed in double quotes: line 1 column 2 (char 1)
{"city": "Chennai", "word": "caf\u00e9"}
{"word": "café"}
```

- **Explain** `datetime` is not one of the six JSON types, so `dumps` stops. The `default` function is called only for objects it does not understand; here it returns the ISO text. The `JSONDecodeError` message tells you the line and column of the problem, so read it.
- **Answer** Dates must be turned into strings (ISO format is the standard choice). JSON requires double quotes. By default non-ASCII letters are escaped (for example `é` becomes `\u00e9`); both forms are valid JSON.
- **Check** The date `2026-01-05` became `2026-01-05T00:00:00`, with time at midnight, which matches what `datetime(2026,1,5)` means.
- **Why it matters** API responses often contain dates and prices. Knowing this error saves you an hour the first time it happens.

Problem 5.2.3 A JSON Lines log has 3 records. Find the average delivery time, and safely read a deep key.

Step 1	JSON Lines means one JSON object per line. Parse each line with `json.loads`.
Step 2	Average = (20 + 55 + 15) / 3 = 90 / 3 = 30.0.
Step 3	Use chained `.get(key, {})` so a missing key gives a default, not a crash.

```
import json
lines = '{"id":1,"delivery_minutes":20}\n{"id":2,"delivery_minutes":55}\n{"id":3,"delivery_minutes":15}\n'
rows = [json.loads(l) for l in lines.splitlines()]
print(sum(r["delivery_minutes"] for r in rows) / len(rows))
nested = {"user": {"address": {"city": "Pune"}}}
print(nested.get("user", {}).get("address", {}).get("zip", "n/a"))
```

```
30.0
n/a
```

- **Answer** The average is 30.0 minutes, and the missing `zip` safely returns `"n/a"`.
- **Check** 20 + 55 + 15 = 90, and 90 / 3 = 30. The code printed 30.0.
- **Why it matters** In plain words: the average delivery took 30 minutes. But look at the 55-minute one: an average hides slow outliers, which is why you also report percentiles.

The typical wrong version: `nested["user"]["address"]["zip"]` would stop the program with `KeyError: 'zip'`. Use `.get` with defaults when data may be incomplete.

The picture

```
Python object --dumps/dump--> text (JSON) --send/save--> other system
Python object <--loads/load-- text (JSON) <--receive/read--
        dict <-> {}     list <-> []     None <-> null
```

Connection to the real job

APIs return JSON. Config files, MLflow metadata, notebook files, and many logs are JSON. Event data in data lakes is often JSON Lines (one record per line), because you can append and stream it. In pandas you later use `pd.json_normalize` for nested data.

Applications

- **Data Analytics:** download JSON from a web service and flatten it into a table.
- **Data Engineering:** land event data as JSON Lines, one record per line, then load it into a warehouse.
- **Machine Learning:** save experiment settings and metrics as a small JSON file.
- **GenAI:** requests and replies of LLM APIs are JSON.

COMMON MISTAKE	Mixing up `dumps` / `loads` (strings) with `dump` / `load` (files). The `s` means string.
COMMON MISTAKE	Expecting tuples, sets, dates or integer keys to survive. Sets and dates raise errors; tuples and integer keys change silently.
COMMON MISTAKE	Using `data["a"]["b"]["c"]` on API data. One missing key crashes the job. Use `.get` with defaults, or catch `KeyError`.

Check yourself

1. Which function turns a JSON string into a dict? Answer: `json.loads`.
2. What does `None` become in JSON? Answer: `null`.
3. How do you save a `datetime` in JSON? Answer: convert to an ISO string (for example with `default=` and `isoformat()`).

5.3 CSV

INTUITION	CSV is a spreadsheet saved as text: one row per line, cells split by commas. It looks too simple to go wrong, but a city name like "Delhi, North" has a comma inside it. Splitting on commas by hand would cut it in two. The `csv` module handles quotes for you.

How it works

CSV means *comma-separated values*. Why a module and not `split(",")`? Because the format has rules: a cell that contains a comma, a quote or a line break must be wrapped in double quotes, and a quote inside a cell is doubled. The module knows all these rules.

Item	Example value	Where it appears in the job
Header row	`order_id,city,amount`	Names the columns for `DictReader`
Quoted cell	`"Delhi, North"`	City names, addresses, free text
Delimiter	`;` or tab	European exports and `.tsv` files
Customer list	`customers.csv`	Sample data for testing a program

- `csv.reader(file)` gives each row as a list of strings.
- `csv.DictReader(file)` uses the header row and gives each row as a dict. Best for readable code.
- `csv.writer` and `csv.DictWriter` write rows and add quotes when needed.
- When opening a real file for CSV, use `open(path, newline="", encoding="utf-8")`. The `newline=""` stops blank lines on Windows.
- **Everything is read as a string.** You must convert: `float(row["amount"])`.
- Another delimiter (like `;` or tab) is set with `delimiter=";"`.

Here we use `io.StringIO`, which is a fake file in memory, so the examples run without any file on disk. A real file works the same way.

Worked examples

Problem 5.3.1 Read an orders CSV with a quoted comma. Compute the total amount and the total per city.

Step 1	Use `DictReader`. Row 3 has `"Delhi, North"` inside quotes, so it stays as one cell.
Step 2	Convert amounts with `float`. Total = 250.50 + 1200 + 75.25 = 1525.75.
Step 3	Use `defaultdict(float)` to sum per city. (A `defaultdict(float)` is a dict that creates `0.0` for a missing key. Chapter 4 explains it fully.)

Predict: What will `rows[2]["city"]` be? One cell or two?

```
import csv, io
from collections import defaultdict
text = """order_id,city,amount
1,Pune,250.50
2,Chennai,1200
3,"Delhi, North",75.25
"""
rows = list(csv.DictReader(io.StringIO(text)))
print(rows[2])
print(sum(float(x["amount"]) for x in rows))
for row in csv.reader(io.StringIO(text)):
    print(row)
agg = defaultdict(float)
for x in rows:
    agg[x["city"]] += float(x["amount"])
print(dict(agg))
```

```
{'order_id': '3', 'city': 'Delhi, North', 'amount': '75.25'}
1525.75
['order_id', 'city', 'amount']
['1', 'Pune', '250.50']
['2', 'Chennai', '1200']
['3', 'Delhi, North', '75.25']
{'Pune': 250.5, 'Chennai': 1200.0, 'Delhi, North': 75.25}
```

- **Explain** `StringIO` pretends to be a file. `DictReader` reads the header, then makes one dict per row. The quoted comma stays inside one cell. `csv.reader` shows the same data as plain lists. The loop adds each amount to its city's running total.
- **Answer** The total is 1525.75. Notice every value in the dict is still a string until we convert it. Pune brought 250.5, Chennai 1200.0, and Delhi, North 75.25.
- **Check** By hand: 250.50 + 1200 = 1450.50, and 1450.50 + 75.25 = 1525.75. The code agrees, and 250.5 + 1200.0 + 75.25 from the dict also equals 1525.75.
- **Why it matters** In plain words: Chennai gave about 79% of the revenue (1200 / 1525.75). That kind of "group and sum" is the heart of analytics.

The typical wrong version. You forget that CSV values are strings and add them directly:

```
total = 0
for r in rows:
    total += r["amount"]
```

```
TypeError: unsupported operand type(s) for +=: 'int' and 'str'
```

The fix is `total += float(r["amount"])`. And splitting by hand also fails: `"Delhi, North".split(",")` gives `['Delhi', ' North']`, two pieces, which is wrong.

Problem 5.3.2 Write rows that contain quotes and commas, change the delimiter, and meet the "numbers as text" trap.

Step 1	`DictWriter` quotes cells automatically. A quote inside a cell is doubled (`""`).
Step 2	Read `"a;b;c"` with `delimiter=";"` .
Step 3	Sort the text values `"10", "9", "100"` and see the wrong order, then sort with `key=int` .

```
import csv, io
out = io.StringIO()
w = csv.DictWriter(out, fieldnames=["order_id", "city", "amount"], lineterminator="\n")
w.writeheader()
w.writerow({"order_id": 4, "city": 'Goa "Beach"', "amount": 10})
w.writerow({"order_id": 5, "city": "Kochi, Kerala", "amount": 20})
print(out.getvalue())
print(next(csv.reader(io.StringIO("a;b;c"), delimiter=";")))
print(sorted(["10", "9", "100"]), sorted(["10", "9", "100"], key=int))
```

```
order_id,city,amount
4,"Goa ""Beach""",10
5,"Kochi, Kerala",20

['a', 'b', 'c']
['10', '100', '9'] ['9', '10', '100']
```

- **Explain** `DictWriter` wrote the quote inside `Goa "Beach"` as `""` and wrapped both cells in quotes. `delimiter=";"` changed the separator. Sorting text compares the first character first: `"1"` is less than `"9"`, so `"10"` and `"100"` come before `"9"` .
- **Answer** Text sorts character by character ("10" < "100" < "9"), so convert to numbers first.
- **Check** Sorted as numbers, 9 < 10 < 100, which matches the second list.
- **Why it matters** A "top 5 orders" report that sorts text would show wrong orders and nobody would see an error message.

The picture

```
order_id,city,amount          ->  DictReader row:
3,"Delhi, North",75.25            {"order_id":"3",
     ^ comma inside quotes         "city":"Delhi, North",
       is NOT a separator          "amount":"75.25"}
```

Connection to the real job

CSV is the most common exchange format in analytics. You will load it with `pandas.read_csv`, but knowing the plain module helps when the file is too big, too messy, or you do not want to install pandas in a small script. Exports from a shop's order system are read the same way.

Applications

- **Data Analytics:** read Excel exports and BI tool downloads.

- **Data Engineering:** small ETL (extract, transform, load) jobs that clean a file and re-save it.
- **Software Testing:** build sample-data CSV files to feed automated tests.
- **Machine Learning:** read small training sets and label files.

COMMON MISTAKE Doing `line.split(",")`. It breaks on quoted commas. Use the `csv` module.

COMMON MISTAKE Forgetting that all CSV values are strings. `"250.50" + "1200"` joins text; it does not add.

COMMON MISTAKE Wrong encoding on Excel files (they may be `utf-8-sig` or `cp1252`). If you see strange characters at the start, try `encoding="utf-8-sig"`.

Check yourself

1. What type is every value from `csv.reader`? Answer: `str`.
2. Which class gives dict rows? Answer: `csv.DictReader`.
3. What does the writer do with a cell containing a comma? Answer: it wraps the cell in double quotes.

5.4 Dates, Times and Time Zones

INTUITION "9:00" means nothing until you say where. A test that starts at 9:00 in Chennai is 3:30 in the morning in London. Servers usually store time in UTC (Coordinated Universal Time, the world's reference clock), and show local time only when a person looks at it. Think of UTC as the one common ruler that every system measures against.

Words you need

Word	Plain meaning
UTC	The world's reference clock. No daylight saving. India is UTC+5:30
offset	How far a place is from UTC (for example +05:30)
time zone	A named region with its own offset rules, like `Asia/Kolkata`
daylight saving (DST)	Some places move clocks forward one hour in summer
timestamp	Seconds since 1 Jan 1970 UTC, as one number
ISO 8601	The standard text form: `2026-11-07T14:05:00+05:30`

Item	Example value	Where it appears in the job
ISO datetime	`2026-11-07T14:05:00+05:30`	API fields and log lines
Timestamp	`1767225600`	Timestamp column in a log export (some systems use milliseconds)
timedelta	`0:00:02.350000`	Delivery time or job duration
Time zone	`America/New_York`	Joining data from several regions

Formal definition

The `datetime` module has these classes:

- `date` : year, month, day.
- `datetime` : date plus time.
- `timedelta` : a length of time (days, seconds). Subtracting two dates gives a `timedelta` .
- `timezone.utc` : the UTC zone.
- `zoneinfo.ZoneInfo("Asia/Kolkata")` : a named time zone from the standard IANA database. It knows daylight saving rules.

A datetime is **naive** if it has no zone (`tzinfo` is empty) and **aware** if it has one. You cannot mix them in arithmetic.

Converting text and dates: `strptime(text, format)` parses text; `strftime(format)` makes text. Common codes: `%Y` year, `%m` month, `%d` day, `%H` hour (24h), `%M` minute, `%S` second, `%A` weekday name, `%B` month name. `datetime.fromisoformat(...)` reads ISO 8601 text such as `2026-11-07T14:05:00+05:30` .

Worked examples

Problem 5.4.1 Do date arithmetic and parsing.

Step 1	Make a date and ask for the weekday (`weekday()` gives Monday=0, so Sunday=6).
Step 2	Add `timedelta(days=20, hours=5)` to 15 March 2026 09:30.
Step 3	Count days from 7 Nov 2026 to 25 Dec 2026: 23 days left in November (30 - 7) plus 25 = 48.
Step 4	Parse `07/11/2026 14:05` with the day-first format.

Predict: What day of the week is 15 March 2026? (`weekday()` counts Monday as 0.)

```
from datetime import datetime, date, timedelta
d = date(2026, 3, 15)
print(d, d.weekday(), d.strftime("%A, %d %B %Y"))
dt = datetime(2026, 3, 15, 9, 30)
print(dt.isoformat(), dt + timedelta(days=20, hours=5))
print(date(2026, 12, 25) - date(2026, 11, 7))
p = datetime.strptime("07/11/2026 14:05", "%d/%m/%Y %H:%M")
print(p, p.year)
print(datetime.fromisoformat("2026-11-07T14:05:00+05:30"))
print(date(2026, 2, 28) + timedelta(days=1))
start = datetime(2026, 1, 1, 10, 0, 0)
end = datetime(2026, 1, 1, 10, 0, 2, 350000)
print((end - start).total_seconds(), end - start)
```

```
2026-03-15 6 Sunday, 15 March 2026
2026-03-15T09:30:00 2026-04-04 14:30:00
48 days, 0:00:00
2026-11-07 14:05:00 2026
2026-11-07 14:05:00+05:30
2026-03-01
2.35 0:00:02.350000
```

- **Explain** `strftime` turns a date into text using codes. `strptime` does the reverse, and its format must match the text exactly (`%d/%m/%Y` means day first). Adding a `timedelta` moves a date by a length of time. Subtracting two dates gives a `timedelta` .
- **Answer** The gap is 48 days, and 28 Feb 2026 plus one day is 1 March (2026 is not a leap year). `total_seconds()` gives a number you can use in delivery-time maths. 15 March 2026 is a Sunday.
- **Check** Days left: 30 - 7 = 23 in November, plus 25 in December = 48. And `2026 % 4 = 2` , so 2026 is not a leap year, so February has 28 days.

- **Why it matters** In plain words: a delivery-form step that took `0:00:02.350000` took 2.35 seconds, or 2350 ms. Converting to one unit makes it comparable with other results.

The typical wrong version. Parse a day-first date with the wrong format:

```
from datetime import datetime
datetime.strptime("07/11/2026", "%m/%d/%Y")   # reads 11 July
datetime.strptime("25/11/2026", "%m/%d/%Y")
```

```
ValueError: time data '25/11/2026' does not match format '%m/%d/%Y'
```

The first line does not fail, it silently reads 11 July instead of 7 November, which is worse than an error. Always know whether your source is day-first or month-first.

Problem 5.4.2 Convert 9:00 in India on 1 July 2026 to New York and UTC.

Step 1	India is UTC+5:30 all year. 09:00 IST is 03:30 UTC.
Step 2	New York in July is on daylight saving time, UTC-4. So 03:30 UTC is 23:30 on the previous day, 30 June.
Step 3	Let Python confirm with `astimezone`.

Predict: Which calendar day is it in New York when it is 09:00 on 1 July in India?

```
from datetime import datetime, timezone
from zoneinfo import ZoneInfo
ist = ZoneInfo("Asia/Kolkata")
ny = ZoneInfo("America/New_York")
t = datetime(2026, 7, 1, 9, 0, tzinfo=ist)
print(t, t.astimezone(ny), t.astimezone(timezone.utc))
```

```
2026-07-01 09:00:00+05:30 2026-06-30 23:30:00-04:00 2026-07-01 03:30:00+00:00
```

- **Explain** `tzinfo=ist` tells Python what zone the clock reading is in. `astimezone(zone)` keeps the instant and changes only how it is shown.
- **Answer** It is 23:30 on 30 June in New York. The same moment has three faces but is one instant.
- **Check** 09:00 - 5:30 = 03:30 UTC. 03:30 - 4:00 = -00:30, which is 23:30 of the previous day. Matches.
- **Why it matters** "Orders per day" for a New York customer differs from "orders per day" in India. You must decide the zone before you group by day.

Problem 5.4.3 Show the daylight saving trap and the naive/aware error.

Step 1	In New York, clocks jumped forward on 8 March 2026 at 02:00 (to 03:00). So 02:30 does not exist that day.
Step 2	Take 01:30 and add one hour with plain wall-clock arithmetic. Python gives 02:30, a time that never happened.
Step 3	Convert both to UTC and subtract. This is the safe way to measure real time passed.
Step 4	Try subtracting a naive and an aware datetime.

```
from datetime import datetime, timedelta, timezone
from zoneinfo import ZoneInfo
ny = ZoneInfo("America/New_York")
a = datetime(2026, 3, 8, 1, 30, tzinfo=ny)
b = a + timedelta(hours=1)
print(a, b, b.utcoffset())
print(b.astimezone(timezone.utc) - a.astimezone(timezone.utc))
try:
    datetime(2026, 1, 1) - datetime(2026, 1, 1, tzinfo=timezone.utc)
except TypeError as e:
    print("TypeError:", e)
print(datetime.fromtimestamp(1_800_000_000, tz=timezone.utc))
print(datetime(2026, 1, 1, tzinfo=timezone.utc).timestamp())
```

```
2026-03-08 01:30:00-05:00 2026-03-08 02:30:00-05:00 -1 day, 19:00:00
1:00:00
TypeError: can't subtract offset-naive and offset-aware datetimes
2027-01-15 08:00:00+00:00
1767225600.0
```

- **Explain** Python's `+ timedelta(hours=1)` moves the *clock reading* by one hour and does not check if that reading exists. Only converting to UTC reveals the true time passed.
- **Answer** `-1 day, 19:00:00` is how Python prints a negative offset of -5 hours. The "02:30" result is a wall-clock time that does not exist in real New York time, so the lesson is: do arithmetic in UTC and convert to local only for display. A Unix timestamp (seconds since 1 Jan 1970 UTC) is a plain number, for example 1767225600.0 for 1 Jan 2026.
- **Check** UTC difference printed `1:00:00`; the offset `-1 day, 19:00:00` equals -24h + 19h = -5h. Correct for New York before DST starts.
- **Why it matters** Twice a year, hourly reports from daylight saving regions get 23 or 25 hours in a day. Using UTC avoids this.

For timing code (how long did this take), use `time.perf_counter()`, which is a steady clock that does not jump. Do not use dates for that.

The picture

```
Store/compute in UTC:   2026-07-01 03:30 UTC   <- one real instant
Show to people:   Kolkata 09:00 (+5:30)   New York 23:30 prev day (-4:00)

naive (no zone)  --X-->  mixing with aware = TypeError
```

Connection to the real job

Logs from servers in different regions need one clock before you can join them. Daily aggregations ("orders per day") depend on which time zone defines "day". Time-series features for ML (hour of day, day of week) are wrong if time zones are mixed. Timestamps from many systems are stored as seconds or milliseconds since 1970 (a timestamp).

Applications

- **Data Analytics:** daily, weekly and monthly reports with a clear time zone for "day".
- **Data Science:** time features such as hour of day and weekday for forecasting.
- **Data Engineering:** schedulers (cron, Airflow) run jobs in UTC, and partitions are named by date.
- **Performance Testing:** line up test start and end times with server monitoring graphs.

COMMON MISTAKE	Using naive datetimes and hoping everyone means the same zone. Make datetimes aware, and store UTC.

COMMON MISTAKE	Adding timedelta to local aware times near a daylight saving change. Convert to UTC first.

COMMON MISTAKE	Writing `%m` when you mean minutes. Month is `%m`, minute is `%M`.

Check yourself

1. What does `date(2026,3,1) - date(2026,2,1)` give? Answer: `timedelta(days=28)`.
2. Why store times in UTC? Answer: it is one common ruler without daylight saving jumps.
3. What error appears when you subtract naive from aware? Answer: `TypeError: can't subtract offset-naive and offset-aware datetimes`.

5.5 Configuration: Environment Variables, `.env` Files, YAML and TOML

INTUITION	The same program runs on your laptop, a test server, and production. The database address is different in each. If you type the address inside the code, you must edit code for each place. Instead, keep settings outside the code and let each machine provide its own. This is like a restaurant kitchen reading the day's specials from a board instead of rewriting the recipe book.

How it works

Words you need: an *environment variable* is a named text value that the operating system attaches to every program it starts (like a sticky note on the program's desk). A *secret* is a value that must stay private, such as a password or API key.

Item	Example value	Where it appears in the job
Env variable	`DB_HOST=db.internal`	Docker, Kubernetes, GitHub Actions
`.env` file	`API_KEY=fake-key-123`	Your laptop during development
TOML file	`[db] port = 5432`	`pyproject.toml` and tool settings
YAML file	`db:` then `port: 5432`	Airflow, Kubernetes, CI pipelines

Three common places for settings, from most to least specific:

1. **Environment variables**: named values the operating system gives to a program. In Python: `os.environ.get("NAME", "default")`. `os.environ["NAME"]` raises `KeyError` if it is missing (good when the setting is required).
2. **`.env` file**: a text file of `NAME=value` lines used on a developer machine. A helper library (`python-dotenv`) loads it into the environment. The file must be listed in `.gitignore` and **never committed**, because it often holds secrets.
3. **Config files**: YAML (needs the third-party `PyYAML` package, not run here), TOML (`tomllib` is in the standard library from Python 3.11, read-only), or JSON.

Environment variables are always strings. Convert: `int(os.environ.get("DB_PORT", "5432"))`.

Worked examples

Problem 5.5.1 Write a small `.env` loader, read the file, and let a real environment variable override it.

Step 1	Skip blank lines and `#` comments. Split each line at the first `=`.
Step 2	Strip quotes around values.
Step 3	Convert the port to an `int`. Then override `APP_ENV` from the real environment. The key below is fake.

Predict: After we set `os.environ["APP_ENV"] = "prod"`, which value of `APP_ENV` wins, `dev` from the file or `prod`?

```
import os, tempfile
from pathlib import Path

def load_dotenv(path):
    env = {}
    for line in Path(path).read_text().splitlines():
        line = line.strip()
        if not line or line.startswith("#") or "=" not in line:
            continue
        k, v = line.split("=", 1)
        env[k.strip()] = v.strip().strip('"').strip("'")
    return env

with tempfile.TemporaryDirectory() as d:
    f = Path(d) / ".env"
    f.write_text('# demo\nAPP_ENV=dev\nDB_HOST="localhost"\nDB_PORT=5432\nAPI_KEY=fake-key-123\n')
    cfg = load_dotenv(f)
print(cfg)
port = int(os.environ.get("DB_PORT", cfg["DB_PORT"]))
print(port + 1)
os.environ["APP_ENV"] = "prod"
merged = {**cfg, **{k: v for k, v in os.environ.items() if k in cfg}}
print(merged["APP_ENV"], merged["DB_HOST"])
try:
    os.environ["REQUIRED_X"]
except KeyError as e:
    print("KeyError", e)
```

```
{'APP_ENV': 'dev', 'DB_HOST': 'localhost', 'DB_PORT': '5432', 'API_KEY': 'fake-key-123'}
5433
prod localhost
KeyError 'REQUIRED_X'
```

- **Explain** `split("=", 1)` cuts at the first `=` only, so values may contain `=`. The `{**cfg, **other}` merge keeps `cfg` first and lets the second dict overwrite same keys. `os.environ["X"]` fails loudly, `os.environ.get("X")` returns `None`.
- **Check** 5432 + 1 = 5433, matching the output. `merged["APP_ENV"]` is `prod`, so the override worked, and `DB_HOST` stayed `localhost`.
- **Why it matters** The same code now runs in dev and prod with no edits, and the secret is never in the code.
- **Answer** The environment value `prod` beat the file value `dev`, which is the usual rule: real environment wins over `.env`. The required missing variable fails loudly with `KeyError`.

Problem 5.5.2 Read a TOML config and understand what a YAML file looks like.

Step 1	`tomllib.loads` parses a TOML string into nested dicts.
Step 2	YAML expresses the same data with indentation (shown as text only; PyYAML is not installed here, so we do not run it).

```
import tomllib
print(tomllib.loads('[db]\nhost="h"\nport=5432\n'))
```

```
{'db': {'host': 'h', 'port': 5432}}
```

```
# not run here: needs PyYAML (yaml.safe_load)
db:
  host: h
  port: 5432
```

- **Answer** Both forms give `{"db": {"host": "h", "port": 5432}}`. If you use PyYAML, always call `yaml.safe_load`, never plain `yaml.load` on untrusted text.

The picture

```
code  <-- reads --  environment variables (set by server, CI, Docker)
                    .env file (laptop only, in .gitignore)
                    config.toml / config.yaml (non-secret defaults)
Priority:  env var  >  .env  >  config file  >  default in code
```

Connection to the real job

Every deployed data pipeline, API and ML service takes its database URL, bucket name and API keys from the environment. Docker, Kubernetes, GitHub Actions and Airflow all inject settings as environment variables. Chapter 13 covers secrets in depth.

The typical wrong version. Trusting that an environment value is a bool:

```
import os
print(bool(os.environ.get("DEBUG_X", "False")))
```

```
True
```

The text `"False"` is a non-empty string, so it is truthy. Compare the text instead: `os.environ.get("DEBUG_X", "false").lower() == "true"`.

Applications

- **Data Engineering:** pipeline settings (bucket names, schedule) differ per environment.
- **MLOps:** model-serving containers receive their model path and API keys as env variables.
- **Machine Learning:** notebooks read API keys with `os.environ` instead of pasting them in cells.
- **Performance Testing:** point the same test script at dev, QA or staging with one variable.

COMMON MISTAKE	Committing `.env` to Git. Add `.env` to `.gitignore` before the first commit, and commit a `.env.example` with fake values instead.
COMMON MISTAKE	Forgetting that environment values are strings. `os.environ.get("DEBUG")` returns `"False"`, which is truthy!
COMMON MISTAKE	Hard-coding a "temporary" password in code. It always stays.

Check yourself

1. What type is `os.environ["PORT"]`? Answer: `str`.
2. What does `os.environ["X"]` do if `X` is missing? Answer: raises `KeyError`.
3. Which should win: `.env` or a real environment variable? Answer: the real environment variable.

5.6 Handling Archives, Temporary Files, Deprecated Time Calls and Pickle

Real data work moves compressed files around, writes scratch files, and keeps old code alive. This section covers the standard-library tools for that and two traps: `datetime.utcnow()` and `pickle`.

COMMON MISTAKE	`datetime.utcnow()` and `datetime.utcfromtimestamp()` are deprecated since Python 3.12. They return a "naive" time (no time zone attached) that looks like UTC but is easy to confuse with local time. Use `datetime.now(timezone.utc)`, which returns an "aware" time that carries its zone. Section 5.4 explains aware versus naive.

Problem 5.6.1 Compress a CSV with `gzip`, bundle files in a `zipfile`, copy with `shutil`, all inside a `TemporaryDirectory` that deletes itself. Also show the deprecation warning.

Step 1	`warnings.catch_warnings(record=True)` with the filter set to "always" captures the warning `utcnow()` raises. The category is `DeprecationWarning`.
Step 2	Write a 4,000 byte CSV (the line pair is repeated 200 times). Copy it into a `.gz` file with `shutil.copyfileobj`, which streams in chunks, so it works for files larger than memory.
Step 3	Read the gzip back in text mode (`"rt"`). Build a zip with `ZipFile.write` (a file) and `writestr` (text made in memory).
Step 4	Leave the `with` block: the directory and everything in it is gone, so `root.exists()` is `False`.

```
import gzip, shutil, tempfile, zipfile, warnings
from datetime import datetime, timezone
from pathlib import Path
with warnings.catch_warnings(record=True) as w:
    warnings.simplefilter("always")
    datetime.utcnow()
print(w[0].category.__name__)
print(datetime.now(timezone.utc).tzinfo)
with tempfile.TemporaryDirectory() as d:
    root = Path(d)
    (root / "data.csv").write_text("id,amount\n1,10\n2,20\n" * 200)
    raw = (root / "data.csv").stat().st_size
    with open(root / "data.csv", "rb") as src, gzip.open(root / "data.csv.gz", "wb") as dst:
        shutil.copyfileobj(src, dst)
    gz = (root / "data.csv.gz").stat().st_size
    print(raw, gz, gz < raw)
    with gzip.open(root / "data.csv.gz", "rt") as f:
        print(f.readline().strip(), f.readline().strip())
    with zipfile.ZipFile(root / "bundle.zip", "w", zipfile.ZIP_DEFLATED) as z:
        z.write(root / "data.csv", arcname="data.csv")
        z.writestr("notes/readme.txt", "hello")
    with zipfile.ZipFile(root / "bundle.zip") as z:
        print(z.namelist())
        print(z.read("notes/readme.txt").decode())
    copy = shutil.copy(root / "data.csv", root / "backup.csv")
    print(Path(copy).name, sorted(p.name for p in root.iterdir()))
print(root.exists())
```

```
DeprecationWarning
UTC
4000 77 True
id,amount 1,10
['data.csv', 'notes/readme.txt']
hello
backup.csv ['backup.csv', 'bundle.zip', 'data.csv', 'data.csv.gz']
False
```

The CSV shrank from 4,000 to 77 bytes because it is one repeated pair of lines. Real data compresses less. Text such as CSV and logs usually compresses well; images and already compressed files do not.

Pickle: convenient and dangerous

`pickle` saves almost any Python object to bytes and loads it back. That convenience is also the danger: unpickling can **run code**. An object can define `__reduce__` to tell pickle "to rebuild me, call this function with these arguments", and that function can be anything. So loading a pickle from a stranger is like running their program.

Problem 5.6.2 Round-trip a dict, then show that loading a crafted pickle executes code.

Safe use	`pickle.loads(pickle.dumps(data))` equals the original.
Attack	A class whose `__reduce__` returns `(print, ("...",))`. Loading it calls `print`. Real attackers return `os.system` instead.
Safe alternative	JSON carries only data (dicts, lists, strings, numbers), never code.

```
import pickle, json
data = {"user": "ada", "scores": [1, 2, 3]}
print(pickle.loads(pickle.dumps(data)) == data)
class Evil:
    def __reduce__(self):
        return (print, ("code ran during unpickling",))
pickle.loads(pickle.dumps(Evil()))
print(json.loads(json.dumps(data)) == data)
```

```
True
code ran during unpickling
True
```

KEY IDEA	Only unpickle bytes you created yourself and stored where nobody else can write. For anything that crosses a trust boundary (uploads, network, shared folders) use JSON, CSV or Parquet. Chapter 13 repeats this rule in the security section.

Chapter recap

- Use `pathlib.Path` and the `/` operator for paths; it works on every operating system.
- Always open text files with `encoding="utf-8"` and use `with` so they close.
- Loop over a file object line by line for big files; do not read it all at once.
- `"w"` erases, `"a"` appends. `shutil` copies and deletes trees; `tempfile` cleans up after itself.
- JSON maps to dict, list, str, number, bool, None. `dumps/loads` work on strings, `dump/load` on files.
- Tuples become lists in JSON, integer keys become strings, and dates need `default=` or conversion.
- The `csv` module handles quotes and commas; every value arrives as a string.
- Store and compute time in UTC with aware datetimes; convert to local time only for display.
- Do arithmetic across daylight saving changes in UTC, and time code with `time.perf_counter()`.
- Keep settings in environment variables or config files, never in code, and never commit `.env`.

Cheat-sheet

Task	Pattern
Build a path	`Path("a") / "b.csv"`
Read / write small text	`p.read_text(encoding="utf-8")`, `p.write_text(...)`
Loop over big file	`with open(p, encoding="utf-8") as f: for line in f:`

Task	Pattern
Find files	`root.rglob("*.log")`
JSON to / from string	`json.dumps(obj)`, `json.loads(text)`
CSV rows as dicts	`csv.DictReader(f)` then `float(row["x"])`
Aware time	`datetime(..., tzinfo=ZoneInfo("Asia/Kolkata"))`
Change zone	`dt.astimezone(ZoneInfo("America/New_York"))`
Setting with default	`os.environ.get("NAME", "default")`

Practice set

Problem 5.P.1 What is `Path("/data/raw/sales_2026.csv").suffix`?

Answer: `.csv`.

Problem 5.P.2 What is the output? `json.dumps({"a": (1, 2), "b": None})`

Answer: `{"a": [1, 2], "b": null}`.

Problem 5.P.3 What does `date(2026, 3, 1) - date(2026, 2, 1)` print?

Answer: `28 days, 0:00:00`.

Problem 5.P.4 Find the bug: `total = 0` then `for row in csv.DictReader(f): total += row["amount"]`.

Answer: `row["amount"]` is a string, so `0 + "250"` raises `TypeError`. Use `float(row["amount"])`.

Problem 5.P.5 What is the output? `sorted(["b10", "b9", "b100"])`

Answer: `['b10', 'b100', 'b9']`, because strings compare character by character.

Problem 5.P.6 Convert 18:00 UTC on 1 January 2026 to India time. How would you write it?

Answer: India is UTC+5:30, so it is 23:30 the same day. In code:
`datetime(2026,1,1,18,0,tzinfo=timezone.utc).astimezone(ZoneInfo("Asia/Kolkata"))`.

Problem 5.P.7 Find the bug: `with open("log.txt", "w") as f:` was used to add one line to an existing log.

Answer: Mode `"w"` erases the file when opened. Use `"a"`.

Problem 5.P.8 Scenario: a daily job fails on Mondays only, with `UnicodeDecodeError`. The file comes from a partner. What do you check first?

Answer: The file encoding. Open with `encoding="utf-8"` and check the raw bytes; the partner's Monday export may use another encoding (such as `cp1252`) or add a BOM (try `utf-8-sig`). Log the file name when it fails.

Problem 5.P.9 Write a function that returns the number of lines in a text file without loading all of it.

Answer:

```
def count_lines(path):
    with open(path, encoding="utf-8") as f:
        return sum(1 for _ in f)
```

Problem 5.P.10 Interview: why should servers store timestamps in UTC?

Answer: UTC has no daylight saving jumps, so ordering and subtraction are always correct. Different regions can be compared. Convert to the user's zone only when displaying.

Problem 5.P.11 Interview: your API response contains `datetime` and `Decimal` values and `json.dumps` fails. What are two ways to fix it?

Answer: Pass `default=` to `json.dumps` with a function that converts them (`isoformat()` for dates, `str` or `float` for `Decimal`), or convert the data to simple types before dumping.

More practice: build, stretch and review

Problem 5.P.12 (build) Write `load_config(path, defaults)` that returns a copy of `defaults` updated with the JSON object stored at `path` , and returns just the defaults if the file does not exist. Test with no file, then with a file containing `{"port": 8080}` .

```
import json, tempfile
from pathlib import Path
def load_config(path, defaults):
    cfg = dict(defaults)
    p = Path(path)
    if p.exists():
        cfg.update(json.loads(p.read_text()))
    return cfg
with tempfile.TemporaryDirectory() as d:
    f = Path(d) / "c.json"
    print(load_config(f, {"port": 80, "debug": False}))
    f.write_text('{"port": 8080}')
    print(load_config(f, {"port": 80, "debug": False}))
```

```
{'port': 80, 'debug': False}
{'port': 8080, 'debug': False}
```

Answer: as shown. `dict(defaults)` copies so the caller's defaults are not changed. File values override defaults, and keys missing from the file keep their default. Self-check: what should happen for a file with invalid JSON? (Let `json.JSONDecodeError` escape, or wrap it with a message that names the path.)

Problem 5.P.13 (what is the output?) For `t = datetime(2026, 3, 8, 23, 30, tzinfo=timezone.utc)` , what is `(t + timedelta(hours=1)).isoformat()` , and does `datetime.fromisoformat("2026-03-08T23:30:00+00:00") == t` ?

```
from datetime import datetime, timezone, timedelta
t = datetime(2026, 3, 8, 23, 30, tzinfo=timezone.utc)
print((t + timedelta(hours=1)).isoformat())
print(datetime.fromisoformat("2026-03-08T23:30:00+00:00") == t)
```

```
2026-03-09T00:30:00+00:00
True
```

Answer: One hour after 23:30 crosses midnight, so the date rolls to March 9. The parsed time is aware and in UTC, so it equals `t` .

Problem 5.P.14 (interview) A colleague stores machine-learning models with `pickle` and loads files that users upload. What is wrong, and what do you propose?

Answer: Unpickling an uploaded file can run arbitrary code on your server (Problem 5.6.2). Never unpickle untrusted data. Options: accept only a data format (JSON, a safe model format such as ONNX or safetensors), or sign files with an HMAC key you hold and verify before loading, and run the loader in a low-privilege process.

Problem 5.P.15 (stretch, rubric) Write `archive_logs(folder)` that gzips every `.log` file in a folder into `name.log.gz` , deletes the original only after the gzip is written, and returns the number of bytes saved. Rubric (1 point each): (1) uses `pathlib` to find files; (2) streams with `shutil.copyfileobj` ; (3) original removed only after a successful write; (4) works on an empty folder; (5) tested inside a `TemporaryDirectory` .

Answer: No single answer; full marks is 5/5. The common bug is deleting the source inside the same `with` block before the gzip file is closed and flushed; delete after the block exits.

Prompt pack

✨ **AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving into data roles in India, and English is my second language, so use simple words. Teach me Python file handling and paths: pathlib, open modes, encodings, glob/rglob, shutil and tempfile. Explain step by step with small runnable examples, then check my understanding with 5 short questions one at a time. Wait for my answer before the next question and correct me kindly.
```

✨ **AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving into data roles in India, and English is my second language. Topic: Python JSON handling with json.dumps, loads, dump, load, indent, default and ensure_ascii. Give me 8 graded exercises from easy to hard, using realistic API responses and JSON Lines logs. Do not give solutions yet. After I answer each one, I will paste my code and you will review it for bugs and style.
```

✨ **AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving into data roles in India. Topic: reading and writing CSV in Python with the csv module (reader, DictReader, writer, DictWriter), quoting, delimiters, encodings and type conversion. Review my work: [PASTE HERE]. Find bugs, explain why they are bugs in simple English, and show a cleaner version. Then give me one harder follow-up task.
```

✨ **AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving into data roles in India, and English is my second language. Topic: Python datetime, timedelta, strptime/strftime, zoneinfo, naive versus aware datetimes, UTC, and daylight saving traps. Quiz me one question at a time with real numbers (for example converting times between India, UTC and New York). Increase difficulty slowly, and explain each wrong answer step by step.
```

✨ **AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving into data roles in India. Topic: configuration in Python using environment variables, .env files, TOML and YAML, and keeping secrets out of Git. Run a mock interview with me: ask 6 questions about configuration and secrets, one at a time, grade each answer out of 10, and tell me what a strong answer would include. End with a summary of my weak points.
```

CHAPTER 6

ERRORS, EXCEPTIONS, LOGGING AND DEBUGGING

Programs fail. Professionals are the people who know how to read the failure, handle it calmly, leave a trail in the logs, and find the cause fast.

Think of the worst moment of a big task: the program is half done and it dies with a wall of red text. If you can read that text, you fix the problem in two minutes. If you cannot, you lose the afternoon. In data work it is the same: a nightly job reads 2 million rows, row 1,400,001 has a strange value, and the job stops. This chapter teaches you four skills: reading a traceback, handling errors with `try/except`, writing your own errors, and leaving evidence with `logging`. Then you learn `pdb`, Python's built-in debugger, so you can stop a program in the middle and look inside it. By the end you can take any crash and walk to its cause step by step.

Running example: a small function that adds up shop prices like `"Rs100"`. We make it fail, read the failure, handle it, log it and debug it. All code was run with `python3` (Python 3.14). File paths in tracebacks (like `/private/tmp/w6/tb.py`) will be different on your computer.

6.1 Reading Tracebacks and the Common Exceptions

INTUITION

A traceback is like the receipt of a failed journey. It lists every road the program took, in order, and says where the car finally broke down. A beginner reads the top and panics. A professional reads the **last line first** (what broke), then the line above it (where it broke), then walks up the list (how we got there).

Words you need

Word	Plain meaning
exception	An object Python creates when something goes wrong; it stops the normal flow
raise	To create and throw an exception (Python does it for you, or you do it)
traceback	The printed list of function calls that led to the exception
call stack	The chain of "function A called B, B called C" at one moment
bug	A mistake in your code. An exception is how Python tells you about it

How to read one

Here is a real failure. `total` calls `parse_price` for each text. The third text is `"N/A"`.

Predict: Before you run it, which line of the three functions do you think will be named last in the traceback?

```
def parse_price(text):
    return float(text.replace("Rs", ""))

def total(prices):
    s = 0
    for p in prices:
        s += parse_price(p)
    return s

print(total(["Rs100", "Rs250", "N/A"]))
```

```
Traceback (most recent call last):
  File "/private/tmp/w6/tb.py", line 10, in <module>
    print(total(["Rs100", "Rs250", "N/A"]))
          ~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/private/tmp/w6/tb.py", line 7, in total
    s += parse_price(p)
         ~~~~~~~~~~~^^^
  File "/private/tmp/w6/tb.py", line 2, in parse_price
    return float(text.replace("Rs", ""))
ValueError: could not convert string to float: 'N/A'
```

- **Explain** Read from the bottom. The last line says the *type* (`ValueError`) and the *message* (`could not convert string to float: 'N/A'`). The block above it points to line 2 in `parse_price` : that is where Python stopped. The two blocks above that show the callers: `total` at line 7, and the main script at line 10. The `~~~~^^^^` marks show which part of the line failed (newer Python versions draw them).
- **Answer** The program failed in `parse_price` because `"N/A"` is not a number. The bad value is in the *input data*, but the failing line is in *our function*. The fix is to handle the case, which is the topic of 6.2.

The common exception types

Learn these ten by sight. Each line below was produced by running code and catching the error.

```
def attempt(f):
    try:
        print("ok:", f())
    except Exception as e:
        print(type(e).__name__, "->", e)

attempt(lambda: 1/0)
attempt(lambda: int("abc"))
attempt(lambda: [1,2,3][5])
attempt(lambda: {"a":1}["b"])
attempt(lambda: None.upper())
attempt(lambda: "5"+5)
attempt(lambda: undefined_name)
attempt(lambda: open("/no/such/file.txt"))
attempt(lambda: __import__("not_a_module"))
attempt(lambda: (1,2).append(3))
```

```
ZeroDivisionError -> division by zero
ValueError -> invalid literal for int() with base 10: 'abc'
IndexError -> list index out of range
KeyError -> 'b'
AttributeError -> 'NoneType' object has no attribute 'upper'
TypeError -> can only concatenate str (not "int") to str
NameError -> name 'undefined_name' is not defined
FileNotFoundError -> [Errno 2] No such file or directory: '/no/such/file.txt'
ModuleNotFoundError -> No module named 'not_a_module'
AttributeError -> 'tuple' object has no attribute 'append'
```

Exception	Usually means	Typical cause in data work
`ValueError`	Right type, wrong value	`int("abc")` , a bad cell in a CSV
`TypeError`	Wrong type	adding a string from a CSV to a number
`KeyError`	Missing dict key	an API reply without an expected field
`IndexError`	List position too big	an empty result list

Exception	Usually means	Typical cause in data work
`AttributeError`	Object has no such attribute	a function returned `None` and you used its result
`NameError`	Name not defined	typo in a variable name
`FileNotFoundError`	Path is wrong	wrong working folder
`ModuleNotFoundError`	Package not installed	wrong virtual environment (Chapter 9)
`ZeroDivisionError`	Divide by zero	average of an empty list

All of these are subclasses of `Exception`. Think of a family tree: `Exception` is the parent, `LookupError` is the parent of `KeyError` and `IndexError`, and so on. `except LookupError` catches both children.

The picture

```
<module> line 10  ──calls──▶  total line 7  ──calls──▶  parse_price line 2  ✗ ValueError
   (oldest call)                                           (where it broke: look here first,
                                                            then walk back up to find who passed 'N/A')
```

Read the traceback *bottom to top*: the last line is the symptom, the frames above are the history.

Connection to the real job

In production you rarely sit next to the program. The traceback in a log file or an Airflow task page is often all you get. A teammate who reads it correctly fixes the job in minutes. It is like a hospital reading a patient's chart to find where a problem began; reading a traceback is the same skill.

Applications

- **Data Analytics:** a notebook cell dies with `KeyError: 'revenue'`; you check the column names.
- **Data Engineering:** an Airflow task fails with `FileNotFoundError`; you check whether the upstream job produced the file.
- **Machine Learning:** `ValueError: shapes not aligned` points to a matrix mismatch.
- **MLOps:** a deployed API returns 500; the error type in the logs tells you which input broke it.

COMMON MISTAKE Reading from the top and getting lost in library code. Start at the last line, then find the *first* frame that is your own file.

COMMON MISTAKE Fixing the line named in the traceback when the real problem is in the data or in a caller. Ask who passed the bad value.

COMMON MISTAKE Copying only the last line when asking for help. Always paste the whole traceback.

Check yourself

1. In a traceback, which line tells you the exception type? Answer: the last line.
2. Which exception does `{"a": 1}["b"]` raise? Answer: `KeyError`.
3. `None.upper()` raises which error, and what does it usually mean? Answer: `AttributeError`; a variable you thought held text is `None`.

6.2 Handling Errors: `try`, `except`, `else`, `finally`

INTUITION	Picture a shop counter. Most customers pay normally. Sometimes a card is declined. A good cashier does not close the shop; she says "please try another card" and carries on. `try/except` lets your program react to a *known* kind of problem and keep working, while still stopping on problems you did not expect.

How it works

```
try:        code that might fail
except SomeError:   what to do if THAT error happens
else:       runs only if the try block had NO error
finally:    runs ALWAYS (cleanup: close, release, unlock)
```

- Catch **specific** exceptions. `except ValueError` is good. A bare `except:` or `except Exception:` hides bugs.
- You can catch several types: `except (ValueError, TypeError) as e:`. The `as e` gives you the exception object.
- `else` is for code that should run only on success. It keeps the `try` block short.
- `finally` runs even when there is a `return` or an error. Use it for cleanup.
- Python style prefers **EAFP**: "Easier to Ask Forgiveness than Permission" (just try, and catch the error). The other style is **LBYL**: "Look Before You Leap" (check first with `if`). EAFP is often shorter and avoids race conditions (the file could vanish between the check and the use).

Worked examples

Problem 6.2.1 Make `parse_price` safe: return `None` for bad text, and show what `else` and `finally` do.

Step 1	Wrap the risky call (`int(s)`) in `try` . Catch only `ValueError` .
Step 2	Put the "success" message in `else` .
Step 3	Put the "always" message in `finally` .

Predict: For the input `"x"` , does the `finally` message print? In what order do the lines appear?

```
def read_num(s):
    try:
        n = int(s)
    except ValueError:
        print("  bad number")
        return None
    else:
        print("  parsed", n)
        return n
    finally:
        print("  finally runs for", repr(s))

read_num("42")
read_num("x")
```

```
  parsed 42
  finally runs for '42'
  bad number
  finally runs for 'x'
```

- **Explain** For `"42"` , `int` works, so Python skips `except` , runs `else` , and then `finally` . For `"x"` , `int` raises `ValueError` , `except` runs and returns `None` , and `finally` still runs *before* the function actually returns.

- **Answer** `finally` ran in both cases, and after the `except` or `else` block.
- **Check** Count the lines: two calls produce two `finally` lines. They did.
- **Why it matters** Anything that must happen (closing a connection, releasing a lock) belongs in `finally`, because it runs even on failure.

The typical wrong version. A bare `except:` hides real mistakes:

```
try:
    total = prise + 1      # typo: should be price
except:
    total = 0
```

This never shows an error, and the total is silently 0. Python would have told you `NameError: name 'prise' is not defined`. Catching everything turns a loud bug into a quiet one. The fix: `except ValueError:` (or the exact type you expect).

Problem 6.2.2 Run `100 // int(v)` for the inputs `"10"`, `"0"`, `"zz"` and `None`, and handle each kind of failure separately.

Step 1	`int("zz")` raises `ValueError`. `int(None)` raises `TypeError`. `100 // 0` raises `ZeroDivisionError`.
Step 2	Group the first two into one `except` with a tuple, because we treat them the same.
Step 3	Handle `ZeroDivisionError` on its own.

```
for v in ["10", "0", "zz", None]:
    try:
        print(100 // int(v))
    except (ValueError, TypeError) as e:
        print("input problem:", type(e).__name__)
    except ZeroDivisionError:
        print("zero")
```

```
10
zero
input problem: ValueError
input problem: TypeError
```

- **Answer** `100 // 10 = 10`. `"0"` gives `zero`. `"zz"` and `None` are input problems.
- **Check** `100 // 10` is 10 by hand. Each of the other three inputs hit exactly the branch we predicted.
- **Why it matters** In plain words: bad rows are separated by cause, so you can count them and report "3 bad numbers, 1 zero" instead of crashing.

Problem 6.2.3 Compare LBYL and EAFP for a dictionary lookup, and use the loop `else`.

Step 1	LBYL: `if "b" in d`. EAFP: `try ... except KeyError`.
Step 2	Both give the same answer here. EAFP is better when the check itself is as costly as the action, or when things change between check and use.
Step 3	A `for ... else` runs its `else` only if the loop did not `break`.

```
d = {"a": 1}
if "b" in d:
    x = d["b"]
else:
    x = 0
try:
    y = d["b"]
except KeyError:
    y = 0
print(x, y)

for n in [3, 5, 7]:
    if n % 2 == 0:
        print("found even"); break
else:
    print("no even")
```

```
0 0
no even
```

- **Answer** Both styles gave `0`. The loop printed `no even` because no number was even, so `break` never ran.
- **Check** 3, 5, 7 are all odd: `n % 2` is 1 each time, so the condition is never true.
- **Why it matters** For simple dict defaults you will use `d.get("b", 0)` instead; both patterns are good to know for interviews.

The picture

```
try block ──no error──▶ else block ──▶ finally ──▶ continue
    │
  error ──▶ matching except block ──▶ finally ──▶ continue
    │
  no match ──▶ finally ──▶ error keeps travelling up to the caller
```

Connection to the real job

ETL jobs process millions of rows. A good job catches the expected row-level problem (`ValueError` on a bad cell), counts it, and moves on, but lets unexpected problems (a database outage) stop the job loudly. Retry loops (Chapter 7) are built from `try/except`.

Applications

- **Data Engineering:** skip and quarantine bad rows; always close database connections in `finally`.
- **Data Analytics:** cleaning scripts that convert messy text to numbers and report the failures.
- **Machine Learning:** a training loop that catches a corrupt image file and continues.
- **API / Backend:** return a clear 400 message for bad input instead of crashing with a 500.

COMMON MISTAKE	`except Exception: pass`. You are hiding the problem. At least log it (6.4).
COMMON MISTAKE	Putting a lot of code inside `try`. Then you do not know which line failed. Keep `try` small, and use `else` for the rest.
COMMON MISTAKE	Catching an exception and then using a variable that was never set (because the failing line was the one that set it).

Check yourself

1. When does `else` in a try block run? Answer: only if no exception happened in `try`.
2. When does `finally` run? Answer: always, even after `return` or an unhandled error.
3. What does EAFP stand for? Answer: Easier to Ask Forgiveness than Permission.

6.3 Raising, Custom Exceptions and Chaining

INTUITION	Sometimes your code finds the problem, not Python. A function that receives an age of -3 must refuse it with a clear message. Raising your own exception is like a quality inspector stopping the production line and writing a note about *why*.

How it works

- `raise ValueError("age must be >= 0")` throws an exception you choose.
- A bare `raise` inside an `except` block re-throws the *same* exception. Use it after logging.
- A **custom exception** is a small class that inherits from `Exception`. Give your project one base error (`AppError`) and children for each case. Callers can then catch the whole family or one member.
- **Chaining:** `raise NewError("...") from e` links the new error to the old cause. The traceback then says "The above exception was the direct cause of the following exception", so nothing is lost. Without `from`, Python still links them as `__context__`, but with the message "During handling of the above exception, another exception occurred".
- `assert condition, "message"` raises `AssertionError` if the condition is false. Use it for checks of your own assumptions in tests and quick scripts, not for validating user input (Python can run with asserts switched off).
- `e.add_note("text")` adds extra hints to an exception (Python 3.11 and later). `ExceptionGroup` and `except*` handle several errors that happened together (3.11 and later; you will meet them with `asyncio`).

Worked examples

Problem 6.3.1 Create `AppError` and `ValidationError`, raise one, and catch it through the base class.

Step 1	Define `class AppError(Exception)` as the base.
Step 2	Define `ValidationError(AppError)` that stores the `field`.
Step 3	Catch `AppError` (the parent) and read the child's `field`.

Predict: The `except` names `AppError`, but we raise `ValidationError`. Will it be caught?

```
class AppError(Exception):
    """Base error for our app."""

class ValidationError(AppError):
    def __init__(self, field, message):
        super().__init__(f"{field}: {message}")
        self.field = field

def validate(rec):
    if rec.get("age", 0) < 0:
        raise ValidationError("age", "must be >= 0")
    return rec

try:
    validate({"age": -3})
except AppError as e:
    print(type(e).__name__, e, e.field)
```

```
ValidationError age: must be >= 0 age
```

- **Explain** `ValidationError` is a kind of `AppError`, so `except AppError` catches it. `super().__init__(...)` sets the message that `print(e)` shows (the chapter on classes explains `super()` fully). We added our own attribute `field` so callers can know *which* field failed.
- **Answer** It was caught, and we could read both the message and `e.field`.
- **Check** The printed class name is `ValidationError` (the child), proving that a parent `except` catches children.
- **Why it matters** In plain words: your API can turn any `AppError` into a clean "400 Bad Request" and show `field` to the user.

Problem 6.3.2 Translate a low-level error into a clear one and keep the original cause.

Step 1	Parse a bad JSON string. Python raises `JSONDecodeError`.
Step 2	Catch it, and raise `AppError("config is not valid JSON") from e`.
Step 3	Print the cause, then print the full traceback to see the chain.

```
import json, traceback

def load(text):
    try:
        return json.loads(text)
    except json.JSONDecodeError as e:
        raise AppError("config is not valid JSON") from e

try:
    load("{bad")
except AppError as e:
    print(e, "| cause:", type(e.__cause__).__name__)
    traceback.print_exc()
```

```
config is not valid JSON | cause: JSONDecodeError
Traceback (most recent call last):
  File "/private/tmp/w6/c.py", line 19, in load
    return json.loads(text)
           ~~~~~~~~~~^^^^^^
  File ".../json/__init__.py", line 352, in loads
    return _default_decoder.decode(s)
    ...
json.decoder.JSONDecodeError: Expecting property name enclosed in double quotes: line 1 column 2 (char 1)

The above exception was the direct cause of the following exception:

Traceback (most recent call last):
  File "/private/tmp/w6/c.py", line 23, in <module>
    load("{bad")
  File "/private/tmp/w6/c.py", line 21, in load
    raise AppError("config is not valid JSON") from e
AppError: config is not valid JSON
```

The `...` lines are shortened by hand to save space; everything else is the real output.

- **Answer** The new error has a business meaning ("config is bad") and the old technical error is kept as its cause.
- **Check** `e.__cause__` is a `JSONDecodeError`, matching the first line of output.
- **Why it matters** You get friendly messages for users *and* full detail for the engineer reading the log.

Problem 6.3.3 Re-raise after logging, add a note, use `assert`, and handle an `ExceptionGroup`.

Step 1	In `except`, do your action (print "log"), then a bare `raise`.
Step 2	`assert 1 + 1 == 3, "math broke"` raises `AssertionError` with your text.
Step 3	`raise ExceptionGroup("batch", [ValueError("a"), TypeError("b")])` and catch with `except*`.

```
def risky():
    try:
        1/0
    except ZeroDivisionError:
        print("log and re-raise")
        raise

try:
    risky()
except ZeroDivisionError as e:
    print("caught again:", e)

try:
    assert 1 + 1 == 3, "math broke"
except AssertionError as e:
    print("AssertionError:", e)

try:
    raise ExceptionGroup("batch", [ValueError("a"), TypeError("b")])
except* ValueError as eg:
    print("VE", eg.exceptions)
except* TypeError as eg:
    print("TE", eg.exceptions)

try:
    try:
        int("x")
    except ValueError as e:
        e.add_note("while reading row 7")
        raise
except ValueError as e:
    print(e.__notes__)
```

```
log and re-raise
caught again: division by zero
AssertionError: math broke
VE (ValueError('a'),)
TE (TypeError('b'),)
['while reading row 7']
```

- **Answer** Each tool did its job. The caller still saw the `ZeroDivisionError` after our logging.
- **Check** `1 + 1 == 3` is false, so the assert fired, as it should.
- **Why it matters** The note "while reading row 7" tells you *where in the data* the failure happened, which `int("x")` alone never says.

The picture

```
ValueError (low-level)  ──from e──►  AppError (meaningful)
       ▲ __cause__ keeps the link        │
                                         ▼
                          caught by the caller that understands AppError

Exception
  └─ AppError
       ├─ ValidationError
       └─ DatabaseError
```

Connection to the real job

Libraries like `requests`, `pydantic` and `sqlalchemy` each have their own exception families. Your own services should too, so that a failure in a data-quality check is different from a database outage and gets a different reaction (skip, retry, page someone).

Applications

- **Data Engineering:** `DataQualityError` fails the pipeline with a precise message; `TransientError` triggers a retry.
- **Machine Learning:** raise early when the input features have the wrong columns, before a silent wrong prediction.
- **API / Backend:** map custom exceptions to HTTP status codes.
- **Testing:** `assert` states what you expect after the code runs.

COMMON MISTAKE Raising a bare `Exception("something failed")`. Callers cannot catch it precisely. Use a specific type.

COMMON MISTAKE `raise NewError(...)` inside `except` without `from e`. It still works, but the message says "during handling... another exception occurred", which suggests a bug in your handler, not a translation.

COMMON MISTAKE Using `assert` to check user input. Python started with `-O` removes all asserts.

Check yourself

1. What does a bare `raise` inside `except` do? Answer: re-raises the same exception.
2. What does `raise X from e` add? Answer: it records `e` as the direct cause of `X`.
3. Why have one base exception class for your project? Answer: callers can catch all your errors with one `except`.

6.4 Logging

INTUITION `print` is shouting in a room. `logging` is a flight recorder: every message has a time, a severity, a source, and can be sent to the screen, a file or a monitoring system, and switched on or off without editing code. When a nightly job fails at 3 a.m., the log is the only witness.

Words you need

Word	Plain meaning
logger	The object you call (`log.info(...)`); it has a name like `etl.load`
level	Importance: DEBUG 10, INFO 20, WARNING 30, ERROR 40, CRITICAL 50

Word	Plain meaning
handler	Decides *where* a message goes (screen, file)
formatter	Decides *how* a message looks
propagate	A child logger passes its messages up to its parent's handlers

Level	Use for	Example
DEBUG	Details for developers	`row 5 parsed as ...`
INFO	Normal milestones	`loaded 120 rows`
WARNING	Strange but continuing	`missing column city`
ERROR	An operation failed	`cannot connect`
CRITICAL	The program may stop	`disk full`

How it works

- `logging.basicConfig(level=..., format=...)` sets up logging quickly. Call it once, at the start of the program.
- `logging.getLogger("etl.load")` returns a named logger. Convention in a module: `log = logging.getLogger(__name__)`. Dots in names make a tree: `etl.load` is a child of `etl`.
- Messages below the chosen level are ignored. At level INFO, `debug` calls do nothing.
- `log.exception("message")` logs at ERROR level and adds the traceback. Use it inside `except`.
- Use **lazy formatting**: `log.info("loaded %d rows", n)`, not `log.info(f"loaded {n} rows")`. With the first form, the text is built only if the message is actually emitted.
- Never log secrets, passwords, or personal data.

Worked examples

Problem 6.4.1 Set up basic logging at level INFO and write one message of each level, plus an exception.

Step 1	`basicConfig` with a format that shows level, logger name and message.
Step 2	Call `debug`, `info`, `warning`, `error`.
Step 3	In an `except` block call `log.exception`.

Predict: Which of these will NOT appear: debug, info, warning, error?

```
import logging, sys
logging.basicConfig(stream=sys.stdout, level=logging.INFO,
    format="%(levelname)s | %(name)s | %(message)s")
log = logging.getLogger("etl.load")
log.debug("hidden at INFO level")
log.info("loaded %d rows from %s", 120, "orders.csv")
log.warning("missing column %r", "city")
log.error("cannot connect")
try:
    1/0
except ZeroDivisionError:
    log.exception("calc failed")
print(logging.getLevelName(30), logging.WARNING, logging.ERROR, logging.CRITICAL)
```

```
INFO | etl.load | loaded 120 rows from orders.csv
WARNING | etl.load | missing column 'city'
ERROR | etl.load | cannot connect
ERROR | etl.load | calc failed
Traceback (most recent call last):
  File "/private/tmp/w6/lg.py", line 10, in <module>
    1/0
    ~^~
ZeroDivisionError: division by zero
WARNING 30 40 50
```

- **Explain** The debug line is missing because DEBUG (10) is below INFO (20). `%d` and `%r` are placeholders filled from the extra arguments. `log.exception` printed an ERROR line and the full traceback. By default `basicConfig` writes to stderr; we used `stream=sys.stdout` only so the output appears in one place.
- **Answer** Only the debug message was hidden.
- **Check** The numbers: 30, 40, 50 are the real values of WARNING, ERROR, CRITICAL.
- **Why it matters** Change `level=logging.DEBUG` and all details appear, with no other code change. That is how you investigate a problem in production.

The typical wrong version. Using `print` everywhere. It has no level, no time, no way to send to a file, and you must delete the lines later. Also avoid `logging.info(f"...")` with f-strings for expensive objects: the text is built even when the message is dropped.

Problem 6.4.2 Send everything (DEBUG) to a file and only warnings to the screen, using two handlers, and see how a child logger propagates.

Step 1	Make a logger `svc` at level DEBUG.
Step 2	Add a `FileHandler` (level DEBUG) and a `StreamHandler` (level WARNING).
Step 3	Log at debug, info, warning. Print the file. Then log an error from the child `svc.db` .

```
import logging, sys, tempfile, os
d = tempfile.mkdtemp(); fn = os.path.join(d, "app.log")
log = logging.getLogger("svc")
log.setLevel(logging.DEBUG)
fh = logging.FileHandler(fn, encoding="utf-8"); fh.setLevel(logging.DEBUG)
ch = logging.StreamHandler(sys.stdout); ch.setLevel(logging.WARNING)
fmt = logging.Formatter("%(levelname)s:%(name)s:%(message)s")
fh.setFormatter(fmt); ch.setFormatter(fmt)
log.addHandler(fh); log.addHandler(ch)
log.debug("detail for file only")
log.info("info for file only")
log.warning("warning to both")
print("--- file content ---")
fh.flush(); print(open(fn).read(), end="")
child = logging.getLogger("svc.db")
child.error("child logger propagates to parent handlers")
```

```
WARNING:svc:warning to both
--- file content ---
DEBUG:svc:detail for file only
INFO:svc:info for file only
WARNING:svc:warning to both
ERROR:svc.db:child logger propagates to parent handlers
```

- **Explain** The first line is the screen copy of the warning (the screen only shows WARNING and above). The three lines after the divider are the file copy (all levels). The final ERROR line is printed to the *screen* by the child logger `svc.db` : it has no handlers of its own, so its message travels up to `svc` and uses its handlers (it was also written to the file after we printed it).
- **Answer** One logger tree, two destinations, each with its own threshold.

- **Check** The file shows 3 lines at the time we read it (debug, info, warning), matching the 3 calls.
- **Why it matters** In plain words: operators see only warnings and errors on the console, while engineers can search the file for the full story.

Problem 6.4.3 Produce structured (JSON) log lines so a tool can parse them.

Step 1	Subclass `logging.Formatter` and override `format`.
Step 2	Return `json.dumps` of level, message and logger name.
Step 3	Attach it to a handler. Set `propagate = False` to avoid double printing.

```
import logging, sys, json
logging.basicConfig(stream=sys.stdout, level=logging.INFO, format="%(message)s")
class JsonFormatter(logging.Formatter):
    def format(self, r):
        return json.dumps({"level": r.levelname, "msg": r.getMessage(), "logger": r.name})
h = logging.StreamHandler(sys.stdout); h.setFormatter(JsonFormatter())
l2 = logging.getLogger("json"); l2.addHandler(h); l2.propagate = False; l2.setLevel(logging.INFO)
l2.info("order %s paid", 17)
```

```
{"level": "INFO", "msg": "order 17 paid", "logger": "json"}
```

- **Answer** The message became one JSON object per line, which log tools (ELK, CloudWatch, Datadog) can index.
- **Check** `"order %s paid" % 17` is `order 17 paid`. Matches.
- **Why it matters** Searching for `level = ERROR` in JSON logs is easy; searching free text is slow and fragile.

The picture

Connection to the real job

Every production system writes logs: Airflow tasks, FastAPI services, Spark jobs, training runs. During an incident, you search logs by time and level. Logging itself has a cost, like a restaurant kitchen that stops to write a note for every single egg it cracks: a DEBUG line inside a loop of 10 million rows slows the job. Set INFO in production and DEBUG when investigating.

Applications

- **Data Engineering:** row counts and durations of each pipeline step, with warnings for rejected rows.
- **MLOps:** log model version, input size and prediction time of each request.
- **Machine Learning:** log loss and metrics per epoch to a file.
- **Security / Compliance:** audit trail of who ran what (without logging secrets).

COMMON MISTAKE	Calling `basicConfig` after other code already configured logging. It then does nothing. Call it once, first.

COMMON MISTAKE	Logging passwords, tokens or customer personal data. Logs are copied around and kept for years.

COMMON MISTAKE	Using `log.error(str(e))` inside `except` and losing the traceback. Use `log.exception(...)`.

Check yourself

1. At level WARNING, is `log.info(...)` shown? Answer: no.
2. Which call logs a message together with the traceback? Answer: `log.exception(...)`.
3. Why write `log.info("n=%d", n)` rather than an f-string? Answer: the text is built only if the message is emitted (lazy formatting).

6.5 Debugging with `pdb`

INTUITION	`print` debugging is like looking at a crime scene through a keyhole. A debugger lets you walk in: freeze the program at a line, look at every variable, move forward one line at a time, and ask "what is the value now?". It is the difference between guessing and seeing.

How it works

`pdb` is Python's built-in debugger. Three ways to start it:

1. Put `breakpoint()` in your code. Execution stops there.
2. Run `python3 -m pdb script.py` to start the script under the debugger.
3. **Post-mortem:** `python3 -m pdb -c continue script.py`. If the script crashes, pdb opens *at the crash line* and you can inspect the variables.

Set `PYTHONBREAKPOINT=0` to switch every `breakpoint()` off without editing code.

Command	Short	Meaning
`help`	`h`	list commands
`list` / `ll`	`l`	show source around the current line / the whole function
`next`	`n`	run the current line, stay at this level
`step`	`s`	run the current line, go *into* a function call
`continue`	`c`	run until the next breakpoint or the end
`print x`	`p x`	show the value of an expression
`break 12`	`b 12`	set a breakpoint at line 12
`where`	`w`	show the call stack
`up` / `down`	`u` / `d`	move between stack frames
`quit`	`q`	leave the debugger

The VS Code debugger does the same with clicks: click in the gutter to set a breakpoint, press F5, look at the Variables panel, and use the step buttons (not run here, because it needs the VS Code window; the ideas are identical to pdb).

Worked examples

For these examples we feed the commands to pdb through a pipe, so you can see a real session. When you work by hand, you type the same commands at the `(Pdb)` prompt.

Problem 6.5.1 Stop inside `average` just before the division, inspect the variables, and move one line.

Step 1	The function has a `breakpoint()` before `return total / len(nums)`.
Step 2	At the prompt we type `p nums`, `p total`, `n` (next), `ll` (list function), `c` (continue).
Step 3	The second call, `average([])`, stops at the breakpoint again. We look at `nums`.

```
def average(nums):
    total = 0
    for n in nums:
        total += n
    breakpoint()
    return total / len(nums)

print(average([10, 20, 30]))
print(average([]))
```

Predict: What will `p total` show for `[10, 20, 30]`?

```
> /private/tmp/w6/bug.py(5)average()
-> breakpoint()
(Pdb) [10, 20, 30]
(Pdb) 60
(Pdb) > /private/tmp/w6/bug.py(6)average()
-> return total / len(nums)
(Pdb)   1       def average(nums):
  2         total = 0
  3         for n in nums:
  4             total += n
  5         breakpoint()
  6  ->     return total / len(nums)
(Pdb) 20.0
> /private/tmp/w6/bug.py(5)average()
-> breakpoint()
(Pdb) []
(Pdb) Quitting pdb will kill the process. Quit anyway? [y/n]
```

- **Explain** The line `> ...bug.py(5)average()` says: file, line number, function. The `->` line is the *next line to run*. `p nums` printed the list, `p total` printed 60. `n` moved to line 6. `ll` printed the function with an arrow at the current line. `c` ran on: the program printed `20.0` (60 / 3), then reached the breakpoint for the empty list. `p nums` shows `[]`: the cause of the next crash is visible before it happens. (The last line appears because our pipe ended; typing `y` quits.)
- **Answer** The average of `[10, 20, 30]` is 20.0, and for `[]` we can see that `len(nums)` would be 0.
- **Check** 10 + 20 + 30 = 60 and 60 / 3 = 20.0. The debugger's values match.
- **Why it matters** In plain words: you *saw* the empty list before the program crashed, instead of guessing from the error message.

The typical wrong version. Without looking, the second call fails:

```
ZeroDivisionError: division by zero
```

and with `PYTHONBREAKPOINT=0` the `breakpoint()` line does nothing, so the script runs straight to that same `ZeroDivisionError` (we ran it: the last lines were `return total / len(nums)` and `ZeroDivisionError: division by zero`). The fix is a guard such as `if not nums: return 0.0` (or raise a clear error). Remove `breakpoint()` before you commit anyway.

Problem 6.5.2 Find a logic bug with no crash. `buggy_max([-5, -2, -9])` should return -2, but returns 0.

Step 1	Start `python3 -m pdb -c "b 4" -c continue bug2.py`. This sets a breakpoint on line 4 (the `if`) and runs.
Step 2	At each stop, print `n` and `best`.
Step 3	Notice `best` starts at 0, bigger than every number in the list.

```
def buggy_max(nums):
    best = 0
    for n in nums:
        if n > best:
            best = n
    return best
print(buggy_max([-5, -2, -9]))
```

Predict: What is `best` at the first stop?

```
Breakpoint 1 at /private/tmp/w6/bug2.py:4
> /private/tmp/w6/bug2.py(4)buggy_max()
-> if n > best:
(Pdb) (-5, 0)
(Pdb) > /private/tmp/w6/bug2.py(3)buggy_max()
-> for n in nums:
(Pdb) 0
(Pdb) > /private/tmp/w6/bug2.py(4)buggy_max()
-> if n > best:
(Pdb) (-2, 0)
```

- **Explain** The command `p n, best` printed `(-5, 0)`: n is -5, best is 0. `-5 > 0` is false, so `best` never changes. We continued to the next stop: `(-2, 0)`, again false. The initial value `0` is the bug.
- **Answer** The function returns 0 because `best` starts at 0. Fix: `best = nums[0]` (and handle the empty list), or use the built-in `max(nums)`.
- **Check** With the fix, `max([-5, -2, -9])` is -2. Run `print(max([-5, -2, -9]))` to confirm.
- **Why it matters** There was no error message at all. Only looking at the variables revealed the problem. Many data bugs are silent like this.

Problem 6.5.3 Post-mortem: let a script crash and inspect it afterwards.

Step 1	Script: `data = {"a": [1, 2]}` then `print(data["a"][3])`.
Step 2	Run with `python3 -m pdb -c continue crash.py`. Pdb catches the crash and stops at the failing line.
Step 3	Type `p data` and `w` to see the stack.

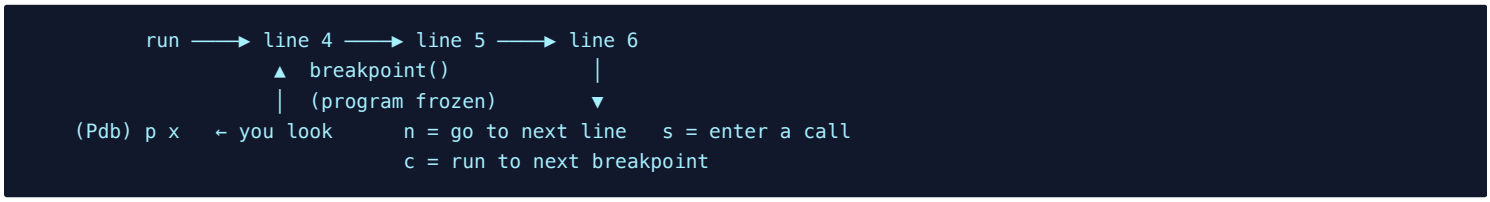
```
IndexError: list index out of range
Uncaught exception. Entering post mortem debugging
Running 'cont' or 'step' will restart the program
> /private/tmp/w6/crash.py(2)<module>()
-> print(data["a"][3])
(Pdb) {'a': [1, 2]}
(Pdb)   /Library/.../pdb.py(3652)main()
-> pdb._run(target)
  ...
> /private/tmp/w6/crash.py(2)<module>()
-> print(data["a"][3])
```

(The lines inside the pdb library and the `...` were shortened by hand.)

- **Answer** `p data` shows the list has only 2 items, so index 3 is out of range. The bottom frame, marked with `>`, is your own line.
- **Check** A list with 2 items has valid indexes 0 and 1. Index 3 fails.

- **Why it matters** In plain words: you see the exact state at the moment of failure, not a reconstruction.

The picture



Connection to the real job

Notebook debugging uses the same ideas (`%debug` in Jupyter opens pdb after an error). For bugs that only appear on a server, you add logging (6.4) since you cannot attach a debugger. In a data pipeline, write the failing row to a file and reproduce it locally with `pdb` .

Applications

- **Data Analytics:** step through a cleaning function on one failing row.
- **Machine Learning:** stop inside a loss function to see tensor shapes.
- **Data Engineering:** reproduce a bad-record failure from yesterday's job with the saved record.
- **Backend:** debug a failing request by running the handler with a breakpoint.

COMMON MISTAKE	Committing code with `breakpoint()` left in. A server would freeze waiting for input. Search for it before you commit (a pre-commit hook can check this, Chapter 10).

COMMON MISTAKE	Typing `n` when you wanted `s` . `n` runs a whole function call as one step; `s` goes inside it.

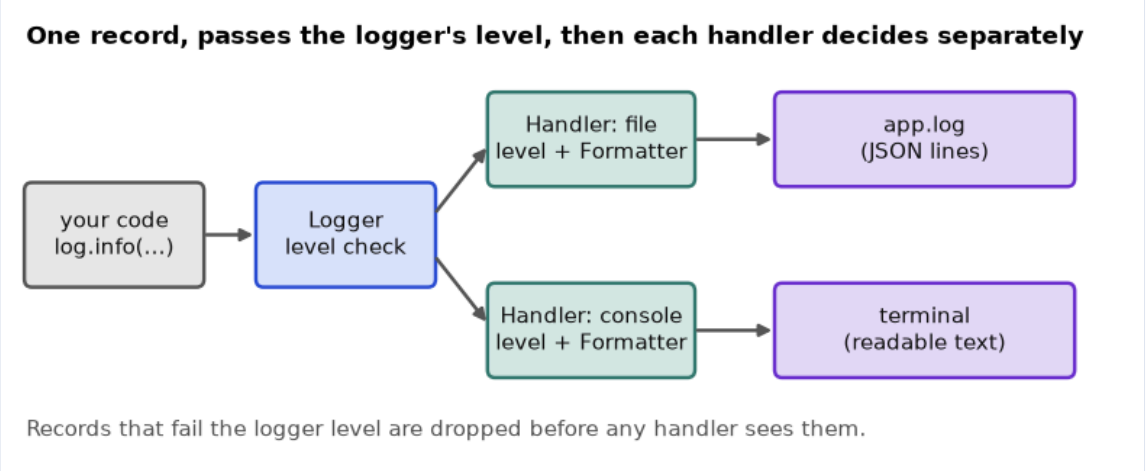
COMMON MISTAKE	Using variable names that match pdb commands (like `c` or `n`) and then printing them. Use `p n` explicitly with `p` .

Check yourself

1. How do you stop the program at a chosen line in code? Answer: write `breakpoint()` there.
2. What is the difference between `n` and `s` ? Answer: `n` runs the line as one step; `s` steps into a function call.
3. Which command shows the call stack? Answer: `w` (where).

6.6 Production Logging: Structured Logs, dictConfig, Warnings and Crash Tools

Section 6.4 taught the basics of `logging` . In production, nobody reads logs line by line. Programs read them: a log system searches, counts and filters. That needs logs with **structure** (fields, not just sentences) and a **correlation ID** so you can follow one request through many log lines.



How one log call travels: the logger checks its level, then each handler checks its own level and formats the record.

How to read it: the logger is the first gate. A record that passes goes to every handler attached to it. Each handler has its own level and formatter, so the same event can appear as readable text on the terminal and as JSON in a file.

Structured logging with contextvars and dictConfig

Two tools make this work. A `contextvars.ContextVar` holds a value (here the request ID) that belongs to the current request or task, even when many run at once. `logging.config.dictConfig` configures loggers, handlers and formatters from one dictionary, which you can later load from a JSON or YAML file instead of code. The key `"()"` in a formatter entry names a class to build.

Problem 6.6.1 Make every log line a JSON object that includes the request ID and any extra fields, send it to the console, and show that DEBUG lines are filtered by the handler.

Step 1	`request_id = ContextVar("request_id", default="-")`. Each request calls `request_id.set(...)` once.
Step 2	A `JsonFormatter` builds a dict from the record (`levelname`, logger name, message) plus the request ID and any `extra` fields, and returns `json.dumps(..., sort_keys=True)`.
Step 3	`dictConfig` wires a console handler at level INFO to the `shop` logger at level DEBUG. The logger accepts DEBUG, the handler drops it.
Step 4	Calling `handle("req-17", 40)` then `handle("req-18", 5)` gives two lines each, tagged with their own ID.

```
import contextvars, json, logging, logging.config
request_id = contextvars.ContextVar("request_id", default="-")
class JsonFormatter(logging.Formatter):
    def format(self, record):
        payload = {"level": record.levelname, "logger": record.name,
                   "msg": record.getMessage(), "request_id": request_id.get()}
        payload.update(getattr(record, "fields", {}))
        return json.dumps(payload, sort_keys=True)
config = {
    "version": 1,
    "formatters": {"json": {"()": JsonFormatter}},
    "handlers": {"console": {"class": "logging.StreamHandler", "stream": "ext://sys.stdout",
                             "formatter": "json", "level": "INFO"}},
    "loggers": {"shop": {"handlers": ["console"], "level": "DEBUG", "propagate": False}},
}
logging.config.dictConfig(config)
log = logging.getLogger("shop.orders")
def handle(rid, amount):
    request_id.set(rid)
    log.debug("not shown, handler level is INFO")
    log.info("order received", extra={"fields": {"amount": amount}})
    log.error("charge failed", extra={"fields": {"amount": amount}})
handle("req-17", 40)
handle("req-18", 5)
```

```
{"amount": 40, "level": "INFO", "logger": "shop.orders", "msg": "order received", "request_id": "req-17"}
{"amount": 40, "level": "ERROR", "logger": "shop.orders", "msg": "charge failed", "request_id": "req-17"}
{"amount": 5, "level": "INFO", "logger": "shop.orders", "msg": "order received", "request_id": "req-18"}
{"amount": 5, "level": "ERROR", "logger": "shop.orders", "msg": "charge failed", "request_id": "req-18"}
```

Now "show me everything that happened to req-17" is a one-line search, and "how many ERROR lines had amount over 30" is a query, not a reading task. In a web service you would set the request ID in middleware (Chapter 12) from an incoming header such as `X-Request-ID` so the same ID appears in every service that handles the request.

COMMON MISTAKE	Building the message with an f-string and putting data inside it: `log.info(f"order {order} by {user}")`. You cannot search by field, the f-string runs even when the level is filtered out, and a user name with a newline can forge a fake log line. Put data in `extra` or in the lazy form `log.info("order %s", order_id)`, and never log passwords, tokens or full card numbers.

Warnings, deprecations and crash tools

The `warnings` module is for "this still works but you should change it". Library authors call `warnings.warn(msg, DeprecationWarning, stacklevel=2)` (stacklevel 2 blames the caller's line, not the library's). Python hides `DeprecationWarning` by default except in code run directly, so in tests turn warnings into errors: `python -W error` or `warnings.simplefilter("error")`. That is how upgrades stop surprising you.

Problem 6.6.2 Capture a deprecation warning, then promote it to an exception, then find the innermost function of a traceback in code.

Step 1	With filter "always" and `record=True`, the warning is stored, not printed. Read its category and message.
Step 2	With filter "error", the same call raises `DeprecationWarning` as an exception you can catch (or let fail a test).
Step 3	A traceback is a linked list: follow `tb_next` to the last entry, which is where the exception was raised. Its frame's code name is the function.

```
import sys, warnings, faulthandler
def old_api():
    warnings.warn("old_api is deprecated, use new_api", DeprecationWarning, stacklevel=2)
    return 1
with warnings.catch_warnings(record=True) as caught:
    warnings.simplefilter("always")
    old_api()
print(caught[0].category.__name__, str(caught[0].message))
with warnings.catch_warnings():
    warnings.simplefilter("error")
    try:
        old_api()
    except DeprecationWarning as e:
        print("promoted to error:", e)
print(faulthandler.is_enabled())
faulthandler.enable()
print(faulthandler.is_enabled())
def crash():
    return {}["missing"]
try:
    crash()
except KeyError:
    tb = sys.exc_info()[2]
    while tb.tb_next:
        tb = tb.tb_next
    print(tb.tb_frame.f_code.co_name)
```

```
DeprecationWarning old_api is deprecated, use new_api
promoted to error: old_api is deprecated, use new_api
False
True
crash
```

`faulthandler.enable()` (or running `python -X faulthandler`) makes Python print the stack of every thread if the process crashes hard, for example from a segmentation fault in a C extension, a case where no `except` can help. Also useful: `faulthandler.dump_traceback_later(30)` prints the stacks if your program is still running after 30 seconds, which is the quickest way to see where a hung program is stuck.

Going deeper: post-mortem debugging

If a program crashes, you can open the debugger *on the crash*. Run `python -m pdb -c continue script.py`: when an unhandled exception ends the run, pdb drops you into the frame where it happened. In an interactive session, call `import pdb; pdb.pm()` after an exception to inspect it. Useful commands once there are `w` (show the stack), `u` and `d` (move up and down frames), `p expr` (print a value) and `q` (quit). This is interactive, so it is not run here.

REAL JOB	When a user reports "it failed at 14:03", you want three things: the request ID (from them or from the response header), every log line carrying that ID, and the stack trace at the ERROR line. Structured logs plus a correlation ID give you the first two in seconds. Chapter 13 (13.9) adds metrics and tracing on top.

Chapter recap

- Read a traceback from the last line upward: type and message first, then the first frame that is your own code.
- Learn the ten common exceptions: `ValueError`, `TypeError`, `KeyError`, `IndexError`, `AttributeError`, `NameError`, `FileNotFoundError`, `ModuleNotFoundError`, `ZeroDivisionError`, `AssertionError`.
- Catch specific exceptions, keep the `try` block small, use `else` for success code and `finally` for cleanup.
- Never use a bare `except:`; it hides bugs.

- EAFP (try and catch) is the Python style; LBYL (check first) is fine for simple cases.
- Raise your own exceptions with a clear message; make a base `AppError` class and children.
- Use `raise X from e` to keep the original cause, and bare `raise` to re-raise after logging.
- Use `logging`, not `print`: levels, handlers, `log.exception`, lazy `%s` formatting, and never log secrets.
- Use `breakpoint()`, `python3 -m pdb` and post-mortem to look inside a running program.

Cheat-sheet

Task	Pattern
Handle a known error	`try: ... except ValueError as e: ...`
Cleanup always	`try: ... finally: close()`
Raise	`raise ValidationError("age", "must be >= 0")`
Keep cause	`raise AppError("...") from e`
Log with traceback	`log.exception("what failed")`
Basic log setup	`logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")`
Stop in debugger	`breakpoint()` or `python3 -m pdb script.py`
pdb moves	`n` next, `s` step, `c` continue, `p x` print, `w` stack, `q` quit

Practice set

Problem 6.P.1 What exception does `int("3.5")` raise?

Answer: `ValueError` (`int` does not accept text with a decimal point; use `int(float("3.5"))`).

Problem 6.P.2 What is the output?

```
try:
    print("A")
except ValueError:
    print("B")
else:
    print("C")
finally:
    print("D")
```

Answer: `A`, `C`, `D` on separate lines. No error, so `else` runs, then `finally`.

Problem 6.P.3 What is the output?

```
def f():
    try:
        return "try"
    finally:
        print("cleanup")
print(f())
```

Answer: `cleanup` then `try`. The `finally` block runs before the function returns.

Problem 6.P.4 Find the bug:

```
try:
    value = int(text)
except:
    pass
print(value * 2)
```

Answer: The bare `except` hides all errors, and if `int(text)` failed, `value` is not defined, so `print` raises `NameError`. Catch `ValueError` and set a default or stop.

Problem 6.P.5 Which exception does `d = {}; d["x"] += 1` raise?

Answer: `KeyError: 'x'`, because `d["x"]` is read first. Use `d.get("x", 0) + 1` or `defaultdict(int)`.

Problem 6.P.6 Write a function `to_float(s)` that returns `None` for text that is not a number, but lets other errors through.

Answer:

```
def to_float(s):
    try:
        return float(s)
    except ValueError:
        return None
```

Note that `to_float(None)` raises `TypeError`, which is correct: `None` is a programming error, not bad data. Decide this on purpose.

Problem 6.P.7 Logging level is WARNING. Which of these appear: `debug`, `info`, `warning`, `error`?

Answer: `warning` and `error`.

Problem 6.P.8 Find the bug: a function does `except ValueError as e: raise AppError("bad input")` and an engineer says the original error is lost.

Answer: Without `from e` the link is only implicit. Write `raise AppError("bad input") from e` so the traceback shows the direct cause.

Problem 6.P.9 Scenario: your nightly job crashes at 2 a.m. with `KeyError: 'amount'` after 1.2 million rows. You have only the log. List your steps.

Answer: (1) Read the traceback bottom-up and find the line in your code. (2) Check the log for which file or row it was processing (this is why you log file names and row counts). (3) Reproduce with that one row on your laptop. (4) Use `pdb` or a print of the row to see which keys exist. (5) Decide: skip and count bad rows with a warning, or fail the job with a clear `DataQualityError`. (6) Add a test with that bad row.

Problem 6.P.10 Interview: what is the difference between `except Exception` and a bare `except:`? Why avoid both when possible?

Answer: A bare `except:` also catches `KeyboardInterrupt` and `SystemExit` (they inherit from `BaseException`), so you cannot stop the program with Ctrl+C. `except Exception` skips those but still hides bugs. Catch the specific exception you can handle.

Problem 6.P.11 Interview: explain EAFP versus LBYL with an example, and say when LBYL is better.

Answer: EAFP: `try: value = d[key] except KeyError: ...`. LBYL: `if key in d: value = d[key]`. EAFP is idiomatic and avoids a race (a file may disappear between `exists()` and `open()`). LBYL is clearer when failure is common and cheap to check, and when an exception would be costly in a hot loop.

Problem 6.P.12 Interview: you must find why a function returns the wrong number, but there is no error. How do you debug it?

Answer: Reproduce with the smallest input. Put `breakpoint()` before the suspicious line, print the inputs and intermediate values (`p`), step with `n`, and compare with what you expect by hand. Then write a failing test that captures the bug, fix, and keep the test.

Prompt pack

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data roles in India, and English is my second language, so use simple words. Teach me how to read a Python traceback and the ten most common exceptions (ValueError, TypeError, KeyError, IndexError, AttributeError, NameError, FileNotFoundError and others). Explain step by step with small examples, then show me 6 tracebacks one at a time and ask me what the cause and the fix are. Wait for my answer, and correct me kindly.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data roles in India. Topic: Python try/except/else/finally, EAFP versus LBYL, catching specific exceptions, and why bare except is dangerous. Give me 8 graded exercises from easy to hard about cleaning messy data rows safely. Do not give solutions yet. When I paste my answers, review them for bugs and style and explain every problem in simple English.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data roles in India, and English is my second language. Topic: custom exceptions in Python, raising errors, exception hierarchies, raise from, re-raising, assert, and ExceptionGroup. Explain with a data pipeline example, then review my design for the custom errors of a small ETL job: [PASTE HERE]. Tell me what is good, what is confusing, and how a senior engineer would improve it.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data roles in India. Topic: the Python logging module: levels, loggers, handlers, formatters, propagation, log.exception, lazy formatting and JSON logs. Quiz me one question at a time, starting easy and getting harder. After each answer, explain the right answer step by step. After 8 questions, give me a small project to set up logging for a script with a file handler and a console handler.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data roles in India, and English is my second language. Topic: debugging Python with pdb and breakpoint(): commands n, s, c, p, ll, w, b, post-mortem debugging. Run a mock debugging interview: describe a buggy function in words, let me ask for variable values, and guide me to find the bug. Do three rounds of increasing difficulty and give me honest feedback on my method at the end.
```

CHAPTER 7

ITERATORS, GENERATORS, DECORATORS AND CONTEXT MANAGERS

Four features that turn beginner Python into professional Python: process data lazily, wrap behaviour around functions, and guarantee cleanup.

Suppose you must read a 20 GB log file, keep only the delivered orders, and compute their average delivery time. If you load the file into a list, your laptop runs out of memory. With iterators and generators you read one line at a time and the job runs in a few megabytes. Now suppose five of your functions need the same extras: timing, retry on network failure, and a log line. Copy-pasting those into every function is a maintenance disaster. A decorator adds them in one line each. Finally, every file, database connection and lock must be released even when errors happen, and context managers guarantee it. By the end of this chapter you can build lazy data pipelines, write your own timing and retry decorators, and write your own `with` blocks.

Running example: a tiny order log with lines like `OK north A1 120` (status, region, order id, delivery time in minutes). We read it lazily, time the functions, retry the flaky ones, and manage resources safely. Every program here was run with `python3` (Python 3.14) and the output is real.

7.1 Iterators

INTUITION

Think of a vending machine that gives one item each time you press the button, and says "empty" when it is done. You never see the whole stock at once. An iterator is that machine: you ask for the next item, it gives one, and at the end it says "no more". A `for` loop is just a person pressing the button until the machine says empty.

Words you need

Word	Plain meaning
iterable	Anything you can loop over: list, string, dict, file, `range`
iterator	An object that hands out items one at a time with `next()`
`iter(x)`	Asks an iterable for a fresh iterator
`next(it)`	Asks the iterator for the next item
`StopIteration`	The exception that means "no more items"
exhausted	An iterator that has given all its items; it cannot restart

How it works

A list is an iterable, but it is not an iterator. The rules (called the *iterator protocol*) are simple:

- An **iterable** has an `__iter__` method that returns an iterator.
- An **iterator** has `__next__`, which returns the next item or raises `StopIteration`, and `__iter__` that returns itself.
- A `for x in thing:` loop calls `iter(thing)`, then calls `next` again and again, and stops quietly when `StopIteration` is raised.

Why does this exist? So that one `for` loop works on lists, files, database cursors, and network streams alike, and so that huge or endless sources do not need to be stored in memory.

Worked examples

Problem 7.1.1 Use `iter` and `next` by hand on `[10, 20, 30]`, and see what happens at the end.

Step 1	`it = iter(nums)` creates the iterator.
Step 2	Call `next(it)` three times.
Step 3	Call it a fourth time and catch `StopIteration`.
Step 4	Rebuild the `for` loop from `while True`.

Predict: What does the fourth `next(it)` do? What does `list(it)` give after the iterator is already used up?

```
nums = [10, 20, 30]
it = iter(nums)
print(type(it).__name__, next(it), next(it), next(it))
try:
    next(it)
except StopIteration:
    print("StopIteration: exhausted")
print(iter(it) is it, iter(nums) is nums)

it = iter(nums)
while True:
    try:
        x = next(it)
    except StopIteration:
        break
    print("got", x)

it = iter([1, 2, 3])
print(list(it), list(it))
print(next(iter([]), "default"))
```

```
list_iterator 10 20 30
StopIteration: exhausted
True False
got 10
got 20
got 30
[1, 2, 3] []
default
```

- **Explain** `iter(nums)` gave a `list_iterator`. `iter(it) is it` is `True`: an iterator is its own iterator. `iter(nums) is nums` is `False`: the list makes a new iterator each time, so you can loop over a list many times. The `while True` loop is exactly what `for` does inside. The second `list(it)` is empty because the first call used up all items. `next(iter([]), "default")` gives a fallback instead of raising.
- **Answer** The fourth `next` raises `StopIteration`. A used iterator cannot be reused.
- **Check** `[1, 2, 3]` then `[]` proves the iterator was spent after one pass.
- **Why it matters** In plain words: if you pass an iterator to two functions, the second one sees nothing. This is a very common silent bug.

The typical wrong version. Loop twice over a generator or iterator:

```
it = iter([1, 2, 3])
print(sum(it))   # 6
print(sum(it))   # 0, silently
```

No error appears, only a wrong result (0). The fix: keep the data in a list, or recreate the iterator.

Problem 7.1.2 Build your own iterator class `Countdown` and use it with `list`, `sum` and `max`.

Step 1	Store the start value in `__init__`.
Step 2	`__iter__` returns `self`.
Step 3	`__next__` raises `StopIteration` at 0, otherwise returns the next number down.

```
class Countdown:
    def __init__(self, start):
        self.n = start
    def __iter__(self):
        return self
    def __next__(self):
        if self.n <= 0:
            raise StopIteration
        self.n -= 1
        return self.n + 1

print(list(Countdown(4)))
print(sum(Countdown(4)), max(Countdown(3)))
d = {"a": 1, "b": 2}
print(list(iter(d)), list(d.items()))
print(list(enumerate("ab", start=1)), list(zip("ab", [1, 2, 3])))
```

```
[4, 3, 2, 1]
10 3
['a', 'b'] [('a', 1), ('b', 2)]
[(1, 'a'), (2, 'b')] [('a', 1), ('b', 2)]
```

- **Answer** `list(Countdown(4))` is `[4, 3, 2, 1]`, and `sum` is 4+3+2+1 = 10.
- **Check** `max(Countdown(3))` is 3 (values 3, 2, 1). `zip` stops at the shorter input, so `3` has no partner and is dropped.
- **Why it matters** Anything that follows the protocol works with all built-ins (`list`, `sum`, `max`, `sorted`, `for`, `in`). You rarely write classes like this (7.2 is easier), but you now know what happens under every `for` loop.

The picture

```
iterable ──iter()──▶ iterator ──next()──▶ 10
                        │       next()──▶ 20
                        │       next()──▶ 30
                        └─ next() ──▶ raises StopIteration  (for-loop ends quietly)
```

Item	Example value	Where it appears in the job
Iterable	a `list`, a `str`, a DataFrame's columns	passed to `for`, `sum`, `sorted`
Iterator	an open file, `zip(...)`, `map(...)`	reading big files, streaming rows
`enumerate` / `zip`	`enumerate(rows, 1)`	line numbers, pairing columns

Connection to the real job

An open file is an iterator over lines; a database cursor is an iterator over rows; `pandas.read_csv(..., chunksize=100_000)` returns an iterator of DataFrames. Understanding the protocol explains why these tools can handle data bigger than memory.

Applications

- **Data Engineering:** stream rows from a database cursor into a file in batches.
- **Data Analytics:** `for chunk in pd.read_csv(path, chunksize=...)` aggregates a file too big for memory.
- **Machine Learning:** a data loader is an iterator that yields batches.
- **GenAI:** streaming an LLM reply is an iterator of text pieces.

COMMON MISTAKE	Using an iterator twice. After the first pass it is empty. Convert to a list if you need two passes.

COMMON MISTAKE	Confusing iterable and iterator. A list is iterable, not an iterator: `next([1,2])` raises `TypeError`.

COMMON MISTAKE	Raising `StopIteration` inside a generator by accident. Since Python 3.7 this becomes a `RuntimeError`.

Check yourself

1. What does `for` call first on the object? Answer: `iter()`.
2. What exception ends iteration? Answer: `StopIteration`.
3. Can you loop over a list twice? Answer: yes, because each loop gets a new iterator; but an iterator itself can be used once.

7.2 Generators

INTUITION	A normal function is a cook who prepares the whole meal and brings every dish at once. A generator is a cook who brings one dish, waits until you ask, and then cooks the next. Nothing is cooked before you ask, and nothing is stored. This is called **lazy evaluation**: work is done only when the result is needed.

How it works

A **generator function** looks like a normal function but uses `yield` instead of (or in addition to) `return`. Calling it does *not* run the body. It returns a **generator object** (an iterator). Each `next()` runs the body until the next `yield`, hands out that value, and freezes the function with all its local variables. The next `next()` unfreezes it.

- A **generator expression** is a comprehension in round brackets: `(i*i for i in range(10))`. It makes a generator, not a list.
- `yield from other` hands out everything from another iterable (a shortcut for a loop with `yield`).
- `gen.send(value)` sends a value *into* a paused generator (advanced; used for coroutines).
- Why it exists: you can describe an endless or huge sequence in a few lines and use only constant memory.

Worked examples

Problem 7.2.1 Write `squares(n)` as a generator, and see that the body runs only on demand.

Step 1	Use `yield i * i` in a loop.
Step 2	Call it and look at the type.
Step 3	Make a `trace` generator with `print` calls to see *when* the code runs.

Predict: After `t = trace()`, does anything print?

```
def squares(n):
    for i in range(n):
        yield i * i

g = squares(4)
print(type(g).__name__, next(g), next(g), list(g))

def trace():
    print("start"); yield 1
    print("middle"); yield 2
    print("end")

t = trace()
print("created, nothing printed yet")
print(next(t)); print(next(t))
try:
    next(t)
except StopIteration:
    print("done")
```

```
generator 0 1 [4, 9]
created, nothing printed yet
start
1
middle
2
end
done
```

- **Explain** `g` is a `generator`. `next(g)` gave 0 and 1 (0^2 and 1^2). `list(g)` collected the rest: $2^2 = 4$ and $3^2 = 9$. For `trace`, creating it prints nothing. The first `next` runs until the first `yield` (printing `start`, giving 1). The third `next` runs to the end of the function (printing `end`) and Python raises `StopIteration`.
- **Answer** Nothing prints at creation. Code runs only when you ask for the next value.
- **Check** `0, 1, 4, 9` are the first four squares; the output shows `0 1 [4, 9]`: 0 and 1 from `next`, then 4 and 9 from `list`.
- **Why it matters** In plain words: you can set up a long chain of steps and no work happens until the final consumer asks for results.

The typical wrong version. Build a list when a generator is enough:

```
big_list = [i*i for i in range(1_000_000)]   # all 1 million numbers in memory
big_gen  = (i*i for i in range(1_000_000))   # nothing computed yet
```

Problem 7.2.2 Measure the memory difference between a list and a generator of one million squares.

Step 1	Use `sys.getsizeof` on both. (It measures the container only, not the integers inside the list, so the real gap is even larger.)
Step 2	Confirm both give the same sum.

Predict: Roughly how many bytes will each take? (Think: 8 bytes per list slot.)

```
import sys
big_list = [i*i for i in range(1_000_000)]
big_gen = (i*i for i in range(1_000_000))
print(sys.getsizeof(big_list), sys.getsizeof(big_gen))
print(sum(big_gen) == sum(big_list))
```

```
8448728 208
True
```

- **Explain** One million slots at 8 bytes each is about 8,000,000 bytes, plus some growth room, giving 8,448,728 here. The generator is 208 bytes, no matter how many items it will produce. The exact numbers depend on your Python version and computer; the size of the gap does not.
- **Answer** The list costs about 8.4 MB for the container, the generator 208 bytes, and the sums are equal.
- **Check** 1,000,000 × 8 = 8,000,000 bytes, close to the measured 8,448,728 (the extra is spare capacity and the list header).
- **Why it matters** Scale this to 1 billion order lines: 8 GB versus 208 bytes. This is why data code uses generators for big inputs.

Problem 7.2.3 Build a generator pipeline over a log: parse, keep status OK, take the delivery times, average them.

Step 1	`parse` yields `(status, region, order, minutes)` and skips bad lines.
Step 2	`only_ok` keeps status OK.
Step 3	`times` extracts the `minutes` values.
Step 4	Chain them. Compute the average by hand: (120 + 340 + 95) / 3.

```
lines = ["OK north A1 120", "LATE south B2 900", "OK east C3 340",
         "bad line", "OK west D4 95"]

def parse(ls):
    for l in ls:
        parts = l.split()
        if len(parts) == 4:
            yield parts[0], parts[1], parts[2], int(parts[3])

def only_ok(rs):
    return (r for r in rs if r[0] == "OK")

def times(rs):
    return (r[3] for r in rs)

ts = list(times(only_ok(parse(lines))))
print(ts, sum(ts) / len(ts))
```

```
[120, 340, 95] 185.0
```

- **Answer** Three delivered orders took 120, 340 and 95 minutes; the average is 185.0 minutes.
- **Check** 120 + 340 + 95 = 555 and 555 / 3 = 185. The late order (900 minutes) and the bad line were correctly left out.
- **Why it matters** In plain words: delivered orders average 185 minutes. Each line flows through all stages one at a time, so with a real 20 GB file memory stays tiny. Each stage is small and testable on its own.

Problem 7.2.4 More generator tools: `yield from`, an infinite generator with `islice`, `send`, and batching.

Step 1	`yield from` flattens a list of lists.
Step 2	The Fibonacci generator never ends; `itertools.islice` takes the first ten.
Step 3	`running_avg` receives numbers with `send` and yields the running average.
Step 4	`gen_read` groups items into batches of 3.

```
from itertools import islice

def inner():
    yield from [1, 2]
    yield from "ab"
print(list(inner()))

def chain_flat(nested):
    for sub in nested:
        yield from sub
print(list(chain_flat([[1, 2], [3], [4, 5]])))

def fib():
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b
print(list(islice(fib(), 10)))

def running_avg():
    total = n = 0
    avg = None
    while True:
        x = yield avg
        total += x
        n += 1
        avg = total / n
r = running_avg(); next(r)
print(r.send(10), r.send(20), r.send(60))

def gen_read(lines, size):
    batch = []
    for l in lines:
        batch.append(l)
        if len(batch) == size:
            yield batch
            batch = []
    if batch:
        yield batch
print(list(gen_read(range(7), 3)))
```

```
[1, 2, 'a', 'b']
[1, 2, 3, 4, 5]
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
10.0 15.0 30.0
[[0, 1, 2], [3, 4, 5], [6]]
```

- **Answer** All five outputs match our expectations. The running averages are 10/1 = 10.0, (10+20)/2 = 15.0, (10+20+60)/3 = 30.0.
- **Check** Fibonacci: each number is the sum of the two before it: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34. And 7 items in batches of 3 give 3 + 3 + 1.
- **Why it matters** Batching is how you insert millions of rows into a database in chunks of 1,000 instead of one at a time. An infinite generator plus `islice` is a safe way to take "the first N" of a stream.

The picture

```
 file lines ─► parse ─► only_ok ─► times ─► sum / list
   (lazy)       (lazy)    (lazy)     (lazy)    (consumer pulls: nothing runs until here)

 one line travels through ALL stages before the next line is read
```

Item	Example value	Where it appears in the job
Generator function	`def read_rows(path): ... yield row`	Row-by-row file readers
Generator expression	`(r[3] for r in rows)`	One-off lazy filters
`yield from`	`yield from sub`	Flattening nested results
Batching	`gen_read(rows, 1000)`	Bulk database inserts

Connection to the real job

Pipelines are the shape of data engineering: read, parse, validate, transform, write. Writing each step as a generator keeps memory flat and makes steps easy to test. Frameworks such as Airflow, Spark and PyTorch data loaders follow the same stream-of-items idea.

Applications

- **Data Engineering:** read 50 GB of JSON Lines logs, filter errors, write a summary, using constant memory.
- **Data Science:** generate sliding windows of a time series without storing all of them.
- **Machine Learning:** `yield` training batches forever, one epoch after another.
- **Backend / APIs:** stream a big response row by row instead of building it in memory.

COMMON MISTAKE

Calling `len()` on a generator (`TypeError: object of type 'generator' has no len()`) or indexing it. A generator has no size and no positions.

COMMON MISTAKE

Forgetting that a generator is used up after one pass. Printing it with `list(g)` for debugging empties it for the real consumer.

COMMON MISTAKE

Using a generator for tiny data or when you need to loop many times; a list is simpler and faster there.

Check yourself

1. What does calling a generator function return before any code runs? Answer: a generator object (no body code has run yet).
2. What does `yield` do to the function? Answer: it hands out a value and pauses the function, keeping its local variables.
3. What is the difference between `[x for x in a]` and `(x for x in a)` ? Answer: the first builds a whole list now; the second makes a lazy generator.

7.3 Decorators

INTUITION

Imagine gift-wrapping. The gift (your function) stays the same, but you wrap it in paper that adds something: a bow for timing, a ribbon for retry, a card for logging. Anyone who receives the wrapped gift uses it exactly as before. A decorator is a function that takes your function and returns a wrapped version of it. In everyday terms, it is like a price tag or gift note attached to a product without changing the product.

Words you need

Word	Plain meaning
first-class function	A function is a value; you can pass it around and return it, like a number
closure	An inner function that remembers variables from the function that made it

Word	Plain meaning
wrapper	The inner function that runs extra code around the original
`*args, **kwargs`	"Accept any positional and keyword arguments and pass them on"
`@name`	Short syntax: `@shout` above `def add` means `add = shout(add)`

How it works

The `@` line is only a shortcut. These two are identical:

```
@shout
def add(a, b): ...
# same as:
def add(a, b): ...
add = shout(add)
```

Every decorator has the same skeleton:

```
def decorator(func):
    @functools.wraps(func)           # copy name and docstring of func onto wrapper
    def wrapper(*args, **kwargs):
        ... do something before ...
        result = func(*args, **kwargs)
        ... do something after ...
        return result
    return wrapper
```

A decorator that takes its *own* arguments (like `@retry(3)`) needs one more level: a function that returns the decorator.

Worked examples

Problem 7.3.1 Write `shout`, which prints the function name before each call. Then see why `functools.wraps` is needed.

Step 1	Define `shout(func)` with an inner `wrapper`.
Step 2	Decorate `add` and print `add.__name__` and `add.__doc__`.
Step 3	Repeat with `@functools.wraps(func)` added.

Predict: What will `add.__name__` print for the first version?

```
import functools

def shout(func):
    def wrapper(*args, **kwargs):
        print("calling", func.__name__)
        return func(*args, **kwargs)
    return wrapper

@shout
def add(a, b):
    "Add two numbers."
    return a + b
print(add(2, 3), add.__name__, add.__doc__)

def shout2(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)
    return wrapper

@shout2
def add2(a, b):
    "Add two numbers."
    return a + b
print(add2.__name__, add2.__doc__)
```

```
calling add
5 wrapper None
add2 Add two numbers.
```

- **Explain** After `@shout`, the name `add` points to `wrapper`, so it reports itself as `wrapper` with no docstring. That breaks help text, logging, and debugging tools. `functools.wraps(func)` copies the original name and docstring to the wrapper.
- **Answer** Without `wraps` you get `wrapper None`; with it you get the real name and docstring.
- **Check** The call returned 5, which is 2 + 3, so the wrapped function still works.
- **Why it matters** Without `wraps`, every logged function name is "wrapper" and you cannot tell which function was slow. Always use it.

Problem 7.3.2 Write a `timed` decorator and a `repeat(times)` decorator with an argument.

Step 1	`timed` measures with `time.perf_counter()` and uses `try/finally`, so the time is reported even if the function fails.
Step 2	`repeat(times)` is a decorator factory: it returns `deco`, which returns `wrapper`.
Step 3	Test with a function that sleeps 10 ms, and a function that greets.

```
import functools, time

def timed(func):
    @functools.wraps(func)
    def wrapper(*a, **k):
        t0 = time.perf_counter()
        try:
            return func(*a, **k)
        finally:
            ms = (time.perf_counter() - t0) * 1000
            print(f"{func.__name__} took >=5ms: {ms >= 5}")
    return wrapper

@timed
def slow():
    time.sleep(0.01)
    return "done"
print(slow())

def repeat(times):
    def deco(func):
        @functools.wraps(func)
        def wrapper(*a, **k):
            return [func(*a, **k) for _ in range(times)]
        return wrapper
    return deco

@repeat(3)
def hi(name):
    return f"hi {name}"
print(hi("Ravi"))
```

```
slow took >=5ms: True
done
['hi Ravi', 'hi Ravi', 'hi Ravi']
```

(We print `>=5ms: True` instead of the exact milliseconds because the exact number changes on every run. In real code you would log the number itself.)

- **Explain** `@repeat(3)` first *calls* `repeat(3)`, which returns `deco`, and then applies `deco` to `hi`. So three layers of functions: factory, decorator, wrapper.
- **Answer** `slow` took at least 5 ms (it sleeps 10 ms), and `hi("Ravi")` ran three times.
- **Check** `time.sleep(0.01)` is 10 ms, which is greater than 5 ms, so `True` is correct.
- **Why it matters** In plain words: you got timing in one line (`@timed`) on any function, without editing its body.

The typical wrong version. Forgetting the parentheses on a decorator that takes arguments:

```
@repeat          # should be @repeat(3)
def bad(): pass
bad()
```

```
TypeError: repeat.<locals>.deco() missing 1 required positional argument: 'func'
```

`@repeat` passes the function as the `times` argument, so `bad` becomes `deco`, and calling it fails. Read the message: it names `deco` and the missing `func`. The fix is `@repeat(3)`.

Problem 7.3.3 Write a `retry` decorator for a flaky network call that fails twice, then works. Show the case where all attempts fail.

Step 1	Loop `attempt` from 1 to `max_tries`, call the function in `try`.
Step 2	On the listed exception types, print the failure; on the last attempt re-raise.
Step 3	`flaky` raises `ConnectionError` on calls 1 and 2, then returns.

```
import functools

def retry(max_tries=3, exceptions=(Exception,)):
    def deco(func):
        @functools.wraps(func)
        def wrapper(*a, **k):
            for attempt in range(1, max_tries + 1):
                try:
                    return func(*a, **k)
                except exceptions as e:
                    print(f"  attempt {attempt} failed: {e}")
                    if attempt == max_tries:
                        raise
        return wrapper
    return deco

calls = {"n": 0}

@retry(max_tries=3, exceptions=(ConnectionError,))
def flaky():
    calls["n"] += 1
    if calls["n"] < 3:
        raise ConnectionError("network down")
    return "ok on call %d" % calls["n"]
print(flaky())

@retry(max_tries=2)
def always_bad():
    raise ValueError("nope")
try:
    always_bad()
except ValueError as e:
    print("gave up:", e)
```

```
  attempt 1 failed: network down
  attempt 2 failed: network down
ok on call 3
  attempt 1 failed: nope
  attempt 2 failed: nope
gave up: nope
```

- **Answer** The third attempt succeeded. When every attempt fails, the last exception is re-raised to the caller.
- **Check** `calls["n"]` was 1, 2 (errors), then 3 (success), so `ok on call 3` is right. With `max_tries=2` the loop stops after two failures.
- **Why it matters** In plain words: temporary network problems no longer kill your program, but real bugs still stop it. Only retry errors that can be temporary (connection, timeout), never `ValueError`; our demo does this only to show the re-raise. Real retries add a waiting time that grows between attempts (called backoff); this demo leaves it out.

Problem 7.3.4 Cache results with `functools.lru_cache`, stack two decorators, use a class decorator and a registry.

Step 1	`fib(80)` is computed with and without a cache. Without a cache the work doubles at each step; with `lru_cache` each `n` is computed once.
Step 2	Stacking: `@tag("b")` above `@tag("i")` means `text = tag("b")(tag("i")(text))`, so `i` is applied first (inner).
Step 3	A class decorator receives a class and returns a class; here it adds `__repr__`.
Step 4	A decorator can *register* functions in a dictionary and return them unchanged.

```
import functools, time

@functools.lru_cache(maxsize=None)
def fib(n):
    return n if n < 2 else fib(n-1) + fib(n-2)
print(fib(80), fib.cache_info())

def fib_slow(n):
    return n if n < 2 else fib_slow(n-1) + fib_slow(n-2)
t0 = time.perf_counter(); fib_slow(27); slow_t = time.perf_counter() - t0
print("uncached fib(27) slower than 5ms:", slow_t > 0.005)

@functools.cache
def sq(x):
    print("computing", x)
    return x * x
print(sq(4), sq(4))

def tag(name):
    def deco(f):
        @functools.wraps(f)
        def w(*a, **k):
            return f"<{name}>{f(*a, **k)}</{name}>"
        return w
    return deco

@tag("b")
@tag("i")
def text():
    return "hi"
print(text())

def add_repr(cls):
    def __repr__(self):
        items = ", ".join(f"{k}={v!r}" for k, v in vars(self).items())
        return f"{cls.__name__}({items})"
    cls.__repr__ = __repr__
    return cls

@add_repr
class Point:
    def __init__(self, x, y):
        self.x, self.y = x, y
print(Point(1, 2))

registry = {}
def register(f):
    registry[f.__name__] = f
    return f

@register
def clean(): return "cleaning"
@register
def load(): return "loading"
print(sorted(registry), registry["load"]())
```

```
23416728348467685 CacheInfo(hits=78, misses=81, maxsize=None, currsize=81)
uncached fib(27) slower than 5ms: True
computing 4
16 16
<b><i>hi</i></b>
Point(x=1, y=2)
['clean', 'load'] loading
```

- **Explain** `fib(80)` is a 17-digit number, computed instantly because each of 81 values (0 to 80) was calculated once (`misses=81`) and reused 78 times (`hits=78`). `sq(4)` printed `computing 4` only once; the second call came from the cache. The stacked decorators wrapped from the bottom up: `i` inside, `b` outside. The registry pattern is how web frameworks collect routes (`@app.get("/users")`).
- **Answer** Caching turns an exponential job into a linear one; stacked decorators apply bottom-up; class decorators and registries change or collect things at definition time.
- **Check** 81 misses is exactly the number of distinct inputs (0 to 80). The uncached version of `fib(27)` is not demonstrated with a number here because timing varies; try `fib_slow(35)` yourself to feel the difference.
- **Why it matters** Caching a function is only safe if it is *pure*: the same inputs always give the same output and no side effects. Never cache a function that reads the current time or a database that changes. Also, `lru_cache` needs hashable arguments (a list argument raises `TypeError`).

The picture

```
caller ──► [ wrapper ]  ──►  [ original function ]
            │  before: start timer / check cache / log
            │  after:  stop timer / store result / retry on error
           ◄── result returned to caller unchanged

@a        a(b(f)):   caller ▶ a's wrapper ▶ b's wrapper ▶ f
@b
def f
```

Item	Example value	Where it appears in the job
`@timed`	prints durations	Finding slow pipeline steps
`@retry(3)`	repeats on `ConnectionError`	API calls, database connects
`@lru_cache`	remembers results	Repeated lookups, recursive math
`@app.get("/x")`	registers a route	FastAPI / Flask (Chapter 12)
`@pytest.fixture`	marks a test helper	Testing (Chapter 10)
`@property` , `@dataclass`	built-in decorators	Classes (Chapter 8)

Connection to the real job

You will *use* decorators daily (`@pytest.fixture` , `@app.get` , `@dataclass` , `@task` in Airflow) and *write* them for cross-cutting needs: timing, retry, logging, access checks. A `@timed` decorator that logs the duration of each pipeline step is a natural first tool.

Applications

- **Data Engineering:** retry decorators around API and database calls; `@task` in Airflow.
- **Backend / APIs:** `@app.get("/orders")` routes; an `@require_login` check.
- **Machine Learning:** `@torch.no_grad()` , `@tf.function` modify how a function runs.
- **Performance Testing:** `@timed` on functions inside a Python benchmark script (for example Locust tasks, named only as an example tool).

COMMON MISTAKE	Forgetting `functools.wraps`. Names and docstrings are lost.

COMMON MISTAKE	Forgetting to `return` the result inside `wrapper`. The decorated function then always returns `None`.

COMMON MISTAKE	Retrying everything, including non-temporary errors and non-idempotent actions (such as "charge the customer"). A retry can double the effect. Idempotent means repeating gives the same result as doing it once; retry only those.

Check yourself

1. What is `@shout` above `def add` equal to? Answer: `add = shout(add)`.
2. Why do we write `*args, **kwargs` in the wrapper? Answer: so the wrapper accepts and passes on whatever arguments the original function takes.
3. What does `functools.wraps` do? Answer: it copies the name, docstring and other details of the original function to the wrapper.

7.4 Context Managers

INTUITION	When you rent a hall for an event, there is a checkout rule: you must switch off the lights and return the key, even if the party went badly. A context manager is that rule in code. The `with` block says: "set things up, let me do my work, and *always* tidy up at the end, even if something exploded in the middle."

Words you need

Word	Plain meaning
resource	Something you must give back: a file, a connection, a lock
context manager	An object that sets up and cleans up around a block of code
`__enter__`	Runs at the start of `with`; its return value goes to `as name`
`__exit__`	Runs at the end, always; it receives the exception details, if any
suppress	Swallow an exception so it does not reach the caller

How it works

`with thing as x:` means: call `thing.__enter__()`, run the block, then call `thing.__exit__(exc_type, exc, traceback)` whatever happened. If `__exit__` returns `True`, the exception is swallowed; if it returns `False` (or nothing), the exception continues.

Two ways to make one:

1. **A class** with `__enter__` and `__exit__`.
2. **A generator** with `@contextlib.contextmanager`: code before `yield` is the setup, the `yield` gives the value, and code after `yield` (in `finally`) is the cleanup. This is shorter and used most often.

Ready-made helpers in `contextlib`: `suppress(...)`, `redirect_stdout`, `ExitStack` (a stack of many managers when you do not know how many beforehand).

Why does it exist? Before it, people wrote `try/finally` by hand and forgot one. `with` makes the right way the easy way.

Worked examples

Problem 7.4.1 Write a class-based `Timer` context manager. See what happens with and without an error.

Step 1	`__enter__` records the start time and returns `self`.
Step 2	`__exit__` records the elapsed time and prints whether an exception happened. It returns `False`, so errors still propagate.
Step 3	Run a normal block, and then a block that divides by zero.

Predict: In the failing block, does `__exit__` run? Does the `ZeroDivisionError` still reach us?

```
import time

class Timer:
    def __enter__(self):
        self.t0 = time.perf_counter()
        print("enter")
        return self
    def __exit__(self, exc_type, exc, tb):
        self.elapsed = time.perf_counter() - self.t0
        print("exit, exception:", exc_type.__name__ if exc_type else None)
        return False

with Timer() as t:
    time.sleep(0.01)
print(t.elapsed > 0.005)

try:
    with Timer():
        1/0
except ZeroDivisionError:
    print("error still propagates")
```

```
enter
exit, exception: None
True
enter
exit, exception: ZeroDivisionError
error still propagates
```

- **Explain** In both runs `__exit__` ran, so cleanup is guaranteed. In the failing run it received `ZeroDivisionError` as `exc_type`. Because we returned `False`, the error continued outwards and our `except` caught it.
- **Answer** `__exit__` always runs, and errors are not hidden unless you return `True`.
- **Check** The `sleep(0.01)` is 10 ms, so `elapsed > 0.005` (5 ms) is `True`.
- **Why it matters** In plain words: the timer reports even when the code fails, which is exactly when you want a measurement.

Problem 7.4.2 Swallow only one kind of error with a class, and with `contextlib.suppress`.

Step 1	`Suppress.__exit__` returns `True` only when the exception is a `KeyError`.
Step 2	`with contextlib.suppress(FileNotFoundError):` removes a file that may not exist.

```
import contextlib, os

class Suppress:
    def __enter__(self):
        return self
    def __exit__(self, et, e, tb):
        return et is KeyError

with Suppress():
    {}["x"]
print("KeyError swallowed")

with contextlib.suppress(FileNotFoundError):
    os.remove("/tmp/definitely_not_here_123")
print("suppress ok")
```

```
KeyError swallowed
suppress ok
```

- **Answer** Both blocks swallowed their expected error and the program carried on.
- **Check** Removing a missing file would normally raise `FileNotFoundError`; since we saw `suppress ok`, it was swallowed.
- **Why it matters** `suppress` is perfect for "delete if exists" and similar harmless cases. Never use it for errors you do not understand.

Problem 7.4.3 Write context managers with `@contextlib.contextmanager`: one that changes into a temporary folder and always returns, one that prints open/close, and combine several in one `with`.

Step 1	Save the old folder, create a temp folder, `os.chdir` into it, `yield`, and in `finally` go back and remove it.
Step 2	`opened(name)` prints `open` before the `yield` and `close` after, inside `finally`.
Step 3	`with opened("a") as a, opened("b") as b:` enters left to right and exits right to left.
Step 4	`redirect_stdout` captures prints; `ExitStack` handles a list of managers.

```
import contextlib, os, tempfile, io

@contextlib.contextmanager
def chdir_temp():
    old = os.getcwd()
    d = tempfile.mkdtemp()
    os.chdir(d)
    try:
        yield d
    finally:
        os.chdir(old)
        os.rmdir(d)
        print("restored")

with chdir_temp() as d:
    print(os.path.samefile(os.getcwd(), d))

@contextlib.contextmanager
def opened(name):
    print("open", name)
    try:
        yield name
    finally:
        print("close", name)

with opened("a") as a, opened("b") as b:
    print("inside", a, b)

buf = io.StringIO()
with contextlib.redirect_stdout(buf):
    print("captured")
print(repr(buf.getvalue()))

with contextlib.ExitStack() as st:
    fs = [st.enter_context(opened(n)) for n in "xy"]
    print("both open", fs)
```

```
True
restored
open a
open b
inside a b
close b
close a
'captured\n'
open x
open y
both open ['x', 'y']
close y
close x
```

- **Explain** Inside `chdir_temp` the working folder really is the temp folder (`samefile` is `True`), and afterwards we are back (`restored`). `a` is opened first and closed last, like putting on and taking off a coat and a jacket in the right order. `redirect_stdout` sent the `print` into the buffer, which is useful in tests. `ExitStack` opened `x` and `y` and closed them in reverse order.
- **Answer** Cleanup always ran, in reverse order of setup.
- **Check** The open/close order is a stack: `open a, open b ... close b, close a` . The same pattern appears for `x` and `y` .
- **Why it matters** The `try/finally` around `yield` is what guarantees the cleanup if the block raises an error. Leave it out and a failure leaves your program in the wrong folder.

The typical wrong version. Opening a file without `with` , when an error happens before `close()` :

```
f = open("data.txt", encoding="utf-8")
total = 1 / 0        # error: the next line never runs
f.close()
```

The file stays open until the program ends. We checked this behaviour: after an error that was caught elsewhere the file object still reported `closed == False` (our run printed `without with, still open: True`). With `with open(...) as f:` the file is closed on the way out. Leaked handles become "Too many open files" errors in long-running jobs.

The picture

```
with A() as x:          __enter__()  ──► x
    body                run body ──► (ok or error)
                        __exit__(error info)  ◄── ALWAYS runs
                            return True  → error swallowed
                            return False → error continues up

@contextmanager:  [setup] ── yield value ── [cleanup in finally]
```

Item	Example value	Where it appears in the job
File	`with open(p) as f:`	Reading and writing files
DB connection	`with engine.begin() as conn:`	Commit on success, rollback on error
Lock	`with lock:`	Threads (Chapter 11)
Temp folder	`with tempfile.TemporaryDirectory()`	Tests
`mock.patch`	`with patch("x.y"):`	Testing (Chapter 10)
GPU / no-grad	`with torch.no_grad():`	Machine learning

Connection to the real job

Database sessions, HTTP sessions, file handles, locks, temporary folders, test mocks and ML inference modes are all context managers. A data engineer who forgets to close connections eventually brings down a database. A senior reviewer looks for `with` around every resource.

Applications

- **Data Engineering:** `with` around database transactions: commit if fine, roll back on error.
- **Machine Learning:** `with torch.no_grad():` switches off gradient tracking for inference.
- **Testing / MLOps:** temporary directories and patched settings that are restored after each test.
- **Performance Testing:** a `Timer` block that records the duration of a code section.

COMMON MISTAKE Returning `True` from `__exit__` by accident. Every error then disappears silently. Return `False` or `None` unless you mean to swallow.

COMMON MISTAKE In `@contextmanager`, putting the cleanup after `yield` without `try/finally`. If the block raises, the cleanup is skipped.

COMMON MISTAKE Using the resource after the `with` block. The file is closed; reading it gives `ValueError: I/O operation on closed file`.

Check yourself

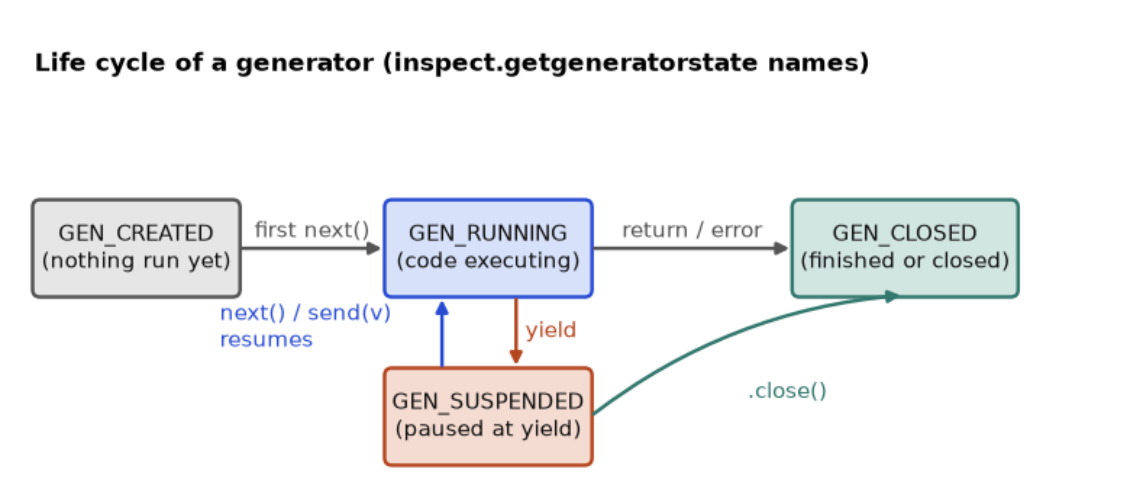
1. When does `__exit__` run? Answer: always at the end of the `with` block, even after an error.
2. How do you swallow an exception in `__exit__`? Answer: return `True`.
3. In `@contextmanager`, what does the `yield` mark? Answer: the point where the `with` body runs; code before is setup, code after is cleanup.

7.5 Two-Way Generators, Parametrised Decorators and Stacked Cleanup

Sections 7.2 to 7.4 used each feature in its simplest form. This section covers the forms that appear in real code: generators that receive values, decorators that take settings, and context managers that you assemble at run time.

Generators that receive values

A generator is not only a producer. `g.send(value)` resumes it and makes the paused `yield` expression evaluate to `value`. The figure shows the four states a generator moves through. You must call `next(g)` once first, to run the code up to the first `yield`; sending a non-`None` value to a generator that has not started raises `TypeError`.



Life cycle of a generator. The state names are the ones printed by `inspect.getgeneratorstate`.

Problem 7.5.1 Write a generator `running_avg()` that you feed numbers with `send` and that answers with the average so far.

Idea	Keep `total` and `count` as local variables; they survive between pauses.
Step 1	`x = yield avg` hands the current average out and receives the next number in.
Step 2	Prime it with `next(g)` (state moves from CREATED to SUSPENDED).
Check	Sending 10, 20, 60 gives running sums 10, 30, 90 and averages 10/1 = 10.0, 30/2 = 15.0, 90/3 = 30.0.

```
import inspect
def running_avg():
    total = 0.0
    count = 0
    avg = None
    while True:
        x = yield avg
        total += x
        count += 1
        avg = total / count
g = running_avg()
print(inspect.getgeneratorstate(g))
next(g)
print(inspect.getgeneratorstate(g))
print(g.send(10), g.send(20), g.send(60))
g.close()
print(inspect.getgeneratorstate(g))
```

```
GEN_CREATED
GEN_SUSPENDED
10.0 15.0 30.0
GEN_CLOSED
```

`yield from` hands control to another generator and gives you its `return` value. `close()` raises `GeneratorExit` at the paused `yield`, so a `finally` block there runs.

```
def inner():
    yield 1
    yield 2
    return "done"
def outer():
    result = yield from inner()
    print("inner returned", result)
    yield 3
print(list(outer()))
def cleanup_demo():
    try:
        yield 1
        yield 2
    finally:
        print("cleanup ran")
g = cleanup_demo()
print(next(g))
g.close()
```

```
inner returned done
[1, 2, 3]
1
cleanup ran
```

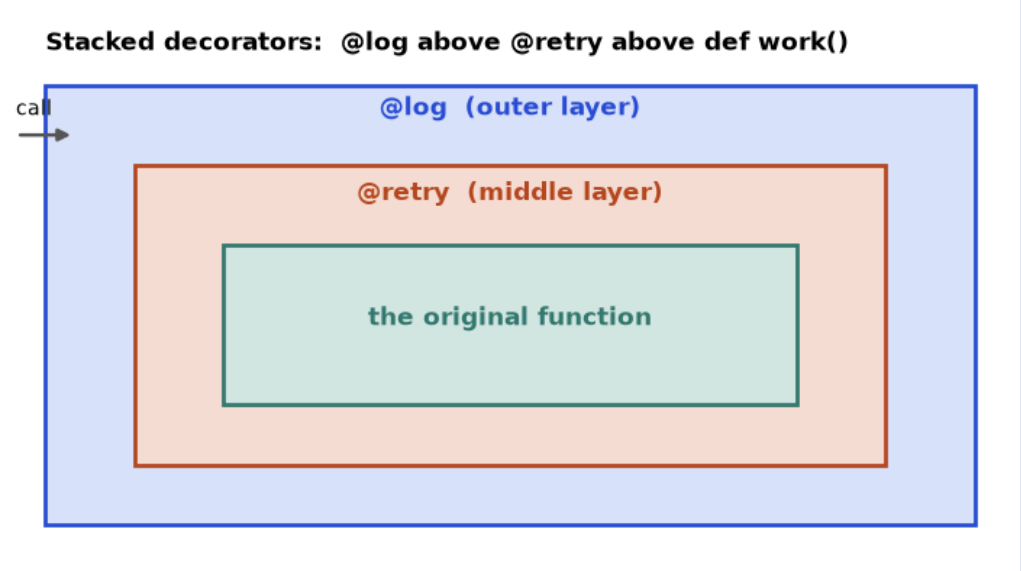
COMMON MISTAKE

Iterating a generator twice. `gen = (x * 10 for x in range(3))` then `list(gen)` gives `[0, 10, 20]`, and a second `list(gen)` gives `[]` with no error, because the generator is exhausted. Fix: create a new generator for each pass, or store the results in a list if you need them twice.

Decorators that take arguments

A decorator with settings, such as `@retry(times=3)`, needs three nested functions: the outer one takes the settings and returns the real decorator, which takes the function and returns the wrapper. The picture below is the mental model for stacked decorators: the call enters the outermost layer first

and reaches your function last.



With `@log` above `@retry` above `def work()`, a call passes through log, then retry, then the original function. Results travel back out in reverse order.

Problem 7.5.2 Write `retry(times, exceptions)` that retries only the listed exceptions, re-raises on the last failure, and keeps the function's name and docstring.

Layer 1	`retry(times, exceptions)` stores the settings in its closure.
Layer 2	`decorator(func)` receives the function.
Layer 3	`wrapper` loops over attempts; on the last attempt it uses bare `raise` so the original traceback is kept.
Check	A function that fails twice then succeeds returns on attempt 3. A `ValueError` is not in the default tuple, so it is not retried unless asked.

```
import functools
def retry(times=3, exceptions=(ConnectionError,)):
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(1, times + 1):
                try:
                    return func(*args, **kwargs)
                except exceptions as e:
                    print(f"attempt {attempt} failed: {e}")
                    if attempt == times:
                        raise
        return wrapper
    return decorator
calls = {"n": 0}
@retry(times=3)
def flaky(x):
    """Return x after two failures."""
    calls["n"] += 1
    if calls["n"] < 3:
        raise ConnectionError("down")
    return x * 2
print(flaky(21), flaky.__name__, flaky.__doc__)
@retry(times=2, exceptions=(ValueError,))
def bad():
    raise ValueError("nope")
try:
    bad()
except ValueError as e:
    print("gave up:", e)
```

```
attempt 1 failed: down
attempt 2 failed: down
42 flaky Return x after two failures.
attempt 1 failed: nope
attempt 2 failed: nope
gave up: nope
```

COMMON MISTAKE

Forgetting `@functools.wraps(func)`. Without it, the decorated function reports `__name__` as `wrapper` and loses its docstring, which breaks help text, logging and debuggers. Wrong: `greet.__name__` prints `wrapper` and `greet.__doc__` prints `None`. Right: add `wraps` and you see `greet` and `Say hello.`

Stacking context managers

A `@contextmanager` generator runs up to `yield` on entry and after it on exit. An exception inside the `with` body is raised at the `yield`, so you can catch it there. When the number of resources is only known at run time, `ExitStack` enters them one by one and closes them in reverse order.

```
from contextlib import contextmanager, ExitStack
@contextmanager
def step(name):
    print("enter", name)
    try:
        yield name
    except ZeroDivisionError:
        print("handled in", name)
    finally:
        print("exit", name)
with step("A") as a:
    1 / 0
    print("not reached")
with ExitStack() as stack:
    names = [stack.enter_context(step(n)) for n in ("x", "y", "z")]
    print(names)
```

```
enter A
handled in A
exit A
enter x
enter y
enter z
['x', 'y', 'z']
exit z
exit y
exit x
```

Going deeper

- A `@contextmanager` that does not wrap `yield` in `try/finally` skips its cleanup when the body raises, because the exception is thrown in at the `yield` and leaves the generator immediately. Always use `try/finally` for the cleanup.
- If the generator catches the exception and does not re-raise, the `with` statement swallows it (as `step("A")` did above). Swallow deliberately or not at all.
- `yield from` is also how a generator delegates `send` and `throw` to a sub-generator. The old coroutine style of asynchronous code was built from this; `async` / `await` (Chapter 11) replaced it.

More practice: build and stretch

Problem 7.P.11 (build) Write an infinite generator `evens()` and take the first five values without building an infinite list.

```
from itertools import islice
def evens():
    n = 0
    while True:
        yield n
        n += 2
print(list(islice(evens(), 5)))
```

```
[0, 2, 4, 6, 8]
```

Answer: as shown. `islice` stops pulling after five items, so the infinite loop never runs away. Self-check: did you use `yield` inside a loop rather than `return`?

Problem 7.P.12 (what is the output?) A decorator `loud` upper-cases the result of the function it wraps and does not use `functools.wraps`. What do `greet("ana")`, `greet.__name__` and `greet.__doc__` give for `def greet(name): "Say hello."; return "hi " + name`?

```
def loud(func):
    def wrapper(*a, **k):
        return func(*a, **k).upper()
    return wrapper
@loud
def greet(name):
    "Say hello."
    return "hi " + name
print(greet("ana"), greet.__name__, greet.__doc__)
```

```
HI ANA wrapper None
```

Answer: `HI ANA wrapper None`. The name and docstring belong to `wrapper`. One line, `@functools.wraps(func)` above `def wrapper`, fixes both.

Problem 7.P.13 (stretch) Write a context manager `timer(label, sink)` that stores the elapsed seconds in the dict `sink[label]` even when the body raises. Show it records a time for a body that raises `RuntimeError`.

```
import time
from contextlib import contextmanager
@contextmanager
def timer(label, sink):
    start = time.perf_counter()
    try:
        yield
    finally:
        sink[label] = time.perf_counter() - start
t = {}
try:
    with timer("job", t):
        raise RuntimeError("boom")
except RuntimeError:
    pass
print("job" in t, t["job"] >= 0)
```

```
True True
```

Answer: The `finally` clause runs before the exception leaves the `with`, so the time is stored. The exact seconds vary, so the code prints only checks. Without `try/finally` the dict would stay empty.

Problem 7.P.14 (stretch, interview) What is the difference between `return` inside a generator and `yield`, and what does `list(outer())` show when `outer` uses `result = yield from inner()` and `inner` ends with `return "done"`?

Answer: `yield` pauses and produces a value; `return` ends the generator (the value becomes `StopIteration.value`). With `yield from`, `result` is that returned value, `"done"`, and it is not part of the list. In the example in 7.5 the list is `[1, 2, 3]`.

Chapter recap

- An iterable can be looped over; an iterator hands out items with `next()` and raises `StopIteration` at the end.
- An iterator can be used only once. A list gives a fresh iterator for each loop.
- A generator function uses `yield`; calling it runs no code until you ask for items (lazy evaluation).
- Generators use tiny, constant memory: 208 bytes versus about 8.4 MB for one million squares (container only).

- Chain generators into a pipeline (parse, filter, extract) so each item flows through all stages one by one.
- A decorator is a function that wraps a function; `@d` is `f = d(f)`.
- Always use `functools.wraps`, `*args, **kwargs`, and remember to return the result.
- A decorator with arguments needs a third layer; do not forget the parentheses: `@retry(3)`.
- Useful ones: timing, retry (only for temporary errors), `lru_cache` (only for pure functions), registries.
- A context manager guarantees cleanup via `with`; build one with `__enter__/__exit__` or `@contextlib.contextmanager` with `try/finally`.

Cheat-sheet

Task	Pattern
Manual iteration	`it = iter(x); next(it); next(it, default)`
Generator function	`def g(): yield value`
Generator expression	`(f(x) for x in data if cond)`
Delegate	`yield from other`
First N of a stream	`list(itertools.islice(gen, N))`
Decorator skeleton	`def d(f): @wraps(f) def w(*a, **k): return f(*a, **k); return w`
Decorator with args	`def d(n): def deco(f): ...; return deco`
Cache pure function	`@functools.lru_cache(maxsize=None)` or `@functools.cache`
Context manager (class)	`__enter__(self)` and `__exit__(self, et, e, tb)`
Context manager (function)	`@contextmanager def cm(): setup; try: yield; finally: cleanup`

Practice set

Problem 7.P.1 What is `next(iter([5, 6]))`?

Answer: `5`.

Problem 7.P.2 What does this print?

```
g = (x * 2 for x in [1, 2, 3])
print(list(g))
print(list(g))
```

Answer: `[2, 4, 6]` then `[]`. The generator is used up after the first pass.

Problem 7.P.3 What is the output?

```
def f():
    yield 1
    yield 2
print(f())
```

Answer: Something like `<generator object f at 0x...>`. The function body did not run, and the address changes every run.

Problem 7.P.4 Write a generator `evens(limit)` that yields even numbers below `limit`. What is `list(evens(10))`?

Answer:

```
def evens(limit):
    for n in range(limit):
        if n % 2 == 0:
            yield n
```

`list(evens(10))` is `[0, 2, 4, 6, 8]`.

Problem 7.P.5 Find the bug:

```
def shout(func):
    def wrapper(*args, **kwargs):
        print("calling", func.__name__)
        func(*args, **kwargs)
    return wrapper

@shout
def add(a, b):
    return a + b
print(add(2, 3))
```

Answer: The wrapper does not `return` the result, so `add(2, 3)` prints `calling add` and then `None`. Write `return func(*args, **kwargs)`. Also add `@functools.wraps(func)`.

Problem 7.P.6 In what order do these print?

```
@tag("b")
@tag("i")
def text(): return "hi"
```

where `tag(name)` wraps the output as `<name>...</name>`.

Answer: The result is `<b><i>hi</i></b>`. The decorator nearest the function (`i`) is applied first and ends up inside.

Problem 7.P.7 What does `with opened("a"), opened("b"):` print when the body prints `inside`, if `opened` prints `open` / `close`?

Answer: `open a`, `open b`, `inside`, `close b`, `close a`. Exit is in reverse order.

Problem 7.P.8 Find the bug: a `@contextmanager` function does `yield conn` and then `conn.close()` on the next line, with no `try/finally`. What goes wrong?

Answer: If the `with` body raises an error, the exception is thrown at the `yield`, and the `conn.close()` line never runs, so the connection leaks. Use `try: yield conn finally: conn.close()`.

Problem 7.P.9 Scenario: a daily job processes a 30 GB file of JSON Lines to count errors per hour. How do you structure it?

Answer: Open the file with `with open(...)`, read lines lazily, and build generator stages: `parse_json(lines)` (skip bad lines and count them), `only_errors(rows)`, and a final loop that updates a `Counter` keyed by hour. Memory stays small because only one line is alive at a time. Log the number of skipped bad lines at the end.

Problem 7.P.10 Interview: what is the difference between an iterator and an iterable? Give an example of each.

Answer: An iterable can be looped over and returns a new iterator from `iter()` (list, str, dict, file path object contents). An iterator keeps its position, gives items with `next()`, and is used up once (`iter([1,2])`, a generator, an open file). Every iterator is iterable, but not every iterable is an iterator.

Problem 7.P.11 Interview: write a `timed` decorator that logs the function name and duration, and explain why you use `functools.wraps`.

Answer:

```
import functools, logging, time
log = logging.getLogger(__name__)

def timed(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        t0 = time.perf_counter()
        try:
            return func(*args, **kwargs)
        finally:
            log.info("%s took %.1f ms", func.__name__, (time.perf_counter() - t0) * 1000)
    return wrapper
```

`wraps` keeps the original `__name__` and docstring; without it every function would be logged as `wrapper`. `try/finally` logs even on failure and `perf_counter` is a steady clock.

Problem 7.P.12 Interview: when is `lru_cache` a bad idea?

Answer: When the function is not pure (it reads the clock, a database, a file, or random numbers) because stale results would be returned; when arguments are not hashable (lists, dicts); and when the number of distinct inputs is huge with no `maxsize`, since the cache grows forever and eats memory.

Prompt pack

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data roles, and English is my second language, so use simple words. Teach me Python iterators and the iterator protocol: iterable versus iterator, iter(), next(), StopIteration, and how a for loop works inside. Use a vending machine analogy first, then small runnable examples. After each part, ask me one "predict the output" question and wait for my answer before continuing.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data roles. Topic: Python generators, yield, yield from, generator expressions, lazy evaluation and memory benefits, and generator pipelines for processing large log files. Give me 8 graded exercises from easy to hard without solutions. After I paste my answer to each, review it for bugs, memory use and clarity, and explain mistakes in simple English.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data roles, and English is my second language. Topic: Python decorators: closures, functools.wraps, *args and **kwargs, decorators with arguments, stacking order, class decorators, and practical decorators for timing, caching, retry and logging. Explain step by step with the gift-wrapping analogy, then review my decorator code: [PASTE HERE]. Point out bugs and suggest improvements.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data roles. Topic: Python context managers: the with statement, __enter__ and __exit__, contextlib.contextmanager, suppress, redirect_stdout and ExitStack, and why cleanup must be guaranteed. Quiz me one question at a time, from easy to hard, including find-the-bug questions. Explain each wrong answer step by step, and finish with a short summary of what I should practise next.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data roles, and English is my second language. Topic: iterators, generators, decorators and context managers in Python. Run a mock technical interview with 8
```

```
questions of rising difficulty, one at a time. Grade each answer out of 10, show a model answer, and ask one follow-up question. At the end, list my three weakest points and give me a one-week practice plan of one hour per day.
```

CHAPTER 8

OBJECT-ORIENTED PROGRAMMING

Classes, special methods, inheritance, composition, SOLID and the design patterns you will meet in real code.

Open any serious Python library and you will find classes: pandas has `DataFrame`, scikit-learn has `LinearRegression`, FastAPI has `FastAPI`, Locust has `HttpUser`. To read that code, and to write code that other engineers can extend without breaking it, you need object-oriented programming (OOP). OOP is a way of organising a program so that **data and the functions that work on that data live together**. By the end of this chapter you will write your own classes, make them behave like built-in types, choose between inheritance and composition, apply the SOLID rules, and recognise the nine design patterns that interviewers ask about. We use one running idea throughout: a small online shop (orders, money, users, discounts). All code was run with Python 3.14 and the outputs are real.

8.1 Classes, Instances and Attributes

INTUITION

Think of a cookie cutter. It is a template: one shape. When you press it into dough, you get 10 real cookies from that template. Each cookie has its own decorations, but all follow the same shape. A **class** is the template. An **instance** (also called an **object**) is one real thing made from it.

Words you need

Word	Plain meaning
class	A template that describes what data and behaviour its objects have
instance / object	One real thing made from a class
attribute	A named piece of data stored on a class or object
method	A function defined inside a class
`self`	The object the method is working on. Python passes it automatically
`__init__`	A method that runs right after an object is created, to set up its data

How it works. `class Name:` defines a class. Calling `Name(...)` creates an instance and runs `__init__`. An **instance attribute** (`self.count`) belongs to one object. A **class attribute** (written directly in the class body) is shared by all objects. Why does `self` exist? A method must know which object it should change. If there were no `self`, `hit()` would not know whose counter to increase.

Real-world mapping

Concept	Example value	Where it appears in the job
Class	`LinearRegression`	scikit-learn model template
Instance	`model = LinearRegression()`	One trained model
Instance attribute	`model.coef_`	Weights learned from your data
Method	`model.fit(X, y)`	Changes the object's own data
Class attribute	`HttpUser.wait_time`	A setting shared by all virtual users

Problem 8.1.1 Create a `Counter` class that counts hits per endpoint and counts how many counters exist.

Before you run it, predict: after `a.total_created = 99`, what will `b.total_created` print?

- **Step 1** Put `total_created` in the class body (shared). Put `name` and `count` in `__init__` (one set per object).
- **Step 2** Create two counters and hit them.
- **Step 3** Assign `a.total_created = 99` and compare.

```
class Counter:
    total_created = 0          # class attribute
    def __init__(self, name):
        self.name = name       # instance attribute
        self.count = 0
        Counter.total_created += 1
    def hit(self):
        self.count += 1
        return self.count

a = Counter("login")
b = Counter("search")
a.hit(); a.hit(); b.hit()
print(a.name, a.count, b.name, b.count)
print(Counter.total_created, a.total_created)
a.total_created = 99          # creates an instance attribute, shadows class one
print(a.total_created, b.total_created, Counter.total_created)
print(a.__dict__)
print(type(a), type(Counter))
```

```
login 2 search 1
2 2
99 2 2
{'name': 'login', 'count': 2, 'total_created': 99}
<class '__main__.Counter'> <class 'type'>
```

Line by line: `a` was hit twice and `b` once, so each keeps its own `count`. Two counters were created, so the shared value is 2. Writing `a.total_created = 99` did not change the class value. It made a **new instance attribute** on `a` (you can see it inside `a.__dict__`), which hides the class one for `a` only.

- **Answer** `b.total_created` prints 2. Assignment through an object never edits the class attribute.
- **Check** `Counter.total_created` is still 2 in the output, and `a.__dict__` contains the new key.
- **Why it matters** Mixing up "per-object" and "shared" data is the number one OOP bug in data code.

Problem 8.1.2 Show the classic shared-list bug and its fix.

Predict: do `x` and `y` end up with one tag each?

```
class Bad:
    tags = []
    def add(self, t): self.tags.append(t)
x, y = Bad(), Bad()
x.add("a"); y.add("b")
print(x.tags, y.tags, x.tags is y.tags)
class Good:
    def __init__(self): self.tags = []
    def add(self, t): self.tags.append(t)
x, y = Good(), Good()
x.add("a"); y.add("b")
print(x.tags, y.tags)
```

```
['a', 'b'] ['a', 'b'] True
['a'] ['b']
```

- **Answer** No. In `Bad`, one list lives on the class, so both objects add to the same list (`is` prints `True`). In `Good`, each object makes its own list in `__init__`.
- **Check** `x.tags is y.tags` is `True` in `Bad` and the two lists differ in `Good`.
- **Why it matters** Two "independent" shop customers that secretly share one basket list will corrupt each other's orders.

The typical beginner errors and their real messages

```
class A:
    def hit(): return 1       # forgot self
A().hit()
```

```
A.hit() takes 0 positional arguments but 1 was given
```

Python passed the object as the first argument, but you declared none. Fix: `def hit(self):`.

```
class B:
    def __init__(self, n): self.n = n
B()
```

```
B.__init__() missing 1 required positional argument: 'n'
```

Fix: `B(5)`, or give `n` a default.

The picture

```
Class Counter (template)          Instances (real objects)
+--------------------+            a --> {name:'login',  count:2}
| total_created = 2  | <-- shared  b --> {name:'search', count:1}
| __init__, hit()    |            (each has its own __dict__)
+--------------------+
```

Read it like this: the box on the left exists once. Each arrow on the right is one object holding its own dictionary of attributes. When Python looks up `a.something`, it checks `a`'s own dictionary first, then the class.

Connection to the real job. Pipelines hold state in objects: a database connection, a model with learned weights, a configured API client. When results differ across runs, a shared mutable class attribute is a top suspect.

Applications

- **Data Analytics:** a `Report` class that holds a DataFrame, a title and an `export()` method.
- **Data Science / ML:** every scikit-learn estimator is a class with `fit` and `predict`.
- **Deep Learning:** a PyTorch model is a class that inherits from `nn.Module`.
- **Data Engineering:** a `PostgresConnection` object that owns one open connection.
- **Load simulation:** tools such as Locust define users as classes; each instance is one simulated user.

COMMON MISTAKE	Putting a list or dict in the class body, or as a default argument (`def __init__(self, d={})`), and expecting one copy per object. Mutable starting values go inside `__init__`. Class attributes should be constants.

Check yourself

1. What does `self` refer to?

Answer: The specific object the method was called on.

1. Where do you put data that must be separate for each object?

Answer: In `__init__`, as `self.something`.

1. If you write `obj.x = 5` and `x` was a class attribute, what changes?

Answer: Only `obj` gets a new instance attribute. The class attribute is unchanged.

8.2 Dunder (Special) Methods

INTUITION

When you type `len(x)` or `a + b` or `print(obj)`, Python quietly calls a hidden method on your object. These methods have double underscores on each side, so people say "dunder" (double-underscore). A restaurant analogy: you just say "the usual", and the waiter knows what that means. Dunders are how your class defines what "the usual" means for `+`, `len`, `print` and `for`.

Why they exist. Without them, your `Money` object could not be sorted, added or printed nicely. With them it behaves like a number. Python decides what an operator means by calling the method, not by hard-coded rules.

You write	Python calls	Purpose
`repr(x)`, list display, debugger	`__repr__`	Unambiguous text for developers
`print(x)`, `str(x)`	`__str__`	Friendly text for users
`a == b`	`__eq__`	Equality
`hash(x)`, set member, dict key	`__hash__`	Fixed number for fast lookup
`a < b`	`__lt__`	Ordering
`a + b`	`__add__`	Addition
`len(x)`	`__len__`	Size
`x[i]`	`__getitem__`	Indexing and slicing
`for v in x`	`__iter__`	Iteration
`v in x`	`__contains__`	Membership
`x(...)`	`__call__`	Make an object callable like a function
`with x:`	`__enter__`, `__exit__`	Context manager
`if x:`	`__bool__`	Truthiness

Two rules to remember. First, if `__str__` is missing, Python uses `__repr__`. So always write `__repr__`. Second, if you define `__eq__` and not `__hash__`, Python makes your object **unhashable** (it cannot go in a set or be a dict key). Objects that are equal must have equal hashes.

Problem 8.2.1 Build a `Money` class that stores paise (1 rupee = 100 paise) as an integer to avoid float errors, with printing, comparison, addition and hashing.

Predict: what does `sorted([Money(10,50), Money(3,75), Money(1)])` print?

- **Step 1** Store `self.paise = rupees*100 + paise`. Integers are exact; `0.1 + 0.2` floats are not.
- **Step 2** Write `__repr__`, `__str__`, `__eq__`, `__lt__`, `__hash__`, `__add__`, `__bool__`.
- **Step 3** Add `@total_ordering` from `functools`: given `__eq__` and `__lt__`, it fills in `>`, `>=`, `<=`.

```
from functools import total_ordering
@total_ordering
class Money:
    def __init__(self, rupees, paise=0):
        self.paise = rupees*100 + paise
    def __repr__(self): return f"Money({self.paise//100}, {self.paise%100})"
    def __str__(self): return f"Rs {self.paise/100:.2f}"
    def __eq__(self, o): return isinstance(o, Money) and self.paise == o.paise
    def __lt__(self, o): return self.paise < o.paise
    def __hash__(self): return hash(self.paise)
    def __add__(self, o): return Money(0, self.paise + o.paise)
    def __bool__(self): return self.paise != 0

m1, m2 = Money(10, 50), Money(3, 75)
print(repr(m1), str(m1), m1 + m2)
print(m1 > m2, m1 == Money(10,50), m1 >= m2)
print(sorted([m1, m2, Money(1)]))
print({m1, Money(10,50)})
print(bool(Money(0)))
print([m1, m2])
```

```
Money(10, 50) Rs 10.50 Rs 14.25
True True True
[Money(1, 0), Money(3, 75), Money(10, 50)]
{Money(10, 50)}
False
[Money(10, 50), Money(3, 75)]
```

Line by line: `repr` gives the developer view, `str` gives the user view, and `print(m1 + m2)` uses `__str__`, so we see `Rs 14.25` (1050 + 375 = 1425 paise). Sorting works because `__lt__` exists. The set keeps one item because two equal `Money(10,50)` have the same hash. `Money(0)` is falsy because of `__bool__`. A list shows items using `__repr__`.

- **Answer** It prints `[Money(1, 0), Money(3, 75), Money(10, 50)]`, ordered 100, 375, 1050 paise.
- **Check** 1050 + 375 = 1425 paise = Rs 14.25, matching the output.
- **Why it matters** Money, timestamps and IDs in real systems need exact equality and ordering. This is how pandas and Decimal-like types are built.

Problem 8.2.2 Make a container that behaves like a sequence, and an object that behaves like a function.

```
class Playlist:
    def __init__(self, *songs): self._s = list(songs)
    def __len__(self): return len(self._s)
    def __getitem__(self, i): return self._s[i]
    def __iter__(self): return iter(self._s)
    def __contains__(self, x): return x in self._s
p = Playlist("a","b","c")
print(len(p), p[0], p[-1], p[0:2], "b" in p, list(p))

class Timer:
    def __init__(self, f): self.f = f; self.calls = 0
    def __call__(self, *a):
        self.calls += 1
        return self.f(*a)
sq = Timer(lambda x: x*x)
print(sq(4), sq(5), sq.calls)
```

```
3 a c ['a', 'b'] True ['a', 'b', 'c']
16 25 2
```

- **Answer** `len`, indexing, slicing, `in` and `list()` all work on `Playlist` because we defined four dunders. `sq` is called like a function but remembers state: it was called twice.
- **Check** `4*4 = 16`, `5*5 = 25`, two calls. The slice `p[0:2]` passes a slice object to `__getitem__`, which the inner list handles.
- **Why it matters** Callable objects with state are how decorators with memory, Keras layers and PyTorch modules work.

Typical wrong version

```
class P:
    def __init__(self, x): self.x = x
    def __eq__(self, o): return self.x == o.x
{P(1)}
```

```
cannot use 'P' as a set element (unhashable type: 'P')
```

Defining `__eq__` removed the default hash. Fix: add `def __hash__(self): return hash(self.x)`. Also note that a class with no `__eq__` compares by identity, so `Q(1) == Q(1)` is `False` for a plain class.

Connection to the real job. pandas, NumPy and PyTorch are full of dunders (`df[col]` is `__getitem__`, `a + b` on arrays is `__add__`). `__enter__`/`__exit__` power `with open(...)` and database transactions (full treatment in Chapter 7).

Applications

- **Data Analytics:** custom types like `Money` or `Percent` that sort and add correctly.
- **Machine Learning:** `Dataset` classes with `__len__` and `__getitem__` (the exact contract PyTorch `Dataset` expects).
- **Data Engineering:** hashable record keys for deduplication in sets.
- **MLOps:** callable objects as pipeline steps.

COMMON MISTAKE	Writing `__eq__` without `__hash__`, or writing `__str__` and skipping `__repr__`. Always write `__repr__` first. It is what you see in logs and the debugger.

Check yourself

1. Which dunder does `print(obj)` call first?

Answer: `__str__`. If missing, it falls back to `__repr__`.

1. Why can an object with only `__eq__` not go in a set?

Answer: Defining `__eq__` sets `__hash__` to `None`, so it is unhashable.

1. Which method makes `obj(5)` legal?

Answer: `__call__`.

8.3 Inheritance, `super()`, MRO and Mixins

INTUITION	A child inherits eye colour from parents but may have its own habits. In code, a **subclass** inherits all methods of its **parent class** (also called base or superclass) and can add new ones or **override** (replace) old ones. It saves you from copying code.

How it works. `class Dog(Animal):` means Dog is-an Animal. `super()` gives access to the parent's version of a method. **MRO** (Method Resolution Order) is the list Python searches, in order, to find a method. For multiple inheritance Python uses the C3 rule: children before parents, and the order you wrote the parents in is respected.

Problem 8.3.1 Single inheritance with an override.

```
class Animal:
    def __init__(self, name): self.name = name
    def speak(self): return "..."
    def intro(self): return f"{self.name} says {self.speak()}"
class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name)
        self.breed = breed
    def speak(self): return "Woof"
d = Dog("Rex", "Lab")
print(d.intro(), isinstance(d, Animal), issubclass(Dog, Animal))
```

```
Rex says Woof True True
```

- **Answer** `intro` is inherited from `Animal`, but it calls `self.speak()`, which finds `Dog.speak`. So Python picks the most specific version. This is called **polymorphism**: same call, different behaviour per class.
- **Check** `isinstance(d, Animal)` is `True`, since a Dog is an Animal.
- **Why it matters** Frameworks call your overridden methods (`fit`, `run`, `handle`) without knowing your class.

Problem 8.3.2 The diamond: what order does `super()` follow?

Predict the output for `D().who()` where `D(B, C)` and both `B` and `C` extend `A`.

```
class A:
    def who(self): return "A"
class B(A):
    def who(self): return "B>" + super().who()
class C(A):
    def who(self): return "C>" + super().who()
class D(B, C):
    def who(self): return "D>" + super().who()
print(D().who())
print([k.__name__ for k in D.__mro__])
```

```
D>B>C>A
['D', 'B', 'C', 'A', 'object']
```

- **Answer** `super()` does not mean "my parent". It means "the next class in the MRO". So `B`'s `super()` goes to `C`, not straight to `A`. `A` is called once, last.
- **Check** The printed call chain `D>B>C>A` equals the MRO list.
- **Why it matters** This is a classic interview question. The answer: use `ClassName.__mro__` to see the order.

Mixins. A **mixin** is a small class that adds one ability and is not meant to stand alone.

```
class JsonMixin:
    def to_json(self):
        import json
        return json.dumps(self.__dict__)
class LogMixin:
    def log(self, msg): print(f"[{type(self).__name__}] {msg}")
class User(JsonMixin, LogMixin):
    def __init__(self, n, a): self.name=n; self.age=a
u = User("Asha", 30)
u.log(u.to_json())
try:
    class X(A, B): pass
except TypeError as e: print("TypeError:", e)
```

```
[User] {"name": "Asha", "age": 30}
TypeError: Cannot create a consistent method resolution order (MRO) for bases A, B
```

The second case fails because `A` is a parent of `B`, yet we asked for `A` before `B`. Python cannot satisfy "children before parents" and says so.

The picture

```
     A
    / \
   B   C        MRO of D:  D -> B -> C -> A -> object
    \ /
     D          super() in B calls the next one in this list (C)
```

Connection to the real job. Django, scikit-learn (`BaseEstimator`, `TransformerMixin`) and PyTorch rely on inheritance and mixins. You will subclass them and override one or two methods.

Applications

- **Machine Learning:** custom transformer inherits `BaseEstimator` and `TransformerMixin`.
- **Data Engineering:** Airflow operators subclass `BaseOperator`.
- **Load simulation:** tools such as Locust use `HttpUser` subclasses to define tasks.
- **GenAI:** custom tools and callbacks subclass framework base classes.

COMMON MISTAKE	Using inheritance just to reuse code when the relation is not "is-a". A `Car` is not an `Engine`. Use composition (section 8.7).

Check yourself

1. What does `super()` return?

Answer: A proxy that looks up the method in the next class in the MRO.

1. How do you see the MRO?

Answer: `ClassName.__mro__` or `ClassName.mro()`.

1. What is a mixin?

Answer: A small class that adds one ability to other classes through inheritance.

8.4 Abstract Base Classes and Protocols

INTUITION	A plug socket does not care what appliance you connect, only that the plug has the right shape. An **interface** is that shape: a list of methods any object must have. An **abstract base class (ABC)** enforces the shape by inheritance. A **Protocol** enforces it by "if it has the right methods, it fits" (called structural typing, or duck typing with a checker).

How it works. With `abc.ABC` and `@abstractmethod`, you cannot create an object until every abstract method is implemented. This catches mistakes at the moment of creation, not halfway through a pipeline.

Problem 8.4.1 Define an `Exporter` interface and see it refuse to be incomplete.

```
from abc import ABC, abstractmethod

class Exporter(ABC):
    @abstractmethod
    def export(self, rows): ...
    def run(self, rows):
        return f"{len(rows)} rows -> {self.export(rows)}"
class CsvExporter(Exporter):
    def export(self, rows): return "\n".join(",".join(map(str,r)) for r in rows)
try:
    Exporter()
except TypeError as e: print("TypeError:", e)
class Half(Exporter): pass
try: Half()
except TypeError as e: print("TypeError:", e)
print(CsvExporter().run([(1,2),(3,4)]))
```

```
TypeError: Can't instantiate abstract class Exporter without an implementation for abstract method 'export'
TypeError: Can't instantiate abstract class Half without an implementation for abstract method 'export'
2 rows -> 1,2
3,4
```

- **Answer** You cannot build the abstract class or a half-finished subclass. `CsvExporter` works, and it reuses `run` from the base class (this is the "template method" idea).
- **Check** The error message names the missing method.
- **Why it matters** Teams use ABCs so that every connector (CSV, Parquet, database) offers the same methods.

Problem 8.4.2 A Protocol needs no inheritance.

```
from typing import Protocol, runtime_checkable

class Closeable(Protocol):
    def close(self) -> None: ...
class FileLike:
    def close(self): print("closed file-like")
def shutdown(x: Closeable): x.close()
shutdown(FileLike())    # no inheritance needed

@runtime_checkable
class Sized2(Protocol):
    def __len__(self) -> int: ...
print(isinstance([1], Sized2), isinstance(5, Sized2))
```

```
closed file-like
True False
```

- **Answer** `FileLike` never mentions `Closeable`, yet it fits because it has `close()`. A type checker such as mypy (Chapter 9) verifies the fit. With `@runtime_checkable`, `isinstance` can check that the methods exist: a list has `__len__`, an int does not.
- **Check** `isinstance(5, Sized2)` is `False`, as `int` has no `__len__`.
- **Why it matters** Protocols let you accept "anything that looks like a file" without forcing everyone to inherit from your class.

Tool	Enforced when	Needs inheritance	Use when
ABC	At object creation	Yes	You own all the classes and want shared code
Protocol	By type checker (mypy)	No	You accept third-party objects

Connection to the real job. Storage backends (local disk, S3), model loaders and readers all share an interface so the pipeline code never changes.

Applications

- **Data Engineering:** `Source` and `Sink` interfaces for connectors.
- **MLOps:** a `ModelStore` interface with local and cloud versions.
- **Data Science:** a `Metric` protocol: anything with `__call__(y_true, y_pred)`.

COMMON MISTAKE

Forgetting that `Protocol` is checked by tools, not by Python at runtime. Without mypy, a wrong object fails only when the missing method is called.

Check yourself

1. What error appears when you instantiate an incomplete ABC?

Answer: `TypeError: Can't instantiate abstract class ...`.

1. Does a class need to inherit from a Protocol?

Answer: No. Having the right methods is enough.

1. When prefer a Protocol over an ABC?

Answer: When you want to accept objects you do not control.

8.5 Properties, `@classmethod` and `@staticmethod`

INTUITION

A bank clerk checks your form before accepting it. A **property** lets you keep simple attribute syntax (`t.celsius = 30`) while a hidden function validates or computes the value. You can start with a plain attribute and add checks later without changing any calling code.

Three kinds of method

Decorator	First argument	Use
none	`self` (the object)	Normal behaviour
`@classmethod`	`cls` (the class)	Alternative constructors like `from_url`
`@staticmethod`	nothing	A helper that lives in the class for tidiness
`@property`	`self`	Looks like an attribute, runs code

Problem 8.5.1 A temperature with validation and a computed attribute.

```
class Temperature:
    def __init__(self, c): self.celsius = c
    @property
    def celsius(self): return self._c
    @celsius.setter
    def celsius(self, v):
        if v < -273.15: raise ValueError("below absolute zero")
        self._c = v
    @property
    def fahrenheit(self): return self._c*9/5+32
t = Temperature(100)
print(t.celsius, t.fahrenheit)
t.celsius = 0
print(t.fahrenheit)
try: t.celsius = -300
except ValueError as e: print("ValueError:", e)
try: t.fahrenheit = 5
except AttributeError as e: print("AttributeError:", e)
```

```
100 212.0
32.0
ValueError: below absolute zero
AttributeError: property 'fahrenheit' of 'Temperature' object has no setter
```

- **Answer** `100 C = 100*9/5 + 32 = 212 F` and `0 C = 32 F`. Setting an impossible temperature is rejected, and the read-only `fahrenheit` cannot be assigned.
- **Check** 100*9 = 900, 900/5 = 180, 180+32 = 212. Correct.
- **Why it matters** Validation in one place protects every caller.

Note that `__init__` assigns `self.celsius`, so even the first value goes through the setter and is checked.

Problem 8.5.2 An alternative constructor and a helper.

```
class Server:
    def __init__(self, host, port): self.host, self.port = host, port
    @classmethod
    def from_url(cls, url):
        host, port = url.split(":")
        return cls(host, int(port))
    @staticmethod
    def valid_port(p): return 0 < p < 65536
    def __repr__(self): return f"Server({self.host!r}, {self.port})"
print(Server.from_url("db.local:5432"), Server.valid_port(70000))
```

```
Server('db.local', 5432) False
```

- **Answer** `from_url` builds a `Server` from text; it uses `cls`, so subclasses get their own type. `70000` is above the maximum port 65535, so the helper returns `False`.
- **Check** `0 < 70000 < 65536` is false.
- **Why it matters** `pd.DataFrame.from_dict` and `datetime.fromisoformat` are classmethods of the same kind.

Connection to the real job. Config classes use `from_env()` or `from_yaml()` classmethods. Properties compute derived columns (for example `cost_per_request`) without storing them twice.

Applications

- **Data Engineering:** `Config.from_env()` for pipeline settings.
- **Machine Learning:** read-only properties like `n_features_`.
- **Backend:** validated fields on domain objects.

COMMON MISTAKE	Storing into `self.celsius` inside the `celsius` setter. It calls itself forever (`RecursionError`). Store into a different name, like `self._c`.

Check yourself

1. What does `cls` mean in a classmethod?

Answer: The class itself, not an object.

1. How do you make a read-only attribute?

Answer: A property with no setter.

1. Why use a classmethod for `from_url`?

Answer: It is an alternative way to build the object and works with subclasses.

8.6 Dataclasses, NamedTuple, Enum and `__slots__`

INTUITION	Writing `__init__`, `__repr__` and `__eq__` for every small data holder is boring. A **dataclass** writes them for you from a list of fields. A **NamedTuple** is an immutable record with names. An **Enum** is a fixed menu of allowed choices.

Problem 8.6.1 A dataclass with default list, validation and copy.

```
from dataclasses import dataclass, field, asdict, replace

@dataclass
class Request:
    url: str
    latency_ms: float = 0.0
    tags: list = field(default_factory=list)
    def __post_init__(self):
        if self.latency_ms < 0: raise ValueError("negative latency")
r = Request("/login", 120.5)
print(r, r == Request("/login", 120.5))
print(asdict(r), replace(r, latency_ms=99))
try: Request("/x", -1)
except ValueError as e: print("ValueError:", e)
```

```
Request(url='/login', latency_ms=120.5, tags=[]) True
{'url': '/login', 'latency_ms': 120.5, 'tags': []} Request(url='/login', latency_ms=99, tags=[])
ValueError: negative latency
```

- **Answer** You get `__init__`, `__repr__` and `__eq__` for free. `field(default_factory=list)` gives each object its own list (the fix for the bug in 8.1). `__post_init__` runs after the generated `__init__`, so it is the place for validation.
- **Check** The two requests compare equal because all fields match.
- **Why it matters** `dataclass` is the standard way to model records and configs in modern Python.

Problem 8.6.2 Frozen dataclass, NamedTuple and Enum.

```
from dataclasses import dataclass
from typing import NamedTuple
from enum import Enum, auto

@dataclass(frozen=True, order=True)
class Point:
    x: int
    y: int
p = Point(1,2)
try: p.x = 5
except Exception as e: print(type(e).__name__, e)
print(sorted([Point(2,1), Point(1,9)]), {p: "ok"})

class Pt(NamedTuple):
    x: int
    y: int
q = Pt(3,4)
x, y = q
print(q, q.x, q[1], x+y, q._asdict())

class Status(Enum):
    OK = 200
    NOT_FOUND = 404
    ERR = auto()
print(Status.OK, Status(404), Status["ERR"].value, Status.OK.name)
print([s.name for s in Status])
```

```
FrozenInstanceError cannot assign to field 'x'
[Point(x=1, y=9), Point(x=2, y=1)] {Point(x=1, y=2): 'ok'}
Pt(x=3, y=4) 3 4 7 {'x': 3, 'y': 4}
Status.OK Status.NOT_FOUND 405 OK
['OK', 'NOT_FOUND', 'ERR']
```

- **Answer** A frozen dataclass cannot be changed after creation and therefore is hashable (usable as a dict key). `order=True` sorts by fields in order. A NamedTuple unpacks like a tuple and also has names. `auto()` gave `ERR` the value 405, because it continues counting from the last number (404).
- **Check** `3 + 4 = 7`. `Status(404)` looks up by value, `Status["ERR"]` by name.
- **Why it matters** Enums replace "magic strings" like `"paid"` that cause silent typos.

Choose	When
`dataclass`	Mutable record with methods or defaults
`dataclass(frozen=True)`	Immutable record, dict key
`NamedTuple`	Light immutable record, behaves like a tuple
`Enum`	Fixed set of named choices

`__slots__` **and memory.** Normally each object carries a dictionary of its attributes. `__slots__` says "these are the only attributes", and Python stores them in a compact fixed layout.

```
import sys
class Plain:
    def __init__(self,a,b): self.a=a; self.b=b
class Slim:
    __slots__ = ("a","b")
    def __init__(self,a,b): self.a=a; self.b=b
pl, sl = Plain(1,2), Slim(1,2)
print(sys.getsizeof(pl), sys.getsizeof(pl.__dict__), sys.getsizeof(sl))
try: sl.c = 3
except AttributeError as e: print("AttributeError:", e)
```

```
48 296 48
AttributeError: 'Slim' object has no attribute 'c' and no __dict__ for setting new attributes
```

`sys.getsizeof` measures only the object itself in bytes (not things it points to). Here `Plain` is 48 bytes plus a 296-byte dictionary, while `Slim` is 48 bytes in total. In a test I made 100,000 objects of each and measured with `tracemalloc`: about 12 MB for `Plain` versus 8 MB for `Slim` (numbers vary by machine and Python version). The saving matters when you hold millions of small objects.

Connection to the real job. Config, API responses, events and rows are modelled as dataclasses. Pydantic (Chapter 9) builds on the same idea, adding validation.

Applications

- **Data Engineering:** typed records for events passing between stages.
- **Machine Learning:** `TrainingConfig` dataclass holding learning rate, epochs.
- **GenAI:** message and tool-call records.
- **Backend:** `Enum` for order status, HTTP method.

COMMON MISTAKE	Using a plain `list` as a dataclass default: `tags: list = []`. Python raises `ValueError: mutable default <class 'list'> for field tags is not allowed: use default_factory`. Use `field(default_factory=list)`.

Check yourself

1. Which makes a dataclass immutable?

Answer: `@dataclass(frozen=True)`.

1. What does `__post_init__` do?

Answer: Runs right after the generated `__init__`, for validation.

1. What does `__slots__` save?

Answer: The per-object attribute dictionary, so less memory.

8.7 Composition, the Four Pillars and SOLID

INTUITION	A car **has an** engine; it **is not an** engine. When an object is built from other objects, that is **composition**. You can swap the engine without rebuilding the car. This is why experienced engineers say "prefer composition over inheritance".

The four pillars of OOP

Pillar	Meaning	Example in this chapter
Encapsulation	Hide internal data; expose a clean surface	`_c` hidden behind property `celsius`

Pillar	Meaning	Example in this chapter
Abstraction	Show what, hide how	`Exporter.export`
Inheritance	Reuse and specialise a parent	`Dog(Animal)`
Polymorphism	One call, many behaviours	`speak()`

Problem 8.7.1 Composition makes swapping (and testing) easy.

```
class Engine:
    def start(self): return "engine on"
class Car:
    def __init__(self, engine): self.engine = engine
    def start(self): return "car: " + self.engine.start()
class FakeEngine:
    def start(self): return "fake"
print(Car(Engine()).start(), "|", Car(FakeEngine()).start())
```

```
car: engine on | car: fake
```

- **Answer** `Car` works with any object that has `start()`. In tests we pass a fake. This idea (give an object its parts from outside) is called **dependency injection**.
- **Check** Both outputs begin with `car:` and differ only in the engine.
- **Why it matters** Tests can run without a real database or network.

SOLID is a set of five design rules. Each is easy to state; the examples show them in code.

Letter	Rule	Plain meaning	Broken sign
S	Single responsibility	A class has one reason to change	A class that parses, calculates and emails
O	Open/closed	Add features by adding code, not editing old code	A long `if/elif` on type names
L	Liskov substitution	A child can replace its parent without surprises	Child raises errors parent never did
I	Interface segregation	Small interfaces, not one huge one	Classes forced to implement unused methods
D	Dependency inversion	Depend on abstractions, not concrete classes	`Service` creates `MySQL()` inside itself

Problem 8.7.2 Spot a Liskov violation.

```
class Bird:
    def fly(self): return "flying"
class Penguin(Bird):
    def fly(self): raise NotImplementedError("penguins cannot fly")
def migrate(birds): return [b.fly() for b in birds]
try: migrate([Bird(), Penguin()])
except NotImplementedError as e: print("LSP broken:", e)
```

```
LSP broken: penguins cannot fly
```

- **Answer** `migrate` expects every `Bird` to fly. `Penguin` breaks that promise, so it cannot safely replace `Bird`. Fix: split the hierarchy (`Bird` and `FlyingBird`).
- **Check** The failure happens inside code that only knows about `Bird`.
- **Why it matters** This kind of silent contract break causes production bugs in pipeline plug-ins.

The open/closed and dependency-inversion examples appear in the next section as Strategy and Dependency Injection.

Connection to the real job. In design interviews, "how would you make this extensible and testable?" is answered with these five letters.

Applications

- **Machine Learning:** swap models behind a common `predict` interface.
- **Data Engineering:** swap storage backends without touching the transform code.
- **Testing:** inject fakes.

COMMON MISTAKE	Over-engineering. Do not create three classes and an interface for a ten-line script. Use these rules when code must grow or be tested, not by default.

Check yourself

1. Is a `Car` an `Engine` or does it have one?

Answer: It has one: use composition.

1. What does the "L" in SOLID check?

Answer: Whether a subclass can replace its parent without breaking callers.

1. What is dependency injection in one line?

Answer: Passing an object's parts in from outside instead of building them inside.

8.8 Design Patterns You Must Recognise

INTUITION	A **design pattern** is a named, proven recipe for a problem that keeps coming back, like "roundabout" for traffic design. Names matter because they let two engineers say "use a Factory" and understand each other at once.

All code below ran together; outputs are in the blocks.

Strategy (swap an algorithm) and **Singleton** (one shared instance):

```
from typing import Callable

def no_discount(t): return t
def festival(t): return t*0.9
def bulk(t): return t*0.8 if t > 1000 else t
STRATEGIES: dict[str, Callable[[float], float]] = {"none": no_discount, "festival": festival, "bulk": bulk}
def price(total, rule): return STRATEGIES[rule](total)
print(price(1500, "festival"), price(1500, "bulk"), price(500, "bulk"))

class Config:
    _inst = None
    def __new__(cls):
        if cls._inst is None:
            cls._inst = super().__new__(cls)
            cls._inst.values = {}
        return cls._inst
c1, c2 = Config(), Config()
c1.values["env"] = "prod"
print(c1 is c2, c2.values)
import logging
print(logging.getLogger("a") is logging.getLogger("a"))
```

```
1350.0 1200.0 500
True {'env': 'prod'}
True
```

- **Strategy:** each discount rule is a function in a dictionary. To add a new rule you add one entry. No old code changes (open/closed). `1500*0.9 = 1350`, `1500*0.8 = 1200`, and 500 is below the bulk limit so it stays 500.
- **Singleton:** `Config()` always returns the same object, so changes made through `c1` are visible through `c2`. `logging.getLogger("a")` is Python's own example. Use sparingly: global state makes tests hard.

Factory (build the right object from a name):

```
from abc import ABC, abstractmethod
class Reader(ABC):
    @abstractmethod
    def read(self, text): ...
class CsvReader(Reader):
    def read(self, text): return [l.split(",") for l in text.splitlines()]
class PipeReader(Reader):
    def read(self, text): return [l.split("|") for l in text.splitlines()]
def make_reader(kind):
    readers = {"csv": CsvReader, "pipe": PipeReader}
    try: return readers[kind]()
    except KeyError: raise ValueError(f"unknown reader {kind!r}") from None
print(make_reader("pipe").read("a|b\nc|d"))
try: make_reader("xml")
except ValueError as e: print("ValueError:", e)
```

```
[['a', 'b'], ['c', 'd']]
ValueError: unknown reader 'xml'
```

The caller says what it wants (`"pipe"`) and the factory decides which class to create.

Builder (assemble a complex object step by step):

```
class QueryBuilder:
    def __init__(self, table): self.table=table; self._where=[]; self._limit=None; self._params=[]
    def where(self, cond, *params): self._where.append(cond); self._params += params; return self
    def limit(self, n): self._limit = n; return self
    def build(self):
        sql = f"SELECT * FROM {self.table}"
        if self._where: sql += " WHERE " + " AND ".join(self._where)
        if self._limit is not None: sql += f" LIMIT {int(self._limit)}"
        return sql, tuple(self._params)
print(QueryBuilder("orders").where("status = ?", "paid").where("amount > ?", 500).limit(10).build())
```

```
('SELECT * FROM orders WHERE status = ? AND amount > ? LIMIT 10', ('paid', 500))
```

Each method returns `self`, so calls chain. Notice that user values travel as separate parameters (the `?`), never pasted into the SQL text. This protects against SQL injection (Chapter 13). Table names cannot be parameters, so they must come from trusted code only.

Observer (publish and subscribe), **Decorator** (wrap an object to add behaviour) and **Adapter** (make an old interface fit a new one):

```
class EventBus:
    def __init__(self): self._subs = {}
    def subscribe(self, event, fn): self._subs.setdefault(event, []).append(fn)
    def publish(self, event, data):
        for fn in self._subs.get(event, []): fn(data)
bus = EventBus()
bus.subscribe("order", lambda d: print("email sent for", d))
bus.subscribe("order", lambda d: print("stock reduced by", d["qty"]))
bus.publish("order", {"id": 7, "qty": 2})

class Notifier:
    def send(self, m): return m
class Upper:
    def __init__(self, inner): self.inner = inner
    def send(self, m): return self.inner.send(m.upper())
class Stars:
    def __init__(self, inner): self.inner = inner
    def send(self, m): return "*** " + self.inner.send(m)
print(Stars(Upper(Notifier())).send("alert"))

class OldLogger:
    def write_line(self, s): print("OLD:", s)
class LoggerAdapter:
    def __init__(self, old): self.old = old
    def info(self, msg): self.old.write_line(f"INFO {msg}")
LoggerAdapter(OldLogger()).info("started")
```

```
email sent for {'id': 7, 'qty': 2}
stock reduced by 2
*** ALERT
OLD: INFO started
```

- **Observer:** the publisher does not know who listens. Add a third subscriber without changing `publish`.
- **Decorator:** objects wrap objects like layers: `Stars` wraps `Upper` wraps `Notifier`, so "alert" becomes "ALERT" and then gets stars. (Not the same as Python's `@decorator` syntax in Chapter 7, but the same wrapping spirit.)
- **Adapter:** `OldLogger` has `write_line`; our code wants `info`. The adapter translates.

Repository (hide the database behind an interface) with **Dependency Injection**:

```
import sqlite3
from dataclasses import dataclass
from typing import Protocol

@dataclass
class User:
    id: int
    name: str

class UserRepo(Protocol):
    def add(self, name: str) -> User: ...
    def get(self, id: int) -> User | None: ...

class SqliteUserRepo:
    def __init__(self, conn):
        self.conn = conn
        conn.execute("CREATE TABLE IF NOT EXISTS users(id INTEGER PRIMARY KEY, name TEXT)")
    def add(self, name):
        cur = self.conn.execute("INSERT INTO users(name) VALUES (?)", (name,))
        return User(cur.lastrowid, name)
    def get(self, id):
        row = self.conn.execute("SELECT id, name FROM users WHERE id = ?", (id,)).fetchone()
        return User(*row) if row else None

class InMemoryUserRepo:
    def __init__(self): self.d = {}
    def add(self, name):
        u = User(len(self.d)+1, name); self.d[u.id] = u; return u
    def get(self, id): return self.d.get(id)

class SignupService:
    def __init__(self, repo: UserRepo):
        self.repo = repo
    def signup(self, name):
        if not name.strip(): raise ValueError("empty name")
        return self.repo.add(name.strip())

for repo in (InMemoryUserRepo(), SqliteUserRepo(sqlite3.connect(":memory:"))):
    svc = SignupService(repo)
    u = svc.signup("  Meera ")
    print(type(repo).__name__, u, repo.get(u.id), repo.get(99))
```

```
InMemoryUserRepo User(id=1, name='Meera') User(id=1, name='Meera') None
SqliteUserRepo User(id=1, name='Meera') User(id=1, name='Meera') None
```

`SignupService` holds the business rule (strip the name, reject empty). It never mentions SQL. We **inject** the repository from outside, so tests use the in-memory one and production uses SQLite (or Postgres). Both give identical results here.

Pattern map

Pattern	Problem it solves	Python shortcut
Singleton	One shared instance	A module-level object
Factory	Create right class by name	Dict of classes
Builder	Many optional steps	Method chaining, or dataclass defaults
Strategy	Swap algorithm	Pass a function
Observer	Notify many listeners	List of callbacks

Pattern	Problem it solves	Python shortcut
Decorator	Add behaviour by wrapping	Wrapper class or `@decorator`
Adapter	Make interfaces fit	Small wrapper class
Repository	Hide data storage	Class with `add/get`
Dependency Injection	Supply parts from outside	Constructor argument

Connection to the real job. Airflow, scikit-learn pipelines, LangChain callbacks and FastAPI dependencies are these patterns. In system design interviews, Repository plus DI is the standard answer to "how do you keep the database out of business logic".

Applications

- **Data Engineering:** Factory for readers, Repository for warehouse access, Observer for pipeline alerts.
- **MLOps:** Strategy for model choice, Adapter for legacy systems.
- **GenAI:** Strategy for swapping model providers, Observer for streaming callbacks.
- **Backend:** Dependency Injection in FastAPI (`Depends`).

COMMON MISTAKE	Making everything a Singleton. It is a global variable in disguise and makes tests depend on each other. Prefer injecting one shared object.

Check yourself

1. Which pattern lets you swap a discount rule by passing a function?

Answer: Strategy.

1. Why does `SignupService` not call `sqlite3` directly?

Answer: So the storage can be swapped and tested separately (Repository plus DI).

1. What does Adapter do?

Answer: Translates one interface into the one your code expects.

8.9 Under the Hood: Descriptors, Subclass Hooks, Hashing and Caching

Every feature in this chapter, from `@property` to `super()` , is built on a small set of rules in Python's data model. Learning those rules lets you build validation, plugin systems and caches that feel native.

Descriptors: how property really works

A **descriptor** is an object stored on a class that defines `__get__` , and optionally `__set__` . When you read or assign `obj.attr` and `attr` is a descriptor on the class, Python calls those methods instead of touching the instance dictionary. `property` , methods and `classmethod` are all descriptors. Writing your own lets you reuse one validation rule on many fields, which a property cannot do without copy and paste.

Problem 8.9.1 Write a reusable `Positive` descriptor and use it for both `price` and `qty` of an `Item` .

Step 1	`__set_name__(owner, name)` is called once when the class is built; store a private name such as `_price` .
Step 2	`__set__` checks the value and stores it in the instance dict under the private name.
Step 3	`__get__` reads it back. When `obj is None` the access came from the class, so return the descriptor itself.
Check	`Item(2.5, 4)` works and total is 10.0. `Item(-1, 3)` raises `ValueError` .

```
class Positive:
    def __set_name__(self, owner, name):
        self.name = "_" + name
    def __get__(self, obj, objtype=None):
        if obj is None:
            return self
        return getattr(obj, self.name)
    def __set__(self, obj, value):
        if value <= 0:
            raise ValueError(f"{self.name[1:]} must be positive, got {value}")
        setattr(obj, self.name, value)
class Item:
    price = Positive()
    qty = Positive()
    def __init__(self, price, qty):
        self.price = price
        self.qty = qty
it = Item(2.5, 4)
print(it.price * it.qty, it.__dict__)
try:
    Item(-1, 3)
except ValueError as e:
    print(e)
print(type(Item.price).__name__)
```

```
10.0 {'_price': 2.5, '_qty': 4}
price must be positive, got -1
Positive
```

COMMON MISTAKE

Storing the value on the descriptor itself (`self.value = value`). A descriptor lives on the class, so every instance would share one value. Wrong: two items overwrite each other's price. Right: store per-instance data in the instance, as the code above does with `setattr(obj, self.name, value)` .

Hooks that run when a subclass is created

`__init_subclass__` is a class method that Python calls on the parent every time someone subclasses it. It is the simple way to build a plugin registry, with no metaclass needed. Extra keyword arguments in the `class` line are passed to it.

Problem 8.9.2 Build an `Exporter` base class that registers each subclass under a format name given in the class line.

Step 1	Define `__init_subclass__(cls, fmt=None, **kw)` on the base.
Step 2	Call `super().__init_subclass__(**kw)` so cooperating parents still work.
Step 3	If `fmt` is given, store `cls` in a registry dict.
Check	The registry lists both formats and `registry["json"]()` builds the right object.

```
class Exporter:
    registry = {}
    def __init_subclass__(cls, fmt=None, **kw):
        super().__init_subclass__(**kw)
        if fmt:
            Exporter.registry[fmt] = cls
class CsvExporter(Exporter, fmt="csv"):
    def run(self): return "a,b"
class JsonExporter(Exporter, fmt="json"):
    def run(self): return '{"a": 1}'
print(sorted(Exporter.registry))
print(Exporter.registry["json"]().run())
```

```
['csv', 'json']
{"a": 1}
```

Equality and hashing must agree

Sets and dict keys rely on a contract: objects that are equal must have equal hashes. If you define `__eq__` and nothing else, Python sets `__hash__` to `None` and your class becomes unhashable. If you define both, hash only fields that do not change.

```
class Bad:
    def __init__(self, k): self.k = k
    def __eq__(self, o): return self.k == o.k
try:
    {Bad(1)}
except TypeError as e:
    print(e)
class Good:
    def __init__(self, k): self.k = k
    def __eq__(self, o): return isinstance(o, Good) and self.k == o.k
    def __hash__(self): return hash(self.k)
print(len({Good(1), Good(1), Good(2)}))
class Mutable:
    def __init__(self, k): self.k = k
    def __eq__(self, o): return self.k == o.k
    def __hash__(self): return hash(self.k)
m = Mutable(1); s = {m}
m.k = 2
print(m in s, Mutable(1) in s, len(s))
```

```
cannot use 'Bad' as a set element (unhashable type: 'Bad')
2
False False 1
```

The last line is the dangerous case: after `m.k = 2`, the object sits in the set in the slot for hash 1, so neither `m` (hash now 2) nor `Mutable(1)` (whose hash matches, but it no longer equals the stored object) is found. The set is silently broken.

COMMON MISTAKE

Hashing a mutable field. Wrong: `__hash__` uses `self.k` and you later change `k`, so membership tests fail. Right: hash only immutable fields, or make the class frozen (`@dataclass(frozen=True)`) so it cannot change.

Computing once with cached_property

`functools.cached_property` runs the method on first access and stores the result in the instance dict, so later reads are plain attribute lookups. It is a non-data descriptor, which is why the stored value wins afterwards. Delete the attribute to force recomputation.

```
from functools import cached_property
class Report:
    def __init__(self, rows): self.rows = rows
    @cached_property
    def total(self):
        print("computing")
        return sum(self.rows)
r = Report([1, 2, 3])
print(r.total, r.total)
del r.total
r.rows.append(4)
print(r.total)
```

```
computing
6 6
computing
10
```

Notice the stale-value risk: after `rows.append(4)` the cached total would still say 6 until it was deleted. Cache only values that cannot change, or invalidate on purpose.

Dataclasses beyond the basics

Three tools solve most real problems. `field(default_factory=list)` gives each instance its own list, and Python refuses a plain `[]` default. `__post_init__` validates after the generated `__init__`. `frozen=True` blocks assignment and `slots=True` removes the per-instance dict (Python 3.10 and later). Inside a frozen class use `object.__setattr__` to set a derived field in `__post_init__`.

Problem 8.9.3 Make an immutable `Order` with a list of items, an id that must be positive, and a derived `total` equal to the number of items.

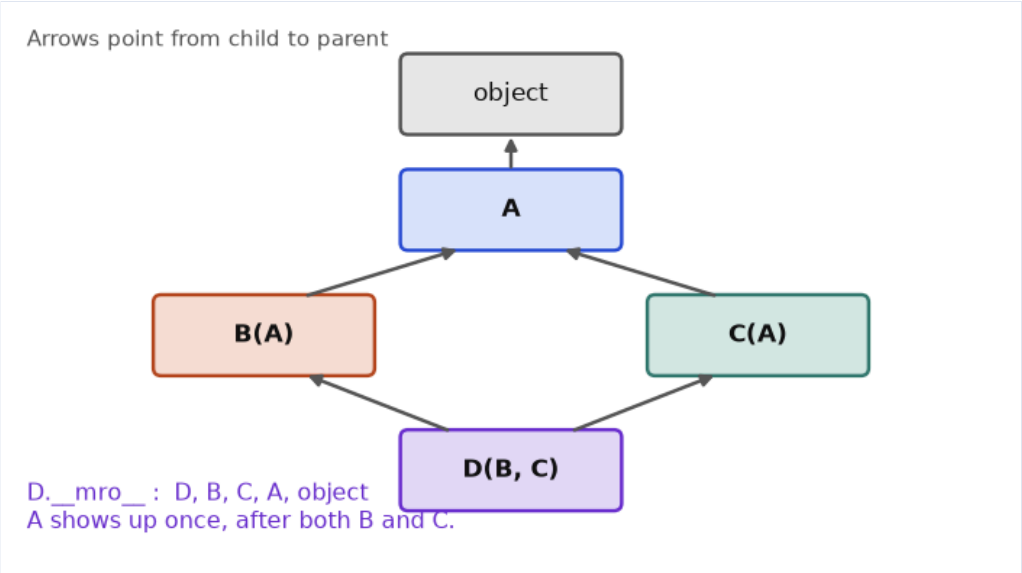
Step 1	`items` uses `default_factory=list`.
Step 2	`total` uses `init=False`, so callers cannot pass it.
Step 3	`__post_init__` checks the id and sets `total` with `object.__setattr__`.
Check	Assigning `o.id` raises `FrozenInstanceError`; `replace` makes a modified copy; no `__dict__` exists.

```
from dataclasses import dataclass, field, replace, fields
import sys
@dataclass(frozen=True, slots=True)
class Order:
    id: int
    items: list = field(default_factory=list)
    total: float = field(init=False, default=0.0)
    def __post_init__(self):
        if self.id <= 0:
            raise ValueError("id must be positive")
        object.__setattr__(self, "total", float(len(self.items)))
o = Order(1, ["a", "b"])
print(o)
try:
    o.id = 5
except Exception as e:
    print(type(e).__name__)
print(replace(o, id=2).id, hasattr(o, "__dict__"))
@dataclass
class Plain:
    id: int
try:
    @dataclass
    class Oops:
        xs: list = []
except ValueError as e:
    print(e)
print([f.name for f in fields(Order)])
```

```
Order(id=1, items=['a', 'b'], total=2.0)
FrozenInstanceError
2 False
mutable default <class 'list'> for field xs is not allowed: use default_factory
['id', 'items', 'total']
```

Going deeper

- Attribute lookup order: data descriptors on the class, then the instance dict, then non-data descriptors and class attributes. That is why a `property` (data descriptor) beats an instance value, while `cached_property` (non-data) is overridden by the value it stores.
- A metaclass is the class of a class. Use one only if `__init_subclass__`, a class decorator or a descriptor cannot do the job; those three cover almost every real need.
- `__getattr__` runs only when normal lookup fails. `__getattribute__` runs on every access and is easy to turn into infinite recursion, so avoid it unless you must.
- Cooperative `super()` follows the MRO (section 8.3), not "the parent". The figure below shows a diamond, and the code after it proves each class runs once.



The diamond. The linearised order D, B, C, A, object puts A once, after both of its children.

```
class A:
    def hi(self): return "A"
class B(A):
    def hi(self): return "B>" + super().hi()
class C(A):
    def hi(self): return "C>" + super().hi()
class D(B, C):
    def hi(self): return "D>" + super().hi()
print(D().hi())
print([k.__name__ for k in D.__mro__])
```

```
D>B>C>A
['D', 'B', 'C', 'A', 'object']
```

More practice: build and stretch

Problem 8.P.11 (what is the output?) What do the two lines print for the diamond above, where each `hi` calls `super().hi()`?

Answer: `D>B>C>A` and `['D', 'B', 'C', 'A', 'object']`. Note that `B`'s `super()` goes to `C`, not to `A`, because `super()` follows the MRO of the instance's class.

Problem 8.P.12 (find the bug) A class defines `__eq__` comparing `k` and you put an instance in a set. What error appears and what is the minimal fix?

Answer: `TypeError: cannot use 'Bad' as a set element (unhashable type: 'Bad')`. Add `def __hash__(self): return hash(self.k)` and make `k` unchangeable (or freeze the class). With that, `{Good(1), Good(1), Good(2)}` has 2 elements.

Problem 8.P.13 (build) Write a `Range` descriptor that takes `low` and `high` in its constructor and rejects values outside them. Use it for `age = Range(0, 130)`.

Answer: Copy `Positive` from 8.9.1, add `def __init__(self, low, high)` storing both, and in `__set__` raise `ValueError` when `not self.low <= value <= self.high`. Self-check: does a second instance keep its own age (stored in the instance, not on the descriptor)?

Problem 8.P.14 (stretch) Why does `@dataclass class Oops: xs: list = []` fail, and what is the sibling bug with function defaults?

Answer: Python raises `ValueError: mutable default <class 'list'> for field xs is not allowed: use default_factory`, because all instances would share one list. The function-default twin is `def f(x=[])` from Chapter 2. Fix with `field(default_factory=list)`.

Problem 8.P.15 (interview) When would you choose `__init_subclass__` over a metaclass?

Answer: Almost always. It runs on subclass creation, needs no extra concept, composes with other bases, and covers registries and validation. Choose a metaclass only to control how the class object itself is created or its namespace, which is rare.

Chapter recap

- A class is a template; an instance is one real object. `self` is that object.
- Put per-object data in `__init__`. Never share mutable data through class attributes or default arguments.
- Dunder methods let your objects work with `print`, `len`, `+`, `<`, `for`, `with` and `()`.
- If you define `__eq__`, also define `__hash__` or the object is unhashable.
- `super()` follows the MRO, not "the parent". Check `Class.__mro__`.
- ABCs enforce interfaces at creation time; Protocols match by shape and are checked by tools.
- Use `dataclass` for records, `NamedTuple` for light immutable records, `Enum` for fixed choices, `__slots__` for memory.
- Prefer composition and dependency injection; SOLID guides design; patterns are shared names for recipes.

Pattern	Syntax
Record	`@dataclass(frozen=True, order=True)`
Safe list default	`field(default_factory=list)`
Read-only attribute	`@property` without setter
Other constructor	`@classmethod def from_x(cls, ...)`
Interface	`class X(ABC)` + `@abstractmethod`
See MRO	`Class.__mro__`
Object size	`sys.getsizeof(obj)`

Practice set

Problem 8.9.1 (easy) What prints?

```
class C:
    n = 0
c1, c2 = C(), C()
c1.n = 5
print(c2.n)
```

Answer: `0`. The assignment created an instance attribute on `c1` only.

Problem 8.9.2 (easy) Find the bug: `class A:` with `def hit(): return 1`, called as `A().hit()`.

Answer: The method is missing `self`. Python raises `A.hit() takes 0 positional arguments but 1 was given`. Fix: `def hit(self):`.

Problem 8.9.3 (easy) For `class Q:` with only `__init__(self, x)` storing `x`, what does `Q(1) == Q(1)` give?

Answer: `False`. Without `__eq__`, equality means "same object", and these are two objects.

Problem 8.9.4 (medium) Give the MRO of `D(B, C)` where `B(A)` and `C(A)`.

Answer: `D, B, C, A, object`.

Problem 8.9.5 (medium) What happens when you run `p.x = 5` on an instance of `@dataclass(frozen=True) class Point`?

Answer: `FrozenInstanceError: cannot assign to field 'x'`.

Problem 8.9.6 (medium) An `Enum` has `OK = 200`, `NOT_FOUND = 404`, `ERR = auto()`. What is `ERR.value`?

Answer: `405`. `auto()` continues from the last integer value.

Problem 8.9.7 (medium) Find the bug and fix it.

```
class Cfg:
    def __init__(self, d={}): self.d = d
a, b = Cfg(), Cfg()
a.d["k"] = 1
print(b.d)
```

Answer: It prints `{'k': 1}`. The default dictionary is created once and shared by all objects. Fix: `def __init__(self, d=None): self.d = {} if d is None else d`.

Problem 8.9.8 (medium) What does `sl.c = 3` do when `sl` is an object of a class with `__slots__ = ("a","b")`?

Answer: It raises `AttributeError: 'Slim' object has no attribute 'c' and no __dict__ for setting new attributes`.

Problem 8.9.9 (interview) What is the difference between `__repr__` and `__str__`?

Answer: `__repr__` is for developers: unambiguous, ideally looks like code that recreates the object. It is used in lists, logs and the debugger. `__str__` is for users and is used by `print`. If only `__repr__` exists, `print` uses it too.

Problem 8.9.10 (interview) Why do experts say "prefer composition over inheritance"? Give an example.

Answer: Inheritance ties the child to the parent's every detail and to the "is-a" rule, so changes in the parent break children (and can break Liskov substitution, like `Penguin(Bird)`). Composition builds an object from parts that can be swapped. Example: `Car` takes an `engine` object, so tests pass a fake engine.

Problem 8.9.11 (scenario) Your `SignupService` creates `MySQL()` inside `__init__` and the unit tests are slow and flaky. What do you change?

Answer: Define a repository interface (`add`, `get`), pass the repository into the constructor (dependency injection), and use an in-memory fake in tests. This is the Repository pattern plus the "D" in SOLID.

Prompt pack

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data roles, and English is my second language, so use simple words. Teach me Python classes and instances: class vs instance attributes, self, __init__, and the mutable-default bug. Explain step by step with one small shop example, then ask me 5 short questions one at a time and wait for my answer before continuing. Correct me kindly and explain why.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data roles, and English is my second language. Explain Python dunder methods (__repr__, __str__, __eq__, __hash__, __lt__, __len__, __getitem__, __iter__, __call__, __enter__) using a Money class. Show runnable code with real output for each method. Then give me 8 graded exercises, from easy to hard, without solutions, and offer to review my answers.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data roles, and English is my second language. Teach me inheritance, super(), the MRO, mixins and the diamond problem in Python. Use simple analogies, then run
```

a small example and show the output. Quiz me one question at a time on "what is the output?", and explain every wrong answer line by line.

✨ **AI prompt: copy and paste into any AI**

You are my patient, practical professor. I am a working professional moving into data roles, and English is my second language. Explain SOLID and composition over inheritance with Python examples that I can run. For each of the five principles, show a bad version and a good version, and explain what breaks in the bad one. Then review my own class design: [PASTE HERE]. Tell me which principle it violates and how to fix it.

✨ **AI prompt: copy and paste into any AI**

You are my patient, practical professor. I am a working professional moving into data roles, and English is my second language. Run a mock interview on Python OOP and design patterns (Singleton, Factory, Builder, Strategy, Observer, Decorator, Adapter, Repository, Dependency Injection). Ask one question at a time, from easy to hard, wait for my answer, grade it out of 10, give the ideal answer, and track my score. Finish with the three topics I should revise.

CHAPTER 9

MODULES, PACKAGES, ENVIRONMENTS, TYPE HINTS AND VALIDATION

How to organise code into files, keep each project's libraries separate, and let tools catch mistakes before you run anything.

A notebook or a single script is fine for an experiment. A real job gives you a repository with fifty files, ten libraries, three teammates and a deadline. Then new questions appear. How do I split code into files and import between them? Why does my laptop work but the server break? Why did my pipeline crash because someone passed a string instead of a number? This chapter answers these questions. By the end you will build a real package and run it, create an isolated environment, read a `pyproject.toml`, write type hints, and validate incoming data. Everything marked "run" was executed with Python 3.14. No network was available here, so tools that need downloads (pip installs, mypy, pydantic, uv, poetry, conda) are shown in standard form and marked **not run here**.

9.1 Modules, Packages and `__main__`

INTUITION A module is a chapter, a package is the whole book, and an import is a bookmark to a chapter in another book. You split code into files so each file does one job and you can find things. Without modules you would scroll through a 5,000-line file.

Words you need

Word	Plain meaning
module	One `.py` file. `import utils` loads `utils.py`
package	A folder of modules (it has an `__init__.py`)
`__init__.py`	Runs when the package is first imported; defines what the package exposes
`__name__`	Variable holding the module's name. It is `"__main__"` when the file is run directly
absolute import	`from shop.utils import round2` (full path)
relative import	`from .utils import round2` (dot means "this package")
`sys.modules`	Python's cache of already imported modules

How an import works. Python looks in `sys.modules` first. If the module is not there, it searches folders (the script's folder, then installed libraries), runs the file top to bottom **once**, and stores the result in the cache. Any later `import` reuses it.

Problem 9.1.1 Build a tiny package called `shop` and run it.

Step 1	Make a folder `shop/` containing `__init__.py`, `pricing.py`, `utils.py` and `__main__.py`.
Step 2	`pricing.py` imports from `utils.py` with a relative import. `__init__.py` re-exports `total_with_tax`.
Step 3	Run with `python3 -m shop 250`.

The files (each with a `print` so you can see when it runs):

```
# shop/__init__.py
"""shop package."""
from .pricing import total_with_tax
__all__ = ["total_with_tax"]
__version__ = "0.1.0"
print("shop/__init__ ran")
```

```
# shop/pricing.py
from .utils import round2
TAX = 0.18
def total_with_tax(amount):
    return round2(amount * (1 + TAX))
print("pricing imported, __name__ =", __name__)
```

```
# shop/utils.py
def round2(x):
    return round(x, 2)
```

```
# shop/__main__.py
from shop import total_with_tax
import sys
def main():
    amount = float(sys.argv[1]) if len(sys.argv) > 1 else 100.0
    print("total:", total_with_tax(amount))
if __name__ == "__main__":
    main()
```

Before you run it, predict: which line prints first, `shop/__init__ ran` or `pricing imported`?

```
python3 -m shop 250
python3 -c "import shop; print(shop.__version__, shop.total_with_tax(1000))"
```

```
pricing imported, __name__ = shop.pricing
shop/__init__ ran
total: 295.0
pricing imported, __name__ = shop.pricing
shop/__init__ ran
0.1.0 1180.0
```

Line by line: `__init__.py` begins running, hits `from .pricing import ...`, and jumps into `pricing.py` first. So "pricing imported" appears before "shop/__init__ ran", which is the last line of `__init__`. Inside pricing, `__name__` is `shop.pricing` (not `__main__`), since it was imported. `python3 -m shop` runs `__main__.py`. `250 * 1.18 = 295.0` and `1000 * 1.18 = 1180.0`.

- **Answer** `pricing imported` prints first, because the package's `__init__` has to import `pricing` before it can reach its own last line.
- **Check** 250 x 1.18 = 295.0. Correct. The import message does not repeat within one run, which shows the cache at work.
- **Why it matters** `python -m package` is the clean way to run a package. Entry points like `uvicorn app.main:app` rely on the same import rules.

Why `if __name__ == "__main__":`? When the file is imported, `__name__` is the module name and `main()` does not run. When the file is run directly, `__name__` is `"__main__"` and `main()` runs. This lets one file be both a library and a script. Without the guard, importing the file would start the whole program.

Problem 9.1.2 The most common relative-import error. Run a module file directly instead of as part of a package:

```
python3 shop/pricing.py
```

```
    from .utils import round2
ImportError: attempted relative import with no known parent package
```

Running a file directly makes Python treat it as a top-level script, with no package around it, so the dot has nothing to point to.

- **Answer** Run it as `python3 -m shop.pricing` from the project root, or run the package with `-m shop`.
- **Check** `-m` sets `__package__`, so the relative import has a parent.
- **Why it matters** Every new Python engineer hits this error in week one.

The picture

```
project/
  shop/                 <- package (folder with __init__.py)
    __init__.py         <- runs on "import shop"
    __main__.py         <- runs on "python -m shop"
    pricing.py  --uses--> utils.py   (from .utils import round2)
```

Read it as: you run from `project/`, `import shop` finds the folder, runs `__init__.py`, and from there Python can reach sibling files.

Connection to real projects. A data repository usually has `src/yourproject/` with `ingest.py`, `transform.py`, `models.py`. Airflow DAGs, FastAPI apps and CLI tools are all packages. `__all__` and `__init__` decide the public surface other teams depend on.

Applications

- **Data Engineering:** one package of reusable transforms shared by many pipelines.
- **Machine Learning:** `features.py`, `train.py`, `evaluate.py` imported by notebooks and jobs.
- **MLOps:** `python -m mypackage.serve` as the container start command.
- **Backend:** `app/` package with `routers/`, `models/`, `services/`.

COMMON MISTAKE	Naming your file after a library, for example `random.py` or `json.py`. Python then imports your file instead of the standard one and strange errors follow. Never name your files after standard or installed libraries.

Check yourself

1. What does `__name__` equal when a file is run with `python file.py`?

Answer: `"__main__"`.

1. Why does the import message print only once even if three files import the same module?

Answer: Python caches the module in `sys.modules` after the first import.

1. What does `from .utils import x` need?

Answer: The file must be part of a package that was imported or run with `-m`.

9.2 Circular Imports and How to Avoid Them

INTUITION	Two people each say "I will leave only after you leave first." Nobody leaves. A **circular import** is the same deadlock: `a.py` needs `b.py` and `b.py` needs `a.py`, and Python is stuck halfway through loading one of them.

Problem 9.2.1 Create the problem. `a.py` imports from `b.py`, and `b.py` imports from `a.py`.

```
# a.py
from b import fb
def fa(): return "fa"
```

```
# b.py
from a import fa
def fb(): return "fb"
```

Predict: what happens on `import a`?

```
python3 -c "import a"
```

```
  File "/private/tmp/c9/circ/b.py", line 1, in <module>
    from a import fa
ImportError: cannot import name 'fa' from 'a' (consider renaming '/private/tmp/c9/circ/a.py' if it has the same name as a
library you intended to import)
```

Step by step: Python starts `a`, reaches line 1, and starts loading `b`. `b` asks for `fa` from `a`, but `a` is only half loaded and `fa` is defined on line 2, which has not run yet. So the name does not exist, and Python raises `ImportError`.

- **Answer** An `ImportError` because `a` is still only partly loaded when `b` asks for `fa`.
- **Check** The traceback points at line 1 of `b.py`.
- **Why it matters** In real projects the cycle is often three or four files long, which makes it hard to see.

Fixes, best first

1. **Move shared code** into a third module that both import (the usual best fix).
2. **Import inside the function** that needs it (a deferred import).
3. **Import the module, not the name** (`import a` and use `a.fa` later).

Here is fix 2 in action:

```
# b.py
def fb():
    from a import fa      # deferred import
    return "fb+" + fa()
```

```
python3 -c "import a; print(a.fb())"
```

```
fb+fa
```

By the time `fb()` is called, both modules are fully loaded, so the import succeeds.

- **Answer** It prints `fb+fa`.
- **Check** `fb()` returns "fb+" followed by what `fa()` returns, "fa".
- **Why it matters** A deferred import is a quick patch. A cycle often means the design should change: one module is doing two jobs (see Single Responsibility in Chapter 8).

Connection to real projects. Cycles appear when `models.py` needs `services.py` for a helper while `services.py` imports `models.py`. The cure is a layered design: `models` imports nothing from `services`; `services` may import `models`.

Applications

- **Backend:** layered packages with one-way dependencies.
- **Machine Learning:** keep `config.py` free of imports from training code.
- **Type hints:** `if TYPE_CHECKING:` imports avoid cycles that only exist for annotations.

COMMON MISTAKE	"Fixing" a cycle by shuffling import lines until it happens to work. It will break again after the next edit. Break the cycle by design.

Check yourself

1. Why does a circular import fail?

Answer: One module is only partly loaded when the other asks for a name from it.

1. Name the cleanest fix.

Answer: Move the shared code into a third module.

1. What is a deferred import?

Answer: An import written inside a function, so it runs at call time.

9.3 Virtual Environments and Package Managers

INTUITION	Imagine each project has its own toolbox. Project A needs a hammer version 1; project B needs version 2. If you keep one shared toolbox for the whole house, you cannot have both. A **virtual environment** (venv) is a private toolbox for one project, so libraries and their versions never clash.

Words you need

Word	Plain meaning
package (library)	Code written by others that you install, like `pandas`
`pip`	The standard tool that installs packages from PyPI (the public package index)
virtual environment	A folder with its own Python and its own installed packages
`site-packages`	The folder where installed packages live
dependency	A library your project needs
lock file	A file that records the exact versions installed

Problem 9.3.1 Create a venv and prove it is isolated.

Step 1	`python3 -m venv venvdemo` creates the folder.
Step 2	Look inside, and ask the venv's Python whether it is in a venv.
Step 3	Ask the normal Python the same question.

```
python3 -m venv venvdemo
ls venvdemo
./venvdemo/bin/python -c "import sys; print(sys.prefix != sys.base_prefix)"
./venvdemo/bin/pip freeze | wc -l
python3 -c "import sys; print(sys.prefix != sys.base_prefix)"
```

```
bin
include
lib
pyvenv.cfg
True
0
False
```

- **Answer** The venv is a small folder with its own `bin` (Python and pip), `lib` (packages) and a `pyvenv.cfg` file. Inside it, `sys.prefix` differs from `sys.base_prefix` (so the check is `True`). `pip freeze` lists nothing: the venv starts empty. The normal Python prints `False`.
- **Check** `pip freeze` printing 0 lines proves your global libraries do not leak in.
- **Why it matters** "It works on my machine" is usually a different set of library versions.

On macOS and Linux you "activate" with `source venvdemo/bin/activate`; on Windows `venvdemo\Scripts\activate`. Activating just puts the venv's `bin` first on your PATH, so `python` and `pip` point at it. You can skip activation by calling `./venvdemo/bin/python` directly, which is what scripts and CI do.

The standard workflow (not run here, needs network)

```
python3 -m venv .venv
source .venv/bin/activate
pip install requests pandas
pip freeze > requirements.txt     # exact versions recorded
pip install -r requirements.txt   # on another machine
```

Tool comparison (not run here: none of these were installed, and there was no network)

Tool	What it does	When you meet it
`venv` + `pip`	Built in, simple	Every project; always learn first
`pip-tools`	`pip-compile` turns a short `requirements.in` into a fully pinned `requirements.txt`	Teams that want pinned files without a new tool
`poetry`	Manages dependencies, lock file and packaging via `pyproject.toml`	Many libraries and apps
`uv`	Very fast modern tool for envs, installs and lock files; compatible with pip workflows	Fast-growing; common in new projects
`conda`	Manages Python itself and non-Python libraries (C, CUDA)	Data science and GPU work

Typical commands, shown for orientation only (check each tool's own docs, as options change by version):

```
pip-compile requirements.in          # pip-tools
poetry add requests                  # poetry
uv venv && uv pip install requests   # uv
conda create -n ds python=3.12       # conda
```

Connection to real projects. CI servers and Docker images build a fresh environment from your lock file, so your program runs the same way everywhere. ML projects often need conda or Docker because GPU libraries are not pure Python.

Applications

- **Data Science:** one environment per project so a library upgrade never breaks an old analysis.
- **Machine Learning:** pinned versions of `numpy`, `scikit-learn`, `torch` so results can be reproduced.
- **MLOps:** a lock file copied into the Docker build.
- **Data Engineering:** the same pinned environment locally and in the scheduler.

COMMON MISTAKE	Running `pip install` while no venv is active, so packages go into your global Python. Check `which python` (or `python -c "import sys; print(sys.prefix)"`) before installing. Also, never commit the `.venv` folder to Git; add it to `.gitignore`.

Check yourself

1. What problem does a venv solve?

Answer: It stops different projects from sharing, and clashing over, library versions.

1. What does `pip freeze` print?

Answer: Every installed package with its exact version.

1. When prefer conda over pip?

Answer: When you need non-Python binaries such as CUDA or C libraries.

9.4 `pyproject.toml`, Version Pinning and Supply-Chain Awareness

INTUITION	`pyproject.toml` is the project's ID card: its name, version, which Python it needs and which libraries it depends on. One file, readable by pip, poetry, uv and build tools.

How it works. The file uses TOML, a simple config format. Python 3.11+ can read it with the built-in `tomllib`.

Problem 9.4.1 Write a minimal `pyproject.toml` and parse it.

```
import tomllib
text = """
[project]
name = "shop"
version = "0.1.0"
requires-python = ">=3.10"
dependencies = ["requests>=2.31"]
"""
print(tomllib.loads(text))
```

```
{'project': {'name': 'shop', 'version': '0.1.0', 'requires-python': '>=3.10', 'dependencies': ['requests>=2.31']}}
```

- **Answer** TOML becomes a normal nested dictionary. Section `[project]` holds metadata. `dependencies` is a list of requirement strings.
- **Check** `d["project"]["name"]` would give `"shop"`.
- **Why it matters** Everything that builds, installs or publishes your project reads this one dictionary.

A fuller file also names a build tool (not run here, since building needs a build backend that may need download):

```
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"

[project.scripts]
shop = "shop.__main__:main"
```

`[project.scripts]` makes `pip install .` create a `shop` command that calls `main()`. Chapter 13 covers building and publishing.

Version specifiers

Written	Meaning	Use
`requests==2.31.0`	Exactly this version	Lock files (reproducible)
`requests>=2.31`	This or newer	Libraries (flexible)
`requests~=2.31`	Compatible: at least 2.31, below 3.0	Apps wanting safe updates
`requests>=2.31,<3`	A range	When a major version breaks

Rule of thumb: libraries declare flexible ranges; applications ship a fully pinned lock file.

Dependency security and supply chain

A **supply-chain attack** is when attackers hide bad code inside a library you install, or inside one of that library's own dependencies. Your project may depend on hundreds of packages you never chose. Protect yourself:

- Pin exact versions and review upgrades.
- Run `pip-audit` (not run here) to check installed packages against known vulnerability lists. Dependabot on GitHub opens pull requests for risky or old versions.
- Install only from sources you trust. Watch for look-alike names (`reqeusts` instead of `requests`) called typosquatting.
- Keep dependencies few. Every library is trust you give away.

Problem 9.4.2 List the installed distributions with the standard library (works in any environment).

```
import importlib.metadata as m
print(sorted(d.metadata['Name'] for d in m.distributions())[:5])
```

```
['Jinja2', 'MarkupSafe', 'PyYAML', 'Pygments', 'Send2Trash']
```

- **Answer** This is what is installed in the global Python on this machine. Inside a clean venv the list would be much shorter (just pip, maybe setuptools).
- **Check** Compare with `pip freeze` output.
- **Why it matters** In an audit you must be able to say exactly what code runs in production.

Connection to real projects. Security teams at large companies scan dependencies. A pinned lock file plus `pip-audit` in CI is a normal requirement, and it is also a strong signal in an interview.

Applications

- **MLOps:** reproducible model builds and audit trails.
- **Data Engineering:** upgrading one library without breaking twenty pipelines.
- **Security / compliance:** list of dependencies with versions.

COMMON MISTAKE	Leaving versions unpinned in an application, so the next `pip install` silently downloads a new major version and your code breaks on Monday. Pin applications. Flexible ranges are for libraries.

Check yourself

1. What does `~=2.31` allow?

Answer: Versions from 2.31 up to, but not including, 3.0.

1. Why pin versions in an application?

Answer: So every install gets the same code and results are reproducible.

1. What is typosquatting?

Answer: A malicious package with a name close to a popular one, hoping you mistype.

9.5 Type Hints and mypy

INTUITION	A label on a jar tells you what is inside before you open it. A **type hint** is a label on a variable or function: "this takes a list of floats and gives back a float". Python does not enforce the label when running, but tools and your teammates can read it, and an editor can warn you before you run anything.

Words you need

Word	Plain meaning
type hint (annotation)	A note like `x: int` or `-> float` about the expected type
static type checker	A tool (mypy, pyright) that reads hints and finds mismatches without running the code
`Optional[X]`	Either an `X` or `None` (same as `X \
`Union[A, B]`	Either an A or a B (same as `A \
`Callable[[int], int]`	A function taking an int and returning an int
`TypeVar`	A placeholder type, so "the output type equals the input type"
`Protocol`	An interface matched by shape (Chapter 8)

Problem 9.5.1 Common hints, run together.

```
from typing import Optional, Union, Callable, TypeVar, Protocol, Iterable

def mean(xs: list[float]) -> float:
    return sum(xs) / len(xs)

def find_user(users: dict[int, str], uid: int) -> Optional[str]:
    return users.get(uid)

def parse(v: Union[int, str]) -> int:
    return int(v)
def parse2(v: int | str) -> int:    # modern spelling
    return int(v)

def apply(f: Callable[[int], int], xs: Iterable[int]) -> list[int]:
    return [f(x) for x in xs]

T = TypeVar("T")
def first(xs: list[T]) -> T:
    return xs[0]
def first_new[T](xs: list[T]) -> T:   # Python 3.12+ syntax
    return xs[0]

class HasName(Protocol):
    name: str
def greet(x: HasName) -> str: return "hi " + x.name

print(mean([1,2,3]), find_user({1:"a"}, 2), parse("7"), parse2(8))
print(apply(lambda n: n*n, [1,2,3]), first(["x","y"]), first_new([10,20]))
```

```
2.0 None 7 8
[1, 4, 9] x 10
```

- **Answer** `mean([1,2,3]) = 6/3 = 2.0`. `find_user` returns `None` because key 2 is missing, which is exactly why its hint is `Optional[str]`. `first` returns the same type as the list holds (`str` for strings, `int` for ints). On Python 3.9 and later, built-in `list[float]` and `dict[int, str]` replace the old `typing.List` and `typing.Dict`. On 3.10+ you can write `int | str`. The `def first_new[T]` syntax needs Python 3.12 or later.
- **Check** 1+2+3 = 6, divided by 3 = 2.0. `1*1, 2*2, 3*3 = 1, 4, 9`.
- **Why it matters** Old code uses `List[int]` and `Optional`. You will see both styles at work, so know both.

Hints are not enforced at runtime. Predict: does `mean("abc")` complain about the hint?

```
try:
    print(mean("abc"))
except TypeError as e: print("TypeError:", e)
from typing import get_type_hints
print(get_type_hints(mean))
```

```
TypeError: unsupported operand type(s) for +: 'int' and 'str'
{'xs': list[float], 'return': <class 'float'>}
```

- **Answer** No. Python ignored the hint. The error came from `sum()` trying `0 + "a"`, not from the label. The hints are stored in `mean.__annotations__` for tools to read.
- **Check** The error is about `int` and `str`, not about `list[float]`.
- **Why it matters** If you want checking, you need a checker such as mypy or runtime validation (section 9.6).

mypy (not run here, no network). `mypy` is the standard static checker. You run it on your files and it reports mismatches without executing the program:

```
pip install mypy
mypy shop/
```

For the call `mean("abc")` above, mypy would report an error similar to `Argument 1 to "mean" has incompatible type "str"; expected "list[float]"`. Exact wording varies by version. Treat mypy as a test that runs in CI. Start with the default settings, and tighten gradually (for example `--strict`) on new code.

Connection to real projects. Teams add hints to shared functions, so the editor can autocomplete and catch mistakes. FastAPI (Chapter 12) reads hints to build request validation and API docs automatically.

Applications

- **Data Engineering:** typed transform functions reduce bugs in large pipelines.
- **Machine Learning:** hints on function inputs ("expects a DataFrame with these columns" in docstrings plus `pd.DataFrame` hints).
- **Backend:** FastAPI uses hints for validation and docs.
- **Code review:** hints act as documentation that cannot go stale unnoticed.

COMMON MISTAKE	Thinking hints make Python check types at runtime. They do not. A function annotated `-> int` can happily return a string. Also avoid the old `List`, `Dict` imports in new code.

Check yourself

1. What does `Optional[str]` mean?

Answer: A `str` or `None`.

1. Does Python raise an error when you pass the wrong type to an annotated function?

Answer: No. Only a checker like mypy, or your own validation, would.

1. What is `TypeVar` for?

Answer: To say that the output type depends on the input type.

9.6 Data Validation: By Hand and with pydantic

INTUITION	A security guard at a building door checks every visitor. Data arriving from outside (a web request, a CSV, a model output) cannot be trusted. **Validation** is the guard: it checks types and rules at the door, and rejects bad data with a clear message, before it spreads through your pipeline.

Why not just trust type hints? Hints are not enforced (section 9.5). Real input is a string like `"42"` where you wanted the number 42, or `"nomail"` where you wanted an email. A validation library converts what it can and rejects the rest.

Problem 9.6.1 Write a small validator using only the standard library. It converts ("coerces") values to the declared type and collects every error.

```
from dataclasses import dataclass, fields

class ValidationError(Exception):
    pass

@dataclass
class Order:
    order_id: int
    email: str
    amount: float
    def __post_init__(self):
        errors = []
        for f in fields(self):
            v = getattr(self, f.name)
            try:
                setattr(self, f.name, f.type(v))   # coerce like pydantic lax mode
            except (TypeError, ValueError):
                errors.append(f"{f.name}: cannot convert {v!r} to {f.type.__name__}")
        if not errors and "@" not in self.email: errors.append("email: missing @")
        if not errors and self.amount <= 0: errors.append("amount: must be > 0")
        if errors: raise ValidationError("; ".join(errors))

print(Order("42", "a@b.com", "99.5"))
for bad in [("x", "a@b.com", 5), (1, "nomail", 5), (1, "a@b.com", -3)]:
    try: Order(*bad)
    except ValidationError as e: print("rejected:", e)
```

```
Order(order_id=42, email='a@b.com', amount=99.5)
rejected: order_id: cannot convert 'x' to int
rejected: email: missing @
rejected: amount: must be > 0
```

Step by step: `fields(self)` lists the declared fields. For each one we call its type as a function: `int("42")` gives 42 and `float("99.5")` gives 99.5, but `int("x")` raises `ValueError`, which we turn into a message. After conversion we apply business rules (an email needs `@`, amount must be positive).

- **Answer** The good order is accepted and its text values are converted to `int` and `float`. The three bad ones are each rejected with a clear reason.

- **Check** `42` is now an `int` (the repr shows `order_id=42`, without quotes).
- **Why it matters** This is the pattern behind every validation library. Note that this tiny version assumes plain types such as `int`, `str`, `float`; real libraries handle lists, nested objects and more.

pydantic (not run here, not installed and no network). The library `pydantic` does this with far more features. This is the standard usage in version 2 (check the docs for your version):

```
from pydantic import BaseModel, EmailStr, Field, ValidationError

class Order(BaseModel):
    order_id: int
    email: EmailStr          # needs: pip install "pydantic[email]"
    amount: float = Field(gt=0)

try:
    Order(order_id="42", email="a@b.com", amount=-3)
except ValidationError as e:
    print(e)                 # lists every failing field
```

The expected behaviour: `"42"` is converted to 42, and `amount=-3` fails the rule `gt=0` with an error that names the field. You also get `.model_dump()` to convert back to a dictionary. FastAPI uses pydantic models for request and response bodies.

Approach	Strength	Weakness
Hints only	Free documentation	Nothing is checked at runtime
Hand-written validator	No dependency, full control	You maintain it, limited features
`dataclass` + `__post_init__`	Standard library	Basic only
pydantic	Rich rules, nested data, clear errors, JSON	Another dependency

Connection to real projects. Every data pipeline should validate input at its boundary: API payloads, config files, rows from vendors. A bad row caught at the door costs minutes; the same row found in a dashboard costs days.

Applications

- **Data Engineering:** validate each record from an upstream feed and send failures to a quarantine file.
- **Machine Learning:** validate prediction-request schemas in a model-serving API.
- **GenAI:** validate that an LLM's JSON output matches the expected structure before using it.
- **Backend:** request and response models in FastAPI.

COMMON MISTAKE

Validating only the happy path. Test with a missing field, a wrong type, an empty string, a negative number and a huge number. Also, never put secrets or passwords in error messages you log.

Check yourself

1. Why validate even if you have type hints?

Answer: Hints are not enforced at runtime, and outside data may be wrong.

1. What is coercion?

Answer: Converting a value to the expected type, like `"42"` to `42`.

1. Which standard-library tool gave us the list of fields?

Answer: `dataclasses.fields`.

9.7 Jupyter Notebooks vs Scripts, `nbconvert` and Reproducibility

INTUITION	A notebook is a lab notebook: you try an idea, see the result next to it, and keep notes. A script is a recipe card: anyone can follow it from start to finish and get the same dish. You explore in notebooks, and you ship scripts and packages.

What a notebook really is. A `.ipynb` file is just JSON: a list of cells with source text and saved outputs. Prove it:

```
import json
nb = {"cells":[{"cell_type":"code","source":["print(1+1)\n"],"outputs":[],"execution_count":None,"metadata":
{}}],"metadata":{},"nbformat":4,"nbformat_minor":5}
open("/tmp/c9/x.ipynb","w").write(json.dumps(nb))
d = json.load(open("/tmp/c9/x.ipynb"))
for c in d["cells"]:
    print("".join(c["source"]), end="")
    exec("".join(c["source"]))
```

```
print(1+1)
2
```

- **Answer** The notebook file loads as ordinary JSON, and each code cell's source is plain text we can run. (`exec` is used here only to show the idea. Never `exec` untrusted notebooks, because they can run any code.)
- **Check** `1+1 = 2`.
- **Why it matters** Because it is JSON, it works badly with Git diffs (outputs and execution counts change) and can hide problems such as cells run out of order.

	Notebook	Script / package
Best for	Exploration, charts, teaching, reports	Repeatable jobs, production, testing
Runs top to bottom?	Not guaranteed (you can run cells in any order)	Always
Git-friendly	Poor (JSON, outputs)	Good
Unit tests	Hard	Easy
Scheduling	Awkward	Natural (cron, Airflow)

Tools (not run here, need installation)

```
jupyter nbconvert --to script analysis.ipynb      # notebook -> .py
jupyter nbconvert --to html analysis.ipynb        # shareable report
jupyter nbconvert --to notebook --execute analysis.ipynb   # re-run top to bottom
```

Reproducibility checklist

1. Restart the kernel and "Run All" before sharing. If it fails, the notebook was not reproducible.
2. Set random seeds (for example `random.seed(42)`) and record them.
3. Keep a pinned `requirements.txt` or lock file next to the notebook.
4. Move reusable code into a package (`from shop import total_with_tax`) and import it in the notebook.
5. Never keep API keys in a notebook. Read them from environment variables (Chapter 13).
6. Strip outputs before committing, or use a tool made for it (for example `nbstripout`), so diffs stay readable.

Connection to real projects. The common professional flow: explore in a notebook, then move the working logic into functions in a package with tests, and let the notebook import them. Hiring managers look for exactly this separation.

Applications

- **Data Analytics:** a notebook turned into an HTML report for managers.
- **Data Science:** exploration notebooks plus a clean `train.py`.
- **Machine Learning:** `nbconvert --execute` in CI to check notebooks still run.
- **Teaching and portfolios:** notebooks as readable demos.

COMMON MISTAKE	Running cells out of order, then sharing a notebook that only works with your hidden state. If "Restart and Run All" fails, fix it before anyone else sees it.

Check yourself

1. What file format is an `.ipynb`?

Answer: JSON.

1. Why restart the kernel and run all before sharing?

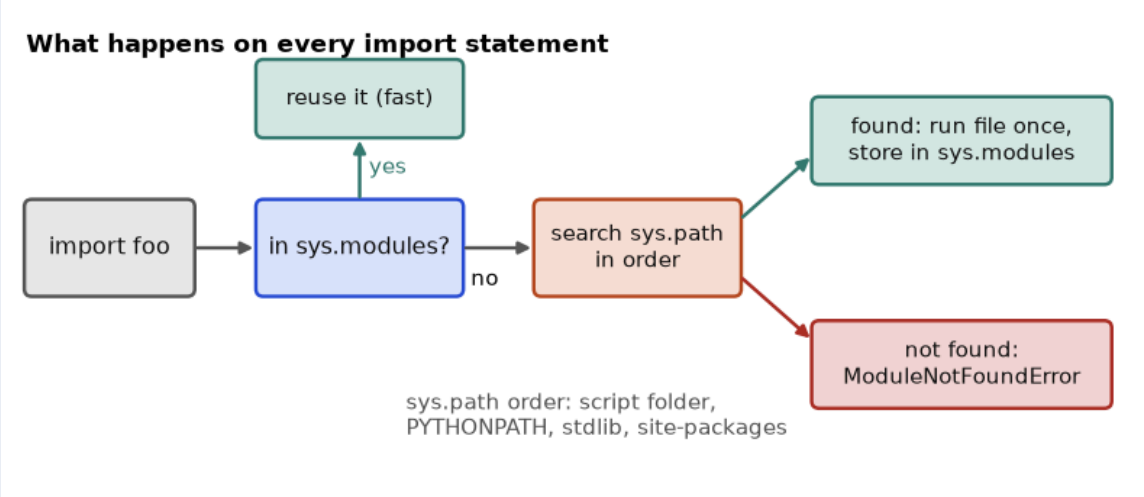
Answer: To prove the notebook works from a clean state, in order.

1. Where should reusable logic live?

Answer: In a package or script that the notebook imports.

9.8 Typing in Depth, Import Order and Packaging Layout

Section 9.5 gave you the basics: annotations and a type checker. Real code needs more precise promises: "this dictionary has exactly these keys", "this string is one of three values", "this decorator keeps the signature of the function it wraps". This section adds those tools. A key fact first: **Python itself ignores annotations at run time**. A separate program, a type checker such as mypy or pyright, reads them. Every code block below really ran, so its output is real. Checker messages are labelled *not run here* because mypy is not installed in this environment.



Every `import foo` follows this path. The first import runs the file and stores the module in `sys.modules`. Later imports reuse it.

How to read it: start at the grey box. If the name is already in `sys.modules` the answer is immediate. If not, Python walks `sys.path` in order and the first match wins. That is why a file of yours called `random.py` in the script folder breaks `import random` everywhere: the script folder is searched first.

Typing a JSON payload with TypedDict

A `TypedDict` describes a dictionary with fixed string keys. At run time it is an ordinary `dict`. A checker, however, flags a misspelt key or a wrong value type. `NotRequired` marks a key that may be missing.

Problem 9.8.1 Describe a user payload that always has `id` and `name` and sometimes `email`, then read it from JSON.

Step 1	Subclass `TypedDict` and list the keys with their types.
Step 2	Mark `email` as `NotRequired[str]`.
Step 3	Parse the JSON and annotate the result as `User`. Python does no checking here: you are promising the checker what the data looks like.
Check	Print the required and optional key sets Python recorded.

```
from typing import TypedDict, NotRequired
import json

class User(TypedDict):
    id: int
    name: str
    email: NotRequired[str]

payload = json.loads('{"id": 7, "name": "Ada"}')
u: User = payload
print(u["name"], type(u).__name__, "email" in u)
print(sorted(User.__required_keys__), sorted(User.__optional_keys__))
```

```
Ada dict False
['id', 'name'] ['email']
```

Compare with pydantic from 9.6: a `TypedDict` costs nothing and checks nothing at run time, so it suits data you already trust (your own functions passing dicts around). For data from outside, validate first, then type it.

COMMON MISTAKE	Believing an annotation validates data. `u: User = {"id": "oops"}` runs without any error. Only the checker (not run here: `mypy file.py` would report an incompatible type) objects. Validate untrusted input with code, not with hints.

Literal, Final, Self and Protocol

Problem 9.8.2 Restrict a log level to three strings, mark a constant as not reassignable, make a chainable builder whose methods return the subclass, and accept any object that has a `close()` method.

Literal	`Literal["debug", "info", "error"]` says "exactly one of these values". Passing `"verbose"` is a checker error but still runs.
Final	`MAX_TRIES: Final = 3` tells the checker never to reassign it.
Self	A method returning `Self` returns whatever class it was called on, so chaining keeps the subclass methods.
Protocol	Structural typing: any class with a matching `close()` fits, with no inheritance needed (duck typing, made checkable).

```
from typing import Literal, Final, Self, Protocol

Level = Literal["debug", "info", "error"]
def set_level(level: Level) -> str:
    return level.upper()
print(set_level("info"))
print(set_level("verbose"))   # a checker rejects this line; Python runs it

MAX_TRIES: Final = 3
print(MAX_TRIES)

class Builder:
    def __init__(self) -> None:
        self.parts: list[str] = []
    def add(self, p: str) -> Self:
        self.parts.append(p)
        return self
class SqlBuilder(Builder):
    def build(self) -> str:
        return " ".join(self.parts)
print(SqlBuilder().add("SELECT 1").add("FROM t").build())

class SupportsClose(Protocol):
    def close(self) -> None: ...
class Conn:
    def close(self) -> None:
        print("closed")
def shutdown(x: SupportsClose) -> None:
    x.close()
shutdown(Conn())
```

```
INFO
VERBOSE
3
SELECT 1 FROM t
closed
```

Without `Self`, you would annotate `add` as returning `Builder`, and the checker would then complain that `.build()` does not exist on `Builder` after the first `.add(...)`. Not run here: that mypy message.

overload: one function, return type depends on the input

Problem 9.8.3 Write `parse` that turns one string into an `int` and a list of strings into a list of `int`, and tell the checker which return type goes with which input.

Idea	Stack `@overload` stubs (signature only, body `...`), then write one real implementation with no decorator.
Run time	Only the real implementation exists. The stubs are thrown away.
Benefit	`parse("5") + 1` type checks. Without overloads the result would be `int | list[int]` and the addition would be flagged.

```
from typing import overload

@overload
def parse(v: str) -> int: ...
@overload
def parse(v: list[str]) -> list[int]: ...
def parse(v):
    return int(v) if isinstance(v, str) else [int(i) for i in v]
print(parse("5"), parse(["1", "2"]))
```

```
5 [1, 2]
```

Typing the retry decorator with ParamSpec

The retry decorator from Chapter 7 loses its signature for a checker if you annotate the wrapper as `Callable[..., Any]`. `ParamSpec` captures "whatever parameters the wrapped function has" and reuses them. Since Python 3.12 you write it as `[**P, R]` after the function name (earlier versions import `ParamSpec` and `TypeVar` by hand).

Problem 9.8.4 Write `retry(times)` that retries on `ValueError`, keeps the name and signature of the wrapped function, and test it with a function that fails twice.

Step 1	Generic parameters `[**P, R]`: `P` is the parameter list, `R` the return type.
Step 2	The wrapper takes `*args: P.args, **kwargs: P.kwargs` and returns `R`.
Step 3	`functools.wraps` copies the name and docstring at run time.
Check	The call `flaky(5, scale=10)` succeeds on the third attempt and prints 50.

```
import functools
from collections.abc import Callable

def retry[**P, R](times: int) -> Callable[[Callable[P, R]], Callable[P, R]]:
    def deco(fn: Callable[P, R]) -> Callable[P, R]:
        @functools.wraps(fn)
        def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
            last: Exception | None = None
            for attempt in range(1, times + 1):
                try:
                    return fn(*args, **kwargs)
                except ValueError as e:
                    last = e
                    print(f"attempt {attempt} failed")
            assert last is not None
            raise last
        return wrapper
    return deco

calls = {"n": 0}
@retry(3)
def flaky(x: int, *, scale: int = 2) -> int:
    calls["n"] += 1
    if calls["n"] < 3:
        raise ValueError("boom")
    return x * scale
print(flaky(5, scale=10), flaky.__name__)
```

```
attempt 1 failed
attempt 2 failed
50 flaky
```

With this typing, a checker would reject `flaky("a", scale="b")` exactly as it would reject a direct call (not run here). Without it, the decorated function accepts anything.

Going deeper

- **Gradual typing strategy.** Do not try to type a legacy code base in one week. Turn on the checker for new files first, then enable strict options one at a time (`disallow_untyped_defs` first, then `warn_return_any`). Strict mode (`strict = true` in the `[tool.mypy]` table of `pyproject.toml`) is a goal, not a day-one requirement.
- **Escape hatches.** `Any` switches checking off for a value. `cast(T, x)` and `# type: ignore[code]` silence the checker without changing behaviour. Each one is a place where a bug can hide, so grep for them in reviews.
- **Packaging layout.** Prefer a `src/` layout: `src/mypkg/__init__.py` plus `pyproject.toml` at the top. Tests then import the *installed* package, so a missing file in the build shows up in testing instead of in production. A command-line entry point is declared in `[project.scripts]` , for example `mytool = "mypkg.cli:main"` , and the installer creates a `mytool` executable that calls `main()` . Building a wheel with `python -m build` and uploading with `twine` (or trusted publishing from CI) is not run here because it needs those tools and a package index.

More practice: types and imports

Problem 9.P.1 (warm-up) Your script folder contains a file named `json.py` and your code says `import json` . What happens, and how do you diagnose it?

Answer: Python finds your file first, because the script folder is first on `sys.path` . Anything that expects the standard library module fails with an `AttributeError` such as "module 'json' has no attribute 'loads'". Diagnose by printing `json.__file__` , then rename your file.

Problem 9.P.2 (build) Define a `TypedDict` called `Order` with `sku: str` , `qty: int` and an optional `note: str` , then write `total_units(orders: list[Order]) -> int` . Run it on two orders.

```
from typing import TypedDict, NotRequired

class Order(TypedDict):
    sku: str
    qty: int
    note: NotRequired[str]

def total_units(orders: list[Order]) -> int:
    return sum(o["qty"] for o in orders)

print(total_units([{"sku": "A1", "qty": 3}, {"sku": "B2", "qty": 4, "note": "gift"}]))
```

```
7
```

Answer: 7. Reading `o["note"]` without checking would be flagged by a checker, because the key is not required. Use `o.get("note")` .

Problem 9.P.3 (what is the output?) `def f(x: int) -> int: return x * 2` then `print(f("ab"))` . Does the annotation stop it?

```
def f(x: int) -> int:
    return x * 2
print(f("ab"))
```

```
abab
```

Answer: It prints `abab` . Annotations are not enforced at run time, and `str * int` is legal. A checker would flag the call (not run here).

Problem 9.P.4 (stretch) Add `@overload` stubs to `def first(x)` so that a `str` input returns `str` and a `list[int]` input returns `int`. Rubric: (1) two stubs with `...` bodies, (2) one undecorated implementation, (3) implementation handles both cases, (4) you called both forms.

Answer: Pattern: `@overload def first(x: str) -> str: ...`, `@overload def first(x: list[int]) -> int: ...`, then `def first(x): return x[0]`. The implementation body is the same for both, because indexing works on both. Common error: forgetting the final undecorated implementation, which raises no error until you call the function and find it is a stub that returns `None`.

Problem 9.P.5 (interview) When would you pick `Protocol` over an abstract base class?

Answer: Pick `Protocol` when the classes that should fit are not yours to change (third-party or standard library), or when you want callers to pass anything with the right methods without inheriting. Pick an abstract base class when you want shared code in the base, run-time `isinstance` checks without extra decorators, or to force subclasses to declare they implement the contract.

Chapter recap

- A module is a file, a package is a folder with `__init__.py`. Python runs a module once and caches it in `sys.modules`.
- Use `if __name__ == "__main__":` so a file can be both imported and run. Run packages with `python -m package`.
- Relative imports (`from .x import y`) only work inside a package; running the file directly breaks them.
- Circular imports mean two modules need each other. Fix the design with a third shared module; deferred imports are a patch.
- Give every project its own virtual environment, and never install into global Python by accident.
- Libraries declare flexible versions, applications ship pinned lock files; scan dependencies with `pip-audit` and beware look-alike package names.
- Type hints are labels. Python does not enforce them; mypy and runtime validation (pydantic) do.
- Notebooks are for exploration, scripts and packages are for production. Always "Restart and Run All" before sharing.

Task	Command / pattern
Run a package	`python3 -m shop`
Script guard	`if __name__ == "__main__":`
Create venv	`python3 -m venv .venv`
Activate (mac/Linux)	`source .venv/bin/activate`
Save versions	`pip freeze > requirements.txt`
Install saved	`pip install -r requirements.txt`
Read pyproject	`tomllib.loads(text)`
Type hint, modern	`list[int]`, `int \`
Static check	`mypy package/` (not run here)
Notebook to script	`jupyter nbconvert --to script file.ipynb`

Practice set

Problem 9.8.1 (easy) What is `__name__` inside `shop/pricing.py` when it is imported by `shop/__init__.py`?

Answer: `shop.pricing`, as shown in the run output.

Problem 9.8.2 (easy) What does the line `if __name__ == "__main__":` protect against?

Answer: It stops the code under it from running when the file is imported. It runs only when the file is executed directly.

Problem 9.8.3 (easy) Which command creates a virtual environment in a folder called `.venv`?

Answer: `python3 -m venv .venv`.

Problem 9.8.4 (what is the output?) What prints? `def f(x: int) -> int: return x` followed by `print(f("a"))`.

Answer: `a`. The hints are not enforced at runtime, so the string passes through.

Problem 9.8.5 (find the bug) A teammate runs `python3 shop/pricing.py` and sees `ImportError: attempted relative import with no known parent package`. Why, and what is the fix?

Answer: The file was run as a top-level script, so the dot has no parent package. Run `python3 -m shop.pricing` from the project root.

Problem 9.8.6 (medium) `a.py` has `from b import fb` at the top and `b.py` has `from a import fa` at the top. Give two fixes.

Answer: Move `fa` and `fb` (or the code they share) into a third module that neither imports from the other; or move one of the imports inside the function that uses it.

Problem 9.8.7 (medium) What do `==2.31.0`, `>=2.31` and `~=2.31` mean?

Answer: Exactly 2.31.0; 2.31 or newer; at least 2.31 but below 3.0.

Problem 9.8.8 (medium) In `Order("42", "a@b.com", "99.5")` from 9.6, what are the types of `order_id` and `amount` afterwards?

Answer: `int` (value 42) and `float` (value 99.5), because the validator coerces them.

Problem 9.8.9 (interview) Why does `import pandas` appear to be instant the second time within the same program?

Answer: The first import runs the module and stores it in `sys.modules`. Later imports just return the cached module object.

Problem 9.8.10 (interview) Your lead says "pin your dependencies and run `pip-audit` in CI". Explain both.

Answer: Pinning fixes exact versions so every build is the same and updates are deliberate. `pip-audit` checks those installed versions against known vulnerability data, so risky packages are found before release. Together they give reproducibility and basic supply-chain safety.

Problem 9.8.11 (scenario) Your notebook works on your laptop but fails for a colleague with `ModuleNotFoundError`. Give three steps to fix the process.

Answer: (1) Create a venv and install from a pinned `requirements.txt` that you commit next to the notebook. (2) Restart the kernel and run all cells top to bottom to find hidden dependencies on state. (3) Move reusable code into a package and install it into the environment, so imports are the same for everyone.

Prompt pack

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional at an Indian company moving into data and AI roles, and English is my second language, so use simple words. Teach me Python modules and packages: __init__.py, __name__ == "__main__", python -m, relative vs absolute imports, and circular imports. Walk me step by step through building a small package with three files, predict-then-run style. Then ask me five questions one at a time, waiting for each answer.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data and AI roles, and English is my second language. Explain virtual environments, pip, pip-tools, poetry, uv and conda: what problem each solves, how they differ, and when to choose each. Use a simple analogy, a comparison table, and a realistic ML project example. Then give me 8 graded exercises without solutions, and offer to review my answers.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data and AI roles, and English is my second language. Teach me pyproject.toml, version specifiers (==, >=, ~=), lock files, pip-audit, Dependabot,
```

typosquatting and supply-chain risk. Use concrete examples and a short checklist I can follow for my own repository. Then review my pyproject.toml: [PASTE HERE] and point out problems.

✨ AI prompt: copy and paste into any AI

You are my patient, practical professor. I am a working professional moving into data and AI roles, and English is my second language. Teach me Python type hints (list[int], Optional, Union, Callable, TypeVar, Protocol), what mypy checks, and why hints are not enforced at runtime. Show small examples with wrong and fixed versions. Then quiz me one question at a time, and explain every mistake I make.

✨ AI prompt: copy and paste into any AI

You are my patient, practical professor. I am a working professional moving into data and AI roles, and English is my second language. Teach me data validation with pydantic and with plain Python, and the differences between notebooks and scripts, nbconvert and reproducibility. Then run a mock interview with six questions, one at a time, graded out of 10 with the ideal answer, and finish with my three weakest topics.

CHAPTER 10

TESTING AND CODE QUALITY

How to prove your code works today, and keep proving it every day after.

You have written a data-cleaning function. It works on your five test rows. Next month a teammate changes it, and a wrong number reaches a manager's dashboard. Professional engineers prevent this with automatic tests and automatic style checks. In this chapter you will write tests with `unittest` and `pytest`, replace slow or risky parts with mocks and fakes, build code step by step with TDD, measure coverage, and set up the tools that keep a team's code clean. You will use one running example, a small `slugify` function and a tiny price-lookup service, so each idea builds on the last.

> **A note on what was run.** This machine has only the Python 3.14 standard library and no network. So `pytest`, `pytest-cov`, `pytest-mock`, `black`, `ruff`, `mypy` and `pre-commit` were **not run here**. Their snippets are standard usage, marked "not run here". The same ideas were run with the standard library (`unittest`, `unittest.mock`, `sqlite3`, `trace`), and all output below is real.

10.1 Why We Test, and the Testing Pyramid

INTUITION

A restaurant kitchen tastes every dish before it leaves the pass. A test is a small robot that checks one promise your code makes, and it never gets tired. You change code, run the robots, and in seconds you know if you broke something. Without tests, every change is a gamble.

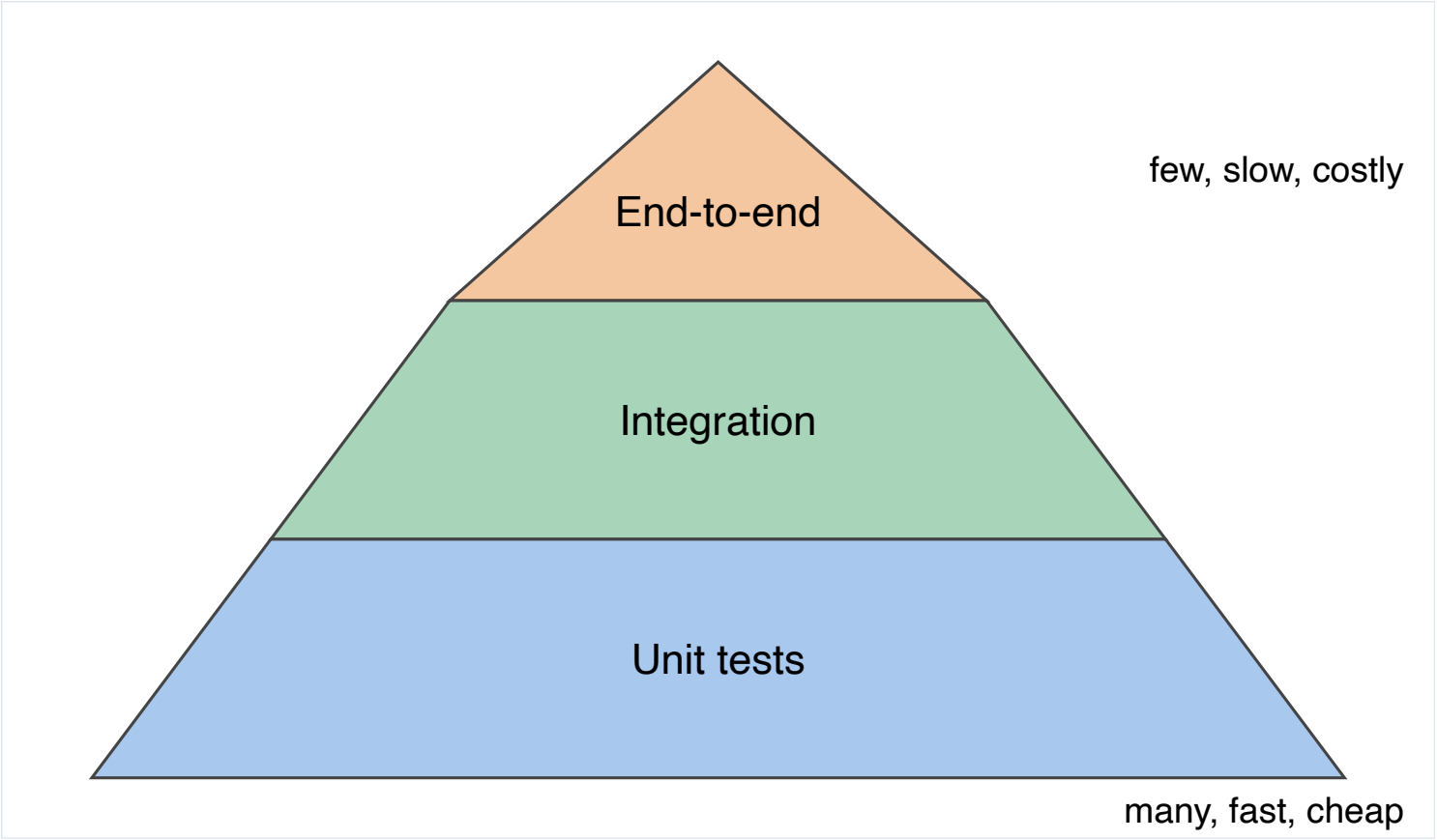
What a test is

A **test** is a piece of code that calls your code with known input and checks the output against the answer you expect. If they differ, the test **fails**. A set of tests is a **test suite**. When you run the suite after every change, a failure points to the change that broke things. This is called catching a **regression** (an old feature that stops working).

The testing pyramid

Tests come in three sizes.

- **Unit test**: checks one small piece (one function or class) alone. Runs in milliseconds. No network, no disk, no real database.
- **Integration test**: checks that two or more pieces work together, for example your code plus a real (small) database.
- **End-to-end (E2E) test**: runs the whole system like a user would, for example call the real API and check the stored result.



The testing pyramid

```
      /\          few: slow, costly, flaky
     /E2E\
    /------\
   /integr. \     some: medium speed
  /----------\
 /   unit     \   many: fast, cheap, precise
/--------------\
```

Why a pyramid and not an upside-down one? Unit tests are fast and tell you exactly where the bug is. E2E tests are slow and a failure only says "something is wrong somewhere". You want many cheap tests at the bottom and few expensive tests on top.

Test data

Tests need input. Good test data is small, explicit, and lives next to the test. Use three kinds of cases:

Kind	Example for `slugify`	Why
Normal	`"Hello World"`	The main promise
Edge	`""`, one character, very long text	Boundaries break most code
Bad	`None`, wrong type	Check clear errors

Never use real customer data in tests. Make small fake data by hand, or generate it with a fixed random seed so the test is repeatable.

Mapping the idea to a real project

Item	Example	Where it appears in the job
Unit test	`slugify("Hello World") == "hello-world"`	Checking one cleaning function
Integration test	Function plus in-memory SQLite	Checking a data-access layer
E2E test	Call the real API, read the stored row	Smoke test after a deploy (like a quick taste check before opening the restaurant)
Regression	Yesterday's bug returns	Why CI runs on every change

Predict, run, explain

Problem 10.1.1 Write a function `add(a, b)` and four tests: two integers, floats, a deliberately wrong expectation, and a wrong type. Run them and read the report.

Step 1	Write the function and the `unittest.TestCase` class. `unittest` is the testing module that ships with Python. Each method starting with `test_` is one test.
Step 2	Run the file with `verbosity=2` to list each test.
Step 3	Read the failure. It shows the line, and `4 != 5`.

```
import unittest
def add(a, b):
    return a + b
class TestAdd(unittest.TestCase):
    def test_ints(self):
        self.assertEqual(add(2, 3), 5)
    def test_floats(self):
        self.assertAlmostEqual(add(0.1, 0.2), 0.3)
    def test_bad(self):
        self.assertEqual(add(2, 2), 5)
    def test_type(self):
        with self.assertRaises(TypeError):
            add("a", 1)
if __name__ == "__main__":
    unittest.main(verbosity=2)
```

Real output:

```
test_bad (__main__.TestAdd.test_bad) ... FAIL
test_floats (__main__.TestAdd.test_floats) ... ok
test_ints (__main__.TestAdd.test_ints) ... ok
test_type (__main__.TestAdd.test_type) ... ok

======================================================================
FAIL: test_bad (__main__.TestAdd.test_bad)
----------------------------------------------------------------------
Traceback (most recent call last):
  File "/private/tmp/w10/t1.py", line 10, in test_bad
    self.assertEqual(add(2, 2), 5)
    ~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^
AssertionError: 4 != 5

----------------------------------------------------------------------
Ran 4 tests in 0.003s

FAILED (failures=1)
```

- **Answer** Three tests passed and one failed, as designed. We used `assertAlmostEqual` for floats because `0.1 + 0.2` is `0.30000000000000004`, not exactly `0.3`.
- **Check** `2 + 2` is 4, and the report says `4 != 5`, so the failure message is telling the truth: our expected value (5) was the wrong one.
- **Why it matters** A failing test is not bad news. It is a clear message that says which promise broke and on which line.

Before you run it, predict: how many tests will say `ok`? (Answer: three.) If you remove `assertAlmostEqual` and write `assertEqual(add(0.1, 0.2), 0.3)`, the report would show `0.30000000000000004 != 0.3`, so use the almost-equal form.

Problem 10.1.2 Test `slugify(text)`, which turns a title into a URL piece, using normal, edge and several cases at once with `subTest`.

Step 1	Write the function with `re.sub` (regex from Chapter 3). Replace every run of characters that are not letters or digits with `-`, then trim `-` at the ends.
Step 2	Write tests. `subTest` runs each case separately, so one failure does not hide the others.

```
# slug.py
import re
def slugify(text):
    text = text.strip().lower()
    text = re.sub(r"[^a-z0-9]+", "-", text)
    return text.strip("-")
```

```
# test_slug.py
import unittest
from slug import slugify
class T(unittest.TestCase):
    def test_basic(self):
        self.assertEqual(slugify("Hello World"), "hello-world")
    def test_punct(self):
        self.assertEqual(slugify("  Python, 101!  "), "python-101")
    def test_empty(self):
        self.assertEqual(slugify(""), "")
    def test_subtests(self):
        cases = [("A  B", "a-b"), ("--x--", "x"), ("C++ Tips", "c-tips")]
        for raw, want in cases:
            with self.subTest(raw=raw):
                self.assertEqual(slugify(raw), want)
```

```
python3 -m unittest -v test_slug
```

```
test_basic (test_slug.T.test_basic) ... ok
test_empty (test_slug.T.test_empty) ... ok
test_punct (test_slug.T.test_punct) ... ok
test_subtests (test_slug.T.test_subtests) ... ok

----------------------------------------------------------------------
Ran 4 tests in 0.000s

OK
```

- **Answer** All four tests pass. `python3 -m unittest` finds files named `test*.py` when you run `python3 -m unittest discover`.
- **Check** By hand: `"C++ Tips"` lower-cased is `c++ tips`. The run `"++ "` is not a letter or digit, so it becomes one `-`, giving `c-tips`. This matches the test.
- **Why it matters** Using one test per rule means that if you later change the regex, you see exactly which rule broke.

Connection to the real job

- Data pipelines silently produce wrong numbers when a function changes. Tests on transformation functions catch this before the dashboard does.
- ML code is tested too: shape checks, no-NaN checks, and "same seed gives same result".
- Teams will not merge your pull request if the CI (Continuous Integration, an automatic runner) shows red tests.

Applications

- **Data Analytics:** tests on a cleaning function that turns `"₹1,200"` into `1200`.
- **Data Engineering:** unit tests for each transform, integration tests with a small database.
- **Machine Learning:** checks that feature code gives the same output for the same seed.
- **MLOps:** regression suites that run in CI before a model-serving service is released.

COMMON MISTAKE

Writing tests that only check the "happy path". Bugs live at the edges: empty list, zero, negative, `None`, duplicate keys, a time zone change. Add at least one edge and one bad case for each function.

COMMON MISTAKE

Testing with `==` on floats. Use `assertAlmostEqual` (unittest) or `pytest.approx` (pytest).

Check yourself

1. Name the three levels of the pyramid, from cheapest to most expensive. Answer: unit, integration, end-to-end.
2. Why is `assertEqual(0.1 + 0.2, 0.3)` a bad test? Answer: floats are stored in binary, and the sum is `0.30000000000000004`, so equality fails.
3. What does a regression mean? Answer: something that worked before stops working after a change.

10.2 pytest: Plain Asserts, Fixtures and Parametrize

INTUITION

`unittest` makes you write a class and remember names like `assertEqual`. `pytest` lets you write a plain function and a plain `assert`. It then prints a rich explanation when the assert fails. It is the standard in industry.

How pytest works

`pytest` is a third-party tool (`pip install pytest`). It finds files named `test_*.py` and functions named `test_*`. You write `assert condition`. If the condition is false, the test fails, and pytest shows the values on both sides.

```
# test_slug_pytest.py   (not run here)
import pytest
from slug import slugify

def test_basic():
    assert slugify("Hello World") == "hello-world"

def test_empty():
    assert slugify("") == ""

def test_none_raises():
    with pytest.raises(AttributeError):
        slugify(None)

def test_float_close():
    assert 0.1 + 0.2 == pytest.approx(0.3)
```

Run with `pytest -q` (quiet) or `pytest -v` (verbose). Useful flags: `-x` stops at the first failure, `-k name` runs tests whose name contains `name`, `--lf` re-runs only the tests that failed last time.

Parametrize: one test, many cases

`@pytest.mark.parametrize` runs the same test body with different inputs. Each case shows as its own test, so a failure names the exact case. It is the pytest version of the `subTest` loop above.

```
# not run here
import pytest
from slug import slugify

@pytest.mark.parametrize(
    "raw, want",
    [
        ("A  B", "a-b"),
        ("--x--", "x"),
        ("C++ Tips", "c-tips"),
        ("", ""),
    ],
)
def test_slugify_cases(raw, want):
    assert slugify(raw) == want
```

Fixtures: shared setup, with clean-up

A **fixture** is a function that prepares something a test needs (a database, a temp file, sample data). Tests ask for it by writing its name as an argument. pytest runs the fixture and passes the result. Code after `yield` is the clean-up.

```
# not run here
import sqlite3
import pytest

@pytest.fixture
def conn():
    c = sqlite3.connect(":memory:")
    c.execute("CREATE TABLE users(id INTEGER PRIMARY KEY, name TEXT UNIQUE)")
    yield c            # the test runs here
    c.close()          # clean-up, even if the test failed

def test_insert(conn):
    conn.execute("INSERT INTO users(name) VALUES (?)", ("ravi",))
    rows = conn.execute("SELECT name FROM users").fetchall()
    assert rows == [("ravi",)]
```

Fixture **scope** controls how often it is rebuilt: `function` (default, every test), `module`, `session` (once for the whole run). Use the widest scope only for expensive, read-only things. Built-in fixtures worth knowing: `tmp_path` (a fresh temporary folder), `capsys` (capture printed text), `monkeypatch` (change things for one test, see 10.3). A file called `conftest.py` holds fixtures shared by many test files.

The same ideas, run with unittest

Since pytest is not installed here, this is the `unittest` form of fixture and clean-up, which we can run. `setUp` runs before each test, `tearDown` after, and `tempfile.TemporaryDirectory` gives a fresh folder.

Words you need. *Setup* is code that prepares the world before a test. *Teardown* is code that cleans up after. A *fixture* is a named setup/teardown that pytest gives to tests.

Item	Example	Where it appears in the job
Fixture	In-memory DB with a table	Every DB or API test
Parametrize	15 messy phone numbers, 15 clean answers	Data-cleaning rules
`tmp_path`	Fresh temp folder	Testing file readers and writers
`conftest.py`	Shared fixtures for the whole project	Large test suites

Problem 10.2.1 Test a save/load pair of JSON functions using a temporary folder, so no test leaves files behind.

Step 1	Write `save(path, obj)` and `load(path)` using `pathlib` and `json` (Chapter 5).
Step 2	In the test, create a temp folder with a context manager, so it is deleted at the end.
Step 3	Save, load, compare.

```
import tempfile, pathlib, unittest, json
def save(path, obj): path.write_text(json.dumps(obj))
def load(path): return json.loads(path.read_text())
class T(unittest.TestCase):
    def test_roundtrip(self):
        with tempfile.TemporaryDirectory() as d:
            p = pathlib.Path(d) / "x.json"
            save(p, {"a": [1, 2]})
            self.assertEqual(load(p), {"a": [1, 2]})
unittest.main(argv=["x"], verbosity=1)
```

```
----------------------------------------------------------------------
Ran 1 test in 0.010s

OK
```

- **Answer** The round trip works, and the temp folder is gone after the test. In pytest the same job is done by the `tmp_path` fixture.
- **Check** We saved `{"a": [1, 2]}` and loaded the same dictionary back, so nothing was lost in JSON. The test touched no real project file.
- **Why it matters** Tests that clean up after themselves can run in any order and on any machine.

Problem 10.2.2 Read what a failing assertion on two dictionaries looks like in `unittest`.

Step 1	Compare `{'a':1,'b':2}` with `{'a':1,'b':3}`.
Step 2	Read the diff marks. The `^` shows the exact character that differs.

```
AssertionError: {'a': 1, 'b': 2} != {'a': 1, 'b': 3}
- {'a': 1, 'b': 2}
?              ^

+ {'a': 1, 'b': 3}
?              ^
```

- **Answer** The value of key `b` differs: 2 against 3. pytest prints a similar diff for a plain `assert`.
- **Check** Compare by eye: key `a` is 1 in both, key `b` is 2 and 3. The `^` marks sit under the `2` and the `3`.
- **Why it matters** Learning to read the diff saves minutes on every failing test.

The picture

```
 test_x(conn)                fixture conn()
     |   asks for "conn"          |
     |--------------------------->|  build DB
     |<-------- DB object --------|  (yield)
     | run test body              |
     |--------------------------->|  clean-up after yield
```

Connection to the real job

- A fixture that builds a tiny DataFrame or a tiny SQLite table is how data teams test transformations without touching production.
- `parametrize` is perfect for data-cleaning rules: list 15 messy inputs and the clean output you want.
- CI runs `pytest` on every pull request.

Applications

- **Data Analytics:** parametrized tests for date and currency parsing rules.
- **Data Engineering:** fixtures that build a tiny table to test an ETL transform.
- **Machine Learning:** tests for model-input validation (missing columns, wrong types).
- **MLOps / backend:** FastAPI endpoint tests with a test client (Chapter 12); CLI tests through `capsys`.

COMMON MISTAKE	Forgetting the pytest-only names. If you write `def test_x(conn)` and no fixture called `conn` exists, pytest stops with an error that says `fixture 'conn' not found` (not run here). Check the spelling or add the fixture to `conftest.py`.

COMMON MISTAKE	Tests that depend on each other or on order. Test B passes only because test A ran first and left a row in the database. Each test must build what it needs, using a fixture, and leave nothing behind.

COMMON MISTAKE	Giving a fixture the wide `session` scope when tests change it. One test then pollutes the next. Start with the default scope.

Check yourself

1. In pytest, how does a test receive a fixture? Answer: by naming it as a function argument.
2. What does code after `yield` in a fixture do? Answer: it runs as clean-up after the test finishes.
3. What is `parametrize` for? Answer: to run one test body with many input cases, each reported separately.

10.3 Mocks, Fakes, monkeypatch and Database Tests

INTUITION	In a rehearsal of an online shop you do not call a real payment gateway with real money. You stub it. A unit test is the same: replace the slow, costly or random parts (network, clock, email) with stand-ins you control, so the test is fast and repeatable.

Terms (words you need)

- **Test double**: any stand-in used instead of the real thing.
- **Fake**: a small, working, simple replacement (for example a dictionary instead of a database). It has real behaviour.
- **Stub**: returns fixed answers. No logic.
- **Mock**: records how it was called, so you can check "was it called once, with these arguments?". In Python, `unittest.mock.Mock` is both a stub and a mock.
- **Spy**: wraps the real thing and records calls.

Item	Example	Where it appears in the job
Stub	Always returns `{"status": 200}`	Skipping a slow partner API
Fake	Dict-backed `FakeClient`	Testing logic without a DB
Mock	`client.get.assert_called_once()`	Checking "email sent once"
Spy	Real function wrapped to count calls	Checking a retry loop

Rule of thumb: prefer a **fake** when the behaviour is simple, because it tests the result. Use a **mock** when the important thing is that a call happened (email sent, once).

Mock in action

Problem 10.3.1 `fetch_price(client, sku)` calls `client.get(...)` and returns the price. `total` sums prices for many SKUs. Test `total` with no network, using a `Mock` and also a `Fake`.

Step 1	Pass the client in as an argument. This is **dependency injection** (Chapter 8): the function does not build its own client, so tests can pass a different one.
Step 2	Use `side_effect` with a list so each call returns the next item. Then assert the call count.
Step 3	Write a `FakeClient` class with a dictionary and test again.

```
import unittest
from unittest import mock

def fetch_price(client, sku):
    resp = client.get(f"/price/{sku}")
    if resp["status"] != 200:
        raise ValueError("bad status")
    return resp["body"]["price"]

def total(client, skus):
    return sum(fetch_price(client, s) for s in skus)

class TestMock(unittest.TestCase):
    def test_total_with_mock(self):
        client = mock.Mock()
        client.get.side_effect = [
            {"status": 200, "body": {"price": 100}},
            {"status": 200, "body": {"price": 250}},
        ]
        self.assertEqual(total(client, ["a", "b"]), 350)
        self.assertEqual(client.get.call_count, 2)
        client.get.assert_any_call("/price/a")
    def test_error(self):
        client = mock.Mock()
        client.get.return_value = {"status": 500, "body": {}}
        with self.assertRaises(ValueError):
            fetch_price(client, "x")

class FakeClient:
    def __init__(self, prices): self.prices = prices
    def get(self, path):
        sku = path.rsplit("/", 1)[1]
        if sku in self.prices:
            return {"status": 200, "body": {"price": self.prices[sku]}}
        return {"status": 404, "body": {}}

class TestFake(unittest.TestCase):
    def test_total(self):
        self.assertEqual(total(FakeClient({"a": 1, "b": 2}), ["a", "b"]), 3)
    def test_missing(self):
        with self.assertRaises(ValueError):
            total(FakeClient({}), ["zzz"])
```

Real output (this file also contained the patch and sqlite tests shown later, 9 tests in all):

```
test_missing (__main__.TestFake.test_missing) ... ok
test_total (__main__.TestFake.test_total) ... ok
test_error (__main__.TestMock.test_error) ... ok
test_total_with_mock (__main__.TestMock.test_total_with_mock) ... ok
test_default (__main__.TestPatch.test_default) ... ok
test_env (__main__.TestPatch.test_env) ... ok
test_time (__main__.TestPatch.test_time) ... ok
test_add (__main__.TestRepo.test_add) ... ok
test_unique (__main__.TestRepo.test_unique) ... ok

----------------------------------------------------------------------
Ran 9 tests in 0.004s

OK
```

- **Answer** `total` returned 350 with the mock and 3 with the fake, and both error paths raised `ValueError`. No network was used.
- **Check** By hand: 100 + 250 = 350 and 1 + 2 = 3. The mock was called twice (two SKUs), which `call_count` confirmed.

- **Why it matters** The tests ran in milliseconds and would give the same answer on a plane with no internet.

Patching: replacing a name for one test

`mock.patch("module.name")` swaps a name for a mock during a test and restores it afterwards. pytest's `monkeypatch` fixture does the same job for attributes and environment variables, and `pytest-mock` gives a `mocker.patch(...)` fixture.

```
# not run here: pytest style
def test_user_from_env(monkeypatch):
    monkeypatch.setenv("APP_USER", "asha")
    assert home_name() == "asha"

def test_with_mocker(mocker):            # needs pytest-mock
    mocker.patch("time.time", return_value=1000.0)
```

The `unittest` forms, which ran in the output above, are `@mock.patch.dict(os.environ, {"APP_USER": "asha"})` and `@mock.patch("time.time", return_value=1000.0)` .

Problem 10.3.2 The famous trap: *patch where the name is used, not where it is defined.* `bill.py` does `from util import rate` . Which patch changes the result of `to_inr(2)` when the real `rate()` is 83.0?

Step 1	`from util import rate` copies the name `rate` into module `bill` . Now `bill.rate` and `util.rate` are two names for the same function.
Step 2	Patching `util.rate` changes only the `util` name. `bill` still points to the old function.
Step 3	Patch `bill.rate` instead, and run all three cases.

```
from unittest import mock
import bill, util
with mock.patch("util.rate", return_value=100.0):
    print("patch util.rate  ->", bill.to_inr(2))
with mock.patch("bill.rate", return_value=100.0):
    print("patch bill.rate  ->", bill.to_inr(2))
print("no patch         ->", bill.to_inr(2))
```

```
patch util.rate  -> 166.0
patch bill.rate  -> 200.0
no patch         -> 166.0
```

- **Answer** Only `bill.rate` works (result 200.0 = 2 × 100). Patching `util.rate` left the answer at 166.0 = 2 × 83.
- **Check** 2 × 83.0 = 166.0 matches the "no patch" line, so the first patch changed nothing.
- **Why it matters** This is the most common reason a patch "does not work". A plain-language rule: patch the name in the module that *looks it up*.

Before you run it, predict the first line. Most beginners guess 200.0 and get 166.0, which is the typical wrong version.

Safer mocks with a spec

A bare `Mock()` accepts any attribute, so a typo can pass silently. Give it a `spec` so only real attributes exist. Modern Python also refuses misspelled assert helpers.

```
from unittest import mock
m2 = mock.Mock(spec=["get"])
try:
    m2.post()
except AttributeError as e:
    print("AttributeError:", e)
m3 = mock.Mock()
m3.assert_called_once_wiht()
```

```
AttributeError: Mock object has no attribute 'post'
...
AttributeError: 'assert_called_once_wiht' is not a valid assertion. Use a spec for the mock if 'assert_called_once_wiht'
is meant to be an attribute.. Did you mean: 'assert_called_once_with'?
```

`mock.create_autospec(real_object)` copies the real signature, so wrong argument counts also fail.

Database tests with in-memory SQLite

`sqlite3.connect(":memory:")` makes a database in RAM that vanishes when closed. It is perfect for fast integration tests of your data-access code. (It is not Postgres, so dialect-specific features need a real Postgres in CI, for example via Docker.)

Problem 10.3.3 Test a `Repo` class that adds users and rejects duplicates.

Step 1	In `setUp`, open a fresh in-memory DB and create the table. In `tearDown`, close it.
Step 2	Test insertion order and the `UNIQUE` rule.
Step 3	Notice the code uses `?` placeholders. This is also the defence against SQL injection (Chapter 13).

```
import sqlite3, unittest
class Repo:
    def __init__(self, conn):
        self.conn = conn
        conn.execute("CREATE TABLE users(id INTEGER PRIMARY KEY, name TEXT UNIQUE)")
    def add(self, name):
        cur = self.conn.execute("INSERT INTO users(name) VALUES (?)", (name,))
        return cur.lastrowid
    def names(self):
        return [r[0] for r in self.conn.execute("SELECT name FROM users ORDER BY id")]
class TestRepo(unittest.TestCase):
    def setUp(self):
        self.conn = sqlite3.connect(":memory:")
        self.repo = Repo(self.conn)
    def tearDown(self):
        self.conn.close()
    def test_add(self):
        self.repo.add("ravi"); self.repo.add("meena")
        self.assertEqual(self.repo.names(), ["ravi", "meena"])
    def test_unique(self):
        self.repo.add("ravi")
        with self.assertRaises(sqlite3.IntegrityError):
            self.repo.add("ravi")
```

- **Answer** Both tests passed (see the 9-test run above). Each test got a brand-new database, so they cannot disturb each other.
- **Check** `test_unique` inserts `"ravi"` twice and `sqlite3.IntegrityError` is raised, which proves the `UNIQUE` rule is active.
- **Why it matters** Your SQL code is tested against a real engine, not a guess.

The picture

```
Real run:    your code --> HTTP client --> internet --> price service
Unit test:   your code --> Mock/Fake  (answers you chose, instantly)
Integration: your code --> real SQLite (in memory)
```

Connection to the real job

- Pipelines call APIs, clouds and databases. You mock the call, test your logic, and keep test runs fast and free.
- Clock-dependent code (daily jobs) is tested by passing the date in as an argument or by patching the clock.
- ML code: replace the slow model with a fake that returns fixed scores to test the surrounding service.

Applications

- **Data Analytics:** test a report function with a tiny in-memory table.
- **Data Engineering:** mock a cloud storage client to test "upload then log" logic.
- **Machine Learning:** fake model returning fixed scores to test the code around it.
- **GenAI / MLOps:** stub the LLM API so tests are free, fast and repeatable.

COMMON MISTAKE	Over-mocking. If the test mocks everything, it only checks that your code calls your mocks. It will pass even when the real thing is broken. Mock only the boundary (network, clock, disk), and keep a few integration tests.

COMMON MISTAKE	Patching the wrong name (Problem 10.3.2). If a patch seems to do nothing, check where the name is imported.

Check yourself

1. Fake or mock: which one has real simple behaviour? Answer: the fake.
2. Where do you patch after `from util import rate` in `bill.py`? Answer: `bill.rate`.
3. Why is `:memory:` SQLite good for tests? Answer: it is fast, needs no setup, and disappears on close, so tests stay isolated.

10.4 TDD and Coverage

INTUITION	A good builder first writes the inspection checklist, then builds until the checklist passes. **Test-Driven Development (TDD)** works like this. You write a failing test, then the smallest code that passes it, then clean up.

The red-green-refactor loop

1. **Red**: write one small test for behaviour that does not exist yet. Run it. It must fail. (A test that was never red may test nothing.)
2. **Green**: write the simplest code that makes it pass. Do not be clever.
3. **Refactor**: clean the code (names, duplication) while the tests stay green.
4. Repeat with the next small behaviour.

TDD is not required everywhere. It shines for logic with clear rules (parsers, calculations, validators). It is awkward for exploratory data work in notebooks. Also, when you find a bug, write a failing test that reproduces it first, then fix it. That bug can never come back silently.

Worked walkthrough: FizzBuzz

Problem 10.4.1 Build `fizzbuzz(n)` by TDD: `3` gives `"Fizz"`, `5` gives `"Buzz"`, `15` gives `"FizzBuzz"`, others give the number as text.

Step 1 (Red)	Write four tests and a stub that raises `NotImplementedError`. Run.
Step 2 (Green, part 1)	Handle only the multiples of 3. Run again and see the remaining failures.
Step 3 (Green and refactor)	Build the text from parts, and fall back to the number. Run until green.

```
# test_fizz.py
import unittest
from fizz import fizzbuzz
class T(unittest.TestCase):
    def test_plain(self): self.assertEqual(fizzbuzz(1), "1")
    def test_fizz(self): self.assertEqual(fizzbuzz(3), "Fizz")
    def test_buzz(self): self.assertEqual(fizzbuzz(5), "Buzz")
    def test_both(self): self.assertEqual(fizzbuzz(15), "FizzBuzz")
```

Red: `fizz.py` is `def fizzbuzz(n): raise NotImplementedError`.

```
  File "/private/tmp/w10/tdd/fizz.py", line 2, in fizzbuzz
    raise NotImplementedError
NotImplementedError

----------------------------------------------------------------------
Ran 4 tests in 0.002s

FAILED (errors=4)
```

Step 2: only `if n % 3 == 0: return "Fizz"` then `return str(n)`.

```
AssertionError: '5' != 'Buzz'
- 5
+ Buzz

----------------------------------------------------------------------
Ran 4 tests in 0.002s

FAILED (failures=2)
```

Two failures remain: `5` and `15`. Final code:

```
def fizzbuzz(n):
    out = ""
    if n % 3 == 0:
        out += "Fizz"
    if n % 5 == 0:
        out += "Buzz"
    return out or str(n)
```

```
....
----------------------------------------------------------------------
Ran 4 tests in 0.000s

OK
```

- **Answer** Four errors, then two failures, then green. The 4 errors in step 1 were "not implemented" errors, and in step 2 two tests turned green and two became real assertion failures. The test list guided every step.
- **Check** By hand: 3 → Fizz, 5 → Buzz, 15 is divisible by both so `"Fizz" + "Buzz"`, 1 → `"1"`. All four match the final code.
- **Why it matters** At no point did you write code without a failing test asking for it, so no code is untested.

Coverage: which lines did the tests run?

Code coverage is the percent of your code lines that ran during the tests. With pytest you use the `pytest-cov` plugin:

```
# not run here
pip install pytest-cov
pytest --cov=mypackage --cov-report=term-missing
```

`term-missing` lists the line numbers no test touched. The standard library has a rough equivalent, `trace`, which we can run.

Problem 10.4.2 `grade(score)` returns A for 90+, B for 75+, else C. Two tests only: 95 and 10. Which line is never executed?

Step 1	Write `cov.py` and `test_cov.py` with the two tests.
Step 2	Run the tests under `python3 -m trace --count --missing -C covout --module unittest test_cov`.
Step 3	Read `covout/cov.cover`. A line starting with `>>>>>>` never ran.

```
    1: def grade(score):
    2:     if score >= 90:
    1:         return "A"
    1:     if score >= 75:
>>>>>>         return "B"
    1:     return "C"
```

- **Answer** The line `return "B"` never ran, because no test used a score between 75 and 89. Add `test_b` with 80 and a boundary test with 75 and 74.
- **Check** 95 takes the first branch and 10 skips both `if`s. Neither can reach `return "B"`, so the report is right.
- **Why it matters** In plain words: 4 of 5 executable lines were tested, and the missing one is a real business rule.

Before you run it, predict which line is marked. Then compare with the report above.

Item	Example	Where it appears in the job
Line coverage	4 of 5 lines ran = 80%	CI quality gate
Branch coverage	Both sides of each `if` ran	Finding untested error paths
Regression test	Test written from a bug report	Support-ticket fixes

What coverage does not tell you

100% coverage means every line ran. It does not mean every line was *checked*. A test with no assert gives coverage and no safety. Use coverage to find untested areas, not as a score to chase. Many teams set a floor (for example 80%) and fail CI below it. Branch coverage (`--cov-branch`) also checks that both sides of every `if` ran.

The picture

```
  RED ----------> GREEN ----------> REFACTOR
(failing test)   (smallest code)    (clean, tests stay green)
    ^                                    |
    +-------------- next behaviour --------+
```

Connection to the real job

- When a production bug is reported, the professional move is: write the failing test, fix, keep the test. Over a year this builds a safety net.
- Data validators (is this row valid?) are ideal for TDD: rules are explicit.
- Performance work (Chapter 11) is safe only if tests prove the faster code gives the same answers.

Applications

- **Data Analytics:** validators for report rules (tax, discount, billing).
- **Data Engineering:** parsers for messy log or CSV formats, built one rule at a time.
- **Machine Learning:** a failing test that reproduces a data-leak bug before you fix it.
- **MLOps:** a safety net that lets you refactor a legacy pipeline without fear.

COMMON MISTAKE

Writing the test after the code and copying the code's current output as the "expected" value. If the code is wrong, the test now locks in the wrong answer. Work out the expected value by hand first.

COMMON MISTAKE

Chasing 100% coverage with assert-free tests. Coverage measures execution, not correctness.

Check yourself

1. What do you do in the Red step? Answer: write a small test for missing behaviour and see it fail.
2. A line shows `>>>>>>` in a trace report. What does it mean? Answer: no test executed that line.
3. Can 100% coverage still hide bugs? Answer: yes, if tests do not assert the results.

10.5 Code Quality: Formatters, Linters, Type Checkers and pre-commit

INTUITION

In a team, nobody wants to argue about spaces versus quotes in a code review. Let machines settle style, so humans discuss design. A formatter rewrites style, a linter flags suspicious code, a type checker flags wrong types, and pre-commit runs all three before each commit.

The four tools

Tool	Job	Changes your code?	Typical command
`black`	Formats code in one fixed style	Yes	`black .`
`ruff`	Very fast linter (and formatter, `ruff format`)	Lint: with `--fix`	`ruff check .`
`mypy`	Checks type hints (Chapter 9)	No	`mypy src`
`pre-commit`	Runs checks automatically at `git commit`	Depends on hooks	`pre-commit install`

All four are third-party (`pip install`), **not run here**. Many teams now use `ruff` for both lint and format and drop `black`. Either is fine; the point is one agreed tool.

Configuration: one place, `pyproject.toml`

```
# pyproject.toml   (not run here; standard keys)
[tool.black]
line-length = 88

[tool.ruff]
line-length = 88

[tool.ruff.lint]
select = ["E", "F", "I", "B"]   # pycodestyle errors, pyflakes, import order, bugbear

[tool.mypy]
python_version = "3.12"
strict = true
```

`F` rules find unused imports and undefined names. `I` sorts imports. `B` (bugbear) catches classic traps such as the mutable default argument. The set of rule letters can change between versions, so check the tool's documentation.

pre-commit

`pre-commit` reads `.pre-commit-config.yaml`. Each hook runs on the files you are about to commit. If a hook fails or edits a file, the commit stops, you re-add the files and commit again.

```
# .pre-commit-config.yaml   (not run here)
repos:
  - repo: https://github.com/astral-sh/ruff-pre-commit
    rev: vX.Y.Z          # put the current release tag here
    hooks:
      - id: ruff
        args: [--fix]
      - id: ruff-format
  - repo: https://github.com/pre-commit/mirrors-mypy
    rev: vX.Y.Z          # put the current release tag here
    hooks:
      - id: mypy
```

```
# not run here
pip install pre-commit
pre-commit install          # sets up the git hook once
pre-commit run --all-files  # check the whole repo now
```

Item	Example	Where it appears in the job
Formatter	`black .` rewrites quotes and spacing	Zero style arguments in review
Linter	`ruff check .` flags an unused import	Catching bugs before running
Type checker	`mypy src` flags `double("5")`	Data code with wrong column types
pre-commit	Runs hooks at `git commit`	Every team repo

What these tools catch

Here is the kind of bug a linter finds, and how running the code shows it.

Problem 10.5.1 The code below has the "mutable default" bug. Show the bug, then fix it. A bugbear rule (`B006`) would flag the first version.

Step 1	Run the buggy function twice.
Step 2	Fix with `None` as default.

```
def add_tag(tag, tags=[]):
    tags.append(tag)
    return tags
print(add_tag("a"), add_tag("b"))

def add_tag_ok(tag, tags=None):
    if tags is None:
        tags = []
    tags.append(tag)
    return tags
print(add_tag_ok("a"), add_tag_ok("b"))
```

```
['a', 'b'] ['a', 'b']
['a'] ['b']
```

- **Answer** The default list is created once and shared, so the first call's data leaks into the second. Both prints in the buggy version show the same list object. The fix gives `['a'] ['b']` .
- **Check** The buggy line prints `['a', 'b']` twice because Python evaluates both calls first, and both returned the same list, which by then held both tags.
- **Why it matters** A linter finds this in a second. Without it, the bug appears as data from one user showing up in another's result.

Problem 10.5.2 What does `mypy` say about this code? (Not run here; this is the standard message shape.)

Step 1	Read the hint: `def double(x: int) -> int` .
Step 2	The call `double("5")` passes a `str` .
Step 3	mypy reports it before the program runs.

```
def double(x: int) -> int:
    return x * 2

double("5")
```

Expected report (wording varies by mypy version): `error: Argument 1 to "double" has incompatible type "str"; expected "int"` .

- **Answer** mypy flags the call without running it. At runtime `"5" * 2` would silently give `"55"` , a wrong answer with no error. That is why type checking helps.
- **Check** In plain Python, `"5" * 2` is `"55"` (string repeat). So the program would not crash, it would be quietly wrong.
- **Why it matters** The worst bugs are the ones that do not raise errors.

The picture

```
you edit --> git commit --> pre-commit hooks --> pass? --> commit saved
                              |  ruff-format
                              |  ruff check
                              |  mypy              fail --> fix, re-add
```

Then CI (Chapter 13) runs the same tools again on the server, so nothing slips past a skipped local hook.

Connection to the real job

- Every professional repo has a formatter, a linter, and usually a type checker in CI. A pull request that fails them is not reviewed.
- Notebooks can be linted too (`ruff` supports `.ipynb`); convert long notebooks to scripts as they mature.
- Type hints plus `mypy` catch many "column is a string, not int" bugs in data code early.

Applications

- **Data Analytics:** `ruff` on notebooks exported to scripts, so shared analysis code stays tidy.
- **Data Engineering:** pre-commit plus CI on every pipeline repository.
- **Machine Learning:** `mypy` catching wrong tensor/DataFrame argument types in training code.
- **MLOps:** hooks that block secrets and debug prints from reaching the main branch.

COMMON MISTAKE	Turning on `strict` mypy and every lint rule in an old codebase on day one. You get 2,000 errors and give up. Start with a small rule set on new code, then widen it.

COMMON MISTAKE	Skipping hooks with `git commit --no-verify` "just this once". CI will fail anyway, and now it fails in public.

Check yourself

1. What is the difference between a formatter and a linter? Answer: a formatter rewrites style only; a linter flags suspicious or wrong code.
2. When does `pre-commit` run? Answer: automatically at `git commit`, once you ran `pre-commit install`.
3. Does `mypy` run your program? Answer: no, it only reads the code and the type hints.

10.6 Property-Based Tests, Fake Clocks, Mocks and Flaky Tests

Example-based tests check the cases you thought of. Bugs live in the cases you did not. This section shows four techniques that find those bugs: property-based testing, fake clocks, careful use of mocks, and a method for taming flaky tests. Everything runs with the standard library only. The popular library for property-based testing is Hypothesis; it is not installed here, so its API is *not run here*, and we build the core idea by hand in 15 lines so you see exactly how it works.

Property-based testing: test rules, not examples

A **property** is a rule that must hold for every valid input. For a function that splits a list into chunks, two rules are: the chunks joined together give back the original list, and every chunk has between 1 and `size` items. A property-based tool generates hundreds of random inputs, checks the rules, and when one fails, **shrinks** the input: it repeatedly removes pieces while the failure persists, so you see the smallest failing case, not a confusing 9-item list.

Problem 10.6.1 The function below should split a list into chunks. Find its off-by-one bug with a seeded random search and shrink the failing input.

Property	Flattened chunks equal the input, and every chunk length is in `1..size`.
Generate	A seeded `random.Random(42)` picks a size from 1 to 4 and a list of 0 to 10 digits. The seed makes the failure reproducible.
Search	Run up to 200 trials and stop at the first failure.
Shrink	Try deleting each element. If the property still fails, keep the smaller list. Repeat until nothing can be deleted.

```
import random

def chunk(items, size):
    out = []
    for i in range(0, len(items) - 1, size):   # bug: should be len(items)
        out.append(items[i:i + size])
    return out

def check(items, size):
    pieces = chunk(items, size)
    flat = [x for p in pieces for x in p]
    return flat == items and all(1 <= len(p) <= size for p in pieces)

def shrink(items, size):
    changed = True
    while changed:
        changed = False
        for i in range(len(items)):
            smaller = items[:i] + items[i + 1:]
            if not check(smaller, size):
                items, changed = smaller, True
                break
    return items

rng = random.Random(42)
for trial in range(1, 201):
    size = rng.randint(1, 4)
    items = [rng.randint(0, 9) for _ in range(rng.randint(0, 10))]
    if not check(items, size):
        print("failed on trial", trial, "size", size, "items", items)
        print("shrunk to", shrink(items, size), "size", size)
        break
```

```
failed on trial 3 size 1 items [6, 0, 0, 1, 3, 3, 8, 9, 0]
shrunk to [0] size 1
```

The shrunk case says it all: a one-item list is lost. With `len(items) - 1` as the stop value, the loop never reaches the final start index when the last chunk starts at the last item. The fix is `range(0, len(items), size)`. In Hypothesis the same test is a decorated function (not run here): `@given(st.lists(st.integers()), st.integers(1, 5))` above a test that asserts the two rules. Hypothesis also remembers past failures in a local database, so a bug found once is rechecked on every later run.

COMMON MISTAKE

Writing a property that repeats the implementation, for example asserting `chunk(x, 2) == [x[i:i+2] for i in range(0, len(x), 2)]`. If the code and the test share a bug, both agree and nothing fails. State rules that are *different in kind* from the code: round trips (decode of encode is the original), invariants (length, order, sum), and comparison with a slow obvious version.

Fake clocks: test waiting without waiting

Retry code sleeps between attempts. A test that really sleeps is slow, and a test that depends on the real clock is flaky. Pass the clock in as an argument (dependency injection, section 8.7) and give tests a fake one that records requested sleeps and advances a counter instead of waiting.

Problem 10.6.2 Test that `retry` uses exponential back-off 1, 2, 4 seconds for a function that fails three times, without waiting.

Step 1	`FakeClock` has `sleep(s)`, which appends `s` to a list and adds it to `now`.
Step 2	`retry` sleeps `base * 2**attempt` after failed attempts 0, 1, 2. After 3 failures the fourth call succeeds.
Step 3	Assert the sleep list is `[1.0, 2.0, 4.0]`, total 7.0 virtual seconds.
Check	The real elapsed time is tiny.

```
import time

class FakeClock:
    def __init__(self):
        self.now = 0.0
        self.sleeps = []
    def sleep(self, s):
        self.sleeps.append(s)
        self.now += s

def retry(fn, tries, base, clock):
    for attempt in range(tries):
        try:
            return fn()
        except ConnectionError:
            if attempt == tries - 1:
                raise
            clock.sleep(base * 2 ** attempt)

calls = {"n": 0}
def flaky():
    calls["n"] += 1
    if calls["n"] < 4:
        raise ConnectionError
    return "ok"

clock = FakeClock()
t0 = time.perf_counter()
print(retry(flaky, 5, 1.0, clock), clock.sleeps, clock.now)
print("real time under 0.1 s:", time.perf_counter() - t0 < 0.1)
```

```
ok [1.0, 2.0, 4.0] 7.0
real time under 0.1 s: True
```

Mocks and patches, used sparingly

Three techniques, from safest to riskiest. (1) Pass the time or the dependency as a parameter, so no patching is needed. (2) Use `unittest.mock.Mock` as a stand-in object whose answers you script. (3) Use `mock.patch("module.name")` to replace a name in place, which is a last resort because you must patch the name *where it is looked up*.

Problem 10.6.3 Test a greeting that depends on the hour, a dice roll, and an HTTP client, with no real time, randomness or network.

Time	Give `greeting` an optional `now` parameter. Tests pass a fixed `datetime`.
Randomness	`mock.patch("random.randint", return_value=4)` replaces the function for the `with` block, and records how it was called.
Client	A `Mock` client: `client.get.return_value.status = 500` scripts one answer. `side_effect` with a list scripts a sequence.

```
from unittest import mock
import datetime, random

def greeting(now=None):
    now = now or datetime.datetime.now()
    return "good morning" if now.hour < 12 else "good afternoon"

print(greeting(datetime.datetime(2026, 1, 1, 9)))
print(greeting(datetime.datetime(2026, 1, 1, 15)))

def roll():
    return random.randint(1, 6)

with mock.patch("random.randint", return_value=4) as m:
    print(roll(), m.call_args)

def fetch(client, url):
    r = client.get(url)
    return r.status if r.status == 200 else None

client = mock.Mock()
client.get.return_value.status = 500
print(fetch(client, "http://x"), client.get.call_count)
client.get.side_effect = [mock.Mock(status=200)]
print(fetch(client, "http://x"))
```

```
good morning
good afternoon
4 call(1, 6)
None 1
200
```

COMMON MISTAKE

Patching the wrong name. If `app.py` says `from random import randint`, then `mock.patch("random.randint")` changes the original but `app.randint` still points at the real function. Patch `"app.randint"`. Rule: patch where the name is used, not where it is defined.

Triage a flaky test

A flaky test passes and fails with no code change. Do not re-run until green. Find the cause class first.

Symptom	Likely cause	Fix
Fails only on certain days or at midnight	Real clock or time zone	Inject the clock, fix the date
Fails when tests run in a different order	Shared state between tests (global, file, database)	Fixtures with fresh state, `tmp_path`
Fails about 1 time in 20, more under load	Real sleeps, threads, race condition	Wait for a condition with a timeout, not a fixed sleep
Fails with random data	Unseeded random numbers	Seed and print the seed on failure
Fails only in CI	Different environment, missing network	Pin versions, fake the network

Going deeper

- **Shrinking is why property tests are usable.** Raw random failures are hard to read. Hypothesis shrinks integers toward 0, lists toward empty and strings toward short ones, using the same delete-and-recheck loop shown above, only smarter.
- **Mutation testing** (tools such as mutmut, not run here) changes your code on purpose, for example `<` to `<=`, and checks that some test fails. A mutant that survives points at a missing test. It answers the question coverage cannot: do your assertions actually check anything?

- **Golden-file (snapshot) tests** save a known-good output to a file and compare against it. They suit report generators, but review every snapshot update by hand or they become rubber stamps.
- **Async tests** need a runner plugin such as pytest-asyncio (not run here), or you can call `asyncio.run(...)` inside an ordinary test, as Chapter 11 does.

More practice: advanced testing

Problem 10.P.12 (warm-up) Name two properties of `sorted(xs)` that hold for any list of numbers.

Answer: (1) The output has the same length and the same items as the input (the same multiset). (2) Each item is less than or equal to the next one. Together they define sorting: only the first would also accept the unchanged list, and only the second would also accept an empty list.

Problem 10.P.13 (build) Using a seeded `random.Random(1)`, write a loop that checks 100 random lists satisfy `sorted(xs) == sorted(sorted(xs))` (sorting twice changes nothing). Print the number of trials that passed.

```
import random

rng = random.Random(1)
passed = 0
for _ in range(100):
    xs = [rng.randint(-50, 50) for _ in range(rng.randint(0, 12))]
    if sorted(xs) == sorted(sorted(xs)):
        passed += 1
print(passed)
```

```
100
```

Answer: 100. This property is called idempotence. It is weak on its own (a broken sort that always returned the empty list would also pass), so combine it with the two properties from the previous problem.

Problem 10.P.14 (find the bug) A test patches `"time.sleep"` but the code under test did `from time import sleep`, and the test still takes 5 seconds. Why, and what is the fix?

Answer: The module under test holds its own reference to the real `sleep`, so patching `time.sleep` does not touch it. Patch `"mymodule.sleep"`. Better: pass a `sleep` function in as a parameter and use a fake.

Problem 10.P.15 (stretch) Extend the fake clock test: with `tries=3` and a function that always raises `ConnectionError`, what exception does `retry` raise and what is `clock.sleeps`?

Answer: It re-raises the last `ConnectionError` after the third failure. Sleeps happen only after attempts 0 and 1, so `clock.sleeps == [1.0, 2.0]` with base 1.0. There is no sleep after the final failure, which is a common off-by-one in hand-written retry code.

Problem 10.P.16 (interview) A test passes locally and fails in CI one run in ten. Give your first four debugging steps.

Answer: (1) Re-run that test alone many times to confirm it is flaky and not an order effect, then run the whole suite in random order. (2) Check for real time, sleeps, threads and unseeded randomness. (3) Print the seed and environment details on failure so the failure can be replayed. (4) If you cannot fix it today, quarantine it with a ticket and a deadline instead of deleting it or ignoring red builds.

Chapter recap

- A test calls your code with known input and checks the output. Tests catch regressions fast.
- The testing pyramid: many unit tests, fewer integration tests, very few end-to-end tests.
- Test normal, edge and bad cases. Use small fake data, never real customer data.
- pytest uses plain `assert`, fixtures (with `yield` for clean-up), `parametrize`, `tmp_path` and `monkeypatch`. `unittest` is the built-in alternative.

- A fake has simple real behaviour; a mock records calls. Prefer fakes, mock only boundaries.
- Patch the name where it is used (`bill.rate`), not where it is defined.
- In-memory SQLite gives fast, isolated database tests.
- TDD is red, green, refactor. Reproduce every bug with a failing test before fixing it.
- Coverage shows which lines ran, not whether results were checked.
- `black` / `ruff` for style and lint, `mypy` for types, `pre-commit` to run them before each commit, config in `pyproject.toml`.

Cheat-sheet

Task	Command or pattern
Run unittest	`python3 -m unittest discover -v`
Run pytest, stop at first failure	`pytest -x -q`
Many cases, one test	`@pytest.mark.parametrize("a,want", [...])`
Fixture with clean-up	`@pytest.fixture` + `yield`
Replace a name	`mock.patch("module_where_used.name")`
Safer mock	`mock.Mock(spec=[...])` or `create_autospec`
Coverage with missing lines	`pytest --cov=pkg --cov-report=term-missing`
Format / lint / types	`ruff format .` / `ruff check .` / `mypy src`
Install hooks	`pre-commit install`

Practice set

Problem 10.P.1 (easy) What is the output of `slugify(" Python, 101! ")` with the function from 10.1?

Answer: `python-101`. Strip and lower give `python, 101!`, the regex turns `", "` into `-` and `!` into `-`, and the trailing `-` is trimmed.

Problem 10.P.2 (easy) Which pyramid level should have the most tests, and why?

Answer: unit tests, because they are fast, cheap, and point to the exact failing function.

Problem 10.P.3 (easy) In `grade(score)` from 10.4 (A at 90 and above, B at 75 and above), what does `grade(75)` return, and which boundary tests should you add?

Answer: `"B"`. Add tests for 90 (A), 89 (B), 75 (B), 74 (C).

Problem 10.P.4 (medium) Write a pytest `parametrize` test for `fizzbuzz` with the cases 1, 3, 5, 15.

Answer: (not run here)

```
import pytest
from fizz import fizzbuzz

@pytest.mark.parametrize("n, want", [(1, "1"), (3, "Fizz"), (5, "Buzz"), (15, "FizzBuzz")])
def test_fizzbuzz(n, want):
    assert fizzbuzz(n) == want
```

Problem 10.P.5 (medium, what is the output?) With `bill.py` doing `from util import rate` and `util.rate()` returning 83.0, what does `to_inr(2)` return inside `mock.patch("util.rate", return_value=100.0)`?

Answer: `166.0`. The patch changed `util.rate`, but `bill` kept its own reference to the old function.

Problem 10.P.6 (medium, find the bug) A test says `self.assertEqual(0.1 * 3, 0.3)`. It fails. Why, and how do you fix it?

Answer: `0.1 * 3` is `0.30000000000000004` in binary floating point. Use `assertAlmostEqual(0.1 * 3, 0.3)` or, in pytest, `pytest.approx(0.3)`.

Problem 10.P.7 (medium, find the bug) This fixture is meant to give each test a clean list, but tests leak data. What is wrong?

```
# not run here
import pytest
shared = []

@pytest.fixture
def items():
    return shared
```

Answer: it returns the same module-level list every time, so tests mutate shared state. Create a new list inside the fixture: `return []`.

Problem 10.P.8 (medium) A trace report shows `>>>>>>` on the line `raise ValueError`. What test do you write?

Answer: a test that triggers the error path, wrapped in `with self.assertRaises(ValueError):` (or `pytest.raises(ValueError)`), using an input that makes the condition true.

Problem 10.P.9 (hard, interview) Explain the difference between a mock, a stub and a fake, and when you would choose each.

Answer: a stub returns fixed answers. A fake is a lightweight working version (an in-memory dictionary instead of a database). A mock records how it was called so you can assert on interactions. Choose a fake when behaviour is simple and you want to test results, a stub when you only need a canned answer, and a mock when the call itself is the outcome (for example "the email was sent once"). Avoid mocking everything, because then the test checks your mocks, not your code.

Problem 10.P.10 (hard, interview) A teammate says "we have 95% coverage so the code is safe". Reply in three sentences.

Answer: Coverage only says those lines executed, not that the results were asserted. Tests without assertions, or tests that only cover the happy path, still give high coverage. I would check assertion quality, edge cases and branch coverage, and look at which 5% is untested, because it may be the risky error handling.

Problem 10.P.11 (hard, scenario) Your nightly pipeline function `build_report(today)` reads the system date inside and your test fails every Sunday. Redesign it so it is testable.

Answer: pass the date in as an argument (`build_report(today)`), and let the caller supply `date.today()`. The test then passes a fixed date such as `date(2026, 11, 8)`. If the code cannot change yet, patch the clock for the test. Passing values in beats patching.

Prompt pack

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data, ML and AI roles, and English is my second language, so use simple words. Teach me Python unit testing with pytest: assert, fixtures with yield, parametrize, tmp_path and monkeypatch. Explain step by step with small examples using real numbers, then ask me one question at a time to check my understanding. Wait for my answer before continuing, and correct me kindly. Relate ideas to everyday examples such as shop orders where it helps.
```

✨ AI prompt: copy and paste into any AI

You are my patient, practical professor. I am a working professional moving into data, ML and AI roles, and English is my second language. The topic is mocks, fakes and stubs in Python (unittest.mock, patch, side_effect, autospec) and the rule "patch where the name is used". Give me 8 graded exercises from easy to hard, each with a small code snippet and a clear goal. Do not give solutions yet. After I reply with my answers, review them one by one and explain every mistake in simple English.

✨ AI prompt: copy and paste into any AI

You are my patient, practical professor. I am a working professional moving into data, ML and AI roles. Chapter topic: Test-Driven Development (red, green, refactor). Pick a small data-cleaning problem for me, such as normalising phone numbers. Walk me through TDD for it, one tiny step at a time. At each step show only the next failing test and wait for me to write the code. Review my code, point out edge cases I missed, and finish with a summary of what I learned.

✨ AI prompt: copy and paste into any AI

You are my patient, practical professor. I am a working professional moving into data, ML and AI roles, and English is my second language. Please review my Python tests and my code quality setup. Here is my code and tests: [PASTE HERE]. Check for missing edge cases, tests that depend on each other, over-mocking, float comparison mistakes, and weak assertions. Then suggest a pyproject.toml configuration for ruff and mypy and a .pre-commit-config.yaml, and explain each setting in simple English.

✨ AI prompt: copy and paste into any AI

You are my patient, practical professor. I am a working professional moving into data, ML and AI roles. Run a mock interview on Python testing and code quality for a data engineer job. Ask me one question at a time: the testing pyramid, fixtures, mocking versus faking, coverage limits, TDD, linters, type checkers and pre-commit. Grade each answer from 1 to 5, show a model answer, and give one tip to improve. After 10 questions, tell me my weakest areas and what to practise next.

CHAPTER 11

CONCURRENCY AND PERFORMANCE

Measure first, then make it faster: threads, processes, asyncio, profiling and vectorization.

Your script takes 40 minutes to call 2,000 URLs, and a data job that should finish overnight is still running at breakfast. Every programmer learns the golden rule: do not guess where the time goes, measure it. In this chapter you will learn to measure Python code (`timeit`, `cProfile`, `tracemalloc`), to understand why Python threads do not speed up some jobs (the GIL), and to choose between threads, processes and `asyncio`. You will also see why a loop in pure Python is slow, and connect all this to Little's law, a formula from queueing theory. All the timings below are real runs on one laptop (12 CPU cores, Python 3.14). Your numbers will differ, but the *shapes* will match.

> **A note on what was run.** Only the Python 3.14 standard library was available, with no network. `numpy`, `pandas`, `numba`, `line_profiler` and `memory_profiler` were **not run here**. Their snippets are standard usage and marked as such.

11.1 Measure First: timeit, cProfile and tracemalloc

INTUITION	A doctor does not operate before the scan. Slow code is the same: the slow part is almost never where you think. You first measure the whole, then find the one hot spot, then fix only that. In code terms: first time the whole script, then drill into the slowest function.

Words you need

- **Wall-clock time**: time as measured by a clock on the wall (what the user waits).
- **Profiler**: a tool that records which functions ran and how long each took.
- **Hot spot**: the small part of the code where most time is spent.
- **Benchmark**: a repeatable timing of a small piece of code.
- **Call**: one execution of a function; `ncalls` is how many times it ran.

timeit: time a small snippet fairly

`timeit` runs a snippet many times and reports the total, which smooths out noise. Use it to *compare two ways* of doing the same small job.

Problem 11.1.1 Compare four ways to compute the sum of squares of 1,000,000 numbers. Predict first: which is fastest, a plain `for` loop or a list comprehension?

Step 1	Write each version as a string. Run each 3 times and divide by 3.
Step 2	Print the average in milliseconds.

```
import timeit
N=1_000_000
t1=timeit.timeit("s=0\nfor i in range(N): s+=i*i", globals=globals(), number=3)/3
t2=timeit.timeit("sum([i*i for i in range(N)])", globals=globals(), number=3)/3
t3=timeit.timeit("sum(i*i for i in range(N))", globals=globals(), number=3)/3
t4=timeit.timeit("sum(map(int.__mul__, r, r))", setup="r=range(N)", globals=globals(), number=3)/3
for n,t in [("for loop",t1),("list comp + sum",t2),("generator + sum",t3),("map(int.__mul__)",t4)]:
    print(f"{n:<20}{t*1000:8.1f} ms")
```

```
for loop               27.2 ms
list comp + sum        26.3 ms
generator + sum        28.8 ms
map(int.__mul__)       45.7 ms
```

- **Answer** The first three are within about 10% of each other (26 to 29 ms). The "clever" `map(int.__mul__)` was the slowest at 45.7 ms.
- **Check** 1,000,000 items in 27 ms means about 27 nanoseconds per item, which is the right order of magnitude for interpreted Python.
- **Why it matters** In plain words: style tricks change speed by percentages. They do not change it by 100 times. Big wins come from a better algorithm or data structure, as the next example shows.

Problem 11.1.2 Is `x in my_list` slow for a list of 100,000 numbers? Compare with a `set`.

Step 1	Build a list and a set of `range(100000)`.
Step 2	Search for `99999`, the worst case for a list, 200 times each.

```
import timeit
print(timeit.timeit("99999 in L", setup="L=list(range(100000))", number=200), "s list")
print(timeit.timeit("99999 in S", setup="S=set(range(100000))", number=200), "s set")
```

```
0.08791520900012983 s list
4.417000127432402e-06 s set
```

- **Answer** The list took 0.0879 s for 200 searches (0.44 ms each). The set took 0.0000044 s (22 nanoseconds each).
- **Check** 0.0879 ÷ 0.0000044 is about 19,900. A list checks items one by one (up to 100,000 steps). A set jumps straight to the right slot using a hash (Chapter 4), about 1 step.
- **Why it matters** Switching a data structure made the code about 20,000 times faster. No formatting trick can do that.

cProfile: find the hot spot

`cProfile` records every function call. You sort the report by **cumulative time** (time in the function *including* the functions it calls) or **tottime** (time in the function itself only).

Problem 11.1.3 Profile a `main()` that calls `slow_sum(300_000)` and a recursive `fib(22)`. Which is the hot spot?

Step 1	Create a `Profile`, enable it, run `main`, disable it.
Step 2	Print the top 5 rows sorted by cumulative time.

```
import cProfile, pstats, io
def slow_sum(n):
    return sum([i*i for i in range(n)])
def fib(n):
    return n if n < 2 else fib(n-1)+fib(n-2)
def main():
    slow_sum(300_000)
    fib(22)
pr = cProfile.Profile(); pr.enable(); main(); pr.disable()
s = io.StringIO()
pstats.Stats(pr, stream=s).sort_stats("cumulative").print_stats(5)
print(s.getvalue())
```

```
         57317 function calls (5 primitive calls) in 0.014 seconds

   Ordered by: cumulative time

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        1    0.000    0.000    0.014    0.014 /private/tmp/w11/b.py:6(main)
        1    0.006    0.006    0.008    0.008 /private/tmp/w11/b.py:2(slow_sum)
  57313/1    0.006    0.000    0.006    0.006 /private/tmp/w11/b.py:4(fib)
        1    0.002    0.000    0.002    0.002 {built-in method builtins.sum}
        1    0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}
```

How to read it: `ncalls 57313/1` means `fib` was called 57,313 times in total, of which 1 was the outer call (the rest are recursive). `tottime` is time inside that function only. `cumtime` includes callers' children. `main` has cumulative 0.014 s and almost 0 tottime, so it is just a coordinator.

- **Answer** `slow_sum` (0.008 s cumulative) and `fib` (0.006 s) share the time. The profile also shows that `fib(22)` made 57,313 calls to compute one number, a clue that caching (`functools.lru_cache` , Chapter 4) would help.
- **Check** 0.008 + 0.006 = 0.014 s, which equals `main` 's cumulative time. Time adds up.
- **Why it matters** The profile points at the function to fix, so you stop guessing. (Profiling adds overhead, so use it to compare *relative* time, not absolute.)

From the command line you can profile a whole script: `python3 -m cProfile -s cumulative myscript.py` . The tool `line_profiler` (not run here) goes deeper: you decorate a function with `@profile` , run `kernprof -l -v script.py` , and it shows time per *line*.

tracemalloc: where does memory go?

Problem 11.1.4 How much memory does a list of 1,000,000 integers use, and how much does a generator that sums the same numbers use? Predict: is it more than 10 MB?

Step 1	Start `tracemalloc` . Build the list, read current memory.
Step 2	Sum a generator, and read the peak above the list.
Step 3	Compare `sys.getsizeof` for both objects.

```
import tracemalloc, sys
tracemalloc.start()
data = [i for i in range(1_000_000)]
cur, peak = tracemalloc.get_traced_memory()
print(f"list of 1M ints: current {cur/1e6:.1f} MB")
tracemalloc.reset_peak()
gen = (i for i in range(1_000_000))
total = sum(gen)
cur2, peak2 = tracemalloc.get_traced_memory()
print(f"generator sum peak extra: {(peak2-cur)/1e6:.4f} MB")
print(sys.getsizeof(data), sys.getsizeof(gen))
snap = tracemalloc.take_snapshot()
for st in snap.statistics("lineno")[:1]: print(st)
```

```
list of 1M ints: current 40.4 MB
generator sum peak extra: 0.0006 MB
8448728 200
/private/tmp/w11/d.py:3: size=38.6 MiB, count=999731, average=40 B
```

- **Answer** The list needs about 40 MB. The generator needed about 0.0006 MB extra, essentially nothing.
- **Check** `sys.getsizeof(data)` is 8.4 MB, which counts only the list's 8-byte pointers (1,000,000 × 8). Each integer object is a separate object of about 32 bytes, so 8 + 32 = 40 bytes per item, matching "average=40 B" and 40 MB total. `getsizeof` does not count the objects inside, so always trust `tracemalloc` for the full picture.

- **Why it matters** Generators (Chapter 7) keep memory flat, so you can stream a 50 GB file through a 200-byte object. `memory_profiler` (not run here) gives memory per line with `@profile` and `python -m memory_profiler script.py`.

The picture

```
"It is slow"
    |
    v
timeit / wall clock  --> how slow? (a number, repeatable)
    |
    v
cProfile             --> which function? (tottime, cumtime, ncalls)
    |
    v
line_profiler        --> which line?
    |
    v
fix ONE thing, measure again, keep tests green (Chapter 10)
```

Connection to the real job

- A pandas `apply` that takes 2 hours is usually one hot function. Profile before rewriting.
- ML training loops: profile the data loader before blaming the model.
- Memory errors ("Killed", `MemoryError`) are found with `tracemalloc` or by switching to generators and chunked reads.

Applications

- **Data Analytics:** find the slow cell in a notebook before buying a bigger machine.
- **Data Engineering:** find the slow transform in an ETL job; use generators to process big files.
- **Machine Learning:** profile feature code and data loading.
- **MLOps / backend:** profile an API endpoint that violates its latency target.

COMMON MISTAKE	Optimizing without measuring, or timing once. One run is noisy (disk cache, other programs). Repeat, and compare under the same conditions.

COMMON MISTAKE	Reading `getsizeof` as the real size. It counts only the container, not its contents.

Check yourself

1. What is the difference between `tottime` and `cumtime`? Answer: `tottime` is time inside the function only; `cumtime` also includes the functions it calls.
2. Why did `x in set` beat `x in list` by about 20,000 times in our test? Answer: a set uses a hash to jump to the item; a list scans every item.
3. Why use `tracemalloc` and not `sys.getsizeof` for memory? Answer: `getsizeof` ignores the objects inside a container.

11.2 The GIL: Threads, Processes and Who Wins

INTUITION	Imagine a kitchen with one chef's knife and several cooks. Only the cook holding the knife can chop. If the cooks mostly *wait* (for water to boil, for a delivery), they can take turns with the knife and everyone finishes sooner. If they all need to chop all day, extra cooks do not help, because there is still one knife. The knife is the **GIL**.

What the GIL is

The **Global Interpreter Lock (GIL)** is a lock inside the standard Python interpreter (CPython). Only **one thread at a time** can run Python bytecode (the low-level instructions Python code is turned into). Why does it exist? It makes memory management (reference counting, Chapter 1) simple and fast for one thread, and it protects Python's own data structures. The price is that threads cannot run Python code in parallel on several CPU cores.

Two key facts follow:

- When a thread **waits** (sleep, network, disk), it releases the GIL, so another thread runs. So threads *do* help for **I/O-bound** work (work that mostly waits).
- For **CPU-bound** work (work that mostly computes, like loops and maths), threads give no speed-up. Use **processes**, each with its own interpreter and its own GIL.

Version note: recent Python versions offer an optional **free-threaded** build with no GIL (check `sys._is_gil_enabled()`). It is not the default in standard installs and library support is still growing, so in this chapter we use the normal build. On this machine the check prints `True`, meaning the GIL is on.

Words you need

Term	Meaning	Where it appears in the job
Thread	A path of execution inside one process; shares memory with other threads	Downloading many files at once
Process	A separate program instance with its own memory and its own GIL	Parallel CPU work like feature building
I/O-bound	Time is spent waiting for network, disk or database	API calls, S3 reads, SQL queries
CPU-bound	Time is spent computing	Parsing, simulation, training loops in pure Python
Race condition	Wrong result because threads touched shared data in a bad order	Shared counters, shared lists

The experiment

Problem 11.2.1 Eight "requests" each take 0.5 s of waiting (`time.sleep`). Four CPU jobs each loop 30 million times. Compare sequential, threads and processes. Predict first: will 4 threads make the CPU jobs about 4 times faster?

Step 1	For I/O: run 8 sleeps in a loop, then in a `ThreadPoolExecutor(8)`.
Step 2	For CPU: run 4 loops in sequence, then in a `ThreadPoolExecutor(4)`, then in a `ProcessPoolExecutor(4)`.
Step 3	Time each with `time.perf_counter()`.

```
import time
from concurrent.futures import ThreadPoolExecutor, ProcessPoolExecutor

def io_task(_):
    time.sleep(0.5)
def cpu_task(n):
    s = 0
    for i in range(n):
        s += i*i
    return s

def timed(label, f):
    t=time.perf_counter(); f(); print(f"{label:<28}{time.perf_counter()-t:6.2f} s")

N=30_000_000
if __name__ == "__main__":
    timed("IO x8 sequential", lambda: [io_task(0) for _ in range(8)])
    def tp():
        with ThreadPoolExecutor(8) as ex: list(ex.map(io_task, range(8)))
    timed("IO x8 threads(8)", tp)
    timed("CPU x4 sequential", lambda: [cpu_task(N) for _ in range(4)])
    def tc():
        with ThreadPoolExecutor(4) as ex: list(ex.map(cpu_task,[N]*4))
    timed("CPU x4 threads(4)", tc)
    def pc():
        with ProcessPoolExecutor(4) as ex: list(ex.map(cpu_task,[N]*4))
    timed("CPU x4 processes(4)", pc)
```

Real output (a separate run added the asyncio line, shown in 11.3):

```
IO x8 sequential             4.03 s
IO x8 threads(8)             0.51 s
CPU x4 sequential            3.18 s
CPU x4 threads(4)            3.15 s
CPU x4 processes(4)          1.30 s
```

- **Answer** Threads made the I/O job 7.9 times faster (4.03 ÷ 0.51). Threads gave **no** speed-up on the CPU job (3.18 s against 3.15 s). Processes cut it to 1.30 s, about 2.4 times faster.
- **Check** The sequential I/O time must be 8 × 0.5 = 4.0 s, and it was 4.03 s. In parallel it must be about one sleep, 0.5 s, and it was 0.51 s. For CPU, 4 jobs of about 0.8 s each give about 3.2 s, matching.
- **Why it matters** Why only 2.4 times and not 4 times for processes? Starting processes, sending the function and result between them (pickling), and the machine's other work cost time. Parallel speed-up is almost always less than the number of workers, and small jobs may even get slower.

Here is the typical wrong version with processes, and its error message:

```
from concurrent.futures import ProcessPoolExecutor
with ProcessPoolExecutor(2) as ex:
    print(list(ex.map(lambda x: x*2, [1,2])))
```

```
PicklingError – Can't pickle <function <lambda> at 0x1039e6770>: it's not found as __m
```

Processes need to send the function to another process by **pickling** (turning an object into bytes). A `lambda` has no importable name, so it fails. The fix is a normal named function defined at the top level of a module. Also, on macOS and Windows the code that starts processes must sit under `if __name__ == "__main__":`, because each child re-imports the file.

A race condition, on purpose

Threads share memory. If two threads update the same variable, results can go wrong. This code does "read, wait a tiny moment, write", which gives other threads the chance to run in the middle:

```
import threading, time
counter = 0
lock = threading.Lock()
def unsafe(n):
    global counter
    for _ in range(n):
        v = counter
        time.sleep(0)       # gives other threads a chance to run mid-update
        counter = v + 1
def safe(n):
    global counter
    for _ in range(n):
        with lock:
            v = counter
            time.sleep(0)
            counter = v + 1
def run(fn):
    global counter
    counter = 0
    ts=[threading.Thread(target=fn,args=(2000,)) for _ in range(4)]
    [t.start() for t in ts]; [t.join() for t in ts]
    return counter
print("unsafe:", run(unsafe), "expected 8000")
print("safe:  ", run(safe), "expected 8000")
```

```
unsafe: 2015 expected 8000
safe:   8000 expected 8000
```

- **Answer** The unsafe version lost about three quarters of the updates (2015 instead of 8000, and the number changes from run to run). With a `Lock`, the result is exactly 8000.
- **Check** 4 threads × 2000 = 8000. Only the locked version equals it.
- **Why it matters** Honest note: when I removed the `time.sleep(0)` and did a plain `counter += 1` loop, the unsafe version gave the correct answer in my two runs on this Python version. That does *not* make it safe. Whether it fails depends on timing and on the Python version, which is exactly why race bugs are dangerous: they hide in testing and appear in production. Rule: when threads share data that changes, use a `Lock` or a `queue.Queue` (11.5).

The picture

```
 I/O-bound, 3 threads (one GIL)          CPU-bound, 3 threads (one GIL)
 T1: run..wait.........run               T1: run run run ...  (holds GIL)
 T2: ..run..wait........run              T2: wait ........... (no GIL)
 T3: ....run..wait......run              T3: wait ...........
 waits overlap, total time shrinks       no overlap of running, no speed-up

 CPU-bound, 3 processes (3 GILs)
 P1: run run run          each process has its own GIL
 P2: run run run          and its own core: real parallelism
 P3: run run run
```

Connection to the real job

- Calling 500 REST APIs or reading 500 S3 files: threads (or asyncio).
- Transforming 8 big files with pure-Python code: processes.

- Most numeric libraries (NumPy, pandas internals) release the GIL inside their fast C code, so threads can sometimes help there. Measure.

Applications

- **Data Analytics:** download many report files in parallel with a thread pool.
- **Data Engineering:** process daily partitions in a process pool; parallel S3 or API ingestion with threads.
- **Machine Learning:** `DataLoader` workers are processes for exactly this GIL reason; parallel hyper-parameter runs.
- **MLOps:** serve models with several worker processes (for example `gunicorn` / `uvicorn` workers) to use all cores.

COMMON MISTAKE	Using threads to speed up a CPU loop and seeing no gain (3.18 s against 3.15 s above). Check: is the job waiting or computing?

COMMON MISTAKE	Sharing and changing a list or counter in threads without a lock. Prefer passing results back (`executor.map` returns them in order) or use a queue.

Check yourself

1. What does the GIL allow and not allow? Answer: only one thread runs Python bytecode at a time, so no CPU parallelism with threads in one process.
2. Why do threads still help I/O-bound jobs? Answer: a waiting thread releases the GIL, so others run during the wait.
3. Why did the `lambda` fail in a process pool? Answer: pickling needs a function with an importable name; a lambda has none.

11.3 concurrent.futures and asyncio

INTUITION	A restaurant waiter can serve 20 tables alone. He takes an order, and while the kitchen cooks, he goes to the next table. He never stands idle. This is `asyncio` : one worker that switches to another task whenever the current one has to wait. Threads are like hiring 20 waiters. `asyncio` is cheaper when you have thousands of tables, but the waiter must *never* freeze at one table.

concurrent.futures: the easy, high-level tool

`concurrent.futures` gives one simple interface for thread pools and process pools. You learned `ex.map(fn, items)` above. For more control use `submit` , which returns a **future** (a promise of a result that is not ready yet), and `as_completed` , which hands you futures as they finish. Errors inside a job are raised when you ask for `result()` .

Problem 11.3.1 Run 6 jobs in a pool of 4 threads. Job 3 fails on purpose. Collect all results and report the failure without crashing.

Step 1	Submit each job, and remember which item belongs to which future in a dictionary.
Step 2	Loop over `as_completed` . Wrap `f.result()` in `try/except` .

```
from concurrent.futures import ThreadPoolExecutor, as_completed
import time, random
def job(i):
    time.sleep(random.random()*0.3)
    if i == 3: raise ValueError("bad item 3")
    return i*10
with ThreadPoolExecutor(4) as ex:
    futs = {ex.submit(job, i): i for i in range(6)}
    for f in as_completed(futs):
        i = futs[f]
        try: print(i, "->", f.result())
        except ValueError as e: print(i, "failed:", e)
```

Real output, sorted so the order is stable (the real arrival order changes each run):

```
0 -> 0
1 -> 10
2 -> 20
3 failed: bad item 3
4 -> 40
5 -> 50
```

- **Answer** Five jobs returned 10 times their number, and item 3's error was caught and reported.
- **Check** 4 → 40 and 5 → 50 are 10 × input. The arrival order differs run to run because each job sleeps a random time. If you need results in input order, use `ex.map`.
- **Why it matters** One bad URL in 5,000 must not kill the whole batch. Catch errors per future and keep a list of failures to retry.

asyncio: many waiting tasks, one thread

A **coroutine** is a function defined with `async def`. Calling it does not run it; it gives an object that the **event loop** (the manager that switches between tasks) runs. Inside a coroutine, `await something` means "I will wait here; run another task meanwhile". `asyncio.gather` runs many coroutines together, and `asyncio.run(main())` starts the loop.

Problem 11.3.2 Eight tasks each wait 0.5 s, using `asyncio`. How long in total? Then limit concurrency to 5 at a time for 20 tasks of 0.2 s, and add a timeout.

Step 1	`gather` 8 tasks of `await asyncio.sleep(0.5)`. Total should be about 0.5 s.
Step 2	Use `asyncio.Semaphore(5)`, a gate that lets only 5 tasks inside at once. 20 tasks ÷ 5 per round = 4 rounds × 0.2 s = 0.8 s.
Step 3	Use `asyncio.timeout(0.1)` around a 1 s wait.

```
import asyncio, time
async def fetch(i, sem):
    async with sem:
        await asyncio.sleep(0.2)
        return i*i
async def main():
    sem = asyncio.Semaphore(5)
    t=time.perf_counter()
    res = await asyncio.gather(*(fetch(i, sem) for i in range(20)))
    print(res[:5], f"{time.perf_counter()-t:.2f}s")
    try:
        async with asyncio.timeout(0.1):
            await asyncio.sleep(1)
    except TimeoutError:
        print("timed out")
asyncio.run(main())
```

```
[0, 1, 4, 9, 16] 0.80s
timed out
```

The 8-task asyncio run from the experiment script printed `IO x8 asyncio 0.50 s`, the same as 8 threads.

- **Answer** 8 tasks took 0.50 s (not 4 s). The gated run took 0.80 s, exactly as predicted, and the timeout fired.
- **Check** 20 ÷ 5 = 4 rounds, 4 × 0.2 = 0.8 s. Matches.
- **Why it matters** The semaphore is how you stay polite to a server: a limit such as "5 requests at a time" is the same idea as a restaurant that seats only 5 tables at a time.

The typical wrong version: blocking inside async

Predict: five tasks, each doing 0.2 s of waiting. Which version finishes in about 0.2 s?

```
import asyncio, time
async def bad(i):
    time.sleep(0.2)          # blocks the whole event loop
    return i
async def good(i):
    await asyncio.sleep(0.2)
    return i
async def main():
    t=time.perf_counter(); await asyncio.gather(*(bad(i) for i in range(5))); print(f"time.sleep in async :
{time.perf_counter()-t:.2f}s")
    t=time.perf_counter(); await asyncio.gather(*(good(i) for i in range(5))); print(f"await asyncio.sleep :
{time.perf_counter()-t:.2f}s")
    t=time.perf_counter(); await asyncio.gather(*(asyncio.to_thread(time.sleep,0.2) for i in range(5)));
print(f"to_thread(time.sleep): {time.perf_counter()-t:.2f}s")
asyncio.run(main())
```

```
time.sleep in async : 1.02s
await asyncio.sleep : 0.20s
to_thread(time.sleep): 0.21s
```

- **Answer** `time.sleep` inside a coroutine froze the loop, so the five tasks ran one after another (5 × 0.2 = 1.0 s). `await asyncio.sleep` took 0.20 s. If you must call a blocking function (an old library without async support), `asyncio.to_thread` runs it in a thread and gives 0.21 s.
- **Check** 5 × 0.2 = 1.0 s for sequential, and 1.02 s was measured.
- **Why it matters** The whole point of asyncio is that nothing blocks. One blocking call in a coroutine ruins every other task.

Which tool to choose

Situation	Tool	Why
10 to 200 API calls or files, simple code	`ThreadPoolExecutor`	Easiest; works with normal libraries
Thousands of connections, async libraries (`httpx.AsyncClient` , FastAPI)	`asyncio`	Very cheap tasks
CPU-heavy pure Python on many cores	`ProcessPoolExecutor` / `multiprocessing`	Separate GILs
Heavy NumPy/pandas maths	Vectorize first (11.4)	Fast C code beats any pool

The picture

```
asyncio event loop (one thread)
task A: start -> await(wait) ............... -> finish
task B:          start -> await(wait) ..... -> finish
task C:                   start -> await ... -> finish
The loop jumps to a ready task whenever another one awaits.
One blocking call (time.sleep) would freeze A, B and C together.
```

Connection to the real job

- Calling a web API 10,000 times (for example, an LLM or geocoding API) is a classic async or thread-pool job, always with a concurrency limit and a timeout.
- FastAPI (Chapter 12) endpoints declared with `async def` run on an event loop, so blocking code inside them hurts all users.
- Real services also have rate limits (for example, "50 requests per second"). Respect them with a semaphore and sleeps.

Applications

- **Data Analytics:** pull data from several SaaS APIs at once for a daily report.
- **Data Engineering:** async ingestion of many small files or API pages.
- **Machine Learning / GenAI:** send many prompts or embedding batches in parallel with a limit.
- **MLOps:** async health checks and web endpoints.

COMMON MISTAKE	Forgetting `await`. Calling `fetch(1)` without `await` gives a coroutine object and a warning that the coroutine "was never awaited". The code inside never ran.

COMMON MISTAKE	Launching 10,000 tasks with no limit. The remote server blocks you, or you run out of sockets. Use a semaphore.

Check yourself

1. What does `await` do? Answer: pauses this coroutine and lets the event loop run other tasks until the awaited thing is ready.
2. Why did `time.sleep` in an async function take 1.0 s for five tasks? Answer: it blocked the event loop, so tasks ran one by one.
3. What does a `Semaphore(5)` do? Answer: lets at most five tasks be inside the guarded block at the same time.

11.4 Vectorization, Algorithms and Native Speed

INTUITION	A cashier who scans one item at a time, thinking about each, is slow. A barcode reader that scans a whole tray at once is fast. Python's `for` loop is the thinking cashier: for each item it checks types, looks up names, and makes objects. **Vectorization** means handing the whole tray to fast compiled code (written in C), so the loop happens outside Python.

What vectorization means

A **vectorized** operation applies one operation to a whole array at once, for example "multiply every element by 2.5 and add 1". Libraries such as NumPy and pandas store numbers in compact blocks of memory with one type (not as separate Python objects), and run the loop in C. The idea is the same as a SQL query: say *what* you want on a whole column, not *how* to loop over rows.

Pure-Python hints you can run now

NumPy is not installed here, but we can already show the "loop overhead" effect with built-in functions that run their loop in C.

Problem 11.4.1 Compute `x * 2.5 + 1` for 1,000,000 numbers with a list comprehension and with an index loop. Then compute the mean with `sum/len` and with `statistics.mean`. Predict which of each pair is faster.

Step 1	Build `xs = list(range(1_000_000))`.
Step 2	Time each approach once with `time.perf_counter()`.

```
import time, statistics
n=1_000_000
xs=list(range(n))
t=time.perf_counter(); ys=[x*2.5+1 for x in xs]; print(f"list comp {time.perf_counter()-t:.3f}s")
t=time.perf_counter()
ys=[0.0]*n
for i in range(n): ys[i]=xs[i]*2.5+1
print(f"index loop {time.perf_counter()-t:.3f}s")
t=time.perf_counter(); statistics.mean(xs); print(f"statistics.mean {time.perf_counter()-t:.3f}s")
t=time.perf_counter(); sum(xs)/len(xs); print(f"sum/len {time.perf_counter()-t:.4f}s")
```

```
list comp 0.026s
index loop 0.056s
statistics.mean 0.091s
sum/len 0.0032s
```

- **Answer** The comprehension (0.026 s) was about twice as fast as the index loop (0.056 s). `sum(xs)/len(xs)` (0.0032 s) was about 28 times faster than `statistics.mean` (0.091 s).
- **Check** The mean of `0..999,999` is 499,999.5 for both. Speed differs because `statistics.mean` does careful exact arithmetic to avoid rounding errors, so it is built for accuracy, not speed. `sum` runs its loop in C.
- **Why it matters** The rule: the fewer times the Python interpreter itself runs per item, the faster. Use built-ins (`sum` , `min` , `max` , `sorted` , `str.join`) and comprehensions before hand-written loops, and compare cost of safety against speed. For big numeric data, NumPy goes much further.

The NumPy and pandas version (not run here)

```
# not run here: needs numpy and pandas
import numpy as np
x = np.arange(1_000_000)
y = x * 2.5 + 1           # whole array, no Python loop
print(y.mean())

import pandas as pd
df = pd.DataFrame({"qty": [2, 5, 1], "price": [100.0, 40.0, 250.0]})
df["total"] = df["qty"] * df["price"]      # vectorized column maths
# slow pattern to avoid:
# df["total"] = df.apply(lambda r: r["qty"] * r["price"], axis=1)
```

On typical machines, vectorized NumPy code is often tens to hundreds of times faster than a pure-Python loop for this kind of maths. I did not measure it here, so measure on your own data.

When vectorization is not enough

Tool	What it does	When to consider it
Better algorithm / data structure	Set instead of list (11.1)	Always first
`lru_cache`	Remembers results of repeated calls	Repeated expensive calls like `fib`
Numba (awareness)	`@njit` compiles numeric Python loops to machine code	A numeric loop you cannot vectorize
Cython (awareness)	Python-like code compiled to C	Library authors, hot loops needing C speed
`multiprocessing` for pandas	Split data into chunks, run each in a process	CPU-heavy per-row Python functions
Polars, DuckDB (awareness)	Faster dataframe/SQL engines	Large local data

Multiprocessing for CPU-bound chunks

Problem 11.4.2 Count the primes below 2,000,000 by splitting the range into 8 chunks of 250,000. Compare one process with a `Pool(4)` .

Step 1	Write `count_primes((lo, hi))` that trial-divides up to the square root.
Step 2	Build the 8 chunks. Run `map` serially, then `Pool(4).map` .
Step 3	Check both results match, and compute speed-up = serial time ÷ parallel time.

```
import time
from multiprocessing import Pool
def count_primes(rng):
    lo, hi = rng
    c = 0
    for n in range(lo, hi):
        if n < 2: continue
        for d in range(2, int(n**0.5)+1):
            if n % d == 0: break
        else: c += 1
    return c
if __name__ == "__main__":
    chunks = [(i*250_000, (i+1)*250_000) for i in range(8)]
    t=time.perf_counter(); r1=sum(map(count_primes, chunks)); t1=time.perf_counter()-t
    t=time.perf_counter()
    with Pool(4) as p: r2=sum(p.map(count_primes, chunks))
    t2=time.perf_counter()-t
    print(r1, r2, f"serial {t1:.2f}s  pool(4) {t2:.2f}s  speedup {t1/t2:.1f}x")
```

```
148933 148933 serial 3.75s  pool(4) 1.49s  speedup 2.5x
```

- **Answer** Both counted 148,933 primes. The pool was 2.5 times faster (3.75 ÷ 1.49), not 4 times.
- **Check** The number of primes below 2,000,000 is 148,933, a known value, so the result is right. Why not 4 times? Later chunks hold bigger numbers and take longer, so some workers finish early and wait (uneven load), and starting processes costs time.
- **Why it matters** Splitting work into many *similar-sized* chunks (more chunks than workers) balances the load. The same pattern works for a pandas DataFrame: split by groups or row ranges, run a function in each process, and concatenate the pieces. The data is copied to each process, so this pays off only when the per-chunk work is heavy.

The picture

```
Python loop:    [x1] -> python -> [y1]   one item at a time, overhead each time
                [x2] -> python -> [y2]

Vectorized:     [x1 x2 x3 x4 ... xN]  --one call--> [y1 y2 y3 ... yN]
                (tight loop in C over a compact block of memory)
```

Connection to the real job

- "Never loop over DataFrame rows" is a standard code-review comment in data teams. Use column operations, `groupby`, or SQL.
- ML pipelines: feature maths in NumPy; custom Python per-row logic is the usual bottleneck.
- Know when to stop: if the job runs 5 minutes once a day, readable code beats a 3-minute saving.

Applications

- **Data Analytics:** replace `apply(axis=1)` with column maths.
- **Data Science:** vectorized simulation (many random draws at once).
- **Machine Learning / Deep Learning:** array and tensor maths on CPU and GPU is vectorization at scale.
- **Data Engineering:** chunked processing in a process pool, or push the work to SQL or Spark.

COMMON MISTAKE	Rewriting everything in a faster style before profiling. First prove the loop is the hot spot (11.1).

COMMON MISTAKE	Using a process pool on tiny tasks. The start-up and copying cost can be larger than the work. Measure with real data size.

Check yourself

1. What does "vectorized" mean? Answer: one operation applied to a whole array, with the loop running in fast compiled code.
2. Why was `sum(xs)/len(xs)` faster than `statistics.mean(xs)`? Answer: `sum` loops in C, while `statistics.mean` does exact, careful arithmetic in Python.
3. Why was the pool speed-up 2.5x, not 4x? Answer: uneven chunk sizes plus process start-up and data-copy costs.

11.5 Queues, Back-Pressure and Little's Law

INTUITION	At a bank, customers wait in a line, and tellers take the next customer when free. If customers arrive faster than tellers can serve, the line grows without end. A programmer's queue is the same line. And Little's law tells you how many tellers you need for a target speed.

Producer, consumer and queue

A **producer** creates work items. A **consumer** (or worker) processes them. A **queue** sits between them. Python's `queue.Queue` is thread-safe: many threads can put and get without a lock of your own. With `maxsize`, `put` waits when the queue is full. That is **back-pressure**: the fast producer is forced to slow down to the speed of consumers instead of filling memory.

Problem 11.5.1 One producer puts 8 items into a queue with `maxsize=3`. Two consumers process each item in 10 ms. How do they stop?

Step 1	The producer puts items 0 to 7, then puts one `None` per consumer as a "stop" signal (a **sentinel**).
Step 2	Each consumer loops `while (item := q.get()) is not None`.
Step 3	Join all threads and print the sorted results.

```
import queue, threading, time
q = queue.Queue(maxsize=3)
results = []
def producer():
    for i in range(8):
        q.put(i)              # blocks when queue is full (back-pressure)
    for _ in range(2): q.put(None)
def consumer(name):
    while (item := q.get()) is not None:
        time.sleep(0.01)
        results.append((name, item))
ts=[threading.Thread(target=producer)]+[threading.Thread(target=consumer,args=(f"c{i}",)) for i in range(2)]
[t.start() for t in ts]; [t.join() for t in ts]
print(sorted(i for _,i in results))
print(len(results), "items processed")
```

```
[0, 1, 2, 3, 4, 5, 6, 7]
8 items processed
```

- **Answer** Every item was processed exactly once, and both consumers stopped cleanly. Which consumer took which item varies between runs, but the final sorted list is always the same.
- **Check** 8 items in, 8 results out, none duplicated and none lost.
- **Why it matters** This is the pattern behind message queues (Kafka, SQS, RabbitMQ) in data engineering, just on a small scale. (`list.append` is safe here because it is a single step; for anything bigger, use a queue or a lock.)

Little's law

Little's law says: in a stable system, the average number of requests inside the system `L` equals the arrival rate (throughput) `λ` times the average time each spends inside `W`.

```
L = λ × W        so      λ = L / W      (throughput = concurrency / latency)
```

Check the units: `L` is a count of requests. `λ` is requests per second. `W` is seconds. Requests per second × seconds = requests. In a bank, `L` is the number of customers inside, `W` is the average time each spends there, and `λ` is how many customers arrive per minute.

Problem 11.5.2 Each "request" takes `W = 0.05 s`. Predict the throughput for concurrency 1, 5, 10 and 20, then measure with real threads for 2 seconds each.

Formula	throughput `λ = L / W`
Plug in and compute	L=1: 1 ÷ 0.05 = 20 req/s. L=5: 5 ÷ 0.05 = 100. L=10: 10 ÷ 0.05 = 200. L=20: 20 ÷ 0.05 = 400.
Step 1	Start `L` threads. Each loops: `time.sleep(0.05)`, then add 1 to a locked counter, until the 2 s are over.
Step 2	Divide the total count by elapsed time.

```
import threading, time
LAT = 0.05          # each request takes 50 ms
DURATION = 2.0
def run(workers):
    done = 0
    lock = threading.Lock()
    end = time.perf_counter() + DURATION
    def w():
        nonlocal done
        while time.perf_counter() < end:
            time.sleep(LAT)
            with lock: done += 1
    ts=[threading.Thread(target=w) for _ in range(workers)]
    t0=time.perf_counter()
    [t.start() for t in ts]; [t.join() for t in ts]
    el=time.perf_counter()-t0
    return done/el
for c in (1,5,10,20):
    print(f"concurrency {c:>2}: measured {run(c):7.1f} req/s   predicted {c/LAT:7.1f} req/s")
```

```
concurrency  1: measured    18.6 req/s   predicted    20.0 req/s
concurrency  5: measured    92.6 req/s   predicted   100.0 req/s
concurrency 10: measured   185.3 req/s   predicted   200.0 req/s
concurrency 20: measured   370.6 req/s   predicted   400.0 req/s
```

- **Answer** Throughput grows in a straight line with concurrency, as the law says. Measured values are about 93% of predicted (for example 370.6 against 400).
- **Check** 370.6 ÷ 400 = 0.9265, and 18.6 ÷ 20 = 0.93. The same 7% gap at every level means one constant cause: `time.sleep(0.05)` always takes a little longer than 50 ms, so the real `W` is about 0.054 s. Using that `W`, 20 ÷ 0.054 = 370 req/s, which matches.
- **Why it matters** In plain words: to get more throughput with the same latency, you need more concurrency, in the same proportion. And if latency doubles (a slow database), throughput at the same concurrency halves. Real servers also have a limit (CPU cores, connections), so the straight line eventually bends flat. That bend is the saturation point you look for when you push traffic at a service. This simple experiment has no such limit, because sleeping uses no CPU.

Use Little's law to size a worker pool: to process 200 requests per second when each takes 0.25 s, you need `L = 200 × 0.25 = 50` concurrent workers (threads or async tasks), plus some safety margin.

The picture

```
producers --> [ queue (maxsize=3) ] --> consumer 1
(fast)        [ x ][ x ][ x ]        --> consumer 2
                 ^
                 | queue full: put() waits  (back-pressure)

Throughput vs concurrency (fixed latency, no resource limit):
req/s |                     /
      |                /
      |           /          slope = 1 / W
      |      /
      +-------------------- concurrency
```

Connection to the real job

- Worker pool size, connection pool size, and Kafka/SQS consumer counts are all chosen with `L = λ × W`.
- When a pipeline "gets slower at night", check whether the queue between stages is growing. A growing queue means arrivals exceed service rate.
- You can check the formula on your own Python service by sending it many requests with a traffic tool such as Locust (named only as an example) and comparing with the prediction.

Applications

- **Data Engineering:** sizing consumers for a message queue; back-pressure in streaming jobs.
- **MLOps:** sizing replicas and worker counts for a model API for a target requests-per-second.
- **Machine Learning:** prefetch queues in data loaders keep the GPU busy.
- **Data Analytics / BI:** explaining why a dashboard slows down when many users open it at 9 am.

COMMON MISTAKE	Using an unbounded queue. A slow consumer then lets the queue grow until memory runs out. Set `maxsize`.

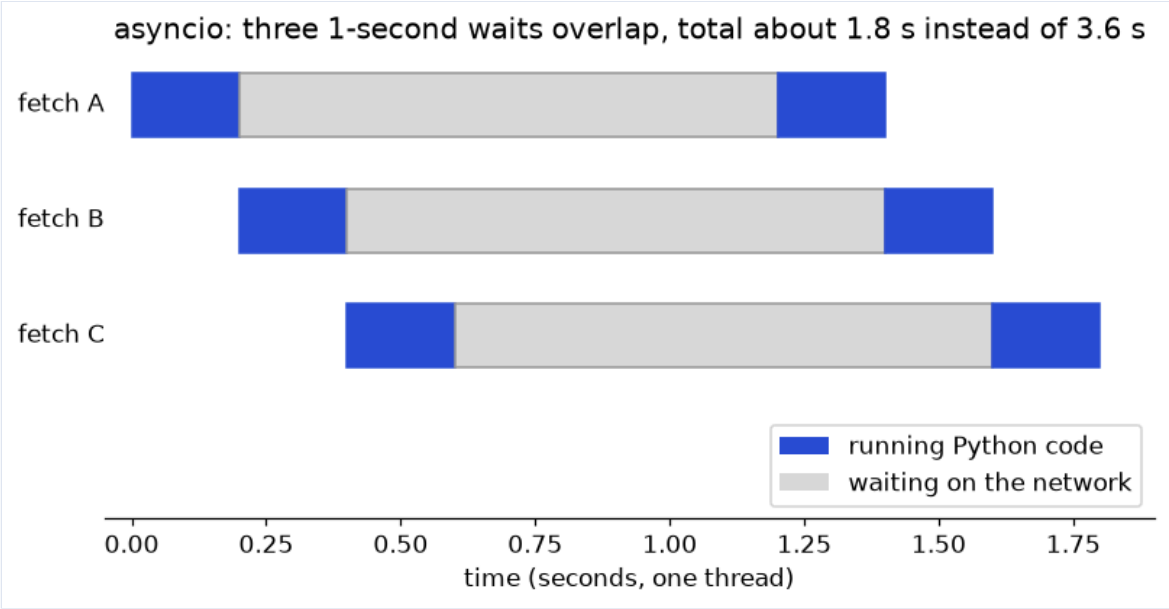
COMMON MISTAKE	Forgetting the stop signal. Consumers wait forever on `q.get()` and the program never exits. Use a sentinel per consumer, or `q.task_done()` with `q.join()`.

Check yourself

1. State Little's law and its units. Answer: `L = λ × W`: requests in system = requests per second × seconds each.
2. Latency is 0.1 s and you run 30 concurrent workers. What throughput do you expect? Answer: 30 ÷ 0.1 = 300 requests per second (at best).
3. What is back-pressure? Answer: a full bounded queue makes the producer wait, so it slows to the speed of the consumers.

11.6 Modern asyncio: TaskGroup, Timeouts, Limits, Queues and Cancellation

Section 11.3 introduced `async` and `await`. Real programs also need to stop work that takes too long, cap how much runs at once, pass work between tasks, and clean up after cancellation. Python 3.11 and newer give you clean tools for each. All blocks below were run.



One thread, three tasks. The loop runs a task until it hits `await`, then switches. The waits overlap, so the total is about 1.8 s, not 3.6 s.

How to read it: blue means Python code is running and nothing else can. Grey means the task is parked waiting on the network and the loop is free to run another. Asyncio helps exactly when the grey is much longer than the blue.

TaskGroup and timeout

A `TaskGroup` runs several tasks and waits for all of them when the `async with` block ends. If one task fails, the group cancels the others and raises an `ExceptionGroup` (section 6.6), which you catch with `except*`. This beats `gather`, which can leave siblings running after a failure. `asyncio.timeout(seconds)` cancels the block inside it and raises `TimeoutError`.

Problem 11.6.1 Run two fetches together and read both results. Then run a slow fetch beside one that fails, and show that the failure surfaces cleanly and the slow one does not keep running. Finally, put a 0.05 s limit on a 1 s wait.

Step 1	`async with asyncio.TaskGroup() as tg:` then `tg.create_task(...)` for each job. After the block, every task is done, so `.result()` is safe.
Step 2	The failing task raises `ValueError("bad")` after 0.01 s. The group cancels the 0.5 s task and raises an `ExceptionGroup`. `except* ValueError` unpacks it.
Step 3	`async with asyncio.timeout(0.05):` around `await asyncio.sleep(1)` raises `TimeoutError` after 0.05 s.

```
import asyncio

async def fetch(name, delay, fail=False):
    await asyncio.sleep(delay)
    if fail:
        raise ValueError(name)
    return name

async def main():
    async with asyncio.TaskGroup() as tg:
        t1 = tg.create_task(fetch("a", 0.02))
        t2 = tg.create_task(fetch("b", 0.01))
    print(t1.result(), t2.result())
    try:
        async with asyncio.TaskGroup() as tg:
            tg.create_task(fetch("ok", 0.5))
            tg.create_task(fetch("bad", 0.01, fail=True))
    except* ValueError as eg:
        print("caught", [str(e) for e in eg.exceptions])
    try:
        async with asyncio.timeout(0.05):
            await asyncio.sleep(1)
    except TimeoutError:
        print("timed out")
asyncio.run(main())
```

```
a b
caught ['bad']
timed out
```

COMMON MISTAKE	Calling a blocking function such as `time.sleep(1)` or `requests.get(...)` inside an `async def`. The whole loop freezes, and every other task stalls with it. Use `await asyncio.sleep`, an async client such as httpx (not run here), or hand the blocking call to a thread with `await asyncio.to_thread(fn, ...)`.

Limiting concurrency, queues and threads

Starting 10,000 requests at once will get you banned or crash a server. A `Semaphore(n)` lets at most `n` tasks into a section at the same time. An `asyncio.Queue` with `maxsize` lets a producer and a consumer work at different speeds: when the queue is full, `put` waits, which is back-pressure from section 11.5. `to_thread` runs an ordinary blocking function in a worker thread and gives you an awaitable.

Problem 11.6.2 Run 10 jobs with at most 3 at a time and prove the limit held. Then pass 6 numbers through a queue of size 2 and square them. Then run a blocking `sum` in a thread.

Limit	Each worker enters `async with sem`, raises a counter, sleeps, lowers it. The largest counter value seen is the peak.
Queue	The producer puts 0..5 and then `None` as a stop signal. The consumer loops until it sees `None`.
Thread	`await asyncio.to_thread(sum, range(1000))` returns 499500 (the sum 0+1+...+999 is 999 x 1000 / 2).

```
import asyncio

async def worker(i, sem, state):
    async with sem:
        state["now"] += 1
        state["peak"] = max(state["peak"], state["now"])
        await asyncio.sleep(0.01)
        state["now"] -= 1
    return i

async def producer(q, n):
    for i in range(n):
        await q.put(i)
    await q.put(None)

async def consumer(q, out):
    while (item := await q.get()) is not None:
        out.append(item * item)

async def main():
    sem = asyncio.Semaphore(3)
    state = {"now": 0, "peak": 0}
    res = await asyncio.gather(*(worker(i, sem, state) for i in range(10)))
    print(res, "peak concurrency", state["peak"])
    q = asyncio.Queue(maxsize=2)
    out = []
    await asyncio.gather(producer(q, 6), consumer(q, out))
    print(out)
    r = await asyncio.to_thread(sum, range(1000))
    print("thread result", r)
asyncio.run(main())
```

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9] peak concurrency 3
[0, 1, 4, 9, 16, 25]
thread result 499500
```

Cancellation

`task.cancel()` does not kill a task. It raises `CancelledError` inside the task at its next `await`, so `finally` blocks and `except` handlers can clean up. Always re-raise the error after cleanup, otherwise the task refuses to stop.

Problem 11.6.3 Start a task that waits 10 seconds, cancel it after 0.01 s, and show that its cleanup code ran.

Step 1	`create_task` starts the task. `await asyncio.sleep(0.01)` lets it begin.
Step 2	`t.cancel()` schedules `CancelledError` inside the task.
Step 3	The task prints, then `raise` re-raises. Awaiting the task from outside raises `CancelledError` there too, and `t.cancelled()` is `True`.

```
import asyncio

async def forever():
    try:
        await asyncio.sleep(10)
    except asyncio.CancelledError:
        print("cleanup ran")
        raise

async def main():
    t = asyncio.create_task(forever())
    await asyncio.sleep(0.01)
    t.cancel()
    try:
        await t
    except asyncio.CancelledError:
        print("task cancelled:", t.cancelled())
asyncio.run(main())
```

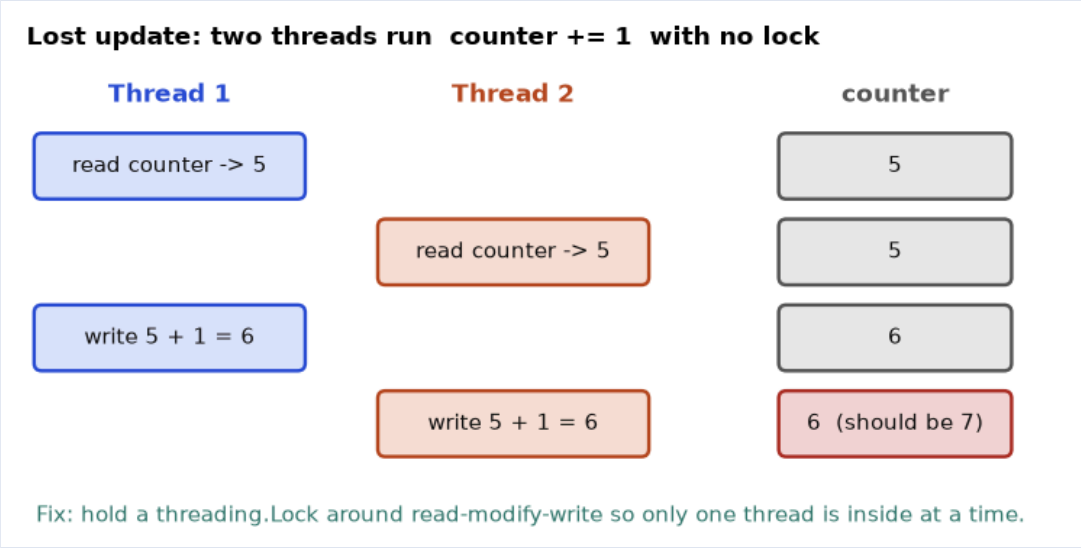
```
cleanup ran
task cancelled: True
```

Going deeper

- **contextvars.** A `ContextVar` holds a value that is separate per task (and per thread), for example a request id for logging. Unlike a global, each task started with `create_task` gets a copy of the current context, so concurrent requests do not overwrite each other.
- **Hidden tasks.** A task made with `create_task` and not stored may be garbage collected before it finishes, because the loop keeps only a weak reference. A `TaskGroup` holds the references for you, another reason to prefer it.
- **Not run here:** a rate-limited crawler needs a real network and httpx. The pattern is the Semaphore above plus one `async with httpx.AsyncClient() as c:` shared by all tasks.

11.7 Locks, Deadlocks, Memory and Data-Structure Speed

Threads share memory, and that is where the trouble starts. This section shows the lost update, the lock that cures it, a deadlock and its fix, then moves to memory: reference counting, cycles, weak references and the data structure that makes queues cheap.



A lost update. Both threads read 5 before either writes, so one increment vanishes.

How to read it: time runs downward. The danger is the gap between reading the counter and writing it back. A lock closes that gap by making the read-modify-write one indivisible step.

Problem 11.7.1 Four threads each add 1 to a shared counter 200,000 times. Compute the correct total, then run it with a lock, and without.

Expected	4 threads x 200,000 = 800,000.
With lock	`with lock:` around `counter += 1` means only one thread is inside at a time. Result must be exactly 800,000.
Without lock	The result can be lower than 800,000, or happen to equal it. The only safe claim is "at most 800,000".

```
import threading

def run(use_lock, n=200_000):
    counter = {"v": 0}
    lock = threading.Lock()
    def work():
        for _ in range(n):
            if use_lock:
                with lock:
                    counter["v"] += 1
            else:
                v = counter["v"]
                counter["v"] = v + 1
    ts = [threading.Thread(target=work) for _ in range(4)]
    for t in ts: t.start()
    for t in ts: t.join()
    return counter["v"]
print(run(True))
print(run(False) <= 800_000)
```

```
800000
True
```

COMMON MISTAKE	Concluding "my code is thread safe" because an unlocked test printed the right number once. Races depend on timing, and on this machine and Python version the unlocked run may well print exactly 800,000 and still be wrong on another. Protect shared writes with a lock by design, not by experiment.

Problem 11.7.2 Thread 1 takes lock A then wants B. Thread 2 takes B then wants A. Reproduce the deadlock safely (with timeouts so the script ends) and then fix it.

Cause	Each thread holds one lock and waits for the other: a cycle, so neither can ever proceed.
Safe repro	Use `acquire(timeout=0.3)` so a stuck thread gives up and logs it. Real code with a plain `with` would hang forever.
Fix	Every thread takes the locks in the same global order (A, then B). A cycle becomes impossible.

```
import threading, time
a, b = threading.Lock(), threading.Lock()
log = []
def t1():
    with a:
        time.sleep(0.05)
        if b.acquire(timeout=0.3):
            b.release(); log.append("t1 got both")
        else:
            log.append("t1 gave up")
def t2():
    with b:
        time.sleep(0.05)
        if a.acquire(timeout=0.3):
            a.release(); log.append("t2 got both")
        else:
            log.append("t2 gave up")
x = [threading.Thread(target=t1), threading.Thread(target=t2)]
for t in x: t.start()
for t in x: t.join()
print(sorted(log))
def ordered(name):
    with a:
        with b:
            log.append(name + " ordered ok")
log.clear()
y = [threading.Thread(target=ordered, args=(n,)) for n in ("t1", "t2")]
for t in y: t.start()
for t in y: t.join()
print(sorted(log))
```

```
['t1 gave up', 't2 gave up']
['t1 ordered ok', 't2 ordered ok']
```

Reference counting, cycles and weak references

CPython frees most objects the instant their reference count hits zero. Two objects that point at each other never reach zero, so a second mechanism, the cyclic garbage collector in the `gc` module, finds and frees such cycles. A `weakref` points at an object without keeping it alive, which makes it ideal for caches: when nothing else uses the value, the cache entry vanishes.

Problem 11.7.3 Build a two-object cycle, delete your names, and show it survives until `gc.collect()`. Then show a `WeakValueDictionary` cache that drops an entry on its own.

Step 1	Disable automatic collection so the demo is deterministic. `a.other = b` and `b.other = a` make a cycle. Keep a weak reference `r` to `a`.
Step 2	`del a, b`: the objects are unreachable from your code, but each still has one reference from the other, so `r()` still returns `a`.
Step 3	`gc.collect()` returns the number of objects it freed (more than 0), and `r()` becomes `None`.
Step 4	A `WeakValueDictionary` holding an object has length 1. After `del obj` (the last strong reference) it has length 0.

```
import gc, weakref

class Node:
    def __init__(self): self.other = None
gc.disable()
a, b = Node(), Node()
a.other, b.other = b, a
r = weakref.ref(a)
del a, b
print("alive after del:", r() is not None)
print("collected objects > 0:", gc.collect() > 0)
print("alive after gc:", r() is not None)
gc.enable()

class Big: pass
cache = weakref.WeakValueDictionary()
obj = Big()
cache["k"] = obj
print(len(cache))
del obj
print(len(cache))
```

```
alive after del: True
collected objects > 0: True
alive after gc: False
1
0
```

Choosing the right structure: deque versus list

`list.pop(0)` shifts every remaining item one place left, so it costs time proportional to the list length (O(n)). `collections.deque.popleft()` is O(1). Draining a queue of 50,000 items with `list.pop(0)` is therefore quadratic in total. The exact times depend on your machine, so the check below prints only whether the ratio exceeded 10x.

```
import timeit

n = 50_000
tl = timeit.timeit("l.pop(0)", setup=f"l=list(range({n}))", number=n-1)
td = timeit.timeit("d.popleft()", setup=f"from collections import deque; d=deque(range({n}))", number=n-1)
print("deque faster than list.pop(0) by more than 10x:", tl / td > 10)
```

```
deque faster than list.pop(0) by more than 10x: True
```

Going deeper

- **Free-threaded build.** Python 3.13 added an optional build without the GIL, supported officially from 3.14. It is not the default, some C extensions are not yet compatible, and single-thread code can run somewhat slower. Even without the GIL, `counter += 1` is still not atomic, so locks stay necessary.
- **multiprocessing start methods.** `spawn` starts a fresh interpreter and re-imports your module (the default on macOS and Windows, and from 3.14 on Linux), `fork` copies the parent. With `spawn`, guard code with `if __name__ == "__main__":` and remember that arguments and results must be picklable: lambdas and open files are not.
- **Leak patterns.** A module-level dict used as a cache that is never trimmed is the classic Python memory leak. Use `functools.lru_cache(maxsize=...)` or a weak-value dictionary.

More practice: concurrency and memory

Problem 11.P.12 (warm-up) A function does 2 s of CPU-bound maths. Would 4 threads speed up 4 calls on a standard (GIL) build? What would?

Answer: No. Only one thread runs Python bytecode at a time, so the total stays near 8 s. Use `ProcessPoolExecutor` (about 2 s on 4 free cores) or a vectorised library. Threads help for waiting, not computing.

Problem 11.P.13 (build) Using `asyncio.gather`, run `fetch("a", 0.03)` and `fetch("b", 0.01)` from 11.6.1 and print the result list. In what order do the results appear?

```
import asyncio

async def fetch(name, delay):
    await asyncio.sleep(delay)
    return name

async def main():
    print(await asyncio.gather(fetch("a", 0.03), fetch("b", 0.01)))
asyncio.run(main())
```

```
['a', 'b']
```

Answer: `['a', 'b']`. `gather` returns results in the order you passed the awaitables, not the order they finished, even though b finished first.

Problem 11.P.14 (find the bug) A worker does `except asyncio.CancelledError: print("bye")` and never re-raises. What goes wrong?

Answer: The task swallows the cancellation and carries on (or finishes normally), so a `TaskGroup` or `timeout` that relies on cancelling it cannot stop it cleanly. Add `raise` after cleanup.

Problem 11.P.15 (stretch) Two threads each need locks A and B for a transfer between two accounts. Design a rule that prevents deadlock when the order of accounts varies by call. Rubric: (1) a total order on locks, (2) acquire in that order whatever the argument order, (3) one sentence on why a cycle then cannot form.

Answer: Sort the two accounts by a fixed key such as account id and always lock the lower id first: `first, second = sorted([acc1, acc2], key=lambda a: a.id)`. A deadlock needs a cycle of waits, and a global order means a thread holding the higher lock never waits for a lower one.

Problem 11.P.16 (interview) Your service leaks memory slowly. Name three Python-specific causes and one tool for each.

Answer: (1) An unbounded cache or global list: find it with `tracemalloc` snapshots compared over time (section 11.1). (2) Reference cycles holding large objects with `__del__` or long-lived callbacks: inspect with `gc.get_objects()` and `gc.collect()`, and break cycles with `weakref`. (3) Tasks or threads that are started and never finished or awaited: list them with `asyncio.all_tasks()` or `threading.enumerate()`.

Chapter recap

- Measure first: `timeit` for small comparisons, `cProfile` to find the hot spot, `tracemalloc` for memory.
- The biggest wins come from better algorithms and data structures (set against list: about 20,000 times faster in our test), not style tricks.
- The GIL lets only one thread run Python bytecode at a time. Threads help I/O-bound work and do not help CPU-bound work. Free-threaded builds exist but are not the default.
- Processes bypass the GIL, with costs: start-up, pickling, copying. Speed-up is less than the worker count.
- `concurrent.futures` is the easy interface to thread and process pools. Handle errors per future.
- `asyncio` runs many waiting tasks on one thread. Never block inside a coroutine; limit concurrency with a semaphore and add timeouts.
- Vectorization moves the loop into compiled code (NumPy, pandas). Avoid row loops and `apply(axis=1)` on big data.
- Shared mutable data between threads needs a lock or a queue. Bounded queues give back-pressure.
- Little's law `L = λ × W` links concurrency, latency and throughput, the same three numbers as in a bank queue: customers inside, time spent, arrival rate.

Cheat-sheet

Need	Tool or formula
Compare two snippets	`timeit.timeit(stmt, number=n)`
Find the slow function	`python3 -m cProfile -s cumulative script.py`
Memory by line of code	`tracemalloc.start()`, `take_snapshot().statistics("lineno")`
Many waits (I/O)	`ThreadPoolExecutor` or `asyncio.gather`
Many calculations (CPU)	`ProcessPoolExecutor` or `Pool.map`
Limit parallel tasks	`asyncio.Semaphore(n)`
Call blocking code from async	`await asyncio.to_thread(fn, ...)`
Protect shared data	`with threading.Lock():` or `queue.Queue(maxsize=n)`
Throughput	`λ = L / W`
Workers needed	`L = λ × W`

Practice set

Problem 11.P.1 (easy) A job has `L = 10` concurrent workers and each request takes `W = 0.2 s`. What is the throughput?

Answer: λ = L ÷ W = 10 ÷ 0.2 = 50 requests per second.

Problem 11.P.2 (easy) You must download 100 files, each taking about 1 s of network wait, with a pool of 20 threads. About how long will it take?

Answer: 100 ÷ 20 = 5 rounds of 1 s each, about 5 s (compared with 100 s sequentially).

Problem 11.P.3 (easy, what is the output?) What does this print and why?

```
import asyncio
async def f():
    return 5
print(f())
```

Answer: it prints something like `<coroutine object f at 0x...>` (plus a warning that it was never awaited). Calling an `async def` function does not run it; you must `await f()` or use `asyncio.run(f())`.

Problem 11.P.4 (medium) Which tool for each: (a) 300 REST API calls, (b) a pure-Python loop over 10 big files on 8 cores, (c) summing a column of 10 million numbers?

Answer: (a) thread pool or asyncio with a limit, (b) process pool, (c) vectorized NumPy/pandas (or the built-in `sum`), no pool.

Problem 11.P.5 (medium, find the bug) This code uses a thread pool to speed up a CPU-heavy function and gets no gain. Why, and what is the fix?

```
with ThreadPoolExecutor(8) as ex:
    results = list(ex.map(heavy_math, data))
```

Answer: `heavy_math` is CPU-bound pure Python, so the GIL allows one thread at a time. Use `ProcessPoolExecutor` (with `heavy_math` defined at module top level), or vectorize it.

Problem 11.P.6 (medium, find the bug) An async function calls `requests.get(url)` (a blocking call) inside `gather` of 100 tasks. It takes 100 times one call. Fix it two ways.

Answer: the blocking call freezes the event loop, so the tasks run one by one. Fix 1: use an async client such as `httpx.AsyncClient` with `await client.get(url)`. Fix 2: wrap the blocking call with `await asyncio.to_thread(requests.get, url)`.

Problem 11.P.7 (medium) A profile shows `fib` with `ncalls = 57313/1`. What does this mean, and what is the cheapest fix?

Answer: 57,313 total calls of which 1 was the outer call, so the rest are recursive repeats of the same sub-problems. Add `@functools.lru_cache(maxsize=None)` so each value is computed once.

Problem 11.P.8 (medium) You need 500 requests per second. The average latency of your service is 0.04 s. How many concurrent workers do you need at minimum?

Answer: L = λ × W = 500 × 0.04 = 20 workers. Add margin (for example 30) because latency varies and the formula is for averages.

Problem 11.P.9 (hard, scenario) Your ETL reads 5,000 small JSON files from cloud storage and parses each (parsing takes 2 ms CPU, reading takes 100 ms wait). Design the concurrency.

Answer: reading is I/O-bound (100 ms wait against 2 ms CPU), so a thread pool of 50 to 100 workers works: 5,000 × 0.1 s ÷ 50 = 10 s of wall time. Parsing at 2 ms × 5,000 = 10 s CPU total, which fits in the same threads (the GIL allows it). Only if parsing were heavy would you add processes. Add retries and per-file error handling, and measure before tuning.

Problem 11.P.10 (hard, interview) Explain the GIL to a junior engineer in four sentences, and say when you would use threads, processes or asyncio.

Answer: The GIL is a lock in CPython that lets only one thread run Python bytecode at a time. Threads still help when they wait, because a waiting thread releases the lock. For CPU-bound work you use processes, each with its own GIL. I use threads for a moderate number of I/O tasks with normal libraries, asyncio for thousands of connections with async libraries, and processes (or vectorized code) for CPU-bound work.

Problem 11.P.11 (hard, interview) A teammate says "just add more threads, the API is slow." Using Little's law, explain what may go wrong.

Answer: more threads raise concurrency L, which raises throughput only while latency W stays flat. If the server is saturated, extra concurrency just raises W (requests queue), and throughput stays flat or drops, with more errors and timeouts. I would try 10, 20, 40, 80 threads, find where throughput stops growing, and use a rate limit and a timeout.

Prompt pack

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data, ML and AI roles, and English is my second language, so use simple words. Teach me the Python GIL, threads, processes and the difference between I/O-bound and CPU-bound work. Explain step by step with small timed experiments and real numbers, and connect each idea to everyday examples like a kitchen or a bank where it helps. After each part, ask me one question to check my understanding and wait for my answer before moving on.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data, ML and AI roles, and English is my second language. The topic is asyncio in Python: coroutines, await, gather, Semaphore, timeout and to_thread. Give me 8 graded exercises from easy to hard, using sleep to simulate network calls so I can run them offline. Do not give solutions yet. After I send my answers, review each one, show the bug if any, and explain the event loop in simple English.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data, ML and AI roles. Chapter topic: profiling and speeding up Python code with timeit, cProfile, tracemalloc and vectorization. Here is my slow code: [PASTE HERE]. Guide me to find the hot spot by measurement, not guessing. Ask me to run each profiling step and paste the output, then explain how to read it, propose one change at a time, and help me verify that results stay identical and the code is faster.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data, ML and AI roles, and English is my second language. Topic: Little's law (L = lambda x W), queues, back-pressure and worker pool sizing in Python services. Teach it using everyday examples (a bank queue, tellers, waiting time, customers served per hour). Then give me 6 sizing problems with real numbers, one at a time, and check each answer. Include one problem where the system saturates and explain why throughput stops growing.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data, ML and AI roles. Run a mock interview on Python concurrency and performance for a data engineer job. Ask me one question at a time: the GIL, threads versus processes versus asyncio, race conditions and locks, profiling workflow, vectorization, multiprocessing with pandas, and queues. Grade each answer from 1 to 5, give a model answer, and one improvement tip. After 10 questions, summarise my weak areas and what to practise next.
```

CHAPTER 12

APIS, THE WEB, SCRAPING AND COMMAND-LINE TOOLS

How programs talk to other programs over HTTP, and how you build tools that other people can run.

Imagine your manager says: "Pull yesterday's orders from the partner's system, score them with our model, and run it every night." To do that you must call a web service, understand its replies, handle failures, build a small service of your own, and wrap it all in a command you can schedule. By the end of this chapter you will be able to read and design a REST API, call it safely from Python, protect it with tokens, scrape a page politely, and package your work as a command-line tool. You have seen HTTP from the outside: every time you open a web page, your browser sends a request and waits for a reply. Now you will learn the inside.

Words you need. **API** (Application Programming Interface) is a doorway that lets one program use another program. **HTTP** is the language web programs use to talk. **JSON** is a plain-text format for data that looks like Python dicts and lists. **Client** is the program that asks. **Server** is the program that answers.

In this chapter, everything marked "run" was executed with Python 3.14 and only the standard library. Third-party libraries (`requests`, `httpx`, `FastAPI`, `Flask`, `typer`, `BeautifulSoup`, `Scrapy`, `Selenium`, `Playwright`) were **not run here** because this study machine had no network access to install them. Their code is short, standard and minimal. Check each library's current docs for version changes.

12.1 HTTP and REST Design

INTUITION

HTTP is a postal system with strict forms. You write a request on a form (method, address, headers, body), post it, and get back a reply form (status code, headers, body). REST is a set of habits for filling these forms in a predictable way, so that a stranger can guess how your service works. Every call a program makes to a web service is exactly one such form, like one order slip handed to a shop counter.

How it works

An HTTP request has four parts:

- **Method**: what you want to do. `GET` reads, `POST` creates, `PUT` replaces, `PATCH` changes part, `DELETE` removes.
- **URL**: the address, such as `/v1/items/2?limit=3`. The part after `?` is the **query string**.
- **Headers**: small labels, such as `Content-Type: application/json` or `Authorization: ...`.
- **Body**: the data (usually JSON) for `POST`, `PUT`, `PATCH`.

A response has a **status code**, headers and a body. Status codes come in families:

Code	Name	When to use
200	OK	Success with a body
201	Created	A new thing was made (add a `Location` header)
204	No Content	Success, nothing to return (common for DELETE)
400	Bad Request	The client sent nonsense
401	Unauthorized	Not logged in (really means "unauthenticated")
403	Forbidden	Logged in, but not allowed
404	Not Found	No such thing
409	Conflict	Clash with current state (duplicate)

Code	Name	When to use
422	Unprocessable	Valid JSON but wrong content
429	Too Many Requests	Rate limit hit
500	Internal Server Error	Your bug
503	Service Unavailable	Overloaded or down

REST design habits:

1. Use **nouns** in URLs, not verbs: `/items`, not `/getItems`.
2. Use plural collections: `/items` (all), `/items/2` (one).
3. The method carries the action: `GET /items/2`, `DELETE /items/2`.
4. Return JSON and correct status codes.
5. Be **stateless**: each request carries everything the server needs (for example a token).

Two properties matter a lot. A method is **safe** if it does not change data (`GET`). A method is **idempotent** if doing it many times has the same effect as doing it once (`GET`, `PUT`, `DELETE`). `POST` is normally not idempotent. This matters for retries (see 12.4).

Worked examples

Problem 12.1.1 Start a tiny web server inside a Python script, call it, and read the status, header and body. Also see what a missing page looks like.

Step 1	`http.server.HTTPServer` listens on a port. Port `0` means "pick any free port".
Step 2	A handler class defines `do_GET`. We send the status, headers, then the body.
Step 3	We run the server in a background thread and call it with `urllib.request`.

```
import threading, json, urllib.request, urllib.error
from http.server import BaseHTTPRequestHandler, HTTPServer

class H(BaseHTTPRequestHandler):
    def do_GET(self):
        if self.path == "/hello":
            body = b"hello, world"
            self.send_response(200)
            self.send_header("Content-Type", "text/plain")
            self.send_header("Content-Length", str(len(body)))
            self.end_headers()
            self.wfile.write(body)
        else:
            self.send_response(404)
            self.end_headers()
    def log_message(self, *a): pass

srv = HTTPServer(("127.0.0.1", 0), H)
port = srv.server_address[1]
threading.Thread(target=srv.serve_forever, daemon=True).start()
base = f"http://127.0.0.1:{port}"
with urllib.request.urlopen(base + "/hello") as r:
    print(r.status, r.reason)
    print(r.headers["Content-Type"])
    print(r.read().decode())
try:
    urllib.request.urlopen(base + "/nope")
except urllib.error.HTTPError as e:
    print("error:", e.code, e.reason)
srv.shutdown()
```

```
200 OK
text/plain
hello, world
error: 404 Not Found
```

- **Answer** The server replied 200 with text for `/hello`, and `urllib` raised `HTTPError` with code 404 for the unknown path. Note: `urllib` raises an exception for 4xx and 5xx codes, while `requests` does not (you call `raise_for_status()`).
- **Check** `404` is in the 4xx family ("client asked for something wrong"), and the output matches the status table above.
- **Why it matters** In a pipeline, an unhandled 404 or 503 stops the whole job. You must decide in code what each status means.

Problem 12.1.2 You must design URLs for a "delivery results" service. Fix these bad designs: `POST /createRun`, `GET /getRunById?id=5`, `GET /deleteRun/5`.

Step 1	Replace verbs with a noun collection: `runs`.
Step 2	Let the method express the action.
Step 3	Never use `GET` to delete: crawlers and browsers prefetch `GET` links.
Answer	`POST /v1/runs` (create, return 201), `GET /v1/runs/5` (read), `DELETE /v1/runs/5` (delete, return 204).
Check	Every URL is now a noun and the three actions use three different methods. No verb is left in a URL.
Why it matters	A new teammate can guess your API without reading docs. That is the whole point of REST.

Real-world mapping

HTTP piece	Example value	Where it appears in the job
Method	`POST`	The verb on the request form; `POST /predict` for a model
URL + query	`/v1/items?limit=3`	Ingestion code pages through data
Header	`Authorization: Bearer ...`	Login token in every pipeline call
Status	`429`	Your retry logic must slow down
Body	`{"name": "widget"}`	Data you send to create records

Predict, Run, Explain. In Problem 12.1.1, before you run it, ask: what does `urlopen` do for the unknown path `/nope`? Many beginners expect it to return a page with status 404. The real result is an exception, shown in the output above (`error: 404 Not Found`). The typical wrong version is:

```
r = urllib.request.urlopen(base + "/nope")   # no try/except
print(r.status)
```

It never prints a status. It stops with `urllib.error.HTTPError: HTTP Error 404: Not Found`. The fix is the `try/except` `urllib.error.HTTPError` you saw above. `urllib` treats 4xx and 5xx as errors; line by line: `urlopen` sends the request, sees code 404, and raises instead of returning.

The picture

Read this diagram top to bottom: the client writes the request lines, the server finds the item, and writes back a status line, headers, and the body.

```
Client                                  Server
  |  GET /v1/items/2 HTTP/1.1             |
  |  Host: example.com                    |
  |  Authorization: Bearer <token>   --->  |
  |                                       |  find item 2
  |  <---  HTTP/1.1 200 OK                |
  |        Content-Type: application/json  |
  |        {"id": 2, "name": "item2"}      |
```

Connection to the real job

- Data engineers pull data from REST APIs (CRM, payments, weather) in ingestion jobs.
- ML engineers serve a model as `POST /predict`.
- QA and support teams test these endpoints. Knowing 429, 503 and timeouts helps you read results.

Applications

- **Data Engineering:** nightly pull of orders from a partner API, page by page.
- **Machine Learning / MLOps:** a `POST /predict` endpoint behind a load balancer.
- **Data Analytics:** refreshing a dashboard from a SaaS tool's API.
- **Testing and QA:** every automated API test script is an HTTP client.

COMMON MISTAKE

Returning status 200 with `{"error": "..."}` in the body. Clients, monitors and automated checks look at the status code first. Use the right 4xx or 5xx code.

Check yourself

1. Which of GET, POST, PUT, DELETE are idempotent? Answer: GET, PUT and DELETE. POST normally is not.

2. What is the difference between 401 and 403? Answer: 401 means "I do not know who you are". 403 means "I know you, but you may not do this".
3. Why is `/getItems` poor REST design? Answer: The verb is already in the HTTP method. The URL should name the thing: `/items`.

12.2 Calling APIs: urllib, requests and httpx

INTUITION

A client library is a good envelope-maker. You say what you want to send. The library handles connections, encoding, headers, and reading the answer. `urllib` is the plain envelope that comes with Python. `requests` and `httpx` are nicer ones with better handles.

How it works

The standard library offers `urllib.request`. It works, but it is low level. Most people install `requests` (simple, synchronous) or `httpx` (same style, plus `async` support and HTTP/2). Their calls look alike.

Rules for good API clients, whichever library you use:

1. **Always set a timeout.** Without one, a hung server hangs your program forever.
2. **Check the status.** Call `raise_for_status()`.
3. **Retry only safe or idempotent calls**, with growing waits (exponential backoff).
4. **Reuse a connection** with a `Session` (requests) or `Client` (httpx). It is much faster for many calls.
5. **Never hardcode keys** (see Chapter 13).

Not run here (needs install and a network):

```
import requests

with requests.Session() as s:
    s.headers["Authorization"] = "Bearer <TOKEN>"
    r = s.get("https://api.example.com/v1/items", params={"limit": 3}, timeout=5)
    r.raise_for_status()
    items = r.json()["data"]
```

```
import httpx

with httpx.Client(timeout=5.0) as client:
    r = client.post("https://api.example.com/v1/items", json={"name": "widget"})
    r.raise_for_status()
    print(r.status_code, r.json())
```

Worked examples

Problem 12.2.1 Build a small JSON API with the standard library (list with pagination, get one, create, delete) and call it with `urllib`. Show status codes for success and failure.

Step 1	The server keeps items in a dict. It routes by path.
Step 2	It returns JSON with the right codes: 200, 201 (with `Location`), 204, 400, 404, 422.
Step 3	A helper `call()` sends a request and returns `(status, json, location)`, catching `HTTPError`.

```
import threading, json, urllib.request, urllib.error
from http.server import BaseHTTPRequestHandler, HTTPServer
from urllib.parse import urlparse, parse_qs

ITEMS = {i: {"id": i, "name": f"item{i}"} for i in range(1, 8)}
NEXT = [8]

class API(BaseHTTPRequestHandler):
    def _send(self, code, obj=None, headers=None):
        body = json.dumps(obj).encode() if obj is not None else b""
        self.send_response(code)
        self.send_header("Content-Type", "application/json")
        self.send_header("Content-Length", str(len(body)))
        for k, v in (headers or {}).items():
            self.send_header(k, v)
        self.end_headers()
        self.wfile.write(body)

    def do_GET(self):
        u = urlparse(self.path)
        q = parse_qs(u.query)
        if u.path == "/v1/items":
            limit = int(q.get("limit", ["3"])[0])
            offset = int(q.get("offset", ["0"])[0])
            data = list(ITEMS.values())[offset:offset + limit]
            nxt = offset + limit if offset + limit < len(ITEMS) else None
            return self._send(200, {"data": data, "next_offset": nxt, "total": len(ITEMS)})
        if u.path.startswith("/v1/items/"):
            try:
                i = int(u.path.rsplit("/", 1)[1])
            except ValueError:
                return self._send(400, {"error": "id must be an integer"})
            if i in ITEMS:
                return self._send(200, ITEMS[i])
            return self._send(404, {"error": "not found"})
        self._send(404, {"error": "no such route"})

    def do_POST(self):
        if self.path != "/v1/items":
            return self._send(404, {"error": "no such route"})
        n = int(self.headers.get("Content-Length", 0))
        try:
            payload = json.loads(self.rfile.read(n))
            name = payload["name"]
        except (ValueError, KeyError):
            return self._send(422, {"error": "need JSON with 'name'"})
        i = NEXT[0]; NEXT[0] += 1
        ITEMS[i] = {"id": i, "name": name}
        self._send(201, ITEMS[i], {"Location": f"/v1/items/{i}"})

    def do_DELETE(self):
        i = int(self.path.rsplit("/", 1)[1])
        ITEMS.pop(i, None)       # idempotent: same result every time
        self._send(204)

    def log_message(self, *a): pass

srv = HTTPServer(("127.0.0.1", 0), API)
threading.Thread(target=srv.serve_forever, daemon=True).start()
base = f"http://127.0.0.1:{srv.server_address[1]}"

def call(method, path, obj=None):
    data = json.dumps(obj).encode() if obj is not None else None
    req = urllib.request.Request(base + path, data=data, method=method,
```

```
                                headers={"Content-Type": "application/json"})
    try:
        with urllib.request.urlopen(req) as r:
            raw = r.read()
            return r.status, (json.loads(raw) if raw else None), r.headers.get("Location")
    except urllib.error.HTTPError as e:
        return e.code, json.loads(e.read()), None

print(call("GET", "/v1/items?limit=3"))
print(call("GET", "/v1/items?limit=3&offset=6"))
print(call("GET", "/v1/items/2"))
print(call("GET", "/v1/items/99"))
print(call("GET", "/v1/items/abc"))
print(call("POST", "/v1/items", {"name": "widget"}))
print(call("POST", "/v1/items", {"nom": "x"}))
print(call("DELETE", "/v1/items/1"))
print(call("DELETE", "/v1/items/1"))
srv.shutdown()
```

```
(200, {'data': [{'id': 1, 'name': 'item1'}, {'id': 2, 'name': 'item2'}, {'id': 3, 'name': 'item3'}], 'next_offset': 3,
'total': 7}, None)
(200, {'data': [{'id': 7, 'name': 'item7'}], 'next_offset': None, 'total': 7}, None)
(200, {'id': 2, 'name': 'item2'}, None)
(404, {'error': 'not found'}, None)
(400, {'error': 'id must be an integer'}, None)
(201, {'id': 8, 'name': 'widget'}, '/v1/items/8')
(422, {'error': "need JSON with 'name'"}, None)
(204, None, None)
(204, None, None)
```

- **Answer** Each call returned the status we designed. Deleting item 1 twice returned 204 both times: DELETE is idempotent.
- **Check** `ITEMS` started with 7 items. After the POST there were 8 and after the delete 7 again, which is why the page walk in 12.4.1 finds 7 names.
- **Why it matters** Every status above is something your client code must handle: retry, skip, or fail loudly.

Predict, Run, Explain. Before you run it, predict what `call("GET", "/v1/items/abc")` returns. The real answer was `(400, {'error': 'id must be an integer'}, None)`. Line by line: the server runs `int("abc")`, which raises `ValueError`; the `except` turns it into a 400. The typical wrong version has no `try/except` in the handler. Then the server thread crashes inside `int("abc")` with `ValueError: invalid literal for int() with base 10: 'abc'`, and the client gets a dropped connection instead of a clear 400. The fix is the `try/except` you see in `do_GET`.

Two more typical wrong versions you can reproduce (real output from a test run): a server that sleeps for one second, and a client that sets `timeout=0.2`, fails with `TimeoutError timed out`, and shows why a timeout is better than waiting forever. And `json.loads("")` on a 204 reply (empty body) fails with `JSONDecodeError: Expecting value: line 1 column 1 (char 0)`. That is why `call()` checks `if raw` before parsing.

Problem 12.2.2 Write a retry helper with exponential backoff and show it on a function that fails twice and then works.

Step 1	Wait `base * 2**attempt` seconds between tries (here tiny numbers so the demo is fast).
Step 2	Retry only on errors that are temporary.
Step 3	After the last try, raise the error.

```
import time

def retry(fn, tries=4, base=0.01, retry_on=(ConnectionError, TimeoutError)):
    for attempt in range(tries):
        try:
            return fn()
        except retry_on as e:
            if attempt == tries - 1:
                raise
            wait = base * 2 ** attempt
            print(f"attempt {attempt+1} failed ({e}); waiting {wait:.2f}s")
            time.sleep(wait)

calls = {"n": 0}
def flaky():
    calls["n"] += 1
    if calls["n"] < 3:
        raise ConnectionError("server reset")
    return "ok after %d calls" % calls["n"]

print(retry(flaky))
```

Before you run it, predict: how many times is `flaky()` called, and how long is each wait? Real output:

```
attempt 1 failed (server reset); waiting 0.01s
attempt 2 failed (server reset); waiting 0.02s
ok after 3 calls
```

- **Answer** The helper waited 0.01s, then 0.02s (doubling) and succeeded on the third call.
- **Check** Wait = base x 2^attempt: 0.01 x 2^0 = 0.01 and 0.01 x 2^1 = 0.02. `flaky` fails while n < 3, so success on call 3 is correct.
- **Why it matters** Doubling the wait gives a struggling server time to recover. Retrying instantly makes an outage worse (every client piles on).

Real-world mapping

Client habit	Example value	Where it appears in the job
Timeout	`timeout=5`	Stops a nightly job from hanging forever
Retry with backoff	wait 0.01, 0.02, 0.04 s	Surviving a 503 from an overloaded partner
Session / Client	one connection for 1,000 calls	Faster ingestion, lower load on the server
`raise_for_status()`	raises on 4xx/5xx	The job fails loudly instead of loading bad data

The picture

Read it left to right: your code sends through a session (with a timeout), the server answers, and the status code is checked before the data is used.

```
 your code --> Session (keeps connection open) --> server
      ^             |  timeout=5                       |
      |             v                                  |
      +--- raise_for_status() <---- status code <------+
```

Connection to the real job

- Ingestion scripts call APIs page by page and must survive 429 and 503.
- Calling a hosted LLM or model API is the same pattern: JSON in, JSON out, API key in a header.

- A Python client with `httpx` is also the base of traffic-generating tools such as Locust (named only as an example).

Applications

- **Data Engineering:** ingestion with retries and pagination.
- **GenAI:** calling a hosted language-model API with a key and a timeout.
- **MLOps:** health checks and smoke tests after a deploy.
- **Testing and QA:** API test automation with Python HTTP clients.

COMMON MISTAKE	Forgetting `timeout=`. The default in `requests` is to wait forever. A single stuck call can freeze a nightly pipeline. Always pass a timeout.

Check yourself

1. Why use a `Session`? Answer: It reuses the connection and shared headers, so many calls are faster and shorter to write.
2. Which calls are safe to retry blindly? Answer: Idempotent ones: GET, PUT, DELETE. For POST use an idempotency key.
3. What does `raise_for_status()` do? Answer: It raises an exception if the status is 4xx or 5xx.

12.3 Building APIs: FastAPI and Flask Basics

INTUITION	Writing a server by hand with `http.server` (as we did) shows how it works, but you would never ship it. A framework is a restaurant kitchen set up for you. You write the recipes (functions) and the framework handles orders, routing, validation, and docs.

How it works

FastAPI is a modern framework. You write a function, add a **decorator** (Chapter 7) that says "this is `GET /items/{item_id}`", and use **type hints** (Chapter 9) for validation. It uses `pydantic` to check bodies, and it creates an **OpenAPI** page automatically at `/docs`. It runs on an ASGI server such as `uvicorn`.

Flask is older and smaller. It gives routes and a request object. You add validation and other pieces yourself. It is simple and still common.

Not run here (needs `pip install fastapi uvicorn`):

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

app = FastAPI(title="Items API")
DB = {1: {"id": 1, "name": "pen"}}

class ItemIn(BaseModel):
    name: str

@app.get("/v1/items/{item_id}")
def get_item(item_id: int):
    if item_id not in DB:
        raise HTTPException(status_code=404, detail="not found")
    return DB[item_id]

@app.post("/v1/items", status_code=201)
def create_item(item: ItemIn):
    new_id = max(DB) + 1
    DB[new_id] = {"id": new_id, **item.model_dump()}
    return DB[new_id]
```

Start it with `uvicorn main:app --reload`, then open `http://127.0.0.1:8000/docs`. If you send `item_id=abc`, FastAPI answers 422 on its own because of the `int` hint.

The same in Flask (not run here):

```
from flask import Flask, jsonify, request, abort

app = Flask(__name__)
DB = {1: {"id": 1, "name": "pen"}}

@app.get("/v1/items/<int:item_id>")
def get_item(item_id):
    if item_id not in DB:
        abort(404)
    return jsonify(DB[item_id])

@app.post("/v1/items")
def create_item():
    data = request.get_json()
    new_id = max(DB) + 1
    DB[new_id] = {"id": new_id, "name": data["name"]}
    return jsonify(DB[new_id]), 201
```

Worked examples

Problem 12.3.1 You have the FastAPI app above. What does `POST /v1/items` with body `{"name": 5}` return, and why?

Step 1	`ItemIn.name` is declared `str`.
Step 2	Pydantic validates the body before your function runs. In its default (lax) mode, an integer is not turned into a string.
Step 3	Validation fails, so FastAPI replies with a 422 error and a JSON description of the problem. (Not run here; this is the documented behaviour.)
Answer	A 422 response. Your function is never called. This is why type hints are worth the effort.
Check	The docs list 422 as FastAPI's code for validation errors, and the same `str` declaration appears in `ItemIn`.
Why it matters	A bad client cannot crash your handler or poison your database.

Problem 12.3.2 Using only the standard library, write the validation step that FastAPI does for you: accept a dict and return clean data or a list of errors.

Step 1	Define the required fields and types.
Step 2	Loop and collect messages.
Step 3	Return `(data, errors)`.

```
def validate_item(payload):
    errors = []
    if not isinstance(payload, dict):
        return None, ["body must be a JSON object"]
    name = payload.get("name")
    if not isinstance(name, str) or not name.strip():
        errors.append("name: must be a non-empty string")
    if "price" in payload and not isinstance(payload["price"], (int, float)):
        errors.append("price: must be a number")
    return ({"name": name.strip(), **({"price": payload["price"]} if "price" in payload else {})}
            if not errors else None), errors

print(validate_item({"name": " pen ", "price": 10}))
print(validate_item({"name": 5, "price": "ten"}))
print(validate_item([1, 2]))
```

```
({'name': 'pen', 'price': 10}, [])
(None, ['name: must be a non-empty string', 'price: must be a number'])
(None, ['body must be a JSON object'])
```

- **Answer** Validation returns clean data or all the errors at once, which is also how good APIs behave: report every problem, not just the first.
- **Check** Input 2 has two wrong fields and the output lists exactly two messages. Input 1 has none, and the name is trimmed from `' pen '` to `'pen'`.
- **Why it matters** Pydantic does this work for you with less code, but now you know what it does.

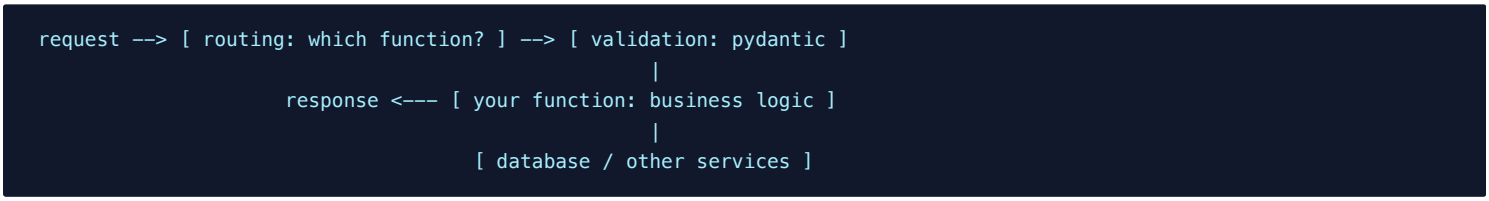
Real-world mapping

Framework piece	Example	Where it appears in the job
Route decorator	`@app.get("/v1/items/{item_id}")`	One function per endpoint of a model service
Type hint	`item_id: int`	Auto 422 for bad input, less defensive code
pydantic model	`class ItemIn(BaseModel)`	The "schema" for a request body
`/docs` page	auto OpenAPI UI	Analysts try your endpoint without code

Predict, Run, Explain. Before you read on: in the Flask example, what happens if the client posts a body with no `name` key? The code does `data["name"]`, which raises `KeyError: 'name'`, and Flask answers with a 500 error (a server bug, not the client's fault). This is the typical wrong version. The fix is validation like the function in Problem 12.3.2 and returning 422. FastAPI avoids it because the pydantic model rejects the body before your code runs (not run here).

The picture

Read it top to bottom: the request is routed, then validated, then your function runs, which may call a database.



Connection to the real job

- Model serving: `POST /predict` loads a model once at start-up and calls it per request.
- Data teams expose "data products" as small internal APIs.

- A FastAPI service with a database and tests is one of the skill-map mastery tests (see Chapter 14).

Applications

- **Machine Learning / MLOps:** a `POST /predict` endpoint that returns a score.
- **Data Engineering:** a small service that receives webhook events and writes to a queue.
- **GenAI:** a thin API that adds your prompt and guardrails in front of a language model.
- **Testing and QA:** a mock service so you can test without touching production.

COMMON MISTAKE	Loading a big ML model inside the request function. It reloads on every call and your latency explodes. Load it once at start-up, outside the handler.

Check yourself

1. What gives FastAPI its automatic validation? Answer: Type hints plus pydantic models.
2. What does FastAPI's `/docs` page show? Answer: Interactive documentation made from the OpenAPI description of your routes.
3. Where should a heavy model be loaded? Answer: Once, at application start-up, not per request.

12.4 Pagination, Idempotency, Versioning, OpenAPI and GraphQL

INTUITION	A server cannot return a million rows in one reply, so it hands them over page by page (pagination). Networks fail, so clients retry, and the server must not charge a card twice (idempotency). Services live for years, so old clients must keep working while new ones arrive (versioning). A contract page (OpenAPI) lets everyone agree on the shape.

How it works

Pagination styles:

Style	Example	Good	Weak
Offset/limit	`?limit=3&offset=6`	Simple, jump to any page	Slow for big offsets; rows shift when data changes
Cursor	`?after=abc123`	Stable and fast	Cannot jump to page 50
Page number	`?page=3&size=20`	Familiar	Same weakness as offset

Idempotency key. For `POST` calls that must not repeat (payments, orders), the client sends a unique header such as `Idempotency-Key: k1`. The server remembers keys. A repeat with the same key returns the first result and does not act again.

Versioning. Put the version in the URL (`/v1/items`) or in a header. Never break `v1` . Add `v2` when you must change the shape.

OpenAPI is a standard JSON/YAML description of an API: routes, parameters, bodies and responses. Tools generate documentation, client code and tests from it. FastAPI makes it for you.

GraphQL (awareness). Instead of many URLs, there is one endpoint and the client writes a query that names exactly the fields it wants. It avoids over-fetching, but it is harder to cache and to rate-limit. REST is still the default choice.

Worked examples

Problem 12.4.1 Walk every page of a 7-item collection with `limit=3`, using the `next_offset` field of our API. How many requests are needed?

Step 1	Start at offset 0. Ask for 3 items. Server says `next_offset=3`.
Step 2	Offset 3 gives 3 more, `next_offset=6`.
Step 3	Offset 6 gives 1 item (item 7) and `next_offset=None`, so stop.

The loop (it used the server of Problem 12.2.1, after item 1 was deleted and "widget" was added, so there were 7 items):

```
off, names = 0, []
while off is not None:
    s, body, _ = call("GET", f"/v1/items?limit=3&offset={off}")
    names += [d["name"] for d in body["data"]]
    off = body["next_offset"]
print(names)
```

```
['item2', 'item3', 'item4', 'item5', 'item6', 'item7', 'widget']
```

- **Answer** 3 requests: ceil(7 / 3) = 3. All 7 names were collected.
- **Check** Pages hold 3 + 3 + 1 = 7 items, matching `total: 7`.
- **Why it matters** If you stop after page 1 you lose data silently. The loop must end on `next_offset is None`.

Problem 12.4.2 Implement an idempotency key on a payment endpoint and show that a repeated key does not create a second payment.

Step 1	The server keeps a dict `seen` of key to result.
Step 2	First time: create, return 201. Repeat: return stored result with 200.
Step 3	The client sends `k1`, `k1`, `k2`.

```
import urllib.request, json, threading
from http.server import BaseHTTPRequestHandler, HTTPServer

seen = {}
class H(BaseHTTPRequestHandler):
    def do_POST(self):
        key = self.headers.get("Idempotency-Key")
        n = int(self.headers["Content-Length"]); self.rfile.read(n)
        if key in seen:
            code, body = 200, seen[key]
        else:
            seen[key] = {"payment_id": len(seen) + 1}
            code, body = 201, seen[key]
        out = json.dumps(body).encode()
        self.send_response(code); self.send_header("Content-Length", str(len(out)))
        self.end_headers(); self.wfile.write(out)
    def log_message(self, *a): pass

srv = HTTPServer(("127.0.0.1", 0), H)
threading.Thread(target=srv.serve_forever, daemon=True).start()
url = f"http://127.0.0.1:{srv.server_address[1]}/pay"
for key in ["k1", "k1", "k2"]:
    req = urllib.request.Request(url, data=b"{}", method="POST", headers={"Idempotency-Key": key})
    with urllib.request.urlopen(req) as r:
        print(key, r.status, r.read().decode())
srv.shutdown()
```

```
k1 201 {"payment_id": 1}
k1 200 {"payment_id": 1}
k2 201 {"payment_id": 2}
```

- **Answer** The second `k1` call returned the same payment (id 1) and did not create id 2. The retry was safe.
- **Check** Three calls, two distinct keys, two payments (ids 1 and 2).
- **Why it matters** In plain words: the customer is charged once even if the phone retries five times.

Real-world mapping

Idea	Example value	Where it appears in the job
Offset paging	`limit=3&offset=6`	Simple partner APIs; 3 requests for 7 rows
Cursor paging	`after=abc123`	Large event feeds, social data
Idempotency key	`Idempotency-Key: k1`	Payments; safe to retry
Version in URL	`/v1/items`	Old clients keep working
`Retry-After` header	`Retry-After: 30`	Tells your ingest job how long to pause on 429

Predict, Run, Explain. Before running Problem 12.4.2, predict the status of the second `k1` call. A typical wrong prediction is 201 again. The real output shows `200` with the same `payment_id`. Line by line: `key in seen` is true, so we return the stored result and do not create anything. The typical wrong version has no `seen` dict and increments a counter every time, so a retry would make payment 2 and the customer pays twice (no error message, which is exactly why it is dangerous: nothing fails loudly).

The picture

Offset paging (left) numbers pages by position, so inserting a row at the front moves everything. Cursor paging (right) remembers the last id seen.

```
Offset paging: rows shift          Cursor paging: bookmark
page1 [1 2 3] | insert row 0       after=3 -> [4 5 6]
page2 [3 4 5]  (3 repeated!)       after=6 -> [7]
```

Connection to the real job

- Ingestion code must loop over pages, and must handle the last page and rate limits (429, `Retry-After` header).
- Idempotent writes make pipelines safe to re-run. This is a core data-engineering idea (Chapter 13 of the data engineering documents expands it).
- OpenAPI files let you generate test data and client test scripts.

Applications

- **Data Engineering:** page loops and idempotent loads that can be re-run safely.
- **Data Analytics:** export APIs that return reports in pages.
- **Machine Learning / MLOps:** versioned model endpoints (`/v1/predict` , `/v2/predict`) during a rollout.
- **Testing and QA:** OpenAPI files used to generate test scripts.

COMMON MISTAKE

Using offset paging on a table that changes while you read it. You can skip or repeat rows. Prefer cursors for large or live data, and always stop when the next-page marker is empty rather than when a page "looks short".

Check yourself

1. Why cursor over offset for big data? Answer: It is faster at depth and does not shift when rows are inserted.

2. What is an idempotency key for? Answer: To make a repeated POST return the first result instead of acting twice.
3. Where can the API version live? Answer: In the URL (`/v1/`) or a header.

12.5 Authentication and Authorization Basics

INTUITION	Authentication answers "who are you?". Authorization answers "what are you allowed to do?". A hotel desk checks your ID (authentication), then gives you a key card that opens only your room (authorization).

How it works

- **API key**: a long random string sent in a header (`X-API-Key: ...`). Simple. It identifies the calling application. Treat it like a password.
- **Bearer token**: `Authorization: Bearer <token>` . The token proves you logged in.
- **OAuth 2.0**: a standard for letting an app act for a user without sharing the password. The user logs in at the provider, and the app receives a short-lived **access token** (and often a **refresh token**). **OIDC** adds identity on top of OAuth 2.0.
- **JWT (JSON Web Token)**: a token made of three base64 parts, `header.payload.signature` . The payload holds **claims** (who, role, expiry `exp`). The signature (for example HMAC-SHA256 with a secret key) lets the server detect any change. A JWT is **signed, not encrypted**: anyone can read the payload. Never put secrets in it.
- **Always use HTTPS** so tokens are not readable in transit.

Worked examples

Problem 12.5.1 Build a JWT-like token with HMAC-SHA256 by hand, verify it, then show that expiry and tampering are detected. The key is a fake demo value.

Step 1	Encode header and payload as compact JSON, then base64url.
Step 2	Sign `header.payload` with HMAC.
Step 3	To verify, recompute the signature and compare with `hmac.compare_digest` (safe against timing attacks). Then check `exp` .

```
import hmac, hashlib, base64, json

SECRET = b"demo-secret-not-real"

def b64(b): return base64.urlsafe_b64encode(b).rstrip(b"=").decode()
def unb64(s): return base64.urlsafe_b64decode(s + "=" * (-len(s) % 4))

def make_token(claims):
    header = b64(json.dumps({"alg": "HS256", "typ": "JWT"}, separators=(",", ":")).encode())
    payload = b64(json.dumps(claims, separators=(",", ":")).encode())
    sig = b64(hmac.new(SECRET, f"{header}.{payload}".encode(), hashlib.sha256).digest())
    return f"{header}.{payload}.{sig}"

def verify(token, now):
    h, p, s = token.split(".")
    good = b64(hmac.new(SECRET, f"{h}.{p}".encode(), hashlib.sha256).digest())
    if not hmac.compare_digest(s, good):
        return "bad signature"
    claims = json.loads(unb64(p))
    if claims["exp"] < now:
        return "expired"
    return claims

t = make_token({"sub": "asha", "role": "analyst", "exp": 1000})
print(t)
print(verify(t, now=500))
print(verify(t, now=1500))
h, p, s = t.split(".")
evil = b64(json.dumps({"sub": "asha", "role": "admin", "exp": 1000}, separators=(",", ":")).encode())
print(verify(f"{h}.{evil}.{s}", now=500))
print(json.loads(unb64(p)))
```

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJhc2hhIiwicm9sZSI6ImFuYWx5c3QiLCJleHAiOjEwMDB9.MvOjd9So6hnZyxNsG6Q5vsm-EzJ80cyE8aR_RK5JuRI
{'sub': 'asha', 'role': 'analyst', 'exp': 1000}
expired
bad signature
{'sub': 'asha', 'role': 'analyst', 'exp': 1000}
```

- **Check** `exp` is 1000: 500 < 1000 passes, 1500 > 1000 fails, which matches the output.
- **Why it matters** Expiry limits damage if a token leaks; the signature stops forged roles.
- **Answer** At time 500 the token is valid; at 1500 it is expired. When we swapped the payload to `role: admin` and kept the old signature, verification said "bad signature". The last line shows that anyone can read the payload: signed is not secret. In real projects use a maintained library (such as PyJWT) and never write your own crypto; this demo only teaches the idea.

Problem 12.5.2 A route `DELETE /v1/runs/5` is called by user "ravi" (role "viewer") with a valid token. Which status code, and why?

Step 1	The token is valid, so authentication passed.
Step 2	Role "viewer" cannot delete, so authorization fails.
Answer	403 Forbidden. A 401 would be wrong because the user is known.
Check	Ask: "Does the server know who this is?" Yes, so not 401. "Is the action allowed?" No, so 403.
Why it matters	When many requests fail at once, a wave of 401s means your login or token reuse is broken; a wave of 403s means a permissions problem. Different bugs, different fixes.

Real-world mapping

Credential	Example value	Where it appears in the job
API key	`X-API-Key: <KEY>`	Calling a model or SaaS API from a script
Bearer token	`Authorization: Bearer <TOKEN>`	After login (token extracted, then reused on every call)
JWT claims	`{"sub": "asha", "role": "analyst", "exp": 1000}`	Data API deciding what you may read
Refresh token	long-lived, stored safely	Notebook that must run for days

Predict, Run, Explain. Before running Problem 12.5.1, predict what `verify` says for the tampered token. The real output is `bad signature`. Line by line: the attacker changed the payload but cannot recompute the HMAC without the secret, so the stored signature no longer matches the new `header.payload`. A typical wrong version compares with `==` and, worse, a very common bug is to decode the payload and trust it without checking the signature. Then the tampered token (`role: admin`) would be accepted and no error appears at all.

The picture

A JWT is three base64 pieces joined by dots. The signature is a fingerprint of the first two pieces.

```
eyJhbGciOi...  .  eyJzdWIiOi...  .  MvOjd9So6h...
\___header___/    \__payload___/    \_signature_/
alg, type         claims: sub,     HMAC(secret, header.payload)
                  role, exp        changes if any byte changes
```

Connection to the real job

- Every cloud and data API (warehouse, model service, BI tool) needs a token or key.
- Service accounts and short-lived tokens are the safe standard.
- Role checks in data APIs decide who sees which data (privacy).

Applications

- **Data Engineering:** service accounts with short-lived tokens for warehouse and cloud access.
- **Machine Learning / MLOps:** API keys for model endpoints, rotated regularly.
- **GenAI:** keys for hosted model APIs, kept out of notebooks and Git.
- **Testing and QA:** token extraction and reuse in test scripts; webhook signature checks use the same HMAC idea.

COMMON MISTAKE

Thinking a JWT is encrypted. It is only signed. Do not store passwords or personal data inside it, and always verify the signature and the expiry on the server.

Check yourself

1. 401 or 403 for a known user without permission? Answer: 403.
2. Can a client read a JWT payload? Answer: Yes. It is base64, not encrypted.
3. Why `compare_digest` ? Answer: It compares in constant time so attackers cannot guess the signature by timing.

12.6 Scraping Responsibly

INTUITION

Scraping is reading a web page with a program instead of your eyes. It is like photocopying pages in a library: allowed in many cases, but there are rules posted at the door, and if you run ten copy machines at once you annoy everyone.

How it works

Order of preference: first use an official API, then a downloadable dataset, and only then scrape.

Be a good citizen:

1. Read **robots.txt** (the site's posted rules) and obey it.
2. Read the **terms of service**. Some sites forbid scraping.
3. Identify yourself with a clear `User-Agent` and a contact.
4. Go slowly: add a delay (respect `Crawl-delay`) and limit parallelism.
5. Do not collect personal data without a lawful reason. Do not bypass logins or paywalls.
6. Cache pages so you do not fetch the same page twice.

Tools: `html.parser` (standard library), **BeautifulSoup** (friendly parsing of HTML), **Scrapy** (a framework for large crawls), **Selenium** and **Playwright** (control a real browser, needed when a page builds its content with JavaScript). BeautifulSoup, Scrapy, Selenium and Playwright were not run here.

Not run here:

```
import requests
from bs4 import BeautifulSoup

r = requests.get("https://example.com/shop", headers={"User-Agent": "StudyBot/1.0 (me@example.com)"}, timeout=10)
soup = BeautifulSoup(r.text, "html.parser")
for li in soup.select("li.p"):
    print(li.a.text, li.select_one(".price").text)
```

Worked examples

Problem 12.6.1 Parse a robots.txt text and decide which URLs your bot may fetch.

Step 1	Use `urllib.robotparser.RobotFileParser` and feed it lines with `parse()` (no network needed).
Step 2	Ask `can_fetch(user_agent, url)`.
Step 3	Read the crawl delay.

```
from urllib.robotparser import RobotFileParser

rules = """User-agent: *
Disallow: /private/
Crawl-delay: 5

User-agent: BadBot
Disallow: /
"""
rp = RobotFileParser()
rp.parse(rules.splitlines())
for ua, url in [("MyStudyBot", "https://example.com/products"),
                ("MyStudyBot", "https://example.com/private/data"),
                ("BadBot", "https://example.com/products")]:
    print(ua, url, rp.can_fetch(ua, url))
print("crawl delay:", rp.crawl_delay("MyStudyBot"))
```

```
MyStudyBot https://example.com/products True
MyStudyBot https://example.com/private/data False
BadBot https://example.com/products False
crawl delay: 5
```

- **Answer** Our bot may fetch `/products` but not `/private/data`, and must wait 5 seconds between requests. "BadBot" is banned everywhere.

- **Check** `/private/data` starts with `/private/`, so the rule matches. `/products` does not.
- **Why it matters** Obeying these rules keeps you legal, polite and unblocked.

Problem 12.6.2 Extract name, link and price from an HTML snippet with the standard library parser.

Step 1	Subclass `HTMLParser`. `handle_starttag` sees tags and attributes. `handle_data` sees text.
Step 2	When we see `<a href=...>`, remember the link. The next text is the name.
Step 3	When we see `<span class="price">`, the next text is the price.

```
from html.parser import HTMLParser

html = """<html><body><h1>Shop</h1>
<ul><li class="p"><a href="/a">Pen</a> <span class="price">10</span></li>
<li class="p"><a href="/b">Book</a> <span class="price">250</span></li></ul></body></html>"""

class P(HTMLParser):
    def __init__(self):
        super().__init__()
        self.links = []; self.prices = []; self.names = []
        self._in_price = False; self._href = None
    def handle_starttag(self, tag, attrs):
        a = dict(attrs)
        if tag == "a": self._href = a.get("href")
        if tag == "span" and a.get("class") == "price": self._in_price = True
    def handle_data(self, d):
        if self._href:
            self.names.append(d); self.links.append(self._href); self._href = None
        elif self._in_price:
            self.prices.append(int(d)); self._in_price = False

p = P(); p.feed(html)
print(list(zip(p.names, p.links, p.prices)))
```

```
[('Pen', '/a', 10), ('Book', '/b', 250)]
```

- **Check** The input has two `<li>` items and the result has two tuples. Prices are `int`, not text, so you can sum them.
- **Answer** We got two products with names, links and prices as numbers. BeautifulSoup does the same job in three lines, but it is good to know what is under the hood.

Real-world mapping

Rule	Example value	Where it appears in the job
robots.txt	`Disallow: /private/`	Check before any crawl
Crawl delay	5 seconds	300 pages take at least 25 minutes
User-Agent	`StudyBot/1.0 (me@example.com)`	Lets site owners contact you
Cache	save HTML to disk	Re-parse without re-fetching

Predict, Run, Explain. Before running Problem 12.6.1, predict the answer for `BadBot`. It is `False` because the group for `BadBot` says `Disallow: /`, which blocks everything; the `*` group does not apply to a bot that has its own group. A typical wrong version forgets `parse()` and calls `can_fetch` on an empty parser. An empty parser (no rules loaded) behaves as if everything is allowed, so you get `True` with no error. The fix: always call `parse()` or `read()` first and check the rules were loaded.

The picture

Read it as a series of gates. You only reach the last box if every gate lets you through.

```
official API?  --yes--> use it
     | no
robots.txt allows?  --no--> stop
     | yes
terms of service ok? --no--> stop
     | yes
fetch slowly, identify yourself, cache, store only what you need
```

Connection to the real job

- Price monitoring, job-market analysis, and research datasets often start with scraping. The result above means: 2 products, total price 10 + 250 = 260.
- A legal or privacy mistake can end a project. Teams check terms before building.
- Pages built by JavaScript need a browser tool, which is slower and heavier.

Applications

- **Data Analytics:** collecting public prices to compare competitors (where terms allow).
- **Data Science:** building a small text dataset for an analysis.
- **Machine Learning / GenAI:** collecting documents for search or retrieval, with permission and attention to copyright and privacy.
- **Data Engineering:** a polite, scheduled crawler with caching and monitoring.

COMMON MISTAKE	Hammering a site with parallel requests and no delay. It looks like an attack and may get your IP banned. Add delays, a User-Agent, and caching. Scrape only what you are allowed to.

Check yourself

1. What file lists the site's crawling rules? Answer: `robots.txt`.
2. When do you need Selenium or Playwright? Answer: When the content is created by JavaScript in the browser.
3. What comes before scraping in your order of choice? Answer: An official API or a downloadable dataset.

12.7 Command-Line Tools: argparse, typer and subprocess

INTUITION	A command-line tool is a function with a keyboard-friendly front door. Instead of editing the script each time, you pass options: `summ 120 80 300 -p 90`. Scripts become reusable tools for you and your team, and schedulers (cron, Airflow) can run them.

How it works

`argparse` (standard library) builds the parser. You declare arguments, their types, defaults and help text. It gives you `--help` and clear errors for free, and exits with code 2 on a usage error.

typer (third-party) makes the same thing from a function with type hints: the parameters become the options. Not run here:

```
import typer

app = typer.Typer()

@app.command()
def summ(numbers: list[float], percentile: int = 90, verbose: bool = False):
    """Summarise delivery times."""
    ...

if __name__ == "__main__":
    app()
```

Good CLI habits: put the work in a function `main(argv=None)` so tests can call it, return proper exit codes (0 good, non-zero bad), print results to stdout and logs to stderr (through `logging`).

subprocess runs other programs from Python. Use `subprocess.run([...], capture_output=True, text=True, check=True)`. Pass a **list**, not a string with `shell=True`, so that user input cannot inject shell commands.

Worked examples

Problem 12.7.1 Write a CLI called `summ` that prints count, mean and a percentile of delivery times (nearest-rank). Show normal use, `--help`, and an error.

Step 1	Declare `numbers` (one or more floats), `-p` percentile (default 90, between 1 and 99), `-v` verbose.
Step 2	Sort. Nearest-rank index is `ceil(n * p / 100) - 1`.
Step 3	Run it.

```
import argparse, logging

def build():
    p = argparse.ArgumentParser(prog="summ", description="Summarise delivery times")
    p.add_argument("numbers", type=float, nargs="+", help="delivery times in minutes")
    p.add_argument("-p", "--percentile", type=int, default=90, choices=range(1, 100), metavar="P")
    p.add_argument("-v", "--verbose", action="store_true")
    return p

def main(argv=None):
    a = build().parse_args(argv)
    logging.basicConfig(level=logging.DEBUG if a.verbose else logging.WARNING,
                        format="%(levelname)s %(message)s")
    xs = sorted(a.numbers)
    logging.debug("got %d numbers", len(xs))
    k = max(0, -(-len(xs) * a.percentile // 100) - 1)
    print(f"count={len(xs)} mean={sum(xs)/len(xs):.1f} p{a.percentile}={xs[k]}")

if __name__ == "__main__":
    main()
```

Saved as `s5.py` and run in the shell:

```
python3 s5.py 120 80 300 95 110 -p 90 -v
python3 s5.py --help
python3 s5.py abc; echo "exit=$?"
```

```
DEBUG got 5 numbers
count=5 mean=141.0 p90=300.0
usage: summ [-h] [-p P] [-v] numbers [numbers ...]

Summarise delivery times

positional arguments:
  numbers             delivery times in minutes

options:
  -h, --help          show this help message and exit
  -p, --percentile P
  -v, --verbose
usage: summ [-h] [-p P] [-v] numbers [numbers ...]
summ: error: argument numbers: invalid float value: 'abc'
exit=2
```

- **Check by hand** Sorted: 80, 95, 110, 120, 300. Mean = 705 / 5 = 141.0. For p90: ceil(5 x 0.9) = 5, index 4, value 300.
- **Check** Mean by hand: 705 / 5 = 141.0. Percentile index: ceil(5 x 90 / 100) - 1 = 4, value 300.0.
- **Why it matters** This is the same nearest-rank percentile idea used in exam-mark and delivery reports; a p90 of 300 minutes means about 9 of 10 deliveries were at least as fast (here 4 of 5 samples are below it).
- **Answer** The tool prints `count=5 mean=141.0 p90=300.0`, gives help text, and rejects bad input with exit code 2. (The `--help` layout may differ slightly between Python versions.)

Problem 12.7.2 Run a child Python process, capture its output and exit code, and see what `check=True` does. Then use `sqlite3` (standard library database) from the same script to store and query results with `?` placeholders.

Step 1	`subprocess.run` with a list and `capture_output=True`.
Step 2	`check=True` raises `CalledProcessError` on a non-zero code.
Step 3	With a list, `"a; echo b"` is one harmless argument, not two commands.
Step 4	`sqlite3` needs no server: `connect(":memory:")` makes a temporary database.

```
import subprocess, sys, sqlite3

r = subprocess.run([sys.executable, "-c", "print('hi from child'); import sys; sys.exit(3)"],
                   capture_output=True, text=True)
print(r.returncode, repr(r.stdout), repr(r.stderr))
try:
    subprocess.run([sys.executable, "-c", "raise SystemExit(2)"], check=True)
except subprocess.CalledProcessError as e:
    print("failed with", e.returncode)
r = subprocess.run(["echo", "a; echo b"], capture_output=True, text=True)
print(r.stdout.strip())

con = sqlite3.connect(":memory:")
con.row_factory = sqlite3.Row
con.execute("CREATE TABLE req(id INTEGER PRIMARY KEY, url TEXT, ms REAL)")
con.executemany("INSERT INTO req(url, ms) VALUES (?, ?)",
                [("/a", 120.0), ("/a", 80.0), ("/b", 300.0), ("/b", 100.0), ("/a", 100.0)])
con.commit()
for row in con.execute("SELECT url, COUNT(*) n, AVG(ms) avg FROM req GROUP BY url ORDER BY url"):
    print(dict(row))
with con:
    con.execute("UPDATE req SET ms = ms * 2 WHERE url = ?", ("/b",))
print(con.execute("SELECT SUM(ms) FROM req").fetchone()[0])
```

```
3 'hi from child\n' ''
failed with 2
a; echo b
{'url': '/a', 'n': 3, 'avg': 100.0}
{'url': '/b', 'n': 2, 'avg': 200.0}
1100.0
```

- **Answer** The child returned code 3 with its output captured. `check=True` turned code 2 into an exception. The `with con:` block committed the update. The sums check out: `/a` is 120+80+100 = 300, `/b` is (300+100) x 2 = 800, total 1100.

Real-world mapping

CLI piece	Example value	Where it appears in the job
Positional argument	`120 80 300`	The input files or numbers
Option with default	`-p 90`	`--date` , `--env prod` for ETL jobs
Flag	`-v`	Turn on debug logging
Exit code	`0` or `2`	Scheduler/CI decides pass or fail

Predict, Run, Explain. Before you run it, what does `python3 s5.py abc` print and what is the exit code? Real output: `summ: error: argument numbers: invalid float value: 'abc'` and exit code 2. Line by line: `type=float` runs `float("abc")` , which fails, so `argparse` prints usage and calls `sys.exit(2)` . The typical wrong version skips `type=float` and then sorts strings: `sorted(["120", "80"])` gives `['120', '80']` (text order), so the percentile is wrong and no error appears. The fix is `type=float` .

The shell injection warning has a real demonstration. With `shell=True` , `subprocess.run("echo hi; echo INJECTED", shell=True)` printed both `hi` and `INJECTED` : two commands ran from one string. If part of that string came from a user, they could run anything.

The picture

Read left to right: the shell passes words, `argparse` turns them into named values, `main` does the work, and the results leave through three channels.

```
shell:  summ 120 80 300 -p 90
          |        |        |
          v        v        v
       argparse --> Namespace(numbers=[...], percentile=90)
                          |
                     main() --> stdout (results), stderr (logs), exit code
```

Connection to the real job

- Every scheduled data job is a CLI: `python etl.py --date 2026-01-31` .
- Docker containers and CI steps run commands, so exit codes decide pass or fail.
- `subprocess` glues Python to tools like `git` , `dbt` or `ffmpeg` .

Applications

- **Data Engineering:** ETL entry points scheduled by cron or Airflow.
- **Machine Learning / MLOps:** `python train.py --config prod.yaml` inside a container.
- **Data Analytics:** a report generator that takes `--month 2026-01` .
- **Testing and QA:** a wrapper that runs an external test tool and summarises its results.

COMMON MISTAKE	Building a shell command from user text with `shell=True` (for example `f"grep {term} file"`). A value like `x; rm -rf ~` becomes a second command. Pass a list of arguments instead.

Check yourself

1. What exit code means success? Answer: 0.
2. Why put the logic in `main(argv=None)`? Answer: So a test can call `main([...])` without a real shell.
3. Why does `?` matter in `sqlite3`? Answer: It passes values separately from the SQL text, preventing SQL injection (Chapter 13).

12.8 Talking to a Database from Python: sqlite3, Transactions and the N+1 Trap

Almost every API ends up reading and writing a database. Python ships with `sqlite3`, a complete SQL database stored in one file (or in memory with `":memory:"`), so you can learn the whole workflow without installing anything. The SQL language itself is taught in the SQL document; here we focus on how Python talks to it safely.

The four moves: connect, parameterise, commit, fetch

- **Connect:** `con = sqlite3.connect("app.db")` opens the file (creating it if missing).
- **Parameterise:** put `?` placeholders in the SQL and pass values as a tuple. Never build SQL with f-strings (see Chapter 13 for why).
- **Commit:** changes are only permanent after `con.commit()`. Using `with con:` commits when the block ends normally and rolls back if an exception escapes.
- **Fetch:** `fetchone()`, `fetchall()`, or loop over the cursor. Set `con.row_factory = sqlite3.Row` to read columns by name instead of by position.

Problem 12.8.1 A bank has accounts ana (100) and ben (50). Transfer 30 from ana to ben. Then try to transfer 500, which would make ana negative. The table forbids negative balances. Show that the failed transfer leaves no half-finished state.

Idea	A transfer is two UPDATEs that must both happen or neither. That is a transaction.
Step 1	Wrap both UPDATEs in `with con:`.
Step 2	The first UPDATE of the second call breaks `CHECK (balance >= 0)`, so sqlite3 raises `IntegrityError`.
Step 3	The exception leaves the `with` block, so the transaction is rolled back automatically.
Check	Balances after the failure equal balances before it: ana 70, ben 80, total still 150.

```
import sqlite3
con = sqlite3.connect(":memory:")
con.row_factory = sqlite3.Row
con.execute("CREATE TABLE account (id INTEGER PRIMARY KEY, owner TEXT NOT NULL, balance INTEGER NOT NULL CHECK (balance >=
0))")
con.executemany("INSERT INTO account (owner, balance) VALUES (?, ?)", [("ana", 100), ("ben", 50)])
con.commit()

def transfer(con, src, dst, amount):
    with con:  # commits on success, rolls back on any exception
        con.execute("UPDATE account SET balance = balance - ? WHERE owner = ?", (amount, src))
        con.execute("UPDATE account SET balance = balance + ? WHERE owner = ?", (amount, dst))

def balances(con):
    return {r["owner"]: r["balance"] for r in con.execute("SELECT owner, balance FROM account ORDER BY id")}

transfer(con, "ana", "ben", 30)
print(balances(con))
try:
    transfer(con, "ana", "ben", 500)
except sqlite3.IntegrityError as e:
    print("rolled back:", e)
print(balances(con))
```

```
{'ana': 70, 'ben': 80}
rolled back: CHECK constraint failed: balance >= 0
{'ana': 70, 'ben': 80}
```

COMMON MISTAKE

Forgetting `commit()`. Your script inserts rows, prints them back happily (the same connection sees its own uncommitted work), exits, and the data is gone. Wrong: `con.execute("INSERT ...")` then exit. Right: `with con: con.execute("INSERT ...")`. Note also that `with con:` does *not* close the connection; call `con.close()` yourself or use `contextlib.closing`.

A repository class

Chapter 8 introduced the repository idea: one class owns all the SQL for one table, and the rest of the program calls plain methods. Here is a complete create, read, update, delete (CRUD) repository. Because it receives the connection from outside, tests can pass `":memory:"`.

Problem 12.8.2 Build `NoteRepo` with `add`, `get`, `mark_done` and `delete`. `get` returns a dict or `None`. The mutators return `True` if exactly one row changed.

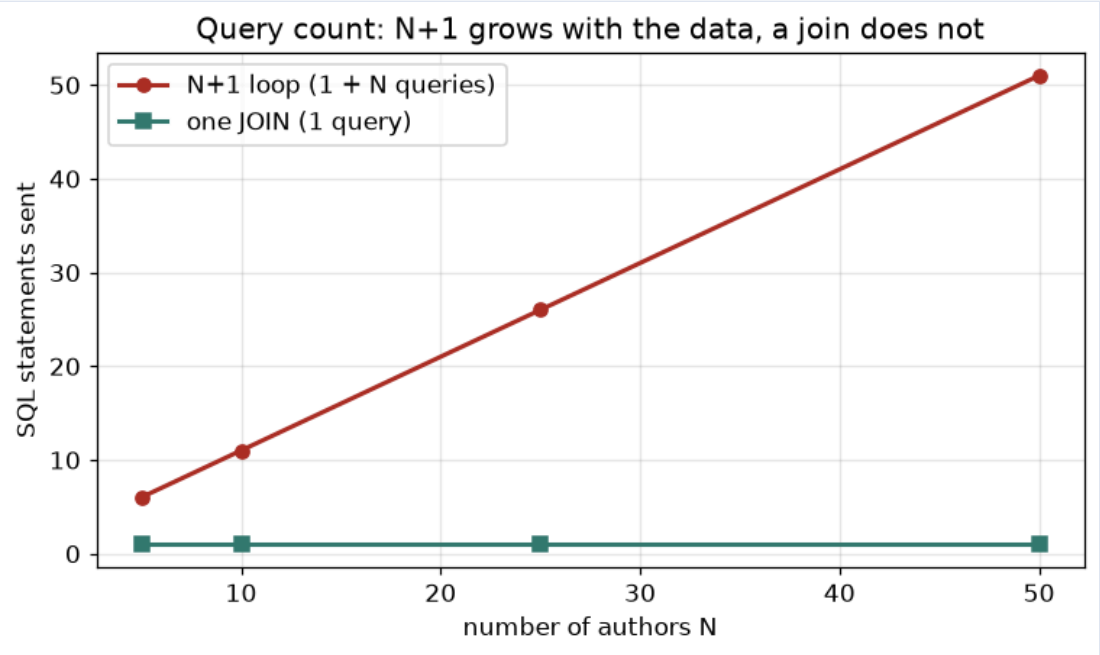
Idea	`cursor.rowcount` says how many rows an UPDATE or DELETE touched, so a missing id is detected without a second query.
Step 1	`add` uses `lastrowid` to return the new id.
Step 2	`get` converts `sqlite3.Row` to a plain dict so callers do not depend on sqlite3.
Check	Deleting the same id twice returns `True` then `False`.

```
import sqlite3

class NoteRepo:
    def __init__(self, con):
        self.con = con
        self.con.row_factory = sqlite3.Row
        self.con.execute("CREATE TABLE IF NOT EXISTS note (id INTEGER PRIMARY KEY, title TEXT NOT NULL, done INTEGER NOT
NULL DEFAULT 0)")
    def add(self, title):
        with self.con:
            return self.con.execute("INSERT INTO note (title) VALUES (?)", (title,)).lastrowid
    def get(self, note_id):
        row = self.con.execute("SELECT * FROM note WHERE id = ?", (note_id,)).fetchone()
        return dict(row) if row else None
    def mark_done(self, note_id):
        with self.con:
            return self.con.execute("UPDATE note SET done = 1 WHERE id = ?", (note_id,)).rowcount == 1
    def delete(self, note_id):
        with self.con:
            return self.con.execute("DELETE FROM note WHERE id = ?", (note_id,)).rowcount == 1

repo = NoteRepo(sqlite3.connect(":memory:"))
a = repo.add("write tests")
b = repo.add("ship it")
print(repo.get(a))
print(repo.mark_done(a), repo.get(a)["done"])
print(repo.delete(b), repo.get(b), repo.delete(b))
```

```
{'id': 1, 'title': 'write tests', 'done': 0}
True 1
True None False
```

The N+1 problem, with a measured query count

Suppose you list authors and their books. The beginner loop runs one query for the authors, then one more query per author: 1 + N queries. With 5 authors that is 6 round trips. With 50,000 authors against a networked database it is the reason a page takes 30 seconds. A single `JOIN` asks for everything at once. The figure shows how the two grow.



Query count against number of authors N: the N+1 loop grows linearly, the JOIN stays at one statement.

How to read it: the red line climbs by one query for every extra author, the teal line stays flat. To count queries for real, `set_trace_callback` hands every executed SQL statement to a function you supply.

Problem 12.8.3 With 5 authors and 2 books each, count the statements sent by the N+1 loop and by the JOIN, and show both give the same data.

Predict	N+1: 1 author query plus 5 book queries = 6. JOIN: 1.
Method	Append each traced statement to a list, clear it before each strategy, and compare lengths.
Check	The two results dictionaries are equal.

```
import sqlite3
con = sqlite3.connect(":memory:")
con.executescript("""
CREATE TABLE author (id INTEGER PRIMARY KEY, name TEXT);
CREATE TABLE book (id INTEGER PRIMARY KEY, author_id INTEGER, title TEXT);
""")
for i in range(1, 6):
    con.execute("INSERT INTO author VALUES (?, ?)", (i, f"author{i}"))
    for j in range(2):
        con.execute("INSERT INTO book (author_id, title) VALUES (?, ?)", (i, f"book{i}{j}"))
con.commit()
log = []
con.set_trace_callback(log.append)

log.clear()
result = {}
for aid, name in con.execute("SELECT id, name FROM author").fetchall():
    result[name] = [t for (t,) in con.execute("SELECT title FROM book WHERE author_id = ?", (aid,))]
print("N+1 queries:", len(log))

log.clear()
joined = {}
for name, title in con.execute("SELECT a.name, b.title FROM author a JOIN book b ON b.author_id = a.id ORDER BY a.id,
b.id"):
    joined.setdefault(name, []).append(title)
print("JOIN queries:", len(log))
print(result == joined)
```

```
N+1 queries: 6
JOIN queries: 1
True
```

KEY IDEA

Count queries in a test. A test that asserts "this endpoint runs at most 2 statements" catches an N+1 regression the day someone adds a loop, long before production traffic does.

Going deeper

- **SQLAlchemy and Alembic (not run here):** SQLAlchemy is the standard Python toolkit. Core builds SQL from Python objects, the ORM maps tables to classes and uses a `Session` as the transaction unit. Its ORM has `selectinload` and `joinedload` options that remove N+1 for you. Alembic records schema changes as numbered migration scripts (`alembic revision --autogenerate`, `alembic upgrade head`), so every environment reaches the same schema.
- sqlite3 allows one writer at a time. For a web service with many writers, use PostgreSQL and a connection pool so each request borrows a connection instead of opening a new one.
- SQLite is loosely typed: a column declared `INTEGER` will happily store text that cannot convert. Use `CHECK` constraints and `STRICT` tables (SQLite 3.37+) to enforce types.

Errors and retries on the API side

Two habits make APIs pleasant to use. On the server, return errors in one predictable shape. RFC 9457 "problem details" defines JSON with `type`, `title`, `status`, `detail` and `instance`. On the client, retry only errors that can succeed later (429, 500, 502, 503, 504), and wait longer each time (exponential backoff: 0.5s, 1s, 2s). To test this without a network, use a fake transport that fails on purpose and inject `sleep` so the test does not wait.

Problem 12.8.4 Write `get_with_retry` that retries 503 up to 4 attempts with doubling delay, and test it against a transport that fails twice then succeeds.

Step 1	Loop over attempts. Call the transport inside `try`.
Step 2	If the status is not retryable, or this was the last attempt, re-raise.
Step 3	Otherwise wait `base_delay * 2 ** (attempt - 1)`: 0.5 then 1.0.
Check	Two waits printed, third call succeeds, total calls = 3.

```
import json, time, urllib.error

class FlakyTransport:
    """Fake network: fails twice with 503, then succeeds."""
    def __init__(self):
        self.calls = 0
    def get(self, url):
        self.calls += 1
        if self.calls <= 2:
            raise urllib.error.HTTPError(url, 503, "Service Unavailable", {}, None)
        return {"ok": True, "calls": self.calls}

def get_with_retry(transport, url, attempts=4, base_delay=0.5, sleep=time.sleep):
    for attempt in range(1, attempts + 1):
        try:
            return transport.get(url)
        except urllib.error.HTTPError as e:
            if e.code not in (429, 500, 502, 503, 504) or attempt == attempts:
                raise
            delay = base_delay * 2 ** (attempt - 1)
            print(f"attempt {attempt} got {e.code}, waiting {delay}s")
            sleep(delay)

print(get_with_retry(FlakyTransport(), "https://api.example.com/x", sleep=lambda s: None))

def problem(status, title, detail, instance):
    return {"type": "about:blank", "title": title, "status": status, "detail": detail, "instance": instance}
print(json.dumps(problem(422, "Validation failed", "field 'age' must be an integer", "/users/7"), indent=2))
```

```
attempt 1 got 503, waiting 0.5s
attempt 2 got 503, waiting 1.0s
{'ok': True, 'calls': 3}
{
  "type": "about:blank",
  "title": "Validation failed",
  "status": 422,
  "detail": "field 'age' must be an integer",
  "instance": "/users/7"
}
```

COMMON MISTAKE	Retrying everything, including 400 and 404, and retrying non-idempotent calls (a POST that creates an order). Wrong: a blanket `except Exception: retry` that creates three orders. Right: retry only the listed status codes, and send an idempotency key with POSTs so a repeat is recognised.

Problem 12.P.12 (build) Extend `NoteRepo` with `list_open()` returning titles of notes with `done = 0` ordered by id. Test it with three notes, one done.

Answer: `return [r["title"] for r in self.con.execute("SELECT title FROM note WHERE done = 0 ORDER BY id")]`. With notes a, b, c and b done, it returns `['a', 'c']`.

Problem 12.P.13 (find the bug) `con.execute(f"SELECT * FROM note WHERE title = '{t}'")`. What input breaks it?

Answer: Any title containing a quote, such as `x' OR '1'='1`, changes the meaning of the query (SQL injection). Use `con.execute("SELECT * FROM note WHERE title = ?", (t,))`.

Chapter recap

- HTTP is request (method, URL, headers, body) and response (status, headers, body). Status codes are part of the contract.
- REST means nouns in URLs, methods as actions, JSON bodies and stateless calls.
- GET, PUT and DELETE are idempotent. POST is not, so use an idempotency key when a repeat would hurt.
- Always set a timeout, check the status, and retry only temporary errors with backoff.
- FastAPI turns type hints and pydantic models into validation and automatic OpenAPI docs. Flask is smaller and needs more manual pieces.
- Paginate big lists. Cursors are steadier than offsets for live data.
- Authentication is "who", authorization is "what". A JWT is signed, not encrypted. Verify signature and expiry.
- Scrape politely: API first, then robots.txt, terms, delays, a User-Agent, and no personal data without cause.
- A good CLI has a `main(argv)` function, clear help, proper exit codes, and logs on stderr.
- Pass argument lists to `subprocess`, and use `?` placeholders in SQL.

Cheat sheet

Task	Pattern
Call an API safely	`client.get(url, timeout=5)`, then `raise_for_status()`
Retry	wait `base * 2**attempt`, only for temporary errors
Paginate	loop until `next_offset is None`
Safe POST retry	`Idempotency-Key` header
Sign a token	`hmac.new(key, msg, sha256)`, verify with `compare_digest`
Check robots	`RobotFileParser().parse(lines)`, `can_fetch(ua, url)`
CLI	`argparse`, `main(argv=None)`, exit code 0 or non-zero
Run a program	`subprocess.run([...], capture_output=True, text=True, check=True)`
SQL values	`execute("... WHERE x = ?", (value,))`

Practice set

Problem 12.P.1 (easy) Which status code for each: item created; item not found; rate limit hit; server crashed on a bug?

Answer: 201; 404; 429; 500.

Problem 12.P.2 (easy) What is the output type of `json.loads('{"a": [1, 2]}')["a"]`, and what is its value?

Answer: A Python `list`: `[1, 2]`.

Problem 12.P.3 (easy) Name the three parts of a JWT, in order.

Answer: Header, payload, signature, joined by dots.

Problem 12.P.4 (medium) An API has 95 records and you ask `limit=20`. How many requests, and how many items on the last page?

Answer: ceil(95 / 20) = 5 requests. The last page has 95 - 80 = 15 items.

Problem 12.P.5 (medium) Find the bug: `r = requests.get(url); data = r.json()` in a nightly job that sometimes hangs.

Answer: There is no `timeout=` and no `raise_for_status()`. Add `timeout=10` and `r.raise_for_status()`. Hanging comes from the missing timeout.

Problem 12.P.6 (medium) Using the 12.7 `summ` tool, what is `p50` for the numbers `10 20 30 40`?

Answer: Nearest rank: ceil(4 x 0.5) = 2, index 1 after sorting, value 20.0. (So `summ 10 20 30 40 -p 50` prints `count=4 mean=25.0 p50=20.0`.)

Problem 12.P.7 (medium) Your bot reads robots.txt with `Disallow: /admin/` and `Crawl-delay: 10`. You want 300 pages from allowed paths. What is the least time the crawl should take?

Answer: 300 x 10 = 3,000 seconds, which is 50 minutes, if you fetch one page at a time and respect the delay.

Problem 12.P.8 (medium, scenario) A mobile app retries `POST /orders` after a timeout, and customers get two orders. Give two fixes.

Answer: (1) Add an idempotency key header, and make the server return the first result for a repeated key. (2) Add a unique constraint on a client-generated order ID so the database refuses duplicates (and return 409 or the original order).

Problem 12.P.9 (hard, interview) Explain the difference between authentication and authorization with an API example, and say where a JWT fits.

Answer: Authentication verifies the identity (a valid login gives a token). Authorization decides actions: a "viewer" token can `GET` but not `DELETE`, so `DELETE` returns 403. A JWT carries identity and claims (like role and expiry) in a signed token, so the server can check them without a database lookup on each call. It is signed, not encrypted.

Problem 12.P.10 (hard, interview) Why is `subprocess.run(f"ls {user_input}", shell=True)` dangerous, and what is the fix?

Answer: The shell parses the string, so `user_input = "x; rm -rf ~"` runs a second command (command injection). Fix: `subprocess.run(["ls", user_input])`, which treats the input as one argument, and validate the input too.

Problem 12.P.11 (hard) Your offset-paged export misses some rows when new rows arrive during the export. Why, and how do you fix it?

Answer: New rows at the front shift every later row by one, so a row can fall between pages (missed) or appear twice. Use cursor (keyset) pagination, for example `WHERE id > last_id ORDER BY id LIMIT 100`, or export from a snapshot.

Prompt pack

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data and AI roles, and English is my second language, so use simple English. Teach me "HTTP and REST API design" step by step: methods, status codes, headers, idempotency, and how REST naming habits work. Use small Python examples with the standard library, show real output, and relate each idea to everyday examples like ordering at a restaurant. After each part, ask me one check question and wait for my answer before continuing.
```

✨ **AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving into data and AI roles, and English is my second language. Give me 8 graded exercises (easy to hard) on calling APIs from Python with requests or httpx: timeouts, raise_for_status, Session reuse, retries with exponential backoff, pagination loops, and rate limiting (429). Do not give solutions. When I paste my answers, review them one by one and point out bugs and risks.
```

✨ **AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional learning Python for data and ML jobs, and English is my second language. Quiz me one question at a time on "authentication and authorization": API keys, bearer tokens, OAuth 2.0, OIDC, JWT structure, signature versus encryption, 401 versus 403, and token expiry. Start easy, increase difficulty, and explain every wrong answer simply before moving on. Keep score out of 10.
```

✨ **AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving into data roles, and English is my second language. Review my work on a small FastAPI service with one database table and two endpoints. Check routing, pydantic validation, status codes, error handling, how the model or database connection is created, and test coverage. Be specific and kind, and list the top 5 improvements in order of importance. My code: [PASTE HERE]
```

✨ **AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional preparing for Python and data engineering interviews, and English is my second language. Run a 30-minute mock interview on "APIs, scraping responsibly, and command-line tools in Python". Ask one question at a time, mixing concept questions (idempotency, pagination, JWT, robots.txt, argparse exit codes, subprocess safety) and a small coding task. After each answer give a score out of 5, a model answer, and one tip on how to say it clearly in English.
```

CHAPTER 13

PROFESSIONAL PRACTICE: CLEAN CODE, CI/CD, SECRETS, SECURITY AND PACKAGING

The habits that turn "code that works on my laptop" into code a team can trust, change and ship.

Picture your first week in a data team. You open a colleague's script. It has one function of 300 lines, names like `x2` and `tmp`, a database password typed into the file, and no tests. Nobody dares to change it. Now picture the opposite: small named functions, tests that run on every push, secrets in the environment, and a package you can install in one command. This chapter shows you how to build the second kind. By the end you will be able to refactor messy code safely, explain how a team ships software (Agile, code review, CI/CD), keep secrets out of Git, stop SQL injection, store passwords correctly, and package your work with `pyproject.toml`.

Everything marked "run" was executed with Python 3.14 and the standard library. Anything needing a network or extra installs (GitHub Actions, `gitleaks`, `pip-audit`, `build`, `twine`) is marked **not run here** and shown in minimal, standard form. Check each tool's docs for current options.

Words you need. **Refactoring** means changing how code is written without changing what it does. **Technical debt** means shortcuts that make future changes slower, like a loan with interest. **CI/CD** means Continuous Integration (tests run automatically on every change) and Continuous Delivery/Deployment (releases happen automatically or with one click). **Secret** means any value that grants access: password, API key, token, private key.

13.1 Clean Code and Refactoring

INTUITION

Code is read ten times more than it is written. Think of a kitchen. In a clean kitchen every tool has a labelled place, so a new cook can work on day one. In a messy kitchen only the person who made the mess can cook. Clean code is a labelled kitchen: good names, small functions, no repeated recipes.

How it works

The main rules, with the reason for each:

Rule	What it means	Why it exists
Good names	`slow_delivered_orders`, not `f`	A name is the cheapest documentation
Small functions	One job, fits on a screen	Easy to test and to reuse
DRY (Don't Repeat Yourself)	One place for each piece of knowledge	A bug fixed once is fixed everywhere
Comments say *why*	Not *what* the code does	Code already says what; only you know why
No magic numbers	`TIMEOUT_S = 5`	The number gets a meaning
Early return	Handle bad cases first	Less nesting
Types and docstrings	Say what goes in and out	Readers and tools can check

Refactoring is done in small steps with tests that stay green. The safe loop is: (1) make sure tests exist, (2) change one thing, (3) run the tests, (4) repeat. If you have no tests, first write a few that capture today's behaviour.

When NOT to refactor: when you have no tests and a deadline tomorrow, add the tests first. Do not tidy code you are not otherwise touching just because you can.

Worked examples

Problem 13.1.1 Refactor a cryptic function. It takes shop order rows `(order_id, delivery_minutes, status)` and a threshold, and returns the order ids of delivered orders slower than the threshold.

Predict first: for the rows below with threshold 60, which order ids come out?

- **Step 1** Write the "before" code and run it to record today's behaviour.
- **Step 2** Name the data with a `dataclass` and the function by what it does.
- **Step 3** Flatten the nested `if`s into one comprehension.
- **Step 4** Run before and after on the same input and compare.

```
# BEFORE
def f(d, t):
    r = []
    for x in d:
        if x[2] == "DELIVERED":
            if x[1] > t:
                r.append(x[0])
    return r

rows = [("A101", 45, "DELIVERED"), ("A102", 12, "DELIVERED"), ("A103", 230, "LOST"), ("A104", 150, "DELIVERED")]
print(f(rows, 60))

# AFTER
from dataclasses import dataclass

@dataclass(frozen=True)
class Order:
    order_id: str
    delivery_minutes: int
    status: str

def slow_delivered_orders(orders, threshold_minutes):
    """Return ids of delivered orders slower than threshold_minutes."""
    return [o.order_id for o in orders if o.status == "DELIVERED" and o.delivery_minutes > threshold_minutes]

orders = [Order(*r) for r in rows]
print(slow_delivered_orders(orders, 60))
print(slow_delivered_orders(orders, 10))
print(f(rows, 10) == slow_delivered_orders(orders, 10))
```

```
['A104']
['A104']
['A101', 'A102', 'A104']
True
```

- **Check** By hand: `A101` is 45 (not above 60), `A102` is 12, `A103` is slow (230) but status is LOST so it is skipped, `A104` is 150 and DELIVERED. Only `A104`. For threshold 10, the delivered rows are `A101`, `A102`, `A104`, all above 10.
- **Answer** The new function gives the same results as the old one (the last line prints `True`), but a reader now knows what it does from its name, and the fields have names (`o.delivery_minutes` instead of `x[1]`).
- **Why it matters** Equality of old and new outputs on the same inputs is a poor-man's test that your refactoring did not break behaviour. You will turn it into a real pytest test (Chapter 10).

Problem 13.1.2 Remove repetition (DRY) from percentile code and give magic numbers names.

Predict: three copies of the same sort-and-index logic. After refactoring, how many copies of the formula exist?

```
import math

# BEFORE: the formula is copy-pasted three times
def p50(xs): xs = sorted(xs); return xs[max(0, math.ceil(len(xs) * 50 / 100) - 1)]
def p90(xs): xs = sorted(xs); return xs[max(0, math.ceil(len(xs) * 90 / 100) - 1)]
def p95(xs): xs = sorted(xs); return xs[max(0, math.ceil(len(xs) * 95 / 100) - 1)]

# AFTER: one function, one place to fix bugs
def percentile(xs, p):
    """Nearest-rank percentile, p in 1..100."""
    xs = sorted(xs)
    return xs[max(0, math.ceil(len(xs) * p / 100) - 1)]

data = [80, 95, 110, 120, 300]
print(p50(data), p90(data), p95(data))
print([percentile(data, p) for p in (50, 90, 95)])
```

I ran this and checked the equality by hand: sorted data is 80, 95, 110, 120, 300 (n = 5). For p50: ceil(2.5) = 3, index 2, value 110. For p90 and p95: ceil(4.5) = 5 and ceil(4.75) = 5, index 4, value 300. Real output:

```
110 300 300
[110, 300, 300]
```

- **Answer** One copy of the formula remains. Both versions print `110 300 300`. If a bug is found in the formula, you now fix one line, not three.
- **Why it matters** In real codebases, the copy you forgot to fix is the one that fails in production.

Predict, Run, Explain. Look at the typical wrong refactor: renaming a function in the definition but not at one call site. Python tells you only when that line runs, for example `NameError: name 'p50' is not defined`. That is exactly why you need tests (or a linter such as `ruff`, which flags undefined names before you run anything): the tests run every call site.

Real-world mapping

Concept	Example	Where it appears in the job
Good name	`slow_delivered_orders`	Reviewing a teammate's pipeline in five minutes
Small function	`percentile(xs, p)`	Reused by report, alert and dashboard code
DRY	one percentile formula	One bug fix, many places repaired
Refactor with tests	same output before and after	Cleaning a legacy ETL script

Applications

- **Data Analytics:** turn a 400-line notebook cell into named functions you can reuse next month.
- **Data Engineering:** split an ETL job into extract, transform and load functions that can be tested.
- **Machine Learning:** separate feature code from training code so the same features run at serving time.
- **Data Analytics:** share one percentile function across all your report scripts.

COMMON MISTAKE	Refactoring and adding features in the same change. If something breaks you do not know which caused it. Do them in separate commits: first refactor with tests green, then add the feature.

Check yourself

1. What does DRY stand for and why does it matter? Answer: Don't Repeat Yourself. A fix or change then happens in one place.
2. Should a comment say what or why? Answer: Why. The code already shows what.

3. What must exist before you refactor? Answer: Tests (or at least recorded outputs) that show behaviour did not change.

13.2 Architecture: Layers, Monoliths, Microservices, Events and Queues

INTUITION	A restaurant has a front desk (takes orders), a kitchen (cooks), and a storeroom (keeps food). If a waiter walked into the storeroom and cooked, chaos follows. Software layers are the same: each part has one job and talks only to its neighbour. A queue is the order rail in the kitchen: waiters put tickets on it, cooks take them when ready, so a rush does not collapse the kitchen.

How it works

Layered architecture. Three layers are common:

1. **API / presentation layer**: receives requests, checks input, returns responses.
2. **Service / business layer**: the rules ("an order total is the sum of its items").
3. **Repository / data layer**: the only place that talks to the database.

Why: you can test the middle layer with a fake data layer, and you can change the database without touching the rules.

Dependency injection (DI) means giving a class the things it needs (a repository, a notifier) from outside instead of creating them inside. The **repository pattern** hides the database behind methods like `save` and `get`. A **fake** is a simple working stand-in (an in-memory repository). A **mock** is a recorder that checks how it was called.

Monolith vs microservices.

	Monolith	Microservices
Shape	One deployable program	Many small services over the network
Good for	Small teams, early products, simple deployment	Big teams, parts that scale differently
Costs	Big codebase, one slow release train	Network failures, many deployments, hard debugging

Start with a well-organised monolith. Split only when you feel real pain. A distributed system is slower to build and harder to debug than a single program.

Event-driven design means parts communicate by announcing events ("order placed") and other parts react. **Message queues** (RabbitMQ, Kafka, cloud queues) store messages between producer and consumer. Benefits: smoothing spikes, retries, and decoupling. Cost: messages may arrive twice (so consumers must be idempotent, see Chapter 12) or late.

Worked examples

Problem 13.2.1 Build a service with injected dependencies and test it using a fake repository and a mock notifier.

Predict: what total does `place(1, [100, 250, 50])` return, and what message is sent?

- **Step 1** `OrderService` receives `repo` and `notifier` in `__init__`.
- **Step 2** `place` sums prices, saves, and notifies.
- **Step 3** In the test we pass a fake repository (works in memory) and a `Mock` notifier (records calls).

```
from unittest.mock import Mock

class OrderRepository:                      # a real one would use a database
    def __init__(self): self.rows = {}
    def save(self, oid, total): self.rows[oid] = total
    def get(self, oid): return self.rows.get(oid)

class FakeRepo(OrderRepository): pass       # in-memory fake: real behaviour, no DB

class OrderService:
    def __init__(self, repo, notifier):     # dependencies injected
        self.repo, self.notifier = repo, notifier
    def place(self, oid, prices):
        total = sum(prices)
        self.repo.save(oid, total)
        self.notifier.send(f"order {oid}: {total}")
        return total

notifier = Mock()
svc = OrderService(FakeRepo(), notifier)
print(svc.place(1, [100, 250, 50]))
print(svc.repo.get(1))
notifier.send.assert_called_once_with("order 1: 400")
print(notifier.send.call_args)
```

```
400
400
call('order 1: 400')
```

- **Check** 100 + 250 + 50 = 400. `assert_called_once_with` raised nothing, which means the message was sent exactly once with the text we expected. (If it was wrong, you would see an `AssertionError` that shows expected and actual calls.)
- **Answer** The total is 400, it was saved, and exactly one notification `order 1: 400` was sent.
- **Why it matters** No email was sent and no database was touched, so the test runs in milliseconds and cannot fail because of a network.

Problem 13.2.2 Show the idea of a queue and of events in a few lines: a worker takes jobs from a `queue.Queue`, and an event bus calls every handler registered for an event.

Predict: in what order do the results appear, and how many handlers run on one event?

```
import queue, threading

q = queue.Queue()
results = []
def worker():
    while True:
        job = q.get()
        if job is None:
            break
        results.append(job * job)
        q.task_done()
t = threading.Thread(target=worker); t.start()
for n in range(1, 6): q.put(n)
q.put(None); t.join()
print(results)

handlers = {}
def on(evt):
    def deco(fn): handlers.setdefault(evt, []).append(fn); return fn
    return deco
def emit(evt, **data):
    for h in handlers.get(evt, []): h(**data)
@on("order_placed")
def send_email(order_id): print("email for", order_id)
@on("order_placed")
def update_stock(order_id): print("stock updated for", order_id)
emit("order_placed", order_id=42)
```

```
[1, 4, 9, 16, 25]
email for 42
stock updated for 42
```

- **Answer** The queue is first-in-first-out, so squares of 1 to 5 appear in order. Two handlers ran for one event. The code that emits "order placed" does not know about email or stock. You can add a third handler without touching it.
- **Check** 1, 4, 9, 16, 25 are the squares of 1 to 5.
- **Why it matters** This is the shape of real systems: producers and consumers separated by a queue, with workers that scale independently.

Typical wrong version: creating the repository inside the service (`self.repo = PostgresRepo()`). The service can no longer be tested without a database, and the test fails with a connection error like `ConnectionRefusedError`. The fix is the injection shown above.

Real-world mapping

Concept	Example	Where it appears in the job
Layers	route to service to repository	A FastAPI model service with a feature store behind it
DI + fake	`FakeRepo`	Fast unit tests for a data pipeline step
Queue	Kafka topic, SQS queue	Buffering events before they are loaded into a warehouse
Event	"file arrived"	Starting an ETL run when data lands in a bucket

Applications

- **Data Engineering:** streaming pipelines where a queue buffers bursts and consumers write idempotently.
- **Machine Learning / MLOps:** a "retrain requested" event triggers a training job.
- **GenAI:** a queue holds long-running document-processing jobs behind a chat API.
- **Operations:** you monitor queue depth and consumer lag, like counting orders waiting at a counter, not only how fast one order is served.

COMMON MISTAKE	Jumping to microservices because it sounds modern. A small team with ten services spends more time on networking and deployment than on features. Start simple; split when a clear need appears.

Check yourself

1. Why use layers? Answer: Each layer has one job, so you can test and change parts separately.
2. What is the difference between a fake and a mock? Answer: A fake is a simple working replacement. A mock records calls so you can check them.
3. Why must queue consumers be idempotent? Answer: Messages can be delivered more than once, and a repeat must not cause a double effect.

13.3 Agile, Estimation, Technical Debt, Code Review and Documentation

INTUITION	Building software is like planning a long road trip when the map is incomplete. You plan a short leg, drive it, look at what you learned, and plan the next leg. That is Agile. Technical debt is borrowing time: a shortcut now saves a day, but every later change pays a bit of interest.

How it works

Agile is a way of working in short cycles with frequent feedback. **Scrum** is a popular form of it:

- A **sprint** is a fixed period (often two weeks) with a goal.
- The **backlog** is the ordered list of work. Each item is a **user story** ("As an analyst I want a daily report so that I can see errors").
- Meetings: **planning** (choose work), **daily stand-up** (15 minutes: what I did, will do, what blocks me), **review** (show the work), **retrospective** (how to improve).
- **Estimation:** teams guess size in points or days. People are bad at single-number guesses, so use a range. The **three-point (PERT)** estimate is `(optimistic + 4 × likely + pessimistic) / 6`. It weights the likely case but respects the risk of a long tail.

Technical debt: quick-and-dirty code you plan to fix later. Some debt is wise (to hit a launch), but it must be written down and repaid. Signs of too much debt: fear of changing code, bugs that return, slow onboarding.

Code review is where a teammate reads your change before it merges. It catches bugs, spreads knowledge, and keeps the style consistent. Good reviews:

- Authors: keep changes small (under about 400 lines), explain *why* in the description, run tests first.
- Reviewers: be kind and specific, review the code and not the person, ask questions ("What happens if the list is empty?"), separate must-fix from nice-to-have, and say what is good.

Documentation layers: a README (what it is, how to install and run in 5 lines), docstrings (what each function does), comments (why), and an architecture note. A **docstring** is the string at the top of a function. It can even hold runnable examples called **doctests**.

Worked examples

Problem 13.3.1 A task has optimistic 2 days, likely 4 days, pessimistic 12 days. Give the PERT estimate and compare with the simple average of the three.

Formula	PERT = (O + 4M + P) / 6
Plug in	(2 + 4 × 4 + 12) / 6
Compute	2 + 16 + 12 = 30, and 30 / 6 = 5.0. Simple average: (2 + 4 + 12) / 3 = 6.0.
Answer	The PERT estimate is 5.0 days. Promise 5 to 6 days, not the "likely" 4.
Check	The result lies between the likely value (4) and the average (6), pulled up a little by the long tail. A one-line check in Python: `round((2 + 4*4 + 12) / 6, 2)` printed `5.0` in a real run.
Why it matters	A promise of 4 days would be missed whenever the pessimistic case happens. Ranges and a weighted estimate give more honest dates.

Problem 13.3.2 Write a docstring with doctests for `percentile` and run them automatically.

Predict: how many examples are tested, and do they pass?

The function below was saved in a small package (you will build it in 13.7) and tested with `doctest`:

```
import math

def percentile(values, p):
    """Nearest-rank percentile.

    >>> percentile([80, 95, 110, 120, 300], 90)
    300
    >>> percentile([1, 2, 3, 4], 50)
    2
    """
    xs = sorted(values)
    k = max(0, math.ceil(len(xs) * p / 100) - 1)
    return xs[k]
```

Running `doctest.testmod` on the module printed:

```
TestResults(failed=0, attempted=2)
```

- **Check** By hand: the first example is the p90 case from 13.1.2 (300). Second: sorted `1,2,3,4`, ceil(4 × 0.5) = 2, index 1, value 2.
- **Answer** Two examples were tested and both passed. Your documentation can never go out of date silently: if the code changes, the example fails.
- **Why it matters** The learner reading the docs sees how to call the function. The team gets a free tiny test.

Typical wrong version: a docstring example with the wrong expected value (for example `300` replaced by `200`). `doctest` reports the line, the expected output and the real output, and `failed=1`. The fix is to correct the example or the code, whichever is wrong.

Real-world mapping

Practice	Example	Where it appears in the job
Sprint	2 weeks, goal "daily report live"	How most data teams plan work
Estimate	PERT 5.0 days	Telling a manager when a pipeline will be ready
Tech debt ticket	"Replace hard-coded path"	Keeps a backlog of cleanups

Practice	Example	Where it appears in the job
Review comment	"What if `rows` is empty?"	Catches the empty-file bug before production
README	install, run, test in 5 lines	A teammate runs your program on day one

Applications

- **Data Analytics:** short sprints with stakeholders reviewing a dashboard each cycle.
- **Data Engineering:** reviews catch schema and idempotency mistakes before a load runs.
- **Machine Learning:** model cards and READMEs record data, metrics and limits.
- **Project planning:** estimating delivery dates; ranges beat single numbers.

COMMON MISTAKE

Treating estimates as promises, or giving one number. State a range and your assumptions ("assuming the data is available by Monday"), and update the estimate when you learn more.

Check yourself

1. What is a sprint? Answer: A short fixed period (often two weeks) with a goal.
2. What does the retrospective do? Answer: The team looks at how it worked and decides what to improve.
3. Why use a range estimate? Answer: Work has uncertainty, and a single number hides the risk.

13.4 CI/CD Basics

INTUITION

CI is a strict friend who runs your tests every time you save to the shared folder. If you broke something, you hear within minutes, not weeks. CD is the delivery truck that takes the passing build to users automatically. Think of CI as a quick taste test of the soup that happens every time the recipe changes.

How it works

Pipeline stages, in order: **build** (install, compile or package), **test** (unit and integration tests), **lint** (style and error checks with `ruff`, types with `mypy`), then **deploy**. A failing stage stops everything after it ("fail fast"). Cheap checks (lint) go first.

Tools: **GitHub Actions** (YAML files in `.github/workflows/`), **Jenkins** (self-hosted, a `Jenkinsfile`), GitLab CI, and others. Key ideas: a **trigger** (push, pull request, schedule), a **runner** (the machine), **steps**, and **secrets** stored in the CI system, never in the repo.

Not run here (it needs GitHub). A minimal, standard GitHub Actions file:

```yaml
name: ci
on:
  push:
    branches: [main]
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: pip install -e ".[dev]"
      - run: ruff check .
      - run: ruff format --check .
      - run: mypy src
      - run: pytest --cov=src
```

Version tags such as `@v4` change over time. Check the current ones on each action's page.

Worked examples

Problem 13.4.1 Simulate a tiny pipeline runner that stops at the first failing stage, and run it with a test stage that fails.

Predict: which stages run, and what is the final result?

```python
import subprocess, sys

def stage(name, code):
    r = subprocess.run([sys.executable, "-c", code], capture_output=True, text=True)
    print(f"{name:6} -> {'PASS' if r.returncode == 0 else 'FAIL'}")
    return r.returncode == 0

stages = [
    ("lint", "pass"),
    ("test", "assert 2 + 2 == 5"),
    ("deploy", "print('shipping')"),
]
for name, code in stages:
    if not stage(name, code):
        print("pipeline stopped at", name)
        break
else:
    print("pipeline passed")
```

```
lint   -> PASS
test   -> FAIL
pipeline stopped at test
```

- **Step 1** Each stage is a command. Exit code 0 means pass, which is the same convention real CI uses.
- **Step 2** The `for ... else` runs `else` only if the loop did not `break`.
- **Check** `assert 2 + 2 == 5` raises `AssertionError`, so the child exits with code 1. Deploy never ran.
- **Answer** Lint passed, test failed, and the pipeline stopped before deploy. Bad code never reached users.
- **Why it matters** This is exactly how a real CI system treats exit codes, which is why Chapter 12 stressed returning proper codes from CLIs.

Problem 13.4.2 A pipeline takes 12 minutes: lint 1, unit tests 4, integration tests 6, package 1. A change that breaks lint is pushed. How long until the developer knows with lint first, and with lint last?

Compute (lint first)	Lint is the first stage and takes 1 minute, so the failure appears after 1 minute.
Compute (lint last)	Stages before lint take 4 + 6 + 1 = 11 minutes, then lint 1 minute: 12 minutes.
Answer	1 minute versus 12 minutes. Put cheap, fast checks first.
Check	11 + 1 = 12, which equals the total pipeline time, as expected when the last stage fails.
Why it matters	Developer waiting time is real cost. Fast feedback means more small, safe changes.

Typical wrong version: printing a secret inside a CI step (`echo $API_KEY`) or putting the key in the YAML. CI logs are readable by many people. Use the CI system's secret store and never print it (see 13.5).

Real-world mapping

Stage	Example command	Where it appears in the job
Lint	`ruff check .`	Stops unused imports and typos early
Test	`pytest`	Protects a transformation from silent changes
Build	`python –m build`	Produces the package that gets deployed
Deploy	push container image	Releases a new model service version

Applications

- **Data Engineering:** CI runs pipeline unit tests and checks SQL or dbt models before merge.
- **Machine Learning / MLOps:** CI trains a tiny model on sample data and checks metrics do not collapse.
- **Data Analytics:** CI checks that reports still build after a change.
- **Software Engineering:** a short smoke test as a CI stage; a bigger one on a schedule.

COMMON MISTAKE	A CI pipeline that is slow or flaky, so people ignore red builds. Keep it fast, fix flaky tests quickly, and treat red as "stop".

Check yourself

1. What decides pass or fail of a CI step? Answer: The command's exit code (0 means pass).
2. Why run lint before integration tests? Answer: It is quick, so you get feedback sooner.
3. Where do CI secrets live? Answer: In the CI system's secret store, never in the repository.

13.5 Configuration and Secrets Handling

INTUITION	A configuration value is like a thermostat setting: you can change it without rebuilding the house. A secret is a house key. You do not write it on the front door (the code), and you do not make copies for everyone (Git history lives forever and is copied to every clone).

How it works

The 12-factor rule: keep configuration in the environment, not in code. The same code then runs on your laptop, in testing and in production with different settings.

Where config lives, from simple to strong:

1. **Environment variables**: `os.environ["PIPELINE_ENV"]`.
2. `.env` **file** for local development: lines like `KEY=value`, loaded by a library such as `python-dotenv` (not run here; we show a small parser). **Add** `.env` **to** `.gitignore` and commit a `.env.example` with fake values.
3. **YAML/TOML files** for non-secret structure (batch sizes, URLs).
4. **Secret managers** (cloud secret managers, vaults) for production: access is audited, and secrets can be **rotated** (replaced regularly or right after a leak) without code changes.

Rules for secrets:

- Never commit them. If one is committed, **treat it as stolen**: revoke and rotate it at once. Deleting the file in a later commit does not remove it from history.
- Run a scanner such as `gitleaks` or `detect-secrets` in a pre-commit hook and in CI (not run here).
- Do not print secrets or put them in logs; mask them.
- Use least privilege: a key that can only read one bucket is less dangerous than an admin key.
- In notebooks, read keys from environment variables; clear output before sharing.

Worked examples

Problem 13.5.1 Load settings from the environment into a typed, immutable object, with a default and a required value.

Predict: what is printed, and what happens when a required variable is missing?

```
import os
from dataclasses import dataclass

os.environ["PIPELINE_ENV"] = "prod"      # in real life the shell or CI sets this

@dataclass(frozen=True)
class Settings:
    env: str
    batch_size: int = 500
    @classmethod
    def from_env(cls):
        return cls(env=os.environ["PIPELINE_ENV"],
                   batch_size=int(os.environ.get("BATCH_SIZE", "500")))

print(Settings.from_env())

try:
    os.environ["MISSING_REQUIRED"]
except KeyError as e:
    print("KeyError", e)
print(os.environ.get("MISSING", "default"))
```

```
Settings(env='prod', batch_size=500)
KeyError 'MISSING_REQUIRED'
default
```

- **Step 1** `os.environ[...]` is strict: missing means `KeyError`. Use it for required values so the program fails early and loudly.
- **Step 2** `os.environ.get(name, default)` is lenient. Use it for optional values.
- **Step 3** Environment variables are always text, so we convert `BATCH_SIZE` with `int(...)`.

- **Check** `BATCH_SIZE` was not set, so the default `"500"` became the integer 500.
- **Answer** Settings loaded as shown, and a missing required variable raised `KeyError`.
- **Why it matters** A pipeline that crashes at start-up with "missing PIPELINE_ENV" is far better than one that silently runs against the wrong database.

Problem 13.5.2 Parse a `.env` text (fake values), then write a tiny secret scanner that flags suspicious lines, then mask personal data in text. Predict: which line is flagged, and what do the masked values look like?

```
import re

def load_dotenv(text):
    out = {}
    for line in text.splitlines():
        line = line.strip()
        if not line or line.startswith("#") or "=" not in line: continue
        k, v = line.split("=", 1)
        out[k.strip()] = v.strip().strip('"')
    return out
print(load_dotenv('# demo\nAPI_KEY="fake-key-123"\nDEBUG=1\n'))

def mask_email(s): return re.sub(r"(\w)[\w.]*@", r"\1***@", s)
def mask_phone(s): return re.sub(r"\b\d{6}(\d{4})\b", r"XXXXXX\1", s)
print(mask_email("asha.k@example.com"), mask_phone("9876543210"))

pat = re.compile(r"(?i)(api[_-]?key|secret|token)\s*=\s*['\"][A-Za-z0-9_\-]{8,}['\"]")
code = 'x = 1\nAPI_KEY = "fake-key-123"\nname = "abc"\n'
for i, l in enumerate(code.splitlines(), 1):
    if pat.search(l): print("line", i, "looks like a secret")
```

```
{'API_KEY': 'fake-key-123', 'DEBUG': '1'}
a***@example.com XXXXXX3210
line 2 looks like a secret
```

- **Check** Line 2 matches `API_KEY = "..."` with 12 allowed characters (at least 8 needed). Lines 1 and 3 have no key-like name. The phone number has 10 digits: six are hidden and the last four stay.
- **Answer** The scanner flagged line 2. Emails and phone numbers were partly hidden. Real scanners like `gitleaks` use hundreds of such patterns plus entropy checks (not run here).
- **Why it matters** Masking limits damage when data reaches logs or test environments. Partial masking is not anonymisation: with enough other columns a person can still be identified.

Redacting secrets in logs. A log filter can remove a token before it is written:

```
import logging, re
class Redact(logging.Filter):
    def filter(self, record):
        record.msg = re.sub(r"(token=)\w+", r"\1***", str(record.msg)); return True
logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
log = logging.getLogger("app"); log.addFilter(Redact())
log.info("calling api token=abc123xyz")
```

```
INFO calling api token=***
```

Typical wrong version: `API_KEY = "sk-live-..."` typed in the script and pushed to GitHub. Bots scan public repos within minutes. Fix: move the key to an environment variable, rotate the leaked key immediately, and add a scanner. A `.gitignore` needs the line `.env` (add it before the first commit).

Real-world mapping

Item	Example	Where it appears in the job
Env var	`DATABASE_URL`	Same code for dev, test, prod
`.env.example`	fake values committed	New teammate knows what to set
Secret manager	cloud vault entry	Production database password, rotated
Pre-commit scanner	`gitleaks`	Stops a key before it leaves your laptop

Applications

- **Data Engineering:** warehouse credentials from a secret manager, injected as env vars at run time.
- **Machine Learning / MLOps:** model-registry tokens in CI secrets.
- **GenAI:** language-model API keys read from the environment in notebooks, never pasted into cells.
- **Data Analytics:** BI connection strings kept in the tool's credential store.

COMMON MISTAKE

"I deleted the key from the file in the next commit, so it is safe." Git history keeps it, and so do clones. Revoke and rotate the key first, clean the history second.

Check yourself

1. Which is stricter, `os.environ[k]` or `os.environ.get(k)`? Answer: `os.environ[k]` raises `KeyError` if missing.
2. What do you do if a key is committed? Answer: Revoke and rotate it immediately.
3. What goes in `.env.example`? Answer: The variable names with fake or empty values.

13.6 SQL Injection, Password Hashing and Safe Basics

INTUITION

Imagine a form where the clerk reads out what you write as an instruction to the vault. If you write "open all drawers", the clerk obeys. SQL injection happens when user text is glued into a SQL command and the database cannot tell data from instruction. A parameterised query is a form with a labelled blank: whatever you write goes in the blank and is treated only as data.

How it works

SQL injection is when untrusted input changes the meaning of a query. It is high on the OWASP Top 10 list (a well-known list of the most common web security risks, in the "injection" category). The fix is always the same: **parameterised queries** (placeholders `?` in `sqlite3`; `%s` or `:name` in other drivers). The values travel separately from the SQL text. Never build SQL with f-strings, `+` or `.format()` from user input.

Other safe basics:

- **Hashing** turns data into a fixed fingerprint, one way. The same input gives the same output; you cannot go back. **Encryption** is two-way: with the key you get the data back. Use hashing for passwords, encryption for data you must read later.
- **Passwords** must be hashed with a slow, salted algorithm: `scrypt`, `argon2`, `bcrypt` or `pbkdf2`. A **salt** is random data added per user so equal passwords give different hashes. Fast hashes like MD5 or plain SHA-256 are for checksums, not passwords.
- **Never write your own crypto.** Use maintained libraries.
- Also know: authorization checks on every request (broken access control is OWASP's top item), HTTPS everywhere, least privilege, and updating dependencies.

Worked examples

Problem 13.6.1 Show SQL injection against string-built SQL, and show that a parameterised query stops it.

Predict: for the input `x' OR '1'='1`, how many users does the vulnerable query return, and the safe one?

```
import sqlite3
con = sqlite3.connect(":memory:")
con.executescript("""
CREATE TABLE users(id INTEGER PRIMARY KEY, name TEXT, secret TEXT);
INSERT INTO users(name, secret) VALUES ('asha','pw-asha'),('ravi','pw-ravi');
""")

def vulnerable(name):
    q = f"SELECT id, name FROM users WHERE name = '{name}'"
    print("SQL:", q)
    return con.execute(q).fetchall()

def safe(name):
    q = "SELECT id, name FROM users WHERE name = ?"
    return con.execute(q, (name,)).fetchall()

print(vulnerable("asha"))
attack = "x' OR '1'='1"
print(vulnerable(attack))
print(safe("asha"))
print(safe(attack))
attack2 = "x' UNION SELECT id, secret FROM users --"
print(vulnerable(attack2))
print(safe(attack2))
```

```
SQL: SELECT id, name FROM users WHERE name = 'asha'
[(1, 'asha')]
SQL: SELECT id, name FROM users WHERE name = 'x' OR '1'='1'
[(1, 'asha'), (2, 'ravi')]
[(1, 'asha')]
[]
SQL: SELECT id, name FROM users WHERE name = 'x' UNION SELECT id, secret FROM users --'
[(1, 'pw-asha'), (2, 'pw-ravi')]
[]
```

- **Step 1** Read the printed SQL for the first attack. The quote in the input closed the string early, and `OR '1'='1'` is always true, so every row matches.
- **Step 2** The second attack uses `UNION` to read the `secret` column, which the query never intended to expose. The trailing `--` comments out the rest.
- **Step 3** With `?`, the whole input is one value. No user is called `x' OR '1'='1`, so the result is empty.
- **Check** The safe function returned `[(1, 'asha')]` for the honest input and `[]` for both attacks.
- **Answer** The vulnerable query returned all users (2) and then leaked the secret column; the parameterised query returned nothing for the attacks.
- **Why it matters** One careless f-string can leak a whole database.

Predict, Run, Explain (the "innocent" bug). You do not need an attacker to break string-built SQL. A user named O'Brien does it by accident. Real output:

```
import sqlite3
con = sqlite3.connect(":memory:")
con.execute("CREATE TABLE users(id INTEGER PRIMARY KEY, name TEXT)")
con.execute("INSERT INTO users(name) VALUES (?)", ("O'Brien",))
name = "O'Brien"
try:
    con.execute(f"SELECT * FROM users WHERE name = '{name}'")
except sqlite3.Error as e:
    print(type(e).__name__, e)
print(con.execute("SELECT * FROM users WHERE name = ?", (name,)).fetchall())
```

```
OperationalError near "Brien": syntax error
[(1, "O'Brien")]
```

Line by line: the apostrophe in `O'Brien` ends the SQL string early, leaving `Brien'` as garbage, hence the syntax error. The `?` version just works. Also note: `sqlite3` refuses `"SELECT 1; DROP TABLE users"` with `ProgrammingError: You can only execute one statement at a time.`, but other databases may allow stacked statements. Do not rely on that.

Problem 13.6.2 Hash and check a password with `scrypt`, see that salts change the hash, and see the difference between hashing and plain SHA-256 and between hashing and encryption.

Predict: if two users choose the same password, will their stored hashes match?

```
import hashlib, os, hmac, uuid

def hash_pw(pw, salt=None):
    salt = salt or os.urandom(16)
    dk = hashlib.scrypt(pw.encode(), salt=salt, n=2**14, r=8, p=1)
    return salt, dk

def check(pw, salt, dk):
    return hmac.compare_digest(hash_pw(pw, salt)[1], dk)

salt, dk = hash_pw("correct horse")
print(len(salt), len(dk))
print(check("correct horse", salt, dk), check("wrong", salt, dk))
s2, d2 = hash_pw("correct horse")
print("same password, different salt, same hash?", dk == d2)

print(hashlib.sha256(b"hello").hexdigest())
print(hashlib.sha256(b"hello!").hexdigest())
print(hashlib.md5(b"hello").hexdigest(), "(md5: fine for checksums, never for security)")
u = uuid.uuid4(); print(len(str(u)), u.version)
print(uuid.uuid5(uuid.NAMESPACE_DNS, "example.com"))
print(uuid.uuid5(uuid.NAMESPACE_DNS, "example.com"))
```

```
16 64
True False
same password, different salt, same hash? False
2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824
ce06092fb948d9ffac7d1a376e404b26b7575bcc11ee05a4615fef4fec3a308b
5d41402abc4b2a76b9719d911017c592 (md5: fine for checksums, never for security)
36 4
cfbff0d1-9375-5685-968c-48ce8b15ae17
cfbff0d1-9375-5685-968c-48ce8b15ae17
```

- **Step 1** We store the salt and the derived key (`dk`), never the password. The salt is 16 random bytes; the key is 64 bytes.
- **Step 2** To log in, hash the attempt with the stored salt and compare in constant time.

- **Step 3** Changing one character (`hello` to `hello!`) changes the whole SHA-256 output. That is the "fingerprint" property, and why SHA-256 is also used for file checksums.
- **Step 4** `uuid4` is random (version 4). `uuid5` is derived from a name, so the same name always gives the same id, which is useful for stable keys in data pipelines.
- **Check** The right password gives `True` , the wrong one `False` . Two hashes of the same password with different salts differ, so attackers cannot spot users with equal passwords.
- **Answer** No, the hashes do not match, because each has a different random salt. Hashing is one-way; encryption (not shown, needs a third-party library or cloud KMS) is reversible with a key.
- **Why it matters** If your user table leaks, salted slow hashes make cracking expensive. Plain SHA-256 or MD5 can be attacked billions of times per second.

Typical wrong version: `hashlib.sha256(password.encode()).hexdigest()` stored as the password hash. It is fast and unsalted, so identical passwords look identical and common passwords are cracked by lookup tables. Use `scrypt` / `argon2` / `bcrypt` through a vetted library.

Real-world mapping

Risk	Safe habit	Where it appears in the job
SQL injection	`execute(sql, params)`	Any query built from a filter box, API parameter or file
Weak password storage	salted slow hash	A user table in an internal tool
Data in logs	redact tokens	Production incident logs shared widely
Stable IDs	`uuid5` of a business key	Deduplicating records across loads

Applications

- **Data Analytics:** parameterised queries behind a dashboard filter.
- **Data Engineering:** SQL built from table names or dates must be validated against an allow-list, since placeholders cannot replace table names.
- **Machine Learning / GenAI:** prompt injection is the same idea (untrusted text changing instructions) for language models; keep untrusted text separate and limit what the model can do.
- **Security-minded roles:** hashing for passwords, encryption for data at rest, HTTPS for data in transit.

COMMON MISTAKE

Believing "my data comes from my own app, so it is safe". Data from a CSV, an API, or a user is untrusted until validated. Always parameterise.

Check yourself

1. What is the one-line fix for SQL injection? Answer: Use parameterised queries (placeholders), never string-built SQL.
2. Hashing or encryption for passwords? Answer: Hashing (salted and slow), because you never need the original back.
3. Why is a salt used? Answer: So the same password gives different hashes, which defeats lookup tables.

13.7 Packaging, Dependencies and Supply-Chain Safety

INTUITION

A script is a loose recipe on paper. A package is a boxed meal kit: it has a name, a version, a list of ingredients (dependencies), and a standard way to open it (`pip install`). Anyone in your company can then use your code without copying files around.

How it works

Modern Python packaging uses a single file, `pyproject.toml` , which holds:

- `[build-system]` : which tool builds it (for example `setuptools` , `hatchling` , `poetry-core`).
- `[project]` : name, version, description, Python version, dependencies, and **entry points** (`[project.scripts]` makes a command like `mini-stats`).
- `[tool.*]` : settings for tools such as `ruff` .

Layout: a `src/` **layout** (`src/mini_stats/__init__.py`) prevents you from accidentally importing the code from the current folder instead of the installed copy.

To build: `python -m build` creates a **wheel** (`.whl` , a ready-to-install zip) and an **sdist** (source archive). To share inside a company, you publish to a **private index** (for example a cloud artifact registry, or a self-hosted index) with `twine upload` and install with `pip install --index-url <your-index> mini-stats` . Public publishing goes to PyPI. Both steps not run here: `build` was not installed on this machine, and there was no network.

Dependencies and safety:

- Pin versions for applications (`requests==2.x.y`), and use ranges for libraries. Tools: `pip-tools` (`pip-compile`), `poetry` , `uv` produce a lock file.
- **Supply-chain risk**: your code trusts every package it installs. Defences: `pip-audit` (checks installed packages against known vulnerabilities), Dependabot (opens pull requests for updates), pinned hashes, using trusted indexes only, and watching for look-alike names (typosquatting). Not run here.
- **Jupyter reproducibility:** notebooks hide state (cells can run out of order). Make them reproducible by restarting and running all cells, pinning the environment, fixing random seeds, and moving reusable code into the package with the notebook only calling it (`nbconvert` can run a notebook from the command line).

Worked examples

Problem 13.7.1 Create a tiny package on disk, read its `pyproject.toml` with the standard `tomllib` , pack it into a zip the way a wheel is a zip, and import it from that archive.

Because the `build` tool was not available, this is a **demonstration of the idea**, not a real wheel (a real wheel also contains a `*.dist-info/` folder with metadata).

Predict: which files go in the archive, and what does the import print?

The files created:

```
pk/
  pyproject.toml
  src/mini_stats/__init__.py
  src/mini_stats/core.py
```

`pyproject.toml` (the part we read):

```
[project]
name = "mini-stats"
version = "0.1.0"
requires-python = ">=3.10"
dependencies = []

[project.scripts]
mini-stats = "mini_stats.cli:main"
```

```
import tomllib, zipfile, sys, pathlib, doctest
root = pathlib.Path("pk")
cfg = tomllib.loads((root / "pyproject.toml").read_text())
print(cfg["project"]["name"], cfg["project"]["version"], cfg["project"]["scripts"])
whl = root / "mini_stats-0.1.0-demo.zip"
with zipfile.ZipFile(whl, "w") as z:
    for f in (root / "src").rglob("*.py"):
        z.write(f, f.relative_to(root / "src"))
print(zipfile.ZipFile(whl).namelist())
sys.path.insert(0, str(whl))
import mini_stats
print(mini_stats.__version__, mini_stats.percentile([5, 1, 9, 3], 75))
import mini_stats.core as c
print(doctest.testmod(c))
```

```
mini-stats 0.1.0 {'mini-stats': 'mini_stats.cli:main'}
['mini_stats/__init__.py', 'mini_stats/core.py']
0.1.0 5
TestResults(failed=0, attempted=2)
```

- **Step 1** `tomllib` (standard library since Python 3.11) reads TOML into a dict.
- **Step 2** We zip the contents of `src/`, so the top folder inside the archive is `mini_stats`, the importable name. (Notice: the project name `mini-stats` uses a dash, the import name `mini_stats` an underscore.)
- **Step 3** Python can import from a zip on `sys.path`, which is the same mechanism pip uses when it installs files into `site-packages`.
- **Check** Sorted `[5, 1, 9, 3]` is `1, 3, 5, 9`. For p75: ceil(4 × 0.75) = 3, index 2, value 5. The output `5` agrees.
- **Answer** Two modules were archived, the import worked, `percentile([5, 1, 9, 3], 75)` gave 5, and both doctests passed.
- **Why it matters** Once the code is a package, a teammate runs one command, and your tests import the package like any other library.

The real build commands (not run here):

```
python -m pip install build twine
python -m build                 # creates dist/*.whl and dist/*.tar.gz
twine upload --repository-url <PRIVATE_INDEX_URL> dist/*
pip install --index-url <PRIVATE_INDEX_URL> mini-stats
pip-audit                       # scan installed packages for known vulnerabilities
```

Problem 13.7.2 You have a `requirements.txt` with a loose line `pandas`. In March the app works; in June a new major version breaks it. How do you prevent this, and what is the cost?

Step 1	Pin the exact version that works: `pandas==X.Y.Z` (use your tested version).
Step 2	Generate pins for the whole tree with `pip-compile` or a `poetry.lock` / `uv.lock` file, so even the dependencies of dependencies are fixed.
Step 3	Update on purpose: let Dependabot open a pull request, let CI run the tests, then merge.
Answer	Pin and lock the versions, and update deliberately through CI. The cost is that you must schedule updates; otherwise you also freeze security fixes.
Check	Ask: "if I build this again in a year, will I get the same packages?" With a lock file the answer is yes.
Why it matters	"It worked yesterday" bugs from silent upgrades waste days. Reproducible environments are also what makes ML experiments repeatable.

Typical wrong version: running the code from the folder above `src/`, forgetting to install the package, and seeing `ModuleNotFoundError: No module named 'mini_stats'`. The fix is `pip install -e .` (editable install) in a virtual environment, then import from anywhere.

Real-world mapping

Piece	Example	Where it appears in the job
`pyproject.toml`	name, version, deps	Shared utilities package used by many pipelines
Entry point	`mini-stats = ...`	A team CLI installed on every machine
Lock file	`uv.lock`, `poetry.lock`	Same versions in dev, CI and production
`pip-audit`	scan report	Security teams ask for it before release
Private index	internal registry	Company-only libraries

Applications

- **Data Engineering:** a shared package of connectors and validators installed in every job.
- **Machine Learning / MLOps:** training code packaged and versioned so a model can be traced to its code.
- **Data Analytics:** a team library of metric definitions, so everyone computes "active user" the same way.
- **Data Analytics:** your reporting helpers packaged once, used across projects.

COMMON MISTAKE

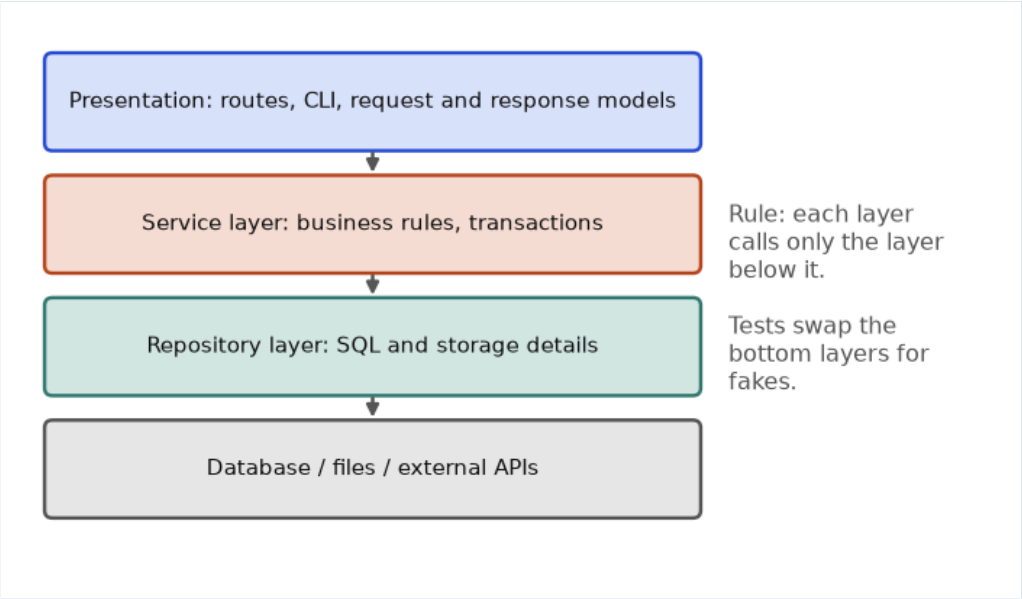
Installing packages with plain `pip install` into the global Python. Versions clash between projects. Always use a virtual environment per project (Chapter 9) and record dependencies in `pyproject.toml` or a lock file.

Check yourself

1. What does a wheel contain? Answer: A ready-to-install zip of your package files plus metadata.
2. Why pin versions for an application? Answer: So a new upstream release cannot silently break or change it.
3. What is a typosquatting attack? Answer: A malicious package with a name close to a popular one, hoping you mistype.

13.8 Production Habits: Structured Logs, Secrets, Safe Subprocesses and a Slim Container

Chapter 6 taught logging for a script you run yourself. A service run by a team at 3 a.m. needs logs a machine can search, secrets nobody can guess, and a container nobody can easily break out of. This section adds those habits. The figure below is the map to keep in mind: requests flow down through layers, and every layer should log with the same request id.



A layered design: each layer calls only the layer below it, so tests can swap the bottom layers for fakes.

How to read it: a route handler (top) calls a service function, which calls a repository, which talks to storage. A correlation id set at the top should appear in every log line from every layer.

Structured logging with a correlation id

A log line such as `charging card` is useless when 500 requests run at once. Two changes fix that. First, emit each line as JSON so tools can filter by field. Second, attach a **correlation id** (here `request_id`) to every line. Passing the id through every function argument is tedious, so use `contextvars.ContextVar`: a variable whose value belongs to the current request (each thread and each asyncio task sees its own value).

Problem 13.8.1 Log "request start" and "charging card" (with an `amount` field) for two requests, `req-a1` and `req-b2`, so every line carries its request id even though `charge()` never receives it.

Step 1	Create `request_id = ContextVar("request_id", default="-")`.
Step 2	A custom `Formatter` builds a dict from the record, adds `request_id.get()`, and returns `json.dumps`.
Step 3	Extra fields travel in `extra={"ctx": {...}}`; the formatter merges them.
Step 4	`handle()` sets the variable, and resets it in `finally`.
Check	Lines for req-a1 never show req-b2, and a line logged outside a request shows `-`.

```
import contextvars, json, logging, sys

request_id = contextvars.ContextVar("request_id", default="-")

class JsonFormatter(logging.Formatter):
    def format(self, record):
        payload = {"level": record.levelname, "logger": record.name,
                   "msg": record.getMessage(), "request_id": request_id.get()}
        payload.update(getattr(record, "ctx", {}))
        return json.dumps(payload, sort_keys=True)

handler = logging.StreamHandler(sys.stdout)
handler.setFormatter(JsonFormatter())
log = logging.getLogger("shop")
log.addHandler(handler)
log.setLevel(logging.INFO)
log.propagate = False

def charge(amount):
    log.info("charging card", extra={"ctx": {"amount": amount}})

def handle(req_id, amount):
    token = request_id.set(req_id)
    try:
        log.info("request start")
        charge(amount)
    finally:
        request_id.reset(token)

handle("req-a1", 20)
handle("req-b2", 35)
log.info("outside any request")
```

```
{"level": "INFO", "logger": "shop", "msg": "request start", "request_id": "req-a1"}
{"amount": 20, "level": "INFO", "logger": "shop", "msg": "charging card", "request_id": "req-a1"}
{"level": "INFO", "logger": "shop", "msg": "request start", "request_id": "req-b2"}
{"amount": 35, "level": "INFO", "logger": "shop", "msg": "charging card", "request_id": "req-b2"}
{"level": "INFO", "logger": "shop", "msg": "outside any request", "request_id": "-"}
```

COMMON MISTAKE

Using a plain global (`CURRENT_ID = "req-a1"`) for the id. With threads or async tasks the requests overwrite each other and logs get mixed. Right: a `ContextVar` , which isolates each task. Also never log passwords, tokens or full card numbers in the `ctx` fields.

Configuring logging in one place with dictConfig

`logging.config.dictConfig` describes formatters, handlers and per-logger levels as one dictionary, which you can load from a JSON or YAML file and change per environment without touching code. Child loggers (`shop.db`) inherit handlers from their parent (`shop`) but can set their own level.

Problem 13.8.2 Configure logger `shop` at INFO and `shop.db` at WARNING. Which of three messages appear?

Predict

`shop` INFO passes. `shop.db` INFO is below its WARNING level and is dropped. `shop.db` WARNING passes and propagates to the `shop` handler.

```
import logging, logging.config, io

stream = io.StringIO()
CONFIG = {
    "version": 1,
    "disable_existing_loggers": False,
    "formatters": {"plain": {"format": "%(levelname)s %(name)s: %(message)s"}},
    "handlers": {"mem": {"class": "logging.StreamHandler", "stream": stream, "formatter": "plain"}},
    "loggers": {"shop": {"level": "INFO", "handlers": ["mem"], "propagate": False},
                "shop.db": {"level": "WARNING"}},
}
logging.config.dictConfig(CONFIG)
logging.getLogger("shop").info("started")
logging.getLogger("shop.db").info("hidden: db logger is WARNING")
logging.getLogger("shop.db").warning("slow query")
print(stream.getvalue(), end="")
```

```
INFO shop: started
WARNING shop.db: slow query
```

Secrets: use the secrets module, never random

The `random` module is predictable by design (Chapter 4). For tokens, password-reset links and API keys use `secrets`, which draws from the operating system's secure source. Compare secrets with `secrets.compare_digest`, which takes the same time whether the first or last character differs, so an attacker cannot learn the answer by timing.

Problem 13.8.3 Make a URL-safe token, compare two strings safely, and make an 8-character lowercase code.

Step 1	`token_urlsafe(16)` uses 16 random bytes, which is 22 URL-safe characters.
Step 2	`compare_digest` returns a bool in constant time.
Step 3	`secrets.choice` picks one character from an alphabet, 8 times.
Check	The token values are random, so the code prints only checks that are always the same.

```
import secrets, string
token = secrets.token_urlsafe(16)
print(len(token) >= 21, set(token) <= set(string.ascii_letters + string.digits + "-_"))
print(secrets.compare_digest("abc", "abc"), secrets.compare_digest("abc", "abd"))
alphabet = string.ascii_lowercase + string.digits
code = "".join(secrets.choice(alphabet) for _ in range(8))
print(len(code))
```

```
True True
True False
8
```

subprocess: the shell is the danger

With `shell=True`, Python hands one string to the system shell, which treats `;`, `&&` and `$(...)` as commands. If any part of the string came from a user, they control your machine. Pass a list instead and no shell is involved.

Problem 13.8.4 A user supplies the filename `notes.txt; echo INJECTED`. Show what each style does (the example uses `echo`, which is harmless).

Shell string	The shell splits at `;` and runs a second command, so two lines are printed.
List form	The whole filename is one argument, so one line is printed, semicolon included.

```
import subprocess, sys
filename = "notes.txt; echo INJECTED"
# DANGEROUS: the shell reads the whole string as commands
bad = subprocess.run(f"echo reading {filename}", shell=True, capture_output=True, text=True)
print(bad.stdout.strip().splitlines())
# SAFE: a list of arguments, no shell
good = subprocess.run(["echo", "reading", filename], capture_output=True, text=True)
print(good.stdout.strip().splitlines())
```

```
['reading notes.txt', 'INJECTED']
['reading notes.txt; echo INJECTED']
```

Unix-like systems only: the example uses the `echo` program, which Windows does not ship as a file.

COMMON MISTAKE	Thinking "I will just quote the user's text with f-string quotes" is enough. Wrong: `f"grep '{term}' file"`, broken by a term that contains a quote. Right: `subprocess.run(["grep", "--", term, "file"], check=True)`. If you must use a shell, `shlex.quote(term)` escapes it.

pickle is code execution

`pickle.loads` rebuilds objects by calling functions named in the data. An attacker who controls the bytes controls the functions. The demo prints a harmless line, but it could as easily delete files.

```
import pickle
class Evil:
    def __reduce__(self):
        return (print, ("code ran during pickle.loads",))
blob = pickle.dumps(Evil())
pickle.loads(blob)
```

```
code ran during pickle.loads
```

Rule: never unpickle data from the network, a user upload or a shared cache you do not fully control. Use JSON (Chapter 5) for data exchange. The same warning applies to `yaml.load` without `safe_load`.

Going deeper: a multi-stage Dockerfile (not run here)

This needs Docker, which was not available here, so treat it as a template. A multi-stage build installs dependencies in a large builder image and copies only the result into a small runtime image that runs as a non-root user. Smaller images download faster and contain fewer programs for an attacker to use. See the Docker document for details.

```
# Stage 1: build wheels with compilers available
FROM python:3.12 AS builder
WORKDIR /app
COPY requirements.txt .
RUN pip wheel --no-cache-dir --wheel-dir /wheels -r requirements.txt

# Stage 2: slim runtime, no compilers
FROM python:3.12-slim
RUN useradd --create-home appuser
WORKDIR /app
COPY --from=builder /wheels /wheels
RUN pip install --no-cache-dir /wheels/* && rm -rf /wheels
COPY src/ ./src/
USER appuser
HEALTHCHECK CMD python -c "import urllib.request as u; u.urlopen('http://localhost:8000/health')"
CMD ["python", "-m", "src.main"]
```

Problem 13.P.12 (stretch) Why does the Dockerfile copy `requirements.txt` and install before copying `src/`?

Answer: Docker caches each layer. Your code changes often, dependencies rarely. Installing first means a code edit reuses the cached dependency layer and the rebuild takes seconds.

Problem 13.P.13 (what is the output?) `secrets.compare_digest("abc", "abc")` and `secrets.compare_digest("abc", "abd")`.

Answer: `True` then `False`, as printed in Problem 13.8.3. The difference from `==` is not the result but the constant timing.

Chapter recap

- Clean code means good names, small functions, no repetition, and comments that say why.
- Refactor in small steps with tests green. Never mix refactoring with new features.
- Layers separate API, rules and data. Dependency injection lets you test with fakes and mocks.
- Start with a monolith. Queues and events help with spikes and decoupling, but consumers must be idempotent.
- Agile works in short cycles. Estimate with ranges (PERT: `(O + 4M + P) / 6`). Write down technical debt.
- Good code review is small, kind, specific, and focused on behaviour and risk.
- CI runs build, lint, test (cheap first) and stops at the first failure. Exit codes decide.
- Configuration lives in the environment. Secrets never go in Git. If one leaks, rotate it.
- Parameterised queries stop SQL injection. Hash passwords with a salted slow algorithm.
- A package has `pyproject.toml`, a `src/` layout, pinned dependencies and scans (`pip-audit`).

Cheat sheet

Topic	Pattern
PERT estimate	`(O + 4M + P) / 6`: (2, 4, 12) gives 5.0
Safe SQL	`con.execute("... WHERE name = ?", (name,))`
Password hash	`hashlib.scrypt(pw, salt=os.urandom(16), n=2**14, r=8, p=1)` and `hmac.compare_digest`
Required env var	`os.environ["NAME"]` (raises `KeyError`)
Optional env var	`os.environ.get("NAME", default)`
`.gitignore`	add `.env`; commit `.env.example`
Doctest	`python -m doctest -v file.py` or `doctest.testmod()`

Topic	Pattern
Build a package	`python -m build`, then `twine upload`
Editable install	`pip install -e .`
CI order	lint, types, unit tests, integration tests, build, deploy

Practice set

Problem 13.P.1 (easy) What is wrong with the name `def f(d, t):`, and what would you change?

Answer: The names tell nothing. Use names that say what they do and mean, such as `def slow_delivered_orders(orders, threshold_minutes):`, and add a docstring.

Problem 13.P.2 (easy) What is printed? `import os; print(os.environ.get("NOPE", "fallback"))`

Answer: `fallback`. `get` returns the default when the variable is missing.

Problem 13.P.3 (easy) A teammate commits `.env` with a real password. List the first two things you do.

Answer: (1) Revoke or rotate the password immediately. (2) Remove the file from the repo and add `.env` to `.gitignore`; then clean the history if needed and add a secret scanner.

Problem 13.P.4 (medium) Estimate with PERT: optimistic 1 day, likely 3, pessimistic 11.

Answer: (1 + 4 x 3 + 11) / 6 = (1 + 12 + 11) / 6 = 24 / 6 = 4.0 days.

Problem 13.P.5 (medium) What is the output? `sqlite3` query `"SELECT * FROM t WHERE n = '" + user + "'"` with `user = "a' OR '1'='1"`. How many rows come back from a table of 5 rows?

Answer: The condition becomes `n = 'a' OR '1'='1'`, which is true for every row, so all 5 rows. The fix is `execute("SELECT * FROM t WHERE n = ?", (user,))`, which returns only rows where `n` equals the full text.

Problem 13.P.6 (medium) Find the bug: `password_hash = hashlib.sha256(pw.encode()).hexdigest()`.

Answer: SHA-256 is fast and, here, unsalted. Equal passwords give equal hashes and attackers can try billions of guesses per second. Use `scrypt`, `argon2` or `bcrypt` with a random salt.

Problem 13.P.7 (medium) A CI pipeline: lint 1 min, unit 3 min, integration 8 min, build 2 min. A unit test fails. How long until the developer sees red if the order is lint, unit, integration, build? And if integration comes before unit?

Answer: Order lint, unit: 1 + 3 = 4 minutes. If integration first: 1 + 8 + 3 = 12 minutes. Put faster checks earlier.

Problem 13.P.8 (medium) After `pip install -e .` you still get `ModuleNotFoundError` for your package. Name two likely causes.

Answer: You may be using a different Python or virtual environment from the one where you installed it (check `which python` and `pip -V`), or the package folder lacks `__init__.py` or is not found by the `src` layout configuration in `pyproject.toml`.

Problem 13.P.9 (hard, scenario) Your service sends order emails when an order is placed. The email provider is slow, so the API times out under load. Redesign using the ideas in this chapter.

Answer: Emit an "order placed" event or put a message on a queue and return at once. A separate worker consumes the queue and sends the email, with retries. Make the worker idempotent (store a sent flag per order id) because messages can arrive twice. Monitor queue depth.

Problem 13.P.10 (hard, interview) Explain hashing, encryption and encoding (base64) and say which you would use to store a password, a credit-card number you must show later, and a JWT payload.

Answer: Hashing is one-way (password: salted slow hash). Encryption is two-way with a key (credit-card number: encrypt, with keys in a managed key service, and consider not storing it at all). Encoding (base64) is a reversible format change with no secrecy; a JWT payload is only base64-encoded and signed, so it is readable by anyone.

Problem 13.P.11 (hard, interview) A teammate says "we do not need tests, we do code review." Give a balanced reply.

Answer: Review and tests catch different things. Reviewers are good at design and readability but miss edge cases and cannot rerun everything after each change. Tests are repeatable and protect against regressions (old bugs returning), especially when refactoring. Use both: tests make the review faster because the reviewer can focus on design.

Prompt pack

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional at an Indian company moving into data and AI roles, and English is my second language, so use simple words. Teach me "clean code and refactoring in Python": naming, small functions, DRY, docstrings, and safe refactoring with tests. Walk through one messy 30-line function, refactor it step by step, and show the output before and after to prove behaviour did not change. After each step ask me one check question and wait for my reply.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data and AI roles, and English is my second language. Quiz me one question at a time on "secrets and configuration handling": environment variables, .env and .gitignore, secret scanners like gitleaks, secret managers, rotation, and what to do when a key leaks. Start easy, get harder, explain each wrong answer simply, and give me a score out of 10 at the end.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional learning Python for data jobs, and English is my second language. Explain "SQL injection and parameterised queries" step by step with sqlite3 examples: show a vulnerable f-string query, an attack input, and the safe version, with real outputs. Then give me 8 graded exercises without solutions, including one where I must fix vulnerable code, and one on password hashing with a salt. Review my answers when I paste them.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data engineering, and English is my second language. Review my CI/CD and packaging setup. Check my pyproject.toml, my GitHub Actions workflow, my dependency pinning, and whether secrets are handled safely. Point out the five most important problems in order, explain each in simple English, and show the corrected snippet. My files: [PASTE HERE]
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional preparing for Python and data engineering interviews, and English is my second language. Run a mock interview on "professional software practice": clean code, testing pyramid, dependency injection, monolith versus microservices, queues and idempotency, code review, CI/CD, SQL injection and password storage. Ask one question at a time, grade my answer out of 5, give a model answer, and give one tip to explain it more clearly in English.
```

CHAPTER 14

CAPSTONE PROJECTS AND THE PYTHON INTERVIEW BANK

Prove your skills with four real projects, a 150-problem plan, and 45 graded interview questions with answers.

Reading about Python does not get you hired. Showing working projects and answering questions calmly does. This chapter gives you both. First, four capstone projects. Each has a clear specification, milestones you can finish in one hour a day, and a way to check that you are done. They tie together everything from Chapters 1 to 13 and use one friendly theme, a restaurant kitchen and its orders, so that your portfolio tells one story: "a professional who can build data tools". Second, a plan to solve 150 or more practice problems, with solved examples that were run for real. Third, an interview bank, graded from easy to hard, with answers written the way you would say them aloud.

Everything marked "run" was executed with Python 3.14 and the standard library. FastAPI, pytest, `requests`, `BeautifulSoup` and similar tools are marked **not run here**. Interview answers describe general Python behaviour. Some details change by version (for example the free-threaded build of Python and the GIL), so check the current docs before an interview.

Words you need. **Capstone** means a final project that uses many skills together. **Milestone** is a small checkpoint on the way. **Definition of done** is the list of things that must be true before you call a project finished.

14.1 Capstone 1: A Command-Line Tool With Logging, Tests and a Typed API

INTUITION A craftsperson proves skill by making one complete object, not by owning many tools. This project is one small tool, finished to a professional standard: it has a clear interface, logs what happens, is tested, and has type hints.

Specification

Build `order-summary`, a tool that reads a CSV file of kitchen orders and prints, per dish, the count, average preparation time, p90 and failure percentage. (You already built the first version of the maths in Chapters 12 and 13.)

Input columns: `placed_at_s,prep_seconds,dish,served_ok`.

Interface:

```
order-summary orders.csv --percentile 90 --format table|json --verbose
```

Definition of done:

- A `main(argv)` function that returns an exit code (0 success, 2 bad usage, 1 for data errors).
- `logging` to stderr; results to stdout.
- A typed core API: `def summarise(rows: Iterable[Order], p: int) -> list[DishStats]`, with `dataclass` types, passing `mypy`.
- At least 10 `pytest` tests, including empty file, one row, bad number, and a missing column.
- `ruff` clean, a `pyproject.toml` with an entry point, and a README with install and usage in five lines.

Milestones (about 8 hours)

1. Day 1: Parse the CSV with `csv.DictReader` and compute stats for a hard-coded file.
2. Day 2: Add `argparse` options and `main(argv)`.
3. Day 3: Add `Order` and `DishStats` dataclasses and type hints; run `mypy`.
4. Day 4: Add logging and good error messages (no raw tracebacks for user mistakes).
5. Day 5: Write tests with `pytest` and `parametrize`.
6. Day 6: Add `ruff`, `pre-commit`, `pyproject.toml`, and a README.

7. Day 7: Add a GitHub Actions workflow (Chapter 13) and tag `v0.1.0`.
8. Day 8: Ask a friend to install and use it from your README only. Fix what confuses them.

The picture

```
 orders.csv --> read_rows() --> [Order ...] --> summarise() --> [DishStats ...]
                    |                                |                |
              logging (stderr)                  pure function     format: table / json
                                                 (easy to test)        |
                                                                    stdout + exit code
```

Read it left to right. The "pure" middle function has no file or print inside it, which is why it is easy to test.

Worked example

Problem 14.1.1 Write the core of the tool for a tiny inline file and check every number by hand. (This was run; it is the prototype for milestone 1 and also serves Capstone 3.)

Predict: which dish has the highest failure rate, and how many orders per minute arrive?

```
import csv, io, statistics, math
from collections import defaultdict
data = """placed_at_s,prep_seconds,dish,served_ok
1000,120,salad,true
1100,95,salad,true
1200,2300,pasta,false
1500,180,salad,true
1800,450,pasta,true
2000,510,pasta,true
2400,130,soup,true
2900,700,soup,true
"""
def pct(xs, p):
    xs = sorted(xs); return xs[max(0, math.ceil(len(xs)*p/100)-1)]
by = defaultdict(list); err = defaultdict(int); stamps = []
for r in csv.DictReader(io.StringIO(data)):
    by[r["dish"]].append(int(r["prep_seconds"]))
    err[r["dish"]] += r["served_ok"] != "true"
    stamps.append(int(r["placed_at_s"]))
print(f"{'dish':8}{'n':>3}{'avg':>8}{'p90':>7}{'fail%':>7}")
for k, v in sorted(by.items()):
    print(f"{k:8}{len(v):>3}{statistics.mean(v):>8.1f}{pct(v,90):>7}{100*err[k]/len(v):>7.1f}")
n = sum(len(v) for v in by.values())
print("orders/min:", round(n / ((max(stamps)-min(stamps))/60), 2))
```

```
dish      n     avg    p90  fail%
pasta     3  1086.7   2300   33.3
salad     3   131.7    180    0.0
soup      2   415.0    700    0.0
orders/min: 0.25
```

- **Formula** average = sum / n; failure % = failures / n x 100; arrival rate = n / duration in minutes; duration = (last time - first time) / 60 (the times are in seconds).
- **Compute (salad)** (120 + 95 + 180) / 3 = 395 / 3 = 131.67, shown as 131.7.
- **Compute (pasta)** (2300 + 450 + 510) / 3 = 3260 / 3 = 1086.67. One failure out of 3 is 33.3%. For p90, n = 3, ceil(2.7) = 3, index 2, so the slowest order, 2300.
- **Compute (arrival rate)** 8 orders, duration (2900 - 1000) / 60 = 31.67 minutes, so 8 / 31.67 = 0.25 orders per minute (one order every 4 minutes).

- **Answer** `pasta` has the highest failure rate (33.3%) and the worst average (about 1087 seconds, over 18 minutes). Orders arrive at 0.25 per minute.
- **Check** The counts 3 + 3 + 2 = 8 match the 8 data rows.
- **Why it matters** In plain words: one pasta order in three failed, and the pasta orders that did work were slow. That is the headline a manager needs. Notice that with only 3 orders a p90 equals the maximum: percentiles need many samples to mean much.

Typical wrong version: `err[r["dish"]] += r["served_ok"]` (adding the text `"false"`). It fails with `TypeError: unsupported operand type(s) for +=: 'int' and 'str'`. CSV values are always strings, so you must compare or convert. Fix: `r["served_ok"] != "true"` as above.

Real-world mapping

Piece	Example	Where it appears in the job
`main(argv)`	called from tests	Any production CLI
Dataclass rows	`Order`, `DishStats`	Typed data between steps of a pipeline
Exit codes	0 / 1 / 2	Schedulers and CI
Pure core	`summarise()`	Reusable in a notebook and an API

Applications

- **Data Engineering:** a validation CLI run before every load.
- **Data Analytics:** a tool that turns raw exports into a standard report.
- **Machine Learning / MLOps:** `train.py` and `evaluate.py` with the same discipline.
- **Operations:** turning any raw export (orders, tickets, test results) into your own report format.

COMMON MISTAKE

Putting the logic inside `main()` with `print` calls and file reads. You cannot test it without files. Keep a pure function in the middle and thin input/output around it.

Check yourself

1. Why return an exit code? Answer: Schedulers and CI use it to decide pass or fail.
2. Why separate the pure function? Answer: It is easy to test and reuse with no files or printing.
3. Why does p90 of 3 samples equal the maximum? Answer: ceil(3 x 0.9) = 3, so the third (largest) value is chosen.

14.2 Capstone 2: A FastAPI Service With a Database and Tests

INTUITION

A service is a restaurant that stays open: customers (clients) come at any time. Here you build a small one that stores kitchen shifts and serves summaries, and prove with tests that it behaves.

Specification

Build `kitchen-api`, a service to store kitchen shifts (lunch, dinner) and their order summaries.

Endpoints:

Method and path	What it does	Status codes
`POST /v1/shifts`	create a shift (`name`, `started_at`)	201, 422
`GET /v1/shifts`	list shifts with `limit` and `offset`	200

Method and path	What it does	Status codes
`GET /v1/shifts/{id}`	one shift	200, 404
`POST /v1/shifts/{id}/orders`	upload a list of orders	201, 404, 422
`GET /v1/shifts/{id}/summary`	per dish stats from Capstone 1	200, 404
`DELETE /v1/shifts/{id}`	delete	204

Definition of done:

- Layers: routes, a service module, a repository module (SQLite, then optional PostgreSQL).
- Pydantic models for input and output. Parameterised SQL only.
- Config from environment variables. No secrets in the repository.
- Tests with `pytest` and FastAPI's `TestClient`, using an in-memory database through a fixture; at least 12 tests, including 404, 422, pagination end and idempotent delete.
- An OpenAPI page at `/docs`, a Dockerfile (optional), and CI.

Not run here (needs `fastapi`, `httpx`, `pytest`). The shape of a test:

```
from fastapi.testclient import TestClient
from kitchen_api.main import create_app

def test_create_and_get_shift():
    client = TestClient(create_app(db_url=":memory:"))
    r = client.post("/v1/shifts", json={"name": "lunch", "started_at": "2026-01-31T10:00:00Z"})
    assert r.status_code == 201
    shift_id = r.json()["id"]
    assert client.get(f"/v1/shifts/{shift_id}").json()["name"] == "lunch"

def test_unknown_shift_is_404():
    client = TestClient(create_app(db_url=":memory:"))
    assert client.get("/v1/shifts/999").status_code == 404
```

Milestones (about 10 hours)

1. Design the tables (`shifts`, `orders`) and write the repository with plain `sqlite3`, tested by itself.
2. Add the service layer that calls the Capstone 1 `summarise()`.
3. Add FastAPI routes and pydantic models. Look at `/docs`.
4. Write tests for every route and every error code.
5. Add pagination and the `Location` header on create.
6. Add configuration from the environment and structured logging.
7. Add CI, linting, type checks. Time 200 requests against it with a small script (or a tool such as Locust) and write down the p90 and the requests per second.

The picture

```
client --HTTP--> [routes: validate] --> [service: rules] --> [repository: SQL] --> SQLite
                       ^                       ^                     ^
                 pydantic models          calls summarise()    parameterised queries
                       |                                             |
                  TestClient  <--- tests use a fake or in-memory repo --+
```

Worked example

Problem 14.2.1 Prove the design idea of the repository layer without any framework: write a `ShiftRepository` over `sqlite3` with parameterised SQL, and show create, get, and list with pagination.

Predict: what does `list(limit=2, offset=2)` return after three shifts are created?

```
import sqlite3

class ShiftRepository:
    def __init__(self, path=":memory:"):
        self.con = sqlite3.connect(path)
        self.con.row_factory = sqlite3.Row
        self.con.execute("CREATE TABLE IF NOT EXISTS shifts("
                         "id INTEGER PRIMARY KEY, name TEXT NOT NULL)")
    def create(self, name):
        with self.con:
            cur = self.con.execute("INSERT INTO shifts(name) VALUES (?)", (name,))
        return cur.lastrowid
    def get(self, shift_id):
        row = self.con.execute("SELECT id, name FROM shifts WHERE id = ?", (shift_id,)).fetchone()
        return dict(row) if row else None
    def list(self, limit=10, offset=0):
        rows = self.con.execute("SELECT id, name FROM shifts ORDER BY id LIMIT ? OFFSET ?",
                                (limit, offset)).fetchall()
        return [dict(r) for r in rows]

repo = ShiftRepository()
for n in ("lunch", "dinner", "brunch"):
    print("created", repo.create(n))
print(repo.get(2))
print(repo.get(99))
print(repo.list(limit=2, offset=2))
try:
    repo.create(None)
except sqlite3.IntegrityError as e:
    print("IntegrityError:", e)
```

I ran this and the real output was:

```
created 1
created 2
created 3
{'id': 2, 'name': 'dinner'}
None
[{'id': 3, 'name': 'brunch'}]
IntegrityError: NOT NULL constraint failed: shifts.name
```

- **Step 1** `with self.con:` commits on success and rolls back on error.
- **Step 2** `ORDER BY id LIMIT 2 OFFSET 2` skips two rows and takes up to two: only shift 3 remains.
- **Step 3** The database itself rejects `None` for `name`, which is a second line of defence after pydantic.
- **Check** Three rows exist, offset 2 skips ids 1 and 2, so one row is returned.
- **Answer** The last page holds one shift, `{'id': 3, 'name': 'brunch'}`. A missing id returns `None`, which the route turns into 404.
- **Why it matters** The route code never sees SQL, so you can swap SQLite for PostgreSQL later without touching it.

Typical wrong version: forgetting `ORDER BY` with `LIMIT` / `OFFSET`. Databases do not promise an order without it, so pages can overlap or skip rows. No error message appears, which is why you must always add it.

Real-world mapping

Piece	Example	Where it appears in the job
Repository	`ShiftRepository`	Data access in any service
`TestClient`	in-memory database	Fast tests of every status code
`Location` header	`/v1/shifts/7`	Standard on 201 Created
Timing your own API	200 requests, p90 and requests per second	Shows your service is fast enough

Applications

- **Machine Learning / MLOps:** the same layout serves `POST /predict`.
- **Data Engineering:** an internal metadata service that records pipeline runs.
- **GenAI:** a small gateway that logs prompts and responses to a database (without storing personal data).
- **Backend development:** a service that you both build and time, then publish the report.

COMMON MISTAKE Testing only the happy path. Most production bugs live in 404, 422, empty list and duplicate cases. Write those tests first.

Check yourself

1. Why is `ORDER BY` needed with `LIMIT/OFFSET`? Answer: Without it the order is undefined, so pages can overlap or skip.
2. What does `with con:` do? Answer: Commits if no error, rolls back if there is one.
3. Why test with an in-memory database? Answer: It is fast and isolated, so each test starts clean.

14.3 Capstone 3: A Queue and Service-Time Analyser

INTUITION Think of a restaurant manager reading the kitchen's ticket log at the end of the night. This project turns that log into a data product: it reads raw orders, finds the slow dish, and writes the conclusion in plain English. It shows you can combine domain knowledge with Python and statistics.

Specification

Build `order-analyse`, which reads the same orders CSV as Capstone 1 (or any similar export, such as bank-queue or hospital-appointment logs) and produces a short report.

It must compute:

- per dish: count, average, median, p90, p95, p99, maximum, error %;
- orders per minute overall, and per 10-minute window;
- a time series of p90 per window, to see degradation over time;
- Little's law check: average number of orders in the kitchen = arrival rate x average preparation time (in the same time unit, minutes);
- a verdict: flag any dish where p95 is over a service level agreement (SLA, a promised limit, for example 15 minutes) or failure % is over 1%.

Extra credit: compare two weeks and say whether the change is real or just noise (use a bootstrap or a Mann-Whitney test, see the statistics documents); draw a chart with matplotlib (not run here).

Definition of done: a CLI or notebook, tests with a small fixed CSV, and a one-page write-up with your findings and one chart.

Milestones (about 6 hours)

1. Load the CSV with the standard `csv` module, then repeat with pandas (not run here) and compare the code size.
2. Reuse your tested `percentile` function.
3. Group by 10-minute windows using integer division: `window = (ts - first_ts) // 600` (times are in seconds).
4. Add the Little's law line and the verdict rules.
5. Generate a Markdown report.
6. Test with a CSV where you know the answers by hand.

The picture

```
prep_seconds
  2300 |                *  <- pasta: slow and failed
   700 |            *
   450 |       *  *
   120 |  *  *        *
       +------------------------> time
       window 0     window 1      p90 per window shows when it degrades
```

Each star is one order. The vertical axis is preparation time; the horizontal axis is when the order was placed. A rising cloud means the kitchen is slowing down as the evening gets busier.

Worked example

Problem 14.3.1 Using the 8-order data from 14.1.1, check Little's law and compute p90 per 10-minute window. Then do the same for a busier evening.

Formula	Little's law: average number in system `L = λ × W`, where `λ` is the arrival rate (orders per minute) and `W` is the average time an order spends in the kitchen, in minutes.
Plug in	λ = 8 / 31.67 = 0.2526 per minute (from 14.1.1). Overall average prep time = (120 + 95 + 2300 + 180 + 450 + 510 + 130 + 700) / 8.
Compute	The sum is 4485 seconds, so the average is 4485 / 8 = 560.6 s = 9.34 minutes. Then `L = 0.2526 × 9.34 = 2.36`.
Answer	On average about 2.4 orders were in the kitchen at once.
Check	Another way: total time spent by all orders is 4485 s = 74.75 minutes over a 31.67 minute window, so 74.75 / 31.67 = 2.36. Both agree.
Why it matters	If the kitchen has 10 cooks but Little's law says only 2.4 orders are in progress on average, most cooks are idle, so you may be over-staffed (or the orders are not reaching the kitchen). This check catches wrong assumptions about how busy a system really is.

Python check for the arithmetic:

```
prep = [120, 95, 2300, 180, 450, 510, 130, 700]
avg_min = sum(prep) / len(prep) / 60
lam = 8 / (1900 / 60)
print(sum(prep), round(avg_min, 2), round(lam, 4), round(lam * avg_min, 2), round(sum(prep) / 60 / (1900 / 60), 2))
```

```
4485 9.34 0.2526 2.36 2.36
```

Typical wrong version: mixing units: using seconds for W and minutes for the arrival rate. You would get `0.2526 x 560.6 = 141.6`, which is 60 times too large, with no error. Always convert to the same unit first and check the unit.

Real-world mapping

Piece	Example	Where it appears in the job
Percentiles per dish	p95 of `pasta`	SLA reports
Arrival rate per window	orders per 10 minutes	Finding the busy hours
Little's law	L = λ × W	Checking how busy the system is
Verdict rules	p95 > 15 min or failures > 1%	Automatic pass/fail in a daily check

Applications

- **Data Analytics:** a reusable dashboard of delivery-time distributions.
- **Data Science:** a significance test for "is the new recipe really faster?".
- **MLOps:** delay and error analysis of a model endpoint after each release.
- **Operations:** a nightly queue-time report, automated and made reproducible.

COMMON MISTAKE

Reporting only the average. A few very slow requests hide behind a pleasant mean. Always show percentiles and error rate together.

Check yourself

1. State Little's law. Answer: Average number in the system equals throughput times average time in the system.
2. Why percentiles per window? Answer: They show when and how the system degrades over time.
3. What unit must W be in if throughput is per second? Answer: Seconds.

14.4 Capstone 4: A Responsible Scraper

INTUITION

This project is about manners as much as code. You collect public data without harming the site or the people in the data, and you can show your rules.

Specification

Build `polite-scrape`, which collects titles and prices from a **site you are allowed to scrape** (your own test site, a practice site made for scraping, or one whose terms allow it). You may start with the local HTML page of Chapter 12 and later move to a real allowed site.

Rules the program must enforce:

1. It reads `robots.txt` first and refuses disallowed URLs.
2. It sends a `User-Agent` with a contact.
3. It waits at least the crawl delay (default 2 seconds) between requests.
4. It caches each page on disk, so a rerun makes no new requests.
5. It stops on 429 or 503 and backs off.
6. It stores only the fields needed, and no personal data.

Definition of done: tests that use a local fake server (no real network), a README that states which site, why scraping is allowed, and the rate used.

Milestones (about 5 hours)

1. Parse the demo HTML with `html.parser`, then repeat with BeautifulSoup (not run here).
2. Add the `robots.txt` check using `urllib.robotparser`.

3. Add the delay and the cache.
4. Add error handling and backoff.
5. Write tests against a local `http.server`.
6. Write the README with the policy.

The picture

Read it as a loop. Every request passes through the same gate.

```
 url list --> [robots allows?] --no--> skip + log
                    | yes
              [in cache?] --yes--> parse cached html
                    | no
              [wait crawl-delay] --> fetch --> status ok? --no(429/503)--> back off
                                                    | yes
                                      save to cache --> parse --> store rows
```

Worked example

Problem 14.4.1 Implement the gate and cache logic with no network: a function that decides whether to fetch, using a robots text and a dictionary as cache.

Predict: for the three URLs below, which are fetched, which are skipped, and which come from cache?

```
from urllib.robotparser import RobotFileParser

ROBOTS = """User-agent: *
Disallow: /private/
Crawl-delay: 2
"""
rp = RobotFileParser(); rp.parse(ROBOTS.splitlines())
cache = {"https://site.test/b": "<html>cached b</html>"}
UA = "StudyBot/1.0 (me@example.com)"

def plan(url):
    if not rp.can_fetch(UA, url):
        return "SKIP (robots)"
    if url in cache:
        return "CACHE"
    return "FETCH (wait %ss first)" % rp.crawl_delay(UA)

for u in ["https://site.test/a", "https://site.test/b", "https://site.test/private/x"]:
    print(u, "->", plan(u))
```

```
https://site.test/a -> FETCH (wait 2s first)
https://site.test/b -> CACHE
https://site.test/private/x -> SKIP (robots)
```

- **Step 1** `can_fetch` is checked first, because a disallowed URL must never be fetched, even from cache logic.
- **Step 2** The cache is a dictionary here. In the project it becomes files named by a hash of the URL.
- **Step 3** Only `/a` needs a real request, and it must wait the crawl delay first.
- **Check** Three URLs, three different outcomes, and exactly one real request, which for 3 pages at a 2 second delay is the least load possible.
- **Answer** `/a` is fetched after a 2 second wait, `/b` comes from the cache, `/private/x` is skipped.
- **Why it matters** Reruns during development cost the site nothing, and your bot cannot wander into places the owner closed.

Typical wrong version: testing against the real site every time while you debug. That can get you blocked. Save one page to disk and develop against the file.

Real-world mapping

Piece	Example	Where it appears in the job
robots check	`can_fetch`	A legal and ethical gate before any crawl
Cache	files keyed by hash	Saves requests and speeds up development
Backoff	wait longer after 429	How production crawlers stay polite
Policy README	site, reason, rate	What a reviewer or lawyer asks to see

Applications

- **Data Analytics:** a small, lawful price dataset for an analysis.
- **Data Engineering:** the same gate-and-cache design for a scheduled ingestion from an API with rate limits.
- **GenAI:** collecting permitted documents for a retrieval project, with a record of source and licence.
- **Machine Learning:** building a training set with documented origin (data provenance).

COMMON MISTAKE

Saying "it is public, so I can take it". Public does not mean free to copy. Check the terms, robots.txt, copyright and privacy rules, and prefer APIs and open datasets.

Check yourself

1. What do you check before the first request? Answer: robots.txt and the terms of service.
2. Why cache pages? Answer: It avoids repeated requests and lets you develop offline.
3. What should the bot do on a 429? Answer: Stop or back off for longer, and respect `Retry-After` if given.

14.5 The Mastery Tests and the 150-Problem Plan

INTUITION

Strength comes from repetition with feedback, like going to the gym. A plan of 150 problems sounds big. At two problems a day it is about 11 weeks, which fits your one hour a day.

The four mastery tests

From your skill map, you are "Python-strong" when you can do all four:

1. Write a CLI tool with argparse, logging, tests and a typed API. (Capstone 1.)
2. Build a small FastAPI service with a database and tests. (Capstone 2.)
3. Explain generators, decorators, the GIL and the MRO without notes. (Use the interview bank below and teach a friend.)
4. Solve 150+ Python or LeetCode problems.

The 150-problem plan

Weeks	Topic	Problems	Examples of patterns
1-2	Strings and arrays	25	two pointers, sliding window, counting
3-4	Hash maps and sets	25	two-sum, grouping, frequency
5	Stacks, queues, linked lists	20	valid parentheses, queue by stack
6	Sorting, binary search, intervals	20	merge intervals, search in a rotated array

Weeks	Topic	Problems	Examples of patterns
7	Recursion, backtracking	15	subsets, permutations
8	Trees and graphs (BFS, DFS)	20	shortest path in a grid, level order
9	Heaps and greedy	10	top-k, schedule
10-11	Dynamic programming	15	climbing stairs, knapsack, edit distance

Method for each problem: (1) read and restate it, (2) work an example by hand, (3) write the simple solution, (4) state time and space cost, (5) improve it, (6) a day later, solve it again from memory. Keep a log with the pattern name and what you missed. Chapter 1 of the DSA document explains complexity in depth; here we focus on Python.

Worked examples (run)

Problem 14.5.1 Solve six classic problems and check every answer by hand.

Predict: what are the six outputs?

```
from collections import Counter, deque
import heapq

def two_sum(nums, target):
    seen = {}
    for i, x in enumerate(nums):
        if target - x in seen:
            return [seen[target - x], i]
        seen[x] = i

def valid_parens(s):
    pairs = {")": "(", "]": "[", "}": "{"}
    st = []
    for ch in s:
        if ch in "([{": st.append(ch)
        elif not st or st.pop() != pairs[ch]: return False
    return not st

def top_k(words, k):
    c = Counter(words)
    return [w for w, _ in sorted(c.items(), key=lambda kv: (-kv[1], kv[0]))[:k]]

def merge_intervals(iv):
    out = []
    for s, e in sorted(iv):
        if out and s <= out[-1][1]: out[-1][1] = max(out[-1][1], e)
        else: out.append([s, e])
    return out

def bfs_shortest(grid, start, goal):
    q = deque([(start, 0)]); seen = {start}
    while q:
        (r, c), d = q.popleft()
        if (r, c) == goal: return d
        for dr, dc in ((1,0),(-1,0),(0,1),(0,-1)):
            nr, nc = r+dr, c+dc
            if 0 <= nr < len(grid) and 0 <= nc < len(grid[0]) and grid[nr][nc] == 0 and (nr,nc) not in seen:
                seen.add((nr,nc)); q.append(((nr,nc), d+1))
    return -1

def anagram_groups(ws):
    g = {}
    for w in ws: g.setdefault("".join(sorted(w)), []).append(w)
    return list(g.values())

print(two_sum([2, 7, 11, 15], 9))
print(valid_parens("{[()]}"), valid_parens("([)]"))
print(top_k("a b a c b a d".split(), 2))
print(merge_intervals([[1,3],[8,10],[2,6],[15,18],[17,20]]))
grid = [[0,0,0],[1,1,0],[0,0,0]]
print(bfs_shortest(grid, (0,0), (2,0)))
print(anagram_groups(["eat","tea","tan","ate","nat","bat"]))
print(heapq.nlargest(3, [5, 1, 9, 3, 7]))
```

```
[0, 1]
True False
['a', 'b']
[[1, 6], [8, 10], [15, 20]]
6
[['eat', 'tea', 'ate'], ['tan', 'nat'], ['bat']]
[9, 7, 5]
```

- **Two sum** 2 + 7 = 9, at indexes 0 and 1. The dictionary lets us find the partner in one step per item, so time is O(n) instead of O(n squared) for two nested loops.
- **Valid parentheses** `{[()]}`: each closer matches the most recent opener (stack, last in first out), so it is valid. In `([)]` the `)` meets `[` on top of the stack, so it is invalid.
- **Top k** Counts are a=3, b=2, c=1, d=1. The top 2 are `a` and `b`.
- **Merge intervals** After sorting: [1,3], [2,6], [8,10], [15,18], [17,20]. [1,3] and [2,6] overlap, so they merge into [1,6]. [8,10] stands alone. [15,18] and [17,20] merge into [15,20].
- **BFS in a grid** From (0,0) to (2,0), the middle row is blocked except at column 2. The path is (0,0), (0,1), (0,2), (1,2), (2,2), (2,1), (2,0): 6 steps. BFS visits cells in order of distance, so the first time we reach the goal it is the shortest.
- **Anagram groups** Sorting letters gives the same key for words with the same letters.
- **Answer** All outputs match the hand checks.
- **Check** Each result above was verified by hand in the bullets.
- **Why it matters** These six patterns (hash map, stack, counting, sort and sweep, BFS, grouping by key) cover a large share of interview questions.

Typical wrong version of valid parentheses: forgetting `not st`. For input `")"` the code would call `st.pop()` on an empty list and fail with `IndexError: pop from empty list`. The fix is the `not st or ...` guard.

Real-world mapping

Pattern	Example	Where it appears in the job
Hash map	two-sum, counting	Joins and group-bys in code form
Stack	parentheses	Parsing, undo, expression evaluation
Sort and merge	intervals	Merging overlapping time windows in logs
BFS	shortest path	Dependency graphs, lineage
Heap	top-k	Top slow requests, leaderboards

Applications

- **Data Analytics:** counting and grouping logic behind quick scripts.
- **Data Engineering:** interval merging for sessionisation and gap detection.
- **Machine Learning:** nearest neighbours and top-k retrieval use heaps and hash maps.
- **Interviews at all levels:** the screening round is usually one or two of these patterns.

COMMON MISTAKE

Reading solutions before trying. You learn by struggling for 20 to 30 minutes first. Then read, then re-solve from a blank page the next day.

Check yourself

1. Why is the two-sum dictionary approach fast? Answer: Looking up the partner in a dict is on average constant time, so one pass is enough.
2. Which structure gives "last in, first out"? Answer: A stack (a Python list with `append` and `pop`).
3. Why does BFS find the shortest path in an unweighted grid? Answer: It explores cells in order of increasing distance.

14.6 The Python Interview Bank (45 Graded Questions)

INTUITION

An interview is a conversation where you show how you think. The best answers are short, correct, and come with a tiny example. Practise saying them aloud in simple English.

How to use it

Cover the answer, say your answer aloud, then compare. Levels: **Easy** (you must know these cold), **Medium** (expected for a junior to mid role), **Hard** (separates strong candidates). Some answers include code that was run; the outputs are real.

Easy (questions 1 to 15)

Q1. What is the difference between a list and a tuple? **Answer:** A list is mutable (you can change it) and a tuple is immutable. Tuples can be dictionary keys if all their items are hashable, and they signal "this should not change". Careful: a tuple can hold a mutable item; `t = (1, [2]); t[1].append(3)` makes `t` become `(1, [2, 3])` (I ran this).

Q2. What is the difference between `is` and `==`? **Answer:** `==` asks "are the values equal?" and `is` asks "is it the same object in memory?". Use `is` for `None` (`x is None`). Do not use `is` to compare numbers or strings.

Q3. What is the output of `0.1 + 0.2 == 0.3`? **Answer:** `False`. Floats are stored in binary and 0.1 is not exact. Compare with `math.isclose` or round. (I ran it: `round(0.1+0.2, 10) == 0.3` is `True`.)

Q4. Name Python's main built-in collection types. **Answer:** `list`, `tuple`, `set`, `frozenset`, `dict`; and `str` and `bytes` are sequences. `collections` adds `deque`, `Counter`, `defaultdict`.

Q5. Is a `dict` ordered? **Answer:** Yes. Since Python 3.7 a dict keeps insertion order as part of the language.

Q6. What is the mutable default argument problem? **Answer:** Default values are created once, when the function is defined. So `def f(x, acc=[])` shares one list across calls. I ran it: `print(f(1), f(2))` printed `[1, 2] [1, 2]`, both names point at the same list. Fix: `acc=None` and then `if acc is None: acc = []`.

Q7. What does `*args` and `**kwargs` mean? **Answer:** `*args` collects extra positional arguments into a tuple. `**kwargs` collects extra keyword arguments into a dict. They also unpack when calling: `f(*lst, **d)`.

Q8. What is a list comprehension, and give an example. **Answer:** A short way to build a list: `[x*x for x in range(5) if x % 2 == 0]` is `[0, 4, 16]`. Dict, set and generator versions exist too.

Q9. What is the difference between a list and a generator expression? **Answer:** A list holds all items in memory now. A generator makes them one at a time when asked (lazy). `(x*x for x in range(10**6))` uses almost no memory. In a run, `next(gen)` gave `0` and then `1`.

Q10. What does `with open(...) as f:` do? **Answer:** It is a context manager: the file is closed automatically, even if an error happens inside the block.

Q11. What is the difference between `append` and `extend`? **Answer:** `append(x)` adds one item (a list stays one item). `extend(iterable)` adds each element. `[1].append([2])` gives `[1, [2]]`; `[1].extend([2])` gives `[1, 2]`.

Q12. What is `__name__ == "__main__"` for? **Answer:** The code under it runs only when the file is run directly, not when it is imported. It makes a file both a script and a module.

Q13. What happens when you slice beyond the end, like `x[5:]` for a 3-item list? **Answer:** Slices are forgiving and return an empty list: `[]`. Indexing a single item beyond the end raises `IndexError`. (I ran `x[5:]`, it printed `[]`.)

Q14. What is the difference between `7 // 2`, `-7 // 2` and `7 % -3`? **Answer:** `//` is floor division, so they are `3`, `-4` (floor of -3.5) and `-2` (the sign follows the divisor). I ran all three.

Q15. How do you read a traceback? **Answer:** Read from the bottom: the last line says the error type and message. The lines above show the chain of calls, with the last call closest to the bottom. Find the lowest line that is in your code. That is usually where to look first.

Medium (questions 16 to 30)

Q16. Explain shallow copy and deep copy. **Answer:** A shallow copy makes a new outer container but shares the inner objects. A deep copy copies everything recursively. I ran: after `a = [[1,2],[3]]`, `s = copy.copy(a)`, `d = copy.deepcopy(a)` and `a[0].append(99)`, the shallow copy showed `[[1, 2, 99], [3]]` and the deep copy `[[1, 2], [3]]`.

Q17. What is a closure, and what is the late-binding trap? **Answer:** A closure is an inner function that remembers variables from the outer function. Trap: `fs = [lambda: i for i in range(3)]` gives `[2, 2, 2]` when called, because all lambdas look up `i` at call time and the loop ended with

`i = 2`. Fix: bind it with a default: `lambda i=i: i` gives `[0, 1, 2]` (I ran both).

Q18. What is a decorator? Write one that keeps the function's name. **Answer:** A decorator is a function that takes a function and returns a new function, usually adding behaviour (timing, logging, caching, retry). Use `functools.wraps` so the name and docstring survive. I ran a timing decorator: `add.__name__` printed `add` and `add.__doc__` printed `adds`. Without `@functools.wraps`, the name would show `w`.

Q19. Explain generators and `yield`. **Answer:** A function with `yield` returns a generator: it pauses at each `yield`, keeps its state, and resumes on the next `next()`. It is lazy, so large or infinite streams use little memory. Pipelines chain generators. I ran a three-stage pipeline: `list(gen_pipeline(["a ", "", " b"]))` gave `['A', 'B']`: strip, drop blanks, uppercase, one item at a time.

Q20. What is the difference between an iterator and an iterable? **Answer:** An iterable can give you an iterator (it has `__iter__`, like a list). An iterator has `__next__` and remembers its position. `iter(x)` converts the first into the second. A `for` loop does this for you.

Q21. What is the MRO and how does `super()` work with multiple inheritance? **Answer:** MRO (Method Resolution Order) is the order Python searches classes for a method, computed by the C3 rule so that a class comes before its parents and the order of bases is respected. `super()` means "the next class in the MRO", not "the parent". In my run of a diamond (D inherits B and C, both inherit A) `D().who()` printed `D>B>C>A` and the MRO was `['D', 'B', 'C', 'A', 'object']`, so `B`'s `super()` call reached `C`, a sibling.

Q22. What is `lru_cache` and when do you use it? **Answer:** A decorator from `functools` that remembers results of a function by its arguments (memoisation). Use it for pure functions that are costly and called with repeated arguments. I ran `fib(50)`: it printed `12586269025` and `CacheInfo(hits=48, misses=51, ...)` meaning only 51 real computations. Arguments must be hashable.

Q23. How do exceptions chain, and what does `raise ... from` do? **Answer:** `raise NewError(...) from e` links the original cause, which Python shows in the traceback. I ran: converting `ValueError` into `RuntimeError('bad config')` kept `__cause__` as the `ValueError` with its message. It keeps the root cause while giving callers a meaningful error type. Use `logging.exception` inside `except` to log with the traceback.

Q24. What is EAFP versus LBYL? **Answer:** EAFP: "Easier to Ask Forgiveness than Permission" (`try: d[k] except KeyError`). LBYL: "Look Before You Leap" (`if k in d:`). Python style prefers EAFP in many cases, and EAFP also avoids race conditions (the file may vanish between the check and the use).

Q25. How does a context manager work? Write one. **Answer:** `with` calls `__enter__` at the start and `__exit__` at the end, even after an error. If `__exit__` returns true, the exception is suppressed. I ran a class that printed `enter`, then `exit ZeroDivisionError`, and the program continued with `after`. You can also write one with `@contextlib.contextmanager` and a `yield`.

Q26. What is the difference between `@staticmethod`, `@classmethod` and a normal method? **Answer:** A normal method receives the instance (`self`). A `classmethod` receives the class (`cls`), useful for alternative constructors (`Settings.from_env()`). A `staticmethod` receives nothing special; it is a plain function placed in the class for organisation.

Q27. What do `__repr__` and `__str__` do? **Answer:** `__repr__` is for developers, unambiguous, ideally looking like code. `__str__` is for users. `print` uses `__str__` and falls back to `__repr__`. Always define `__repr__` (a `dataclass` does it for you).

Q28. When would you use a dataclass? **Answer:** For classes that mostly hold data. `@dataclass` generates `__init__`, `__repr__` and `__eq__`. Options: `frozen=True` for immutable (hashable) objects, `field(default_factory=list)` for mutable defaults, `__post_init__` for validation.

Q29. What is the difference between `pytest` fixtures and `setUp`? **Answer:** A fixture is a function that provides test data or objects, requested by name as a test parameter; it can have scope (function, module, session) and cleanup after `yield`. It is more flexible and reusable than `unittest`'s `setUp` / `tearDown` methods.

Q30. How do you mock an external call in a test? **Answer:** Replace it with a stand-in, with `unittest.mock.Mock` or `patch` (in pytest, also `monkeypatch`). In my run (Chapter 13), `Mock().send` recorded `call('order 1: 400')` and `assert_called_once_with` verified it. Prefer a fake when behaviour matters, a mock when you only need to check the call, and inject dependencies so mocking stays simple.

Hard (questions 31 to 45)

Q31. Explain the GIL and its consequences. **Answer:** The Global Interpreter Lock in standard CPython lets only one thread run Python bytecode at a time. Consequences: threads do not speed up CPU-bound pure-Python code, but they do help I/O-bound work because the GIL is released while waiting on I/O. For CPU-bound work use `multiprocessing` (separate processes), native code (NumPy and similar release the GIL), or vectorisation. Recent Python versions have an optional free-threaded build, so check what your Python version supports.

Q32. Threads, processes or asyncio: how do you choose? **Answer:** Many network waits and moderate count: `asyncio` (one thread, cooperative tasks, scales to thousands) or threads with `concurrent.futures.ThreadPoolExecutor` if libraries are blocking. CPU-bound: processes (`ProcessPoolExecutor`). Always measure first. Costs: processes need data to be pickled and use more memory; asyncio needs async-aware libraries; threads need locks for shared state.

Q33. What is a race condition and how do you prevent one in Python? **Answer:** Two threads change shared data and the result depends on timing. Even `counter += 1` is not atomic (read, add, write). Prevent with a `threading.Lock`, a `queue.Queue` to pass work, or by avoiding shared state (use processes or immutable data).

Q34. How does Python manage memory? **Answer:** Objects are allocated on a heap and tracked by reference counts. When the count reaches zero, the object is freed at once. A cyclic garbage collector finds objects that reference each other in a cycle. `__slots__` removes the per-instance `__dict__` and saves memory for many small objects. `sys.getsizeof` shows the size of one object only, not of what it points to, and the numbers differ between versions.

Q35. What are `*` and `**` unpacking, and keyword-only and positional-only parameters? **Answer:** In `def f(a, b, /, c, *, d)`: `a` and `b` are positional-only (before `/`), `c` can be either, and `d` is keyword-only (after `*`). Keyword-only arguments make calls readable: `f(1, 2, 3, d=4)`.

Q36. What is structural pattern matching? **Answer:** `match` / `case` (Python 3.10 and later) compares a value to patterns, including shapes and unpacking: `case {"type": "click", "x": x}:` or `case [first, *rest]:`. It is more than a switch; it can destructure lists, dicts and classes. Use it for parsing structured data like events.

Q37. Explain SOLID with one line each and one Python example. **Answer:** S: a class has one reason to change (separate parsing from reporting). O: extend by adding code, not editing (new strategy in a dict of strategies). L: subclasses must work where the parent works (a `Square` that breaks a `Rectangle`'s contract violates it). I: small interfaces (a `Protocol` with only `read()`, not a giant base class). D: depend on abstractions (inject a repository, not a concrete database), as in Chapter 13.

Q38. Describe three design patterns you have used in Python. **Answer:** Strategy: pass a function or choose from a dict, such as `{"max": max, "p90": ...}` (I ran this and got `{'max': 300, 'min': 80, 'p90': 300}`). Factory: a function that returns the right object by name. Singleton: usually a module-level object or `lru_cache`d function (I ran `get_config()` and it printed "loading config once", and `get_config() is get_config()` was `True`). Also Observer (event bus) and Repository (Chapter 13).

Q39. Composition versus inheritance: which and why? **Answer:** Prefer composition ("has a") over inheritance ("is a") unless there is a true "is a" relationship. Inheritance couples child to parent, and deep hierarchies are hard to change. Composition lets you swap parts (a `Pipeline` that has a `Reader` and a `Writer`). Mixins and `Protocol`s are the middle path.

Q40. What is the difference between `abc.ABC` and `typing.Protocol`? **Answer:** An ABC requires explicit inheritance and can force subclasses to implement abstract methods (it fails at object creation if they do not). A `Protocol` is structural ("duck typing" checked by type checkers): any class with the right methods fits, with no inheritance needed. Use ABCs when you want shared code and enforcement, Protocols for flexible interfaces.

Q41. How do you make a Python program faster? Walk through your process. **Answer:** (1) Measure first: `timeit` for small pieces, `cProfile` for the whole program, and `line_profiler` for hot lines. (2) Fix the algorithm and data structure (a `set` or `dict` lookup instead of a list scan). (3) Use built-ins and vectorised libraries (NumPy, pandas) rather than Python loops. (4) Reduce I/O and cache repeated results. (5) Only then consider concurrency, Numba or Cython. This is the same loop you use in performance testing: measure, find the bottleneck, change one thing, measure again.

Q42. What is the N+1 problem and how does it relate to APIs and databases? **Answer:** You make one query for a list and then one more query per item (1 + N calls). It is slow because each call has a round-trip cost. Fix: fetch in one query (a join or `IN` list), or batch API calls. A request log shows it as a huge request count per user action.

Q43. Explain how you would design pagination, idempotency and retries for an API client that loads data every night. **Answer:** Page with limit and a cursor (or offset if the data is static) until the next marker is empty. Always set timeouts, retry only temporary errors (429, 503, connection errors) with exponential backoff and respect `Retry-After`. Make loads idempotent (use upserts or a unique key, and an idempotency key for POSTs), so a rerun after a crash does not duplicate data. Log counts per page and verify the total at the end.

Q44. A teammate pushed an API key to GitHub. What do you do? **Answer:** (1) Revoke and rotate the key now, because history and clones keep it. (2) Check logs for misuse. (3) Remove it from the repo, and clean the history if policy requires. (4) Move secrets to environment variables or a secret manager. (5) Add a scanner (gitleaks) to pre-commit and CI. (6) Hold a blameless review about how to prevent it.

Q45. How do you test code that talks to a database or the network? **Answer:** Follow the testing pyramid: many fast unit tests (pure functions, with fakes or mocks for the edges), fewer integration tests (a real database in a container or in-memory SQLite, a local fake server), and very few end-to-

end tests. Inject dependencies so you can substitute them, use fixtures for setup and cleanup, use `parametrize` for many inputs, and run it all in CI. Measure coverage but do not worship the number.

The picture

```
Easy (15)    ->  vocabulary and traps        -> must be instant
Medium (15)  ->  how Python works inside     -> explain with a small example
Hard (15)    ->  design, trade-offs, process -> show your judgement
```

Worked example

Problem 14.6.1 Practise answering "Explain the GIL in 60 seconds" with structure. Use the pattern: define it, say the consequence, give the fix, give one number or example.

Step 1 (define)	"The GIL is a lock in CPython that lets one thread run Python code at a time."
Step 2 (consequence)	"So threads do not make pure-Python number crunching faster, but they do help when the program waits on the network or disk, because the lock is released while waiting."
Step 3 (fix)	"For CPU-bound work I use processes, or vectorised libraries like NumPy that do the work outside Python."
Step 4 (judgement)	"I measure first, because for most data jobs the bottleneck is I/O or the database, not the GIL."
Answer	The four sentences take about 30 seconds aloud and cover definition, consequence, remedy and judgement. Interviewers listen for judgement most.
Check	Time yourself with a phone. If you need more than 90 seconds, cut examples. If under 20, add the consequence.
Why it matters	The structure works for almost any "explain X" question: define, consequence, fix, judgement.

Real-world mapping

Question type	Example	What the interviewer checks
Trap	mutable default	Have you been bitten before?
Internals	MRO, GIL	Do you understand, not just use?
Design	SOLID, patterns	Can you build things others maintain?
Process	speed-up plan, leaked key	Will you act calmly and correctly?

Applications

- **Data Analytics roles:** easy and medium questions plus pandas and SQL.
- **Data Engineering roles:** medium and hard questions on idempotency, concurrency, testing.
- **Machine Learning and MLOps roles:** design, packaging, CI/CD, testing of data and models.
- **GenAI roles:** async calls, API clients, retries, secrets, evaluation code.

COMMON MISTAKE Memorising answers word for word. Interviewers ask follow-ups ("why?") and you freeze. Learn the idea, keep one tiny code example, and say it in your own simple words.

Check yourself

1. In `def f(a, b, /, c, *, d)`, which parameters are keyword-only? Answer: Only `d`.

2. Does `super()` mean "the parent class"? Answer: No, it means the next class in the MRO.
3. What is the first step in speeding up code? Answer: Measure it (profile) to find the real bottleneck.

14.7 Three More Interview Rounds, Worked in Full

The bank in 14.6 gives you many short questions. Real interviews also test how you think aloud. This section walks three rounds the way a strong candidate would: restate, predict, code, test, then name the trade-off.

Round 1: closures and the late-binding trap

Problem 14.7.1 "Write a counter factory, then tell me what `[lambda: i for i in range(3)]` returns when you call each function."

Restate	A closure is a function that remembers variables from the function that created it.
Counter	Each call to `make_counter` creates its own `count`. `nonlocal` lets the inner function rebind it.
Predict lambdas	All three lambdas look up `i` when they are *called*, not when created. By then the loop has finished and `i` is 2, so `[2, 2, 2]`.
Fix	A default argument is evaluated at creation: `lambda i=i: i` gives `[0, 1, 2]`.
Check	Counters a and b are independent: 1, 2, then 1.

```
def make_counter():
    count = 0
    def inc():
        nonlocal count
        count += 1
        return count
    return inc

a, b = make_counter(), make_counter()
print(a(), a(), b())
fns = [lambda: i for i in range(3)]
print([f() for f in fns])
fixed = [lambda i=i: i for i in range(3)]
print([f() for f in fixed])
```

```
1 2 1
[2, 2, 2]
[0, 1, 2]
```

COMMON MISTAKE

Answering `[0, 1, 2]` for the first lambda list. Wrong reasoning: "each lambda captured the current i". Right: closures capture the *variable*, not its value at that moment.

Round 2: design an LRU cache

Problem 14.7.2 "Implement a cache with a fixed capacity that discards the least recently used item. `get` and `put` should be O(1)."

Clarify	What counts as "use"? Both reading and writing. What does a miss return? A default.
Idea	A hash map gives O(1) lookup. We also need order of use. `OrderedDict` is a dict that remembers order and can move a key to the end in O(1).
get	If present, move the key to the end (most recent) and return the value.
put	Store, move to end, and if over capacity pop from the front (least recent).
Test	Capacity 2: put a, put b, get a, put c. Then b was least recent and is evicted, leaving a, c.

```
from collections import OrderedDict

class LRUCache:
    """Least-recently-used cache with O(1) get and put."""
    def __init__(self, capacity):
        self.capacity = capacity
        self._data = OrderedDict()
    def get(self, key, default=None):
        if key not in self._data:
            return default
        self._data.move_to_end(key)
        return self._data[key]
    def put(self, key, value):
        self._data[key] = value
        self._data.move_to_end(key)
        if len(self._data) > self.capacity:
            self._data.popitem(last=False)
    def keys(self):
        return list(self._data)

c = LRUCache(2)
c.put("a", 1); c.put("b", 2)
c.get("a")
c.put("c", 3)
print(c.keys(), c.get("b"))
c.put("a", 10)
print(c.keys(), c.get("a"))
assert LRUCache(1).get("x", "none") == "none"
```

```
['a', 'c'] None
['c', 'a'] 10
```

Follow-up an interviewer will ask: "What if you could not use `OrderedDict`?" Answer: a dict from key to node plus a doubly linked list, which is how `OrderedDict` works inside. And "How would this change with threads?" Answer: guard `get` and `put` with a `threading.Lock`, because the move-then-evict sequence is not atomic. For simple function caching, `functools.lru_cache` already does this.

Round 3: the mutable default argument

Problem 14.7.3 "What does this print, and why? Fix it."

Predict	A default value is created once, when `def` runs, and shared by all calls that omit it. The second call appends to the same list: `[1, 2] [1, 2]` (both names point at one list, so the first result also shows 2).
Fix	Use `None` as the default and create a fresh list inside.

```
def append_item(x, bucket=[]):
    bucket.append(x)
    return bucket
print(append_item(1), append_item(2))

def append_safe(x, bucket=None):
    if bucket is None:
        bucket = []
    bucket.append(x)
    return bucket
print(append_safe(1), append_safe(2))
```

```
[1, 2] [1, 2]
[1] [2]
```

Problem 14.P.12 (interview) Your service is slow only after it has run for days, and memory keeps growing. Give three causes in Python and how you would confirm each.

Answer: (1) An unbounded cache or module-level list: check its size, add a size limit or `lru_cache(maxsize=...)`. (2) Reference cycles holding large objects or unclosed resources: use `tracemalloc` snapshots to compare allocations over time. (3) Mutable default arguments or global registries that grow per request: search for them and log their length. Confirm first with a graph of memory over time, then fix one cause and re-measure.

Problem 14.P.13 (stretch) Add a `delete(key)` method to `LRUCache` that returns `True` if the key existed.

Answer: `def delete(self, key): return self._data.pop(key, _MISSING) is not _MISSING` where `_MISSING = object()`; or simply check `if key in self._data: del self._data[key]; return True` then `return False`. Test: after `delete("a")`, `keys()` no longer lists it and a second delete returns `False`.

Chapter recap

- Four capstones prove your skills: a typed and tested CLI, a FastAPI service with a database, a queue and service-time analyser, and a responsible scraper.
- Keep logic in pure functions and put input/output at the edges. That makes code testable.
- Little's law (L = lambda x W) is a quick sanity check on any queue, from a kitchen to a server. Watch the units.
- A polite scraper checks robots.txt, identifies itself, waits, caches, and stores only what it needs.
- The 150-problem plan is about 2 problems a day for 11 weeks, with a log and spaced repetition.
- Six patterns (hash map, stack, counting, sort and sweep, BFS, grouping) solve a large part of screening questions.
- In interviews: define, give the consequence, show the fix, and show judgement.
- Easy questions must be instant, medium need a small example, hard need trade-offs.

Cheat sheet

Need	Pattern
Nearest-rank percentile	`xs[max(0, ceil(n*p/100) - 1)]` after sorting
Throughput	`n / ((last_ts - first_ts) / 1000)`
Little's law	`L = throughput * avg_seconds`
Two-sum	dict `seen[value] = index`
Valid parentheses	stack plus `pairs` dict
Merge intervals	sort, then extend last if `start <= last_end`

Need	Pattern
Shortest path (unweighted)	BFS with a `deque` and a `seen` set
Explain-X answer	define, consequence, fix, judgement

Practice set

Problem 14.P.1 (easy) For the 8 orders in 14.1.1, what is the p90 of `salad` and why?

Answer: `salad` has 3 orders (95, 120, 180 sorted). ceil(3 x 0.9) = 3, index 2, value 180.

Problem 14.P.2 (easy) What does `merge_intervals([[1,4],[4,5]])` return?

Answer: `[[1, 5]]`. The intervals touch at 4 (`s <= last_end`), so they merge.

Problem 14.P.3 (easy) What is the output? `print(valid_parens("(()"))`

Answer: `False`. After reading all characters one `(` is still on the stack, so `not st` is false.

Problem 14.P.4 (medium) A cafe is open for 60 minutes and serves 300 orders. The average order spends 4 minutes from placing to serving. How many orders are in progress on average, by Little's law?

Answer: Arrival rate = 300 / 60 = 5 per minute. W = 4 minutes. L = 5 x 4 = 20 orders in progress on average.

Problem 14.P.5 (medium) Find the bug: `def add_order(name, orders=[]): orders.append(name); return orders`.

Answer: The default list is shared between calls. Use `orders=None` and create a new list inside when it is `None`.

Problem 14.P.6 (medium) In Capstone 4, your bot receives `Crawl-delay: 5` and you want 120 pages, none cached. Minimum time?

Answer: 120 x 5 = 600 seconds, which is 10 minutes (if you fetch one page at a time).

Problem 14.P.7 (medium) `top_k("x y x z y x".split(), 2)` with the function from 14.5.1. What is returned?

Answer: Counts are x=3, y=2, z=1, so `['x', 'y']`.

Problem 14.P.8 (medium) Your analyser says p95 delivery time of the pasta orders is 23 minutes and the promise is 15 minutes, but the average is 11 minutes. Which do you report and why?

Answer: Report the p95 as the SLA breach, along with the average and the failure rate. The average hides the slow tail that real customers feel.

Problem 14.P.9 (hard, interview) A BFS on a grid returns -1 for a start and goal that you think are connected. List three things to check.

Answer: (1) Bounds checking and the direction list (typo in the four moves). (2) The meaning of 0 and 1: is 0 free and 1 a wall, and is the goal cell itself blocked? (3) Whether `seen` is updated when you push to the queue (not only when you pop); and whether rows and columns are mixed up for non-square grids.

Problem 14.P.10 (hard, interview) You must process a 50 GB log file in Python on a laptop with 16 GB RAM and produce counts per URL. How?

Answer: Stream it: read line by line with a generator, update a `Counter` (memory grows only with the number of distinct URLs), or process in chunks, or push into SQLite or DuckDB. Never `f.read()` the whole file. If distinct URLs are huge, aggregate in chunks and merge results on disk. If it needs speed, split the file and use processes.

Problem 14.P.11 (hard, scenario) Your FastAPI service returns correct data but p95 response time rises sharply at 200 simultaneous users, while CPU stays at 30%. List what you check.

Answer: Low CPU with high latency means waiting. Check database connection pool size and slow queries (N+1, missing index), blocking calls inside `async def` handlers (they stall the event loop), downstream API timeouts, thread pool limits, and lock contention. Profile and trace one slow request, change one thing, and measure again.

Prompt pack

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional at an Indian company moving into data and AI roles, and English is my second language, so use simple words. Help me plan the capstone "command-line tool that summarises a CSV of shop or kitchen orders, with argparse, logging, pytest tests and type hints". Break it into 8 one-hour milestones, tell me the exact definition of done for each, and then guide me through milestone 1 only. Ask me to paste my code afterwards so you can review it kindly and specifically.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data and AI roles, and English is my second language. Quiz me one question at a time from a Python interview bank of 45 questions graded easy, medium and hard, covering types, mutability, closures, decorators, generators, MRO, GIL, asyncio, testing and design. Start at easy. After each answer give a score out of 5, a model answer in simple English, and one follow-up question like a real interviewer would ask.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional learning Python for data jobs, and English is my second language. Give me a 4-week plan to solve 60 practice problems on arrays, strings, hash maps, stacks, intervals and BFS, one hour a day. For week 1, give 12 graded problems without solutions, and for each one tell me the pattern name to learn first. When I paste my solution, review it, tell me the time and space cost, and show a cleaner version.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data roles, and English is my second language. Review my capstone project: a FastAPI service with a SQLite or PostgreSQL database and pytest tests. Check layering (routes, service, repository), parameterised SQL, status codes (404, 422, 201 with Location), secrets and configuration, pagination, and the test cases for failure paths. List the five most important improvements in order, with short code fixes. My code: [PASTE HERE]
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional preparing for data engineering interviews, and English is my second language. Run a 30-minute mock interview that mixes a data-analysis case on order or queue times (percentiles, arrival rate, Little's law, failure rate), one live Python coding problem, and one design question about an API client that loads data every night with retries and idempotency. Ask one question at a time, grade me out of 5, give a model answer, and coach me on how to explain it clearly and calmly in English.
```

CHAPTER 15

PROJECT LAB: FOUR PORTFOLIO PROJECTS, BUILT AND TESTED STEP BY STEP

How to plan, build, test and present small Python projects that an employer can open, run and trust.

A certificate says you studied. A repository says you can build. When a hiring manager opens your GitHub page, they spend about two minutes there: they read the README, look at the folder layout, run one command, and glance at the tests. This chapter gives you four projects that survive that two-minute check, and a method for running any project well. By the end you will be able to plan a project with measurable success criteria, lay out a repository, write tests before you trust your code, and explain your work in a README, a short post and an interview. Chapter 14 gave four capstones. This lab adds four more: the projects here use neutral, everyday data (spending records, shop orders, product pages) so that they show general Python skill that any employer in data, analytics or software recognises.

Words you need. **Repository (repo)** is a folder of code tracked by git. **Commit** is one saved snapshot with a message. **Package** is a folder of Python code that others can install and import. **Fixture** is fixed test data you control. **Definition of done** is the checklist that must be true before you call a project finished.

Everything marked "run" was executed with Python 3.14 using the standard library, plus `pandas` for Project 2. The tests use `unittest` (built into Python) because `pytest` was not installed on this study machine. Chapter 10 shows `pytest`, and every test below converts to it by removing the class. FastAPI, `requests`, `BeautifulSoup`, `pip install -e .` and GitHub features were **not run here**. Check each tool's current documentation, because commands change between versions.

15.1 How to Run a Project in This Area

INTUITION

Building a house without a plan wastes bricks. Building a project without a plan wastes weekends. A good plan has three parts: the question the project answers, the evidence that says "it works", and the small steps in between. Do the thinking first, and the typing becomes easy.

Step 1: Plan the question and the success criteria

Write three lines before you open an editor:

1. **Who is the user?** ("Me, tracking monthly spending." "An analyst who receives messy order files.")
2. **What question does it answer or what job does it do?** One sentence.
3. **How will I know it works?** Numbers, not feelings. "Summary totals match a hand calculation on 6 sample rows." "All 5 tests pass." "A new user installs and runs it from the README in under 5 minutes."

Then list what is **out of scope**. A project that tries to do everything is never finished. Two weeks of one hour a day is a good size for each project below.

Step 2: Repository layout

Use the same skeleton for every project, so reviewers know where to look.

```
project-name/
  README.md            <- what, why, how to run (see template below)
  pyproject.toml       <- name, version, dependencies (packages only)
  .gitignore           <- .venv/, __pycache__/, *.db, data/raw/
  src/ or package/     <- your code, split into small modules
    core.py            <- logic with no printing, no argparse
    cli.py             <- thin layer that talks to the user
  tests/
    __init__.py
    test_core.py       <- fast tests with small fixed data
  data/
    sample/            <- tiny files you commit
    raw/               <- big files you do NOT commit
  docs/                <- screenshots, design notes
```

The most important habit is to **keep logic separate from input and output**. The `core` module takes values and returns values. The `cli` module reads arguments and prints. Then you can test the logic without a terminal, a network or a database server.

Step 3: Environment and dependencies

- Create one virtual environment per project (Chapter 9): `python3 -m venv .venv`, then activate it.
- Prefer the standard library where it does the job. Every extra dependency is something a stranger must install and something that can break.
- Record dependencies in `pyproject.toml` (or `requirements.txt`), with a minimum version, for example `pandas>=2.0`.
- Never commit secrets. Keys go in environment variables (Chapter 13).

Step 4: Data sourcing or generation

Option	Example	Careful about
Generate it	A script that writes 200 fake orders with a fixed random seed	Seed it so results repeat
Public dataset	Government open data, Kaggle files, school exam marks	Read the licence; credit the source
Your own records	Your spending list	Remove names, phone numbers, account numbers before publishing
Scraped	Pages you are allowed to fetch	robots.txt, terms of use, rate limits (Project 4)

Always commit a **tiny sample** (5 to 20 rows) so that tests and the README demo work offline.

Step 5: Commit rhythm and good commit messages

Commit when one small thing works, not at the end of the week. A healthy day has 2 to 5 commits.

- Good: `Add validate() with date and amount checks`, `Fix: reject negative amounts (test added)`, `Docs: add install steps to README`.
- Bad: `update`, `final`, `changes`, `asdf`.
- Pattern: start with a verb, keep the first line under about 60 characters, say what and why, not how.
- Never commit a failing test suite to the main branch. Use a branch for risky work.

Step 6: Definition of done

A project is done when all of these are true:

- [ ] The success criteria from Step 1 are met and written in the README with the real numbers you got.
- [ ] All tests pass from a fresh clone and fresh virtual environment.

- [ ] A new person can run the demo by copying commands from the README.
- [ ] Bad input produces a clear message and a non-zero exit code, not a traceback.
- [ ] No secrets, no giant files, no `__pycache__` in the repo.
- [ ] You can explain every file and every design choice out loud in two minutes.

Step 7: README template

```
# Project Name

One sentence: what it does and for whom.

## Why I built it
Two or three sentences on the problem.

## Quick start
```

git clone <url> && cd project-name python3 -m venv .venv && source .venv/bin/activate pip install -e . python -m package_name --help

```
## Example
Paste a real command and its real output.

## How it works
A small diagram or a list of the main modules.

## Tests
`python -m unittest discover -s tests -v`   (current result: N passed)

## Results / acceptance criteria
A table of the measurable checks and whether they pass.

## Limits and next steps
What it does not do yet. Honest limits build trust.

## Licence and data sources
```

Step 8: Demo and deploy options

Project type	Free, simple demo
Command-line tool	A short screen recording or the real output pasted in the README
Package	Publish a tagged release on GitHub; install with `pip install git+<url>` (not run here)
API	Run locally and paste `curl` commands; optional free hosting tiers change often, so read current terms
Scraper / dataset	Publish the cleaned sample CSV and the code, not the raw pages

Step 9: Writing the LinkedIn or blog post

Use this five-part shape, about 150 to 250 words:

1. **Hook:** one sentence about the problem ("My spending notes were scattered over three apps.").
2. **What I built:** one sentence plus the link.
3. **One thing that surprised me:** a real bug and how you found it. This is the part people remember.
4. **One number:** "6 tests, 3 commands, runs offline."
5. **What next:** one honest limit and one plan.

No hype words. No claim you cannot show in the repo.

Step 10: Testing and debugging checklist for Python projects

- **Test the three kinds of input:** normal, empty, and wrong (bad date, negative amount, missing column).
- **Test the edges:** one row, zero rows, duplicate rows, a very long text.
- **Use fixed data and a temporary folder** (`tempfile.TemporaryDirectory`) so tests never touch real files.
- **Check exit codes** for command-line tools: 0 is success, non-zero is failure.
- **Debug in this order:** read the last line of the traceback first; reproduce with the smallest input; print or `logging` the values just before the failure; use `python3 -m pdb` or `breakpoint()` (Chapter 6) only after that.
- **Run the whole suite before every commit.**
- **Test from a clean clone** once before you share the link.

REAL JOB TIP	Reviewers rarely run your code on day one. They read the README, the folder tree and the test names. A test called `test_negative_amount_is_rejected` tells them more about your care than a hundred lines of clever code.

Check yourself

1. Why should `core.py` never call `print()` or `argparse` ? Answer: So the logic can be tested by simply calling functions and checking returned values, without capturing terminal output.
2. Name two things that must not be committed. Answer: Secrets (keys, passwords) and large raw data files or `.venv` folders.
3. Give one measurable success criterion for a data-cleaning tool. Answer: "On the sample file, the report shows 1 duplicate dropped, 1 bad date and 1 outlier, matching my hand count."

15.2 Project: A Command-Line Expense Tracker

Goal and why employers care

Build `expenses` , a command that records spending in a CSV file and prints totals per category. It looks small, but it shows the five things employers check first: a clean interface (`argparse`), validated input, file storage, separation of logic from output, and tests. Almost every internal tool in a company is a bigger cousin of this one.

Success criteria: `add` saves a valid row and returns exit code 0; invalid date or amount prints a clear message and returns 1; `summary --month 2025-03` matches a hand calculation; 6 tests pass.

Skills used

Chapter 2 (functions), Chapter 4 (`defaultdict` , sorting), Chapter 5 (files, CSV, dates), Chapter 6 (custom exceptions), Chapter 10 (testing), Chapter 12 (command-line tools with `argparse`).

Data

No download needed. You generate data by using the tool. For tests, small fixed rows are enough. Example data for the demo (shop-style spending): groceries 120.50 and 80.00, transport 45.00, rent 9000.00.

Architecture

```
expense-tracker/
  expense_tracker/
    __init__.py
    core.py        validate(), load(), append(), summarise()   (no printing)
    cli.py         build_parser(), main(argv) -> exit code
    __main__.py    lets you run: python3 -m expense_tracker
  tests/
    __init__.py
    test_core.py
  README.md
```

```
 user types:  add 2025-03-01 food 120.5
                 |
              cli.main(argv) --parse--> core.validate() --> core.append() --> expenses.csv
                 |  (bad input: ExpenseError)
                 +--> prints "error: ..." to stderr, returns 1
```

Step-by-step build

Step 1: The core, starting with validation. Raise your own error type so the CLI can tell "user mistake" from "program bug".

```
# expense_tracker/core.py (first part)
import csv
from collections import defaultdict
from datetime import date
from pathlib import Path

FIELDS = ["date", "category", "amount", "note"]

class ExpenseError(Exception):
    """A mistake the user can fix (bad date, bad amount)."""

def validate(day: str, category: str, amount: str, note: str = "") -> dict:
    try:
        date.fromisoformat(day)
    except ValueError:
        raise ExpenseError(f"bad date {day!r}, use YYYY-MM-DD") from None
    try:
        value = round(float(amount), 2)
    except ValueError:
        raise ExpenseError(f"bad amount {amount!r}") from None
    if value <= 0:
        raise ExpenseError("amount must be positive")
    if not category.strip():
        raise ExpenseError("category is empty")
    return {"date": day, "category": category.strip().lower(),
            "amount": value, "note": note}
```

Step 2: Storage and summary. Append rows, write the header only when the file is new, and add up per category.

```
def load(path: Path) -> list[dict]:
    if not path.exists():
        return []
    with path.open(newline="", encoding="utf-8") as f:
        rows = list(csv.DictReader(f))
    for r in rows:
        r["amount"] = float(r["amount"])
    return rows

def append(path: Path, row: dict) -> None:
    is_new = not path.exists()
    with path.open("a", newline="", encoding="utf-8") as f:
        w = csv.DictWriter(f, fieldnames=FIELDS)
        if is_new:
            w.writeheader()
        w.writerow(row)

def summarise(rows: list[dict], month: str | None = None) -> dict:
    totals: dict[str, float] = defaultdict(float)
    for r in rows:
        if month and not r["date"].startswith(month):
            continue
        totals[r["category"]] += r["amount"]
    return {k: round(v, 2) for k, v in sorted(totals.items())}
```

Step 3: The command-line layer. Sub-commands (`add`, `list`, `summary`) each get their own arguments. `main` takes `argv` so tests can call it directly, and it returns an exit code instead of calling `sys.exit` itself.

```
# expense_tracker/cli.py
import argparse, sys
from pathlib import Path
from . import core

def build_parser() -> argparse.ArgumentParser:
    p = argparse.ArgumentParser(prog="expenses", description="Track spending.")
    p.add_argument("--file", type=Path, default=Path("expenses.csv"))
    sub = p.add_subparsers(dest="command", required=True)
    add = sub.add_parser("add", help="record one expense")
    add.add_argument("date"); add.add_argument("category"); add.add_argument("amount")
    add.add_argument("--note", default="")
    lst = sub.add_parser("list", help="show all expenses")
    lst.add_argument("--category")
    summ = sub.add_parser("summary", help="totals per category")
    summ.add_argument("--month", help="YYYY-MM")
    return p

def main(argv=None) -> int:
    args = build_parser().parse_args(argv)
    try:
        if args.command == "add":
            row = core.validate(args.date, args.category, args.amount, args.note)
            core.append(args.file, row)
            print(f"saved: {row['date']} {row['category']} {row['amount']:.2f}")
        elif args.command == "list":
            for r in core.load(args.file):
                if args.category and r["category"] != args.category.lower():
                    continue
                print(f"{r['date']}  {r['category']:<10} {r['amount']:>8.2f}  {r['note']}")
        else:
            totals = core.summarise(core.load(args.file), args.month)
            for cat, total in totals.items():
                print(f"{cat:<10} {total:>8.2f}")
            print(f"{'TOTAL':<10} {sum(totals.values()):>8.2f}")
    except core.ExpenseError as e:
        print(f"error: {e}", file=sys.stderr)
        return 1
    return 0
```

`__main__.py` has two lines: `import sys` / `from .cli import main`, then `sys.exit(main())`.

Step 4: Run it. Before you run it, predict: what will the last two commands print?

```
python3 -m expense_tracker add 2025-03-01 Food 120.50 --note "groceries"
python3 -m expense_tracker add 2025-03-03 transport 45
python3 -m expense_tracker add 2025-03-09 food 80
python3 -m expense_tracker add 2025-04-02 rent 9000
python3 -m expense_tracker add 2025-13-02 food 5
python3 -m expense_tracker summary --month 2025-03
```

Real output:

```
saved: 2025-03-01 food 120.50
saved: 2025-03-03 transport 45.00
saved: 2025-03-09 food 80.00
saved: 2025-04-02 rent 9000.00
error: bad date '2025-13-02', use YYYY-MM-DD
food         200.50
transport     45.00
TOTAL        245.50
```

Why: month 13 does not exist, so `date.fromisoformat` raises `ValueError`, we convert it to `ExpenseError`, and `main` prints it and returns 1 (the shell shows `exit=1`). March has food 120.50 + 80.00 = 200.50 and transport 45.00, total 245.50. April's rent is excluded by the `startswith("2025-03")` filter. The CSV on disk is:

```
date,category,amount,note
2025-03-01,food,120.5,groceries
2025-03-03,transport,45.0,
2025-03-09,food,80.0,
2025-04-02,rent,9000.0,
```

Step 5: Tests. Put each kind of input in a test: good, bad, and the storage round trip, always in a temporary folder.

```
# tests/test_core.py (excerpt)
class ValidateTests(unittest.TestCase):
    def test_bad_date(self):
        with self.assertRaises(core.ExpenseError):
            core.validate("01/03/2025", "food", "5")

class StorageTests(unittest.TestCase):
    def test_round_trip_and_summary(self):
        with tempfile.TemporaryDirectory() as d:
            p = Path(d) / "e.csv"
            core.append(p, core.validate("2025-03-01", "food", "100"))
            core.append(p, core.validate("2025-03-02", "food", "50.25"))
            core.append(p, core.validate("2025-04-01", "travel", "300"))
            rows = core.load(p)
            self.assertEqual(core.summarise(rows), {"food": 150.25, "travel": 300.0})
            self.assertEqual(core.summarise(rows, "2025-03"), {"food": 150.25})
```

The full suite has 6 tests: good row, bad date, negative amount, round trip, missing file is empty, and the CLI exit codes (0, 0, 1). Real run:

```
$ python3 -m unittest discover -v
...
Ran 6 tests in 0.019s

OK
```

Your first run may instead print `Ran 0 tests` / `NO TESTS RAN`. We got exactly that when we ran `python3 -m unittest -v` from the project root, and then an `ImportError: Start directory is not importable` with `discover -s tests`, until we added an empty `tests/__init__.py` and used plain `python3 -m unittest discover`. That is Pitfall 4 below.

Evaluation / acceptance criteria

Check	Expected	How to verify
March total (hand calculation: 120.50 + 80 + 45)	245.50	`summary --month 2025-03`
Bad date	message on stderr, exit code 1	`echo $?` after the command
Bad amount `abc`	`error: bad amount 'abc'`	run it
Tests	6 pass, under 1 second	`python3 -m unittest discover`
Fresh clone works	README commands succeed	try in a new folder

Common pitfalls

1. **Float rounding.** `0.1 + 0.2` prints `0.30000000000000004`. We round when saving and again in `summarise` (`round(v, 2)`); the summary of 0.1 and 0.2 gives `{'food': 0.3}`. Detect with a test using decimals. For real money use integer paise or `decimal.Decimal`.
2. **Date formats.** `2025-3-1` is rejected (`Invalid isoformat string: '2025-3-1'`). Decide on one format and say so in the error message.
3. **Mixing printing and logic.** If `summarise` prints, you cannot test it. Detect by asking: can I call this function without a terminal?
4. **Tests not discovered.** Missing `tests/__init__.py` or wrong working folder gives `NO TESTS RAN`. Detect: the count of tests run is 0, always read it.
5. **Header written twice.** Appending with a header each time corrupts the CSV. The `is_new` check prevents it; a test that adds two rows and loads two rows catches it.

Three extensions

1. A `delete` sub-command and an `edit` command, using a row id.
2. A monthly budget per category, and a warning when you pass 80%.
3. Switch storage to `sqlite3` behind the same `core` functions, without changing the tests for `summarise`.

Interview talking points

Q1. Why did you separate `core` from `cli`? Because logic that does not print is easy to test and to reuse. I can call `core.summarise` from a script, a web page or a test. The CLI is a thin layer that I replace without touching the logic.

Q2. How do you handle bad input? I validate at the edge. `validate` raises one custom error. The CLI catches only that error, prints a short message and returns exit code 1. A real bug still shows a traceback, which is what I want.

Q3. What would you change for real money? I would not store floats. I would store integer paise, or use `Decimal`, because floats cannot represent 0.1 exactly. I would also add a file lock or a database so two processes cannot write at once.

README outline

Title and one-line purpose; quick start (3 commands); usage table of sub-commands; real output block from Step 4; storage format (the CSV header); how to run tests; known limits (single user, floats); next steps.

15.3 Project: A Reusable Data-Cleaning Package

Goal and why employers care

Every analyst cleans the same mess again and again: untidy column names, text with stray spaces, bad dates, repeated rows, empty cells, wild values. Instead of copying cells between notebooks, build a small **package**, `tidyshop`, with tested functions and one pipeline that also returns a **report of what it changed**. Employers care because reusable, tested cleaning code is what separates a notebook user from a data professional, and the report makes cleaning auditable.

Success criteria: on a 6-row sample the report says: 1 duplicate dropped, 1 bad date, 1 missing amount, 1 outlier flagged, 5 rows out; 8 tests pass; the package imports from a clean environment.

Skills used

Chapter 4 (collections), Chapter 3 (regular expressions for column names), Chapter 5 (dates), Chapter 9 (packages, `pyproject.toml`, type hints), Chapter 10 (testing). It also uses `pandas`, which the Data Science track teaches in depth. See also the learner's Math PDF for the median and quartile definitions behind the IQR rule.

Data

A generated table of shop orders with typical problems (spaces, mixed case, a bad date, a repeated order id, a missing amount, one wild amount). It is built right inside the test and the demo, so there is nothing to download.

Architecture

```
tidyshop/
  pyproject.toml
  README.md
  src/
    tidyshop/
      __init__.py      exports the public functions
      clean.py         one function per cleaning job + clean_orders()
  tests/
    test_clean.py      8 tests
  docs/
    usage.md           examples (the demo output below)
```

The `src/` layout means tests import the installed package, not the folder next to them, so packaging mistakes show up early.

```
 raw DataFrame
   -> standardise_columns  (" Order ID " -> order_id)
   -> strip_text           (" Pune" -> pune)
   -> parse_dates          (bad -> NaT)
   -> drop_duplicate_rows  (by order_id)
   -> fill_missing         (median)
   -> flag_outliers_iqr    (adds amount_outlier)
   -> (clean DataFrame, report dict)
```

Step-by-step build

Step 1: `pyproject.toml`. This tells tools the package name, version and what it needs.

```
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"

[project]
name = "tidyshop"
version = "0.1.0"
description = "Small, tested helpers for cleaning shop-order tables"
requires-python = ">=3.10"
dependencies = ["pandas>=2.0"]

[tool.setuptools.packages.find]
where = ["src"]
```

`pip install -e .` installs it in editable mode: **not run here**, because pip was blocked. We ran everything with `PYTHONPATH=src`, which gives Python the same import path.

Step 2: Small functions that each return a NEW table. Never change the caller's data.

```
# src/tidyshop/clean.py (excerpt)
import re
import pandas as pd

def standardise_columns(df: pd.DataFrame) -> pd.DataFrame:
    """' Order ID ' -> 'order_id'."""
    out = df.copy()
    out.columns = [re.sub(r"\W+", "_", str(c).strip().lower()).strip("_")
                   for c in out.columns]
    return out

def parse_dates(df: pd.DataFrame, column: str) -> pd.DataFrame:
    """Bad values become NaT, never an exception."""
    out = df.copy()
    out[column] = pd.to_datetime(out[column], errors="coerce", format="%Y-%m-%d")
    return out

def fill_missing(df: pd.DataFrame, column: str, how: str = "median") -> pd.DataFrame:
    if how not in {"median", "mean"}:
        raise ValueError(f"how must be 'median' or 'mean', got {how!r}")
    out = df.copy()
    out[column] = out[column].fillna(getattr(out[column], how)())
    return out

def flag_outliers_iqr(df: pd.DataFrame, column: str, k: float = 1.5) -> pd.DataFrame:
    out = df.copy()
    q1, q3 = out[column].quantile([0.25, 0.75])
    iqr = q3 - q1
    out[f"{column}_outlier"] = ((out[column] < q1 - k * iqr)
                                | (out[column] > q3 + k * iqr))
    return out
```

The IQR rule: a value is flagged if it is below `Q1 − 1.5 × IQR` or above `Q3 + 1.5 × IQR`, where IQR = Q3 − Q1 (the spread of the middle half of the data). `strip_text` (trim and lower-case the named text columns with `.astype("string").str.strip().str.lower()`) and `drop_duplicate_rows` (`df.drop_duplicates(subset=..., keep="first").reset_index(drop=True)`) follow the same pattern.

Step 3: The pipeline with a report. Counting inside the pipeline is what makes it trustworthy.

```
def clean_orders(df):
    report = {"rows_in": len(df)}
    out = standardise_columns(df)
    out = strip_text(out, ["city", "status"])
    out = parse_dates(out, "order_date")
    report["bad_dates"] = int(out["order_date"].isna().sum())
    before = len(out)
    out = drop_duplicate_rows(out, ["order_id"])
    report["duplicates_dropped"] = before - len(out)
    report["missing_amount"] = int(out["amount"].isna().sum())
    out = fill_missing(out, "amount", "median")
    out = flag_outliers_iqr(out, "amount")
    report["outliers_flagged"] = int(out["amount_outlier"].sum())
    report["rows_out"] = len(out)
    return out, report
```

Step 4: Run it on the sample. Predict first: after de-duplication there are 5 rows with amounts 100, 120, missing, 110, 9000. What median will fill the gap?

```
import pandas as pd
from tidyshop import clean_orders
raw = pd.DataFrame({
    "Order ID": [1, 2, 2, 3, 4, 5],
    "City": [" Pune", "pune ", "pune ", "DELHI", "Delhi", "Pune"],
    "Status": ["Shipped"] * 6,
    "Order Date": ["2025-01-01", "2025-01-02", "2025-01-02", "bad", "2025-01-04", "2025-01-05"],
    "Amount": [100.0, 120.0, 120.0, None, 110.0, 9000.0],
})
out, rep = clean_orders(raw)
print(rep)
print(out.to_string())
```

Real output:

```
{'rows_in': 6, 'bad_dates': 1, 'duplicates_dropped': 1, 'missing_amount': 1, 'outliers_flagged': 1, 'rows_out': 5}
   order_id   city   status order_date  amount  amount_outlier
0         1   pune  shipped 2025-01-01   100.0           False
1         2   pune  shipped 2025-01-02   120.0           False
2         3  delhi  shipped        NaT   115.0           False
3         4  delhi  shipped 2025-01-04   110.0           False
4         5   pune  shipped 2025-01-05  9000.0            True
```

Check by hand: the four known amounts are 100, 110, 120, 9000. The median of four values is the average of the middle two: (110 + 120) / 2 = 115. Order 3 received 115. The 9000 order lies far above `Q3 + 1.5 × IQR`, so it is flagged. The city values collapsed from 5 spellings to 2 (`pune` , `delhi`). The bad date `"bad"` became `NaT` ("not a time"), a safe empty date.

Step 5: Tests. Each small function has a test, the input is checked for mutation, and the pipeline has one test on exact numbers.

```
def test_input_is_not_changed(self):
    df = pd.DataFrame({"A B": [1]})
    standardise_columns(df)
    self.assertEqual(list(df.columns), ["A B"])

def test_outlier_flag(self):
    df = pd.DataFrame({"x": [10, 11, 12, 13, 14, 500]})
    flags = flag_outliers_iqr(df, "x")["x_outlier"].tolist()
    self.assertEqual(flags, [False] * 5 + [True])
```

```
$ PYTHONPATH=src python3 -m unittest discover -s tests -v
...
Ran 8 tests in 0.006s

OK
```

Step 6: Docs. Put the Step 4 code and output in `docs/usage.md` , and write a docstring on every function. A reader should learn the package from the README and one example.

Evaluation / acceptance criteria

Check	Expected
Report on sample	`duplicates_dropped=1, bad_dates=1, missing_amount=1, outliers_flagged=1, rows_out=5`
Filled value (hand calculation)	115.0
Input table unchanged after any function	test passes

Check	Expected
Tests	8 pass
Import works after `pip install -e .`	`python -c "import tidyshop"` (not run here)
Every public function has a docstring	`help(tidyshop.fill_missing)` shows text

Common pitfalls

1. **Changing the caller's data.** Without `.copy()`, the original table changes silently. Detect with the "input is not changed" test.
2. **Filling missing values before removing duplicates (or outliers).** The median changes depending on order. Decide the order, write it in the README, and test it.
3. `errors="coerce"` **hides bad data.** Bad dates become `NaT` quietly. That is why the report counts them. If the count is large, stop and look at the raw file.
4. **Flat layout imports.** Tests passing only because they run next to the code. The `src/` layout and a clean-environment test catch it.
5. **Silent assumptions.** The pipeline assumes columns named `city`, `status`, `order_date`, `amount`. Detect with a test for a missing column and raise a clear `KeyError` message (extension 1).

Three extensions

1. Check that required columns exist and raise a friendly error listing the missing ones.
2. Add a `--report json` command-line entry point (`[project.scripts]` in `pyproject.toml`).
3. Make the outlier method pluggable (IQR, z-score, fixed limits) and compare them on the same data.

Interview talking points

Q1. Why return a new table instead of changing the old one? Changing inputs creates bugs that appear far from their cause. If every function returns a new table, I can compare before and after, and tests are simple.

Q2. How do you know your cleaning did not damage the data? The pipeline returns a report: rows in and out, duplicates dropped, missing values filled, outliers flagged. I check these counts against a hand calculation on a small sample, and I flag outliers rather than delete them, because the owner of the data should decide.

Q3. Why the `src/` layout? It forces tests to use the installed package, as a user would. With a flat layout, Python can import the local folder by accident and hide packaging errors.

README outline

Purpose; install (`pip install -e .`, not run here); five-line example with real report output; list of functions in a table; the order of the pipeline and why; tests; limits (assumes four column names); changelog entry for `0.1.0`.

15.4 Project: A Small REST API With a Database

Goal and why employers care

Build a small web service that stores shop orders in a database and offers four actions: list, read one, create, delete. This is the backbone of most back-end work: an HTTP interface, a database, input checks and correct status codes. We use only `http.server` and `sqlite3` so that it runs anywhere. In industry you would use a framework such as FastAPI, and the second half of this project shows that version (not run here), so you see what stays the same and what the framework does for you.

Success criteria: the five tests pass; `POST` returns 201, bad input returns 422, a missing order returns 404, a delete returns 204 and the next delete returns 404; text such as `x'); DROP TABLE orders;--` is stored as plain text.

Skills used

Chapter 12 (HTTP, REST design, status codes, building a small service), Chapter 5 (JSON), Chapter 6 (exceptions), Chapter 7 (context managers, used by `with conn:`), Chapter 10 (testing), Chapter 13 (security: parameterised SQL). The learner's SQL document covers `CREATE TABLE`, `INSERT` and constraints.

Data

The data is the orders your tests send. A table with three columns:

Column	Type	Rule
`id`	integer	assigned by the database
`customer`	text	not empty
`amount`	real	greater than 0 (also a database `CHECK`)

Architecture

```
orders-api/
  orders_api.py         connect(), make_handler(db_path), serve()
  test_orders_api.py    starts a real server on a free port, calls it
  README.md
  orders.db             (created at run time; in .gitignore)
```

```
 client --HTTP--> Handler.do_GET / do_POST / do_DELETE
                       |  parse path, parse JSON, validate
                       v
                  sqlite3 (one short connection per request, "?" placeholders)
                       |
                       v
          JSON body + status (200, 201, 204, 404, 422)
```

Two routes: `/orders` (the collection) and `/orders/<id>` (one order). Nouns in the URL, the method carries the action (Chapter 12).

Step-by-step build

Step 1: The database layer. A `CHECK` constraint makes the database refuse bad amounts even if your Python code forgets.

```
import json, sqlite3
from http.server import BaseHTTPRequestHandler, HTTPServer

SCHEMA = """CREATE TABLE IF NOT EXISTS orders (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    customer TEXT NOT NULL,
    amount REAL NOT NULL CHECK (amount > 0)
)"""

def connect(db_path):
    conn = sqlite3.connect(db_path)
    conn.row_factory = sqlite3.Row      # rows behave like dicts
    conn.execute(SCHEMA)
    return conn
```

Step 2: A handler that sends JSON and status codes. One helper does the repeated work.

```
def make_handler(db_path):
    class Handler(BaseHTTPRequestHandler):
        def _send(self, status, payload=None):
            body = b"" if payload is None else json.dumps(payload).encode()
            self.send_response(status)
            self.send_header("Content-Type", "application/json")
            self.send_header("Content-Length", str(len(body)))
            self.end_headers()
            self.wfile.write(body)

        def _order_id(self):
            parts = self.path.strip("/").split("/")
            if len(parts) == 2 and parts[0] == "orders" and parts[1].isdigit():
                return int(parts[1])
            return None
        # do_GET, do_POST, do_DELETE ...
```

The handler is built inside a function so it knows `db_path`. That is a closure (Chapter 2), and it keeps tests from sharing a global database.

Step 3: Create with validation, using `?` placeholders.

```
        def do_POST(self):
            if self.path.rstrip("/") != "/orders":
                return self._send(404, {"error": "no such route"})
            size = int(self.headers.get("Content-Length", 0))
            try:
                data = json.loads(self.rfile.read(size))
                customer, amount = str(data["customer"]).strip(), float(data["amount"])
                if not customer or amount <= 0:
                    raise ValueError
            except (ValueError, KeyError, TypeError):
                return self._send(422, {"error": "need customer (text) and amount > 0"})
            conn = connect(db_path)
            try:
                with conn:      # commits on success, rolls back on error
                    cur = conn.execute(
                        "INSERT INTO orders (customer, amount) VALUES (?, ?)",
                        (customer, amount))
                return self._send(201, {"id": cur.lastrowid,
                                        "customer": customer, "amount": amount})
            finally:
                conn.close()
```

`do_GET` selects with `WHERE id = ?` and returns 404 when `fetchone()` is `None`. `do_DELETE` checks `cur.rowcount` (rows deleted): 204 if one, 404 if zero. `log_message` is overridden to stay quiet.

Step 4: Why `?` and never an f-string. Predict: what happens if a customer is called `O'Brien` and you build the SQL with an f-string?

```
name = "O'Brien"
c.execute(f"insert into o values ('{name}')")      # wrong
c.execute("insert into o values (?)", (name,))     # right
```

Real output:

```
OperationalError near "Brien": syntax error
[("O'Brien",)]
```

The apostrophe ended the SQL text early. A malicious name does worse than crash: it can run commands (SQL injection). With `?` the value always travels separately from the command, so it can never become part of it. Our test stores `x'); DROP TABLE orders;--` and reads it back unchanged.

Step 5: Try it end to end. `test_orders_api.py` has a small `call(method, url, body)` helper built on `urllib.request`, and the demo starts the server on port 0 (any free port) in a thread.

```
(201, {'id': 1, 'customer': 'Asha', 'amount': 250.0})
(201, {'id': 2, 'customer': 'Ravi', 'amount': 99.5})
(200, [{'id': 1, 'customer': 'Asha', 'amount': 250.0}, {'id': 2, 'customer': 'Ravi', 'amount': 99.5}])
(404, {'error': 'order not found'})
(422, {'error': 'need customer (text) and amount > 0'})
(204, None)
(200, [{'id': 2, 'customer': 'Ravi', 'amount': 99.5}])
```

Read it line by line: two creates (201 Created, ids 1 and 2), a list of both, a read of id 7 (404), a create with amount −1 (422), deleting id 1 (204 No Content, no body), and a list that now holds only Ravi.

To run it for real: `python3 orders_api.py` serves on port 8000 and you can call it with `curl` (not run here, because the tests use a thread instead).

Step 6: Tests that start a real server. `setUp` makes a temporary database and a server; `tearDown` shuts both down.

```
def setUp(self):
    self.tmp = tempfile.TemporaryDirectory()
    db = str(Path(self.tmp.name) / "t.db")
    self.srv = HTTPServer(("127.0.0.1", 0), make_handler(db))
    threading.Thread(target=self.srv.serve_forever, daemon=True).start()
    self.base = f"http://127.0.0.1:{self.srv.server_address[1]}"

def tearDown(self):
    self.srv.shutdown(); self.srv.server_close(); self.tmp.cleanup()
```

```
$ python3 -m unittest
.....
Ran 5 tests in 2.585s

OK
```

Our first run printed `ResourceWarning: Implicitly cleaning up <HTTPError 422>` twice. The test helper read an error reply but never closed it. We added `e.close()` after reading the body, and the warnings disappeared. A warning is a bug report: read it.

The same API in FastAPI (not run here)

FastAPI needs `pip install fastapi uvicorn`, which was not possible here, so this code is **not run here**. It is the standard shape from the documentation.

```
# main.py  (not run here)
import sqlite3
from fastapi import FastAPI, HTTPException, Response
from pydantic import BaseModel, Field

app = FastAPI()

class OrderIn(BaseModel):
    customer: str = Field(min_length=1)
    amount: float = Field(gt=0)

def db():
    conn = sqlite3.connect("orders.db")
    conn.row_factory = sqlite3.Row
    return conn

@app.post("/orders", status_code=201)
def create(order: OrderIn):
    with db() as conn:
        cur = conn.execute("INSERT INTO orders (customer, amount) VALUES (?, ?)",
                           (order.customer, order.amount))
    return {"id": cur.lastrowid, **order.model_dump()}

@app.get("/orders/{order_id}")
def read(order_id: int):
    row = db().execute("SELECT * FROM orders WHERE id = ?", (order_id,)).fetchone()
    if row is None:
        raise HTTPException(404, "order not found")
    return dict(row)
```

Run with `uvicorn main:app --reload` and open `/docs` for a generated test page. What the framework did for you: parsing the JSON, checking types and rules (`OrderIn` replaces our `try/except`, and bad input gets a 422 automatically), routing, and documentation. What it did not change: SQL with `?`, status codes, and tests.

Evaluation / acceptance criteria

Check	Expected
Create valid order	201 and an `id`
Create with amount −5 or missing `amount`	422
Read id 99 (does not exist)	404
Delete twice	204, then 404
Injection-style name	stored and returned as the same text
Tests	5 pass

Common pitfalls

1. **SQL built with f-strings.** Gives errors on quotes and opens injection. Detect: search the code for `execute(f"`.
2. **Forgetting `Content-Length`.** The client waits for more data and seems to hang. Detect with a test that has a timeout.
3. **Shared database between tests.** Tests pass alone and fail together. Use a temporary database per test.
4. **Wrong status codes.** Returning 200 for everything makes errors invisible to clients. Test the status, not only the body.
5. **Leaving a server thread running.** A test run that never exits. Always `shutdown()` and `server_close()` in `tearDown`.

Three extensions

1. `PUT /orders/<id>` to change an amount, plus `GET /orders?customer=Asha`.
2. A simple API token in the `Authorization` header, read from an environment variable (Chapter 12).
3. Move to the FastAPI version above and keep the same five tests running against it (use a test client).

Interview talking points

Q1. What is the difference between 404 and 422? 404 means the thing you asked for does not exist. 422 means I understood your request, but the content is not acceptable, for example a negative amount.

Q2. Why parameterised queries? The database receives the command and the values separately, so a value can never change the command. It also handles quotes correctly. It is the first defence against SQL injection.

Q3. Why use a framework if the standard library works? The standard library works for learning and small tools, but I would have to write routing, validation, documentation and concurrency handling myself. A framework gives them tested and consistent. I still need to understand the basics underneath, or I cannot debug it.

README outline

Purpose; routes table (method, path, success code, error codes); how to run the server and sample `curl` calls (not run here); schema; how to run the tests; security notes (parameterised SQL, no authentication yet); FastAPI version and why.

15.5 Project: A Responsible Web Scraper and Dataset Builder

Goal and why employers care

Build a tool that turns product pages into a clean CSV dataset, while respecting the website. Many analytics jobs start with "we need data that is only available on web pages". Employers want people who can do it **and** know where the line is: read `robots.txt`, honour the terms of use, slow down, identify yourself, and never collect personal data. This project teaches the parsing on a local HTML string (so it runs without any network), and the etiquette that makes the difference.

Success criteria: the parser extracts 3 products from the sample page; prices become numbers, and "sold out" becomes an empty value; `robots.txt` rules are checked before every fetch; the pause between requests is at least the site's requested delay.

Skills used

Chapter 3 (strings and regular expressions), Chapter 5 (CSV), Chapter 7 (classes with state), Chapter 12 (HTTP and scraping), Chapter 10 (tests), Chapter 13 (responsible use).

Data

No real site is touched. We use a small HTML string with three products (one sold out) and a `robots.txt` string. Real scraping would feed the same parser with downloaded pages.

Responsible-scraping rules (read before writing code)

1. **Look for an official API or a downloadable dataset first.** It is easier and welcome.
2. **Read the terms of use.** If they forbid automated access, stop. A technical possibility is not permission.
3. **Check `/robots.txt`.** It lists paths the owner asks bots to avoid and sometimes a `Crawl-delay`. It is a request, not a lock, and you follow it anyway.
4. **Identify yourself** with a clear `User-Agent` that includes a contact address.
5. **Go slowly:** a pause between requests (2 seconds or the site's `Crawl-delay`), few requests, and cache pages you already have.
6. **Collect only what you need.** No personal data (names, phone numbers, photos of people).
7. **Do not republish copyrighted content.** Publish your code and a small derived sample, with the source credited.

Architecture

```
web-scraper/
  scraper/
    fetch.py        polite_get(): robots check, User-Agent, delay, cache (not run here)
    parse.py        ProductParser, parse_price()
    export.py       to_csv()
  tests/
    test_parse.py   runs on the local HTML string
  data/
    sample/products.csv
  README.md
```

```
 robots.txt --allowed?--> polite_get(url) --HTML--> ProductParser --> items --> CSV
                |              |  sleep(delay)                |
             skip if not    User-Agent + contact        parse_price(): "Rs. 1,299" -> 1299.0
```

Step-by-step build

Step 1: Check robots.txt rules. The standard library includes `urllib.robotparser`. Here we feed it a string.

```
import urllib.robotparser

ROBOTS = """
User-agent: *
Disallow: /private/
Crawl-delay: 2
"""

def allowed(robots_text, url, agent="study-bot"):
    rp = urllib.robotparser.RobotFileParser()
    rp.parse(robots_text.splitlines())
    return rp.can_fetch(agent, url), rp.crawl_delay(agent)
```

```
(True, 2)      # allowed("https://shop.example/products")
(False, 2)     # allowed("https://shop.example/private/admin")
```

So `/products` may be fetched and `/private/admin` may not, and we must wait at least 2 seconds between requests.

Step 2: Parse HTML with `html.parser`. `HTMLParser` calls your methods when it meets a start tag, text, or an end tag. You keep a small state: "am I inside a product?" and "which field am I reading?"

```
from html.parser import HTMLParser

class ProductParser(HTMLParser):
    def __init__(self):
        super().__init__()
        self.items, self._cur, self._field = [], None, None

    def handle_starttag(self, tag, attrs):
        a = dict(attrs)
        if tag == "div" and a.get("class") == "product":
            self._cur = {"name": "", "price_text": "", "url": ""}
        elif self._cur is not None:
            if tag in ("h2", "span") and a.get("class") in ("name", "price"):
                self._field = "name" if a["class"] == "name" else "price_text"
            elif tag == "a":
                self._cur["url"] = a.get("href", "")

    def handle_data(self, data):
        if self._cur is not None and self._field:
            self._cur[self._field] += data.strip()

    def handle_endtag(self, tag):
        if tag in ("h2", "span"):
            self._field = None
        elif tag == "div" and self._cur is not None:
            self.items.append(self._cur)
            self._cur = None
```

Step 3: Turn price text into numbers.

```
def parse_price(text):
    """'Rs. 1,299' -> 1299.0 ; anything without digits -> None."""
    digits = "".join(ch for ch in text if ch.isdigit() or ch == ".").strip(".")
    return float(digits) if digits else None
```

Step 4: Run on the sample page and export. Predict: what will the price of "Desk Lamp" be in the CSV?

```
p = ProductParser()
p.feed(PAGE)          # PAGE is the HTML string with three product blocks
for it in p.items:
    print(it)
```

Real output:

```
{'name': 'Steel Bottle', 'price_text': 'Rs. 450', 'url': '/p/101'}
{'name': 'Notebook (A5)', 'price_text': 'Rs. 1,299', 'url': '/p/102'}
{'name': 'Desk Lamp', 'price_text': 'sold out', 'url': '/p/103'}
```

And `to_csv(p.items)` (a `csv.DictWriter` writing `name`, `price`, `url`) produces:

```
name,price,url
Steel Bottle,450.0,/p/101
Notebook (A5),1299.0,/p/102
Desk Lamp,,/p/103
```

"sold out" has no digits, so `parse_price` returns `None`, and the CSV writer leaves the cell empty. An empty cell is honest: we did not guess a price. Note also that we kept the raw text (`price_text`) next to the parsed value while debugging; keep it until you trust the parser.

Step 5: The polite fetcher (not run here). It needs the network, so it was **not run here**. This is the standard-library version.

```
import time, urllib.request

UA = "study-bot/0.1 (contact: you@example.com)"
DELAY = 2.0          # at least the site's Crawl-delay

def polite_get(url, cache={}):
    if url in cache:
        return cache[url]
    time.sleep(DELAY)
    req = urllib.request.Request(url, headers={"User-Agent": UA})
    with urllib.request.urlopen(req, timeout=10) as r:
        cache[url] = r.read().decode("utf-8", errors="replace")
    return cache[url]
```

The `requests` and BeautifulSoup version (not run here, needs `pip install requests beautifulsoup4`):

```
import requests
from bs4 import BeautifulSoup

html = requests.get(url, headers={"User-Agent": UA}, timeout=10).text
soup = BeautifulSoup(html, "html.parser")
for box in soup.select("div.product"):
    name = box.select_one(".name").get_text(strip=True)
    price = box.select_one(".price").get_text(strip=True)
```

`select` takes CSS selectors, so the parser class collapses to two lines. The `User-Agent`, delay and timeout are exactly the same; libraries do not remove your responsibility.

Step 6: Tests. Because parsing runs on a string, tests are fast and need no network.

```
class ParseTests(unittest.TestCase):
    def test_three_products(self):
        p = ProductParser(); p.feed(PAGE)
        self.assertEqual(len(p.items), 3)
    def test_prices(self):
        self.assertEqual(parse_price("Rs. 1,299"), 1299.0)
        self.assertIsNone(parse_price("sold out"))
    def test_robots(self):
        self.assertFalse(allowed(ROBOTS, "https://shop.example/private/x")[0])
```

These three tests were run and pass (`Ran 3 tests ... OK`). Put them in `tests/test_parse.py` and run them as you did in Project 1.

Evaluation / acceptance criteria

Check	Expected
Products parsed from sample page	3
Price of Bottle, Notebook, Lamp	450.0, 1299.0, empty
`/private/admin`	blocked by robots check
Delay between real requests	at least `Crawl-delay` (2 s here)
Request headers	custom `User-Agent` with contact
Dataset	no personal data; source and date written in README

Common pitfalls

1. **Page layout changes.** A site changes `class="price"` and your parser returns blanks. Detect: a test and a run-time check that "items found > 0" and "share of empty prices is below a limit".
2. **Fragile price cleaning.** Our `parse_price` keeps digits and dots, so `Rs.1.5` style text or a currency like `1.299,00` (European format) would be read wrongly. Detect by testing real examples from the page, and handle each format on purpose.
3. **Ignoring etiquette.** No delay, no `User-Agent`, ignoring robots.txt. Detect: you get HTTP 429 (too many requests) or get blocked. Respond by slowing down and rereading the terms; do not hide from the block.
4. **Scraping JavaScript pages with a plain downloader.** The HTML you receive has no products because they load later. Detect: `len(items) == 0` on a page that shows products in a browser. Look for an official API first; browser tools are a last resort.
5. **Saving raw pages and personal data.** Keep only the fields you need.

Three extensions

1. Follow pagination ("next page") with a hard limit of pages and the delay.
2. Save to SQLite and keep a history of prices by date.
3. A small chart of price changes over time with `matplotlib` (see the Data Science track).

Interview talking points

Q1. How do you decide if you may scrape a site? First I look for an API or a download. Then I read the terms of use and `robots.txt`. If automated access is forbidden, I do not scrape. If it is allowed, I go slowly, identify myself, and collect only what I need.

Q2. What happens when the site changes its layout? My parser returns fewer items or empty fields. So I add checks: items found must be above zero and the share of missing prices must stay below a limit. A failed check stops the job and tells me, instead of silently saving bad data.

Q3. Why parse with a parser and not regular expressions? HTML is nested and messy. A regular expression breaks when attributes change order or tags nest. A parser follows the structure of the page.

README outline

Purpose; legal and ethical section (what the site's terms and robots.txt say, date checked, your delay and User-Agent); install; example run with real output; data dictionary (columns); how to run the tests; known limits; how the dataset can be used.

15.6 Project: A Log Analyser Built Test-First

The projects above start from data. This one starts from a failing test, the way professional teams work. The tool, `loglens`, reads log lines such as `ERROR db: timeout` (the format from Chapter 6) and reports how many lines each level has, which service fails most, and how many lines it could not understand. It is small on purpose, so you can see the whole cycle: write a test, watch it fail, write just enough code, repeat.

Problem 15.6.1 Build `parse_line` and `analyse` test-first. Success criteria (written before any code): (1) a valid line gives a dict with level, service and msg; (2) a malformed line gives `None`, never an exception; (3) blank lines are ignored, not counted as bad; (4) the top error service is reported; (5) lowercase levels are rejected.

Step 1: list cases	Empty input, blank lines, garbage, a tie between services, lowercase level. The unusual inputs are where bugs live, so they go first.
Step 2: red	Write one test, for example `test_garbage_is_counted`. Run it. It must fail (here with `NameError`, since nothing exists yet). A test you have never seen fail may not test anything.
Step 3: green	Write the smallest code that passes: a regular expression with named groups for the line shape, and a `Counter` for each tally.
Step 4: repeat	Add the next test. After each one, run the whole suite.
Step 5: refactor	With tests green, tidy names and split functions. The tests tell you at once if you broke something.

The finished code, run on a sample with one blank line and one garbage line:

```
import re
from collections import Counter

LINE = re.compile(r"^(?P<level>INFO|WARNING|ERROR) (?P<service>[\w.]+): (?P<msg>.*)$")

def parse_line(line):
    m = LINE.match(line.strip())
    return m.groupdict() if m else None

def analyse(lines):
    levels, errors, bad = Counter(), Counter(), 0
    for line in lines:
        if not line.strip():
            continue
        rec = parse_line(line)
        if rec is None:
            bad += 1
            continue
        levels[rec["level"]] += 1
        if rec["level"] == "ERROR":
            errors[rec["service"]] += 1
    return {"levels": dict(levels), "top_error_service": errors.most_common(1), "bad_lines": bad}

sample = """INFO api: started
ERROR db: timeout
ERROR db: timeout again
WARNING api: slow response

garbage here
ERROR api: 500 returned""".splitlines()
print(parse_line("ERROR db: timeout"))
print(parse_line("garbage"))
print(analyse(sample))
```

```
{'level': 'ERROR', 'service': 'db', 'msg': 'timeout'}
None
{'levels': {'INFO': 1, 'ERROR': 3, 'WARNING': 1}, 'top_error_service': [('db', 2)], 'bad_lines': 1}
```

Notice that `top_error_service` is `[('db', 2)]`: three errors in total, two from `db`. Now the tests that drove it:

```
import unittest, re
from collections import Counter
LINE = re.compile(r"^(?P<level>INFO|WARNING|ERROR) (?P<service>[\w.]+): (?P<msg>.*)$")
def parse_line(line):
    m = LINE.match(line.strip())
    return m.groupdict() if m else None
def analyse(lines):
    levels, errors, bad = Counter(), Counter(), 0
    for line in lines:
        if not line.strip():
            continue
        rec = parse_line(line)
        if rec is None:
            bad += 1
            continue
        levels[rec["level"]] += 1
        if rec["level"] == "ERROR":
            errors[rec["service"]] += 1
    return {"levels": dict(levels), "top_error_service": errors.most_common(1), "bad_lines": bad}

class AnalyseTests(unittest.TestCase):
    def test_empty_input(self):
        self.assertEqual(analyse([]), {"levels": {}, "top_error_service": [], "bad_lines": 0})
    def test_blank_lines_are_not_bad(self):
        self.assertEqual(analyse(["", "   "])["bad_lines"], 0)
    def test_garbage_is_counted(self):
        self.assertEqual(analyse(["nonsense"])["bad_lines"], 1)
    def test_top_error_service(self):
        lines = ["ERROR db: a", "ERROR db: b", "ERROR api: c"]
        self.assertEqual(analyse(lines)["top_error_service"], [("db", 2)])
    def test_lowercase_level_is_rejected(self):
        self.assertIsNone(parse_line("error db: x"))

import io
suite = unittest.defaultTestLoader.loadTestsFromTestCase(AnalyseTests)
result = unittest.TextTestRunner(stream=io.StringIO()).run(suite)
print("ran", result.testsRun, "ok" if result.wasSuccessful() else "FAILED")
```

```
ran 5 ok
```

COMMON MISTAKE

Writing all the code first and the tests afterwards. The tests then copy what the code does, bugs included. Wrong: run the code, paste its output into `assertEqual`. Right: work out the expected value by hand from the problem statement, then compare.

Adding the command line

Keep the logic in `analyse` and make `main` a thin wrapper that takes its inputs as arguments, so a test can pass a fake argument list and a fake `stdin` (an `io.StringIO`) instead of the real ones. The exit code matters in automation: 0 means success, non-zero tells a script or CI job to stop.

```
import argparse, io, json, re, sys
from collections import Counter
LINE = re.compile(r"^(?P<level>INFO|WARNING|ERROR) (?P<service>[\w.]+): (?P<msg>.*)$")

def analyse(lines):
    levels, bad = Counter(), 0
    for line in lines:
        if not line.strip():
            continue
        m = LINE.match(line.strip())
        if m is None:
            bad += 1
        else:
            levels[m["level"]] += 1
    return {"levels": dict(levels), "bad_lines": bad}

def main(argv, stdin):
    p = argparse.ArgumentParser(prog="loglens")
    p.add_argument("--max-bad", type=int, default=0, help="allowed unparsable lines")
    args = p.parse_args(argv)
    report = analyse(stdin)
    print(json.dumps(report, sort_keys=True))
    return 0 if report["bad_lines"] <= args.max_bad else 1

fake = io.StringIO("INFO a: ok\nERROR a: boom\nnonsense\n")
print("exit", main([], fake))
fake = io.StringIO("INFO a: ok\nERROR a: boom\nnonsense\n")
print("exit", main(["--max-bad", "1"], fake))
```

```
{"bad_lines": 1, "levels": {"ERROR": 1, "INFO": 1}}
exit 1
{"bad_lines": 1, "levels": {"ERROR": 1, "INFO": 1}}
exit 0
```

The first run finds 1 bad line with an allowance of 0, so it exits 1. The second allows 1 bad line and exits 0. In a real file `main` would read `sys.stdin` and the script would end with `sys.exit(main(sys.argv[1:], sys.stdin))` (not run here, since it needs a real terminal pipe).

COMMON MISTAKE

Testing the program by running it by hand and reading the output. Wrong: "it looked right when I ran it". Right: every case you checked by eye becomes a test, so the next change cannot silently break it. Also do not let a bad line crash the whole run: one corrupt line in a million should be counted, not fatal.

Self-check rubric (1 point each, 8 total)

- Success criteria were written down before code.
- You saw each test fail before you made it pass.
- Edge cases tested: empty input, blank lines, garbage, lowercase level.
- Logic is separate from input and output, so `analyse` takes a list of lines, not a file.
- A bad line never raises.
- The command line exits non-zero when too many bad lines are found.
- A README shows one example command and its output.
- All tests run with one command (`python3 -m unittest`).

Problem 15.9 (stretch) Extend `loglens` to report the slowest request. Lines now look like `INFO api: GET /users took 120ms`. Write the failing test first. What must `parse_line` do with a line that has no `took` part?

Answer: Add a second regex `took (\d+)ms$` applied to `msg` and store `ms` as an `int` or `None`. A line without it must still parse (ms is `None`) so criterion 2 still holds, and the slowest-request code must skip `None` values. Test with input where the only line lacks a time: the result should be

`None`, not a crash from `max([])`.

Problem 15.10 (find the bug) A teammate's test is `self.assertEqual(analyse(["ERROR db: a", "ERROR api: b"])["top_error_service"], [("db", 1)])` and it passes today. Why is it a fragile test?

Answer: The two services tie at 1. `most_common(1)` returns the first of the tied items in insertion order, so the test silently depends on the order of the input lines and on an unstated tie rule. Decide the tie rule (for example alphabetical) in the code and test it explicitly, or assert on a case with no tie.

Chapter recap

- A project is a question plus measurable success criteria plus a small plan; write them before coding.
- The same layout works for every project: README, `pyproject.toml`, source, tests, sample data.
- Keep logic (`core`) separate from input and output (`cli`, web handler), so tests are simple.
- Validate at the edge, raise one clear error type, return exit codes (0 good, non-zero bad).
- Use temporary folders, temporary databases and local strings in tests; never touch real data or the network.
- Parameterised SQL (`?`) prevents errors with quotes and prevents injection.
- A cleaning pipeline should return a report of what it changed, and flag rather than delete suspicious data.
- Scraping is allowed only after checking terms and robots.txt, and it must be slow, identified and minimal.
- Read warnings and the count of tests run; `NO TESTS RAN` is not a pass.

Pattern	Command or code
Run all tests	`python3 -m unittest discover -v` (needs `tests/__init__.py`)
Run a module	`python3 -m package_name args`
Exit code of last command	`echo $?`
Safe SQL	`conn.execute("... WHERE id = ?", (value,))`
Free port in tests	`HTTPServer(("127.0.0.1", 0), handler)`
Commit message	`Verb + what + why`, under about 60 characters
robots check	`RobotFileParser().parse(lines)`, then `can_fetch(agent, url)`

Practice set

Problem 15.1 (easy) Write three success criteria for the expense tracker. **Answer:** Any measurable three, for example: `summary --month 2025-03` returns 245.50 for the sample rows; a bad date returns exit code 1 with a message; 6 tests pass.

Problem 15.2 (easy) What does this print: `python3 -m expense_tracker add 2025-02-30 food 10`? **Answer:** `error: bad date '2025-02-30', use YYYY-MM-DD` (February 30 does not exist, so `fromisoformat` raises `ValueError`), and the exit code is 1.

Problem 15.3 (easy) Which of these commit messages is better, and why: `update` or `Fix: reject negative amounts (test added)`? **Answer:** The second. It says what changed and that a test protects it.

Problem 15.4 (medium) Your tests print `NO TESTS RAN`. List two causes. **Answer:** The `tests` folder has no `__init__.py` (so discovery does not treat it as a package), or you ran the command from the wrong folder, or test files and methods do not start with `test`.

Problem 15.5 (medium) Find the bug: `conn.execute(f"SELECT * FROM orders WHERE customer = '{name}'")`. **Answer:** SQL built by string formatting breaks on a name like `O'Brien` and allows injection. Use `conn.execute("SELECT * FROM orders WHERE customer = ?", (name,))`.

Problem 15.6 (medium) For the amounts 100, 110, 120 and one missing value, the owner asks you to fill the gap with the median of known values. What value? **Answer:** 110, the middle of 100, 110, 120. Adding an outlier such as 9000 before taking the median would change the answer, which is why the order of cleaning steps matters.

Problem 15.7 (interview) A teammate says "robots.txt does not block me technically, so I can scrape the whole site fast." Reply. **Answer:** `robots.txt` is a request from the owner, and the terms of use may add rules. Being able to fetch a page is not permission. I check the terms, follow robots.txt and its crawl delay, identify my bot, go slowly, and look for an API first.

Problem 15.8 (interview) Explain the difference between 201, 204, 404 and 422 for the orders API. **Answer:** 201: created, returns the new order. 204: success with no body (delete). 404: no such order or route. 422: the request was understood but its content is invalid (negative amount, missing field).

Prompt pack

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data roles. The topic is planning a small Python portfolio project (a command-line expense tracker). Help me write: the user, the question, three measurable success criteria, an out-of-scope list, a repository tree, and a 10-day plan of one hour a day. Ask me up to five clarifying questions first, one at a time, then produce the plan. After that, ask me to explain in my own words why logic should be separate from input and output, and correct me kindly.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data roles. The topic is reviewing a Python project repository before I share it with employers. Here is my repository tree and README: [PASTE HERE]. Review it using a definition-of-done checklist: README clarity, tests from a clean clone, separation of logic from input and output, error handling and exit codes, secrets, and commit messages. Give a short list ordered by importance, with one concrete fix for each item, and explain the reason in simple English.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data roles. The topic is debugging failing tests in a Python project that uses unittest. Here is my test output and the code under test: [PASTE HERE]. Do not give the fix immediately. First teach me how to read the traceback from the last line upward, then ask me what I think the cause is, then give hints one at a time, and only after my attempt show the fix and a test that would have caught it.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data roles. The topic is a small REST API built with http.server and sqlite3, and its FastAPI equivalent. Quiz me one question at a time on status codes (200, 201, 204, 404, 422), parameterised SQL, test setup and teardown, and why a framework helps. Wait for my answer each time, mark it, explain mistakes with a tiny example, and after eight questions tell me which topic I should revise.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data roles. The topic is writing a responsible web scraper and a data-cleaning package as portfolio projects. Give me a mock interview with six questions: three about scraping ethics and robustness (robots.txt, terms, rate limits, layout changes) and three about data cleaning (reports, outliers, order of steps, packaging). Ask one at a time, wait for my answer, grade each from 1 to 5 in simple English, and show a model answer I can learn from. Finish with a one-paragraph summary of my strengths and gaps.
```

CHAPTER 16

NUMPY: ARRAYS, VECTORISATION AND NUMERICAL COMPUTING

Library Handbook: arrays, broadcasting, linear algebra and the recipes you use every day.

It is Monday morning and a retail team hands you a file: three million sales lines, each with a price, a quantity and a store number. They want every price turned into a z-score within its store, the ten biggest baskets flagged, and the distance between every pair of stores computed from their coordinates. You write a `for` loop, press run, and go to make tea. The tea is cold before the loop finishes. A colleague rewrites the same logic in six lines of NumPy and it finishes while the kettle is still boiling.

That gap is not magic. It is the difference between asking Python to do one tiny thing three million times and asking compiled code to do one big thing once. NumPy makes the second way possible, and pandas, SciPy, scikit-learn, PyTorch and matplotlib are all built on its array.

By the end of this chapter you will create arrays of any shape, select exactly the elements you want, broadcast, summarise along any axis, do linear algebra, use reproducible random numbers, avoid view and copy traps, and solve everyday tasks without loops. Everything printed here ran on Python 3.14 and NumPy 2.5.3. Timings were measured on one laptop and will differ on yours.

16.1 What NumPy Is, When to Use It, and the Mental Model

INTUITION

A Python list is a shopping bag of separate boxes: each item is its own object with its own label. A NumPy array is an egg carton: every slot has the same size and type, they sit side by side in one block, and so the library can scan the whole carton in a single sweep. Same-type, side-by-side, one block: that is the whole secret of its speed.

What it is

NumPy (Numerical Python) gives you one main object, the **ndarray** (n-dimensional array): a grid of numbers that all share one **dtype** (data type), stored in one flat block of memory. Around that object it provides thousands of fast functions that work on whole arrays at once. Working on a whole array at once, with no Python loop, is called **vectorisation**.

When to use it, and when not to

Use NumPy when	Think twice when
You have numbers (or booleans) in bulk and want maths on all of them	Your data is mixed text, dates and numbers in named columns: use pandas (the next chapter of this Handbook), which sits on top of NumPy
You need matrices, linear algebra, random numbers, image pixels	You have a handful of values: a plain list is fine and simpler
You want to replace a loop over rows with one expression	Each step depends on the previous one (a recursion): a loop, or a tool such as Numba, may be the right answer
You feed numbers to scikit-learn, SciPy, PyTorch	Data is bigger than memory: use chunking, Dask, or a database (see the Data Engineering document)
You need a fixed-size table of one type	You need to append one item at a time millions of times: collect in a list, convert once

Install, version check and import convention

Install with `pip install numpy` (inside a virtual environment, see Chapter 9). The universal import is `import numpy as np`. Never use `from numpy import *`: it overwrites Python names such as `sum`, `min`, `max` and `abs` with different functions and causes bugs that are very hard to see.

Problem 16.1.1 Check which NumPy you have, and see the speed difference between a list and an array on a tiny sum of squares.

Idea	Print `np.__version__`. Then compute the same sum of squares with a list comprehension and with one array expression, and confirm they agree.

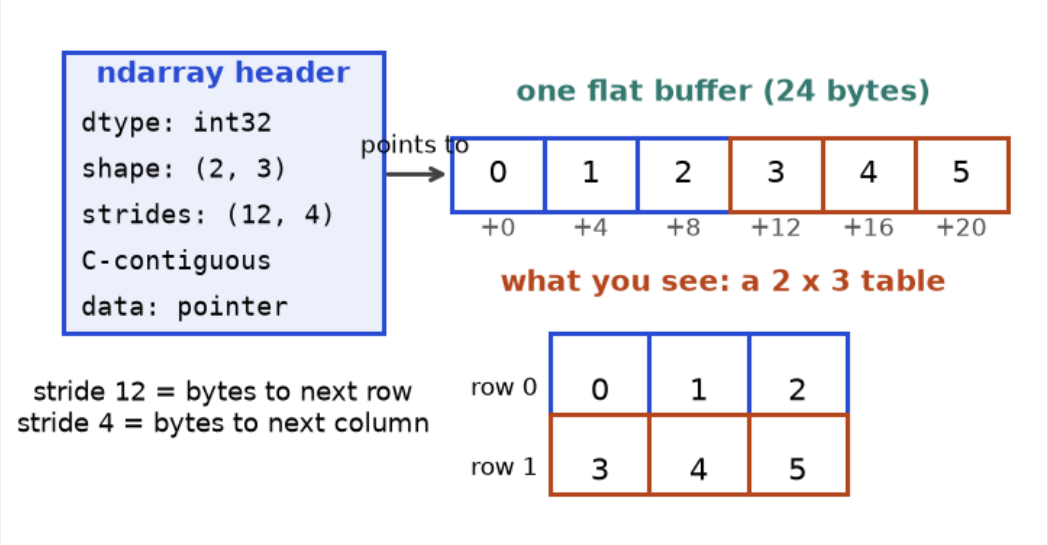
```
import numpy as np
print("major version:", np.__version__.split(".")[0])
values = [1.5, 2.0, 3.5, 4.0]
as_list = sum(v * v for v in values)
arr = np.array(values)
as_array = (arr * arr).sum()
print(as_list, as_array, as_list == as_array)
print(type(arr), arr.dtype, arr.shape)
```

```
major version: 2
34.5 34.5 True
<class 'numpy.ndarray'> float64 (4,)
```

Line by line.

1. The version string looks like `2.5.3`. Splitting on dots and taking the first part tells you the major version, which is 2 here.
2. `arr * arr` multiplies all four pairs in one step; `.sum()` adds them. Both ways give 34.5.
3. The last line shows the three facts every array carries: its type (`numpy.ndarray`), its dtype (`float64`) and its shape (`(4,)` , a one-dimensional array of four items).

The mental model



An ndarray is a small header (dtype, shape, strides) plus a pointer to one flat buffer of bytes. The 2 x 3 table is only a way of reading that buffer.

How to read it: the six boxes are the real data, side by side in memory, four bytes each (`int32`). The header holds instructions: *shape* says "show this as 2 rows and 3 columns"; *strides* say how many bytes to jump to the next row (12) or column (4). Because the header is separate from the data, NumPy can show the same bytes in another shape, order or slice without copying.

KEY IDEA	An array = one buffer of same-type numbers + a header (dtype, shape, strides). Reshaping, transposing and slicing usually change only the header. Arithmetic, sorting and fancy indexing create new buffers.

16.2 The ndarray: Attributes, dtypes and Memory Layout

Before you create arrays in bulk, learn to read one. These attributes answer most "why is my array behaving oddly" questions.

Name	What it tells you (plain English)	Minimal example
`a.shape`	Tuple of sizes per axis	`np.ones((2,3)).shape` gives `(2, 3)`
`a.ndim`	Number of axes	`np.ones((2,3)).ndim` gives `2`
`a.size`	Total number of elements	`np.ones((2,3)).size` gives `6`
`a.dtype`	Type of every element	`np.array([1,2]).dtype` gives `int64`
`a.itemsize`	Bytes per element	`np.float32(1).itemsize` gives `4`
`a.nbytes`	Total bytes of the data buffer	`np.zeros(10).nbytes` gives `80`
`a.strides`	Bytes to step along each axis	`np.zeros((2,3)).strides` gives `(24, 8)`
`a.flags`	Contiguity, writeable, owns data	`a.flags['C_CONTIGUOUS']`
`a.base`	The array whose memory this one borrows, or `None`	`a[1:].base is a`
`a.T`	Transpose (a view)	`np.ones((2,3)).T.shape` gives `(3, 2)`
`a.astype(t)`	New array converted to dtype `t` (always a copy by default)	`np.array([1.7]).astype(int)` gives `[1]`
`a.copy()`	Independent copy of data	`b = a.copy()`
`a.view(t)`	Same bytes read as another dtype	`np.float32(1).view(np.int32)`
`a.tolist()`	Convert to nested Python lists of Python numbers	`np.arange(3).tolist()`
`a.item()`	One element as a Python scalar	`np.array([7]).item()`
`np.iinfo(t)`, `np.finfo(t)`	Limits of an integer or float type	`np.iinfo(np.int8).max` gives `127`
`np.result_type(a, b)`	Which dtype an operation would produce	`np.result_type(np.int8, np.float32)`

The dtypes you will actually meet

dtype	Bytes	Range or meaning	Typical use
`bool`	1	True or False	Masks from comparisons
`int8, uint8`	1	-128..127, 0..255	Image pixels (uint8), small codes
`int32, int64`	4, 8	about +/-2.1 billion, +/-9.2 quintillion	Counts, indexes (int64 is the usual default)
`float32`	4	about 7 decimal digits	Deep learning, big images, half the memory
`float64`	8	about 15-16 decimal digits	Default for decimals, statistics
`complex128`	16	real + imaginary parts	Signal processing (FFT)
`datetime64[ns]`	8	timestamps	Time series (pandas uses it)
`object`	8 (pointer)	any Python objects	Avoid: no speed benefit

Problem 16.2.1 Inspect an array, change its dtype, and see how much memory it uses.

Idea	Create a 1000 by 1000 float64 array, read its attributes, then convert to float32 and uint8 and compare `nbytes`.

```
import numpy as np
a = np.zeros((1000, 1000))
print(a.shape, a.ndim, a.size, a.dtype)
print(a.itemsize, a.nbytes, a.strides)
b = a.astype(np.float32)
c = a.astype(np.uint8)
print(a.nbytes, b.nbytes, c.nbytes)
print(np.array([1, 2, 3]).dtype, np.array([1, 2.5]).dtype, np.array([1, "a"]).dtype)
```

```
(1000, 1000) 2 1000000 float64
8 8000000 (8000, 8)
8000000 4000000 1000000
int64 float64 <U21
```

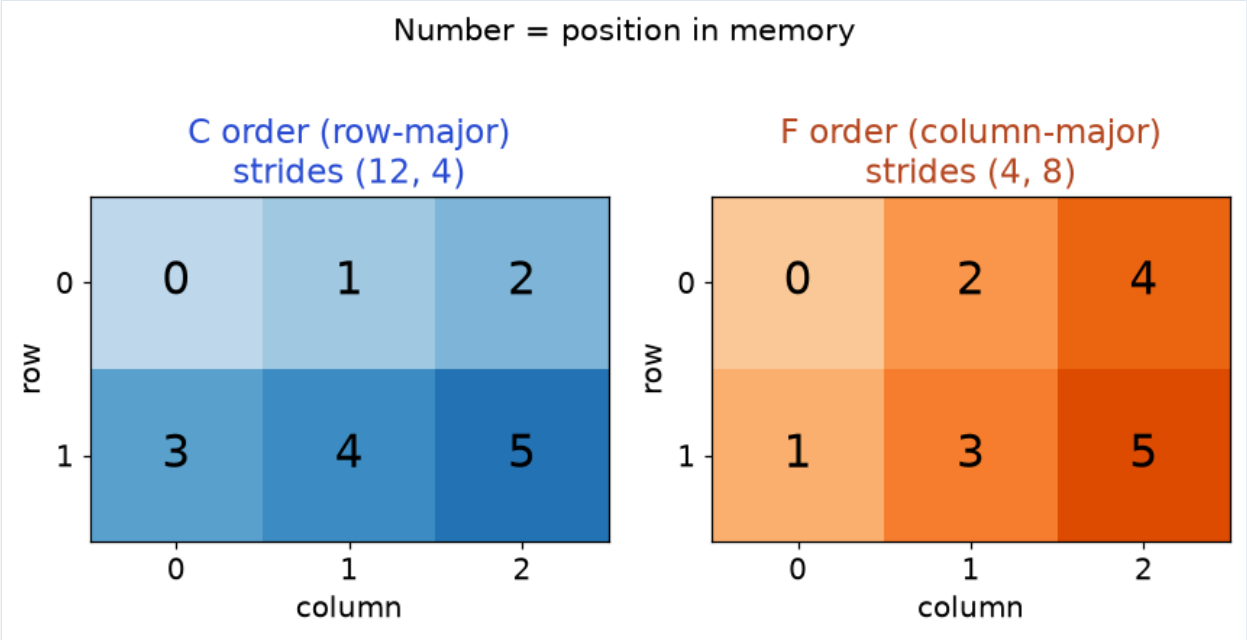
Line by line.

1. Each float64 takes 8 bytes, so the buffer is 8,000,000 bytes (8 MB). Strides `(8000, 8)`: to go down one row, jump 1000 items of 8 bytes; to go right, jump 8.
2. `astype` returns new arrays. float32 halves memory to 4 MB, uint8 cuts it to 1 MB.
3. The last line shows **type promotion** on creation: all ints give `int64`; a mix of int and float becomes `float64`; a mix with text becomes a string dtype (`<U21`), which is rarely what you want.

COMMON MISTAKE	**Wrong:** `np.array([200, 100], dtype=np.uint8) + np.array([100, 100], dtype=np.uint8)` and trusting the answer. **Symptom:** you get `[44 200]`, because 300 does not fit in 0..255 and silently *wraps around*. **Fix:** convert before arithmetic: `a.astype(np.int32) + b`. Integer overflow in NumPy arrays gives no warning.

Memory layout: C order and F order

A 2D array must be flattened into one line of memory. **C order** (row-major, the default) stores row 0 first, then row 1. **Fortran order** (column-major) stores column 0 first. It matters because walking along memory is fast and jumping around is slow, and because some libraries expect one order.



The number in each cell is its position in memory. In C order, neighbours in a row are neighbours in memory. In F order, neighbours in a column are.

How to read it: the left grid counts 0,1,2 along the first row and then 3,4,5 along the second, so moving right is a step of one slot (4 bytes) and moving down jumps three slots (12 bytes). The right grid counts down each column first, so the two step sizes swap. Same table of values, two different physical orders.

Problem 16.2.2 Show that a transpose copies nothing, only swaps the strides, and that a non-contiguous array can be made contiguous.

Idea Make a 2 by 3 `int32` array, transpose it, compare strides and flags, then call `np.ascontiguousarray`.

```
import numpy as np
a = np.arange(6, dtype=np.int32).reshape(2, 3)
t = a.T
print(a.strides, t.strides)
print(a.flags["C_CONTIGUOUS"], t.flags["C_CONTIGUOUS"], t.flags["F_CONTIGUOUS"])
print(t.base is a, np.shares_memory(a, t))
c = np.ascontiguousarray(t)
print(c.strides, np.shares_memory(a, c))
print(t.ravel(), t.ravel(order="K"))
```

```
(12, 4) (4, 12)
True False True
False True
(8, 4) False
[0 3 1 4 2 5] [0 1 2 3 4 5]
```

Line by line.

1. Original strides are `(12, 4)` as in the figure. The transpose has `(4, 12)`: the same bytes, read with the step sizes swapped.
2. The transpose is no longer C-contiguous but is F-contiguous, because reading it column by column walks memory in order.
3. `ascontiguousarray` really copies, giving strides `(8, 4)` for the new 3 by 2 shape, and it no longer shares memory.
4. `ravel()` flattens in logical (row-by-row) order, so the transposed array gives `[0 3 1 4 2 5]`. `order="K"` means "whatever order is in memory", giving `[0 1 2 3 4 5]`.

WHY IT MATTERS Walking memory in order is friendly to the CPU cache, so a reduction along a strided axis can be slower (it depends on shape and machine). When a function is slow on a transposed or sliced array, check `flags` and consider `np.ascontiguousarray`.

16.3 Creating Arrays

There are four ways arrays are born: from existing Python data, from a pattern (zeros, ranges), from random numbers (16.10), and from files. Pick the constructor that says what you mean.

Function	What it does (plain English)	Minimal example
`np.array(obj)`	Copy a list or nested list into an array	`np.array([[1,2],[3,4]])`
`np.asarray(obj)`	Like `array` but does not copy if it already is an array	`np.asarray(a)`
`np.zeros(shape)`	All zeros	`np.zeros((2,3))`
`np.ones(shape)`	All ones	`np.ones(4)`
`np.full(shape, v)`	Every cell is `v`	`np.full((2,2), 7)`
`np.empty(shape)`	Uninitialised memory: garbage until you fill it	`np.empty(3)`
`np.zeros_like(a)`	Zeros with the same shape and dtype as `a`	`np.zeros_like(a)`

Function	What it does (plain English)	Minimal example
`np.arange(start, stop, step)`	Evenly stepped values, stop excluded	`np.arange(0, 10, 3)` gives `[0 3 6 9]`
`np.linspace(a, b, n)`	`n` evenly spaced points, both ends included	`np.linspace(0, 1, 5)`
`np.eye(n)`	Identity matrix	`np.eye(3)`
`np.diag(v)`	Diagonal matrix from a vector (or the diagonal of a matrix)	`np.diag([1,2,3])`
`np.meshgrid(x, y)`	Coordinate grids from two vectors	`X, Y = np.meshgrid(x, y)`
`np.tile(a, reps)`	Repeat the whole array	`np.tile([1,2], 3)`
`np.repeat(a, n)`	Repeat each element	`np.repeat([1,2], 3)`
`np.loadtxt(f)`, `np.genfromtxt(f)`	Read a text or CSV file of numbers (`genfromtxt` copes with gaps)	`np.loadtxt(f, delimiter=",")`
`np.save`, `np.load`	Binary `.npy` file, exact and fast	`np.save("x.npy", a)`

Problem 16.3.1 Build the common starting arrays and look at them.

Idea Use `arange`, `linspace`, `full`, `eye` and `zeros_like`, and watch dtypes.

```
import numpy as np
print(np.arange(0, 10, 3))
print(np.arange(0, 1, 0.25))
print(np.linspace(0, 1, 5))
print(np.full((2, 3), 7))
print(np.eye(3, dtype=int))
x = np.array([1.5, 2.5])
print(np.zeros_like(x), np.zeros_like(x).dtype)
print(np.logspace(0, 3, 4))
```

```
[0 3 6 9]
[0.   0.25 0.5  0.75]
[0.   0.25 0.5  0.75 1.  ]
[[7 7 7]
 [7 7 7]]
[[1 0 0]
 [0 1 0]
 [0 0 1]]
[0. 0.] float64
[   1.   10.  100. 1000.]
```

Line by line.

1. `arange(0, 10, 3)` starts at 0, steps by 3 and stops before 10: `0 3 6 9`.
2. With a float step, `arange` works but can be off by one element because of rounding. Here 0.25 is exact in binary, so it is safe: four values.
3. `linspace(0, 1, 5)` includes both ends and chooses the step for you (0.25). Prefer it when you know how many points you want.
4. `logspace(0, 3, 4)` gives 10^0 to 10^3: `1 10 100 1000`, written as floats.

COMMON MISTAKE

Wrong: `np.arange(0, 1, 0.1)` when you need exactly 11 points, or `np.zeros(3, 4)`. **Symptom:** the first gives 10 values and never reaches 1; the second raises a `TypeError` because the second argument is the dtype. **Fix:** `np.linspace(0, 1, 11)` and `np.zeros((3, 4))`, with the shape as one tuple.

16.4 Indexing: Slicing, Boolean Masks and Fancy Indexing

Indexing is how you say "give me these elements". NumPy has three families and they behave differently in one crucial way: slices are **views** (no copy), while boolean and fancy indexing return **copies**.

Form	What it does	Minimal example (a = np.arange(10))
`a[3]`	One element (a scalar)	gives `3`
`a[-1]`	Last element	gives `9`
`a[2:5]`	Slice, stop excluded (view)	gives `[2 3 4]`
`a[::2]`	Every second element (view)	gives `[0 2 4 6 8]`
`a[::-1]`	Reversed (view)	gives `[9 8 ... 0]`
`m[1, 2]`	Row 1, column 2 of a matrix	one cell
`m[1]` or `m[1, :]`	Whole row 1 (view)	1D array
`m[:, 2]`	Whole column 2 (view)	1D array
`m[:, None]`, `m[..., None]`	Add a new axis of length 1	column from row
`a[a > 5]`	Boolean mask: keep where True (copy)	gives `[6 7 8 9]`
`a[[1, 3, 3]]`	Fancy: pick by list of positions, repeats allowed (copy)	gives `[1 3 3]`
`m[[0, 2], [1, 3]]`	Fancy with two lists: pairs (0,1) and (2,3)	1D array of two cells
`m[np.ix_(rows, cols)]`	Sub-grid: all combinations of these rows and columns	rectangular block
`a[mask] = v`	Assign to selected cells (writes into `a`)	`a[a < 3] = 0`
`np.where(cond)`	Positions where condition is True	`np.where(a > 7)`
`np.take_along_axis`	Pick one element per row using an index matrix (for example after `argsort`)	top-k per row

Problem 16.4.1 Use boolean masks to filter and to assign. A store has daily sales and wants to cap outliers and count good days.

Step 1	A comparison produces a boolean array of the same shape.
Step 2	Combine conditions with `&` (and), `|` (or), `~` (not), each condition in brackets.
Step 3	Summing booleans counts the Trues; averaging them gives a proportion.

```
import numpy as np
sales = np.array([120, 340, 95, 410, 80, 275, 1500, 310])
mask = sales > 300
print(mask)
print(sales[mask])
print(sales[(sales > 100) & (sales < 400)])
print(sales[~mask])
print(mask.sum(), mask.mean())
capped = sales.copy()
capped[capped > 500] = 500
print(capped)
print(sales[sales > 300].mean())
```

```
[False  True False  True False False  True  True]
[ 340  410 1500  310]
[120 340 275 310]
[120  95  80 275]
4 0.5
[120 340  95 410  80 275 500 310]
640.0
```

Line by line.

1. For "between", write both comparisons in brackets and join with `&`. Writing `100 < sales < 400` raises an error, because Python tries to turn an array into a single True or False.
2. `mask.sum()` is 4 (True counts as 1), and `mask.mean()` is 0.5, the share of days above 300.
3. We copy before assigning so the original stays intact, then replace everything above 500 by 500; the 1500 becomes 500.

Problem 16.4.2 Fancy indexing: pick rows by position, pick individual cells, take a sub-grid, and see that it is a copy.

Idea	A list of integers selects those positions in that order. Two lists select pairs. `np.ix_` turns two lists into a rectangle.

```
import numpy as np
m = np.arange(20).reshape(4, 5)
print(m[[3, 0]])
print(m[[0, 2], [1, 4]])
print(m[np.ix_([0, 2], [1, 4])])
sub = m[[0, 1]]
sub[0, 0] = 999
print(m[0, 0], sub.base is None)
view = m[:2]
view[0, 0] = 999
print(m[0, 0])
```

```
[[15 16 17 18 19]
 [ 0  1  2  3  4]]
[ 1 14]
[[ 1  4]
 [11 14]]
0 True
999
```

Line by line.

1. `m[[0, 2], [1, 4]]` pairs the lists: cell (0,1) and cell (2,4), giving `1` and `14`. The result is 1D, not a rectangle.
2. To get the rectangle of rows 0, 2 and columns 1, 4, use `np.ix_`: the result is a 2 by 2 array `[[1 4] [11 14]]`.
3. `sub = m[[0, 1]]` is fancy indexing, so it is a copy. Writing 999 into `sub` leaves `m[0, 0]` at 0, and `sub.base is None` shows it owns its memory.

4. `view = m[:2]` is a slice, so it is a view. Writing 999 through it changes `m[0, 0]` to 999.

COMMON MISTAKE

Wrong: `m[m[:, 0] > 5][:, 1] = 0` (chained indexing). **Symptom:** nothing changes in `m`, with no error, because `m[mask]` is a copy and you assigned into the copy. **Fix:** do the whole selection in one assignment: `m[m[:, 0] > 5, 1] = 0`.

REAL JOB

A common pipeline step is "set every negative sensor reading to NaN, then drop incomplete rows". In NumPy it is two lines: `x[x < 0] = np.nan` then `x = x[~np.isnan(x).any(axis=1)]`. You will see this pattern again in 16.8.

16.5 Broadcasting: Combining Arrays of Different Shapes

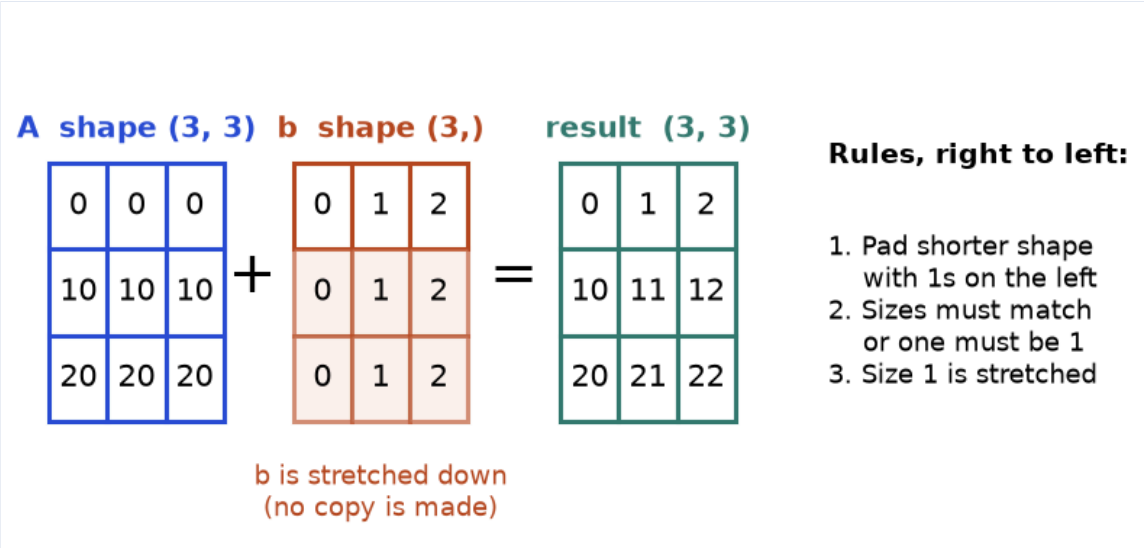
INTUITION

Imagine painting a wall with a stencil that is only one row tall. To cover the whole wall you slide the same stencil down, row after row, without making copies of it. Broadcasting does exactly that: when one array is smaller, NumPy conceptually stretches it to fit the other, but reuses the same numbers.

Broadcasting is the set of rules that lets `a + b` work when the shapes differ. It is the single most useful idea in NumPy, and the source of the most puzzling errors, so learn the three rules by heart.

KEY IDEA

The three rules. Line the shapes up from the right. (1) If one shape is shorter, pad it with 1s on the left. (2) Compare axis by axis: sizes must be equal, or one of them must be 1. (3) An axis of size 1 is stretched to the other size. If any axis fails rule 2, NumPy raises `ValueError: operands could not be broadcast together`.



Adding a row of shape (3,) to a (3, 3) array: the row is reused for every row of A, with no copy.

How to read it: the blue grid holds 0, 10, 20 down its rows. The orange row `[0 1 2]` has shape `(3,)`, which rule 1 pads to `(1, 3)`; rule 3 stretches the size-1 row axis to 3 (the faded copies). Each result cell is the matching blue cell plus the orange value in its column, so row 1 becomes `10 11 12`.

Problem 16.5.1 Predict and check the result shapes for four combinations.

Step 1	(3, 3) with (3,): pad to (1, 3), stretch to (3, 3). Fine.
Step 2	(3, 1) with (1, 4): both stretch, result (3, 4). This makes an "outer" table.
Step 3	(3,) with (4,): neither is 1, so it fails.
Step 4	(2, 3) with (2,): aligned from the right, 3 vs 2 fails. Fix with a new axis: (2, 1).

```
import numpy as np
A = np.array([[0, 0, 0], [10, 10, 10], [20, 20, 20]])
b = np.array([0, 1, 2])
print(A + b)
col = np.array([[1], [2], [3]])
row = np.array([[10, 20, 30, 40]])
print((col * row).shape)
print(col * row)
try:
    np.ones(3) + np.ones(4)
except ValueError as e:
    print("ValueError:", e)
m = np.arange(6).reshape(2, 3)
w = np.array([100, 200])
try:
    m + w
except ValueError as e:
    print("ValueError:", e)
print(m + w[:, None])
```

```
[[ 0  1  2]
 [10 11 12]
 [20 21 22]]
(3, 4)
[[ 10  20  30  40]
 [ 20  40  60  80]
 [ 30  60  90 120]]
ValueError: operands could not be broadcast together with shapes (3,) (4,)
ValueError: operands could not be broadcast together with shapes (2,3) (2,)
[[100 101 102]
 [203 204 205]]
```

Line by line.

1. `col` has shape (3, 1) and `row` has shape (1, 4). Each stretches along its size-1 axis, so the product has shape (3, 4): a multiplication table of 1..3 by 10..40.
2. Shapes (3,) and (4,) cannot match, and NumPy says so, printing both shapes. Reading that message as "which two shapes?" is the fastest way to debug.
3. The matrix `m` is (2, 3) and `w` is (2,). Aligned from the right, 3 meets 2 and the rule fails: the error is the classic one.
4. `w[:, None]` turns `w` into a column of shape (2, 1). Now row 0 gets 100 added and row 1 gets 200 added.

COMMON MISTAKE

Wrong: subtract a vector of row means from a matrix with `X - X.mean(axis=1)`. **Symptom:** an error, or worse, silently wrong values when the matrix happens to be square (the vector gets matched to columns instead of rows). **Fix:** keep the axis with `X - X.mean(axis=1, keepdims=True)`, which makes the mean shape (n, 1).

16.6 Ufuncs, Reductions and the axis Argument

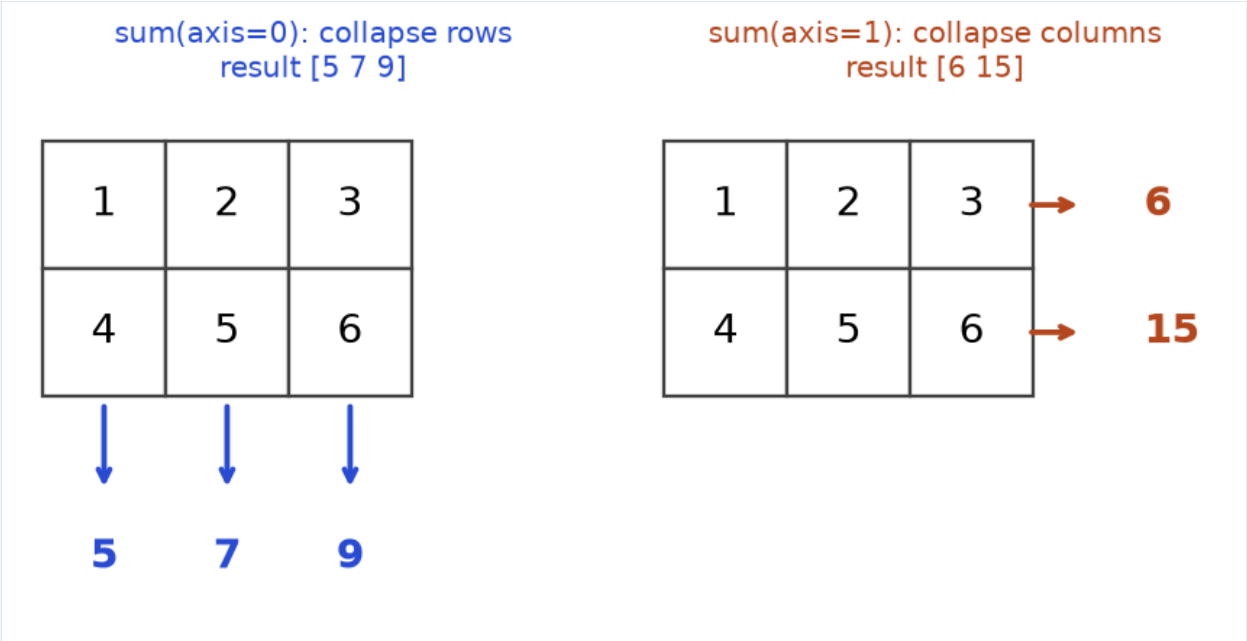
A **ufunc** (universal function) is a function that applies the same operation to every element, in compiled code, with broadcasting built in. `+`, `*`, `np.sqrt` and `np.exp` are all ufuncs. A **reduction** collapses an axis into one number, such as a sum or maximum.

Function	What it does (plain English)	Minimal example
`+ - * / // % **`	Element-wise arithmetic (`/` is always float division)	`a / b`
`np.abs`	Absolute value	`np.abs([-2, 3])`

Function	What it does (plain English)	Minimal example
`np.sqrt, np.square`	Square root, square	`np.sqrt(16.0)`
`np.exp, np.expm1`	e to the power, and accurate e^x - 1 for tiny x	`np.exp(1)`
`np.log, np.log2, np.log10, np.log1p`	Logarithms (`log1p` is accurate for tiny x)	`np.log(np.e)`
`np.sin, np.cos, np.tan, np.arctan2`	Trigonometry in radians	`np.sin(np.pi / 2)`
`np.round, np.floor, np.ceil, np.trunc`	Rounding family (`round` rounds halves to even)	`np.round(2.5)` gives `2.0`
`np.clip(a, lo, hi)`	Limit values to a range	`np.clip(a, 0, 1)`
`np.maximum, np.minimum`	Element-wise larger or smaller of two arrays	`np.maximum(a, 0)` (ReLU)
`np.isnan, np.isinf, np.isfinite`	Boolean tests	`np.isnan(a)`
`np.isclose, np.allclose`	Compare floats with tolerance	`np.isclose(0.1 + 0.2, 0.3)`
`np.sum, np.prod`	Total, product	`a.sum(axis=0)`
`np.mean, np.median`	Average, middle value	`a.mean()`
`np.std, np.var`	Standard deviation, variance (`ddof=1` for sample)	`a.std(ddof=1)`
`np.min, np.max, np.ptp`	Smallest, largest, range (max - min)	`np.ptp(a)`
`np.argmin, np.argmax`	Position of smallest or largest	`a.argmax()`
`np.cumsum, np.cumprod`	Running total, running product	`np.cumsum(a)`
`np.diff`	Differences between neighbours	`np.diff([1, 4, 9])`
`np.percentile, np.quantile`	Value below which a share of data falls	`np.percentile(a, 90)`
`np.any, np.all`	Is any / is every element True	`(a > 0).all()`
`np.corrcoef, np.cov`	Correlation and covariance matrices	`np.corrcoef(x, y)`
`np.add.at(a, idx, v)`	Unbuffered add: repeated indexes accumulate	histogram by hand

The axis argument

Every reduction accepts `axis` . The rule that never fails: **the axis you name is the one that disappears**. For a table of shape (rows, columns), `axis=0` collapses the rows (you get one answer per column) and `axis=1` collapses the columns (one answer per row).



sum(axis=0) squashes the table downward and leaves column totals. sum(axis=1) squashes it sideways and leaves row totals.

How to read it: in the left panel the arrows run down each column, adding 1+4, 2+5, 3+6 to get 5, 7, 9. The arrow direction is the axis that is being consumed. In the right panel the arrows run across each row, giving 6 and 15. With three or more dimensions the same rule applies: the named axis vanishes and the other axes remain.

Problem 16.6.1 A 3D array holds 2 stores, 3 days and 4 products of sales. Answer three business questions with the axis rule.

Shape	(store, day, product) = (2, 3, 4).
Q1	Total per store: collapse day and product, `axis=(1, 2)` gives shape (2,).
Q2	Total per product across everything: collapse store and day, `axis=(0, 1)` gives (4,).
Q3	Best product on each store-day: `argmax(axis=2)` gives (2, 3).

```
import numpy as np
sales = np.arange(24).reshape(2, 3, 4)
print(sales.sum(axis=(1, 2)))
print(sales.sum(axis=(0, 1)))
print(sales.argmax(axis=2))
print(sales.mean(axis=1).shape, sales.mean(axis=1, keepdims=True).shape)
print(sales.cumsum(axis=1)[0])
print(np.diff(sales, axis=2)[0, 0])
```

```
[ 66 210]
[60 66 72 78]
[[3 3 3]
 [3 3 3]]
(2, 4) (2, 1, 4)
[[ 0  1  2  3]
 [ 4  6  8 10]
 [12 15 18 21]]
[1 1 1]
```

Line by line.

1. Store 0 holds 0..11 (sum 66) and store 1 holds 12..23 (sum 210). Naming a tuple of axes collapses several at once.
2. Summing store and day leaves one total per product: the four sums are 60, 66, 72, 78.

3. Without `keepdims` the day axis vanishes: (2, 4). With it, a length-1 axis stays: (2, 1, 4). Keeping it lets the result broadcast back against the original.
4. `cumsum(axis=1)` accumulates across days within each store; we print store 0, whose last row is the 3-day running total.

COMMON MISTAKE

Wrong: `np.mean(x)` on a 2D array when you wanted per-column means, or `np.std(x)` to estimate a sample's spread. **Symptom:** a single number instead of a vector; and a standard deviation that is slightly too small. **Fix:** pass `axis=0`, and use `ddof=1` for the sample standard deviation (NumPy defaults to `ddof=0`, the population version; pandas defaults to 1).

16.7 Reshape, Transpose, Stack and Split

Real data rarely arrives in the shape a function wants. These tools rearrange arrays; most of them change only the header, so they are cheap.

Function	What it does (plain English)	Minimal example
`a.reshape(r, c)`	Same numbers, new shape (total size must match)	`np.arange(6).reshape(2, 3)`
`a.reshape(-1, 3)`	The `-1` means "work out this size for me"	`a.reshape(-1, 3)`
`a.ravel()`	Flatten to 1D (view when possible)	`m.ravel()`
`a.flatten()`	Flatten to 1D, always a copy	`m.flatten()`
`a.T`, `np.transpose(a, axes)`	Swap or permute axes	`m.T`
`a[:, None]`, `np.newaxis`	Insert an axis of length 1	`v[:, None]`
`np.squeeze(a)`	Remove axes of length 1	`np.squeeze(np.ones((1, 3)))`
`np.concatenate([a, b], axis)`	Join along an existing axis	`np.concatenate([a, b], axis=0)`
`np.stack([a, b], axis)`	Join along a NEW axis	`np.stack([a, b])`
`np.vstack, np.hstack, np.dstack`	Stack vertically, horizontally, depth-wise	`np.vstack([a, b])`
`np.column_stack`	1D arrays become columns of a matrix	`np.column_stack([x, y])`
`np.split(a, n)`	Cut into n equal pieces (must divide evenly)	`np.split(a, 3)`
`np.array_split(a, n)`	Cut into n nearly equal pieces	`np.array_split(a, 3)`
`np.flip(a, axis)`	Reverse order along an axis	`np.flip(m, axis=1)`
`np.roll(a, k)`	Shift elements circularly	`np.roll(a, 1)`
`np.pad(a, width)`	Add a border of zeros (or edge values)	`np.pad(m, 1)`
`np.append(a, v)`, `np.insert`, `np.delete`	Return a new array with items added or removed (slow in a loop)	`np.delete(a, 0)`

Problem 16.7.1 Combine and cut arrays: concatenate vs stack, then split a dataset into folds.

Idea	`concatenate` glues along an existing axis; `stack` adds a new one. `array_split` tolerates sizes that do not divide evenly.

```
import numpy as np
a = np.array([[1, 2], [3, 4]])
b = np.array([[5, 6], [7, 8]])
print(np.concatenate([a, b], axis=0).shape)
print(np.concatenate([a, b], axis=1).shape)
print(np.stack([a, b]).shape, np.stack([a, b], axis=2).shape)
print(np.vstack([a, b]).shape, np.hstack([a, b]).shape)
x = np.array([1, 2, 3])
y = np.array([10, 20, 30])
print(np.column_stack([x, y]))
parts = np.array_split(np.arange(10), 3)
print([p.tolist() for p in parts])
print(np.roll(np.arange(5), 2), np.pad(a, 1).shape)
```

```
(4, 2)
(2, 4)
(2, 2, 2) (2, 2, 2)
(4, 2) (2, 4)
[[ 1 10]
 [ 2 20]
 [ 3 30]]
[[0, 1, 2, 3], [4, 5, 6], [7, 8, 9]]
[3 4 0 1 2] (4, 4)
```

Line by line.

1. `stack` creates a new axis: at the front it gives (2, 2, 2) as "two matrices"; with `axis=2` the new axis is last.
2. `column_stack` places each 1D array as a column: rows are `[1 10]`, `[2 20]`, `[3 30]`.
3. Ten items into three folds cannot be equal, so `array_split` gives sizes 4, 3, 3 (the first folds get the extra item).
4. `np.roll(..., 2)` shifts right by two, wrapping the end around to the front. `pad(a, 1)` adds a one-cell border: (2, 2) becomes (4, 4).

16.8 Sorting, Searching, Unique, where and NaN Handling

Function	What it does (plain English)	Minimal example
`np.sort(a)`	Sorted copy (ascending)	`np.sort([3, 1, 2])`
`a.sort()`	Sort in place (returns None)	`a.sort()`
`np.argsort(a)`	The positions that would sort the array	`np.argsort([30, 10, 20])` gives `[1 2 0]`
`np.argpartition(a, k)`	Positions version; fast top-k	`np.argpartition(-a, 2)`
`np.searchsorted(a, v)`	Where would `v` be inserted in sorted `a` (binary search)	`np.searchsorted(a, 5)`
`np.unique(a)`	Sorted distinct values	`np.unique([2, 1, 2])`
`np.unique(a, return_counts=True)`	Distinct values and how often each occurs	frequency table
`np.unique(a, return_inverse=True)`	Also a code (0, 1, 2...) for each original element	label encoding
`np.bincount(ints)`	Count of each non-negative integer (very fast)	`np.bincount([0, 1, 1, 3])`

Function	What it does (plain English)	Minimal example
`np.digitize(a, bins)`	Which bin each value falls into	`np.digitize(x, [0, 10, 20])`
`np.histogram(a, bins)`	Counts and bin edges	`np.histogram(x, bins=5)`
`np.where(cond, x, y)`	Choose x where True, else y	`np.where(a > 0, a, 0)`
`np.select(conds, choices)`	Several conditions, first True wins	grade bands
`np.argwhere(cond)`	Coordinates of True cells, one row each	`np.argwhere(m > 5)`
`np.isin(a, values)`	Is each element in this set	`np.isin(a, [1, 2])`
`np.nanmean, np.nansum, np.nanstd, np.nanmax, np.nanmin`	Statistics that skip NaN	`np.nanmean(x)`
`np.nan_to_num(a)`	Replace NaN and infinity by numbers	`np.nan_to_num(x, nan=0.0)`
`np.isnan(a)`	Boolean mask of NaN	`x[~np.isnan(x)]`

Problem 16.8.1 Sort, rank, and find the three best students without sorting everything.

Idea	`argsort` returns positions, which you then use to reorder other arrays that must stay aligned with the scores. `argpartition` finds the top k in linear time when you do not need a full sort.

```
import numpy as np
names = np.array(["Asha", "Ben", "Chen", "Dana", "Eli"])
score = np.array([72, 95, 88, 95, 60])
order = np.argsort(score)
print(order, names[order])
desc = np.argsort(-score, kind="stable")
print(names[desc])
top3 = np.argpartition(-score, 3)[:3]
print(sorted(top3.tolist()), names[top3[np.argsort(-score[top3])]])
ranks = np.empty_like(order)
ranks[order] = np.arange(len(score))
print(ranks)
sorted_scores = np.sort(score)
print(np.searchsorted(sorted_scores, [60, 90, 100]))
```

```
[4 0 2 1 3] ['Eli' 'Asha' 'Chen' 'Ben' 'Dana']
['Ben' 'Dana' 'Chen' 'Asha' 'Eli']
[1, 2, 3] ['Dana' 'Ben' 'Chen']
[1 3 2 4 0]
[0 3 5]
```

Line by line.

1. `argsort` returns `[4 0 2 1 3]`: position 4 (score 60) is smallest. Indexing the names with it sorts them in the same order, keeping names and scores aligned.
2. Negating the scores sorts high to low. `kind="stable"` keeps tied items (Ben and Dana, both 95) in their original order, so Ben comes before Dana.
3. Ranks: assigning `ranks[order] = 0..n-1` inverts the permutation. Asha (72) is the second lowest, rank 1.
4. `searchsorted` says where each value would be inserted into the sorted array: 60 at 0, 90 at 3, 100 at 5.

Problem 16.8.2 Count, group and bin with `unique`, `bincount`, `digitize` and `where`.

Idea	`return_counts` gives a frequency table; `return_inverse` gives integer codes; `digitize` turns numbers into bin numbers; `np.select` labels them.

```
import numpy as np
city = np.array(["Pune", "Oslo", "Pune", "Lima", "Oslo", "Pune"])
vals, counts = np.unique(city, return_counts=True)
print(vals, counts)
vals, codes = np.unique(city, return_inverse=True)
print(codes)
print(np.bincount(codes))
spend = np.array([0.0, 4.5, 12.0, 25.0, 99.0, 7.0])
bins = np.array([5, 20])
idx = np.digitize(spend, bins)
print(idx)
label = np.select([spend < 5, spend < 20], ["low", "mid"], default="high")
print(label)
print(np.where(spend > 10, "big", "small"))
print(np.isin(city, ["Lima", "Oslo"]))
```

```
['Lima' 'Oslo' 'Pune'] [1 2 3]
[2 1 2 0 1 2]
[1 2 3]
[0 0 1 2 2 1]
['low' 'low' 'mid' 'high' 'high' 'mid']
['small' 'small' 'big' 'big' 'big' 'small']
[False  True False  True  True False]
```

Line by line.

1. With `return_inverse` each original city becomes its index in the distinct list (Lima 0, Oslo 1, Pune 2), so `city` becomes `[2 1 2 0 1 2]`.
2. `digitize(spend, [5, 20])` returns 0 for values below 5, 1 for 5 up to (not including) 20, and 2 for 20 or more.
3. `np.select` tests the conditions in order, and the first True wins; anything else gets the default `high`. Note that 4.5 is low, 7.0 is mid.
4. Three-argument `where` is a vectorised if/else and works on strings too. `isin` gives a mask for membership.

NaN handling

NaN (Not a Number) is how a float array stores a missing or undefined value. Its two key properties surprise people: NaN is not equal to anything, even itself, and any arithmetic with NaN gives NaN. Integer arrays cannot hold NaN; a column with gaps must be float.

Problem 16.8.3 Compute honest statistics on a series with gaps, and fill the gaps three ways.

Idea	Plain `mean` poisons the answer; `nanmean` skips NaN. To fill, build a mask first, then choose a fill rule.

```
import numpy as np
x = np.array([4.0, np.nan, 6.0, 10.0, np.nan])
print(np.nan == np.nan, np.isnan(x))
print(x.mean(), np.nanmean(x), np.nanstd(x))
print(np.nanmax(x), np.nanargmax(x))
mask = np.isnan(x)
filled_mean = np.where(mask, np.nanmean(x), x)
print(filled_mean)
print(np.nan_to_num(x, nan=-1.0))
idx = np.arange(len(x))
interp = x.copy()
interp[mask] = np.interp(idx[mask], idx[~mask], x[~mask])
print(interp)
print(np.count_nonzero(mask), mask.mean())
```

```
False [False  True False False  True]
nan 6.666666666666667 2.494438257849294
10.0 3
[ 4.          6.66666667  6.         10.          6.66666667]
[ 4. -1.  6. 10. -1.]
[ 4.  5.  6. 10. 10.]
2 0.4
```

Line by line.

1. `np.nan == np.nan` is `False`. Use `np.isnan` to detect missing values; `x == np.nan` never finds anything.
2. `x.mean()` is `nan` because one missing value poisons the sum. `nanmean` averages the three present values: (4+6+10)/3 = 6.667. `nanstd` likewise ignores the gaps.
3. Linear interpolation: `np.interp(positions_to_fill, known_positions, known_values)`. The gap at index 1 lands between 4 and 6, so 5. The gap at index 4 is beyond the last known point, so `interp` holds the last value, 10.
4. Finally, we count the missing: two of five, a proportion of 0.4.

COMMON MISTAKE

Wrong: `x[x == np.nan] = 0`. **Symptom:** nothing is replaced, no error. **Fix:** `x[np.isnan(x)] = 0`. A related trap: `np.sort` puts NaN at the end, and `np.max` returns NaN if any value is NaN, so use `nanmax` when you want to ignore gaps.

16.9 Linear Algebra with np.linalg

Linear algebra is the maths of tables of numbers, and it is the engine inside regression, PCA, recommender systems and neural networks (see the Mathematics document, Chapters 5 to 9, for the theory). NumPy hands the heavy work to optimised BLAS and LAPACK libraries.

Function	What it does (plain English)	Minimal example
`A @ B`, `np.matmul`	Matrix product (rows of A with columns of B)	`A @ B`
`np.dot(a, b)`	Dot product for vectors; matrix product for 2D	`np.dot(u, v)`
`np.vdot, np.inner, np.outer`	Flat dot, inner product, outer product	`np.outer(u, v)`
`np.cross(u, v)`	Cross product of 3-vectors	`np.cross(u, v)`
`np.trace(A)`	Sum of the diagonal	`np.trace(A)`
`np.linalg.norm(x)`	Length of a vector, or matrix norm	`np.linalg.norm([3, 4])` gives `5.0`
`np.linalg.det(A)`	Determinant	`np.linalg.det(A)`

Function	What it does (plain English)	Minimal example
`np.linalg.inv(A)`	Matrix inverse (rarely what you need)	`np.linalg.inv(A)`
`np.linalg.solve(A, b)`	Solve A x = b (use instead of inverse)	`np.linalg.solve(A, b)`
`np.linalg.lstsq(X, y)`	Least-squares fit (regression)	`np.linalg.lstsq(X, y, rcond=None)`
`np.linalg.eig, eigh`	Eigenvalues and eigenvectors (`eigh` for symmetric matrices)	`np.linalg.eigh(C)`
`np.linalg.svd`	Singular value decomposition	`U, s, Vt = np.linalg.svd(X)`
`np.linalg.qr, cholesky`	Other factorisations	`np.linalg.qr(A)`
`np.linalg.matrix_rank`	Number of independent rows	`np.linalg.matrix_rank(A)`
`np.linalg.cond`	Condition number: how fragile the system is	`np.linalg.cond(A)`
`np.linalg.pinv`	Pseudo-inverse for non-square or singular matrices	`np.linalg.pinv(X)`
`np.linalg.matrix_power`	A multiplied by itself n times	`np.linalg.matrix_power(A, 3)`

Problem 16.9.1 Multiply matrices, solve a small system, and compare `solve` with the inverse. Two shops sell coffee and tea: 2 coffees plus 1 tea cost 7, and 1 coffee plus 3 teas cost 11. Find the prices.

Setup	A = [[2, 1], [1, 3]], b = [7, 11], unknown x = [coffee, tea].
By hand	From the first equation tea = 7 - 2 coffee. Then coffee + 3(7 - 2 coffee) = 11, so 21 - 5 coffee = 11, coffee = 2, tea = 3.

```
import numpy as np
A = np.array([[2.0, 1.0], [1.0, 3.0]])
b = np.array([7.0, 11.0])
x = np.linalg.solve(A, b)
print(x)
print(np.allclose(A @ x, b))
print(np.linalg.det(A), np.linalg.matrix_rank(A))
Ainv = np.linalg.inv(A)
print(np.round(A @ Ainv, 10))
B = np.array([[0.0, 1.0], [1.0, 0.0]])
print(A @ B)
print(A * B)
print(np.linalg.norm([3, 4]), np.trace(A))
```

```
[2. 3.]
True
5.000000000000001 2
[[ 1.  0.]
 [-0.  1.]]
[[1. 2.]
 [3. 1.]]
[[0. 1.]
 [1. 0.]]
5.0 5.0
```

Line by line.

1. `solve(A, b)` returns `[2. 3.]`, matching the hand calculation: coffee 2, tea 3.
2. Always verify by multiplying back: `A @ x` equals `b` up to float tolerance, so `allclose` is `True`.
3. `A @ B` is the matrix product (here B swaps the columns of A). `A * B` is the element-wise product, a different thing.

4. `norm([3, 4])` is the length of the vector, 5. The trace is 2 + 3 = 5.

COMMON MISTAKE

Wrong: `x = np.linalg.inv(A) @ b` to solve a system, or `np.linalg.inv` on a nearly singular matrix. **Symptom:** slower, less accurate answers, or huge garbage numbers without an error. **Fix:** `np.linalg.solve(A, b)`, and check `np.linalg.cond(A)`: very large values (around 1e10 and above) mean results are unreliable. If the matrix is exactly singular, `inv` raises `LinAlgError`.

16.10 Random Numbers: Generator and Seeds

Modern NumPy uses a **Generator**: an object that produces random numbers from a seeded stream. Create it once with `rng = np.random.default_rng(seed)` and call methods on it. The old global functions (`np.random.seed`, `np.random.rand`) still work but are legacy; the Generator is the recommended API because it is faster, has better statistical quality, and avoids hidden global state.

Method	What it does (plain English)	Minimal example
`default_rng(seed)`	Make a generator; same seed gives same stream	`rng = np.random.default_rng(42)`
`rng.random(size)`	Uniform floats in [0, 1)	`rng.random(3)`
`rng.integers(lo, hi, size)`	Random integers, `hi` excluded	`rng.integers(1, 7, 10)` (dice)
`rng.normal(mu, sigma, size)`	Bell-curve numbers	`rng.normal(0, 1, 5)`
`rng.standard_normal(size)`	Mean 0, standard deviation 1	`rng.standard_normal((3, 2))`
`rng.uniform(a, b, size)`	Uniform floats between a and b	`rng.uniform(5, 10, 4)`
`rng.choice(a, size, replace, p)`	Sample from values, optional weights	`rng.choice(5, 3, replace=False)`
`rng.shuffle(a)`	Shuffle IN PLACE	`rng.shuffle(a)`
`rng.permutation(n)`	Shuffled copy, or a shuffled range	`rng.permutation(10)`
`rng.binomial(n, p, size)`	Number of successes in n trials	`rng.binomial(10, 0.3, 5)`
`rng.poisson(lam, size)`	Event counts at a given rate	`rng.poisson(4, 5)`
`rng.exponential(scale, size)`	Waiting times	`rng.exponential(2.0, 5)`
`rng.multivariate_normal(mean, cov, size)`	Correlated normals	fake correlated features
`rng.spawn(n)`	Independent child generators (parallel jobs)	`children = rng.spawn(4)`
`np.random.SeedSequence(n).spawn(k)`	Independent seeds for workers	reproducible parallel runs

Problem 16.10.1 Show that the same seed repeats, different seeds differ, and the main draws work. Simulate 10 dice rolls and a weighted choice.

Idea A generator remembers its position in the stream. A new generator with the same seed starts again from the beginning.

```
import numpy as np
r1 = np.random.default_rng(42)
r2 = np.random.default_rng(42)
print(r1.random(3), r2.random(3))
print(np.array_equal(np.random.default_rng(1).random(5), np.random.default_rng(2).random(5)))
rng = np.random.default_rng(0)
rolls = rng.integers(1, 7, size=10)
print(rolls, rolls.min() >= 1, rolls.max() <= 6)
print(rng.choice(["red", "green", "blue"], size=8, p=[0.6, 0.3, 0.1]))
print(rng.choice(10, size=4, replace=False))
z = rng.normal(100, 15, size=100000)
print(round(float(z.mean()), 1), round(float(z.std()), 1))
```

```
[0.77395605 0.43887844 0.85859792] [0.77395605 0.43887844 0.85859792]
False
[6 4 4 2 2 1 1 1 2 5] True True
['blue' 'green' 'green' 'red' 'blue' 'green' 'red' 'green']
[0 6 3 7]
100.0 15.0
```

Line by line.

1. Two generators made from seed 42 give identical numbers, so `r1.random(3)` and `r2.random(3)` print the same three values. This is what makes experiments reproducible.
2. `integers(1, 7, size=10)` gives dice rolls: the upper limit 7 is excluded. The two checks confirm the range.
3. `choice` with `p` draws with weights: mostly red. Probabilities must sum to 1.
4. One hundred thousand draws from a normal with mean 100 and standard deviation 15 land very close to those values (100.0 and 15.0 after rounding).

COMMON MISTAKE

Wrong: creating a new generator inside a function that you call in a loop, such as `np.random.default_rng(0).normal()` on each iteration. **Symptom:** every call returns exactly the same number, so your "simulation" has no randomness. **Fix:** create `rng` once, outside the loop, and pass it into your functions as an argument. For parallel workers, give each its own child from `rng.spawn(n)`, never the same seed.

16.11 einsum: Index Notation for Array Algebra

`np.einsum` writes a calculation as a recipe of axis letters. Letters that appear in the inputs but not after the arrow `->` are **summed over**; letters that appear after the arrow stay. Once you can read it, many tangled `reshape`, `transpose` and `sum` chains become one line.

Recipe	Meaning	Same as
`"i,i->"`	Multiply matching items and add: dot product	`np.dot(u, v)`
`"i,j->ij"`	Every pair product: outer product	`np.outer(u, v)`
`"ij->ji"`	Swap axes: transpose	`A.T`
`"ij->"`	Add everything	`A.sum()`
`"ij->i"`	Sum over j: row sums	`A.sum(axis=1)`
`"ii->"`	Diagonal summed: trace	`np.trace(A)`
`"ij,jk->ik"`	Matrix product	`A @ B`
`"bij,bjk->bik"`	A batch of matrix products	`np.matmul`

Recipe	Meaning	Same as
`"ij,ij->i"`	Row-by-row dot products	`(A * B).sum(axis=1)`

Problem 16.11.1 Check each recipe against the plain NumPy equivalent, then do a batch of matrix products and a row-wise dot product.

Rule	Same letter on two inputs means those axes are matched. Missing from the output means summed.

```
import numpy as np
rng = np.random.default_rng(0)
A = rng.integers(0, 5, (3, 4))
B = rng.integers(0, 5, (4, 2))
u = np.array([1, 2, 3])
v = np.array([4, 5, 6])
print(np.einsum("i,i->", u, v), u @ v)
print(np.array_equal(np.einsum("ij,jk->ik", A, B), A @ B))
print(np.array_equal(np.einsum("ij->i", A), A.sum(axis=1)))
print(np.einsum("i,j->ij", u, v))
batch = rng.integers(0, 3, (5, 2, 3))
other = rng.integers(0, 3, (5, 3, 4))
print(np.einsum("bij,bjk->bik", batch, other).shape)
P = rng.random((4, 3))
Q = rng.random((4, 3))
print(np.allclose(np.einsum("ij,ij->i", P, Q), (P * Q).sum(axis=1)))
```

```
32 32
True
True
[[ 4  5  6]
 [ 8 10 12]
 [12 15 18]]
(5, 2, 4)
True
```

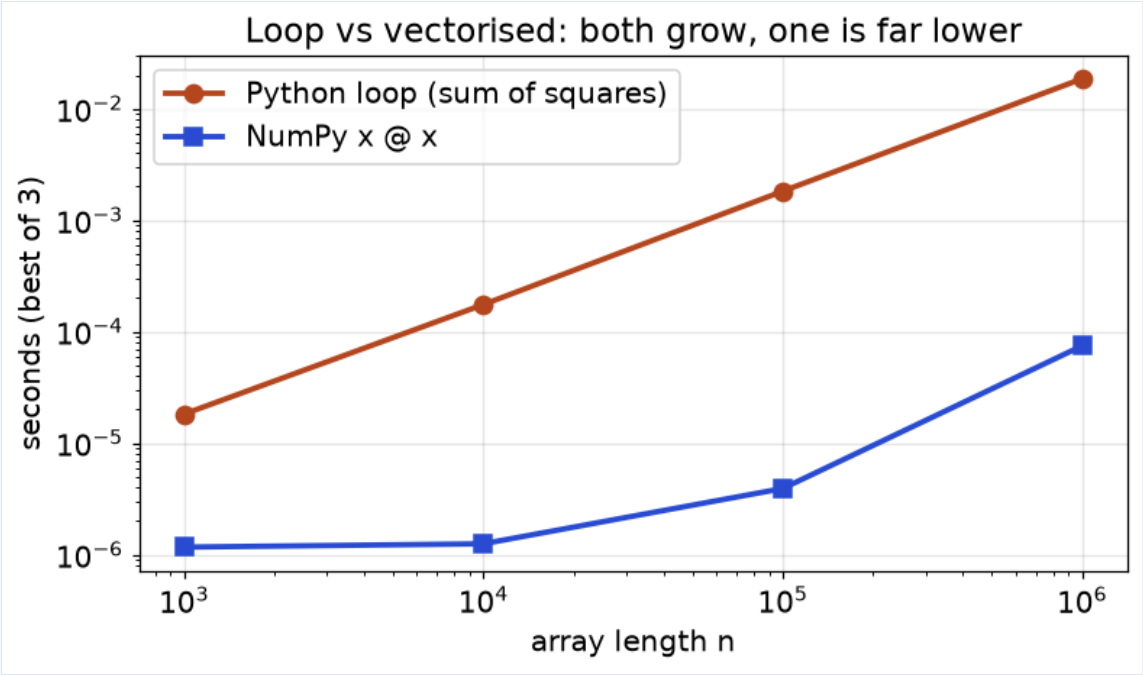
Line by line.

1. `"i,i->"` matches the two vectors item by item and sums: 4 + 10 + 18 = 32. The empty output means a single number.
2. `"ij,jk->ik"`: the shared letter `j` is summed, which is exactly the matrix product. The check prints `True`.
3. `"i,j->ij"` has no shared letter, so nothing is summed and every pair multiplies: the 3 by 3 outer product.
4. With a leading batch letter `b` kept in the output, five independent (2, 3) by (3, 4) products give shape (5, 2, 4).

INTUITION	Read einsum as "for every combination of the letters on the right-hand side, add up the product of the inputs over the remaining letters". For attention scores in a transformer, `"bhqd,bhkd->bhqk"` is just that: for each batch and head, dot every query with every key.

16.12 Vectorisation vs Loops, with Timings

Vectorising means replacing a Python `for` loop with array operations. The loop is slow not because Python is careless but because for each element it must look up types, create a Python float object and call the interpreter. A NumPy operation does that bookkeeping once, then runs a tight compiled loop over raw numbers.



Time to compute a sum of squares as the array grows (measured on one laptop, best of 3; your numbers will differ). Both axes are logarithmic.

How to read it: each dot is one measurement. The horizontal axis is how many numbers; the vertical axis is seconds, and each gridline is ten times bigger than the last. The loop line climbs steadily because its work is proportional to `n`. The NumPy line starts flat, because for small arrays the fixed cost of one call dominates, and then rises. The gap is the constant factor you buy by vectorising: for a million numbers it was more than a hundredfold in these measurements.

Problem 16.12.1 Time a loop against a vectorised version of the same job, on three tasks, and confirm the answers match. The code prints only facts that do not change between runs; the measured numbers are given in the text below.

Setup	One million random floats. Tasks: scale-and-shift every value, sum of squares, and growing an array one item at a time.
Method	Use `time.perf_counter` and keep the best of several repeats to reduce noise.

```
import time
import numpy as np

def best(f, repeats=3):
    t = float("inf")
    for _ in range(repeats):
        start = time.perf_counter()
        f()
        t = min(t, time.perf_counter() - start)
    return t

rng = np.random.default_rng(0)
x = rng.random(1_000_000)
xl = x.tolist()

def loop_scale():
    out = []
    for v in xl:
        out.append(v * 2 + 1)
    return out

t_loop = best(loop_scale)
t_vec = best(lambda: x * 2 + 1)
print("same result:", np.allclose(loop_scale(), x * 2 + 1))
print("vectorised at least 10x faster:", t_loop / t_vec > 10)

t_sum_loop = best(lambda: sum(v * v for v in xl))
t_sum_vec = best(lambda: x @ x)
print("sum of squares equal:", np.isclose(sum(v * v for v in xl), x @ x))
print("at least 10x faster:", t_sum_loop / t_sum_vec > 10)

def grow():
    a = np.array([])
    for i in range(10000):
        a = np.append(a, i)
    return a

def preallocate():
    a = np.empty(10000)
    for i in range(10000):
        a[i] = i
    return a

print("growing is slower than preallocating:", best(grow) > best(preallocate))
```

```
same result: True
vectorised at least 10x faster: True
sum of squares equal: True
at least 10x faster: True
growing is slower than preallocating: True
```

Line by line.

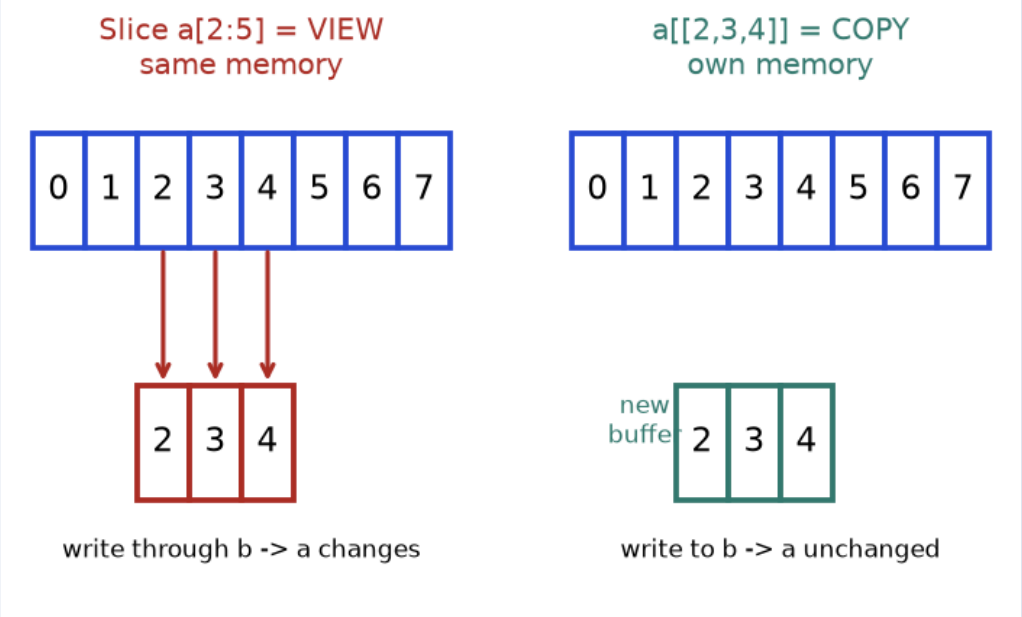
1. `best` runs a function a few times and keeps the fastest run. The minimum is the best estimate of the cost, because noise from other programs only ever slows a run down.
2. The scaling task: a loop with `append` against `x * 2 + 1`. The results agree, and the vector form is more than ten times faster.
3. Growing an array with `np.append` copies the whole array every time, so 10,000 appends do about 50 million copies of items.

Measured on the laptop used for this chapter (they vary): scale-and-shift 46 ms in the loop against 0.7 ms vectorised; sum of squares 20 ms against 0.1 ms; 10,000 appends 19 ms against 0.35 ms preallocated; a 300-point pairwise distance table 92 ms with nested loops against 1.1 ms vectorised.

COMMON MISTAKE	**Wrong:** `np.sum(my_list)` or `np.array(my_list).sum()` for a one-off sum of a small Python list. **Symptom:** slower than the built-in `sum`, because NumPy first converts the whole list to an array (measured here: about 17 ms against 2.7 ms for a million floats). **Fix:** convert once and keep the data as an array, or use `sum` for a single pass over a list.

16.13 Views, Copies and Memory Pitfalls

Because arrays share buffers, the question "does this change my original?" matters. The answer follows a simple rule.



A slice is a window onto the original numbers. Fancy or boolean indexing copies the chosen numbers into new memory.

How to read it: on the left the arrows mean the red array points into the blue one: no new numbers exist, so writing through the red array edits the blue one. On the right there are no arrows: the teal array owns separate memory and edits stay private.

Operation	View or copy?	How to check
Basic slice `a[1:5]`, `a[::2]`, `a[:, 0]`	View	`np.shares_memory(a, b)`
`reshape`, `ravel`, `T`, `swapaxes`, `squeeze`, `a[:, None]`	View when possible	`b.base is a`
Boolean mask `a[a > 0]`	Copy	`b.base is None`
Fancy index `a[[1, 3]]`, `np.take`	Copy	`np.shares_memory`
`flatten`, `copy`, `astype`, `np.sort`, `np.unique`, `np.concatenate`	Copy	always new
Arithmetic `a + 1`, `a * b`	New array	always new
`a += 1`, `a[...] = v`, `np.add(a, 1, out=a)`	In place: no new array	same `id`

Problem 16.13.1 A function normalises an array and unexpectedly alters the caller's data. Find out why and fix it.

Cause	Passing an array to a function passes a reference to the same buffer. In-place operators such as `-=` and `/=` then change the caller's array.
Fix	Do not use in-place operators on arguments, or take a copy first.

```
import numpy as np

def centre_bad(x):
    x -= x.mean()
    return x

def centre_good(x):
    return x - x.mean()

data = np.array([2.0, 4.0, 6.0])
out = centre_bad(data)
print(data, out, out is data)

data = np.array([2.0, 4.0, 6.0])
out = centre_good(data)
print(data, out)

ints = np.array([1, 2, 3])
try:
    ints /= 2
except Exception as e:
    print(type(e).__name__)
ints = ints / 2
print(ints, ints.dtype)

a = np.arange(6)
row = a[:3]
row *= 10
print(a)
print(np.shares_memory(a, a[:3]), np.shares_memory(a, a[[0, 1, 2]]))
```

```
[-2.  0.  2.] [-2.  0.  2.] True
[2. 4. 6.] [-2.  0.  2.]
UFuncTypeError
[0.5 1.  1.5] float64
[ 0 10 20  3  4  5]
True False
```

Line by line.

1. `centre_bad` uses `-=`, which writes into the buffer it was given. After the call, `data` itself holds `[-2. 0. 2.]`, and `out is data` is `True`: the "result" is the same object.
2. `ints /= 2` on an integer array tries to store floats back into integer memory and NumPy refuses, raising `UFuncTypeError`.
3. Multiplying the slice `row` in place changes `a` as well: `[0 10 20 3 4 5]`.
4. `shares_memory` is `True` for the slice and `False` for the fancy selection of the same three cells.

Views also save memory: a slice of a 1 GB array costs almost nothing. The flip side is that a small view keeps the whole big buffer alive. If you keep only a tiny slice of a huge array for a long time, `.copy()` it so the big buffer can be freed.

16.14 Recipes: How Do I ...?

Each recipe is a complete, runnable answer to a task you will meet again. Copy the pattern, change the names.

Recipe 1. How do I normalise and standardise columns?

Min-max scaling maps each column to 0..1: (x - min) / (max - min). **Standardisation** (z-score) gives each column mean 0 and standard deviation 1: (x - mean) / std. Both are one broadcast expression per column.

```
import numpy as np
X = np.array([[170.0, 65.0, 30.0],
              [180.0, 80.0, 45.0],
              [160.0, 55.0, 22.0],
              [175.0, 90.0, 38.0]])
lo, hi = X.min(axis=0), X.max(axis=0)
minmax = (X - lo) / (hi - lo)
mu, sd = X.mean(axis=0), X.std(axis=0)
z = (X - mu) / sd
print(minmax.round(2))
print(z.mean(axis=0).round(6), z.std(axis=0).round(6))
const = np.array([[5.0, 1.0], [5.0, 2.0]])
rng_ = np.ptp(const, axis=0)
safe = np.where(rng_ == 0, 1.0, rng_)
print((const - const.min(axis=0)) / safe)
```

```
[[0.5  0.29 0.35]
 [1.   0.71 1.  ]
 [0.   0.   0.  ]
 [0.75 1.   0.7 ]]
[0. 0. 0.] [1. 1. 1.]
[[0. 0.]
 [0. 1.]]
```

1. In `minmax` every column spans exactly 0 to 1 (the smallest height 160 becomes 0.0, the tallest 180 becomes 1.0).
2. After standardising, each column's mean is 0 and its standard deviation is 1; the rounding hides float dust.
3. A constant column has range 0 and would divide by zero. Replacing a zero range by 1 gives zeros instead of NaN. Always guard this in real pipelines.
4. Fit the statistics (lo, hi, mu, sd) on training data only and reuse them for test data; otherwise information leaks.

Recipe 2. How do I one-hot encode labels?

One-hot encoding turns a category code into a row with a single 1. The trick: index an identity matrix with the codes.

```
import numpy as np
labels = np.array(["cat", "dog", "bird", "dog", "cat"])
classes, codes = np.unique(labels, return_inverse=True)
onehot = np.eye(len(classes), dtype=int)[codes]
print(classes)
print(onehot)
back = classes[onehot.argmax(axis=1)]
print(back)
alt = np.zeros((len(codes), len(classes)), dtype=int)
alt[np.arange(len(codes)), codes] = 1
print(np.array_equal(alt, onehot))
```

```
['bird' 'cat' 'dog']
[[0 1 0]
 [0 0 1]
 [1 0 0]
 [0 0 1]
 [0 1 0]]
['cat' 'dog' 'bird' 'dog' 'cat']
True
```

1. `np.eye(3)[codes]` is fancy indexing: row `k` of the identity matrix is a one-hot vector for class `k`, so the result is (5, 3).
2. `argmax(axis=1)` finds the position of the 1 in each row, and indexing `classes` with it decodes back to the original labels.

3. The second construction sets cell (row i, column codes[i]) to 1. It is the version to use when the number of classes is large and you want to avoid building an identity matrix.

Recipe 3. How do I compute a pairwise distance matrix without loops?

For n points in d dimensions you want an n by n table of distances. Insert axes so that broadcasting forms every pair: shape (n, 1, d) minus (1, n, d) gives (n, n, d); square, sum over the last axis, take the square root.

```
import numpy as np
pts = np.array([[0.0, 0.0], [3.0, 4.0], [6.0, 8.0], [0.0, 1.0]])
diff = pts[:, None, :] - pts[None, :, :]
D = np.sqrt((diff ** 2).sum(axis=-1))
print(D.round(2))
print(diff.shape)
sq = (pts ** 2).sum(axis=1)
D2 = np.sqrt(np.maximum(sq[:, None] + sq[None, :] - 2 * pts @ pts.T, 0))
print(np.allclose(D, D2))
from scipy.spatial.distance import cdist
print(np.allclose(D, cdist(pts, pts)))
nearest = np.where(np.eye(len(pts), dtype=bool), np.inf, D).argmin(axis=1)
print(nearest)
```

```
[[ 0.    5.   10.    1.  ]
 [ 5.    0.    5.    4.24]
 [10.    5.    0.    9.22]
 [ 1.    4.24  9.22  0.  ]]
(4, 4, 2)
True
True
[3 3 1 0]
```

1. `pts[:, None, :]` has shape (4, 1, 2) and `pts[None, :, :]` has shape (1, 4, 2). Subtracting broadcasts to (4, 4, 2): the difference vector for every pair.
2. Squaring, summing over the last axis and taking the root gives the Euclidean distance. Point 1 is (3, 4), 5 units from the origin, and point 2 is 10 units away.
3. The second form uses the identity $|a - b|^2 = |a|^2 + |b|^2 - 2\,a.b$.
4. To find each point's nearest neighbour, set the diagonal (distance to itself) to infinity before `argmin`. Point 0's nearest is point 3.

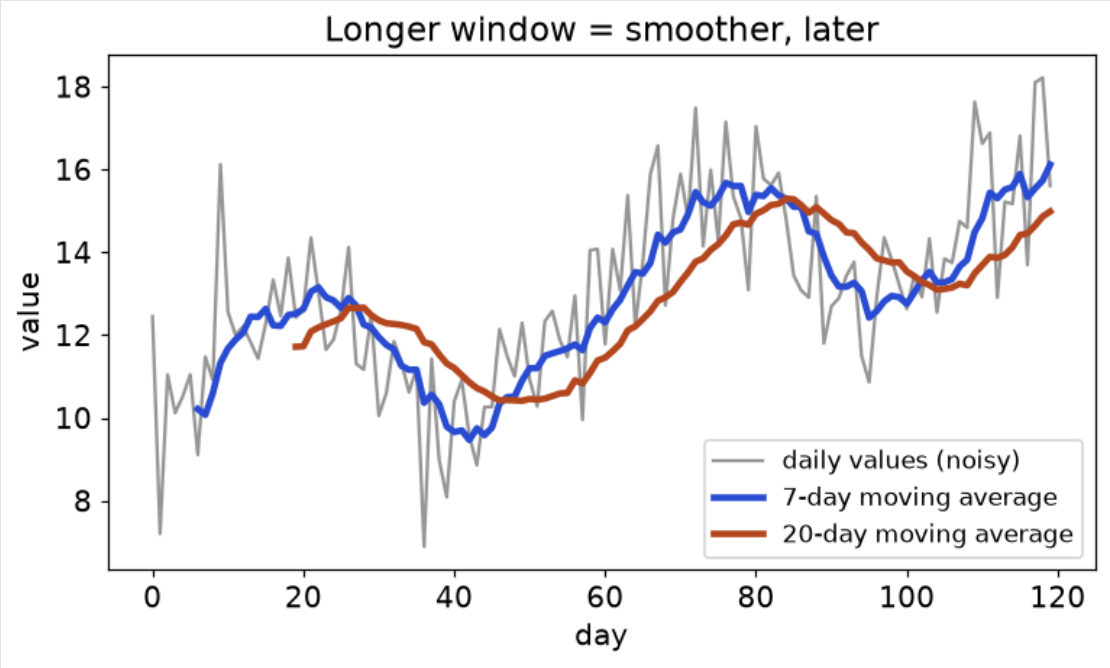
Recipe 4. How do I compute a moving average?

A window of width w averages w neighbours. The fast trick: take the cumulative sum, and subtract it from a copy shifted by w. Each window sum is then two lookups.

```
import numpy as np
x = np.array([1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0])
w = 3
c = np.cumsum(np.insert(x, 0, 0.0))
ma = (c[w:] - c[:-w]) / w
print(ma)
conv = np.convolve(x, np.ones(w) / w, mode="valid")
print(np.allclose(ma, conv))
from numpy.lib.stride_tricks import sliding_window_view
win = sliding_window_view(x, w)
print(win.shape, win.mean(axis=1))
```

```
[2. 3. 4. 5. 6. 7.]
True
(6, 3) [2. 3. 4. 5. 6. 7.]
```

1. `c[w:] - c[:-w]` is the sum of each window of 3 (for example 1+2+3 = 6). Dividing by 3 gives `[2. 3. 4. 5. 6. 7.]`.
2. `sliding_window_view` returns a (6, 3) read-only view with no copying; averaging along axis 1 gives the same answer and extends to medians, maxima, and so on.
3. The output has n - w + 1 values: a window needs w points, so the first w - 1 positions have no average.



The cumulative-sum moving average applied to 120 noisy daily values with windows of 7 and 20.

How to read it: the grey line is the raw noisy data. The blue 7-day line follows the ups and downs more closely; the orange 20-day line is much smoother but starts later (it needs 20 points) and lags behind turning points. Choosing the window is a trade between noise and delay.

Recipe 5. How do I get the top k items (overall and per row)?

```
import numpy as np
scores = np.array([0.2, 0.9, 0.4, 0.95, 0.1, 0.7])
k = 3
idx = np.argpartition(scores, -k)[-k:]
top = idx[np.argsort(scores[idx])[::-1]]
print(top, scores[top])
M = np.array([[5, 1, 9, 3], [2, 8, 4, 6], [7, 7, 1, 0]])
order = np.argsort(-M, axis=1, kind="stable")[:, :2]
print(order)
print(np.take_along_axis(M, order, axis=1))
print(np.sort(M, axis=1)[:, ::-1][:, :2])
```

```
[3 1 5] [0.95 0.9  0.7 ]
[[2 0]
 [1 3]
 [0 1]]
[[9 5]
 [8 6]
 [7 7]]
[[9 5]
 [8 6]
 [7 7]]
```

1. Those k positions are then sorted by score (descending) so the answer is ordered: positions 3, 1, 5 with scores 0.95, 0.9, 0.7.
2. For a matrix, `argsort(-M, axis=1)` orders every row at once; the first two columns of the result are each row's top-2 positions.
3. `take_along_axis` turns those positions into the values. If you only need values, sorting and slicing does it as in the last line.

Recipe 6. How do I work with image arrays?

A colour image is an array of shape (height, width, 3) of `uint8` (0 to 255). Real files are loaded with Pillow or matplotlib (`plt.imread`); here we build a tiny synthetic image so the example is standalone.

```
import numpy as np
rng = np.random.default_rng(0)
img = rng.integers(0, 256, size=(4, 6, 3), dtype=np.uint8)
print(img.shape, img.dtype, img.min(), img.max())
gray = img @ np.array([0.299, 0.587, 0.114])
print(gray.shape, gray.dtype)
gray8 = gray.round().astype(np.uint8)
flipped = img[:, ::-1]
crop = img[1:3, 2:5]
print(flipped.shape, crop.shape, np.shares_memory(img, crop))
bright = np.clip(img.astype(np.int16) + 100, 0, 255).astype(np.uint8)
wrong = img + np.uint8(100)
print((bright[0, 0] >= img[0, 0]).all(), (wrong[0, 0] >= img[0, 0]).all())
chw = np.moveaxis(img, -1, 0)
print(chw.shape, (img / 255.0).dtype)
mask = gray8 > 128
print(mask.sum(), img[mask].shape)
```

```
(4, 6, 3) uint8 1 248
(4, 6) float64
(4, 6, 3) (2, 3, 3) True
True False
(3, 4, 6) float64
12 (12, 3)
```

1. Matrix-multiplying the last axis by the luma weights (0.299, 0.587, 0.114) collapses the channels to one grey value per pixel: shape (4, 6), float64.
2. `img[:, ::-1]` mirrors left to right as a view; `img[1:3, 2:5]` crops a 2 by 3 window, also a view that shares memory.
3. Brightening: `uint8 + 100` wraps around past 255 (the pixel gets darker, as the second check shows). Converting to `int16`, clipping to 0..255, then converting back is the correct recipe.
4. `np.moveaxis(img, -1, 0)` gives channels-first (3, 4, 6), the layout used by PyTorch. Dividing by 255.0 gives floats in 0..1.

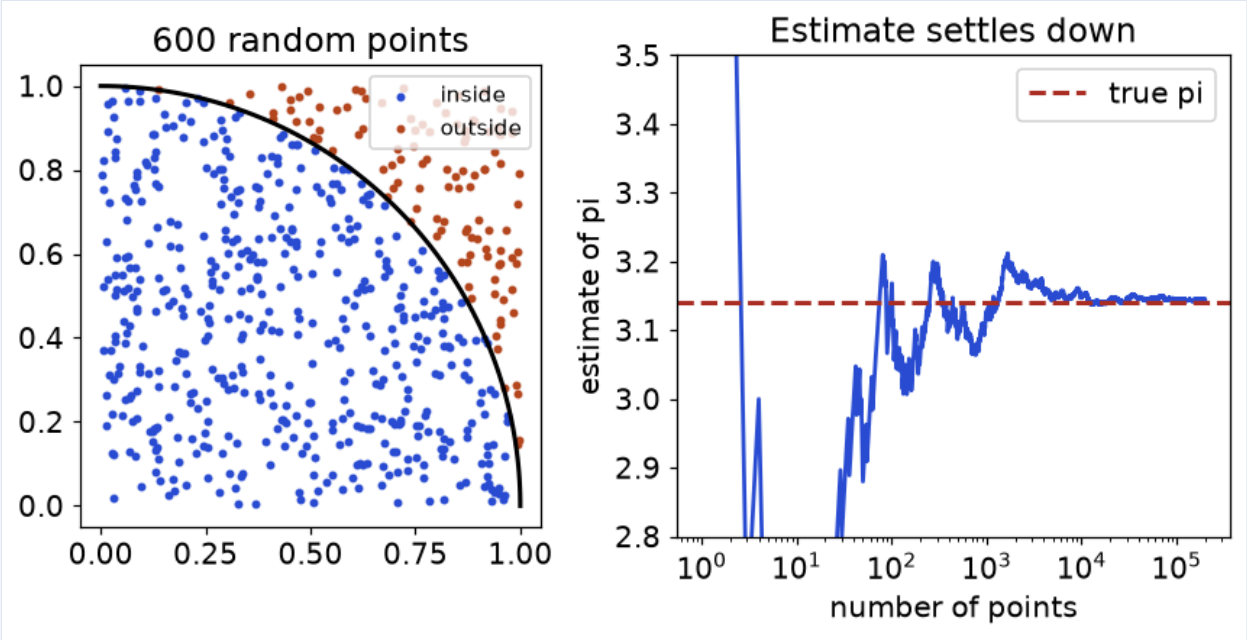
Recipe 7. How do I run a Monte Carlo simulation?

Monte Carlo estimates a quantity by random sampling. To estimate pi: throw random points in the unit square; the share landing within distance 1 of the corner (in a quarter circle of area pi/4) is about pi/4.

```
import numpy as np
rng = np.random.default_rng(7)
for n in (100, 10_000, 1_000_000):
    pts = rng.random((n, 2))
    inside = (pts ** 2).sum(axis=1) <= 1.0
    print(n, round(4 * inside.mean(), 4))
rolls = rng.integers(1, 7, size=(100000, 2)).sum(axis=1)
print(round(float((rolls == 7).mean()), 3), round(6 / 36, 3))
```

```
100 3.08
10000 3.162
1000000 3.1437
0.166 0.167
```

1. `(pts ** 2).sum(axis=1) <= 1.0` tests $x^2 + y^2 <= 1$ for every point; the mean of that boolean is the inside share; times 4 estimates pi.
2. The second experiment rolls two dice 100,000 times as a (100000, 2) array and sums across axis 1.



Monte Carlo estimate of pi. The error shrinks roughly like one over the square root of the number of points.

How to read it: on the left, blue points fall inside the quarter circle and orange points outside. On the right, the horizontal axis is logarithmic. The blue estimate swings widely at first and then hugs the red dashed line at pi. To halve the error you need about four times as many points, which is why Monte Carlo is simple but slow to be very precise.

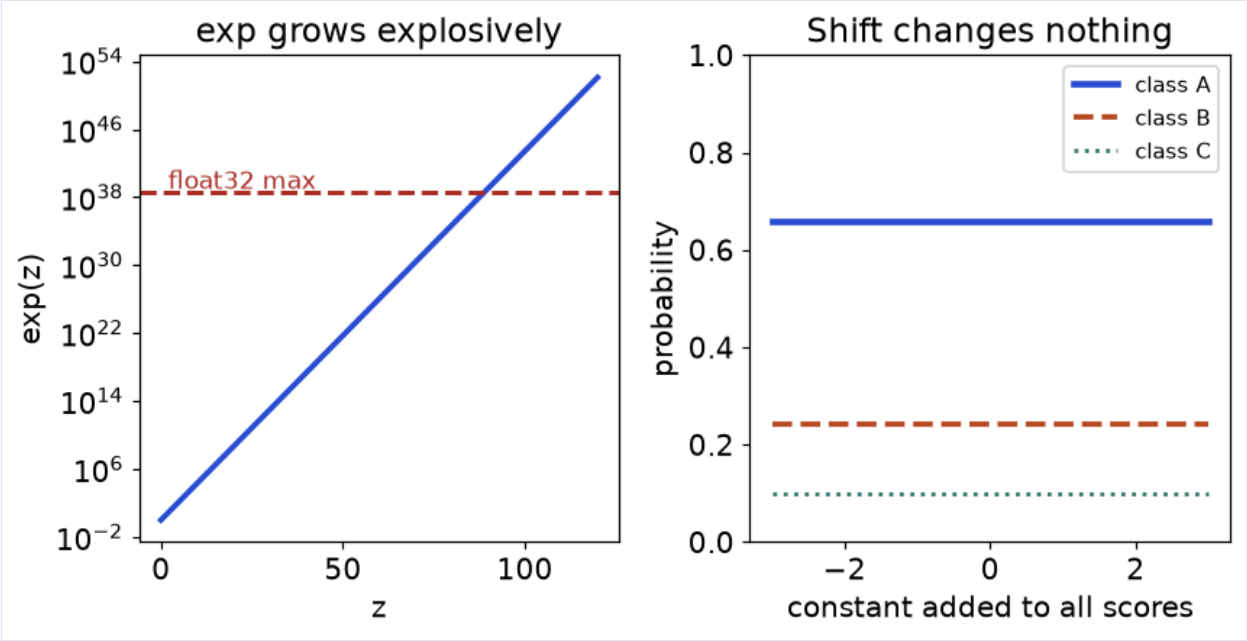
Recipe 8. How do I compute a numerically stable softmax?

Softmax turns scores into probabilities: exp(z_i) / sum of exp(z). The danger: exp of a large number overflows. The fix uses the fact that subtracting the same constant from every score changes nothing, so subtract the maximum.

```
import numpy as np
def softmax_naive(z):
    e = np.exp(z)
    return e / e.sum(axis=-1, keepdims=True)

def softmax(z):
    z = z - z.max(axis=-1, keepdims=True)
    e = np.exp(z)
    return e / e.sum(axis=-1, keepdims=True)

z = np.array([1000.0, 1001.0, 1002.0])
with np.errstate(all="ignore"):
    print(softmax_naive(z))
print(softmax(z).round(4))
print(np.allclose(softmax(np.array([2.0, 1.0, 0.1])), softmax(np.array([102.0, 101.0, 100.1]))))
batch = np.array([[2.0, 1.0, 0.1], [0.0, 0.0, 0.0]])
print(softmax(batch).round(3), softmax(batch).sum(axis=1))
def log_softmax(z):
    z = z - z.max(axis=-1, keepdims=True)
    return z - np.log(np.exp(z).sum(axis=-1, keepdims=True))
print(log_softmax(z).round(4))
```

```
[nan nan nan]
[0.09   0.2447 0.6652]
True
[[0.659 0.242 0.099]
 [0.333 0.333 0.333]] [1. 1.]
[-2.4076 -1.4076 -0.4076]
```

1. The naive version computes `exp(1000)`, which is infinity in float64, and then infinity divided by infinity is NaN: the output is all `nan`.
2. The stable version subtracts the row maximum first. The largest score becomes 0, so `exp` of it is 1 and nothing overflows.
3. Shifting all scores by 100 gives identical probabilities, which the `allclose` check proves (see the right panel of the figure).
4. `axis=-1` with `keepdims=True` makes the function work on a single vector or on a whole batch, one softmax per row.



Why subtract the maximum. Left: exp(z) leaves the float32 range quickly. Right: adding any constant to all three scores leaves the probabilities unchanged.

How to read it: on the left the blue line is exp(z) on a log scale, a straight line meaning "multiplies by the same factor each step". It crosses the red dashed float32 limit at about z = 88; float64 survives to about z = 709. On the right, the horizontal axis is the constant added to the scores and the

three lines are the three class probabilities. They stay flat: this is the freedom the max-subtraction trick uses.

Recipe 9. How do I split data into train and test sets by hand?

```
import numpy as np
rng = np.random.default_rng(42)
X = np.arange(20).reshape(10, 2)
y = np.array([0, 1] * 5)
perm = rng.permutation(len(X))
n_test = int(len(X) * 0.3)
test_idx, train_idx = perm[:n_test], perm[n_test:]
X_train, X_test = X[train_idx], X[test_idx]
y_train, y_test = y[train_idx], y[test_idx]
print(X_train.shape, X_test.shape)
print(sorted(train_idx.tolist() + test_idx.tolist()) == list(range(10)))
print(y_test, y_train.mean())
def stratified_split(y, test_frac, rng):
    test = []
    for cls in np.unique(y):
        members = rng.permutation(np.flatnonzero(y == cls))
        test.append(members[: round(len(members) * test_frac)])
    test = np.concatenate(test)
    train = np.setdiff1d(np.arange(len(y)), test)
    return train, test
tr, te = stratified_split(np.array([0] * 8 + [1] * 2), 0.5, np.random.default_rng(0))
print(len(tr), len(te), np.bincount(np.array([0] * 8 + [1] * 2)[te]))
```

```
(7, 2) (3, 2)
True
[1 0 0] 0.5714285714285714
5 5 [4 1]
```

1. Shuffle the row numbers once with `permutation`, then cut the shuffled list. The same index arrays split `X` and `y` so rows stay paired.
2. 30 percent of 10 rows is 3 test rows, 7 training rows. The check confirms every row appears exactly once, with no overlap and nothing lost.
3. Plain random splits can leave rare classes missing from the test set. The stratified version splits each class separately, so class proportions are kept.

Recipe 10. How do I do a group-by sum or a histogram without a loop?

```
import numpy as np
store = np.array([0, 2, 1, 0, 2, 2, 1])
amount = np.array([10.0, 5.0, 7.5, 2.5, 1.0, 4.0, 2.5])
print(np.bincount(store, weights=amount))
print(np.bincount(store))
totals = np.zeros(3)
np.add.at(totals, store, amount)
print(totals)
ages = np.array([3, 17, 25, 31, 44, 52, 67, 71])
counts, edges = np.histogram(ages, bins=[0, 18, 40, 65, 120])
print(counts, edges)
print(np.digitize(ages, [18, 40, 65]))
```

```
[12.5 10.  10. ]
[2 2 3]
[12.5 10.  10. ]
[2 2 2 2] [  0  18  40  65 120]
[0 0 1 1 2 2 3 3]
```

1. `bincount(store, weights=amount)` adds the amounts per store number: store 0 has 10 + 2.5 = 12.5, store 1 has 10, store 2 has 10.
2. `np.add.at` is the general "scatter-add"; it handles repeated indexes correctly, which plain `totals[store] += amount` does not (it applies only one update per distinct index).
3. `np.histogram` with explicit bin edges counts values in each age band; `digitize` gives each value's band number. For labelled group-bys on strings, use pandas (the next chapter).

16.15 Performance and Pitfalls

Speed rules that hold up

- **Move loops into NumPy** (16.12), **preallocate** instead of growing, and collect pieces in a list before one `np.concatenate`.
- **Avoid needless copies.** Slices and reshapes are free; fancy indexing, `astype` and temporaries are not. Use `out=` or in-place operators when you own the array.
- **Pick the smallest sufficient dtype** (float32, int32) and check precision. Prefer `@` to element loops, since it calls optimised BLAS code.
- **Measure, do not guess.** Use `time.perf_counter` or `timeit`, repeat, keep the best.

Problem 16.15.1 Run five classic traps in one script so you can see the symptoms with your own eyes.

Idea	Each line below reproduces one pitfall. The explanation after the output names the fix.

```
import numpy as np
a = np.array([1, 2, 3])
a[0] = 2.9
print(a)
f = np.float32(16777216)
print(f + np.float32(1) == f)
y = np.array([1.0, 2.0, 3.0])
pred = np.array([[1.0], [2.0], [3.0]])
print((y - pred).shape, ((y - pred) ** 2).mean())
print(((y - pred.ravel()) ** 2).mean())
b = a
b[1] = 99
print(a)
data = np.array([3, 1, 2])
result = data.sort()
print(result, data)
v = np.array([1.0, np.nan, 3.0])
print(v.argmax(), np.nanargmax(v))
M = np.arange(6).reshape(2, 3)
print(sum(M), M.sum())
```

```
[2 2 3]
True
(3, 3) 1.3333333333333333
0.0
[ 2 99  3]
None [1 2 3]
1 2
[3 5 7] 15
```

Line by line.

1. Assigning 2.9 into an integer array silently truncates it to 2. The array keeps its dtype and does not upgrade.
2. float32 has only 24 bits of precision: 16,777,216 + 1 equals 16,777,216, so the addition is lost. Fix: accumulate in float64 (`dtype=np.float64`) when summing many values.

3. The shapes (3,) and (3, 1) broadcast to (3, 3). The "mean squared error" is computed over nine pairs, not three, and comes out as 1.333 instead of 0.
4. `b = a` copies nothing; it adds a second name for the same array. Changing `b[1]` changes `a`. Fix: `b = a.copy()`.
5. `data.sort()` sorts in place and returns `None`. Writing `result = data.sort()` leaves you with `None`. Fix: `np.sort(data)` for a new array.
6. The built-in `sum(M)` adds the rows as Python objects and returns an array `[3 5 7]`; `M.sum()` returns the grand total 15.

Ten common mistakes, each with symptom and fix

COMMON MISTAKE 1: TRUTH VALUE OF AN ARRAY	**Wrong:** `if a == b:` or `if x > 0:` where `x` is an array. **Symptom:** `ValueError: The truth value of an array with more than one element is ambiguous`. **Fix:** say what you mean: `(x > 0).all()`, `(x > 0).any()`, `np.array_equal(a, b)` or `np.allclose(a, b)`.
COMMON MISTAKE 2: AND / OR AND OPERATOR PRECEDENCE	**Wrong:** `x[(x > 1) and (x < 5)]` or `x[x > 1 & x < 5]`. **Symptom:** the same ambiguous-truth error (Python's `and` wants one boolean), or for the second form a confusing error because `&` binds tighter than `>`. **Fix:** `x[(x > 1) & (x < 5)]`, with brackets around every comparison.
COMMON MISTAKE 3: INTEGER WRAP-AROUND AND TRUNCATION	**Wrong:** `np.uint8` or `int32` arithmetic that exceeds the range; `int_array / 2` assigned back with `[:] =`. **Symptom:** negative sales totals, 255 + 1 = 0, silent truncation. **Fix:** convert up first (`astype(np.int64)` or `float64`), and check ranges with `np.iinfo`.
COMMON MISTAKE 4: ACCIDENTAL (N, N) BROADCASTING	**Wrong:** `np.mean((y - pred) ** 2)` with `y` of shape (n,) and `pred` of shape (n, 1). **Symptom:** a wrong number and a sudden memory spike on large n (an n by n matrix), but no error. **Fix:** assert shapes (`assert y.shape == pred.shape`) at function boundaries.
COMMON MISTAKE 5: ALIASING INSTEAD OF COPYING	**Wrong:** `backup = arr`, `best = weights` inside a training loop. **Symptom:** your "backup" changes along with the original; your "best weights" always equal the latest weights. **Fix:** `backup = arr.copy()`.
COMMON MISTAKE 6: GROWING ARRAYS IN A LOOP	**Wrong:** `for r in rows: arr = np.append(arr, r)` or `np.vstack([arr, r])` inside a loop. **Symptom:** time grows with the square of the data; a job that takes seconds for 10,000 rows takes hours for 10 million. **Fix:** collect results in a Python list and call `np.array(results)` once, or preallocate with `np.empty` and fill by index.
COMMON MISTAKE 7: SHUFFLING OR SORTING THAT RETURNS NONE	**Wrong:** `x = rng.shuffle(x)` or `x = x.sort()`. **Symptom:** `x` becomes `None` and the next line fails with `TypeError: 'NoneType' object is not subscriptable`. **Fix:** in-place methods return nothing; call them as statements, or use `rng.permutation(x)` and `np.sort(x)` to get a new array.
COMMON MISTAKE 8: SAME SEED EVERYWHERE, OR NO SEED	**Wrong:** calling `np.random.default_rng()` with no seed in a reported experiment, or reusing seed 0 for every parallel worker. **Symptom:** results nobody can reproduce, or workers producing identical "random" numbers. **Fix:** one seeded generator per run, and `rng.spawn(n)` to hand independent children to workers.

COMMON MISTAKE 9: FLOAT EQUALITY AND ACCUMULATED ERROR	**Wrong:** `if total == 0.3:` after adding 0.1 three times; summing a million float32 values in a plain loop. **Symptom:** `False` where you expect `True`; sums that are slightly off. **Fix:** `np.isclose` with a stated tolerance; use float64 for accumulation. NumPy's `sum` adds in pairs, which is more accurate than a naive left-to-right loop.

COMMON MISTAKE 10: OBJECT ARRAYS FROM RAGGED DATA	**Wrong:** `np.array([[1, 2], [3]])`. **Symptom:** a `ValueError` about an inhomogeneous shape in NumPy 1.24 and later (older versions silently built a slow object array). **Fix:** pad to a common length (`np.pad` or NaN), or keep a list of arrays when lengths really differ.

16.16 Cross-Library Integration

NumPy is the common language of the scientific Python world. Knowing where memory is shared and where it is copied lets you pass data between libraries without surprises. This section was run with pandas 3.0.6, SciPy 1.18.1, scikit-learn 1.9.1 and matplotlib 3.11.2.

pandas

A pandas `DataFrame` is a labelled table whose columns are NumPy-based arrays (strings use pandas' own string dtype, which is not a NumPy dtype). Use `.to_numpy()` to leave pandas, and NumPy functions work directly on Series. Since pandas 3.0, copy-on-write is always on: changing a derived object never changes the parent, and arrays handed out as views are marked read-only so you cannot corrupt the table by accident.

Problem 16.16.1 Move data between pandas and NumPy and observe dtypes and write protection.

Idea	`to_numpy()` on mixed columns picks a common dtype; a single numeric column comes back as a read-only view. Make a `.copy()` if you need to write.

```
import numpy as np
import pandas as pd
df = pd.DataFrame({"price": [10.0, 12.5, 9.0], "qty": [3, 1, 4], "name": ["a", "b", "c"]})
print({k: str(v) for k, v in df.dtypes.items()})
nums = df[["price", "qty"]].to_numpy()
print(nums.dtype, nums.shape)
col = df["price"].to_numpy()
print(col.flags.writeable)
try:
    col[0] = 99.0
except ValueError as e:
    print("ValueError:", e)
safe = df["price"].to_numpy().copy()
safe[0] = 99.0
print(df["price"].iloc[0], safe[0])
print(np.log(df["price"]).round(3).tolist())
df["total"] = df["price"] * df["qty"]
print(df["total"].to_numpy(), np.sum(df["total"]))
back = pd.DataFrame(np.arange(6).reshape(3, 2), columns=["x", "y"])
print(back.dtypes.to_dict())
```

```
{'price': 'float64', 'qty': 'int64', 'name': 'str'}
float64 (3, 2)
False
ValueError: assignment destination is read-only
10.0 99.0
[2.303, 2.526, 2.197]
[30.  12.5 36. ] 78.5
{'x': dtype('int64'), 'y': dtype('int64')}
```

Line by line.

1. Taking two numeric columns and calling `to_numpy()` upcasts the integers to a shared `float64`, giving a (3, 2) array.
2. A single numeric column is exposed without a copy, so pandas marks it read-only (`writeable` is `False`).
3. Copy first, then edit: the DataFrame still holds 10.0 while the copy holds 99.0.
4. NumPy ufuncs accept Series and return Series, keeping the index. Arithmetic between columns is vectorised, exactly like arrays.

scikit-learn

scikit-learn takes NumPy arrays (or DataFrames) of shape (n_samples, n_features) and returns arrays. Remember: `X` must be 2D, `y` 1D.

```
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import StandardScaler
rng = np.random.default_rng(3)
X = rng.normal(10, 2, size=(100, 3))
y = X @ np.array([1.0, -2.0, 0.5]) + 4.0 + rng.normal(0, 0.1, 100)
sc = StandardScaler().fit(X)
print(np.allclose(sc.transform(X), (X - X.mean(axis=0)) / X.std(axis=0)))
model = LinearRegression().fit(X, y)
print(model.coef_.round(2), round(float(model.intercept_), 2))
A = np.column_stack([X, np.ones(len(X))])
coef = np.linalg.lstsq(A, y, rcond=None)[0]
print(coef.round(2))
try:
    LinearRegression().fit(np.arange(5.0), np.arange(5.0))
except ValueError:
    print("ValueError: X must be 2D")
print(LinearRegression().fit(np.arange(5.0).reshape(-1, 1), np.arange(5.0) * 2).coef_)
```

```
True
[ 1.   -1.99  0.5 ] 3.85
[ 1.   -1.99  0.5   3.85]
ValueError: X must be 2D
[2.]
```

Line by line.

1. The data follow y = X . [1, -2, 0.5] + 4 plus small noise, so the fitted coefficients recover roughly 1, -2, 0.5 and an intercept near 4 (3.85 here, because noise and the small sample move it a little).
2. The same least-squares fit with `np.linalg.lstsq` and a column of ones gives the same four numbers; scikit-learn is wrapping this kind of linear algebra with a friendly interface.
3. Passing a 1D array as `X` raises a `ValueError` asking for a 2D array. The fix is `reshape(-1, 1)` (one feature, many samples), as the last line shows with the slope 2.

PyTorch and other array libraries (not run here)

PyTorch is not installed in this study environment, so the following is clearly **not run here**. The idiom is standard and documented by PyTorch:

```
# not run here (needs: pip install torch)
import numpy as np, torch
a = np.arange(6, dtype=np.float32).reshape(2, 3)
t = torch.from_numpy(a)      # shares memory with a, no copy
t[0, 0] = 100.0              # a[0, 0] is now 100.0 as well
b = t.numpy()                # back to NumPy, again shared (CPU tensors only)
c = torch.tensor(a)          # explicit copy
```

16.17 How to Solve Any NumPy Problem: a Playbook

The decision flow

1. **Write down the shapes** of the input and the output you want. Most NumPy problems are shape problems.
2. **Ask: is it element-by-element?** Use ufuncs, `where`, `select`, `clip`.
3. **Does it summarise along a direction?** Use a reduction and decide the axis: the axis that should disappear.
4. **Does every item need to meet every other item?** Add axes with `None` and broadcast, or use a matrix product.
5. **Does it select or rearrange?** Boolean mask (by value), fancy index (by position), `argsort` (by rank).
6. **Does each step depend on earlier steps?** Look for `cumsum`, `accumulate`, `convolve`, `sliding_window_view`; if none fits, write a loop, and consider Numba.
7. **Check it** on a tiny case by hand and against a slow reference.

Eight recurring problem patterns

Pattern	Signal in the problem	Tool
1. Conditional rule per element	"if ... then ... else" on each value	`np.where`, `np.select`
2. Summary per row or column	"per customer", "for each column"	reduction with `axis`, `keepdims`
3. All pairs	"distance between every two", "table of products"	`a[:, None] op b[None, :]`
4. Count or group by key	"how many of each", "total per store id"	`unique`, `bincount`, `add.at`
5. Running or windowed	"cumulative", "last 7 days"	`cumsum`, `sliding_window_view`
6. Filter	"only rows where", "remove missing"	boolean mask, `isnan`
7. Rank or top	"best 5", "order by"	`argsort`, `argpartition`
8. Solve or fit	"find the weights", "solve for x"	`solve`, `lstsq`, `@`

Problem 16.17.1 One solved example for each of the eight patterns, on a small order table. Orders have a customer id (0 to 2), a price, a quantity and a rating (NaN if missing).

Data	8 orders: customer ids, prices, quantities and ratings, with two missing ratings.

```
import numpy as np
cust = np.array([0, 1, 0, 2, 1, 0, 2, 1])
price = np.array([20.0, 35.0, 15.0, 50.0, 10.0, 25.0, 40.0, 30.0])
qty = np.array([1, 2, 3, 1, 5, 2, 1, 1])
rating = np.array([4.0, np.nan, 5.0, 3.0, 4.0, np.nan, 2.0, 5.0])
revenue = price * qty

size = np.where(revenue >= 60, "large", "small")
print("1", size)

table = np.zeros((3, 2))
for c in range(3):
    table[c] = [revenue[cust == c].sum(), qty[cust == c].sum()]
by_cust = np.bincount(cust, weights=revenue)
print("2", by_cust, table[:, 0])

P = np.column_stack([price, qty]).astype(float)
D = np.abs(P[:, None, 0] - P[None, :, 0])
print("3", D.shape, D[0, :3])

print("4", np.bincount(cust), np.unique(cust, return_counts=True)[1])

running = np.cumsum(revenue)
print("5", running, np.cumsum(revenue)[-1] == revenue.sum())

ok = ~np.isnan(rating)
print("6", rating[ok], round(float(rating[ok].mean()), 2), round(float(np.nanmean(rating)), 2))

top2 = np.argsort(-revenue, kind="stable")[:2]
print("7", top2, revenue[top2])

A = np.column_stack([price, np.ones(len(price))])
coef = np.linalg.lstsq(A, qty, rcond=None)[0]
print("8", coef.round(3))
```

```
1 ['small' 'large' 'small' 'small' 'small' 'small' 'small' 'small']
2 [115. 150.  90.] [115. 150.  90.]
3 (8, 8) [ 0. 15.  5.]
4 [3 3 2] [3 3 2]
5 [ 20.  90. 135. 185. 235. 285. 325. 355.] True
6 [4. 5. 3. 4. 2. 5.] 3.83 3.83
7 [1 3] [70. 50.]
8 [-0.076  4.143]
```

Line by line.

1. **Pattern 1.** Revenue is price times quantity for every order at once. `where` labels orders of 60 or more as large: only order 1, with revenue 70, qualifies.
2. **Pattern 2.** A loop over three customers builds the by-customer table; `bincount` with weights gives the same revenue per customer in one call, and the second print shows that they agree.
3. **Pattern 3.** `price[:, None]` against `price[None, :]` forms the 8 by 8 table of absolute price gaps; the first row, restricted to three entries, starts with 0 (an order versus itself).
4. **Pattern 4.** Counts of orders per customer, computed two ways, agree: customer 0 has 3, customer 1 has 3, customer 2 has 2.
5. **Pattern 5.** A running revenue total; its last element must equal the plain sum, a cheap built-in check.
6. **Pattern 6.** The mask `~isnan` keeps the six known ratings; their mean is 3.83 and equals `nanmean`.
7. **Pattern 7.** The two biggest orders by revenue: order 1 (70) and then order 3 (50). Orders 3, 4 and 5 all earn 50, and the stable sort keeps the earliest of the tie.

8. **Pattern 8.** A least-squares line of quantity against price: a slope of about -0.076 units per currency unit and an intercept of about 4.14, so pricier orders tend to have smaller quantities in this tiny table.

How to verify

- **Tiny case by hand:** shrink the input to 3 or 4 items you can calculate on paper.
- **Slow reference:** write the obvious loop, compare with `np.testing.assert_allclose` on random inputs.
- **Invariants:** probabilities sum to 1, a distance matrix is symmetric with a zero diagonal, a split covers every row exactly once, a sorted array has non-negative `np.diff`.
- **Shapes:** assert expected shapes at function boundaries.

```
import numpy as np
rng = np.random.default_rng(11)
P = rng.random((40, 3))
D = np.sqrt(((P[:, None] - P[None]) ** 2).sum(-1))
ref = np.zeros((40, 40))
for i in range(40):
    for j in range(40):
        ref[i, j] = np.sqrt(sum((P[i, k] - P[j, k]) ** 2 for k in range(3)))
np.testing.assert_allclose(D, ref, rtol=1e-12)
print("matches loop:", True)
print("symmetric:", np.allclose(D, D.T), "zero diagonal:", np.allclose(np.diag(D), 0))
print("triangle inequality holds:", bool((D[0, 1] + D[1, 2] >= D[0, 2] - 1e-12)))
```

```
matches loop: True
symmetric: True zero diagonal: True
triangle inequality holds: True
```

Line by line.

1. The slow reference uses three nested loops. If the arrays disagree, `assert_allclose` raises with a readable report of the worst mismatch.
2. Then property checks that must hold for any distance matrix: symmetric, zero diagonal, and the triangle inequality on one triple.

How to debug

- Print `shape`, `dtype` and a few values after every step. Most bugs show up as an unexpected shape.
- Reproduce on a tiny seeded input, so that you can read the whole array.
- Turn silent float warnings into errors with `np.errstate(all="raise")`, so that a divide by zero stops the program at the exact line.
- Check `x.base` and `np.shares_memory` when data changes unexpectedly.
- Split a long chain into named intermediate variables and inspect each one.

16.18 Going Deeper: Internals, Memory, Speed and Interview Questions

How a ufunc works

A ufunc such as `np.add` is an object with several compiled inner loops, one per dtype combination. When you call it, NumPy (1) works out the result dtype from the inputs, (2) works out the broadcast shape, (3) allocates the output and (4) calls the matching loop over the buffers, with the strides handling the broadcast. Modern CPUs also use SIMD instructions in many of these loops. The ufunc object also carries methods that generalise it.

```
import numpy as np
print(np.add.reduce([1, 2, 3]), np.add.accumulate([1, 2, 3]))
print(np.multiply.outer([1, 2], [3, 4]))
print(np.maximum.accumulate([1, 3, 2, 5, 4]))
a = np.zeros(3)
np.add.at(a, [0, 0, 2], 1.0)
print(a)
print(np.add.types[:4])
```

```
6 [1 3 6]
[[3 4]
 [6 8]]
[1 3 3 5 5]
[2. 0. 1.]
['??->?', 'bb->b', 'BB->B', 'hh->h']
```

Line by line.

1. `reduce` folds an array with the operation: `add.reduce` is the sum, 6. `accumulate` keeps the running result: `[1 3 6]`, which is `cumsum`.
2. `maximum.accumulate` gives a running maximum, a pattern with no dedicated function name: `[1 3 3 5 5]`.
3. `np.add.at` adds with repeated indexes counted properly: index 0 appears twice, so `a[0]` is 2.

Type promotion in NumPy 2

NumPy 2 changed how Python numbers mix with arrays (NEP 50). A plain Python int or float is "weak": it takes the dtype of the array. So `uint8_array + 100` stays `uint8` (and wraps around), and a Python integer that does not fit raises an error rather than silently changing the result type. Two arrays of different dtypes still promote to a common type, which can be surprising: unsigned 64-bit with signed 64-bit goes to `float64`.

```
import numpy as np
a = np.array([200], dtype=np.uint8)
print((a + 100), (a + 100).dtype)
print((np.array([1.0], dtype=np.float32) + 3.0).dtype)
try:
    np.array([1], dtype=np.uint8) + 300
except OverflowError as e:
    print("OverflowError:", e)
print(np.result_type(np.uint64, np.int64), np.result_type(np.int32, np.float32))
```

```
[44] uint8
float32
OverflowError: Python integer 300 out of bounds for uint8
float64 float64
```

Line by line.

1. 200 + 100 does not fit in uint8, so the answer wraps to 44, and the dtype remains `uint8`.
2. Adding the integer 300 to a uint8 array raises `OverflowError`, because 300 cannot be represented in uint8 at all.
3. Mixing unsigned 64-bit and signed 64-bit integers has no integer type that holds both, so the result is `float64`. int32 with float32 also goes to `float64`, to avoid losing integer digits.

Speed: what actually matters

- **Contiguity and bandwidth.** Contiguous data is friendlier to the CPU cache; for big arrays memory traffic dominates, so fewer passes and smaller dtypes help more than clever arithmetic.
- **BLAS threads.** Matrix products use multi-threaded libraries, usually controlled by environment variables such as `OMP_NUM_THREADS`. Check your build with `np.show_config()`.

- **Sorting.** The default `quicksort` is not stable; `kind="stable"` selects a stable method.
- **When NumPy is not enough:** Numba or Cython for loops that cannot be vectorised (not run here); memmap, Dask or a database for data larger than memory; CuPy or PyTorch for the GPU.

What interviewers probe

INTERVIEW

- **"Why is NumPy faster than lists?"** Same-type values in one contiguous buffer, compiled loops, no per-element Python object, SIMD and cache-friendly access.
- **"View or copy?"** Basic slicing, reshape and transpose are views; boolean and fancy indexing copy.
- **"Explain broadcasting."** Right-aligned shapes; each axis must match or be 1; size-1 axes are stretched with stride 0.
- **"Subtract row means from a matrix."** `X - X.mean(axis=1, keepdims=True)`.
- **"Why not `inv` to solve a system?"** `solve` is faster and more accurate.
- **"Reproducible randomness in parallel code?"** One `default_rng(seed)`, then `spawn`.

Chapter recap

- An ndarray is one buffer of same-type numbers plus a header (dtype, shape, strides). Same-type, contiguous and compiled is where the speed comes from.
- Know your dtypes: float64 and int64 are the defaults; float32 halves memory; uint8 wraps silently; NaN exists only in float arrays.
- Basic slices, `reshape`, `ravel` and `.T` are views. Boolean and fancy indexing, `astype`, `flatten` and arithmetic make copies. Use `.copy()` when you need independence.
- Broadcasting: align shapes from the right, pad with 1s, sizes must match or be 1. Use `None` to add an axis and `keepdims=True` to keep one.
- For a reduction, the axis you name is the one that disappears. `axis=0` gives one answer per column of a 2D table; `axis=1` one per row.
- Masks select by value, fancy indexes select by position, `argsort` selects by rank, and `where` is a vectorised if/else.
- NaN is never equal to itself: use `isnan`, `nanmean` and friends, and decide deliberately how to fill gaps.
- Use `solve` not `inv`, `@` not `*` for matrix products, and `lstsq` for fits.
- One seeded `default_rng(seed)` per experiment, passed into functions; `spawn` for parallel workers.
- Vectorise loops, preallocate instead of appending, pick dtypes deliberately, and always verify against a tiny case or a slow reference.
- Debug by printing shapes and dtypes, using `errstate`, and checking `shares_memory`.

Formula / pattern	Meaning
`(X - X.mean(0)) / X.std(0)`	Standardise every column
`(X - X.min(0)) / (X.max(0) - X.min(0))`	Min-max scale every column
`np.eye(k)[codes]`	One-hot encoding
`np.sqrt(((P[:, None] - P[None]) ** 2).sum(-1))`	All-pairs Euclidean distance
`c = np.cumsum(np.insert(x, 0, 0)); (c[w:] - c[:-w]) / w`	Moving average of width w
`np.argpartition(a, -k)[-k:]`	Positions of the k largest
`e = np.exp(z - z.max(-1, keepdims=True)); e / e.sum(-1, keepdims=True)`	Stable softmax
`rng.permutation(n)` then cut	Train/test split
`np.bincount(keys, weights=w)`	Group-by sum for small integer keys
`np.where(cond, a, b)`	Vectorised if/else

Formula / pattern	Meaning
`np.linalg.solve(A, b)`	Solve A x = b
`np.einsum("ij,jk->ik", A, B)`	Matrix product by index notation
`a[:, None] + b[None, :]`	Outer combination of two vectors

Practice set

Try each problem before you read the answer. Problems are graded from easy to expert. Code shown in the problems is plain text; run it yourself to check.

Easy

Problem 16.P.1 An array is created by `np.zeros((3, 4, 2))`. Give its `shape`, `ndim`, `size` and `nbytes`.

Answer: shape `(3, 4, 2)`, ndim 3, size 3 x 4 x 2 = 24, and since the default dtype is float64 (8 bytes), nbytes = 24 x 8 = 192.

Problem 16.P.2 For `x = np.array([4, 9, 2, 7, 10])`, how many values exceed 5, and what is the mean of those values? Write the code.

Answer: `(x > 5).sum()` is 3 (values 9, 7, 10). `x[x > 5].mean()` is 26/3 = 8.667.

Problem 16.P.3 What are the results of broadcasting these shapes, or do they fail? (a) (4, 1) with (3,), (b) (2, 3) with (2,), (c) (5, 1, 3) with (4, 1).

Answer: (a) pad (3,) to (1, 3); result (4, 3). (b) align from the right: 3 versus 2 fails, so `ValueError`. (c) pad (4, 1) to (1, 4, 1); axes: 5 with 1 gives 5, 1 with 4 gives 4, 3 with 1 gives 3; result (5, 4, 3).

Problem 16.P.4 For `M = np.array([[1, 2, 3], [4, 5, 6]])`, evaluate `M.sum(axis=0)`, `M.sum(axis=1)` and `M.mean()`.

Answer: `[5 7 9]` (columns: 1+4, 2+5, 3+6), `[6 15]` (rows), and 21 / 6 = `3.5` (no axis means all elements).

Intermediate

Problem 16.P.5 What does this print?

```
a = np.arange(6)
b = a[2:5]
b[0] = 99
c = a[[3, 4]]
c[0] = -1
print(a)
```

Answer: `[ 0 1 99 3 4 5]`. The slice `b` is a view, so writing 99 through it changes `a[2]`. `c` comes from fancy indexing, a copy, so the -1 never reaches `a`.

Problem 16.P.6 What do `[1, 2, 3] * 2` and `np.array([1, 2, 3]) * 2` print, and why do they differ?

Answer: the list gives `[1, 2, 3, 1, 2, 3]` (list repetition), the array gives `[2 4 6]` (element-wise arithmetic). Operators on lists mean concatenation and repetition; on arrays they mean maths.

Problem 16.P.7 Find the bug and fix it.

```
x = np.array([3, -1, 4])
if x > 0:
    print("all positive")
```

Answer: `x > 0` is an array of three booleans, and `if` needs one. NumPy raises `ValueError: The truth value of an array with more than one element is ambiguous`. Fix: `if (x > 0).all():` (here False, so nothing prints), or `.any()` if you mean "at least one".

Problem 16.P.8 Find the bug. `X` has shape (3, 4) and we want each row to have mean 0.

```
Xc = X - X.mean(axis=1)
```

Answer: `X.mean(axis=1)` has shape (3,), which aligns with the last axis of size 4, so broadcasting (3, 4) with (3,) fails with a `ValueError`. (If X were 3 by 3 it would silently subtract the wrong thing, row means applied to columns.) Fix: `Xc = X - X.mean(axis=1, keepdims=True)`, where the mean has shape (3, 1).

Problem 16.P.9 Find the bug. We want to replace missing readings by zero.

```
x = np.array([1.0, np.nan, 3.0])
x[x == np.nan] = 0
```

Answer: NaN is not equal to anything, including itself, so `x == np.nan` is all False and nothing is replaced. Fix: `x[np.isnan(x)] = 0`, giving `[1. 0. 3.]`.

Problem 16.P.10 What is printed by `print(repr(np.array([1, 2, 3], dtype=np.uint8) * 100))`?

Answer: `array([100, 200, 44], dtype=uint8)`. The Python int 100 is weak and takes the array's dtype uint8. 3 x 100 = 300 does not fit in 0..255, so it wraps: 300 - 256 = 44. There is no warning for array arithmetic.

Problem 16.P.11 What is the shape of `np.arange(24).reshape(2, -1, 4)`? Which call would fail and why: `np.arange(24).reshape(5, -1)`?

Answer: the `-1` is solved as 24 / (2 x 4) = 3, so shape `(2, 3, 4)`. With 5 rows, 24 / 5 is not a whole number, so NumPy raises `ValueError: cannot reshape array of size 24 into shape (5,newaxis)`-style message.

Problem 16.P.12 For `s = np.array([30, 10, 20])` give `np.argsort(s)` and the rank of each element (0 for the smallest), computed without a loop.

Answer: `order = np.argsort(s)` is `[1 2 0]`. Ranks: `r = np.empty_like(order); r[order] = np.arange(len(s))` gives `[2 0 1]`: 30 is the largest (rank 2), 10 the smallest (rank 0), 20 is in the middle (rank 1).

Advanced

Problem 16.P.13 Compute the moving average of width 2 of `x = np.array([1., 3., 5., 7.])` using `cumsum`, and state the length of the result.

Answer: `c = np.cumsum(np.insert(x, 0, 0))` is `[0 1 4 9 16]`. Then `(c[2:] - c[:-2]) / 2` is `[4-0, 9-1, 16-4] / 2 = [2. 4. 6.]`. The length is n - w + 1 = 3.

Problem 16.P.14 Compute the stable softmax of `z = np.array([1., 2., 3.])` and say why subtracting the maximum is allowed.

Answer: subtract 3: `[-2, -1, 0]`, exponentials 0.1353, 0.3679, 1.0, total 1.5032, so the result is `[0.09 0.245 0.665]` (rounded to three places). It is allowed because softmax is unchanged when a constant c is added to all scores: the factor e^{-c} cancels from numerator and denominator.

Problem 16.P.15 Assign points to the nearest centre without loops: `X = [[0,0],[1,0],[10,10],[9,9]]`, `C = [[0,0],[10,10]]`. Write the one-line code and the result.

Answer: `labels = ((X[:, None, :] - C[None, :, :]) ** 2).sum(axis=-1).argmin(axis=1)`. The difference array has shape (4, 2, 2); summing the last axis gives squared distances (4, 2); `argmin` over the centres gives `[0 0 1 1]`. This is the assignment step of k-means.

Problem 16.P.16 Compute the sliding maximum with window 3 of `[1, 3, 2, 5, 4, 1]` without a Python loop.

Answer: `sliding_window_view(x, 3).max(axis=1)` gives `[3 5 5 5]` (windows 1,3,2 / 3,2,5 / 2,5,4 / 5,4,1). The view costs no memory; the maximum runs along its last axis.

Problem 16.P.17 A teammate writes this function, and the results of her experiments are identical in every run of the loop. Why? Fix it.

```
def noisy(x):
    rng = np.random.default_rng(0)
    return x + rng.normal(0, 1, x.shape)

results = [noisy(np.zeros(3)) for _ in range(3)]
```

Answer: each call creates a new generator with the same seed, so each call starts the stream from the beginning and returns the same noise. Fix: create the generator once outside, `rng = np.random.default_rng(0)`, and pass it in: `def noisy(x, rng): ...`. Then each call continues the stream, and the whole run is still reproducible.

Expert

Problem 16.P.18 You need `np.trace(A @ B)` for two 5000 by 5000 matrices. Show a way that avoids the full matrix product, and state the saving.

Answer: the trace needs only the diagonal of the product, which is the sum over i and j of A_{ij} B_{ji}: `np.einsum("ij,ji->", A, B)` or `(A * B.T).sum()`. The full product costs about n^3 multiplications (1.25 x 10^{11} for n = 5000); the shortcut costs n^2 (2.5 x 10^7). On the small example A = [[1,2],[3,4]], B = [[0,1],[1,0]], both ways give 5.

Problem 16.P.19 Your all-pairs distance code `np.sqrt(((P[:, None] - P[None]) ** 2).sum(-1))` works for 1,000 points and crashes with a memory error for 10,000 points in 3 dimensions. Estimate the memory needed and give two remedies.

Answer: the difference array has shape (10000, 10000, 3) of float64: 10000 x 10000 x 3 x 8 bytes = 2.4 GB, and the squared copy needs the same again. Remedies: (1) use the expansion $|a|^2 + |b|^2 - 2\,a.b$ with one matrix product, which needs only an n by n result (0.8 GB), or `scipy.spatial.distance.cdist`; (2) process the rows in blocks of, say, 500 and write each (500, 10000) block of the answer, or use a tree structure such as `scipy.spatial.cKDTree` if you only need nearest neighbours.

Problem 16.P.20 Explain why the following two lines can give different results for the first element, and which of them modifies `img`. `img` is a uint8 array with `img[0, 0, 0] == 250`.

```
a = img + 10
b = np.clip(img.astype(np.int16) + 10, 0, 255).astype(np.uint8)
```

Answer: `a` wraps around: 250 + 10 = 260, which becomes 4 in uint8. `b` computes in int16 (room for 260), clips to 255, and converts back to give 255, the intended saturation. Neither line modifies `img`, because both build new arrays; only an in-place operation such as `img += 10` would write into it (and that would wrap too).

Problem 16.P.21 Interview: "You slice `small = big[:10]` from a 2 GB array, delete the name `big`, and your process still uses 2 GB. Why? What do you do?" Also state how to prove the cause.

Answer: the slice is a view; `small.base` is the big array, which keeps its whole buffer alive while any view exists. Prove it with `small.base is not None` and `small.base.nbytes`. Fix: `small = big[:10].copy()` before deleting `big`, so that the copy owns just its 10 rows and the big buffer can be freed.

Problem 16.P.22 A colleague computes a weighted group total with `totals[idx] += values`, where `idx = np.array([0, 0, 2])`, `values = np.array([1., 1., 1.])` and `totals = np.zeros(3)`. What does it give, what is right, and what is the fix?

Answer: it gives `[1. 0. 1.]`: with a repeated index, buffered fancy-index addition applies only one update to position 0. The correct answer is `[2. 0. 1.]`. Fix: `np.add.at(totals, idx, values)`, or `np.bincount(idx, weights=values, minlength=3)`, which is usually faster.

Prompt pack

✨ **AI prompt: copy and paste into any AI**

You are my patient, practical professor. I am a working professional moving into data roles, and English is my second language, so use simple English. Teach me NumPy step by step: ndarray, dtypes, shapes and strides, slicing and boolean masks, broadcasting, the axis argument, views versus copies, and vectorisation. Use short runnable examples with real output and everyday analogies. After each part, ask me one check question and wait for my answer before continuing. Correct my mistakes gently and tell me which pitfall I fell into.

✨ AI prompt: copy and paste into any AI

You are my patient, practical professor. I am a working professional moving into data roles, and English is my second language. Review my NumPy code below. Check for Python loops that could be vectorised, hidden copies, wrong axis arguments, broadcasting mistakes (shape (n,) versus (n, 1)), dtype problems such as integer overflow or float32 precision, NaN handling, and unseeded or badly seeded random numbers. For each issue show the smaller or faster code, explain why in simple English, and tell me how to verify the fix against a slow reference version. List the top 5 improvements in order of importance. My code: [PASTE HERE]

✨ AI prompt: copy and paste into any AI

You are my patient, practical professor. I am a working professional preparing for data science and machine learning interviews, and English is my second language. Run a 30-minute mock interview on NumPy. Ask one question at a time, mixing concept questions (views versus copies, broadcasting rules, axis, why arrays are fast, float precision) and small coding tasks (stable softmax, pairwise distances, one-hot encoding, moving average, k-means assignment, top-k per row) without loops. After each answer give a score out of 5, a model answer, and one tip on how to explain it clearly in English.

CHAPTER 17

PANDAS: THE COMPLETE PRACTICAL GUIDE

Library Handbook: selection, cleaning, groupby, merges, time series and a SQL-to-pandas translation.

It is Monday morning and the finance team has sent you three files: a CSV of last month's orders (40,000 rows), an Excel sheet with customer regions, and a JSON export from the payment provider. The question sounds simple: "Which regions grew, which customers stopped buying, and do our books match the payment provider?" With plain Python lists and dicts you could answer it, but you would write hundreds of lines and probably get a join wrong. With **pandas** the whole job is about thirty readable lines. Every analyst, data engineer and data scientist uses it daily, which is why interviews for data roles almost always include a pandas exercise.

This chapter is a practical handbook. It is organised by *task*: reading files, looking at data, selecting, cleaning, grouping, joining, reshaping, time series, speed. Each area starts with a reference table of the functions you will actually use, then worked examples that climb from trivial to realistic. Every code block runs on its own, and every printed result is the real output from pandas 3.0 on NumPy 2.x. By the end you will be able to solve a new pandas problem methodically instead of by search-and-paste.

Words you need. **Vectorised** means one call works on a whole column at once, with the loop running in fast compiled code. **Index** is the set of row labels. **dtype** is the data type of a column. **NaN** is "not a number", the usual marker for a missing value. **Copy-on-write** is the rule (always on in pandas 3.0) that a changed object never silently changes another one.

17.1 What pandas Is, When to Use It, and the Mental Model

INTUITION

pandas is a spreadsheet you drive with code. A **DataFrame** is the sheet, a **Series** is one column, and the **Index** is the row labels. Unlike a spreadsheet it can hold millions of rows, never loses your steps (they are code), and can be re-run tomorrow on new data.

What it is for, and what it is not for

Use pandas when	Think twice when
Data fits in memory (rule of thumb: comfortably under a quarter of your RAM)	Data is larger than memory: consider Polars (lazy engine), DuckDB (SQL on files) or Spark
Tabular data with mixed types: CSV, Excel, JSON, SQL results, Parquet	Pure numeric arrays with no labels: NumPy is simpler and faster
Exploration, cleaning, reshaping, joins, group summaries, time series	Heavy row-by-row logic with branching: plain Python or a database may be clearer
Glue between files, SQL, plotting and machine-learning libraries	Multi-user transactional storage: use a database

Install, version check, import convention

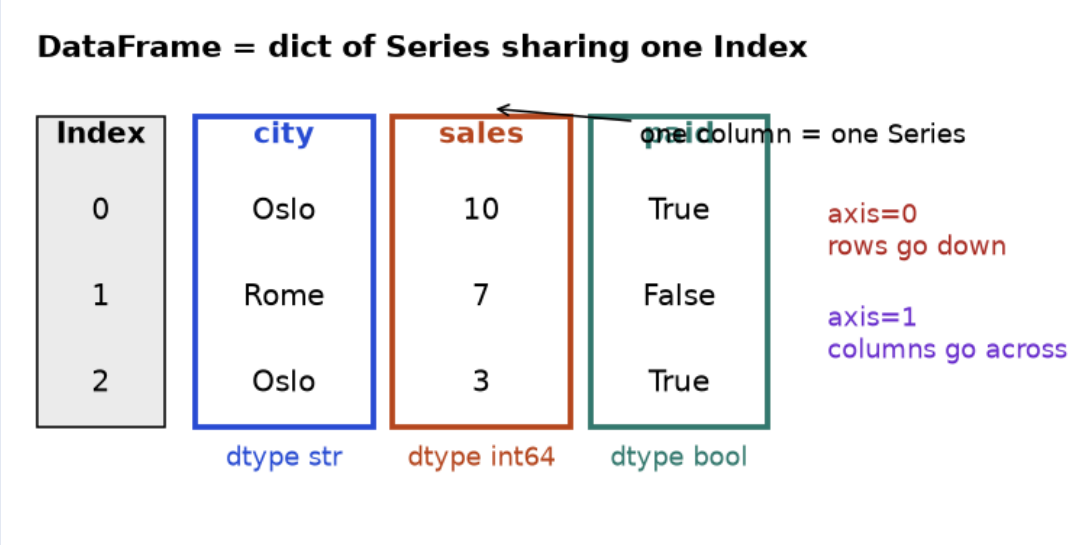
Install with `pip install pandas` (add `openpyxl` for `.xlsx` files and `pyarrow` for Parquet; the second is *not run here*, because we only use what is installed). The universal import convention is `import pandas as pd` together with `import numpy as np`. Always check the version, because behaviour changed in 3.0.

```
import pandas as pd
import numpy as np

print(pd.__version__)
print(np.__version__)
print(pd.options.display.max_rows)
```

```
3.0.6
2.5.3
60
```

1. `import pandas as pd` loads the library under the short name every tutorial uses.
2. `import numpy as np` is imported alongside because pandas columns are built on NumPy arrays and you will use `np.nan`, `np.where` and friends.
3. `pd.__version__` prints the version string. Here it is a 3.0 release, so string columns default to the new `str` dtype and copy-on-write is always on.
4. The NumPy version should be 2.x. `pd.options` holds display and behaviour settings; `max_rows` is the number of rows printed before pandas truncates with `...`.

The mental model



A DataFrame is a dict of Series that share one Index. Each Series has exactly one dtype.

How to read it: the grey box on the left is the Index (row labels 0, 1, 2). Each coloured box is a column, and its dtype is written underneath. Because columns are separate, a DataFrame can mix text, numbers and booleans, but inside one column everything has the same type. Operations *align on the Index labels*, not on position: that single rule explains most surprising pandas behaviour, including joins, arithmetic between Series and assigning a Series as a new column.

KEY IDEA	Think in columns, not rows. Instead of "for each row, do X", say "take this column, apply X to all of it". Almost every fast and readable pandas line is a whole-column statement.

17.2 Series, DataFrame and Index

Name	Plain English	Minimal example
`pd.Series(data, index=)`	One labelled column	`pd.Series([3,5], index=["a","b"])`

Name	Plain English	Minimal example
`pd.DataFrame(dict)`	Table from a dict of columns	`pd.DataFrame({"x":[1,2]})`
`pd.DataFrame(list_of_dicts)`	Table from records (one dict per row)	`pd.DataFrame([{"x":1},{"x":2}])`
`df.shape`	(rows, columns)	`df.shape`
`df.columns`	Column labels (an Index)	`list(df.columns)`
`df.index`	Row labels	`df.index`
`df.dtypes`	Type of each column	`df.dtypes`
`df.values` / `df.to_numpy()`	Plain NumPy array of the data	`df.to_numpy()`
`s.to_frame()`	Series to one-column DataFrame	`s.to_frame("n")`
`df.set_index("col")`	Move a column into the index	`df.set_index("id")`
`df.reset_index()`	Move the index back to a column	`df.reset_index(drop=True)`
`df.rename(columns=)`	Rename columns	`df.rename(columns={"a":"b"})`
`df.reindex(new_labels)`	Conform to a new index, inserting NaN	`s.reindex(["a","z"])`
`df.sort_index()`	Sort by labels	`df.sort_index()`
`df.sort_values(by)`	Sort by column values	`df.sort_values("x", ascending=False)`
`df.T`	Transpose	`df.T`
`df.copy()`	Explicit independent copy	`df2 = df.copy()`
`pd.RangeIndex`	Default 0..n-1 index, stored cheaply	`pd.RangeIndex(3)`
`pd.DatetimeIndex`	Index of timestamps (enables time slicing)	`pd.date_range("2025-01-01", periods=3)`
`idx.is_unique`	Does every label appear once?	`df.index.is_unique`
`pd.concat([..], axis=1)`	Glue Series into a DataFrame	`pd.concat([a, b], axis=1)`

Beginner: build the three objects

```
import pandas as pd

s = pd.Series([10, 7, 3], index=["Oslo", "Rome", "Lima"], name="sales")
print(s)
print(s["Rome"], s.iloc[0])
print(s.index)
print(s.dtype)
```

```
Oslo    10
Rome     7
Lima     3
Name: sales, dtype: int64
7 10
Index(['Oslo', 'Rome', 'Lima'], dtype='str')
int64
```

1. A Series is built from a list plus an optional list of labels. `name="sales"` becomes the column name if the Series is later put in a DataFrame.
2. `print(s)` shows labels on the left, values on the right, and a footer with the name and dtype.
3. `s["Rome"]` looks up by *label*. `s.iloc[0]` looks up by *position*. Labels and positions are different things; keep them apart.
4. `s.index` is an `Index` object. In pandas 3.0 an index of text has dtype `str`.
5. `s.dtype` is `int64`, the type of the values.

Intermediate: a DataFrame from records, and index alignment

```
import pandas as pd

rows = [
    {"city": "Oslo", "sales": 10, "paid": True},
    {"city": "Rome", "sales": 7, "paid": False},
    {"city": "Oslo", "sales": 3, "paid": True},
]
df = pd.DataFrame(rows)
print(df)
print(df.dtypes)

a = pd.Series([1, 2, 3], index=["x", "y", "z"])
b = pd.Series([10, 20, 30], index=["y", "z", "w"])
print(a + b)
```

```
   city  sales   paid
0  Oslo     10   True
1  Rome      7  False
2  Oslo      3   True
city       str
sales    int64
paid      bool
dtype: object
w     NaN
x     NaN
y    12.0
z    23.0
dtype: float64
```

1. A list of dicts is the natural shape of JSON and of API responses. Each dict is a row; keys become columns.
2. `df.dtypes` confirms that pandas guessed `str`, `int64` and `bool`. If a key were missing from some dict, that cell would be NaN.
3. Series `a` has labels x, y, z; `b` has y, z, w.
4. `a + b` matches by *label*, not position. Labels y and z exist in both, so they add (12 and 23). Labels x and w exist in only one, so the result is NaN. The ints became floats because NaN needs a float column.

REAL JOB

Alignment is why pandas is safe for joining two columns from different tables: it will not silently pair row 5 of one with row 5 of the other if their ids differ. It is also why a stray duplicate or mismatched index suddenly fills a column with NaN. When you see unexpected NaN after arithmetic, compare the two indexes first.

Expert: custom index, set_index, reindex

```
import pandas as pd

df = pd.DataFrame({"id": [101, 102, 103], "name": ["Ana", "Bo", "Cy"], "score": [88, 92, 75]})
by_id = df.set_index("id")
print(by_id)
print(by_id.loc[102, "name"])
print(by_id["score"].reindex([101, 999]))
print(by_id.reset_index().columns.tolist())
print(by_id.index.is_unique, df.index)
```

```
    name  score
id
101  Ana     88
102   Bo     92
103   Cy     75
Bo
id
101    88.0
999     NaN
Name: score, dtype: float64
['id', 'name', 'score']
True RangeIndex(start=0, stop=3, step=1)
```

1. The `id` column becomes the index, so `loc[102, ...]` means "the row whose id is 102".
2. `by_id.loc[102, "name"]` returns a single value, `Bo`.
3. `reindex([101, 999])` asks for exactly those labels. 999 does not exist, so it gets NaN instead of an error. This is the safe way to "look up and tolerate misses".
4. `reset_index()` puts `id` back as an ordinary column at the front.
5. `is_unique` is worth checking before you rely on an index as a key. `df.index` shows the default `RangeIndex`, which stores only start, stop and step, so it costs almost no memory.

DEFINITION	**axis** names a direction. `axis=0` (or `"index"`) means "going down the rows", so `df.sum(axis=0)` gives one number per column. `axis=1` means "going across the columns", so `df.sum(axis=1)` gives one number per row. Writing the word (`axis="columns"`) is clearer than the digit.

17.3 Reading and Writing CSV, Excel and JSON

Name	Plain English	Minimal example
`pd.read_csv(path)`	Load a CSV (path, URL or file-like)	`pd.read_csv("a.csv")`
`sep=`	Field separator	`read_csv(p, sep=";")`
`usecols=`	Load only some columns (saves memory)	`read_csv(p, usecols=["id","x"])`
`dtype=`	Force column types	`read_csv(p, dtype={"zip":"str"})`
`parse_dates=`	Parse columns as datetimes	`read_csv(p, parse_dates=["when"])`
`na_values=`	Extra strings that mean missing	`read_csv(p, na_values=["n/a","-"])`
`index_col=`	Use a column as the index	`read_csv(p, index_col="id")`

Name	Plain English	Minimal example
`nrows=`	Read only the first n rows (peek)	`read_csv(p, nrows=5)`
`skiprows=`	Skip junk header lines	`read_csv(p, skiprows=2)`
`chunksize=`	Iterate over the file in pieces	`for c in read_csv(p, chunksize=1000): ...`
`encoding=`	Text encoding	`read_csv(p, encoding="latin-1")`
`df.to_csv(path, index=False)`	Write CSV without the index column	`df.to_csv("o.csv", index=False)`
`pd.read_excel(path, sheet_name=)`	Load a sheet (needs openpyxl)	`read_excel("a.xlsx", sheet_name="Q1")`
`df.to_excel(path)`	Write a sheet	`df.to_excel("o.xlsx", index=False)`
`pd.ExcelWriter`	Write several sheets to one file	`with pd.ExcelWriter(p) as w: ...`
`pd.read_json(path)`	Load JSON (records, columns, lines)	`read_json(p, lines=True)`
`pd.json_normalize(list)`	Flatten nested JSON into columns	`json_normalize(data, sep="_")`
`df.to_json(orient=)`	Write JSON in a chosen shape	`df.to_json(orient="records")`
`pd.read_parquet / to_parquet`	Fast typed columnar format (needs pyarrow; not run here)	`df.to_parquet("o.parquet")`
`pd.read_sql(query, conn)`	Run SQL, get a DataFrame	`read_sql("select * from t", conn)`
`pd.read_clipboard()`	Paste from a spreadsheet (handy, not for pipelines)	`pd.read_clipboard()`
`io.StringIO(text)`	Pretend a string is a file (great for examples)	`read_csv(StringIO(txt))`

Beginner: read a CSV and look at what pandas guessed

```
import io
import pandas as pd

text = """order_id,customer,amount,placed
1001,ana,25.5,2025-03-01
1002,bo,,2025-03-02
1003,ana,12.0,2025-03-04
"""
df = pd.read_csv(io.StringIO(text))
print(df)
print(df.dtypes)
```

```
   order_id customer  amount      placed
0      1001      ana    25.5  2025-03-01
1      1002       bo     NaN  2025-03-02
2      1003      ana    12.0  2025-03-04
order_id      int64
customer        str
amount      float64
placed          str
dtype: object
```

1. `io.StringIO(text)` makes the string behave like an open file, so the example needs no file on disk. With a real file you would write `pd.read_csv("orders.csv")`.
2. The first line becomes the column names. Each later line becomes a row.
3. The empty amount for order 1002 becomes `NaN`, and because NaN is a float, the whole `amount` column is `float64`.
4. The `placed` column stays text (`str`), because we did not ask for dates. That is the most common surprise: pandas does not parse dates unless told.

Intermediate: control types, dates and missing markers on the way in

```
import io
import pandas as pd

text = """zip;customer;amount;placed
01234;ana;1,5;2025-03-01
02345;bo;n/a;2025-03-02
"""
df = pd.read_csv(
    io.StringIO(text),
    sep=";",
    dtype={"zip": "str"},
    decimal=",",
    na_values=["n/a"],
    parse_dates=["placed"],
)
print(df)
print(df.dtypes)
```

```
     zip customer  amount     placed
0  01234      ana     1.5 2025-03-01
1  02345       bo     NaN 2025-03-02
zip                   str
customer              str
amount            float64
placed      datetime64[us]
dtype: object
```

1. `sep=";"`: European exports often use semicolons because commas are decimal marks.
2. `dtype={"zip": "str"}` keeps the leading zero. Without it, `01234` becomes the integer 1234 and the zero is lost for ever.
3. `decimal=","` reads `1,5` as the number 1.5.
4. `na_values=["n/a"]` adds a custom missing marker. pandas already treats empty cells, `NA`, `NaN` and `null` as missing.
5. `parse_dates=["placed"]` converts the column to datetimes. In pandas 3.0 the unit is chosen automatically (here microseconds, shown as `datetime64[us]`).

Realistic: write and read back, Excel, and nested JSON

```
import json
import tempfile
from pathlib import Path
import pandas as pd

df = pd.DataFrame({"id": [1, 2], "region": ["north", "south"], "sales": [10.5, 20.0]})
folder = Path(tempfile.mkdtemp())

df.to_csv(folder / "a.csv", index=False)
print((folder / "a.csv").read_text())

df.to_excel(folder / "a.xlsx", sheet_name="Q1", index=False)
print(pd.read_excel(folder / "a.xlsx", sheet_name="Q1"))

payload = [
    {"id": 1, "user": {"name": "Ana", "city": "Oslo"}, "items": [{"sku": "A", "qty": 2}, {"sku": "B", "qty": 1}]},
    {"id": 2, "user": {"name": "Bo", "city": "Rome"}, "items": [{"sku": "A", "qty": 5}]},
]
print(pd.json_normalize(payload, sep="_").drop(columns="items"))
flat = pd.json_normalize(payload, record_path="items", meta=["id", ["user", "name"]])
print(flat)
print(df.to_json(orient="records"))
```

```
id,region,sales
1,north,10.5
2,south,20.0

   id region  sales
0   1  north   10.5
1   2  south   20.0
   id user_name user_city
0   1       Ana      Oslo
1   2        Bo      Rome
  sku  qty id user.name
0   A    2  1       Ana
1   B    1  1       Ana
2   A    5  2        Bo
[{"id":1,"region":"north","sales":10.5},{"id":2,"region":"south","sales":20.0}]
```

1. `tempfile.mkdtemp()` gives a throwaway folder so the example leaves nothing behind.
2. `to_csv(..., index=False)` writes only your columns. Forgetting `index=False` adds an unnamed first column, which then comes back as `Unnamed: 0` when you reload.
3. `to_excel` and `read_excel` use the installed `openpyxl` engine. The sheet name is how you pick one sheet out of many.
4. `json_normalize(payload, sep="_")` flattens nested dicts: `user` becomes `user_name` and `user_city`. The list column `items` stays as a list, so we drop it for display.
5. `record_path="items"` explodes the nested list into one row per item, and `meta=[...]` copies parent fields onto each row. This is the standard way to turn "orders with line items" JSON into a flat table.
6. `to_json(orient="records")` writes a list of row dicts, the shape most APIs expect.

COMMON MISTAKE

Reading codes as numbers. Zip codes, phone numbers and product ids with leading zeros must be read with `dtype=str`. Fix it at read time; repairing later with `zfill` only works if you know the original width.

17.4 Inspecting a DataFrame

Name	Plain English	Minimal example
`df.head(n) / tail(n)`	First or last n rows	`df.head(3)`
`df.sample(n, random_state=)`	Random rows (reproducible with a seed)	`df.sample(5, random_state=0)`
`df.shape`	Rows and columns	`df.shape`
`df.info()`	Columns, non-null counts, dtypes, memory	`df.info()`
`df.describe()`	Summary stats of numeric columns	`df.describe()`
`df.describe(include="all")`	Also text and dates	`df.describe(include="all")`
`s.value_counts()`	Frequency table of a column	`df["city"].value_counts()`
`value_counts(normalize=True)`	Shares instead of counts	`s.value_counts(normalize=True)`
`s.nunique()`	Number of distinct values	`df["id"].nunique()`
`s.unique()`	The distinct values themselves	`df["city"].unique()`
`df.isna().sum()`	Missing count per column	`df.isna().sum()`
`df.duplicated().sum()`	Number of repeated rows	`df.duplicated().sum()`
`df.memory_usage(deep=True)`	Bytes per column (deep counts text)	`df.memory_usage(deep=True)`
`df.select_dtypes(include=)`	Pick columns by dtype	`df.select_dtypes("number")`
`df.corr(numeric_only=True)`	Correlation matrix	`df.corr(numeric_only=True)`
`s.min() / max() / mean() / median()`	Basic aggregates	`df["x"].median()`
`s.quantile(q)`	Percentile	`s.quantile([.25,.75])`
`s.idxmax() / idxmin()`	Index label of the biggest or smallest	`s.idxmax()`
`df.nlargest(n, col)`	Top n rows by a column	`df.nlargest(3, "sales")`
`df.agg(["min","max"])`	Several aggregates at once	`df.agg(["min","max"])`
`pd.set_option`	Display settings	`pd.set_option("display.max_columns", 50)`

The 60-second health check

Whenever a new table arrives, run the same checks in the same order: size, types, missing, duplicates, ranges, categories. It catches 90 percent of data problems before they become wrong answers.

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(42)
n = 12
df = pd.DataFrame({
    "order_id": np.arange(1, n + 1),
    "city": rng.choice(["Oslo", "Rome", "Lima"], n),
    "amount": rng.normal(50, 20, n).round(2),
})
df.loc[3, "amount"] = np.nan
df = pd.concat([df, df.iloc[[0]]], ignore_index=True)

print(df.shape)
df.info()
print(df.describe().round(2))
print(df["city"].value_counts())
print(df.isna().sum().to_dict(), df.duplicated().sum())
```

```
(13, 3)
<class 'pandas.DataFrame'>
RangeIndex: 13 entries, 0 to 12
Data columns (total 3 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   order_id  13 non-null     int64
 1   city      13 non-null     str
 2   amount    12 non-null     float64
dtypes: float64(1), int64(1), str(1)
memory usage: 444.0 bytes
       order_id  amount
count     13.00   12.00
mean       6.08   52.99
std        3.77   12.84
min        1.00   30.82
25%        3.00   48.16
50%        6.00   52.56
75%        9.00   60.90
max       12.00   72.54
city
Oslo    5
Lima    4
Rome    4
Name: count, dtype: int64
{'order_id': 0, 'city': 0, 'amount': 1} 1
```

1. The seeded generator `default_rng(42)` makes the "random" data identical every run.
2. `df.loc[3, "amount"] = np.nan` plants one missing value, and `concat` with `df.iloc[[0]]` appends a copy of the first row, so we have a duplicate to find.
3. `df.shape` is a tuple: 13 rows, 3 columns.
4. `df.info()` prints directly (it returns `None`, so do not wrap it in `print`). Read the "Non-Null Count" column: 12 of 13 for `amount` reveals the missing cell.
5. `describe()` gives count, mean, std, min, quartiles and max. Check that min and max are plausible: negative amounts or a max of 10 million are red flags.
6. `value_counts()` shows how many rows per city; a misspelling like "oslo" would appear as its own row here.
7. `isna().sum()` counts missing cells per column, and `duplicated().sum()` counts rows that repeat an earlier row exactly.

KEY IDEA

Count before you clean. Write down how many rows you have at the start, after each filter, and at the end. A pipeline that silently drops 30 percent of rows is the most common expensive pandas bug.

17.5 Selecting: loc, iloc, Boolean Masks and query

Name	Plain English	Minimal example
`df["col"]`	One column as a Series	`df["city"]`
`df[["a","b"]]`	Several columns as a DataFrame	`df[["city","sales"]]`
`df.loc[rows, cols]`	Select by label (end of a slice is included)	`df.loc[0:2, "city"]`
`df.iloc[rows, cols]`	Select by position (end excluded)	`df.iloc[0:2, 0]`
`df.at[r, c] / df.iat[i, j]`	Fast single value by label / position	`df.at[0, "city"]`
`df[mask]`	Rows where a Boolean Series is True	`df[df["sales"] > 5]`
`& | ~`	and, or, not on masks (brackets required)	`(a > 1) & (b < 3)`
`s.isin(values)`	Membership test	`df["city"].isin(["Oslo","Rome"])`
`s.between(lo, hi)`	Range test (inclusive by default)	`df["sales"].between(5, 10)`
`s.str.contains(pat)`	Text pattern test	`df["city"].str.contains("os")`
`s.isna() / notna()`	Missing tests	`df[df["x"].notna()]`
`df.query("expr")`	Filter with a readable string	`df.query("sales > 5 and city == 'Oslo'")`
`query("@var")`	Use a Python variable inside query	`df.query("sales > @cut")`
`df.filter(items/like/regex)`	Select columns by name pattern	`df.filter(like="amt")`
`df.drop(columns=)`	Remove columns	`df.drop(columns=["tmp"])`
`df.drop(index=)`	Remove rows by label	`df.drop(index=[0, 3])`
`df.loc[mask, "col"] = v`	Assign into selected cells	`df.loc[df.x < 0, "x"] = 0`
`df.where(cond, other)`	Keep where True, else replace	`df["x"].where(df.x > 0, 0)`
`np.where(cond, a, b)`	Vectorised if/else	`np.where(df.x > 0, "pos", "neg")`
`df.take / df.xs`	Positional take / cross-section of a MultiIndex	`df.xs("Oslo", level="city")`
`df.nsmallest / nlargest`	Top or bottom n rows	`df.nlargest(2, "sales")`
`df.head(n)` / `df.iloc[:n]`	First n rows	`df.iloc[:5]`
`df.sort_values().head()`	Top n by sorting	`df.sort_values("x").head(3)`

Beginner: columns, labels, positions

```
import pandas as pd

df = pd.DataFrame(
    {"city": ["Oslo", "Rome", "Lima", "Oslo"], "sales": [10, 7, 3, 12]},
    index=[10, 11, 12, 13],
)
print(df["city"].tolist())
print(df.loc[11, "sales"])
print(df.iloc[1, 1])
print(df.loc[10:12, "city"].tolist())
print(df.iloc[0:2].index.tolist())
```

```
['Oslo', 'Rome', 'Lima', 'Oslo']
7
7
['Oslo', 'Rome', 'Lima']
[10, 11]
```

1. `df["city"]` returns a Series; `.tolist()` turns it into a plain list so the print is short.
2. `df.loc[11, "sales"]` means "the row labelled 11, column sales", which is 7.
3. `df.iloc[1, 1]` means "second row, second column" by position. It is also 7 here because label 11 sits in position 1.
4. `loc[10:12]` is a *label* slice and includes the end label 12, so three rows come back.
5. `iloc[0:2]` is a *position* slice and excludes the end, so two rows come back, with labels 10 and 11.

COMMON MISTAKE

Mixing up loc and iloc end points. `loc[0:2]` returns three rows, `iloc[0:2]` returns two. And with a non-default index, `s[0]` no longer falls back to position in pandas 3.0: it raises a `KeyError`. Say `loc` or `iloc` every time.

Intermediate: Boolean masks and query

```
import pandas as pd

df = pd.DataFrame({
    "city": ["Oslo", "Rome", "Lima", "Oslo", "Rome"],
    "sales": [10, 7, 3, 12, 9],
    "paid": [True, False, True, True, False],
})
mask = (df["sales"] > 5) & df["paid"]
print(mask.tolist())
print(df[mask])
print(df[df["city"].isin(["Rome", "Lima"]) | (df["sales"] > 11)])
print(df[~df["paid"]]["sales"].sum())
cut = 8
print(df.query("sales > @cut and city != 'Rome'"))
```

```
[True, False, False, True, False]
   city  sales  paid
0  Oslo     10  True
3  Oslo     12  True
   city  sales   paid
1  Rome      7  False
2  Lima      3   True
3  Oslo     12   True
4  Rome      9  False
16
   city  sales  paid
0  Oslo     10  True
3  Oslo     12  True
```

1. Each comparison returns a Series of True/False. `&` combines them element by element. Every comparison needs its own brackets because `&` binds tighter than `>`.
2. The column `paid` is already Boolean, so it works as a mask on its own.
3. `df[mask]` keeps the rows where the mask is True and keeps their original labels (0 and 3).
4. Line 3 uses `|` (or) with `isin`: rows in Rome or Lima, or with sales above 11.
5. `~df["paid"]` flips True and False, selecting the unpaid rows (sales 7 and 9, total 16).
6. `query` takes the condition as a string, which reads like SQL. `@cut` pulls in the Python variable. Use `and` / `or` inside query, but `&` / `|` outside it.

COMMON MISTAKE

Using and / or on Series. `df[(df.a > 1) and (df.b < 2)]` raises "The truth value of a Series is ambiguous". Python's `and` wants one True or False, but a Series holds many. Use `&`, `|`, `~` with brackets.

Expert: assigning safely with loc, where and np.where

```
import numpy as np
import pandas as pd

df = pd.DataFrame({"x": [-5, 2, -1, 8], "grp": ["a", "b", "a", "b"]})
df.loc[df["x"] < 0, "x"] = 0
print(df["x"].tolist())

df["sign"] = np.where(df["x"] > 0, "pos", "zero")
df["capped"] = df["x"].where(df["x"] < 5, 5)
df["label"] = np.select([df["x"] == 0, df["x"] < 5], ["none", "low"], default="high")
print(df)
```

```
[0, 2, 0, 8]
   x grp  sign  capped label
0  0   a  zero       0  none
1  2   b   pos       2   low
2  0   a  zero       0  none
3  8   b   pos       5  high
```

1. `df.loc[mask, "x"] = 0` changes only the cells where the mask is True. This one-step form is the safe way to assign into a selection.
2. `np.where(cond, a, b)` builds a whole new column: "pos" where x is positive, else "zero".
3. `Series.where(cond, other)` keeps values where the condition is True and replaces the rest. Here it caps x at 5.
4. `np.select(conditions, choices, default)` handles more than two outcomes. The conditions are checked in order and the first True wins, so `x == 0` is tested before `x < 5`.

17.6 Assigning Columns and Choosing dtypes

Name	Plain English	Minimal example
`df["new"] = ...`	Add or overwrite a column in place	`df["tax"] = df.price * 0.2`
`df.assign(new=...)`	Return a new frame with extra columns (chainable)	`df.assign(tax=lambda d: d.price*.2)`
`df.insert(pos, name, vals)`	Add a column at a chosen position	`df.insert(0, "k", 1)`
`df.rename(columns=)`	Rename	`df.rename(columns=str.lower)`
`df.astype(dict)`	Convert types	`df.astype({"id": "int32"})`
`pd.to_numeric(s, errors=)`	Parse numbers; `"coerce"` turns junk into NaN	`pd.to_numeric(s, errors="coerce")`
`pd.to_datetime(s, format=)`	Parse dates	`pd.to_datetime(s, format="%d/%m/%Y")`
`s.astype("category")`	Few distinct values, stored as codes	`df["city"].astype("category")`
`pd.Categorical(values, categories, ordered)`	Ordered categories (S < M < L)	`pd.Categorical(["S"], ["S","M"], ordered=True)`
`"Int64", "Float64", "boolean"`	Nullable dtypes that hold `pd.NA` without turning to float	`s.astype("Int64")`
`"string[pyarrow]" / "str"`	Text dtypes (default `str` in 3.0)	`s.astype("str")`
`df.convert_dtypes()`	Switch to best nullable dtypes	`df.convert_dtypes()`
`df.infer_objects()`	Re-guess dtypes of object columns	`df.infer_objects()`
`pd.api.types.is_numeric_dtype`	Type check helpers	`is_numeric_dtype(df.x)`
`s.replace(old, new)`	Swap values	`s.replace({"N":"No"})`
`s.map(dict_or_func)`	Look up each value	`s.map({"a": 1})`
`s.clip(lo, hi)`	Limit to a range	`s.clip(0, 100)`
`s.round(n)`	Round	`s.round(2)`
`s.diff() / pct_change()`	Change vs previous row	`s.pct_change()`
`s.cumsum() / cummax()`	Running total / maximum	`s.cumsum()`
`s.rank()`	Rank values	`s.rank(ascending=False)`
`s.shift(k)`	Move values down by k	`s.shift(1)`

Beginner: add columns two ways

```
import pandas as pd

df = pd.DataFrame({"item": ["pen", "book", "lamp"], "price": [2.0, 12.5, 30.0], "qty": [10, 2, 1]})
df["total"] = df["price"] * df["qty"]
print(df)

priced = df.assign(vat=lambda d: (d["total"] * 0.2).round(2), big=lambda d: d["total"] > 20)
print(priced[["item", "vat", "big"]])
print(list(df.columns))
```

```
   item  price  qty  total
0   pen    2.0   10   20.0
1  book   12.5    2   25.0
2  lamp   30.0    1   30.0
   item  vat    big
0   pen  4.0  False
1  book  5.0   True
2  lamp  6.0   True
['item', 'price', 'qty', 'total']
```

1. Multiplying two columns works row by row: 2.0 x 10, 12.5 x 2, 30.0 x 1.
2. `df["total"] = ...` adds the column to `df` itself.
3. `assign` returns a *new* DataFrame and leaves `df` alone. The `lambda d: ...` receives the frame as it is at that moment, so a later argument can use a column created by an earlier one.
4. The last line shows `df` has no `vat` or `big`: `assign` did not change it.

Intermediate: dtypes that save memory and prevent bugs

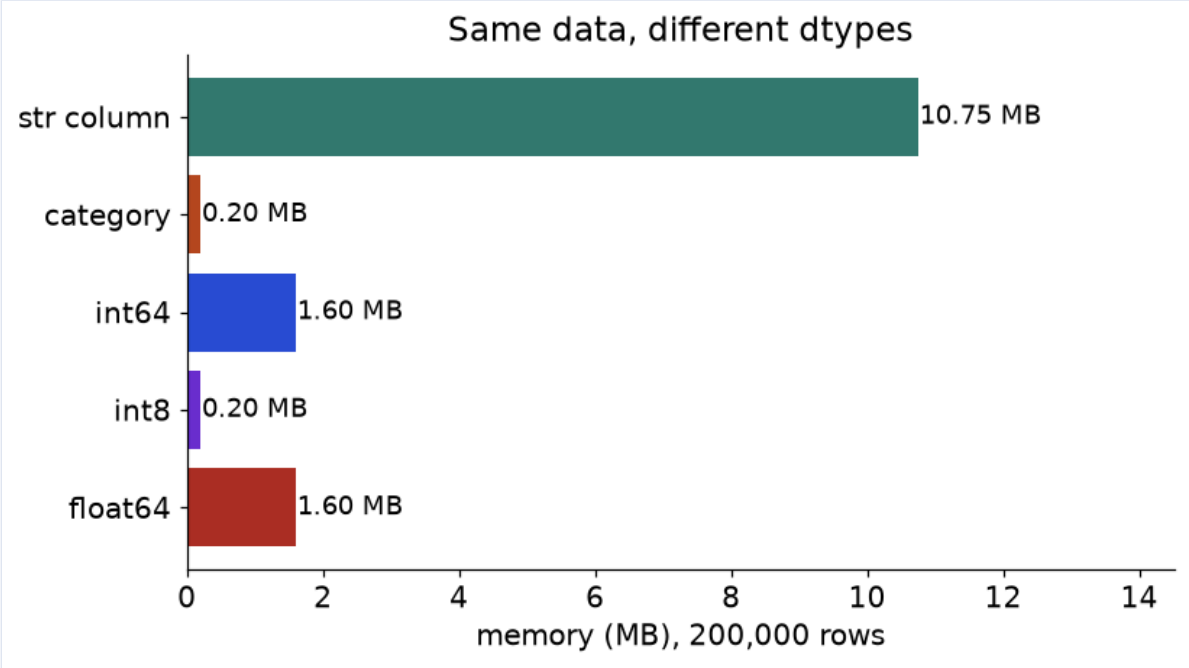
```
import pandas as pd

raw = pd.Series(["12", "7", "oops", "30", None])
nums = pd.to_numeric(raw, errors="coerce")
print(nums.tolist())

ints = nums.astype("Int64")
print(ints.tolist(), ints.dtype)

size = pd.Series(["M", "S", "L", "M", "S"])
cat = pd.Series(pd.Categorical(size, categories=["S", "M", "L"], ordered=True))
print(cat.dtype, (cat >= "M").tolist())
print(cat.sort_values().tolist(), cat.cat.codes.tolist())

dates = pd.to_datetime(pd.Series(["03/04/2025", "15/04/2025"]), format="%d/%m/%Y")
print(dates.dt.month.tolist(), dates.dtype)
```

```
[12.0, 7.0, nan, 30.0, nan]
[12, 7, <NA>, 30, <NA>] Int64
category [True, False, True, True, False]
['S', 'S', 'M', 'M', 'L'] [1, 0, 2, 1, 0]
[4, 4] datetime64[us]
```

1. `to_numeric(..., errors="coerce")` turns "oops" and the missing value into NaN instead of crashing. The default `errors="raise"` is better when you want to be told about bad data.
2. `astype("Int64")` (capital I) is the *nullable* integer: it keeps integers as integers and shows the gap as `<NA>`. The plain `int64` cannot hold a missing value.

3. An ordered `Categorical` knows S < M < L, so comparisons (`>= "M"`) and sorting follow that order, not the alphabet (where L would come before M).
4. `cat.cat.codes` are the small integers pandas actually stores: S=0, M=1, L=2. That is why categories save memory.
5. Always give `format=` to `to_datetime` for non-ISO dates. "03/04/2025" is 3 April in Europe but 4 March in the US, and guessing is how silent bugs are born.

Expert: what each dtype costs



Memory for the same 200,000 values stored with different dtypes. Measured on one machine; exact bytes vary by platform and version.

How to read it: each bar is one column of 200,000 values. The text column built from four repeated words is the longest bar. Converting it to `category` stores each word once plus a small integer code per row, so the bar shrinks a lot. For numbers, shrinking `int64` to `int8` divides memory by eight when values fit between -128 and 127. The lesson: low-cardinality text becomes category, and numbers get the smallest safe type.

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(0)
n = 200_000
city = pd.Series(rng.choice(["Oslo", "Rome", "Lima", "Kyiv"], n))
qty = pd.Series(rng.integers(0, 100, n))

print(round(city.memory_usage(deep=True) / 1e6, 2), "MB as str")
print(round(city.astype("category").memory_usage(deep=True) / 1e6, 2), "MB as category")
print(round(qty.memory_usage(deep=True) / 1e6, 2), "MB int64")
print(round(qty.astype("int8").memory_usage(deep=True) / 1e6, 2), "MB int8")
print(qty.astype("int8").max(), np.iinfo("int8").max)
```

```
10.6 MB as str
0.2 MB as category
1.6 MB int64
0.2 MB int8
99 127
```

1. 200,000 values drawn from 4 words; `default_rng(0)` makes it repeatable.
2. `memory_usage(deep=True)` measures real bytes including the text; without `deep=True` pandas under-counts text columns.

3. The category version stores 4 strings once plus one byte per row, which is why it is far smaller.
4. The int8 conversion is only safe because the largest value, 99, is below `np.iinfo("int8").max` (127). If a value did not fit, the conversion would silently wrap or raise; always check the range first, or use `pd.to_numeric(s, downcast="integer")` which picks a safe size for you.

REAL JOB

Rule of thumb: if a text column has fewer distinct values than about half of its rows, try `category`. It cuts memory and speeds up `groupby`. If the column is nearly all unique (ids, free text), keep it as `str`.

17.7 Missing Data and Duplicates

Name	Plain English	Minimal example
`df.isna() / notna()`	Boolean map of missing cells	`df.isna()`
`df.isna().sum()`	Missing per column	`df.isna().sum()`
`df.isna().mean()`	Share missing per column	`df.isna().mean().round(2)`
`df.dropna()`	Drop rows with any missing value	`df.dropna()`
`dropna(subset=)`	Only look at some columns	`df.dropna(subset=["id"])`
`dropna(how="all")`	Drop only fully empty rows	`df.dropna(how="all")`
`dropna(thresh=k)`	Keep rows with at least k real values	`df.dropna(thresh=2)`
`fillna(value)`	Fill with a constant	`df["x"].fillna(0)`
`fillna(df.mean())`	Fill each column with its own value	`df.fillna(df.mean(numeric_only=True))`
`ffill() / bfill()`	Carry the last / next value	`s.ffill()`
`interpolate()`	Estimate between neighbours	`s.interpolate()`
`groupby().transform("median")`	Fill with a group-specific statistic	`g.transform("median")`
`replace(value, np.nan)`	Turn sentinels such as -999 into NaN	`df.replace(-999, np.nan)`
`pd.NA`	The missing marker of nullable dtypes	`pd.Series([1, None], dtype="Int64")`
`df.duplicated(subset, keep)`	Mark repeated rows	`df.duplicated(["id"], keep=False)`
`df.drop_duplicates(subset, keep)`	Remove repeated rows	`df.drop_duplicates("id", keep="last")`
`s.is_unique`	Is every value distinct?	`df["id"].is_unique`
`s.value_counts(dropna=False)`	Count missing as a category too	`s.value_counts(dropna=False)`
`df.sort_values().drop_duplicates()`	Keep the best row per key	`df.sort_values("ts").drop_duplicates("id", keep="last")`

Beginner: find, drop, fill

```
import numpy as np
import pandas as pd

df = pd.DataFrame({"id": [1, 2, 3, 4], "age": [25, np.nan, 40, np.nan], "city": ["Oslo", "Rome", None, "Lima"]})
print(df.isna().sum().to_dict())
print(df.dropna().shape)
print(df.dropna(subset=["city"]).shape)
filled = df.assign(age=df["age"].fillna(df["age"].median()), city=df["city"].fillna("unknown"))
print(filled)
```

```
{'id': 0, 'age': 2, 'city': 1}
(1, 3)
(3, 3)
   id   age     city
0   1  25.0     Oslo
1   2  32.5     Rome
2   3  40.0  unknown
3   4  32.5     Lima
```

1. `isna().sum()` counts missing cells: 2 in age, 1 in city.
2. `dropna()` with no arguments removes every row that has *any* gap. Rows 1 and 3 lack an age and row 2 lacks a city, so only row 0 survives: the shape is (1, 3).
3. `subset=["city"]` only cares about city, so rows with a missing age are kept and only row 2 is dropped: (3, 3).
4. Filling age with the median (32.5, the middle of 25 and 40) is a simple, outlier-resistant choice. Filling text with the word "unknown" keeps the row and makes the gap visible.

Intermediate: group-aware fill, sentinels, forward fill

```
import numpy as np
import pandas as pd

df = pd.DataFrame({
    "dept": ["a", "a", "b", "b", "b"],
    "pay": [100.0, np.nan, 200.0, -999.0, np.nan],
})
df["pay"] = df["pay"].replace(-999, np.nan)
df["pay_filled"] = df["pay"].fillna(df.groupby("dept")["pay"].transform("median"))
print(df)

ts = pd.Series([1.0, np.nan, np.nan, 4.0, np.nan])
print(ts.ffill().tolist())
print(ts.interpolate().tolist())
```

```
  dept    pay  pay_filled
0    a  100.0       100.0
1    a    NaN       100.0
2    b  200.0       200.0
3    b    NaN       200.0
4    b    NaN       200.0
[1.0, 1.0, 1.0, 4.0, 4.0]
[1.0, 2.0, 3.0, 4.0, 4.0]
```

1. Some systems store "unknown" as -999. If you average that you get nonsense, so `replace(-999, np.nan)` converts the sentinel to a real missing marker first.

2. `groupby("dept")["pay"].transform("median")` returns a Series the same length as `df` in which each row holds the median of its own department (100 for a, 200 for b).
3. `fillna(that_series)` fills each gap with the matching department median. This is better than one global median because departments differ.
4. `ffill()` carries the last known value forward (1, 1, 1, 4, 4): right for state that persists, such as a price.
5. `interpolate()` draws a straight line between known points (1, 2, 3, 4, then 4 at the end because nothing follows): right for smooth measurements such as temperature.

COMMON MISTAKE	**Filling before you understand why it is missing.** If "no discount" is stored as NaN, filling it with the mean invents a discount. Decide per column: is missing zero, unknown, or not applicable? Keep an indicator column (`df["age_missing"] = df["age"].isna()`) when missingness itself carries information.

Duplicates: exact, by key, and keep the latest

```
import pandas as pd

df = pd.DataFrame({
    "id": [1, 1, 2, 3, 3],
    "ts": ["2025-01-01", "2025-01-05", "2025-01-02", "2025-01-03", "2025-01-03"],
    "val": [10, 12, 7, 5, 5],
})
print(df.duplicated().tolist())
print(df.duplicated("id", keep=False).tolist())
latest = df.sort_values("ts").drop_duplicates("id", keep="last")
print(latest.sort_values("id"))
print(df.drop_duplicates().shape[0])
```

```
[False, False, False, False, True]
[True, True, False, True, True]
   id          ts  val
1   1  2025-01-05   12
2   2  2025-01-02    7
4   3  2025-01-03    5
4
```

1. `duplicated()` with no arguments compares whole rows. Only the last row repeats an earlier one exactly, so only it is True.
2. `duplicated("id", keep=False)` compares only the key and marks *every* row that shares an id (`keep=False`), which is what you want when you need to inspect all conflicting versions.
3. To keep the most recent row per id: sort by time first, then `drop_duplicates("id", keep="last")` . Order matters; without the sort, "last" means last in file order.
4. Dropping exact duplicates leaves 4 rows. Note the difference: key duplicates (id 1 appears with different values) are a *business* question, exact duplicates are usually an extraction error.

17.8 String and Datetime Accessors, and apply vs Vectorised

Name	Plain English	Minimal example
`s.str.lower() / upper() / title()`	Change case	`s.str.lower()`
`s.str.strip()`	Trim spaces at both ends	`s.str.strip()`
`s.str.replace(pat, repl, regex=)`	Replace text	`s.str.replace(r"\D", "", regex=True)`

Name	Plain English	Minimal example
`s.str.contains(pat)`	Boolean pattern match	`s.str.contains("^A")`
`s.str.startswith / endswith`	Prefix and suffix test	`s.str.endswith(".csv")`
`s.str.split(sep, expand=True)`	Split into columns	`s.str.split(" ", expand=True)`
`s.str.extract(regex)`	Pull capture groups into columns	`s.str.extract(r"(\d+)")`
`s.str.len()`	String length	`s.str.len()`
`s.str.cat(sep=)`	Join strings	`s.str.cat(sep=", ")`
`s.str.get_dummies(sep)`	One-hot from delimited tags	`s.str.get_dummies("|")`
`s.str.zfill(w)`	Left-pad with zeros	`s.str.zfill(5)`
`s.dt.year / month / day`	Date parts	`s.dt.month`
`s.dt.dayofweek / day_name()`	Weekday (Monday = 0)	`s.dt.day_name()`
`s.dt.to_period("M")`	Month bucket (as Period)	`s.dt.to_period("M")`
`s.dt.floor("h") / ceil / round`	Snap to a unit	`s.dt.floor("D")`
`s.dt.normalize()`	Drop the time of day	`s.dt.normalize()`
`s.dt.strftime(fmt)`	Format as text	`s.dt.strftime("%Y-%m")`
`s.dt.tz_localize / tz_convert`	Attach and change time zone	`s.dt.tz_localize("UTC")`
`pd.Timedelta / s.dt.days`	Durations	`(b - a).dt.days`
`pd.date_range(start, periods, freq)`	Regular calendar	`pd.date_range("2025-01-01", periods=3, freq="D")`
`pd.offsets.MonthEnd`	Calendar arithmetic	`s + pd.offsets.MonthEnd(0)`
`s.map(func_or_dict)`	Element-wise on a Series	`s.map({"a": 1})`
`df.apply(f, axis=)`	Run a function per column or per row	`df.apply(f, axis=1)`
`df.map(f)`	Element-wise on a whole DataFrame (replaces `applymap`)	`df.map(len)`

Beginner: clean messy text

```
import pandas as pd

names = pd.Series(["  Ana Silva ", "BO chen", "cy O'Neil"])
clean = names.str.strip().str.title()
print(clean.tolist())
parts = clean.str.split(" ", n=1, expand=True)
parts.columns = ["first", "last"]
print(parts)
print(clean.str.contains("an", case=False).tolist())
print(clean.str.len().tolist())
```

```
['Ana Silva', 'Bo Chen', "Cy O'Neil"]
  first    last
0   Ana   Silva
1    Bo    Chen
2    Cy  O'Neil
[True, False, False]
[9, 7, 9]
```

1. `.str` is an *accessor*: it applies a text method to every element of the column. You chain methods: strip first, then title-case.
2. `title()` capitalises each word; it handles these three names, but it turns "mcdonald" into "Mcdonald", so check on your real data.
3. `split(" ", n=1, expand=True)` splits at the first space only and returns a DataFrame with one column per piece; we then name them.
4. `contains("an", case=False)` tests for the substring ignoring case: True for "Ana Silva", False for the others.
5. `len()` counts characters (after cleaning, so the spaces are gone).

Intermediate: regex extraction and dates

```
import pandas as pd

log = pd.DataFrame({
    "line": ["ERR 2025-03-07 14:05 code=502", "OK 2025-03-08 09:30 code=200", "ERR 2025-03-09 23:59 code=404"],
})
ex = log["line"].str.extract(r"^(?P<level>\w+) (?P<when>[\d-]+ [\d:]+) code=(?P<code>\d+)$")
ex["when"] = pd.to_datetime(ex["when"])
ex["code"] = ex["code"].astype(int)
print(ex)
print(ex["when"].dt.day_name().tolist())
ex["day"] = ex["when"].dt.normalize()
ex["hour"] = ex["when"].dt.hour
print(ex[["day", "hour"]])
gap = ex["when"].diff()
print(gap.dt.total_seconds().div(3600).round(1).tolist())
```

```
  level               when  code
0   ERR 2025-03-07 14:05:00   502
1    OK 2025-03-08 09:30:00   200
2   ERR 2025-03-09 23:59:00   404
['Friday', 'Saturday', 'Sunday']
         day  hour
0 2025-03-07    14
1 2025-03-08     9
2 2025-03-09    23
[nan, 19.4, 38.5]
```

1. `str.extract` takes a regular expression with *named groups* `(?P<name>...)` and returns one column per group. Rows that do not match become NaN.
2. Extracted pieces are text, so we convert: `to_datetime` for the timestamp and `astype(int)` for the code.
3. `.dt` is the date accessor, the twin of `.str`. `day_name()` gives weekday names (7 March 2025 was a Friday).
4. `normalize()` sets the time to midnight, which is the easiest way to get a "date" key for grouping; `hour` is an integer part.
5. `diff()` on a datetime column gives a Timedelta between consecutive rows. `total_seconds()` converts to seconds, and dividing by 3600 gives hours. The first gap is NaN because no row precedes it.

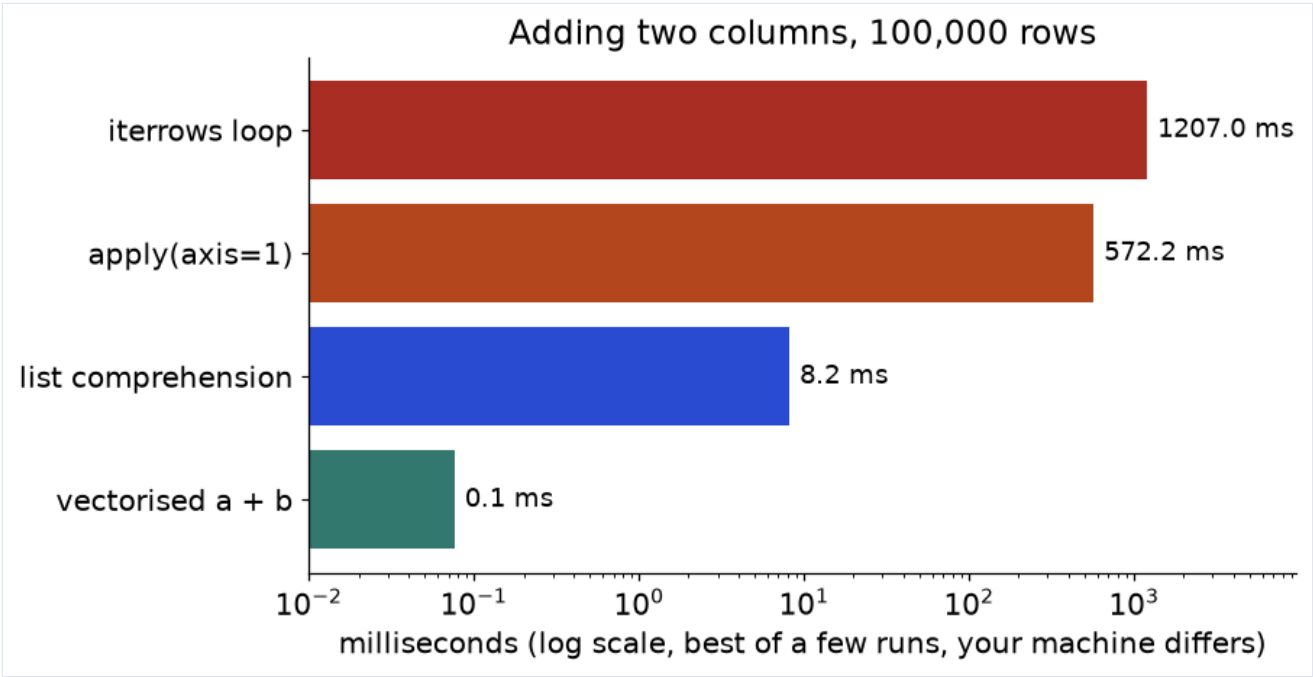
apply vs map vs vectorised: the speed ladder

The same task, "total = price times quantity", can be written four ways. They give the same answer at very different cost.

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(0)
df = pd.DataFrame({"price": rng.uniform(1, 10, 5), "qty": rng.integers(1, 5, 5)})

a = df["price"] * df["qty"]
b = df.apply(lambda r: r["price"] * r["qty"], axis=1)
c = pd.Series([p * q for p, q in zip(df["price"], df["qty"])])
d = pd.Series([r.price * r.qty for r in df.itertuples()])
print(np.allclose(a, b), np.allclose(a, c), np.allclose(a, d))
print(a.round(2).tolist())

tier = pd.Series([1, 2, 3, 1, 4]).map({1: "single", 2: "pair"}).fillna("bulk")
print(tier.tolist())
```

```
True True True
[20.2, 13.71, 4.11, 3.45, 33.28]
['single', 'pair', 'bulk', 'single', 'bulk']
```

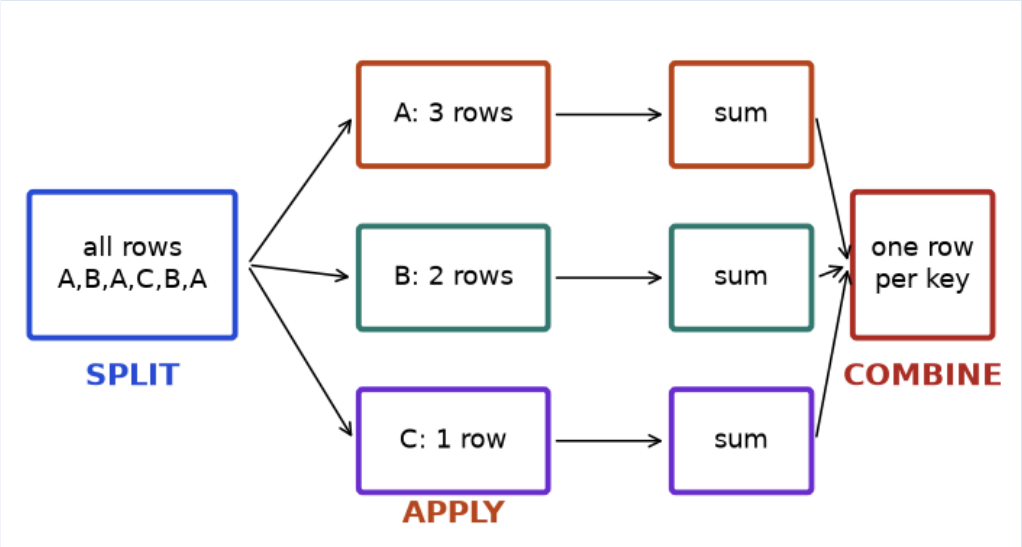
1. `a` is vectorised: one multiplication of two whole columns, executed in compiled code. This is the default choice.
2. `b` uses `apply(axis=1)`, which builds a Series object for every row and calls your Python function once per row. It is the slowest and the most overused.
3. `c` is a list comprehension over `zip` of two columns. It is a Python loop but avoids row objects, so it is typically much faster than `apply(axis=1)` and is a good fallback for logic that cannot be vectorised.
4. `d` uses `itertuples()`, the right way to loop over rows if you must (much faster than `iterrows`).
5. `np.allclose` checks that all four agree. The timings are in the figure below.
6. `Series.map(dict)` is the vectorised way to recode values. Anything not in the dict becomes NaN, which we fill with "bulk".



Adding two columns of 100,000 rows. Times measured on one laptop; they vary by machine, version and data, but the ordering is stable.

How to read it: the horizontal axis is logarithmic, so each gridline is ten times larger than the one before. The vectorised bar is orders of magnitude shorter than the two row-wise approaches. The gap grows with the number of rows. Reach for `apply(axis=1)` only after you have tried column arithmetic, `np.where`, `np.select`, `map` and the `.str` / `.dt` accessors.

17.9 groupby: agg, transform, filter and Named Aggregation



groupby is split, apply, combine: cut the rows by key, summarise each piece, stack the summaries.

How to read it: follow the arrows left to right. The blue box holds all rows. They are split by key into three boxes, a function (here `sum`) runs inside each box independently, and the three small results are combined into the final table with one row per key. `agg` shrinks each group to one row. `transform` returns one value for *every* original row. `filter` keeps or drops whole groups.

Name	Plain English	Minimal example
`df.groupby("k")`	Create the groups (nothing computed yet)	`g = df.groupby("city")`
`g["x"].sum() / mean() / count()`	Built-in per-group stats	`g["sales"].sum()`
`g.size()`	Rows per group (counts NaN too)	`g.size()`
`g.count()`	Non-null values per column per group	`g.count()`
`g.nunique()`	Distinct values per group	`g["user"].nunique()`
`g.agg({"x": ["sum","mean"]})`	Several stats per column	`g.agg({"x": ["sum","mean"]})`
`g.agg(total=("x","sum"))`	Named aggregation: clear output names	`g.agg(total=("sales","sum"))`
`g.agg(func)`	Custom function per group	`g["x"].agg(lambda s: s.max()-s.min())`
`g.transform("mean")`	Group stat broadcast to every row	`g["x"].transform("mean")`
`g.filter(func)`	Keep only groups that pass a test	`g.filter(lambda d: len(d) > 2)`
`g.apply(func)`	Flexible per-group function (slowest)	`g.apply(lambda d: d.nlargest(2, "x"))`
`g.head(n) / tail(n)`	First n rows of each group	`g.head(2)`
`g.nth(k)`	k-th row of each group	`g.nth(0)`
`g.cumcount()`	0,1,2... within each group	`g.cumcount()`
`g.rank()`	Rank within group	`g["x"].rank(ascending=False)`

Name	Plain English	Minimal example
`g.cumsum() / shift() / diff()`	Running/lagged values within group	`g["x"].shift(1)`
`g.first() / last()`	First/last non-null per group	`g.last()`
`g.idxmax()`	Row label of the max per group	`g["x"].idxmax()`
`groupby(["a","b"])`	Group by several keys	`df.groupby(["city","year"])`
`groupby(..., as_index=False)`	Keys stay as ordinary columns	`df.groupby("k", as_index=False).sum()`
`groupby(..., dropna=False)`	Keep a NaN group	`df.groupby("k", dropna=False)`
`groupby(..., observed=)`	For categorical keys: show unused categories? (default True in 3.0)	`groupby("cat", observed=False)`
`groupby(pd.Grouper(key, freq))`	Group a date column by period	`pd.Grouper(key="ts", freq="W")`
`g.ngroups / g.groups`	Number of groups / their row labels	`g.ngroups`
`g.get_group(key)`	Pull one group out	`g.get_group("Oslo")`
`g.describe()`	Summary statistics per group	`g["x"].describe()`

Beginner: one key, one statistic

```
import pandas as pd

df = pd.DataFrame({
    "city": ["Oslo", "Rome", "Oslo", "Lima", "Rome", "Oslo"],
    "sales": [10, 7, 3, 5, 9, 12],
})
print(df.groupby("city")["sales"].sum())
print(df.groupby("city")["sales"].agg(["count", "mean", "max"]))
print(df.groupby("city", as_index=False)["sales"].sum())
```

```
city
Lima     5
Oslo    25
Rome    16
Name: sales, dtype: int64
      count      mean  max
city
Lima      1  5.000000    5
Oslo      3  8.333333   12
Rome      2  8.000000    9
   city  sales
0  Lima      5
1  Oslo     25
2  Rome     16
```

1. `groupby("city")` says "split by city". `["sales"]` selects the column to summarise and `.sum()` adds within each group. Oslo: 10+3+12 = 25. Groups are sorted by key by default.
2. Passing a list to `agg` computes several statistics at once; each becomes a column.
3. `as_index=False` keeps `city` as an ordinary column instead of the index, which is easier to merge or plot afterwards.

Intermediate: named aggregation, transform and filter

```
import pandas as pd

df = pd.DataFrame({
    "city": ["Oslo", "Rome", "Oslo", "Lima", "Rome", "Oslo"],
    "channel": ["web", "web", "shop", "web", "shop", "web"],
    "sales": [10, 7, 3, 5, 9, 12],
})
summary = df.groupby("city").agg(
    orders=("sales", "size"),
    total=("sales", "sum"),
    best=("sales", "max"),
    channels=("channel", "nunique"),
)
print(summary)

df["share_of_city"] = (df["sales"] / df.groupby("city")["sales"].transform("sum")).round(2)
df["vs_city_mean"] = df["sales"] - df.groupby("city")["sales"].transform("mean")
print(df)

big = df.groupby("city").filter(lambda d: len(d) >= 2)
print(sorted(big["city"].unique()))
```

```
      orders  total  best  channels
city
Lima       1      5     5         1
Oslo       3     25    12         2
Rome       2     16     9         2
   city channel  sales  share_of_city  vs_city_mean
0  Oslo     web     10           0.40      1.666667
1  Rome     web      7           0.44     -1.000000
2  Oslo    shop      3           0.12     -5.333333
3  Lima     web      5           1.00      0.000000
4  Rome    shop      9           0.56      1.000000
5  Oslo     web     12           0.48      3.666667
['Oslo', 'Rome']
```

1. Named aggregation has the form `output_name=(column, function)`. It gives flat, readable column names, so prefer it over the dict-of-lists form which creates a two-level header.
2. `size` counts rows, `nunique` counts distinct channels. Lima has one order and one channel.
3. `transform("sum")` returns a Series aligned to the original rows: each Oslo row sees 25. Dividing gives each row's share of its city total.
4. `sales - transform("mean")` centres each value on its group mean: a common feature-engineering step.
5. `filter` receives each group as a small DataFrame and returns True to keep it. Lima has only one row, so it is dropped; Oslo and Rome stay.

Problem 17.9.1 For the table above, find the city whose average sale is highest, using one chain.

Idea	Group by city, take the mean of sales, then pick the label of the largest mean.
Code	`df.groupby("city")["sales"].mean().idxmax()`
By hand	Oslo (10+3+12)/3 = 8.33, Rome (7+9)/2 = 8.0, Lima 5/1 = 5.0.
Answer	Oslo. `idxmax()` returns the *label* (the city), whereas `max()` would return the number 8.33.

Expert: top-N per group, custom aggregations, and what apply costs

```
import pandas as pd

df = pd.DataFrame({
    "dept": ["a", "a", "a", "b", "b", "b"],
    "emp": ["u1", "u2", "u3", "u4", "u5", "u6"],
    "pay": [50, 80, 65, 90, 40, 70],
})
top2 = df.sort_values("pay", ascending=False).groupby("dept").head(2)
print(top2.sort_values(["dept", "pay"], ascending=[True, False]))

df["rank"] = df.groupby("dept")["pay"].rank(ascending=False, method="first").astype(int)
print(df[df["rank"] == 1][["dept", "emp"]])

spread = df.groupby("dept")["pay"].agg(lambda s: s.max() - s.min())
print(spread.to_dict())
print(df.groupby("dept")["pay"].agg(["min", "max"]).assign(spread=lambda d: d["max"] - d["min"])["spread"].to_dict())
```

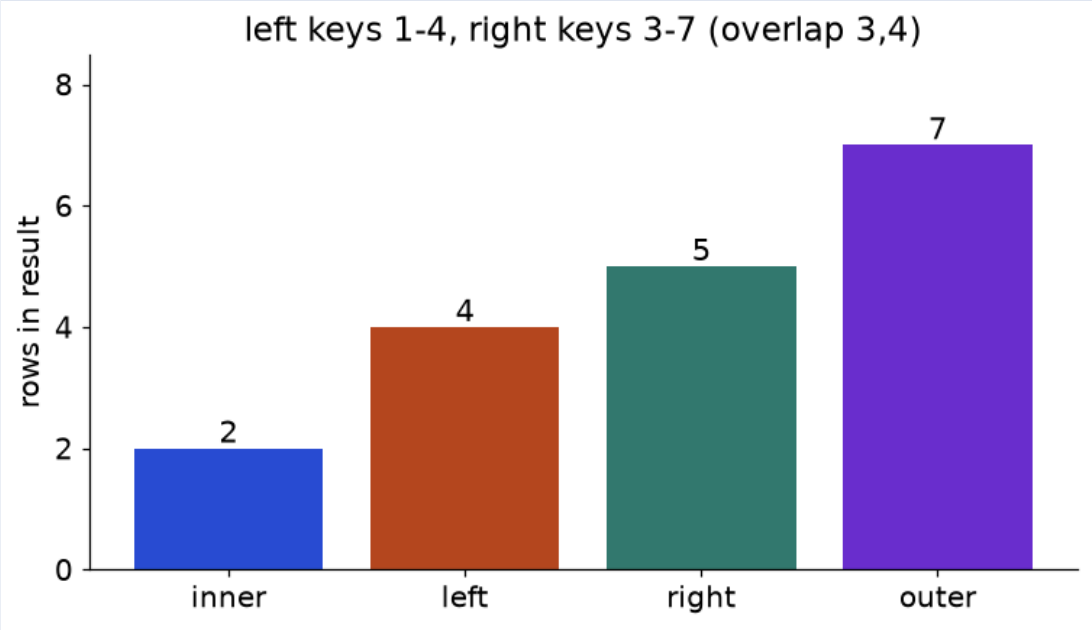
```
  dept emp  pay
1    a  u2   80
2    a  u3   65
3    b  u4   90
5    b  u6   70
  dept emp
1    a  u2
3    b  u4
{'a': 30, 'b': 50}
{'a': 30, 'b': 50}
```

1. Sorting by pay descending and then taking `groupby("dept").head(2)` is the fast idiom for top-N per group. `head` is implemented without calling Python per group.
2. The second `sort_values` is only for tidy display.
3. `rank(ascending=False, method="first")` numbers pay within each department; ties get distinct ranks in order of appearance. Filtering `rank == 1` gives the top earner per department. Ranks are floats, hence `astype(int)`.
4. `agg(lambda ...)` calls your Python function once per group: simple and flexible, but slower for many groups.
5. The last line computes the same spread with built-ins (`min`, `max`, then subtract). Prefer built-in aggregations whenever possible; they run in compiled code.

REAL JOB

Whenever you hear "per customer / per day / per region", say "groupby". Whenever you hear "compared with the group average" or "share of the group total", say "transform". Whenever you hear "only groups that ...", say "filter".

17.10 merge, join and concat, with Join Validation



The same two small tables joined four ways. Left keys are 1 to 4, right keys are 3 to 7, so only 3 and 4 match.

How to read it: each bar is the number of rows after a join. `inner` keeps only matching keys (2). `left` keeps all four left keys (4). `right` keeps all five right keys (5). `outer` keeps every key from either side (7). The row count is the quickest sanity check: if your left join returns more rows than the left table had, the right table had duplicate keys.

Name	Plain English	Minimal example
`pd.merge(a, b, on="k")`	SQL-style join on a shared column	`a.merge(b, on="k")`
`how="inner"`	Only keys in both	`a.merge(b, on="k", how="inner")`
`how="left"`	All left rows, matches from right	`a.merge(b, on="k", how="left")`
`how="right"`	All right rows	`how="right"`
`how="outer"`	All keys from both	`how="outer"`
`how="cross"`	Every pairing (Cartesian product)	`a.merge(b, how="cross")`
`left_on= / right_on=`	Different key names	`a.merge(b, left_on="id", right_on="user_id")`
`on=["a","b"]`	Composite key	`a.merge(b, on=["y","m"])`
`suffixes=`	Names for clashing columns	`suffixes=("_a","_b")`
`validate="1:1" / "m:1" / "1:m"`	Raise if key uniqueness is not as expected	`a.merge(b, on="k", validate="m:1")`
`indicator=True`	Add a column saying which side matched	`a.merge(b, on="k", how="outer", indicator=True)`
`a.join(b)`	Join on the index (shortcut)	`a.join(b, how="left")`
`pd.concat([a, b])`	Stack frames vertically	`pd.concat([a, b], ignore_index=True)`
`pd.concat(axis=1)`	Place side by side, aligned on index	`pd.concat([a, b], axis=1)`
`concat(keys=)`	Label each piece in a MultiIndex	`pd.concat([a, b], keys=["x","y"])`

Name	Plain English	Minimal example
`pd.merge_asof`	Join to the nearest earlier key (time)	`pd.merge_asof(t, q, on="ts")`
`pd.merge_ordered`	Ordered merge with fill	`pd.merge_ordered(a, b, on="ts")`
`df.isin(other)`	Semi-join test	`a[a.k.isin(b.k)]`
`~a.k.isin(b.k)`	Anti-join	`a[~a.k.isin(b.k)]`
`df.combine_first(other)`	Fill gaps from another frame	`a.combine_first(b)`
`df.update(other)`	Overwrite in place with non-NaN values	`a.update(b)`

Beginner: the four join types

```
import pandas as pd

customers = pd.DataFrame({"cid": [1, 2, 3], "name": ["Ana", "Bo", "Cy"]})
orders = pd.DataFrame({"cid": [2, 3, 3, 4], "amount": [20, 30, 5, 99]})

for how in ["inner", "left", "outer"]:
    out = customers.merge(orders, on="cid", how=how)
    print(how, out.shape)
print(customers.merge(orders, on="cid", how="left"))
```

```
inner (3, 3)
left (4, 3)
outer (5, 3)
   cid name  amount
0    1  Ana     NaN
1    2   Bo    20.0
2    3   Cy    30.0
3    3   Cy     5.0
```

1. Customers 1 to 3 and orders from customers 2, 3, 3, 4. Customer 3 has two orders; order 4 belongs to a customer that does not exist in the table.
2. `inner` gives 3 rows: customer 2 once and customer 3 twice. Customer 1 (no orders) and order 4 (no customer) vanish.
3. `left` gives 4 rows: the same 3 plus customer 1 with NaN amount.
4. `outer` gives 5 rows: left plus the orphan order for customer 4.
5. Notice that a one-to-many join *multiplies* rows: customer 3 appears twice. That is correct here, but it is the root of the classic "my totals doubled" bug.

Intermediate: validate, indicator, different key names

```
import pandas as pd

users = pd.DataFrame({"user_id": [1, 2, 3], "plan": ["free", "pro", "pro"]})
events = pd.DataFrame({"uid": [1, 1, 2, 9], "event": ["a", "b", "c", "d"]})

m = events.merge(users, left_on="uid", right_on="user_id", how="left", validate="m:1", indicator=True)
print(m)
print(m["_merge"].value_counts().to_dict())

dup_users = pd.concat([users, users.iloc[[0]]], ignore_index=True)
try:
    events.merge(dup_users, left_on="uid", right_on="user_id", validate="m:1")
except Exception as e:
    print(type(e).__name__, "-", str(e)[:60])
```

```
   uid event  user_id  plan     _merge
0    1     a      1.0  free       both
1    1     b      1.0  free       both
2    2     c      2.0   pro       both
3    9     d      NaN   NaN  left_only
{'both': 3, 'left_only': 1, 'right_only': 0}
MergeError - Merge keys are not unique in right dataset; not a many-to-on
```

1. `left_on="uid", right_on="user_id"` joins differently named keys. Both key columns stay in the result; drop one afterwards.
2. `validate="m:1"` promises "many events, one user each". If any user id repeats in `users`, pandas raises `MergeError` instead of silently multiplying rows.
3. `indicator=True` adds `_merge` with values `both`, `left_only` or `right_only`. Here event "d" has uid 9, which is not a user, so it is `left_only` with NaN plan.
4. We then add a duplicate user and merge again. The `validate` check fires, and the message tells you the right side is not unique. Catching this at the join is far cheaper than discovering inflated revenue in a report.

COMMON MISTAKE

A left join that changes the row count. If the right table has duplicate keys the output has more rows than the left input. Always pass `validate="m:1"` (or "1:1") when you expect uniqueness, and assert the row count after the join.

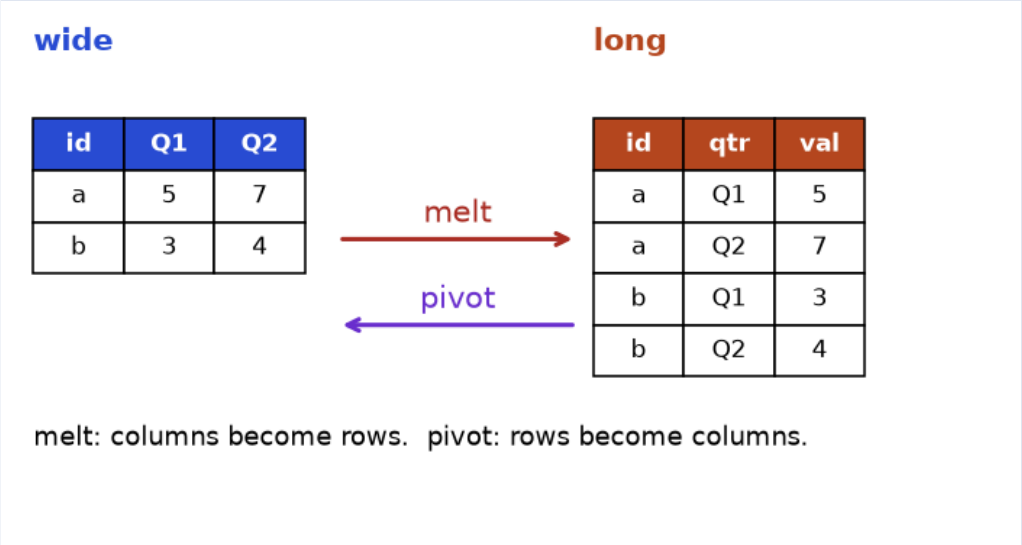
concat, merge_asof and anti-joins

```
import pandas as pd

jan = pd.DataFrame({"id": [1, 2], "v": [10, 20]})
feb = pd.DataFrame({"id": [2, 3], "v": [25, 30]})
stacked = pd.concat([jan, feb], keys=["jan", "feb"])
print(stacked)
print(pd.concat([jan, feb], ignore_index=True).shape)

trades = pd.DataFrame({"ts": pd.to_datetime(["2025-01-01 10:00:05", "2025-01-01 10:00:20"]), "qty": [5, 7]})
quotes = pd.DataFrame({"ts": pd.to_datetime(["2025-01-01 10:00:00", "2025-01-01 10:00:10"]), "px": [100.0, 101.5]})
print(pd.merge_asof(trades, quotes, on="ts"))

print(jan[~jan["id"].isin(feb["id"])]["id"].tolist())
```

```
       id   v
jan 0   1  10
    1   2  20
feb 0   2  25
    1   3  30
(4, 2)
                   ts  qty     px
0 2025-01-01 10:00:05    5  100.0
1 2025-01-01 10:00:20    7  101.5
[1]
```

1. `concat` stacks rows. `keys=["jan","feb"]` records which piece each row came from as an extra index level, useful when you read many files and need to know the source.
2. `ignore_index=True` discards the old labels and renumbers 0..n-1; the shape (4 rows, 2 columns) confirms nothing was lost.
3. `merge_asof` matches each trade with the *latest quote at or before* it: the 10:00:05 trade gets the 10:00:00 price, the 10:00:20 trade gets the 10:00:10 price. Both tables must be sorted by the key.
4. `~isin` is the anti-join: ids in January that are not in February (id 1).

17.11 Reshaping: pivot, pivot_table, melt, stack and crosstab



Wide and long are two shapes of the same data. melt goes wide to long; pivot goes long to wide.

How to read it: the blue table keeps one row per id with a column per quarter (wide, good for people). The orange table has one row per id and quarter (long, good for groupby, plotting and databases). The top arrow is `melt`, the bottom arrow is `pivot`. If a tool complains that it needs "tidy data", it wants the long shape.

Name	Plain English	Minimal example
`df.pivot(index, columns, values)`	Long to wide; fails on duplicate pairs	`df.pivot(index="id", columns="q", values="v")`
`df.pivot_table(..., aggfunc)`	Long to wide with aggregation of duplicates	`pivot_table(index="a", columns="b", values="v", aggfunc="sum")`
`pivot_table(margins=True)`	Add totals row and column	`margins=True`
`pivot_table(fill_value=0)`	Replace empty cells	`fill_value=0`

Name	Plain English	Minimal example
`pd.melt(df, id_vars, value_vars)`	Wide to long	`df.melt(id_vars="id", var_name="q", value_name="v")`
`df.stack()`	Columns into an inner index level	`df.stack()`
`df.unstack(level)`	Inner index level into columns	`s.unstack()`
`pd.crosstab(a, b)`	Count table of two columns	`pd.crosstab(df.a, df.b)`
`crosstab(normalize="index")`	Row percentages	`normalize="index"`
`df.explode("col")`	One row per list item	`df.explode("tags")`
`pd.get_dummies(s)`	One-hot encode	`pd.get_dummies(df["city"], dtype=int)`
`pd.wide_to_long`	Several stubbed column families to long	`wide_to_long(df, ["q"], i="id", j="n")`
`df.T`	Swap rows and columns	`df.T`
`df.reset_index()`	Flatten after pivot	`pt.reset_index()`
`df.columns.to_flat_index()`	Flatten MultiIndex columns	`["_".join(c) for c in df.columns]`

Beginner: melt and pivot are inverses

```
import pandas as pd

wide = pd.DataFrame({"id": ["a", "b"], "Q1": [5, 3], "Q2": [7, 4]})
long = wide.melt(id_vars="id", var_name="qtr", value_name="val")
print(long)
back = long.pivot(index="id", columns="qtr", values="val").reset_index()
back.columns.name = None
print(back)
print(back.equals(wide))
```

```
  id qtr  val
0  a  Q1    5
1  b  Q1    3
2  a  Q2    7
3  b  Q2    4
  id  Q1  Q2
0  a   5   7
1  b   3   4
True
```

1. `id_vars` are the columns that stay as identifiers. Every other column is "melted" into two new columns: `var_name` holds the old column name, `value_name` holds the cell value.
2. The long table has 2 ids x 2 quarters = 4 rows.
3. `pivot` reverses it: the unique values of `qtr` become columns. `reset_index()` turns the id index back into a column and `columns.name = None` removes the leftover label "qtr".
4. `equals` confirms we are back to the original table.

Intermediate: pivot_table, crosstab, stack and unstack

```
import pandas as pd

df = pd.DataFrame({
    "city": ["Oslo", "Oslo", "Rome", "Rome", "Oslo"],
    "year": [2024, 2025, 2024, 2025, 2025],
    "sales": [10, 12, 7, 9, 3],
    "paid": ["y", "n", "y", "y", "y"],
})
pt = df.pivot_table(index="city", columns="year", values="sales", aggfunc="sum", fill_value=0, margins=True)
print(pt)
print(pd.crosstab(df["city"], df["paid"]))
print(pd.crosstab(df["city"], df["paid"], normalize="index").round(2))
s = df.groupby(["city", "year"])["sales"].sum()
print(s.unstack())
print(s.unstack().stack().equals(s))
```

```
year  2024  2025  All
city
Oslo    10    15   25
Rome     7     9   16
All     17    24   41
paid  n  y
city
Oslo  1  2
Rome  0  2
paid     n     y
city
Oslo  0.33  0.67
Rome  0.00  1.00
year  2024  2025
city
Oslo    10    15
Rome     7     9
True
```

1. `pivot_table` is `groupby` plus a reshape: rows by city, columns by year, cells are the sum of sales. Oslo in 2025 has two rows (12 and 3), so they are added to 15. Plain `pivot` would have raised an error here because of the duplicate pair.
2. `margins=True` adds an "All" row and column of totals; `fill_value=0` replaces empty combinations.
3. `crosstab` counts how many rows fall in each pair of values. `normalize="index"` divides each row by its total, giving row percentages (Oslo: 2 of 3 paid).
4. A groupby on two keys returns a Series with a two-level index. `unstack()` moves the inner level (year) into columns; `stack()` moves it back. The final `equals` shows they round-trip.

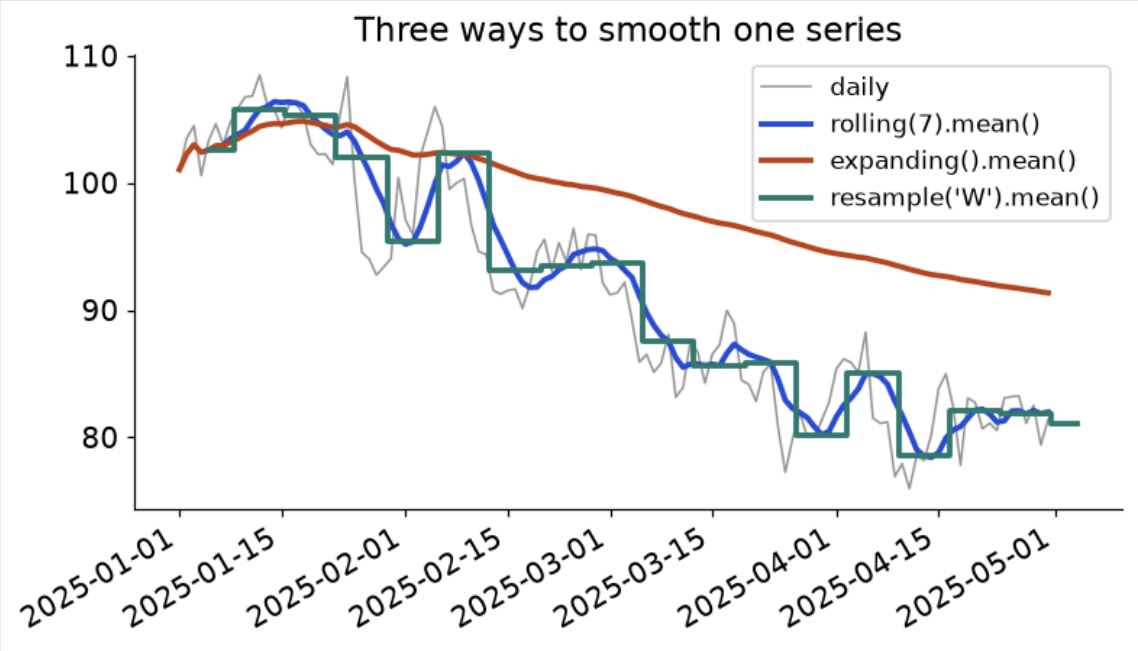
Realistic: explode tags and one-hot encode

```
import pandas as pd

posts = pd.DataFrame({"id": [1, 2, 3], "tags": ["py|sql", "py", "sql|dbt|py"]})
posts["tag"] = posts["tags"].str.split("|")
exploded = posts.explode("tag")[["id", "tag"]]
print(exploded.groupby("tag").size().sort_values(ascending=False).to_dict())
onehot = posts["tags"].str.get_dummies("|")
print(onehot)
print(pd.get_dummies(pd.Series(["Oslo", "Rome", "Oslo"]), dtype=int))
```

```
{'py': 3, 'sql': 2, 'dbt': 1}
   dbt  py  sql
0    0   1    1
1    0   1    0
2    1   1    1
   Oslo  Rome
0     1     0
1     0     1
2     1     0
```

1. `str.split("|")` turns each cell into a Python list.
2. `explode` repeats the row once per list element, so post 3 becomes three rows. Counting rows per tag then answers "how popular is each tag" (py: 3, sql: 2, dbt: 1).
3. `str.get_dummies("|")` does split plus one-hot in one call: a 0/1 column per tag, ready for a model.
4. `pd.get_dummies` one-hot encodes an ordinary categorical column. Use `dtype=int` to get 0/1 instead of True/False.

17.12 Window Functions: rolling, expanding and resample



One noisy daily series, smoothed three ways. Synthetic data with a fixed seed.

How to read it: the grey line is the raw daily value. The blue `rolling(7).mean()` follows it closely but irons out day-to-day noise, because each point is the mean of the last seven days. The orange `expanding().mean()` averages *everything so far*, so it reacts slowly as history piles up. The teal steps are `resample("W").mean()`: one value per week, so the series has fewer points. Rolling keeps the original length; resample changes it.

Name	Plain English	Minimal example
`s.rolling(n).mean()`	Moving average of the last n rows	`s.rolling(7).mean()`
`rolling(n, min_periods=k)`	Allow shorter windows at the start	`s.rolling(7, min_periods=1).mean()`
`rolling("7D")`	Time-based window (needs a DatetimeIndex)	`s.rolling("7D").sum()`
`rolling(n).sum() / std() / min() / max()`	Other window statistics	`s.rolling(3).max()`

Name	Plain English	Minimal example
`rolling(n, center=True)`	Window centred on each point	`s.rolling(3, center=True).mean()`
`rolling(n).apply(f, raw=True)`	Custom window function	`s.rolling(5).apply(np.ptp, raw=True)`
`s.expanding().sum()`	Running aggregate from the start	`s.expanding().max()`
`s.ewm(span=n).mean()`	Exponentially weighted average (recent counts more)	`s.ewm(span=7).mean()`
`s.cumsum() / cummax()`	Cumulative shortcuts	`s.cumsum()`
`s.shift(k)`	Previous or next value	`s.shift(1)`
`s.diff(k)`	Change vs k rows ago	`s.diff()`
`s.pct_change()`	Relative change	`s.pct_change()`
`s.resample("D").sum()`	Aggregate into time buckets	`s.resample("W").mean()`
`resample("ME")`	Month end buckets (`"M"` was removed)	`s.resample("ME").sum()`
`resample(...).agg([..])`	Several stats per bucket	`.agg(["min","max"])`
`resample(...).asfreq() / ffill()`	Up-sample (more buckets than data)	`s.resample("h").ffill()`
`s.asfreq("D")`	Insert missing dates as NaN	`s.asfreq("D")`
`groupby(pd.Grouper(freq=))`	Resample inside groups	`groupby(["k", pd.Grouper(key="ts", freq="W")])`
`s.loc["2025-03"]`	Slice by partial date string	`s.loc["2025-03-01":"2025-03-07"]`
`pd.date_range(..., freq=)`	Build a calendar ("D", "h", "ME", "W", "QE")	`pd.date_range("2025", periods=4, freq="QE")`
`s.tz_localize / tz_convert`	Time zones	`s.tz_localize("UTC").tz_convert("Europe/Oslo")`
`s.between_time("09:00","17:00")`	Filter by time of day	`df.between_time("09:00","17:00")`

Beginner and intermediate: rolling, expanding and shift

```
import pandas as pd

s = pd.Series([10, 12, 9, 15, 20, 18],
              index=pd.date_range("2025-03-01", periods=6, freq="D"), name="visits")
print(s.rolling(3).mean().round(2).tolist())
print(s.rolling(3, min_periods=1).mean().round(2).tolist())
print(s.expanding().max().tolist())
print(s.shift(1).tolist())
print(s.pct_change().round(3).tolist())
print((s - s.shift(7)).isna().all())
```

```
[nan, nan, 10.33, 12.0, 14.67, 17.67]
[10.0, 11.0, 10.33, 12.0, 14.67, 17.67]
[10.0, 12.0, 12.0, 15.0, 20.0, 20.0]
[nan, 10.0, 12.0, 9.0, 15.0, 20.0]
[nan, 0.2, -0.25, 0.667, 0.333, -0.1]
True
```

1. `rolling(3).mean()` averages each day with the two before it. The first two values are NaN because a full window of three does not exist yet. Fourth value: (12+9+15)/3 = 12.
2. `min_periods=1` accepts shorter windows at the start, so there are no NaN, but the first values are averages of fewer points, so treat them with care.
3. `expanding().max()` is the running maximum: 10, 12, 12, 15, 20, 20.
4. `shift(1)` moves values down one row, giving "yesterday's visits" next to today's. This is how you build lag features.
5. `pct_change()` is (today - yesterday) / yesterday: the second value is 12/10 - 1 = 0.2.
6. The last line shows that comparing with 7 days ago gives only NaN in a 6-row series: there is no history that far back.

Realistic: resample irregular events into daily and monthly totals

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(3)
ts = pd.to_datetime("2025-01-01") + pd.to_timedelta(rng.integers(0, 90 * 24 * 60, 400), unit="min")
ev = pd.DataFrame({"ts": ts, "amount": rng.integers(5, 100, 400)}).sort_values("ts").set_index("ts")

daily = ev["amount"].resample("D").sum()
print(len(daily), daily.index.min().date(), daily.index.max().date())
monthly = ev["amount"].resample("ME").agg(["count", "sum"])
print(monthly)
print(ev.loc["2025-02-10":"2025-02-11", "amount"].sum())
roll = daily.rolling("7D").sum()
print(roll.iloc[[0, 6, 30]].tolist())
print(ev.groupby(ev.index.dayofweek)["amount"].sum().idxmax())
```

```
90 2025-01-01 2025-03-31
            count   sum
ts
2025-01-31    140  7064
2025-02-28    122  6418
2025-03-31    138  7147
370
[254.0, 1465.0, 1652.0]
2
```

1. 400 random timestamps over 90 days become the index. A `DatetimeIndex` unlocks time-aware slicing and resampling.
2. `resample("D").sum()` puts every event into its calendar day and adds the amounts. Days with no events get 0 (for sum), so the result has no gaps.
3. `resample("ME")` buckets by month end; `agg(["count", "sum"])` gives the number of events and the total per month. The 90 days run from 1 January to 31 March, so there are exactly three monthly rows and the counts add up to 400.
4. Partial string slicing `loc["2025-02-10":"2025-02-11"]` selects whole days; both ends are included.
5. `rolling("7D")` is a time-based window: the sum of the last seven calendar days, correct even when rows are unevenly spaced.
6. `dayofweek` gives 0 for Monday to 6 for Sunday; grouping on it finds the weekday with the largest total (2 means Wednesday here).

17.13 MultiIndex, cut/qcut, Method Chaining and pipe

Name	Plain English	Minimal example
`set_index(["a","b"])`	Build a MultiIndex from columns	`df.set_index(["city","year"])`
`pd.MultiIndex.from_product`	All combinations of lists	`from_product([a, b], names=[..])`
`df.loc[("Oslo", 2025)]`	Select by a tuple of labels	`df.loc[("Oslo", 2025), "sales"]`
`df.xs(key, level=)`	Cross-section on one level	`df.xs(2025, level="year")`
`df.loc[pd.IndexSlice[:, 2025], :]`	Slice across levels	`idx = pd.IndexSlice`
`df.sort_index()`	Required for fast, correct slicing	`df.sort_index()`
`df.groupby(level=)`	Group by an index level	`df.groupby(level="city").sum()`
`df.swaplevel()`	Swap level order	`df.swaplevel()`
`df.reset_index(level=)`	Turn a level back into a column	`df.reset_index(level="year")`
`df.index.get_level_values(n)`	Extract one level as an Index	`df.index.get_level_values("city")`
`pd.cut(s, bins, labels)`	Fixed-width or custom bins	`pd.cut(age, [0,18,65,120])`
`pd.qcut(s, q)`	Equal-count bins (quantiles)	`pd.qcut(s, 4, labels=False)`
`cut(..., right=False)`	Left-closed intervals	`right=False`
`pd.IntervalIndex`	Interval lookups	`pd.IntervalIndex.from_breaks([0,1,2])`
`df.pipe(f, *args)`	Put your own function in a chain	`df.pipe(clean, col="x")`
`df.assign(...)`	Add columns inside a chain	`.assign(z=lambda d: d.x*2)`
`df.query(...)`	Filter inside a chain	`.query("z > 1")`
`df.rename(columns=)`	Rename inside a chain	`.rename(columns=str.lower)`
`df.loc[lambda d: d.x > 1]`	Filter with a callable	`.loc[lambda d: d.x > 1]`
`df.sort_values().reset_index(drop=True)`	Tidy order and index	`.reset_index(drop=True)`

MultiIndex: when a table has two row keys

```
import pandas as pd

df = pd.DataFrame({
    "city": ["Oslo", "Oslo", "Rome", "Rome"],
    "year": [2024, 2025, 2024, 2025],
    "sales": [10, 15, 7, 9],
}).set_index(["city", "year"]).sort_index()
print(df)
print(df.loc[("Rome", 2025), "sales"])
print(df.xs(2025, level="year"))
print(df.groupby(level="city")["sales"].sum().to_dict())
print(df.reset_index(level="year").columns.tolist())
```

```
           sales
city year
Oslo 2024     10
     2025     15
Rome 2024      7
     2025      9
9
      sales
city
Oslo     15
Rome      9
{'Oslo': 25, 'Rome': 16}
['year', 'sales']
```

1. `set_index(["city","year"])` makes a two-level row index; `sort_index()` sorts it, which pandas needs for correct and fast slicing.
2. A tuple selects a single combination: Rome in 2025 has sales 9.
3. `xs(2025, level="year")` takes all rows with year 2025 and drops that level, leaving a city-indexed table.
4. `groupby(level="city")` aggregates by one index level without resetting the index.
5. If a MultiIndex gets in your way, `reset_index()` flattens it back to columns. A good rule: use MultiIndex for the output of reshaping, but keep your working data flat.

Binning with cut and qcut

```
import pandas as pd

age = pd.Series([4, 17, 18, 35, 64, 65, 90])
groups = pd.cut(age, bins=[0, 17, 64, 120], labels=["child", "adult", "senior"])
print(groups.tolist())
print(pd.cut(age, bins=[0, 18, 65, 120], right=False).astype(str).tolist()[:3])

spend = pd.Series([5, 8, 12, 20, 33, 50, 80, 400])
q = pd.qcut(spend, 4, labels=["Q1", "Q2", "Q3", "Q4"])
print(q.tolist())
print(q.value_counts().sort_index().to_dict())
print(pd.qcut(spend, 4).cat.categories[0])
```

```
['child', 'child', 'adult', 'adult', 'adult', 'senior', 'senior']
['[0, 18)', '[0, 18)', '[18, 65)']
['Q1', 'Q1', 'Q2', 'Q2', 'Q3', 'Q3', 'Q4', 'Q4']
{'Q1': 2, 'Q2': 2, 'Q3': 2, 'Q4': 2}
(4.999, 11.0]
```

1. `cut` with explicit `bins` uses your edges. By default intervals are closed on the *right*: (0, 17] contains 17 but not 0 and not 18. So 17 is a "child" and 18 is an "adult".
2. `right=False` flips this to [0, 18), so 18 starts a new bin. The printed interval strings show which convention is in force. Always check a boundary value.
3. `qcut` chooses edges so that each bin holds about the same number of rows (quartiles). With 8 values, each quartile gets 2: the large value 400 does not distort the bins the way equal-width bins would.
4. Counting per label confirms 2 values per quartile. The last line shows the first computed interval, whose lower edge is slightly below the minimum so the minimum is included.

Method chaining and pipe

A chain reads like a recipe: each line is one step and the whole expression returns the finished table. There are no half-finished variables to confuse you later.

```
import pandas as pd

raw = pd.DataFrame({
    "Order ID": [1, 2, 3, 4, 5],
    "Customer": [" ana", "BO ", "ana", "Cy", None],
    "Amount": ["10.5", "7", "oops", "30", "12"],
    "Date": ["2025-03-01", "2025-03-02", "2025-03-02", "2025-03-05", "2025-03-06"],
})

def drop_small(d, min_amount):
    return d[d["amount"] >= min_amount]

clean = (
    raw
    .rename(columns=lambda c: c.strip().lower().replace(" ", "_"))
    .assign(
        customer=lambda d: d["customer"].str.strip().str.lower(),
        amount=lambda d: pd.to_numeric(d["amount"], errors="coerce"),
        date=lambda d: pd.to_datetime(d["date"]),
    )
    .dropna(subset=["customer", "amount"])
    .pipe(drop_small, min_amount=8)
    .sort_values("date")
    .reset_index(drop=True)
)
print(clean)
print(clean.dtypes.astype(str).to_dict())
```

```
   order_id customer  amount       date
0         1      ana    10.5 2025-03-01
1         4       cy    30.0 2025-03-05
{'order_id': 'int64', 'customer': 'str', 'amount': 'float64', 'date': 'datetime64[us]'}
```

1. The whole pipeline is one parenthesised expression, so each line can start with a dot. Parentheses let Python continue the expression over lines without backslashes.
2. `rename(columns=lambda c: ...)` normalises every header: strip spaces, lowercase, spaces to underscores. Consistent snake_case names save endless quoting later.
3. `assign` creates three cleaned columns. Each lambda receives the current frame, so `d["customer"]` sees the renamed column.
4. `dropna(subset=...)` removes the row with no customer and the row whose amount failed to parse ("oops").
5. `pipe(drop_small, min_amount=8)` inserts your own function into the chain; the frame is passed as the first argument. This is how you keep chains readable while reusing logic.
6. Finally sort and renumber. Order 3 ("oops") and order 5 (no customer) were dropped by `dropna`, and order 2 (amount 7) by the size filter, so orders 1 and 4 remain. Printing dtypes proves the types are what you intended.

17.14 Memory, Speed and pandas 3.0 Specifics

Name	Plain English	Minimal example
`df.memory_usage(deep=True)`	Real bytes per column	`df.memory_usage(deep=True).sum()`
`pd.to_numeric(downcast=)`	Smallest safe number type	`pd.to_numeric(s, downcast="integer")`
`astype("category")`	Shrink repeated text	`df["c"].astype("category")`
`read_csv(usecols=, dtype=)`	Load less, with the right types	`usecols=["a","b"]`

Name	Plain English	Minimal example
`read_csv(chunksize=)`	Stream large files	`for part in read_csv(p, chunksize=10_000)`
`df.eval("c = a + b")`	Evaluate an expression in one pass	`df.eval("c = a + b")`
`pd.read_parquet(columns=)`	Columnar file, read some columns (not run here)	`read_parquet(p, columns=["a"])`
`df.itertuples()`	Fast row loop when unavoidable	`for r in df.itertuples(): r.a`
`s.to_numpy()`	Drop to NumPy for tight math	`s.to_numpy()`
`df.copy(deep=False)`	Cheap shallow copy (safe with copy-on-write)	`df.copy(deep=False)`
`pd.options.mode.string_storage`	Choose the backing for text	`pd.options.mode.string_storage`
`pd.options.future.infer_string`	The 2.x switch for the 3.0 string default	(always on in 3.0)
`pd.api.types.is_string_dtype`	Is a column text, in any representation?	`is_string_dtype(s)`
`df.sort_values(kind=)`	Sorting algorithm	`kind="stable"`
`df.groupby(sort=False)`	Skip sorting group keys	`groupby("k", sort=False)`
`DataFrame.map`	Element-wise function (`applymap` was removed)	`df.map(str.upper)`

Shrink a table: a worked memory diet

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(7)
n = 100_000
df = pd.DataFrame({
    "user_id": rng.integers(0, 5000, n),
    "country": rng.choice(["NO", "IT", "PE", "UA", "US"], n),
    "qty": rng.integers(1, 20, n),
    "price": rng.uniform(1, 100, n).round(2),
})
before = df.memory_usage(deep=True).sum()
slim = df.assign(
    user_id=pd.to_numeric(df["user_id"], downcast="unsigned"),
    country=df["country"].astype("category"),
    qty=pd.to_numeric(df["qty"], downcast="unsigned"),
    price=df["price"].astype("float32"),
)
after = slim.memory_usage(deep=True).sum()
print(slim.dtypes.astype(str).to_dict())
print(f"{before/1e6:.1f} MB -> {after/1e6:.1f} MB")
print(round(after / before, 2))
```

```
{'user_id': 'uint16', 'country': 'category', 'qty': 'uint8', 'price': 'float32'}
7.5 MB -> 0.8 MB
0.11
```

1. A 100,000-row table with an id, a country, a small integer and a price.
2. `memory_usage(deep=True).sum()` is the total in bytes; we remember it as the "before" size.
3. `downcast="unsigned"` asks pandas for the smallest unsigned integer that fits: user ids below 5000 need 16 bits, quantities below 20 need 8.
4. Country has five values, so `category` stores them once and uses one byte per row.

5. `float32` halves the price column. It carries about 7 significant digits, which is fine for prices rounded to cents in this range but *not* for money you must add up exactly (use integer cents or `Decimal` for that).
6. The ratio shows the new table is a fraction of the old one. Always compare results before and after to make sure the diet did not change any answer.

pandas 3.0 in practice: copy-on-write and the string dtype

KEY IDEA	**Copy-on-write (CoW)** is now the only behaviour. Every DataFrame or Series derived from another one behaves as an independent copy: changing the child never changes the parent, and vice versa. pandas achieves this lazily (it copies only when you write), so it is cheap. The consequence: *chained assignment* such as `df["a"][mask] = 0` or `df[mask]["a"] = 0` no longer modifies `df`. Use `df.loc[mask, "a"] = 0`.

```
import pandas as pd

df = pd.DataFrame({"a": [1, 2, 3], "b": ["x", "y", "z"]})
view = df["a"]
view.iloc[0] = 100
print(df["a"].tolist(), view.tolist())

sub = df[df["a"] > 1]
sub.loc[sub.index[0], "a"] = -1
print(df["a"].tolist())

df.loc[df["a"] > 2, "a"] = 0
print(df["a"].tolist())

s = df["b"]
print(s.dtype, s.str.upper().dtype)
print(pd.Series(["a", None]).isna().tolist(), pd.Series(["a", None]).dtype)
print(pd.Series(["a", 1]).dtype)
```

```
[1, 2, 3] [100, 2, 3]
[1, 2, 3]
[1, 2, 0]
str str
[False, True] str
object
```

1. `view = df["a"]` looks like a window onto the column, but under copy-on-write it is a lazy copy. Writing to `view` triggers the real copy, so `df` is unchanged: the first list still starts with 1.
2. The same holds for a filtered frame `sub`. Editing it does not touch `df`. This removes the old `SettingWithCopyWarning` confusion: in 3.0 there is no ambiguity and no warning.
3. `df.loc[mask, "a"] = 0` is a single operation on `df` itself, so it works. It changes 3 to 0.
4. Text columns now have dtype `str` by default (before 3.0 they were `object`). It stores strings only, supports missing values as NaN, and is faster and smaller, especially when `pyarrow` is installed.
5. `Series(["a", None])` is still `str` and the missing value is detected by `isna()`.
6. A mixed list such as `["a", 1]` cannot be `str`, so it falls back to `object`. If you see `object` columns in 3.0, you have mixed types, which is worth investigating.

REAL JOB

Migration checklist from pandas 2.x: (1) replace chained assignment with `.loc`; (2) do not rely on `dtype == object` to find text columns, use `pd.api.types.is_string_dtype` or `select_dtypes(include="str")`; (3) replace `applymap` by `DataFrame.map`; (4) use `"ME"`, `"h"`, `"QE"` frequency aliases; (5) do not assume datetimes are nanoseconds: the unit (`ns`, `us`, `ms`, `s`) is inferred, so avoid `.astype("int64")` on a datetime and expecting nanoseconds; (6) grouping by a categorical now defaults to `observed=True` (unused categories are not shown); pass `observed=False` if you want them.

17.15 Recipes: "How do I ...?"

Each recipe is a complete, runnable answer. Copy the pattern, then change the column names.

Recipe 1. How do I clean a messy table end to end?

The task: a CSV export with inconsistent headers, padded text, mixed number formats, repeated rows, impossible values and missing cells. Write the cleaning as one function so it can be tested and rerun.

```
import io
import numpy as np
import pandas as pd

text = """Order ID,Customer Name,Country,Amount,Order Date,Status
1001, Ana Silva ,no,"1,250.50",2025-03-01,paid
1002,bo chen,NO,80,2025-03-02,PAID
1002,bo chen,NO,80,2025-03-02,PAID
1003,Cy ONeil,it,-5,2025-03-03,refunded
1004,,IT,,2025-03-04,paid
1005,Di Park,pe,300,not a date,Paid
"""

def clean_orders(raw: pd.DataFrame) -> pd.DataFrame:
    return (
        raw
        .rename(columns=lambda c: c.strip().lower().replace(" ", "_"))
        .assign(
            customer_name=lambda d: d["customer_name"].str.strip().str.title(),
            country=lambda d: d["country"].str.upper(),
            status=lambda d: d["status"].str.lower(),
            amount=lambda d: pd.to_numeric(d["amount"].astype("str").str.replace(",", ""), errors="coerce"),
            order_date=lambda d: pd.to_datetime(d["order_date"], errors="coerce"),
        )
        .drop_duplicates()
        .assign(amount_ok=lambda d: d["amount"].ge(0))
    )

raw = pd.read_csv(io.StringIO(text), dtype={"Amount": "str"})
out = clean_orders(raw)
print(len(raw), "->", len(out))
print(out)
print(out.isna().sum().to_dict())
print(out.loc[out["amount_ok"] & out["order_date"].notna(), "amount"].sum())
```

```
6 -> 5
   order_id customer_name country  amount order_date    status  amount_ok
0      1001     Ana Silva      NO  1250.5 2025-03-01      paid       True
1      1002       Bo Chen      NO    80.0 2025-03-02      paid       True
3      1003      Cy Oneil      IT    -5.0 2025-03-03  refunded      False
4      1004           NaN      IT     NaN 2025-03-04      paid      False
5      1005       Di Park      PE   300.0        NaT      paid       True
{'order_id': 0, 'customer_name': 1, 'country': 0, 'amount': 1, 'order_date': 1, 'status': 0, 'amount_ok': 0}
1330.5
```

1. Reading `Amount` as text (`dtype="str"`) is deliberate: "1,250.50" contains a thousands comma, so it must be cleaned before conversion.
2. The function takes a raw frame and returns a cleaned one. Because it never modifies its input, you can call it twice safely and unit-test it on a tiny frame.
3. `rename` normalises headers. `str.strip().str.title()` fixes padding and case in names. Country codes are upper-cased, status lower-cased, so "PAID" and "paid" are the same value.
4. Amounts: remove the comma, then `to_numeric(errors="coerce")` turns unparseable or empty cells into NaN. Dates: the bad date "not a date" becomes `NaT` (not-a-time) instead of crashing.
5. `drop_duplicates()` removes the repeated row 1002, so six rows become five. Duplicates are removed *after* normalising, so that rows that differ only by case or spaces are caught.
6. Instead of deleting the impossible negative amount, we flag it with `amount_ok`. Flagging keeps evidence; you can decide later whether to drop.
7. The last two lines count missing values and total the valid, dated amounts: 1250.5 + 80 = 1330.5. Order 1005 is excluded (bad date), as are the negative amount and the missing amount.

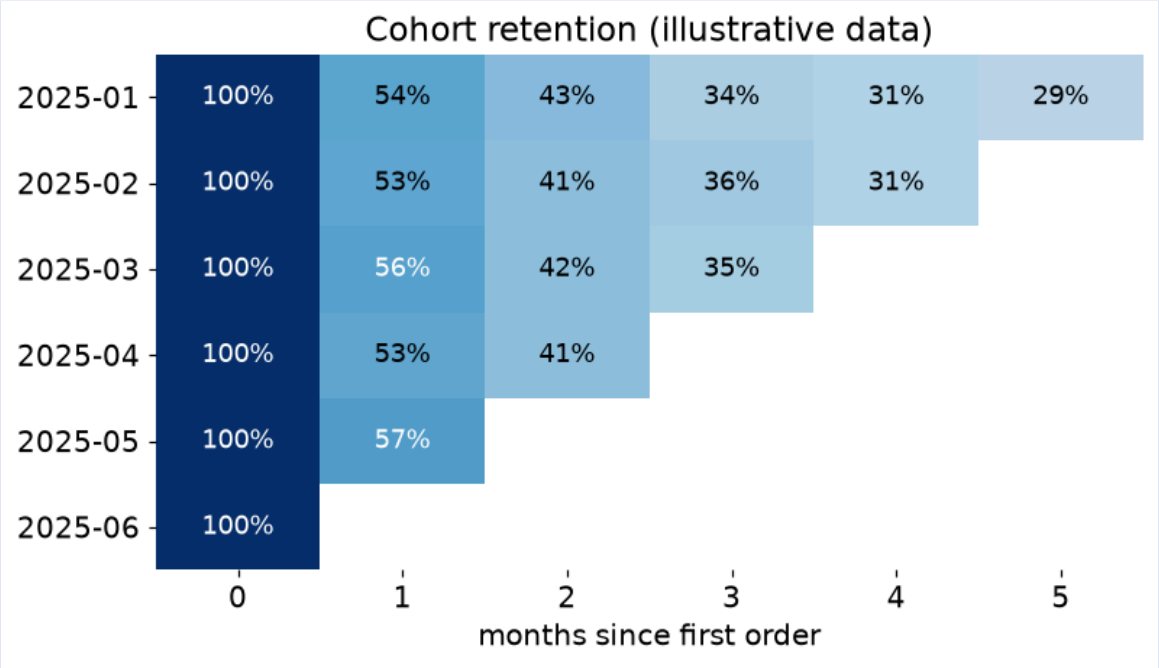
Recipe 2. How do I compute cohort retention?

The task: for each month of first purchase (the cohort), what percentage of customers buy again 1, 2, 3 months later?

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(5)
rows = []
for cust in range(300):
    start = rng.integers(0, 4)
    for m in range(start, 6):
        if m == start or rng.random() < 0.5 ** (m - start) * 0.8 + 0.1:
            rows.append((cust, pd.Timestamp("2025-01-01") + pd.DateOffset(months=int(m))))
orders = pd.DataFrame(rows, columns=["customer", "order_month"])

first = orders.groupby("customer")["order_month"].transform("min")
orders["cohort"] = first.dt.to_period("M")
orders["age"] = (orders["order_month"].dt.year - first.dt.year) * 12 + (orders["order_month"].dt.month - first.dt.month)

counts = orders.pivot_table(index="cohort", columns="age", values="customer", aggfunc="nunique")
retention = counts.div(counts[0], axis=0).round(2)
print(retention)
print(counts[0].tolist())
```

```
age        0     1     2     3     4     5
cohort
2025-01  1.0  0.53  0.22  0.17  0.18  0.09
2025-02  1.0  0.52  0.36  0.19  0.04   NaN
2025-03  1.0  0.58  0.36  0.19   NaN   NaN
2025-04  1.0  0.53  0.27   NaN   NaN   NaN
[90.0, 75.0, 73.0, 62.0]
```

1. We simulate 300 customers who start in one of four months and then return with a probability that falls with time. Real data would come from an orders table with a customer id and a date.
2. `transform("min")` gives each order row the customer's first order month, aligned to the original rows.
3. `to_period("M")` turns the date into a month label, the cohort.
4. `age` is the number of whole months between the order and the cohort start: years difference times 12 plus months difference.
5. `pivot_table(..., aggfunc="nunique")` counts distinct customers per cohort and age. Using `nunique` matters when a customer can place several orders in one month.
6. Dividing every row by its age-0 column (`div(counts[0], axis=0)`) turns counts into retention rates. Age 0 is 1.00 by definition. Cells that lie in the future are NaN.



A cohort retention triangle (illustrative data). Darker means more customers came back.

How to read it: each row is a group of customers who started in the same month. Moving right shows what share of that group is still buying 1, 2, 3 months later. The empty bottom-right corner is the future: those months have not happened yet. Compare columns down a column to see whether newer cohorts retain better than older ones.

Recipe 3. How do I score customers with RFM?

RFM scores each customer by **R**ecency (days since last order), **F**requency (number of orders) and **M**onetary value (total spent).

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(11)
n = 400
orders = pd.DataFrame({
    "customer": rng.integers(0, 60, n),
    "date": pd.Timestamp("2025-01-01") + pd.to_timedelta(rng.integers(0, 180, n), unit="D"),
    "amount": rng.gamma(2.0, 30.0, n).round(2),
})
today = orders["date"].max() + pd.Timedelta(days=1)

rfm = orders.groupby("customer").agg(
    recency=("date", lambda s: (today - s.max()).days),
    frequency=("date", "size"),
    monetary=("amount", "sum"),
)
rfm["r"] = pd.qcut(rfm["recency"], 4, labels=[4, 3, 2, 1]).astype(int)
rfm["f"] = pd.qcut(rfm["frequency"].rank(method="first"), 4, labels=[1, 2, 3, 4]).astype(int)
rfm["m"] = pd.qcut(rfm["monetary"], 4, labels=[1, 2, 3, 4]).astype(int)
rfm["score"] = rfm["r"] + rfm["f"] + rfm["m"]
print(rfm.sort_values("score", ascending=False).head(4).round(1))
print(rfm["score"].describe().round(2)[["min", "max"]].tolist())
print(rfm["r"].value_counts().sort_index().tolist())
```

```
          recency  frequency  monetary  r  f  m  score
customer
17             10         10     776.8  4  4  4     12
58              1          9     650.8  4  4  4     12
4               7         11     571.9  4  4  4     12
54             10          8     663.1  4  4  4     12
[3.0, 12.0]
[14, 16, 13, 17]
```

1. We fix "today" as the day after the latest order so results do not change with the calendar.
2. Named aggregation computes three measures per customer in one pass. Recency uses a small lambda: days between today and the customer's latest order.
3. `qcut(..., 4)` splits customers into four equal-size groups. For recency, *smaller is better*, so labels are reversed (4 for the most recent quartile).
4. Frequency has many ties (customers with the same count), and `qcut` fails when bin edges repeat. Ranking with `method="first"` first breaks ties so that the edges are unique.
5. The score is the sum, from 3 (worst) to 12 (best). Sorting by it lists the best customers first. A segment such as "score 10 or more" is then a one-line filter.

Recipe 4. How do I get the top N rows per group?

```
import pandas as pd

sales = pd.DataFrame({
    "region": ["n", "n", "n", "s", "s", "s", "s"],
    "product": ["a", "b", "c", "a", "b", "c", "d"],
    "revenue": [100, 250, 80, 40, 40, 300, 90],
})
top2 = (sales.sort_values(["region", "revenue"], ascending=[True, False])
             .groupby("region").head(2))
print(top2)
ranked = sales.assign(rk=sales.groupby("region")["revenue"].rank(method="dense", ascending=False))
print(ranked[ranked["rk"] <= 2][["region", "product", "rk"]])
best = sales.loc[sales.groupby("region")["revenue"].idxmax()]
print(best[["region", "product"]].to_dict("records"))
```

```
  region product  revenue
1      n       b      250
0      n       a      100
5      s       c      300
6      s       d       90
  region product   rk
0      n       a  2.0
1      n       b  1.0
5      s       c  1.0
6      s       d  2.0
[{'region': 'n', 'product': 'b'}, {'region': 's', 'product': 'c'}]
```

1. Option 1 sorts so the largest revenue is first within each region, then `head(2)` keeps two rows per region. Ties are broken by file order.
2. Option 2 ranks within the group. `method="dense"` gives tied values the same rank without skipping numbers. In the south group, products a and b tie at 40 and share rank 3, so they are excluded. But if two products tied for rank 2, both would be kept and you would get *more than 2 rows*. Choose the tie rule deliberately.
3. Option 3, `idxmax`, finds the row label of the largest value per group, and `loc` pulls those rows. This is the way to get exactly one row per group.

Recipe 5. How do I sessionise click events?

A **session** is a run of events by one user with no gap longer than 30 minutes. The trick: a new session starts whenever the gap is large, and a cumulative sum of those starts gives each event a session number.

```
import pandas as pd

ev = pd.DataFrame({
    "user": ["u1", "u1", "u1", "u1", "u2", "u2"],
    "ts": pd.to_datetime(["2025-03-01 09:00", "2025-03-01 09:10", "2025-03-01 10:00",
                          "2025-03-01 10:05", "2025-03-01 09:00", "2025-03-01 09:29"]),
}).sort_values(["user", "ts"])

gap = ev.groupby("user")["ts"].diff()
new_session = gap.isna() | (gap > pd.Timedelta(minutes=30))
ev["session"] = new_session.groupby(ev["user"]).cumsum()
print(ev)
summary = ev.groupby(["user", "session"]).agg(start=("ts", "min"), end=("ts", "max"), events=("ts", "size"))
summary["minutes"] = (summary["end"] - summary["start"]).dt.total_seconds() / 60
print(summary)
```

```
  user                 ts  session
0   u1 2025-03-01 09:00:00        1
1   u1 2025-03-01 09:10:00        1
2   u1 2025-03-01 10:00:00        2
3   u1 2025-03-01 10:05:00        2
4   u2 2025-03-01 09:00:00        1
5   u2 2025-03-01 09:29:00        1
                          start                 end  events  minutes
user session
u1   1       2025-03-01 09:00:00 2025-03-01 09:10:00       2     10.0
     2       2025-03-01 10:00:00 2025-03-01 10:05:00       2      5.0
u2   1       2025-03-01 09:00:00 2025-03-01 09:29:00       2     29.0
```

1. Sort by user and time first. Everything below assumes that order.
2. `groupby("user")["ts"].diff()` is the time since the previous event of the *same user*; each user's first event has NaT.
3. An event starts a new session if there is no previous event (`isna()`) or the gap exceeds 30 minutes. Result: a True/False Series.
4. Within each user, a cumulative sum of those flags numbers the sessions 1, 2, 3. For u1, events at 9:00 and 9:10 are session 1; 10:00 starts session 2 (50-minute gap) and 10:05 stays in it.
5. Finally, an ordinary groupby on (user, session) gives start, end, event count and duration. User u2 has a 29-minute gap, so it stays one session.

Recipe 6. How do I reconcile two tables?

Finance wants to know: which invoices are in the ledger but not in the bank file, which are in the bank file but not the ledger, and which are in both with different amounts?

```
import pandas as pd

ledger = pd.DataFrame({"invoice": ["A1", "A2", "A3", "A4"], "amount": [100.0, 250.0, 75.0, 60.0]})
bank = pd.DataFrame({"invoice": ["A2", "A3", "A5"], "amount": [250.0, 70.0, 40.0]})

m = ledger.merge(bank, on="invoice", how="outer", suffixes=("_ledger", "_bank"), indicator=True, validate="1:1")
only_ledger = m.loc[m["_merge"] == "left_only", "invoice"].tolist()
only_bank = m.loc[m["_merge"] == "right_only", "invoice"].tolist()
both = m[m["_merge"] == "both"].assign(diff=lambda d: d["amount_ledger"] - d["amount_bank"])
mismatch = both[both["diff"].abs() > 0.005]
print(only_ledger, only_bank)
print(mismatch[["invoice", "amount_ledger", "amount_bank", "diff"]])
print(m["_merge"].value_counts().to_dict())
```

```
['A1', 'A4'] ['A5']
  invoice  amount_ledger  amount_bank  diff
2      A3           75.0         70.0   5.0
{'left_only': 2, 'both': 2, 'right_only': 1}
```

1. An `outer` join keeps invoices from either table. `suffixes` labels the two amount columns. `validate="1:1"` ensures neither file has repeated invoice numbers, which would otherwise corrupt the comparison.
2. `indicator=True` adds `_merge` , so we can split the result into three groups without any second join.
3. A4 is `left_only` (in ledger, missing from the bank). A5 is `right_only` (bank money with no invoice).
4. For invoices in both, compute the difference and compare its absolute value with a small tolerance. Never test float money with `==` ; rounding noise makes equal amounts differ in the last digit.
5. The value counts give the headline numbers for the report: 2 matched, 1 only in each.

Recipe 7. How do I translate SQL to pandas?

SQL	pandas
`SELECT a, b FROM t`	`t[["a","b"]]`
`SELECT * FROM t WHERE x > 5 AND y = 'z'`	`t[(t.x > 5) & (t.y == "z")]` or `t.query("x > 5 and y == 'z'")`
`SELECT DISTINCT a FROM t`	`t["a"].drop_duplicates()`
`SELECT a, SUM(x) AS s FROM t GROUP BY a`	`t.groupby("a", as_index=False).agg(s=("x","sum"))`
`... HAVING SUM(x) > 10`	`...agg(s=("x","sum")).query("s > 10")`
`ORDER BY x DESC LIMIT 5`	`t.nlargest(5, "x")` or `t.sort_values("x", ascending=False).head(5)`
`a JOIN b ON a.k = b.k`	`a.merge(b, on="k")`
`a LEFT JOIN b ON ...`	`a.merge(b, on="k", how="left")`
`a UNION ALL b`	`pd.concat([a, b], ignore_index=True)`
`UNION` (no duplicates)	`pd.concat([a, b]).drop_duplicates()`
`WHERE k IN (SELECT k FROM b)`	`a[a.k.isin(b.k)]`
`WHERE k NOT IN (...)`	`a[~a.k.isin(b.k)]`
`CASE WHEN x > 0 THEN 'p' ELSE 'n' END`	`np.where(t.x > 0, "p", "n")` or `np.select`
`COALESCE(x, 0)`	`t["x"].fillna(0)`
`COUNT(*)` / `COUNT(x)` / `COUNT(DISTINCT x)`	`len(t)` or `size()` / `count()` / `nunique()`
`ROW_NUMBER() OVER (PARTITION BY k ORDER BY x)`	`t.sort_values("x").groupby("k").cumcount() + 1`
`RANK() OVER (PARTITION BY k ORDER BY x DESC)`	`t.groupby("k")["x"].rank(method="min", ascending=False)`
`SUM(x) OVER (PARTITION BY k)`	`t.groupby("k")["x"].transform("sum")`
`LAG(x) OVER (PARTITION BY k ORDER BY d)`	`t.sort_values("d").groupby("k")["x"].shift(1)`
`SUM(x) OVER (ORDER BY d ROWS 6 PRECEDING)`	`t.sort_values("d")["x"].rolling(7).sum()`
`DATE_TRUNC('month', d)`	`t["d"].dt.to_period("M")` or `.dt.to_period("M").dt.to_timestamp()`
`PIVOT` (conditional aggregation)	`t.pivot_table(index=, columns=, values=, aggfunc=)`

The next block proves a few of these translations by running the same query in SQLite and in pandas and comparing.

```
import sqlite3
import pandas as pd

t = pd.DataFrame({"k": ["a", "a", "b", "b", "b"], "x": [3, 9, 4, 4, 1], "d": [1, 2, 1, 2, 3]})
con = sqlite3.connect(":memory:")
t.to_sql("t", con, index=False)

sql = pd.read_sql("""
    SELECT k, SUM(x) AS s, COUNT(*) AS n
    FROM t GROUP BY k HAVING SUM(x) > 8 ORDER BY s DESC""", con)
pdf = (t.groupby("k", as_index=False).agg(s=("x", "sum"), n=("x", "size"))
        .query("s > 8").sort_values("s", ascending=False).reset_index(drop=True))
print(sql.equals(pdf))
print(pdf)

win = pd.read_sql("SELECT k, d, x, ROW_NUMBER() OVER (PARTITION BY k ORDER BY x DESC, d) AS rn FROM t ORDER BY k, rn",
con)
t2 = t.sort_values(["k", "x", "d"], ascending=[True, False, True])
t2["rn"] = t2.groupby("k").cumcount() + 1
print(win.equals(t2[["k", "d", "x", "rn"]].reset_index(drop=True)))
```

```
True
   k   s  n
0  a  12  2
1  b   9  3
True
```

1. The pandas table is written to an in-memory SQLite database with `to_sql`, so the same data lives on both sides.
2. The SQL query groups, filters groups with `HAVING` and sorts. The pandas version chains `groupby/agg`, `query` (the HAVING step, applied after aggregation) and `sort_values`.
3. `equals` prints True: same rows, same order, same dtypes.
4. For the window function, `ROW_NUMBER() OVER (PARTITION BY k ORDER BY x DESC, d)` becomes: sort by key, then x descending, then d; then `cumcount() + 1` inside each group. Including `d` as a tie-breaker in both versions makes the result deterministic.

Recipe 8. How do I find the biggest changes between two periods?

```
import pandas as pd

sales = pd.DataFrame({
    "product": ["a", "b", "c", "a", "b", "d"],
    "month": ["jan", "jan", "jan", "feb", "feb", "feb"],
    "units": [100, 50, 20, 130, 40, 10],
})
wide = sales.pivot(index="product", columns="month", values="units").fillna(0)
wide["change"] = wide["feb"] - wide["jan"]
wide["pct"] = ((wide["change"] / wide["jan"]).replace([float("inf"), -float("inf")], float("nan")) * 100).round(1)
print(wide.sort_values("change", ascending=False))
```

```
month      feb    jan  change    pct
product
a        130.0  100.0    30.0   30.0
d         10.0    0.0    10.0    NaN
b         40.0   50.0   -10.0  -20.0
c          0.0   20.0   -20.0 -100.0
```

1. Pivot to one row per product and one column per month; products missing in a month get NaN, which we fill with 0 because "not sold" means zero units.

2. Absolute change is February minus January.
3. Percentage change divides by January. Product d did not exist in January, so the division gives infinity; replacing infinities by NaN avoids a meaningless number.
4. Sorting by `change` shows the winners at the top (a: +30) and the biggest loser last (c: -20).

Recipe 9. How do I find outliers?

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(1)
s = pd.Series(np.append(rng.normal(100, 10, 200), [250, -40]), name="x")
q1, q3 = s.quantile([0.25, 0.75])
iqr = q3 - q1
low, high = q1 - 1.5 * iqr, q3 + 1.5 * iqr
out = s[(s < low) | (s > high)]
print(round(low, 1), round(high, 1))
print(out.round(1).to_dict())
z = (s - s.mean()) / s.std()
print((z.abs() > 3).sum())
clipped = s.clip(low, high)
print(round(clipped.min(), 1), round(clipped.max(), 1))
```

```
74.9 123.8
{24: 72.9, 124: 125.5, 183: 128.7, 193: 72.9, 200: 250.0, 201: -40.0}
2
74.9 123.8
```

1. 200 normal values around 100 plus two planted outliers (250 and -40).
2. The IQR rule: the interquartile range is Q3 minus Q1; anything more than 1.5 IQRs beyond the quartiles is flagged. It is robust because quartiles ignore extreme values.
3. The z-score rule flags values more than 3 standard deviations from the mean. It is less robust, since outliers inflate the mean and standard deviation themselves.
4. The IQR rule flagged six points, four of them ordinary values near the fences, while the z-score rule flagged just the two planted ones (a normal sample always has a few points beyond 1.5 IQR). Neither rule is "right"; they answer different questions.
5. `clip(low, high)` winsorises: it limits values to the fences instead of deleting rows. Whether to remove, clip or keep an outlier is a business decision; investigate before you drop.

Recipe 10. How do I fill gaps in a time series and compute week-over-week growth?

```
import pandas as pd

s = pd.Series([5, 8, 6], index=pd.to_datetime(["2025-03-03", "2025-03-05", "2025-03-10"]), name="orders")
daily = s.asfreq("D", fill_value=0)
print(daily.tolist())
weekly = daily.resample("W-SUN").sum()
print(weekly.to_dict())
print(weekly.pct_change().round(3).tolist())
```

```
[5, 0, 8, 0, 0, 0, 0, 6]
{Timestamp('2025-03-09 00:00:00'): 13, Timestamp('2025-03-16 00:00:00'): 6}
[nan, -0.538]
```

1. The series has only three observed days. `asfreq("D", fill_value=0)` inserts every calendar day between the first and last, with 0 for days with no orders. Be sure that "no row" truly means zero, not "unknown".

2. `resample("W-SUN")` groups days into weeks that end on Sunday: the week of 3 to 9 March sums to 13, and 10 March starts the next week with 6.
3. `pct_change()` compares each week with the previous one: 6 / 13 - 1.

Recipe 11. How do I read many CSV files into one table?

```
import tempfile
from pathlib import Path
import pandas as pd

folder = Path(tempfile.mkdtemp())
for month in ["2025-01", "2025-02", "2025-03"]:
    pd.DataFrame({"id": [1, 2], "v": [10, 20]}).to_csv(folder / f"sales_{month}.csv", index=False)

parts = [pd.read_csv(p).assign(source=p.stem) for p in sorted(folder.glob("sales_*.csv"))]
all_data = pd.concat(parts, ignore_index=True)
print(all_data.shape)
print(all_data["source"].unique().tolist())
```

```
(6, 3)
['sales_2025-01', 'sales_2025-02', 'sales_2025-03']
```

1. `Path.glob("sales_*.csv")` lists matching files; `sorted` makes the order deterministic.
2. A list comprehension reads each file and adds a `source` column with the file name (`p.stem`), so you can trace any row back to its file.
3. One `concat` at the end is much faster than appending inside a loop, because each append copies all earlier rows.

Recipe 12. How do I flag rows that changed between two snapshots?

```
import pandas as pd

old = pd.DataFrame({"id": [1, 2, 3], "price": [10.0, 20.0, 30.0], "stock": [5, 0, 7]}).set_index("id")
new = pd.DataFrame({"id": [1, 2, 4], "price": [10.0, 22.0, 15.0], "stock": [5, 0, 3]}).set_index("id")

added = new.index.difference(old.index).tolist()
removed = old.index.difference(new.index).tolist()
common = old.index.intersection(new.index)
diff = old.loc[common].compare(new.loc[common])
print(added, removed)
print(diff)
```

```
[4] [3]
   price
    self other
id
2   20.0  22.0
```

1. With `id` as the index, set operations on the indexes find rows that were added (4) and removed (3).
2. For ids present in both, restrict both frames to the same labels; `compare` needs identically labelled frames.
3. `DataFrame.compare` returns only the cells that differ, with `self` and `other` columns: price of id 2 went from 20 to 22. Unchanged columns (stock) do not appear at all.

17.16 Performance and Pitfalls

Most pandas disasters are not crashes; they are quiet wrong answers. Each pitfall below has the same layout: a red box describing the trap, then one block that runs the *wrong* code, shows the symptom, and runs the *fix*.

COMMON MISTAKE 1: CHAINED ASSIGNMENT

Selecting a column and then assigning into the result, such as `df["a"][mask] = 0` or `df[mask]["a"] = 0`. In pandas 3.0 the first selection is a lazy copy, so your change lands on a throwaway object. Symptom: no error, data unchanged.

```
import pandas as pd

df = pd.DataFrame({"a": [5, -1, 7]})
df[df["a"] < 0]["a"] = 0          # wrong: edits a temporary copy
print("wrong:", df["a"].tolist())
df.loc[df["a"] < 0, "a"] = 0      # fix: one .loc assignment on df itself
print("fixed:", df["a"].tolist())
```

```
wrong: [5, -1, 7]
fixed: [5, 0, 7]
```

COMMON MISTAKE 2: INPLACE AND NONE

Methods with `inplace=True` return `None`. Writing `df = df.dropna(inplace=True)` replaces your table with `None`. Symptom: `AttributeError: 'NoneType' ...` on the next line. Prefer the returned-copy style: `df = df.dropna()`.

```
import pandas as pd

df = pd.DataFrame({"a": [1.0, None, 3.0]})
res = df.dropna(inplace=True)      # wrong to assign this
print("wrong:", res)
df = pd.DataFrame({"a": [1.0, None, 3.0]})
df = df.dropna()                    # fix
print("fixed:", len(df))
```

```
wrong: None
fixed: 2
```

COMMON MISTAKE 3: A JOIN THAT MULTIPLIES ROWS

The right table has a duplicate key, so each matching left row is repeated. Symptom: the total revenue is too high and nothing warns you. Fix: `validate="m:1"`, or de-duplicate first.

```
import pandas as pd

orders = pd.DataFrame({"cid": [1, 2], "amount": [100, 50]})
cust = pd.DataFrame({"cid": [1, 1, 2], "tier": ["gold", "gold", "basic"]})
bad = orders.merge(cust, on="cid", how="left")
print("wrong total:", bad["amount"].sum())
try:
    orders.merge(cust, on="cid", how="left", validate="m:1")
except Exception as e:
    print("caught:", type(e).__name__)
good = orders.merge(cust.drop_duplicates("cid"), on="cid", how="left", validate="m:1")
print("fixed total:", good["amount"].sum())
```

```
wrong total: 250
caught: MergeError
fixed total: 150
```

COMMON MISTAKE 4: COMPARING WITH NAN

`df[df["x"] == np.nan]` returns nothing, because NaN is not equal to anything, not even itself. Symptom: an empty result when you know there are gaps. Fix: `isna()`.

```
import numpy as np
import pandas as pd

df = pd.DataFrame({"x": [1.0, np.nan, 3.0]})
print("wrong:", len(df[df["x"] == np.nan]))
print("fixed:", len(df[df["x"].isna()]))
```

```
wrong: 0
fixed: 1
```

COMMON MISTAKE 5: SILENT INTEGER WRAP-AROUND

Down-casting to a smaller integer type without checking the range. NumPy does not raise: values that do not fit wrap around. Symptom: negative numbers where there were none. Fix: check the range or use `to_numeric(downcast=...)`.

```
import pandas as pd

s = pd.Series([100, 200, 300])
print("wrong:", s.astype("int8").tolist())
print("fixed:", pd.to_numeric(s, downcast="integer").dtype)
```

```
wrong: [100, -56, 44]
fixed: int16
```

COMMON MISTAKE 6: GROUPBY SILENTLY DROPS MISSING KEYS

Rows whose group key is NaN vanish from the result by default. Symptom: group totals add up to less than the grand total. Fix: `dropna=False`, or fill the key first.

```
import pandas as pd

df = pd.DataFrame({"k": ["a", None, "a", "b"], "v": [1, 2, 3, 4]})
print("wrong:", df.groupby("k")["v"].sum().sum(), "of", df["v"].sum())
print("fixed:", df.groupby("k", dropna=False)["v"].sum().sum())
```

```
wrong: 8 of 10
fixed: 10
```

COMMON MISTAKE 7: ITERROWS CHANGES TYPES

`iterrows()` turns each row into a Series, and a Series has one dtype, so mixed columns are upcast. Symptom: integers become floats (or everything becomes text). It is also slow. Fix: `itertuples()` or, better, a vectorised expression.

```
import pandas as pd

df = pd.DataFrame({"n": [1, 2], "x": [1.5, 2.5]})
for _, row in df.iterrows():
    print("wrong:", type(row["n"]).__name__, end="  ")
print()
for row in df.itertuples():
    print("fixed:", type(row.n).__name__, end="  ")
print()
```

```
wrong: float64  wrong: float64
fixed: int  fixed: int
```

COMMON MISTAKE 8: AMBIGUOUS DATES

`to_datetime("01/02/2025")` reads month first, so it becomes 2 January, not 1 February. Symptom: European dates silently swap day and month whenever both are 12 or less. Fix: always pass `format=`.

```
import pandas as pd

s = pd.Series(["01/02/2025"])
print("wrong:", pd.to_datetime(s)[0].date())
print("fixed:", pd.to_datetime(s, format="%d/%m/%Y")[0].date())
```

```
wrong: 2025-01-02
fixed: 2025-02-01
```

COMMON MISTAKE 9: ASSIGNING A SERIES WITH A DIFFERENT INDEX

Assignment aligns on index labels, not position. If the labels differ, you get NaN. Symptom: a new column that is mostly empty. Fix: assign the underlying values with `.to_numpy()` when you really mean "by position" (and check the lengths), or fix the index.

```
import pandas as pd

df = pd.DataFrame({"a": [1, 2, 3]}, index=[10, 11, 12])
other = pd.Series([7, 8, 9])                # index 0, 1, 2
df["b"] = other
print("wrong:", df["b"].tolist())
df["b"] = other.to_numpy()
print("fixed:", df["b"].tolist())
```

```
wrong: [nan, nan, nan]
fixed: [7, 8, 9]
```

COMMON MISTAKE 10: AGGREGATING TEXT AS NUMBERS

Numbers stored as text (from a CSV with a stray character) make `mean()` fail (and in older object columns `sum()` would glue the strings together). Symptom: a `TypeError` or a nonsense result. Fix: convert with `to_numeric`, then aggregate.

```
import pandas as pd

df = pd.DataFrame({"qty": ["3", "4", "x"]})
try:
    df["qty"].mean()
except TypeError as e:
    print("wrong:", type(e).__name__)
qty = pd.to_numeric(df["qty"], errors="coerce")
print("fixed:", qty.mean(), "with", int(qty.isna().sum()), "bad value")
```

```
wrong: TypeError
fixed: 3.5 with 1 bad value
```

COMMON MISTAKE 11: FLOAT MONEY

Sums and comparisons of decimal amounts in binary floating point carry tiny errors. Symptom: `0.1 + 0.2 == 0.3` is False, and reconciliations show phantom differences of 0.0000001. Fix: compare with a tolerance (`np.isclose`) or store integer cents.

```
import numpy as np
import pandas as pd

s = pd.Series([0.1, 0.2])
print("wrong:", s.sum() == 0.3, repr(s.sum()))
print("fixed:", bool(np.isclose(s.sum(), 0.3)), (pd.Series([10, 20]).sum() == 30))
```

```
wrong: False np.float64(0.30000000000000004)
fixed: True True
```

COMMON MISTAKE 12: GROWING A DATAFRAME IN A LOOP

Calling `pd.concat` or appending one row at a time copies the whole table every time, so work grows with the square of the row count. Symptom: a loop that is fine for 1,000 rows and hopeless for 100,000. Fix: collect rows in a Python list and build the DataFrame once.

```
import pandas as pd

rows = []
for i in range(1000):
    rows.append({"i": i, "sq": i * i})
df = pd.DataFrame(rows)          # fix: one construction at the end
print(df.shape, df["sq"].iloc[-1])
```

```
(1000, 2) 998001
```

Speed and memory rules that actually matter

1. **Load less.** `usecols` and `dtype` at read time beat any later optimisation.
2. **Shrink types.** `category` for repeated text, smaller number types where the range allows.
3. **Vectorise.** Column arithmetic, `np.where`, `map`, the string and datetime accessors; avoid `apply(axis=1)` and `iterrows`.

4. **Use built-in aggregations** (`"sum"`, `"mean"`) in groupby instead of lambdas.
5. **Filter early, join late.** Reduce rows and columns before an expensive merge or groupby.
6. **Build once.** Collect pieces in a list, then one `concat`.
7. **Measure.** Use `time.perf_counter` or `%timeit`; do not guess.
8. **Know your ceiling.** When a table is too large for memory even after the diet, move to chunks, Parquet with column selection, DuckDB or Polars rather than fighting pandas.

A quick measured example of rules 3 and 4 follows. The numbers vary by machine; only the ordering is the lesson.

```
import time
import numpy as np
import pandas as pd

rng = np.random.default_rng(0)
n = 300_000
df = pd.DataFrame({"k": rng.integers(0, 1000, n), "x": rng.normal(size=n)})

def timeit(f):
    t = time.perf_counter(); f(); return time.perf_counter() - t

fast = timeit(lambda: df.groupby("k")["x"].agg("sum"))
slow = timeit(lambda: df.groupby("k")["x"].agg(lambda s: s.sum()))
print("lambda slower than built-in:", slow > fast)
a = df.groupby("k")["x"].agg("sum")
b = df.groupby("k")["x"].agg(lambda s: s.sum())
print(np.allclose(a, b))
```

```
lambda slower than built-in: True
True
```

1. 300,000 rows in 1,000 groups, so a lambda would be called 1,000 times.
2. `timeit` times a call with a monotonic clock. We print only a comparison, not the raw seconds, so that the output is the same on every machine.
3. The built-in `"sum"` runs in compiled code for all groups at once; the lambda runs Python for each group.
4. `allclose` confirms both versions give the same answer: the speed-up costs no correctness.

17.17 Cross-library Integration

pandas rarely works alone. The arrows below are the handoffs you will use most.

Partner	Direction	Code	Watch out for
NumPy	both	`df.to_numpy()`, `pd.DataFrame(arr, columns=...)`, `np.log1p(df["x"])`	Ufuncs keep the index; `to_numpy()` loses labels
matplotlib / seaborn	out	`df.plot(...)`, `sns.barplot(data=df, x=, y=)`	Seaborn wants long (tidy) data
scikit-learn	both	`X = df[features]`, `ColumnTransformer`	Split before fitting; keep column order the same at predict time
SQL (sqlite3, SQLAlchemy)	both	`read_sql`, `to_sql`	Push filters into SQL; do not read a whole table to filter in pandas
Parquet / Arrow	both	`to_parquet`, `read_parquet(columns=)` (not run here)	Needs `pyarrow`; preserves dtypes unlike CSV

Partner	Direction	Code	Watch out for
Excel	both	`read_excel`, `ExcelWriter`	Dates and merged cells need care; slow for large files
JSON / requests	in	`pd.json_normalize(resp.json())`	Nested lists need `record_path`
Polars / DuckDB	both	`pl.from_pandas(df)`, `duckdb.sql("select ...")` (not run here)	Different semantics for nulls and ordering

Below, one DataFrame goes through NumPy, SQL, scikit-learn and matplotlib in a few lines. Everything here is installed.

```
import sqlite3
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

rng = np.random.default_rng(0)
df = pd.DataFrame({"ad": rng.uniform(0, 10, 200)})
df["sales"] = 3 * df["ad"] + 5 + rng.normal(0, 1, 200)

con = sqlite3.connect(":memory:")
df.to_sql("t", con, index=False)
big = pd.read_sql("SELECT * FROM t WHERE ad > 5", con)

X_tr, X_te, y_tr, y_te = train_test_split(df[["ad"]], df["sales"], test_size=0.25, random_state=0)
model = LinearRegression().fit(X_tr, y_tr)
pred = pd.Series(model.predict(X_te), index=X_te.index, name="pred")
res = pd.concat([X_te, y_te, pred], axis=1)
print(len(big), res.shape)
print(round(float(model.coef_[0]), 2), round(float(model.intercept_), 2))
print(res.head(2).round(2))

import matplotlib.pyplot as plt
ax = res.plot.scatter("ad", "sales", color="#2b4fd6")
print(type(ax).__name__)
print(np.log1p(df["ad"]).head(2).round(3).tolist())
```

```
112 (50, 3)
2.95 5.15
       ad  sales   pred
18   3.00  14.56  14.00
170  3.44  14.68  15.32
Axes
[1.997, 1.308]
```

1. Synthetic data: sales are 3 times ad spend plus 5 plus noise.
2. The table is written to SQLite and a filtered subset is read back with `read_sql`: the filter runs in the database.
3. `train_test_split` accepts DataFrames and Series and returns pieces that keep their index labels, so you can join predictions back to the right rows.
4. `df[["ad"]]` (double brackets) gives a DataFrame, which scikit-learn requires for features; `df["sales"]` is the 1-D target.
5. The learned slope and intercept are close to the true 3 and 5.
6. Wrapping predictions in a Series with the test index lets `concat(axis=1)` align them with the inputs and truth, a clean table for error analysis.
7. `res.plot.scatter` returns a matplotlib Axes, so everything in the matplotlib chapter applies. NumPy functions such as `np.log1p` accept a Series and return a Series with the same index.

17.18 How to Solve Any pandas Problem

The decision flow

1. **State the output table.** Write down the columns and what one row means: "one row per customer per month". The grain of the output decides everything.
2. **Identify the input grain.** What does one row of the input mean? If input grain is finer than output grain, you need `groupby` (or `resample`). If equal, you need column operations. If the information is in another table, you need a `merge`.
3. **Pick the verb.** Filter rows (mask, `query`), create columns (`assign`, `np.where`), summarise (`groupby.agg`), compare to group (`transform`), combine tables (`merge`, `concat`), change shape (`pivot`, `melt`), time logic (`resample`, `rolling`, `shift`).
4. **Write it as a chain, step by step.** Print the head after each step until the shape is right.
5. **Verify** with counts, totals and a hand-checked row before you trust it.

Six to eight recurring patterns, each solved

Pattern A: "one row per X with a total" (aggregate). Group by X, aggregate. **Pattern B: "compare each row with its group" (transform). Pattern C: "latest/first/best row per X" (sort then drop_duplicates, or idxmax). Pattern D: "attach information from another table" (merge with validate). Pattern E: "what did not match" (indicator or anti-join). Pattern F: "wide to long and back" (melt/pivot). Pattern G: "what happened before/after this row" (sort, groupby, shift/diff/cumsum). Pattern H: "bucket the values" (cut/qcut, dt.to_period, resample).** Here is one solved example for each, on one small dataset.

```
import pandas as pd

o = pd.DataFrame({
    "cust": ["a", "a", "a", "b", "b", "c"],
    "day": pd.to_datetime(["2025-03-01", "2025-03-05", "2025-03-20", "2025-03-02", "2025-03-03", "2025-03-09"]),
    "amt": [10, 30, 20, 50, 5, 40],
})
regions = pd.DataFrame({"cust": ["a", "b", "d"], "region": ["north", "south", "west"]})

A = o.groupby("cust")["amt"].sum()
B = (o["amt"] / o.groupby("cust")["amt"].transform("sum")).round(2)
C = o.sort_values("day").drop_duplicates("cust", keep="last")[["cust", "amt"]]
D = o.merge(regions, on="cust", how="left", validate="m:1")
E = regions.merge(o, on="cust", how="left", indicator=True).query("_merge == 'left_only'")["cust"].tolist()
F = o.pivot_table(index="cust", columns=o["day"].dt.to_period("M"), values="amt", aggfunc="sum")
G = o.sort_values(["cust", "day"]).assign(days_since=lambda d: d.groupby("cust")["day"].diff().dt.days)
H = pd.cut(o["amt"], [0, 10, 30, 100], labels=["small", "mid", "large"])

print(A.to_dict())
print(B.tolist())
print(C.to_dict("records"))
print(D["region"].tolist())
print(E)
print(F.round(0).to_dict())
print(G["days_since"].tolist())
print(H.tolist())
```

```
{'a': 60, 'b': 55, 'c': 40}
[0.17, 0.5, 0.33, 0.91, 0.09, 1.0]
[{'cust': 'b', 'amt': 5}, {'cust': 'c', 'amt': 40}, {'cust': 'a', 'amt': 20}]
['north', 'north', 'north', 'south', 'south', nan]
['d']
{Period('2025-03', 'M'): {'a': 60, 'b': 55, 'c': 40}}
[nan, 4.0, 15.0, nan, 1.0, nan]
['small', 'mid', 'mid', 'large', 'small', 'large']
```

1. **A** sums by customer: a = 60, b = 55, c = 40.

2. **B** is each order's share of its customer total (customer c has one order, so 1.0).
3. **C** sorts by day and keeps the last row per customer: the latest order.
4. **D** attaches regions with a left join. Customer c has no region, so NaN. `validate="m:1"` proves the regions table is unique per customer.
5. **E** finds customers in the regions table that have no orders: customer d. The indicator column distinguishes unmatched rows.
6. **F** pivots to customers by month; every order is in March, so there is one column.
7. **G** sorts, then diff within each customer gives days since the previous order; each customer's first order is NaN.
8. **H** buckets amounts: 10 is small (closed right edge), 30 is mid, 50 is large.

How to verify

- **Row counts** before and after each filter or join: `len(df)`. Joins should not change them unexpectedly.
- **Totals:** sum of group totals equals the grand total (watch for NaN keys).
- **Uniqueness:** `df["id"].is_unique` or `df.duplicated(["id"]).any()`.
- **Spot-check one entity** by hand: filter to one customer and compare with the aggregated row.
- **Assertions as guard rails:** `assert out["share"].between(0, 1).all()` at the end of a pipeline fails loudly when data changes.
- **Compare with a second method:** compute the same number with SQL or plain Python on a small sample.

How to debug

```
import pandas as pd

df = pd.DataFrame({"city": ["Oslo", "oslo ", "Rome"], "x": ["1", "2", "3"]})
steps = (
    df
    .assign(city=lambda d: d["city"].str.strip().str.lower())
    .pipe(lambda d: (print("after clean:", d["city"].tolist()), d)[1])
    .assign(x=lambda d: pd.to_numeric(d["x"]))
    .groupby("city", as_index=False)["x"].sum()
)
print(steps.to_dict("records"))
print(df["x"].dtype, df.shape, df.index.is_unique)
```

```
after clean: ['oslo', 'oslo', 'rome']
[{'city': 'oslo', 'x': 3}, {'city': 'rome', 'x': 3}]
str (3, 2) True
```

1. A chain is debugged by inserting a `pipe` with a tiny function that prints and returns its input (here a lambda that prints and returns the frame). You see the state midway without breaking the chain.
2. Before cleaning, "Oslo" and "oslo " would be separate groups; after cleaning they merge into one with x = 3, and Rome has x = 3.
3. The last line is the standard triage: dtype, shape and index uniqueness. If a column is not the dtype you expect, fix that first.
4. Other quick tools: `df.head()` after each step, `df.sample(5, random_state=0)`, `df["col"].unique()[:20]` to see odd values, `df.query("...")` to isolate the failing rows, and reducing the data to the smallest frame that still shows the bug.

17.19 Going Deeper: Internals, Memory, Speed and What Interviewers Probe

How a DataFrame is really stored

A DataFrame is not a grid of Python objects. Each column is a typed array (a NumPy array for numbers, booleans and datetimes, an extension array for categories, nullable integers and strings). pandas keeps these arrays together with the Index objects that label the rows and columns. Operations on a column therefore run as loops over a contiguous typed array in compiled code, which is the whole source of pandas' speed. A Python-level function applied row by row throws that advantage away, because every cell must be boxed into a Python object again.

KEY IDEA

The Index is a typed, **immutable** array with a lookup structure that is built lazily on first use. Looking up a label in a unique index is a hash lookup, not a scan, which is why `df.loc[label]` on a set index is fast and why a non-unique index makes lookups slower and sometimes returns several rows instead of one.

```
import numpy as np
import pandas as pd

codes, uniques = pd.factorize(pd.Series(["b", "a", "b", "c"]))
print(codes.tolist(), list(uniques))

df = pd.DataFrame({"a": [1, 2, 3]})
shallow = df.copy(deep=False)
print(np.shares_memory(df["a"].to_numpy(), shallow["a"].to_numpy()))
shallow.loc[0, "a"] = 99
print(np.shares_memory(df["a"].to_numpy(), shallow["a"].to_numpy()), df["a"].tolist())

idx = pd.Index(["x", "y"])
try:
    idx[0] = "z"
except TypeError as e:
    print("Index is immutable:", type(e).__name__)
```

```
[0, 1, 0, 2] ['b', 'a', 'c']
True
False [1, 2, 3]
Index is immutable: TypeError
```

1. `factorize` converts labels to small integer codes (b=0, a=1, c=2) plus a table of unique values. This is exactly the first step `groupby` and `category` use: integers are cheap to compare, hash and count.
2. A shallow copy shares the underlying memory with the original, so `shares_memory` is True: copying costs almost nothing.
3. When we write to the shallow copy, copy-on-write notices that the data is shared, copies just that column first, and then writes. The memory is no longer shared, and the original is untouched. This is the mechanism behind "never a surprise write".
4. Index objects refuse item assignment; to change labels you build a new Index (`rename` , `set_index`).

What the main operations do underneath

- **groupby** factorizes the key column(s) into integer group codes, then reduces each value column by those codes. Built-in reducers (`sum` , `mean` , `count` , `min` , `max`) are compiled loops. A lambda forces a Python call per group.
- **merge** is a hash join: it builds a hash table of keys from one side and probes it with the other. Cost grows roughly with the sum of the two sizes, plus the size of the output, which is why a many-to-many join can explode memory.
- **sort_values** sorts one or more keys; use `kind="stable"` when ties must keep their input order.
- **Boolean masks** allocate a full-length mask and a new frame; `query` and `eval` can avoid temporaries for large frames when the optional `numexpr` package is installed (not run here).
- **String dtype:** in pandas 3.0 the default `str` dtype stores text in a dedicated string array rather than as one Python object per cell. Missing values are NaN-like, and `.str` methods return typed results. With `pyarrow` installed it uses Arrow storage, which is smaller and faster; without it, a Python-backed storage is used. Check `pd.options.mode.string_storage` .
- **Datetime resolution:** timestamps are no longer always nanoseconds; the unit is inferred (our examples show microseconds). Arithmetic between different units converts to the finer one. Dates far outside the nanosecond range (year 1500 or 3000) now work without special handling.
- **Nullable dtypes** (`Int64` , `Float64` , `boolean`) keep a separate mask of missing values, so integers stay integers, at some cost in speed and memory compared with plain NumPy types.

Memory arithmetic you should be able to do in your head

An `int64` or `float64` column takes 8 bytes per row, so 10 million rows take 80 MB per column. A table with 20 numeric columns and 10 million rows is about 1.6 GB before any temporaries. A `groupby`, `merge` or `sort` can need an extra copy or two, so plan for two to three times the table size. That arithmetic tells you when to downcast, read fewer columns, process in chunks, or switch tools.

INTERVIEW	**What interviewers probe.** (1) Difference between `loc` and `iloc` and what slice end points include. (2) Why `apply(axis=1)` is slow and what to use instead. (3) `merge` types, how to detect and prevent row multiplication. (4) `transform` vs `agg` vs `apply`. (5) What copy-on-write means and why chained assignment no longer works. (6) How to handle a file bigger than memory. (7) How to deduplicate keeping the latest record. (8) How `NaN` behaves in comparisons, sums and groupby keys. (9) Time series: `resample` vs `rolling`, time zones. (10) Reading code and predicting its output, which is why the practice set has many "what does this print" tasks.

Processing a file bigger than memory

```
import io
import pandas as pd

lines = ["user,amount"] + [f"u{i % 4},{i}" for i in range(1, 21)]
text = "\n".join(lines)

totals = {}
for chunk in pd.read_csv(io.StringIO(text), chunksize=6):
    part = chunk.groupby("user")["amount"].sum()
    for user, value in part.items():
        totals[user] = totals.get(user, 0) + int(value)
print(totals)
print(sum(totals.values()), sum(range(1, 21)))
```

```
{'u0': 60, 'u1': 45, 'u2': 50, 'u3': 55}
210 210
```

1. We fake a 20-row file; a real one might have billions of rows on disk.
2. `chunksize=6` turns `read_csv` into an iterator of small DataFrames, so only six rows are in memory at a time.
3. For each chunk we compute the partial sum per user, then merge it into a running dictionary. This works for any aggregation that can be combined from partial results (sum, count, min, max). A mean needs a running sum and a running count; a median does not combine this way.
4. The grand total matches the arithmetic series 1 + 2 + ... + 20 = 210, a cheap correctness check.

Chapter recap

- A DataFrame is a dict of typed Series sharing one Index. Think in columns; operations align on labels.
- Read with explicit `dtype`, `parse_dates`, `na_values` and `usecols`; write with `index=False`.
- Inspect first: `shape`, `info`, `describe`, `value_counts`, `isna().sum()`, `duplicated().sum()`.
- Select with `loc` (labels, end included), `iloc` (positions, end excluded), Boolean masks with `& | ~` and brackets, or `query`.
- Assign with `df.loc[mask, col] = v`, `assign`, `np.where`, `np.select`; never chain indexing in pandas 3.0.
- Pick dtypes on purpose: `category` for low-cardinality text, nullable integers for gaps, `str` for text, datetimes with `format=`.
- Decide what missing means before filling; flag it; remember `groupby` drops NaN keys unless `dropna=False`.
- `groupby` is split-apply-combine: `agg` shrinks, `transform` keeps length, `filter` keeps groups. Use named aggregation.
- Joins: choose `how`, always `validate`, use `indicator` to see mismatches, and check the row count.
- Reshape with `melt` / `pivot` / `pivot_table`; windows with `rolling`, `expanding`, `resample`, `shift`.
- Vectorise, use built-in aggregations, shrink dtypes, build once, measure; leave pandas when data does not fit memory.

Formula / pattern	Meaning
`df.loc[mask, "c"] = v`	Safe conditional assignment
`df.groupby("k").agg(n=("x","size"), s=("x","sum"))`	One row per key, named results
`df["x"] / df.groupby("k")["x"].transform("sum")`	Share of group total
`df.sort_values("t").drop_duplicates("k", keep="last")`	Latest row per key
`a.merge(b, on="k", how="left", validate="m:1")`	Lookup that cannot multiply rows
`a.merge(b, how="outer", indicator=True)`	Reconciliation of two tables
`df.pivot_table(index=, columns=, values=, aggfunc=)`	Cross-tab with aggregation
`gap = g["ts"].diff(); (gap > lim).groupby(key).cumsum()`	Sessions
`s.resample("ME").sum()`	Monthly totals
`s.rolling(7).mean()`	Seven-row moving average
`pd.qcut(s, 4)` / `pd.cut(s, bins)`	Equal-count / chosen-edge bins
`df.pipe(f).assign(...).query(...)`	Readable method chain

Practice set

Graded from easy to expert. Try each one before reading the answer. Outputs shown were produced by running the code.

Problem 17.P.1 (Easy, what does this print?) What do the two lines print?

```
import pandas as pd

s = pd.Series([4, 8, 15], index=["a", "b", "c"])
print(s["b"], s.iloc[2])
print(s.loc["a":"b"].tolist())
```

Answer: `s["b"]` is a label lookup (8), `s.iloc[2]` is positional (15). The label slice `"a":"b"` includes both ends, giving two values.

```
8 15
[4, 8]
```

Problem 17.P.2 (Easy) A DataFrame `df` has 1000 rows. Which line gives the number of missing values in each column, and which gives the number of fully repeated rows?

Answer: `df.isna().sum()` counts missing cells per column; `df.duplicated().sum()` counts rows identical to an earlier row. `len(df)` or `df.shape[0]` gives 1000.

Problem 17.P.3 (Easy, find the bug) This line raises an error. Explain and fix it.

```
import pandas as pd

df = pd.DataFrame({"a": [1, 5, 9], "b": [2, 2, 8]})
try:
    print(df[(df["a"] > 2) and (df["b"] < 5)])
except ValueError as e:
    print("ValueError")
print(df[(df["a"] > 2) & (df["b"] < 5)])
```

Answer: `and` asks Python for one truth value of a whole Series, which is ambiguous, so pandas raises `ValueError`. Use `&` with brackets around each comparison. The fixed line keeps only row 1 (a=5, b=2).

```
ValueError
   a  b
1  5  2
```

Problem 17.P.4 (Easy) A column `age` has values 20, 30, NaN, 40. What is `df["age"].mean()`, and what is the mean after `fillna(0)`?

Answer: pandas skips NaN by default: (20+30+40)/3 = 30. After `fillna(0)` the four values sum to 90, and 90/4 = 22.5. Filling with 0 silently changed the mean, which is why the fill value must be chosen with care.

Problem 17.P.5 (Easy, what does this print?) What is printed?

```
import pandas as pd

a = pd.Series([1, 2, 3], index=["x", "y", "z"])
b = pd.Series([10, 20], index=["y", "z"])
print((a + b).tolist())
print(a.add(b, fill_value=0).tolist())
```

Answer: addition aligns on labels. `x` is missing from `b`, so `x` becomes NaN; y is 2+10 = 12 and z is 3+20 = 23. With `add(..., fill_value=0)` a missing side is treated as 0, so x stays 1.

```
[nan, 12.0, 23.0]
[1.0, 12.0, 23.0]
```

Problem 17.P.6 (Medium) For orders `cust = [a, a, b]` and `amt = [10, 30, 20]`, write one expression that gives each row's amount divided by the customer's total, and compute it by hand.

Answer: `df["amt"] / df.groupby("cust")["amt"].transform("sum")`. Customer a total is 40, so the first two rows are 10/40 = 0.25 and 30/40 = 0.75; customer b is 20/20 = 1.0.

Problem 17.P.7 (Medium, compute by hand) Left table has keys [1, 2, 2, 3]. Right table has keys [2, 2, 3, 4]. How many rows result from an inner join, a left join and an outer join? Verify with code.

```
import pandas as pd

L = pd.DataFrame({"k": [1, 2, 2, 3]})
R = pd.DataFrame({"k": [2, 2, 3, 4]})
print({how: len(L.merge(R, on="k", how=how)) for how in ["inner", "left", "outer"]})
```

Answer: key 2 appears twice on each side, so it produces 2 x 2 = 4 rows. Key 3 gives 1 x 1 = 1. Inner = 4 + 1 = 5. Left adds the unmatched left key 1: 6. Outer also adds the unmatched right key 4: 7. Many-to-many joins multiply.

```
{'inner': 5, 'left': 6, 'outer': 7}
```

Problem 17.P.8 (Medium) Why does `df.pivot(index="city", columns="year", values="sales")` fail when Oslo has two rows for 2025, and what do you use instead?

Answer: `pivot` only rearranges: each (city, year) pair must be unique, otherwise it does not know which value to put in the cell, and raises an error about duplicate entries. `pivot_table(..., aggfunc="sum")` first aggregates duplicates (or use `groupby` first, then `unstack`).

Problem 17.P.9 (Medium, find the bug) The programmer wants to cap negative values at zero in `df`. It does not work in pandas 3.0. Why? Fix it and show it works.

```
import pandas as pd

df = pd.DataFrame({"v": [3, -2, 5, -7]})
df["v"][df["v"] < 0] = 0
print(df["v"].tolist())
df.loc[df["v"] < 0, "v"] = 0
print(df["v"].tolist())
```

Answer: `df["v"]` returns a lazy copy under copy-on-write, so assigning into it never reaches `df`. A single `df.loc[mask, "v"] = 0` writes to `df` directly. (An equivalent is `df["v"] = df["v"].clip(lower=0)`.)

```
[3, -2, 5, -7]
[3, 0, 5, 0]
```

Problem 17.P.10 (Medium, what does this print?) What does this print, and why?

```
import pandas as pd

s = pd.Series([1, 2, 3, 4, 5])
print(s.rolling(3).sum().tolist())
print(s.shift(1).tolist())
print(s.diff().tolist())
print(s.expanding().mean().tolist())
```

Answer: rolling(3).sum() needs a full window of 3, so the first two are NaN, then 1+2+3 = 6, 9, 12. shift(1) moves values down: the first is NaN. diff() is today minus yesterday: NaN then 1 each time. expanding().mean() is the running average: 1, 1.5, 2, 2.5, 3.

```
[nan, nan, 6.0, 9.0, 12.0]
[nan, 1.0, 2.0, 3.0, 4.0]
[nan, 1.0, 1.0, 1.0, 1.0]
[1.0, 1.5, 2.0, 2.5, 3.0]
```

Problem 17.P.11 (Medium) Keep only the most recent row per `id` from a table with columns `id, ts, val`. Write the code, then explain what goes wrong if you skip the sort.

Answer: `df.sort_values("ts").drop_duplicates("id", keep="last")`. Without the sort, "last" means last in file order, which may not be the latest timestamp; the answer would depend on how the file happened to be arranged.

Problem 17.P.12 (Medium, what does this print?) Where do 18, 64 and 65 fall?

```
import pandas as pd

age = pd.Series([18, 64, 65])
print(pd.cut(age, [0, 18, 64, 120]).astype(str).tolist())
print(pd.cut(age, [0, 18, 64, 120], right=False).astype(str).tolist())
```

Answer: by default intervals include the right edge: 18 is in (0, 18], 64 in (18, 64], 65 in (64, 120]. With `right=False` intervals include the left edge: 18 moves to [18, 64), 64 to [64, 120), 65 stays in [64, 120). Always test boundary values.

```
['(0, 18]', '(18, 64]', '(64, 120]']
['[18, 64)', '[64, 120)', '[64, 120)']
```

Problem 17.P.13 (Hard, find the bug) Revenue per tier is overstated after this join. Find why, show the symptom, and fix with a validation that would have caught it.

```
import pandas as pd

orders = pd.DataFrame({"cid": [1, 2, 3], "amt": [100, 200, 300]})
tiers = pd.DataFrame({"cid": [1, 2, 2, 3], "tier": ["g", "s", "s", "b"]})
bad = orders.merge(tiers, on="cid")
print(bad.groupby("tier")["amt"].sum().to_dict())
good = orders.merge(tiers.drop_duplicates("cid"), on="cid", validate="m:1")
print(good.groupby("tier")["amt"].sum().to_dict())
```

Answer: customer 2 is repeated in `tiers`, so their 200 appears twice and silver shows 400 instead of 200. `validate="m:1"` would raise `MergeError` as soon as `tiers` had a duplicate key. Fix by de-duplicating (or fix the source) and keeping the validation.

```
{'b': 300, 'g': 100, 's': 400}
{'b': 300, 'g': 100, 's': 200}
```

Problem 17.P.14 (Hard) Return the two highest-paid employees in each department, and make the handling of ties explicit.

```
import pandas as pd

df = pd.DataFrame({"dept": list("aaabbb"), "emp": ["u1", "u2", "u3", "u4", "u5", "u6"], "pay": [50, 80, 80, 90, 40, 70]})
r = df.sort_values(["dept", "pay", "emp"], ascending=[True, False, True]).groupby("dept").head(2)
print(r[["dept", "emp", "pay"]].to_dict("records"))
```

Answer: sort by department, pay descending and employee as a tie-breaker, then `head(2)` per group. Department a has a tie at 80 (u2 and u3), and the tie-breaker makes the result deterministic. If instead you wanted *all* tied employees, use `rank(method="min")` and keep rank at most 2.

```
[{'dept': 'a', 'emp': 'u2', 'pay': 80}, {'dept': 'a', 'emp': 'u3', 'pay': 80}, {'dept': 'b', 'emp': 'u4', 'pay': 90},
{'dept': 'b', 'emp': 'u6', 'pay': 70}]
```

Problem 17.P.15 (Hard) Sessionise with a 15-minute limit. User u1 has events at 09:00, 09:10, 09:30, 09:35. How many sessions, and what are their sizes?

```
import pandas as pd

ts = pd.to_datetime(["2025-01-01 09:00", "2025-01-01 09:10", "2025-01-01 09:30", "2025-01-01 09:35"])
ev = pd.DataFrame({"user": "u1", "ts": ts})
gap = ev.groupby("user")["ts"].diff()
ev["session"] = (gap.isna() | (gap > pd.Timedelta(minutes=15))).groupby(ev["user"]).cumsum()
print(ev.groupby("session").size().to_dict())
```

Answer: gaps are NaT, 10, 20 and 5 minutes. Only the first event and the 20-minute gap start sessions, so there are two sessions of sizes 2 and 2. The test is "gap greater than the limit", so a gap of exactly 15 minutes stays in the session; decide the boundary rule explicitly.

```
{1: 2, 2: 2}
```

Problem 17.P.16 (Hard) A text column `country` has 5 distinct values over 2 million rows, and `qty` holds integers from 1 to 40. Estimate memory and give the dtypes you choose.

Answer: `qty` as `int64` costs 8 bytes x 2,000,000 = 16 MB; as `uint8` (0 to 255) it is 2 MB. `country` as category costs about one byte per row (about 2 MB, five labels are negligible), against a much larger cost for 2 million separate text values. Verify with `memory_usage(deep=True)` and compare results before and after.

Problem 17.P.17 (Hard, find the bug) The group totals below do not add to the grand total. What is wrong, and what is the fix?

```
import pandas as pd

df = pd.DataFrame({"region": ["n", "s", None, "n"], "rev": [10, 20, 30, 40]})
print(df.groupby("region")["rev"].sum().sum(), df["rev"].sum())
print(df.groupby("region", dropna=False)["rev"].sum().sum())
```

Answer: the row with a missing region is dropped from the groupby by default, so the totals add to 70, not 100. Use `dropna=False` (a NaN group appears), or fill the key with a label such as "unknown" first, and then reconcile the totals.

```
70 100
100
```

Problem 17.P.18 (Hard) Translate to pandas: `SELECT dept, AVG(pay) AS avg_pay FROM emp GROUP BY dept HAVING COUNT(*) >= 2 ORDER BY avg_pay DESC`.

Answer:

```
(emp.groupby("dept").agg(avg_pay=("pay", "mean"), n=("pay", "size"))
    .query("n >= 2")
    .sort_values("avg_pay", ascending=False)[["avg_pay"]])
```

HAVING becomes a filter on the aggregated table, so it comes after `agg`; the extra column `n` is needed to apply it and is dropped at the end.

Problem 17.P.19 (Hard) Two payment files should have equal amounts per invoice. Floats differ in the 10th decimal. How do you list true mismatches?

Answer: outer-merge on invoice with `indicator=True` and `validate="1:1"`, take rows where both sides exist, and keep those where `(a - b).abs() > 0.005` (half a cent) or compare after `round(2)`. Never use `==` on floats for money. See Recipe 6 for the full code.

Problem 17.P.20 (Expert, what does this print?) Predict the three lines, then explain the second.

```
import pandas as pd

df = pd.DataFrame({"a": [1, 2, 3]})
v = df["a"]
w = df.copy(deep=False)
v.iloc[0] = 10
w.loc[1, "a"] = 20
print(df["a"].tolist())
print(v.tolist(), w["a"].tolist())
s = pd.Series(["x", None, "y"])
print(s.dtype, s.isna().sum(), s.str.upper().tolist())
```

Answer: with copy-on-write, neither write reaches `df`, so it stays [1, 2, 3]. `v` is [10, 2, 3] and `w` is [1, 20, 3]: each object copied on its own first write. The Series is the default `str` dtype with one missing value, and upper-casing keeps the missing value as NaN.

```
[1, 2, 3]
[10, 2, 3] [1, 20, 3]
str 1 ['X', nan, 'Y']
```

Problem 17.P.21 (Expert, interview) Explain why `df.apply(f, axis=1)` is slow, and give three ways to avoid it with a concrete example for each.

Answer: `apply(axis=1)` builds a Series object for every row and calls a Python function once per row, so the work is dominated by interpreter overhead and object creation, not arithmetic. Alternatives: (1) vectorised column arithmetic, `df["a"] * df["b"]`; (2) `np.where` / `np.select` for if/else logic, `np.where(df.x > 0, "pos", "neg")`; (3) `map` with a dict for recoding, or the `.str` / `.dt` accessors for text and dates. If real per-row logic remains, a list comprehension over `zip` of columns, or `itertuples()`, is much cheaper than `apply`.

Problem 17.P.22 (Expert) A cohort table shows retention above 100 percent for one cell. Name two likely causes in the code of Recipe 2 and the fix for each.

Answer: (1) Counting rows (`size`) instead of distinct customers (`nunique`): a customer with several orders in the same month counts several times. Fix: `nunique`. (2) Dividing by the wrong denominator, for example the cohort size taken from a different filter, or computing the cohort from a table that was already filtered to repeat buyers. Fix: derive cohort size from the age-0 column of the same pivot, as the recipe does with `counts.div(counts[0], axis=0)`. Sanity check: age 0 must be exactly 100 percent and no cell can exceed it.

Problem 17.P.23 (Expert, design) You must deduplicate and sum a 40 GB CSV of events on a 16 GB laptop. Sketch the approach and its limits.

Answer: stream with `read_csv(chunksize=..., usecols=..., dtype=...)`, aggregate each chunk, and combine partial results, as in the "bigger than memory" example; this is exact for sums and counts. Exact global deduplication needs all keys at once: if the set of distinct keys fits in memory, keep a set of seen keys across chunks; if not, partition the data by a hash of the key into several files first, then deduplicate each partition separately. Better still, convert to Parquet once (not run here) and use a tool built for it, such as DuckDB or Polars lazy mode. State clearly that pandas holds everything it processes in memory.

Problem 17.P.24 (Expert, find the bug) A monthly report uses `df.resample("M").sum()` and `df.applymap(str.strip)` and fails after upgrading to pandas 3.0. What changed?

Answer: the month alias `"M"` was replaced by `"ME"` (month end) and the old `applymap` was removed in favour of `DataFrame.map`. Fix: `df.resample("ME").sum()` and `df.map(str.strip)` (on text-only columns). General lesson: pin your pandas version in `requirements.txt`, run the test suite after upgrades, and read the release notes.

Prompt pack

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical pandas tutor. I am a working professional moving into data roles and I already know basic Python. Quiz me on pandas one question at a time, starting easy and getting harder: loc vs iloc, Boolean masks, missing data, groupby agg/transform/filter, merge with validate, pivot/melt, resample/rolling, dtypes and memory, and pandas 3.0 copy-on-write. After each answer tell me whether it is right, explain the idea in plain words, show a tiny runnable example, and then ask a harder follow-up. Keep a running score and finish with the three topics I should revise.
```

✨ AI prompt: copy and paste into any AI

```
You are a senior data engineer doing a code review. Below is my pandas code (pandas 3.0, NumPy 2). Review it for: wrong results (chained assignment, row-multiplying joins, NaN group keys, float equality, ambiguous dates), slow patterns (apply axis=1, iterrows, growing a DataFrame in a loop), memory waste (object or int64 where smaller dtypes fit), and readability (method chaining, named aggregation). For each issue give the line, why it is a problem, and a corrected snippet. Then list the checks (row counts, totals, uniqueness) I should add to prove the output is right. My code: [PASTE CODE HERE]
```

✨ AI prompt: copy and paste into any AI

```
Act as an interviewer for a data analyst or data scientist role. Run a 30-minute pandas mock interview. Ask me to predict the output of short snippets, find the bug in snippets, translate SQL (GROUP BY, HAVING, window functions, LEFT JOIN) into pandas, and design a pipeline for a file too large for memory. Ask one question at a time, wait for my answer, then give a score out of 10, the model answer, and the common wrong answer candidates give. At the end summarise my strengths and a 7-day plan to improve.
```

CHAPTER 18

SCIPY, STATSMODELS AND THE SCIENTIFIC STACK

Library Handbook: distributions, tests, optimisation, interpolation, signals and sparse matrices.

It is Monday morning and Maya, an analyst at a food-delivery company, has four questions on her desk. The delivery-time column is skewed, so which distribution describes it, and what is the chance that an order takes more than two hours? The new route-planning app was tried on half the drivers: is it really faster, or is that luck? The support-call volume after a bad-weather day decays over time: how fast, with what error bars? And finally, 40 driver-hours must be shared between two zones to earn the most profit. None of this is machine learning. It is the older, steadier craft of scientific computing, and almost every data domain (finance, health, engineering, marketing, operations) leans on the same few libraries to do it.

This chapter teaches those libraries point by point: **SciPy** (the scientific toolbox on top of NumPy) and **statsmodels** (statistical models with full inference tables), plus a quick matplotlib reference for showing results. By the end you will be able to fit and test distributions, run and interpret hypothesis tests, build confidence intervals by formula and by bootstrap, fit curves with uncertainty, solve small optimisation problems, integrate and interpolate, work with sparse matrices, find a frequency in a noisy signal, search neighbours quickly, and cluster points. Five recipes tie it together: fit a distribution, A/B test, curve fitting with uncertainty, a small optimisation, and resampling.

HONESTY NOTE

Everything marked with an output was executed on this study machine with Python 3, NumPy 2.x, SciPy 1.18, pandas 3.0 and matplotlib 3.11. **statsmodels is not installed here**, so every statsmodels block is labelled *not run here* and is paired with an equivalent computed in NumPy and SciPy that did run. Timings are measured on this machine and vary on yours. The full guide to charts is the Data Analytics document, Chapter 8; here you only get the matplotlib quick reference. For the ideas behind tests and confidence intervals see the Statistics document.

18.1 What SciPy and statsmodels Are, and the Mental Model

What each library is

NumPy gives you the array. **SciPy** is a collection of well-tested numerical routines that work on NumPy arrays: probability distributions and tests, optimisers, integrators, interpolators, linear algebra, sparse matrices, signal processing, Fourier transforms, spatial searches and clustering. Most of it wraps battle-tested C, C++ and Fortran code (LAPACK, ODEPACK, QHull and others), so you get decades of numerical care with a Python call. **statsmodels** is a separate package focused on statistical modelling: regression (OLS, GLM, logistic), time series (ARIMA, SARIMAX), and above all *inference output*: standard errors, p-values, confidence intervals and diagnostic tests in one summary table.

Use it when	Do NOT use it when
You need a distribution, a classical test, or a confidence interval	You want a predictive model with cross-validation and pipelines: use scikit-learn
You need to fit a custom formula to measured points (`curve_fit`)	Your data has millions of rows and you need group-by logic: use pandas first, then call SciPy on the arrays
You need a small constrained optimisation (a few hundred variables)	You are training a neural network: use a deep-learning framework with automatic differentiation
You need a regression with standard errors and p-values (statsmodels)	You only need predictions from a regression: `numpy.linalg.lstsq` or scikit-learn is lighter
You need fast nearest-neighbour search on low-dimensional points	You have 768-dimensional embeddings at scale: use an approximate nearest-neighbour index

Install, version check, import conventions

Install with `python3 -m pip install scipy statsmodels` (inside a virtual environment, see the Python document, Chapter 10). SciPy submodules are not imported automatically by `import scipy`, so import the pieces you need. The conventions below are what you will see in almost all code.

```
import numpy as np
import scipy
import pandas as pd
import matplotlib
from scipy import stats, optimize, integrate, interpolate, linalg, signal, spatial, fft
from scipy import sparse
from scipy.cluster import vq, hierarchy

print("numpy ", np.__version__.split(".")[0] + ".x")
print("scipy ", scipy.__version__)
print("pandas", pd.__version__.split(".")[0] + ".x")
print("matplotlib", matplotlib.__version__)
print(stats.norm.cdf(0))
```

```
numpy  2.x
scipy  1.18.1
pandas 3.x
matplotlib 3.11.2
0.5
```

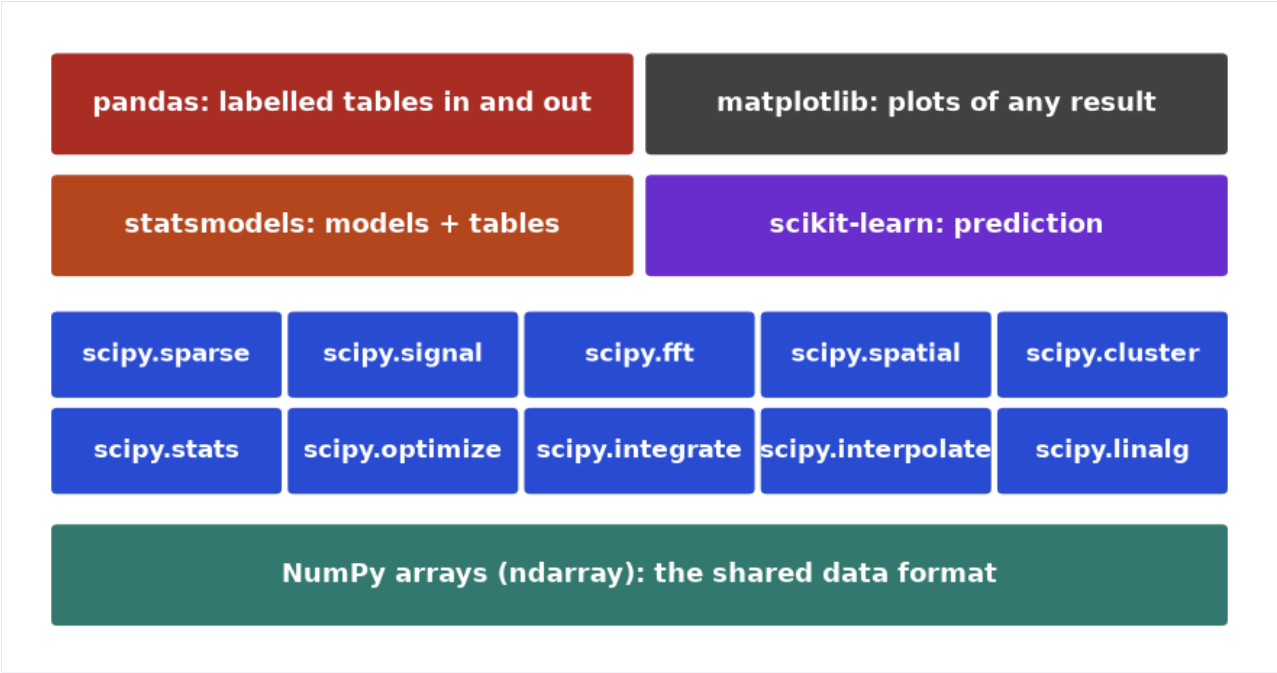
1. The `from scipy import ...` lines pull in the submodules. `import scipy` alone would not make `scipy.stats` usable in older versions, so always import the submodule you use.
2. Printing only the major version of NumPy and pandas keeps the output stable; the full version string is in `np.__version__`.
3. `stats.norm.cdf(0)` is a first smoke test: half the standard normal lies below zero, so it prints 0.5.

KEY IDEA

Conventions: `np` for NumPy, `pd` for pandas, `plt` for `matplotlib.pyplot`, `sm` for `statsmodels.api`, `smf` for `statsmodels.formula.api`, and import SciPy as `from scipy import stats, optimize`. A statsmodels block (not run here) always starts `import statsmodels.api as sm`.

The mental model

Picture a stack. NumPy arrays are the floor: every SciPy function takes arrays in and returns arrays or small result objects out. Each SciPy submodule is a self-contained tool drawer. statsmodels, scikit-learn, pandas and matplotlib stand on the same floor, so data moves between them without copying or converting. Learning this chapter is mostly learning which drawer holds which tool and what shape of object each tool returns.



The scientific stack: one array format shared by every layer.

How to read it: start at the bottom. Whatever the top layer is (a pandas column, a scikit-learn feature matrix), you can hand its values to a SciPy routine because they are NumPy arrays underneath. Results come back the same way, which is why a pipeline can mix all of these libraries in ten lines.

Three habits make SciPy easy. First, *result objects*: modern SciPy functions return named results (`res.statistic` , `res.pvalue` , `res.x` , `res.success`) rather than anonymous tuples; read the attribute names. Second, *random state*: always pass a seeded generator, `rng = np.random.default_rng(0)` , and give it to `random_state=rng` or `rvs(..., random_state=rng)` . Third, *vectorise*: pass whole arrays, never loop over scalars.

18.2 scipy.stats I: Distributions

A **probability distribution** is a recipe that says how likely each value is. SciPy gives you about a hundred ready-made ones. Every continuous distribution is an object with the same methods, so once you learn `norm` you know `lognorm` , `gamma` , `expon` and the rest. A *discrete* distribution (`binom` , `poisson`) uses `pmf` (probability of exactly this value) instead of `pdf` (density).

Name	What it does in plain English	Minimal example
`norm`	Bell curve with mean `loc` and spread `scale`	`stats.norm(loc=0, scale=1)`
`expon`	Waiting time between random events	`stats.expon(scale=5)`
`lognorm`	Positive, right-skewed values (durations, prices)	`stats.lognorm(s=0.5, scale=3)`
`gamma`	Positive skewed, sums of waiting times	`stats.gamma(a=2, scale=1.5)`
`beta`	Values between 0 and 1 (rates, proportions)	`stats.beta(2, 5)`
`uniform`	All values in an interval equally likely	`stats.uniform(loc=2, scale=3)`
`t` , `chi2`	Reference curves used inside tests	`stats.t(df=9)`
`binom`	Number of successes in n yes/no trials	`stats.binom(n=10, p=0.3)`
`poisson`	Count of events in a fixed window	`stats.poisson(mu=4)`

Name	What it does in plain English	Minimal example
`.pdf(x)` / `.pmf(k)`	Density (or exact probability) at a point	`stats.norm.pdf(0)`
`.cdf(x)`	Probability of being at or below x	`stats.norm.cdf(1.96)`
`.sf(x)`	Survival function, 1 minus cdf, accurate in the tail	`stats.norm.sf(5)`
`.ppf(q)`	Inverse cdf: the value below which q of the mass lies	`stats.norm.ppf(0.975)`
`.rvs(size, random_state)`	Draw random samples	`d.rvs(100, random_state=rng)`
`.interval(conf)`	Central interval holding a fraction of the mass	`stats.norm.interval(0.95)`
`.mean() .std() .var()`	Moments of a frozen distribution	`stats.expon(scale=5).mean()`
`.logpdf(x)`	Log density, stable for likelihoods	`stats.norm.logpdf(x).sum()`
`.fit(data)`	Maximum-likelihood parameters from data	`stats.lognorm.fit(x, floc=0)`
"Freezing"	Fix the parameters once, reuse the object	`d = stats.gamma(2, scale=3)`
`stats.describe(x)`	Count, min/max, mean, variance, skewness, kurtosis	`stats.describe(x).skewness`
`stats.skew / kurtosis`	Asymmetry and tail weight of a sample	`stats.skew(x)`
`stats.zscore(x)`	Standardise: (x minus mean) / std	`stats.zscore(x)`
`stats.kstest(x, cdf)`	Does the sample match a given distribution?	`stats.kstest(x, "norm", args=(m, s))`
`stats.probplot(x)`	Data for a Q-Q plot (normality check)	`stats.probplot(x, dist="norm")`
`stats.scoreatpercentile / percentileofscore`	Percentile of a value or value of a percentile	`stats.percentileofscore(x, 3)`
`stats.trim_mean(x, 0.1)`	Mean after cutting 10% off each end	`stats.trim_mean(x, 0.1)`
`stats.iqr(x)`	Interquartile range, a robust spread	`stats.iqr(x)`
`stats.mode(x)`	Most frequent value	`stats.mode([1, 2, 2]).mode`
`stats.sem(x)`	Standard error of the mean	`stats.sem(x)`
`stats.qmc.Sobol`	Low-discrepancy points for simulation	`stats.qmc.Sobol(2, seed=0)`

Worked example 1: the four verbs on a normal curve

Problem 18.2.1 Test scores follow a normal distribution with mean 70 and standard deviation 10. Find the density at 70, the share scoring below 85, the share above 90, the score that 90% of people stay under, and 5 random scores.

```
import numpy as np
from scipy import stats

scores = stats.norm(loc=70, scale=10)
print("pdf at 70      :", round(scores.pdf(70), 4))
print("P(score <= 85) :", round(scores.cdf(85), 4))
print("P(score > 90)  :", round(scores.sf(90), 4))
print("90th percentile:", round(scores.ppf(0.90), 2))
rng = np.random.default_rng(0)
print("5 draws        :", scores.rvs(5, random_state=rng).round(1))
```

```
pdf at 70      : 0.0399
P(score <= 85) : 0.9332
P(score > 90)  : 0.0228
90th percentile: 82.82
5 draws        : [71.3 68.7 76.4 71.  64.6]
```

1. `stats.norm(loc=70, scale=10)` freezes the parameters into an object called `scores`. Every later call reuses them.
2. `pdf(70)` is the height of the curve at its peak: 1 / (10 times the square root of 2π), about 0.0399. A density is *not* a probability, which is why it can exceed 1 for narrow curves.
3. `cdf(85)` is the area to the left of 85, the chance of scoring 85 or less. That is 1.5 standard deviations above the mean, so about 0.93.
4. `sf(90)` is "survival", the area to the right of 90, which is two standard deviations up: roughly 2.3%.
5. `ppf(0.90)` goes the other way (percent point function): from a probability to a value. About 1.28 standard deviations above the mean gives 82.8.
6. `rvs(5, random_state=rng)` draws five scores. The seeded generator makes the draws repeatable.

INTUITION

Remember the pair: `cdf` turns a *value* into a *probability*, `ppf` turns a *probability* into a *value*. They are inverses: `ppf(cdf(x))` returns x. A confidence interval is just two `ppf` calls.

Worked example 2: discrete distributions and the tail

Problem 18.2.2 A call centre gets on average 4 complaints per hour (Poisson). What is the chance of exactly 7 in an hour, of 10 or more, and what is the smallest staffing level that covers 99% of hours?

```
from scipy import stats

calls = stats.poisson(mu=4)
print("P(exactly 7)  :", round(calls.pmf(7), 4))
print("P(10 or more) :", round(calls.sf(9), 5))
print("99% covered by:", int(calls.ppf(0.99)), "calls")
print("mean, std     :", calls.mean(), round(calls.std(), 3))
```

```
P(exactly 7)  : 0.0595
P(10 or more) : 0.00813
99% covered by: 9 calls
mean, std     : 4.0 2.0
```

1. A Poisson distribution needs one parameter, `mu`, its mean count.
2. `pmf(7)` is the probability of exactly 7, a real probability because the variable is discrete.
3. "10 or more" is "more than 9", so we use `sf(9)`. A common slip is `sf(10)`, which would drop the case of exactly 10 (see the pitfalls section).

4. `ppf(0.99)` returns the smallest integer k with `cdf(k)` at least 0.99.
5. For a Poisson the mean equals the variance, so the variance is also 4, so the standard deviation is the square root of 4, which is 2.0.

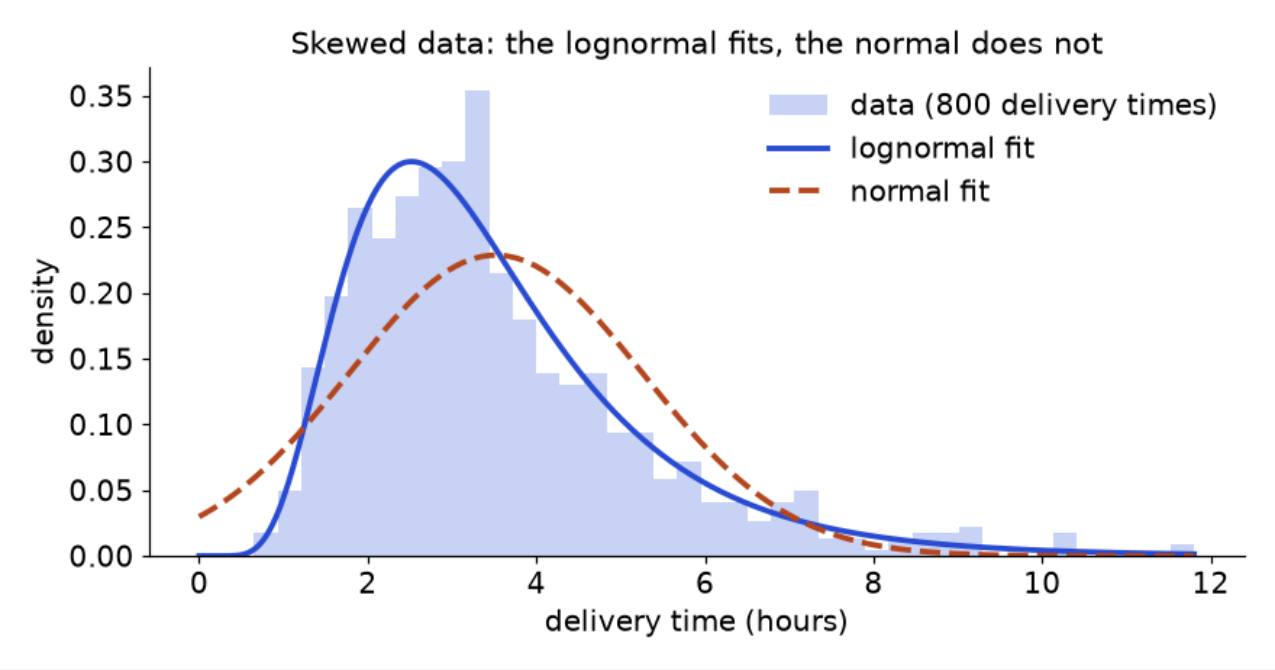
Worked example 3: fit a distribution to real-looking data (Recipe 1)

Problem 18.2.3 Maya has 800 delivery times (hours). Fit a lognormal and a normal, compare them, and compute the chance an order takes more than 6 hours.

```
import numpy as np
from scipy import stats

rng = np.random.default_rng(7)
data = rng.lognormal(mean=1.2, sigma=0.5, size=800)

s, loc, scale = stats.lognorm.fit(data, floc=0)
mu, sd = stats.norm.fit(data)
print("lognorm shape s=%.3f scale=%.3f" % (s, scale))
print("normal  mu=%.3f sd=%.3f" % (mu, sd))

ll_log = stats.lognorm.logpdf(data, s, loc, scale).sum()
ll_nor = stats.norm.logpdf(data, mu, sd).sum()
print("log-likelihood lognorm: %.1f  normal: %.1f" % (ll_log, ll_nor))
print("AIC lognorm: %.1f  normal: %.1f" % (2 * 2 - 2 * ll_log, 2 * 2 - 2 * ll_nor))

print("KS p-value lognorm: %.3f" % stats.kstest(data, "lognorm", args=(s, loc, scale)).pvalue)
print("P(time > 6h):", round(stats.lognorm.sf(6, s, loc, scale), 4), "empirical:", (data > 6).mean())
```

```
lognorm shape s=0.472 scale=3.149
normal  mu=3.519 sd=1.744
log-likelihood lognorm: -1452.4  normal: -1580.0
AIC lognorm: 2908.9  normal: 3163.9
KS p-value lognorm: 0.589
P(time > 6h): 0.086 empirical: 0.08875
```

1. We simulate data from a true lognormal so we know the right answer: shape 0.5 and scale $e^{1.2}$, about 3.32. The fit lands close to both (sampling noise explains the gap).
2. `lognorm.fit(data, floc=0)` estimates shape, location and scale. Fixing `floc=0` says "values start at zero", which is physically right for durations and makes the fit stable and faster.
3. `norm.fit` returns the sample mean and standard deviation.
4. The log-likelihood adds `logpdf` over all points; larger is better. The AIC (2 times parameters minus 2 times log-likelihood, lower is better) penalises extra parameters. Both models have two free parameters here, so the comparison is fair.
5. `kstest` compares the sample with the fitted cdf. Caution: because the parameters were estimated from the same data, the p-value is optimistic; treat it as a rough check, not proof.
6. `sf(6, ...)` gives the tail probability, and we compare it with the plain empirical share as a sanity check.



Delivery times with a lognormal fit (solid) and a normal fit (dashed).

How to read it: the bars are the data. The solid curve follows the sharp rise and long right tail. The dashed normal curve is symmetric, so it puts probability on negative hours and underestimates the peak. A good fit means the curve rides the tops of the bars everywhere, not just near the middle.

COMMON MISTAKE

Reading `pdf` as a probability. `stats.norm(0, 0.1).pdf(0)` is about 3.99, which is fine: density can exceed 1. Probabilities come from *areas*: use `cdf(b) - cdf(a)`.

18.3 scipy.stats II: Tests, Intervals, Bootstrap, Correlation

A **hypothesis test** asks "could this difference be just luck?". It starts from a boring assumption (the *null hypothesis*, for example "both groups have the same mean"), computes a *test statistic*, and reports a **p-value**: the probability of seeing a result at least this extreme if the boring assumption were true. A small p-value (below a threshold you chose in advance, often 0.05) is evidence against the null. It is *not* the probability that the null is true, and it says nothing about how large or important the effect is.

Name	What it does in plain English	Minimal example
`ttest_1samp`	Is the mean different from a fixed number?	`stats.ttest_1samp(x, 50)`
`ttest_ind`	Do two independent groups have the same mean? (`equal_var=False` is Welch)	`stats.ttest_ind(a, b, equal_var=False)`
`ttest_rel`	Paired before/after comparison	`stats.ttest_rel(before, after)`
`mannwhitneyu`	Rank-based two-group test, no normality needed	`stats.mannwhitneyu(a, b)`
`wilcoxon`	Rank-based paired test	`stats.wilcoxon(before - after)`
`shapiro`	Is the sample plausibly normal?	`stats.shapiro(x)`
`normaltest`	Normality test from skew and kurtosis	`stats.normaltest(x)`
`levene`	Do groups have equal variance?	`stats.levene(a, b)`
`f_oneway`	One-way ANOVA across 3+ groups	`stats.f_oneway(a, b, c)`

Name	What it does in plain English	Minimal example
`kruskal`	Rank-based ANOVA	`stats.kruskal(a, b, c)`
`chi2_contingency`	Are two categorical variables related?	`stats.chi2_contingency(table)`
`chisquare`	Do counts match expected counts?	`stats.chisquare(obs, exp)`
`fisher_exact`	Exact test for a 2x2 table with small counts	`stats.fisher_exact([[3, 1], [1, 3]])`
`binomtest`	Is an observed rate different from a target?	`stats.binomtest(k, n, 0.5)`
`ks_2samp`	Do two samples share a distribution?	`stats.ks_2samp(a, b)`
`pearsonr`	Linear correlation, with p-value	`stats.pearsonr(x, y)`
`spearmanr`	Rank correlation, handles monotone curves and outliers	`stats.spearmanr(x, y)`
`kendalltau`	Rank correlation for small samples and ties	`stats.kendalltau(x, y)`
`linregress`	Straight-line fit with slope p-value	`stats.linregress(x, y)`
`bootstrap`	Confidence interval by resampling	`stats.bootstrap((x,), np.mean)`
`permutation_test`	Test by shuffling labels	`stats.permutation_test((a, b), stat)`
`t.interval`	Confidence interval for a mean	`stats.t.interval(0.95, n-1, m, se)`
`stats.sem`	Standard error of the mean	`stats.sem(x)`
`false_discovery_control`	Adjust many p-values (Benjamini-Hochberg)	`stats.false_discovery_control(p)`
`power_divergence`	Family behind chi-square and G-test	`stats.power_divergence(o, e, lambda_=0)`

KEY IDEA

Every test returns a result object with `.statistic` and `.pvalue`, and many add `.confidence_interval()`. Choose by the data shape: two numeric groups (t-test or Mann-Whitney), paired numeric (paired t or Wilcoxon), two categories (chi-square or Fisher), many groups (ANOVA or Kruskal), two numeric columns (Pearson or Spearman).

Worked example 1: a one-sample and a two-sample t-test (Recipe 2, A/B test)

Problem 18.3.1 Control drivers average about 38 minutes per delivery; drivers using the new route app are simulated at about 35. With 120 drivers in each group, is the app faster? Report the difference with a 95% interval.

```
import numpy as np
from scipy import stats

rng = np.random.default_rng(42)
control = rng.normal(38, 8, 120)
variant = rng.normal(35, 9, 120)

res = stats.ttest_ind(variant, control, equal_var=False)
print("t = %.3f  p = %.4f" % (res.statistic, res.pvalue))
diff = variant.mean() - control.mean()
ci = res.confidence_interval(0.95)
print("mean diff = %.2f min, 95%% CI (%.2f, %.2f)" % (diff, ci.low, ci.high))

pooled = np.sqrt((variant.var(ddof=1) + control.var(ddof=1)) / 2)
print("Cohen d = %.2f" % (diff / pooled))
```

```
t = -2.802  p = 0.0055
mean diff = -2.82 min, 95% CI (-4.80, -0.84)
Cohen d = -0.36
```

1. Two normal samples with different spreads imitate an A/B test. Seeding with 42 makes the numbers repeatable.
2. `ttest_ind(variant, control, equal_var=False)` is Welch's t-test, the safe default because it does not assume equal variances. The first argument minus the second is the tested difference.
3. The statistic is negative because the variant mean is lower; the p-value is two-sided (the app could be slower or faster).
4. `res.confidence_interval(0.95)` gives the range of plausible true differences. Report this, not just p: it shows size and uncertainty.
5. Cohen's d divides the difference by a pooled standard deviation. About 0.2 is small, 0.5 medium, 0.8 large (rules of thumb).

Worked example 2: when the data is not normal

Problem 18.3.2 Customer wait times are skewed. Check normality with Shapiro-Wilk, then compare two groups with Mann-Whitney U.

```
import numpy as np
from scipy import stats

rng = np.random.default_rng(1)
a = rng.lognormal(1.0, 0.6, 80)
b = rng.lognormal(1.3, 0.6, 80)

print("shapiro a: p = %.5f" % stats.shapiro(a).pvalue)
print("shapiro log(a): p = %.3f" % stats.shapiro(np.log(a)).pvalue)
mw = stats.mannwhitneyu(a, b, alternative="two-sided")
print("Mann-Whitney U = %.0f  p = %.5f" % (mw.statistic, mw.pvalue))
auc = mw.statistic / (len(a) * len(b))
print("P(random a beats random b) = %.3f" % auc)
```

```
shapiro a: p = 0.00000
shapiro log(a): p = 0.532
Mann-Whitney U = 2292  p = 0.00196
P(random a beats random b) = 0.358
```

1. Two lognormal samples whose log-means differ (1.0 against 1.3).

2. Shapiro-Wilk tests the null "the data is normal". A tiny p-value rejects normality for raw `a`; the log of a lognormal is normal, so the second p-value is large (we cannot reject).
3. `mannwhitneyu` compares ranks, so skew and outliers matter far less. `alternative="two-sided"` is explicit.
4. The statistic U divided by the number of pairs is the probability that a random value from `a` beats a random value from `b`, a readable effect size (0.5 means no difference).

Worked example 3: categorical data with chi-square

Problem 18.3.3 Of 1000 visitors, 500 saw page A and 500 saw page B. A converted 60, B converted 85. Is the conversion rate different?

```
import numpy as np
from scipy import stats

table = np.array([[60, 440],
                  [85, 415]])
chi2, p, dof, expected = stats.chi2_contingency(table, correction=False)
print("chi2 = %.3f  dof = %d  p = %.4f" % (chi2, dof, p))
print("expected counts:\n", expected)
print("fisher exact p = %.4f" % stats.fisher_exact(table).pvalue)
print("rates:", table[:, 0] / table.sum(axis=1))
```

```
chi2 = 5.041  dof = 1  p = 0.0247
expected counts:
 [[ 72.5 427.5]
 [ 72.5 427.5]]
fisher exact p = 0.0309
rates: [0.12 0.17]
```

1. The table has one row per page and columns "converted" and "not converted". Always put counts in, never percentages.
2. `chi2_contingency` computes the counts we would expect if page and conversion were unrelated (row total times column total divided by grand total), then measures the gap. `correction=False` switches off Yates' continuity correction so the number matches the textbook formula.
3. The returned `expected` table must have all cells of at least about 5 for the chi-square approximation to be trusted; here the smallest is 72.5.
4. Fisher's exact test needs no approximation; it reaches the same conclusion (a p-value a little higher).
5. The last line prints the raw rates (0.12 and 0.17) so you never report p without the effect.

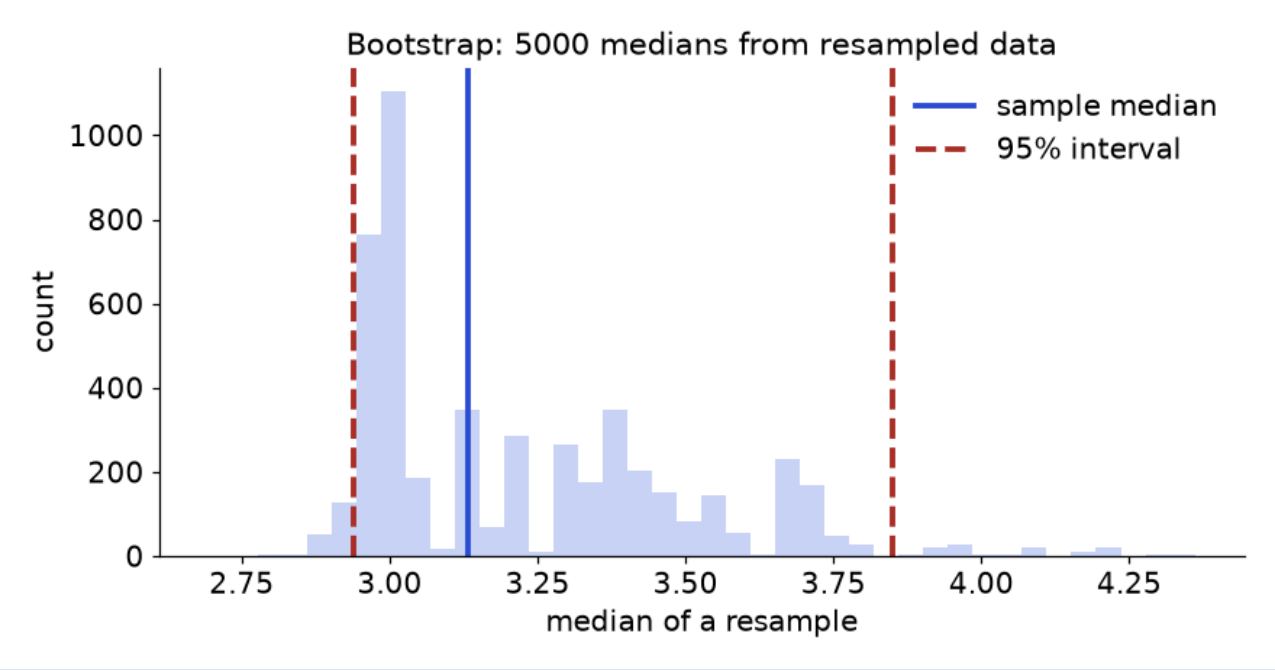
Worked example 4: confidence intervals and the bootstrap (Recipe 5, resampling)

Problem 18.3.4 A small sample of 60 skewed delivery times. Get a 95% interval for the mean by formula, and for the *median* by bootstrap (no formula exists for it).

```
import numpy as np
from scipy import stats

rng = np.random.default_rng(3)
data = rng.lognormal(1.2, 0.5, 60)

m, se = data.mean(), stats.sem(data)
lo, hi = stats.t.interval(0.95, df=len(data) - 1, loc=m, scale=se)
print("mean %.3f, t interval (%.3f, %.3f)" % (m, lo, hi))

res = stats.bootstrap((data,), np.median, n_resamples=5000,
                      confidence_level=0.95, method="percentile", random_state=1)
ci = res.confidence_interval
print("median %.3f, bootstrap interval (%.3f, %.3f)" % (np.median(data), ci.low, ci.high))
print("bootstrap std error: %.3f" % res.standard_error)
```

```
mean 3.819, t interval (3.183, 4.455)
median 3.132, bootstrap interval (2.939, 3.851)
bootstrap std error: 0.281
```

1. Formula route: the 95% interval of a mean is the mean plus or minus a t quantile times the standard error. `stats.t.interval` does that in one call, using `df = n - 1`.
2. `stats.bootstrap` takes the data *as a tuple of samples* (note `(data,)`), a statistic function, and the number of resamples. It resamples with replacement, recomputes the median each time, and reads the interval from percentiles of those medians.
3. `method="percentile"` is the simplest; the default `"BCa"` corrects bias and skew and is usually preferred for real work.
4. `random_state=1` makes it repeatable, and `res.standard_error` is the spread of the resampled medians.



The bootstrap distribution of the median with its 95% interval.

How to read it: each bar counts how often a resample gave that median. The dashed lines cut off the outer 2.5% on each side. A narrow pile means the median is well pinned down; a wide one means more data is needed.

Worked example 5: correlation and many tests at once

Problem 18.3.5 Compare Pearson and Spearman on a straight relationship with one wild reading, then correct 6 p-values for multiple testing.

```
import numpy as np
from scipy import stats

rng = np.random.default_rng(5)
x = np.linspace(1, 10, 40)
y = x + rng.normal(0, 0.5, 40)
y[0] = 30                                # one broken sensor reading
print("pearson  r = %.3f" % stats.pearsonr(x, y).statistic)
print("spearman r = %.3f" % stats.spearmanr(x, y).statistic)

p = np.array([0.001, 0.008, 0.039, 0.041, 0.20, 0.74])
adj = stats.false_discovery_control(p)
print("adjusted:", adj.round(3))
print("significant at 5%:", adj < 0.05)
```

```
pearson  r = 0.290
spearman r = 0.843
adjusted: [0.006 0.024 0.062 0.062 0.24  0.74 ]
significant at 5%: [ True  True False False False False]
```

1. The relationship is a clean rising line, except that the first reading was replaced by a broken value of 30.
2. Pearson uses the actual values, so one extreme point drags it far down.
3. Spearman first converts values to ranks, so it captures any rising trend and shrugs off the single bad value.
4. `false_discovery_control` applies the Benjamini-Hochberg procedure. Testing 6 hypotheses at 5% each would give a high chance of at least one false alarm, so the adjusted values are larger; only those under 0.05 count.

COMMON MISTAKE

Testing many things and reporting only the p below 0.05. With 20 independent tests at 5%, the chance of at least one false positive is about 64%. Decide the tests in advance, or adjust with `false_discovery_control`.

18.4 scipy.optimize: Minimise, Fit, Solve

Optimisation means "find the inputs that make some number as small as possible". Training a model, fitting a curve, planning a route and pricing a product are all optimisation. `scipy.optimize` offers one function per family of problems. Pick by asking: is it a smooth function of a few variables (`minimize`), a model to fit to points (`curve_fit`), an equation to solve (`root`, `brentq`), or a straight-line objective with straight-line limits (`linprog`)?

Name	What it does in plain English	Minimal example
`minimize`	General minimiser; choose a `method`	`optimize.minimize(f, x0)`
`minimize(method="BFGS")`	Smooth, unconstrained, uses gradients	`minimize(f, x0, method="BFGS")`
`minimize(method="L-BFGS-B")`	Smooth with simple bounds, many variables	`minimize(f, x0, bounds=[(0, 5)] * 3)`
`minimize(method="SLSQP")`	Smooth with equality or inequality constraints	`minimize(f, x0, constraints=[c])`
`minimize(method="Nelder-Mead")`	No gradients, noisy or non-smooth	`minimize(f, x0, method="Nelder-Mead")`
`minimize_scalar`	One variable, bracket or bounds	`optimize.minimize_scalar(g, bounds=(0, 4), method="bounded")`
`curve_fit`	Least-squares fit of a Python function to points	`optimize.curve_fit(f, x, y, p0=[1, 1])`
`least_squares`	Robust or bounded nonlinear least squares	`optimize.least_squares(resid, x0, loss="soft_l1")`
`root`	Solve a system f(x) = 0	`optimize.root(F, [1, 1])`
`root_scalar`	Solve one equation in one variable	`optimize.root_scalar(g, bracket=[0, 3])`
`brentq`	Fast bracketed root, needs a sign change	`optimize.brentq(g, 0, 3)`
`fsolve`	Older system solver, still common	`optimize.fsolve(F, [1, 1])`

Name	What it does in plain English	Minimal example
`linprog`	Linear objective with linear limits (HiGHS solver)	`optimize.linprog(c, A_ub=A, b_ub=b)`
`milp`	Linear problem where some variables are integers	`optimize.milp(c, integrality=1, ...)`
`linear_sum_assignment`	Best one-to-one matching from a cost matrix	`optimize.linear_sum_assignment(cost)`
`nnls`	Least squares with non-negative coefficients	`optimize.nnls(A, b)`
`lsq_linear`	Linear least squares with bounds	`optimize.lsq_linear(A, b, bounds=(0, 1))`
`differential_evolution`	Global search for rugged functions, with bounds	`optimize.differential_evolution(f, bounds)`
`basinhopping`	Global search by random restarts	`optimize.basinhopping(f, x0)`
`dual_annealing`	Global search by simulated annealing	`optimize.dual_annealing(f, bounds)`
`Bounds` , `LinearConstraint`	Objects that describe limits cleanly	`optimize.Bounds(0, 1)`
`NonlinearConstraint`	Constraint given by a function	`NonlinearConstraint(g, 0, np.inf)`
`approx_fprime`	Numerical gradient check	`optimize.approx_fprime(x, f, 1e-8)`
`check_grad`	Compare your gradient with a numerical one	`optimize.check_grad(f, grad, x0)`
`rosen` , `rosen_der`	A classic test function and its gradient	`optimize.rosen([1, 1])`

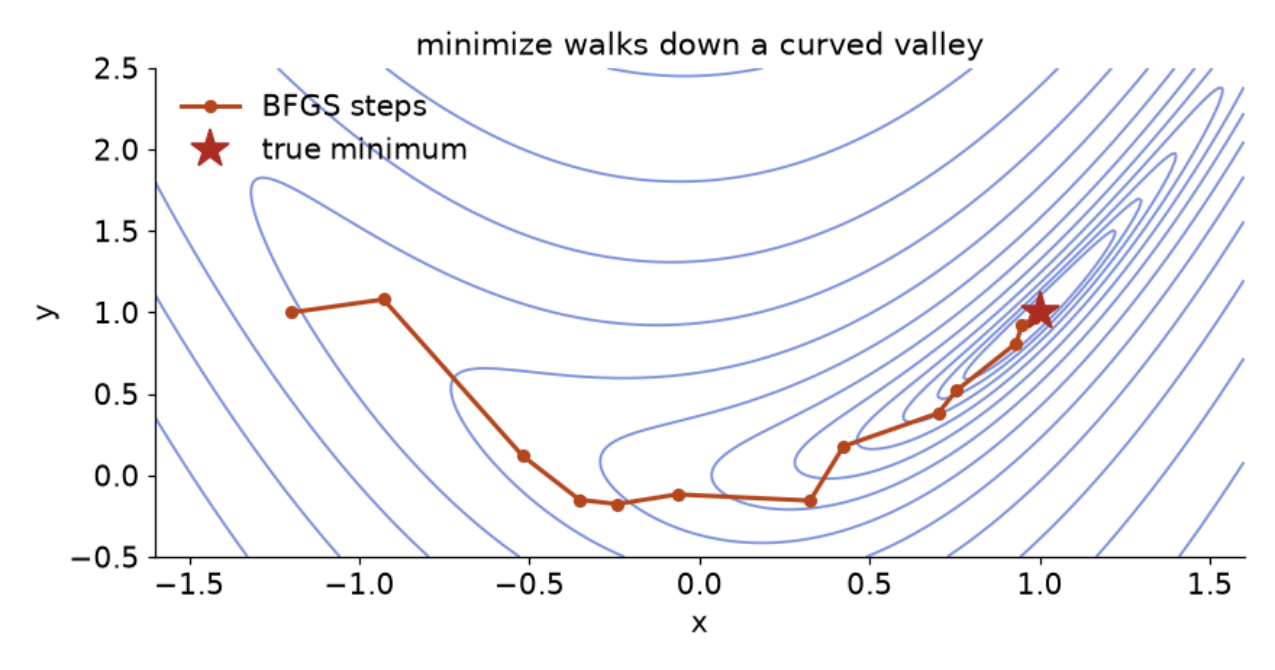
Worked example 1: minimise a function of two variables

Problem 18.4.1 Minimise f(x, y) = $(1 - x)^2 + 5(y - x^2)^2$, a curved valley whose bottom is at (1, 1). Start from (-1.2, 1).

```
import numpy as np
from scipy import optimize

def f(v):
    x, y = v
    return (1 - x) ** 2 + 5 * (y - x ** 2) ** 2

res = optimize.minimize(f, x0=[-1.2, 1.0], method="BFGS")
print("success:", res.success)
print("x      :", res.x.round(4))
print("f(x)   :", float(res.fun) < 1e-8)
print("iterations:", res.nit, " function calls:", res.nfev)
print(res.message)
```

```
success: True
x      : [1. 1.]
f(x)   : True
iterations: 15  function calls: 57
Optimization terminated successfully.
```

1. `f` takes *one* array `v`, not two separate numbers. This is the contract for `minimize`.
2. `x0` is the starting guess. Optimisers walk downhill from there, so a bad start can end in a poor place for non-convex functions.
3. `method="BFGS"` estimates the slope numerically and builds a curved model of the valley as it goes.
4. Always check `res.success` and `res.message`; a run can stop early and still return numbers.
5. We print only whether `res.fun` is nearly zero, because the exact tiny value is noise; `res.nit` and `res.nfev` show the work done.



The BFGS path from (-1.2, 1) to the minimum at (1, 1).

How to read it: the closed curves are levels of equal f: tight lines mean steep ground. The orange path starts on the left, drops into the banana-shaped valley and follows it to the star. Short steps in the valley show the optimiser feeling for the curve.

Worked example 2: constraints and bounds

Problem 18.4.2 Minimise $x^2 + y^2$ subject to x + y at least 1, and both between 0 and 2. By symmetry the answer is (0.5, 0.5).

```
import numpy as np
from scipy import optimize

obj = lambda v: v[0] ** 2 + v[1] ** 2
cons = optimize.LinearConstraint([[1, 1]], lb=1, ub=np.inf)
bnds = optimize.Bounds([0, 0], [2, 2])
res = optimize.minimize(obj, x0=[2.0, 0.0], method="SLSQP", bounds=bnds, constraints=[cons])
print(res.x.round(4), round(res.fun, 4), res.success)
```

```
[0.5 0.5] 0.5 True
```

1. `LinearConstraint([[1, 1]], lb=1, ub=np.inf)` reads: 1 times x plus 1 times y must lie between 1 and infinity.
2. `Bounds` sets a box for each variable separately.
3. SLSQP handles both. The solution sits on the boundary x + y = 1, which is typical: constraints are "active" at the optimum.

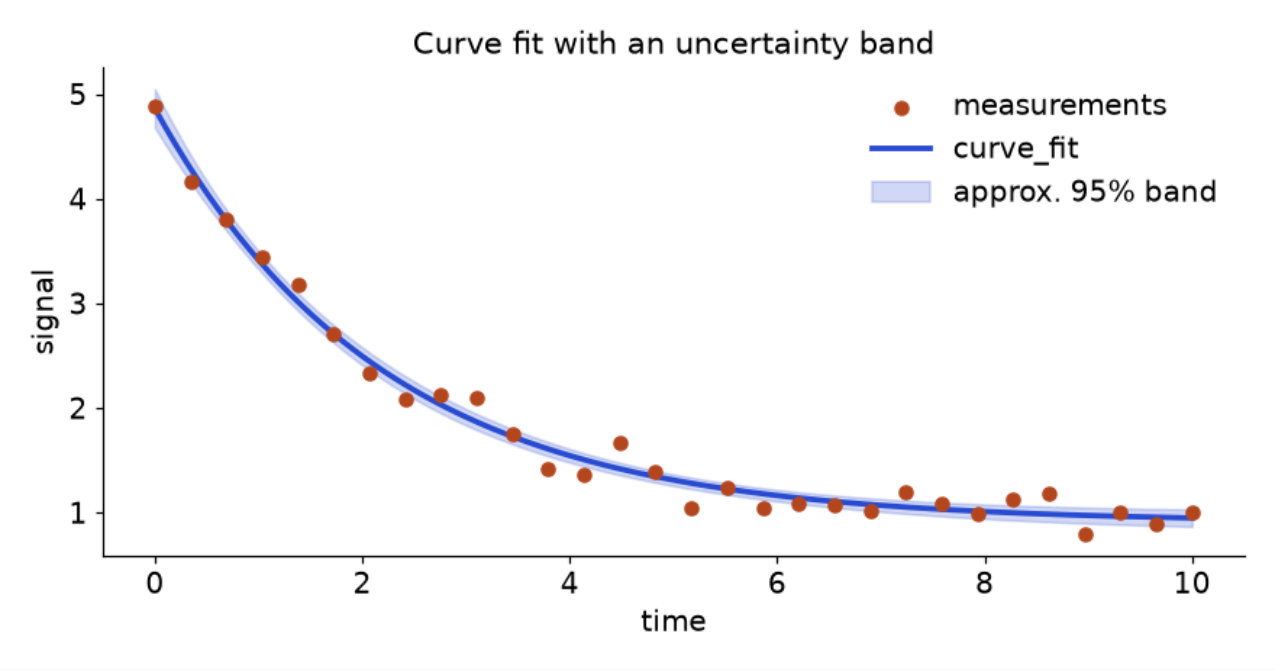
Worked example 3: curve fitting with uncertainty (Recipe 3)

Problem 18.4.3 Support calls after a storm decay as a e^{-kt} + c. Fit a, k, c to 30 noisy measurements and report 95% intervals.

```
import numpy as np
from scipy import optimize, stats

def model(t, a, k, c):
    return a * np.exp(-k * t) + c

rng = np.random.default_rng(5)
t = np.linspace(0, 10, 30)
y = model(t, 4.0, 0.5, 1.0) + rng.normal(0, 0.15, t.size)

popt, pcov = optimize.curve_fit(model, t, y, p0=[1, 1, 0])
perr = np.sqrt(np.diag(pcov))
dof = t.size - len(popt)
tcrit = stats.t.ppf(0.975, dof)
for name, v, e in zip("a k c".split(), popt, perr):
    print("%s = %.3f +/- %.3f   95%% CI (%.3f, %.3f)" % (name, v, e, v - tcrit * e, v + tcrit * e))
resid = y - model(t, *popt)
print("residual std: %.3f (true noise 0.15)" % resid.std(ddof=len(popt)))
```

```
a = 3.954 +/- 0.091   95% CI (3.767, 4.141)
k = 0.457 +/- 0.025   95% CI (0.405, 0.508)
c = 0.908 +/- 0.049   95% CI (0.806, 1.009)
residual std: 0.130 (true noise 0.15)
```

1. `model(t, a, k, c)` must take the x data first, then the parameters as separate arguments.
2. We generate data from known truth (a=4, k=0.5, c=1) plus noise so you can see whether the fit recovers it.
3. `curve_fit(model, t, y, p0=...)` returns `popt` (best parameters) and `pcov` (their covariance matrix). Nonlinear fits need a sensible starting guess `p0`; the default of all ones can fail.
4. The square roots of the diagonal of `pcov` are the standard errors.
5. The 95% interval uses a t quantile with n minus the number of parameters degrees of freedom: n = 30 points, 3 parameters, so 27.
6. The residual standard deviation (dividing by degrees of freedom) should be near the true noise 0.15; if it is much larger, the model shape is wrong.



The fitted curve with an approximate 95% band built from the parameter covariance.

How to read it: the orange dots are measurements, the blue line the best curve and the shaded band the range the curve could plausibly take. The band is narrow where data is dense and reliable. Points outside the band are fine: the band describes the *curve*, not individual measurements.

Worked example 4: roots of equations

Problem 18.4.4 Solve $x\,e^x = 5$ for x. Then find the point where a circle $x^2 + y^2 = 4$ meets the line $y = x$.

```
import numpy as np
from scipy import optimize

g = lambda x: x * np.exp(x) - 5
x1 = optimize.brentq(g, 0, 3)
print("brentq x =", round(x1, 6), " check:", round(x1 * np.exp(x1), 6))

F = lambda v: [v[0] ** 2 + v[1] ** 2 - 4, v[1] - v[0]]
sol = optimize.root(F, [1.0, 1.0])
print("root:", sol.x.round(6), sol.success, " sqrt(2) =", round(np.sqrt(2), 6))
```

```
brentq x = 1.326725  check: 5.0
root: [1.414214 1.414214] True  sqrt(2) = 1.414214
```

1. A root solver wants a function that equals zero at the answer, so we move the 5 to the left side.
2. `brentq(g, 0, 3)` needs a bracket where `g` changes sign (g(0) is negative, g(3) positive). It is fast and cannot fail inside a valid bracket.
3. For several equations in several unknowns, write a function returning a list of residuals, then call `root` with a starting guess.
4. The circle meets the line where x = y = square root of 2, about 1.414214.

Worked example 5: a small linear optimisation (Recipe 4)

Problem 18.4.5 You have 40 driver-hours for zones A and B. A earns 12 per hour, B earns 9. Fuel limits: 3 units per hour in A and 1 per hour in B, with 90 units in total. Zone A can absorb at most 25 hours. Maximise profit.

Variables	a = hours in A, b = hours in B
Objective	maximise 12a + 9b, so minimise -12a - 9b (linprog only minimises)
Limits	a + b <= 40; 3a + b <= 90; 0 <= a <= 25; b >= 0

```
from scipy import optimize

c = [-12, -9]
A_ub = [[1, 1],
        [3, 1]]
b_ub = [40, 90]
res = optimize.linprog(c, A_ub=A_ub, b_ub=b_ub, bounds=[(0, 25), (0, None)], method="highs")
print(res.status, res.message)
print("hours A, B:", res.x.round(2))
print("profit:", round(-res.fun, 2))
print("slack (unused) per limit:", res.slack.round(2))
```

```
0 Optimization terminated successfully. (HiGHS Status 7: Optimal)
hours A, B: [25. 15.]
profit: 435.0
slack (unused) per limit: [0. 0.]
```

1. `c` holds the cost per unit of each variable. Negating turns maximise into minimise.
2. `A_ub` and `b_ub` encode every "at most" row: each row of `A_ub` times the variables must be at most the matching entry of `b_ub`.
3. `bounds` gives each variable a (low, high) pair; `None` means no limit.
4. Status 0 means an optimum was found. The solution is at a corner of the feasible region, which is where linear problems always have their answer.
5. `res.slack` shows unused capacity: a slack of 0 means that limit is binding (fuel and hours are both used up).

COMMON MISTAKE

Forgetting that `linprog` minimises, then reporting a negative "profit" or the wrong plan. Negate the objective going in, and negate `res.fun` coming out.

18.5 scipy.integrate and scipy.interpolate

Integration adds up a quantity that changes continuously: area under a curve, total distance from a speed, probability from a density. **Interpolation** estimates values *between* known points. Both turn scattered or sampled numbers into continuous functions you can query anywhere.

Name	What it does in plain English	Minimal example
`integrate.quad`	Area under a function between two limits, with an error estimate	`integrate.quad(np.sin, 0, np.pi)`
`integrate.dblquad`	Double integral over a region	`integrate.dblquad(lambda y, x: x*y, 0, 1, 0, 1)`
`integrate.nquad`	Integral in many dimensions	`integrate.nquad(f, [[0, 1]] * 3)`
`integrate.quad_vec`	Integrate a vector-valued function	`integrate.quad_vec(lambda x: [x, x**2], 0, 1)`

Name	What it does in plain English	Minimal example
`integrate.trapezoid`	Area from sampled points (straight segments)	`integrate.trapezoid(y, x)`
`integrate.simpson`	Area from sampled points (parabolic pieces)	`integrate.simpson(y, x=x)`
`integrate.cumulative_trapezoid`	Running area, e.g. distance from speed	`integrate.cumulative_trapezoid(v, t, initial=0)`
`integrate.solve_ivp`	Solve a differential equation from a start value	`integrate.solve_ivp(rhs, (0, 5), [1.0])`
`integrate.odeint`	Older ODE solver, still common	`integrate.odeint(rhs, y0, t)`
`interpolate.interp1d`	Legacy 1-D interpolator (no new features)	`interpolate.interp1d(x, y, kind="cubic")`
`np.interp`	Fast linear interpolation on sorted x	`np.interp(2.5, x, y)`
`interpolate.CubicSpline`	Smooth curve through every point	`interpolate.CubicSpline(x, y)`
`interpolate.PchipInterpolator`	Smooth but never overshoots the data	`interpolate.PchipInterpolator(x, y)`
`interpolate.Akima1DInterpolator`	Smooth, little wiggle near outliers	`interpolate.Akima1DInterpolator(x, y)`
`interpolate.make_interp_spline`	B-spline of chosen degree through points	`interpolate.make_interp_spline(x, y, k=3)`
`interpolate.make_smoothing_spline`	Smooth curve that does not chase noise	`make_smoothing_spline(x, y, lam=1)`
`interpolate.UnivariateSpline`	Smoothing spline with a smoothing factor `s`	`UnivariateSpline(x, y, s=1)`
`interpolate.RegularGridInterpolator`	Look up values on a regular grid	`RegularGridInterpolator((xs, ys), Z)`
`interpolate.griddata`	Interpolate scattered 2-D points to a grid	`interpolate.griddata(pts, vals, (gx, gy))`
`interpolate.RBFInterpolator`	Smooth interpolation of scattered data in any dimension	`RBFInterpolator(pts, vals)`
`spline.derivative()`, `.antiderivative()`, `.integrate(a, b)`	Calculus on a fitted spline	`cs.integrate(0, 6)`
`spline.roots()`	Where the fitted curve crosses zero	`cs.roots()`

Worked example 1: integrals of functions, checked against a known answer

Problem 18.5.1 Compute the area under the standard normal density from -1.96 to 1.96 with `quad`, and check it against the cdf. Then compute the average of a function over a range.

```
import numpy as np
from scipy import integrate, stats

area, err = integrate.quad(stats.norm.pdf, -1.96, 1.96)
print("quad area = %.6f  (error estimate %.1e)" % (area, err))
print("cdf diff  = %.6f" % (stats.norm.cdf(1.96) - stats.norm.cdf(-1.96)))

val, _ = integrate.quad(lambda x: x ** 2, 0, 3)
print("integral of x^2 on [0,3]:", round(val, 6), " mean value:", round(val / 3, 6))

tail, _ = integrate.quad(lambda x: np.exp(-x), 0, np.inf)
print("integral of exp(-x) to infinity:", round(tail, 6))
```

```
quad area = 0.950004  (error estimate 1.0e-11)
cdf diff  = 0.950004
integral of x^2 on [0,3]: 9.0  mean value: 3.0
integral of exp(-x) to infinity: 1.0
```

1. `quad(f, a, b)` returns two numbers: the value and an estimate of the absolute error. Unpack both and look at the error.
2. The first check matches the analytic answer from the cdf: a 95% central area of about 0.95.
3. The integral of x^2 from 0 to 3 is $3^3/3 = 9$, so the mean value over the range is 9/3 = 3.
4. Limits may be infinite (`np.inf`); the integral of e^{-x} from 0 to infinity is exactly 1.

Worked example 2: integrate sampled data

Problem 18.5.2 A delivery van logs its speed (km/h) every 6 minutes for an hour. How far did it travel? Use the trapezoid and Simpson rules, and the running total.

```
import numpy as np
from scipy import integrate

t = np.linspace(0, 1, 11)                    # hours
speed = np.array([0, 20, 35, 42, 45, 40, 38, 30, 22, 10, 0.0])
print("trapezoid:", round(integrate.trapezoid(speed, t), 3), "km")
print("simpson  :", round(integrate.simpson(speed, x=t), 3), "km")
dist = integrate.cumulative_trapezoid(speed, t, initial=0)
print("distance at each stop:", dist.round(1))
```

```
trapezoid: 28.2 km
simpson  : 28.267 km
distance at each stop: [ 0.   1.   3.8  7.6 12.  16.2 20.1 23.5 26.1 27.7 28.2]
```

1. Eleven readings give ten 0.1-hour intervals. The arguments of `trapezoid` are `(y, x)` : the values first, the positions second. Swapping them is a classic bug.
2. The trapezoid rule joins neighbouring points with straight lines and adds the areas.
3. Simpson's rule fits parabolas through triples of points and is more accurate for smooth data; it takes `x` as a keyword.
4. `cumulative_trapezoid` returns the distance travelled so far at each reading; `initial=0` keeps the array the same length as the input.

Worked example 3: a differential equation

Problem 18.5.3 App users grow logistically: dN/dt = r N (1 - N/K) with r = 0.8 per week and capacity K = 1000. Starting from 50 users, find N after 10 weeks and when it first passes 500.

```
import numpy as np
from scipy import integrate

r, K = 0.8, 1000.0
rhs = lambda t, N: r * N * (1 - N / K)
sol = integrate.solve_ivp(rhs, (0, 10), [50.0], t_eval=np.linspace(0, 10, 101), rtol=1e-8)
print("N(10) = %.1f" % sol.y[0, -1])
exact = K / (1 + (K / 50 - 1) * np.exp(-r * 10))
print("exact = %.1f" % exact)
cross = sol.t[np.argmax(sol.y[0] > 500)]
print("passes 500 users at about week", cross)
```

```
N(10) = 993.7
exact = 993.7
passes 500 users at about week 3.7
```

1. `solve_ivp` wants `rhs(t, y)` with time first, then the state. The state is an array even for one variable, hence `[50.0]`.
2. `(0, 10)` is the time span; `t_eval` asks for output at 101 evenly spaced times; `rtol` tightens accuracy.
3. `sol.y` has one row per variable and one column per time, so `sol.y[0, -1]` is the last value.
4. This equation has a known closed form, so we compare. Matching it to a few decimals is the verification habit you want.
5. `np.argmax(condition)` returns the first index where the condition is True: the first grid time with more than 500 users.

Worked example 4: interpolation and filling gaps

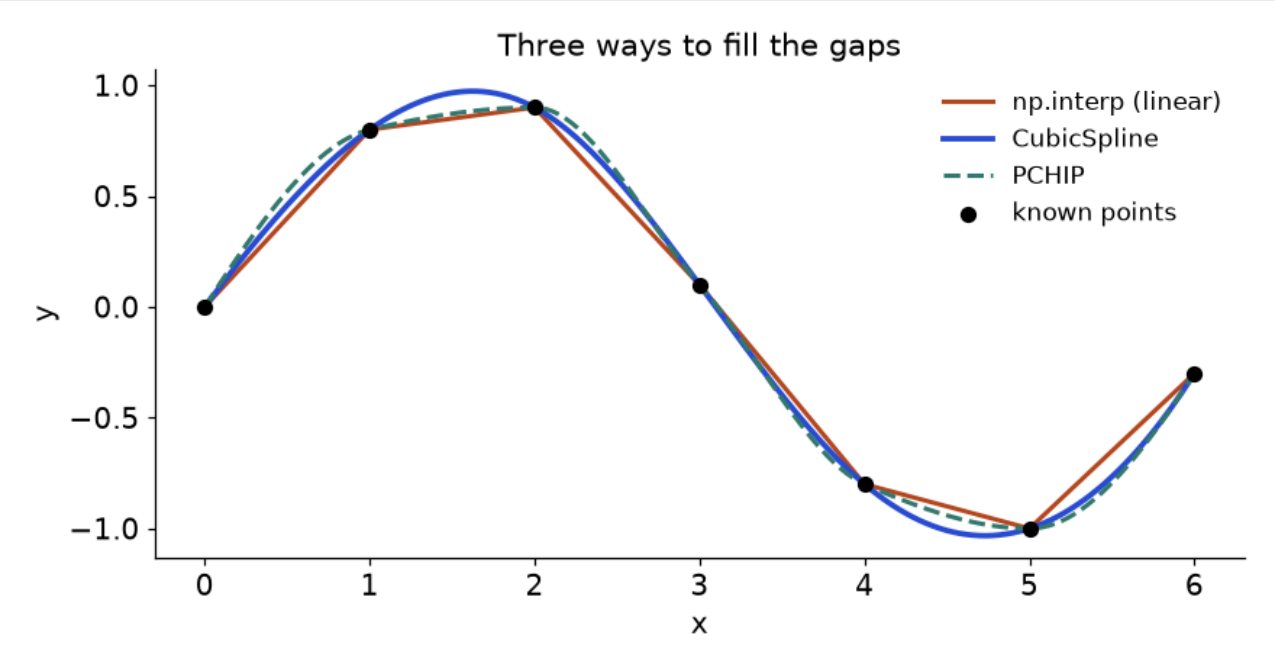
Problem 18.5.4 Seven sensor readings are known. Estimate the value at x = 2.5 with three methods, then use a spline to get the slope there and the total area.

```
import numpy as np
from scipy import interpolate

xk = np.array([0, 1, 2, 3, 4, 5, 6.0])
yk = np.array([0, 0.8, 0.9, 0.1, -0.8, -1.0, -0.3])

print("linear:", round(float(np.interp(2.5, xk, yk)), 4))
cs = interpolate.CubicSpline(xk, yk)
pc = interpolate.PchipInterpolator(xk, yk)
print("cubic :", round(float(cs(2.5)), 4))
print("pchip :", round(float(pc(2.5)), 4))
print("slope at 2.5:", round(float(cs.derivative()(2.5)), 4))
print("area 0..6   :", round(float(cs.integrate(0, 6)), 4))
print("zero crossings:", cs.roots(extrapolate=False).round(3))
```

```
linear: 0.5
cubic : 0.5768
pchip : 0.6059
slope at 2.5: -0.8452
area 0..6   : -0.1643
zero crossings: [0.    3.098]
```

1. Arrays `xk` and `yk` are the known points. Interpolators need `x` sorted in increasing order.
2. `np.interp` joins neighbours with straight lines: at 2.5 that is the midpoint of 0.9 and 0.1, which is 0.5.

3. A cubic spline is a smooth curve through every point; PCHIP is smooth too but is built so it never swings above or below neighbouring data.
4. A fitted spline is an object you can differentiate, integrate and root-find. `extrapolate=False` keeps roots to the known range; the first root, 0, is simply the first data point (its value is 0).



Linear, cubic spline and PCHIP interpolation of the same seven points.

How to read it: every curve passes through all black points, so they only disagree between points. The orange linear path has corners. The blue spline is smoothest but may overshoot slightly near sharp turns; the dashed PCHIP stays within the neighbouring values. Choose PCHIP for data that must not overshoot (prices, counts).

COMMON MISTAKE	Using interpolation to *predict outside the data*. Splines are only trustworthy between the first and last x. A cubic extrapolated past the end can swing wildly. Use `extrapolate=False` or a real model.

18.6 scipy.linalg and scipy.sparse

Linear algebra solves systems like A x = b, finds the best-fitting line (least squares) and splits a matrix into simpler parts (SVD, eigenvalues). **Sparse matrices** store only the non-zero entries, so a matrix of 10 million cells that is 99.99% zero takes a few kilobytes. `numpy.linalg` covers the basics; `scipy.linalg` is a superset with more decompositions and options, and always uses optimised LAPACK.

Name	What it does in plain English	Minimal example
`linalg.solve`	Solve A x = b (preferred over inv)	`linalg.solve(A, b)`
`linalg.inv` , `pinv`	Inverse and pseudo-inverse	`linalg.pinv(A)`
`linalg.det`	Determinant, zero means singular	`linalg.det(A)`
`linalg.lstsq`	Best-fit solution of an over-determined system	`linalg.lstsq(X, y)`
`linalg.eig` , `eigh`	Eigenvalues; `eigh` for symmetric matrices (faster, real)	`linalg.eigh(C)`
`linalg.svd`	Singular value decomposition	`U, s, Vt = linalg.svd(A)`
`linalg.qr`	QR decomposition	`Q, R = linalg.qr(A)`

Name	What it does in plain English	Minimal example
`linalg.lu_factor / lu_solve`	Factor once, solve many right-hand sides	`lu = linalg.lu_factor(A)`
`linalg.cholesky`	Square-root of a positive-definite matrix	`linalg.cholesky(C, lower=True)`
`linalg.solve_triangular`	Fast solve for triangular systems	`linalg.solve_triangular(L, b, lower=True)`
`linalg.norm`	Vector or matrix size	`linalg.norm(A, "fro")`
`linalg.expm`	Matrix exponential	`linalg.expm(A)`
`linalg.null_space` , `orth`	Basis of the null space or column space	`linalg.null_space(A)`
`linalg.block_diag`	Build a block-diagonal matrix	`linalg.block_diag(A, B)`
`sparse.csr_array`	Compressed sparse rows: fast row slices and products	`sparse.csr_array(dense)`
`sparse.csc_array`	Compressed sparse columns: fast column slices and solves	`sparse.csc_array(dense)`
`sparse.coo_array`	Triplets (row, col, value): easiest to build	`sparse.coo_array((v, (r, c)), shape=s)`
`sparse.lil_array` , `dok_array`	Cheap to change element by element	`sparse.lil_array((3, 3))`
`sparse.eye_array` , `diags_array`	Identity and banded matrices	`sparse.eye_array(5)`
`sparse.random_array`	Random sparse test matrix	`sparse.random_array((100, 100), density=0.01)`
`sparse.linalg.spsolve`	Solve a sparse system	`spsolve(A.tocsc(), b)`
`sparse.linalg.cg`	Iterative solver for symmetric positive-definite systems	`cg(A, b)`
`sparse.linalg.eigsh` , `svds`	A few eigenvalues or singular values of a big sparse matrix	`svds(A, k=5)`
`sparse.csgraph.connected_components`	Group nodes of a graph	`connected_components(G)`
`sparse.csgraph.dijkstra`	Shortest paths on a weighted graph	`dijkstra(G, indices=0)`
`sparse.hstack / vstack`	Join sparse matrices	`sparse.hstack([A, B])`

Worked example 1: solving a system, and why you never invert

Problem 18.6.1 Three products use three ingredients. Find the batch sizes x that use exactly 100, 120 and 80 units of the ingredients. Then see what happens with a nearly singular matrix.

```
import numpy as np
from scipy import linalg

A = np.array([[2.0, 1.0, 1.0],
              [1.0, 3.0, 2.0],
              [1.0, 0.0, 4.0]])
b = np.array([100.0, 120.0, 80.0])
x = linalg.solve(A, b)
print("x       :", x.round(4))
print("A @ x   :", (A @ x).round(6))
print("det, cond:", round(linalg.det(A), 3), round(np.linalg.cond(A), 2))

B = np.array([[1.0, 2.0], [2.0, 4.0000001]])
print("cond of nearly singular B: %.2e" % np.linalg.cond(B))
```

```
x       : [33.6842 21.0526 11.5789]
A @ x   : [100. 120.  80.]
det, cond: 19.0 3.88
cond of nearly singular B: 2.50e+08
```

1. Each row of `A` is one ingredient; each column is one product.
2. `linalg.solve` factors the matrix (LU) and back-substitutes. It is faster and more accurate than `inv(A) @ b`.
3. Check by multiplying back: `A @ x` must reproduce `b`. This is the cheapest verification there is.
4. The condition number tells you how much errors are amplified. A value near 1 is healthy. Matrix `B` has rows that are almost multiples of each other, so its condition number is enormous (around 10^8): small noise in the data would swing the answer wildly.

Worked example 2: least squares and the SVD

Problem 18.6.2 Fit price = w0 + w1 * size + w2 * rooms to 200 simulated homes with `lstsq`, then use the SVD to see how many directions the 5 columns of a redundant feature matrix really carry.

```
import numpy as np
from scipy import linalg

rng = np.random.default_rng(0)
size = rng.uniform(40, 160, 200)
rooms = rng.integers(1, 6, 200)
price = 20 + 3.0 * size + 12 * rooms + rng.normal(0, 10, 200)
X = np.column_stack([np.ones(200), size, rooms])
w, res, rank, sv = linalg.lstsq(X, price)
print("weights:", w.round(2), " rank:", rank)

base = rng.normal(size=(200, 3))
M = np.column_stack([base, base[:, 0] + base[:, 1], 2 * base[:, 2]])   # 5 columns, 3 independent
s = linalg.svdvals(M)
print("singular values:", s.round(2))
print("numerical rank :", int((s > 1e-10 * s[0]).sum()))
```

```
weights: [23.4   2.99 11.27]  rank: 3
singular values: [30.55 25.31 12.47  0.    0.  ]
numerical rank : 3
```

1. The design matrix `X` has a column of ones for the intercept, then one column per feature.
2. `lstsq(X, price)` minimises the sum of squared errors without forming the normal equations, which is numerically safer. It returns the weights, residuals, rank and singular values.
3. The slopes land close to the truth (3 and 12); the intercept is noisier (about 23 against a true 20) because the noise has standard deviation 10. More data tightens all three.
4. The matrix `M` has five columns but two are sums of others. `svdvals` shows only three clearly non-zero singular values; the other two collapse to about zero. That count is the numerical rank, and it explains why such a matrix breaks ordinary regression.

Worked example 3: sparse matrices save memory and time

Problem 18.6.3 A user-by-product matrix has 20,000 users and 5,000 products, and each user bought about 10 products. Compare dense and sparse storage, then multiply by a vector.

```
import numpy as np
from scipy import sparse

rng = np.random.default_rng(1)
n_users, n_items, per_user = 20000, 5000, 10
rows = np.repeat(np.arange(n_users), per_user)
cols = rng.integers(0, n_items, n_users * per_user)
vals = np.ones(rows.size)
M = sparse.coo_array((vals, (rows, cols)), shape=(n_users, n_items)).tocsr()

dense_mb = n_users * n_items * 8 / 1e6
sparse_mb = (M.data.nbytes + M.indices.nbytes + M.indptr.nbytes) / 1e6
print("dense  needs %.0f MB" % dense_mb)
print("sparse needs %.1f MB" % sparse_mb)
print("stored non-zeros:", M.nnz, " density: %.4f" % (M.nnz / (n_users * n_items)))

popularity = M.sum(axis=0)                # purchases per product
print("most popular product id:", int(np.argmax(popularity)), "with", int(popularity.max()))
user_scores = M @ rng.normal(size=n_items)
print("scores shape:", user_scores.shape)
```

```
dense  needs 800 MB
sparse needs 3.4 MB
stored non-zeros: 199815  density: 0.0020
most popular product id: 2622 with 62
scores shape: (20000,)
```

1. The COO format stores triples (row, column, value), the easiest way to build a matrix from a list of events. Duplicate (row, col) pairs are summed when converted.
2. `.tocsr()` converts to compressed sparse rows, the format for fast arithmetic and row slicing.
3. Dense float64 storage costs 8 bytes per cell: 20,000 times 5,000 times 8 is 800 MB. The sparse version stores three small arrays.
4. `M.sum(axis=0)` and `M @ vector` run directly on the sparse structure without ever building the dense matrix.

KEY IDEA	Build in COO or LIL, compute in CSR or CSC. Row-wise work (feature rows for a model) suits CSR; column-wise work and direct solvers suit CSC. Prefer the `*_array` classes in new code; the older `*_matrix` classes have different `*` semantics (matrix multiply) and exist for compatibility.

Worked example 4: a big sparse system and a graph

Problem 18.6.4 Solve a 100,000-unknown tridiagonal system (a 1-D heat-flow style problem), then find connected groups and shortest paths in a small road network.

```
import numpy as np
from scipy import sparse
from scipy.sparse import linalg as spla
from scipy.sparse import csgraph

n = 100_000
A = sparse.diags_array([-np.ones(n - 1), 2 * np.ones(n), -np.ones(n - 1)], offsets=[-1, 0, 1], format="csc")
b = np.ones(n)
x = spla.spsolve(A, b)
print("residual max:", float(np.abs(A @ x - b).max()) < 1e-6, " x[n//2] = %.0f" % x[n // 2])

# roads: 0–1 (4 km), 1–2 (3), 0–2 (9), 3–4 (2); nodes 3,4 are a separate island
rows, cols, km = [0, 1, 0, 3], [1, 2, 2, 4], [4, 3, 9, 2]
G = sparse.coo_array((km, (rows, cols)), shape=(5, 5)).tocsr()
n_comp, labels = csgraph.connected_components(G, directed=False)
print("components:", n_comp, labels)
print("distance from 0:", csgraph.dijkstra(G, directed=False, indices=0))
```

```
residual max: True  x[n//2] = 1250025000
components: 2 [0 0 0 1 1]
distance from 0: [ 0.  4.  7. inf inf]
```

1. `diags_array` builds a banded matrix from three diagonals. Only about 300,000 numbers are stored instead of 10^{10}.
2. `spsolve` wants CSC format for its direct solver. A dense solve of this size would not even fit in memory.
3. We verify by multiplying back and checking the residual.
4. A graph is just a sparse adjacency matrix with distances as values. `connected_components` finds the separate islands (two here).
5. `dijkstra` returns the shortest distance from node 0 to every node: to node 2 it is 4 + 3 = 7 rather than the direct 9. Unreachable nodes show `inf`.

18.7 scipy.signal and scipy.fft

A **signal** is any measurement taken over time: sales per day, a heartbeat, a machine's vibration. Two ideas cover most uses. **Filtering** removes unwanted parts (noise, slow drift). The **Fourier transform** re-expresses the signal as a sum of waves, so you can see *which frequencies* it contains. Frequencies need a **sampling rate** `fs` (samples per second, or per day for daily data).

Name	What it does in plain English	Minimal example
`fft.rfft`	Fourier transform of real data (non-negative frequencies)	`fft.rfft(x)`
`fft.rfftfreq`	The frequency of each rfft output bin	`fft.rfftfreq(len(x), 1/fs)`
`fft.fft`, `ifft`	Complex transform and its inverse	`fft.ifft(fft.fft(x))`
`fft.fftfreq`	Frequencies for the full complex transform	`fft.fftfreq(n, d)`
`fft.next_fast_len`	A length that transforms quickly	`fft.next_fast_len(1001)`
`fft.dct`	Cosine transform (compression)	`fft.dct(x, norm="ortho")`
`signal.butter`	Design a Butterworth filter	`signal.butter(4, 5, fs=100, output="sos")`

Name	What it does in plain English	Minimal example
`signal.sosfiltfilt`	Apply a filter forward and back (no delay)	`signal.sosfiltfilt(sos, x)`
`signal.filtfilt`, `lfilter`	Filter with b, a coefficients	`signal.filtfilt(b, a, x)`
`signal.savgol_filter`	Smooth with local polynomials, keeps peak shape	`signal.savgol_filter(x, 11, 3)`
`signal.medfilt`	Median filter, removes spikes	`signal.medfilt(x, 5)`
`signal.find_peaks`	Locate peaks by height, distance, prominence	`signal.find_peaks(x, prominence=1)`
`signal.peak_prominences`	How much each peak stands out	`signal.peak_prominences(x, peaks)`
`signal.welch`	Power spectrum with less noise than a raw FFT	`signal.welch(x, fs)`
`signal.periodogram`	Simple power spectrum	`signal.periodogram(x, fs)`
`signal.spectrogram`	Frequencies changing over time	`signal.spectrogram(x, fs)`
`signal.convolve`	Sliding weighted sum (moving averages)	`signal.convolve(x, w, mode="same")`
`signal.correlate`	Find a pattern or a lag between signals	`signal.correlate(a, b, mode="full")`
`signal.detrend`	Remove a straight-line trend	`signal.detrend(x)`
`signal.resample`	Change the number of samples	`signal.resample(x, 200)`
`signal.hilbert`	Envelope and phase	`np.abs(signal.hilbert(x))`
`signal.windows.hann`	Taper to reduce spectral leakage	`signal.windows.hann(len(x))`

Worked example 1: which frequencies are in my data?

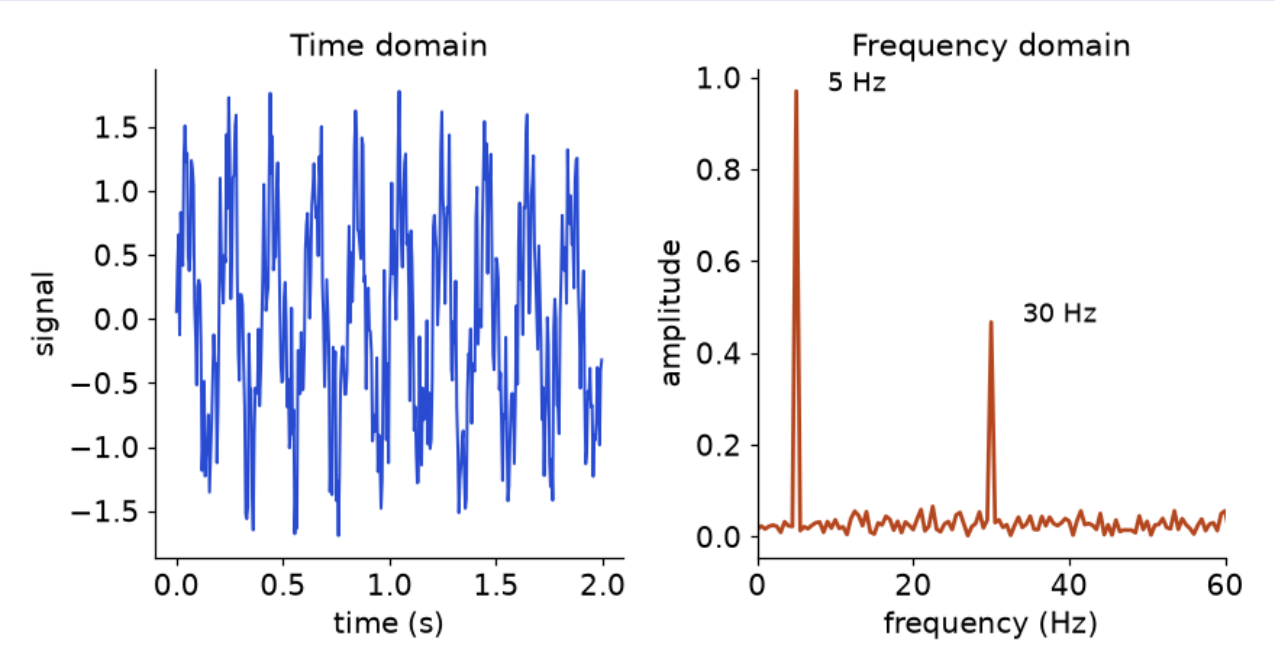
Problem 18.7.1 A sensor sampled at 200 Hz for 2 seconds hides a 5 Hz wave of amplitude 1 and a 30 Hz wave of amplitude 0.5 under noise. Find them.

```
import numpy as np
from scipy import fft

rng = np.random.default_rng(2)
fs = 200
t = np.arange(0, 2, 1 / fs)
x = np.sin(2 * np.pi * 5 * t) + 0.5 * np.sin(2 * np.pi * 30 * t) + rng.normal(0, 0.3, t.size)

amp = np.abs(fft.rfft(x)) * 2 / t.size
freq = fft.rfftfreq(t.size, 1 / fs)
top = np.argsort(amp)[-2:][::-1]
for i in top:
    print("peak at %.1f Hz, amplitude %.2f" % (freq[i], amp[i]))
print("bin spacing:", freq[1], "Hz; highest frequency:", freq[-1], "Hz (Nyquist = fs/2)")
```

```
peak at 5.0 Hz, amplitude 0.97
peak at 30.0 Hz, amplitude 0.47
bin spacing: 0.5 Hz; highest frequency: 100.0 Hz (Nyquist = fs/2)
```

1. Two seconds at 200 samples per second gives 400 points; `t` is in seconds.
2. `rfft` returns complex numbers for frequencies from 0 up to fs/2. Taking `np.abs` gives strength; multiplying by 2/n converts it to the amplitude of the original wave (valid away from 0 Hz).

3. `rfftfreq(n, 1/fs)` labels each output bin. The bin spacing is fs/n = 0.5 Hz, so frequencies that fall exactly on the grid (5 and 30) appear cleanly.
4. We sort amplitudes and take the two largest: 5 Hz with amplitude about 1 and 30 Hz with about 0.5.
5. Nothing above 100 Hz (the Nyquist limit, half the sampling rate) can be seen; faster waves would masquerade as slower ones (aliasing).



The same signal in the time domain and in the frequency domain.

How to read it: on the left the pattern is invisible in the noise. On the right, each tall spike is one hidden wave: its position tells the frequency and its height the amplitude. The flat low carpet is the noise.

Worked example 2: filter noise away

Problem 18.7.2 Remove everything above 10 Hz from the same signal with a Butterworth low-pass filter, and measure the improvement.

```
import numpy as np
from scipy import signal

rng = np.random.default_rng(2)
fs = 200
t = np.arange(0, 2, 1 / fs)
clean = np.sin(2 * np.pi * 5 * t)
noisy = clean + rng.normal(0, 0.5, t.size)

sos = signal.butter(4, 10, btype="low", fs=fs, output="sos")
smooth = signal.sosfiltfilt(sos, noisy)
print("noise rms before: %.3f" % np.sqrt(np.mean((noisy - clean) ** 2)))
print("noise rms after : %.3f" % np.sqrt(np.mean((smooth - clean) ** 2)))
```

```
noise rms before: 0.504
noise rms after : 0.125
```

1. The "clean" signal is a 5 Hz sine; we add noise of standard deviation 0.5, covering all frequencies.
2. `butter(4, 10, btype="low", fs=fs, output="sos")` designs a 4th-order filter that keeps frequencies below 10 Hz. Giving `fs` means the cutoff is in Hz, not a fraction of Nyquist. The `sos` output (second-order sections) is the numerically stable form.
3. `sosfiltfilt` runs the filter forward and then backward, which cancels the time delay (zero phase). A plain forward filter shifts the signal in time.

4. The root-mean-square error against the known clean signal drops by a large factor: most of the noise lived above 10 Hz.

Worked example 3: find the peaks (for example daily orders)

Problem 18.7.3 Daily orders have a weekly rhythm plus noise. Find the peak days and confirm that the period is 7 days.

```
import numpy as np
from scipy import signal, fft

rng = np.random.default_rng(8)
days = np.arange(140)
orders = 100 + 20 * np.sin(2 * np.pi * days / 7) + rng.normal(0, 4, days.size)

smooth = signal.savgol_filter(orders, window_length=5, polyorder=2)
peaks, props = signal.find_peaks(smooth, distance=5, prominence=15)
print("peak days:", peaks[:6], "... total", len(peaks))
print("gaps between peaks:", np.diff(peaks)[:8])

spec = np.abs(fft.rfft(orders - orders.mean()))
f = fft.rfftfreq(days.size, d=1.0)            # cycles per day
print("dominant period: %.1f days" % (1 / f[np.argmax(spec)]))
```

```
peak days: [ 2  9 16 23 30 36] ... total 20
gaps between peaks: [7 7 7 7 6 8 7 7]
dominant period: 7.0 days
```

1. Orders are a baseline of 100, a 7-day wave of amplitude 20 and noise.
2. `savgol_filter` fits a small polynomial in a sliding window of 5 days; it smooths the noise while keeping the peak heights fairly intact (a moving average would flatten them more).
3. `find_peaks` returns indices and a dictionary of properties. `distance=5` forbids two peaks closer than 5 days; `prominence=15` requires each to stand 15 orders above its surroundings. These two arguments are what stop noise wiggles from counting.
4. The gaps between neighbouring peaks are mostly 7 days (6 or 8 where noise nudges a peak by a day).
5. The FFT agrees: with daily samples, `d=1.0`, frequencies are in cycles per day, and 1 divided by the strongest frequency is the period. We subtract the mean first so the zero-frequency bin does not dominate.

18.8 scipy.spatial and scipy.cluster

Spatial tools answer "who is near whom?". A **KDTree** splits space into boxes once so that each nearest-neighbour query skips most of the data. **Distance functions** measure similarity between rows. **Clustering** groups points that belong together; `scipy.cluster` offers k-means and hierarchical clustering (for the full machine-learning treatment see scikit-learn and the Machine Learning document).

Name	What it does in plain English	Minimal example
`spatial.KDTree`	Index for fast neighbour search in low dimensions	`tree = spatial.KDTree(points)`
`tree.query(x, k)`	The k nearest points and their distances	`d, i = tree.query(q, k=3)`
`tree.query_ball_point(x, r)`	All points within radius r	`tree.query_ball_point(q, r=2.0)`
`tree.query_pairs(r)`	All pairs closer than r	`tree.query_pairs(0.5)`
`tree.sparse_distance_matrix`	Distances for near pairs only	`tree.sparse_distance_matrix(tree, 1.0)`
`distance.cdist`	Distance between every row of A and every row of B	`distance.cdist(A, B)`
`distance.pdist`	All pairwise distances inside one set (condensed)	`distance.pdist(X)`

Name	What it does in plain English	Minimal example
`distance.squareform`	Convert condensed to square matrix and back	`distance.squareform(d)`
`distance.euclidean`	Straight-line distance between two points	`distance.euclidean(a, b)`
`distance.cityblock`	Manhattan (grid) distance	`distance.cityblock(a, b)`
`distance.cosine`	1 minus cosine similarity (direction only)	`distance.cosine(a, b)`
`distance.mahalanobis`	Distance that accounts for correlation	`distance.mahalanobis(a, b, VI)`
`distance.jaccard` , `hamming`	Distance for yes/no vectors	`distance.hamming(u, v)`
`spatial.ConvexHull`	Outer boundary of a point cloud	`spatial.ConvexHull(points).vertices`
`spatial.Delaunay`	Triangulation of points	`spatial.Delaunay(points)`
`spatial.Voronoi`	Region closest to each point	`spatial.Voronoi(points)`
`spatial.transform.Rotation`	3-D rotations without gimbal trouble	`Rotation.from_euler("z", 90, degrees=True)`
`cluster.vq.kmeans2`	k-means clustering, returns centres and labels	`vq.kmeans2(X, 3, seed=1, minit="++")`
`cluster.vq.whiten`	Scale each column to unit spread first	`vq.whiten(X)`
`cluster.vq.vq`	Assign points to the nearest centre	`vq.vq(X, centres)`
`cluster.hierarchy.linkage`	Build the merge tree (Ward, average, ...)	`hierarchy.linkage(X, "ward")`
`cluster.hierarchy.fcluster`	Cut the tree into flat clusters	`hierarchy.fcluster(Z, 3, "maxclust")`
`cluster.hierarchy.dendrogram`	Plot the merge tree	`hierarchy.dendrogram(Z)`
`cluster.hierarchy.cophenet`	How faithfully the tree keeps distances	`hierarchy.cophenet(Z, d)`

Worked example 1: nearest neighbours the slow and the fast way

Problem 18.8.1 A courier app has 200,000 restaurant locations. For 2,000 customers, find the 3 nearest restaurants. Compare brute force with a KDTree.

```
import time
import numpy as np
from scipy import spatial

rng = np.random.default_rng(0)
shops = rng.uniform(0, 100, (200_000, 2))
custs = rng.uniform(0, 100, (2_000, 2))

t0 = time.perf_counter()
tree = spatial.KDTree(shops)
dist, idx = tree.query(custs, k=3)
t_tree = time.perf_counter() - t0

t0 = time.perf_counter()
brute = np.array([np.argsort(((shops - c) ** 2).sum(axis=1))[:3] for c in custs[:200]])
t_brute = (time.perf_counter() - t0) * 10          # scaled up from 200 to 2000 customers

print("same answer:", np.array_equal(brute, idx[:200]))
print("tree at least 10x faster:", t_brute > 10 * t_tree)
print("first customer, nearest shop distance: %.3f" % dist[0, 0])
```

```
same answer: True
tree at least 10x faster: True
first customer, nearest shop distance: 0.149
```

1. Points are rows of a 2-D array: 200,000 shops and 2,000 customers.
2. Building the tree is a one-off cost; `query(custs, k=3)` then handles all customers in one vectorised call, returning distances and indices (each of shape 2000 by 3).
3. The brute-force version computes the distance to all 200,000 shops for each customer, then sorts. We time only 200 customers and multiply by 10 to save waiting; this is an estimate.
4. We first check that both methods agree, then compare speed. We print a comparison rather than raw seconds because timings vary between machines and runs; on this machine the tree was far faster.

REAL JOB	KDTrees shine in 2-D and 3-D (geo points, sensor positions) and degrade as dimensions grow past about 20, when nearly every point is almost equally far away. For embeddings use an approximate index or scikit-learn's neighbour search.

Worked example 2: distance matrices and hierarchical clusters

Problem 18.8.2 Six customers have (annual spend in thousands, orders per month). Compute distances, then cluster them into 2 groups with Ward linkage.

```
import numpy as np
from scipy.spatial import distance
from scipy.cluster import hierarchy

X = np.array([[1.0, 2], [1.2, 2.5], [0.8, 1.8],
              [8.0, 9], [8.5, 9.4], [7.6, 8.8]])
D = distance.squareform(distance.pdist(X))
print("distance 0–1: %.3f   0–3: %.3f" % (D[0, 1], D[0, 3]))
Z = hierarchy.linkage(X, method="ward")
labels = hierarchy.fcluster(Z, t=2, criterion="maxclust")
print("labels:", labels)
print("merge heights:", Z[:, 2].round(2))
print("cosine distance of rows 0 and 3: %.4f" % distance.cosine(X[0], X[3]))
```

```
distance 0–1: 0.539   0–3: 9.899
labels: [1 1 1 2 2 2]
merge heights: [ 0.28  0.45  0.77  0.99 17.15]
cosine distance of rows 0 and 3: 0.0344
```

1. Each row of `X` is a customer; two groups are obvious by eye.
2. `pdist` returns a flat list of the 15 pairwise distances; `squareform` turns it into a 6 by 6 table where `D[i, j]` is the distance between customers i and j.
3. `linkage(X, "ward")` repeatedly merges the two closest groups. Each row of `Z` says which two were merged, at what height, and the new size.
4. `fcluster(Z, t=2, criterion="maxclust")` cuts the tree so that exactly two clusters remain; labels start at 1.
5. Merge heights jump sharply at the last merge (joining the two big groups): a big jump is the signal that the previous cut was natural.
6. The cosine distance compares direction only, so two rows with similar proportions look close even if one is much larger. It is small here because both customers have a similar balance of spend and orders.

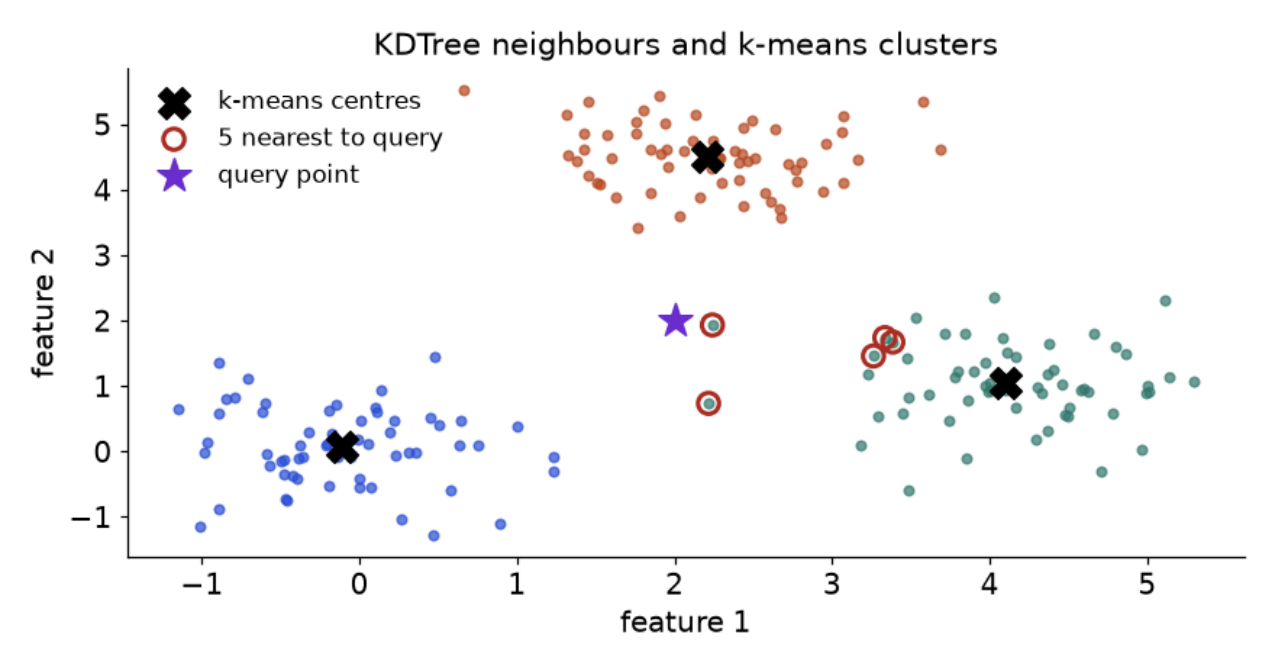
Worked example 3: k-means

Problem 18.8.3 Cluster 180 points drawn around three centres, then ask which cluster a new point belongs to.

```
import numpy as np
from scipy.cluster import vq

rng = np.random.default_rng(4)
pts = np.vstack([rng.normal(c, 0.6, (60, 2)) for c in ([0, 0], [4, 1], [2, 4.5])])
centres, labels = vq.kmeans2(pts, 3, seed=1, minit="++")
print("sizes:", np.bincount(labels))
print("centres:\n", centres[np.argsort(centres[:, 0])].round(2))
new_point = np.array([[3.8, 0.7]])
code, dist = vq.vq(new_point, centres)
print("new point joins centre", int(code[0]), "at distance %.2f" % dist[0])
```

```
sizes: [60 60 60]
centres:
 [[-0.11  0.07]
 [ 2.2   4.51]
 [ 4.1   1.04]]
new point joins centre 2 at distance 0.45
```

1. Three groups of 60 points each, centred near (0, 0), (4, 1) and (2, 4.5).
2. `kmeans2(pts, 3, seed=1, minit="++")` picks three starting centres smartly (k-means++), then alternates between assigning points to the nearest centre and moving centres to the average of their points.
3. Cluster numbering is arbitrary, so we sort centres by x before printing. They land near the true centres.
4. `vq.vq` assigns new points to the nearest centre; it returns the centre index and the distance.



k-means centres, plus the five nearest neighbours of a query point found with a KDTree.

How to read it: colours are the k-means groups and the X marks are centres. The star sits between clusters, so its five nearest neighbours (red circles) come from more than one group. Nearest-neighbour search and cluster membership answer different questions.

18.9 statsmodels: OLS, GLM, ARIMA and the Formula API

scikit-learn asks "how well does this predict?". statsmodels asks "what do the coefficients mean, and how sure are we?". It returns standard errors, t-statistics, p-values, confidence intervals and diagnostic tests in one summary table, which is what analysts, economists and scientists must report. **statsmodels is not installed on this study machine.** Each statsmodels block below is labelled *not run here*: it is standard, idiomatic code, but no output is shown for it, because I will not invent output. Directly after each one comes a NumPy/SciPy version that did run, which computes the same quantities from first principles. Run the statsmodels code yourself after `python3 -m pip install statsmodels` and compare.

Name	What it does in plain English	Minimal example
`sm.add_constant(X)`	Add the intercept column (statsmodels does not add one for you)	`X = sm.add_constant(X)`
`sm.OLS(y, X).fit()`	Ordinary least squares regression	`res = sm.OLS(y, X).fit()`
`res.summary()`	Full table: coefficients, errors, p, R-squared	`print(res.summary())`
`res.params` , `bse`	Coefficients and their standard errors	`res.params["x"]`

Name	What it does in plain English	Minimal example
`res.pvalues` , `tvalues`	Test of each coefficient being zero	`res.pvalues`
`res.conf_int(alpha=0.05)`	95% interval for each coefficient	`res.conf_int()`
`res.rsquared` , `rsquared_adj`	Share of variance explained	`res.rsquared_adj`
`res.resid` , `fittedvalues`	Residuals and fitted values for diagnostics	`res.resid.std()`
`res.predict(X_new)`	Predictions for new rows	`res.predict(X_new)`
`res.get_prediction(X).summary_frame()`	Predictions with confidence and prediction intervals	`pred.summary_frame(alpha=0.05)`
`fit(cov_type="HC3")`	Robust standard errors when the spread is uneven	`sm.OLS(y, X).fit(cov_type="HC3")`
`smf.ols("y ~ x1 + C(g)", data=df)`	Formula interface: column names, categories built in	`smf.ols("price ~ size + C(city)", df).fit()`
`sm.GLM(y, X, family=...)`	Generalised linear model (Poisson, Binomial, Gamma)	`sm.GLM(y, X, family=sm.families.Poisson())`
`sm.Logit(y, X)`	Logistic regression with inference	`sm.Logit(y, X).fit()`
`sm.stats.anova_lm(res)`	Analysis of variance table	`sm.stats.anova_lm(res, typ=2)`
`variance_inflation_factor`	Detect collinear features	`variance_inflation_factor(X.values, i)`
`het_breuschpagan`	Test for uneven residual spread	`sm.stats.diagnostic.het_breuschpagan(res.resid, X)`
`durbin_watson`	Residual autocorrelation (near 2 is fine)	`sm.stats.durbin_watson(res.resid)`
`sm.tsa.ARIMA(y, order=(p, d, q))`	Time-series model with trend and noise structure	`ARIMA(y, order=(1, 0, 0)).fit()`
`sm.tsa.SARIMAX`	ARIMA with seasonality and outside regressors	`SARIMAX(y, order=(1,1,1), seasonal_order=(1,0,0,7))`
`res.forecast(steps)`	Predict the next steps of a time series	`res.forecast(7)`
`sm.tsa.seasonal_decompose`	Split into trend, season and rest	`seasonal_decompose(y, period=7)`
`sm.tsa.adfuller`	Test whether a series is stationary	`adfuller(y)[1]`
`sm.tsa.acf` , `pacf`	Correlation of a series with its own past	`sm.tsa.acf(y, nlags=20)`

Name	What it does in plain English	Minimal example
`sm.stats.proportions_ztest`	Compare two conversion rates	`proportions_ztest([60, 85], [500, 500])`
`sm.stats.multipletests`	Adjust many p-values	`multipletests(p, method="fdr_bh")`
`sm.stats.power.TTestIndPower`	Sample size or power planning	`TTestIndPower().solve_power(effect_size=0.3, power=0.8, alpha=0.05)`

Worked example 1: OLS with a full inference table

Problem 18.9.1 Regress y on x for 80 observations (true line y = 2 + 0.7 x). Report coefficients, standard errors, p-values, 95% intervals and R-squared. First the statsmodels code, which is **not run here**.

```
import numpy as np
import statsmodels.api as sm

rng = np.random.default_rng(11)
x = rng.uniform(0, 10, 80)
y = 2 + 0.7 * x + rng.normal(0, 1.0, 80)

X = sm.add_constant(x)            # adds a column of ones for the intercept
res = sm.OLS(y, X).fit()
print(res.summary())              # coefficients, std errors, t, p, CI, R-squared
print(res.params, res.bse, res.pvalues)
print(res.conf_int(alpha=0.05))
```

1. `sm.add_constant` is needed: unlike scikit-learn, statsmodels fits *no* intercept unless you add the column. Forgetting it is the classic statsmodels bug.
2. `sm.OLS(y, X)` takes the response first, then the design matrix (the reverse of scikit-learn's order for `fit(X, y)`).
3. `.fit()` returns a results object. `summary()` prints the full table; the attributes give each part as arrays.

The same numbers, computed by hand with NumPy and SciPy (this block ran):

```
import numpy as np
from scipy import stats

rng = np.random.default_rng(11)
x = rng.uniform(0, 10, 80)
y = 2 + 0.7 * x + rng.normal(0, 1.0, 80)

X = np.column_stack([np.ones_like(x), x])
n, k = X.shape
beta = np.linalg.solve(X.T @ X, X.T @ y)              # normal equations
resid = y - X @ beta
sigma2 = resid @ resid / (n - k)                      # unbiased noise variance
cov = sigma2 * np.linalg.inv(X.T @ X)
se = np.sqrt(np.diag(cov))
t = beta / se
p = 2 * stats.t.sf(np.abs(t), df=n - k)
tcrit = stats.t.ppf(0.975, n - k)
r2 = 1 - resid @ resid / ((y - y.mean()) @ (y - y.mean()))

for name, b, s, tv, pv in zip(["const", "x"], beta, se, t, p):
    print("%-5s coef=%.4f se=%.4f t=%.2f p=%.2g CI=(%.4f, %.4f)" % (name, b, s, tv, pv, b - tcrit * s, b + tcrit * s))
print("R-squared = %.4f" % r2)
```

```
const coef=2.0100 se=0.2054 t=9.78 p=3.3e-15 CI=(1.6010, 2.4190)
x     coef=0.7166 se=0.0391 t=18.35 p=5.1e-30 CI=(0.6389, 0.7944)
R-squared = 0.8119
```

1. The design matrix has a ones column and the x column, exactly what `add_constant` builds.
2. The least-squares coefficients solve (X^TX) beta = X^Ty. (In production prefer `lstsq`; this form keeps the algebra visible.)
3. The noise variance is the sum of squared residuals divided by n minus k (observations minus parameters); dividing by n would underestimate it.
4. The coefficient covariance matrix is that variance times the inverse of X^TX. Standard errors are square roots of its diagonal.
5. The t-statistic is coefficient over standard error, and the two-sided p-value comes from the t distribution with n minus k degrees of freedom.
6. The confidence interval is coefficient plus or minus a t quantile times the standard error. statsmodels reports these same quantities in `params`, `bse`, `pvalues` and `conf_int()`.

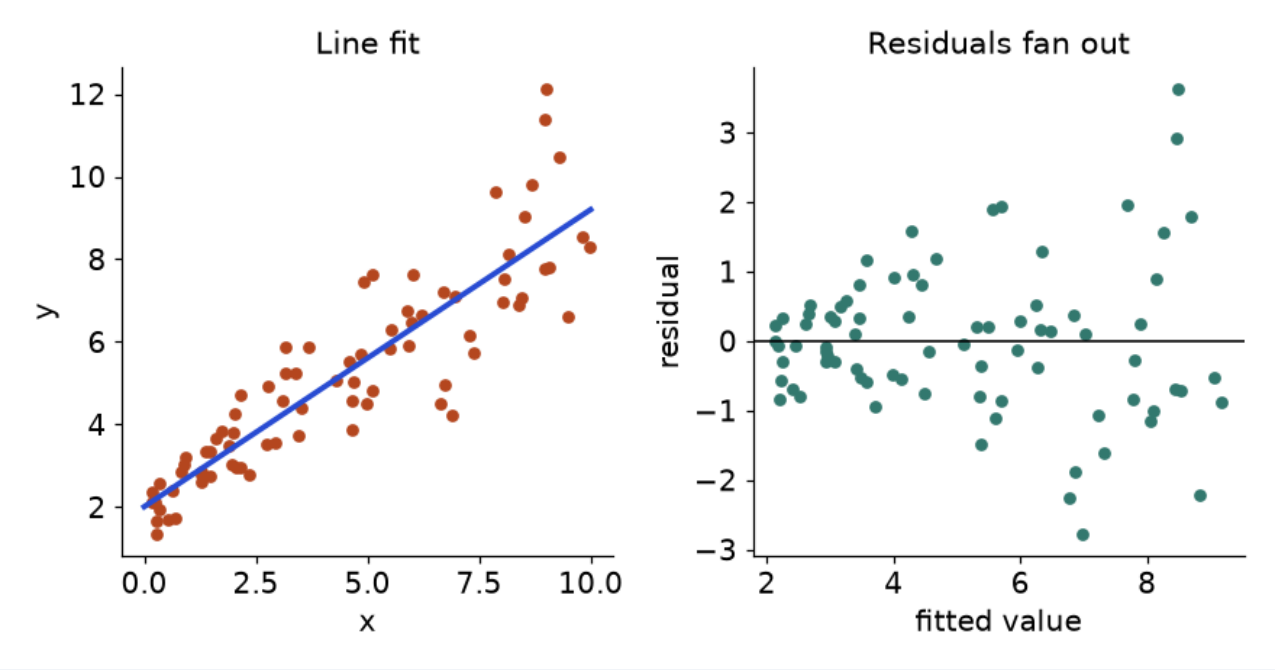
Worked example 2: uneven spread and robust standard errors

Problem 18.9.2 When the noise grows with x, ordinary standard errors are unreliable. See the fan shape in the residuals, then compute heteroscedasticity-robust (HC3) standard errors. statsmodels: `sm.OLS(y, X).fit(cov_type="HC3")` (not run here).

```
import numpy as np
from scipy import stats

rng = np.random.default_rng(11)
x = rng.uniform(0, 10, 80)
y = 2 + 0.7 * x + rng.normal(0, 0.4 + 0.12 * x)
X = np.column_stack([np.ones_like(x), x])
n, k = X.shape
XtX_inv = np.linalg.inv(X.T @ X)
beta = XtX_inv @ X.T @ y
resid = y - X @ beta

se_classic = np.sqrt(np.diag(resid @ resid / (n - k) * XtX_inv))
h = np.einsum("ij,jk,ik->i", X, XtX_inv, X)            # leverage of each row
w = resid / (1 - h)
meat = (X * w[:, None]).T @ (X * w[:, None])
se_hc3 = np.sqrt(np.diag(XtX_inv @ meat @ XtX_inv))
print("slope %.3f" % beta[1])
print("classic se: %.4f   robust HC3 se: %.4f" % (se_classic[1], se_hc3[1]))

fit = X @ beta
lo = np.abs(resid[fit < np.median(fit)]).mean()
hi = np.abs(resid[fit >= np.median(fit)]).mean()
print("mean |residual| low half: %.2f, high half: %.2f" % (lo, hi))
print("Spearman(fitted, |resid|) = %.2f" % stats.spearmanr(fit, np.abs(resid)).statistic)
```

```
slope 0.719
classic se: 0.0406   robust HC3 se: 0.0486
mean |residual| low half: 0.51, high half: 1.08
Spearman(fitted, |resid|) = 0.47
```

1. The true noise standard deviation is 0.4 plus 0.12 times x, so large x values are noisier.
2. The classic standard error assumes equal noise everywhere.
3. The HC3 "sandwich": the middle part (meat) weights each row by its own squared residual divided by (1 minus leverage) squared; bread and meat are multiplied as XtX_inv, meat, XtX_inv. The leverage `h` says how unusual a row is.
4. Two diagnostics expose the problem: residuals are larger in the high half of fitted values, and the rank correlation between fitted values and absolute residuals is clearly positive. This is what `het_breuschpagan` tests formally.



A straight-line fit (left) and its residuals (right). The fan shape means uneven spread.

How to read it: a healthy residual plot is a flat, even band around zero. Here the band widens to the right, so predictions at large x are less certain than the classic formula suggests. That is the cue to use robust errors, a weighted fit, or a transformed response.

Worked example 3: the formula API and a GLM

Problem 18.9.3 Daily support tickets depend on the temperature and whether it is a weekend. Counts are not normal, so use a Poisson GLM. Formulas let you write the model like a sentence: statsmodels code below (**not run here**).

```
import numpy as np
import pandas as pd
import statsmodels.api as sm
import statsmodels.formula.api as smf

rng = np.random.default_rng(0)
df = pd.DataFrame({"temp": rng.uniform(-5, 30, 300), "weekend": rng.integers(0, 2, 300)})
df["tickets"] = rng.poisson(np.exp(1.0 + 0.04 * df["temp"] - 0.5 * df["weekend"]))

ols = smf.ols("tickets ~ temp + C(weekend)", data=df).fit()          # formula API
glm = smf.glm("tickets ~ temp + weekend", data=df,
              family=sm.families.Poisson()).fit()
print(glm.summary())
print(np.exp(glm.params))          # rate ratios: multiplicative effects
```

1. `smf.ols("tickets ~ temp + C(weekend)", data=df)` says "tickets depends on temp and the category weekend". `C(...)` turns a column into dummy variables. The intercept is added automatically by formulas.
2. `family=sm.families.Poisson()` chooses the distribution of the response and, by default, a log link: the mean is exp(linear predictor).
3. Exponentiating the coefficients gives rate ratios: a coefficient of -0.5 on weekend means about 0.61 times as many tickets ($e^{-0.5}$).

The equivalent from first principles with SciPy (ran): maximise the Poisson log-likelihood, then take standard errors from the information matrix.

```
import numpy as np
from scipy import optimize, stats

rng = np.random.default_rng(0)
temp = rng.uniform(-5, 30, 300)
weekend = rng.integers(0, 2, 300)
tickets = rng.poisson(np.exp(1.0 + 0.04 * temp - 0.5 * weekend))
X = np.column_stack([np.ones(300), temp, weekend])

def nll(b):                                  # negative log-likelihood (constant dropped)
    eta = X @ b
    return np.sum(np.exp(eta) - tickets * eta)

res = optimize.minimize(nll, np.zeros(3), method="BFGS")
b = res.x
mu = np.exp(X @ b)
cov = np.linalg.inv(X.T @ (X * mu[:, None]))   # inverse Fisher information
se = np.sqrt(np.diag(cov))
z = b / se
for name, bi, si, zi in zip(["const", "temp", "weekend"], b, se, z):
    print("%-8s coef=%.3f se=%.3f z=%.1f p=%.3g" % (name, bi, si, zi, 2 * stats.norm.sf(abs(zi))))
print("rate ratio for weekend: %.2f" % np.exp(b[2]))
```

```
const    coef=1.024 se=0.066 z=15.5 p=3.2e-54
temp     coef=0.034 se=0.003 z=11.3 p=2.15e-29
weekend  coef=-0.418 se=0.059 z=-7.1 p=1.76e-12
rate ratio for weekend: 0.66
```

1. The truth is intercept 1.0, temperature 0.04, weekend -0.5. Counts come from `rng.poisson`.
2. For a Poisson model with a log link, the negative log-likelihood is the sum of exp(eta) minus y times eta, where eta is the linear predictor. Terms that do not depend on the coefficients are dropped.
3. `minimize` finds the maximum-likelihood coefficients. This is the same objective statsmodels optimises (it uses a method called IRLS and arrives at the same maximum).
4. The covariance is the inverse of X^T W X with W the fitted means; that is the Fisher information for a Poisson GLM with a log link. Standard errors and z-values follow.
5. The weekend rate ratio is exp of the weekend coefficient; the truth is $e^{-0.5}$, about 0.61, and sampling noise explains the small gap.

Worked example 4: ARIMA, and an AR(1) computed by hand

Problem 18.9.4 A daily metric follows an AR(1) process: today = 0.7 times yesterday plus noise. Fit it and forecast the next 5 days. statsmodels (**not run here**):

```
import numpy as np
from statsmodels.tsa.arima.model import ARIMA

rng = np.random.default_rng(5)
y = np.zeros(300)
for i in range(1, 300):
    y[i] = 0.7 * y[i - 1] + rng.normal()

res = ARIMA(y, order=(1, 0, 0)).fit()        # (p, d, q) = (1, 0, 0), constant included
print(res.summary())
print(res.forecast(steps=5))
```

1. `order=(p, d, q)` means p lagged values (AR), d differences to remove a trend, q lagged errors (MA). AR(1) is (1, 0, 0).
2. `.fit()` estimates by maximum likelihood; the summary lists the AR coefficient and noise variance with standard errors.

3. `.forecast(steps=5)` predicts the next five values, which decay toward the series mean.

The same model, by regression on the lagged series (ran):

```
import numpy as np

rng = np.random.default_rng(5)
y = np.zeros(300)
for i in range(1, 300):
    y[i] = 0.7 * y[i - 1] + rng.normal()

ybar = y.mean()
z = y - ybar
phi = (z[1:] @ z[:-1]) / (z[:-1] @ z[:-1])      # least-squares AR(1) coefficient
sigma = np.std(z[1:] - phi * z[:-1], ddof=1)
print("phi = %.3f  noise std = %.3f" % (phi, sigma))

forecast, last = [], z[-1]
for h in range(5):
    last = phi * last
    forecast.append(ybar + last)
print("forecast:", np.round(forecast, 3))
print("last observed value: %.3f, series mean: %.3f" % (y[-1], ybar))
```

```
phi = 0.715  noise std = 0.960
forecast: [-1.439 -1.067 -0.8   -0.61  -0.473]
last observed value: -1.960, series mean: -0.130
```

1. The loop builds an AR(1) series: each value is 0.7 times the previous plus standard normal noise.
2. Subtract the mean, then regress today on yesterday through the origin; the coefficient is the sum of products over the sum of squares. It should be near the true 0.7.
3. The noise standard deviation is that of the one-step prediction errors.
4. Forecasting multiplies the last deviation by phi once per step, so the forecast shrinks geometrically toward the mean, exactly the decay you would see from `res.forecast`.

COMMON MISTAKE

Running ARIMA on a series that trends or has a changing level without differencing. Check stationarity first (`adfuller` , or look at the plot), or set `d=1` . A model fitted to a trending series gives confident-looking but meaningless coefficients.

18.10 matplotlib Quick Reference for Plotting Results

This is only a cheat sheet for plotting the results in this chapter. The full visualisation guide (chart choice, design, storytelling) is in the Data Analytics document, Chapter 8. Always use the **object-oriented style**: `fig, ax = plt.subplots()` , then call methods on `ax` . It scales to many panels and avoids the confusion of hidden "current figure" state.

Name	What it does in plain English	Minimal example
`plt.subplots(r, c, figsize)`	Make a figure and a grid of axes	`fig, ax = plt.subplots(figsize=(8, 4))`
`ax.plot(x, y)`	Line	`ax.plot(x, y, lw=2, label="fit")`
`ax.scatter(x, y)`	Points, optionally sized or coloured	`ax.scatter(x, y, s=20, c=z)`
`ax.hist(x, bins, density)`	Histogram; `density=True` to compare with a pdf	`ax.hist(x, bins=40, density=True)`
`ax.bar(x, h)`	Bars for categories	`ax.bar(names, values)`

Name	What it does in plain English	Minimal example
`ax.errorbar`	Points with error bars (estimates with CIs)	`ax.errorbar(x, y, yerr=se, fmt="o")`
`ax.fill_between`	Shaded band (uncertainty)	`ax.fill_between(x, lo, hi, alpha=0.2)`
`ax.axhline / axvline`	Reference line (zero, mean, threshold)	`ax.axhline(0, color="black")`
`ax.contour / contourf`	Level lines of a two-variable function	`ax.contour(X, Y, Z)`
`ax.imshow`	Matrix as an image (heat map)	`ax.imshow(M, cmap="Blues")`
`ax.set_xlabel / set_ylabel / set_title`	Label everything, always	`ax.set_xlabel("hours")`
`ax.set_xscale("log")`	Log axis for skewed or multiplicative data	`ax.set_yscale("log")`
`ax.set_xlim / set_ylim`	Zoom to a range	`ax.set_xlim(0, 60)`
`ax.legend(frameon=False)`	Show labels given in `label=`	`ax.legend()`
`ax.annotate`	Text with an arrow at a point	`ax.annotate("peak", (x0, y0), xytext=(x1, y1))`
`fig.tight_layout()`	Stop labels overlapping	`fig.tight_layout()`
`fig.savefig(path, dpi)`	Write PNG/SVG/PDF	`fig.savefig("a.png", dpi=110)`
`stats.probplot(x, plot=ax)`	Q-Q plot for normality checks	`stats.probplot(x, plot=ax)`
`matplotlib.use("Agg")`	Render without a screen (servers, scripts)	`matplotlib.use("Agg")`
`plt.close(fig)`	Free memory in loops	`plt.close(fig)`

Problem 18.10.1 Plot a Q-Q plot, a histogram with the fitted density, and a means-with-intervals chart in one figure, then save it to memory.

```
import io
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

rng = np.random.default_rng(7)
data = rng.lognormal(1.2, 0.5, 400)
s, loc, scale = stats.lognorm.fit(data, floc=0)

fig, axes = plt.subplots(1, 3, figsize=(12, 3.6))
stats.probplot(np.log(data), dist="norm", plot=axes[0])
axes[0].set_title("Q-Q plot of log(data)")

axes[1].hist(data, bins=30, density=True, alpha=0.5)
xs = np.linspace(0, data.max(), 200)
axes[1].plot(xs, stats.lognorm.pdf(xs, s, loc, scale), lw=2)
axes[1].set_title("histogram + fitted pdf")

groups = [rng.normal(m, 2, 30) for m in (10, 11, 13)]
means = [g.mean() for g in groups]
errs = [1.96 * stats.sem(g) for g in groups]
axes[2].errorbar([0, 1, 2], means, yerr=errs, fmt="o", capsize=4)
axes[2].set_xticks([0, 1, 2], ["A", "B", "C"])
axes[2].set_title("means with 95% CI")

fig.tight_layout()
buf = io.BytesIO()
fig.savefig(buf, format="png", dpi=80)
plt.close(fig)
print("panels:", len(axes), " png bytes > 5000:", buf.tell() > 5000)
print("x tick labels of last panel:", [t.get_text() for t in axes[2].get_xticklabels()])
```

```
panels: 3  png bytes > 5000: True
x tick labels of last panel: ['A', 'B', 'C']
```

1. `plt.subplots(1, 3, figsize=(12, 3.6))` returns the figure and an array of three axes; all later calls go through `axes[i]`.
2. `stats.probplot(..., plot=axes[0])` draws a Q-Q plot on that axis. Points on the straight line mean the data matches the distribution; here we test the *log* of the data against a normal.
3. The histogram uses `density=True` so its area is 1 and the fitted pdf can be overlaid on the same scale.
4. `errorbar` draws a mean as a dot and 1.96 standard errors as whiskers. `set_xticks(positions, labels)` names the groups.
5. Saving to a `BytesIO` buffer is how you embed a figure in a web page or report without writing a file, and `plt.close(fig)` frees its memory.

18.11 Recipes: How Do I ...?

Each recipe is a complete, standalone answer to a real task. Copy it, change the data, and keep the verification line.

How do I choose between several candidate distributions? (Recipe 1)

```
import numpy as np
from scipy import stats

rng = np.random.default_rng(10)
data = rng.gamma(shape=3.0, scale=2.0, size=500)

rows = []
for name in ["norm", "lognorm", "gamma", "weibull_min", "expon"]:
    dist = getattr(stats, name)
    params = dist.fit(data, floc=0) if name != "norm" else dist.fit(data)
    ll = dist.logpdf(data, *params).sum()
    aic = 2 * len(params) - 2 * ll
    rows.append((aic, name))
for aic, name in sorted(rows):
    print("%-12s AIC = %.1f" % (name, aic))
```

```
gamma        AIC = 2544.8
weibull_min  AIC = 2562.7
lognorm      AIC = 2565.1
norm         AIC = 2671.7
expon        AIC = 2795.8
```

1. The data is gamma by construction, so we know which should win.
2. For each candidate, `getattr(stats, name)` fetches the distribution, `fit` estimates its parameters (with location fixed at 0 for positive data), and the AIC scores it.
3. Lower AIC is better. Differences under about 2 are a tie; large gaps are decisive. Always follow with a plot (histogram plus pdf) before trusting the winner.

How do I run an A/B test on conversion rates and plan the sample size? (Recipe 2)

```
import numpy as np
from scipy import stats

conv_a, n_a = 120, 2000
conv_b, n_b = 156, 2000
pa, pb = conv_a / n_a, conv_b / n_b
pool = (conv_a + conv_b) / (n_a + n_b)
se = np.sqrt(pool * (1 - pool) * (1 / n_a + 1 / n_b))
z = (pb - pa) / se
print("rates %.3f vs %.3f  z = %.2f  p = %.4f" % (pa, pb, z, 2 * stats.norm.sf(abs(z))))

se_diff = np.sqrt(pa * (1 - pa) / n_a + pb * (1 - pb) / n_b)
lo, hi = (pb - pa) + np.array([-1, 1]) * stats.norm.ppf(0.975) * se_diff
print("lift 95%% CI: (%.4f, %.4f)" % (lo, hi))

# sample size per group to detect 6% -> 7.8% with 80% power, 5% two-sided
p1, p2 = 0.06, 0.078
zb = stats.norm.ppf(0.80) + stats.norm.ppf(0.975)
n = zb ** 2 * (p1 * (1 - p1) + p2 * (1 - p2)) / (p2 - p1) ** 2
print("needed per group:", int(np.ceil(n)))
```

```
rates 0.060 vs 0.078  z = 2.25  p = 0.0247
lift 95% CI: (0.0023, 0.0337)
needed per group: 3109
```

1. The two-proportion z-test pools both groups to estimate the standard error under the null (no difference).

2. The confidence interval for the lift uses the unpooled standard error, because we no longer assume the groups are equal.
3. The planning formula says: sample more when the effect is small or the rates are close to 50%. Do this *before* the test, never after. statsmodels' `proportions_ztest` and `NormalIndPower` do the same (not run here).

How do I get a prediction band from a curve fit? (Recipe 3)

```
import numpy as np
from scipy import optimize, stats

def model(t, a, k):
    return a * (1 - np.exp(-k * t))

rng = np.random.default_rng(2)
t = np.linspace(0.2, 8, 25)
y = model(t, 10, 0.6) + rng.normal(0, 0.3, t.size)
p, cov = optimize.curve_fit(model, t, y, p0=[5, 1])

t_new = 6.0
eps = 1e-6
grad = np.array([(model(t_new, *(p + eps * np.eye(2)[i])) - model(t_new, *p)) / eps for i in range(2)])
se_mean = np.sqrt(grad @ cov @ grad)                    # delta method
s_resid = np.sqrt(np.sum((y - model(t, *p)) ** 2) / (t.size - 2))
tc = stats.t.ppf(0.975, t.size - 2)
fit = model(t_new, *p)
print("fit at t=6: %.2f" % fit)
print("95%% CI of the curve : (%.2f, %.2f)" % (fit - tc * se_mean, fit + tc * se_mean))
print("95%% prediction range: (%.2f, %.2f)" % (fit - tc * np.hypot(se_mean, s_resid), fit + tc * np.hypot(se_mean,
s_resid)))
```

```
fit at t=6: 9.73
95% CI of the curve : (9.58, 9.87)
95% prediction range: (9.16, 10.29)
```

1. The delta method turns parameter uncertainty (`cov`) into uncertainty of the curve at one point using the gradient of the model with respect to the parameters.
2. The curve interval answers "where is the true average curve?". The prediction interval adds the scatter of individual points (`s_resid`) and is wider. Use the second to warn about a single future measurement.

How do I pick the best items under a budget? (Recipe 4, integer optimisation)

```
import numpy as np
from scipy import optimize

value = np.array([60, 100, 120, 80, 30.0])        # expected profit per campaign
cost = np.array([10, 20, 30, 25, 5.0])            # budget needed
budget = 55
res = optimize.milp(c=-value,
                    constraints=optimize.LinearConstraint(cost[None, :], ub=budget),
                    integrality=np.ones(5), bounds=optimize.Bounds(0, 1))
chosen = np.round(res.x).astype(int)
print("chosen campaigns:", chosen, " profit:", int(value @ chosen), " cost:", int(cost @ chosen))
```

```
chosen campaigns: [0 1 1 0 1]  profit: 250  cost: 55
```

1. This is a knapsack: pick items (0 or 1 each) to maximise value within a budget. `integrality=1` forces whole numbers and the bounds 0 to 1 make them yes/no.
2. `milp` minimises, so we pass the negated values.

3. The check is arithmetic: cost 55 or less, and no other combination beats the profit (here the search space has only 32 combinations, so you could confirm by brute force).

How do I get a permutation test and a bootstrap interval for a correlation? (Recipe 5)

```
import numpy as np
from scipy import stats

rng = np.random.default_rng(6)
x = rng.normal(size=40)
y = 0.4 * x + rng.normal(size=40)

stat = lambda a, b: np.corrcoef(a, b)[0, 1]
perm = stats.permutation_test((x, y), stat, permutation_type="pairings",
                              n_resamples=5000, random_state=1)
print("r = %.3f  permutation p = %.4f  (pearsonr p = %.4f)" % (perm.statistic, perm.pvalue, stats.pearsonr(x, y).pvalue))

boot = stats.bootstrap((x, y), lambda a, b: np.corrcoef(a, b)[0, 1], paired=True,
                       vectorized=False, n_resamples=2000, random_state=1)
print("bootstrap 95%% CI for r: (%.3f, %.3f)" % (boot.confidence_interval.low, boot.confidence_interval.high))
```

```
r = 0.470  permutation p = 0.0008  (pearsonr p = 0.0022)
bootstrap 95% CI for r: (0.184, 0.673)
```

1. `permutation_type="pairings"` shuffles y relative to x many times; if the observed r is rare among shuffled values, it is unlikely to be chance.
2. The permutation p-value should be close to the formula-based `pearsonr` p-value, a nice agreement check.
3. The bootstrap resamples (x, y) *pairs* together (`paired=True`) and recomputes r each time. `vectorized=False` tells SciPy the statistic handles one resample at a time.

How do I flag outliers robustly?

```
import numpy as np
from scipy import stats

rng = np.random.default_rng(3)
x = np.append(rng.normal(100, 5, 200), [160, 40])
z = stats.zscore(x)
mad = stats.median_abs_deviation(x, scale="normal")
robust_z = (x - np.median(x)) / mad
print("classic |z|>3 count:", int((np.abs(z) > 3).sum()))
print("robust  |z|>3 count:", int((np.abs(robust_z) > 3).sum()))
print("flagged values:", np.sort(x[np.abs(robust_z) > 3]).round(1))
```

```
classic |z|>3 count: 2
robust  |z|>3 count: 3
flagged values: [ 40.  116.6 160. ]
```

1. The classic z-score uses mean and standard deviation, both distorted by the very outliers you hunt (masking).
2. The median and the median absolute deviation (MAD, scaled to match a standard deviation for normal data) barely move, so the robust score is sharper.

How do I fill gaps in a time series?

```
import numpy as np
import pandas as pd
from scipy import interpolate

s = pd.Series([10.0, 12.0, np.nan, np.nan, 18.0, 17.0, np.nan, 14.0],
              index=pd.date_range("2025-01-01", periods=8, freq="D"))
known = s.dropna()
xk = (known.index - s.index[0]).days.to_numpy()
pchip = interpolate.PchipInterpolator(xk, known.to_numpy())
filled = pd.Series(pchip((s.index - s.index[0]).days), index=s.index).round(2)
print(filled.tolist())
print("pandas linear:", s.interpolate().round(2).tolist())
```

```
[10.0, 12.0, 14.44, 16.89, 18.0, 17.0, 15.66, 14.0]
pandas linear: [10.0, 12.0, 14.0, 16.0, 18.0, 17.0, 15.5, 14.0]
```

1. Convert dates to numbers (days since the start), because SciPy interpolators need numeric x.
2. Fit PCHIP only to the known points, then evaluate it on every date. PCHIP does not overshoot, which matters for non-negative quantities.
3. For plain linear filling, `Series.interpolate()` in pandas is enough.

How do I match drivers to orders at the lowest total distance?

```
import numpy as np
from scipy import optimize
from scipy.spatial import distance

drivers = np.array([[0, 0], [5, 5], [9, 1.0]])
orders = np.array([[8, 2], [1, 1], [6, 6.0]])
cost = distance.cdist(drivers, orders)
rows, cols = optimize.linear_sum_assignment(cost)
print("driver -> order:", list(zip(rows.tolist(), cols.tolist())))
print("total distance: %.2f" % cost[rows, cols].sum())
print("greedy first choice would pair driver 0 with order", int(cost[0].argmin()))
```

```
driver -> order: [(0, 1), (1, 2), (2, 0)]
total distance: 4.24
greedy first choice would pair driver 0 with order 1
```

1. `cdist` builds the cost matrix: one row per driver, one column per order.
2. `linear_sum_assignment` finds the one-to-one matching with minimum total cost (the Hungarian algorithm); it is exact, unlike matching greedily driver by driver.

How do I find the nearest warehouse from latitude and longitude?

```
import numpy as np
from scipy.spatial import KDTree

def to_xyz(lat, lon):
    lat, lon = np.radians(lat), np.radians(lon)
    return np.column_stack([np.cos(lat) * np.cos(lon), np.cos(lat) * np.sin(lon), np.sin(lat)])

wh = {"Rotterdam": (51.92, 4.48), "Berlin": (52.52, 13.40), "Madrid": (40.42, -3.70), "Milan": (45.46, 9.19)}
names = list(wh)
tree = KDTree(to_xyz(*np.array(list(wh.values())).T))
d, i = tree.query(to_xyz(48.86, 2.35))                # Paris
print("nearest to Paris:", names[int(i[0])])
print("chord distance %.4f -> about %.0f km" % (d[0], 6371 * 2 * np.arcsin(d[0] / 2)))
```

```
nearest to Paris: Rotterdam
chord distance 0.0584 -> about 372 km
```

1. Latitude/longitude are angles on a sphere, so straight-line distance in degrees is wrong (a degree of longitude shrinks toward the poles). Convert to 3-D points on the unit sphere first.
2. Distance through the sphere (the chord) ranks neighbours in the same order as distance along the surface. Convert a chord *c* to kilometres with 2 R arcsin(c/2), where R = 6371 km.

How do I find the break-even point of a profit curve?

```
import numpy as np
from scipy import optimize

price = lambda q: 50 * np.exp(-0.01 * q)          # price falls as quantity rises
profit = lambda q: q * price(q) - (400 + 12 * q)
low = optimize.root_scalar(profit, bracket=[1, 20])
best = optimize.minimize_scalar(lambda q: -profit(q), bounds=(1, 200), method="bounded")
print("break-even at q = %.2f" % low.root)
print("max profit %.1f at q = %.1f" % (-best.fun, best.x))
```

```
break-even at q = 12.44
max profit 527.8 at q = 57.4
```

1. Break-even is where profit equals zero: a root. The bracket [1, 20] must contain a sign change (profit is negative at 1, positive at 20).
2. The best quantity is a one-variable maximisation; negate to use a minimiser.

How do I compute AUC or any area from a curve?

```
import numpy as np
from scipy import integrate

fpr = np.array([0, 0.1, 0.3, 0.6, 1.0])
tpr = np.array([0, 0.5, 0.75, 0.9, 1.0])
print("AUC by trapezoid: %.4f" % integrate.trapezoid(tpr, fpr))
print("random-guess AUC: %.4f" % integrate.trapezoid(fpr, fpr))
```

```
AUC by trapezoid: 0.7775
random-guess AUC: 0.5000
```

1. Values go first, positions second: `trapezoid(y, x)`. The area under a ROC curve is exactly this trapezoid sum.
2. The diagonal (a coin flip) must give 0.5; always run that sanity check.

How do I test every pair of columns for correlation?

```
import numpy as np
import pandas as pd
from scipy import stats

rng = np.random.default_rng(9)
df = pd.DataFrame({"a": rng.normal(size=60)})
df["b"] = df["a"] * 0.8 + rng.normal(size=60)
df["c"] = rng.normal(size=60)
res = stats.spearmanr(df)                    # a table in, a matrix out
print(pd.DataFrame(res.statistic, index=df.columns, columns=df.columns).round(2))
print("p(a,b) = %.2g   p(a,c) = %.2f" % (res.pvalue[0, 1], res.pvalue[0, 2]))
```

```
      a     b     c
a  1.00  0.54 -0.19
b  0.54  1.00 -0.12
c -0.19 -0.12  1.00
p(a,b) = 8.7e-06   p(a,c) = 0.14
```

1. Passing a whole DataFrame to `spearmanr` returns a correlation matrix and a matching p-value matrix. `df.corr(method="spearman")` gives the same correlations without p-values.
2. With many columns, apply a multiple-testing correction to the off-diagonal p-values.

18.12 Performance and Pitfalls

Speed rules that matter

- **Vectorise.** Pass arrays to `cdf`, `pdf`, `interp`, `cdist`. Per-call overhead of SciPy distribution objects is large for scalars.
- **Choose the right structure.** A KDTree beats brute force for neighbours; a sparse matrix beats dense when over 90% of cells are zero; `sosfiltfilt` beats a Python loop.
- **Factor once.** Solving many right-hand sides with the same matrix: `lu_factor` once, `lu_solve` many times.
- **Use `workers`.** Several functions accept `workers=-1` to use all cores: `KDTree.query`, `fft` functions, `differential_evolution`.
- **Give gradients.** If you can write the derivative, pass `jac=` to `minimize`; numerical differences cost one extra call per variable.

```
import time
import numpy as np
from scipy import stats

x = np.random.default_rng(0).normal(size=20000)
t0 = time.perf_counter()
slow = [stats.norm.cdf(v) for v in x]
t_loop = time.perf_counter() - t0
t0 = time.perf_counter()
fast = stats.norm.cdf(x)
t_vec = time.perf_counter() - t0
print("same values:", np.allclose(slow, fast))
print("vectorised at least 20x faster:", t_loop > 20 * t_vec)
```

```
same values: True
vectorised at least 20x faster: True
```

1. The loop calls `cdf` 20,000 times, paying input checking each time. The vectorised call checks once and computes in compiled code.
2. We print a comparison rather than seconds, since timings differ by machine; on this machine the gap was well above 20 times.

Common mistakes with symptom and fix

COMMON MISTAKE 1: OFF-BY-ONE ON DISCRETE TAILS

Wrong: `sf(10)` to get "10 or more". Symptom: the probability is too small because exactly 10 is excluded. Fix: `sf(k - 1)` for "k or more".

```
from scipy import stats
d = stats.poisson(4)
print("wrong  sf(10):", round(d.sf(10), 5), "  right sf(9):", round(d.sf(9), 5), "  by pmf sum:",
round(sum(d.pmf(k) for k in range(10, 60)), 5))
```

```
wrong  sf(10): 0.00284   right sf(9): 0.00813   by pmf sum: 0.00813
```

COMMON MISTAKE 2: STUDENT T-TEST WITH UNEQUAL SPREAD

Wrong: `ttest_ind(a, b)` assumes equal variances. Symptom: p-values drift when group sizes and spreads differ. Fix: `equal_var=False` (Welch) by default.

```
import numpy as np
from scipy import stats
rng = np.random.default_rng(0)
a = rng.normal(0, 1, 10); b = rng.normal(0, 6, 100)
print("student p = %.3f   welch p = %.3f" % (stats.ttest_ind(a, b).pvalue, stats.ttest_ind(a, b,
equal_var=False).pvalue))
```

```
student p = 0.906   welch p = 0.729
```

COMMON MISTAKE 3: CURVE_FIT WITHOUT A STARTING GUESS

Wrong: default `p0` of all ones for an oscillating model. Symptom: no error, but a terrible fit stuck in a local minimum. Fix: give `p0` from a plot or a quick estimate, and always check the residuals.

```
import numpy as np
from scipy import optimize
rng = np.random.default_rng(0)
t = np.linspace(0, 10, 100); y = 2 * np.sin(3 * t) + rng.normal(0, 0.1, 100)
f = lambda t, a, w: a * np.sin(w * t)
for p0 in (None, [1, 2.9]):
    p, _ = optimize.curve_fit(f, t, y, p0=p0)
    print("p0=%-10s params=%s rmse=%.3f" % (p0, p.round(2), np.sqrt(np.mean((y - f(t, *p)) ** 2))))
```

```
p0=None       params=[0.05 1.02] rmse=1.442
p0=[1, 2.9]   params=[2.03 3.  ] rmse=0.094
```

COMMON MISTAKE 4: LOGNORM.FIT WITHOUT FLOC=0

Wrong: letting the location float. Symptom: a third parameter that shifts the curve, unstable shape and scale. Fix: `floc=0` for quantities that start at zero.

```
import numpy as np
from scipy import stats
d = np.random.default_rng(0).lognormal(1.2, 0.5, 200)
print("free loc  :", np.round(stats.lognorm.fit(d), 3))
print("floc=0    :", np.round(stats.lognorm.fit(d, floc=0), 3))
```

```
free loc  : [ 0.434 -0.322  3.703]
floc=0    : [0.481 0.    3.346]
```

COMMON MISTAKE 5: NP.INTERP WITH UNSORTED X

Wrong: x not increasing. Symptom: silently wrong answers, no error. Fix: sort first.

```
import numpy as np
x = np.array([3, 1, 2.0]); y = np.array([30, 10, 20.0])
print("wrong:", np.interp(1.5, x, y), "  right:", np.interp(1.5, *zip(*sorted(zip(x, y)))))
```

```
wrong: 30.0   right: 15.0
```

COMMON MISTAKE 6: * MEANS DIFFERENT THINGS FOR SPARSE MATRIX AND ARRAY

Wrong: assuming `A * B` is matrix multiplication for every sparse type. Symptom: elementwise result from `csr_array`, matrix product from `csr_matrix`. Fix: use `@` always.

```
import numpy as np
from scipy import sparse
d = np.array([[1, 2], [3, 4]])
print("csr_matrix A*A:", (sparse.csr_matrix(d) * sparse.csr_matrix(d)).toarray().tolist())
print("csr_array  A*A:", (sparse.csr_array(d) * sparse.csr_array(d)).toarray().tolist())
print("csr_array  A@A:", (sparse.csr_array(d) @ sparse.csr_array(d)).toarray().tolist())
```

```
csr_matrix A*A: [[7, 10], [15, 22]]
csr_array  A*A: [[1, 4], [9, 16]]
csr_array  A@A: [[7, 10], [15, 22]]
```

COMMON MISTAKE 7: INVERTING A MATRIX TO SOLVE A SYSTEM

Wrong: `inv(A) @ b`. Symptom: larger rounding error, especially for ill-conditioned A. Fix: `solve(A, b)`.

```
import numpy as np
from scipy import linalg
H = linalg.hilbert(10); truth = np.ones(10); b = H @ truth
err_inv = np.abs(np.linalg.inv(H) @ b - truth).max()
err_solve = np.abs(linalg.solve(H, b) - truth).max()
print("inverse error is larger than solve error:", err_inv > err_solve)
```

```
inverse error is larger than solve error: True
```

COMMON MISTAKE 8: FFT FREQUENCY AXIS WITHOUT THE SAMPLING STEP

Wrong: `fftfreq(n)` alone. Symptom: the 5 Hz wave shows up at 0.025 "something", meaningless units. Fix: pass the spacing `d = 1/fs`.

```
import numpy as np
from scipy import fft
fs = 200; t = np.arange(0, 2, 1 / fs); x = np.sin(2 * np.pi * 5 * t)
k = np.argmax(np.abs(fft.rfft(x)))
print("wrong:", fft.rfftfreq(t.size)[k], "  right:", fft.rfftfreq(t.size, 1 / fs)[k], "Hz")
```

```
wrong: 0.025   right: 5.0 Hz
```

COMMON MISTAKE 9: A TINY P-VALUE FOR A TINY EFFECT

Wrong: reporting only p. Symptom: with 200,000 rows per group even a 0.01 standard-deviation difference is "significant". Fix: report the effect size and interval, and ask whether the size matters.

```
import numpy as np
from scipy import stats
rng = np.random.default_rng(0)
a = rng.normal(0, 1, 200000); b = rng.normal(0.01, 1, 200000)
r = stats.ttest_ind(a, b)
print("p = %.4f   difference = %.4f standard deviations" % (r.pvalue, abs(a.mean() - b.mean())))
```

```
p = 0.0015   difference = 0.0101 standard deviations
```

COMMON MISTAKE 10: UNSEEDED RANDOMNESS

Wrong: `rvs(size)` or `bootstrap` with no seed. Symptom: every run gives different numbers, so results cannot be reviewed or tested. Fix: pass `random_state=` (an integer or a Generator).

```
import numpy as np
from scipy import stats
a = stats.norm.rvs(size=3, random_state=42); b = stats.norm.rvs(size=3, random_state=42)
c = stats.norm.rvs(size=3); d = stats.norm.rvs(size=3)
print("seeded runs identical:", np.array_equal(a, b), "  unseeded identical:", np.array_equal(c, d))
```

```
seeded runs identical: True   unseeded identical: False
```

COMMON MISTAKE 11: WRITING INTO A PANDAS ARRAY (PANDAS 3 COPY-ON-WRITE)

Wrong: `a = df["x"].to_numpy(); a[0] = 9` to "fix" a value before a SciPy call. Symptom: `ValueError: assignment destination is read-only`, and the DataFrame is never modified by accident. Fix: take an explicit `.copy()`, or use `df.loc` to change the table.

```
import pandas as pd
df = pd.DataFrame({"x": [1.0, 2.0, 3.0]})
a = df["x"].to_numpy()
try:
    a[0] = 9
except ValueError as e:
    print("error:", e)
a = df["x"].to_numpy().copy(); a[0] = 9
print("copy edited:", a.tolist(), " table unchanged:", df["x"].tolist())
```

```
error: assignment destination is read-only
copy edited: [9.0, 2.0, 3.0]  table unchanged: [1.0, 2.0, 3.0]
```

COMMON MISTAKE 12: K-MEANS ON UNSCALED COLUMNS

Wrong: clustering income (tens of thousands) with age (tens). Symptom: clusters follow only the large-scale column. Fix: `vq.whiten` or standardise first.

```
import numpy as np
from scipy.cluster import vq
rng = np.random.default_rng(0)
age = np.concatenate([rng.normal(25, 3, 100), rng.normal(55, 3, 100)])   # two clear age groups
income = rng.normal(50000, 15000, 200)                                    # unrelated, huge scale
X = np.column_stack([age, income])
for name, data in [("raw", X), ("whitened", vq.whiten(X))]:
    _, lab = vq.kmeans2(data, 2, seed=1, minit="++")
    truth = np.arange(200) >= 100
    agree = max(np.mean(lab == truth), np.mean(lab != truth))
    print("%-9s agreement with the age groups: %.2f" % (name, agree))
```

```
raw       agreement with the age groups: 0.56
whitened  agreement with the age groups: 1.00
```

18.13 Cross-Library Integration

The libraries share NumPy arrays, so integration is mostly about moving data cleanly. Three patterns cover most real work.

Pandas to SciPy: one test per group

```
import numpy as np
import pandas as pd
from scipy import stats

rng = np.random.default_rng(12)
df = pd.DataFrame({
    "city": np.repeat(["Oslo", "Lima", "Pune"], 60),
    "variant": np.tile(["control", "new"], 90),
    "minutes": np.concatenate([rng.normal(m, 6, 60) for m in (30, 34, 28)]),
})
df.loc[df["variant"] == "new", "minutes"] -= 3.0

rows = []
for city, g in df.groupby("city"):
    a = g.loc[g["variant"] == "control", "minutes"]
    b = g.loc[g["variant"] == "new", "minutes"]
    r = stats.ttest_ind(b, a, equal_var=False)
    rows.append({"city": city, "diff": b.mean() - a.mean(), "p": r.pvalue})
out = pd.DataFrame(rows)
out["p_adj"] = stats.false_discovery_control(out["p"])
print(out.round(3).to_string(index=False))
print("city dtype:", df["city"].dtype)
```

```
city   diff     p  p_adj
Lima -3.415 0.021  0.036
Oslo -3.383 0.024  0.036
Pune -2.228 0.157  0.157
city dtype: str
```

1. The table has a text column (`city`), a group column and a numeric column. In pandas 3 the text column has the dedicated `str` dtype, shown on the last line.
2. Pandas Series go straight into SciPy; they are converted to NumPy arrays internally. We never loop over rows, only over the 3 groups.
3. Collecting results into a list of dictionaries and building one DataFrame at the end is the clean pattern (appending rows to a DataFrame inside a loop is slow).
4. With three simultaneous tests we adjust the p-values with `false_discovery_control`.

SciPy to scikit-learn: distributions as search spaces, sparse matrices as features

```
import numpy as np
from scipy import stats, sparse
from sklearn.linear_model import Ridge
from sklearn.model_selection import RandomizedSearchCV

rng = np.random.default_rng(0)
X = sparse.random_array((400, 50), density=0.1, rng=rng, format="csr")
w = rng.normal(size=50)
y = X @ w + rng.normal(0, 0.1, 400)

search = RandomizedSearchCV(Ridge(), {"alpha": stats.loguniform(1e-3, 1e2)},
                            n_iter=12, cv=3, random_state=0)
search.fit(X, y)
print("input is sparse:", sparse.issparse(X), " nnz =", X.nnz)
print("best alpha: %.4f   cv R2: %.3f" % (search.best_params_["alpha"], search.best_score_))
```

```
input is sparse: True  nnz = 2000
best alpha: 0.0826   cv R2: 0.988
```

1. A scikit-learn estimator accepts a SciPy sparse matrix directly, so a text-feature matrix never needs to be made dense.
2. `stats.loguniform(1e-3, 1e2)` is a frozen SciPy distribution. `RandomizedSearchCV` calls its `rvs` method, so any distribution from section 18.2 can define a search range. Log-uniform is right for scale-like settings such as regularisation strengths.
3. The best settings and scores are the search result; the exact alpha depends on the random draws, which are seeded.

SciPy with statsmodels and joblib

statsmodels (not run here) accepts NumPy arrays and pandas DataFrames, and its results can be post-processed with SciPy: for instance `stats.t.ppf` for custom intervals, or `stats.probplot(res.resid, plot=ax)` for a residual Q-Q plot. To spread many independent bootstraps over cores, wrap the work with `joblib.Parallel`; give each task its own seed so results stay reproducible.

```
import numpy as np
from joblib import Parallel, delayed

data = np.random.default_rng(0).lognormal(1.2, 0.5, 300)

def boot_median(seed):
    r = np.random.default_rng(seed)
    return np.median(r.choice(data, size=data.size, replace=True))

meds = Parallel(n_jobs=2)(delayed(boot_median)(s) for s in range(400))
print("n resamples:", len(meds), " 95%% interval: (%.3f, %.3f)" % tuple(np.percentile(meds, [2.5, 97.5])))
again = Parallel(n_jobs=2)(delayed(boot_median)(s) for s in range(400))
print("reproducible:", meds == again)
```

```
n resamples: 400  95% interval: (2.990, 3.467)
reproducible: True
```

1. Each job gets a different integer seed (0 to 399), so the 400 resamples differ from each other but the whole run repeats exactly.
2. `Parallel(n_jobs=2)` runs tasks in separate worker processes; for work this light, the start-up cost can exceed the gain. Parallelise heavy tasks, not tiny ones.

18.14 How to Solve Any SciPy/statsmodels Problem

The decision flow

Ask yourself	If yes, reach for
Do I need a probability, quantile or random draw?	`stats` distribution: `cdf, sf, ppf, rvs`
Is a difference real or luck?	`stats` test chosen by data type, plus an interval and effect size
Do I need uncertainty and no formula exists?	`stats.bootstrap` or `permutation_test`
Do I have a formula with unknown constants?	`optimize.curve_fit` (or `least_squares`)
Do I want the best inputs?	`minimize` (smooth), `linprog` / `milp` (linear), `differential_evolution` (rugged)
Do I need a total over a range or a changing system?	`integrate.quad`, `trapezoid`, `solve_ivp`
Do I need values between or smoother than my points?	`interpolate` or `signal.savgol_filter`

Ask yourself	If yes, reach for
Is it a repeating pattern or noise in time?	`fft`, `signal.welch`, `find_peaks`, `butter`
Is it about who is close or similar?	`spatial.KDTree`, `cdist`, `cluster`
Do I need coefficients with p-values and intervals?	statsmodels (`OLS, GLM, ARIMA`) or the manual NumPy version
Is the matrix mostly zeros or huge?	`sparse` and `sparse.linalg`

Six recurring problem patterns, each solved

Pattern 1: "What is the chance?" Name the distribution, then use `sf` or `cdf`.

```
from scipy import stats
# A server answers in a lognormal time with median 200 ms and shape 0.4. P(slower than 500 ms)?
d = stats.lognorm(s=0.4, scale=200)
print("P(>500 ms) = %.4f   99th percentile = %.0f ms" % (d.sf(500), d.ppf(0.99)))
```

```
P(>500 ms) = 0.0110   99th percentile = 507 ms
```

1. For a lognormal, `scale` is the median. `sf(500)` is the tail beyond 500; `ppf(0.99)` is the value 99% of requests stay under.

Pattern 2: "Is it different?" Check the shape, choose the test, report effect and interval.

```
import numpy as np
from scipy import stats
rng = np.random.default_rng(21)
before = rng.lognormal(3.0, 0.5, 40)
after = before * rng.lognormal(-0.15, 0.2, 40)          # same users measured twice
diff = np.log(after) - np.log(before)
print("normal log-differences? shapiro p = %.2f" % stats.shapiro(diff).pvalue)
t = stats.ttest_1samp(diff, 0.0)
ci = t.confidence_interval()
print("paired on log scale: p = %.2g, mean ratio %.2f (CI %.2f to %.2f)" % (t.pvalue, np.exp(diff.mean()), np.exp(ci.low),
np.exp(ci.high)))
```

```
normal log-differences? shapiro p = 0.46
paired on log scale: p = 5.4e-08, mean ratio 0.85 (CI 0.81 to 0.89)
```

1. Same users twice means paired data. The ratios are skewed, but their logs are normal, so a one-sample t-test on the log-differences works, and exponentiating gives a ratio with an interval.

Pattern 3: "How sure am I?" If the statistic has no formula, bootstrap it.

```
import numpy as np
from scipy import stats
rng = np.random.default_rng(4)
x = rng.exponential(5, 50)
r = stats.bootstrap((x,), lambda s: np.percentile(s, 90), n_resamples=3000, random_state=0)
print("p90 = %.2f, 95%% CI (%.2f, %.2f)" % (np.percentile(x, 90), *r.confidence_interval))
```

```
p90 = 13.91, 95% CI (9.24, 23.30)
```

1. The 90th percentile has no tidy standard-error formula, but the bootstrap works for any function of the sample. Default method BCa handles the skew.

Pattern 4: "What are the best inputs?" Write the objective as a function of one array, add limits, minimise.

```
import numpy as np
from scipy import optimize
# Split 100 units between 3 channels with diminishing returns a*sqrt(x); maximise total return.
a = np.array([3.0, 2.0, 1.0])
res = optimize.minimize(lambda x: -(a * np.sqrt(x)).sum(), x0=[30, 30, 40], bounds=[(0, 100)] * 3,
                        constraints=[{"type": "eq", "fun": lambda x: x.sum() - 100}], method="SLSQP")
print(res.x.round(1), "return %.2f" % -res.fun)
print("theory (x proportional to a^2):", (100 * a ** 2 / (a ** 2).sum()).round(1))
```

```
[64.3 28.6  7.1] return 37.42
theory (x proportional to a^2): [64.3 28.6  7.1]
```

1. The equality constraint keeps the total at 100. Lagrange theory says the best split is proportional to a squared, which the numerical answer matches: a good verification.

Pattern 5: "Accumulate or evolve." Total over a range: integrate. Something changing by a rule: `solve_ivp`.

```
import numpy as np
from scipy import integrate
# Drug concentration decays at rate 0.3/h; a 10-unit dose is given each 4 h in a continuous 'pump' approximation.
sol = integrate.solve_ivp(lambda t, c: 10 / 4 - 0.3 * c, (0, 24), [0.0], t_eval=[24], rtol=1e-8)
print("c(24h) = %.3f   steady state = %.3f" % (sol.y[0, 0], 10 / 4 / 0.3))
```

```
c(24h) = 8.327   steady state = 8.333
```

1. Inflow minus decay gives dc/dt = 2.5 - 0.3 c. The level climbs toward the steady state 2.5/0.3 = 8.33; after 24 hours it has nearly arrived.

Pattern 6: "Find structure in a signal or in space." Frequencies via FFT; neighbours via KDTree.

```
import numpy as np
from scipy import fft, spatial
rng = np.random.default_rng(1)
sales = 50 + 8 * np.sin(2 * np.pi * np.arange(364) / 52) + rng.normal(0, 2, 364)   # daily data with a 52-day cycle
f = fft.rfftfreq(364, 1.0)
print("cycle length: %.0f days" % (1 / f[1:][np.argmax(np.abs(fft.rfft(sales - sales.mean()))[1:])]))
pts = rng.uniform(size=(1000, 2))
print("pairs closer than 0.02:", len(spatial.KDTree(pts).query_pairs(0.02)))
```

```
cycle length: 52 days
pairs closer than 0.02: 612
```

1. Remove the mean, transform, ignore the zero-frequency bin, and invert the peak frequency to get a period (here 52 days exactly divides 364 days, so there is no leakage).
2. `query_pairs` lists every pair of points within a radius without comparing all pairs.

How to verify

- **Recover a known answer.** Simulate data with known parameters, run your method, and see whether it finds them (as in most examples above).
- **Compute it two ways.** `quad` against `cdf`; permutation p against formula p; `solve` then multiply back.
- **Check boundaries.** Probabilities between 0 and 1; fitted distributions with positive support for positive data; the optimum respects constraints (`res.success`, then test constraints yourself).
- **Plot it.** A fit you have not plotted is a guess.

How to debug

1. **Read the result object.** `res.success`, `res.message`, `res.status`; `print(res)` shows everything.
2. **Shrink the problem.** Run on 10 points, then 100, then all.
3. **Check shapes and dtypes.** `x.shape`, `x.dtype`, `np.isnan(x).sum()`. NaN silently poisons many statistics; use `nan_policy="omit"` where offered or drop them first.
4. **Change the start.** For optimisers try three different `x0`; if the answers differ, the problem has several local minima.
5. **Turn warnings into errors while debugging:** `warnings.simplefilter("error")`. A "covariance could not be estimated" warning from `curve_fit` means the fit failed, even if numbers came out.
6. **Compare to a simpler method** (`np.polyfit` for a line, a histogram for a distribution).

18.15 Going Deeper: Internals, Memory, Speed, Interviews

How distribution objects work

Every SciPy continuous distribution subclasses `rv_continuous`. You supply one function, `_pdf` (or `_cdf`), and the base class derives the rest numerically: cdf by integrating the pdf, ppf by root-finding on the cdf, rvs by inverting the cdf. Distributions with closed forms override these methods for speed. You can define your own:

```
import numpy as np
from scipy import stats

class Triangle(stats.rv_continuous):
    "Density 2x on [0, 1]"
    def _pdf(self, x):
        return 2 * x

tri = Triangle(a=0, b=1, name="tri")
print("cdf(0.5) = %.4f   (exact 0.25)" % tri.cdf(0.5))
print("ppf(0.25) = %.4f  (exact 0.5)" % tri.ppf(0.25))
print("mean = %.4f   (exact 2/3)" % tri.mean())
print("sample mean = %.3f" % tri.rvs(size=2000, random_state=1).mean())
```

```
cdf(0.5) = 0.2500   (exact 0.25)
ppf(0.25) = 0.5000  (exact 0.5)
mean = 0.6667   (exact 2/3)
sample mean = 0.673
```

1. `a` and `b` are the support limits. Only `_pdf` is defined.
2. The cdf and mean come from numerical integration, ppf from root-finding, and rvs from inverting the cdf. This works but is slow; defining `_cdf` and `_ppf` yourself speeds it up greatly.

Optimiser internals in one paragraph

`minimize` is a front door to several algorithms. BFGS builds an approximation of the inverse curvature from successive gradients (cost grows with the square of the number of variables). L-BFGS-B stores only a short history of steps, so it scales to thousands of variables and supports bounds. SLSQP solves a sequence of quadratic problems with linearised constraints. Nelder-Mead moves a simplex and uses no derivatives, but is slow and can stall in more than about 10 variables. `linprog` with `method="highs"` uses the HiGHS simplex or interior-point solver, and `curve_fit` calls Levenberg-Marquardt (unbounded) or a trust-region method (bounded). Convergence is controlled by `tol` and `options`; a run that hits `maxiter` returns `success=False`.

Memory: sparse formats and dtypes

```
import numpy as np
from scipy import sparse

rng = np.random.default_rng(0)
n = 5000
r = rng.integers(0, n, 50000); c = rng.integers(0, n, 50000)
coo = sparse.coo_array((np.ones(50000), (r, c)), shape=(n, n))
csr = coo.tocsr()
lil = coo.tolil()
mb = lambda b: round(b / 1e6, 2)
print("dense float64 :", mb(n * n * 8), "MB")
print("COO           :", mb(coo.data.nbytes + coo.row.nbytes + coo.col.nbytes), "MB")
print("CSR           :", mb(csr.data.nbytes + csr.indices.nbytes + csr.indptr.nbytes), "MB")
print("index dtype   :", csr.indices.dtype, " nnz after duplicates summed:", csr.nnz)
```

```
dense float64 : 200.0 MB
COO           : 1.2 MB
CSR           : 0.84 MB
index dtype   : int64  nnz after duplicates summed: 49949
```

1. COO stores three numbers per entry (value, row, column). CSR replaces the row array by a short pointer array of length n + 1, so it is smaller and gives fast row operations.
2. The last line prints the index dtype (64-bit here, because the inputs were 64-bit). 32-bit indices would take half the index memory, so check `.indices.dtype` on very large matrices.
3. Converting COO to CSR sums duplicate coordinates, so `nnz` can be below the number of entries you inserted (random pairs collided here).
4. LIL (a list of rows) is the flexible format for editing individual elements but is slow and heavy for arithmetic, so convert before computing.

Speed: where the time goes

- FFT cost is about n log n, but only for lengths with small prime factors; a length that is a large prime can be far slower. `fft.next_fast_len(n)` and zero-padding, or choosing a good length, fix it.
- Dense linear algebra calls optimised BLAS/LAPACK libraries; time scales with n cubed for solve, so doubling the size costs about 8 times.
- KDTree build is about n log n and a query about log n in low dimensions, degrading toward brute force as dimension grows.
- Distribution methods have per-call overhead. For a hot loop, pull the maths out (`scipy.special.ndtr` for the normal cdf) or vectorise.
- Many functions accept `workers` for parallelism; BLAS threading is controlled by environment variables such as `OMP_NUM_THREADS`.

What interviewers probe

1. **"What does a p-value mean?"** Probability, under the null, of a statistic at least as extreme as observed. Not the probability the null is true.
2. **"Why Welch rather than Student?"** Equal variances are rarely guaranteed; Welch loses almost nothing when they are equal.
3. **"Why is `solve` better than `inv` ?"** One LU factorisation (about $2n^3/3$ operations) with smaller rounding error, while inversion costs more and amplifies error through the condition number.
4. **"Bootstrap versus formula?"** The bootstrap needs no distribution formula and works for medians, ratios and percentiles, but needs a reasonably representative sample and fails for extremes such as the maximum.
5. **"What if `curve_fit` fails?"** Better `p0` , bounds, rescale parameters to similar magnitudes, check the model is identifiable (two parameters doing the same job make `pcov` infinite).
6. **"KDTree in 500 dimensions?"** Distances concentrate and pruning fails; use approximate nearest-neighbour methods.
7. **"Aliasing?"** Frequencies above half the sampling rate fold back to lower ones; sample faster or low-pass filter before sampling.

Chapter recap

- SciPy is a set of tool drawers on top of NumPy arrays; statsmodels adds regression and time-series models with full inference tables. Import the submodule you need.
- Every distribution has `pdf/pmf, cdf, sf, ppf, rvs, fit`. `cdf` maps value to probability, `ppf` maps probability to value.
- Pick tests by data type, prefer Welch over Student, use rank tests for skewed data, and always report an effect size and an interval, not only p.
- The bootstrap gives intervals for any statistic; permutation tests give p-values without formulas. Seed them.
- `minimize` for smooth problems, `curve_fit` for fitting formulas (give `p0`), `root` / `brentq` for equations, `linprog` / `milp` for linear plans (minimises, so negate).
- `trapezoid(y, x)` and `quad` integrate; `solve_ivp` evolves systems; splines and PCHIP interpolate but should not extrapolate.
- `solve` not `inv`; `lstsq` for fits; sparse matrices for mostly-zero data (build in COO, compute in CSR); use `@` for products.
- `rfft` plus `rfftfreq(n, 1/fs)` finds frequencies; `sosfiltfilt` filters without delay; `find_peaks` needs prominence or distance.
- KDTree for neighbours in low dimensions, `cdist` for small matrices, `linkage` and `kmeans2` for quick clustering (scale the columns first).
- statsmodels needs `add_constant` in the array API; formulas add the intercept. Its numbers can be reproduced with NumPy, which is the best way to understand them.
- Verify by recovering a known answer, computing two ways, and plotting.

Formula / pattern	Meaning
`d.cdf(b) - d.cdf(a)`	Probability of landing between a and b
`d.sf(k - 1)`	Discrete "k or more"
`mean +/- t.ppf(0.975, n-1) * sem`	95% confidence interval for a mean
`ttest_ind(a, b, equal_var=False)`	Welch two-group test
`n = 2 ((z_{1-a/2} + z_{power}) / d)^2`	Sample size per group for standardised effect d
`bootstrap((x,), stat)`	Interval for any statistic
`curve_fit(f, x, y, p0)`, `sqrt(diag(pcov))`	Parameters and standard errors
`linprog(-c, A_ub, b_ub)`	Maximise c.x within limits
`solve(A, b)`	Linear system (never `inv(A) @ b`)
`rfftfreq(n, 1/fs)[argmax(abs(rfft(x)))]`	Dominant frequency in Hz
`tree.query(q, k)`	k nearest neighbours
`(X'X)^-1X'y`, se = sqrt(diag(s^2(X'X)^-1))	OLS coefficients and standard errors

Practice set

Twenty-five problems, graded from easy to expert. Try each before reading the answer.

Problem 18.P.1 (easy) What is the probability that a standard normal value is at most 1.96? What is it for a value above 1.96?

Answer: `stats.norm.cdf(1.96)` is about 0.975, and the upper tail `sf(1.96)` is 0.025. They add to 1. The two tails together hold 5%, which is why 1.96 is the 95% interval multiplier.

Problem 18.P.2 (easy, what does it print?)

```
from scipy import stats
print(stats.norm.ppf(0.5, loc=10, scale=3))
```

Answer: it prints 10.0. The 50th percentile of a symmetric distribution is its mean (and median), so `loc` is returned; `scale` does not matter here.

Problem 18.P.3 (easy) You have 30 waiting times in each of two groups, both strongly right-skewed. Which SciPy test do you choose and why?

Answer: `stats.mannwhitneyu`, because it compares ranks and does not need normality. A good alternative is to log-transform the times and run Welch's `ttest_ind` if the logs look normal (check with `shapiro` or a Q-Q plot). Report an effect size either way.

Problem 18.P.4 (easy, what does it print?)

```
from scipy import optimize
res = optimize.minimize(lambda v: (v[0] - 3) ** 2 + 1, x0=[0.0])
print(round(float(res.x[0]), 3), round(float(res.fun), 3))
```

Answer: 3.0 1.0. The parabola $(x - 3)^2 + 1$ has its lowest point at x = 3 with value 1. Note that `x0` is an array, so `res.x` is an array and we take element 0.

Problem 18.P.5 (easy) By hand, what does `integrate.trapezoid([0, 2, 4], [0, 1, 2])` return?

Answer: two trapezoids, each of width 1. The first has heights 0 and 2: area (0 + 2)/2 = 1. The second has heights 2 and 4: area 3. Total 4, which equals the exact area under y = 2x from 0 to 2 (a straight line has no error): 4.0.

Problem 18.P.6 (easy, what does it print?)

```
import numpy as np
from scipy import sparse
A = sparse.csr_array(np.array([[0, 0, 3], [4, 0, 0], [0, 0, 0]]))
print(A.nnz, A.shape, A.toarray().sum())
```

Answer: 2 (3, 3) 7. Only the two non-zero entries (3 and 4) are stored, the shape is 3 by 3 and their sum is 7. A sparse matrix remembers values and positions, not the zeros.

Problem 18.P.7 (medium, what does it print?)

```
import numpy as np
print(np.interp([0, 1.5, 9], [1, 2], [10, 20]))
```

Answer: [10. 15. 20.]. 1.5 is halfway between 1 and 2, so 15. Outside the known range `np.interp` does not extrapolate: it clamps to the end values (10 on the left, 20 on the right). That silent clamping can hide a data problem.

Problem 18.P.8 (medium, find the bug) The code is meant to maximise 3x + 2y with x + y at most 4, x and y at least 0. It reports a profit of zero. Why?

```
from scipy import optimize
res = optimize.linprog([3, 2], A_ub=[[1, 1]], b_ub=[4], bounds=[(0, None)] * 2)
print(res.x, res.fun)
```

Answer: it prints [0. 0.] 0.0. `linprog` minimises, so the best "minimum" of 3x + 2y is at the origin. Fix: pass `c=[-3, -2]` and negate the result; the true optimum is x = 4, y = 0 with value 12.

Problem 18.P.9 (medium, find the bug) A speed log is integrated and gives a negative distance. What went wrong?

```
import numpy as np
from scipy import integrate
t = np.array([0.0, 1.0, 2.0, 3.0]); v = np.array([5.0, 6.0, 7.0, 6.0])
print(integrate.trapezoid(t, v))
```

Answer: it prints -0.5, a negative number. The signature is `trapezoid(y, x)`: values first. With the arguments swapped the function integrates time with respect to speed, and because speed falls at the end the small steps count backward. Correct call: `trapezoid(v, t)` = 18.5.

Problem 18.P.10 (medium) A sample of 25 values has mean 50 and standard deviation 10. Give the 95% t interval for the mean.

Answer: standard error = 10 / sqrt(25) = 2. The t quantile with 24 degrees of freedom is 2.064. Interval = 50 plus or minus 4.13, so (45.87, 54.13). In code: `stats.t.interval(0.95, 24, loc=50, scale=2)`. Using 1.96 instead of 2.064 would make the interval too narrow for a sample this small.

Problem 18.P.11 (medium) A table has page A: 30 converted, 70 not; page B: 50 converted, 50 not. Compute the chi-square statistic by hand.

Answer: totals: converted 80, not 120, each page 100, grand 200. Expected converted per page = 100 * 80 / 200 = 40, not converted = 60. Chi-square = sum of $(O - E)^2/E$ = 100/40 + 100/60 + 100/40 + 100/60 = 2.5 + 1.667 + 2.5 + 1.667 = 8.333, with 1 degree of freedom. Code check: 8.333, p = 0.0039, so the rates 30% and 50% differ significantly.

Problem 18.P.12 (medium, find the bug) This call fails or returns nonsense. Why?

```
def model(a, k, t):                # parameters first, x last
    return a * np.exp(-k * t)
popt, pcov = optimize.curve_fit(model, t, y)
```

Answer: `curve_fit` always passes the x data as the *first* argument and the parameters after it. Here it would treat `a` as x and fit `k` and `t` as parameters (and complain about the wrong number of values). Fix the signature to `model(t, a, k)`.

Problem 18.P.13 (medium, what does it print?)

```
from scipy import stats
d = stats.poisson(2)
print(round(d.cdf(2), 4), round(d.sf(2), 4), round(d.pmf(2), 4))
```

Answer: 0.6767 0.3233 0.2707. `cdf(2)` is P(0, 1 or 2 events), `sf(2)` is P(3 or more) and the two add to 1. `pmf(2)` is just P(exactly 2). Mixing these up is the most common discrete-distribution error.

Problem 18.P.14 (medium) Write code for a 90% bootstrap interval for the 75th percentile of a sample.

Answer:

```
import numpy as np
from scipy import stats

x = np.random.default_rng(0).gamma(2.0, 3.0, 80)
r = stats.bootstrap((x,), lambda s, axis: np.percentile(s, 75, axis=axis), n_resamples=3000,
                    confidence_level=0.90, random_state=0)
print("p75 = %.2f, 90%% CI (%.2f, %.2f)" % (np.percentile(x, 75), r.confidence_interval.low, r.confidence_interval.high))
```

```
p75 = 6.94, 90% CI (6.39, 8.32)
```

1. The data goes in as a tuple `(x,)`.
2. The default is vectorised: the statistic receives a batch of resamples and an `axis` argument, so the lambda takes `axis` and passes it to `np.percentile`. Forgetting `axis` is the usual cause of errors or wrong shapes here.
3. `confidence_level=0.90` gives a 90% interval.

Problem 18.P.15 (hard, find the bug) A colleague fits a normal to data, then runs `stats.kstest(data, "norm", args=(data.mean(), data.std()))` and reports "p = 0.4, so the data is normal". Find two problems.

Answer: (1) The parameters were estimated from the same data being tested, which makes the KS p-value too large (too forgiving); the proper fix is a Lilliefors-style test or a parametric bootstrap that repeats the estimate on simulated data. (2) A large p-value does not prove normality; it only says the test could not reject it. Complement with a Q-Q plot (`stats.probplot`) and, with small samples, remember the test has little power.

Problem 18.P.16 (hard) How many users per group do you need to detect a standardised difference of 0.3 with 80% power at a two-sided 5% level?

Answer: with the normal approximation, $n = 2 * ((z_{0.975} + z_{0.80}) / d)^2 = 2 * ((1.96 + 0.8416) / 0.3)^2 = 175$ per group (rounded up). Exact t-based calculations (statsmodels `TTestIndPower` , not run here) give a slightly larger number. Halving the effect size quadruples the sample size, which is why small effects are expensive.

Problem 18.P.17 (hard) A user-item matrix has 1,000,000 rows, 1,000,000 columns and 5,000,000 non-zero values. Estimate the memory of dense float64 and of CSR with 32-bit indices.

Answer: dense: 10^{12} cells * 8 bytes = $8 * 10^{12}$ bytes = 8 TB, impossible. CSR: values $5 * 10^6 * 8 = 40$ MB, column indices $5 * 10^6 * 4 = 20$ MB, row pointers $(10^6 + 1) * 4 = 4$ MB, total about 64 MB. Density is $5 * 10^{-6}$, so sparse is the only option.

Problem 18.P.18 (hard, what does it print?)

```
import numpy as np
from scipy import stats
print(stats.zscore([1, 2, 3]).round(4), stats.zscore([1, 2, 3], ddof=1).round(4))
```

Answer: [-1.2247 0. 1.2247] [-1. 0. 1.]. By default `zscore` divides by the population standard deviation (`ddof=0` : sqrt(2/3) = 0.8165), giving plus or minus 1.2247. With `ddof=1` the sample standard deviation is 1, giving plus or minus 1. pandas' `std` uses `ddof=1` by default, so results can differ between libraries unless you set it.

Problem 18.P.19 (hard) Find the x where cos(x) = x.

```
import numpy as np
from scipy import optimize
print(round(optimize.brentq(lambda x: np.cos(x) - x, 0, 1), 6))
```

Answer: 0.739085. Define f(x) = cos x - x, which is positive at 0 (value 1) and negative at 1 (about -0.46), so a root lies in [0, 1] and `brentq` is guaranteed to find it. Check: cos(0.739085) is 0.739085.

Problem 18.P.20 (hard, find the bug) The code is meant to find the weekly peaks, but it returns far too many.

```
import numpy as np
from scipy import signal
rng = np.random.default_rng(0)
x = 10 * np.sin(2 * np.pi * np.arange(140) / 7) + rng.normal(0, 4, 140)
print(len(signal.find_peaks(x)[0]), len(signal.find_peaks(x, prominence=10, distance=5)[0]))
```

Answer: it prints 29 20 (the true count is 20, one per week). Plain `find_peaks` calls every tiny noise wiggle a peak. Constrain it: `prominence` says how much a peak must stand above its surroundings, and `distance` says how far apart peaks must be. Choose these from the domain (a weekly cycle means at least 5 days apart).

Problem 18.P.21 (hard, design) Delivery-time data has 30% exact zeros (pre-cooked items) and positive skew otherwise. How would you model it with SciPy?

Answer: a single lognormal cannot have exact zeros. Use a two-part (hurdle) model: estimate the zero share p0 as a proportion (with a `binomtest` interval), fit `lognorm.fit(positive_values, floc=0)` to the rest, and combine: P(T > t) = (1 - p0) * lognorm.sf(t). Validate by simulating from the model and comparing quantiles with the data.

Problem 18.P.22 (expert) Show that the standard error of the OLS slope in simple regression is sigma / sqrt(S_{xx}), where S_{xx} is the sum of squared deviations of x, and confirm numerically.

Answer: the slope is sum((x - mean x)(y - mean y)) / S_{xx} = sum((x - mean x) y) / S_{xx}, a linear combination of y with weights w = (x - mean x)/S_{xx}. With independent noise of variance sigma^2, the variance is sigma^2 * sum(w^2) = sigma^2 * S_{xx}/S_{xx}^2 = sigma^2/S_{xx}. Numerical confirmation against the matrix formula:

```
import numpy as np

rng = np.random.default_rng(11)
x = rng.uniform(0, 10, 80)
y = 2 + 0.7 * x + rng.normal(0, 1.0, 80)
X = np.column_stack([np.ones(80), x])
beta = np.linalg.solve(X.T @ X, X.T @ y)
s2 = ((y - X @ beta) ** 2).sum() / 78
se_matrix = np.sqrt(s2 * np.linalg.inv(X.T @ X)[1, 1])
se_formula = np.sqrt(s2 / ((x - x.mean()) ** 2).sum())
print(round(se_matrix, 6), round(se_formula, 6))
```

```
0.039062 0.039062
```

Both routes give the same number, and the formula shows why a wider spread of x (larger S_{xx}) gives a more precise slope: design your experiments to spread x out.

Problem 18.P.23 (expert, interview) Why do numerical analysts say "never compute the inverse to solve a system", and what do you do when the same matrix must be solved against 1,000 right-hand sides?

Answer: solving by LU costs about 2n^3/3 operations and has backward-stable error behaviour; forming the inverse costs about 2n^3 and then a multiply, and its errors are larger when the matrix is ill-conditioned (condition number large). For many right-hand sides, factor once with `lu_factor` (or `cholesky` for symmetric positive-definite matrices), then call `lu_solve` for each (or pass all right-hand sides as columns of one matrix). Also check `np.linalg.cond`: if it is near 1/eps (about 10^{16}), the answer has no correct digits whatever method you use.

Problem 18.P.24 (expert, interview) Your KDTree nearest-neighbour search on 768-dimensional text embeddings is no faster than brute force. Why, and what are your options?

Answer: in high dimensions distances to the nearest and farthest points become similar, so the tree cannot rule out whole boxes and must visit almost every node. Options: reduce dimension first (PCA or SVD, accepting some loss), use approximate nearest-neighbour libraries (graph or quantisation indexes), or batch brute force as a matrix multiplication on normalised vectors (`E @ q`), which is often fast enough up to a few hundred thousand vectors because it uses optimised BLAS.

Problem 18.P.25 (expert, design) An experiment has 10 million users per arm, and the data lives in a warehouse. How do you run the t-test without moving all rows?

Answer: compute count, mean and variance per arm in SQL, then run the test from the summaries with `ttest_ind_from_stats`. It gives the same result as the row-level test:

```
import numpy as np
from scipy import stats

rng = np.random.default_rng(0)
a = rng.normal(10.00, 3, 200000); b = rng.normal(10.02, 3, 200000)
full = stats.ttest_ind(b, a, equal_var=False)
summ = stats.ttest_ind_from_stats(b.mean(), b.std(ddof=1), b.size, a.mean(), a.std(ddof=1), a.size, equal_var=False)
print(round(full.pvalue, 8), round(summ.pvalue, 8), np.isclose(full.statistic, summ.statistic))
```

```
0.03389195 0.03389195 True
```

The summaries must use the sample standard deviation (`ddof=1`). With ten million users per arm even tiny differences are significant, so judge practical size with the interval and the business threshold, and consider variance-reduction methods (pre-experiment covariates) before spending more traffic.

Prompt pack

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data roles, and English is my second language, so use simple English. Teach me SciPy and statsmodels step by step: scipy.stats distributions (pdf, cdf, ppf, rvs, fit), hypothesis tests (t-test, Mann-Whitney, chi-square, Shapiro), confidence intervals and the bootstrap, then optimize (minimize, curve_fit, linprog), integrate, interpolate and the OLS output table of statsmodels. Use small runnable Python examples with seeded randomness and show real output. After each part, ask me one check question and wait for my answer before continuing.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional learning scientific Python, and English is my second language. Review my analysis code below. Check: Is the statistical test right for my data (paired or independent, normal or skewed)? Do I report an effect size and a confidence interval, not only a p-value? Are random seeds set? Did I give curve_fit a starting guess and check the residuals? Did I use solve instead of inv, and sparse matrices where the data is mostly zeros? List the top 5 problems in order of importance, with a corrected snippet for each. My code: [PASTE HERE]
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional preparing for data science interviews, and English is my second language. Run a 30-minute mock interview on SciPy and statsmodels: p-values, Welch versus Student, bootstrap versus formula intervals, power and sample size, curve_fit failures, optimisation methods, solve versus inv, sparse matrices, FFT aliasing and KDTree limits. Ask one question at a time, mixing concept questions with a small coding task. After each answer give a score out of 5, a model answer, and one tip on how to say it clearly in English.
```

CHAPTER 19

SCIKIT-LEARN: THE COMPLETE PRACTICAL GUIDE

Library Handbook: pipelines, models, cross-validation, metrics and leakage-proof workflows.

Priya works at a mid-sized telecom. On Monday her manager forwards a spreadsheet of 20,000 customers with a column called `churned` and says: "Tell me who is about to leave, and tell me how sure you are." Priya knows pandas. She can filter, group and plot. But nothing in pandas will *learn* a rule from the data. She needs a tool that takes a table of examples and returns a function that makes predictions, and she needs to know whether that function will still work next month on customers it has never seen.

That tool is **scikit-learn**. It is the standard Python library for classical machine learning on tables: predicting a label (classification), predicting a number (regression), grouping similar rows (clustering), compressing columns (dimensionality reduction) and finding odd rows (anomaly detection). It is also the library where beginners most often fool themselves, because it is very easy to get a score that looks wonderful and means nothing. This chapter teaches both halves: how to use every important part of the library, and how to use it *honestly*.

By the end you will be able to build a complete, leak-free model pipeline, tune and evaluate it with cross-validation, pick metrics that match the business question, calibrate probabilities, handle imbalanced classes, explain a model, save it, and write your own transformer. You will also have a playbook for the recurring problems you will meet at work and 22 practice problems with full answers.

Words you need. A **feature** is one input column. **X** is the table of features (rows are samples). **y** is the target you want to predict. **Training** (or fitting) means letting the algorithm learn from X and y. A **hyper-parameter** is a setting you choose before training (such as the depth of a tree). A **leak** is information from the test data that sneaks into training, which makes scores look better than reality.

WHAT WAS RUN

Every block marked as output was executed with Python 3.14, scikit-learn 1.9.1, NumPy 2.5.3, pandas 3.0.6, SciPy 1.18.1 and matplotlib 3.11.2, and the output shown is the real output. `xgboost` and `lightgbm` are **not installed** on this machine, so code for them is labelled *not run here*, with a runnable scikit-learn equivalent. Timings quoted in prose were measured once on one laptop and will vary on yours.

19.1 What scikit-learn Is, When to Use It, and the Mental Model

INTUITION

Scikit-learn is a toolbox where every tool has the same handle. Whether you use a linear model, a forest or a scaler, you build the object, call `fit` to learn from data, and then call `predict` or `transform` to use what it learned. Learn the handle once and you can pick up any tool in the box.

What it is

Scikit-learn (import name `sklearn`) is a library of machine-learning algorithms plus the machinery around them: splitting data, preprocessing, chaining steps, searching for settings, measuring quality, and inspecting results. It is built on NumPy and SciPy. Its design goal is a **consistent interface**: all algorithms are *estimators* that follow the same few method names.

When to use it

- Your data is a table (rows and columns) of up to a few million rows that fits in memory.
- You want a strong baseline fast: logistic regression, random forest, gradient boosting.
- You need preprocessing, validation and tuning that are correct by construction (pipelines, cross-validation).
- You need classical unsupervised tools: k-means, PCA, DBSCAN, isolation forest.

When NOT to use it

- **Deep learning** on images, audio or long text: use PyTorch or TensorFlow (see the Deep Learning document).
- **Data far larger than memory**, or distributed training: use Spark ML, Dask, or the out-of-core `partial_fit` estimators for simple models only.

- **Statistical inference** (p-values, confidence intervals on coefficients): use `statsmodels`. Scikit-learn optimises prediction and does not report standard errors.
- **Time-ordered forecasting with seasonality**: scikit-learn can do the regression part, but dedicated time-series tools handle the structure better (see the Statistics document).
- **Online or GPU training at scale**: not its job.

Install, version check and import conventions

Install with `pip install scikit-learn` (the package name differs from the import name). Check what you have:

```
import sklearn
import numpy as np
import pandas as pd

print(sklearn.__version__)
print(np.__version__, pd.__version__)
```

```
1.9.1
2.5.3 3.0.6
```

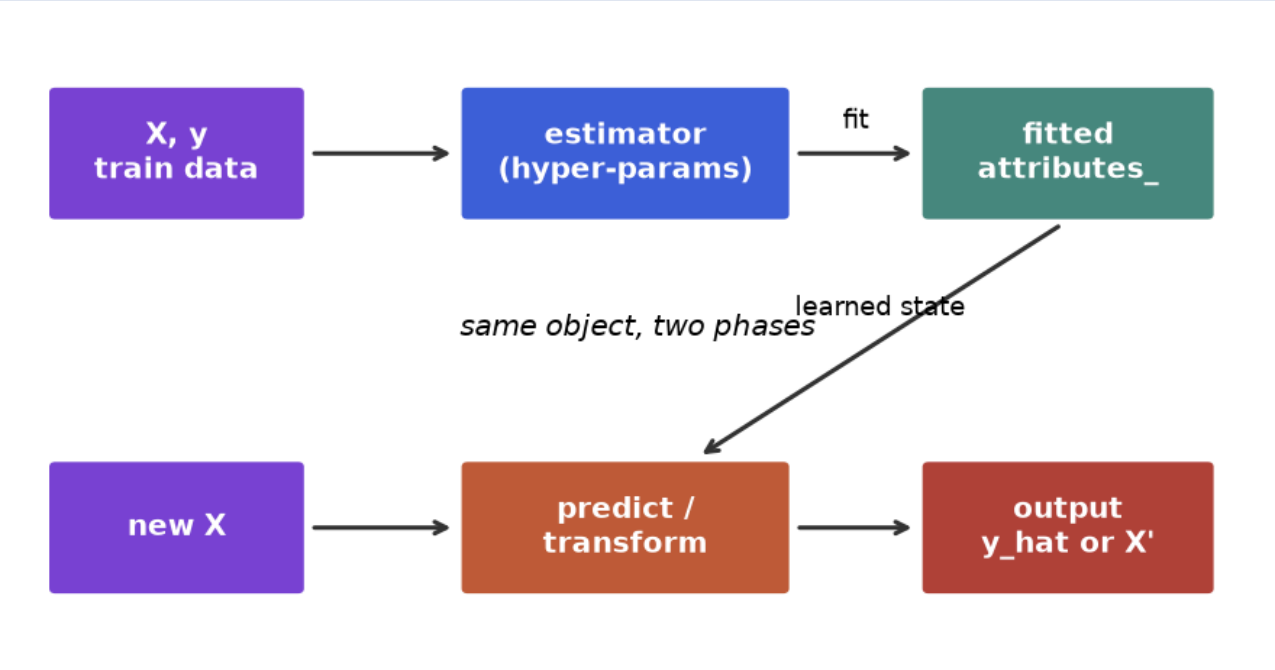
1. `import sklearn` loads the package; the version lives in `sklearn.__version__`.
2. NumPy and pandas are imported with their standard short names `np` and `pd`.
3. The print lines show the versions in use. Always record these in a project, because defaults change between versions.

Scikit-learn is imported *piece by piece*, not as `import sklearn as sk`. The convention is to import the specific class you need from the submodule that owns it:

Submodule	Holds	Typical import
`sklearn.datasets`	toy data and generators	`from sklearn.datasets import load_iris`
`sklearn.model_selection`	splits, CV, searches	`from sklearn.model_selection import train_test_split`
`sklearn.preprocessing`	scalers, encoders	`from sklearn.preprocessing import StandardScaler`
`sklearn.impute`	missing values	`from sklearn.impute import SimpleImputer`
`sklearn.compose`	per-column steps	`from sklearn.compose import ColumnTransformer`
`sklearn.pipeline`	chaining	`from sklearn.pipeline import Pipeline, make_pipeline`
`sklearn.linear_model`, `tree`, `ensemble`, `svm`, `neighbors`, `naive_bayes`	model families	`from sklearn.ensemble import RandomForestClassifier`
`sklearn.cluster`, `decomposition`, `mixture`	unsupervised	`from sklearn.cluster import KMeans`
`sklearn.metrics`	scores and curves	`from sklearn.metrics import accuracy_score`
`sklearn.inspection`	importance, partial dependence	`from sklearn.inspection import permutation_importance`

The mental model

Three ideas explain almost everything. First, an **estimator** is an object holding settings (hyper-parameters, set in the constructor). Second, calling `fit(X, y)` makes the object *learn* and store the results in attributes whose names end in an underscore, such as `coef_`. Third, a fitted object can then `predict` (models) or `transform` (preprocessors) new data using only what it stored.



The estimator life cycle: settings in, fit learns state, predict or transform applies that state to new data.

How to read it: follow the top row left to right: data and an unfitted estimator meet in `fit`, and the learned numbers land in the fitted attributes. The diagonal arrow is the key: the bottom box uses only that stored state, never the training data again. Everything about leakage in this chapter is a violation of this picture, where information flows into the learned state from rows that should have been kept out.

KEY IDEA

Constructor = choose settings. `fit` = learn from *training data only*. `predict` / `transform` = apply what was learned. A trailing underscore (`coef_` , `classes_` , `mean_`) means "this exists only after fit".

19.2 The Estimator API, Datasets and Splitting Data

This section is the foundation. Every later section is a variation of the same four verbs: `fit` , `predict` , `transform` , `score` .

Reference: the 20 most important names

Name	What it does in plain English	Minimal example
`est.fit(X, y)`	Learn from training data; returns the estimator itself	`m.fit(X_tr, y_tr)`
`est.predict(X)`	Output a label or number for each row	`m.predict(X_te)`
`est.predict_proba(X)`	Class probabilities, one column per class	`m.predict_proba(X_te)[:, 1]`
`est.decision_function(X)`	Raw score before the threshold (margin or log-odds)	`svm.decision_function(X_te)`

Name	What it does in plain English	Minimal example
`est.transform(X)`	Apply a learned preprocessing step	`sc.transform(X_te)`
`est.fit_transform(X)`	Fit then transform in one call (training data only)	`sc.fit_transform(X_tr)`
`est.score(X, y)`	Default quality number: accuracy for classifiers, R squared for regressors	`m.score(X_te, y_te)`
`est.get_params()`	Dictionary of the constructor settings	`m.get_params()`
`est.set_params(**kw)`	Change settings (used by searches); nested names use `__`	`pipe.set_params(clf__C=3)`
`sklearn.base.clone`	Make an unfitted copy with the same settings	`m2 = clone(m)`
`check_is_fitted`	Raise `NotFittedError` if not yet fitted	`check_is_fitted(m)`
`load_iris` , `load_wine` , `load_breast_cancer` , `load_diabetes`	Small built-in datasets (ship with the library, no network)	`load_iris(as_frame=True)`
`fetch_california_housing` , `fetch_20newsgroups`	Larger datasets downloaded on first use (need network)	`fetch_20newsgroups()` (not run here)
`make_classification`	Synthetic classification table with chosen difficulty	`make_classification(1000, 10, random_state=0)`
`make_regression`	Synthetic regression table	`make_regression(500, 5, noise=10)`
`make_blobs` , `make_moons`	2-D clusters or two half-moons for clustering and boundary demos	`make_moons(300, noise=0.3)`
`train_test_split`	Shuffle and cut into train and test parts	`train_test_split(X, y, test_size=0.2, random_state=0)`
`stratify=y`	Keep the class ratio equal in both parts	`train_test_split(X, y, stratify=y)`
`KFold` , `StratifiedKFold`	Make k train/validation folds	`StratifiedKFold(5, shuffle=True, random_state=0)`
`GroupKFold` , `TimeSeriesSplit`	Folds that keep groups together or respect time order	`GroupKFold(5).split(X, y, groups)`
`set_config(transform_output="pandas")`	Make transformers return DataFrames globally	`sklearn.set_config(transform_output="pandas")`

Example 1: the smallest possible model

Four houses: size in hundreds of square metres and price in thousands. We fit a straight line and use it.

```
import numpy as np
from sklearn.linear_model import LinearRegression

X = np.array([[1.0], [2.0], [3.0], [4.0]])   # shape (4, 1): 4 rows, 1 feature
y = np.array([110, 190, 310, 390])

model = LinearRegression()
model.fit(X, y)
print("slope:", model.coef_, "intercept:", round(model.intercept_, 1))
print("predict size 5:", model.predict([[5.0]]))
print("R squared:", round(model.score(X, y), 4))
```

```
slope: [96.] intercept: 10.0
predict size 5: [490.]
R squared: 0.9931
```

1. `X` must be two-dimensional: rows by features. A single feature is a column, hence the double brackets.
2. `y` is one-dimensional: one target per row.
3. The constructor `LinearRegression()` stores settings only; nothing is learned yet.
4. `fit` finds the slope and intercept by least squares and stores them in `coef_` and `intercept_`.
5. `predict` takes a 2-D array again (one new row with one feature) and returns an array with one number.
6. `score` for a regressor returns R squared, the share of variance explained (1.0 is perfect). Here it is measured on the training data, so it is optimistic.

Reading the output: the fitted line is price = 10 + 96 × size, so size 5 predicts 490. R squared is 0.9931, which is high because four points almost lie on a line, and it is measured on the same four points the line was fitted to.

COMMON MISTAKE

Wrong: `model.fit([1, 2, 3, 4], y)` or `model.predict([5])`. **Symptom:** `ValueError: Expected 2D array, got 1D array instead`. **Fix:** reshape with `X.reshape(-1, 1)`, or select a column as `df[["size"]]` (double brackets keep it a DataFrame) rather than `df["size"]`.

Example 2: a real classifier, an honest test

The iris dataset has 150 flowers, 4 measurements each, 3 species. We hold out a test set, fit on the rest, and score on the held-out part.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

data = load_iris(as_frame=True)
X, y = data.data, data.target
print(X.shape, list(X.columns))

X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=0)
print(X_tr.shape, X_te.shape, y_te.value_counts().to_dict())

clf = LogisticRegression(max_iter=200)
clf.fit(X_tr, y_tr)
print("train acc:", round(clf.score(X_tr, y_tr), 3))
print("test acc :", round(clf.score(X_te, y_te), 3))
print("classes_:", clf.classes_, " coef_ shape:", clf.coef_.shape)
print(clf.predict_proba(X_te.iloc[:2]).round(3))
```

```
(150, 4) ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']
(112, 4) (38, 4) {0: 13, 1: 13, 2: 12}
train acc: 0.955
test acc : 1.0
classes_: [0 1 2]  coef_ shape: (3, 4)
[[0.968 0.032 0.   ]
 [0.966 0.034 0.   ]]
```

1. `load_iris(as_frame=True)` returns a bundle whose `.data` is a DataFrame and `.target` a Series.
2. We print the shape and column names to know what we have before modelling.
3. `train_test_split` returns four pieces in a fixed order: X train, X test, y train, y test. `test_size=0.25` keeps a quarter for testing. `stratify=y` keeps the three species in equal proportion in both parts. `random_state=0` makes the shuffle repeatable.
4. `max_iter=200` gives the optimiser enough steps to converge (the default is 100).
5. `fit` sees only the training rows. The test rows are untouched.
6. Comparing train and test accuracy shows whether the model memorised. A small gap is healthy.
7. `classes_` lists the labels in the column order of `predict_proba`; `coef_` has one row of weights per class.
8. `predict_proba` gives one probability per class for each row; each row sums to 1.

Reading the output: the model has 112 training rows and 38 test rows, and the test set holds 13, 13 and 12 flowers of the three species. Train accuracy is 0.955 and test accuracy is 1.0. Test beating train is luck of a small test set (38 rows, so one error moves accuracy by 0.026); do not read it as "the model is perfect". The first two test flowers get about 0.97 for class 0, which is confident and correct.

Problem 19.2.1 A dataset has 1,000 rows, 5% of them positive. You call `train_test_split(X, y, test_size=0.2, random_state=0)` without `stratify`. Is it guaranteed that the test set holds exactly 10 positives?

Expected	20% of 1,000 is 200 test rows; 5% of 200 is 10 positives on average.
Without stratify	The split is a random shuffle, so the count of positives in the test set is random (typically close to 10, but 6 or 14 is entirely possible).
With stratify=y	The split preserves the class ratio as closely as integers allow, giving 10 (or within one).
Answer	No. Use `stratify=y` whenever classes are imbalanced, so your test metric is not distorted by an unlucky split.

Example 3: inspecting and copying estimators

Settings are visible and changeable, and `clone` gives a fresh, unfitted copy. This is how cross-validation and grid search work internally.

```
from sklearn.base import clone
from sklearn.exceptions import NotFittedError
from sklearn.tree import DecisionTreeClassifier
from sklearn.utils.validation import check_is_fitted

tree = DecisionTreeClassifier(max_depth=3, random_state=0)
print(tree.get_params()["max_depth"])

try:
    check_is_fitted(tree)
except NotFittedError:
    print("not fitted yet")

tree.set_params(max_depth=5)
copy = clone(tree)
print(copy.max_depth, copy is tree)
print(tree)
```

```
3
not fitted yet
5 False
DecisionTreeClassifier(max_depth=5, random_state=0)
```

1. `get_params()` returns every constructor setting as a dictionary; we read one key.
2. `check_is_fitted` inspects the object for attributes ending in an underscore. None exist, so it raises `NotFittedError`, which we catch and report.
3. `set_params` edits a setting in place. Searches use this to try many values.
4. `clone` builds a new object from the current settings only. The fitted state, if there were any, is not copied. It is a different object (`copy is tree` is `False`).
5. Printing the estimator shows its constructor call, listing only the settings you changed from the defaults.

Example 4: realistic splitting, three parts

Real projects need *three* parts. Train to fit, validation to choose settings, and a test set you open once at the very end. Splitting twice does it.

```
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split

X, y = make_classification(n_samples=2000, n_features=12, n_informative=5,
                           weights=[0.9, 0.1], random_state=42)
X_tmp, X_test, y_tmp, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)
X_train, X_val, y_train, y_val = train_test_split(
    X_tmp, y_tmp, test_size=0.25, stratify=y_tmp, random_state=42)

for name, part in [("train", y_train), ("val", y_val), ("test", y_test)]:
    print(f"{name:5s} rows={len(part):4d} positive rate={part.mean():.3f}")
```

```
train rows=1200 positive rate=0.101
val   rows= 400 positive rate=0.100
test  rows= 400 positive rate=0.100
```

1. `make_classification` builds 2,000 rows and 12 columns, 5 of them informative. `weights=[0.9, 0.1]` makes about 10% positives.
2. The first split sets aside 20% (400 rows) as the final test set. Nothing is allowed to look at it until the end.
3. The second split cuts the remaining 1,600 rows into 1,200 train and 400 validation (25% of 1,600 is 400).
4. Both splits use `stratify` so each part keeps about the same positive rate.
5. The loop prints the size and the positive rate of each part. The three rates agree closely, which is the point.

KEY IDEA	Cross-validation (section 19.5) replaces the train/validation split with many rotated splits, which is more reliable on small data. The test set stays separate either way.

19.3 Preprocessing, ColumnTransformer and Pipeline

Models want numbers without gaps, on comparable scales. Real tables have text categories, missing cells and columns measured in different units. Preprocessing closes that gap. The rule that governs this whole section: **every preprocessing step learns from the training rows only**, then applies what it learned to everything else.

Reference: preprocessing and composition

Name	What it does in plain English	Minimal example
`StandardScaler`	Subtract the mean, divide by the standard deviation (mean 0, spread 1)	`StandardScaler().fit_transform(X)`
`MinMaxScaler`	Squeeze each column into a range, default 0 to 1	`MinMaxScaler((0, 1))`
`RobustScaler`	Centre on the median, scale by the interquartile range; outlier-tolerant	`RobustScaler()`
`MaxAbsScaler`	Divide by the largest absolute value; keeps sparse data sparse	`MaxAbsScaler()`
`Normalizer`	Scale each *row* to unit length (not each column)	`Normalizer(norm="l2")`
`PowerTransformer`	Make a skewed column more bell-shaped (Yeo-Johnson)	`PowerTransformer()`
`QuantileTransformer`	Map values to their rank, uniform or normal shape	`QuantileTransformer(n_quantiles=100)`
`KBinsDiscretizer`	Cut a number into bins	`KBinsDiscretizer(5, encode="ordinal")`
`Binarizer`	1 if above a threshold, else 0	`Binarizer(threshold=0.5)`
`PolynomialFeatures`	Add squares and products of columns	`PolynomialFeatures(2, include_bias=False)`
`SplineTransformer`	Smooth bumps so a linear model can fit curves	`SplineTransformer(n_knots=5)`
`OneHotEncoder`	One 0/1 column per category	`OneHotEncoder(handle_unknown="ignore")`
`OrdinalEncoder`	Categories to integers 0, 1, 2 (for trees or ordered levels)	`OrdinalEncoder(categories=[["lo","mid","hi"]])`
`TargetEncoder`	Replace a category by a smoothed mean of the target (cross-fitted)	`TargetEncoder(random_state=0)`
`LabelEncoder`	Encode the *target* y only (not features)	`LabelEncoder().fit_transform(y)`
`SimpleImputer`	Fill gaps with mean, median, most frequent or a constant	`SimpleImputer(strategy="median")`
`KNNImputer`	Fill gaps from the nearest similar rows	`KNNImputer(n_neighbors=5)`
`IterativeImputer`	Model each gappy column from the others (needs an experimental import)	`IterativeImputer(random_state=0)`

Name	What it does in plain English	Minimal example
`MissingIndicator`	Add a 0/1 flag where a value was missing	`SimpleImputer(add_indicator=True)`
`FunctionTransformer`	Wrap any function as a step	`FunctionTransformer(np.log1p)`
`ColumnTransformer`	Different steps for different columns	`ColumnTransformer([("num", num, cols)])`
`make_column_selector`	Pick columns by dtype or name pattern	`make_column_selector(dtype_include="number")`
`Pipeline`, `make_pipeline`	Chain steps and a final model into one estimator	`make_pipeline(StandardScaler(), SVC())`
`FeatureUnion`	Run transformers in parallel and glue the columns	`FeatureUnion([("a", A), ("b", B)])`
`TransformedTargetRegressor`	Transform y for fitting and undo it for predictions	`TransformedTargetRegressor(m, func=np.log1p, inverse_func=np.expm1)`
`est.set_output(transform="pandas")`	Make one transformer return a DataFrame	`StandardScaler().set_output(transform="pandas")`
`get_feature_names_out()`	Names of the output columns after transformation	`ct.get_feature_names_out()`

Example 1: scaling, and why fit must see training data only

A scaler learns two numbers per column: the mean and the standard deviation. If it learns them from the test rows too, it has peeked.

```
import numpy as np
from sklearn.preprocessing import StandardScaler

train = np.array([[10.0], [20.0], [30.0], [40.0]])
test = np.array([[100.0], [50.0]])

sc = StandardScaler()
sc.fit(train)
print("mean_:", sc.mean_, "scale_:", sc.scale_.round(3))
print(sc.transform(train).ravel().round(3))
print(sc.transform(test).ravel().round(3))

# same arithmetic by hand
print(((test - train.mean()) / train.std()).ravel().round(3))
```

```
mean_: [25.] scale_: [11.18]
[-1.342 -0.447  0.447  1.342]
[6.708 2.236]
[6.708 2.236]
```

1. The training data has mean 25 and population standard deviation about 11.18. The test data has unusual values (100 and 50).
2. `fit(train)` learns `mean_` and `scale_` from the four training numbers only.
3. `transform(train)` gives values centred on zero: the mean of the result is 0 and the standard deviation is 1.
4. `transform(test)` uses the *training* mean and scale. A test value of 100 maps to about 6.7, far from the training range. That is honest information: the new data looks different from what the model saw.

5. The last line repeats the arithmetic with NumPy (`np.std` uses the population formula, like the scaler) and gets the same numbers.

COMMON MISTAKE	**Wrong:** `X_scaled = StandardScaler().fit_transform(X)` and *then* `train_test_split(X_scaled, y)` . **Symptom:** no error, and a score that is slightly too good, because the test rows helped set the mean and spread. With imputers, selectors and target encoders the damage can be large. **Fix:** split first, then fit the preprocessing on the training part only, or better, put it in a `Pipeline` .

Example 2: encoding categories, including ones never seen

```
import pandas as pd
from sklearn.preprocessing import OneHotEncoder, OrdinalEncoder

train = pd.DataFrame({"plan": ["basic", "plus", "basic", "pro"],
                      "size": ["S", "L", "M", "S"]})
new = pd.DataFrame({"plan": ["enterprise"], "size": ["M"]})

ohe = OneHotEncoder(handle_unknown="ignore", sparse_output=False)
ohe.fit(train[["plan"]])
print(ohe.get_feature_names_out())
print(ohe.transform(train[["plan"]]))
print(ohe.transform(new[["plan"]]))

ordenc = OrdinalEncoder(categories=[["S", "M", "L"]])
print(ordenc.fit_transform(train[["size"]]).ravel())
```

```
['plan_basic' 'plan_plus' 'plan_pro']
[[1. 0. 0.]
 [0. 1. 0.]
 [1. 0. 0.]
 [0. 0. 1.]]
[[0. 0. 0.]]
[0. 2. 1. 0.]
```

1. Two small columns: `plan` has no natural order, `size` does (S below M below L).
2. `OneHotEncoder(handle_unknown="ignore")` will output all zeros for a category it did not see during fit instead of crashing. `sparse_output=False` returns a plain array that is easy to print (the default is a sparse matrix, which is better for many categories).
3. Fitting learns the category list, sorted alphabetically: basic, plus, pro. `get_feature_names_out` names the columns `plan_basic` , `plan_plus` , `plan_pro` .
4. Transforming the training rows gives one 1 per row. The unseen plan "enterprise" becomes a row of zeros.
5. `OrdinalEncoder` with an explicit `categories` list uses the order you give (S=0, M=1, L=2). Without it, the order would be alphabetical, which would put L before M.

Example 3: missing values and the missing flag

```
import numpy as np
from sklearn.impute import SimpleImputer

X_train = np.array([[1.0, 7.0], [2.0, np.nan], [3.0, 9.0], [np.nan, 11.0]])
X_new = np.array([[np.nan, np.nan]])

imp = SimpleImputer(strategy="median", add_indicator=True)
print(imp.fit_transform(X_train))
print(imp.statistics_)
print(imp.transform(X_new))
```

```
[[ 1.  7.  0.  0.]
 [ 2.  9.  0.  1.]
 [ 3.  9.  0.  0.]
 [ 2. 11.  1.  0.]]
[2. 9.]
[[2. 9. 1. 1.]]
```

1. The training matrix has one gap in each column.
2. `strategy="median"` fills with the column median. It resists outliers better than the mean.
3. `add_indicator=True` appends one extra 0/1 column for each column that had gaps in training. Whether a value was missing is often itself a signal (for example, "customer did not provide income").
4. `statistics_` holds the medians learned from training: 2 for column 0 and 9 for column 1.
5. Transforming a brand new row of two gaps fills both with the *training* medians and sets both indicator flags to 1.

Example 4: a ColumnTransformer inside a Pipeline on a realistic table

Back to Priya's telecom churn. The table mixes numbers, categories and gaps. A `ColumnTransformer` routes each column group to the right steps, and a `Pipeline` joins that to the model so the whole thing is one object.

```
import numpy as np
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

rng = np.random.default_rng(0)
n = 2000
df = pd.DataFrame({
    "tenure": rng.integers(1, 72, n),
    "monthly": rng.normal(65, 20, n).round(2),
    "calls": rng.poisson(2, n),
    "contract": rng.choice(["monthly", "yearly", "two-year"], n, p=[0.55, 0.25, 0.20]),
    "payment": rng.choice(["card", "bank", "cheque"], n),
})
logit = (-2.0 - 0.04 * df["tenure"] + 0.02 * df["monthly"] + 0.3 * df["calls"]
         + np.where(df["contract"] == "monthly", 1.0, 0.0))
df["churn"] = (rng.random(n) < 1 / (1 + np.exp(-logit))).astype(int)
df.loc[rng.random(n) < 0.08, "monthly"] = np.nan

X, y = df.drop(columns="churn"), df["churn"]
X_tr, X_te, y_tr, y_te = train_test_split(X, y, stratify=y, random_state=0)

num = Pipeline([("impute", SimpleImputer(strategy="median")),
                ("scale", StandardScaler())])
cat = OneHotEncoder(handle_unknown="ignore")
prep = ColumnTransformer([("num", num, ["tenure", "monthly", "calls"]),
                          ("cat", cat, ["contract", "payment"])])
pipe = Pipeline([("prep", prep), ("clf", LogisticRegression(max_iter=500))])

pipe.fit(X_tr, y_tr)
print("churn rate:", round(y.mean(), 3))
print("test accuracy:", round(pipe.score(X_te, y_te), 3))
print(pipe.named_steps["prep"].get_feature_names_out())
print(pipe.named_steps["clf"].coef_.round(2))
```

```
churn rate: 0.294
test accuracy: 0.74
['num__tenure' 'num__monthly' 'num__calls' 'cat__contract_monthly'
 'cat__contract_two-year' 'cat__contract_yearly' 'cat__payment_bank'
 'cat__payment_card' 'cat__payment_cheque']
[[-0.77  0.4   0.41  0.66 -0.3  -0.36 -0.02  0.01  0.01]]
```

1. The first block builds a reproducible synthetic customer table: tenure in months, monthly bill, support calls, contract type and payment method.
2. The churn label is generated from a logistic formula: longer tenure lowers churn, higher bills and more calls raise it, and monthly contracts raise it. About 8% of bills are then blanked to create missing values.
3. We split first (stratified) so the test rows are untouched.
4. `num` is a small pipeline: fill gaps with the median, then scale. `cat` one-hot encodes with unseen categories ignored.
5. `ColumnTransformer` takes a list of (name, transformer, columns). Numeric columns go to `num`; the two text columns go to `cat`. The outputs are concatenated side by side.
6. The outer `Pipeline` ends with the model. Calling `pipe.fit` fits the imputer, scaler and encoder on training rows only and then fits the model on the transformed result.
7. `pipe.score(X_te, y_te)` sends raw test rows through the same learned steps and scores the predictions. You never transform by hand.
8. `get_feature_names_out` lists the 9 output columns (3 numeric plus 3 contract plus 3 payment categories) and `coef_` gives one weight per column in that same order.

Reading the output: 29.4% of customers churn, so a model that always says "stays" would score about 0.706. Our 0.74 is only a little better, which is a first lesson: accuracy needs a baseline next to it (section 19.6). The weights come out with the signs we built in: tenure is negative (-0.77, staying longer lowers churn), monthly bill and calls are positive (0.40, 0.41), and the monthly-contract column is the largest positive category weight (0.66). The payment columns are near zero because they carry no signal by construction.

WHY IT MATTERS	**A Pipeline prevents leakage by construction.** When you call `cross_val_score(pipe, X, y)`, scikit-learn clones the whole pipeline for each fold and fits *all* steps (imputer, scaler, encoder, selector, model) on that fold's training rows alone. There is no moment where you could forget to do it. It also guarantees that training and serving use the identical transformations, and it is the only thing you need to save to disk.

Pipeline mechanics you will use daily

- **Names and nested parameters.** Steps have names. To reach inside use a double underscore: `pipe.set_params(clf__C=0.5)` or `prep__num__impute__strategy="mean"`. This is how grid search tunes a whole pipeline.
- **Slicing.** `pipe[:-1]` is the pipeline without the final model (useful to see transformed data). `pipe[-1]` is the model. `pipe["prep"]` works by name.
- **make_pipeline** names steps automatically from the lowercase class name (`standardscaler`, `svc`). Use `Pipeline` with explicit names when you will tune or inspect.
- **Column selectors.** Instead of typing column names, `make_column_selector(dtype_include="number")` picks numeric columns and `dtype_exclude="number"` picks the rest. With pandas 3 text columns have the `str` dtype, and `dtype_include=object` still selects them in scikit-learn 1.9.1 (checked here).
- **remainder.** Columns not named in a `ColumnTransformer` are dropped by default. Pass `remainder="passthrough"` to keep them untouched.
- **Column order and names.** A fitted pipeline remembers `feature_names_in_`. Feeding a DataFrame with the same names in another order is fine for `ColumnTransformer`-based pipelines because it selects by name, but it fails (or warns) for bare models.

Problem 19.3.1 A column holds the values 2, 4, 6, 8 in training, and a new value 14 arrives at serving time. What does `StandardScaler` output for 14, and what does `MinMaxScaler` output?

Training stats	Mean = 5. Deviations: -3, -1, 1, 3. Squares: 9, 1, 1, 9. Mean of squares = 5, so std = $\sqrt{5} \approx 2.236$. Min = 2, max = 8.
Standard	(14 - 5) / 2.236 = 9 / 2.236 ≈ 4.025.
MinMax	(14 - 2) / (8 - 2) = 12 / 6 = 2.0, outside the 0 to 1 range.
Meaning	Neither clips. Values outside the training range stay outside. That is correct behaviour, and a reason to monitor input drift.

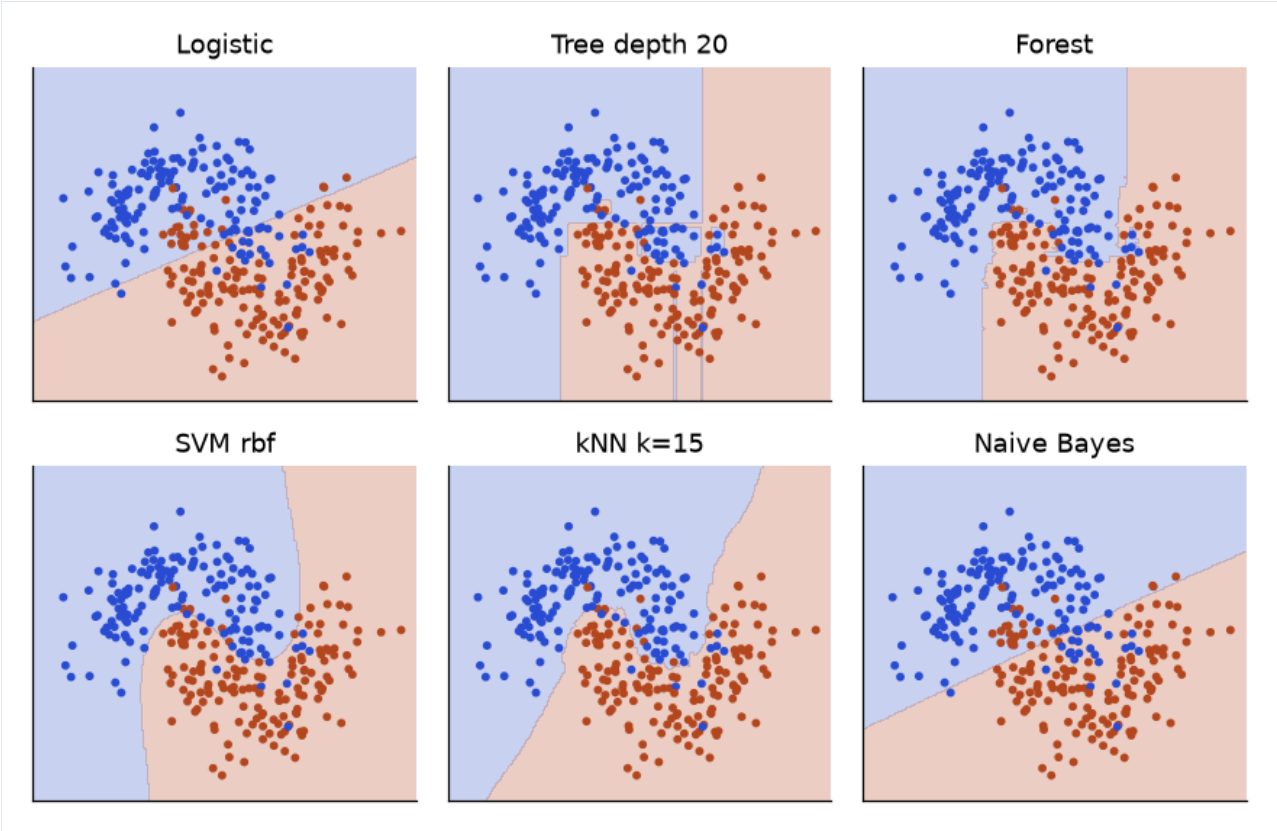
19.4 Model Families: Linear, Trees, Forests, Boosting, SVM, kNN, Naive Bayes, Clustering, PCA

You do not need to know all the algorithms deeply to use them well. You need to know what each assumes, what it is fast or slow at, and which settings matter. All of them share the interface from 19.2, so swapping one for another is a one-line change.

Reference: the models worth knowing

Class	What it does in plain English	Minimal example
`LinearRegression`	Best straight-line fit by least squares	`LinearRegression().fit(X, y)`
`Ridge`	Linear fit with a penalty that shrinks weights (helps with correlated columns)	`Ridge(alpha=1.0)`
`Lasso`	Linear fit that pushes some weights to exactly zero (selects features)	`Lasso(alpha=0.1)`
`ElasticNet`	Mix of Ridge and Lasso penalties	`ElasticNet(alpha=0.1, l1_ratio=0.5)`
`LogisticRegression`	Linear classifier giving probabilities; strong first baseline	`LogisticRegression(C=1.0, max_iter=500)`
`SGDClassifier`	Linear model trained in small steps; handles huge data and `partial_fit`	`SGDClassifier(loss="log_loss")`
`DecisionTreeClassifier`	A flowchart of yes/no questions on features	`DecisionTreeClassifier(max_depth=4)`
`DecisionTreeRegressor`	Same flowchart, leaves hold average values	`DecisionTreeRegressor(min_samples_leaf=20)`
`RandomForestClassifier`	Hundreds of de-correlated trees voting	`RandomForestClassifier(300, n_jobs=-1)`
`RandomForestRegressor`	Forest that averages numbers	`RandomForestRegressor(300)`
`ExtraTreesClassifier`	Forest with random split points; faster, often similar	`ExtraTreesClassifier(300)`
`HistGradientBoostingClassifier`	Fast boosted trees; handles missing values and categories natively	`HistGradientBoostingClassifier(max_iter=200)`
`HistGradientBoostingRegressor`	Same for numbers	`HistGradientBoostingRegressor()`
`GradientBoostingClassifier`	Original (slower) boosting; fine for small data	`GradientBoostingClassifier(n_estimators=100)`
`AdaBoostClassifier`	Boosting that re-weights hard rows	`AdaBoostClassifier()`

Class	What it does in plain English	Minimal example
`BaggingClassifier`	Bootstrap-and-average wrapper around any model	`BaggingClassifier(estimator=SVC())`
`VotingClassifier`, `StackingClassifier`	Combine different models by voting or by a meta-model	`VotingClassifier([("lr", lr), ("rf", rf)])`
`SVC`	Support vector machine with curved boundaries (kernel); needs scaling. For probabilities wrap it in `CalibratedClassifierCV` (its built-in `probability=True` is deprecated in 1.9)	`SVC(C=1, kernel="rbf")`
`LinearSVC`	Linear SVM; scales to many rows and sparse text	`LinearSVC()`
`KNeighborsClassifier`	Vote among the k nearest training rows	`KNeighborsClassifier(n_neighbors=15)`
`GaussianNB`	Naive Bayes for continuous columns	`GaussianNB()`
`MultinomialNB`	Naive Bayes for word counts; tiny and fast for text	`MultinomialNB(alpha=0.5)`
`MLPClassifier`	Small neural network	`MLPClassifier((50,), max_iter=500)`
`DummyClassifier`, `DummyRegressor`	Baselines that ignore features	`DummyClassifier(strategy="prior")`
`KMeans`	Split rows into k round groups around centres	`KMeans(4, n_init=10, random_state=0)`
`MiniBatchKMeans`	Faster k-means for big data	`MiniBatchKMeans(8)`
`DBSCAN`, `HDBSCAN`	Density-based clusters of any shape; mark noise as -1	`DBSCAN(eps=0.5, min_samples=5)`
`AgglomerativeClustering`	Merge nearest groups step by step into a hierarchy	`AgglomerativeClustering(3)`
`GaussianMixture`	Soft clusters as overlapping bell shapes	`GaussianMixture(3, random_state=0)`
`PCA`	Rotate to new axes ordered by variance; keep the top few	`PCA(n_components=0.95)`
`TruncatedSVD`	PCA for sparse matrices (text)	`TruncatedSVD(100)`
`IsolationForest`, `LocalOutlierFactor`	Flag rows that look unusual	`IsolationForest(contamination=0.02)`



The same 300 points, six models. Each one draws a different shape of boundary.

How to read it: each panel colours the plane by the predicted class. Logistic regression and (here) naive Bayes draw a smooth, nearly straight divide and cannot follow the moon curves. The unpruned tree draws boxy, jagged islands that chase individual points (overfitting). The SVM and kNN follow the curve smoothly, and the forest follows it in boxy steps. The picture is the reason to try several families: the right one depends on the shape of the problem.

Example 1: one loop, many models

Because the interface is uniform, comparing families is a short loop. Scale-sensitive models (SVM, kNN, logistic) get a `StandardScaler` in a pipeline.

```
from sklearn.datasets import load_breast_cancer
from sklearn.dummy import DummyClassifier
from sklearn.ensemble import HistGradientBoostingClassifier, RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier

X, y = load_breast_cancer(return_X_y=True)
cv = StratifiedKFold(5, shuffle=True, random_state=0)
models = {
    "dummy": DummyClassifier(strategy="prior"),
    "logistic": make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000)),
    "tree": DecisionTreeClassifier(max_depth=4, random_state=0),
    "forest": RandomForestClassifier(200, random_state=0),
    "hist boosting": HistGradientBoostingClassifier(random_state=0),
    "svm": make_pipeline(StandardScaler(), SVC()),
    "knn": make_pipeline(StandardScaler(), KNeighborsClassifier(15)),
    "naive bayes": GaussianNB(),
}
for name, m in models.items():
    s = cross_val_score(m, X, y, cv=cv)
    print(f"{name:14s} {s.mean():.3f} +/- {s.std():.3f}")
```

```
dummy          0.627 +/- 0.004
logistic       0.979 +/- 0.014
tree           0.926 +/- 0.025
forest         0.967 +/- 0.019
hist boosting  0.970 +/- 0.012
svm            0.977 +/- 0.007
knn            0.961 +/- 0.011
naive bayes    0.939 +/- 0.020
```

1. The breast-cancer dataset has 569 rows, 30 numeric features and a binary target. `return_X_y=True` returns just the arrays.
2. `StratifiedKFold(5, shuffle=True, random_state=0)` defines the same five folds for every model, so the comparison is fair.
3. The dictionary maps a name to an unfitted model. The dummy model always predicts the most common class and sets the floor any real model must beat.
4. Scale-sensitive models are wrapped with `make_pipeline(StandardScaler(), ...)`. Trees, forests, boosting and Naive Bayes do not need scaling.
5. `cross_val_score` fits and scores each model on the five folds and returns five numbers; we print the mean and the spread.

Reading the output: the dummy sits at about 0.627 (the share of the common class). Every real model beats it clearly, and the scaled linear and kernel models are at the top. The differences between the best few are within about one standard deviation of each other, so on 569 rows you cannot declare a winner from this table. Report the spread, not just the mean.

Example 2: linear models, penalties and coefficients

Ridge and Lasso add a penalty to the fit. The strength is `alpha`. A bigger alpha means smaller weights. Lasso can set weights to exactly zero.

```
import numpy as np
from sklearn.datasets import load_diabetes
from sklearn.linear_model import Lasso, LinearRegression, Ridge
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

X, y = load_diabetes(return_X_y=True)
names = load_diabetes().feature_names
X_tr, X_te, y_tr, y_te = train_test_split(X, y, random_state=0)

for label, est in [("ols", LinearRegression()), ("ridge a=10", Ridge(alpha=10)),
                   ("lasso a=2", Lasso(alpha=2.0))]:
    m = make_pipeline(StandardScaler(), est).fit(X_tr, y_tr)
    coef = m[-1].coef_
    print(f"{label:11s} R2={m.score(X_te, y_te):.3f} zeros={int((coef == 0).sum())}",
          np.round(coef[:4], 1))
print(names[:4])
```

```
ols         R2=0.359 zeros=0 [-2.1 -9.9 28.9 14.3]
ridge a=10  R2=0.358 zeros=0 [-1.7 -9.5 28.4 13.9]
lasso a=2   R2=0.355 zeros=3 [-0.  -6.4 28.8 11.5]
['age', 'sex', 'bmi', 'bp']
```

1. The diabetes dataset has 442 patients, 10 numeric features, and a disease-progress score as the target.
2. We split once and loop over three linear models: plain least squares, Ridge with alpha 10, and Lasso with alpha 2.
3. Each model gets a scaler first. Penalised models are **scale-dependent**, because a penalty on weights treats a column in grams differently from the same column in kilograms.
4. `m[-1]` is the last pipeline step (the linear model); `coef_` holds its weights.
5. We print the test R squared, how many weights are exactly zero, and the first four weights.

Reading the output: Ridge shrinks the weights toward zero but keeps all ten. Lasso at alpha 2 sets three of the ten weights to exactly zero (the first one, `age`, prints as -0.0), which is built-in feature selection. The test R squared values are almost identical, 0.359, 0.358 and 0.355: on this noisy target, shrinking does not add accuracy, but Lasso gives a smaller model with the same score, and Ridge gives more stable weights when columns are correlated.

Example 3: a tree you can read, and how depth causes overfitting

```
from sklearn.datasets import load_iris, make_moons
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, export_text

iris = load_iris()
small = DecisionTreeClassifier(max_depth=2, random_state=0).fit(iris.data, iris.target)
print(export_text(small, feature_names=list(iris.feature_names)))

X, y = make_moons(600, noise=0.35, random_state=1)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=1)
for d in (1, 3, 6, None):
    t = DecisionTreeClassifier(max_depth=d, random_state=0).fit(X_tr, y_tr)
    print(f"depth={d!s:4s} leaves={t.get_n_leaves():3d} "
          f"train={t.score(X_tr, y_tr):.3f} test={t.score(X_te, y_te):.3f}")
```

```
|--- petal width (cm) <= 0.80
|   |--- class: 0
|--- petal width (cm) >  0.80
|   |--- petal width (cm) <= 1.75
|   |   |--- class: 1
|   |--- petal width (cm) >  1.75
|   |   |--- class: 2

depth=1    leaves=  2 train=0.783 test=0.783
depth=3    leaves=  7 train=0.874 test=0.850
depth=6    leaves= 23 train=0.910 test=0.828
depth=None leaves= 68 train=1.000 test=0.822
```

1. `export_text` prints the learned flowchart as indented rules, so a non-technical colleague can read it. We fit it on the iris flowers.
2. A depth-2 tree asks at most two questions per flower. Each line shows a question, and each `class:` line is a leaf with its prediction.
3. For the depth experiment we switch to the noisy two-moons data, where overfitting is easy to see. We hold out 30% for testing.
4. The loop trains trees of increasing depth. `None` means grow until every leaf is pure.
5. `get_n_leaves()` counts the final boxes. More leaves means a more flexible model.
6. We print train and test accuracy for each depth.

Reading the output: the iris tree needs only the petal width, two cuts at 0.80 and 1.75. On the moons, train accuracy climbs from 0.783 to 1.000 as the tree grows to 68 leaves, while test accuracy peaks at 0.850 for depth 3 and falls to 0.822 with no limit. A perfect training score from an unlimited tree only says it memorised the rows. This train-versus-test gap is the signature of overfitting, and the fix is to limit depth or `min_samples_leaf`, or to average many trees (a forest).

Example 4: forests and boosting, including missing values

A random forest averages many deep, de-correlated trees. Gradient boosting builds shallow trees one after another, each fixing the errors of the last. `HistGradientBoosting*` bins numbers into 255 buckets, which makes it fast, and it accepts `NaN` directly.

```
import numpy as np
from sklearn.datasets import make_classification
from sklearn.ensemble import HistGradientBoostingClassifier, RandomForestClassifier
from sklearn.model_selection import train_test_split

X, y = make_classification(4000, 20, n_informative=6, random_state=0)
rng = np.random.default_rng(0)
X[rng.random(X.shape) < 0.1] = np.nan          # blank out 10% of cells
X_tr, X_te, y_tr, y_te = train_test_split(X, y, random_state=0)

hgb = HistGradientBoostingClassifier(max_iter=200, learning_rate=0.1,
                                     early_stopping=True, random_state=0)
hgb.fit(X_tr, y_tr)
print("hist boosting test acc:", round(hgb.score(X_te, y_te), 3),
      "iterations used:", hgb.n_iter_)

rf = RandomForestClassifier(200, oob_score=True, random_state=0)
rf.fit(X_tr, y_tr)
print("forest test acc:", round(rf.score(X_te, y_te), 3))
print("forest OOB score:", round(rf.oob_score_, 3))
```

```
hist boosting test acc: 0.889 iterations used: 75
forest test acc: 0.876
forest OOB score: 0.876
```

1. We make 4,000 rows and then blank out 10% of all cells at random to simulate a gappy table.
2. The histogram booster is fitted directly on data containing `NaN`. It learns, at each split, which side the missing rows should go to.

3. `early_stopping=True` holds out 10% of training rows internally and stops adding trees when the score stops improving, so `n_iter_` can be below `max_iter`.
4. Recent scikit-learn versions let `RandomForestClassifier` (and decision trees) take `NaN` too, so the forest is fitted on the same gappy matrix. Not every estimator does: logistic regression, SVM and kNN still raise an error, so check the docs or impute in a pipeline.
5. `oob_score=True` asks the forest to evaluate each row using only the trees whose bootstrap sample did not include it. This gives a free validation estimate without a separate split.

Reading the output: boosting reaches 0.889 test accuracy and stopped after 75 trees because early stopping saw no further gain. The forest's test accuracy is 0.876 and its out-of-bag score is 0.876, the same to three decimals here (they will not always agree this closely), which is why OOB is a useful quick check.

INTUITION	**Forest versus boosting.** A forest is a committee of independent experts who each saw a different random slice of the data; averaging cancels their individual quirks (it lowers variance). Boosting is an apprentice chain: each new small tree is trained on what the previous ones still get wrong (it lowers bias). Boosting usually wins on tables when tuned, and a forest is harder to break when you do not tune.

Example 5: why scaling matters for kNN and SVM

```
import numpy as np
from sklearn.model_selection import cross_val_score
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

rng = np.random.default_rng(0)
n = 600
signal = rng.normal(size=n)                       # small-scale informative column
y = (signal + rng.normal(scale=0.5, size=n) > 0).astype(int)
big_noise = rng.normal(scale=1000, size=n)         # huge-scale useless column
X = np.c_[signal, big_noise]

raw = KNeighborsClassifier(15)
scaled = make_pipeline(StandardScaler(), KNeighborsClassifier(15))
print("raw   :", cross_val_score(raw, X, y, cv=5).mean().round(3))
print("scaled:", cross_val_score(scaled, X, y, cv=5).mean().round(3))
```

```
raw   : 0.518
scaled: 0.877
```

1. One informative column with unit scale, and one useless column with a standard deviation of 1,000.
2. kNN measures distances between rows. Without scaling, the distance is dominated almost entirely by the big useless column.
3. The scaled version puts both columns on equal footing, so the informative one can do its work.
4. The raw model scores near coin-flip, and the scaled model scores far better.

Example 6: unsupervised families, clustering and PCA

```
import numpy as np
from sklearn.cluster import DBSCAN, KMeans
from sklearn.datasets import load_wine, make_moons
from sklearn.decomposition import PCA
from sklearn.metrics import adjusted_rand_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = load_wine(return_X_y=True)
Xs = StandardScaler().fit_transform(X)
pca = PCA().fit(Xs)
print("explained variance ratio:", pca.explained_variance_ratio_[:4].round(3))
print("components for 90%:", int(np.searchsorted(np.cumsum(pca.explained_variance_ratio_), 0.90) + 1))

Z = PCA(n_components=2).fit_transform(Xs)
km = KMeans(3, n_init=10, random_state=0).fit(Z)
print("kmeans vs true wine class, ARI:", round(adjusted_rand_score(y, km.labels_), 3))

M, truth = make_moons(400, noise=0.07, random_state=0)
km2 = KMeans(2, n_init=10, random_state=0).fit(M)
db = DBSCAN(eps=0.2, min_samples=5).fit(M)
print("moons ARI, kmeans:", round(adjusted_rand_score(truth, km2.labels_), 3))
print("moons ARI, dbscan:", round(adjusted_rand_score(truth, db.labels_), 3),
      "clusters:", len(set(db.labels_) - {-1}), "noise:", int((db.labels_ == -1).sum()))
```

```
explained variance ratio: [0.362 0.192 0.111 0.071]
components for 90%: 8
kmeans vs true wine class, ARI: 0.895
moons ARI, kmeans: 0.269
moons ARI, dbscan: 1.0 clusters: 2 noise: 0
```

1. The wine data has 178 rows and 13 chemical measurements. PCA is scale-sensitive, so we standardise first.
2. `explained_variance_ratio_` says what fraction of the total variation each new axis carries, in decreasing order. The cumulative sum tells us how many axes keep 90%.
3. We keep two components, cluster the 2-D points with k-means (k=3), and compare to the true wine classes with the adjusted Rand index (1.0 is a perfect match, around 0 is random).
4. On the two-moons data we know the true grouping. k-means is asked for two clusters; it cuts the plane with a straight divide, which does not follow the moons.
5. `DBSCAN` grows clusters from dense neighbourhoods and can follow curved shapes. Label -1 means noise. We count the clusters found and the noise points, and score both against the truth with the adjusted Rand index.

Reading the output: the first PCA axis carries 36.2% of the variance and the second 19.2%, so two axes keep about 55% and 8 axes are needed for 90%. Even so, k-means on just two axes recovers the true wine classes well (ARI 0.895). On the moons, k-means scores an ARI of only 0.269 because round clusters are the wrong assumption, while DBSCAN scores 1.0, finding the two moons exactly with no noise points. The lesson: pick the clustering method to match the shape you expect.

NOT RUN HERE	XGBoost and LightGBM are separate libraries with the same fit/predict interface. They are usually the fastest, strongest boosters on large tables. The code below is idiomatic but was **not run here** because neither library is installed. For a runnable equivalent use `HistGradientBoostingClassifier`, whose design is similar to LightGBM's.

```
# not run here: needs `pip install xgboost lightgbm`
from xgboost import XGBClassifier
from lightgbm import LGBMClassifier

xgb = XGBClassifier(n_estimators=400, learning_rate=0.05, max_depth=4,
                    subsample=0.8, colsample_bytree=0.8, eval_metric="logloss")
lgbm = LGBMClassifier(n_estimators=400, learning_rate=0.05, num_leaves=31)
# both drop straight into Pipeline, cross_val_score and GridSearchCV
```

19.5 Cross-Validation, GridSearchCV and RandomizedSearchCV

One train/test split gives one number, and one number can be lucky. **Cross-validation (CV)** splits the training data into k folds, trains k times (each time holding out a different fold), and gives you k scores. The mean tells you the typical quality, and the spread tells you how much to trust it. Hyper-parameter searches are CV run once for every candidate setting.

Reference: validation and tuning tools

Name	What it does in plain English	Minimal example
`cross_val_score`	k scores for one model and one metric	`cross_val_score(m, X, y, cv=5)`
`cross_validate`	Several metrics at once, plus train scores and timings	`cross_validate(m, X, y, scoring=["accuracy", "f1"])`
`cross_val_predict`	Out-of-fold prediction for every row (for plots and stacking)	`cross_val_predict(m, X, y, cv=5, method="predict_proba")`
`KFold`	Plain k folds	`KFold(5, shuffle=True, random_state=0)`
`StratifiedKFold`	Folds that keep the class ratio (the default for classifiers when `cv` is an int)	`StratifiedKFold(5, shuffle=True, random_state=0)`
`RepeatedStratifiedKFold`	Repeat CV with different shuffles for a steadier mean	`RepeatedStratifiedKFold(n_splits=5, n_repeats=3)`
`GroupKFold`	Never split one group (patient, customer) across train and validation	`cross_val_score(m, X, y, groups=g, cv=GroupKFold(5))`
`StratifiedGroupKFold`	Groups kept together and classes balanced	`StratifiedGroupKFold(5)`
`TimeSeriesSplit`	Train on the past, validate on the future, in order	`TimeSeriesSplit(n_splits=5)`
`ShuffleSplit`	Many random splits of chosen sizes	`ShuffleSplit(10, test_size=0.2)`
`LeaveOneOut`	Each row is validation once (tiny data only; slow)	`LeaveOneOut()`
`GridSearchCV`	Try every combination in a grid, CV each, keep the best	`GridSearchCV(pipe, {"clf__C": [0.1, 1, 10]}, cv=5)`
`RandomizedSearchCV`	Try n random combinations from lists or distributions	`RandomizedSearchCV(m, dist, n_iter=20, random_state=0)`
`HalvingGridSearchCV`	Start many candidates on little data, keep the best (experimental import)	`HalvingGridSearchCV(m, grid)`

Name	What it does in plain English	Minimal example
`search.best_params_`, `best_score_`	The winning settings and their mean CV score	`gs.best_params_`
`search.best_estimator_`	The winning model, refitted on all training data	`gs.best_estimator_.predict(X_te)`
`search.cv_results_`	Every candidate's scores; wrap in a DataFrame	`pd.DataFrame(gs.cv_results_)`
`scoring=`	Which metric to optimise: a string or a scorer	`scoring="roc_auc"`
`make_scorer`	Turn any metric function into a scorer	`make_scorer(fbeta_score, beta=2)`
`get_scorer_names()`	List the valid scoring strings	`sklearn.metrics.get_scorer_names()`
`learning_curve`	Score as the training set grows (do I need more data?)	`learning_curve(m, X, y, cv=5)`
`validation_curve`	Score as one setting changes (am I over- or under-fitting?)	`validation_curve(m, X, y, param_name="max_depth", param_range=r)`
`permutation_test_score`	Is my CV score better than shuffled-label chance?	`permutation_test_score(m, X, y)`
`TunedThresholdClassifierCV`	Pick the decision threshold by CV for a chosen metric	`TunedThresholdClassifierCV(m, scoring="f1")`

Note on scoring names: scikit-learn maximises scores, so error metrics are negated. `"neg_mean_squared_error"` and `"neg_root_mean_squared_error"` are negative numbers, and "higher is better" holds for all of them.

Example 1: cross_val_score versus cross_validate

```
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score, cross_validate
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = load_breast_cancer(return_X_y=True)
pipe = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))

print(cross_val_score(pipe, X, y, cv=5).round(3))
res = cross_validate(pipe, X, y, cv=5, scoring=["accuracy", "roc_auc", "f1"],
                     return_train_score=True)
print(sorted(res.keys()))
for k in ("test_accuracy", "train_accuracy", "test_roc_auc", "test_f1"):
    print(f"{k:15s} mean={res[k].mean():.3f}")
```

```
[0.982 0.982 0.974 0.974 0.991]
['fit_time', 'score_time', 'test_accuracy', 'test_f1', 'test_roc_auc', 'train_accuracy', 'train_f1', 'train_roc_auc']
test_accuracy   mean=0.981
train_accuracy  mean=0.989
test_roc_auc    mean=0.995
test_f1         mean=0.985
```

1. The pipeline scales then fits logistic regression. Because the scaler is inside, it is re-fitted in every fold on that fold's training rows (no leakage).
2. `cross_val_score(..., cv=5)` returns five accuracy scores, one per fold. With an integer `cv` and a classifier it uses stratified folds without shuffling.
3. `cross_validate` accepts a list of metrics and returns a dictionary of arrays: one entry per metric as `test_*`, optionally `train_*`, plus `fit_time` and `score_time`.
4. We print the sorted keys so you can see what is available, then the mean of four of them.
5. A train accuracy only slightly above the test accuracy means little overfitting. A large gap means the model memorised.

Reading the output: the five fold accuracies range from 0.974 to 0.991, so even a good model moves by about two points depending on the fold. Mean test accuracy is 0.981 against 0.989 on the training folds, a gap of under one point, so overfitting is small. ROC AUC (0.995) and F1 (0.985) tell the same story from other angles.

Example 2: choosing the right splitter (groups and time)

Standard k-fold assumes rows are independent. When several rows come from the same person, or rows are ordered in time, a random split leaks. Here each of 60 customers has 20 rows, and the label is a property of the customer, unrelated to the features beyond the customer's "signature".

```
import numpy as np
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GroupKFold, KFold, TimeSeriesSplit, cross_val_score

rng = np.random.default_rng(0)
n_groups, per = 60, 20
signature = rng.normal(size=(n_groups, 5))            # fixed fingerprint per customer
label_of_group = rng.integers(0, 2, n_groups)          # label unrelated to the fingerprint
groups = np.repeat(np.arange(n_groups), per)
X = signature[groups] + rng.normal(scale=0.1, size=(n_groups * per, 5))
y = label_of_group[groups]

rf = RandomForestClassifier(100, random_state=0)
random_cv = KFold(5, shuffle=True, random_state=0)
print("random KFold :", cross_val_score(rf, X, y, cv=random_cv).mean().round(3))
print("GroupKFold   :", cross_val_score(rf, X, y, cv=GroupKFold(5), groups=groups).mean().round(3))

for tr, va in TimeSeriesSplit(n_splits=3).split(np.arange(12)):
    print("train", tr.tolist(), "-> validate", va.tolist())
```

```
random KFold : 0.992
GroupKFold   : 0.515
train [0, 1, 2] -> validate [3, 4, 5]
train [0, 1, 2, 3, 4, 5] -> validate [6, 7, 8]
train [0, 1, 2, 3, 4, 5, 6, 7, 8] -> validate [9, 10, 11]
```

1. Each customer has a random 5-number fingerprint, and a label that is a coin flip per customer. By construction nothing in the fingerprint predicts the label for a *new* customer.
2. Each of a customer's 20 rows is the fingerprint plus small noise. So rows from one customer are near-duplicates.
3. With shuffled `KFold`, a customer's rows land in both training and validation. The forest simply recognises the fingerprint and recalls the label, scoring very high.
4. `GroupKFold` with `groups=` keeps all rows of a customer in the same fold. Validation customers are truly unseen, and the score falls to chance level (about 0.5).
5. `TimeSeriesSplit` yields expanding training windows that always come before the validation window. The printed index lists show it for 12 ordered rows.

Reading the output: shuffled KFold reports 0.992, a near-perfect score for a problem where the true answer is a coin flip. GroupKFold reports 0.515, which is the honest number. The time-series splits show the training window growing (3, 6, then 9 rows) and always ending before the validation rows begin.

COMMON MISTAKE

Wrong: `cross_val_score(model, X, y, cv=KFold(5, shuffle=True))` on data with repeated customers, patients, sensors or time order. **Symptom:** CV score of 0.98, production score of 0.55. **Fix:** ask "will the new data contain entities already in training?". If not, use `GroupKFold`; if rows are ordered in time, use `TimeSeriesSplit`.

Example 3: GridSearchCV on a whole pipeline

```
import pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import GridSearchCV, train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

X, y = load_breast_cancer(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, stratify=y, random_state=0)

pipe = Pipeline([("scale", StandardScaler()), ("svm", SVC())])
grid = {"svm__C": [0.1, 1, 10], "svm__gamma": ["scale", 0.01, 0.1]}
gs = GridSearchCV(pipe, grid, cv=5, scoring="roc_auc")
gs.fit(X_tr, y_tr)

print("best params:", gs.best_params_)
print("best CV auc:", round(gs.best_score_, 4))
top = pd.DataFrame(gs.cv_results_).sort_values("rank_test_score").head(3)
print(top[["param_svm__C", "param_svm__gamma", "mean_test_score", "std_test_score"]].round(4).to_string(index=False))
print("test auc   :", round(gs.score(X_te, y_te), 4))
```

```
best params: {'svm__C': 1, 'svm__gamma': 'scale'}
best CV auc: 0.9959
 param_svm__C param_svm__gamma  mean_test_score  std_test_score
          1.0            scale           0.9959          0.0052
          1.0             0.01           0.9959          0.0060
         10.0             0.01           0.9951          0.0077
test auc   : 0.9927
```

1. We split off a test set first. The search must never see it.
2. The pipeline has two named steps. In the grid, a parameter of a step is written as `stepname__parameter`, so `svm__C` means the `C` of the step called `svm`.
3. The grid has 3 values of C times 3 of gamma, which is 9 candidates; with 5 folds, 45 fits, plus one final refit on all training rows (`refit=True` is the default).
4. The metric `"roc_auc"` is used to rank candidates.
5. `best_params_` and `best_score_` report the winner and its mean CV score.
6. `cv_results_` is a dictionary of arrays; as a DataFrame sorted by `rank_test_score` we see the top three settings with their means and standard deviations.
7. `gs.score(X_te, y_te)` scores the refitted best pipeline on the untouched test set. This is the number to report.

Reading the output: the winner is C=1 with gamma left at `scale`, CV AUC 0.9959, and the held-out test AUC is 0.9927, close to the CV figure, so the search did not overfit the folds. The top two candidates tie to four decimals (0.9959 each, with standard deviations of about 0.005 to 0.006), so the "best" is not meaningfully better than the second. When means differ by less than their standard deviations, prefer the simpler or cheaper setting.

Example 4: RandomizedSearchCV with distributions

When there are many settings, a grid explodes (5 values for each of 4 settings is 625 candidates). Random search samples a fixed number of combinations and is usually just as good. Continuous settings like `C` should be sampled on a log scale.

```
from scipy.stats import loguniform, randint
from sklearn.datasets import make_classification
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.model_selection import RandomizedSearchCV, train_test_split

X, y = make_classification(3000, 20, n_informative=8, flip_y=0.05, random_state=0)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, random_state=0)

dist = {"learning_rate": loguniform(0.02, 0.3),
        "max_leaf_nodes": randint(8, 64),
        "min_samples_leaf": randint(10, 80),
        "l2_regularization": loguniform(1e-3, 10)}
rs = RandomizedSearchCV(HistGradientBoostingClassifier(max_iter=150, random_state=0),
                        dist, n_iter=12, cv=3, scoring="accuracy", random_state=0)
rs.fit(X_tr, y_tr)
print({k: (round(v, 3) if isinstance(v, float) else int(v)) for k, v in rs.best_params_.items()})
print("CV:", round(rs.best_score_, 3), " test:", round(rs.score(X_te, y_te), 3))
```

```
{'l2_regularization': np.float64(0.081), 'learning_rate': np.float64(0.18), 'max_leaf_nodes': 54, 'min_samples_leaf': 47}
CV: 0.925  test: 0.919
```

1. `loguniform(a, b)` samples numbers so that every power of ten between a and b is equally likely, which is right for rates and penalties. `randint(a, b)` samples whole numbers.
2. The dictionary maps each parameter name to a list or a distribution.
3. `n_iter=12` says to try 12 random combinations, so the cost is fixed regardless of how many settings there are. `random_state=0` makes the sample repeatable.
4. After the search, `best_params_` holds the winner; we round for printing.
5. CV score and test score are printed side by side as a sanity check.

Reading the output: the CV score of the winner is 0.925 and the test accuracy is 0.919. The test number is a little lower, the typical size of the winner's curse.

KEY IDEA	The CV score of the *winning* candidate is slightly optimistic, because you picked it for being the best of many noisy scores. For an honest estimate use a test set that the search never touched, or **nested CV** (a search inside an outer cross-validation).

Example 5: reading a validation curve

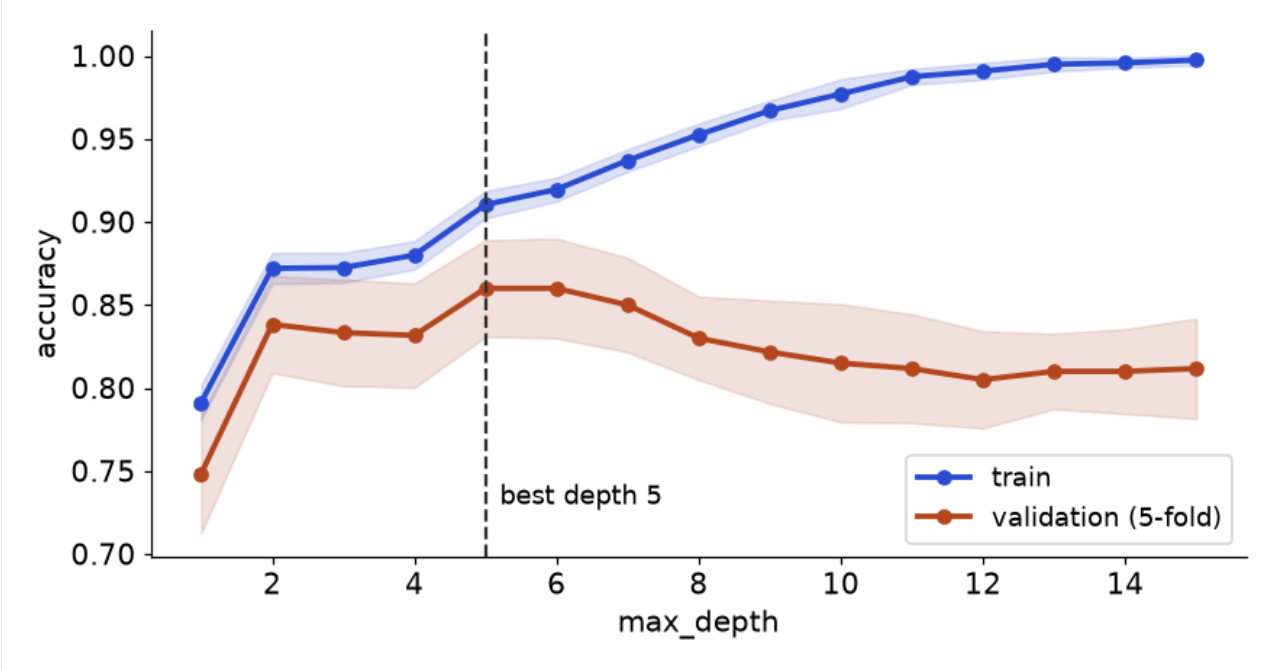
```
import numpy as np
from sklearn.datasets import make_moons
from sklearn.model_selection import validation_curve
from sklearn.tree import DecisionTreeClassifier

X, y = make_moons(600, noise=0.35, random_state=1)
depths = np.arange(1, 16)
tr, va = validation_curve(DecisionTreeClassifier(random_state=0), X, y,
                          param_name="max_depth", param_range=depths, cv=5)
best = depths[va.mean(axis=1).argmax()]
print("best depth:", best, " val acc:", va.mean(axis=1).max().round(3))
print("depth 15  : train", tr.mean(axis=1)[-1].round(3), " val", va.mean(axis=1)[-1].round(3))
```

```
best depth: 5  val acc: 0.86
depth 15  : train 0.997  val 0.812
```

1. `validation_curve` trains the model with each value of one setting, with 5-fold CV, and returns two arrays of shape (values, folds): training scores and validation scores.

2. We average over folds (axis 1) and find the depth where the validation mean peaks.
3. We also show depth 15, where the tree is far too flexible.

Reading the output: validation accuracy peaks at 0.86 for depth 5. At depth 15 the tree scores 0.997 on training folds but only 0.812 on validation folds, a gap of 18 points.



Validation curve for a decision tree on noisy moons. The band is one standard deviation across folds.

How to read it: on the left (shallow trees) both lines are low: the model is too simple (underfitting). Moving right, training accuracy keeps climbing to 1.0 but validation peaks and then declines: the model is memorising noise (overfitting). The best setting is where validation peaks, marked by the dashed line. A big gap between the two lines is the signal to simplify or add data.

Problem 19.5.1 You grid-search 4 values of one setting and 3 values of another, with 5-fold CV and `refit=True`. How many model fits happen?

Candidates	4 × 3 = 12 combinations.
CV fits	12 × 5 = 60 fits (each on 80% of the training rows).
Refit	One more fit of the best candidate on all training rows.
Answer	61 fits. Adding a third setting with 5 values would make it 60 × 5 + 1 = 301, which is why random search exists.

19.6 Metrics, Class Imbalance and Calibration

A model is only "good" relative to a question. "Be right as often as possible" and "catch every fraud without drowning the analysts" are different questions with different metrics. This section gives you the metrics, then shows how imbalance breaks the default one and how to fix it.

Reference: the metrics worth knowing

Function	What it measures, in plain English	Minimal example
`accuracy_score`	Share of predictions that are right (misleading when classes are imbalanced)	`accuracy_score(y, p)`
`balanced_accuracy_score`	Average recall over classes	`balanced_accuracy_score(y, p)`

Function	What it measures, in plain English	Minimal example
`confusion_matrix`	Counts of true vs predicted classes	`confusion_matrix(y, p)`
`precision_score`	Of those I flagged, how many were real?	`precision_score(y, p)`
`recall_score`	Of the real ones, how many did I catch?	`recall_score(y, p)`
`f1_score`, `fbeta_score`	Harmonic mean of precision and recall; beta weights recall	`fbeta_score(y, p, beta=2)`
`classification_report`	Per-class precision, recall, F1 and support in one table	`classification_report(y, p)`
`roc_auc_score`	Chance that a random positive outranks a random negative	`roc_auc_score(y, proba)`
`roc_curve`	True positive rate vs false positive rate over thresholds	`roc_curve(y, proba)`
`average_precision_score`	Area-like summary of the precision-recall curve (best for rare positives)	`average_precision_score(y, proba)`
`precision_recall_curve`	Precision and recall at every threshold	`precision_recall_curve(y, proba)`
`log_loss`	Penalty for confident wrong probabilities	`log_loss(y, proba)`
`brier_score_loss`	Mean squared error of the probability	`brier_score_loss(y, proba)`
`matthews_corrcoef`	One balanced number from the whole confusion matrix	`matthews_corrcoef(y, p)`
`cohen_kappa_score`	Agreement beyond chance	`cohen_kappa_score(a, b)`
`ConfusionMatrixDisplay`, `RocCurveDisplay`	Ready-made plots	`RocCurveDisplay.from_predictions(y, p)`
`mean_absolute_error`	Average size of the error, in target units	`mean_absolute_error(y, p)`
`mean_squared_error`, `root_mean_squared_error`	Squares errors (punishes big misses); RMSE is in target units	`root_mean_squared_error(y, p)`
`median_absolute_error`	Typical error, ignoring outliers	`median_absolute_error(y, p)`
`mean_absolute_percentage_error`	Average relative error (breaks when y is near 0)	`mean_absolute_percentage_error(y, p)`
`r2_score`	Share of variance explained; 0 means "no better than predicting the mean"	`r2_score(y, p)`
`PredictionErrorDisplay`	Plot of predicted vs actual or residuals	`PredictionErrorDisplay.from_estimator(m, X, y)`
`ndcg_score`	Quality of a ranked list, rewarding good items near the top	`ndcg_score([rel], [scores], k=3)`
`silhouette_score`	How well rows sit in their own cluster vs the next (-1 to 1)	`silhouette_score(X, labels)`

Function	What it measures, in plain English	Minimal example
`davies_bouldin_score`, `calinski_harabasz_score`	Cluster tightness vs separation, no labels needed	`davies_bouldin_score(X, labels)`
`adjusted_rand_score`	Agreement between two groupings, corrected for chance	`adjusted_rand_score(true, found)`
`calibration_curve`	Observed rate vs predicted probability, binned	`calibration_curve(y, proba, n_bins=10)`
`CalibratedClassifierCV`	Wrap a model so its probabilities are reliable	`CalibratedClassifierCV(m, method="isotonic", cv=5)`

For multi-class problems, precision/recall/F1 need an `average=`: `"macro"` averages the classes equally (rare classes count as much as common ones), `"micro"` pools all decisions (equal to accuracy for single-label data), and `"weighted"` averages weighted by class size.

Example 1: the confusion matrix by hand

Ten loan applications; 1 means "defaults". The model flags some.

```
import numpy as np
from sklearn.metrics import confusion_matrix, f1_score, precision_score, recall_score

y_true = np.array([1, 1, 1, 1, 0, 0, 0, 0, 0, 0])
y_pred = np.array([1, 1, 0, 0, 1, 0, 0, 0, 0, 0])

print(confusion_matrix(y_true, y_pred))
tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel()
print("tn fp fn tp:", tn, fp, fn, tp)
print("precision:", precision_score(y_true, y_pred), "= tp/(tp+fp) =", tp / (tp + fp))
print("recall   :", recall_score(y_true, y_pred), "= tp/(tp+fn) =", tp / (tp + fn))
print("f1       :", round(f1_score(y_true, y_pred), 3))
```

```
[[5 1]
 [2 2]]
tn fp fn tp: 5 1 2 2
precision: 0.6666666666666666 = tp/(tp+fp) = 0.6666666666666666
recall   : 0.5 = tp/(tp+fn) = 0.5
f1       : 0.571
```

1. Four real defaults and six real non-defaults. The model catches two defaults, misses two, and wrongly flags one good applicant.
2. `confusion_matrix` returns rows for the true class (0 then 1) and columns for the predicted class. So the top-left cell is true negatives and the bottom-right is true positives.
3. `.ravel()` flattens the 2×2 into the order tn, fp, fn, tp, which we unpack into four names.
4. Precision is tp / (tp + fp): of 3 flagged applicants, 2 were real defaults, so 2/3.
5. Recall is tp / (tp + fn): of 4 real defaults, 2 were caught, so 2/4 = 0.5.
6. F1 is the harmonic mean 2PR/(P+R) = 2 × 0.667 × 0.5 / 1.167 = 0.571.

Example 2: the accuracy trap on imbalanced data

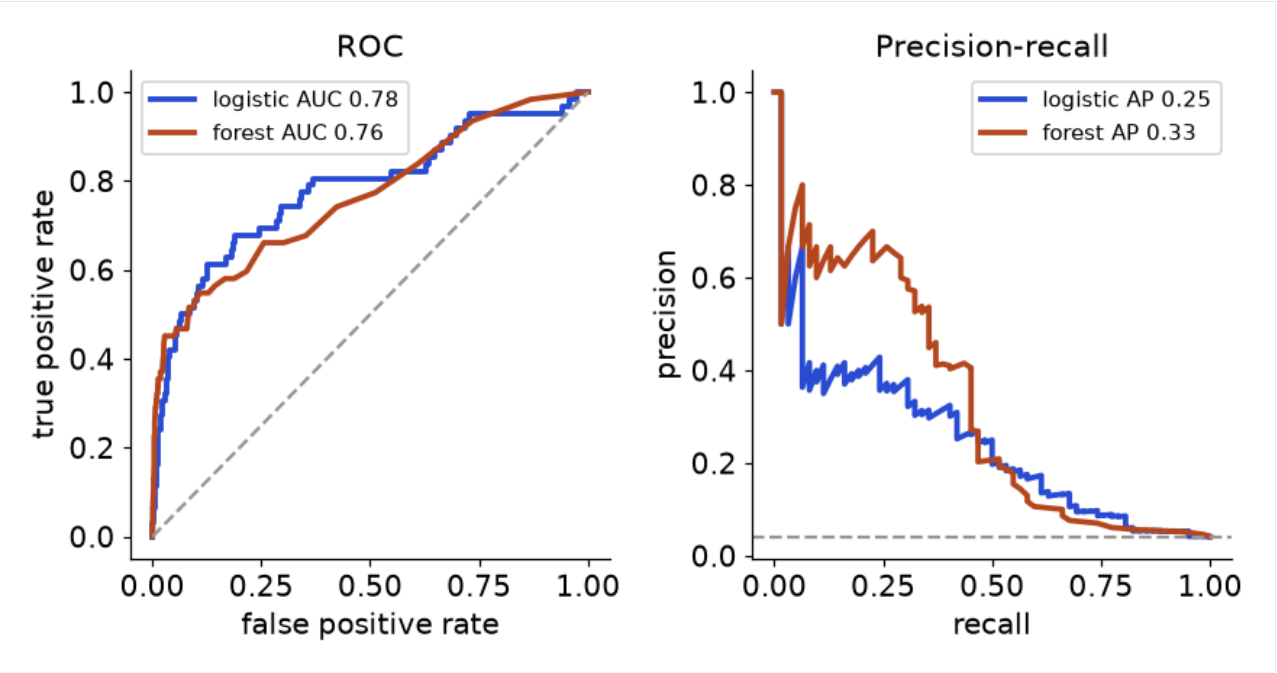
A fraud team has 1 fraudulent transaction in 100. We train a dummy that always says "not fraud" and a real model.

```
from sklearn.dummy import DummyClassifier
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.datasets import make_classification
from sklearn.metrics import (accuracy_score, average_precision_score, balanced_accuracy_score,
                             classification_report, roc_auc_score)
from sklearn.model_selection import train_test_split

X, y = make_classification(20000, 15, n_informative=5, weights=[0.99], flip_y=0.0,
                           class_sep=0.5, random_state=0)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, stratify=y, random_state=0)
print("positive rate:", round(y.mean(), 4))

dummy = DummyClassifier(strategy="most_frequent").fit(X_tr, y_tr)
model = HistGradientBoostingClassifier(random_state=0).fit(X_tr, y_tr)
for name, m in [("dummy", dummy), ("hist boosting", model)]:
    p = m.predict(X_te)
    print(f"{name:14s} acc={accuracy_score(y_te, p):.4f} "
          f"bal_acc={balanced_accuracy_score(y_te, p):.3f}")
proba = model.predict_proba(X_te)[:, 1]
print("roc_auc:", round(roc_auc_score(y_te, proba), 3),
      " avg precision:", round(average_precision_score(y_te, proba), 3))
print(classification_report(y_te, model.predict(X_te), digits=3))
```

```
positive rate: 0.01
dummy          acc=0.9900 bal_acc=0.500
hist boosting  acc=0.9922 bal_acc=0.659
roc_auc: 0.964  avg precision: 0.503
              precision    recall  f1-score   support

           0      0.993     0.999     0.996      4950
           1      0.762     0.320     0.451        50

    accuracy                          0.992      5000
   macro avg      0.878     0.659     0.723      5000
weighted avg      0.991     0.992     0.991      5000
```

1. We generate 20,000 transactions with only about 1% positives, with a stratified split so the test set has positives too.
2. The dummy always predicts the majority class. It never catches any fraud.
3. For each model we print accuracy and balanced accuracy (the mean of recall on each class).
4. Threshold-free scores use probabilities: ROC AUC and average precision.
5. `classification_report` prints precision, recall, F1 and support (row count) for each class.

Reading the output: the dummy scores 0.99 accuracy by doing nothing, and its balanced accuracy is exactly 0.5. The real model scores 0.9922, only 0.22 points higher, yet its balanced accuracy of 0.659 shows it is doing something. The class-1 row of the report is where the story is: of the 50 real frauds in the test set the model catches 32% (recall 0.320), and 76% of its alarms are right (precision 0.762). ROC AUC looks superb at 0.964, but average precision is only 0.503. With rare positives, even a small false-positive rate produces many false alarms compared with the few true cases, and average precision exposes that while ROC AUC hides it.



ROC (left) and precision-recall (right) curves on a problem with about 4% positives. The dashed line in the right panel is the positive rate, the precision of random guessing.

How to read it: in the ROC panel the diagonal is random and the top-left corner is perfect. A curve hugging the corner is strong. In the precision-recall panel the dashed line is the baseline for a random model, which equals the positive rate; a curve far above it is good. On rare-positive data the ROC curve can look comfortable while the precision-recall curve shows that high recall costs low precision. Look at both. In this picture the forest has the lower AUC (0.76 against 0.78) but the higher average precision (0.33 against 0.25), so which model is better depends on which question you ask.

Example 3: choosing a threshold

`predict` uses a 0.5 probability cut-off. That is rarely right when mistakes have different costs. Choose the cut-off from the precision-recall curve, using validation data.

```
import numpy as np
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import precision_recall_curve, precision_score, recall_score
from sklearn.model_selection import train_test_split

X, y = make_classification(10000, 12, n_informative=5, weights=[0.95], random_state=1)
X_tr, X_val, y_tr, y_val = train_test_split(X, y, stratify=y, random_state=1)
m = LogisticRegression(max_iter=1000).fit(X_tr, y_tr)
proba = m.predict_proba(X_val)[:, 1]

prec, rec, thr = precision_recall_curve(y_val, proba)
ok = np.where(rec[:-1] >= 0.80)[0]          # thresholds that keep recall at 80% or more
best = ok[np.argmax(prec[:-1][ok])]          # among them, the highest precision
print("chosen threshold:", round(float(thr[best]), 3))
for t in (0.5, thr[best]):
    p = (proba >= t).astype(int)
    print(f"t={t:.3f} flagged={int(p.sum()):3d} precision={precision_score(y_val, p, zero_division=0):.3f} "
          f"recall={recall_score(y_val, p):.3f}")
```

```
chosen threshold: 0.077
t=0.500 flagged=  5 precision=0.000 recall=0.000
t=0.077 flagged=522 precision=0.209 recall=0.801
```

1. We build a 5% positive dataset and fit logistic regression on the training part. The thresholding is done on the *validation* part.

2. `precision_recall_curve` returns precision and recall for every distinct threshold. Precision and recall each have one more entry than `thr`, which is why we slice `[:-1]`.
3. Our business rule: catch at least 80% of positives. `ok` holds the thresholds that satisfy it.
4. Among them we pick the one with the best precision (fewest false alarms).
5. We compare the default 0.5 against the chosen threshold, turning probabilities into labels with `proba >= t`.

Reading the output: at the default 0.5 the model flags only 5 of the 2,500 validation rows and none is a real positive, so precision and recall are both 0. A threshold of 0.077 flags 522 rows and catches 80.1% of the positives, at a precision of 0.209. That is the price of high recall on a rare event: roughly four false alarms for each true catch. Whether that is acceptable is a business decision, which is exactly why the threshold should be chosen on purpose.

In production code, `TunedThresholdClassifierCV(model, scoring="f1")` does the search inside cross-validation and returns a classifier whose `predict` already uses the best threshold; `FixedThresholdClassifier` fixes one you chose.

Example 4: regression metrics and what each hides

```
import numpy as np
from sklearn.metrics import (mean_absolute_error, mean_absolute_percentage_error,
                             median_absolute_error, r2_score, root_mean_squared_error)

y_true = np.array([100.0, 120, 130, 110, 125, 115])
good = np.array([102.0, 118, 133, 108, 127, 113])
one_bad = good.copy(); one_bad[0] = 160            # one big miss

for name, p in (("good", good), ("one big miss", one_bad)):
    print(f"{name:13s} MAE={mean_absolute_error(y_true, p):5.2f} "
          f"RMSE={root_mean_squared_error(y_true, p):5.2f} "
          f"MedAE={median_absolute_error(y_true, p):4.1f} "
          f"MAPE={mean_absolute_percentage_error(y_true, p):.3f} "
          f"R2={r2_score(y_true, p):6.3f}")
```

```
good          MAE= 2.17 RMSE= 2.20 MedAE= 2.0 MAPE=0.019 R2= 0.950
one big miss  MAE=11.83 RMSE=24.58 MedAE= 2.0 MAPE=0.115 R2=-5.214
```

1. Six true values and two prediction sets. The second is the same as the first except one prediction is 60 off.
2. MAE averages the absolute errors. RMSE squares them first, so a single large error counts far more.
3. MedAE takes the median error, so one wild miss does not move it.
4. MAPE divides each error by the true value (a fraction, so 0.02 is 2%).
5. R squared compares the model's squared error to the error of always predicting the mean.

Reading the output: adding one big miss moves MAE from 2.17 to 11.83 and RMSE from 2.20 to 24.58, while the median absolute error stays at 2.0 and R squared collapses from 0.950 to -5.214 (far worse than predicting the mean). Pick the metric that matches the cost: use RMSE if big misses are disproportionately bad, MAE or MedAE if every unit of error costs the same, and MAPE only when the target is never near zero.

Example 5: ranking and clustering metrics

```
import numpy as np
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs
from sklearn.metrics import (adjusted_rand_score, davies_bouldin_score, ndcg_score,
                             roc_auc_score, silhouette_score)

# ranking: relevance of 5 items (3 = best) and two model score lists
rel = np.array([[3, 2, 0, 0, 1]])
good_scores = np.array([[0.9, 0.8, 0.1, 0.2, 0.5]])
bad_scores = np.array([[0.1, 0.2, 0.9, 0.8, 0.3]])
print("NDCG@3 good:", round(ndcg_score(rel, good_scores, k=3), 3),
      " bad:", round(ndcg_score(rel, bad_scores, k=3), 3))
print("AUC for binary labels:", roc_auc_score([0, 0, 1, 1], [0.1, 0.4, 0.35, 0.8]))

# clustering
X, truth = make_blobs(400, centers=3, cluster_std=1.0, random_state=0)
labels = KMeans(3, n_init=10, random_state=0).fit_predict(X)
print("silhouette:", round(silhouette_score(X, labels), 3),
      " davies-bouldin:", round(davies_bouldin_score(X, labels), 3),
      " ARI:", round(adjusted_rand_score(truth, labels), 3))
```

```
NDCG@3 good: 1.0  bad: 0.105
AUC for binary labels: 0.75
silhouette: 0.473  davies-bouldin: 0.743  ARI: 0.774
```

1. Rank quality: `ndcg_score` takes the true relevance of each item and the model's scores (both shaped as one row per query). It rewards putting high-relevance items near the top and cuts the list at `k=3`. A perfect order scores 1.0.
2. The "good" scores order items 0 and 1 first (relevance 3 and 2), so NDCG is near 1. The "bad" scores put the irrelevant items first, so it is low.
3. `roc_auc_score` on a 4-row example: of the 4 positive-negative pairs, 3 are ordered correctly (0.35 vs 0.1 yes, 0.35 vs 0.4 no, 0.8 vs 0.1 yes, 0.8 vs 0.4 yes), so AUC = 3/4 = 0.75.
4. For clustering with no labels, silhouette (higher is better, up to 1) and Davies-Bouldin (lower is better) judge tightness and separation.
5. When the true grouping is known (here only because the data is synthetic), the adjusted Rand index compares the found clusters to it.

Example 6: handling imbalance with class weights

Instead of throwing data away or copying rows, ask the model to pay more for mistakes on the rare class with `class_weight="balanced"`. The weight of each class becomes n_samples / (n_classes × count_of_class).

```
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import average_precision_score, precision_score, recall_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = make_classification(10000, 12, n_informative=4, weights=[0.96], flip_y=0.02, random_state=3)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, stratify=y, random_state=3)

for cw in (None, "balanced"):
    m = make_pipeline(StandardScaler(), LogisticRegression(class_weight=cw, max_iter=1000))
    m.fit(X_tr, y_tr)
    p, pr = m.predict(X_te), m.predict_proba(X_te)[:, 1]
    print(f"class_weight={cw!s:9s} precision={precision_score(y_te, p):.3f} "
          f"recall={recall_score(y_te, p):.3f} AP={average_precision_score(y_te, pr):.3f}")
```

```
class_weight=None      precision=0.824 recall=0.336 AP=0.603
class_weight=balanced  precision=0.193 recall=0.808 AP=0.542
```

1. A 4% positive problem with a little label noise.
2. We fit the same pipeline twice, once with equal weights and once with `"balanced"`.
3. With balanced weights each positive counts about 24 times as much as each negative in the loss, so the model is pushed to find positives.
4. We report precision, recall and average precision (which ignores the threshold) for each.

Reading the output: without weights the model is cautious: precision 0.824 but recall only 0.336. Balanced weights lift recall to 0.808 and drop precision to 0.193. That is a trade-off, not a free win: the weights effectively move the decision threshold. Average precision, which does not depend on a threshold, moves only from 0.603 to 0.542, which tells you the model's *ranking* is similar and most of the change is the operating point. If you only need a different operating point, tuning the threshold is the cleaner tool; class weights change the training itself.

NOT RUN HERE

Resampling. The package `imbalanced-learn` (`from imblearn.over_sampling import SMOTE` , `from imblearn.pipeline import Pipeline`) is not installed here. Its rule matches scikit-learn's: resample only *inside* the pipeline, on training folds, never before splitting. A runnable alternative with scikit-learn alone is `sklearn.utils.resample` inside a manual loop, or simply `class_weight` plus threshold tuning, which is the first thing to try.

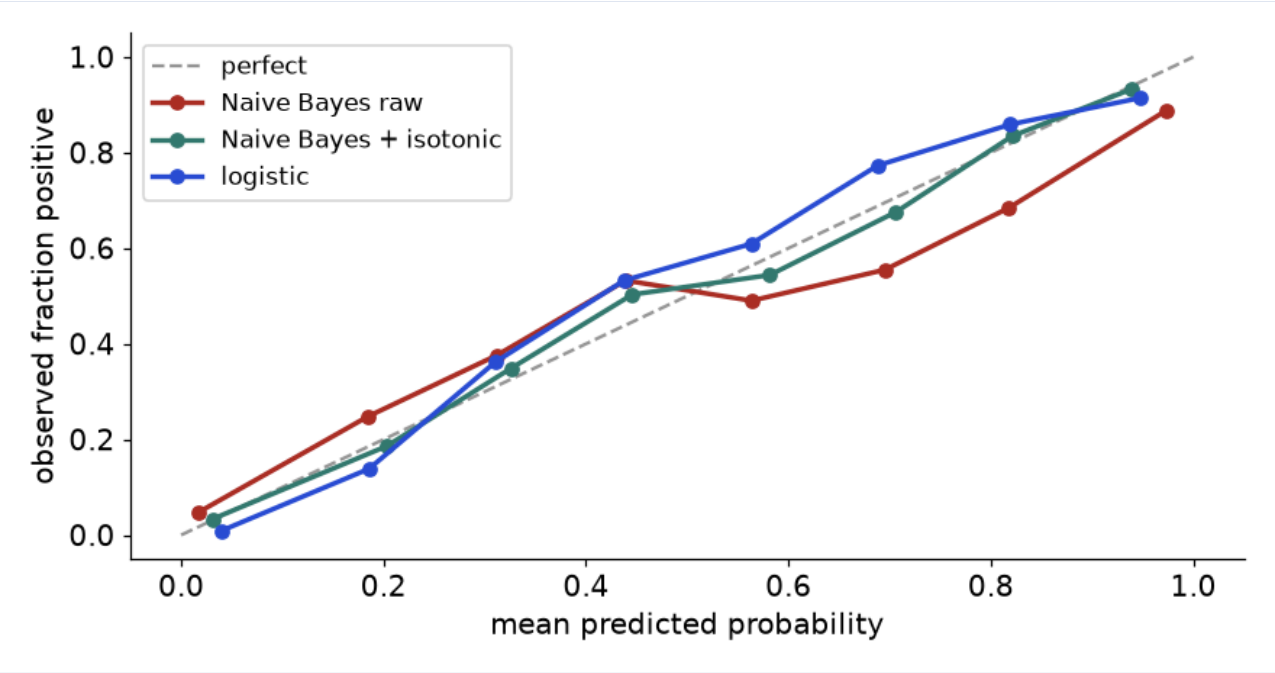
Example 7: calibration, when probabilities must mean something

If a model says 0.8, about 80% of such cases should be positive. Some models (naive Bayes, SVM, boosted trees without care) rank well but give distorted probabilities. A calibrated probability matters when you multiply it by money or use it as a risk score.

```
from sklearn.calibration import CalibratedClassifierCV
from sklearn.datasets import make_classification
from sklearn.metrics import brier_score_loss, log_loss
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB

X, y = make_classification(20000, 20, n_informative=6, n_redundant=10, class_sep=0.8,
                           flip_y=0.0, random_state=0)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, random_state=0)

raw = GaussianNB().fit(X_tr, y_tr)
cal = CalibratedClassifierCV(GaussianNB(), method="isotonic", cv=5).fit(X_tr, y_tr)
for name, m in (("raw", raw), ("isotonic", cal)):
    p = m.predict_proba(X_te)[:, 1]
    print(f"{name:9s} brier={brier_score_loss(y_te, p):.4f} logloss={log_loss(y_te, p):.4f}")
```

```
raw       brier=0.1139 logloss=0.4070
isotonic  brier=0.1078 logloss=0.3460
```

1. The same data as the calibration figure below, with 20 columns of which 10 are redundant copies of others. Naive Bayes assumes independence, which redundant columns violate, so its probabilities become overconfident.
2. `CalibratedClassifierCV` trains the base model on folds and learns a monotone mapping from raw scores to observed frequencies on the held-out part. `method="isotonic"` is flexible (use it with plenty of data); `"sigmoid"` is a two-parameter curve that is safer on small data.
3. We compare Brier score (mean squared error of the probability) and log loss. Lower is better for both. Brier improves from 0.1139 to 0.1078 and log loss from 0.4070 to 0.3460 after calibration.



Calibration curves on 5,000 test rows. A perfectly calibrated model follows the dashed diagonal.

How to read it: each point groups predictions into a bin (x is the average prediction in the bin) and shows the real share of positives in that bin (y). Points on the diagonal are honest probabilities. The raw naive Bayes line sags below the diagonal on the right (it says 0.7, reality is about 0.55), which is overconfidence. The isotonic version sits closer to the diagonal. Fix calibration after you are happy with ranking, and always check it on data the calibrator did not see.

19.7 Feature Selection, Model Inspection, Saving Models and Custom Transformers

Once a model scores well, four practical questions follow. Do I need all these columns? Why did the model decide that? How do I ship it? And what if scikit-learn has no transformer for my cleaning step? This section answers all four.

Reference: selection, inspection, persistence and extension

Name	What it does in plain English	Minimal example
`VarianceThreshold`	Drop columns that barely vary (constants)	`VarianceThreshold(0.0)`
`SelectKBest`	Keep the k columns with the best univariate score	`SelectKBest(f_classif, k=10)`
`SelectPercentile`	Keep the top percent of columns	`SelectPercentile(f_classif, percentile=20)`
`f_classif`, `f_regression`	ANOVA or correlation F-score of a column against y	`f_regression(X, y)`
`mutual_info_classif`	Score that also catches nonlinear relations (slower)	`SelectKBest(mutual_info_classif, k=5)`
`chi2`	Score for non-negative counts (text) against a class	`SelectKBest(chi2, k=1000)`
`SelectFromModel`	Keep columns a model rates as important (L1 weights or importances)	`SelectFromModel(Lasso(0.05))`

Name	What it does in plain English	Minimal example
`RFE`, `RFECV`	Repeatedly drop the weakest column; the CV version picks how many	`RFECV(LogisticRegression(), cv=5)`
`SequentialFeatureSelector`	Greedily add (or remove) one column at a time, scored by CV	`SequentialFeatureSelector(m, n_features_to_select=5)`
`selector.get_support()`	Boolean mask of the kept columns	`sel.get_support()`
`model.feature_importances_`	Tree-based importance (computed on training data; biased)	`rf.feature_importances_`
`model.coef_`	Linear-model weights (meaningful when columns are scaled)	`lr.coef_`
`permutation_importance`	Score drop when one column is shuffled on held-out data	`permutation_importance(m, X_te, y_te, n_repeats=10)`
`partial_dependence`	Average prediction as one feature varies (numbers)	`partial_dependence(m, X, [0])`
`PartialDependenceDisplay`	Plot of partial dependence or ICE curves	`PartialDependenceDisplay.from_estimator(m, X, [0])`
`joblib.dump`, `joblib.load`	Save and load a fitted model or pipeline	`joblib.dump(pipe, "m.joblib")`
`BaseEstimator`	Gives `get_params`, `set_params`, nice printing	`class My(BaseEstimator): ...`
`TransformerMixin`	Gives `fit_transform` and `set_output`	`class My(TransformerMixin, BaseEstimator)`
`validate_data`	Check and convert X, record `n_features_in_` and column names	`X = validate_data(self, X)`
`FunctionTransformer`	Quickest custom step: wrap a function	`FunctionTransformer(np.log1p, validate=False)`

Example 1: feature selection, the leak-free way

We have 30 columns, only 5 informative. We compare using all columns with two selection strategies, both placed *inside* the pipeline so they are re-learned in every CV fold.

```
from sklearn.datasets import make_classification
from sklearn.feature_selection import SelectFromModel, SelectKBest, f_classif
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = make_classification(400, 30, n_informative=5, n_redundant=0, random_state=0)
lr = LogisticRegression(max_iter=1000)
variants = {
    "all 30 columns": make_pipeline(StandardScaler(), lr),
    "SelectKBest k=5": make_pipeline(StandardScaler(), SelectKBest(f_classif, k=5), lr),
    "L1 SelectFromModel": make_pipeline(
        StandardScaler(),
        SelectFromModel(LogisticRegression(l1_ratio=1, solver="liblinear", C=0.1)),
        lr),
}
for name, m in variants.items():
    print(f"{name:20s} {cross_val_score(m, X, y, cv=5).mean():.3f}")

fitted = variants["SelectKBest k=5"].fit(X, y)
print("kept columns:", fitted[1].get_support().nonzero()[0])
```

```
all 30 columns       0.865
SelectKBest k=5      0.885
L1 SelectFromModel   0.883
kept columns: [ 5  6 14 15 16]
```

1. The dataset has 400 rows, 30 columns, 5 informative ones, and the rest noise.
2. Each variant is a pipeline. Selection steps sit between the scaler and the final model, so in each fold they are fitted on the fold's training rows only.
3. `SelectKBest(f_classif, k=5)` scores each column separately with an F-test against y and keeps the top five.
4. `SelectFromModel` fits an L1-penalised logistic regression (which zeroes weak columns) and keeps the columns with non-negligible weights.
5. We print the 5-fold accuracy of each variant.
6. After fitting one pipeline on all the data, `fitted[1]` is the selector step and `get_support()` is a boolean mask; `.nonzero()[0]` turns it into column numbers.

Reading the output: using all 30 columns scores 0.865, while keeping only five columns gives 0.885 with `SelectKBest` and 0.883 with the L1 selector. Removing noise helps a little. The kept indices (5, 6, 14, 15, 16) are the columns the F-test ranked highest. Because the selection happens inside each fold, these scores are honest; section 19.8 shows what happens when you select first on all rows. Univariate selection judges each column alone, so it can miss columns that matter only in combination, which is why the model-based selector is a useful second opinion.

Example 2: why permutation importance beats the built-in importance

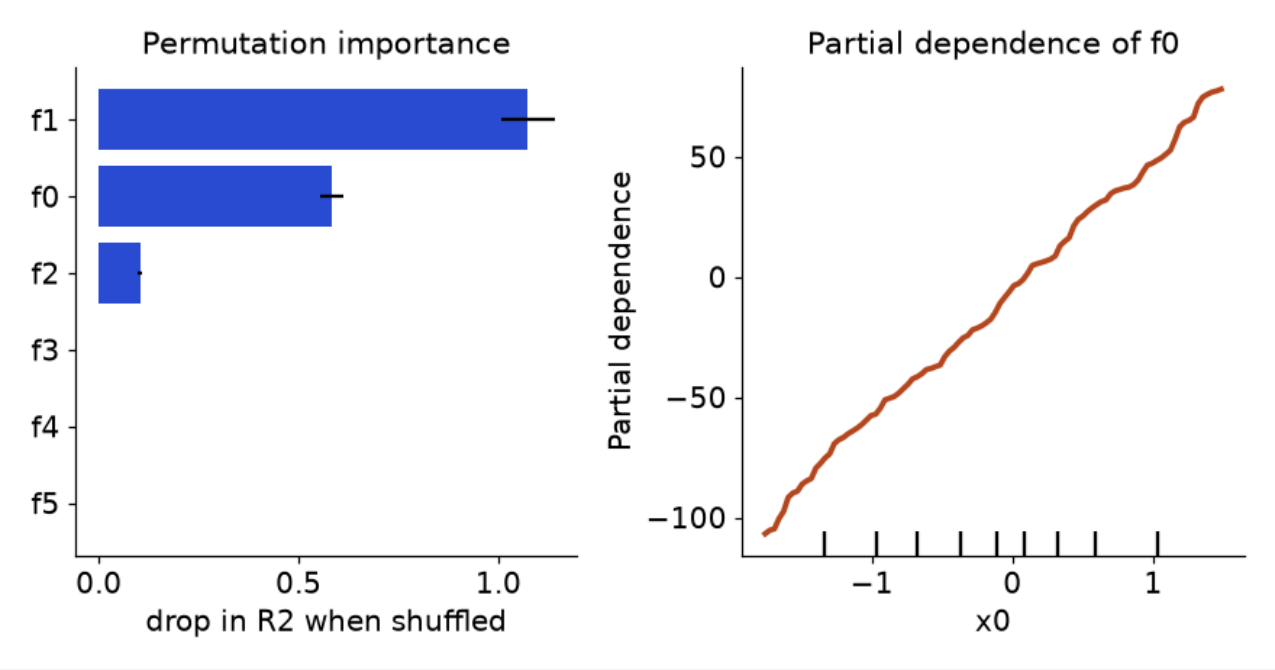
The `feature_importances_` of a forest are computed from the training data and favour columns with many distinct values, even useless ones. Permutation importance shuffles one column of *held-out* data and measures how much the score drops, which is an honest test of "does the model rely on this?". Here we add a random ID-like column that carries no signal.

```
import numpy as np
from sklearn.datasets import make_classification
from sklearn.ensemble import RandomForestClassifier
from sklearn.inspection import permutation_importance
from sklearn.model_selection import train_test_split

X, y = make_classification(1500, 5, n_informative=3, n_redundant=0, random_state=0, shuffle=False)
rng = np.random.default_rng(0)
X = np.c_[X, rng.normal(size=len(X))]                  # column 5: pure noise with many distinct values
names = ["f0", "f1", "f2", "f3", "f4", "noise"]
X_tr, X_te, y_tr, y_te = train_test_split(X, y, random_state=0)

rf = RandomForestClassifier(300, min_samples_leaf=1, random_state=0).fit(X_tr, y_tr)
pi = permutation_importance(rf, X_te, y_te, n_repeats=20, random_state=0)
print(f"{'feature':8s} {'builtin':>8s} {'permutation':>12s}")
for i, n in enumerate(names):
    print(f"{n:8s} {rf.feature_importances_[i]:8.3f} {pi.importances_mean[i]:8.3f} +/- {pi.importances_std[i]:.3f}")
```

```
feature   builtin  permutation
f0          0.277    0.285 +/- 0.025
f1          0.209    0.146 +/- 0.018
f2          0.249    0.227 +/- 0.016
f3          0.087   -0.003 +/- 0.007
f4          0.085   -0.011 +/- 0.006
noise       0.092   -0.009 +/- 0.006
```

1. Five real columns (the first three informative) plus a sixth column of random numbers.
2. The forest is trained on the training part only.
3. `permutation_importance` shuffles each column of the *test* set 20 times, re-scores, and reports the mean drop in accuracy and its standard deviation. A drop near zero means the model does not use that column.
4. We print both importance types side by side.

Reading the output: the three informative columns have permutation importances of 0.285, 0.146 and 0.227 (accuracy lost when shuffled). The pure-noise column gets a built-in importance of 0.092, as large as the two real-but-useless columns f3 and f4 (0.087 and 0.085), so the built-in numbers hand about a quarter of the total credit to three useless columns. Permutation importance gives all three a value of about zero (-0.003, -0.011, -0.009, each within about two standard deviations of zero), which is the right answer: shuffling them costs nothing. Caveat: if two columns are strongly correlated, shuffling one barely hurts because the model leans on the other, so both look unimportant. Check correlated groups together.



Permutation importance (left) and partial dependence of one feature (right) for a random forest regressor.

How to read it: in the left panel each bar is how much R squared falls when that feature is shuffled on test data; the thin line is the spread over repeats. Three features matter and three bars are invisible because they are essentially zero. In the right panel the line is the model's average prediction as f0 sweeps over its range while everything else keeps its real values. The tick marks on the x-axis show where the data lives; trust the curve only where ticks exist.

Example 3: partial dependence as numbers

```
import numpy as np
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.inspection import partial_dependence

rng = np.random.default_rng(0)
X = rng.uniform(0, 10, size=(2000, 2))
y = 3 * np.sin(X[:, 0]) + 0.5 * X[:, 1] + rng.normal(scale=0.3, size=2000)
m = HistGradientBoostingRegressor(random_state=0).fit(X, y)

pd0 = partial_dependence(m, X, features=[0], grid_resolution=6, method="brute")
print("grid  :", pd0["grid_values"][0].round(2))
print("effect:", pd0["average"][0].round(2))
print("truth :", (3 * np.sin(pd0["grid_values"][0]) + 0.5 * X[:, 1].mean()).round(2))
```

```
grid  : [0.49 2.27 4.06 5.85 7.64 9.43]
effect: [3.89 4.77 0.05 1.29 5.36 2.42]
truth : [3.9  4.79 0.11 1.25 5.43 2.47]
```

1. The target depends on a sine wave of the first column and a gentle linear trend in the second.
2. We fit a boosted regressor to the whole data (for the demonstration, not for scoring).
3. `partial_dependence` evaluates the model on a grid of six values of feature 0 (`method="brute"` does this literally, by predicting on modified copies of the data; the default `"auto"` may use a faster tree shortcut for boosted trees that leaves out the model's baseline constant, so levels would not match the truth below). At each grid value it sets feature 0 to that value for *every* row, predicts, and averages.
4. The result dictionary holds `grid_values` and `average` (the partial dependence).
5. For comparison we compute the true average effect from the formula we used to make the data.

Reading the output: the model's averaged curve follows the true sine shape (rise, peak, fall) to within about 0.1 at every grid point (for example 3.89 against 3.90 at the first point and 5.36 against 5.43 at the fifth). That is what partial dependence is for: showing the *shape* of a relationship that a single coefficient cannot.

Example 4: saving and loading a model

Persist the *whole pipeline*, so the preprocessing travels with the model. `joblib` is the standard tool because it stores NumPy arrays efficiently.

```
import os
import tempfile

import joblib
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = load_breast_cancer(return_X_y=True)
pipe = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000)).fit(X, y)

with tempfile.TemporaryDirectory() as d:
    path = os.path.join(d, "model.joblib")
    joblib.dump({"model": pipe, "sklearn": "1.9.1", "features": 30}, path, compress=3)
    print("file exists:", os.path.exists(path), " size under 20 KB:", os.path.getsize(path) < 20000)
    bundle = joblib.load(path)

same = np.array_equal(pipe.predict(X), bundle["model"].predict(X))
print("identical predictions after reload:", same)
print(bundle["sklearn"], bundle["features"])
```

```
file exists: True  size under 20 KB: True
identical predictions after reload: True
1.9.1 30
```

1. We fit a scaler-plus-model pipeline on all rows (in real work you would fit on training rows only).
2. We save inside a temporary folder so the example leaves nothing behind.
3. Saving a *dictionary* that holds the model and metadata (library version, expected column count) is a good habit: it records how the model must be used.
4. `compress=3` trades a little time for a smaller file.
5. `joblib.load` returns the same dictionary, and the reloaded pipeline gives identical predictions.

COMMON MISTAKE	**Wrong:** loading a `.joblib` or pickle file from an untrusted source, or loading in an environment with a different scikit-learn version. **Symptom:** the first is an arbitrary-code-execution risk (pickle can run code on load); the second gives warnings, wrong results or crashes. **Fix:** only load files you created, pin the library versions (store them with the model), and keep the training script so you can rebuild. For sharing between organisations, consider a safer format such as `skops` (not installed here, so not run) or export to ONNX.

Example 5: a custom transformer

Suppose you want to cap extreme values at the 5th and 95th percentile of each column, learned from training data. Scikit-learn has no such step, so we write one. The recipe: inherit from `TransformerMixin` and `BaseEstimator`; store constructor arguments untouched; learn into attributes ending in an underscore in `fit`; apply in `transform`.

```
import numpy as np
import pandas as pd
from sklearn.base import BaseEstimator, TransformerMixin, clone
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import GridSearchCV
from sklearn.pipeline import make_pipeline
from sklearn.utils.validation import check_is_fitted, validate_data


class Winsorizer(TransformerMixin, BaseEstimator):
    def __init__(self, lower=0.05, upper=0.95):
        self.lower = lower
        self.upper = upper

    def fit(self, X, y=None):
        X = validate_data(self, X, dtype=float)
        self.lo_ = np.quantile(X, self.lower, axis=0)
        self.hi_ = np.quantile(X, self.upper, axis=0)
        return self

    def transform(self, X):
        check_is_fitted(self)
        X = validate_data(self, X, dtype=float, reset=False)
        return np.clip(X, self.lo_, self.hi_)

    def get_feature_names_out(self, input_features=None):
        return np.asarray(input_features if input_features is not None
                          else self.feature_names_in_, dtype=object)


df = pd.DataFrame({"a": [1, 2, 3, 4, 100.0], "b": [5, 6, 7, 8, 9.0]})
w = Winsorizer(0.0, 0.8).set_output(transform="pandas")
print(w.fit_transform(df))
print(clone(w).get_params())

rng = np.random.default_rng(0)
X = rng.normal(size=(300, 1)); X[:5] = 40                # 5 wild outliers
y = 2 * X[:, 0] + rng.normal(scale=0.2, size=300)
gs = GridSearchCV(make_pipeline(Winsorizer(), LinearRegression()),
                  {"winsorizer__upper": [0.9, 0.99]}, cv=3).fit(X, y)
print(gs.best_params_)
```

```
a    b
0   1.0  5.0
1   2.0  6.0
2   3.0  7.0
3   4.0  8.0
4  23.2  8.2
{'lower': 0.0, 'upper': 0.8}
{'winsorizer__upper': 0.99}
```

1. The constructor only stores its arguments, with the same names as the parameters. This rule is what makes `get_params`, `clone` and grid search work.
2. `fit` calls `validate_data(self, X, dtype=float)`, which converts to a float array, checks for NaN or infinity, and records `n_features_in_` (and `feature_names_in_` for DataFrames). It then learns the two percentile vectors `lo_` and `hi_` and returns `self`.
3. `transform` first calls `check_is_fitted`, then validates with `reset=False` so that the column count must match what `fit` saw, and clips with `np.clip`.
4. `get_feature_names_out` is what lets `set_output(transform="pandas")` return a DataFrame with the right column names.
5. For column `a` the 80th percentile (with linear interpolation, position 0.8 × 4 = 3.2) is 4 + 0.2 × 96 = 23.2, so 100 becomes 23.2 in the printed DataFrame; column `b` is clipped at its own 80th percentile, 8.2.

6. `clone(w).get_params()` proves the class behaves like a real estimator: parameters are discoverable and copyable.
7. Last, the transformer slots into `GridSearchCV` with the double-underscore name `winsorizer__upper`, so its setting can be tuned like any other. Here the search prefers clipping at the 99th percentile over the 90th: the five wild rows are only 1.7% of the data, so the 90th percentile clips a lot of good data too.

KEY IDEA	For a stateless function (no learning, such as `np.log1p` or "add a ratio column"), `FunctionTransformer` is enough and needs no class. Write a class only when `fit` must learn something from the training data, such as quantiles, a vocabulary or a lookup table.

19.8 Recipes: How Do I ...?

Each recipe is a complete, runnable answer to a real task. Read the six big ones in order (they build skills), then use the quick ones as lookups.

Recipe 1: How do I build an end-to-end tabular classifier?

Task: predict churn, compare two candidates fairly, pick a threshold, test once, save. The data is the telecom table from section 19.3.

```
import joblib, os, tempfile
import numpy as np
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import average_precision_score, roc_auc_score
from sklearn.model_selection import StratifiedKFold, cross_validate, train_test_split
from sklearn.pipeline import Pipeline, make_pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

rng = np.random.default_rng(0)
n = 4000
df = pd.DataFrame({
    "tenure": rng.integers(1, 72, n),
    "monthly": rng.normal(65, 20, n).round(2),
    "calls": rng.poisson(2, n),
    "contract": rng.choice(["monthly", "yearly", "two-year"], n, p=[0.55, 0.25, 0.20]),
    "payment": rng.choice(["card", "bank", "cheque"], n),
})
logit = (-2.5 - 0.04 * df["tenure"] + 0.02 * df["monthly"] + 0.3 * df["calls"]
         + np.where(df["contract"] == "monthly", 1.0, 0.0)
         + 0.02 * df["tenure"] * (df["contract"] == "monthly"))      # an interaction
df["churn"] = (rng.random(n) < 1 / (1 + np.exp(-logit))).astype(int)
df.loc[rng.random(n) < 0.08, "monthly"] = np.nan

X, y = df.drop(columns="churn"), df["churn"]
X_tr, X_te, y_tr, y_te = train_test_split(X, y, stratify=y, random_state=0)

num, cat = ["tenure", "monthly", "calls"], ["contract", "payment"]
lin_prep = ColumnTransformer([
    ("num", make_pipeline(SimpleImputer(strategy="median"), StandardScaler()), num),
    ("cat", OneHotEncoder(handle_unknown="ignore"), cat)])
candidates = {
    "logistic": Pipeline([("prep", lin_prep), ("clf", LogisticRegression(max_iter=1000))]),
    "hist boosting": Pipeline([("prep", ColumnTransformer(
        [("cat", OneHotEncoder(handle_unknown="ignore"), cat)], remainder="passthrough")),
        ("clf", HistGradientBoostingClassifier(random_state=0))]),
}
cv = StratifiedKFold(5, shuffle=True, random_state=0)
for name, pipe in candidates.items():
    r = cross_validate(pipe, X_tr, y_tr, cv=cv, scoring=["roc_auc", "average_precision"])
    print(f"{name:14s} auc={r['test_roc_auc'].mean():.3f} ap={r['test_average_precision'].mean():.3f}")

final = candidates["logistic"].fit(X_tr, y_tr)
p = final.predict_proba(X_te)[:, 1]
print("TEST auc:", round(roc_auc_score(y_te, p), 3), " ap:", round(average_precision_score(y_te, p), 3))
with tempfile.TemporaryDirectory() as d:
    joblib.dump(final, os.path.join(d, "churn.joblib"))
    print("reloaded ok:", np.allclose(joblib.load(os.path.join(d, "churn.joblib")).predict_proba(X_te)[:, 1], p))
```

```
logistic       auc=0.752 ap=0.550
hist boosting  auc=0.709 ap=0.501
TEST auc: 0.751  ap: 0.537
reloaded ok: True
```

1. We build a 4,000-row churn table with 8% missing monthly bills, and a small interaction: for monthly-contract customers a longer tenure raises churn a little. A pure linear model cannot represent the interaction.
2. Split once with stratification. The test rows are not used again until the single test line.

3. Two candidate pipelines. The linear one imputes, scales and one-hot encodes. The boosting one needs no scaling and handles missing values itself, so we only encode the text columns and pass numbers through (`remainder="passthrough"`; the encoder step returns the numeric columns unchanged after the one-hot columns).
4. Both are scored with the same stratified folds on the *training* rows, with two metrics at once in `cross_validate`.
5. We pick one, refit on all training rows, and score the test set once.
6. Last, we save the fitted pipeline and verify the reload gives identical probabilities.

Reading the output: cross-validated ROC AUC is 0.752 for logistic regression and 0.709 for boosting, so the simple model wins here: the signal in this table is mostly additive and the interaction is small next to the noise, and with 3,000 training rows the flexible model has little room to pay off. Never assume the fancier model is better. The test AUC of the chosen pipeline is 0.751, matching its CV score of 0.752, which is what an honest process looks like. Average precision is 0.537 on test against a baseline of about 0.29 (the churn rate). The saved pipeline reloads with identical probabilities.

Recipe 2: How do I do regression with proper cross-validation?

Task: predict house prices (a skewed target), compare to a baseline, and report error in money units.

```
import numpy as np
from sklearn.compose import TransformedTargetRegressor
from sklearn.dummy import DummyRegressor
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.linear_model import Ridge
from sklearn.model_selection import KFold, cross_validate
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

rng = np.random.default_rng(1)
n = 1500
area = rng.uniform(40, 250, n)
rooms = np.clip((area / 40 + rng.normal(0, 0.7, n)).round(), 1, 8)
age = rng.uniform(0, 60, n)
price = np.exp(10.5 + 0.009 * area + 0.05 * rooms - 0.006 * age + rng.normal(0, 0.15, n))
X = np.c_[area, rooms, age]

cv = KFold(5, shuffle=True, random_state=0)
models = {
    "mean baseline": DummyRegressor(),
    "ridge on price": make_pipeline(StandardScaler(), Ridge()),
    "ridge on log(price)": TransformedTargetRegressor(
        make_pipeline(StandardScaler(), Ridge()), func=np.log, inverse_func=np.exp),
    "hist boosting": HistGradientBoostingRegressor(random_state=0),
}
for name, m in models.items():
    r = cross_validate(m, X, price, cv=cv,
                       scoring=["neg_root_mean_squared_error", "neg_mean_absolute_error", "r2"])
    print(f"{name:20s} RMSE={-r['test_neg_root_mean_squared_error'].mean():9.0f} "
          f"MAE={-r['test_neg_mean_absolute_error'].mean():9.0f} R2={r['test_r2'].mean():.3f}")
```

```
mean baseline        RMSE=   106683 MAE=    87385 R2=-0.000
ridge on price       RMSE=    41475 MAE=    31182 R2=0.848
ridge on log(price)  RMSE=    29709 MAE=    19842 R2=0.922
hist boosting        RMSE=    32853 MAE=    21958 R2=0.904
```

1. The price is generated as the exponential of a linear formula plus noise, so it is right-skewed and the effect of each feature is multiplicative, a very common real-world shape.
2. We use plain `KFold` with shuffling (regression has no classes to stratify).
3. The baseline predicts the training mean every time. It is the floor, with R squared at or below 0.
4. `TransformedTargetRegressor` trains on `log(price)` and automatically applies `exp` to predictions, so the output is back in money.
5. Scoring uses negated error names (`neg_root_mean_squared_error`), so we negate again when printing to show positive errors.

Reading the output: the mean baseline has R squared 0.000 and an RMSE of about 106,683. Ridge on the raw price cuts RMSE to 41,475 (R squared 0.848). Ridge on the log of the price does best: RMSE 29,709 and R squared 0.922, because the data really is multiplicative and the log makes it linear. Boosting reaches 32,853 (R squared 0.904): trees capture the shape without being told, but with 1,500 rows they are noisier than a correct linear model. A model that cannot beat the mean baseline by a wide margin is not ready, whatever its R squared looks like.

Recipe 3: How do I classify text with TfidfVectorizer?

Task: route support tickets to billing, technical or shipping. TF-IDF turns each text into a sparse vector: term frequency (how often a word appears in the ticket) times inverse document frequency (rare words across tickets count more). The real `fetch_20newsgroups` corpus needs a download, so we generate a small synthetic ticket set.

```
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report
from sklearn.model_selection import cross_val_score, train_test_split
from sklearn.pipeline import make_pipeline

rng = np.random.default_rng(0)
pools = {
    "billing": "invoice refund charge payment card overcharged receipt billing price".split(),
    "technical": "error crash login password app update install bug screen".split(),
    "shipping": "delivery package courier late tracking address parcel shipped lost".split(),
}
common = "please help my the is not working since yesterday order account thanks".split()
texts, labels = [], []
for lab, pool in pools.items():
    for _ in range(200):
        words = [rng.choice(pool) if rng.random() < 0.45 else rng.choice(common) for _ in range(9)]
        texts.append(" ".join(words)); labels.append(lab)
print(texts[0])

X_tr, X_te, y_tr, y_te = train_test_split(texts, labels, stratify=labels, random_state=0)
pipe = make_pipeline(TfidfVectorizer(ngram_range=(1, 2), min_df=2, sublinear_tf=True),
                     LogisticRegression(max_iter=1000))
print("CV acc:", cross_val_score(pipe, X_tr, y_tr, cv=5).mean().round(3))
pipe.fit(X_tr, y_tr)
print(classification_report(y_te, pipe.predict(X_te), digits=3))

vec, clf = pipe[0], pipe[-1]
names = np.array(vec.get_feature_names_out())
for i, lab in enumerate(clf.classes_):
    print(lab, "->", names[np.argsort(clf.coef_[i])[-4:][::-1]].tolist())
print(pipe.predict(["my parcel never arrived and tracking is wrong"]))
```

```
working charge refund order thanks yesterday the billing working
CV acc: 0.998
              precision    recall  f1-score   support

     billing      0.980     1.000     0.990        50
    shipping      1.000     0.980     0.990        50
   technical      1.000     1.000     1.000        50

    accuracy                          0.993       150
   macro avg      0.993     0.993     0.993       150
weighted avg      0.993     0.993     0.993       150

billing -> ['price', 'payment', 'invoice', 'overcharged']
shipping -> ['courier', 'address', 'tracking', 'shipped']
technical -> ['screen', 'install', 'login', 'bug']
['shipping']
```

1. We make 600 tickets. Each has nine words; with probability 0.45 a word comes from the topic's pool and otherwise from shared filler words. The first ticket is printed so you see what the model sees.
2. The split is stratified over the raw strings. Note we split *before* any vectoriser sees the text; the vocabulary and the IDF weights must be learned from training tickets only.
3. `TfidfVectorizer(ngram_range=(1, 2), min_df=2, sublinear_tf=True)` uses single words and word pairs, ignores terms that appear in fewer than two tickets, and uses `1 + log(tf)` so a word repeated ten times does not count ten times.
4. The vectoriser and classifier are in one pipeline, so CV refits the vocabulary in every fold.
5. The report gives per-class precision and recall.
6. `get_feature_names_out` gives the vocabulary; sorting each class's coefficients shows the four terms that push a ticket toward that class. This is the cheap, honest way to sanity-check a text model: do the top words make sense?
7. Finally we predict the topic of a new sentence the model never saw.

Reading the output: cross-validated accuracy is 0.998 and the test report shows one error in 150 tickets (0.993). That is unrealistically easy because we wrote the data so topic words are clean; on real tickets expect far lower numbers. What transfers is the method. The top terms per class are sensible (`invoice` and `overcharged` for billing, `courier` and `tracking` for shipping, `login` and `bug` for technical), and the new sentence about a parcel is routed to shipping.

REAL JOB

For real text, start with this exact pipeline. A `TfidfVectorizer` plus `LogisticRegression` or `LinearSVC` is a famously strong baseline, trains in seconds, and is explainable. Move to transformer embeddings only if this baseline is clearly not enough (see the GenAI document). Add `stop_words="english"` , `max_features` and `min_df` to control vocabulary size on large corpora.

Recipe 4: How do I cluster customers?

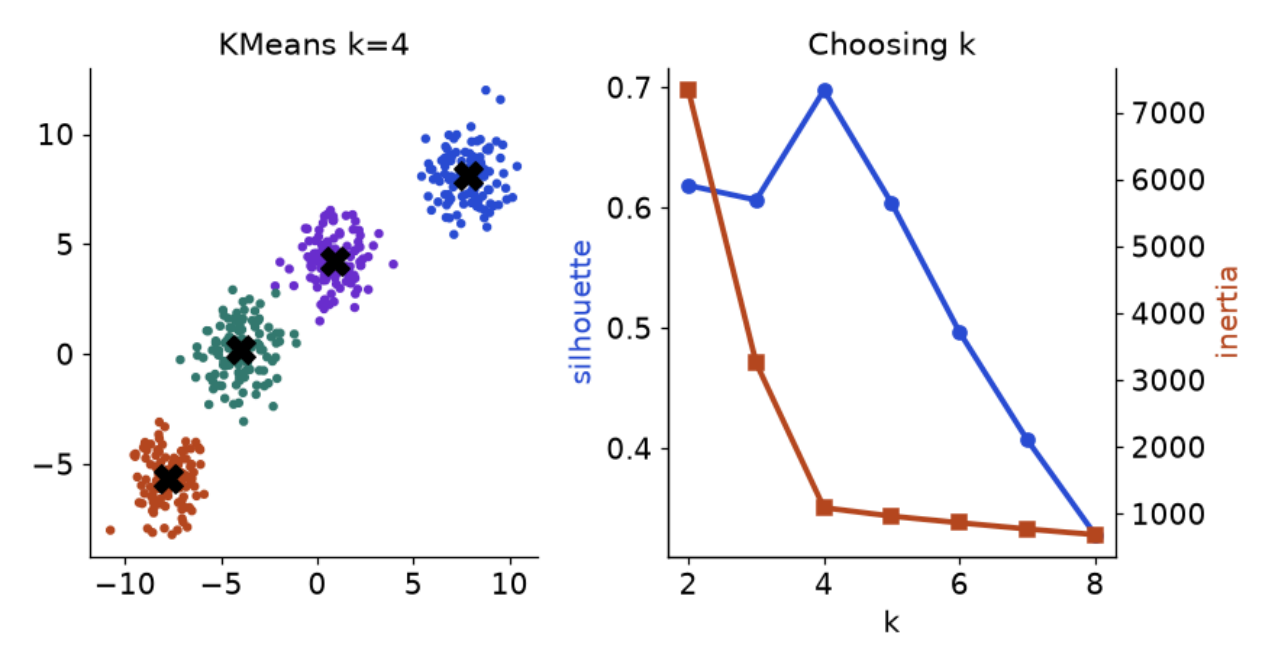
Task: group customers by recency (days since last order), frequency (orders) and monetary value (total spend), then describe each group.

```
import numpy as np
import pandas as pd
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.preprocessing import StandardScaler

rng = np.random.default_rng(0)
parts = []
for recency, freq, money, size in [(10, 20, 2000, 150), (60, 6, 500, 250), (200, 2, 120, 200)]:
    parts.append(pd.DataFrame({
        "recency": rng.gamma(4, recency / 4, size),
        "frequency": rng.poisson(freq, size) + 1,
        "monetary": rng.lognormal(np.log(money), 0.35, size)}))
rfm = pd.concat(parts, ignore_index=True)

Z = StandardScaler().fit_transform(np.log1p(rfm))        # log tames skew, scaler equalises units
for k in range(2, 7):
    km = KMeans(k, n_init=10, random_state=0).fit(Z)
    print(k, "silhouette", round(silhouette_score(Z, km.labels_), 3), " inertia", round(km.inertia_, 1))

best = KMeans(3, n_init=10, random_state=0).fit(Z)
rfm["segment"] = best.labels_
print(rfm.groupby("segment").agg(n=("recency", "size"), recency=("recency", "mean"),
                                 frequency=("frequency", "mean"), monetary=("monetary", "mean")).round(0))
```

```
2 silhouette 0.561  inertia 658.6
3 silhouette 0.567  inertia 274.2
4 silhouette 0.482  inertia 236.2
5 silhouette 0.388  inertia 202.1
6 silhouette 0.385  inertia 183.3
           n  recency  frequency  monetary
segment
0        247     58.0        7.0     551.0
1        202    202.0        3.0     130.0
2        151      9.0       21.0    2096.0
```

1. We simulate three kinds of customers (recent big spenders, middling regulars, lapsed small spenders) so we know the right answer is three groups. Real data has no such label.
2. K-means uses Euclidean distance, so skewed columns and different units distort it. `np.log1p` reduces skew and `StandardScaler` puts columns on one scale.
3. We run k from 2 to 6 and print the silhouette score (higher is better) and the inertia (total squared distance to the centres, always falling as k grows, so look for an elbow, not a minimum).
4. We fit k=3, attach the labels to the original table and aggregate with `groupby` to profile each segment in original units. Describing segments in business terms is the real deliverable.
5. `n_init=10` restarts k-means from ten random starting points and keeps the best, because one start can land in a poor solution.

Reading the output: silhouette is highest at k=3 (0.567, with k=2 close at 0.561) and drops to 0.482 at k=4; inertia falls from 658.6 to 274.2 between k=2 and 3 and then declines slowly, an elbow at 3. Both agree with the three groups we planted. The profile table recovers them: segment 2 (151 customers) is recent, frequent and high spend (about 9 days, 21 orders, 2,096), segment 0 (247) is the middle group, and segment 1 (202) has lapsed (202 days, 3 orders, 130). The sizes are close to the 150, 250 and 200 we generated.



K-means on four blobs (left) and the two curves used to choose k (right).

How to read it: on the right, the blue line (silhouette) peaks at the correct k=4, while the orange line (inertia) falls steadily and only bends at 4. Use silhouette for a peak and inertia for an elbow, and then check that the clusters are *useful*: a cluster you cannot describe or act on is not worth keeping.

Recipe 5: How do I find anomalies?

Task: flag unusual transactions without labels. `IsolationForest` isolates a point by random splits; unusual points are isolated in fewer splits. We inject known anomalies only so we can grade the method.

```
import numpy as np
from sklearn.ensemble import IsolationForest
from sklearn.metrics import precision_score, recall_score, roc_auc_score
from sklearn.neighbors import LocalOutlierFactor

rng = np.random.default_rng(0)
normal = np.c_[rng.lognormal(3.5, 0.5, 2000), rng.normal(14, 3, 2000)]      # amount, hour
odd = np.c_[rng.lognormal(6.0, 0.4, 40), rng.normal(3, 1, 40)]              # big, at 3 am
X = np.vstack([normal, odd])
is_anomaly = np.r_[np.zeros(2000, int), np.ones(40, int)]

iso = IsolationForest(n_estimators=200, contamination=0.02, random_state=0).fit(X)
pred = (iso.predict(X) == -1).astype(int)            # predict gives -1 for outliers
score = -iso.score_samples(X)                         # higher = more anomalous
print("isolation forest: AUC", round(roc_auc_score(is_anomaly, score), 3),
      " precision", round(precision_score(is_anomaly, pred), 3),
      " recall", round(recall_score(is_anomaly, pred), 3))

lof = LocalOutlierFactor(n_neighbors=30, contamination=0.02)
pred2 = (lof.fit_predict(X) == -1).astype(int)
print("local outlier factor: precision", round(precision_score(is_anomaly, pred2), 3),
      " recall", round(recall_score(is_anomaly, pred2), 3))
top = np.argsort(score)[-10:]
print("top-10 flagged rows that are real anomalies:", int(is_anomaly[top].sum()))
```

```
isolation forest: AUC 1.0  precision 0.976  recall 1.0
local outlier factor: precision 0.463  recall 0.475
top-10 flagged rows that are real anomalies: 10
```

1. Normal transactions: amounts around 33 (the log-normal centre) and hours near mid-afternoon. Anomalies: much larger amounts around 3 am.
2. `IsolationForest(contamination=0.02)` sets the threshold so about 2% of rows are labelled outliers. It is a guess about prevalence; choose it from business knowledge. `score_samples` gives a continuous score (more negative means more abnormal), so we negate it for ranking.
3. `predict` returns +1 for normal and -1 for outlier, so we convert to 0/1 to use the usual metrics.
4. `LocalOutlierFactor` compares a point's local density to its neighbours'. With `fit_predict` it labels the training data directly (set `novelty=True` if you need to score new data later).
5. Finally we check how many of the 10 highest-scoring rows are real injected anomalies, a "precision at 10" that matches how an analyst would use the list.

Reading the output: isolation forest ranks perfectly here (AUC 1.0); with the 2% cut-off it flags 41 rows, all 40 real anomalies plus one normal row (precision 0.976, recall 1.0), and all of the top 10 are real. Local outlier factor does much worse (precision 0.463, recall 0.475). The reason is instructive: the 40 anomalies are close to each other, so each has plenty of similar neighbours and looks locally normal. Density methods struggle when anomalies form a small cluster; isolation works because a point far from the bulk is separated in few splits. Also remember the ranking quality (AUC) and the thresholded precision and recall are separate things: precision and recall depend on the 2% contamination guess. With scarce labels, rank by score and let a human review the top of the list.

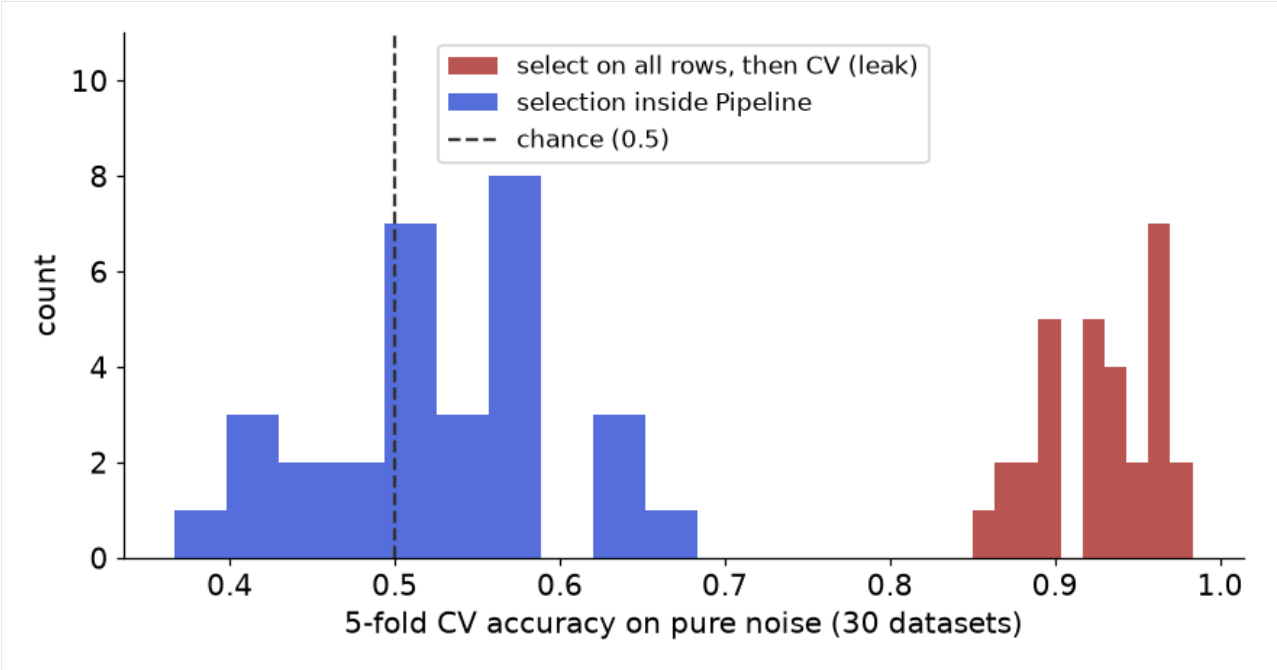
Recipe 6: How do I see data leakage with my own eyes?

Leakage means information from validation rows influences training. The most dramatic form is feature selection on all the data before cross-validation. Here the data is **pure noise**: 60 rows, 2,000 random columns, a random label. Honest accuracy is 50%.

```
import numpy as np
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score
from sklearn.pipeline import make_pipeline

leaky, honest = [], []
for seed in range(10):
    r = np.random.RandomState(seed)
    X = r.normal(size=(60, 2000))
    y = r.randint(0, 2, 60)

    X_sel = SelectKBest(f_classif, k=20).fit_transform(X, y)         # selection sees ALL labels
    leaky.append(cross_val_score(LogisticRegression(), X_sel, y, cv=5).mean())

    pipe = make_pipeline(SelectKBest(f_classif, k=20), LogisticRegression())
    honest.append(cross_val_score(pipe, X, y, cv=5).mean())          # selection redone per fold

print("leaky  mean accuracy:", round(float(np.mean(leaky)), 3))
print("honest mean accuracy:", round(float(np.mean(honest)), 3))
```

```
leaky  mean accuracy: 0.917
honest mean accuracy: 0.53
```

1. For ten different random datasets we generate 60 rows of 2,000 independent normal columns and a coin-flip label. Nothing can predict the label.
2. Leaky route: `SelectKBest(k=20)` looks at all 60 labels and keeps the 20 columns that happen, by chance, to correlate best with y. With 2,000 candidates, some look excellent by luck. Then CV is run on those 20 columns. The validation folds already influenced which columns were kept.
3. Honest route: the selector is inside the pipeline, so each fold re-selects using only that fold's training rows. The validation rows are truly unseen.
4. We average over ten datasets to avoid one lucky run.



Thirty noise datasets scored two ways. The red group is an impossible result created by leakage.

Reading the output: averaged over ten noise datasets the leaky procedure scores 0.917 and the honest one 0.53. The truth is 0.5.

How to read it: the dashed line is chance. The blue bars (selection inside the Pipeline) scatter around 0.5, which is the truth for random data; their spread is also a reminder of how noisy a score on 60 rows is. The red bars sit between 0.85 and 0.98, an "excellent model" built from nothing. If your real project ever shows a result this good on a small, wide dataset, suspect leakage before celebrating.

KEY IDEA	Anything that *learns from the data* (scalers, imputers, encoders with target statistics, selectors, PCA, vectorisers, resamplers, calibrators) belongs inside the Pipeline, so that cross-validation can fit it per fold.

Quick recipes

Recipe 7: How do I compare two models fairly? Use the same folds, repeat the CV, and look at the paired differences per fold, not just the means.

```
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import RepeatedStratifiedKFold, cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = load_breast_cancer(return_X_y=True)
cv = RepeatedStratifiedKFold(n_splits=5, n_repeats=4, random_state=0)   # 20 identical folds for both
a = cross_val_score(make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000)), X, y, cv=cv)
b = cross_val_score(RandomForestClassifier(200, random_state=0), X, y, cv=cv)
diff = a - b
print("means:", a.mean().round(3), b.mean().round(3))
print("paired diff mean:", diff.mean().round(4), " std:", diff.std(ddof=1).round(4),
      " logistic wins in", int((diff > 0).sum()), "of 20 folds")
```

```
means: 0.975 0.963
paired diff mean: 0.0123  std: 0.0188  logistic wins in 13 of 20 folds
```

1. `RepeatedStratifiedKFold` yields 20 splits (5 folds, 4 repetitions). Passing the same object to both calls guarantees identical splits.
2. Subtracting fold by fold gives paired differences, which are more informative than two separate means because both models face the same hard or easy fold.
3. We report the mean difference, its spread and the number of folds the first model wins. A mean difference smaller than its spread (and wins near half) means "no clear winner".

Reading the output: logistic regression averages 0.975 against 0.963 for the forest. The mean difference of 0.0123 is smaller than the spread of the differences (0.0188) and logistic wins in 13 of 20 folds, so the evidence is suggestive but not decisive. Note that the 20 folds overlap in their training data, so they are not independent; treat any formal test on them with caution.

Recipe 8: How do I get out-of-fold predictions for every row? `cross_val_predict` returns, for each row, the prediction made by the model that did not train on it. Use it to draw honest confusion matrices or to build stacking features.

```
from sklearn.datasets import load_wine
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix
from sklearn.model_selection import cross_val_predict
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = load_wine(return_X_y=True)
pipe = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))
oof = cross_val_predict(pipe, X, y, cv=5)
print(confusion_matrix(y, oof))
print("oof accuracy:", round((oof == y).mean(), 3))
```

```
[[58  1  0]
 [ 0 69  2]
 [ 0  0 48]]
oof accuracy: 0.983
```

1. Each of the 178 wines is predicted by a pipeline trained on the other four folds.
2. The confusion matrix has true classes in rows and predicted classes in columns; the diagonal is correct predictions.
3. The accuracy computed from these out-of-fold predictions is a fair estimate. Do not use `cross_val_predict` output as a "score per fold": predictions from different models are pooled.

Reading the output: 3 of 178 wines are misclassified (class 0 mistaken for 1 once, class 1 mistaken for 2 twice), so out-of-fold accuracy is 0.983.

Recipe 9: How do I stack models? `StackingClassifier` trains base models, builds out-of-fold predictions from them, and trains a final model on those.

```
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import HistGradientBoostingClassifier, RandomForestClassifier, StackingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

X, y = load_breast_cancer(return_X_y=True)
base = [("svm", make_pipeline(StandardScaler(), SVC())),
        ("rf", RandomForestClassifier(200, random_state=0)),
        ("hgb", HistGradientBoostingClassifier(random_state=0))]
stack = StackingClassifier(base, final_estimator=LogisticRegression(max_iter=1000), cv=5)
print("stack:", cross_val_score(stack, X, y, cv=5).mean().round(3))
print("svm  :", cross_val_score(base[0][1], X, y, cv=5).mean().round(3))
```

```
stack: 0.977
svm  : 0.974
```

1. Three different base models. The SVM has no probabilities in scikit-learn 1.9 (its `probability=True` option is deprecated), and that is fine: with the default `stack_method="auto"` the stack uses `predict_proba` where available and falls back to `decision_function` for the SVM.
2. The final estimator is a simple logistic regression that learns how much to trust each base model. Keep the final model simple.
3. Internally, `cv=5` makes the base predictions out-of-fold so the final model is not trained on leaked predictions.
4. We compare with the best single base model. Stacking costs several times more training time and often gains little over a well-tuned single model; add it last.

Reading the output: the stack scores 0.977 against 0.974 for the SVM alone. A gain of 0.3 points (about two rows in 569) is far inside the fold-to-fold noise, so stacking added essentially nothing here, a common outcome for a few extra minutes of training.

Recipe 10: How do I get a DataFrame out of my pipeline, with names?

```
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler

df = pd.DataFrame({"age": [25, 40, 31], "city": ["Oslo", "Rome", "Oslo"]})
ct = ColumnTransformer([("num", StandardScaler(), ["age"]),
                        ("cat", OneHotEncoder(sparse_output=False), ["city"])],
                       verbose_feature_names_out=False).set_output(transform="pandas")
out = ct.fit_transform(df)
print(out.round(2))
print(type(out).__name__, list(out.columns))
```

```
   age  city_Oslo  city_Rome
0 -1.14        1.0        0.0
1  1.30        0.0        1.0
2 -0.16        1.0        0.0
DataFrame ['age', 'city_Oslo', 'city_Rome']
```

1. `set_output(transform="pandas")` makes every transformer in the composite return a DataFrame instead of a bare array.
2. `verbose_feature_names_out=False` removes the `num__` and `cat__` prefixes when the names are unique. Leave it at the default `True` if two steps could produce the same name.
3. Now downstream code can use column names, and you can inspect the transformed table the way you would any DataFrame.

19.9 Performance and Pitfalls

Speed and memory: what actually matters

Most scikit-learn slowness comes from three causes: the wrong algorithm for the data size, dense arrays where sparse would do, and unused CPU cores. The timings below were measured once on one laptop (20,000 rows by 30 columns, a synthetic binary problem). They vary by machine, version and data, so use them only for the ratios.

Fit	Seconds (one run)	Lesson
`SVC()` (RBF kernel)	2.3	Kernel SVM cost grows faster than linearly in rows; fine at 20,000, painful at 500,000
`LinearSVC(dual=False)`	0.05	A linear model is orders of magnitude cheaper when it is accurate enough
`GradientBoostingClassifier()`	11.7	The original boosting is single-threaded and exact
`HistGradientBoostingClassifier(max_iter=100)`	1.5	Binning plus multithreading: the default choice for big tables
`RandomForestClassifier(200, n_jobs=1)`	9.7	Trees are independent, so they parallelise well
`RandomForestClassifier(200, n_jobs=-1)`	1.3	Same model on all cores (this laptop has 12 threads)

Memory: one-hot encoding a text column with 5,000 distinct values over 100,000 rows gives a 100,000 by 5,000 matrix. Stored densely as float64 that would be 4 GB; scikit-learn's default sparse output stores it in about 1.6 MB (measured with `OneHotEncoder`), because only one value per row is non-zero. Keep sparse until a step demands dense; tell `ColumnTransformer` with `sparse_threshold` if you need to control it.

- **Use all cores:** `n_jobs=-1` for forests, searches and `cross_val_score`. Nested parallelism (a parallel search around a parallel forest) can oversubscribe the CPU; set it on the outer level only.
- **Search smarter:** randomised search before grid search; halving search for large grids; early stopping inside boosting.
- **Shrink the problem:** develop on a 10% sample, then scale up. Use `float32` inputs where precision allows.
- **Out of memory:** `partial_fit` estimators (`SGDClassifier` , `MultinomialNB` , `MiniBatchKMeans` , `IncrementalPCA`) read the data in chunks.
- **Profile first:** time the steps separately before optimising; the slow part is often a custom transformer or a data conversion, not the model.

Common mistakes, with the wrong code, the symptom and the fix

COMMON MISTAKE 1: FIT_TRANSFORM ON THE TEST SET

Wrong: `X_te_s = StandardScaler().fit_transform(X_te)` . **Symptom:** no error; the test set is scaled with its own mean and spread, so a shift in the new data is hidden and the model sees a different scaling than it was trained on. Scores look fine, but predictions on a single row are impossible (its standard deviation is 0). **Fix:** `fit_transform` on train, only `transform` on test, or use a Pipeline.

```
import numpy as np
from sklearn.preprocessing import StandardScaler

train = np.array([[10.0], [20.0], [30.0]])
test = np.array([[100.0], [110.0], [120.0]])      # the world has shifted upward

right = StandardScaler().fit(train).transform(test)
wrong = StandardScaler().fit_transform(test)
print("right:", right.ravel().round(2))
print("wrong:", wrong.ravel().round(2))
```

```
right: [ 9.8  11.02 12.25]
wrong: [-1.22  0.    1.22]
```

1. Training values average 20; new values are around 110, a large upward shift.
2. The correct route scales with the training statistics, so the shifted values come out far from zero (9.8 to 12.25 standard deviations away), which tells the model the data has drifted.
3. The wrong route re-centres the test set on itself, producing -1.22, 0, 1.22: the shift is erased and the model sees nothing unusual.

COMMON MISTAKE 2: SCORING ON THE TRAINING DATA	**Wrong:** `model.fit(X, y); print(model.score(X, y))` . **Symptom:** an unlimited decision tree or a 1-nearest-neighbour model scores 1.000 and then disappoints in production. **Fix:** report scores from held-out rows or cross-validation only; show train and test side by side to detect overfitting.
COMMON MISTAKE 3: PREPROCESSING OR SELECTING FEATURES BEFORE THE SPLIT	**Wrong:** `X = SelectKBest(k=20).fit_transform(X, y)` and then `cross_val_score(model, X, y)` . **Symptom:** 0.92 accuracy on random noise (Recipe 6). **Fix:** every learned step goes inside the Pipeline.
COMMON MISTAKE 4: TUNING ON THE TEST SET	**Wrong:** try ten models, keep the one with the best test score, report that score. **Symptom:** the reported number is the maximum of ten noisy estimates, so it is optimistic, and a fresh dataset disappoints. **Fix:** tune with CV on the training part; open the test set once, for the final chosen model.
COMMON MISTAKE 5: ACCURACY ON IMBALANCED DATA	**Wrong:** `accuracy_score(y, p)` for a 1% fraud problem. **Symptom:** 99% accuracy from a model that never flags anything (section 19.6). **Fix:** compare to a `DummyClassifier` , use recall, precision, average precision, and choose the threshold on purpose.
COMMON MISTAKE 6: INTEGER CODES FOR UNORDERED CATEGORIES	**Wrong:** `OrdinalEncoder().fit_transform(df[["city"]])` feeding a linear or kNN model. **Symptom:** the model treats "Paris = 2" as twice "Berlin = 1" and "Rome = 3" as between them, which is meaningless, so accuracy is poor and coefficients are nonsense. **Fix:** `OneHotEncoder` for nominal categories (ordinal codes are acceptable for trees, or for categories with a real order).
COMMON MISTAKE 7: FORGETTING TO SCALE	**Wrong:** `KNeighborsClassifier().fit(X, y)` with a salary column in the tens of thousands next to a 0-1 flag. **Symptom:** accuracy near chance (0.518 in section 19.4). **Fix:** `make_pipeline(StandardScaler(), model)` for SVM, kNN, k-means, PCA, MLP, and penalised linear models.

COMMON MISTAKE 8: TARGET LEAKAGE THROUGH A FEATURE

Wrong: predicting churn with a column `cancellation_reason` or `days_since_last_login_at_cancellation`, which only exists after the outcome. **Symptom:** AUC of 0.99 and one dominating feature importance. **Fix:** for every feature ask "will I know this at the moment I must predict?"; remove anything that is a consequence of the label; look at the top permutation importances with suspicion.

COMMON MISTAKE 9: RANDOM FOLDS ON GROUPED OR ORDERED DATA

Wrong: `KFold(5, shuffle=True)` over repeated patients or over time. **Symptom:** CV 0.99 against 0.52 with `GroupKFold` (section 19.5). **Fix:** `GroupKFold` or `TimeSeriesSplit`.

COMMON MISTAKE 10: COLUMN ORDER OR NAMES CHANGED AT PREDICTION TIME

Wrong: fit on a DataFrame, predict on a NumPy array with columns in another order (or a DataFrame with different names). **Symptom:** a `ValueError` about feature names for a bare model, or silently wrong predictions when a plain NumPy array has its columns in another order. **Fix:** always predict with a DataFrame with the same column names, and put a `ColumnTransformer` first so selection is by name.

```
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X = pd.DataFrame({"age": [20, 30, 40, 50], "income": [10, 20, 30, 40]})
y = [0, 0, 1, 1]
bare = LogisticRegression().fit(X, y)
safe = make_pipeline(ColumnTransformer([("s", StandardScaler(), ["age", "income"])]),
                     LogisticRegression()).fit(X, y)

for label, bad in (("renamed", X.rename(columns={"income": "salary"})),
                   ("reordered", X[["income", "age"]])):
    try:
        bare.predict(bad)
    except ValueError as e:
        print(label, "-> bare model:", str(e).splitlines()[1])
print("reordered -> pipeline with ColumnTransformer:", safe.predict(X[["income", "age"]]))
```

```
renamed -> bare model: Feature names unseen at fit time:
reordered -> bare model: Feature names must be in the same order as they were in fit.
reordered -> pipeline with ColumnTransformer: [0 0 1 1]
```

1. The model is fitted on a DataFrame, so it stores `feature_names_in_` and checks it at predict time.
2. A renamed column and a merely reordered DataFrame both make the bare model raise a `ValueError`. We print the second line of each message, which says what is wrong.
3. A pipeline that starts with a `ColumnTransformer` selects columns by name, so it accepts the reordered frame and puts the columns back in the trained order.
4. A plain NumPy array has no names to check, so it is interpreted by position and a shuffled order would give silently wrong answers.

COMMON MISTAKE 11: DENSE ONE-HOT ON HIGH-CARDINALITY COLUMNS

Wrong: `OneHotEncoder(sparse_output=False)` on a column of 50,000 user IDs. **Symptom:** `MemoryError`, or a model that memorises IDs. **Fix:** keep sparse output, set `min_frequency` or `max_categories` to group rare values, or use `TargetEncoder` (which cross-fits internally) for a high-cardinality predictive column. A pure identifier should not be a feature at all.

COMMON MISTAKE 12: READING NEGATIVE SCORES AS BAD

Wrong: `cross_val_score(m, X, y, scoring="neg_mean_squared_error")` returns `[-4.1, -3.9, ...]`, and you conclude the model is broken. **Symptom:** confusion and sign errors in reports. **Fix:** scikit-learn maximises, so errors are negated; negate back to report them (`-scores.mean()`). Higher (closer to zero) is better.

COMMON MISTAKE 13: TRUSTING RAW PROBABILITIES AND UNSCALED COEFFICIENTS

Wrong: using `RandomForest.predict_proba` or `GaussianNB` output as a literal risk percentage, or ranking features by `coef_` when columns have different units. **Symptom:** "80% risk" cases come true 55% of the time; a column measured in small units looks hugely important. **Fix:** check `calibration_curve` and calibrate when probabilities are used as numbers; scale before comparing coefficients, or use permutation importance.

COMMON MISTAKE 14: UNSEEDED RANDOMNESS AND UNFAIR COMPARISONS

Wrong: comparing two models on different random splits, with no `random_state`. **Symptom:** the "winner" changes every run. **Fix:** create one CV splitter with a fixed seed and use it for every model; set `random_state` on every random estimator.

COMMON MISTAKE 15: A 2-D COLUMN VECTOR AS Y, OR 1-D X

Wrong: `model.fit(X, df[["target"]])` or `model.fit(df["age"], y)`. **Symptom:** a `DataConversionWarning` about a column-vector y, or `Expected 2D array, got 1D array`. **Fix:** `y = df["target"]` (one bracket) and `X = df[["age"]]` (two brackets).

```
import numpy as np
from sklearn.dummy import DummyRegressor
from sklearn.model_selection import cross_val_score

X = np.arange(20).reshape(-1, 1)
y = np.arange(20) * 1.0
scores = cross_val_score(DummyRegressor(), X, y, cv=4, scoring="neg_mean_squared_error")
print(scores.round(1))
print("report as MSE:", round(-scores.mean(), 1))
```

```
[-102.   -13.1  -13.1 -102. ]
report as MSE: 57.6
```

1. A mean-only baseline on a perfectly linear target gives sizeable errors.
2. Folds 1 and 4 hold the extremes of the range, so they score -102.0; the middle folds score -13.1.
3. Every score is negative because the scorer returns the negated MSE.
4. We flip the sign before reporting: an MSE of 57.6 is the typical squared error of this baseline.

19.10 Cross-Library Integration

Scikit-learn rarely works alone. This section shows the handshakes with the libraries around it. The first three examples are run; the rest are short notes.

pandas and SQL in, scikit-learn out

A typical job: query a table, build features, fit. Here an in-memory SQLite database stands in for a warehouse. With pandas 3, text columns arrive with the new `str` dtype, and scikit-learn's encoders and selectors accept that.

```
import sqlite3
import numpy as np
import pandas as pd
from sklearn.compose import ColumnTransformer, make_column_selector
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.model_selection import cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import OneHotEncoder

rng = np.random.default_rng(0)
con = sqlite3.connect(":memory:")
n = 800
pd.DataFrame({"plan": rng.choice(["basic", "plus", "pro"], n),
              "age": rng.integers(18, 70, n),
              "visits": rng.poisson(5, n)}).assign(
    bought=lambda d: ((d["visits"] > 5) ^ (rng.random(n) < 0.15)).astype(int)
).to_sql("customers", con, index=False)

df = pd.read_sql("SELECT plan, age, visits, bought FROM customers", con)
print(df["plan"].dtype, df["age"].dtype)

X, y = df.drop(columns="bought"), df["bought"]
prep = ColumnTransformer([("cat", OneHotEncoder(handle_unknown="ignore"),
                           make_column_selector(dtype_include=object))],
                         remainder="passthrough")
pipe = make_pipeline(prep, HistGradientBoostingClassifier(random_state=0))
print("cv accuracy:", cross_val_score(pipe, X, y, cv=5).mean().round(3))
```

```
str int64
cv accuracy: 0.798
```

1. We create an in-memory SQLite database and write a small `customers` table into it, with a purchase flag that depends on the number of visits plus 15% label noise.
2. `pd.read_sql` runs a query and returns a DataFrame. The printed dtypes show `plan` as the pandas 3 `str` dtype and `age` as `int64`.
3. `make_column_selector(dtype_include=object)` picks the text columns by dtype, so the pipeline does not hard-code names; it still matches pandas 3 `str` columns in this scikit-learn version.
4. `remainder="passthrough"` sends the numeric columns on unchanged.
5. The whole thing is cross-validated as one object. The best possible accuracy is about 0.85 because 15% of the labels were flipped on purpose; the 0.798 we get is a little below that ceiling.

matplotlib and seaborn: plots from results

```
import os
import tempfile
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_wine
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import ConfusionMatrixDisplay, confusion_matrix
from sklearn.model_selection import cross_val_predict
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

data = load_wine()
pipe = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))
pred = cross_val_predict(pipe, data.data, data.target, cv=5)
cm = confusion_matrix(data.target, pred)

fig, axes = plt.subplots(1, 2, figsize=(8, 3.5))
ConfusionMatrixDisplay(cm, display_labels=data.target_names).plot(ax=axes[0], colorbar=False)
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", cbar=False, ax=axes[1],
            xticklabels=data.target_names, yticklabels=data.target_names)
with tempfile.TemporaryDirectory() as d:
    fig.savefig(os.path.join(d, "cm.png"), dpi=80)
    print("saved:", os.path.getsize(os.path.join(d, "cm.png")) > 1000)
print(cm.diagonal().sum(), "of", cm.sum(), "correct")
```

```
saved: True
175 of 178 correct
```

1. Out-of-fold predictions give an honest confusion matrix.
2. scikit-learn's own `ConfusionMatrixDisplay` draws on a matplotlib axis you supply, so it fits into your own figure layout.
3. seaborn's `heatmap` draws the same matrix from the raw array, with more control over colour and annotation format (`fmt="d"` prints integers).
4. Saving to a temporary file keeps the example clean; in a notebook you would simply show the figure.

Serving a model over HTTP

A model becomes useful when another program can call it. The Python chapter on APIs (Chapter 12) covers FastAPI and Flask, which are the usual choices. To keep this example self-contained and runnable, we use the standard library server on a free local port and call it with `requests`.

```
import json
import threading
from http.server import BaseHTTPRequestHandler, HTTPServer

import requests
from sklearn.datasets import load_iris
from sklearn.linear_model import LogisticRegression

iris = load_iris()
model = LogisticRegression(max_iter=300).fit(iris.data, iris.target)

class Handler(BaseHTTPRequestHandler):
    def do_POST(self):
        body = json.loads(self.rfile.read(int(self.headers["Content-Length"])))
        pred = model.predict([body["features"]])[0]
        out = json.dumps({"species": str(iris.target_names[pred])}).encode()
        self.send_response(200)
        self.send_header("Content-Type", "application/json")
        self.send_header("Content-Length", str(len(out)))
        self.end_headers()
        self.wfile.write(out)
    def log_message(self, *args):
        pass

server = HTTPServer(("127.0.0.1", 0), Handler)       # port 0 = pick any free port
threading.Thread(target=server.serve_forever, daemon=True).start()
url = f"http://127.0.0.1:{server.server_port}/predict"

for row in ([5.0, 3.4, 1.5, 0.2], [6.7, 3.0, 5.2, 2.3]):
    print(requests.post(url, json={"features": row}, timeout=5).json())
server.shutdown()
```

```
{'species': 'setosa'}
{'species': 'virginica'}
```

1. The model is trained once when the server starts, not on every request.
2. The handler reads the JSON body (using the `Content-Length` header), calls `predict` with one row, and answers with JSON.
3. `HTTPServer(("127.0.0.1", 0), ...)` binds to a free port on the local machine only. A daemon thread serves requests in the background.
4. `requests.post(url, json=...)` sends the request and `.json()` decodes the reply. Two flowers give two species names.
5. In production you would load a saved pipeline from disk once at start-up, validate input with a schema, and run the server behind a proper framework.

Quick notes on the rest

- **NumPy and SciPy.** Estimators accept NumPy arrays and SciPy sparse matrices (`scipy.sparse.csr_matrix`); text vectorisers and one-hot encoders return sparse matrices by default. Search distributions come from `scipy.stats` (`loguniform` , `randint`).
- **pandas.** DataFrames in, names preserved in `feature_names_in_` ; `set_output(transform="pandas")` to get DataFrames out. Use `df[["col"]]` for one feature. Under pandas 3 copy-on-write, `X_train = df[cols]` is a safe independent object, so the old `SettingWithCopyWarning` no longer applies; still, do not modify X after fitting and expect the model to notice.
- **joblib.** Used for parallelism inside scikit-learn (`n_jobs`) and for saving models. You can change the backend with `joblib.parallel_config` .
- **statsmodels.** Use it when you need coefficients with p-values and confidence intervals (see the Statistics document). Use scikit-learn for prediction and validation.
- **XGBoost and LightGBM** (*not run here*). Both ship scikit-learn-compatible classes (`XGBClassifier` , `LGBMClassifier`), so they work inside `Pipeline` , `cross_val_score` and `GridSearchCV` unchanged.
- **Polars** (*not run here*). Recent scikit-learn can return Polars frames with `set_output(transform="polars")` and accepts Polars input in many places; check the version notes of your install.

- **Deep learning.** `skorch` wraps PyTorch models in the scikit-learn interface (not run here); otherwise export features from scikit-learn and train in PyTorch.

19.11 How to Solve Any scikit-learn Problem: A Playbook

The decision flow

Follow these steps in order. Most real problems are solved by doing step 3 or 4 properly, not by finding a cleverer algorithm.

1. **Name the task and the cost of mistakes.** Predict a category (classification), a number (regression), group rows (clustering), find odd rows (anomaly detection), or compress columns (PCA)? Which error hurts more, a false alarm or a miss? That chooses the metric.
2. **Split first and lock the test set.** Use `stratify` for classes, `GroupKFold` for repeated entities, `TimeSeriesSplit` for time.
3. **Baseline.** A `DummyClassifier` or `DummyRegressor`, then a scaled linear model. Record the numbers.
4. **Put all preprocessing in a Pipeline** with a `ColumnTransformer`: impute, scale, encode.
5. **Try one strong nonlinear model** (`HistGradientBoosting*` or a forest). Compare with the same folds.
6. **Diagnose** with train versus validation scores: both low means underfit; high train and low validation means overfit; a suspiciously perfect score means leakage.
7. **Tune** the best one with randomised search, then choose the threshold for the business cost.
8. **Check calibration** if probabilities are used as numbers, and **inspect** with permutation importance.
9. **Open the test set once**, report the score with its uncertainty, save the pipeline with versions.

Situation	Start with	Why
Tabular, mixed types, missing values	`HistGradientBoostingClassifier` or Regressor	Strong default, handles NaN, fast
Small data, little tuning time	`RandomForest*`	Hard to break, free OOB estimate
Need a readable baseline	Scaled `LogisticRegression` / `Ridge`	Weights you can explain
Text	`TfidfVectorizer` + `LogisticRegression` / `LinearSVC`	Fast and strong on sparse features
Unlabelled, round groups	`KMeans` on scaled data	Simple and scalable
Odd shapes or noise	`DBSCAN` / `HDBSCAN`	No k needed, labels noise
Spot unusual rows	`IsolationForest`	Works without labels

Eight recurring problem patterns, each solved

Pattern 1: "The score is too good to be true" (leakage). Look at what the model leans on. Below, a column `refund_issued` is only filled in after a customer has already cancelled.

```
import numpy as np
import pandas as pd
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.inspection import permutation_importance
from sklearn.metrics import roc_auc_score
from sklearn.model_selection import train_test_split

rng = np.random.default_rng(0)
n = 3000
df = pd.DataFrame({"tenure": rng.integers(1, 60, n), "calls": rng.poisson(2, n)})
p = 1 / (1 + np.exp(-(0.4 * df["calls"] - 0.03 * df["tenure"] - 0.5)))
df["churn"] = (rng.random(n) < p).astype(int)
df["refund_issued"] = df["churn"] * (rng.random(n) < 0.9)       # consequence of the label

def run(cols):
    X_tr, X_te, y_tr, y_te = train_test_split(df[cols], df["churn"], random_state=0)
    m = HistGradientBoostingClassifier(random_state=0).fit(X_tr, y_tr)
    return m, X_te, y_te, roc_auc_score(y_te, m.predict_proba(X_te)[:, 1])

m, X_te, y_te, auc = run(["tenure", "calls", "refund_issued"])
imp = permutation_importance(m, X_te, y_te, n_repeats=5, random_state=0, scoring="roc_auc")
print("with leak AUC:", round(auc, 3), {c: round(float(v), 3) for c, v in zip(X_te.columns, imp.importances_mean)})
print("fixed AUC    :", round(run(["tenure", "calls"])[3], 3))
```

```
with leak AUC: 0.945 {'tenure': 0.002, 'calls': -0.002, 'refund_issued': 0.398}
fixed AUC    : 0.646
```

1. We build a churn target from tenure and calls, then add `refund_issued`, which is 1 for 90% of churners and never for others. It exists only after the outcome.
2. With it included, the model reaches an AUC of 0.945.
3. Permutation importance on the test rows shows that one column carries almost all the weight: shuffling `refund_issued` costs 0.398 AUC, while shuffling tenure or calls costs about nothing. That is the signature of a leak.
4. Removing it gives an AUC of 0.646, the real number that will hold in production. The leaky model was reporting an excellent result that no live system could reproduce.

Pattern 2: "Great on train, poor on validation" (overfitting). Make the model simpler or average more.

```
from sklearn.datasets import make_classification
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split

X, y = make_classification(1500, 20, n_informative=5, flip_y=0.15, random_state=0)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, random_state=0)
for leaf in (1, 10, 30):
    m = RandomForestClassifier(200, min_samples_leaf=leaf, random_state=0).fit(X_tr, y_tr)
    print(f"min_samples_leaf={leaf:2d} train={m.score(X_tr, y_tr):.3f} test={m.score(X_te, y_te):.3f}")
```

```
min_samples_leaf= 1 train=1.000 test=0.829
min_samples_leaf=10 train=0.891 test=0.832
min_samples_leaf=30 train=0.832 test=0.824
```

1. 15% of labels are flipped, so no model can exceed about 0.85 to 0.90 honestly.
2. With `min_samples_leaf=1` every tree grows until leaves are single rows, so the forest memorises training noise (train near 1.0).
3. Larger leaves force each tree to average over more rows. The train score falls from 1.000 to 0.891 to 0.832 while the test score stays near 0.83 (0.829, 0.832, 0.824). The gap shrinks from 0.171 to 0.008 and the forest stops memorising, without losing test accuracy. The gap, not the training score, is the thing to watch. Here the test score is already at the noise ceiling, so simplifying cost nothing.

Pattern 3: "Poor on both" (underfitting). The model is too simple. Add flexibility or better features.

```
from sklearn.datasets import make_moons
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import PolynomialFeatures, StandardScaler

X, y = make_moons(800, noise=0.25, random_state=0)
options = {
    "logistic": LogisticRegression(),
    "logistic + degree-3 polynomial": make_pipeline(PolynomialFeatures(3), StandardScaler(),
LogisticRegression(max_iter=1000)),
    "hist boosting": HistGradientBoostingClassifier(random_state=0),
}
for name, m in options.items():
    print(f"{name:32s} {cross_val_score(m, X, y, cv=5).mean():.3f}")
```

```
logistic                         0.859
logistic + degree-3 polynomial   0.909
hist boosting                    0.938
```

1. The moons are not separable by a straight line, so plain logistic regression underfits (both train and validation low).
2. Adding polynomial features lets the same linear model draw a curved boundary, a cheap and interpretable fix.
3. A boosted tree model gets there by itself. Cross-validated accuracy rises from 0.859 (logistic) to 0.909 (polynomial) and 0.938 (boosting). The diagnostic that told you to try them was low training accuracy.

Pattern 4: "Rare positives" (imbalance). Optimise a rare-class metric and set the threshold deliberately.

```
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import f1_score
from sklearn.model_selection import TunedThresholdClassifierCV, train_test_split

X, y = make_classification(8000, 12, n_informative=4, weights=[0.95], flip_y=0.02, random_state=2)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, stratify=y, random_state=2)

plain = LogisticRegression(max_iter=1000).fit(X_tr, y_tr)
tuned = TunedThresholdClassifierCV(LogisticRegression(max_iter=1000), scoring="f1", cv=5).fit(X_tr, y_tr)
print("default 0.5 threshold F1:", round(f1_score(y_te, plain.predict(X_te)), 3))
print("tuned threshold      F1:", round(f1_score(y_te, tuned.predict(X_te)), 3),
      " threshold:", round(float(tuned.best_threshold_), 3))
```

```
default 0.5 threshold F1: 0.481
tuned threshold      F1: 0.53  threshold: 0.22
```

1. A 5% positive problem. Plain logistic regression uses the 0.5 cut-off, which is too high for a rare class.
2. `TunedThresholdClassifierCV` fits the model in cross-validation, finds the threshold that maximises F1 on out-of-fold predictions, and returns a classifier that applies it.
3. The tuned F1 on the untouched test set rises from 0.481 to 0.530, and `best_threshold_` shows the cut-off chosen, 0.22, well below 0.5. The gain is modest because the ranking itself is mediocre; a threshold can only move the operating point.

Pattern 5: "The score jumps around" (small, noisy data). Repeat the CV and prefer the simpler model when scores tie within one standard error.

```
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import RepeatedStratifiedKFold, cross_val_score, StratifiedKFold
from sklearn.tree import DecisionTreeClassifier

X, y = load_breast_cancer(return_X_y=True)
X, y = X[:200], y[:200]                       # a small sample makes estimates noisy
m = DecisionTreeClassifier(max_depth=3, random_state=0)
for seed in (0, 1, 2):
    s = cross_val_score(m, X, y, cv=StratifiedKFold(5, shuffle=True, random_state=seed)).mean()
    print("single 5-fold, seed", seed, round(s, 4))
rep = cross_val_score(m, X, y, cv=RepeatedStratifiedKFold(n_splits=5, n_repeats=10, random_state=0))
print("repeated 5x10 mean:", rep.mean().round(4), " std error:", (rep.std() / np.sqrt(len(rep))).round(4))
```

```
single 5-fold, seed 0 0.895
single 5-fold, seed 1 0.89
single 5-fold, seed 2 0.905
repeated 5x10 mean: 0.9025  std error: 0.0052
```

1. We deliberately use only 200 rows and a tree, which is sensitive. Three single 5-fold runs differ only by the shuffling seed, yet give 0.895, 0.890 and 0.905. That wobble of a point and a half is the noise in your estimate.
2. Ten repetitions average over 50 fold scores, giving a steadier mean (0.9025) and a standard error of 0.0052.
3. Rule of thumb ("one standard error rule"): among models within one standard error of the best, choose the simplest.

Pattern 6: "Works in the notebook, fails in production" (schema and drift). The cause is almost always different columns, new categories, missing values or a different library version. Make the pipeline tolerant and add an input check.

```
import numpy as np
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline, make_pipeline
from sklearn.preprocessing import OneHotEncoder

train = pd.DataFrame({"plan": ["a", "b", "a", "b", "a", "b"], "usage": [1.0, 2.0, 3.0, 4.0, 5.0, 6.0]})
y = [0, 0, 0, 1, 1, 1]
prep = ColumnTransformer([("c", OneHotEncoder(handle_unknown="ignore"), ["plan"]),
                          ("n", SimpleImputer(strategy="median"), ["usage"])])
pipe = Pipeline([("prep", prep), ("clf", LogisticRegression())]).fit(train, y)

def check_input(df, expected=("plan", "usage")):
    missing = [c for c in expected if c not in df.columns]
    if missing:
        raise ValueError(f"missing columns: {missing}")
    return df[list(expected)]

live = pd.DataFrame({"usage": [np.nan, 5.5], "plan": ["zzz", "b"], "extra": [1, 2]})
print(pipe.predict(check_input(live)))
try:
    check_input(live.drop(columns="plan"))
except ValueError as e:
    print(e)
```

```
[1 1]
missing columns: ['plan']
```

1. The encoder uses `handle_unknown="ignore"` and the numeric imputer covers missing values, so a never-seen plan `zzz` and a missing usage no longer crash prediction.

2. `check_input` enforces the schema: it selects and orders the expected columns, dropping surprises like `extra`, and fails loudly with a clear message if one is missing.
3. Also record the library versions with the saved model, and log the input distributions so drift is visible.

Pattern 7: "It is too slow." Change the algorithm before tuning code.

```
import time
from sklearn.datasets import make_classification
from sklearn.linear_model import SGDClassifier
from sklearn.svm import LinearSVC, SVC

X, y = make_classification(15000, 30, n_informative=10, random_state=0)

def seconds(model):
    start = time.perf_counter(); model.fit(X, y); return time.perf_counter() - start

t_rbf = seconds(SVC())
t_lin = seconds(LinearSVC(dual=False))
t_sgd = seconds(SGDClassifier(random_state=0))
print("LinearSVC at least 5x faster than RBF SVC:", t_lin * 5 < t_rbf)
print("SGDClassifier at least 5x faster than RBF SVC:", t_sgd * 5 < t_rbf)
```

```
LinearSVC at least 5x faster than RBF SVC: True
SGDClassifier at least 5x faster than RBF SVC: True
```

1. Exact timings change from run to run, so we print only a robust yes/no about a large ratio.
2. A kernel SVM scales poorly with rows; `LinearSVC` and `SGDClassifier` scale linearly.
3. Other levers in order: switch to `HistGradientBoosting` instead of exact boosting, use `n_jobs`, reduce rows for experiments, keep matrices sparse, and cache slow pipeline steps with `memory`.

Pattern 8: "The business wants to know why." Combine permutation importance, partial dependence and a small surrogate tree that imitates the complex model.

```
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, export_text

data = load_breast_cancer()
X_tr, X_te, y_tr, y_te = train_test_split(data.data, data.target, random_state=0)
black_box = HistGradientBoostingClassifier(random_state=0).fit(X_tr, y_tr)

surrogate = DecisionTreeClassifier(max_depth=2, random_state=0).fit(X_tr, black_box.predict(X_tr))
fidelity = (surrogate.predict(X_te) == black_box.predict(X_te)).mean()
print("black box accuracy:", round(black_box.score(X_te, y_te), 3), " surrogate fidelity:", round(fidelity, 3))
print(export_text(surrogate, feature_names=list(data.feature_names)))
```

```
black box accuracy: 0.986  surrogate fidelity: 0.937
|--- mean concave points <= 0.05
|   |--- worst area <= 952.90
|   |   |--- class: 1
|   |--- worst area >  952.90
|   |   |--- class: 0
|--- mean concave points >  0.05
|   |--- worst area <= 785.80
|   |   |--- class: 1
|   |--- worst area >  785.80
|   |   |--- class: 0
```

1. The complex model is trained normally.
2. The surrogate is a depth-2 tree trained to predict the *black box's predictions*, not the true labels. It is a readable approximation.
3. Fidelity is how often the surrogate agrees with the black box on unseen rows: 0.937 here, with the black box itself at 0.986 accuracy. High fidelity means the rules are a fair summary (two features, `mean concave points` and `worst area`, explain about 94% of its decisions); low fidelity means do not present them as an explanation.
4. The printed rules show which features and thresholds drive decisions, a conversation-friendly artefact.

How to verify any result

- Beat the dummy baseline by a clear margin, on the same folds.
- Check that the score on shuffled labels collapses to chance (the next example does this).
- Compare train and validation scores; read the confusion matrix, not just a single number.
- Look at the top features; if one dominates, ask whether it could be a consequence of the label.
- Test on a time-later or other-group slice if the real use is on new groups or future dates.

How to debug when something is wrong

```
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import permutation_test_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

X, y = load_breast_cancer(return_X_y=True)
print("shape", X.shape, " NaN:", int(np.isnan(X).sum()), " class counts:", np.bincount(y))

score, perm_scores, pvalue = permutation_test_score(
    make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000)), X, y,
    cv=5, n_permutations=20, random_state=0)
print("real score:", round(score, 3), " shuffled-label mean:", perm_scores.mean().round(3),
      " p-value:", round(pvalue, 3))

tiny = RandomForestClassifier(random_state=0).fit(X[:20], y[:20])
print("can it memorise 20 rows?", tiny.score(X[:20], y[:20]))
```

```
shape (569, 30)  NaN: 0  class counts: [212 357]
real score: 0.981  shuffled-label mean: 0.602  p-value: 0.048
can it memorise 20 rows? 1.0
```

1. First look at the raw facts: shape, number of missing cells and class counts. Many "model bugs" are data bugs.
2. `permutation_test_score` repeats the CV with shuffled labels 20 times. If the real score is far above the shuffled scores (and the p-value small), the model has found real signal. If real and shuffled scores are close, there is nothing to find or a bug.
3. Reading the output: the real score is 0.981 and the shuffled-label scores average 0.602, not 0.5, because 62.7% of the rows are one class and a model that learns nothing still gets that. This is why the baseline and the shuffled test matter. The p-value is 0.048 = 1/21, the smallest value possible with 20 permutations.
4. The tiny-subset test asks whether the model can at least memorise 20 rows. If a flexible model cannot, the code path (labels, shapes, dtypes) is broken, not the algorithm.
5. When a pipeline misbehaves, run it in pieces: `pipe[:-1].fit_transform(X)` and look at the output matrix, then fit the last step on that.

19.12 Going Deeper: Internals, Memory, Speed and What Interviewers Probe

This section is for when you want to know *why* the library behaves as it does. Skip it on a first read.

Internals: the estimator contract

Scikit-learn works because of a few strict conventions, and breaking them causes strange failures in `clone`, grid search and pipelines.

- **The constructor only stores arguments**, under the same names, with no validation and no changes. Validation happens in `fit`. This is what lets `get_params`, `set_params` and `clone` rebuild any object from its settings.
- **Learned state ends with an underscore** and is created only in `fit`. `check_is_fitted` looks for such attributes.
- `fit` **returns** `self`, which enables chaining (`model.fit(X, y).predict(X)`).
- **Input checking** is done by `validate_data`: it converts lists and DataFrames to arrays, rejects NaN and infinity unless allowed, and records `n_features_in_` and `feature_names_in_`.
- **Tags** (`__sklearn_tags__`) describe an estimator's capabilities, such as whether it accepts sparse input or NaN. Tools like pipelines and the test suite read them.
- **Randomness:** `random_state` may be an integer (reproducible) or a `RandomState` instance. Using an integer in an estimator that is cloned many times gives each clone the same stream, which is what you want for reproducibility; passing a shared instance makes results depend on call order.

```
from sklearn.base import BaseEstimator, clone
from sklearn.linear_model import LogisticRegression


class Bad(BaseEstimator):
    def __init__(self, scale=1.0):
        self.scale = scale * 2          # forbidden: modifies the argument in the constructor


class Good(BaseEstimator):
    def __init__(self, scale=1.0):
        self.scale = scale


print(clone(Good(3.0)).scale)
try:
    clone(Bad(3.0))
except RuntimeError as e:
    print("clone failed:", str(e).splitlines()[0][:80])

tags = LogisticRegression().__sklearn_tags__()
print("sparse input:", tags.input_tags.sparse, " allows NaN:", tags.input_tags.allow_nan)
```

```
3.0
clone failed: Cannot clone object Bad(scale=6.0), as the constructor either does not set or mo
sparse input: True  allows NaN: False
```

1. `Good` stores its argument unchanged. `clone` reads the parameters with `get_params`, builds a new object, and checks that the new object holds exactly the same parameter values. It does, so the clone has `scale` 3.0.
2. `Bad` doubles the value in its constructor, so `Bad(3.0)` holds 6.0. `clone` reads 6.0, builds `Bad(6.0)` (which would hold 12.0) and finds the stored value no longer matches, so it raises a `RuntimeError` pointing at the constructor, as the first line of the message shows. The same mistake breaks `GridSearchCV` and `cross_val_score`, which clone internally.
3. The last lines read the tags of logistic regression: it accepts sparse input and does not accept NaN, which is why NaN needs an imputer in front of it.

Memory

```
import numpy as np
from scipy import sparse

dense64 = np.zeros((10000, 1000))
dense32 = dense64.astype(np.float32)
rng = np.random.default_rng(0)
mask = rng.random((10000, 1000)) < 0.01                # 1% non-zero
csr = sparse.csr_matrix(np.where(mask, 1.0, 0.0))

print("float64 MB:", dense64.nbytes / 1e6)
print("float32 MB:", dense32.nbytes / 1e6)
print("csr MB    :", round((csr.data.nbytes + csr.indices.nbytes + csr.indptr.nbytes) / 1e6, 2))
```

```
float64 MB: 80.0
float32 MB: 40.0
csr MB    : 1.24
```

1. A 10,000 by 1,000 float64 matrix needs 8 bytes per cell, 80 MB.
2. float32 halves it (40 MB). Many tree models and `HistGradientBoosting` bin or convert to smaller types internally anyway.
3. A compressed sparse row (CSR) matrix stores only the 1% of cells that are non-zero, plus two index arrays, about 1.2 MB here. Text vectorisers and one-hot encoders produce CSR by default; avoid `.toarray()` unless you must.
4. Remember that cross-validation, search and bagging make copies: memory use can be a multiple of the data size. `n_jobs` workers each get their own copy of the arrays unless the data is memory-mapped, which joblib does automatically for large NumPy arrays.

Speed: where time really goes

- **Three layers of parallelism.** joblib processes or threads (`n_jobs`) for independent fits; OpenMP threads inside Cython code (histogram boosting, k-means, some distances); BLAS threads inside NumPy linear algebra. They can multiply into oversubscription; limit one layer with `threadpoolctl.threadpool_limits` or the environment variables `OMP_NUM_THREADS`.
- **Histogram boosting is fast** because it bins every feature into at most 255 buckets once, then each split search scans bucket statistics instead of sorting raw values. The cost per tree no longer depends on how many distinct values a feature has.
- **Solvers matter.** `LogisticRegression` defaults to `lbfgs`; `saga` suits huge sparse data; `liblinear` suits small data and L1. PCA's `svd_solver="auto"` picks randomised SVD when you want few components from a big matrix.
- **K-means** uses k-means++ seeding and Lloyd iterations. Its cost is about rows × clusters × features per iteration, times `n_init` restarts.
- **Complexity notes:** kNN has no training cost but predicts slowly (it scans or searches an index); kernel SVM training grows faster than linearly with rows; a random forest's cost is roughly trees × rows × log(rows) × features tried per split.

Do I need more data? The learning curve

```
import numpy as np
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import learning_curve

X, y = make_classification(3000, 20, n_informative=6, flip_y=0.1, random_state=0)
sizes, tr, va = learning_curve(LogisticRegression(max_iter=1000), X, y,
                               train_sizes=[0.05, 0.1, 0.25, 0.5, 1.0], cv=5, random_state=0)
for n, a, b in zip(sizes, tr.mean(axis=1), va.mean(axis=1)):
    print(f"n={n:5d} train={a:.3f} validation={b:.3f} gap={a - b:.3f}")
```

```
n=  120 train=0.868 validation=0.757 gap=0.111
n=  240 train=0.839 validation=0.789 gap=0.050
n=  600 train=0.816 validation=0.792 gap=0.023
n= 1200 train=0.812 validation=0.801 gap=0.011
n= 2400 train=0.810 validation=0.807 gap=0.003
```

1. `learning_curve` fits the model on growing fractions of the training folds and records train and validation scores.
2. With 120 rows the model fits its training rows well (0.868) but generalises poorly (0.757), a gap of 0.111.
3. As rows grow the two curves converge. If the validation curve has flattened and the gap is small, more data will not help; you need a better model or better features. If the curves are still converging, collecting more data pays off. Here the gap shrinks to 0.003 at 2,400 rows and validation creeps from 0.801 to 0.807, so more rows would add almost nothing; the ceiling is set by the 10% label noise.

What interviewers probe

- **"Why not scale before splitting?"** The scaler's mean and variance would include test rows, leaking information. Fit on train, transform test, ideally in a Pipeline.
- **"Explain bias versus variance with a tree."** A shallow tree has high bias (underfits); a deep tree has high variance (overfits). A forest lowers variance by averaging; boosting lowers bias by sequential correction.
- **"Your AUC is 0.99. What do you check?"** Leakage (target-derived features, duplicates across the split, group or time structure), then a shuffled-label test and permutation importances.
- **"Precision versus recall; when do you prefer each?"** Recall when a miss is costly (disease screening); precision when a false alarm is costly (account lockouts). Choose the threshold from the cost, and use PR curves for rare classes.
- **"How does `fit_transform` differ from `fit` then `transform`?"** Often identical; some transformers implement a faster joint path or behave differently on training data (for example `TargetEncoder` cross-fits during `fit_transform` to avoid leaking the target).
- **"What does `class_weight="balanced"` do?"** Scales each row's loss by n / (k × class count). It changes training, and effectively moves the threshold.
- **"Why is permutation importance preferred over impurity importance?"** Impurity importance is computed on training data and is biased toward high-cardinality and noisy continuous columns; permutation importance uses held-out data, but is distorted by correlated features.
- **"How would you tune a model without overfitting the CV?"** Random search with CV on train only, a final untouched test set, and nested CV for an unbiased estimate of the whole procedure.
- **"What goes wrong with accuracy on imbalanced data, and what do you use?"** The majority baseline looks great. Use recall, precision, PR AUC or average precision, balanced accuracy, and a tuned threshold.
- **"How do you deploy it?"** Save the whole pipeline with `joblib`, record the library versions, validate input schema, monitor input drift and prediction drift, and keep the training script to rebuild.

Chapter recap

- Every scikit-learn object follows one contract: construct with settings, `fit` to learn (state ends with an underscore), then `predict` or `transform`. `X` is 2-D, `y` is 1-D.
- Split first, with `stratify` for classes, `GroupKFold` for repeated entities and `TimeSeriesSplit` for time. The test set is opened once.
- All learned preprocessing (imputers, scalers, encoders, selectors, vectorisers) belongs in a `Pipeline`; a `ColumnTransformer` routes columns. That prevents leakage and ships training and serving together.
- Always run a dummy baseline and a scaled linear model before anything fancy. For tables, `HistGradientBoosting` is the strong default; forests are the robust one.
- Cross-validate with the same folds for every model, report mean and spread, and tune with `RandomizedSearchCV` on training data only. Use nested CV for an honest estimate of the whole procedure.
- Match the metric to the cost of errors. On rare positives use precision, recall, average precision and a chosen threshold, not accuracy.
- Probabilities from many models are not calibrated. Check `calibration_curve` and wrap with `CalibratedClassifierCV` when probabilities are used as numbers.
- Inspect models with permutation importance (on held-out data) and partial dependence; impurity importances and raw coefficients mislead.

- Save the whole pipeline with `joblib`, with versions. Never load files from untrusted sources.
- When a score looks too good, suspect leakage first: target-derived features, selection before splitting, duplicates, group or time structure.

Formula / pattern	Meaning
`m.fit(X_tr, y_tr).predict(X_te)`	Learn on train, apply on unseen rows
`sc.fit(X_tr); sc.transform(X_te)`	Learn statistics on train only
`make_pipeline(prep, model)`	One object, leak-free CV, easy to save
`ColumnTransformer([(name, step, cols)])`	Different preprocessing per column group
`pipe.set_params(step__param=v)`	Double underscore reaches inside a pipeline
`cross_val_score(pipe, X, y, cv=cv, scoring=s)`	k honest scores; report mean and std
`RandomizedSearchCV(pipe, dist, n_iter=n, cv=k)`	Fixed-budget tuning; fits = n × k + 1
precision = TP / (TP + FP); recall = TP / (TP + FN)	Trust in alarms; share of real cases caught
F1 = 2PR / (P + R)	Harmonic mean of precision and recall
class weight = n / (k × count)	What `class_weight="balanced"` applies
AUC = share of (positive, negative) pairs ranked correctly	Threshold-free ranking quality
`permutation_importance(m, X_te, y_te)`	Drop in score when a column is shuffled on held-out data
`joblib.dump(pipe, path)` / `joblib.load(path)`	Persist the whole fitted pipeline

Practice set

Twenty-three problems, easy to expert. Try each before reading the answer. Where a problem shows code, the printed output is given in the answer, and it was produced by running the code.

Problem 19.P.1 (Easy) What does `fit` return, and how can you tell from the attribute names that a model has been fitted?

Answer: `fit` returns the estimator itself (`self`), which allows chaining such as `model.fit(X, y).predict(X)`. Attributes learned from data end with an underscore (`coef_`, `classes_`, `mean_`), and they exist only after `fit`. `check_is_fitted(model)` tests for them and raises `NotFittedError` if there are none.

Problem 19.P.2 (Easy) `model.fit(df["age"], y)` raises `Expected 2D array, got 1D array instead`. Fix it, and say why scikit-learn insists on 2-D.

Answer: use double brackets: `model.fit(df[["age"]], y)`, which keeps a one-column DataFrame (with a NumPy array, `X.reshape(-1, 1)`). Scikit-learn treats X as rows by features, and a flat vector is ambiguous: it could be one sample with many features or many samples with one feature. Requiring 2-D removes the ambiguity.

Problem 19.P.3 (Easy) A spam filter flags 40 emails: 30 are spam and 10 are not. There were 50 spam emails in total. Compute precision, recall and F1.

Answer: TP = 30, FP = 10, FN = 50 - 30 = 20. Precision = 30 / 40 = 0.75. Recall = 30 / 50 = 0.60. F1 = 2 × 0.75 × 0.60 / (0.75 + 0.60) = 0.90 / 1.35 = 0.667. The filter is trustworthy when it fires but misses 40% of the spam.

Problem 19.P.4 (Easy) Which of these need feature scaling to work well: decision tree, random forest, kNN, RBF SVM, logistic regression with the default penalty, k-means, PCA?

Answer: needing scaling: kNN, RBF SVM, regularised logistic regression, k-means and PCA, because they depend on distances, variances or a penalty on weights. Not needing scaling: decision tree and random forest, which only compare each feature to thresholds, so any monotonic rescaling gives the same splits.

Problem 19.P.5 (Easy) What does this print?

```
from sklearn.preprocessing import OneHotEncoder

enc = OneHotEncoder(handle_unknown="ignore", sparse_output=False).fit([["a"], ["b"]])
print(enc.transform([["a"], ["c"]]))
```

Answer: the encoder learned two categories, `a` and `b`, so it outputs two columns. The known value `a` becomes `[1, 0]`. The unseen value `c` becomes all zeros because of `handle_unknown="ignore"` (the default would raise an error).

```
[[1. 0.]
 [0. 0.]]
```

Problem 19.P.6 (Medium) What does this print, and why is the last number not between -2 and 2?

```
import numpy as np
from sklearn.preprocessing import StandardScaler

sc = StandardScaler().fit(np.array([[1.0], [2.0], [3.0]]))
print(sc.mean_, sc.scale_.round(4))
print(sc.transform(np.array([[3.0], [4.0], [10.0]])).ravel().round(3))
```

Answer: the training data has mean 2 and population standard deviation $\sqrt{(2/3)} = 0.8165$. New values are scaled with those numbers: (3 - 2)/0.8165 = 1.225, (4 - 2)/0.8165 = 2.449, (10 - 2)/0.8165 = 9.798. Scaling does not clip, so values outside the training range land far from zero. That is useful information about drift, not an error.

```
[2.] [0.8165]
[1.225 2.449 9.798]
```

Problem 19.P.7 (Medium) Find the bug and fix it.

```
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score

X_scaled = StandardScaler().fit_transform(X)
print(cross_val_score(LogisticRegression(), X_scaled, y, cv=5).mean())
```

Answer: the scaler is fitted on all rows, so every validation fold has influenced the mean and spread used on the training folds. For a scaler the effect is usually small; for a selector, imputer or encoder it can be large. Fix: `pipe = make_pipeline(StandardScaler(), LogisticRegression()); cross_val_score(pipe, X, y, cv=5)`, which refits the scaler inside each fold.

Problem 19.P.8 (Medium) Find the bug: `GridSearchCV(make_pipeline(StandardScaler(), SVC()), {"C": [0.1, 1, 10]}, cv=5).fit(X, y)` raises `ValueError: Invalid parameter 'C'`.

Answer: the grid names a parameter of the pipeline, which has none called `C`. Parameters of a step are addressed as `stepname__param`, and `make_pipeline` names steps by lowercase class name, so the key is `svc__C`. Use `{"svc__C": [0.1, 1, 10]}`. If unsure, print `pipe.get_params().keys()`.

Problem 19.P.9 (Medium) A grid has 3 values for one setting and 2 for another. You use 4-fold CV with the default `refit=True`. How many model fits happen?

Answer: 3 × 2 = 6 candidates; 6 × 4 = 24 fits during the search; plus 1 final refit of the best candidate on all training rows: 25 fits.

Problem 19.P.10 (Medium) What does this print? Say which cell is which.

```
from sklearn.metrics import confusion_matrix

print(confusion_matrix([0, 1, 1, 1], [0, 0, 1, 1]))
```

Answer: rows are true classes, columns are predicted classes. Row 0 (true 0): one predicted 0, none predicted 1. Row 1 (true 1): one predicted 0 (a false negative), two predicted 1 (true positives). So tn = 1, fp = 0, fn = 1, tp = 2.

```
[[1 0]
 [1 2]]
```

Problem 19.P.11 (Medium) A tuned model has CV train accuracy 0.99 and CV validation accuracy 0.71. Diagnose it and give three fixes.

Answer: large gap with high train score is overfitting (high variance). Fixes: (1) simplify the model: limit tree depth, raise `min_samples_leaf`, increase the regularisation penalty (smaller `C`, larger `alpha`); (2) average more (a forest instead of one tree, bagging); (3) get more rows or remove noisy features, and check that the CV scheme is not leaking (a leak would inflate validation, so the real situation is not better than 0.71). Confirm with a validation curve or learning curve.

Problem 19.P.12 (Medium) Fraud occurs in 0.2% of transactions. Pick a primary metric and explain how you would set the decision rule.

Answer: accuracy is useless (99.8% by never flagging). Use average precision (area under the precision-recall curve) to compare models, and report recall and precision at the operating point. Choose the threshold from costs: if reviewers can check 500 alerts a day, set the threshold to give about that many alerts and read off the recall; or maximise expected value using the cost of a missed fraud versus a false alert. Use `stratify`, `class_weight` if helpful, and check calibration if probabilities feed other decisions.

Problem 19.P.13 (Medium) What does this print?

```
import numpy as np
from sklearn.linear_model import LogisticRegression

a = LogisticRegression()
b = a
b.set_params(C=5)
print(a.C)

from sklearn.base import clone
c = clone(a)
c.set_params(C=9)
print(a.C, c.C)
```

Answer: `b = a` does not copy; both names refer to one object, so changing `b` changes `a` and the first line prints 5. `clone(a)` builds a new unfitted object with the same settings (including C=5); changing `c` leaves `a` alone, so the second line prints `5 9`.

```
5
5 9
```

Problem 19.P.14 (Medium) Write a pipeline for a table with numeric columns `age` and `income` (some missing) and a text column `city`, ending in logistic regression, and report 3-fold CV accuracy on the small table below.

Answer: impute and scale the numbers, one-hot encode the text, route each with a `ColumnTransformer`, and put the model last. Running it prints the three fold accuracies (1.0, 0.75 and 1.0). The table has only 12 rows, so these numbers are only a demonstration.

```
import numpy as np
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

df = pd.DataFrame({
    "age": [25, 32, np.nan, 41, 29, 55, 38, np.nan, 47, 23, 36, 60],
    "income": [30, 45, 52, np.nan, 38, 80, 61, 58, 72, 28, 49, 90],
    "city": ["A", "B", "A", "C", "B", "C", "A", "B", "C", "A", "B", "C"],
})
y = [0, 0, 0, 1, 0, 1, 1, 0, 1, 0, 0, 1]

prep = ColumnTransformer([
    ("num", make_pipeline(SimpleImputer(strategy="median"), StandardScaler()), ["age", "income"]),
    ("cat", OneHotEncoder(handle_unknown="ignore"), ["city"])])
pipe = make_pipeline(prep, LogisticRegression())
print(cross_val_score(pipe, df, y, cv=3).round(3))
```

```
[1.   0.75 1.  ]
```

Problem 19.P.15 (Hard) Find the bug: `cross_val_score(model, X, y, cv=GroupKFold(5))` raises `ValueError: The 'groups' parameter should not be None`. Then explain when you need it.

Answer: `GroupKFold` needs to know which rows belong together, so pass them: `cross_val_score(model, X, y, groups=patient_ids, cv=GroupKFold(5))`. Needed whenever several rows come from the same entity (patient, customer, device, document) and new data will bring new entities; otherwise random folds put near-copies of one entity in both train and validation, inflating the score (section 19.5: 0.992 versus 0.515).

Problem 19.P.16 (Hard) What does this print?

```
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

X = pd.DataFrame({"n": [1.0, 2.0, 3.0, 4.0], "c": ["x", "y", "z", "x"]})
y = [0, 1, 0, 1]
pipe = make_pipeline(
    ColumnTransformer([("s", StandardScaler(), ["n"]), ("o", OneHotEncoder(), ["c"])]),
    LogisticRegression()).fit(X, y)
print(pipe[:-1].transform(X).shape, pipe[-1].coef_.shape)
print(pipe.steps[0][0])
```

Answer: the transformed matrix has 4 rows and 1 + 3 columns (one scaled number plus a one-hot column for each of x, y, z), so the shape is (4, 4); the model has one weight per transformed column, shape (1, 4). `make_pipeline` names the first step after the lowercase class name, `columntransformer`. Slicing `pipe[:-1]` is the pipeline without its final model.

```
(4, 4) (1, 4)
columntransformer
```

Problem 19.P.17 (Hard) A model says "90% likely to default" for a group of customers, but only 60% of that group defaulted. What is wrong, how do you confirm it, and what do you do?

Answer: the probabilities are overconfident, meaning they are not calibrated. Confirm with `calibration_curve(y_true, p, n_bins=10)` on held-out data (points far below the diagonal at high predicted values) and with Brier score and log loss. Fix with `CalibratedClassifierCV(model,`

`method="isotonic", cv=5)` (or `"sigmoid"` on small data), then re-check the curve on data the calibrator did not see. The ranking (AUC) is not changed much by calibration; the meaning of the numbers is.

Problem 19.P.18 (Hard) Compute by hand the ROC AUC for labels [0, 0, 1, 1, 1] and scores [0.1, 0.6, 0.4, 0.8, 0.7], then verify with code.

```
from sklearn.metrics import roc_auc_score

print(roc_auc_score([0, 0, 1, 1, 1], [0.1, 0.6, 0.4, 0.8, 0.7]))
```

Answer: AUC is the share of (positive, negative) pairs where the positive scores higher. Positives: 0.4, 0.8, 0.7. Negatives: 0.1, 0.6. That is 3 × 2 = 6 pairs. 0.4 beats 0.1 but not 0.6 (1 of 2). 0.8 beats both (2 of 2). 0.7 beats both (2 of 2). Total 5 of 6 = 0.8333.

```
0.8333333333333333
```

Problem 19.P.19 (Hard) Write a custom transformer `TopK` that keeps the `k` most frequent categories of a single column seen in training and maps everything else to `"other"`. Show it on a small column, including an unseen value.

Answer: learn the vocabulary in `fit` (state ends in an underscore) and apply it in `transform`. The constructor stores `k` unchanged. Output: the two most frequent training cities are Rome (3 rows) and Oslo (2), so Paris and the unseen Berlin both become `other`.

```
import numpy as np
import pandas as pd
from collections import Counter
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.utils.validation import check_is_fitted


class TopK(TransformerMixin, BaseEstimator):
    def __init__(self, k=2):
        self.k = k

    def fit(self, X, y=None):
        values = np.asarray(X).ravel()
        self.keep_ = [v for v, _ in Counter(values).most_common(self.k)]
        return self

    def transform(self, X):
        check_is_fitted(self)
        values = np.asarray(X).ravel()
        return np.array([[v if v in self.keep_ else "other"] for v in values], dtype=object)


train = pd.DataFrame({"city": ["Oslo", "Oslo", "Rome", "Rome", "Rome", "Paris", "Lima"]})
t = TopK(k=2).fit(train[["city"]])
print(t.keep_)
print(t.transform(pd.DataFrame({"city": ["Rome", "Paris", "Berlin"]})).ravel())
```

```
['Rome', 'Oslo']
['Rome' 'other' 'other']
```

Problem 19.P.20 (Hard) Find the leak: a colleague adds the average churn rate per city to the table with `df["city_rate"] = df.groupby("city")["churn"].transform("mean")` before splitting. Why is the CV score inflated, and what is the correct tool?

Answer: the new column is computed from the target of every row, including each row's own label and every validation row's label. For a rare city with two customers, the city rate is almost the row's own label. The model reads the answer from a feature that will not be available at prediction time for new rows. Correct tool: `TargetEncoder` inside the pipeline. It computes smoothed means from training data only, and during `fit_transform` uses internal cross-fitting so a row's encoding never uses its own label. Then the whole thing is evaluated by outer cross-validation.

Problem 19.P.21 (Expert) What does this print, and what does it teach about permutation importance?

```
import numpy as np
from sklearn.ensemble import RandomForestClassifier
from sklearn.inspection import permutation_importance
from sklearn.model_selection import train_test_split

rng = np.random.default_rng(0)
x = rng.normal(size=(1500, 1))
y = (x[:, 0] + rng.normal(scale=0.5, size=1500) > 0).astype(int)
X_single = np.c_[x, rng.normal(size=1500)]                    # [signal, noise]
X_dup = np.c_[x, x + rng.normal(scale=0.01, size=(1500, 1)), rng.normal(size=1500)]  # signal twice

for name, X in (("one copy", X_single), ("two copies", X_dup)):
    X_tr, X_te, y_tr, y_te = train_test_split(X, y, random_state=0)
    m = RandomForestClassifier(200, random_state=0).fit(X_tr, y_tr)
    pi = permutation_importance(m, X_te, y_te, n_repeats=10, random_state=0)
    print(name, pi.importances_mean.round(3))
```

Answer: with one copy the signal column has a large importance and the noise column about zero. With two near-identical copies, the forest splits its reliance between them (0.204 and 0.079), so shuffling one copy hurts far less because the other still carries the signal, and the first copy's importance falls from 0.345 to 0.204 although the information is exactly as valuable. Lesson: correlated features hide each other. Cluster correlated columns and permute them as a group, or drop duplicates before interpreting.

```
one copy [0.345 0.017]
two copies [ 0.204  0.079 -0.002]
```

Problem 19.P.22 (Expert, interview) Design the validation for a model that predicts, at the start of each month, whether a customer will cancel during that month. You have monthly snapshots for three years.

Answer: time and entity both matter. (1) Features for a snapshot may only use information available on that date; labels are the cancellations in the following month. (2) Split by time, not randomly: train on months up to T, validate on a later block, test on the latest months, using `TimeSeriesSplit` for CV (or an expanding-window scheme with a gap of one month so label windows do not overlap). (3) The same customer appears in many months; within the training period it is fine, but never let a customer's later months leak into a fold that is validated on earlier months. (4) Use average precision and recall at the number of customers the retention team can contact. (5) Monitor drift after launch and re-train on a schedule. Everything learned (encoders, imputers) goes in the pipeline, fitted per fold.

Problem 19.P.23 (Expert) Without running it, say what this prints, then explain what would change if `refit=False`.

```
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GridSearchCV

X, y = make_classification(300, 8, random_state=0)
gs = GridSearchCV(LogisticRegression(max_iter=500), {"C": [0.01, 1.0]}, cv=3, refit=True).fit(X, y)
print([int(r) for r in sorted(gs.cv_results_["rank_test_score"])], hasattr(gs, "best_estimator_"), gs.n_features_in_)
```

Answer: there are two candidates, so the ranks are [1, 2] (ties would repeat a rank). `refit=True` creates `best_estimator_`, so `hasattr` is True, and the search exposes `n_features_in_` (8). With `refit=False` there is no `best_estimator_`; `gs.predict` and `gs.score` would raise an error and you would have to refit the best parameters yourself. You keep only `best_params_` and `cv_results_`. A refit is also required when you use a list of metrics as `scoring` (you must name the metric to refit on).

```
[1, 2] True 8
```

Prompt pack

AI prompt: copy and paste into any AI

You are my patient, practical machine-learning professor. I am a working professional moving into data roles and I just studied scikit-learn: the estimator API (fit, predict, transform, score), train_test_split, preprocessing with ColumnTransformer, Pipeline and why it prevents leakage, linear/tree/forest/boosting/SVM/kNN/naive Bayes models, cross-validation with GridSearchCV and RandomizedSearchCV, metrics for imbalanced classes, calibration, permutation importance and joblib. Quiz me one question at a time, starting easy and getting harder: mix "what does this print", "find the bug", and "which metric or splitter should I use and why". Wait for my answer before revealing yours. After each answer tell me what I got right, what I missed, and give a one-line rule I can remember. After 10 questions, summarise my weak areas and suggest three exercises.

AI prompt: copy and paste into any AI

Act as a senior data scientist doing a code review of my scikit-learn project. I will paste my code and a short description of the data and goal. Check, in order: (1) data leakage (any fitting of scalers, imputers, encoders, selectors or target statistics outside a Pipeline or before the split; features that would not be available at prediction time; repeated entities or time order handled with random folds); (2) the splitting scheme; (3) whether the metric matches the cost of errors, and whether a dummy baseline is reported; (4) hyper-parameter tuning hygiene (was the test set touched? is there a CV-optimism problem?); (5) calibration if probabilities are used as numbers; (6) reproducibility (random_state, saved versions); (7) deployment risks (column order, unseen categories, missing values). For each problem quote the exact line, explain the symptom it would cause, and show the corrected code. End with a ranked list of the three changes that matter most.

AI prompt: copy and paste into any AI

Run a mock technical interview for a data scientist role focused on scikit-learn and classical machine learning. Ask me one question at a time, from basic to expert, covering: Pipeline versus manual preprocessing and leakage; bias and variance with trees, forests and boosting; imbalanced classification (metrics, class_weight, thresholds, PR versus ROC curves); cross-validation design for grouped and time-ordered data; hyper-parameter search and nested CV; calibration; model interpretation (permutation importance versus impurity importance, partial dependence); writing a custom transformer; saving and serving a model safely. After each of my answers, score it from 1 to 5, point out gaps, and give a model answer in under 120 words. Occasionally give me a short code snippet containing a subtle bug and ask me to find it. At the end give an overall assessment and a study plan for the weakest topics.

CHAPTER 20

GETTING DATA IN AND OUT: FILES, DATABASES, APIS AND THE WEB

Library Handbook: sqlite3, requests, httpx, BeautifulSoup, validation and incremental loads.

Priya has a Monday deadline. Finance sent a sales export as a CSV that opens with strange characters where the accented names should be. Operations keeps order history in a small SQLite file. The partner's shipping data sits behind a web API that returns 50 rows per page and sometimes answers "too many requests". The competitor's price list exists only as an HTML table on a web page. And her manager wants all four in one table, checked for bad values, loaded again every night without duplicates.

None of this is modelling. It is plumbing, and plumbing decides whether a data project works at all. A model trained on half-read files is worse than no model. In this chapter you learn the libraries that move data in and out of Python: `pathlib`, `csv`, `json`, PyYAML, `openpyxl`, `sqlite3`, SQLAlchemy, `requests`, `httpx`, BeautifulSoup, validation with dataclasses and pydantic, Parquet and Arrow, plus `logging`, progress bars and chunked reading. By the end you can open any common source, read it safely, check it, and write it somewhere else, and you have a playbook for when something goes wrong.

How this chapter is labelled. Every code block marked as a normal Python block was run for this document with Python 3.14, pandas 3.0, NumPy 2.x, requests, httpx, BeautifulSoup 4, openpyxl and PyYAML, and the output under it is the real output. SQLAlchemy, pydantic, pyarrow and tqdm were **not installed** on the study machine, so their code appears in plain text blocks that start with `# NOT RUN HERE`, and wherever possible a runnable equivalent is given next to it. Web examples never touch the internet: each one starts its own tiny server on `127.0.0.1` inside the script, so everything works offline. Timings in this chapter were measured once and vary between machines.

20.1 The data-access toolbox: what, when, and the mental model

INTUITION

Every data source is a different kind of door into the same house. A file is a door you open yourself, a database is a door with a receptionist (SQL), a web API is a door on another building, and a web page is a window you can only look through. Your job is to pick the right door, carry things out in manageable boxes, and check the contents before they enter the house (your DataFrame).

What each library is for, and when not to use it

Library	Use it when	Do NOT use it when
`pathlib`, `glob`	Building paths, finding many files, safe file operations	You need remote storage such as S3 (use that provider's SDK or `fsspec`)
`csv`	Small to medium text tables, streaming row by row, odd dialects	You need fast analysis (use `pandas.read_csv`)
`json`, PyYAML	Nested data, configs, API payloads	Huge tabular data (JSON repeats every key on every row)
`openpyxl`	Reading and writing `.xlsx`, formatting, formulas	Old `.xls` files, or millions of rows
`sqlite3`	Local single-file database, tests, small ETL state	Many concurrent writers or a shared server database
SQLAlchemy	One code base for several databases, ORM models, safe query building	A five-line script on SQLite (plain `sqlite3` is enough)
`requests`	Simple synchronous HTTP calls	Hundreds of parallel calls (use `httpx.AsyncClient`)
`httpx`	Same style as requests plus async, HTTP/2 option, strict timeouts	You need a browser to run JavaScript

Library	Use it when	Do NOT use it when
BeautifulSoup	Pulling facts out of static HTML	The site offers an API or a download (use that), or the page is built by JavaScript
pydantic / dataclasses	Checking that incoming records have the right shape	Checking statistical quality of a whole table (use DataFrame checks)
Parquet / Arrow	Large analytic tables, fast column reads, typed storage	Files people must open in a text editor

Install and version check

Install with `python3 -m pip install pandas requests httpx beautifulsoup4 openpyxl pyyaml`, and add `sqlalchemy pydantic pyarrow tqdm` for the optional parts. Always print versions when something behaves oddly, because behaviour changes between releases (pandas 3.0 and requests 2.x differ from older tutorials).

```
import sys, sqlite3, importlib.util
import pandas as pd, requests, httpx, bs4, openpyxl, yaml

print("python", sys.version.split()[0])
print("sqlite library", sqlite3.sqlite_version)
for mod in (pd, requests, httpx, bs4, openpyxl, yaml):
    print(mod.__name__, mod.__version__)
for name in ("sqlalchemy", "pydantic", "pyarrow", "tqdm"):
    found = importlib.util.find_spec(name) is not None
    print(name, "installed" if found else "missing")
```

```
python 3.14.3
sqlite library 3.50.4
pandas 3.0.6
requests 2.34.2
httpx 0.28.1
bs4 4.15.0
openpyxl 3.1.5
yaml 6.0.3
sqlalchemy missing
pydantic missing
pyarrow missing
tqdm missing
```

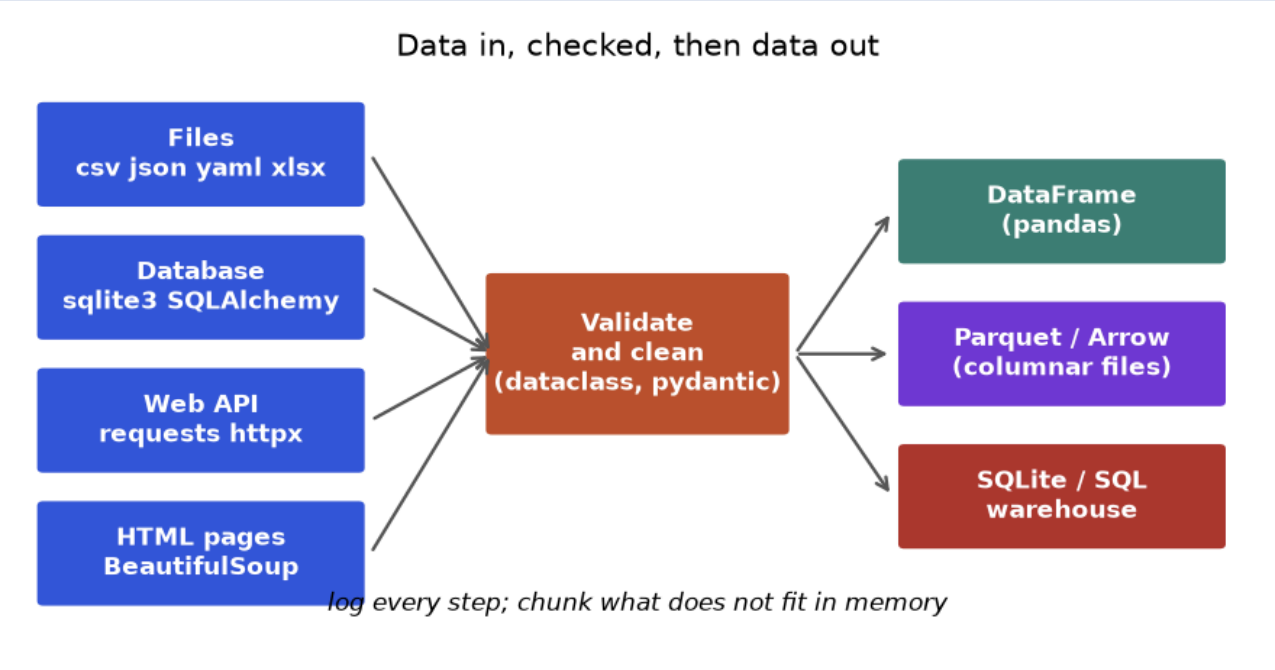
1. Line 1 imports the standard-library helpers: `sys` for the Python version, `sqlite3` for the engine version, `importlib.util` to test for a package without importing it.
2. Line 2 imports the five third-party libraries under their standard names (`pd` is the universal alias for pandas).
3. Lines 4 and 5 print the Python version (only the number, before the first space) and the version of the SQLite C library bundled with Python.
4. The `for` loop prints each module's own name and `__version__`. Every well-behaved package has that attribute.
5. `find_spec` returns `None` when a package cannot be imported, so the last loop reports which optional packages are missing. Your output will differ; the point is to know what you have.

Import conventions

- `import pandas as pd`, `import numpy as np`, `import requests`, `import httpx`, `import sqlite3`, `import yaml` (the PyPI name is `pyyaml`, the import name is `yaml`), `from bs4 import BeautifulSoup` (PyPI `beautifulsoup4`), `from pathlib import Path`, `import openpyxl` or `from openpyxl import load_workbook`.
- Create one logger per module with `logger = logging.getLogger(__name__)`, never `print` in library code.

The mental model

Every data-access script, from ten lines to ten thousand, has the same four stages: **extract** (open the door), **validate** (check the shape), **transform** (clean and combine) and **load** (write somewhere durable). Libraries differ, the stages do not. Most bugs are stage confusion: loading before validating, or re-extracting everything when only new rows were needed.



Sources on the left, one validation gate in the middle, destinations on the right.

How to read it: follow the arrows left to right. All four blue sources pass through the single orange gate. That is deliberate: put your checks in one place, so bad data from any door is stopped the same way. The note under the diagram reminds you that logging and chunking apply to every arrow.

KEY IDEA	Prefer the structured door. If a source offers a database, an API or a file download, use that before scraping HTML. Structured sources have types, versions and documented limits; HTML has layout that changes without notice.

20.2 Paths and files: pathlib and glob

Before any parser can run, you must find the file. `pathlib.Path` treats a path as an object with methods instead of a string you glue with `+`. It works the same on Windows (backslashes) and macOS or Linux (slashes), which is why it replaced `os.path` in modern code.

Reference: the most useful pathlib, glob and file helpers

Name	What it does	Minimal example
`Path(...)`	Make a path object	`Path("data") / "a.csv"`
`/` operator	Join parts with the right separator	`root / "raw" / "x.csv"`
`.name`, `.stem`, `.suffix`	File name, name without extension, extension with dot	`Path("a/b.csv").stem` gives `b`
`.parent`, `.parents`	Folder containing it, all ancestors	`p.parent.parent`
`.parts`	Tuple of path pieces	`Path("a/b").parts`
`.with_suffix(s)`, `.with_name(n)`	Same path, new extension or file name	`p.with_suffix(".parquet")`

Name	What it does	Minimal example
`.exists()`, `.is_file()`, `.is_dir()`	Check what is there	`if not p.is_file(): ...`
`.mkdir(parents=True, exist_ok=True)`	Create a folder and any missing parents, no error if present	`out.mkdir(parents=True, exist_ok=True)`
`.iterdir()`	Items directly inside a folder (any order)	`sorted(d.iterdir())`
`.glob(pattern)`	Items matching a pattern in one folder	`d.glob("*.csv")`
`.rglob(pattern)`	Same, searching all sub-folders	`d.rglob("*.json")`
`.read_text(encoding=...)`	Whole file as one string	`p.read_text(encoding="utf-8")`
`.write_text(s, encoding=...)`	Write a string, replacing the file	`p.write_text("hi", encoding="utf-8")`
`.open(mode, encoding=...)`	File handle for streaming	`with p.open() as f: ...`
`.stat()`	Size, modified time	`p.stat().st_size`
`.replace(target)`, `.rename(target)`	Move a file; `replace` overwrites atomically	`tmp.replace(final)`
`.relative_to(base)`	Path seen from another folder	`p.relative_to(root)`
`glob.glob(pattern)`, `glob.iglob`	String-based matching, list or lazy iterator	`glob.glob("data/*_2025.csv")`

Glob patterns: `*` matches any characters except a slash, `?` matches one character, `[12]` matches one of the listed characters, and `**` (with `rglob` or `recursive=True`) crosses folders.

Example 1: take a path apart

```
from pathlib import Path

p = Path("data") / "raw" / "sales_2025.csv"
print(p)
print(p.name, p.stem, p.suffix, p.parent)
print(p.with_suffix(".parquet"))
print(p.parts)
print(p.is_absolute())
```

```
data/raw/sales_2025.csv
sales_2025.csv sales_2025 .csv data/raw
data/raw/sales_2025.parquet
('data', 'raw', 'sales_2025.csv')
False
```

1. The `/` operator joins three pieces into one path, so you never type a separator.
2. `print(p)` shows the path with forward slashes (on Windows you would see backslashes).
3. The three attributes split the file name: `sales_2025.csv`, its stem `sales_2025` and the suffix `.csv`. `.parent` is the folder part.
4. `with_suffix` returns a new path; `p` itself is unchanged because paths are immutable.
5. `.parts` is a tuple, and `is_absolute()` is `False` because the path starts relative to the current folder.

Example 2: search a tree, then write safely

A nightly job that crashes half way through writing a report leaves a broken file that the next job happily reads. The fix is to write to a temporary name in the same folder and then `replace` it in one step.

```
import glob, os, tempfile
from pathlib import Path

root = Path(tempfile.mkdtemp())
for month in ("2025-01", "2025-02"):
    (root / "raw" / month).mkdir(parents=True, exist_ok=True)
    for day in (1, 2):
        (root / "raw" / month / f"orders_{day:02d}.csv").write_text("id,total\n1,9.5\n")
(root / "raw" / "notes.txt").write_text("hello")

found = sorted(p.relative_to(root).as_posix() for p in root.rglob("*.csv"))
print(len(found), found[0])
print([p.name for p in sorted((root / "raw").glob("*"))])
print(sorted(os.path.basename(f) for f in glob.glob(str(root / "raw" / "*" / "orders_01.csv"))))

target = root / "report.txt"
tmp = target.with_suffix(".tmp")
tmp.write_text("final content")
tmp.replace(target)
print(target.read_text(), tmp.exists())
```

```
4 raw/2025-01/orders_01.csv
['2025-01', '2025-02', 'notes.txt']
['orders_01.csv', 'orders_01.csv']
final content False
```

1. `mkdir(parents=True, exist_ok=True)` builds `raw/2025-01` and its parent, and is safe to re-run. The `:02d` pads days so names sort correctly.
2. `rglob("*.csv")` searches every sub-folder and finds 4 files; `sorted` is needed because file systems return any order; `relative_to` drops the temporary prefix.
3. `glob("*")` lists only direct children of `raw` (two folders and a text file). `glob.glob` works on strings, hence `str()`.
4. The write goes to `report.tmp` first; `replace` swaps it in atomically. Readers see the old report or the new one, never half of it.

REAL JOB

Data teams name files by date (`orders_2025-03-01.csv`) and use `glob` to pick up "everything not yet processed". Sort the matches, because lexical order of ISO dates equals time order, and your pipeline becomes deterministic.

20.3 Text formats: csv, json and yaml

These three formats cover most files you will meet. **CSV** is a flat table in text, **JSON** is nested data (dicts and lists), and **YAML** is human-friendly nested data, mostly used for configuration. The golden rule: parse them with a real parser, never with `line.split(",")`, because real data contains commas and quotes inside values.

Reference: csv, json and yaml essentials

Name	What it does	Minimal example
`csv.reader(f)`	Yields each row as a list of strings	`for row in csv.reader(f): ...`
`csv.DictReader(f)`	Yields each row as a dict keyed by the header	`rows = list(csv.DictReader(f))`
`csv.writer(f)`	Writes lists as rows, quoting when needed	`w.writerow(["a", 1])`
`csv.DictWriter(f, fieldnames)`	Writes dicts as rows	`w.writeheader()`

Name	What it does	Minimal example
`open(path, newline="", encoding=...)`	The correct way to open a file for csv	`open("a.csv", newline="", encoding="utf-8")`
`delimiter=` , `quotechar=`	Handle semicolon, tab or pipe files	`csv.reader(f, delimiter=";")`
`quoting=csv.QUOTE_ALL` (and others)	Control when output is quoted	`csv.writer(f, quoting=csv.QUOTE_ALL)`
`csv.Sniffer().sniff(sample)`	Guess the delimiter from a sample	`dialect = csv.Sniffer().sniff(text[:1024])`
`json.loads(s)` , `json.load(f)`	Text or file to Python objects	`json.loads('{"a": 1}')`
`json.dumps(obj)` , `json.dump(obj, f)`	Python objects to text or file	`json.dumps({"a": 1})`
`indent=` , `sort_keys=`	Pretty and stable output	`json.dumps(d, indent=2, sort_keys=True)`
`ensure_ascii=False`	Keep accents readable instead of `é`	`json.dumps("café", ensure_ascii=False)`
`default=`	Function that converts unknown types (dates)	`json.dumps(d, default=str)`
`json.JSONDecodeError`	Raised on bad JSON; has `msg` , `pos`	`except json.JSONDecodeError as e:`
JSON Lines pattern	One JSON object per line; streams well	`[json.loads(l) for l in f]`
`pd.json_normalize(records)`	Flatten nested dicts into columns	`pd.json_normalize(recs, sep="_")`
`yaml.safe_load(text)`	YAML text to Python objects, safely	`cfg = yaml.safe_load(f)`
`yaml.safe_dump(obj, sort_keys=False)`	Python objects to YAML text	`yaml.safe_dump(cfg)`

Example 1: write and read CSV with quotes inside values

```
import csv, io

buf = io.StringIO()
w = csv.DictWriter(buf, fieldnames=["id", "name", "amount"], lineterminator="\n")
w.writeheader()
w.writerow({"id": 1, "name": "Ana, Jr.", "amount": 12.5})
w.writerow({"id": 2, "name": 'Bo "B" Lee', "amount": 7})
text = buf.getvalue()
print(text)

rows = list(csv.DictReader(io.StringIO(text)))
print(rows[0])
print(type(rows[0]["amount"]).__name__)
print(sum(float(r["amount"]) for r in rows))
```

```
id,name,amount
1,"Ana, Jr.",12.5
2,"Bo ""B"" Lee",7

{'id': '1', 'name': 'Ana, Jr.', 'amount': '12.5'}
str
19.5
```

1. `io.StringIO` is an in-memory "file", handy for demos; with a real file you would use `open(path, "w", newline="", encoding="utf-8")` .

2. `DictWriter` needs the column names. `lineterminator="\n"` avoids the default `\r\n`, so the output is the same on every system.
3. The two rows contain a comma and double quotes. The writer wraps the first name in quotes and doubles the inner quotes (`""`), which is the CSV rule for a quote inside a quoted value.
4. `DictReader` reads it back correctly: the comma stayed inside the name.
5. Every value comes back as a **string**, including `amount`. CSV has no types, so you convert with `float()` yourself (or let pandas guess).
6. The last line adds 12.5 and 7.0 to get 19.5.

Example 2: nested JSON Lines, the JSON date trap and a YAML config

```
import json, yaml
from datetime import date
import pandas as pd

print(json.dumps({"when": date(2025, 3, 1), "note": "café"}, default=str, ensure_ascii=False))
try:
    json.loads("{'a': 1}")
except json.JSONDecodeError as e:
    print("bad json:", e.msg, "at char", e.pos)

lines = (
    '{"id": 1, "user": {"name": "Ana", "tier": "gold"}, "total": 30}\n'
    '{"id": 2, "user": {"name": "Bo", "tier": "basic"}, "total": 12}\n'
)
records = [json.loads(line) for line in lines.splitlines()]
df = pd.json_normalize(records)
print(df)

cfg = yaml.safe_load("""
source:
  path: data/orders.jsonl
  encoding: utf-8
columns: [id, total]
dropna: true
""")
print(cfg)
print(df[cfg["columns"]].sum().to_dict())
print(yaml.safe_dump({"a": [1, 2], "b": {"c": None}}, sort_keys=False))
```

```
{"when": "2025-03-01", "note": "café"}
bad json: Expecting property name enclosed in double quotes at char 1
   id  total user.name user.tier
0   1     30       Ana      gold
1   2     12        Bo     basic
{'source': {'path': 'data/orders.jsonl', 'encoding': 'utf-8'}, 'columns': ['id', 'total'], 'dropna': True}
{'id': 3, 'total': 42}
a:
- 1
- 2
b:
  c: null
```

1. JSON cannot hold a `date`, so `dumps` would raise `TypeError`; `default=str` converts unknown objects (reading it back gives a string, so convert dates yourself). `ensure_ascii=False` keeps the accent. The bad-JSON test uses single quotes, which JSON forbids; the exception carries `msg` and `pos`, which is what you log.
2. The string is JSON Lines: one JSON object per line, so a huge file can be processed line by line.
3. Each line is parsed on its own; `json_normalize` flattens the inner `user` dict into columns `user.name` and `user.tier`.
4. `yaml.safe_load` parses the triple-quoted text. Indentation means nesting; `[id, total]` is a list; `true` becomes Python `True`.
5. The config drives the code: the column list picks two columns and sums them.

6. `safe_dump` writes YAML back. `sort_keys=False` keeps your order and `None` becomes `null`.

COMMON MISTAKE

Wrong: `yaml.load(text)`. **Symptom:** in old PyYAML it could construct arbitrary Python objects from a malicious file, which is a code-execution risk; in PyYAML 6 it fails with a missing `Loader` argument. **Fix:** always call `yaml.safe_load` for data you did not write yourself.

20.4 Excel: openpyxl and pandas

Excel files are everywhere in business: budgets, exports, hand-edited lists. An `.xlsx` file is a zip of XML files. `openpyxl` reads and writes it cell by cell, with formatting and formulas. `pandas.read_excel` and `DataFrame.to_excel` use openpyxl underneath for the whole-table case.

Reference: openpyxl and pandas Excel essentials

Name	What it does	Minimal example
`Workbook()`	New empty workbook (one sheet)	`wb = Workbook()`
`load_workbook(path)`	Open an existing file	`wb = load_workbook("a.xlsx")`
`read_only=True`	Stream large files with low memory	`load_workbook(p, read_only=True)`
`data_only=True`	Read cached formula results, not formulas	`load_workbook(p, data_only=True)`
`wb.active`	The currently selected sheet	`ws = wb.active`
`wb.sheetnames`	List of sheet names	`wb.sheetnames`
`wb["Name"]`	Get a sheet by name	`ws = wb["Sales"]`
`wb.create_sheet(title)`	Add a sheet	`wb.create_sheet("Summary")`
`ws["A1"]`	One cell by address	`ws["A1"] = "id"`
`ws.append(list)`	Add a row after the last used row	`ws.append(["Ana", 30])`
`ws.iter_rows(values_only=True)`	Iterate rows as tuples of values	`for r in ws.iter_rows(min_row=2, values_only=True)`
`ws.freeze_panes`	Keep header visible when scrolling	`ws.freeze_panes = "A2"`
`Font`, `PatternFill`	Bold text, background colour	`cell.font = Font(bold=True)`
`cell.number_format`	Display format of a number	`cell.number_format = "0.00"`
`wb.save(path)`	Write the file to disk	`wb.save("out.xlsx")`
`pd.read_excel(path, sheet_name=...)`	Sheet to DataFrame; `None` gives a dict of all sheets	`pd.read_excel(p, sheet_name=None)`
`skiprows=`, `header=`, `usecols=`, `dtype=`	Handle title rows and pick columns and types	`pd.read_excel(p, skiprows=2, usecols="A:C")`
`df.to_excel(writer, sheet_name=, index=False)`	DataFrame to a sheet	`df.to_excel(w, sheet_name="Data", index=False)`
`pd.ExcelWriter(path, engine="openpyxl")`	Write several sheets into one file	`with pd.ExcelWriter("o.xlsx") as w: ...`

Example 1: create, save, reload, and the formula trap

```
import tempfile
from pathlib import Path
from openpyxl import Workbook, load_workbook

path = Path(tempfile.mkdtemp()) / "sales.xlsx"
wb = Workbook()
ws = wb.active
ws.title = "Sales"
ws.append(["region", "units"])
for row in [("north", 120), ("south", 95), ("east", 143)]:
    ws.append(row)
ws["B5"] = "=SUM(B2:B4)"
wb.save(path)

sheet = load_workbook(path)["Sales"]
print(sheet.max_row, sheet.max_column)
for region, units in sheet.iter_rows(min_row=2, max_row=4, values_only=True):
    print(region, units)
print(sheet["B5"].value, load_workbook(path, data_only=True)["Sales"]["B5"].value)
```

```
5 2
north 120
south 95
east 143
=SUM(B2:B4) None
```

1. A new `Workbook` has one sheet; `append` adds a row per call. Nothing exists on disk until `wb.save`.
2. After reloading, `max_row` is 5 (header, three rows and the formula row) and `max_column` is 2. `iter_rows(values_only=True)` yields plain tuples; without it you get cell objects.
3. Cell B5 stores the *text* of a formula. A normal load returns that text. With `data_only=True` you ask for the cached result, but only Excel writes that cache; openpyxl never calculates, so you get `None`.
4. Lesson: formulas written by a script have no values until a spreadsheet application saves the file. Compute in pandas and write values instead.

Example 2: a multi-sheet report with pandas

```
import tempfile
from pathlib import Path
import pandas as pd

path = Path(tempfile.mkdtemp()) / "report.xlsx"
sales = pd.DataFrame({"region": ["north", "south", "east"], "units": [120, 95, 143]})
notes = pd.DataFrame({"note": ["Prepared by Priya"]})
with pd.ExcelWriter(path, engine="openpyxl") as writer:
    sales.to_excel(writer, sheet_name="Sales", index=False)
    notes.to_excel(writer, sheet_name="Notes", index=False)

everything = pd.read_excel(path, sheet_name=None)
print(list(everything))
print(everything["Sales"])
print(pd.read_excel(path, sheet_name="Sales", usecols="B").columns.tolist())
print(pd.read_excel(path, sheet_name="Sales", dtype={"units": "float64"})["units"].dtype)
```

```
['Sales', 'Notes']
  region  units
0  north    120
1  south     95
2   east    143
['units']
float64
```

1. `ExcelWriter` as a context manager saves and closes the file when the `with` block ends.
2. Each `to_excel` call writes one sheet; `index=False` avoids an unwanted extra column holding 0, 1, 2.
3. `sheet_name=None` reads every sheet into a dict of DataFrames, keyed by sheet name.
4. `usecols="B"` uses Excel column letters and returns only the second column.
5. `dtype` forces a type on read, here turning whole numbers into floats. Use it to keep codes as text, for example `dtype={"zip": str}`.

20.5 Databases with sqlite3

SQLite is a complete SQL database stored in one file (or in memory), and Python ships with the driver. It is the best place to learn database access, and a good home for small pipelines, caches and ETL bookkeeping. The same ideas (connection, cursor, parameters, transaction) apply to PostgreSQL, MySQL and SQL Server drivers, because they all follow the Python DB-API.

Reference: sqlite3 essentials

Name	What it does	Minimal example
`sqlite3.connect(path)`	Open (or create) a database file	`con = sqlite3.connect("app.db")`
`":memory:"`	Temporary database that vanishes on close	`sqlite3.connect(":memory:")`
`con.execute(sql, params)`	Run one statement, get a cursor	`con.execute("select 1")`
`con.executemany(sql, rows)`	Run one statement for many parameter sets	`con.executemany("insert into t values (?,?)", rows)`
`cursor.fetchone()`	Next row, or `None`	`cur.fetchone()`
`cursor.fetchall()`	All remaining rows as a list	`cur.fetchall()`
iterate a cursor	Stream rows one by one	`for row in con.execute(sql): ...`
`? placeholder`	Positional parameter binding	`"where id = ?", (7,)`
`:name placeholder`	Named parameter binding	`"where id = :id", {"id": 7}`
`con.commit()`, `con.rollback()`	Save or undo the open transaction	`con.commit()`
`with con:`	Commit on success, roll back on error (does not close)	`with con: con.execute(...)`
`con.close()`	Release the file	`con.close()`
`con.row_factory = sqlite3.Row`	Rows accessible by column name	`row["name"]`
`cursor.lastrowid`, `cursor.rowcount`	Id of last insert, rows changed	`cur.rowcount`

Name	What it does	Minimal example
`insert ... on conflict do update`	Upsert: insert or update by key	`on conflict(id) do update set v = excluded.v`
`sqlite3.IntegrityError`	Constraint failed (duplicate key, NOT NULL)	`except sqlite3.IntegrityError:`
`pragma table_info(t)`	List columns of a table	`con.execute("pragma table_info(t)")`
`pragma journal_mode = wal`	Readers do not block the writer	`con.execute("pragma journal_mode = wal")`
`explain query plan ...`	Show whether an index is used	`con.execute("explain query plan select ...")`

Parameter binding means you send the SQL text and the values separately. The database treats values only as data, never as SQL. It is the single most important habit in this section.

Example 1: create, insert, query

```
import sqlite3

con = sqlite3.connect(":memory:")
con.execute("create table orders(id integer primary key, customer text, total real)")
rows = [(1, "ana", 30.0), (2, "bo", 12.5), (3, "ana", 8.0)]
con.executemany("insert into orders values (?, ?, ?)", rows)
con.commit()

cur = con.execute("select customer, sum(total) from orders group by customer order by 2 desc")
print([d[0] for d in cur.description])
for customer, spent in cur:
    print(customer, spent)

con.row_factory = sqlite3.Row
row = con.execute("select * from orders where id = :id", {"id": 2}).fetchone()
print(row["customer"], row["total"], tuple(row))
con.close()
```

```
['customer', 'sum(total)']
ana 38.0
bo 12.5
bo 12.5 (2, 'bo', 12.5)
```

1. `":memory:"` makes a throw-away database, ideal for demos and tests. `executemany` runs one insert three times, binding each tuple to the three `?` marks; `commit()` makes the rows permanent for other connections.
2. The query groups by customer. `cursor.description` lists column names; iterating the cursor streams rows; `order by 2` means the second selected column.
3. With `row_factory = sqlite3.Row` a row works like a tuple and a dict. The named placeholder `:id` takes a dict.

Example 2: SQL injection, shown and fixed

```
import sqlite3

con = sqlite3.connect(":memory:")
con.execute("create table users(name text, role text)")
con.executemany("insert into users values (?, ?)", [("ana", "admin"), ("bo", "user"), ("cy", "user")])

evil = "x' OR '1'='1"
bad_sql = f"select name from users where name = '{evil}'"
print(bad_sql)
print(con.execute(bad_sql).fetchall())

safe = con.execute("select name from users where name = ?", (evil,)).fetchall()
print(safe)
```

```
select name from users where name = 'x' OR '1'='1'
[('ana',), ('bo',), ('cy',)]
[]
```

1. `evil` imitates what an attacker, or just a surname with an apostrophe, might type into a form.
2. The f-string pastes it into the SQL, giving `where name = 'x' OR '1'='1'`, which is always true: every user is returned. In a real system the same trick can read or delete other tables.
3. With `?` the driver sends the text as a value only. Nobody has that name, so the result is empty.
4. Values are bound with `?`; table and column names cannot be bound, so check them against a fixed allow-list.

Example 3: transactions, rollback and upsert

```
import sqlite3

con = sqlite3.connect(":memory:")
con.execute("create table stock(sku text primary key, qty integer not null)")
con.execute("insert into stock values ('A1', 10)")
con.commit()

try:
    with con:
        con.execute("update stock set qty = qty - 4 where sku = 'A1'")
        con.execute("insert into stock values ('A1', 99)")
except sqlite3.IntegrityError as e:
    print("rolled back:", e)
print(con.execute("select * from stock").fetchall())

sql = """insert into stock(sku, qty) values (?, ?)
         on conflict(sku) do update set qty = qty + excluded.qty"""
con.executemany(sql, [("A1", 5), ("B2", 7)])
con.commit()
print(con.execute("select * from stock order by sku").fetchall())
```

```
rolled back: UNIQUE constraint failed: stock.sku
[('A1', 10)]
[('A1', 15), ('B2', 7)]
```

1. Inside `with con:` the first update succeeds, then the duplicate-key insert raises `IntegrityError`. The block is rolled back as a whole, so the update is undone too: stock is still 10.
2. The upsert says: insert the row, but if the key exists, add the new quantity to the old one (`excluded` is the rejected row). A1 becomes 15 and B2 is created with 7.

KEY IDEA	A transaction is a promise: either every change in the block happens or none does. Wrap each logical unit of work (one batch load, one order) in `with con:`. Idempotent loads (safe to run twice) use upserts like the one above.

20.6 SQLAlchemy and pandas.read_sql

SQLAlchemy is the standard Python toolkit for relational databases. It was **not installed on the study machine, so its code below is not run here**. It has two layers. **Core** builds SQL from Python objects (tables, `select()`, `insert()`) and executes it. The **ORM** (object-relational mapper) sits on Core and maps tables to classes, so a row becomes an object and a `Session` tracks your changes. The examples use the SQLAlchemy 2.0 style (`select()`, `Mapped`, `mapped_column`).

Connection strings

An engine is created from a URL shaped like `dialect+driver://user:password@host:port/database`. The engine owns a **connection pool**, so create one engine per process, not one per query.

Database	URL example	Driver package
SQLite file (relative)	`sqlite:///data/app.db`	built in
SQLite file (absolute)	`sqlite:////home/me/app.db` (four slashes)	built in
SQLite in memory	`sqlite:///:memory:`	built in
PostgreSQL	`postgresql+psycopg://user:pw@host:5432/sales`	`psycopg`
MySQL	`mysql+pymysql://user:pw@host:3306/sales`	`pymysql`
SQL Server	`mssql+pyodbc://user:pw@dsn_name`	`pyodbc`

Passwords with characters such as `@` or `/` must be percent-encoded, or the URL is parsed wrongly. This runnable check shows the anatomy of a URL using only the standard library:

```
from urllib.parse import urlsplit, quote_plus, unquote

password = "p@ss/word"
url = f"postgresql+psycopg://etl_user:{quote_plus(password)}@db.internal:5432/sales"
print(url)
u = urlsplit(url)
print(u.scheme, u.username, unquote(u.password), u.hostname, u.port, u.path.lstrip("/"))
```

```
postgresql+psycopg://etl_user:p%40ss%2Fword@db.internal:5432/sales
postgresql+psycopg etl_user p@ss/word db.internal 5432 sales
```

1. `quote_plus` turns the unsafe characters into `%40` and `%2F`.
2. The f-string assembles the URL. In real code the password comes from an environment variable, never from the source file.
3. `urlsplit` cuts the URL into parts; the `+` in the scheme is allowed.
4. `unquote` restores the original password, which shows the encoding is reversible. SQLAlchemy's `URL.create(...)` builds the URL from separate fields and does this encoding for you.

Reference: SQLAlchemy 2.0 essentials (not run here)

Name	What it does	Minimal example
`create_engine(url)`	Make the engine and pool	`engine = create_engine("sqlite:///app.db")`

Name	What it does	Minimal example
`engine.connect()`	Borrow a connection (use `with`)	`with engine.connect() as conn:`
`engine.begin()`	Connection plus automatic commit or rollback	`with engine.begin() as conn:`
`text(sql)`	Raw SQL with `:name` parameters	`text("select * from t where id = :id")`
`conn.execute(stmt, params)`	Run a statement	`conn.execute(text(...), {"id": 1})`
`result.mappings().all()`	Rows as dict-like objects	`conn.execute(q).mappings().all()`
`MetaData()`, `Table`, `Column`	Describe tables in Core	`Table("t", md, Column("id", Integer, primary_key=True))`
`metadata.create_all(engine)`	Create missing tables	`md.create_all(engine)`
`select(...)`	Build a SELECT	`select(t.c.name).where(t.c.id > 3)`
`insert(t)`, `update(t)`, `delete(t)`	Build DML statements	`conn.execute(insert(t), [{"name": "a"}])`
`.where()`, `.join()`, `.group_by()`, `.order_by()`	Compose clauses	`select(t).order_by(t.c.id.desc())`
`DeclarativeBase`	Base class of ORM models	`class Base(DeclarativeBase): pass`
`Mapped[...]`, `mapped_column()`	Declare typed columns	`id: Mapped[int] = mapped_column(primary_key=True)`
`Session(engine)`	Unit of work for ORM objects	`with Session(engine) as s:`
`session.add`, `add_all`	Queue new objects	`s.add(Order(...))`
`session.commit()`, `rollback()`	Save or undo	`s.commit()`
`relationship()`	Link two models	`items: Mapped[list["Item"]] = relationship()`
`selectinload`, `joinedload`	Load related rows in one go (avoids N+1)	`select(Order).options(selectinload(Order.items))`

Core and ORM in one sketch (not run here)

```
# NOT RUN HERE: sqlalchemy is not installed on the study machine (pip install sqlalchemy)
from sqlalchemy import create_engine, text, select, func, String
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column, Session

engine = create_engine("sqlite:///:memory:")

# Core with raw SQL and bound parameters
with engine.begin() as conn:                      # commits at the end of the block
    conn.execute(text("create table t(id integer primary key, name text)"))
    conn.execute(text("insert into t(name) values (:n)"), [{"n": "ana"}, {"n": "bo"}])
with engine.connect() as conn:
    rows = conn.execute(text("select * from t where name = :n"), {"n": "ana"}).mappings().all()

# ORM: a class per table, a Session per unit of work
class Base(DeclarativeBase):
    pass

class Order(Base):
    __tablename__ = "orders"
    id: Mapped[int] = mapped_column(primary_key=True)
    customer: Mapped[str] = mapped_column(String(50))
    total: Mapped[float]

Base.metadata.create_all(engine)
with Session(engine) as s:
    s.add_all([Order(customer="ana", total=30.0), Order(customer="bo", total=12.5)])
    s.commit()
    big = s.scalars(select(Order).where(Order.total > 20)).all()
    per_customer = s.execute(
        select(Order.customer, func.sum(Order.total)).group_by(Order.customer)
    ).all()
```

1. One engine is created and reused. With `sqlite:///:memory:` stay in one thread, because each new connection would see its own empty database.
2. `text()` with `:n` binds parameters separately, like `?` in sqlite3. `engine.begin()` commits at the end of the block or rolls back on error; `engine.connect()` does not commit by itself. `.mappings().all()` gives dict-like rows.
3. The class `Order` declares the table; `Mapped[float]` alone is a required column. The `Session` collects objects from `add_all` and writes them on `commit()`; `scalars().all()` unwraps objects into a list.
4. The last query is a group-by written with class attributes; SQLAlchemy parameterises it automatically.

WHY IT MATTERS	The ORM is great for applications that create and update single objects. For analytics, where you read millions of rows into a DataFrame, plain SQL through `text()` or `pandas.read_sql` is shorter and faster. Use the ORM for writes with business rules and Core or SQL for bulk reads.

pandas.read_sql and to_sql (run with sqlite3)

`pd.read_sql_query(sql, con, params=, parse_dates=, chunksize=)` runs a query and returns a DataFrame. It accepts a SQLAlchemy engine/connection, and, for SQLite only, a plain `sqlite3` connection. `df.to_sql(name, con, if_exists=, index=False, chunksize=)` writes a DataFrame. `if_exists` is `"fail"` (default), `"replace"` (drops the table!) or `"append"`.

```
import sqlite3
import pandas as pd

con = sqlite3.connect(":memory:")
df = pd.DataFrame({
    "id": [1, 2, 3, 4],
    "customer": ["ana", "bo", "ana", "cy"],
    "total": [30.0, 12.5, 8.0, 44.0],
    "day": ["2025-03-01", "2025-03-02", "2025-03-02", "2025-03-03"],
})
df.to_sql("orders", con, index=False)
more = pd.DataFrame({"id": [5], "customer": ["bo"], "total": [9.9], "day": ["2025-03-04"]})
more.to_sql("orders", con, if_exists="append", index=False)

q = "select * from orders where total > ? and day >= ?"
out = pd.read_sql_query(q, con, params=(10, "2025-03-02"), parse_dates=["day"])
print(out)
print(out["day"].dtype, out["customer"].dtype)

ids = [1, 3, 5]
marks = ",".join("?" * len(ids))
print(pd.read_sql_query(f"select id, total from orders where id in ({marks})", con, params=ids))

total = 0
for chunk in pd.read_sql_query("select * from orders", con, chunksize=2):
    total += chunk["total"].sum()
print(round(total, 1))
```

```
id customer  total        day
0   2       bo   12.5 2025-03-02
1   4       cy   44.0 2025-03-03
datetime64[us] str
   id  total
0   1   30.0
1   3    8.0
2   5    9.9
104.4
```

1. `to_sql` creates `orders` from the DataFrame (`index=False` keeps the index out); the second call uses `if_exists="append"` to add a fifth row.
2. `params` supplies the two values for the two `?` marks. `parse_dates=["day"]` converts text to datetimes (first printed dtype `datetime64[us]` ; the second, `str` , is the pandas 3 default text type). SQLite has no date type, so say which columns to parse.
3. An `IN` list cannot be one bound value: build one `?` per item (`"?" * 3` is `"???"`) and pass the list. Only the count of marks is formatted into the SQL, never the data.
4. With `chunksize=2` you get an iterator of small DataFrames, so you can aggregate a table larger than memory: the total is 104.4.

20.7 Web APIs: requests and httpx

An API call is one question and one answer over HTTP: you send a method (`GET` to read, `POST` to create), a URL, optional headers and body, and you get back a status code, headers and a body, usually JSON. `requests` is the long-standing simple choice. `httpx` has almost the same API and adds async support and strict default timeouts. The networking basics (methods, status families, REST design) are in Chapter 12; this chapter is about using the clients robustly to collect data.

COMMON MISTAKE

Wrong: `requests.get(url)` with no timeout. **Symptom:** a nightly job hangs forever when the server accepts the connection and then goes silent; `requests` has no default timeout. **Fix:** always pass `timeout=` , for example `timeout=(3.05, 10)` (connect, read). `httpx` defaults to 5 seconds, but set it explicitly anyway.

Reference: requests and httpx essentials

Name	What it does	Minimal example
`requests.get(url)`	Read a resource	`requests.get(u, timeout=5)`
`requests.post(url, json=...)`	Send a JSON body and set the header	`requests.post(u, json={"a": 1}, timeout=5)`
`params={...}`	Build the query string safely	`get(u, params={"page": 2})`
`headers={...}`	Custom headers	`get(u, headers={"Accept": "application/json"})`
`timeout=`	Maximum seconds to connect and to wait for data	`timeout=(3, 10)`
`auth=(user, pw)`	HTTP Basic authentication	`get(u, auth=("me", pw))`
Bearer token header	Token authentication	`headers={"Authorization": f"Bearer {tok}"}`
`r.status_code`, `r.ok`	Result number; `True` if below 400	`if not r.ok: ...`
`r.raise_for_status()`	Raise `HTTPError` for 4xx and 5xx	`r.raise_for_status()`
`r.json()`	Parse the body as JSON	`data = r.json()`
`r.text`, `r.content`	Body as decoded text, as raw bytes	`r.content[:100]`
`r.headers`	Case-insensitive header dict	`r.headers["Retry-After"]`
`stream=True`, `r.iter_content(n)`	Download big bodies in pieces	`for part in r.iter_content(8192): ...`
`requests.Session()`	Reuse connection, headers and cookies	`s = requests.Session(); s.get(u)`
`HTTPAdapter(max_retries=Retry(...))`	Automatic retries with backoff	`s.mount("https://", adapter)`
`requests.Timeout`, `ConnectionError`, `HTTPError`	The exceptions to catch	`except requests.Timeout:`
`httpx.Client(base_url=, timeout=)`	Session-like client with shared base URL	`httpx.Client(base_url="https://api.x.com")`
`httpx.AsyncClient`	Many requests concurrently with `await`	`await ac.get("/x")`
`httpx.Timeout(5, connect=2)`	Separate limits per phase	`Client(timeout=httpx.Timeout(10, connect=3))`
`httpx.HTTPTransport(retries=n)`	Retry failed connections only (not 5xx)	`Client(transport=httpx.HTTPTransport(retries=2))`

The offline test server. All HTTP examples start the same small server on `127.0.0.1` with a free port chosen by the operating system (port `0`). You pass it one function that decides the reply for each path. This is also how you can unit-test your own API code without the internet.

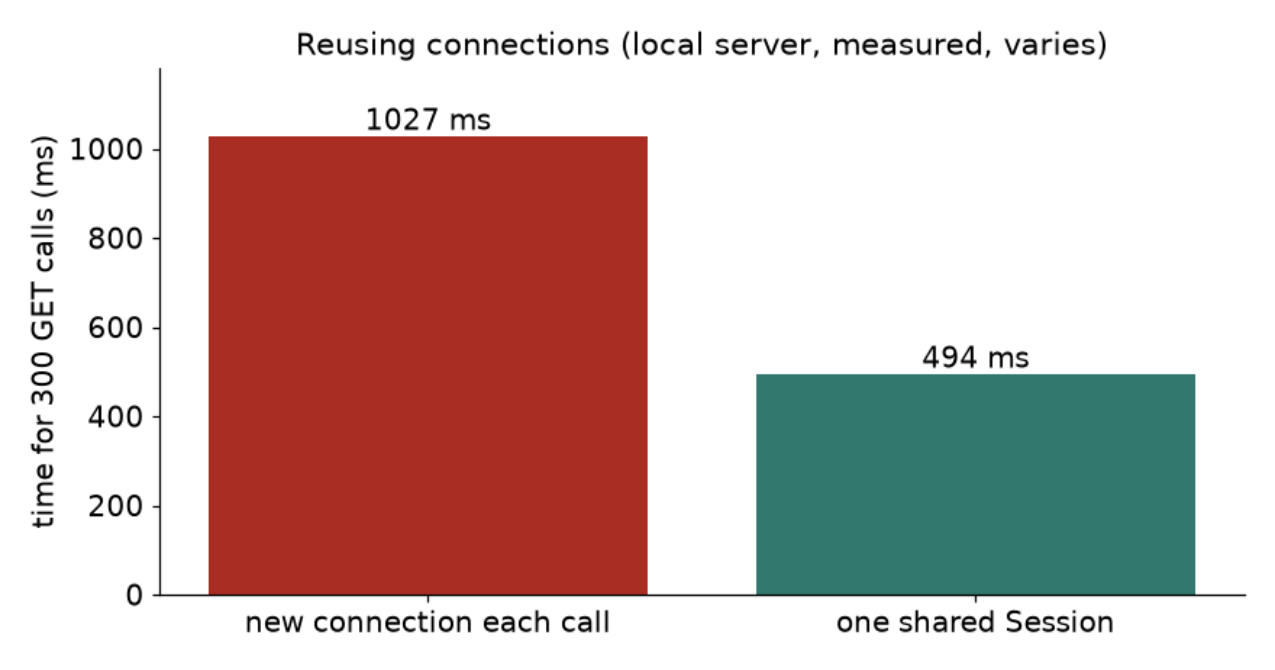
Example 1: one call, parameters, auth, errors, then httpx

```
import asyncio, json, os, threading, time
import httpx, requests
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
from urllib.parse import urlparse, parse_qs

def start(route):
    class H(BaseHTTPRequestHandler):
        def do_GET(self):
            u = urlparse(self.path)
            code, obj, hdrs = route(u.path, parse_qs(u.query), self.headers)
            body = json.dumps(obj).encode()
            self.send_response(code)
            for k, v in {"Content-Type": "application/json", "Content-Length": len(body), **hdrs}.items():
                self.send_header(k, str(v))
            self.end_headers()
            self.wfile.write(body)
        def log_message(self, *args):
            pass
    os.environ["NO_PROXY"] = "127.0.0.1"   # local test traffic must never go through a proxy
    srv = ThreadingHTTPServer(("127.0.0.1", 0), H)
    threading.Thread(target=srv.serve_forever, daemon=True).start()
    return srv, f"http://127.0.0.1:{srv.server_port}"

def route(path, query, headers):
    if path == "/secure" and headers.get("Authorization") != "Bearer s3cret":
        return 401, {"error": "bad token"}, {}
    if path == "/slow":
        time.sleep(0.3)
    return 200, {"path": path, "query": query}, {}

srv, base = start(route)
r = requests.get(f"{base}/items", params={"page": 2, "tag": ["a", "b"]}, timeout=5)
print(r.status_code, r.ok, r.url.split("/items")[1])
print(r.json())

bad = requests.get(f"{base}/secure", timeout=5)
print(bad.status_code, bad.json())
try:
    bad.raise_for_status()
except requests.HTTPError as e:
    print("HTTPError, status", e.response.status_code)

s = requests.Session()
s.headers.update({"Authorization": "Bearer s3cret"})
print(s.get(f"{base}/secure", timeout=5).status_code)

async def many():
    async with httpx.AsyncClient(base_url=base, timeout=5.0) as ac:
        return await asyncio.gather(*(ac.get("/slow", params={"n": i}) for i in range(3)))

t0 = time.perf_counter()
replies = asyncio.run(many())
print([x.json()["query"]["n"][0] for x in replies], "concurrent:", time.perf_counter() - t0 < 0.8)
srv.shutdown()
```

```
200 True ?page=2&tag=a&tag=b
{'path': '/items', 'query': {'page': ['2'], 'tag': ['a', 'b']}}
401 {'error': 'bad token'}
HTTPError, status 401
200
['0', '1', '2'] concurrent: True
```

1. Lines 1 to 22 are the reusable test server: `start(route)` builds a handler whose `do_GET` asks your `route` function for a status code, a JSON-able object and extra headers. It binds port 0 (any free port) in a background thread. Setting `NO_PROXY` makes `requests` and `httpx` skip any environment proxy (common on company networks) for the local address.
2. `route` answers 401 on `/secure` without the right bearer token, sleeps 0.3 s on `/slow`, and otherwise echoes path and query.
3. The first call passes `params` as a dict; a list becomes a repeated key (`tag=a&tag=b`). The server sees every value as a list of strings, as `parse_qs` always reports.
4. `requests` does **not** raise on 4xx by itself: call `raise_for_status()` ; the response stays available as `e.response` .
5. A `Session` stores headers once and keeps the connection open between calls.
6. `httpx` looks almost the same but adds async. `asyncio.gather` sends three 0.3 s calls at once and returns replies in the order given. Done one after another they would need 0.9 s; the printed `True` means they finished in under 0.8 s (exact seconds vary, so I print a yes/no).

The figure shows why sessions matter even on a local machine: opening a new connection for every call costs more time than reusing one.



Measured on one machine against a local server; absolute numbers vary, the gap is what matters.

How to read it: the left bar is 300 calls that each build a fresh connection, the right bar is the same 300 calls through one `Session` . Over a real network the gap is larger, because DNS lookup and the TLS handshake are skipped too.

Example 2: timeouts, retries, backoff, pagination and 429

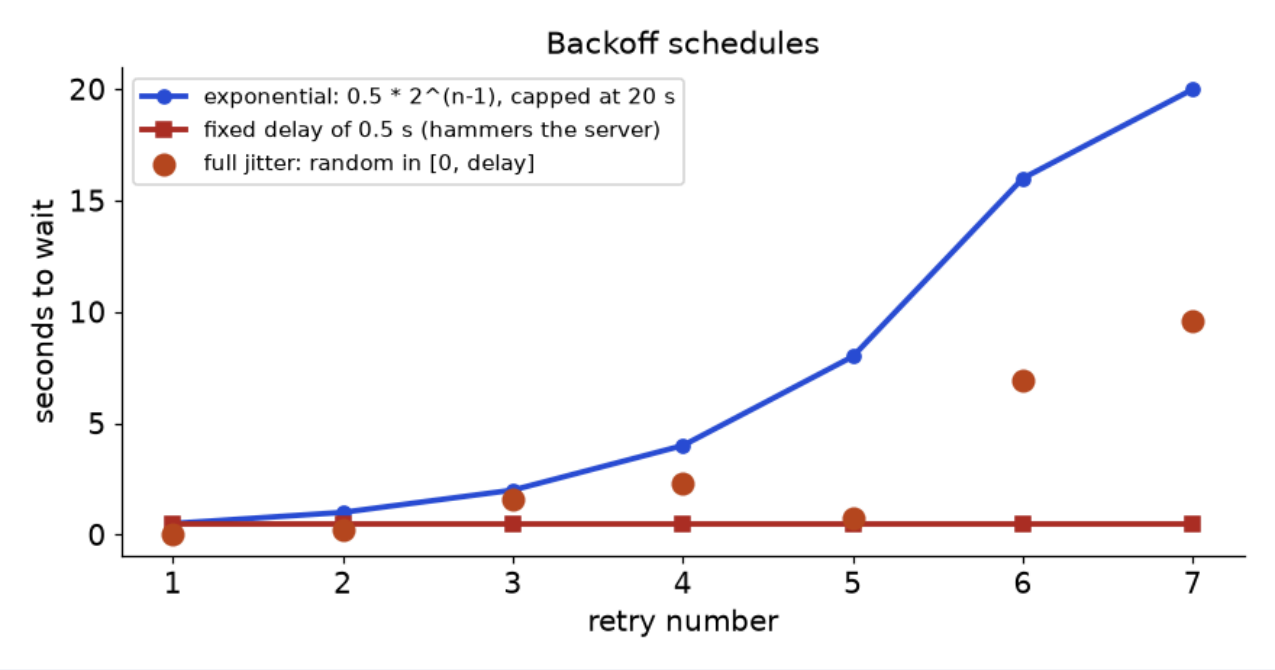
Retry only what can succeed later: timeouts, connection errors, 429 and 5xx. Never retry 400, 401, 403 or 404, because the answer will not change. **Backoff** means waiting longer after each failure (0.05, 0.1, 0.2 seconds ...), and **jitter** means adding randomness so that many clients do not all return at the same instant.

```
import json, os, random, threading, time, requests
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
from urllib.parse import urlparse, parse_qs
from requests.adapters import HTTPAdapter
from urllib3.util.retry import Retry

def start(route):
    class H(BaseHTTPRequestHandler):
        def do_GET(self):
            u = urlparse(self.path)
            code, obj, hdrs = route(u.path, parse_qs(u.query), self.headers)
            body = json.dumps(obj).encode()
            self.send_response(code)
            for k, v in {"Content-Type": "application/json", "Content-Length": len(body), **hdrs}.items():
                self.send_header(k, str(v))
            self.end_headers()
            self.wfile.write(body)
        def log_message(self, *args):
            pass
    os.environ["NO_PROXY"] = "127.0.0.1"
    srv = ThreadingHTTPServer(("127.0.0.1", 0), H)
    threading.Thread(target=srv.serve_forever, daemon=True).start()
    return srv, f"http://127.0.0.1:{srv.server_port}"

ITEMS = [{"id": i, "v": i * i} for i in range(1, 24)]
state = {"flaky": 0, "feed": 0, "limited": 0}
def route(path, query, headers):
    if path == "/flaky":
        state["flaky"] += 1
        return (503, {"error": "busy"}, {}) if state["flaky"] < 3 else (200, {"ok": True}, {})
    if path == "/slow":
        time.sleep(1.0)
    if path == "/feed":
        state["feed"] += 1
        if state["feed"] % 3 == 0:
            state["limited"] += 1
            return 429, {"error": "slow down"}, {"Retry-After": "0"}
        after, limit = int(query["after"][0]), int(query["limit"][0])
        page = [x for x in ITEMS if x["id"] > after][:limit]
        return 200, {"items": page, "next": page[-1]["id"] if len(page) == limit else None}, {}
    return 200, {}, {}

srv, base = start(route)

def get_with_retry(session, url, tries=5, base_delay=0.05, cap=1.0, rng=random.Random(0)):
    delays = []
    for attempt in range(1, tries + 1):
        try:
            r = session.get(url, timeout=2)
            if r.status_code not in (429, 500, 502, 503, 504):
                r.raise_for_status()
                return r, delays
        except (requests.ConnectionError, requests.Timeout):
            pass
        if attempt == tries:
            raise RuntimeError(f"gave up after {tries} tries")
        delay = rng.uniform(0, min(cap, base_delay * 2 ** (attempt - 1)))
        delays.append(round(delay, 3))
        time.sleep(delay)

r, delays = get_with_retry(requests.Session(), f"{base}/flaky")
print(r.json(), "waits:", delays)

state["flaky"] = 0
```

```
s = requests.Session()
s.mount("http://", HTTPAdapter(max_retries=Retry(total=4, backoff_factor=0.05, status_forcelist=[503])))
print(s.get(f"{base}/flaky", timeout=2).json(), "server calls:", state["flaky"])

try:
    requests.get(f"{base}/slow", timeout=(1, 0.3))
except requests.exceptions.ReadTimeout as e:
    print("timeout:", type(e).__name__)

def paged(session, url, limit=10):
    after = 0
    while True:
        r = session.get(url, params={"after": after, "limit": limit}, timeout=5)
        if r.status_code == 429:
            time.sleep(float(r.headers.get("Retry-After", 1)))
            continue
        r.raise_for_status()
        body = r.json()
        yield from body["items"]
        if body["next"] is None:
            break
        after = body["next"]

rows = list(paged(requests.Session(), f"{base}/feed"))
print(len(rows), rows[0], rows[-1], "unique:", len({x["id"] for x in rows}))
print("feed calls:", state["feed"], "rate limited:", state["limited"])
srv.shutdown()
```

```
{'ok': True} waits: [0.042, 0.076]
{'ok': True} server calls: 3
timeout: ReadTimeout
23 {'id': 1, 'v': 1} {'id': 23, 'v': 529} unique: 23
feed calls: 4 rate limited: 1
```

1. The first 22 lines are the same test server as Example 1. The route table has `/flaky` (503 twice, then 200), `/slow` (sleeps one second) and `/feed` (cursor pages of 23 items, with every third call answered by 429 and `Retry-After: 0`).
2. `get_with_retry` retries connection errors, timeouts and the statuses 429, 500, 502, 503, 504. A 4xx such as 404 goes straight to `raise_for_status()`. The delay is `uniform(0, min(cap, base * 2**(attempt-1)))`: exponential growth, a cap and full jitter; the seeded generator makes the printed waits repeatable. Two waits were needed.
3. The second approach mounts a urllib3 `Retry` policy: `total=4` retries, `backoff_factor=0.05` (the first retry is immediate, later waits grow), `status_forcelist=[503]`. The server saw three calls. It also honours `Retry-After`.
4. The slow route needs a second but we allow 0.3 s to *read*, so `ReadTimeout` is raised. A tuple means (connect timeout, read timeout).
5. `paged` is a **generator**: `yield from` hands out items one by one, so the caller can stream them into a database. On 429 it sleeps as long as the server asked and repeats the same request; the cursor did not move, so nothing is skipped.
6. Result: 23 unique items from four feed calls: two full pages, one refused call, and a last short page of 3.



Exponential backoff doubles the wait up to a cap; full jitter picks a random point below the line.

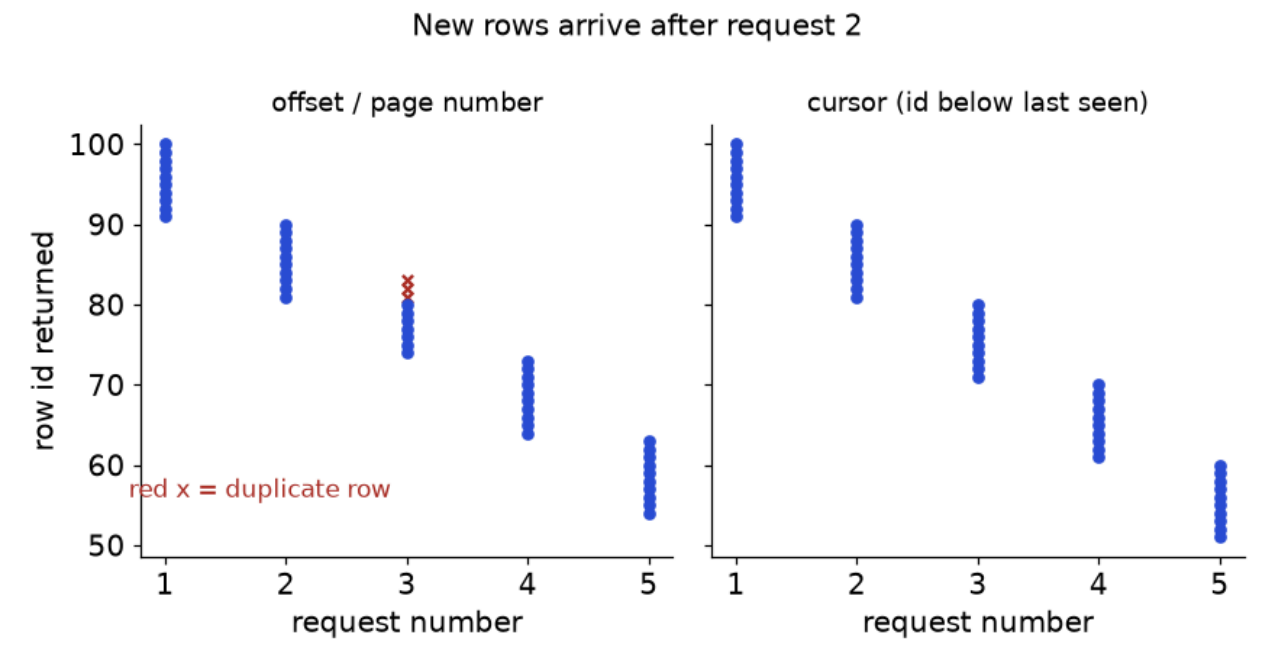
How to read it: the horizontal axis is how many times you have already failed, the vertical axis is how long you wait before the next try. The red line shows a fixed delay, which keeps hitting a struggling server every half second. The blue line doubles each time until it reaches the 20 second cap. The orange dots are jittered waits: each is a random value between zero and the blue line, so a crowd of clients does not move in lockstep.

REAL JOB

Concurrency is permission to hurt a server faster. Cap it (for example with `asyncio.Semaphore(5)`) and respect the provider's rate limit, otherwise your fast script gets your key blocked.

Offset or cursor pagination

APIs return big collections in pages (the `paged` generator above is the pattern). There are two styles. **Offset or page-number** pagination (`?page=3`) is simple, but if rows are added while you read, the pages shift and you see duplicates or miss rows. **Cursor** pagination (`?after=20`) asks for "the next rows after the last one I saw" and stays correct.



Three new rows arrive after request 2. Offset paging repeats rows (red crosses); cursor paging does not.

How to read it: each column of dots is one request and each dot is a row id it returned (newest ids at the top). On the left, request 3 starts three ids too high because the new rows pushed everything down, so it returns rows already seen in request 2 (red crosses). On the right, the cursor says "ids below 81", so request 3 continues exactly where request 2 stopped.

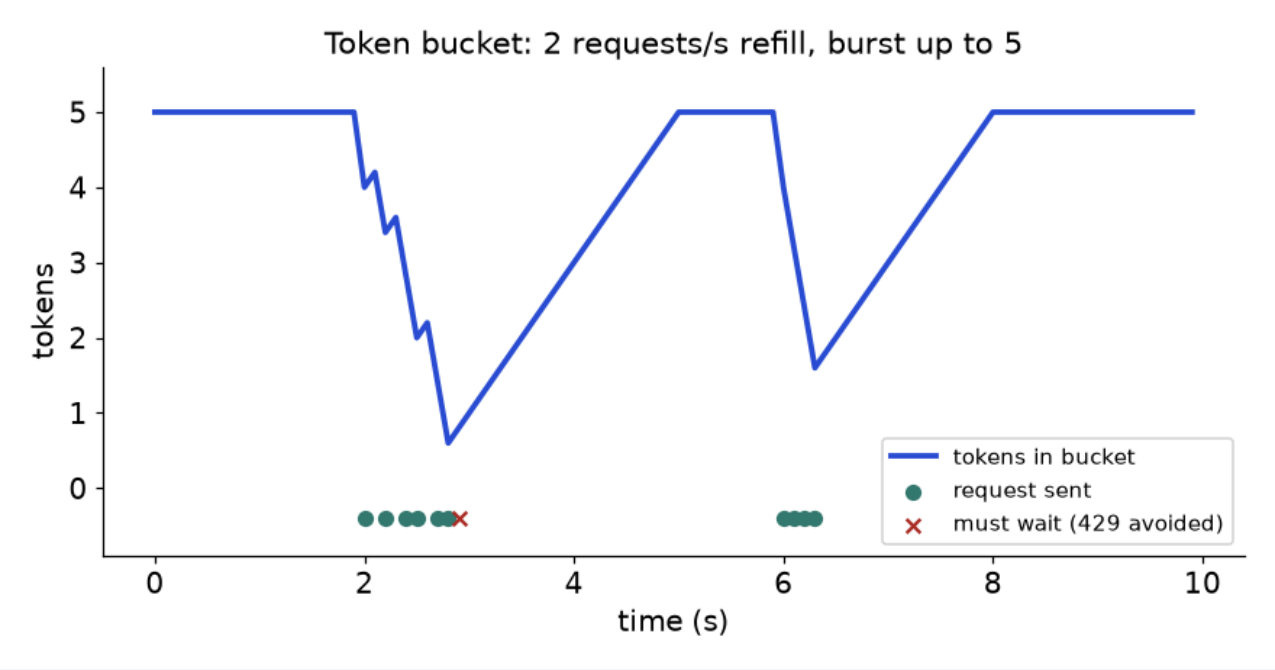
Example 3: staying under a rate limit with a token bucket

A **token bucket** is the classic polite-client tool. The bucket refills at a steady rate, up to a maximum. Each request takes one token. If the bucket is empty you wait. This allows short bursts but a steady average. To keep the example repeatable, the clock is passed in, so we can use a fake clock.

```
class TokenBucket:
    def __init__(self, rate, capacity, clock):
        self.rate, self.capacity, self.clock = rate, capacity, clock
        self.tokens, self.last = capacity, clock()

    def take(self):
        """Return 0.0 if a request may go now, else the seconds to wait."""
        now = self.clock()
        self.tokens = min(self.capacity, self.tokens + (now - self.last) * self.rate)
        self.last = now
        if self.tokens >= 1:
            self.tokens -= 1
            return 0.0
        return round((1 - self.tokens) / self.rate, 3)

now = [0.0]
bucket = TokenBucket(rate=2, capacity=3, clock=lambda: now[0])
print([bucket.take() for _ in range(5)])
now[0] += 1.0
print([bucket.take() for _ in range(3)])
```

```
[0.0, 0.0, 0.0, 0.5, 0.5]
[0.0, 0.0, 0.5]
```

1. The bucket stores the refill rate (2 per second), the capacity (3) and a clock function. It starts full.
2. `take` adds tokens for the time passed (capped), then spends one and returns 0.0, or returns how long to wait for the missing part of a token.
3. At time 0 the first three calls pass (the burst); the fourth and fifth must wait 0.5 s. After one fake second two tokens are back, so two calls pass and the third waits again.
4. In real code pass `time.monotonic` as the clock and `time.sleep(wait)` when the result is above zero.



A simulated bucket (2 tokens per second, capacity 5) absorbing two bursts.

How to read it: the blue line is how many tokens are available. Each green dot is a request that went out and drains the bucket by one. During the burst at second 2 the line falls; the red cross shows a moment when the bucket was nearly empty and a request would have had to wait. Between bursts the line climbs at the refill rate and stops at the capacity of 5. The average rate stays at 2 requests per second, but short bursts are allowed.

Authentication in practice

- **Bearer token**: `Authorization: Bearer <token>` in a header. Store the token in an environment variable (`os.environ["API_TOKEN"]`), never in the script, and never print it.
- **API key**: usually a header such as `X-API-Key` . A key in the URL query string ends up in server logs and browser history, so prefer headers when the API allows.
- **Basic auth**: `auth=("user", "password")` . Only over HTTPS.
- **OAuth 2**: you exchange credentials for a short-lived token and must refresh it when it expires. Use a library built for the provider.
- Proxies: `requests` and `httpx` read `HTTP_PROXY` and `NO_PROXY` from the environment. If a local test call is unexpectedly routed through a corporate proxy, set `NO_PROXY=127.0.0.1` (as the examples do) or pass `trust_env=False` .

20.8 HTML parsing with BeautifulSoup, and scraping ethics

When a page has no API, its facts are inside HTML tags. **BeautifulSoup** (package `beautifulsoup4`) turns HTML text into a tree you can search. It does *not* download pages (that is `requests`) and it does not run JavaScript. Always give it the `"html.parser"` argument (built into Python); `lxml` and `html5lib` are optional faster or more forgiving parsers, and neither was installed on the study machine.

Reference: BeautifulSoup essentials

Name	What it does	Minimal example
`BeautifulSoup(html, "html.parser")`	Parse text into a tree	`soup = BeautifulSoup(page, "html.parser")`
`soup.find(name, ...)`	First matching tag or `None`	`soup.find("h1")`
`soup.find_all(name, ...)`	List of all matching tags (possibly empty)	`soup.find_all("a")`

Name	What it does	Minimal example
`find(class_="x")`	Filter by CSS class (underscore avoids the keyword)	`soup.find_all("tr", class_="row")`
`find(id="x")`, `attrs={...}`	Filter by id or any attribute	`soup.find(attrs={"data-sku": "W-1"})`
`soup.select(css)`	List by CSS selector	`soup.select("table#prices tr.row")`
`soup.select_one(css)`	First match or `None`	`soup.select_one("h1")`
`tag.get_text(strip=True)`	Visible text, trimmed	`td.get_text(strip=True)`
`tag.get_text(" ", strip=True)`	Text pieces joined by a separator	`p.get_text(" ", strip=True)`
`tag["href"]`, `tag.get("href")`	Attribute value (`get` returns `None` if absent)	`a.get("href")`
`tag.parent`, `.children`	Walk up or down the tree	`td.parent.name`
`find_all(string=re.compile(...))`	Search by text content	`soup.find_all(string=re.compile("Price"))`
`tag.decompose()`	Remove a tag from the tree	`script.decompose()`
`urllib.parse.urljoin(base, href)`	Turn relative links into full URLs	`urljoin(page_url, "/about")`
`urllib.robotparser.RobotFileParser`	Read `robots.txt` rules	`rp.can_fetch("bot", url)`
`pd.read_html(source)`	All tables of a page into DataFrames (needs `lxml` or `bs4` with `html5lib`; not run here)	`tables = pd.read_html(html_text)`

Example 1: find things

```
from bs4 import BeautifulSoup
from urllib.parse import urljoin

page = """
<html><head><title>Price list</title></head><body>
<h1 id="top">Widgets</h1>
<p class="intro">Updated <b>today</b>. Prices in USD.</p>
<a href="/about">About</a> <a href="https://other.example/x">Partner</a> <a name="anchor">no link</a>
</body></html>
"""
soup = BeautifulSoup(page, "html.parser")
print(soup.title.string, "|", soup.find("h1").get_text(strip=True), soup.find("h1")["id"])
print(soup.select_one("p.intro").get_text(" ", strip=True))
for a in soup.find_all("a"):
    print(a.get("href"), "->", urljoin("https://shop.example/list/", a.get("href") or ""))
print(soup.find("table"), soup.find_all("table"))
```

```
Price list | Widgets top
Updated today . Prices in USD.
/about -> https://shop.example/about
https://other.example/x -> https://other.example/x
None -> https://shop.example/list/
None []
```

1. The inline string is the whole "web page". `soup.title.string` reads the title; `find("h1")["id"]` reads an attribute; `select_one("p.intro")` uses a CSS selector, and `get_text(" ", strip=True)` joins the text pieces `Updated`, `today` and `. Prices in USD.` with spaces.

2. The third anchor has no `href`: `.get` returns `None` (`["href"]` would raise `KeyError`). `urljoin` resolves `/about` against the page address and leaves a full URL unchanged.
3. There is no table: `find` returns `None` and `find_all` returns `[]`. A method call on that `None` is the classic scraping crash.

Example 2: robots.txt and defensive extraction

```
from urllib.robotparser import RobotFileParser
from bs4 import BeautifulSoup

robots = """
User-agent: *
Disallow: /private/
Crawl-delay: 5

User-agent: BadBot
Disallow: /
"""
rp = RobotFileParser()
rp.parse(robots.splitlines())
for agent, url in [("my-study-bot", "https://shop.example/list"),
                   ("my-study-bot", "https://shop.example/private/data"),
                   ("BadBot", "https://shop.example/list")]:
    print(agent, url.split(".example")[1], rp.can_fetch(agent, url))
print("crawl delay:", rp.crawl_delay("my-study-bot"))

def text_or_none(soup, css):
    tag = soup.select_one(css)
    return tag.get_text(strip=True) if tag else None

soup = BeautifulSoup("<div><span class='price'>9.99</span></div>", "html.parser")
print(text_or_none(soup, "span.price"), text_or_none(soup, "span.discount"))
```

```
my-study-bot /list True
my-study-bot /private/data False
BadBot /list False
crawl delay: 5
9.99 None
```

1. `robots.txt` is a public file where a site owner says which paths automated clients may visit. Here every agent is kept out of `/private/` and asked to wait 5 seconds, and `BadBot` is shut out completely. In real use call `rp.set_url(...)` and `rp.read()`.
2. `can_fetch(agent, url)` answers per user-agent; `crawl_delay` returns 5, which your loop should honour with `time.sleep`.
3. `text_or_none` turns a missing element into `None` instead of a crash. Real pages change; this helper prevents most overnight failures.

REAL JOB

Scraping ethics and law, in practice. (1) Look for an API, a data download or a licence first. (2) Read `robots.txt` and the site's terms of use; robots.txt is a convention, not a security lock, but ignoring it is poor practice and can be a terms breach. (3) Identify yourself with an honest `User-Agent` that includes contact details. (4) Slow down: one request per few seconds, honour `Crawl-delay` and 429, cache pages you already have. (5) Do not collect personal data without a lawful reason, and do not republish copyrighted content. (6) Do not bypass logins, paywalls or captchas. This is general guidance, not legal advice; ask your legal team for commercial projects.

20.9 Validating data: dataclasses and pydantic

Data from outside is guilty until proven innocent. An API can send `"total": "N/A"`, a CSV can have a negative age, a scraped price can be empty. **Validation** means checking each record against a schema (names, types, ranges, allowed values) at the door, so errors are caught with a clear

message instead of ten steps later as a mysterious `NaN`. **pydantic** is the standard library for this. It was **not installed here**, so its code is labelled "not run" and each concept is paired with a runnable dataclass or plain-Python version.

Reference: pydantic v2 essentials (not run here) and the plain-Python equivalent

Name	What it does	Minimal example
`BaseModel`	Class whose fields are checked and converted on creation	`class Order(BaseModel): id: int`
Type hints (`int` , `float` , `date` , `list[int]`)	Declare and coerce types (`"7"` becomes `7`)	`total: float`
`Field(ge=, le=, min_length=, pattern=)`	Numeric, length and regex limits	`age: int = Field(ge=0, le=120)`
`Literal[...]`	Only listed values allowed	`status: Literal["new", "paid"]`
`Model.model_validate(dict)`	Build and check from a dict	`Order.model_validate(rec)`
`Model.model_validate_json(text)`	Build and check from JSON text	`Order.model_validate_json(body)`
`obj.model_dump()`	Back to a plain dict	`row = order.model_dump()`
`ValidationError`	Raised with every problem found	`except ValidationError as e:`
`@field_validator("name")`	Custom check or clean-up for one field	`@classmethod def strip(cls, v): ...`
`@model_validator(mode="after")`	Check relations between fields	`if self.end < self.start: raise ValueError(...)`
`ConfigDict(extra="forbid")`	Reject unknown keys	`model_config = ConfigDict(extra="forbid")`
`ConfigDict(strict=True)`	No coercion: `"7"` is an error for `int`	`model_config = ConfigDict(strict=True)`
`TypeAdapter(list[Order])`	Validate a list or any type without a model class	`TypeAdapter(list[Order]).validate_python(rows)`
`Field(alias="Total Amt")`	Map awkward source names to clean field names	`total: float = Field(alias="Total Amt")`
`dataclasses.dataclass` + `__post_init__`	Runnable equivalent: fields plus your own checks	`def __post_init__(self): ...`
`dataclasses.asdict`	Dataclass to dict	`asdict(order)`

pydantic in practice (not run here)

```
# NOT RUN HERE: pydantic is not installed on the study machine (pip install pydantic)
from datetime import date
from typing import Literal
from pydantic import BaseModel, ConfigDict, Field, TypeAdapter, ValidationError, field_validator

class Order(BaseModel):
    model_config = ConfigDict(extra="forbid")
    id: int
    customer: str = Field(min_length=1)
    total: float = Field(ge=0)
    status: Literal["new", "paid", "shipped"] = "new"
    placed: date

    @field_validator("customer")
    @classmethod
    def clean_name(cls, v: str) -> str:
        return v.strip()

good = Order.model_validate({"id": "7", "customer": " Ana ", "total": "12.5", "placed": "2025-03-01"})
print(good.model_dump())      # id is the int 7, customer "Ana", placed a date object

try:
    Order.model_validate({"id": "x", "customer": "", "total": -1, "placed": "tomorrow"})
except ValidationError as e:
    for err in e.errors():     # one entry per bad field: err["loc"], err["msg"]
        print(err["loc"], err["msg"])

orders = TypeAdapter(list[Order]).validate_python(list_of_dicts)
```

1. The class body lists typed fields. pydantic converts compatible input (`"7"` to `7` , `"2025-03-01"` to a `date`) and rejects the rest; `strict=True` disables conversion.
2. `Field(min_length=1)` and `Field(ge=0)` add limits, `Literal` restricts `status` , and a default makes it optional. `extra="forbid"` turns an unexpected key into an error.
3. The second record has four bad fields, and pydantic reports *all four* in one `ValidationError` . `TypeAdapter` validates a whole page of API results in one call.

Example 1: a runnable dataclass validator

```
from dataclasses import dataclass, field, asdict
from datetime import date

@dataclass
class Order:
    id: int
    customer: str
    total: float
    placed: date
    status: str = "new"
    tags: list = field(default_factory=list)

    def __post_init__(self):
        self.id = int(self.id)
        self.total = float(self.total)
        self.customer = self.customer.strip()
        if isinstance(self.placed, str):
            self.placed = date.fromisoformat(self.placed)
        if not self.customer:
            raise ValueError("customer must not be empty")
        if self.total < 0:
            raise ValueError(f"total must be >= 0, got {self.total}")
        if self.status not in {"new", "paid", "shipped"}:
            raise ValueError(f"bad status {self.status!r}")

ok = Order("7", " Ana ", "12.5", "2025-03-01")
print(ok)
print(asdict(ok)["total"], type(ok.id).__name__)
for bad in [dict(id=1, customer="Bo", total=-5, placed="2025-03-02"),
            dict(id="x", customer="Cy", total=1, placed="2025-03-02"),
            dict(id=2, customer="Di", total=3, placed="tomorrow")]:
    try:
        Order(**bad)
    except ValueError as e:
        print("rejected:", e)
```

```
Order(id=7, customer='Ana', total=12.5, placed=datetime.date(2025, 3, 1), status='new', tags=[])
12.5 int
rejected: total must be >= 0, got -5.0
rejected: invalid literal for int() with base 10: 'x'
rejected: Invalid isoformat string: 'tomorrow'
```

1. A dataclass writes `__init__` and `__repr__`. `field(default_factory=list)` gives each object its own list. `__post_init__` runs after `__init__`: we convert types first, then check rules.
2. The first record is accepted, with converted values. The three bad ones fail for three reasons (negative total, an id that is not an integer, a date that is not ISO format). All are `ValueError`, so one `except` catches them.
3. Compared with pydantic you write the conversions yourself and stop at the first error. Recipe 3 (20.12) collects every error and sets rejects aside.

Example 2: validate a whole DataFrame

```
import numpy as np
import pandas as pd

def check_frame(df):
    problems = []
    required = {"id": "int", "total": "float", "region": "str"}
    for col, kind in required.items():
        if col not in df.columns:
            problems.append(f"missing column {col}")
        elif kind == "int" and not pd.api.types.is_integer_dtype(df[col]):
            problems.append(f"{col} should be integer, is {df[col].dtype}")
        elif kind == "float" and not pd.api.types.is_float_dtype(df[col]):
            problems.append(f"{col} should be float, is {df[col].dtype}")
    if "id" in df and df["id"].duplicated().any():
        problems.append(f"duplicate ids: {df.loc[df['id'].duplicated(), 'id'].tolist()}")
    if "total" in df:
        if df["total"].isna().any():
            problems.append(f"{int(df['total'].isna().sum())} missing totals")
        if (df["total"] < 0).any():
            problems.append("negative totals")
    return problems

good = pd.DataFrame({"id": [1, 2], "total": [5.0, 6.5], "region": ["n", "s"]})
bad = pd.DataFrame({"id": [1, 1, 3], "total": [5.0, np.nan, -2.0], "region": ["n", "s", "e"]})
print(check_frame(good))
print(check_frame(bad))
print(check_frame(bad.drop(columns="region").astype({"id": "float64"})))
```

```
[]
['duplicate ids: [1]', '1 missing totals', 'negative totals']
['id should be integer, is float64', 'missing column region', 'duplicate ids: [1.0]', '1 missing totals', 'negative
totals']
```

1. The function returns a list of problems instead of raising, so a pipeline can warn, drop rows or stop.
2. Column existence and dtype use `pandas.api.types`, which also recognises pandas 3 nullable and string types. `duplicated()` flags repeated ids; `NaN < 0` is false, so missing values need their own check.
3. The good frame gives `[]`; the second gives three problems; the third (region dropped, id as float) adds the dtype and missing-column messages, and its duplicate id shows as `1.0`.

KEY IDEA	Validate at the boundary, once. Convert text to real types, check ranges and keys, and send rejects to a dead-letter file with the reason. Everything downstream may then assume clean data. Libraries such as pydantic (records) and pandera or Great Expectations (tables) do the same job with more features.

20.10 Columnar files: Parquet and Arrow

A CSV stores data **row by row** as text. To read one column you still parse every line. **Parquet** stores data **column by column**, in compressed, typed blocks, with a footer that holds the schema and per-block statistics. **Apache Arrow** is the matching in-memory format: a standard layout for columns that lets libraries (pandas, Polars, DuckDB, Spark) share data without copying or re-parsing. The Python package is `pyarrow`. It was **not installed on the study machine**, so the pyarrow and pandas Parquet code below is labelled "not run here", followed by runnable demonstrations of the ideas that make Parquet fast.

When to use it, and when not to

- **Use Parquet** for analytic tables of more than a few hundred thousand rows, for data you read again and again, for files that keep their types (dates stay dates, integers stay integers, no `"007"` surprises) and for data lakes where engines read only needed columns and row blocks.
- **Do not use it** for tiny files people edit by hand, for append-one-row-at-a-time logging (write batches instead), or when the receiving system only accepts CSV.

Reference: pandas and pyarrow Parquet essentials (not run here)

Name	What it does	Minimal example
`df.to_parquet(path)`	Write a DataFrame (needs pyarrow or fastparquet)	`df.to_parquet("o.parquet", index=False)`
`pd.read_parquet(path)`	Read a file or folder into a DataFrame	`pd.read_parquet("o.parquet")`
`columns=[...]`	Read only these columns	`pd.read_parquet(p, columns=["id", "total"])`
`filters=[...]`	Skip row blocks that cannot match	`filters=[("year", "==", 2025)]`
`compression=`	Codec: `snappy` (default), `zstd`, `gzip`	`to_parquet(p, compression="zstd")`
`partition_cols=[...]`	Write a folder tree split by column values	`to_parquet("out/", partition_cols=["year"])`
`pa.Table.from_pandas(df)`	DataFrame to Arrow table	`t = pa.Table.from_pandas(df)`
`table.to_pandas()`	Arrow table to DataFrame	`t.to_pandas()`
`table.schema`	Names and types of columns	`print(t.schema)`
`pq.write_table(table, path)`	Write Parquet with full control	`pq.write_table(t, "o.parquet", row_group_size=100_000)`
`pq.read_table(path, columns=, filters=)`	Read into an Arrow table	`pq.read_table(p, columns=["id"])`
`pq.ParquetFile(path)`	Open for metadata and batch reading	`pf = pq.ParquetFile(p)`
`pf.metadata`	Row count, row groups, statistics	`pf.metadata.num_row_groups`
`pf.iter_batches(batch_size=)`	Stream a large file in pieces	`for b in pf.iter_batches(50_000): ...`
`pyarrow.dataset.dataset(path)`	Treat a folder of files as one table	`ds.dataset("out/", partitioning="hive")`
`pyarrow.compute ( pc )`	Fast column operations	`pc.sum(t["total"])`
`table.filter(mask)`, `table.select(cols)`	Row and column selection	`t.select(["id", "total"])`
`pd.ArrowDtype`, `dtype_backend="pyarrow"`	Keep Arrow-backed columns in pandas	`pd.read_parquet(p, dtype_backend="pyarrow")`

Typical use (not run here)

```
# NOT RUN HERE: pyarrow is not installed on the study machine (pip install pyarrow)
import pandas as pd
import pyarrow.parquet as pq

df = pd.DataFrame({"year": [2024, 2025, 2025], "region": ["n", "s", "n"], "total": [1.5, 2.5, 3.5]})
df.to_parquet("orders.parquet", compression="zstd", index=False)

only_two = pd.read_parquet("orders.parquet", columns=["region", "total"])
recent = pd.read_parquet("orders.parquet", filters=[("year", "==", 2025)])

pf = pq.ParquetFile("orders.parquet")
print(pf.metadata.num_rows, pf.metadata.num_row_groups, pf.schema_arrow)
for batch in pf.iter_batches(batch_size=100_000, columns=["total"]):
    chunk = batch.to_pandas()            # process one slice at a time

df.to_parquet("orders_dataset", partition_cols=["year"])   # folder: year=2024/, year=2025/
```

1. One call writes a typed, compressed file; `index=False` avoids storing the DataFrame index as a column.
2. `columns=` reads only some column blocks; `filters=` compares the request with statistics in the file so row groups that cannot match are skipped.
3. `ParquetFile` reads only the footer (rows, row groups, schema), which is cheap even for huge files; `iter_batches` is the Parquet equivalent of `chunksize`.
4. `partition_cols` writes one sub-folder per value, so a filter on `year` never opens the other folders.

Runnable demonstration 1: why columns and encodings save space

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(0)
n = 100_000
df = pd.DataFrame({
    "id": np.arange(n),
    "region": rng.choice(["north", "south", "east", "west"], n),
    "total": rng.normal(50, 10, n).round(2),
})
whole = df.to_csv(index=False)
one_column = df["total"].to_csv(index=False)
print("csv bytes:", len(whole), "| bytes of the total column alone:", len(one_column))
print("share of the file you need:", round(len(one_column) / len(whole), 2))

as_text = df["region"]
as_codes = df["region"].astype("category")
print("text column bytes:", as_text.memory_usage(deep=True))
print("dictionary-encoded bytes:", as_codes.memory_usage(deep=True))
print(as_codes.cat.categories.tolist(), as_codes.cat.codes.dtype)
```

```
csv bytes: 1728852 | bytes of the total column alone: 589910
share of the file you need: 0.34
text column bytes: 5350174
dictionary-encoded bytes: 100346
['east', 'north', 'south', 'west'] int8
```

1. The CSV text of the whole frame is compared with the CSV text of the `total` column alone, the size a column-by-column file would need for a query that wants only the total: about a third.

2. A text column repeats four words 100,000 times. **Dictionary encoding** stores each distinct word once and replaces every row by a small integer code. The pandas `category` type is the same idea, and the memory printed shows the saving. The codes fit in `int8`. Parquet applies this automatically and then compresses again.

Runnable demonstration 2: skipping row groups with statistics

Parquet files are cut into **row groups**, and each group stores the minimum and maximum of every column. A filter such as "dates in March" lets the reader skip any group whose range does not overlap. The simulation below does exactly that with NumPy.

```
import numpy as np

rng = np.random.default_rng(1)
days = np.sort(rng.integers(0, 365, 100_000))        # one year of events, sorted by day
group_size = 10_000
groups = [days[i:i + group_size] for i in range(0, len(days), group_size)]
stats = [(g.min(), g.max()) for g in groups]
print("row groups:", len(groups))

lo, hi = 90, 120                                      # "the month of April"
to_read = [i for i, (mn, mx) in enumerate(stats) if mx >= lo and mn <= hi]
print("groups that can match:", to_read)
rows = sum(((g >= lo) & (g <= hi)).sum() for i, g in enumerate(groups) if i in to_read)
check = ((days >= lo) & (days <= hi)).sum()
print("rows found:", rows, "same as full scan:", rows == check)
print("fraction of data read:", len(to_read) / len(groups))
```

```
row groups: 10
groups that can match: [2, 3]
rows found: 8490 same as full scan: True
fraction of data read: 0.2
```

1. The data is sorted by day and cut into 10 blocks of 10,000 rows; per block we keep only (minimum, maximum), which Parquet writes in its footer for you.
2. A block can contain matches only if its range overlaps the query: `max >= lo` and `min <= hi`. Only blocks 2 and 3 qualify, so 20 per cent of the data is read, and the count equals the full scan.
3. Sorting matters: with shuffled days every block spans the whole year and nothing can be skipped. That is why people sort or partition by the column they filter on most.

WHY IT MATTERS

Reading Parquet with `columns=` and `filters=` is often the difference between a notebook that loads in seconds and one that runs out of memory. In pandas 3 you can also keep the Arrow types after reading (`dtype_backend="pyarrow"`), which avoids converting everything to NumPy and keeps missing integers as integers. Both parts need pyarrow installed and were not run here.

20.11 Logging, progress, big files and encodings

These four tools make a pipeline *observable* and *survivable*. **Logging** records what happened, with severity levels and without `print`. A **progress bar** tells you a long loop is alive. **Chunking** processes a file in pieces so memory stays small. **Encodings** decide how bytes turn into characters; get one wrong and `é` becomes `Ã©`.

Reference: logging, progress, chunking and encoding essentials

Name	What it does	Minimal example
`logging.getLogger(__name__)`	Named logger per module	`log = logging.getLogger(__name__)`

Name	What it does	Minimal example
`logging.basicConfig(...)`	One-time setup for scripts (level, format, stream)	`basicConfig(level=logging.INFO)`
Levels	DEBUG, INFO, WARNING, ERROR, CRITICAL	`log.warning("slow")`
Lazy arguments	Format only if the message is emitted	`log.info("%d rows", n)`
`log.exception(msg)`	ERROR with the traceback (inside `except`)	`log.exception("load failed")`
`logging.Formatter(fmt)`	Message layout with time, level, name	`"%(asctime)s %(levelname)s %(message)s"`
`handler.setLevel`	Different thresholds per destination	`console.setLevel(logging.WARNING)`
`tqdm(iterable, total=, desc=)`	Progress bar around a loop (package tqdm)	`for r in tqdm(rows): ...`
`tqdm.pandas()`	Adds `progress_apply` to pandas	`df.progress_apply(f, axis=1)`
`pd.read_csv(chunksize=n)`	Iterator of DataFrames	`for ch in pd.read_csv(p, chunksize=50_000):`
`usecols=`, `dtype=`, `nrows=`	Read fewer columns, smaller types, a sample	`read_csv(p, usecols=["a"], nrows=1000)`
`open(p, encoding=, errors=)`	Choose the codec and how to treat bad bytes	`open(p, encoding="latin-1")`
`bytes.decode(enc, errors=)`	Bytes to text; errors: `strict`, `replace`, `ignore`	`raw.decode("utf-8", errors="replace")`
`utf-8-sig`	UTF-8 that drops a leading byte-order mark	`open(p, encoding="utf-8-sig")`
`pd.read_csv(encoding=, encoding_errors=)`	Encoding control inside pandas	`read_csv(p, encoding="cp1252")`
`hashlib.sha256()`	Checksum to verify a copy or download	`h.update(chunk)`

Example 1: logging that you can actually read

```
import logging, sys, tempfile
from pathlib import Path

logfile = Path(tempfile.mkdtemp()) / "etl.log"
root = logging.getLogger()
root.setLevel(logging.DEBUG)
root.handlers.clear()

console = logging.StreamHandler(sys.stdout)
console.setLevel(logging.INFO)
console.setFormatter(logging.Formatter("%(levelname)-7s %(name)s: %(message)s"))
to_file = logging.FileHandler(logfile)
to_file.setFormatter(logging.Formatter("%(levelname)s|%(name)s|%(message)s"))
root.addHandler(console)
root.addHandler(to_file)

log = logging.getLogger("etl.orders")
log.debug("raw row sample: %s", {"id": 1})
log.info("loaded %d rows from %s", 120, "orders.csv")
log.warning("%d rows rejected (%.1f%%)", 3, 100 * 3 / 120)
try:
    int("12x")
except ValueError as e:
    log.error("parse failed: %s", e)

print("--- file content ---")
print(logfile.read_text().strip())
```

```
INFO    etl.orders: loaded 120 rows from orders.csv
WARNING etl.orders: 3 rows rejected (2.5%)
ERROR   etl.orders: parse failed: invalid literal for int() with base 10: '12x'
--- file content ---
DEBUG|etl.orders|raw row sample: {'id': 1}
INFO|etl.orders|loaded 120 rows from orders.csv
WARNING|etl.orders|3 rows rejected (2.5%)
ERROR|etl.orders|parse failed: invalid literal for int() with base 10: '12x'
```

1. Handlers on the **root** logger receive messages from every logger below it. The console handler shows INFO and above in a compact layout; the file handler keeps everything, including DEBUG.
2. `etl.orders` is a child of `etl`; the dots form a hierarchy, so `getLogger("etl")` tunes a whole subsystem.
3. The `%d` and `%s` arguments are passed separately, so Python builds the string only if the level is enabled.
4. Real formats add `%(asctime)s`; it is omitted so the output is repeatable. Inside `except`, `log.exception(...)` adds the traceback.

COMMON MISTAKE

Wrong: `log.info(f"calling {url} with token {token}")`. **Symptom:** secrets end up in log files and ticketing systems, and the f-string is built even when INFO is off. **Fix:** `log.info("calling %s", url)` and never log tokens, passwords or personal data; mask them (`token[:4] + "..."`).

Example 2: a progress bar without tqdm

The `tqdm` package (not installed here) is the usual choice. The idea is simple enough to build, which also shows how it works: wrap an iterable, count what passes, and report at intervals.

```
import sys

def progress(iterable, total, desc="", steps=4, out=sys.stdout, width=20):
    marks = {max(1, round(total * k / steps)) for k in range(1, steps + 1)}
    for i, item in enumerate(iterable, 1):
        yield item
        if i in marks:
            filled = round(width * i / total)
            print(f"{desc}: [{'#' * filled}{' ' * (width - filled)}] {100 * i // total:3d}% ({i}/{total})", file=out)

total = 0
for x in progress(range(1, 101), total=100, desc="load"):
    total += x
print("sum =", total)
```

```
load: [#####               ]  25% (25/100)
load: [##########          ]  50% (50/100)
load: [###############     ]  75% (75/100)
load: [####################] 100% (100/100)
sum = 5050
```

1. `progress` is a **generator** wrapper: it yields every item unchanged and reports at 25, 50, 75 and 100 per cent. Real tqdm reports by elapsed time, which is why its output cannot be pasted exactly.
2. Send progress to `sys.stderr` (the `out` argument) so stdout stays clean for data. The loop sums 1 to 100: 5050.

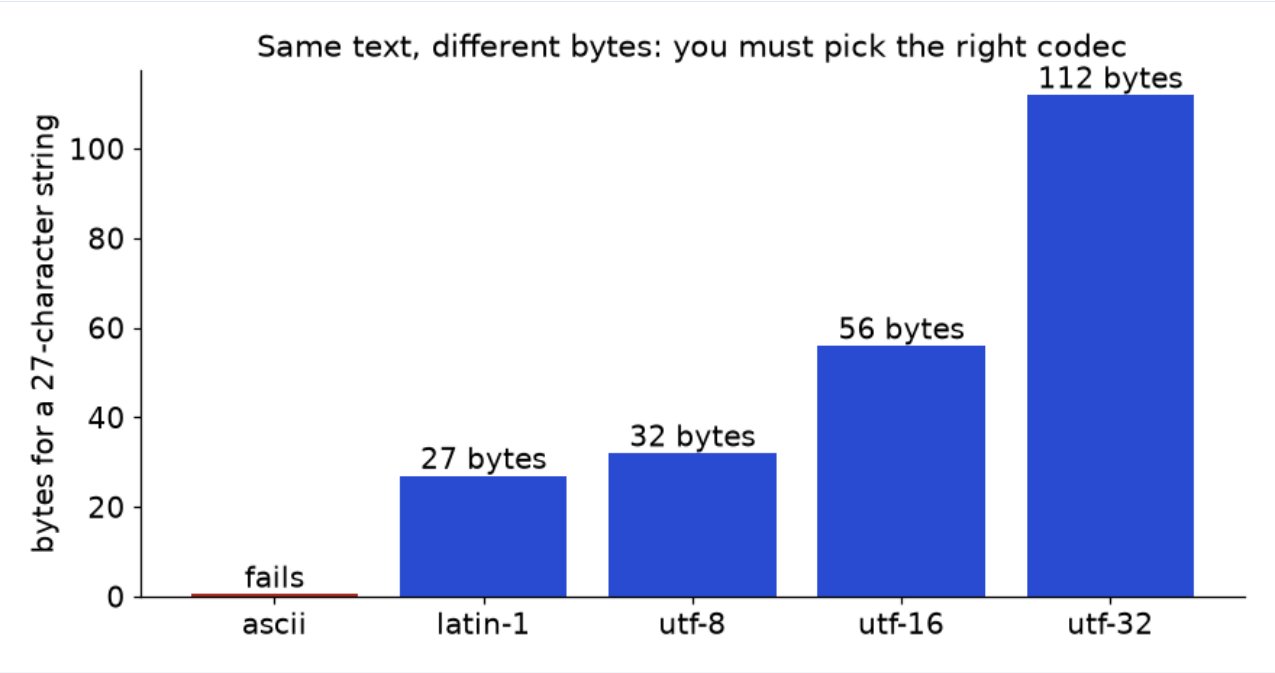
```
# NOT RUN HERE: tqdm is not installed on the study machine (pip install tqdm)
from tqdm import tqdm
import pandas as pd

for row in tqdm(rows, desc="load", total=len(rows)):      # bar with rate and ETA
    process(row)

tqdm.pandas(desc="clean")                                  # adds progress_apply
df["clean"] = df["raw"].progress_apply(clean_text)

with tqdm(total=file_size, unit="B", unit_scale=True) as bar:   # manual updates
    for part in response.iter_content(8192):
        out.write(part)
        bar.update(len(part))
```

Example 3: encodings, byte-order marks and mojibake

```
import io
import pandas as pd

raw = "Zoë,Müller,café\n".encode("latin-1")          # a file saved by an old tool
try:
    raw.decode("utf-8")
except UnicodeDecodeError as e:
    print("utf-8 failed:", e.reason, "at byte", e.start)
print(raw.decode("latin-1").strip())
print(raw.decode("utf-8", errors="replace").strip())

bom = "id,name\n1,Zoë\n".encode("utf-8-sig")
print(bom[:3])
print(repr(bom.decode("utf-8").splitlines()[0]))
print(repr(bom.decode("utf-8-sig").splitlines()[0]))
print(pd.read_csv(io.BytesIO(bom), encoding="utf-8-sig").columns.tolist())

garbled = "café".encode("utf-8").decode("latin-1")
print(garbled, "->", garbled.encode("latin-1").decode("utf-8"))
```

```
utf-8 failed: invalid continuation byte at byte 2
Zoë,Müller,café
Zo�,M�ller,caf�
b'\xef\xbb\xbf'
'\ufeffid,name'
'id,name'
['id', 'name']
cafÃ© -> café
```

1. A file is bytes. "Zoë" saved as Latin-1 uses the single byte `0xEB`; UTF-8 expects a longer sequence there, so decoding fails and the error gives the reason and byte position. Decoding as Latin-1 works. `errors="replace"` never crashes but swaps bad bytes for `&#65533;`; use it to inspect, never to load data you care about.
2. Some Windows tools start a UTF-8 file with a byte-order mark (`EF BB BF`). Plain `utf-8` keeps it as an invisible character, so the first header becomes `'\ufeffid'` and `df["id"]` fails. `utf-8-sig` removes it; pandas accepts the same name.
3. **Mojibake** is text decoded with the wrong codec (UTF-8 bytes read as Latin-1 give `cafÃ©`). It is reversible: encode with the wrong codec, decode with the right one.



The same text takes a different number of bytes in each encoding, and ASCII cannot hold it at all.

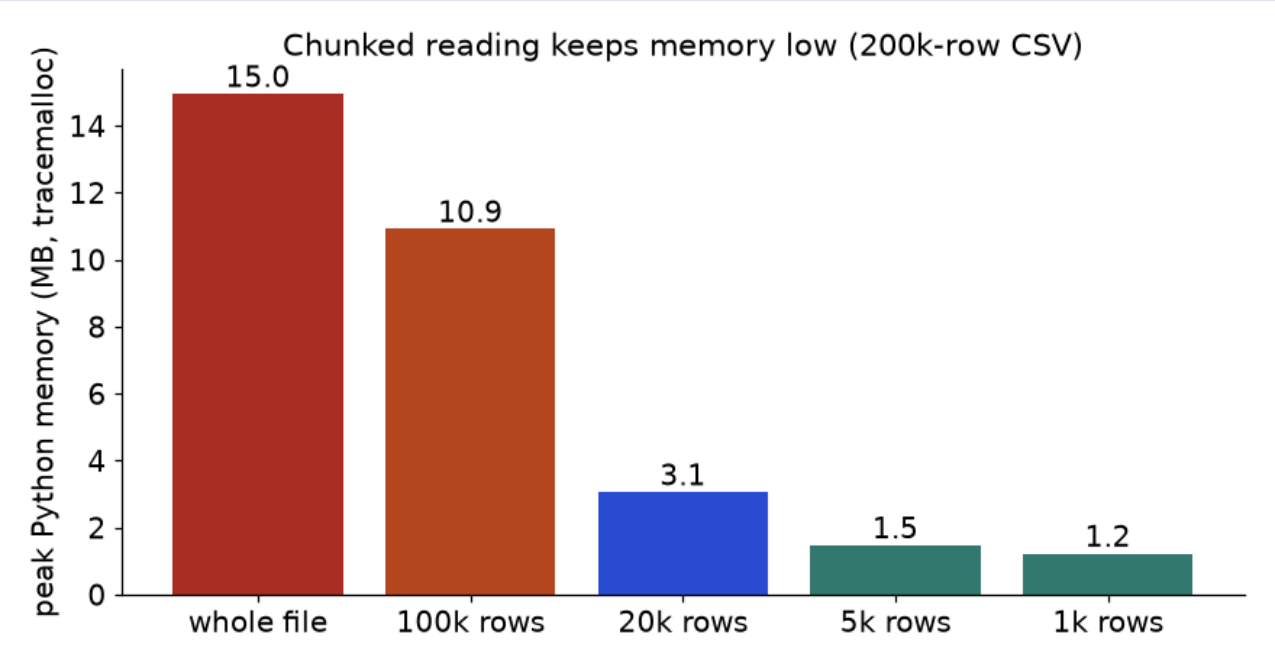
How to read it: every bar is the same 27-character string. ASCII (red) cannot represent accented letters, so it fails. Latin-1 uses one byte per character. UTF-8 uses one byte for plain letters and two for accented ones, so it is slightly bigger. UTF-16 and UTF-32 use two and four bytes per character (plus a header for UTF-16 and UTF-32). UTF-8 is the safe default for new files; when reading old files, expect Latin-1 or cp1252 from Windows tools.

Example 4: big files in chunks

```
import os, tempfile
import numpy as np
import pandas as pd

rng = np.random.default_rng(42)
n = 50_000
df = pd.DataFrame({
    "region": rng.choice(["north", "south", "east"], n),
    "units": rng.integers(1, 20, n),
    "price": rng.uniform(1, 100, n).round(2),
})
path = os.path.join(tempfile.mkdtemp(), "sales.csv")
df.to_csv(path, index=False)

revenue, rows_seen = {}, 0
for chunk in pd.read_csv(path, chunksize=10_000, dtype={"region": "category"}):
    rows_seen += len(chunk)
    part = (chunk["units"] * chunk["price"]).groupby(chunk["region"], observed=True).sum()
    for region, value in part.items():
        revenue[region] = revenue.get(region, 0.0) + value

chunked = pd.Series(revenue).sort_index()
full = (df["units"] * df["price"]).groupby(df["region"]).sum().sort_index()
print(rows_seen)
print(chunked.round(0).to_dict())
print("matches full read:", np.allclose(chunked, full))

sample = pd.read_csv(path, usecols=["region", "units"], nrows=3)
print(sample)
```

```
50000
{'east': 8494800.0, 'north': 8473515.0, 'south': 8326957.0}
matches full read: True
  region  units
0  north      2
1   east     19
2  south     10
```

1. The script writes a 50,000-row CSV so it is self-contained; in real life this is the file that does not fit in memory.
2. `chunksize=10_000` makes `read_csv` an iterator of five DataFrames, one in memory at a time. Each chunk yields a *partial* result (revenue by region) added into a running dictionary. This works for combinable statistics: sums, counts, min, max; a mean is a sum plus a count; a median is not combinable.
3. The chunked result is compared with the all-at-once result using `np.allclose`, because additions in another order differ in the last digits. They match. `usecols` and `nrows` are the quickest way to peek at a big file.



Peak memory tracked by `tracemalloc` while summing one column (measured once; values vary).

How to read it: the red bar is the whole file loaded at once and the other bars use smaller chunks. Memory falls as the chunk shrinks and levels off, because each chunk still has fixed overhead. The trade-off: very small chunks mean more Python-level loop iterations, so pick a chunk that is comfortably small (tens of thousands of rows), not the smallest possible.

20.12 Recipes: How do I ...?

Ten solved tasks. Each is a complete script you can copy. The first four are the big ones that tie several libraries together. The rest are short answers to questions that come up every week.

Recipe 1. How do I ingest a paginated API into SQLite, safely and repeatably?

Plan: fetch page by page with a cursor, validate each record, set aside the bad ones, and upsert the good ones so that running the job twice gives the same table. To keep the recipe short, `fetch_page` stands in for `requests.get(url, params=..., timeout=5).json()`; the real HTTP code with a local server is in 20.7.

```
import sqlite3

ITEMS = [{"id": i, "name": f"item{i}", "price": str(i * 1.5)} for i in range(1, 8)]
ITEMS[4]["price"] = "n/a"                              # one bad record from the provider

def fetch_page(after, limit):
    page = [x for x in ITEMS if x["id"] > after][:limit]
    return {"items": page, "next": page[-1]["id"] if len(page) == limit else None}

con = sqlite3.connect(":memory:")
con.execute("create table items(id integer primary key, name text not null, price real not null)")
UPSERT = """insert into items values (?, ?, ?)
            on conflict(id) do update set name = excluded.name, price = excluded.price"""
rejected = []

def ingest(page_size=3):
    after, loaded = 0, 0
    while True:
        body = fetch_page(after, page_size)
        good = []
        for rec in body["items"]:
            try:
                good.append((int(rec["id"]), rec["name"], float(rec["price"])))
            except (KeyError, ValueError) as e:
                rejected.append((rec.get("id"), str(e)))
        with con:
            con.executemany(UPSERT, good)
        loaded += len(good)
        if body["next"] is None:
            return loaded
        after = body["next"]

print("run 1 loaded:", ingest())
print("run 2 loaded:", ingest())
print("rows in table:", con.execute("select count(*) from items").fetchone()[0])
print("rejected:", rejected[:1])
print(con.execute("select * from items where id = 7").fetchone())
```

```
run 1 loaded: 6
run 2 loaded: 6
rows in table: 6
rejected: [(5, "could not convert string to float: 'n/a'")]
(7, 'item7', 10.5)
```

1. Seven provider items; the fifth has a broken price. `fetch_page` does cursor paging: ids above `after`, and `next` is `None` on a short page.
2. The table has a primary key and `not null` columns as the last line of defence. `UPSERT` overwrites an existing id, so a re-run is harmless.
3. Each record is converted inside `try`; a bad one goes to `rejected` with the reason and the loop continues. `with con:` commits each page as one transaction.
4. Each run loads 6 good rows (3, 2 and 1 per page). Run 2 changes nothing: still 6 rows. The bad record is rejected in both runs (only the first is shown).

Recipe 2. How do I scrape a multi-page table into a DataFrame, politely?

```
import time
import pandas as pd
from bs4 import BeautifulSoup
from urllib.robotparser import RobotFileParser

PAGES = {
    "https://shop.example/list?p=1": "<table><tr><th>SKU</th><th>Price</th></tr><tr><td>A</td><td>$1.10</td></tr><tr>
<td>B</td><td>$2.20</td></tr></table>",
    "https://shop.example/list?p=2": "<table><tr><th>SKU</th><th>Price</th></tr><tr><td>B</td><td>$2.20</td></tr><tr>
<td>C</td><td>$3.30</td></tr></table>",
    "https://shop.example/list?p=3": "<p>No table here</p>",
}
rp = RobotFileParser()
rp.parse(["User-agent: *", "Disallow: /admin/"])

def fetch(url):                       # stand-in for requests.get(url, headers=..., timeout=...).text
    return PAGES[url]

frames = []
for url in PAGES:
    if not rp.can_fetch("study-bot", url):
        continue
    table = BeautifulSoup(fetch(url), "html.parser").find("table")
    if table is None:
        print("no table at", url[-3:])
        continue
    rows = [[c.get_text(strip=True) for c in tr.find_all(["th", "td"])] for tr in table.find_all("tr")]
    frames.append(pd.DataFrame(rows[1:], columns=rows[0]))
    time.sleep(0.01)                  # be polite: real crawls wait seconds, not 0.01

df = pd.concat(frames, ignore_index=True).drop_duplicates().reset_index(drop=True)
df["Price"] = df["Price"].str.lstrip("$").astype(float)
print(df)
```

```
no table at p=3
  SKU  Price
0   A    1.1
1   B    2.2
2   C    3.3
```

1. The dictionary stands in for three web pages; replace `fetch` with `requests.get(...).text` (plus timeout and error handling). `can_fetch` is asked before every page.
2. Page 3 has no table, so `find` returns `None` and is reported instead of crashing. Each row (header included) is read into a list of strings; the first row names the columns.
3. `concat` and `drop_duplicates` remove product B, which appears on both pages; `reset_index` renumbers. The price text is cleaned and converted to float.

Recipe 3. How do I validate a schema and keep the rejects?

```
import json, tempfile
from pathlib import Path

lines = [
    '{"id": 1, "total": 10.5, "region": "north"}',
    '{"id": 2, "total": "oops", "region": "south"}',
    'not json at all',
    '{"id": 4, "total": -1, "region": "east"}',
    '{"id": 5, "total": 3, "region": "mars"}',
    '{"id": 6, "region": "west"}',
]
REGIONS = {"north", "south", "east", "west"}

def parse(line):
    rec = json.loads(line)
    total = float(rec["total"])
    if total < 0:
        raise ValueError("negative total")
    if rec["region"] not in REGIONS:
        raise ValueError(f"unknown region {rec['region']!r}")
    return {"id": int(rec["id"]), "total": total, "region": rec["region"]}

good = []
rejects = Path(tempfile.mkdtemp()) / "rejected.jsonl"
with rejects.open("w", encoding="utf-8") as bad:
    for n, line in enumerate(lines, 1):
        try:
            good.append(parse(line))
        except (ValueError, KeyError) as e:      # JSONDecodeError is a ValueError
            bad.write(json.dumps({"line": n, "error": f"{type(e).__name__}: {e}"}) + "\n")

print(len(good), "good:", good)
for row in rejects.read_text().splitlines():
    print(row)
```

```
1 good: [{'id': 1, 'total': 10.5, 'region': 'north'}]
{"line": 2, "error": "ValueError: could not convert string to float: 'oops'"}
{"line": 3, "error": "JSONDecodeError: Expecting value: line 1 column 1 (char 0)"}
{"line": 4, "error": "ValueError: negative total"}
{"line": 5, "error": "ValueError: unknown region 'mars'"}
{"line": 6, "error": "KeyError: 'total'"}
```

1. The input has every kind of damage: bad number, broken JSON, negative value, unknown category, missing key.
2. One `except` catches `ValueError` (which includes `JSONDecodeError`) and `KeyError`.
3. Each reject is one JSON object with the line number and reason in a JSON Lines file: an audit trail for the data owner. One record passes, five are rejected with five different messages.

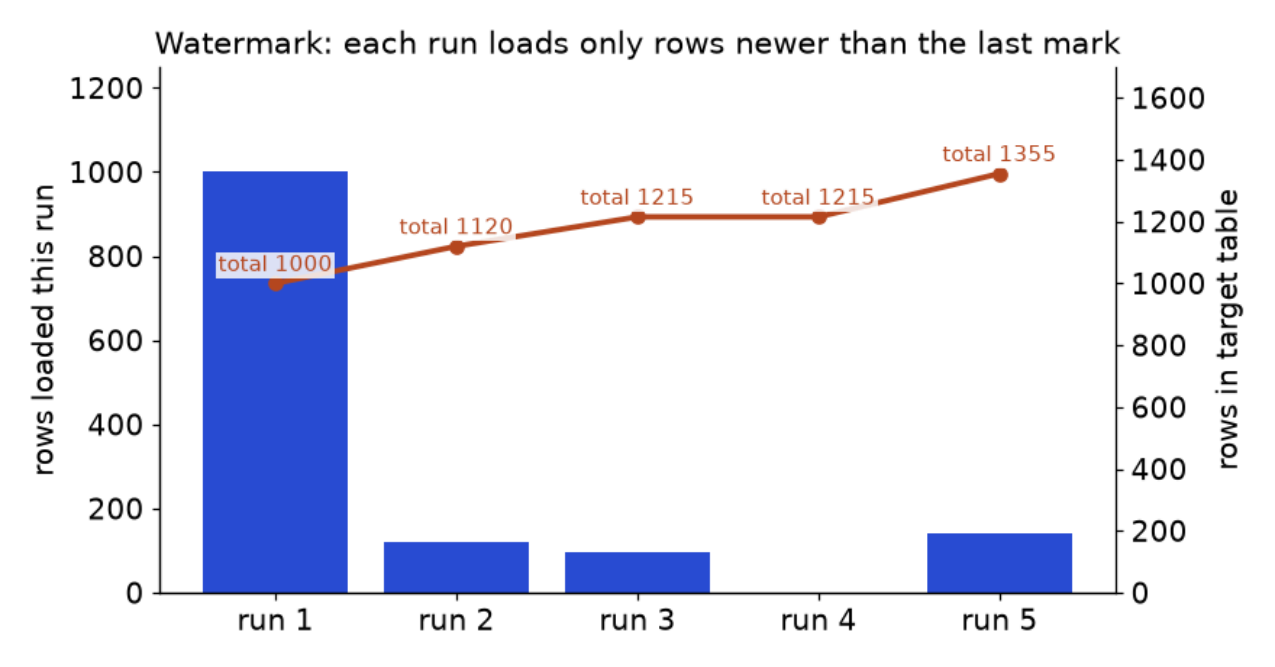
Recipe 4. How do I do an incremental load with a watermark?

A **watermark** is the highest `updated_at` value you have already loaded. Each run asks the source only for rows at or after it. The watermark is stored in the target database and updated in the same transaction as the data, so a crash can never advance it without the rows.

```
import sqlite3
from datetime import datetime, timedelta

src, dst = sqlite3.connect(":memory:"), sqlite3.connect(":memory:")
for c in (src, dst):
    c.execute("create table orders(id integer primary key, total real, updated_at text)")
dst.execute("create table etl_state(name text primary key, watermark text)")
UPSERT = """insert into orders values (?, ?, ?) on conflict(id) do update
            set total = excluded.total, updated_at = excluded.updated_at"""

def load(overlap_hours=0):
    row = dst.execute("select watermark from etl_state where name = 'orders'").fetchone()
    wm = row[0] if row else "1970-01-01T00:00:00"
    start = (datetime.fromisoformat(wm) - timedelta(hours=overlap_hours)).isoformat()
    rows = src.execute("select id, total, updated_at from orders where updated_at >= ? order by updated_at",
                       (start,)).fetchall()
    with dst:                                          # data and watermark commit together
        dst.executemany(UPSERT, rows)
        if rows:
            dst.execute("""insert into etl_state values ('orders', ?)
                           on conflict(name) do update set watermark = max(watermark, excluded.watermark)""",
                        (rows[-1][2],))
    return len(rows)

src.executemany("insert into orders values (?, ?, ?)",
                [(1, 10.0, "2025-03-01T09:00:00"), (2, 20.0, "2025-03-01T10:00:00"), (3, 30.0, "2025-03-01T11:00:00")])
print("run 1 read", load())
print("run 2 read", load(), "(only the boundary row again)")

src.execute("update orders set total = 25.0, updated_at = '2025-03-01T12:00:00' where id = 2")
src.execute("insert into orders values (4, 40.0, '2025-03-01T12:30:00')")
print("run 3 read", load())
print(dst.execute("select * from orders order by id").fetchall())

src.execute("insert into orders values (5, 50.0, '2025-03-01T11:30:00')")      # arrives late, older timestamp
print("run 4 (no overlap) read", load(), "-> rows in target:", dst.execute("select count(*) from orders").fetchone()[0])
print("run 5 (2 h overlap) read", load(overlap_hours=2), "-> rows in target:", dst.execute("select count(*) from
orders").fetchone()[0])
```

```
run 1 read 3
run 2 read 1 (only the boundary row again)
run 3 read 3
[(1, 10.0, '2025-03-01T09:00:00'), (2, 25.0, '2025-03-01T12:00:00'), (3, 30.0, '2025-03-01T11:00:00'), (4, 40.0, '2025-03-
01T12:30:00')]
run 4 (no overlap) read 1 -> rows in target: 4
run 5 (2 h overlap) read 4 -> rows in target: 5
```

1. Two in-memory databases play the source and the warehouse; the warehouse has a state table with one row per job.
2. `load` reads the watermark (1970 on the first run), steps back by an optional overlap, and selects rows with `updated_at >= start`. Using `>=` means rows sharing the exact watermark are never missed; they are read again and the upsert makes that harmless.
3. Inside `with dst:` the rows and the new watermark commit together, or neither does. `max(...)` stops the watermark moving backwards during an overlap run.
4. Run 1 reads 3 rows; run 2 reads only the boundary row. Run 3 sees the changed order 2 (now 25.0), the new order 4 and the boundary row.
5. Run 4 shows the weakness: order 5 arrives late with an older timestamp (11:30, watermark 12:30) and is never read. Run 5 uses a two-hour overlap and catches it, at the price of re-reading recent data.



Illustrative numbers: a first full load, then small incremental loads, one run with nothing new.

How to read it: blue bars are the rows each run touches; the orange line is how many rows the target holds in total. After the big first load the bars are small, which is the whole point: later runs do a tiny amount of work. Run 4 brought nothing new and the orange line stays flat, which is a healthy result and not an error.

Recipe 5. How do I combine many CSV files and remember where each row came from?

```
import tempfile
from pathlib import Path
import pandas as pd

root = Path(tempfile.mkdtemp())
(root / "sales_2025-01.csv").write_text("sku,units\nA,3\nB,5\n")
(root / "sales_2025-02.csv").write_text("sku,units\nA,4\nC,1\n")
(root / "notes.txt").write_text("ignore me")

frames = [pd.read_csv(p).assign(source=p.stem) for p in sorted(root.glob("sales_*.csv"))]
df = pd.concat(frames, ignore_index=True)
df["month"] = df["source"].str[-7:]
print(df)
print(df.groupby("month")["units"].sum().to_dict())
```

```
sku  units         source    month
0   A      3  sales_2025-01  2025-01
1   B      5  sales_2025-01  2025-01
2   A      4  sales_2025-02  2025-02
3   C      1  sales_2025-02  2025-02
{'2025-01': 8, '2025-02': 5}
```

1. `glob("sales_*.csv")` ignores the notes file; `sorted` makes the order reproducible.
2. `assign(source=p.stem)` tags each row with its file, so a wrong number can be traced back; `concat(..., ignore_index=True)` gives a fresh index.

Recipe 6. How do I open a mystery CSV (unknown delimiter, encoding and decimal comma)?

```
import csv, io
import pandas as pd

raw = "id;name;amount\n1;Zoë;12,5\n2;Müller;7,25\n".encode("cp1252")

def guess(raw):
    for enc in ("utf-8", "cp1252"):
        try:
            text = raw.decode(enc)
            break
        except UnicodeDecodeError:
            continue
    sep = csv.Sniffer().sniff(text[:2000], delimiters=",;\t|").delimiter
    return enc, sep

enc, sep = guess(raw)
print(enc, repr(sep))
df = pd.read_csv(io.BytesIO(raw), sep=sep, encoding=enc, decimal=",")
print(df)
print(df["amount"].sum())
```

```
cp1252 ';'
   id    name  amount
0   1     Zoë   12.50
1   2  Müller    7.25
19.75
```

1. The bytes imitate a Windows export: cp1252, semicolons, decimal comma.
2. `guess` tries strict UTF-8 first, fails on the accented byte, then uses `cp1252`. `csv.Sniffer` finds the delimiter among the allowed ones.
3. `decimal=","` makes pandas read `12,5` as 12.5; without it the column would be text.

Recipe 7. How do I write a formatted Excel report?

```
import tempfile
from pathlib import Path
from openpyxl import Workbook, load_workbook
from openpyxl.styles import Font, PatternFill

path = Path(tempfile.mkdtemp()) / "report.xlsx"
wb = Workbook()
ws = wb.active
ws.title = "Summary"
ws.append(["Region", "Revenue"])
for row in [("North", 1234.5), ("South", 987.25)]:
    ws.append(row)
for cell in ws[1]:
    cell.font = Font(bold=True, color="FFFFFF")
    cell.fill = PatternFill("solid", fgColor="2B4FD6")
for (cell,) in ws.iter_rows(min_row=2, min_col=2, max_col=2):
    cell.number_format = "#,##0.00"
ws.freeze_panes = "A2"
ws.column_dimensions["A"].width = 18
ws.auto_filter.ref = ws.dimensions
wb.save(path)

check = load_workbook(path)["Summary"]
print(check["A1"].font.bold, check["B2"].number_format, check.freeze_panes, check.auto_filter.ref)
print(check["B2"].value)
```

```
True #,##0.00 A2 A1:B3
1234.5
```

1. Data goes in with `append`; styling is applied afterwards: bold white header text on a blue fill (hex without `#`).
2. `number_format` changes only the display; the cell still holds the number 1234.5, so finance can still sum it. Freeze panes, a column width and a filter finish the sheet.
3. Reloading the file confirms what was really saved.

Recipe 8. How do I upsert a whole DataFrame into a table?

```
import sqlite3
import pandas as pd

con = sqlite3.connect(":memory:")
con.execute("create table prices(sku text primary key, price real)")
con.execute("insert into prices values ('A', 1.0), ('B', 2.0)")
new = pd.DataFrame({"sku": ["B", "C"], "price": [2.5, 3.0]})

with con:
    new.to_sql("prices_stage", con, if_exists="replace", index=False)
    con.execute("""insert into prices select sku, price from prices_stage where true
                   on conflict(sku) do update set price = excluded.price""")
    con.execute("drop table prices_stage")
print(pd.read_sql_query("select * from prices order by sku", con))
```

```
sku  price
0   A    1.0
1   B    2.5
2   C    3.0
```

1. `to_sql` cannot upsert, so the DataFrame goes to a throw-away **staging table**.
2. One SQL statement moves staging rows into the real table (in SQLite, `where true` after the `select` avoids a parsing ambiguity before `on conflict`), then the stage is dropped, all in one transaction.
3. Result: A unchanged, B updated to 2.5, C inserted. PostgreSQL uses the same `insert ... on conflict`.

Recipe 9. How do I flatten nested JSON that contains lists?

```
import pandas as pd

orders = [
    {"id": 1, "customer": {"name": "Ana"}, "lines": [{"sku": "A", "qty": 2}, {"sku": "B", "qty": 1}]},
    {"id": 2, "customer": {"name": "Bo"}, "lines": [{"sku": "A", "qty": 5}]},
    {"id": 3, "customer": {"name": "Cy"}, "lines": []},
]
lines = pd.json_normalize(orders, record_path="lines", meta=["id", ["customer", "name"]])
print(lines)
print(lines.groupby("customer.name")["qty"].sum().to_dict())
```

```
sku  qty id customer.name
0   A    2  1           Ana
1   B    1  1           Ana
2   A    5  2            Bo
{'Ana': 3, 'Bo': 5}
```

1. Each order has a list of lines, so the natural table has one row per line. `record_path="lines"` expands that list.
2. `meta` copies parent fields onto every child row, including the nested `["customer", "name"]`. Order 3 has an empty list and therefore no rows; join the parent table back if you need it.

Recipe 10. How do I cache API responses so reruns are cheap?

```
import hashlib, json, tempfile, time
from pathlib import Path

cache_dir = Path(tempfile.mkdtemp())
calls = {"n": 0}

def fake_api(url):                                  # stands in for requests.get(url).json()
    calls["n"] += 1
    return {"url": url, "value": len(url)}

def cached_get(url, max_age=3600, fetch=fake_api):
    f = cache_dir / (hashlib.sha256(url.encode()).hexdigest()[:16] + ".json")
    if f.exists() and time.time() - f.stat().st_mtime < max_age:
        return json.loads(f.read_text())
    data = fetch(url)
    tmp = f.with_suffix(".tmp")
    tmp.write_text(json.dumps(data))
    tmp.replace(f)
    return data

for u in ["https://api.example/a", "https://api.example/b", "https://api.example/a"]:
    cached_get(u)
print("api calls:", calls["n"])
cached_get("https://api.example/a", max_age=0)
print("api calls after forcing a refresh:", calls["n"])
```

```
api calls: 2
api calls after forcing a refresh: 3
```

1. The cache key is a hash of the URL, so any URL becomes a safe file name. A file younger than `max_age` seconds is returned without a network call.
2. Otherwise the function fetches and writes through a temporary file plus `replace` (the atomic write from 20.2).
3. Three lookups cost two real calls; `max_age=0` forces a refresh, giving a third. Caching also keeps development runs far below rate limits.

20.13 Performance and pitfalls

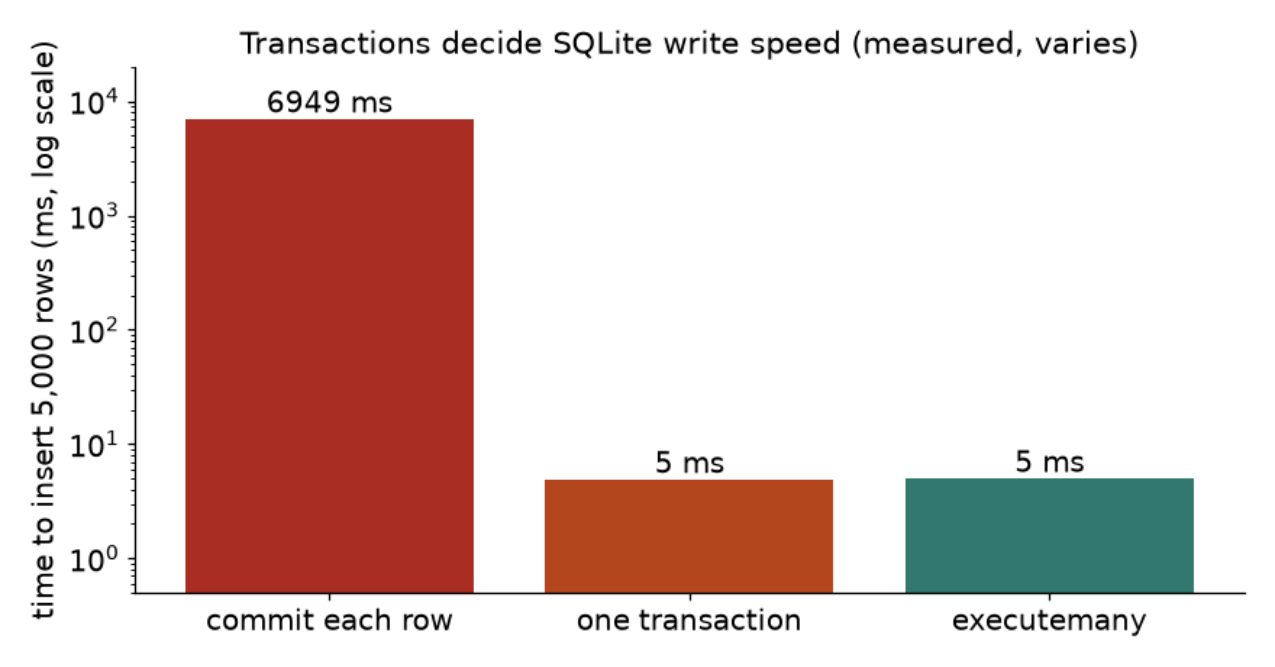
Where time and memory actually go

- **Round trips and syncs.** Every network call and every SQLite commit has a fixed cost (a commit forces a disk sync). Batch, page, and reuse connections.
- **Parsing and memory.** CSV and JSON are parsed character by character, and a DataFrame is often several times bigger than its file. Read fewer columns, use smaller dtypes (`category`, `int32`), and chunk.
- **Waiting.** An HTTP loop is mostly idle; threads or `httpx.AsyncClient` help, within the provider's limits.

```
import os, sqlite3, tempfile, time

rows = [(i, f"name{i}", i * 0.5) for i in range(3000)]

def insert_seconds(per_row_commit):
    con = sqlite3.connect(os.path.join(tempfile.mkdtemp(), "t.db"))
    con.execute("create table t(id integer, name text, v real)")
    t0 = time.perf_counter()
    if per_row_commit:
        for r in rows:
            con.execute("insert into t values (?, ?, ?)", r)
            con.commit()
    else:
        with con:
            con.executemany("insert into t values (?, ?, ?)", rows)
    elapsed = time.perf_counter() - t0
    assert con.execute("select count(*) from t").fetchone()[0] == 3000
    con.close()
    return elapsed

slow, fast = insert_seconds(True), insert_seconds(False)
print("same 3000 rows, commit per row is slower than one batch:", slow > fast)
```

```
same 3000 rows, commit per row is slower than one batch: True
```

1. The function writes the same 3,000 rows into a real file database, once with a commit after each row and once with one `executemany` inside one transaction.
2. The `assert` proves both variants stored all rows, so the comparison is fair.
3. I print a yes/no result because absolute timings depend on your disk and operating system; the figure below shows what was measured on the study machine for 5,000 rows.



Measured once on the study machine (it varies by machine and disk).

How to read it: each bar is the time to insert the same 5,000 rows. The vertical axis is logarithmic, so each gridline is ten times the last. The red bar commits after every row and is hundreds of times slower (or worse) than the other two. The orange bar loops inside one transaction. The green bar uses `executemany` in one transaction, which is the fastest or about equal to the loop, because the statement is prepared once. The fix costs one line (`with con:`).

Pitfall lab: five failures on purpose

```
import io, json, os, sqlite3, tempfile
import numpy as np
import pandas as pd
from bs4 import BeautifulSoup

raw = "zip,qty\n02134,5\n90210,7\n"
print(pd.read_csv(io.StringIO(raw))["zip"].tolist())
print(pd.read_csv(io.StringIO(raw), dtype={"zip": str})["zip"].tolist())

try:
    json.dumps({"n": np.int64(5)})
except TypeError as e:
    print("TypeError:", e)
print(json.dumps({"x": float("nan")}), json.dumps({"n": int(np.int64(5)), "x": None}))

path = os.path.join(tempfile.mkdtemp(), "t.db")
con = sqlite3.connect(path)
con.execute("create table t(a)")
con.commit()
con.execute("insert into t values (1)")
con.close()
print("rows after forgetting commit:", sqlite3.connect(path).execute("select count(*) from t").fetchone()[0])

try:
    BeautifulSoup("<p>hi</p>", "html.parser").find("table").find_all("tr")
except AttributeError as e:
    print("AttributeError:", e)

c2 = sqlite3.connect(":memory:")
c2.execute("create table keep(id integer primary key, v text)")
pd.DataFrame({"id": [1], "v": ["a"]}).to_sql("keep", c2, if_exists="replace", index=False)
print("primary-key flags after to_sql replace:", [r[5] for r in c2.execute("pragma table_info(keep)")])
```

```
[2134, 90210]
['02134', '90210']
TypeError: Object of type int64 is not JSON serializable
{"x": NaN} {"n": 5, "x": null}
rows after forgetting commit: 0
AttributeError: 'NoneType' object has no attribute 'find_all'
primary-key flags after to_sql replace: [0, 0]
```

1. Zip codes with a leading zero are read as integers, so `02134` becomes `2134`. With `dtype={"zip": str}` they survive.
2. `json.dumps` refuses NumPy integers (they are not Python `int`), and it writes `NaN` for a float nan, which is **not valid JSON** and breaks strict parsers. Convert to plain Python types and use `None` (JSON `null`) for missing values.
3. The insert was never committed before `close()`, so the reopened database has zero rows. The data is silently lost.
4. `find` returned `None` for the missing table; calling `.find_all` on it raised `AttributeError`.
5. `to_sql(if_exists="replace")` drops the table and recreates it from the DataFrame, so the primary key and any constraints are gone (flags 0 and 0, where the original had 1 on `id`).

Common mistakes, with symptom and fix

Three more are shown earlier: `yaml.load` (20.3), no `timeout` (20.7) and secrets in logs (20.11).

COMMON MISTAKE 1: ENCODING AND NEWLINES	**Wrong:** `open("export.csv")` then `csv.reader(f)`. **Symptom:** accented names turn into `Ã©` or the script crashes on another computer; on Windows, blank lines appear between rows. The default encoding depends on the machine. **Fix:** `open("export.csv", newline="", encoding="utf-8")` (or the encoding the provider documents).
COMMON MISTAKE 2: SQL BUILT WITH AN F-STRING	**Wrong:** `con.execute(f"select * from users where name = '{name}'")`. **Symptom:** a name such as `O'Brien` crashes the query, and a crafted name returns or destroys other rows (SQL injection). **Fix:** `con.execute("select * from users where name = ?", (name,))`.
COMMON MISTAKE 3: FORGETTING TO COMMIT	**Wrong:** `con.execute("insert ..."); con.close()`. **Symptom:** the script prints no error, but the next run finds an empty table. **Fix:** `with con: con.execute("insert ...")` (commits on success, rolls back on error). Remember that `with con` does not close the connection; call `con.close()` yourself, or use `contextlib.closing`.
COMMON MISTAKE 4: CODES READ AS NUMBERS	**Wrong:** `pd.read_csv("zips.csv")`. **Symptom:** zip code `02134` becomes `2134`, a product id `1E5` becomes the float 100000.0, and a join to another table silently matches nothing. **Fix:** `pd.read_csv("zips.csv", dtype={"zip": str})`. Identifiers are labels, not numbers.
COMMON MISTAKE 5: PAGE-NUMBER PAGINATION ON LIVE DATA	**Wrong:** `for page in range(1, 50): get(url, params={"page": page})`. **Symptom:** the final table has duplicates and some missing rows, and the counts differ on every run. **Fix:** use the cursor or `updated_since` parameter when the API offers one, and de-duplicate on the primary key with an upsert.
COMMON MISTAKE 6: TRUSTING THAT THE ELEMENT EXISTS	**Wrong:** `soup.find("span", class_="price").text`. **Symptom:** `AttributeError: 'NoneType' object has no attribute 'text'` the day the site changes its layout, or on one product with no price. **Fix:** a helper such as `text_or_none` from 20.8, counting how many items had missing fields, and failing the job if the missing share suddenly jumps.
COMMON MISTAKE 7: TO_SQL WITH REPLACE AND AN INDEX COLUMN	**Wrong:** `df.to_sql("orders", con, if_exists="replace")`. **Symptom:** the table loses its primary key, indexes and column types, and an extra `index` column appears. **Fix:** create the table yourself with constraints, then `df.to_sql("orders", con, if_exists="append", index=False)`, or load a staging table and upsert (Recipe 8).
COMMON MISTAKE 8: LOADING EVERYTHING, COMMITTING EVERYTHING	**Wrong:** `df = pd.read_csv("40_million_rows.csv")`, or a commit inside a row loop. **Symptom:** `MemoryError` or the process killed by the operating system; a load that takes hours. **Fix:** `chunksize=`, `usecols=` and `dtype=` when reading; one transaction per batch of thousands of rows when writing.
COMMON MISTAKE 9: RETRYING A POST THAT IS NOT IDEMPOTENT	**Wrong:** a retry policy that repeats `POST /payments` after a timeout. **Symptom:** the first request had actually succeeded, so the customer is charged twice. **Fix:** retry only GET, PUT and DELETE by default; for POST send an idempotency key header if the API supports one, or look up the result before repeating.

COMMON MISTAKE 10: SWALLOWING EVERY ERROR

Wrong: `try: load(row) except Exception: pass`. **Symptom:** the job reports success while half the rows are missing, and nobody can tell why. **Fix:** catch the specific exceptions you expect, log the reason with the record key, count rejects, and fail the run when the reject rate passes a threshold.

20.14 Cross-library integration

The skill of data access is hand-offs. Each library hands a plain Python object to the next, and the cleanest hand-offs use the simplest types: `list[dict]`, `DataFrame`, `bytes` and `str`.

From	To	Hand-off
`requests` / `httpx`	pandas	`pd.DataFrame(r.json()["items"])` or `pd.json_normalize(r.json())`
BeautifulSoup	pandas	list of row lists, then `pd.DataFrame(rows, columns=header)`
`sqlite3` / SQLAlchemy	pandas	`pd.read_sql_query(sql, con, params=...)`
pandas	SQL	`df.to_sql(name, con, if_exists="append", index=False)`
pandas	Excel	`with pd.ExcelWriter(p) as w: df.to_excel(w, ...)`, then openpyxl for styling
pandas	Parquet	`df.to_parquet(p)` and `pd.read_parquet(p, columns=...)`
files (csv/json/yaml)	validation	dict per record into a dataclass or pydantic model
validation	database	validated objects to tuples (`astuple`) for `executemany`
`logging`	everything	one logger per module; log counts at every stage boundary
`pathlib`	everything	pass `Path` objects; pandas, openpyxl, sqlite3 and `open` accept them

Worked example: HTML + API + SQL in one logged pipeline

```
import json, logging, sqlite3, sys
import pandas as pd
from bs4 import BeautifulSoup

logging.basicConfig(stream=sys.stdout, level=logging.INFO, format="%(levelname)s %(message)s", force=True)
log = logging.getLogger("pipeline")

HTML = ("<table><tr><th>sku</th><th>price</th></tr><tr><td>A</td><td>$1.50</td></tr>"
        "<tr><td>B</td><td>$2.00</td></tr><tr><td>C</td><td>oops</td></tr></table>")
API = '[{"sku": "A", "units": 10}, {"sku": "B", "units": 4}, {"sku": "D", "units": 7}]'

def extract_prices(html):
    rows = [[c.get_text(strip=True) for c in tr.find_all(["th", "td"])]
            for tr in BeautifulSoup(html, "html.parser").find_all("tr")]
    df = pd.DataFrame(rows[1:], columns=rows[0])
    df["price"] = pd.to_numeric(df["price"].str.lstrip("$"), errors="coerce")
    return df

prices = extract_prices(HTML)
units = pd.DataFrame(json.loads(API))
log.info("extracted %d price rows and %d unit rows", len(prices), len(units))

bad = prices[prices["price"].isna()]
if len(bad):
    log.warning("dropping %d price rows without a valid price: %s", len(bad), bad["sku"].tolist())
prices = prices.dropna(subset=["price"])

merged = prices.merge(units, on="sku", how="outer", indicator=True)
log.info("join result: %s", merged["_merge"].astype(str).value_counts().sort_index().to_dict())
both = merged[merged["_merge"] == "both"].drop(columns="_merge")

con = sqlite3.connect(":memory:")
both.to_sql("revenue_base", con, index=False)
out = pd.read_sql_query("select sku, price * units as revenue from revenue_base order by sku", con)
log.info("loaded %d rows", len(out))
print(out)
```

```
INFO extracted 3 price rows and 3 unit rows
WARNING dropping 1 price rows without a valid price: ['C']
INFO join result: {'both': 2, 'right_only': 1}
INFO loaded 2 rows
  sku  revenue
0   A     15.0
1   B      8.0
```

1. Two sources: a scraped HTML table and an API's JSON text. Both are inline strings, so the example runs anywhere.
2. `extract_prices` turns table rows into a DataFrame. `to_numeric(errors="coerce")` converts the bad price `oops` into `NaN` instead of failing.
3. The invalid row is logged as a WARNING with the sku (not silently dropped), then removed.
4. An outer merge with `indicator=True` adds a `_merge` column that says whether each key was in both tables, left only or right only. Logging the counts is a cheap, powerful check on a join: sku D has units but no price, and C was dropped earlier.
5. Only the matched rows go to SQLite with `to_sql`; the revenue is computed in SQL and read back with `read_sql_query`.
6. The log lines are the audit trail of one run: how much came in, how much was dropped, how many joined, how many were loaded.

20.15 How to solve any data-access problem: the playbook

The decision flow

1. **Where does it live?** Prefer a database or API, then a file download, then scraping.
2. **How big?** Fits in about a quarter of RAM: read whole. Otherwise chunk, stream, or use a columnar format.
3. **How does it change?** Never: load once. Otherwise pick watermark, cursor or changed-since.
4. **How much do you trust it?** Validate at the boundary and keep rejects.
5. **What can fail?** Network (timeouts, retries, 429), format (encoding, delimiter, schema drift), process (use transactions and idempotent writes).
6. **How will you know it worked?** Counts, sums and key sets compared between source and target.

Seven recurring problem patterns, each solved

Problem 20.15.1 The file will not parse. A supplier file raises `UnicodeDecodeError` at byte 2 and then loads as one column. Fix it.

Diagnose	Print `open(p, "rb").read(200)`. UTF-8 is wrong; semicolons mean the delimiter is not a comma.
Fix	`pd.read_csv(p, sep=";", encoding="cp1252", decimal=",")` (Recipe 6).
Verify	Column count, no `?` or `Ã` in text, a numeric sum against the supplier's total.

Problem 20.15.2 Too big for memory. A 40,000,000-row CSV must be summarised by region. How many chunks with `chunksize=500_000`, and what do you keep?

Count	40,000,000 / 500,000 = 80 chunks.
Keep	Only a running dictionary of region totals (20.11, Example 4). Add `usecols` and a `category` dtype, or convert once to Parquet.
Limit	Sum, count, min and max combine across chunks; a median does not.

Problem 20.15.3 The request fails sometimes. An API returns 503 about one call in ten. You allow 5 retries, base delay 0.5 s, doubling. What is the longest total wait without jitter?

Delays	0.5, 1, 2, 4, 8 seconds, total 15.5 seconds.
Add	Jitter (each wait between 0 and its value), a total deadline, and a log line per retry.
Never retry	400, 401, 403, 404.

Problem 20.15.4 I only want what is new. The source has 2,000,000 orders and about 3,000 change per day. What does a nightly watermark load touch?

Work	About 3,000 rows plus the boundary row and any overlap, against 2,000,000 for a full reload: 3,000 / 2,000,000 = 0.15 per cent.
Needs	An indexed `updated_at` in the source, an upsert in the target, a state table (Recipe 4).
Blind spot	Deletes. Ask for a soft-delete flag or compare keys periodically.

Problem 20.15.5 Nested and messy JSON. Each of 1,000 orders holds 1 to 6 lines. What shape does `json_normalize(orders, record_path="lines", meta=["id"])` give?

Shape	One row per line: between 1,000 and 6,000 rows; line fields plus `id` as columns.
Trap	Orders with an empty list vanish (Recipe 9); join the parent table back if needed.

Problem 20.15.6 Garbage in. Validation rejects 120 of 10,000 records. What is the reject rate, and what should the job do?

Rate	120 / 10,000 = 1.2 per cent.
Action	Load 9,880, write 120 to a rejects file with reasons, log the rate, and stop with an alert above a threshold (say 5 per cent): a jump means the source changed.

Problem 20.15.7 Writes are slow. A load commits once per row for 1,000,000 rows. You commit every 10,000 rows instead. How many commits are saved?

Count	1,000,000 before; 1,000,000 / 10,000 = 100 after; 999,900 saved, each of them a disk sync.
Mind	`executemany` per batch inside one transaction, and an idempotent load so a failed batch can be repeated. If the source changes shape (`total` becomes `amount`), required-field validation catches it instead of loading NULLs.

How to verify a load

- **Counts and totals:** rows read = rows loaded + rows rejected; one numeric sum matches in source and target.
- **Keys and samples:** compare id sets in both directions; eyeball 3 random rows from each side.
- **Idempotence:** run the job twice; the second run must change nothing.

How to debug

- **Look at the raw thing:** `r.status_code` , `r.text[:300]` , the first bytes of a file, `repr()` of a suspicious string.
- **Shrink the problem:** one page, one file, `nrows=5` . **Turn on detail:** `logging.getLogger("urllib3").setLevel(logging.DEBUG)` .
- **Ask the database:** `explain query plan` says whether an index is used.

```
import sqlite3

con = sqlite3.connect(":memory:")
con.execute("create table events(id integer primary key, user_id integer, kind text)")
con.executemany("insert into events(user_id, kind) values (?, ?)", [(i % 100, "click") for i in range(5000)])
q = "select count(*) from events where user_id = ?"

def plan():
    return con.execute("explain query plan " + q, (7,)).fetchall()[0][3]

print(plan())
con.execute("create index ix_events_user on events(user_id)")
print(plan())
print(con.execute(q, (7,)).fetchone()[0])
print(repr("caf\u00e9\u00a0x"), repr("  padded \n"))
src_ids, dst_ids = {1, 2, 3, 4}, {1, 2, 4, 9}
print("missing in target:", sorted(src_ids - dst_ids), "| unexpected in target:", sorted(dst_ids - src_ids))
```

```
SCAN events
SEARCH events USING COVERING INDEX ix_events_user (user_id=?)
50
'café\xa0x' '  padded \n'
missing in target: [3] | unexpected in target: [9]
```

1. The first plan scans the whole table; after `create index` the plan searches the index and touches only matching rows. Reading a plan is the quickest way to explain a slow query. The count for user 7 is 50.
2. `repr` shows a non-breaking space as `\xa0` and the newline as `\n` ; both are invisible in `print` and a classic reason why two "equal" keys do not match.
3. Set differences in both directions are the fastest reconciliation.

20.16 Going deeper: internals, memory, speed and what interviewers ask

Internals worth knowing

- **sqlite3 transactions.** In the default (legacy) mode Python opens a transaction before `INSERT`, `UPDATE` and `DELETE`, so you must commit; since Python 3.12 `connect(autocommit=...)` gives explicit control. SQLite allows many readers but one writer; WAL mode (`pragma journal_mode = wal`) lets readers continue while a writer works. Typing is by *affinity*: a declared type is a preference unless the table is `STRICT` (below).
- **requests and httpx.** `requests` sits on urllib3, which keeps a connection pool per host; a `Session` reuses it (HTTP keep-alive). `requests` blocks one thread per reply; `httpx.AsyncClient` overlaps thousands of waits in one thread. Threads also work for I/O because Python releases the GIL while waiting on a socket.
- **pandas 3.** Strings default to a `str` dtype (Arrow-backed when pyarrow is installed) and copy-on-write is always on: `sub = df[["a"]]` behaves as an independent copy, so changing `sub` never changes `df` and defensive `.copy()` calls are no longer needed.
- **Parquet and memory.** A file holds row groups, each with one compressed, dictionary-encoded column chunk per column, and a footer with the schema and min/max statistics. Repeated strings become millions of Python objects unless you use `category` or Arrow strings (20.10 showed about 5 MB shrink to 0.1 MB on the study machine).

```
import sqlite3

con = sqlite3.connect(":memory:")
con.execute("create table loose(n integer, s text)")
con.execute("insert into loose values ('42', 7)")
con.execute("insert into loose values ('abc', 'x')")
print(con.execute("select n, typeof(n), s, typeof(s) from loose").fetchall())

con.execute("create table strict_t(n integer) strict")
try:
    con.execute("insert into strict_t values ('abc')")
except sqlite3.IntegrityError as e:
    print("strict table says:", e)
```

```
[(42, 'integer', '7', 'text'), ('abc', 'text', 'x', 'text')]
strict table says: cannot store TEXT value in INTEGER column strict_t.n
```

1. The first table is normal. Inserting the text `'42'` into an integer column converts it to the integer 42, because it looks like a number. The integer 7 in a text column is stored as the text `'7'`.
2. The text `'abc'` cannot become an integer, so SQLite stores it as text in an integer column and does not complain. This is why a column typed INTEGER can still contain words.
3. A `STRICT` table (SQLite 3.37 and later) refuses such values with an error. Use it for tables where type safety matters.

What interviewers probe

- **Idempotency:** "What if your job runs twice?" Upsert on a natural key, a transaction per batch, the watermark advanced in the same transaction. Retries give at-least-once delivery; add idempotent writes for an exactly-once effect.
- **Late data and deletes:** overlap windows, event time versus ingestion time, periodic reconciliation; a watermark cannot see deletes (soft deletes or change data capture can).
- **Pagination:** offset versus cursor, and why a large offset gets slow (the database still walks past the skipped rows).
- **Security and failure design:** parameter binding, secrets in the environment, least-privilege users; timeouts everywhere, retry only the retryable, backoff with jitter, dead-letter files.
- **Formats and scale:** why Parquet over CSV (types, compression, column pruning, predicate pushdown), when CSV is still right, and what changes at 10x data (chunking, columnar files, a database server, a distributed engine).

Chapter recap

- Every data-access job is extract, validate, transform, load. Validate once, at the boundary.
- `pathlib` and `glob` find files; write results through a temporary file and `replace`. Give `open` an `encoding` (and `newline=""` for csv).
- CSV has no types, JSON has no dates, YAML needs `safe_load`. openpyxl never calculates formulas.
- sqlite3: bind with `?`, use `with con:`, batch with `executemany`, upsert with `on conflict`. SQLAlchemy Core builds SQL, the ORM maps classes, `text()` binds `:name`.
- pandas: `read_sql_query(params=, chunksize=)` in, `to_sql(if_exists="append", index=False)` out; `replace` destroys constraints.
- HTTP: always a timeout, a Session, `raise_for_status()`, backoff with jitter on retryable errors only, cursor pagination, respect 429 and `Retry-After`.
- BeautifulSoup parses static HTML; handle missing elements; read robots.txt and the terms; prefer an API.
- Validate with dataclasses or pydantic, keep rejects, alarm on a high reject rate. Parquet and Arrow (not run here) store typed columns with statistics so you read fewer bytes.
- Log counts at every stage, chunk what is large, and fix encoding errors by finding the right codec.

Formula / pattern	Meaning
`open(p, newline="", encoding="utf-8")`	Safe way to open text for csv
`con.execute(sql, (value,))`	Parameter binding, no injection
`with con: ...`	Commit on success, roll back on error
`insert ... on conflict(key) do update ...`	Upsert, makes loads idempotent
`pd.read_csv(p, usecols=, dtype=, chunksize=)`	Small, typed, bounded-memory reads
`requests.get(u, params=, headers=, timeout=)`	The minimum safe API call
delay = uniform(0, min(cap, base * 2^{n-1}))	Exponential backoff with full jitter
tokens = min(capacity, tokens + elapsed * rate)	Token bucket refill
`where updated_at >= watermark - overlap`	Incremental load that tolerates late rows
chunks = ceil(rows / chunksize)	Number of reads for a chunked job
reject rate = rejected / total	Data-quality alarm metric
`pd.json_normalize(recs, record_path=, meta=)`	Nested JSON to a flat table

Practice set

Problem 20.P.1 (easy) For `Path("data/2025/sales.csv")`, what are `.stem`, `.suffix` and `.parent.name`?

Answer: `sales`, `.csv` and `2025`.

Problem 20.P.2 (easy) What Python value does `json.loads('[1, 2, {"a": null}]')` return?

Answer: The list `[1, 2, {'a': None}]`. JSON `null` becomes `None`, objects become dicts, arrays become lists.

Problem 20.P.3 (easy) Which is safe and why: `con.execute("select * from u where name = ?", (n,))` or `con.execute(f"select * from u where name = '{n}'")`?

Answer: The first. The value is sent separately from the SQL text, so the database cannot mistake it for SQL. The f-string version lets a value such as `x' OR '1'='1` change the query.

Problem 20.P.4 (easy) Which of 401, 404, 429 and 503 are worth retrying, and which not?

Answer: Retry 429 (after the `Retry-After` delay) and 503 (the server is busy). Do not retry 401 (your credentials are wrong) or 404 (the resource does not exist); repeating gives the same answer.

Problem 20.P.5 (easy) What do `soup.find("table")` and `soup.find_all("table")` return when the page has no table?

Answer: `None` and an empty list `[]`. Calling a method on the `None` raises `AttributeError`.

Problem 20.P.6 (medium) What does this print?

```
from pathlib import Path
p = Path("/tmp/data/report.final.csv")
print(p.suffix, p.stem, p.with_suffix(".txt").name, p.with_name("x.csv").parent.name)
```

Answer: `.csv report.final report.final.txt data`. Only the last dot counts as the suffix, so the stem keeps `.final`. `with_suffix` swaps just `.csv`, and `with_name` keeps the folder.

Problem 20.P.7 (medium) What does this print, and why is the second line not `[(1,), (2,)]`?

```
import sqlite3
con = sqlite3.connect(":memory:")
con.execute("create table t(a integer unique)")
con.execute("insert into t values (1)")
con.commit()
try:
    with con:
        con.execute("insert into t values (2)")
        con.execute("insert into t values (1)")
except sqlite3.IntegrityError:
    print("error")
print(con.execute("select a from t").fetchall())
```

Answer: `error / [(1,)]`. The duplicate raises `IntegrityError` inside the `with` block, which rolls back the whole block, including the successful insert of 2.

Problem 20.P.8 (medium) Find the bug: `name = "O'Brien"; con.execute(f"select * from users where name = '{name}'")`.

Answer: The apostrophe ends the SQL string early, giving a syntax error, and the same hole allows injection. Fix: `con.execute("select * from users where name = ?", (name,))`.

Problem 20.P.9 (medium) Find the bug: `data = requests.get(url).json()` in a nightly job that sometimes hangs and sometimes crashes with `JSONDecodeError`.

Answer: No timeout (the hang) and no status check (the crash comes from an HTML error page). Fix: `r = requests.get(url, timeout=(3, 10)); r.raise_for_status(); data = r.json()`.

Problem 20.P.10 (medium) An API holds 95 records and returns 20 per page, and allows 1 request per second. How many requests are needed, how many records are on the last page, and what is the minimum time between the first and the last request?

Answer: ceil(95 / 20) = 5 requests. The last page has 95 - 4 x 20 = 15 records. Five requests at one per second are spread over at least 4 seconds (the gaps between them).

Problem 20.P.11 (medium) What does this print?

```
import json
print(json.dumps({"a": [1, 2], "b": None, "c": "é"}, ensure_ascii=False, separators=(",", ":")))
```

Answer: `{"a":[1,2],"b":null,"c":"é"}`. Compact separators remove spaces, `None` becomes `null`, and the accent stays because `ensure_ascii=False`.

Problem 20.P.12 (medium) Write a function that counts rows per value of a column in a CSV too large to load, using chunks. Test it on an in-memory CSV with 5 rows and `chunksize=2`.

Answer:

```
import io
import pandas as pd

def count_by(source, column, chunksize=2):
    totals = {}
    for chunk in pd.read_csv(source, usecols=[column], chunksize=chunksize):
        for key, n in chunk[column].value_counts().items():
            totals[key] = totals.get(key, 0) + int(n)
    return dict(sorted(totals.items()))

csv_text = "region,units\nn,1\ns,2\nn,3\ne,4\nn,5\n"
print(count_by(io.StringIO(csv_text), "region"))
```

```
{'e': 1, 'n': 3, 's': 1}
```

Only the one needed column is read (`usecols`), each chunk is counted on its own, and counts are added into one dictionary. Chunks of 2, 2 and 1 rows give the same answer as a full read.

Problem 20.P.13 (hard) What does this print?

```
from bs4 import BeautifulSoup
html = '<ul><li class="a">One <b>1</b></li><li class="b">Two</li><li class="a"><a href="/x">Three</a></li></ul>'
soup = BeautifulSoup(html, "html.parser")
print([li.get_text(strip=True) for li in soup.select("li.a")])
print(soup.select_one("li.a a")["href"], len(soup.find_all("li")), soup.select_one("li.c"))
```

Answer: `['One1', 'Three'] / /x 3 None`. The selector `li.a` matches the first and third items; `get_text(strip=True)` joins the nested text without a space, giving `One1`. The link href is `/x`, there are 3 list items, and the missing class gives `None`.

Problem 20.P.14 (hard) Your watermark is 12:30. A source row with `updated_at` 11:30 appears after the last run. With a 2-hour overlap, which times are re-read, and does the late row load? What protects you from duplicates?

Answer: The query starts at 12:30 - 2 h = 10:30, so the 11:30 row is read and loads. The upsert makes re-read rows harmless and `max(watermark, new)` stops the watermark moving backwards. The cost is re-reading two hours of data per run.

Problem 20.P.15 (hard) Find the bug: a loop `for page in range(1, 100): rows += get(url, params={"page": page})` reads a feed that receives new rows all day. The final data has duplicates and gaps.

Answer: New rows at the top push older rows onto later pages (duplicates), and deletions pull rows back across a boundary (gaps). Fix: cursor pagination (`after=last_id`) or an `as_of` snapshot, plus an upsert on the key.

Problem 20.P.16 (hard) Write a dataclass `Reading(sensor: str, celsius: float)` that rejects a temperature below -50 or above 60 and an empty sensor name, then test it with three records.

Answer:

```
from dataclasses import dataclass

@dataclass
class Reading:
    sensor: str
    celsius: float

    def __post_init__(self):
        self.celsius = float(self.celsius)
        if not self.sensor.strip():
            raise ValueError("sensor must not be empty")
        if not -50 <= self.celsius <= 60:
            raise ValueError(f"celsius {self.celsius} outside -50..60")

for rec in [("s1", "21.5"), ("", 20), ("s3", 99)]:
    try:
        print("ok", Reading(*rec))
    except ValueError as e:
        print("rejected:", e)
```

```
ok Reading(sensor='s1', celsius=21.5)
rejected: sensor must not be empty
rejected: celsius 99.0 outside -50..60
```

The chained comparison `-50 <= x <= 60` tests both limits at once; the string `"21.5"` is converted first, so text input is accepted when it is valid.

Problem 20.P.17 (hard) What does this print?

```
import tempfile
from pathlib import Path
from openpyxl import Workbook, load_workbook
p = Path(tempfile.mkdtemp()) / "t.xlsx"
wb = Workbook()
ws = wb.active
ws["A1"], ws["A2"], ws["A3"] = 2, 3, "=A1*A2"
wb.save(p)
print(load_workbook(p).active["A3"].value, load_workbook(p, data_only=True).active["A3"].value)
```

Answer: `=A1*A2 None`. The file stores the formula text and no cached result, because openpyxl does not calculate.

Problem 20.P.18 (hard) A file has 40 million rows and 3 float64 columns. How many bytes do the numbers need in memory, and how much for a chunk of 1 million rows?

Answer: 40,000,000 x 3 x 8 = 960,000,000 bytes, about 0.96 GB, before pandas overhead. A chunk of 1,000,000 rows needs 1,000,000 x 3 x 8 = 24,000,000 bytes, about 24 MB, and there are 40 chunks.

Problem 20.P.19 (expert) Show that an upsert-based load is idempotent: run the same load twice and report the row count and the total.

Answer:

```
import sqlite3

con = sqlite3.connect(":memory:")
con.execute("create table sales(id integer primary key, amount real)")
batch = [(1, 10.0), (2, 20.0), (3, 5.5)]
SQL = "insert into sales values (?, ?) on conflict(id) do update set amount = excluded.amount"
for run in (1, 2):
    with con:
        con.executemany(SQL, batch)
    print(run, con.execute("select count(*), sum(amount) from sales").fetchone())
batch[1] = (2, 25.0)
with con:
    con.executemany(SQL, batch)
print(3, con.execute("select count(*), sum(amount) from sales").fetchone())
```

```
1 (3, 35.5)
2 (3, 35.5)
3 (3, 40.5)
```

Running the load a second time leaves the count at 3 and the total at 35.5, because each key is overwritten with the same values. A changed record (id 2 now 25.0) updates in place instead of creating a duplicate: the total becomes 40.5. A plain `insert` would fail on the second run (primary key) or, without a key, double the rows.

Problem 20.P.20 (expert) Interview: a 500-million-row events table is queried mostly by date range and country, and rarely needs more than 4 of its 40 columns. Argue for Parquet over CSV and describe how to lay it out.

Answer: (1) Column pruning: 4 of 40 columns reads roughly a tenth of the bytes, while CSV parses every line. (2) Types and compression: dictionary encoding of repeated values such as country, no re-parsing of numbers. (3) Row-group statistics: with data partitioned or sorted by date, a date filter skips whole groups or folders. Layout: partition by year and month, keep files of hundreds of megabytes rather than thousands of tiny ones, and keep CSV only for hand-off. Not run here (pyarrow is not installed); the principle was demonstrated in 20.10.

Problem 20.P.21 (expert) A scheduler retried a timed-out job, so a table that should have unique ids contains duplicates. Repair it, and prevent it from happening again.

Answer:

```
import sqlite3

con = sqlite3.connect(":memory:")
con.execute("create table events(id integer, kind text)")
con.executemany("insert into events values (?, ?)", [(1, "a"), (2, "b"), (2, "b"), (3, "c"), (1, "a")])
print(con.execute("select count(*) from events").fetchone()[0])
with con:
    con.execute("delete from events where rowid not in (select min(rowid) from events group by id)")
    con.execute("create unique index ux_events_id on events(id)")
print(con.execute("select * from events order by id").fetchall())
try:
    con.execute("insert into events values (2, 'b')")
except sqlite3.IntegrityError as e:
    print("blocked:", e)
```

```
5
[(1, 'a'), (2, 'b'), (3, 'c')]
blocked: UNIQUE constraint failed: events.id
```

Repair: keep the first row (smallest `rowid`) of each id and delete the others. Prevention at three levels: a unique index (the database now refuses duplicates), upserts in the load instead of plain inserts, and a run identifier or idempotency key so a retried run is recognised as the same run.

Problem 20.P.22 (expert) Find the bug: the job writes `etl_state.watermark = max(updated_at)` and commits, and then loads the rows in a second transaction. It crashed during the second step.

Answer: The watermark moved forward but the rows never arrived, so the next run starts after them and they are lost. Fix: update rows and watermark in the *same* transaction (Recipe 4).

Prompt pack

AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data roles. Teach me data access in Python step by step: pathlib and glob, csv/json/yaml, openpyxl, sqlite3 with parameter binding and transactions, requests and httpx with timeouts, retries and pagination, BeautifulSoup, and validation. Use small runnable examples that need no internet (inline data, a local http.server, SQLite in memory), show the real output, and explain each line in numbered points. After each topic, ask me one quiz question and wait for my answer before continuing. Start with a diagnostic question to find my level.
```

AI prompt: copy and paste into any AI

```
Act as a senior data engineer doing a code review. I will paste a Python script that reads data from a CSV, an API or a web page and loads it into a database. Review it for: missing timeouts and status checks, retry and backoff problems, SQL injection and parameter binding, missing commits or transactions, encoding and newline mistakes, schema validation, idempotency (what happens if it runs twice), memory use on big files, logging and secrets. List each problem with the line, why it fails in real life, and the corrected code. Then give me a checklist of three tests I should write.
```

AI prompt: copy and paste into any AI

```
Run a mock interview for a data engineer or analytics engineer role, focused on getting data in and out of systems. Ask me one question at a time, from easy to hard: parameter binding, transactions, upserts, incremental loads with watermarks and late data, API pagination and rate limits, idempotency and retries, Parquet versus CSV, handling files that do not fit in memory, and data validation. After each answer, grade it out of 10, show a model answer with a short code example, and note what a strong candidate would add. Keep a running score and finish with my three weakest topics.
```

CHAPTER 21

DEEP LEARNING AND GENAI LIBRARIES: PYTORCH, HUGGING FACE AND THE LLM TOOLCHAIN

Library Handbook: tensors, training loops, transformers, embeddings and LLM clients.

Imagine you join a software company's support team as the first data person. There are 40,000 old tickets in a spreadsheet and 300 new ones arrive every day. Your manager wants three things: (1) label each new ticket as *billing*, *bug* or *how-to* automatically, (2) show the agent the five most similar old tickets, and (3) draft a first reply with a large language model (LLM) and refuse to send it if it is not valid JSON. Each of those three wishes is a different library family. The labeller is a neural network trained with **PyTorch**, or a pretrained model fine-tuned with **Hugging Face**. The "similar tickets" feature is **embeddings plus vector search** (sentence-transformers, FAISS or Chroma). The reply drafter is an **LLM SDK call** wrapped in retries, streaming and output checks. Glue and measurement come from orchestration code and an **evaluation harness**.

This chapter is a handbook, not a theory course. For each library you get a reference table of the functions you will actually use, worked examples that climb from trivial to realistic, a list of traps, and a playbook for solving problems you have not seen. One practical difficulty shapes the whole chapter: `torch`, `transformers`, `faiss` and the vendor SDKs are **not installed** on the machine that produced this book, and some need a GPU or a network key anyway. So every library appears twice. First you see idiomatic code labelled **not run here** (it is the code you would paste into a real project, written from the documented APIs). Then you see a **runnable equivalent** built from NumPy, scikit-learn and the standard library that demonstrates the same mechanics, with real printed output. When a mechanism is understood by hand (autograd, an optimiser step, a retry loop, a nearest-neighbour search), the library call stops being magic.

By the end you will be able to: read and write a PyTorch training loop and know what every line does; debug shape, device and memory errors in a systematic way; tokenise text and explain what a Hugging Face `Trainer` does; build and evaluate a retrieval system; call an LLM API with retries, streaming and structured output; and judge whether your GenAI feature actually works with a small evaluation harness. For the maths behind gradients see the Maths document, Chapter 13 (backpropagation) and Chapter 14 (optimisation). For concepts such as attention and fine-tuning in depth, see the Deep Learning and GenAI documents. Here we focus on the *libraries and the code*.

Words you need. A **tensor** is an n-dimensional array that can also remember how it was computed. A **gradient** is the slope of the loss with respect to a number, telling you which way to nudge it. A **token** is a small piece of text (a word or part of a word) mapped to an integer id. An **embedding** is a list of numbers that places a piece of text (or an image) at a point in space so that similar things sit close together. An **LLM** is a very large model that continues text. **Inference** means using a trained model, **training** means changing it.

21.1 What These Libraries Are, When to Use Them, and the Mental Model

The libraries in one table

Library	What it is	Use it when	Do NOT use it when
PyTorch (`torch`)	Tensors with automatic differentiation, neural network building blocks, optimisers, GPU support	You train or fine-tune neural networks, or need custom losses and architectures	Tabular data with a few thousand rows: gradient boosting or logistic regression is usually faster, simpler and as accurate
`transformers`	Pretrained models, tokenisers and a `Trainer` for text, vision and audio	You want a pretrained model for classification, extraction, generation or embeddings	A regex or a TF-IDF plus logistic regression already solves the task at the needed accuracy
`datasets`	Memory-mapped, cached tables with fast `map`, `filter` and streaming	Data is too big for pandas RAM or you feed a Hugging Face model	The data fits in a pandas DataFrame and you only do analysis
`tokenizers`	Fast tokeniser engine (Rust) behind `transformers`	You train your own vocabulary or need exact offsets	You use a standard pretrained model: load its own tokeniser instead

Library	What it is	Use it when	Do NOT use it when
`peft`	Parameter-efficient fine-tuning (LoRA and relatives)	The base model is too big to fine-tune fully on your hardware	The model is small enough to fine-tune fully, or prompting already works
`sentence-transformers`	Models that turn sentences into embedding vectors	Semantic search, clustering, deduplication, retrieval for RAG	Exact keyword matches matter most (use BM25 or a database `LIKE`)
FAISS	Fast nearest-neighbour search over millions of vectors	Vectors live in memory and you need speed	Under about 100,000 vectors: a NumPy matrix product is enough
Chroma	A small vector database with metadata filters and persistence	Prototypes and small apps that need filtering plus storage	Large multi-user production search (use a managed vector store or pgvector)
OpenAI / Anthropic SDKs	Typed clients for hosted LLM APIs	You want a strong model without hosting it	Data may not leave your network, or the cost per call is too high at your volume
LangChain-style orchestration	Composable prompt, model, parser chains and agent loops	Many steps, several providers, tool calls	One prompt and one call: plain SDK code is clearer

KEY IDEA

The right first question is not "which deep learning library?" but "do I need a neural network at all?". A model you do not train, a rule, or a linear model are all valid answers. Reach for these libraries when the input is text, images, audio or sequences, or when you need to adapt a pretrained model.

Install, version check and import conventions

Install in a virtual environment (see Chapter 9). PyTorch has a choice of builds (CPU only, or a specific CUDA version), so copy the exact command from the selector on pytorch.org rather than guessing. Not run here:

```
# create and activate an environment first (Chapter 9)
python3 -m venv .venv && source .venv/bin/activate
pip install torch                       # use the command from pytorch.org for your CUDA/CPU build
pip install transformers datasets tokenizers peft accelerate
pip install sentence-transformers faiss-cpu chromadb
pip install openai anthropic langchain-core
python -c "import torch; print(torch.__version__, torch.cuda.is_available())"
```

The conventional imports are below. Learn them: every tutorial and every colleague uses exactly these names.

```
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
from transformers import AutoTokenizer, AutoModelForSequenceClassification
from datasets import load_dataset
from peft import LoraConfig, get_peft_model
from sentence_transformers import SentenceTransformer
import faiss
import chromadb
from openai import OpenAI
import anthropic
```

A version check that works whether or not a library exists is a handy first line in any bug report. This one runs here and reports honestly what the machine has:

```
import importlib.util
import importlib.metadata as md

def report(pairs):
    for import_name, pip_name in pairs:
        if importlib.util.find_spec(import_name) is None:
            print(f"{import_name:14s} not installed")
        else:
            try:
                print(f"{import_name:14s} {md.version(pip_name)}")
            except md.PackageNotFoundError:
                print(f"{import_name:14s} installed (no metadata)")

report([("torch", "torch"), ("transformers", "transformers"), ("faiss", "faiss-cpu"),
        ("openai", "openai"), ("anthropic", "anthropic"),
        ("numpy", "numpy"), ("sklearn", "scikit-learn")])
```

```
torch          not installed
transformers   not installed
faiss          not installed
openai         not installed
anthropic      not installed
numpy          2.5.3
sklearn        1.9.1
```

1. `find_spec` asks Python whether an import would succeed without actually importing the (possibly slow) package. It returns `None` when the package is missing.
2. `md.version` reads the version from the installed package metadata. This is the same number `pip list` shows.
3. The *import name* and the *pip name* can differ: you `pip install scikit-learn` but `import sklearn`, and `faiss-cpu` imports as `faiss`. So each pair holds both names, and we catch `PackageNotFoundError` in case the metadata is stored under another name.
4. The output shows the first five are absent on this machine, while NumPy and scikit-learn are present. That is the reason every library below comes with a runnable stand-in. Your versions will differ; always print them in a bug report.

The mental model

Every deep learning library is a variation on one loop. You hold **parameters** (numbers), you run **data** forward through a function to get a **prediction**, you compare it with the truth to get a single number called the **loss**, you compute the **gradient** of the loss with respect to each parameter (the backward pass), and an **optimiser** nudges the parameters. Repeat. The GenAI libraries add two more pictures: text becomes **token ids** then **embeddings**, and calling a hosted model is a network request that can fail, stream and return text that you must **validate**.

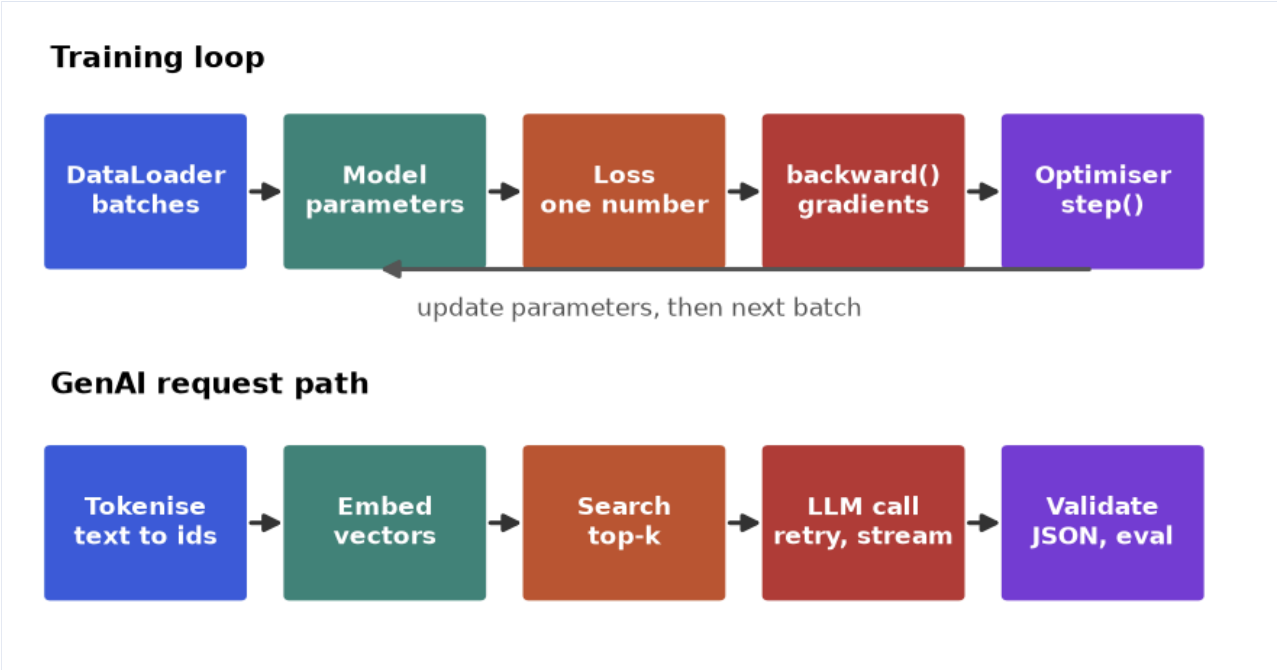


Figure 21.1. The training loop (top row) and the GenAI request path (bottom row) are the two shapes every program in this chapter fits into.

How to read it: follow the top row left to right for one training step; the arrow from the optimiser back to the model is the only place the model changes. In the bottom row nothing is trained: text is turned into numbers, the closest stored items are retrieved, and a hosted model writes the answer. Whenever you get lost in a library, ask "which box am I in?".

REAL JOB	Most production time goes to the boxes that are not glamorous: getting batches to the model with the right shape and dtype, keeping memory under the limit, retrying failed API calls and measuring whether answers are good. This chapter spends most of its pages there on purpose.

21.2 PyTorch Part 1: Tensors and Autograd

INTUITION	A PyTorch tensor is a NumPy array with two extra powers: it can live on a GPU, and it can *remember the operations that created it*. That memory is a recorded recipe (a graph). When you call `.backward()` PyTorch walks the recipe backwards and fills in the slope of the loss for every number you marked as trainable. Think of a cooking show: the forward pass cooks the dish while writing the recipe, the backward pass reads the recipe in reverse to say how much each ingredient mattered.

Reference: the 24 tensor functions you will use daily

Name	What it does in plain English	Minimal example (not run here)
`torch.tensor`	Make a tensor from Python numbers (copies the data)	`torch.tensor([[1., 2.], [3., 4.]])`
`torch.zeros / ones / full`	Filled tensors of a given shape	`torch.zeros(2, 3)`
`torch.arange / linspace`	Evenly spaced values	`torch.arange(0, 10, 2)`
`torch.manual_seed`	Fix the random generator so runs repeat	`torch.manual_seed(0)`
`torch.randn / rand / randint`	Random normal, uniform, integers	`torch.randn(4, 3)`
`torch.from_numpy / .numpy()`	Share memory with a NumPy array (CPU only)	`torch.from_numpy(arr)`

Name	What it does in plain English	Minimal example (not run here)
`x.shape, x.dtype, x.device`	The three facts to check first when anything breaks	`print(x.shape, x.dtype, x.device)`
`x.to(...)`	Move to a device and/or change dtype	`x.to("cuda")` , `x.to(torch.float16)`
`x.view / x.reshape`	Same numbers, new shape (`view` needs contiguous memory)	`x.reshape(-1, 4)`
`x.permute / x.transpose`	Reorder dimensions (a view, often non-contiguous)	`x.permute(0, 2, 1)`
`x.unsqueeze / x.squeeze`	Add or remove a length-1 dimension	`x.unsqueeze(0)`
`torch.cat / torch.stack`	Join along an existing / a new dimension	`torch.stack([a, b])`
`@ / torch.matmul`	Matrix product, batched when there are leading dimensions	`a @ b`
`torch.einsum`	Any product or reduction written as index letters	`torch.einsum("bij,bjk->bik", a, b)`
`x.sum / mean / max (dim, keepdim)`	Reduce along a dimension	`x.sum(dim=1, keepdim=True)`
`x.argmax`	Index of the largest value (turn logits into a class)	`logits.argmax(dim=-1)`
`torch.softmax`	Turn scores into probabilities along a dimension	`torch.softmax(z, dim=-1)`
`torch.where / masked_fill`	Choose elementwise / overwrite where a mask is true	`x.masked_fill(mask, -1e9)`
`x.gather`	Pick one entry per row using index tensor	`logp.gather(1, y.unsqueeze(1))`
`x.item()`	One-element tensor to a Python number	`loss.item()`
`x.requires_grad_()`	Start recording operations on this tensor	`w = torch.randn(3, requires_grad=True)`
`loss.backward()`	Fill `.grad` for every tracked leaf	`loss.backward(); w.grad`
`torch.no_grad()`	Stop recording (evaluation, inference)	`with torch.no_grad(): y = model(x)`
`x.detach()`	Same data, cut from the graph	`x.detach().cpu().numpy()`
`x.clone()`	Real copy (otherwise slices share memory)	`y = x.clone()`

The names and ideas are the same as in NumPy, so we first practise the mechanics where we can run them. The first example covers shapes, dtypes, and the most important distinction in the whole chapter: a **view** shares memory, a **copy** does not.

Problem 21.2.1 Build a batch of 2 tickets, each with 3 tokens, each token a 4-number vector. Reshape it, take a slice, and prove whether the slice shares memory with the original.

Idea	NumPy plays the part of `torch` : `reshape` , slicing and transposing return views; `copy()` returns new memory. PyTorch behaves the same way.
Check	`np.shares_memory(a, b)` answers the question directly. In PyTorch compare `a.data_ptr()` or `a.untyped_storage().data_ptr()` .

```
import numpy as np

x = np.arange(24, dtype=np.float32).reshape(2, 3, 4)   # (batch, tokens, features)
print("shape", x.shape, "dtype", x.dtype, "bytes", x.nbytes)

flat = x.reshape(6, 4)            # merge batch and tokens
first_ticket = x[0]               # a slice
copy = x[0].copy()
swapped = x.transpose(0, 2, 1)    # (batch, features, tokens)

print("flat shares memory:", np.shares_memory(x, flat))
print("slice shares memory:", np.shares_memory(x, first_ticket))
print("copy shares memory:", np.shares_memory(x, copy))
print("transposed shape:", swapped.shape, "contiguous:", swapped.flags["C_CONTIGUOUS"])

first_ticket[0, 0] = 99.0          # write through the view
print("x[0,0,0] is now", x[0, 0, 0], "and copy[0,0] is still", copy[0, 0])
```

```
shape (2, 3, 4) dtype float32 bytes 96
flat shares memory: True
slice shares memory: True
copy shares memory: False
transposed shape: (2, 4, 3) contiguous: False
x[0,0,0] is now 99.0 and copy[0,0] is still 0.0
```

1. Line 3 builds 24 numbers and shapes them as `(2, 3, 4)`. Read shapes left to right as "2 tickets, 3 tokens, 4 features". `float32` uses 4 bytes per number, so 24 numbers need 96 bytes.
2. Lines 7 to 10 create four relatives of `x`. `reshape` and a slice are views. `.copy()` allocates new memory. `transpose` only changes how the same memory is walked (the strides), so it is also a view.
3. The three `shares_memory` lines print `True`, `True`, `False`. A view costs nothing but is dangerous if you modify it.
4. The transposed array is no longer C-contiguous (`False`). In PyTorch this is exactly the case where `.view()` raises an error and `.reshape()` or `.contiguous()` is needed.
5. The last two lines write 99.0 through the slice. The original changes too, while the copy keeps the old value 0.0.

Now the PyTorch version of the same ideas. This is what you would run on a machine with torch installed (not run here):

```
import torch

torch.manual_seed(0)
x = torch.arange(24, dtype=torch.float32).reshape(2, 3, 4)
print(x.shape, x.dtype, x.device, x.element_size() * x.nelement())

flat = x.reshape(6, 4)             # view when possible
copy = x[0].clone()                # real copy
swapped = x.transpose(1, 2)        # (2, 4, 3), non-contiguous
print(swapped.is_contiguous())     # False
try:
    swapped.view(8, 3)             # fails: view needs contiguous memory
except RuntimeError as e:
    print("view failed:", str(e)[:60])
ok = swapped.reshape(8, 3)         # reshape silently copies when it has to
ok2 = swapped.contiguous().view(8, 3)
```

Broadcasting, reductions and a stable softmax

Broadcasting is the rule that lets arrays of different shapes combine. Line the shapes up from the *right*; two dimensions are compatible when they are equal or one of them is 1; a missing dimension counts as 1. It saves memory and loops, and it silently produces nonsense if you get a shape wrong, which is why the shape-debugging guide in 21.6 matters.

Problem 21.2.2 Scores for 3 tickets over 4 classes are given. Convert them to probabilities with a numerically safe softmax, using keepdims, and show why subtracting the row maximum matters.

Formula	softmax(z)$_i$ = exp(z$_i$) / sum$_j$ exp(z$_j$). Subtracting any constant c from every z$_i$ leaves the answer unchanged.
Trap	`exp(1000)` overflows to infinity. Subtract the row maximum first so the largest exponent is `exp(0) = 1`.

```
import numpy as np

z = np.array([[2.0, 1.0, 0.1, -1.0],
              [1000.0, 999.0, 998.0, 0.0],
              [0.0, 0.0, 0.0, 0.0]], dtype=np.float64)

def softmax_naive(z):
    e = np.exp(z)
    return e / e.sum(axis=1, keepdims=True)

def softmax_stable(z):
    z = z - z.max(axis=1, keepdims=True)      # (3,4) - (3,1) broadcasts
    e = np.exp(z)
    return e / e.sum(axis=1, keepdims=True)

with np.errstate(over="ignore", invalid="ignore"):
    print("naive row 1:", softmax_naive(z)[1])
print("stable row 1:", np.round(softmax_stable(z)[1], 4))
p = softmax_stable(z)
print("rows sum to", p.sum(axis=1))
print("argmax per row:", p.argmax(axis=1))
print("uniform row:", p[2])
```

```
naive row 1: [nan nan nan  0.]
stable row 1: [0.6652 0.2447 0.09   0.    ]
rows sum to [1. 1. 1.]
argmax per row: [0 0 0]
uniform row: [0.25 0.25 0.25 0.25]
```

1. `z.max(axis=1, keepdims=True)` has shape `(3, 1)`. Without `keepdims=True` it would have shape `(3,)`, and `(3, 4) - (3,)` would try to align 3 with 4 and fail (or worse, succeed with the wrong meaning when the sizes happen to match).
2. The naive version overflows for 1000: `exp(1000)` is infinity and infinity divided by infinity is `nan`. The first printed line shows `nan` for the three large entries (the last entry, `exp(0)/inf`, becomes 0).
3. The stable version gives the true probabilities, about 0.665, 0.245, 0.09 and 0.0 for row 1, because after subtraction the row becomes `[0, -1, -2, -1000]`.
4. Each row sums to 1, `argmax` picks class 0 for every row (on the all-zero row it is a tie and the first index wins), and that all-zero row becomes the uniform distribution 0.25 each. `torch.softmax` and `F.cross_entropy` do this subtraction for you internally; the lesson is never to write `exp(z) / sum(exp(z))` yourself.

Autograd, first by hand

Before trusting `loss.backward()`, build a tiny version. We create a scalar class `Value` that stores its number, its gradient, the values it came from, and a small function that pushes gradient back to those parents using the chain rule. This is a miniature of what PyTorch records in `grad_fn`.

Problem 21.2.3 Let z = x*y + x^2 with x = 2 and y = -3. By hand: z = -6 + 4 = -2. dz/dx = y + 2x = -3 + 4 = 1 and dz/dy = x = 2. Verify with a hand-built autograd, then with a finite-difference check.

Rule 1	For c = a + b: dc/da = 1 and dc/db = 1, so each parent receives the child's gradient.
Rule 2	For c = a * b: dc/da = b and dc/db = a, so a parent receives (child gradient) times (the other parent's value).
Rule 3	If a variable is used twice (here x), its gradients from the two paths are *added*. This is why PyTorch accumulates into `.grad`.

```
class Value:
    def __init__(self, data, parents=()):
        self.data = float(data)
        self.grad = 0.0
        self._parents = parents
        self._backward = lambda: None

    def __add__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        out = Value(self.data + other.data, (self, other))
        def _backward():
            self.grad += out.grad
            other.grad += out.grad
        out._backward = _backward
        return out

    def __mul__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        out = Value(self.data * other.data, (self, other))
        def _backward():
            self.grad += other.data * out.grad
            other.grad += self.data * out.grad
        out._backward = _backward
        return out

    def __pow__(self, k):
        out = Value(self.data ** k, (self,))
        def _backward():
            self.grad += k * self.data ** (k - 1) * out.grad
        out._backward = _backward
        return out

    def backward(self):
        order, seen = [], set()
        def visit(v):
            if id(v) not in seen:
                seen.add(id(v))
                for p in v._parents:
                    visit(p)
                order.append(v)
        visit(self)
        self.grad = 1.0
        for v in reversed(order):
            v._backward()

def f(xv, yv):
    x, y = Value(xv), Value(yv)
    z = x * y + x ** 2
    return x, y, z

x, y, z = f(2.0, -3.0)
z.backward()
print("z =", z.data, " dz/dx =", x.grad, " dz/dy =", y.grad)

h = 1e-6
num_dx = (f(2.0 + h, -3.0)[2].data - f(2.0 - h, -3.0)[2].data) / (2 * h)
num_dy = (f(2.0, -3.0 + h)[2].data - f(2.0, -3.0 - h)[2].data) / (2 * h)
print("numeric dx, dy:", round(num_dx, 4), round(num_dy, 4))

z.backward()                      # call again without zeroing
print("after second backward, x.grad =", x.grad)
```

```
z = -2.0  dz/dx = 1.0  dz/dy = 2.0
numeric dx, dy: 1.0 2.0
after second backward, x.grad = 3.0
```

1. Each operator (`__add__` , `__mul__` , `__pow__`) computes the forward number and stores a closure `_backward` that knows the local derivative. This is the recipe the cooking-show analogy described.
2. Inside each closure the update uses `+=` , not `=` . Because `x` feeds two operations (the product and the square), it receives two contributions that must be added: y + 2x = -3 + 4 = 1.
3. `backward()` first orders the nodes so that every node comes after its parents (a topological order), seeds the output gradient with 1.0 (the slope of z with respect to itself), then calls each closure from the output back to the inputs.
4. The first printed line matches the hand calculation: z = -2, dz/dx = 1, dz/dy = 2.
5. The finite-difference check nudges the input up and down by `h` and measures the slope. It agrees to four decimals, which is the standard way to test any gradient code.
6. Calling `backward()` a second time without resetting adds to everything already stored, including the intermediate nodes, so `x.grad` becomes 3 instead of 1 (it is not even a clean doubling). This is the exact behaviour behind the famous bug "I forgot `optimizer.zero_grad()` ".

In PyTorch the same computation is three lines (not run here), and the two behaviours (accumulation and the graph being freed) are the ones that surprise newcomers:

```
import torch

x = torch.tensor(2.0, requires_grad=True)
y = torch.tensor(-3.0, requires_grad=True)
z = x * y + x ** 2
z.backward()
print(z.item(), x.grad.item(), y.grad.item())   # -2.0 1.0 2.0

# calling backward again on the SAME graph raises an error, because buffers were freed:
# RuntimeError: Trying to backward through the graph a second time ...
# Use z.backward(retain_graph=True) only if you truly need it.

# gradients ACCUMULATE across separate backward passes:
x.grad.zero_()                      # reset (optimizer.zero_grad() does this for all parameters)
(x ** 2).backward()
print(x.grad.item())                # 4.0

with torch.no_grad():               # nothing is recorded inside
    w = x * 3
print(w.requires_grad)              # False
v = (x * 3).detach()                # cut a tensor out of the graph
```

COMMON MISTAKE

Wrong: storing `losses.append(loss)` in a loop. **Symptom:** memory grows every step, because each stored tensor keeps its entire graph alive. **Fix:** `losses.append(loss.item())` (a plain Python float) or `loss.detach()` .

Realistic: gradients for a whole layer, checked numerically

Real networks use vectors and matrices, so the backward formulas are matrix formulas. For a linear model $\hat{y} = X w + b$ with mean squared error $L = \text{mean}((\hat{y} - y)^2)$, the gradients are $dL/dw = (2/N)\, X^T (\hat{y} - y)$ and $dL/db = (2/N)\, \text{sum}(\hat{y} - y)$. We implement them, then check against numerical gradients, which is how you validate any custom layer.

```
import numpy as np

rng = np.random.default_rng(0)
N, D = 50, 3
X = rng.normal(size=(N, D))
true_w = np.array([2.0, -1.0, 0.5])
y = X @ true_w + 0.3 + rng.normal(scale=0.1, size=N)

def loss_fn(w, b):
    err = X @ w + b - y
    return np.mean(err ** 2)

def analytic_grads(w, b):
    err = X @ w + b - y
    return 2 / N * X.T @ err, 2 / N * err.sum()

def numeric_grads(w, b, h=1e-6):
    gw = np.zeros_like(w)
    for i in range(len(w)):
        e = np.zeros_like(w); e[i] = h
        gw[i] = (loss_fn(w + e, b) - loss_fn(w - e, b)) / (2 * h)
    gb = (loss_fn(w, b + h) - loss_fn(w, b - h)) / (2 * h)
    return gw, gb

w0, b0 = np.zeros(D), 0.0
gw, gb = analytic_grads(w0, b0)
nw, nb = numeric_grads(w0, b0)
print("analytic dw:", np.round(gw, 4), "db:", round(gb, 4))
print("numeric  dw:", np.round(nw, 4), "db:", round(nb, 4))
print("match:", np.allclose(gw, nw, atol=1e-6) and abs(gb - nb) < 1e-6)
```

```
analytic dw: [-4.941   1.0721 -1.4741] db: 0.0996
numeric  dw: [-4.941   1.0721 -1.4741] db: 0.0996
match: True
```

1. We simulate data from a known rule (weights 2, -1, 0.5 and offset 0.3) plus small noise, so we know what a good fit looks like.
2. `analytic_grads` is the formula you would code in a custom autograd function. Note `X.T @ err`: `(D, N) @ (N,)` gives shape `(D,)`, one gradient per weight.
3. `numeric_grads` nudges one weight at a time, which is slow (two forward passes per weight) but needs no calculus. This is the idea behind `torch.autograd.gradcheck`.
4. At `w = 0` each gradient has the opposite sign to the true weight (negative for 2 and 0.5, positive for -1), so a step *against* the gradient moves each weight toward its true value. The two printed rows agree and `match` prints `True`.

KEY IDEA	Autograd is the chain rule applied by a program. It needs three things from you: mark trainable tensors with `requires_grad=True`, produce one scalar loss, and call `backward()`. It gives you three things back: `.grad` on each leaf, accumulation across calls, and a graph that is freed after use.

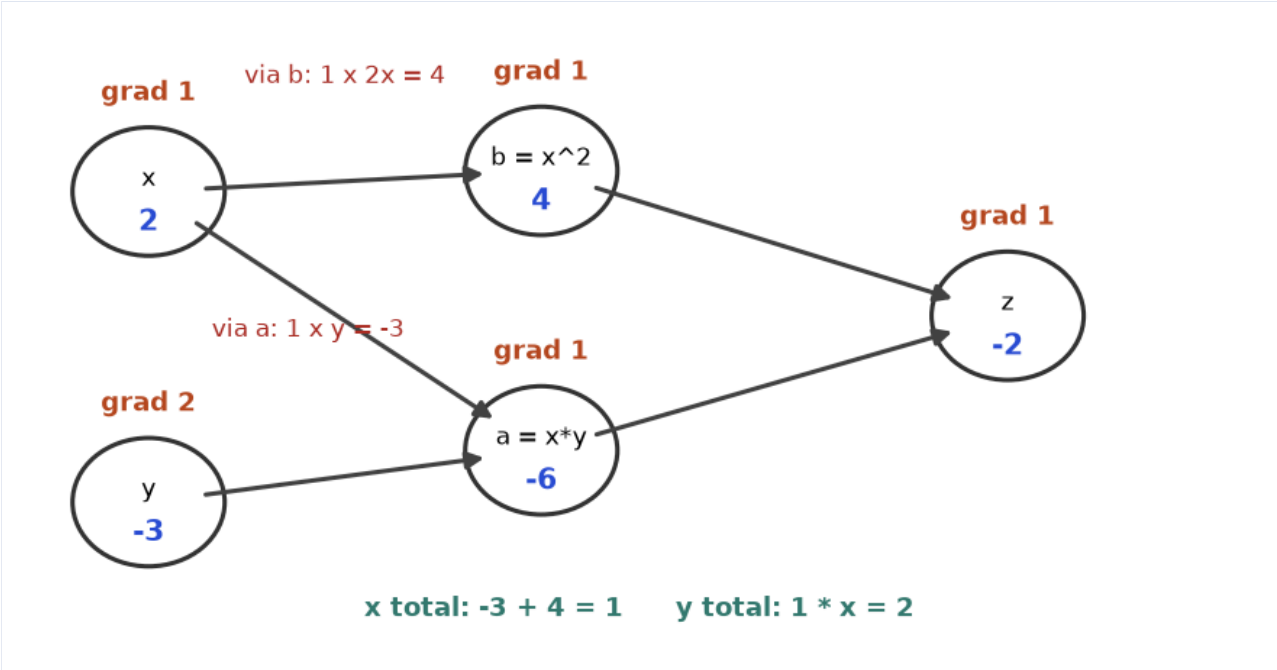


*Figure 21.2. The graph for z = x*y + x^2 at x = 2, y = -3. Blue numbers go forward, orange numbers are the gradients flowing back; x receives two contributions that are added.*

How to read it: start at the left with the inputs, follow the blue values to z = -2, then read the orange numbers from right to left. At x the two incoming gradients are y = -3 (through the product) and 2x = 4 (through the square); they add to 1.

21.3 PyTorch Part 2: nn.Module, Losses, Optimisers and Schedulers

INTUITION

An `nn.Module` is a labelled toolbox of parameters plus a recipe (`forward`) that uses them. The label part matters: when you assign a layer or a parameter as an attribute, the module registers it, so `model.parameters()` can hand every trainable number to the optimiser, `state_dict()` can save them, and `.to("cuda")` can move them. A layer stored in a plain Python list is invisible to all of that.

Reference: modules, losses, optimisers and schedulers

Name	What it does in plain English	Minimal example (not run here)
`nn.Module`	Base class: registers parameters and sub-modules	`class Net(nn.Module): ...`
`nn.Linear(i, o)`	Fully connected layer, y = xW^T + b	`nn.Linear(10, 4)`
`nn.Sequential`	Chain layers; output of one feeds the next	`nn.Sequential(nn.Linear(4, 8), nn.ReLU())`
`nn.ReLU / GELU`	Non-linearities; without them stacked layers collapse into one	`nn.GELU()`
`nn.Dropout(p)`	Randomly zero values in training; identity in eval	`nn.Dropout(0.1)`
`nn.LayerNorm(d)`	Normalise each sample's features (used in transformers)	`nn.LayerNorm(768)`

Name	What it does in plain English	Minimal example (not run here)
`nn.BatchNorm1d(d)`	Normalise across the batch; has running statistics	`nn.BatchNorm1d(64)`
`nn.Embedding(n, d)`	Lookup table from integer id to vector	`nn.Embedding(30000, 128)`
`nn.TransformerEncoderLayer`	One attention + feed-forward block	`nn.TransformerEncoderLayer(d_model=128, nhead=4, batch_first=True)`
`model.parameters()`	All trainable tensors (what the optimiser needs)	`sum(p.numel() for p in model.parameters())`
`model.named_parameters()`	Same, with names (inspect, freeze, group)	`for n, p in model.named_parameters(): ...`
`state_dict / load_state_dict`	Dictionary of tensors; the portable way to save	`model.load_state_dict(sd)`
`model.train() / model.eval()`	Switch dropout and batch-norm behaviour	`model.eval()`
`p.requires_grad_(False)`	Freeze a parameter (transfer learning)	`for p in base.parameters(): p.requires_grad_(False)`
`nn.CrossEntropyLoss`	Classification loss from raw logits and integer class ids	`nn.CrossEntropyLoss()(logits, y)`
`nn.BCEWithLogitsLoss`	Binary / multi-label loss from raw logits	`nn.BCEWithLogitsLoss(pos_weight=w)`
`nn.MSELoss`	Mean squared error for regression	`nn.MSELoss()(pred, y)`
`optim.SGD`	Plain gradient step, optional momentum	`optim.SGD(ps, lr=0.01, momentum=0.9)`
`optim.Adam / AdamW`	Adaptive step per parameter; AdamW decouples weight decay	`optim.AdamW(ps, lr=3e-4, weight_decay=0.01)`
`optimizer.zero_grad()`	Clear gradients before the next backward	`opt.zero_grad(set_to_none=True)`
`optimizer.step()`	Apply the update using `.grad`	`opt.step()`
`lr_scheduler.CosineAnnealingLR`	Smoothly decay the learning rate	`CosineAnnealingLR(opt, T_max=1000)`
`lr_scheduler.OneCycleLR / LambdaLR`	Warm-up then decay; or any custom rule	`LambdaLR(opt, lambda s: min(1, s / 100))`
`clip_grad_norm_`	Rescale gradients so their total length is at most a limit	`nn.utils.clip_grad_norm_(ps, 1.0)`
`nn.init.*`	Choose starting values for weights	`nn.init.kaiming_normal_(w)`

First, the PyTorch way (not run here). The class below is a complete, idiomatic classifier. Read it once, then see the same mechanics built by hand.

```
import torch
import torch.nn as nn

class TicketNet(nn.Module):
    def __init__(self, vocab_size, emb_dim=64, hidden=128, n_classes=3, p_drop=0.2):
        super().__init__()                      # REQUIRED: sets up the registry
        self.emb = nn.Embedding(vocab_size, emb_dim, padding_idx=0)
        self.body = nn.Sequential(
            nn.Linear(emb_dim, hidden),
            nn.ReLU(),
            nn.Dropout(p_drop),
            nn.Linear(hidden, n_classes),
        )

    def forward(self, ids, mask):
        # ids: (B, T) int64, mask: (B, T) with 1 for real tokens, 0 for padding
        e = self.emb(ids)                                   # (B, T, E)
        m = mask.unsqueeze(-1).to(e.dtype)                  # (B, T, 1)
        pooled = (e * m).sum(dim=1) / m.sum(dim=1).clamp(min=1)   # mean over real tokens: (B, E)
        return self.body(pooled)                            # logits: (B, n_classes)

model = TicketNet(vocab_size=5000)
print(sum(p.numel() for p in model.parameters()))           # 5000*64 + (64*128+128) + (128*3+3) = 328707
for name, p in model.named_parameters():
    print(name, tuple(p.shape))                             # emb.weight (5000, 64), body.0.weight (128, 64), ...

loss_fn = nn.CrossEntropyLoss()                             # takes LOGITS, not probabilities
opt = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=0.01)
```

The comment on the parameter count is an arithmetic check you can do yourself: embedding 5000 x 64 = 320,000; first linear 64 x 128 + 128 = 8,320; second linear 128 x 3 + 3 = 387; total 328,707. Counting parameters by hand before running is one of the best shape-debugging habits.

Building the module system from scratch

The registry trick is simple: intercept attribute assignment and file parameters and sub-modules into dictionaries. Below is a 40-line NumPy version with `Linear`, `ReLU`, `Dropout` and `Sequential`, including a hand-written backward pass.

Problem 21.3.1 Build `Sequential(Linear(2,16), ReLU(), Linear(16,2))`. List the named parameters, count them (by hand: 2*16 + 16 + 16*2 +

2 = 82) and verify one weight gradient numerically.

```
import numpy as np

class Parameter:
    def __init__(self, data):
        self.data = data
        self.grad = np.zeros_like(data)

class Module:
    def __init__(self):
        object.__setattr__(self, "_params", {})
        object.__setattr__(self, "_mods", {})
        object.__setattr__(self, "training", True)
    def __setattr__(self, name, value):
        if isinstance(value, Parameter):
            self._params[name] = value
        elif isinstance(value, Module):
            self._mods[name] = value
        object.__setattr__(self, name, value)
    def named_parameters(self, prefix=""):
        for k, p in self._params.items():
            yield prefix + k, p
        for k, m in self._mods.items():
            yield from m.named_parameters(prefix + k + ".")
    def parameters(self):
        return [p for _, p in self.named_parameters()]
    def zero_grad(self):
        for p in self.parameters():
            p.grad[...] = 0.0
    def train(self, mode=True):
        object.__setattr__(self, "training", mode)
        for m in self._mods.values():
            m.train(mode)
        return self
    def eval(self):
        return self.train(False)

class Linear(Module):
    def __init__(self, n_in, n_out, rng):
        super().__init__()
        self.weight = Parameter(rng.normal(scale=np.sqrt(2 / n_in), size=(n_in, n_out)))
        self.bias = Parameter(np.zeros(n_out))
    def forward(self, x):
        self.x = x
        return x @ self.weight.data + self.bias.data
    def backward(self, g):
        self.weight.grad += self.x.T @ g
        self.bias.grad += g.sum(axis=0)
        return g @ self.weight.data.T

class ReLU(Module):
    def forward(self, x):
        self.mask = x > 0
        return x * self.mask
    def backward(self, g):
        return g * self.mask

class Sequential(Module):
    def __init__(self, *layers):
        super().__init__()
        for i, layer in enumerate(layers):
            setattr(self, str(i), layer)
        self.layers = layers
```

```
    def forward(self, x):
        for layer in self.layers:
            x = layer.forward(x)
        return x
    def backward(self, g):
        for layer in reversed(self.layers):
            g = layer.backward(g)
        return g

rng = np.random.default_rng(1)
model = Sequential(Linear(2, 16, rng), ReLU(), Linear(16, 2, rng))
for name, p in model.named_parameters():
    print(f"{name:10s} {p.data.shape}")
print("total parameters:", sum(p.data.size for p in model.parameters()))

x = rng.normal(size=(5, 2))
out = model.forward(x)
print("output shape:", out.shape)
model.zero_grad()
model.backward(np.ones_like(out))            # gradient of sum(out) w.r.t. out is 1
analytic = model.layers[0].weight.grad[0, 0]

w = model.layers[0].weight.data
h = 1e-6
w[0, 0] += h; up = model.forward(x).sum()
w[0, 0] -= 2 * h; down = model.forward(x).sum()
w[0, 0] += h
print("analytic vs numeric:", round(analytic, 5), round((up - down) / (2 * h), 5))
```

```
0.weight   (2, 16)
0.bias     (16,)
2.weight   (16, 2)
2.bias     (2,)
total parameters: 82
output shape: (5, 2)
analytic vs numeric: 0.97713 0.97713
```

1. `Parameter` pairs a number array with a same-shaped gradient array. In PyTorch a `Parameter` is a tensor flagged as trainable.
2. `Module.__setattr__` is the whole registry: when you write `self.weight = Parameter(...)` it files the object into `_params`; when you assign a sub-module it files it into `_mods`. `named_parameters` walks both, producing dotted names like `0.weight`.
3. `Linear.forward` remembers its input because the backward pass needs it: $dW = x^T g$. The bias gradient is the column sum of g, and the gradient passed to the previous layer is $g W^T$. These are the same formulas as in 21.2.
4. `Sequential` registers its layers under the names "0", "1", "2", which is exactly why PyTorch parameter names look like `body.0.weight`. It then runs forward in order and backward in reverse order.
5. The listing prints four parameter tensors (two weights, two biases) and 82 in total, matching the hand count 32 + 16 + 32 + 2.
6. Finally one weight gradient from backward is compared with a finite-difference estimate; they agree to 5 decimals. If a custom layer ever disagrees with the numeric check, the bug is in your backward.
7. `train()` and `eval()` only set a flag on every sub-module. They change nothing by themselves; layers like dropout read the flag.

COMMON MISTAKE

Wrong: `self.layers = [nn.Linear(4, 4) for _ in range(3)]` inside an `nn.Module`. **Symptom:** `model.parameters()` is empty (or misses the layers), the optimiser trains nothing, the loss never moves, and `.to("cuda")` leaves the layers on the CPU. **Fix:** `nn.ModuleList([...])` or `nn.Sequential(...)`, which register their contents. The registry above shows why: only attributes that are `Module` or `Parameter` objects are filed.

Dropout and the train/eval switch

Dropout randomly zeroes a fraction p of the values while training and scales the survivors by 1/(1-p) so the average stays the same ("inverted dropout"). At evaluation it must do nothing. Forgetting `model.eval()` is the single most common reason for predictions that change between two identical calls.

```
import numpy as np

rng = np.random.default_rng(0)

def dropout(x, p, training, rng):
    if not training or p == 0:
        return x
    keep = rng.random(x.shape) >= p
    return x * keep / (1 - p)

x = np.ones((1, 10))
print("train call 1:", dropout(x, 0.5, True, rng))
print("train call 2:", dropout(x, 0.5, True, rng))
print("eval  call 1:", dropout(x, 0.5, False, rng))

big = np.ones((10000, 1))
print("train mean over 10000 values:", round(dropout(big, 0.3, True, rng).mean(), 3))
```

```
train call 1: [[2. 0. 0. 0. 2. 2. 2. 2. 2. 2.]]
train call 2: [[2. 0. 2. 0. 2. 0. 2. 2. 0. 0.]]
eval  call 1: [[1. 1. 1. 1. 1. 1. 1. 1. 1. 1.]]
train mean over 10000 values: 0.999
```

1. The mask `rng.random(shape) >= p` keeps each value with probability 1-p. Dividing by 1-p keeps survivors at 2.0 when p = 0.5.
2. Two training calls on identical input print different patterns of 0.0 and 2.0, which is why a model left in train mode gives different predictions every time.
3. The eval call returns the input unchanged.
4. With p = 0.3 over 10,000 values the average stays close to 1.0 (printed above), the whole point of the scaling.

Losses: why they take logits

Classification losses in PyTorch (`CrossEntropyLoss` , `BCEWithLogitsLoss`) expect **raw scores (logits)**, not probabilities, because they fuse the softmax and the logarithm using the log-sum-exp trick: log softmax(z)$_y$ = z$_y$ - log sum exp(z). The gradient of the cross-entropy with respect to the logits has a famously simple form: softmax(z) minus the one-hot label.

Problem 21.3.2 For logits z = [2.0, 0.5, -1.0] and true class 0, compute the cross-entropy by hand: softmax = [0.7856, 0.1753, 0.0391] (approximately), so the loss is -log 0.7856 = 0.2413 and the gradient is [0.7856 - 1, 0.1753, 0.0391] = [-0.2144, 0.1753, 0.0391]. Verify, and show what happens to a naive loss with huge logits.

```
import numpy as np

def cross_entropy_from_logits(z, y):
    z = z - z.max(axis=1, keepdims=True)
    log_probs = z - np.log(np.exp(z).sum(axis=1, keepdims=True))
    return -log_probs[np.arange(len(y)), y].mean()

def naive_cross_entropy(z, y):
    p = np.exp(z) / np.exp(z).sum(axis=1, keepdims=True)
    return -np.log(p[np.arange(len(y)), y]).mean()

z = np.array([[2.0, 0.5, -1.0]])
y = np.array([0])
p = np.exp(z) / np.exp(z).sum()
print("softmax:", np.round(p, 4))
print("loss:", round(cross_entropy_from_logits(z, y), 4))
print("gradient wrt logits:", np.round(p - np.eye(3)[y], 4))

huge = np.array([[1000.0, 0.0, -5.0]])
with np.errstate(all="ignore"):
    print("naive on huge logits:", naive_cross_entropy(huge, y))
print("stable on huge logits:", cross_entropy_from_logits(huge, y) + 0.0)

# binary version with logits: log(1 + exp(-t*z)) written stably
def bce_with_logits(z, t):
    return np.mean(np.maximum(z, 0) - z * t + np.log1p(np.exp(-np.abs(z))))
print("BCE:", round(bce_with_logits(np.array([2.0, -1.0, 0.5]), np.array([1.0, 0.0, 1.0])), 4))
```

```
softmax: [[0.7856 0.1753 0.0391]]
loss: 0.2413
gradient wrt logits: [[-0.2144  0.1753  0.0391]]
naive on huge logits: nan
stable on huge logits: 0.0
BCE: 0.3048
```

1. The stable version subtracts the row maximum, then computes `z - log(sum(exp(z)))`. This is what `F.log_softmax` does and is why `CrossEntropyLoss` accepts logits.
2. `log_probs[np.arange(len(y)), y]` picks, for each row, the log-probability of its true class. In PyTorch the same pick is `gather` or `nll_loss`.
3. The printed loss is 0.2413 and the gradient is `p - onehot`, matching the hand calculation: the true class is pushed up (negative gradient means increase), the others pushed down.
4. With logits of 1000 the naive formula returns `nan`, while the stable one returns 0.0 (the model is certain and right; we add 0.0 only to avoid printing a negative zero).
5. `bce_with_logits` uses the rewrite max(z,0) - z*t + log(1 + exp(-|z|)), which never overflows. That is the formula inside `BCEWithLogitsLoss`.

COMMON MISTAKE

Wrong: `nn.CrossEntropyLoss()(torch.softmax(logits, -1), y)`. **Symptom:** training works but plateaus at a poor accuracy and the loss looks oddly small, because softmax is applied twice. **Fix:** pass raw logits. Apply `softmax` only when you want probabilities to show a user.

Optimisers: SGD, momentum and Adam by hand

An optimiser takes the gradient and decides the step. **SGD** subtracts lr times the gradient. **Momentum** keeps a running velocity so steps in a consistent direction speed up and zig-zags cancel. **Adam** keeps running averages of the gradient (m) and of its square (v) and divides one by the root of the other, giving every parameter its own step size. With bias correction the update is: $\hat{m} = m/(1 - \beta_1^t)$, $\hat{v} = v/(1 - \beta_2^t)$, w -= lr * $\hat{m}$ / ($\sqrt{\hat{v}}$ + ε).

Problem 21.3.3 Minimise the stretched bowl f(x, y) = 0.5 * (x^2 + 100 y^2) starting at (5, 5). The gradient is (x, 100 y). Compare plain SGD, momentum and Adam after 100 steps. SGD needs lr below 2/100 = 0.02 to stay stable in the steep y direction, which makes it crawl along the shallow x direction.

```
import numpy as np

def f(w):
    return 0.5 * (w[0] ** 2 + 100 * w[1] ** 2)

def grad(w):
    return np.array([w[0], 100 * w[1]])

def run(step_fn, steps=100):
    w = np.array([5.0, 5.0])
    state = {}
    for t in range(1, steps + 1):
        w = step_fn(w, grad(w), state, t)
    return f(w)

def sgd(lr):
    def step(w, g, s, t):
        return w - lr * g
    return step

def momentum(lr, beta=0.9):
    def step(w, g, s, t):
        s["v"] = beta * s.get("v", 0) + g
        return w - lr * s["v"]
    return step

def adam(lr, b1=0.9, b2=0.999, eps=1e-8):
    def step(w, g, s, t):
        s["m"] = b1 * s.get("m", 0) + (1 - b1) * g
        s["v"] = b2 * s.get("v", 0) + (1 - b2) * g ** 2
        mh = s["m"] / (1 - b1 ** t)
        vh = s["v"] / (1 - b2 ** t)
        return w - lr * mh / (np.sqrt(vh) + eps)
    return step

print("start loss:", f(np.array([5.0, 5.0])))
print("SGD lr=0.019      :", round(run(sgd(0.019)), 6))
print("SGD lr=0.021 (too big):", f"{run(sgd(0.021)):.3e}")
print("momentum lr=0.01  :", round(run(momentum(0.01)), 6))
print("Adam lr=0.3       :", round(run(adam(0.3)), 6))
```

```
start loss: 1262.5
SGD lr=0.019      : 0.269596
SGD lr=0.021 (too big): 2.374e+11
momentum lr=0.01  : 0.002365
Adam lr=0.3       : 0.023447
```

1. The two directions have very different curvature: the x direction has curvature 1 and the y direction 100. This "ravine" is the classic reason plain SGD is slow.
2. SGD with `lr = 0.019` is just stable for y (the factor 1 - 100*0.019 = -0.9 shrinks it by 0.9 each step with a sign flip) but x only shrinks by 0.981 per step, so after 100 steps x is still about 5 * 0.981^{100} = 0.73 and the loss is about 0.27.
3. With `lr = 0.021` the y factor is 1 - 2.1 = -1.1, so |y| grows 10 percent every step and the loss explodes (printed in scientific notation). That is the classic divergence symptom: a loss that goes to `inf` then `nan`.
4. Momentum accumulates velocity in the consistent x direction. With an even smaller lr of 0.01 it reaches a loss near 0.002, about a hundred times lower than plain SGD. (Its stability limit is higher, so the safe range is wider.)

5. Adam divides by the square root of the running squared gradient, so both coordinates move at a similar speed regardless of curvature. With lr 0.3 it reaches about 0.02: far better than SGD, but with a constant step size it keeps overshooting near the minimum, which is the reason schedulers exist. The exact ranking depends on the problem and the learning rates tried; the lesson is the behaviour, not a winner.

Learning-rate schedules and gradient clipping

A common recipe for transformers is **warm-up then cosine decay**: increase the rate linearly from zero for the first few percent of steps (the early gradients are noisy and Adam's statistics are still unreliable), then decay smoothly toward a small floor. In PyTorch you express it with `LambdaLR` or use `get_cosine_schedule_with_warmup` from `transformers`. **Gradient clipping** rescales all gradients together if their combined length exceeds a limit, which prevents one bad batch from wrecking the weights.

```
import numpy as np

def lr_at(step, total, warmup, peak, floor=0.0):
    if step < warmup:
        return peak * step / warmup
    prog = (step - warmup) / max(1, total - warmup)
    return floor + 0.5 * (peak - floor) * (1 + np.cos(np.pi * prog))

for s in [0, 50, 100, 300, 550, 1000]:
    print(f"step {s:4d}  lr {lr_at(s, 1000, 100, 3e-4):.2e}")

def clip_grad_norm(grads, max_norm):
    total = np.sqrt(sum((g ** 2).sum() for g in grads))
    scale = min(1.0, max_norm / (total + 1e-6))
    return [g * scale for g in grads], total

grads = [np.array([3.0, 4.0]), np.array([12.0])]      # global norm = sqrt(9+16+144) = 13
clipped, before = clip_grad_norm(grads, 1.0)
after = np.sqrt(sum((g ** 2).sum() for g in clipped))
print("norm before:", round(before, 3), "after:", round(after, 3))
print("direction kept:", np.round(clipped[0] / np.linalg.norm(clipped[0]), 3), "= [0.6, 0.8]")
```

```
step    0  lr 0.00e+00
step   50  lr 1.50e-04
step  100  lr 3.00e-04
step  300  lr 2.65e-04
step  550  lr 1.50e-04
step 1000  lr 0.00e+00
norm before: 13.0 after: 1.0
direction kept: [0.6 0.8] = [0.6, 0.8]
```

1. `lr_at` has two phases. Before `warmup` steps the rate is a straight line from 0 to the peak. After that, `prog` goes from 0 to 1 and the cosine term goes from 1 to -1, so the rate moves from the peak to the floor along a smooth S curve.
2. The printout shows 0 at step 0, half of the peak at step 50, the peak 3.00e-04 at step 100, then decay to 0.00e+00 at step 1000. Halfway through the decay (step 550) the rate is exactly half the peak.
3. `clip_grad_norm` computes the *global* norm across all gradient tensors (13 here), then multiplies every gradient by max_norm / total. The relative direction of the update is unchanged; only its length shrinks to 1.
4. In PyTorch you call it after `backward()` and before `step()`: `torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)`. It also returns the pre-clip norm, which is a useful number to log.

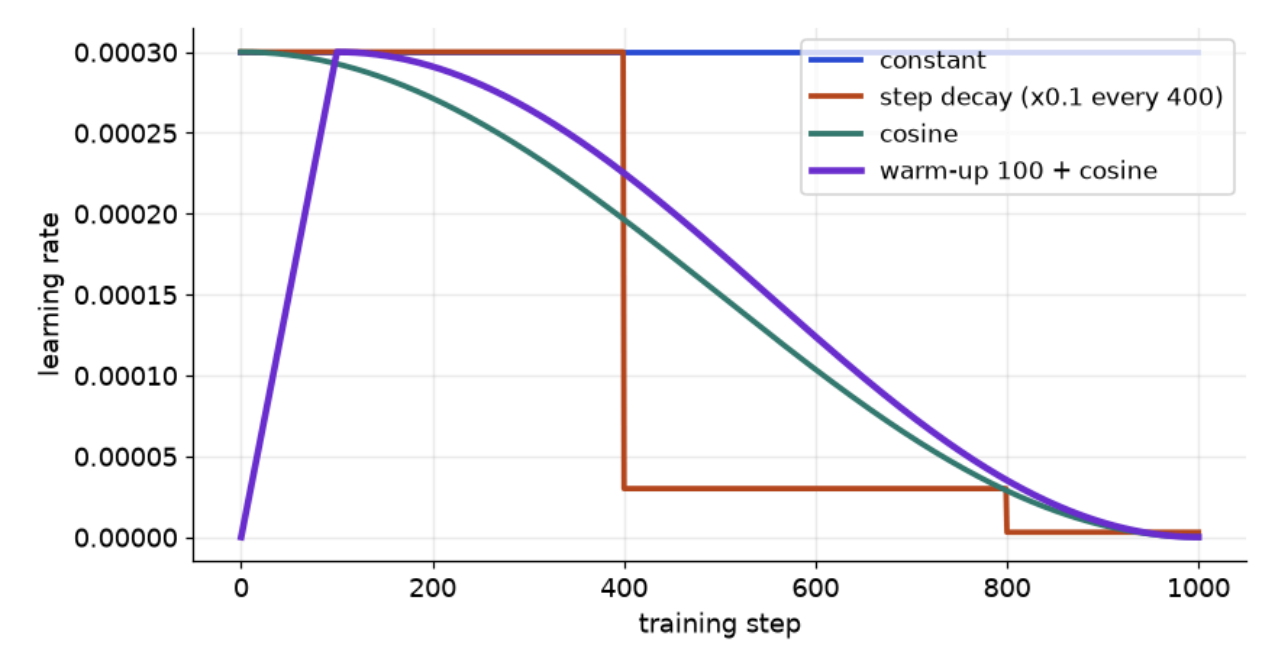


Figure 21.3. Four common learning-rate schedules over 1000 steps with the same peak rate.

How to read it: the vertical axis is the learning rate and the horizontal axis is the training step. The purple warm-up curve rises for 100 steps and then follows a cosine down to zero; the step schedule drops by a factor of 10 at fixed moments; constant never changes. Large rates early explore quickly, small rates late settle into a minimum.

The PyTorch equivalents (not run here):

```
import torch
from torch.optim.lr_scheduler import LambdaLR, CosineAnnealingLR, OneCycleLR

opt = torch.optim.AdamW(model.parameters(), lr=3e-4, weight_decay=0.01)

# 1) warm-up then cosine using LambdaLR (the lambda returns a MULTIPLIER of the base lr)
import math
total, warm = 1000, 100
def mult(step):
    if step < warm:
        return step / warm
    prog = (step - warm) / (total - warm)
    return 0.5 * (1 + math.cos(math.pi * prog))
sched = LambdaLR(opt, lr_lambda=mult)

# 2) param groups: different learning rates (or no weight decay on biases and norms)
decay = [p for n, p in model.named_parameters() if p.ndim >= 2]
no_decay = [p for n, p in model.named_parameters() if p.ndim < 2]
opt = torch.optim.AdamW([{"params": decay, "weight_decay": 0.01},
                         {"params": no_decay, "weight_decay": 0.0}], lr=3e-4)

# inside the training loop, the ORDER matters:
#   opt.zero_grad(); loss.backward(); clip_grad_norm_(...); opt.step(); sched.step()
print(opt.param_groups[0]["lr"])      # inspect the current learning rate
```

21.4 PyTorch Part 3: Datasets, DataLoader, the Training Loop and Saving

INTUITION

The `Dataset` answers "give me item number i". The `DataLoader` is a waiter: it decides the order (shuffling), collects items into trays (batches), and uses a *collate function* to stack uneven items into one rectangular tensor. The training loop is the kitchen routine: for each tray, predict, measure the error, clear old gradients, backpropagate, step. Once you can write that routine from memory you can debug any framework built on it, including the Hugging Face `Trainer`.

Reference: data, training and saving

Name	What it does in plain English	Minimal example (not run here)
`Dataset`	Your class with `__len__` and `__getitem__`	`def __getitem__(self, i): return self.x[i], self.y[i]`
`TensorDataset`	Wrap tensors that share the first dimension	`TensorDataset(X, y)`
`IterableDataset`	Stream items without random access (huge files)	`class Stream(IterableDataset): def __iter__(self): ...`
`DataLoader`	Batches, shuffles and (optionally) parallelises loading	`DataLoader(ds, batch_size=32, shuffle=True)`
`shuffle=True`	New random order each epoch (train only)	`DataLoader(train, shuffle=True)`
`num_workers`	Background processes that prepare batches	`DataLoader(ds, num_workers=4)`
`pin_memory=True`	Page-locked host memory for faster GPU copies	`DataLoader(ds, pin_memory=True)`
`drop_last=True`	Discard the final short batch	`DataLoader(ds, drop_last=True)`
`collate_fn`	Your function that merges a list of items into a batch	`DataLoader(ds, collate_fn=pad_collate)`
`random_split`	Split a dataset into train/validation parts	`random_split(ds, [0.8, 0.2], generator=g)`
`WeightedRandomSampler`	Oversample rare classes	`DataLoader(ds, sampler=sampler)`
`pad_sequence`	Pad a list of tensors to equal length	`pad_sequence(seqs, batch_first=True)`
`model.train() / eval()`	Switch layer behaviour	`model.eval()`
`torch.no_grad() / inference_mode()`	Skip graph building in evaluation	`with torch.inference_mode(): ...`
`loss.backward()`	Compute gradients	`loss.backward()`
`torch.save`	Write a checkpoint (use a dict of state_dicts)	`torch.save({"model": m.state_dict()}, "ck.pt")`
`torch.load(weights_only=True)`	Read it back safely (never load untrusted pickles)	`torch.load("ck.pt", map_location="cpu", weights_only=True)`
`load_state_dict(strict)`	Copy tensors into a model; `strict=False` reports mismatches	`model.load_state_dict(sd)`
`torch.manual_seed`	Reproducible init and shuffling	`torch.manual_seed(42)`

Name	What it does in plain English	Minimal example (not run here)
`worker_init_fn / generator`	Seed each loader worker and the shuffle order	`DataLoader(ds, generator=g)`
`SummaryWriter`	Log scalars for TensorBoard	`writer.add_scalar("loss", v, step)`

A DataLoader with padding, built by hand

Sentences have different lengths, but a tensor must be rectangular. The collate function pads short sequences with a pad id (0 here) and builds a **mask** that is 1 for real tokens and 0 for padding, so the model can ignore the padding. This is the single most common reason a hand-written `DataLoader` breaks ("expected sequence of length 5 at dim 1 (got 3)").

Problem 21.4.1 Seven tickets are lists of token ids with lengths 3, 5, 2, 4, 6, 2, 3. Batch them in threes without shuffling, then with `drop_last`, and show the padded matrix and mask for the first batch.

```
import numpy as np

class TicketDataset:
    def __init__(self, seqs, labels):
        self.seqs, self.labels = seqs, labels
    def __len__(self):
        return len(self.seqs)
    def __getitem__(self, i):
        return self.seqs[i], self.labels[i]

def pad_collate(items, pad_id=0):
    seqs, labels = zip(*items)
    longest = max(len(s) for s in seqs)
    ids = np.full((len(seqs), longest), pad_id, dtype=np.int64)
    mask = np.zeros((len(seqs), longest), dtype=np.int64)
    for r, s in enumerate(seqs):
        ids[r, :len(s)] = s
        mask[r, :len(s)] = 1
    return ids, mask, np.array(labels, dtype=np.int64)

class DataLoader:
    def __init__(self, ds, batch_size, shuffle=False, drop_last=False, collate_fn=pad_collate, seed=0):
        self.ds, self.bs, self.shuffle = ds, batch_size, shuffle
        self.drop_last, self.collate, self.rng = drop_last, collate_fn, np.random.default_rng(seed)
    def __len__(self):
        n = len(self.ds)
        return n // self.bs if self.drop_last else -(-n // self.bs)
    def __iter__(self):
        order = self.rng.permutation(len(self.ds)) if self.shuffle else np.arange(len(self.ds))
        for start in range(0, len(order), self.bs):
            idx = order[start:start + self.bs]
            if self.drop_last and len(idx) < self.bs:
                break
            yield self.collate([self.ds[int(i)] for i in idx])

seqs = [[5, 6, 7], [8, 9, 10, 11, 12], [13, 14], [15, 16, 17, 18], [19, 20, 21, 22, 23, 24], [25, 26], [27, 28, 29]]
ds = TicketDataset(seqs, [0, 1, 2, 0, 1, 2, 0])

dl = DataLoader(ds, batch_size=3)
print("batches:", len(dl), "| with drop_last:", len(DataLoader(ds, 3, drop_last=True)))
for ids, mask, y in dl:
    print("ids", ids.shape, "mask", mask.shape, "labels", y)
ids, mask, y = next(iter(dl))
print(ids)
print(mask)
first = [int(y.sum()) for _, _, y in DataLoader(ds, 3, shuffle=True, seed=1)]
print("shuffled label sums per batch:", first)
```

```
batches: 3 | with drop_last: 2
ids (3, 5) mask (3, 5) labels [0 1 2]
ids (3, 6) mask (3, 6) labels [0 1 2]
ids (1, 3) mask (1, 3) labels [0]
[[ 5  6  7  0  0]
 [ 8  9 10 11 12]
 [13 14  0  0  0]]
[[1 1 1 0 0]
 [1 1 1 1 1]
 [1 1 0 0 0]]
shuffled label sums per batch: [3, 3, 0]
```

1. `TicketDataset` follows the PyTorch contract: `__len__` says how many items exist and `__getitem__` returns one item. Nothing is loaded until asked, which is how a Dataset can wrap a million files.
2. `pad_collate` receives a *list* of items from `__getitem__`. It finds the longest sequence in *this batch only*, allocates a rectangle filled with the pad id, copies each sequence into its row, and sets the mask to 1 on real positions. Padding to the batch maximum, not the global maximum, saves compute.
3. `__len__` uses ceiling division: 7 items in batches of 3 give 3 batches (3, 3 and 1). With `drop_last` the short last batch is discarded, giving 2.
4. Iterating prints shapes `(3, 5)`, `(3, 6)` and `(1, 3)`: the second dimension changes per batch, because lengths differ. A model must therefore accept any sequence length.
5. The matrix shows token ids with zeros on the right and the mask with ones exactly where ids are real.
6. With `shuffle=True` the permutation changes the batch contents. The seed makes the order repeatable; in PyTorch pass `generator=torch.Generator().manual_seed(1)`.

The PyTorch version (not run here). The real `DataLoader` adds workers, pinned memory and many more safety checks, but the collate idea is identical:

```
import torch
from torch.utils.data import Dataset, DataLoader
from torch.nn.utils.rnn import pad_sequence

class TicketDataset(Dataset):
    def __init__(self, seqs, labels):
        self.seqs, self.labels = seqs, labels
    def __len__(self):
        return len(self.seqs)
    def __getitem__(self, i):
        return torch.tensor(self.seqs[i]), self.labels[i]

def pad_collate(items):
    seqs, labels = zip(*items)
    ids = pad_sequence(seqs, batch_first=True, padding_value=0)      # (B, T_max)
    mask = (ids != 0).long()
    return ids, mask, torch.tensor(labels)

g = torch.Generator().manual_seed(0)
train_dl = DataLoader(TicketDataset(seqs, labels), batch_size=32, shuffle=True,
                      collate_fn=pad_collate, num_workers=2, pin_memory=True,
                      drop_last=False, generator=g)
```

COMMON MISTAKE

Wrong: using `shuffle=True` on the validation loader and, on the training loader, forgetting it. **Symptom:** training on sorted or grouped data (all class 0 first, then class 1) makes the loss oscillate wildly and the model memorise the last class it saw; a shuffled validation loader is harmless but wasteful and makes batch-by-batch logs impossible to compare. **Fix:** shuffle training only.

The complete training loop

Now the full routine in a form you can compare line by line with PyTorch: mini-batches, an Adam optimiser, a validation pass without gradient tracking, **early stopping** (stop when validation loss has not improved for 30 epochs), and keeping the best weights. We deliberately use a tiny training set (60 points) and a wide hidden layer so the model overfits and you can watch it happen.

Problem 21.4.2 Train a 2-64-2 network on 60 noisy "two moons" points, validate on 340 others, stop early, then save and reload the best checkpoint

and confirm identical predictions.

```
import numpy as np, tempfile, os
from sklearn.datasets import make_moons

X, y = make_moons(n_samples=400, noise=0.35, random_state=0)
X = (X - X.mean(0)) / X.std(0)
Xtr, ytr, Xva, yva = X[:60], y[:60], X[60:], y[60:]

class MLP:
    def __init__(self, sizes, rng):
        self.W = [rng.normal(scale=np.sqrt(2 / a), size=(a, b)) for a, b in zip(sizes[:-1], sizes[1:])]
        self.b = [np.zeros(b) for b in sizes[1:]]
    def params(self):
        return self.W + self.b
    def forward(self, x):
        self.acts = [x]
        for i, (W, b) in enumerate(zip(self.W, self.b)):
            x = x @ W + b
            if i < len(self.W) - 1:
                x = np.maximum(x, 0)
            self.acts.append(x)
        return x
    def backward(self, g):
        gW, gb = [None] * len(self.W), [None] * len(self.W)
        for i in reversed(range(len(self.W))):
            gW[i] = self.acts[i].T @ g
            gb[i] = g.sum(0)
            if i > 0:
                g = (g @ self.W[i].T) * (self.acts[i] > 0)
        return gW + gb

class Adam:
    def __init__(self, params, lr=0.01, b1=0.9, b2=0.999, eps=1e-8):
        self.p, self.lr, self.b1, self.b2, self.eps, self.t = params, lr, b1, b2, eps, 0
        self.m = [np.zeros_like(q) for q in params]
        self.v = [np.zeros_like(q) for q in params]
    def step(self, grads):
        self.t += 1
        for q, g, m, v in zip(self.p, grads, self.m, self.v):
            m *= self.b1; m += (1 - self.b1) * g
            v *= self.b2; v += (1 - self.b2) * g * g
            q -= self.lr * (m / (1 - self.b1 ** self.t)) / (np.sqrt(v / (1 - self.b2 ** self.t)) + self.eps)

def ce_loss_grad(logits, y):
    z = logits - logits.max(1, keepdims=True)
    p = np.exp(z); p /= p.sum(1, keepdims=True)
    loss = -np.log(p[np.arange(len(y)), y]).mean()
    p[np.arange(len(y)), y] -= 1
    return loss, p / len(y)

def evaluate(model, X, y):                  # like model.eval() + torch.no_grad()
    logits = model.forward(X)
    loss, _ = ce_loss_grad(logits, y)
    return loss, (logits.argmax(1) == y).mean()

rng = np.random.default_rng(0)
model = MLP([2, 64, 2], rng)
opt = Adam(model.params(), lr=0.01)
best_loss, best_state, best_epoch, bad = 1e9, None, -1, 0

for epoch in range(1, 301):
    order = rng.permutation(len(Xtr))
    total = 0.0
```

```
    for i in range(0, len(order), 16):
        idx = order[i:i + 16]
        logits = model.forward(Xtr[idx])             # forward
        loss, g = ce_loss_grad(logits, ytr[idx])     # loss
        grads = model.backward(g)                    # backward
        opt.step(grads)                              # optimiser step
        total += loss * len(idx)
    val_loss, val_acc = evaluate(model, Xva, yva)
    if val_loss < best_loss - 1e-4:
        best_loss, best_epoch, bad = val_loss, epoch, 0
        best_state = [q.copy() for q in model.params()]
    else:
        bad += 1
    if epoch % 20 == 0:
        print(f"epoch {epoch:3d} train {total/len(Xtr):.4f} val {val_loss:.4f} acc {val_acc:.3f}")
    if bad >= 30:
        print(f"early stop at epoch {epoch}; best epoch {best_epoch} (val loss {best_loss:.4f})")
        break

path = os.path.join(tempfile.mkdtemp(), "best.npz")
np.savez(path, *best_state)                                  # like torch.save(state_dict)
fresh = MLP([2, 64, 2], np.random.default_rng(99))           # different random init
loaded = np.load(path)
for q, key in zip(fresh.params(), loaded.files):
    q[...] = loaded[key]                                     # like load_state_dict
for q, s in zip(model.params(), best_state):
    q[...] = s
print("reloaded predictions identical:", np.array_equal(fresh.forward(Xva), model.forward(Xva)))
print("final val accuracy of best model:", round(evaluate(fresh, Xva, yva)[1], 3))
```

```
epoch  20 train 0.3745 val 0.3671 acc 0.847
epoch  40 train 0.3097 val 0.3361 acc 0.862
epoch  60 train 0.2553 val 0.3559 acc 0.847
epoch  80 train 0.2272 val 0.3671 acc 0.847
early stop at epoch 83; best epoch 53 (val loss 0.3332)
reloaded predictions identical: True
final val accuracy of best model: 0.865
```

1. Standardising the features (mean 0, standard deviation 1) is the same preprocessing you would do before any neural network; unscaled inputs make learning slow and unstable.
2. The `MLP` keeps the layer inputs in `self.acts` during `forward` because the backward pass needs them. This is the memory PyTorch calls *activations*, and it is why training uses far more memory than inference (see 21.5).
3. Each mini-batch of 16 runs the four-step heartbeat: forward, loss, backward, step. In PyTorch you would also call `opt.zero_grad()`; here the gradients are fresh arrays each time, so there is nothing to clear.
4. The cross-entropy function returns the loss *and* the gradient `(softmax - onehot) / batch`, which is what `loss.backward()` would start the chain with.
5. `evaluate` never calls `backward` or the optimiser, the equivalent of `model.eval()` plus `torch.no_grad()`.
6. Look at the printed table: training loss keeps falling while validation loss reaches its minimum around epoch 53 and then rises. That divergence is overfitting, and early stopping watches for it. Patience 30 stops the run at epoch 83.
7. We keep a copy of the weights at the best epoch (`q.copy()`; without the copy the saved weights would silently keep changing because they are views of the live parameters).
8. Saving writes the arrays to disk, and loading copies them into a model that started from a different random seed. The check prints `True`: the checkpoint fully determines the model.

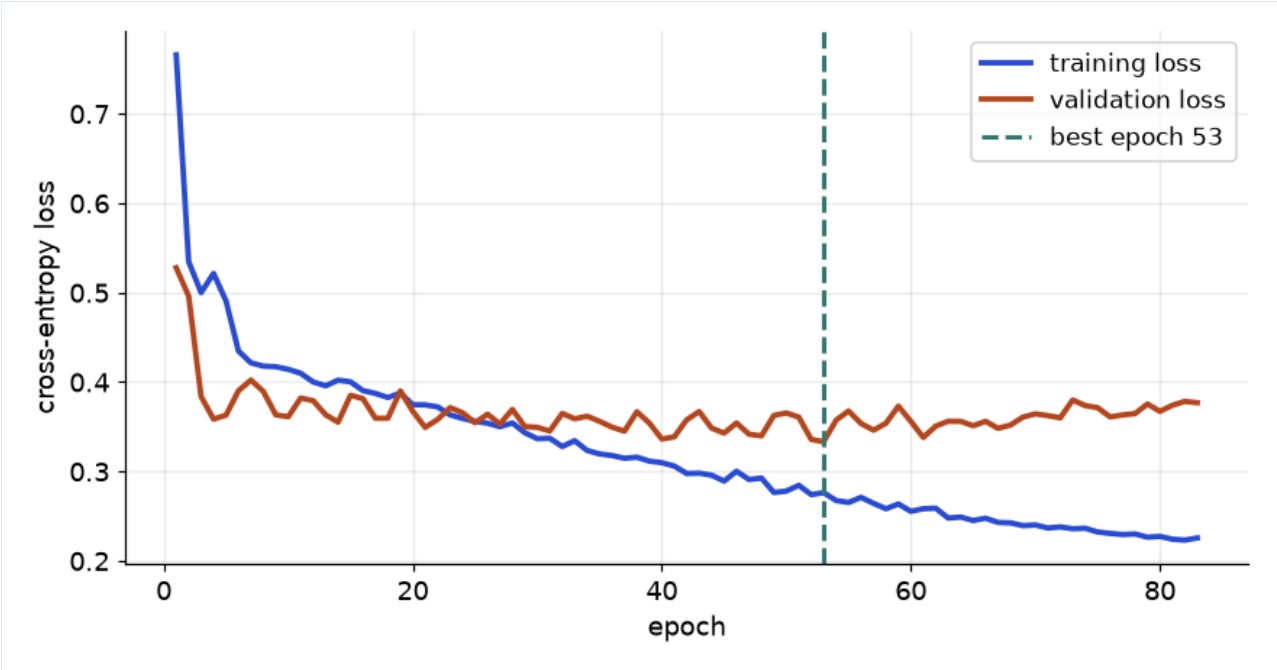


Figure 21.4. Training and validation loss from the run above. The dashed line marks the best epoch that early stopping restores.

How to read it: the blue training curve only goes down, because the network can memorise 60 points. The orange validation curve bottoms out and climbs: from that point extra training makes the model worse on new data. The gap between the curves is the generalisation gap.

The same loop in PyTorch (not run here). Compare it line by line with the NumPy version:

```
import copy, torch
import torch.nn as nn

device = "cuda" if torch.cuda.is_available() else "cpu"
model = TicketNet(vocab_size=5000).to(device)
loss_fn = nn.CrossEntropyLoss()
opt = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=0.01)

@torch.no_grad()
def evaluate(model, loader):
    model.eval()
    total, correct, n = 0.0, 0, 0
    for ids, mask, y in loader:
        ids, mask, y = ids.to(device), mask.to(device), y.to(device)
        logits = model(ids, mask)
        total += loss_fn(logits, y).item() * len(y)
        correct += (logits.argmax(dim=-1) == y).sum().item()
        n += len(y)
    return total / n, correct / n

best_loss, best_state, bad = float("inf"), None, 0
for epoch in range(100):
    model.train()
    for ids, mask, y in train_dl:
        ids, mask, y = ids.to(device), mask.to(device), y.to(device)
        opt.zero_grad(set_to_none=True)
        loss = loss_fn(model(ids, mask), y)
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        opt.step()
    val_loss, val_acc = evaluate(model, val_dl)
    if val_loss < best_loss - 1e-4:
        best_loss, bad = val_loss, 0
        best_state = copy.deepcopy(model.state_dict())     # deepcopy, or it keeps changing
    else:
        bad += 1
        if bad >= 5:
            break

model.load_state_dict(best_state)
torch.save({"model": model.state_dict(), "epoch": epoch, "val_loss": best_loss}, "best.pt")

# later, in another process (weights_only=True is the safe default in recent versions):
ckpt = torch.load("best.pt", map_location="cpu", weights_only=True)
new_model = TicketNet(vocab_size=5000)
new_model.load_state_dict(ckpt["model"])
new_model.eval()
```

REAL JOB

Save the **state_dict**, not the whole model object. Pickling the object ties the file to your exact class and module path, so it breaks when you refactor, and loading untrusted pickles can run arbitrary code. A state_dict is just named tensors, and the safetensors format used by Hugging Face stores them without pickle at all. Also save the config (layer sizes, vocab) next to the weights, because a state_dict cannot rebuild the architecture.

21.5 Devices, Memory and Mixed Precision

INTUITION

A GPU is a separate computer with its own small, fast memory. Data and model must be in *that* memory to be used there, and copying between the two memories is slow compared with computing. Nearly every "device" error is one of two stories: two things that must be in the same place are not, or there is not enough room for everything you asked to keep.

Reference: device and memory tools

Name	What it does in plain English	Minimal example (not run here)
`torch.cuda.is_available()`	Is an NVIDIA GPU usable?	`torch.cuda.is_available()`
`torch.backends.mps.is_available()`	Is the Apple-silicon GPU usable?	`torch.backends.mps.is_available()`
`torch.device`	A device handle	`torch.device("cuda:0")`
`model.to(device)`	Move all parameters in place	`model.to(device)`
`x.to(device, non_blocking=True)`	Copy a tensor (returns a new tensor)	`x = x.to(device)`
`x.cpu()` / `x.cuda()`	Shortcuts for moving	`x.cpu().numpy()`
`torch.cuda.memory_allocated()`	Bytes held by live tensors	`torch.cuda.memory_allocated() / 1e9`
`torch.cuda.max_memory_allocated()`	Peak bytes since last reset	log it once per epoch
`torch.cuda.memory_summary()`	Readable report for OOM debugging	`print(torch.cuda.memory_summary())`
`torch.cuda.empty_cache()`	Return cached blocks to the driver (rarely helps)	`torch.cuda.empty_cache()`
`torch.autocast`	Run matmuls in lower precision automatically	`with torch.autocast("cuda", dtype=torch.bfloat16): ...`
`torch.amp.GradScaler`	Scale the loss so float16 gradients do not underflow	`scaler = torch.amp.GradScaler()`
`torch.utils.checkpoint`	Trade compute for memory by recomputing activations	`checkpoint(block, x, use_reentrant=False)`
`torch.inference_mode()`	Fastest no-gradient mode for serving	`with torch.inference_mode(): ...`
`model.half()` / `.to(torch.bfloat16)`	Cast weights to 16 bits	`model.to(torch.bfloat16)`
`CUDA_VISIBLE_DEVICES`	Environment variable that hides other GPUs	`CUDA_VISIBLE_DEVICES=1 python train.py`

The standard device boilerplate (not run here). Pick the device once, put it in one variable, and move the *model once* and every *batch* each step:

```
import torch

if torch.cuda.is_available():
    device = torch.device("cuda")
elif torch.backends.mps.is_available():
    device = torch.device("mps")
else:
    device = torch.device("cpu")
print("using", device)

model = model.to(device)                  # for a Module, .to() is in place
x = x.to(device)                          # for a Tensor, .to() RETURNS a new tensor: assign it!
with torch.autocast(device_type=device.type, dtype=torch.bfloat16, enabled=device.type != "mps"):
    logits = model(x)
print(logits.device, logits.dtype)

if device.type == "cuda":
    print(torch.cuda.get_device_name(0))
    print(torch.cuda.memory_allocated() / 1e9, "GB held,", torch.cuda.max_memory_allocated() / 1e9, "GB peak")
```

How much memory does training need? Do the arithmetic first

Training memory has four parts: **parameters**, **gradients** (one per parameter), **optimiser state** (Adam keeps two extra numbers per parameter) and **activations** (the intermediate results saved for backward, which grow with batch size and sequence length). The first three depend only on the parameter count P and the bytes per number. In plain fp32 with Adam that is 4 + 4 + 8 = 16 bytes per parameter, before any activations.

Problem 21.5.1 How much memory do the parameters, gradients and optimiser state need for a 7-billion-parameter model? By hand: 7e9 x 16 bytes = 112e9 bytes = 112 GB (decimal gigabytes). Compare with inference in 16-bit (2 bytes per parameter = 14 GB), a 4-bit quantised copy (0.5 bytes = 3.5 GB) and LoRA fine-tuning where the frozen base is in 16-bit and only 0.5 percent of the parameters train in fp32 with Adam.

```
def training_gb(n_params, w_bytes, g_bytes, opt_bytes, trainable_frac=1.0, base_bytes=None):
    """Bytes for weights + grads + optimiser state (activations excluded)."""
    trainable = n_params * trainable_frac
    frozen = n_params - trainable
    base = frozen * (base_bytes if base_bytes is not None else w_bytes)
    total = base + trainable * (w_bytes + g_bytes + opt_bytes)
    return total / 1e9

P = 7e9
rows = {
    "full fine-tune, fp32 + Adam":      training_gb(P, 4, 4, 8),
    "all bf16 (weights, grads, Adam)":   training_gb(P, 2, 2, 4),
    "LoRA 0.5% trainable, bf16 base":    training_gb(P, 4, 4, 8, trainable_frac=0.005, base_bytes=2),
    "inference, bf16":                   P * 2 / 1e9,
    "inference, 4-bit quantised":        P * 0.5 / 1e9,
}
for name, gb in rows.items():
    print(f"{name:34s} {gb:7.1f} GB")

# batch size finder: halve until the estimate fits, like an OOM retry loop
def max_batch(budget_gb, fixed_gb, per_sample_gb, start=256):
    b = start
    while b > 0 and fixed_gb + b * per_sample_gb > budget_gb:
        b //= 2
    return b
print("largest batch that fits 24 GB with 14.5 GB fixed:", max_batch(24, 14.5, 0.35))
```

```
full fine-tune, fp32 + Adam          112.0 GB
all bf16 (weights, grads, Adam)       56.0 GB
LoRA 0.5% trainable, bf16 base        14.5 GB
inference, bf16                       14.0 GB
inference, 4-bit quantised             3.5 GB
largest batch that fits 24 GB with 14.5 GB fixed: 16
```

1. The function multiplies bytes per parameter for the trainable part and uses a smaller footprint for frozen weights. The first row is the hand calculation: 7e9 x 16 = 112 GB.
2. Keeping everything in bf16 (2 + 2 + 4 = 8 bytes per parameter) halves it to 56 GB. This is aggressive and can hurt accuracy because tiny weight updates are lost in 16-bit rounding (see the next example).
3. LoRA stores the frozen base at 2 bytes per parameter (about 13.9 GB for the 99.5 percent that is frozen) and pays the 16-byte price only on 0.5 percent of the parameters (0.56 GB), so the total is about 14.5 GB. This is why a model that cannot be fully fine-tuned on one 24 GB GPU can be LoRA-tuned on it.
4. The two inference rows show the saving from fewer bits: 14 GB and 3.5 GB.
5. `max_batch` imitates the standard defence against out-of-memory errors: start big, halve until it fits. With 24 GB, 14.5 GB already used and an assumed 0.35 GB per sample, it settles on a batch of 16 (0.35 x 16 = 5.6 GB, total 20.1 GB; 32 would need 25.7 GB). The 0.35 GB per sample is an invented illustration; measure your own with `max_memory_allocated`.

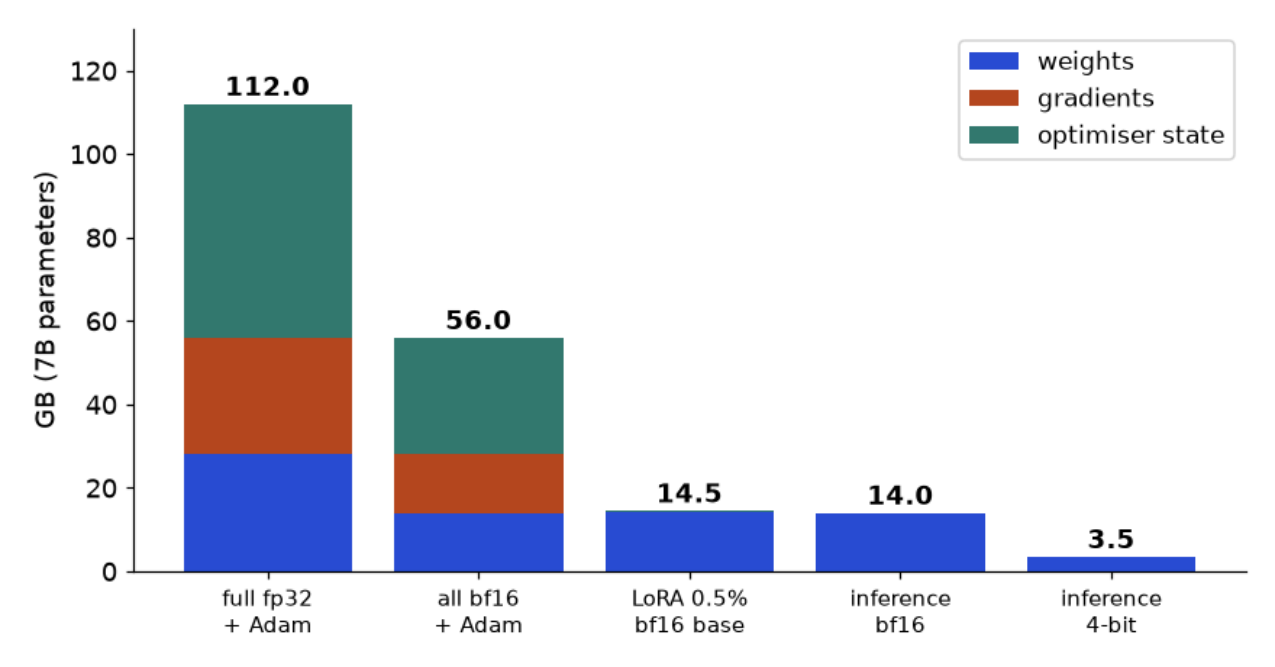


Figure 21.5. Memory for weights, gradients and optimiser state of a 7-billion-parameter model (activations not included). The numbers are arithmetic from bytes per parameter.

How to read it: each bar is one setup. Blue is the weights, orange the gradients, teal the optimiser state. The full fp32 bar is dominated by the optimiser and gradients; LoRA and inference bars are almost all blue, because nothing but the (frozen) weights needs to be stored.

Why precision matters: float16, bfloat16 and float32

float32 has 8 exponent bits and 23 mantissa bits. float16 has 5 exponent bits and 10 mantissa bits: more precise than bfloat16 but its largest number is only 65,504, so large activations overflow to infinity. bfloat16 keeps float32's 8 exponent bits (same range) but only 7 mantissa bits, so it is coarse yet rarely overflows. That is why bfloat16 needs no loss scaling and float16 does.

```
import numpy as np

print("float16 max:", np.finfo(np.float16).max, " eps:", np.finfo(np.float16).eps)
print("float32 max:", f"{np.finfo(np.float32).max:.3e}", " eps:", f"{np.finfo(np.float32).eps:.3e}")
print("bfloat16 max (computed): ", f"{(2 - 2**-7) * 2.0**127:.3e}", " eps:", 2**-7)

big = np.float32(70000.0)
print("70000 in float16:", np.float16(big))

# emulate bfloat16 by keeping only the top 16 bits of a float32
def to_bf16(x):
    u = np.asarray(x, dtype=np.float32).view(np.uint32)
    return (u & np.uint32(0xFFFF0000)).view(np.float32)
print("70000 in bfloat16 (truncated):", to_bf16(70000.0))

# small updates vanish when added to a large weight
w16 = np.float16(1.0)
print("1.0 + 0.0004 in float16:", w16 + np.float16(0.0004))
print("1.0 + 0.0004 in float32:", np.float32(1.0) + np.float32(0.0004))

# accumulating in low precision stalls
s16, s32 = np.float16(0.0), np.float32(0.0)
for _ in range(20000):
    s16 = np.float16(s16 + np.float16(0.0001))
    s32 = np.float32(s32 + np.float32(0.0001))
print("sum of 20000 x 0.0001 -> float16:", s16, " float32:", round(float(s32), 4))
```

```
float16 max: 6.55e+04  eps: 0.000977
float32 max: 3.403e+38  eps: 1.192e-07
bfloat16 max (computed):  3.390e+38  eps: 0.0078125
70000 in float16: inf
70000 in bfloat16 (truncated): 69632.0
1.0 + 0.0004 in float16: 1.0
1.0 + 0.0004 in float32: 1.0004
sum of 20000 x 0.0001 -> float16: 0.25  float32: 2.0002
```

1. `np.finfo` reports the limits of a type. float16 tops out at 65504, so a value of 70000 becomes `inf`, as the next line shows. This is the overflow that makes float16 training unstable without a loss scaler.
2. bfloat16 does not exist in NumPy, so `to_bf16` emulates it: a float32 is 32 bits, and dropping the lowest 16 bits keeps the sign, the full 8-bit exponent and 7 mantissa bits. 70000 survives, as 69632 after the truncation, which shows the price: only about 2 to 3 significant digits.
3. Adding 0.0004 to 1.0 in float16 returns exactly 1.0, because the spacing between neighbouring float16 numbers near 1 is about 0.001. Updates smaller than half that spacing vanish. In float32 the addition works (1.0004).
4. The loop adds 0.0001 twenty thousand times. The correct total is 2.0. In float16 the running sum stops growing at 0.25, where the spacing between neighbouring numbers is about 0.0002 and adding 0.0001 rounds back to the same value. Even float32 drifts slightly (2.0002) because 0.0001 is not exactly representable; it is just far more accurate. This is why mixed precision keeps master weights, loss and reductions in float32.

KEY IDEA

Mixed precision means: do the heavy matrix multiplications in 16 bits, but keep the sensitive things (master weights, optimiser state, loss, sums) in 32 bits. `torch.autocast` chooses the precision per operation for you. On recent NVIDIA GPUs prefer bfloat16 (no scaler needed); use float16 with `GradScaler` on older ones.

Gradient accumulation: a big batch from small pieces

When a batch of 64 does not fit, run 4 micro-batches of 16, divide each loss by 4, call `backward()` on each (gradients add up in `.grad`, which is the accumulation behaviour from 21.2) and call `step()` once. The result equals the big-batch gradient exactly for losses that are averages over examples. Let us prove it.

```
import numpy as np

rng = np.random.default_rng(3)
X = rng.normal(size=(64, 5)); y = rng.normal(size=64)
w = rng.normal(size=5)

def grad(Xb, yb, w):                       # gradient of the MEAN squared error
    return 2 / len(yb) * Xb.T @ (Xb @ w - yb)

full = grad(X, y, w)

acc = np.zeros(5)
for i in range(0, 64, 16):                 # 4 micro-batches of 16
    acc += grad(X[i:i + 16], y[i:i + 16], w) / 4      # divide by the number of micro-batches
print("identical to full batch:", np.allclose(full, acc))

wrong = np.zeros(5)
for i in range(0, 64, 16):
    wrong += grad(X[i:i + 16], y[i:i + 16], w)        # forgot the division
print("without /4 the gradient is", round(float(wrong[0] / full[0]), 3), "times too large")
```

```
identical to full batch: True
without /4 the gradient is 4.0 times too large
```

1. The target is the gradient of the mean squared error over all 64 rows.
2. The loop visits four slices of 16 rows. Each slice has its own mean, so averaging four slice-means (each divided by 4 before adding) equals the overall mean when the slices are the same size.
3. `np.allclose` prints `True`.
4. If you skip the division, the gradient is 4 times too big, which acts like a learning rate four times larger and may diverge. The printed ratio is 4.0.
5. In PyTorch: `(loss / accum_steps).backward()` every micro-batch, then `opt.step(); opt.zero_grad()` only every `accum_steps` micro-batches. Batch-norm statistics still see only the micro-batch, so accumulation is not perfectly identical for networks that use it.

COMMON MISTAKE

Wrong: `x.to(device)` on its own line, or `model = model.to(device)` but `x = x` left on the CPU. **Symptom:** `RuntimeError: Expected all tensors to be on the same device`. **Fix:** tensors are not moved in place, so write `x = x.to(device)`; modules are moved in place, so either style works for a model. Move every field of a batch, including masks and labels.

21.6 The Shape-Debugging Guide

Most deep learning bugs are shape bugs, and the most dangerous shape bugs do not raise errors: broadcasting silently produces an answer of the wrong size or meaning. The cure is a habit: **write the shape of every tensor in a comment, print shapes at the first batch, and assert the ones that matter.**

Data kind	Usual shape	Letters
Tabular batch	(B, F)	B batch, F features
Token ids	(B, T)	T sequence length
Embedded tokens	(B, T, E)	E embedding size
Multi-head attention tensors	(B, H, T, D)	H heads, D = E / H
Attention scores	(B, H, T, T)	every token against every token

Data kind	Usual shape	Letters
Images	(B, C, H, W)	channels first in PyTorch (NumPy and PIL are channels last)
Classifier logits	(B, K)	K classes; labels are (B,) integers
Language-model logits	(B, T, V)	V vocabulary; cross-entropy wants (B*T, V) and (B*T,)

A shape trace through multi-head attention

Attention is where shapes get hardest. The code below runs a real multi-head self-attention in NumPy, prints the shape after each step, and uses a tiny `check` helper that fails loudly with a readable message. Every line has the PyTorch equivalent in its comment.

```
import numpy as np

def check(x, spec, **dims):
    """Assert a shape against letters, e.g. check(x, 'B T E', B=2)."""
    names = spec.split()
    assert x.ndim == len(names), f"expected {len(names)} dims ({spec}), got shape {x.shape}"
    for n, s in zip(names, x.shape):
        if n in dims:
            assert dims[n] == s, f"dim {n}: expected {dims[n]}, got {s} (shape {x.shape})"
        dims[n] = s
    return dims

rng = np.random.default_rng(0)
B, T, E, H = 2, 5, 8, 2
D = E // H
x = rng.normal(size=(B, T, E)); check(x, "B T E", B=B, T=T, E=E)
Wq, Wk, Wv, Wo = (rng.normal(scale=0.3, size=(E, E)) for _ in range(4))
mask = np.array([[1, 1, 1, 1, 1], [1, 1, 1, 0, 0]])          # second ticket has 2 pad tokens

q = x @ Wq                                                    # (B, T, E)
k = x @ Wk
v = x @ Wv
def split_heads(t):                                           # t.view(B, T, H, D).transpose(1, 2)
    return t.reshape(B, T, H, D).transpose(0, 2, 1, 3)        # (B, H, T, D)
qh, kh, vh = split_heads(q), split_heads(k), split_heads(v)
print("heads      :", qh.shape)

scores = qh @ kh.transpose(0, 1, 3, 2) / np.sqrt(D)           # (B, H, T, T)
print("scores     :", scores.shape)
scores = np.where(mask[:, None, None, :] == 1, scores, -1e9)   # mask: (B,1,1,T) broadcasts over H and rows
scores = scores - scores.max(-1, keepdims=True)
att = np.exp(scores); att /= att.sum(-1, keepdims=True)       # softmax over the LAST dim (keys)
print("attention  :", att.shape, " rows sum to 1:", np.allclose(att.sum(-1), 1))
print("pad weight in ticket 2:", round(float(att[1, :, :, 3:].sum()), 6))

ctx = att @ vh                                                # (B, H, T, D)
merged = ctx.transpose(0, 2, 1, 3).reshape(B, T, E)           # back to (B, T, E)
out = merged @ Wo
print("context    :", ctx.shape, " merged:", merged.shape, " out:", out.shape)
print(check(out, "B T E", B=B, T=T, E=E))

try:
    check(out.reshape(B * T, E), "B T E", B=B)
except AssertionError as e:
    print("caught:", e)
```

```
heads      : (2, 2, 5, 4)
scores     : (2, 2, 5, 5)
attention  : (2, 2, 5, 5)  rows sum to 1: True
pad weight in ticket 2: 0.0
context    : (2, 2, 5, 4)  merged: (2, 5, 8)  out: (2, 5, 8)
{'B': 2, 'T': 5, 'E': 8}
caught: expected 3 dims (B T E), got shape (10, 8)
```

1. `check` maps letters to sizes the first time it sees them and verifies them afterwards. It turns a mystery crash 30 lines later into an immediate message that names the dimension.
2. Queries, keys and values are the same input multiplied by three weight matrices: shape `(B, T, E)` each. `split_heads` reshapes the last dimension E into (H, D) and then moves the head dimension forward, giving `(B, H, T, D)`. In PyTorch: `t.view(B, T, H, D).transpose(1, 2)`.
3. Scores are every query against every key: `(B, H, T, D) @ (B, H, D, T)` gives `(B, H, T, T)`. Dividing by the square root of D keeps the numbers in a range where softmax is not saturated.
4. The padding mask has shape `(B, T)`. Inserting two length-1 axes makes it `(B, 1, 1, T)`, which broadcasts over heads and over query rows. Masked positions get a huge negative score, so softmax gives them weight 0. The printout confirms the second ticket puts exactly zero attention on its padding.
5. Softmax runs over the *last* axis (the keys), so each query row sums to one.
6. After attention, `transpose(0, 2, 1, 3)` followed by `reshape` puts the heads side by side again. In PyTorch you must call `.contiguous()` before `.view()` here, or use `.reshape()`.
7. The last test feeds a deliberately wrong shape and the helper raises a readable assertion (printed after "caught:").

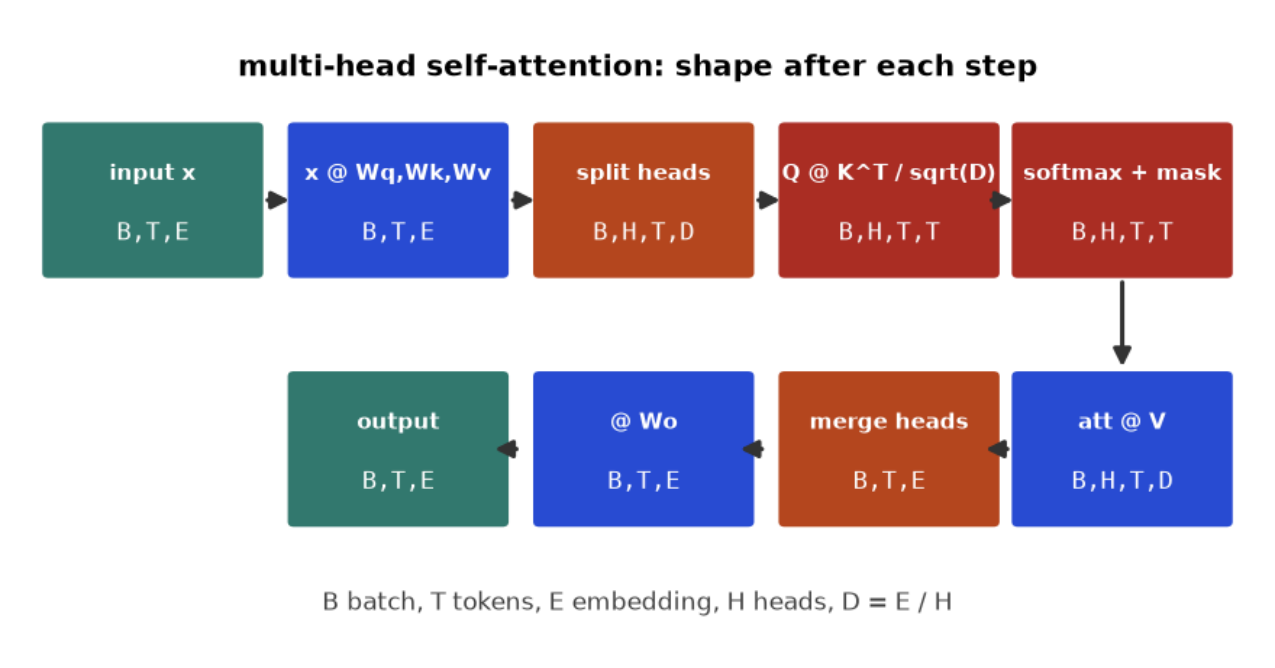


Figure 21.6. The shape of the tensor after each step of multi-head attention. Only the last two or three dimensions change meaning at each step.

How to read it: follow the boxes from left to right. The batch dimension B never changes. The blue boxes are projections, the orange ones are the head split and merge (pure shape changes with no arithmetic), and the red box is the only place where T appears twice, which is why attention memory grows with the square of the sequence length.

The silent broadcasting bug

Not every shape bug crashes. Subtracting a column from a row broadcasts into a full matrix, a classic cause of a loss that "works" but never improves.

```
import numpy as np

pred = np.array([0.9, 0.2, 0.7, 0.4])            # shape (4,)
target = np.array([[1.0], [0.0], [1.0], [0.0]])  # shape (4, 1), e.g. from y.unsqueeze(1)

diff = pred - target                             # (4,) - (4,1) -> (4,4)
print("diff shape:", diff.shape)
print("wrong MSE :", round(float(np.mean(diff ** 2)), 4))

fixed = pred - target.ravel()                    # both (4,)
print("right MSE :", round(float(np.mean(fixed ** 2)), 4))
```

```
diff shape: (4, 4)
wrong MSE : 0.325
right MSE : 0.075
```

1. A vector of shape `(4,)` aligned from the right with `(4, 1)` becomes `(1, 4)` versus `(4, 1)`, so both stretch to `(4, 4)`: every prediction is compared with every target.
2. The wrong mean is computed over 16 meaningless pairs and is much larger than the true error; the right one is over 4 matching pairs.
3. No exception occurs. PyTorch's `nn.MSELoss` warns in this case ("Using a target size that is different to the input size"), but your own arithmetic does not.
4. Defence: assert `pred.shape == target.shape` before computing a custom loss.

21.7 Hugging Face: transformers, tokenizers, datasets and peft

INTUITION

Hugging Face is a library of *parts that fit together by convention*. A checkpoint name such as a model id on the Hub points to three things: a tokeniser (text to ids), a config (the architecture) and weights. The `Auto*` classes read the config and build the right architecture for you. The `datasets` library holds your examples in a fast disk-backed table. The `Trainer` is a ready-made training loop like the one you wrote in 21.4. `peft` freezes the big model and trains tiny add-on matrices.

Reference: the most important Hugging Face names

Name	What it does in plain English	Minimal example (not run here)
`AutoTokenizer.from_pretrained`	Load the tokeniser that matches a model	`tok = AutoTokenizer.from_pretrained(name)`
`tok(text, ...)`	Text to `input_ids` and `attention_mask`	`tok(["a", "bb"], padding=True, return_tensors="pt")`
`padding / truncation / max_length`	Make a batch rectangular and bounded	`tok(x, truncation=True, max_length=128)`
`tok.tokenize / convert_ids_to_tokens`	See the pieces	`tok.tokenize("unbelievable")`
`tok.decode`	Ids back to text	`tok.decode(ids, skip_special_tokens=True)`
`tok.pad_token / eos_token`	Special tokens; set pad if missing	`tok.pad_token = tok.eos_token`
`AutoModel`	Bare encoder: hidden states only	`AutoModel.from_pretrained(name)`
`AutoModelForSequenceClassification`	Encoder plus a classification head	`...from_pretrained(name, num_labels=3)`

Name	What it does in plain English	Minimal example (not run here)
`AutoModelForCausalLM`	A text-continuing language model	`AutoModelForCausalLM.from_pretrained(name)`
`from_pretrained(dtype, device_map)`	Load in 16-bit and spread across devices (older releases call the first argument `torch_dtype`)	`..., dtype=torch.bfloat16, device_map="auto"`
`model.generate`	Produce new tokens	`model.generate(**batch, max_new_tokens=50)`
`pipeline(task)`	One-line tokenise, model, decode	`pipeline("text-classification", model=name)`
`TrainingArguments`	All training settings in one object	`TrainingArguments(output_dir="out", learning_rate=2e-5)`
`Trainer`	Ready-made loop with eval, saving, logging	`Trainer(model=m, args=a, train_dataset=tr)`
`DataCollatorWithPadding`	Dynamic padding per batch	`DataCollatorWithPadding(tok)`
`model.save_pretrained / from_pretrained`	Save and reload weights plus config	`model.save_pretrained("dir")`
`load_dataset`	Load from the Hub, files or scripts	`load_dataset("csv", data_files="t.csv")`
`ds.map(fn, batched=True)`	Apply a function to many rows at a time	`ds.map(tokenize, batched=True)`
`ds.filter / select / shuffle`	Subset and reorder rows	`ds.filter(lambda r: r["n"] > 3)`
`ds.train_test_split`	Make train and test splits	`ds.train_test_split(test_size=0.2, seed=0)`
`ds.set_format("torch")`	Return tensors instead of lists	`ds.set_format("torch", columns=["input_ids"])`
`ds.to_pandas / Dataset.from_pandas`	Convert to and from a DataFrame	`Dataset.from_pandas(df)`
`load_dataset(..., streaming=True)`	Iterate without downloading everything	`for row in ds: ...`
`LoraConfig`	Choose rank r, scaling alpha, target layers	`LoraConfig(r=8, lora_alpha=16, target_modules=["q_proj", "v_proj"])`
`get_peft_model`	Wrap a model with trainable adapters	`model = get_peft_model(base, cfg)`
`print_trainable_parameters`	Show the small trainable share	`model.print_trainable_parameters()`
`merge_and_unload`	Fold adapters into the base weights	`merged = model.merge_and_unload()`

Tokenisers: from text to ids

Models cannot read letters; they read integer ids. A tokeniser splits text into pieces from a fixed vocabulary of **subwords**. Common words are one piece; rare words are cut into smaller known pieces; nothing is ever completely unknown. The most common method, **byte-pair encoding (BPE)**, builds the vocabulary by repeatedly merging the most frequent adjacent pair of symbols. The tokeniser also adds an `attention_mask` telling the model which positions are real.

Idiomatic use (not run here):

```
from transformers import AutoTokenizer

name = "your-model-id"                      # any Hub id, e.g. a BERT-style or small encoder model
tok = AutoTokenizer.from_pretrained(name)

batch = tok(["reset password", "refund the billing charge twice"],
            padding=True, truncation=True, max_length=128, return_tensors="pt")
print(batch.keys())                          # input_ids, attention_mask (and sometimes token_type_ids)
print(batch["input_ids"].shape)              # (2, T) where T = longest in this batch
print(tok.convert_ids_to_tokens(batch["input_ids"][0]))
print(tok.decode(batch["input_ids"][0], skip_special_tokens=True))

# decoder-only models often have no pad token: set one before padding a batch
if tok.pad_token is None:
    tok.pad_token = tok.eos_token
tok.padding_side = "left"                    # for generation with decoder-only models
```

Now a working miniature. This BPE tokeniser trains on a small corpus, splits unseen words into known pieces and returns the same dictionary keys as the Hugging Face call.

Problem 21.7.1 Train 50 merges on eight support sentences. Show the first merges, split an unseen word, and encode a padded batch.

```
from collections import Counter

corpus = ["refund the billing charge", "billing charge refunded twice", "cannot reset password",
          "password reset link broken", "charge appears twice on billing page", "reset the password link",
          "refund requested for twice charged order", "how to reset billing password"]

def train_bpe(texts, n_merges):
    words = Counter()
    for t in texts:
        for w in t.lower().split():
            words[tuple(w) + ("</w>",)] += 1
    merges = []
    for _ in range(n_merges):
        pairs = Counter()
        for w, c in words.items():
            for a, b in zip(w, w[1:]):
                pairs[(a, b)] += c
        if not pairs:
            break
        best = max(pairs, key=lambda p: (pairs[p], p))      # most frequent pair, ties by name
        merges.append(best)
        new = Counter()
        for w, c in words.items():
            out, i = [], 0
            while i < len(w):
                if i < len(w) - 1 and (w[i], w[i + 1]) == best:
                    out.append(w[i] + w[i + 1]); i += 2
                else:
                    out.append(w[i]); i += 1
            new[tuple(out)] += c
        words = new
    return merges

class BPETokenizer:
    def __init__(self, merges, corpus):
        self.rank = {m: i for i, m in enumerate(merges)}
        chars = sorted({ch for t in corpus for ch in t.lower() if ch != " "})
        toks = ["<pad>", "<unk>"] + chars + ["</w>"] + ["".join(m) for m in merges]
        self.vocab = {t: i for i, t in enumerate(dict.fromkeys(toks))}
        self.inv = {i: t for t, i in self.vocab.items()}
    def tokenize(self, text):
        out = []
        for w in text.lower().split():
            syms = list(w) + ["</w>"]
            while len(syms) > 1:
                cand = [(self.rank[p], i) for i, p in enumerate(zip(syms, syms[1:])) if p in self.rank]
                if not cand:
                    break
                _, i = min(cand)                            # apply the earliest-learned merge first
                syms[i:i + 2] = [syms[i] + syms[i + 1]]
            out.extend(syms)
        return out
    def __call__(self, texts, padding=False, truncation=False, max_length=None):
        ids = [[self.vocab.get(t, 1) for t in self.tokenize(x)] for x in texts]
        if truncation and max_length:
            ids = [s[:max_length] for s in ids]
        longest = max(len(s) for s in ids) if padding else 0
        mask = [[1] * len(s) + [0] * (longest - len(s)) for s in ids]
        ids = [s + [0] * (longest - len(s)) for s in ids]
        return {"input_ids": ids, "attention_mask": mask}
    def decode(self, ids):
        return "".join(self.inv[i] for i in ids if i != 0).replace("</w>", " ").strip()
```

```
merges = train_bpe(corpus, 50)
tok = BPETokenizer(merges, corpus)
print("first merges:", merges[:5])
print("vocab size:", len(tok.vocab))
print("seen    :", tok.tokenize("refund the billing"))
print("unseen  :", tok.tokenize("refunding passwords"))
print("unknown :", tok.tokenize("xyz"))
batch = tok(["reset password", "refund the billing charge twice"], padding=True)
print("shape   :", len(batch["input_ids"]), "x", len(batch["input_ids"][0]))
print("mask    :", batch["attention_mask"])
print("decoded :", tok.decode(batch["input_ids"][0]))
print("truncated:", tok(["refund the billing charge twice"], truncation=True, max_length=4)["input_ids"])
```

```
first merges: [('e', '</w>'), ('d', '</w>'), ('r', 'e'), ('o', 'r'), ('l', 'i')]
vocab size: 73
seen    : ['refund</w>', 'the</w>', 'billing</w>']
unseen  : ['refund', 'i', 'n', 'g', '</w>', 'pa', 's', 's', 'wor', 'd', 's</w>']
unknown : ['x', 'y', 'z', '</w>']
shape   : 2 x 5
mask    : [[1, 1, 0, 0, 0], [1, 1, 1, 1, 1]]
decoded : reset password
truncated: [[59, 58, 47, 56]]
```

1. `train_bpe` starts with each word as a tuple of characters plus an end-of-word marker. Each round it counts all adjacent pairs (weighted by word frequency), merges the most frequent pair everywhere, and records the merge. Fifty rounds leave an ordered list of merge rules.
2. The `BPETokenizer` vocabulary contains special tokens first (`<pad>` id 0, `<unk>` id 1), then single characters, then one entry per merge. A real tokeniser works the same way with a vocabulary of tens of thousands.
3. `tokenize` applies the earliest-learned merge first, repeatedly. A word seen in training collapses into few pieces. The unseen word "refunding" still works because it breaks into known smaller pieces (see the printed lists).
4. A word made of letters outside the training alphabet falls back to the `<unk>` id when mapped to ids. Real byte-level tokenisers avoid `<unk>` entirely by starting from the 256 bytes.
5. The call returns the same two keys as Hugging Face. The shorter sequence is padded with zeros and its mask has zeros in those places.
6. `decode` reverses the mapping. `truncation=True, max_length=4` simply cuts the id list. In real models truncation also reserves room for special tokens; keep an eye on this when comparing lengths.

KEY IDEA	Always use the tokeniser that was trained with the model. A different vocabulary maps the same id to a different piece, and the model produces garbage without raising any error. Token counts, not characters or words, decide cost, speed and the context limit of an LLM call.

datasets: a table that does not live in RAM

A Hugging Face `Dataset` is a typed table stored on disk (Apache Arrow) and memory-mapped, with results of `map` cached. Two ideas matter most. **Batched map**: your function receives a dictionary of lists and returns a dictionary of lists, which is much faster than one row at a time. **Immutability**: every operation returns a *new* dataset; writing `ds.map(f)` and ignoring the result does nothing (the same trap as DataFrame methods).

```
from datasets import load_dataset, Dataset

ds = load_dataset("csv", data_files={"train": "tickets_train.csv", "test": "tickets_test.csv"})
print(ds)                                   # DatasetDict with 'train' and 'test'
print(ds["train"].features)                 # column types

label2id = {"billing": 0, "bug": 1, "how-to": 2}

def prepare(batch):                         # batched=True: batch is a dict of lists
    enc = tok(batch["text"], truncation=True, max_length=128)
    enc["labels"] = [label2id[x] for x in batch["label"]]
    return enc

ds = ds.map(prepare, batched=True, remove_columns=["text", "label"])
ds = ds.filter(lambda row: len(row["input_ids"]) > 2)
split = ds["train"].train_test_split(test_size=0.1, seed=0)
ds.set_format("torch")                      # rows now come back as tensors
small = ds["train"].shuffle(seed=0).select(range(1000))      # quick debugging subset
df = ds["train"].to_pandas()
streamed = load_dataset("csv", data_files="huge.csv", split="train", streaming=True)
for row in streamed.take(2):
    print(row)
```

The pandas stand-in below uses the same moves: a batched `map`, `filter`, a seeded `train_test_split` and a streaming read. It also shows the pandas 3 string dtype.

Problem 21.7.2 Prepare ten tickets: add a token count, keep tickets with at least 3 tokens, split 80/20 with a seed, and stream the CSV in chunks of 4 rows.

```
import io
import pandas as pd
from sklearn.model_selection import train_test_split

df = pd.DataFrame({
    "text": ["refund the charge", "app crashes on login", "how to export data", "charged twice", "button does nothing",
             "reset my password please", "invoice missing", "error 500 on save", "where is the api key", "ok"],
    "label": ["billing", "bug", "how-to", "billing", "bug", "how-to", "billing", "bug", "how-to", "bug"],
})
print("text dtype:", df["text"].dtype)

def add_tokens(batch):                      # batched map: dict of lists in, dict of lists out
    return {"n_tokens": [len(t.split()) for t in batch["text"]]}

new_cols = add_tokens({"text": df["text"].tolist()})
df = df.assign(**new_cols)                  # returns a NEW frame, like Dataset.map
df = df[df["n_tokens"] >= 3].reset_index(drop=True)      # like Dataset.filter
print("rows kept:", len(df))

train, test = train_test_split(df, test_size=0.2, random_state=0, stratify=None)
print("train/test:", len(train), len(test), "| test rows:", list(test["text"]))

csv_text = df.to_csv(index=False)
seen = 0
for chunk in pd.read_csv(io.StringIO(csv_text), chunksize=4):     # like streaming=True
    seen += len(chunk)
    print("chunk of", len(chunk), "rows; first label:", chunk["label"].iloc[0])
print("streamed rows:", seen)
```

```
text dtype: str
rows kept: 7
train/test: 5 2 | test rows: ['where is the api key', 'how to export data']
chunk of 4 rows; first label: billing
chunk of 3 rows; first label: how-to
streamed rows: 7
```

1. In pandas 3 a text column has the dedicated string dtype (printed as `str`), not `object`. Hugging Face stores strings in Arrow, which has the same idea.
2. `add_tokens` has the batched-map shape: it receives a dictionary of columns (lists) and returns a dictionary of new columns. Writing the function this way means the same function works unchanged in `Dataset.map(add_tokens, batched=True)`.
3. `df.assign` and boolean filtering return new objects. We reassign to `df`; forgetting that is the same bug as ignoring the result of `ds.map`.
4. The filter removes the three tickets with fewer than 3 words ("charged twice", "invoice missing" and "ok"), leaving 7 rows. A 20 percent test share of 7 rows rounds up to 2 test rows and 5 train rows, and the seed makes the chosen test rows repeatable.
5. `chunksize=4` reads the file in pieces without loading it all, the idea behind `streaming=True`. With 7 rows we see a chunk of 4 and a chunk of 3.

Fine-tuning with Trainer (not run here)

This is the complete recipe for fine-tuning a pretrained encoder as a ticket classifier. Compare it with the loop you wrote in 21.4: `Trainer` does the same forward, loss, backward, step, evaluate, save, early stop and logging, configured by `TrainingArguments`.

```
import numpy as np
from transformers import (AutoTokenizer, AutoModelForSequenceClassification,
                          DataCollatorWithPadding, TrainingArguments, Trainer)

name = "your-model-id"
tok = AutoTokenizer.from_pretrained(name)
model = AutoModelForSequenceClassification.from_pretrained(
    name, num_labels=3, id2label={0: "billing", 1: "bug", 2: "how-to"},
    label2id={"billing": 0, "bug": 1, "how-to": 2})

def compute_metrics(eval_pred):
    logits, labels = eval_pred                      # numpy arrays
    pred = logits.argmax(axis=-1)
    return {"accuracy": float((pred == labels).mean())}

args = TrainingArguments(
    output_dir="out", num_train_epochs=3, learning_rate=2e-5, weight_decay=0.01,
    per_device_train_batch_size=16, per_device_eval_batch_size=64,
    eval_strategy="epoch", save_strategy="epoch",   # called evaluation_strategy in older releases
    load_best_model_at_end=True, metric_for_best_model="accuracy",
    logging_steps=50, seed=42, report_to="none",
    bf16=False,                                     # set True on a GPU that supports bfloat16
)
trainer = Trainer(
    model=model, args=args,
    train_dataset=ds["train"], eval_dataset=ds["test"],
    data_collator=DataCollatorWithPadding(tok),     # pads each batch to its longest
    compute_metrics=compute_metrics,
    processing_class=tok,                           # called tokenizer= in older releases
)
trainer.train()
print(trainer.evaluate())
trainer.save_model("ticket-classifier")
tok.save_pretrained("ticket-classifier")

# inference
from transformers import pipeline
clf = pipeline("text-classification", model="ticket-classifier")
print(clf(["I was charged twice this month"]))
```

COMMON MISTAKE

Wrong: padding every example to `max_length=512` in the `map` step "to be safe". **Symptom:** training is several times slower and uses far more memory, because most positions are padding. **Fix:** tokenise without padding and use `DataCollatorWithPadding`, which pads each batch to its own longest example.

peft and LoRA: train a small update instead of the whole matrix

A weight matrix W of shape (d_{out}, d_{in}) normally changes by a full matrix of updates during fine-tuning. **LoRA** (low-rank adaptation) freezes W and learns the update as a product of two thin matrices: ΔW = (alpha / r) B A, where A has shape (r, d_{in}), B has shape (d_{out}, r) and r is small (4 to 64). B starts at zero so the model starts exactly equal to the base model. For d = 4096 and r = 8, the update has 2 x 4096 x 8 = 65,536 numbers instead of 16.8 million.

```
from peft import LoraConfig, get_peft_model, TaskType, PeftModel

base = AutoModelForCausalLM.from_pretrained("your-model-id", dtype=torch.bfloat16, device_map="auto")
cfg = LoraConfig(task_type=TaskType.CAUSAL_LM, r=8, lora_alpha=16, lora_dropout=0.05,
                 target_modules=["q_proj", "v_proj"])         # names depend on the architecture
model = get_peft_model(base, cfg)
model.print_trainable_parameters()          # prints trainable params, total params and the percentage
# ... train with Trainer or your own loop; only adapter weights have requires_grad=True ...
model.save_pretrained("ticket-lora")        # saves just the small adapter (megabytes)

# serving: reload base + adapter, optionally fold the adapter in for speed
base = AutoModelForCausalLM.from_pretrained("your-model-id", dtype=torch.bfloat16)
model = PeftModel.from_pretrained(base, "ticket-lora")
merged = model.merge_and_unload()           # plain model again, no adapter overhead
```

Now the same mechanics in NumPy. A frozen weight matrix W0 is shifted by a hidden rank-4 change. We learn the change with only A and B, then merge.

Problem 21.7.3 Let W0 be a 64 x 64 frozen matrix and the target be W0 plus a hidden rank-4 matrix. Train a LoRA update with r = 4 and alpha = 8 (scale s = alpha / r = 2) using gradients computed by hand, then verify that merging gives the same outputs.

Model	output = x (W0 + s B A)^T. Parameters: A is 4 x 64 and B is 64 x 4, so 512 numbers instead of 4,096.
Gradient	With G = dL/dW (a 64 x 64 matrix from the usual layer formula), the chain rule gives dL/dB = s G A^T and dL/dA = s B^T G.
Start	B = 0 and A small random, so the first prediction equals the base model.

```
import numpy as np

rng = np.random.default_rng(0)
d, r, N = 64, 4, 512
W0 = rng.normal(size=(d, d)) / np.sqrt(d)                       # frozen base weight
delta = rng.normal(size=(d, r)) @ rng.normal(size=(r, d)) / np.sqrt(d * r)   # hidden rank-4 change
X = rng.normal(size=(N, d))
Y = X @ (W0 + delta).T                                          # targets from the "true" fine-tuned layer

alpha = 8
s = alpha / r
A = rng.normal(scale=0.01, size=(r, d))
B = np.zeros((d, r))

def forward_loss():
    E = X @ (W0 + s * B @ A).T - Y
    return (E ** 2).mean(), E

print("loss at start (base model):", round(forward_loss()[0], 4))
mA, vA, mB, vB = np.zeros_like(A), np.zeros_like(A), np.zeros_like(B), np.zeros_like(B)
for t in range(1, 501):
    loss, E = forward_loss()
    G = 2 / (N * d) * E.T @ X                                   # dL/d(effective weight), shape (d, d)
    gB, gA = s * G @ A.T, s * B.T @ G
    for p, g, m, v in ((A, gA, mA, vA), (B, gB, mB, vB)):       # Adam, lr 0.02
        m *= 0.9; m += 0.1 * g
        v *= 0.999; v += 0.001 * g * g
        p -= 0.02 * (m / (1 - 0.9 ** t)) / (np.sqrt(v / (1 - 0.999 ** t)) + 1e-8)

print("loss after 500 steps:", f"{forward_loss()[0]:.2e}")
print("trainable:", A.size + B.size, "of", W0.size, f"= {100 * (A.size + B.size) / W0.size:.1f}%")
W_merged = W0 + s * B @ A                                       # merge_and_unload
unmerged = X @ W0.T + s * (X @ A.T) @ B.T                       # adapter applied on the side
print("merged equals unmerged:", np.allclose(X @ W_merged.T, unmerged))
print("rank of learned update:", np.linalg.matrix_rank(s * B @ A))
print("max error vs true weights:", f"{np.abs(W_merged - (W0 + delta)).max():.1e}")
```

```
loss at start (base model): 1.0581
loss after 500 steps: 1.48e-10
trainable: 512 of 4096 = 12.5%
merged equals unmerged: True
rank of learned update: 4
max error vs true weights: 2.6e-05
```

1. We build the situation LoRA is designed for: the "true" fine-tuned layer differs from the base by a low-rank matrix. Real fine-tuning updates are not exactly low-rank, but they are often well approximated by low rank, which is the empirical bet behind LoRA.
2. The effective weight is `W0 + s * B @ A`. The loss at the start equals the base-model loss because B is zero. This is why LoRA adapters begin as an exact no-op.
3. `G` is the gradient with respect to the full 64 x 64 weight, computed with the normal layer formula. The two adapter gradients follow from the chain rule for a product of matrices. Only A and B are updated; W0 never changes.
4. The loss drops from about 1.06 to about 1e-10 (printed in scientific notation), and the learned update has rank 4 and recovers the hidden change to within about 1e-5.
5. Trainable numbers are 512 out of 4096, 12.5 percent here. For a real 4096 x 4096 layer at r = 8 the share is 65,536 / 16,777,216 = 0.39 percent.
6. Merging folds the product into the weight; the merged and unmerged outputs agree, which is why `merge_and_unload()` costs nothing at serving time.

REAL JOB

When fine-tuning with LoRA you manage three choices: **rank r** (capacity; start at 8 or 16), **which layers** to adapt (attention projections first, then add more if underfitting), and **learning rate** (LoRA typically tolerates a higher rate than full fine-tuning). Evaluate on held-out data against the untouched base model with a prompt-only baseline: if prompting is good enough, you do not need to fine-tune.

21.8 Embeddings and Vector Search: sentence-transformers, FAISS and Chroma

INTUITION

An embedding model turns a sentence into a point in space, say 384 numbers, so that sentences with similar meaning land near each other. "Charged twice" and "double payment" share no words, yet their points are close. Vector search then answers "which stored points are nearest to this query point?". FAISS is a very fast calculator for that question. Chroma wraps the calculator in a small database that also stores the original text and labels (metadata) and can filter on them.

Reference: embedding and search tools

Name	What it does in plain English	Minimal example (not run here)
`SentenceTransformer(name)`	Load an embedding model	`m = SentenceTransformer(name)`
`m.encode(texts)`	Texts to a NumPy matrix (N, dim)	`m.encode(docs, batch_size=64)`
`normalize_embeddings=True`	Unit length vectors: inner product equals cosine	`m.encode(docs, normalize_embeddings=True)`
`m.get_sentence_embedding_dimension()`	The vector size	use it to build the index
`m.max_seq_length`	Tokens beyond this are silently cut	`m.max_seq_length = 256`
`util.cos_sim`	Cosine similarity matrix of two sets	`util.cos_sim(q, docs)`
`util.semantic_search`	Top-k hits per query (small corpora)	`util.semantic_search(q, docs, top_k=5)`
`faiss.IndexFlatL2(d)`	Exact search by squared Euclidean distance	`index = faiss.IndexFlatL2(384)`
`faiss.IndexFlatIP(d)`	Exact search by inner product	`faiss.IndexFlatIP(384)`
`faiss.normalize_L2(x)`	Normalise rows in place (cosine via IP)	`faiss.normalize_L2(x)`
`index.add(x)`	Store vectors (float32, C-contiguous)	`index.add(x.astype("float32"))`
`index.search(q, k)`	Returns scores `D` and positions `I`	`D, I = index.search(q, 5)`
`index.ntotal`	Number of stored vectors	`assert index.ntotal == len(docs)`

Name	What it does in plain English	Minimal example (not run here)
`faiss.IndexIVFFlat`	Cluster first, search only nearby clusters	`IndexIVFFlat(quant, d, nlist)`
`index.train(x)`	Learn the clusters (IVF and PQ need it)	`index.train(x)`
`index.nprobe`	How many clusters to visit per query	`index.nprobe = 8`
`faiss.IndexHNSWFlat`	Graph index: fast and accurate, more memory	`IndexHNSWFlat(d, 32)`
`faiss.IndexIDMap`	Attach your own integer ids	`IndexIDMap(IndexFlatIP(d))`
`faiss.write_index / read_index`	Save and load an index file	`faiss.write_index(index, "t.faiss")`
`chromadb.PersistentClient`	A Chroma database on disk	`PersistentClient(path="chroma_db")`
`client.get_or_create_collection`	A named table of documents	`..., metadata={"hnsw:space": "cosine"}`
`collection.add / upsert`	Store ids, documents, metadata, embeddings	`col.upsert(ids=ids, documents=docs, metadatas=meta)`
`collection.query`	Nearest documents with optional filters	`col.query(query_texts=["..."], n_results=3, where={"product": "mobile"})`
`collection.get / delete / count`	Fetch by id or filter, remove, size	`col.get(where={"label": "bug"})`

Typical idiomatic use of the real libraries together (not run here):

```
import numpy as np
from sentence_transformers import SentenceTransformer, util
import faiss, chromadb

docs = ["I was charged twice for my subscription", "app crashes on the settings page", "how do I reset my password"]
model = SentenceTransformer("your-embedding-model-id")
emb = model.encode(docs, normalize_embeddings=True).astype("float32")      # (3, dim)

# exact search with FAISS: with unit vectors, inner product == cosine similarity
index = faiss.IndexFlatIP(emb.shape[1])
index.add(emb)
q = model.encode(["double payment on my card"], normalize_embeddings=True).astype("float32")
D, I = index.search(q, 2)                      # D: (1, 2) scores, I: (1, 2) row positions
print([docs[i] for i in I[0]], D[0])

# approximate search for big corpora
nlist = 1024
quantizer = faiss.IndexFlatIP(emb.shape[1])
ivf = faiss.IndexIVFFlat(quantizer, emb.shape[1], nlist, faiss.METRIC_INNER_PRODUCT)
ivf.train(big_matrix)                          # needs at least ~39 x nlist training vectors
ivf.add(big_matrix)
ivf.nprobe = 16
faiss.write_index(ivf, "tickets.faiss")

# Chroma: embeddings + text + metadata + filtering
client = chromadb.PersistentClient(path="chroma_db")
col = client.get_or_create_collection("tickets", metadata={"hnsw:space": "cosine"})
col.upsert(ids=["t1", "t2", "t3"], documents=docs, embeddings=emb.tolist(),
           metadatas=[{"label": "billing"}, {"label": "bug"}, {"label": "how-to"}])
res = col.query(query_embeddings=q.tolist(), n_results=2, where={"label": "billing"})
print(res["documents"][0], res["distances"][0])      # distance = 1 - cosine for the cosine space
```

Worked example: semantic search with a stand-in embedder

A real sentence-transformer needs a model download. The stand-in below uses TF-IDF (weights each word by how rare it is across documents) followed by a truncated SVD (compresses to a few dimensions that capture words that appear together, called latent semantic analysis). The result has the same shape and use as real embeddings: a dense matrix (N, dim) of unit-length rows. It matches vocabulary and co-occurrence, not true paraphrase, which is the main thing a real model adds.

Problem 21.8.1 Embed 24 support tickets into 8 dimensions, normalise, and retrieve the 3 nearest tickets to three queries by cosine similarity. Then show that for unit vectors the inner product ranking equals the Euclidean ranking.

```
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.decomposition import TruncatedSVD
from sklearn.preprocessing import Normalizer
from sklearn.pipeline import make_pipeline

docs = [
 "I was charged twice for my monthly subscription", "please refund the duplicate payment on my invoice",
 "my invoice shows the wrong billing amount", "how do I update the credit card used for payment",
 "the discount code was not applied to my invoice", "I want a refund for the annual plan I never used",
 "payment failed but my card was charged", "need a receipt for last month payment",
 "the app crashes when I open the settings page", "login button does nothing on the mobile app",
 "error 500 appears when I save my profile", "the export freezes and the page never loads",
 "dark mode breaks the layout of the dashboard", "the mobile app crashes after the latest update",
 "search returns an error instead of results", "upload stops at ninety percent and shows an error",
 "how do I reset my password", "where can I find my api key in the dashboard",
 "how to export my data as a csv file", "how can I invite a teammate to my workspace",
 "how do I change the language of the app", "how do I enable two factor login",
 "how to connect the dashboard to my database", "how can I delete my account and data",
]
topic = ["billing"] * 8 + ["bug"] * 8 + ["how-to"] * 8

embedder = make_pipeline(TfidfVectorizer(sublinear_tf=True),
                         TruncatedSVD(n_components=8, random_state=0),
                         Normalizer())
E = embedder.fit_transform(docs).astype("float32")
print("embedding matrix:", E.shape, E.dtype, "row norms ~1:", np.allclose(np.linalg.norm(E, axis=1), 1, atol=1e-5))

def search(query, k=3):
    q = embedder.transform([query]).astype("float32")
    scores = (E @ q.T).ravel()                    # cosine, because rows are unit length
    top = np.argsort(-scores)[:k]
    return [(int(i), topic[i], round(float(scores[i]), 3)) for i in top]

for query in ["refund my payment", "the app crashes", "how can I export data"]:
    print(f"{query!r:26s}", search(query))

q = embedder.transform(["refund my payment"]).astype("float32")[0]
by_ip = np.argsort(-(E @ q))
by_l2 = np.argsort(((E - q) ** 2).sum(1))
print("same ranking by inner product and by L2:", np.array_equal(by_ip, by_l2))
```

```
embedding matrix: (24, 8) float32 row norms ~1: True
'refund my payment'        [(1, 'billing', 0.841), (7, 'billing', 0.711), (5, 'billing', 0.659)]
'the app crashes'          [(13, 'bug', 0.992), (8, 'bug', 0.957), (9, 'bug', 0.791)]
'how can I export data'    [(23, 'how-to', 0.973), (18, 'how-to', 0.962), (19, 'how-to', 0.798)]
same ranking by inner product and by L2: True
```

1. The pipeline chains three steps, exactly as you would chain tokeniser, model and normalisation. `fit_transform` learns the vocabulary and the 8 directions from the 24 tickets.
2. The `Normalizer` makes every row length 1. The printed check confirms the norms are close to 1. This is what `normalize_embeddings=True` does in sentence-transformers.
3. `search` embeds the query with the *same fitted pipeline* (never refit on the query) and multiplies the matrix by the query vector. For unit vectors, the dot product is the cosine similarity. `argsort(-scores)` ranks highest first.
4. Each query returns tuples of (document position, true topic, score). In this run all three hits of each query come from the expected topic: billing, bug and how-to. A score near 1 means almost the same direction. We left `stop_words` off on purpose: with only 24 tiny documents, removing

common words such as "app" and "how" changed the neighbours for the worse in our trials, a reminder that every preprocessing choice must be checked on your own data.

5. The last check explains why `IndexFlatL2` and `IndexFlatIP` give the same ranking on normalised vectors: $||a - b||^2 = 2 - 2\,(a \cdot b)$, so a smaller distance means a bigger inner product.

COMMON MISTAKE	**Wrong:** embedding documents with one model and queries with another, or refitting the vectoriser on the query. **Symptom:** results look random, scores are tiny, or you get a shape error. **Fix:** one model and one normalisation for both sides; store the model name next to the index. Some retrieval models also want a prefix such as "query:" or "passage:", so read the model card.

A FAISS-style index, built to learn the API

FAISS has a small API: `add` to store, `search` to query, results as two matrices `(D, I)` of scores and positions. When fewer than `k` vectors exist, the missing slots are filled with `-1`. This class mimics that contract in NumPy so you can see exactly what you get back.

```
import numpy as np

class FlatIndex:
    """NumPy imitation of faiss.IndexFlatIP / IndexFlatL2."""
    def __init__(self, d, metric="ip"):
        self.d, self.metric = d, metric
        self.x = np.zeros((0, d), dtype=np.float32)
    @property
    def ntotal(self):
        return len(self.x)
    def add(self, x):
        assert x.dtype == np.float32 and x.ndim == 2 and x.shape[1] == self.d, "need float32 (n, d)"
        self.x = np.vstack([self.x, x])
    def search(self, q, k):
        assert q.dtype == np.float32 and q.shape[1] == self.d
        if self.metric == "ip":
            scores = q @ self.x.T
            order = np.argsort(-scores, axis=1)[:, :k]
        else:
            scores = (q ** 2).sum(1)[:, None] - 2 * q @ self.x.T + (self.x ** 2).sum(1)[None, :]
            order = np.argsort(scores, axis=1)[:, :k]
        D = np.take_along_axis(scores, order, 1)
        I = order.copy()
        if k > self.ntotal:                                  # FAISS pads with -1
            pad = k - self.ntotal
            D = np.hstack([D, np.full((len(q), pad), np.nan, dtype=np.float32)])
            I = np.hstack([I, np.full((len(q), pad), -1)])
        return D.astype(np.float32), I

rng = np.random.default_rng(0)
x = rng.normal(size=(100, 16)).astype(np.float32)
x /= np.linalg.norm(x, axis=1, keepdims=True)
q = x[:2] + 0.05 * rng.normal(size=(2, 16)).astype(np.float32)
q /= np.linalg.norm(q, axis=1, keepdims=True)

ip, l2 = FlatIndex(16, "ip"), FlatIndex(16, "l2")
ip.add(x); l2.add(x)
D, I = ip.search(q, 3)
print("positions:", I.tolist())
print("scores   :", np.round(D.astype(float), 3).tolist())
print("L2 gives the same positions:", np.array_equal(I, l2.search(q, 3)[1]))

small = FlatIndex(16); small.add(x[:2])
print("k larger than ntotal:", small.search(q[:1], 4)[1].tolist())

try:
    ip.add(x.astype(np.float64))
except AssertionError as e:
    print("dtype guard:", e)
```

```
positions: [[0, 49, 60], [1, 62, 55]]
scores   : [[0.963, 0.537, 0.481], [0.977, 0.524, 0.519]]
L2 gives the same positions: True
k larger than ntotal: [[0, 1, -1, -1]]
dtype guard: need float32 (n, d)
```

1. The two internal formulas are the whole difference between metrics. Inner product ranks by largest score; squared L2 expands $(q - x)^2$ into `q.q - 2 q.x + x.x` and ranks by smallest.
2. Queries are slightly noisy copies of the first two stored vectors, so the nearest position for query 0 is 0 and for query 1 is 1. The first column of the printed positions shows exactly that.
3. The scores are cosine values close to 1 for the true neighbour and much smaller for the rest.

4. The L2 index returns the same positions, as the algebra from the previous example predicts for unit vectors.
5. When only two vectors exist, asking for four gives two real hits and two `-1` placeholders. Code that does `docs[i]` with `i = -1` silently picks the *last* document in Python, a nasty bug. Always filter `I >= 0`.
6. FAISS requires `float32`. NumPy defaults to `float64`, so `x.astype("float32")` is the line you will write most often; the guard here shows the kind of error you get without it.

Approximate search: IVF and the recall/speed trade-off

Exact search compares the query with every vector, costing time proportional to N. An **IVF** (inverted file) index first clusters the vectors into `nlist` groups with k-means. At query time it ranks the cluster centres, visits only the `nprobe` closest groups and compares with the vectors inside. This trades a little accuracy for a lot of speed. The accuracy is measured as **recall@k**: of the true k nearest neighbours, how many did we find?

Problem 21.8.2 Build an IVF index with 64 lists over 20,000 vectors in 32 dimensions. Measure recall@10 and the share of vectors scanned for different `nprobe` values on two data sets: strongly clustered data (like real embeddings, which have topics) and structureless Gaussian noise (the hard case).

```
import numpy as np
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs

def exact_topk(base, Q, k):
    d = (Q ** 2).sum(1)[:, None] - 2 * Q @ base.T + (base ** 2).sum(1)[None, :]
    idx = np.argpartition(d, k, axis=1)[:, :k]
    order = np.take_along_axis(d, idx, 1).argsort(1)
    return np.take_along_axis(idx, order, 1)

def ivf_curve(X, nlist=64, k=10, probes=(1, 2, 4, 8, 16, 32, 64)):
    base, Q = X[:20000], X[20000:20200]
    truth = exact_topk(base, Q, k)
    km = KMeans(n_clusters=nlist, n_init=1, random_state=0).fit(base)       # index.train
    lists = [np.where(km.labels_ == c)[0] for c in range(nlist)]               # index.add
    cent = km.cluster_centers_.astype(np.float32)
    out = []
    for nprobe in probes:
        rec, scanned = 0.0, 0
        for q, t in zip(Q, truth):
            probe = np.argsort(((cent - q) ** 2).sum(1))[:nprobe]
            cand = np.concatenate([lists[c] for c in probe])
            best = cand[np.argsort(((base[cand] - q) ** 2).sum(1))[:k]]
            rec += len(set(best) & set(t)) / k
            scanned += len(cand)
        out.append((nprobe, rec / len(Q), scanned / len(Q) / len(base)))
    return out

blobs, _ = make_blobs(n_samples=20200, n_features=32, centers=40, cluster_std=6.0, random_state=0)
noise = np.random.default_rng(0).normal(size=(20200, 32))
print("nprobe | clustered recall, scanned | gaussian recall, scanned")
for (p, r1, s1), (_, r2, s2) in zip(ivf_curve(blobs.astype(np.float32)), ivf_curve(noise.astype(np.float32))):
    print(f"{p:6d} | {r1:15.3f} {s1:9.3f} | {r2:15.3f} {s2:9.3f}")
```

```
nprobe | clustered recall, scanned | gaussian recall, scanned
     1 |           0.788     0.019 |           0.179     0.016
     2 |           0.964     0.033 |           0.285     0.031
     4 |           0.992     0.065 |           0.429     0.063
     8 |           0.997     0.129 |           0.620     0.125
    16 |           0.999     0.258 |           0.793     0.250
    32 |           1.000     0.510 |           0.942     0.501
    64 |           1.000     1.000 |           1.000     1.000
```

1. `exact_topk` is the ground truth (what `IndexFlat` would return). We need it to *measure* the approximation; never trust an approximate index without comparing with exact search on a sample.
2. `ivf_curve` performs the three real stages: training (k-means on the vectors), adding (store the member list of each cluster), and searching (rank centres, concatenate the chosen lists, rank the candidates).
3. On the clustered data, visiting only 4 lists (about 6.5 percent of vectors) already recovers about 99 percent of the true neighbours; this is the situation FAISS is designed for.
4. On pure noise there is no cluster structure to exploit: with 8 probes (12.5 percent scanned) recall is only about 0.6, and you need to scan roughly half the data to get above 0.9. The lesson is that the benefit of IVF depends on data structure, so tune `nprobe` on your own embeddings.
5. The last row (`nprobe` = `nlist`) scans everything and has recall 1.0 by construction: it is exact search plus overhead.

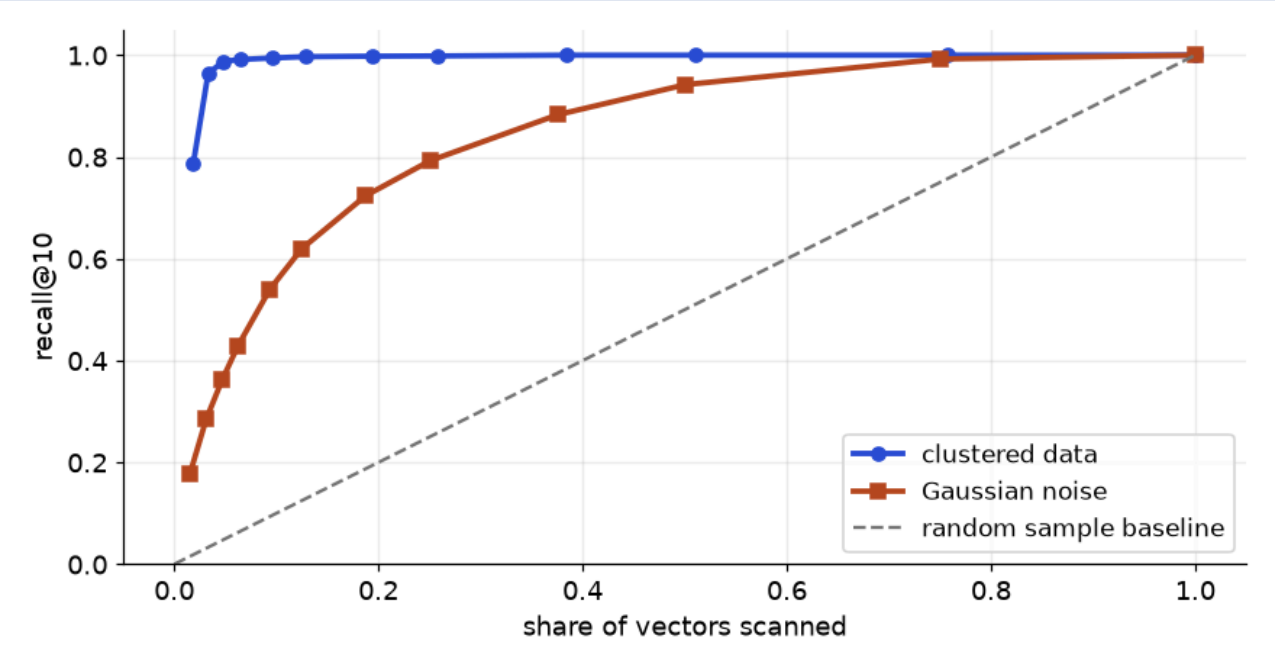


Figure 21.7. Recall at 10 versus the share of vectors scanned. Clustered data reaches high recall after scanning a few percent; structureless noise needs far more.

How to read it: the horizontal axis is the work done (share of the data compared), the vertical axis is the quality (recall). A curve that rises steeply and early is a good fit for an IVF index. The dashed diagonal is a baseline for a random sample: scanning a fraction f of the data at random finds a fraction f of the neighbours.

A Chroma-style collection: documents, metadata and filters

Chroma stores the vector, the original text and a metadata dictionary under a string id. Its two special features are `where` filters on metadata and persistence. The class below imitates the interface, built on our embedder, so you can see how filtering and the result layout work. (Chroma returns *lists of lists*, one inner list per query.)

```
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.decomposition import TruncatedSVD
from sklearn.preprocessing import Normalizer
from sklearn.pipeline import make_pipeline

class Collection:
    def __init__(self, embedder):
        self.embedder, self.ids, self.docs, self.meta, self.E = embedder, [], [], [], None
    def add(self, ids, documents, metadatas):
        dup = set(ids) & set(self.ids)
        if dup:
            raise ValueError(f"duplicate ids: {sorted(dup)}")       # Chroma add() rejects existing ids
        self.ids += ids; self.docs += documents; self.meta += metadatas
        self.E = self.embedder.transform(self.docs)
    def query(self, query_texts, n_results=3, where=None):
        q = self.embedder.transform(query_texts)
        keep = [i for i, m in enumerate(self.meta) if not where or all(m.get(k) == v for k, v in where.items())]
        out = {"ids": [], "documents": [], "distances": []}
        for row in q:
            dist = 1 - self.E[keep] @ row                           # cosine distance, smaller is closer
            top = np.argsort(dist)[:n_results]
            out["ids"].append([self.ids[keep[j]] for j in top])
            out["documents"].append([self.docs[keep[j]] for j in top])
            out["distances"].append([round(float(dist[j]), 3) for j in top])
        return out
    def count(self):
        return len(self.ids)

docs = ["refund the duplicate payment", "charged twice this month", "app crashes on the settings page",
        "mobile login button does nothing", "how do I reset my password", "where is my api key",
        "mobile app crashes after update", "web dashboard layout is broken"]
meta = [{"label": "billing", "platform": "web"}, {"label": "billing", "platform": "web"},
        {"label": "bug", "platform": "mobile"}, {"label": "bug", "platform": "mobile"},
        {"label": "how-to", "platform": "web"}, {"label": "how-to", "platform": "web"},
        {"label": "bug", "platform": "mobile"}, {"label": "bug", "platform": "web"}]
embedder = make_pipeline(TfidfVectorizer(stop_words="english"), TruncatedSVD(n_components=5, random_state=0),
Normalizer())
embedder.fit(docs)

col = Collection(embedder)
col.add([f"t{i}" for i in range(8)], docs, meta)
print("count:", col.count())
res = col.query(["the app keeps crashing"], n_results=2)
print(res["ids"], res["distances"])
res = col.query(["the app keeps crashing"], n_results=2, where={"platform": "web"})
print("filtered to web:", res["ids"], res["documents"])
try:
    col.add(["t0"], ["again"], [{"label": "bug", "platform": "web"}])
except ValueError as e:
    print("add twice:", e)
```

```
count: 8
[['t2', 't6']] [[0.007, 0.04]]
filtered to web: [['t7', 't4']] [['web dashboard layout is broken', 'how do I reset my password']]
add twice: duplicate ids: ['t0']
```

1. `Collection.add` rejects ids that already exist, as Chroma's `add` does. The fix in real code is `upsert`, which inserts new ids and replaces existing ones, making re-runs of an ingestion job safe.
2. `query` first builds `keep`, the list of rows whose metadata matches `where`, then ranks only those. This is *pre-filtering*. Retrieving the top k and filtering afterwards (post-filtering) can return fewer than k results, or none, if the filter is selective.

3. The distance is `1 - cosine`, the convention of Chroma's cosine space: 0 is identical, 1 is unrelated (orthogonal).
4. Results are lists of lists: `res["ids"][0]` belongs to the first query. Forgetting the `[0]` is the most common beginner bug with Chroma.
5. The unfiltered crash query returns mobile bug tickets; with `where={"platform": "web"}` the answer switches to the best *web* ticket, even though it is a weaker match. A filter changes the answer, not just the speed.

REAL JOB

For a retrieval-augmented generation (RAG) system, quality problems are usually in **chunking** (documents cut into pieces of a few hundred tokens, usually with a little overlap), not in the index. Store the source id and chunk position in the metadata, so you can show citations and so re-ingesting a changed file replaces its old chunks. Measure with a small labelled set (see 21.10).

21.9 LLM SDK Calls: Clients, Retries, Streaming and Structured Output

INTUITION

Calling a hosted LLM is an HTTP request to a very busy shop. Sometimes the shop says "come back in two seconds" (429, rate limit), sometimes it is briefly broken (500, 503), sometimes it is slow (timeout). The answer arrives as text, not as a guaranteed data structure, so you must *check* it before trusting it. Good SDK code is therefore three layers: a request, a safety net around failures, and a validator around the reply.

Reference: what you use from the OpenAI and Anthropic style SDKs

Name	What it does in plain English	Minimal example (not run here)
`OpenAI(...)` / `anthropic.Anthropic(...)`	Create a client; reads the API key from an environment variable	`client = anthropic.Anthropic()`
`max_retries`, `timeout`	Built-in retry count and request time limit	`Anthropic(max_retries=3, timeout=30.0)`
`client.with_options(...)`	Override settings for one call	`client.with_options(timeout=5).messages.create(...)`
`client.messages.create` (Anthropic)	One chat turn: model, max_tokens, messages	`create(model=M, max_tokens=500, messages=[...])`
`client.chat.completions.create` (OpenAI)	One chat turn in the OpenAI shape	`create(model=M, messages=[...])`
`client.responses.create` (OpenAI)	OpenAI's newer unified endpoint	`create(model=M, input="...")`
`system` / role `system`	Standing instructions for the model	`system="You label tickets."` (Anthropic), a system message (OpenAI)
`response.content[0].text`	Read the text of an Anthropic reply	check `stop_reason` first
`response.choices[0].message.content`	Read the text of an OpenAI reply	check `finish_reason` first
`response.usage`	Input and output token counts (cost)	`usage.input_tokens`, `usage.output_tokens`

Name	What it does in plain English	Minimal example (not run here)
`stop_reason` / `finish_reason`	Why generation ended: finished, hit the length limit, tool call, refusal	`if r.stop_reason == "max_tokens": ...`
`client.messages.stream(...)`	Stream text pieces as they arrive	`for t in stream.text_stream: ...`
`stream=True` (OpenAI)	Iterate over chunks with `delta.content`	`for chunk in client.chat.completions.create(..., stream=True)`
`stream.get_final_message()`	The full message after streaming (Anthropic)	use it to read usage and stop reason
`messages.parse(..., output_format=Model)`	Reply validated against a Pydantic model (Anthropic)	`r.parsed_output`
`chat.completions.parse(response_format=Model)`	Same idea for OpenAI	`r.choices[0].message.parsed`
`tools=[...]`	Describe functions the model may ask you to call	name, description, JSON schema
`client.embeddings.create`	Embedding vectors from a hosted model	`create(model=E, input=[...])`
`RateLimitError`	429: slow down; retry later	`except anthropic.RateLimitError`
`APIStatusError`	Any non-success HTTP status (has `status_code`)	`e.status_code >= 500`
`APIConnectionError / APITimeoutError`	Network failure / no reply in time	retry
`BadRequestError / AuthenticationError`	Your bug or a bad key: do NOT retry	fix the request
`AsyncOpenAI / AsyncAnthropic`	Async clients for concurrency	`await client.messages.create(...)`
`client.messages.count_tokens` (Anthropic)	Count tokens before sending	budget your prompt

The following is the idiomatic, minimal shape of each call (not run here; no API key or network in this book). The model name is a placeholder: always read the provider's current model list rather than copying an id from a book.

```
import json
import anthropic
from pydantic import BaseModel

MODEL = "your-model-id"                      # choose from the provider's current model list
client = anthropic.Anthropic(max_retries=3, timeout=30.0)   # key read from ANTHROPIC_API_KEY

# 1) a plain call
resp = client.messages.create(
    model=MODEL, max_tokens=500,
    system="You are a support assistant. Answer in two sentences.",
    messages=[{"role": "user", "content": "How do I reset my password?"}],
)
if resp.stop_reason == "max_tokens":
    print("answer was cut off; raise max_tokens")
print(resp.content[0].text)
print(resp.usage.input_tokens, resp.usage.output_tokens)

# 2) streaming: print text as it arrives, then read the final message
with client.messages.stream(model=MODEL, max_tokens=500,
                            messages=[{"role": "user", "content": "Draft a reply to a billing complaint."}]) as stream:
    for text in stream.text_stream:
        print(text, end="", flush=True)
    final = stream.get_final_message()
print("\ntokens out:", final.usage.output_tokens)

# 3) structured output: the SDK validates the reply against a Pydantic model
class TicketLabel(BaseModel):
    label: str            # billing | bug | how-to
    urgency: int          # 1 to 5
    summary: str

parsed = client.messages.parse(
    model=MODEL, max_tokens=500,
    messages=[{"role": "user", "content": "Classify: 'I was charged twice and need money back today.'"}],
    output_format=TicketLabel,
)
print(parsed.parsed_output)       # a TicketLabel instance, not a string

# 4) errors: most specific first
try:
    client.messages.create(model=MODEL, max_tokens=50, messages=[{"role": "user", "content": "hi"}])
except anthropic.RateLimitError as e:
    print("rate limited, retry after", e.response.headers.get("retry-after"))
except anthropic.APIConnectionError:
    print("network problem")
except anthropic.APIStatusError as e:
    print("server said", e.status_code)
```

The OpenAI client has the same bones with different names (not run here):

```
from openai import OpenAI
from pydantic import BaseModel

client = OpenAI(max_retries=3, timeout=30.0)           # key from OPENAI_API_KEY
MODEL = "your-model-id"

resp = client.chat.completions.create(
    model=MODEL,
    messages=[{"role": "system", "content": "You label support tickets."},
              {"role": "user", "content": "I was charged twice."}],
)
print(resp.choices[0].message.content, resp.choices[0].finish_reason, resp.usage.total_tokens)

stream = client.chat.completions.create(model=MODEL, stream=True,
                                        messages=[{"role": "user", "content": "Write a short apology."}])
for chunk in stream:
    delta = chunk.choices[0].delta.content         # may be None on the first and last chunk
    if delta:
        print(delta, end="", flush=True)

class TicketLabel(BaseModel):
    label: str
    urgency: int

r = client.chat.completions.parse(model=MODEL, response_format=TicketLabel,
                                  messages=[{"role": "user", "content": "Classify: app crashes on login"}])
print(r.choices[0].message.parsed)
```

KEY IDEA	Both SDKs already retry connection errors, 429 and 5xx responses with exponential backoff (a small default number of times; check the current default in the docs). Set `max_retries` and `timeout` on the client first. Write your own retry loop only for behaviour the SDK does not offer, and never stack a retry loop of five on top of an SDK retry of three without realising that you now make up to twenty attempts.

Worked example: retries with exponential backoff and jitter

The logic behind SDK retries is simple and worth knowing. After a failure wait `base x 2^attempt` seconds (capped), but choose a *random* number between 0 and that value (**full jitter**). Without randomness, a hundred clients that failed together all retry together and fail again. If the server sends a `retry-after` header, obey it. Errors caused by your own request (400, 401, 403, 404) must never be retried because the answer will not change.

A fake client with scripted failures lets us test this exactly. Time is simulated by passing in a `sleep` function that records the wait instead of waiting.

Problem 21.9.1 Script three situations: (A) a 429 with `retry-after` 2 seconds, then a 503, then success; (B) a 400 error; (C) six 503 errors in a row with a retry limit of 4. Show the waits and the outcome of each.

```
import random
from dataclasses import dataclass

class APIStatusError(Exception):
    def __init__(self, status_code, retry_after=None):
        super().__init__(f"HTTP {status_code}")
        self.status_code, self.retry_after = status_code, retry_after
class RateLimitError(APIStatusError): pass
class InternalServerError(APIStatusError): pass
class BadRequestError(APIStatusError): pass
class APITimeoutError(Exception): pass

@dataclass
class Response:
    text: str
    input_tokens: int
    output_tokens: int

class FakeLLM:
    """Plays back a script of outcomes: an exception instance to raise, or a string to return."""
    def __init__(self, script):
        self.script, self.calls = list(script), 0
    def complete(self, prompt):
        self.calls += 1
        outcome = self.script.pop(0)
        if isinstance(outcome, Exception):
            raise outcome
        return Response(outcome, input_tokens=len(prompt.split()), output_tokens=len(outcome.split()))

RETRYABLE = (RateLimitError, InternalServerError, APITimeoutError)

def call_with_retries(fn, max_retries=4, base=0.5, cap=8.0, sleep=None, rng=None):
    for attempt in range(max_retries + 1):
        try:
            return fn()
        except RETRYABLE as e:
            if attempt == max_retries:
                raise
            hinted = getattr(e, "retry_after", None)
            delay = hinted if hinted is not None else rng.uniform(0, min(cap, base * 2 ** attempt))
            sleep(delay)
        # BadRequestError is not in RETRYABLE, so it propagates immediately

def run(label, script):
    waits, llm = [], FakeLLM(script)
    try:
        r = call_with_retries(lambda: llm.complete("label this ticket please"),
                              sleep=waits.append, rng=random.Random(0))
        outcome = f"ok: {r.text!r}"
    except Exception as e:
        outcome = f"gave up: {type(e).__name__} ({e})"
    print(f"{label}: calls={llm.calls} waits={[round(w, 2) for w in waits]} -> {outcome}")

run("A", [RateLimitError(429, retry_after=2.0), InternalServerError(503), "billing"])
run("B", [BadRequestError(400), "never reached"])
run("C", [InternalServerError(503)] * 6)
```

```
A: calls=3 waits=[2.0, 0.84] -> ok: 'billing'
B: calls=1 waits=[] -> gave up: BadRequestError (HTTP 400)
C: calls=5 waits=[0.42, 0.76, 0.84, 1.04] -> gave up: InternalServerError (HTTP 503)
```

1. The exception classes copy the names used by the real SDKs, so this code reads like the real thing. `RETRYABLE` is the allow-list of failures worth another try: 429, 5xx and timeouts.
2. `FakeLLM` returns the next scripted outcome. This is the standard way to unit-test LLM code without a network: deterministic, free and instant.
3. `call_with_retries` loops up to `max_retries + 1` times. On a retryable failure it uses the server's `retry-after` hint when present, otherwise a random wait between 0 and `min(cap, base x 2^attempt)`. The last attempt re-raises instead of waiting.
4. Scenario A makes three calls. The first wait is exactly 2.0 (the hint); the second is a random value below 1.0 (attempt 1 allows up to 0.5 x 2 = 1.0). Then the success text comes back.
5. Scenario B stops after one call and no wait: a 400 is your bug, and retrying cannot fix it.
6. Scenario C makes five calls (the first plus four retries), waits four times, each a random value below a doubling ceiling (0.5, 1.0, 2.0, 4.0), and finally raises. Callers must handle that final failure: a fallback model, a cached answer, or a clear error for the user.
7. Injecting `sleep` and `rng` is what made the test possible. In production pass `time.sleep` and a `random.Random()`.

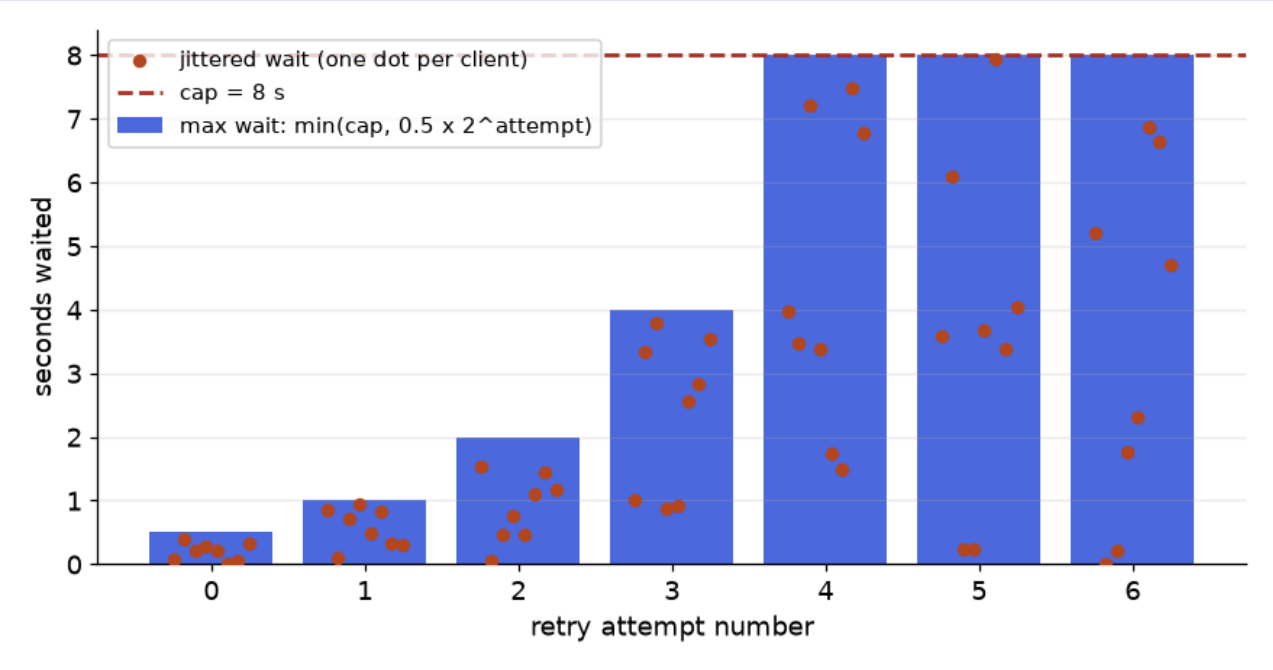


Figure 21.8. Exponential backoff ceilings (bars) and sampled full-jitter waits (dots) for attempts 0 to 6, with base 0.5 s and cap 8 s.

How to read it: each bar is the longest wait allowed before the next attempt, doubling until it hits the cap. Each dot is one random wait actually drawn. The scatter of the dots is the point: different clients wake up at different moments, so they do not hit the server in one wave.

Streaming

A long answer takes many seconds. Streaming returns it in small pieces so a user sees the first words almost at once, and it avoids request timeouts on long outputs. A stream is just a Python generator that yields text pieces. Three things matter: accumulate the pieces into the full text, handle a failure in the middle (the partial text is usually unusable; restart or fall back), and read the usage numbers at the end.

```
import time

class FakeStreamingLLM:
    def __init__(self, text, fail_after=None):
        self.text, self.fail_after = text, fail_after
    def stream(self, prompt):
        words = self.text.split()
        for i, w in enumerate(words):
            if self.fail_after is not None and i == self.fail_after:
                raise ConnectionError("connection dropped mid-stream")
            yield w + (" " if i < len(words) - 1 else "")

def consume(stream_fn, on_piece=None):
    parts = []
    try:
        for piece in stream_fn():
            parts.append(piece)
            if on_piece:
                on_piece(piece)
        return "".join(parts), True
    except ConnectionError:
        return "".join(parts), False          # partial text plus a flag

reply = "Please reset your password using the link on the sign in page"
shown = []
text, ok = consume(lambda: FakeStreamingLLM(reply).stream("q"), on_piece=shown.append)
print("pieces:", len(shown), "| first piece:", repr(shown[0]), "| complete:", ok)
print("joined equals original:", text == reply)

text, ok = consume(lambda: FakeStreamingLLM(reply, fail_after=4).stream("q"))
print("failed stream ->", repr(text), "| complete:", ok)

def stream_with_fallback(prompt, attempts):
    for llm in attempts:                       # restart from scratch on a new attempt
        text, ok = consume(lambda: llm.stream(prompt))
        if ok:
            return text
    return None
print("recovered:", stream_with_fallback("q", [FakeStreamingLLM(reply, fail_after=2), FakeStreamingLLM(reply)]) == reply)
```

```
pieces: 12 | first piece: 'Please ' | complete: True
joined equals original: True
failed stream -> 'Please reset your password ' | complete: False
recovered: True
```

1. The fake `stream` method is a generator: each `yield` hands one piece to the caller and pauses. Real SDK streams (`stream.text_stream`, or iterating OpenAI chunks) behave the same way.
2. `consume` collects pieces in a list and joins once at the end. Joining a list is faster than repeatedly appending to a string; `on_piece` is where a web server would forward each piece to the browser.
3. The first run shows 12 pieces (one per word) and proves that joining them reproduces the exact reply.
4. The second run simulates a dropped connection after four words. The function returns the partial text and `False`. A partial JSON document or half a sentence must not be treated as a result.
5. The third part retries from scratch with a fresh stream, because a stream cannot be resumed from the middle in general. It recovers on the second attempt.
6. In real SDK code you do the same inside a `with client.messages.stream(...)` block, catch the connection exception, and read token usage from the final message once the stream completes.

Structured output: never trust the shape of a reply

You want `{"label": "billing", "urgency": 4}` and sometimes get a fenced code block, a trailing comma, a missing key or a number as a string. There are three defences, strongest first: (1) use the provider's structured output feature (schema-constrained decoding, or `parse()` with a Pydantic model) so the reply is valid by construction; (2) validate every reply yourself against the schema; (3) on failure, retry once or twice with the validation error in the prompt (a *repair loop*). Even with (1), still check `stop_reason`: a reply cut off by `max_tokens` is incomplete and refusals are not data.

```
import json, re
from dataclasses import dataclass

LABELS = {"billing", "bug", "how-to"}

class ValidationFailure(Exception):
    pass

@dataclass
class TicketLabel:
    label: str
    urgency: int

def parse_label(raw):
    text = raw.strip()
    fenced = re.match(r"^```(?:json)?\s*(.*?)\s*```$", text, re.S)       # models like to add code fences
    if fenced:
        text = fenced.group(1)
    try:
        data = json.loads(text)
    except json.JSONDecodeError as e:
        raise ValidationFailure(f"not valid JSON: {e.msg}")
    if not isinstance(data, dict):
        raise ValidationFailure("expected a JSON object")
    missing = {"label", "urgency"} - data.keys()
    if missing:
        raise ValidationFailure(f"missing keys: {sorted(missing)}")
    if data["label"] not in LABELS:
        raise ValidationFailure(f"label must be one of {sorted(LABELS)}, got {data['label']!r}")
    if not isinstance(data["urgency"], int) or isinstance(data["urgency"], bool) or not 1 <= data["urgency"] <= 5:
        raise ValidationFailure("urgency must be an integer from 1 to 5")
    return TicketLabel(data["label"], data["urgency"])

class ScriptedModel:
    def __init__(self, replies):
        self.replies, self.prompts = list(replies), []
    def complete(self, prompt):
        self.prompts.append(prompt)
        return self.replies.pop(0)

def classify(model, ticket, max_attempts=3):
    prompt = f"Return JSON with label and urgency for: {ticket}"
    for attempt in range(1, max_attempts + 1):
        raw = model.complete(prompt)
        try:
            return parse_label(raw), attempt
        except ValidationFailure as e:
            prompt = f"{prompt}\nYour last reply was rejected: {e}. Return only valid JSON."
    raise ValidationFailure("gave up after repairs")

replies = [
    '{"label": "billing", "urgency": 4,}',                    # trailing comma
    '```json\n{"label": "refund", "urgency": 4}\n```',         # fenced, bad label
    '{"label": "billing", "urgency": "high"}',                 # wrong type
    '{"label": "billing", "urgency": 4}',
]
m = ScriptedModel(replies)
try:
    result, attempts = classify(m, "charged twice", max_attempts=3)
except ValidationFailure as e:
    print("3 attempts failed:", e)
m = ScriptedModel(replies[1:])
result, attempts = classify(m, "charged twice", max_attempts=3)
print("result:", result, "after", attempts, "attempts")
```

```
print("last prompt tail:", m.prompts[-1].splitlines()[-1])
print("fenced valid reply parses:", parse_label('```json\n{"label": "bug", "urgency": 2}\n```'))
```

```
3 attempts failed: gave up after repairs
result: TicketLabel(label='billing', urgency=4) after 3 attempts
last prompt tail: Your last reply was rejected: urgency must be an integer from 1 to 5. Return only valid JSON.
fenced valid reply parses: TicketLabel(label='bug', urgency=2)
```

1. `parse_label` is a defensive parser: strip whitespace, remove a code fence if present, `json.loads`, then check type, required keys, allowed values and ranges. Each failure raises one exception with a human-readable reason.
2. `isinstance(True, int)` is true in Python, so the explicit `bool` check stops `true` from passing as urgency 1.
3. `classify` is the repair loop. After a rejection it appends the reason to the prompt, which gives the model the information it needs to fix the reply.
4. First we feed the script of four replies with only three attempts allowed: the first three are all invalid (trailing comma, unknown label, wrong type) so it fails and prints the message "3 attempts failed".
5. Then we start from the second reply, giving a bad label, a bad type, then a good reply: success on attempt 3. The last prompt shows the accumulated rejection notes.
6. The final line shows the fence-stripping path working. With provider-side structured outputs most of these failures disappear, but the validator stays as the last line of defence, because it is cheap and the cost of trusting a malformed reply is high.

COMMON MISTAKE

Wrong: `data = json.loads(resp.content[0].text)` with no checks. **Symptom:** works in the demo, then a production request returns a refusal, a truncated answer or a code fence and the job crashes with `JSONDecodeError`, or worse, a wrong-typed value silently flows into your database. **Fix:** check the stop reason, use the provider's structured output feature or `parse()`, validate against a schema, and bound the number of repairs.

Tool calls, rate limits and cost

When you give a model **tools**, it can reply "please call `lookup_ticket` with id 7" instead of text. You run the function and send the result back; the loop ends when the model answers in text. Keep a hard cap on loop turns. Two more production basics: a **token bucket** to stay under rate limits, and a **cost estimate** from the usage numbers. Prices below are invented for illustration; read your provider's price page.

```
import json

TICKETS = {7: {"status": "open", "plan": "pro"}, 9: {"status": "closed", "plan": "free"}}
def lookup_ticket(ticket_id):
    return TICKETS.get(ticket_id, {"error": "not found"})
TOOLS = {"lookup_ticket": lookup_ticket}

class ScriptedToolModel:
    """Turn 1 asks for a tool; turn 2 answers using the tool result."""
    def __init__(self):
        self.turn = 0
    def respond(self, messages):
        self.turn += 1
        if self.turn == 1:
            return {"type": "tool_call", "name": "lookup_ticket", "arguments": {"ticket_id": 7}}
        last = json.loads(messages[-1]["content"])
        return {"type": "text", "text": f"Ticket 7 is {last['status']} on the {last['plan']} plan."}

def agent_loop(model, question, max_turns=5):
    messages = [{"role": "user", "content": question}]
    for _ in range(max_turns):                                  # hard cap against endless loops
        out = model.respond(messages)
        if out["type"] == "text":
            return out["text"], messages
        fn = TOOLS.get(out["name"])
        result = fn(**out["arguments"]) if fn else {"error": "unknown tool"}
        messages.append({"role": "assistant", "content": f"call {out['name']}"})
        messages.append({"role": "tool", "content": json.dumps(result)})
    return "stopped: too many tool turns", messages

answer, msgs = agent_loop(ScriptedToolModel(), "What is the state of ticket 7?")
print(answer)
print("roles:", [m["role"] for m in msgs])

class TokenBucket:
    def __init__(self, rate_per_s, capacity):
        self.rate, self.capacity, self.tokens, self.t = rate_per_s, capacity, capacity, 0.0
    def allow(self, now, cost=1):
        self.tokens = min(self.capacity, self.tokens + (now - self.t) * self.rate)
        self.t = now
        if self.tokens >= cost:
            self.tokens -= cost
            return True
        return False

bucket = TokenBucket(rate_per_s=2, capacity=3)
times = [0.0, 0.1, 0.2, 0.3, 0.4, 1.0, 1.1]
print("allowed:", [bucket.allow(t) for t in times])

def cost_usd(inp, out, price_in_per_m=3.0, price_out_per_m=15.0):      # illustrative prices only
    return inp / 1e6 * price_in_per_m + out / 1e6 * price_out_per_m
print("10000 calls x (800 in, 150 out):", round(10000 * cost_usd(800, 150), 2), "dollars at the invented prices")
```

```
Ticket 7 is open on the pro plan.
roles: ['user', 'assistant', 'tool']
allowed: [True, True, True, False, False, True, True]
10000 calls x (800 in, 150 out): 46.5 dollars at the invented prices
```

1. The agent loop is the whole idea of tool use: ask the model, if it requests a tool then run it and append the result, ask again. The `max_turns` cap protects against a model that keeps calling tools forever.
2. Unknown tool names return an error result instead of crashing; the model can read the error and try something else.

3. The printed roles show the conversation order: user, then the assistant's tool request, then the tool result.
4. The token bucket holds up to 3 tokens and refills at 2 per second. The five calls at 0.0 to 0.4 seconds use the bucket up: the first three pass and the next two are refused (the refill of 0.2 and 0.4 tokens is not enough). By 1.0 s enough has refilled that the next call passes. Refused calls should wait, not be dropped silently.
5. Cost is linear in tokens: 800 x 3.0 / 1e6 + 150 x 15.0 / 1e6 = 0.00465 per call, so 10,000 calls cost 46.5 dollars at these invented prices. Always multiply per-call cost by your real daily volume before building.

21.10 Orchestration Awareness and Evaluation Harnesses

INTUITION

An LLM application is rarely one call. It is a pipeline: build a prompt from a template, maybe retrieve documents, call a model, parse the reply, maybe call a tool, check the result. Orchestration libraries such as LangChain give these steps a common interface (every step has `invoke`) and a pipe operator so they snap together like Unix commands. They save glue code when you have many steps or many providers; they add a layer you must learn and debug when you have one. Evaluation is the other half: without a test set and a metric, you cannot tell whether a change made things better.

Reference: LangChain-style building blocks

This library changes quickly: package names (`langchain-core`, `langchain-community`, provider packages, `langgraph` for agents) and import paths have moved between releases. Treat the table as a map of *ideas* and check the current documentation for exact imports.

Idea / name	What it does in plain English	Minimal example (not run here)
`ChatPromptTemplate`	A prompt with named holes	`ChatPromptTemplate.from_template("Label: {text}")`
Runnable	Anything with `invoke`, `batch`, `stream`	`chain.invoke({"text": "..."})`
pipe operator `|`	Output of the left step feeds the right step	`prompt | model | parser`
chat model wrapper	One interface over many providers	`init_chat_model("provider:model-id")`
`StrOutputParser`	Reply object to plain string	`... | StrOutputParser()`
`with_structured_output`	Model returns a validated object	`model.with_structured_output(Schema)`
`RunnableLambda`	Wrap any Python function as a step	`RunnableLambda(clean_text)`
`RunnablePassthrough`	Carry the input along unchanged	`{"context": retriever, "question": RunnablePassthrough()}`
`RunnableParallel` (dict syntax)	Run several steps on the same input	a dict of runnables
text splitter	Cut documents into overlapping chunks	`RecursiveCharacterTextSplitter(chunk_size=800, chunk_overlap=100)`
vector store / retriever	Search wrapper over FAISS, Chroma and others	`store.as_retriever(search_kwargs={"k": 4})`
`@tool`	Turn a function into a tool the model can call	`@tool def lookup(id: int): ...`
agent graph (LangGraph)	Loops and branches with explicit state	nodes, edges, conditional edges

Idea / name	What it does in plain English	Minimal example (not run here)
tracing	Record every step's input and output	an environment variable plus a tracing service

The classic RAG chain looks like this (not run here; import paths follow recent `langchain-core` but check your version):

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnablePassthrough
from langchain.chat_models import init_chat_model

model = init_chat_model("provider:your-model-id", temperature=0)
retriever = store.as_retriever(search_kwargs={"k": 3})           # store: Chroma or FAISS wrapper

prompt = ChatPromptTemplate.from_template(
    "Answer using only the context. If the answer is not there, say 'unknown'.\n"
    "Context:\n{context}\n\nQuestion: {question}")

def join_docs(docs):
    return "\n---\n".join(d.page_content for d in docs)

chain = ({"context": retriever | join_docs, "question": RunnablePassthrough()}
         | prompt | model | StrOutputParser())

print(chain.invoke("How do I reset my password?"))
for piece in chain.stream("How do I reset my password?"):       # streaming comes for free
    print(piece, end="")
answers = chain.batch(["question one", "question two"])         # concurrency for free
```

Understanding the idea beats memorising the API. Below is a 30-line pipe framework with the same shape, followed by a RAG chain built from our earlier retriever. The "model" is a scripted function so the output is real and repeatable.

Problem 21.10.1 Build `Runnable` with `invoke`, `batch` and the `|` operator, then compose retrieve, prompt, model and parse into a chain and run it on two questions.

```
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.decomposition import TruncatedSVD
from sklearn.preprocessing import Normalizer
from sklearn.pipeline import make_pipeline

class Runnable:
    def __init__(self, fn, name="step"):
        self.fn, self.name = fn, name
    def invoke(self, x):
        return self.fn(x)
    def batch(self, xs):
        return [self.invoke(x) for x in xs]
    def __or__(self, other):                        # a | b means: run a, then feed its output to b
        return Runnable(lambda x: other.invoke(self.invoke(x)), f"{self.name} | {other.name}")

docs = ["To reset your password open settings and click reset link",
        "Refunds are issued to the original card within five days",
        "Export your data from the dashboard menu as a csv file",
        "The mobile app crash was fixed in the latest update"]
embedder = make_pipeline(TfidfVectorizer(), TruncatedSVD(n_components=3, random_state=0), Normalizer()).fit(docs)
E = embedder.transform(docs)

def retrieve(question, k=2):
    scores = E @ embedder.transform([question])[0]
    return {"question": question, "docs": [docs[i] for i in np.argsort(-scores)[:k]]}

def build_prompt(d):
    context = "\n".join(f"- {t}" for t in d["docs"])
    return f"Answer from the context only.\nContext:\n{context}\nQuestion: {d['question']}"

def fake_model(prompt):
    lines = [l[2:] for l in prompt.splitlines() if l.startswith("- ")]
    return {"content": lines[0] if lines else "unknown", "prompt_chars": len(prompt)}   # echoes the best chunk

parse = Runnable(lambda msg: msg["content"].strip(), "parse")
chain = Runnable(retrieve, "retrieve") | Runnable(build_prompt, "prompt") | Runnable(fake_model, "model") | parse

print("chain:", chain.name)
print(chain.invoke("how do I reset my password"))
print(chain.batch(["where do refunds go", "export csv data"]))

prompt_text = (Runnable(retrieve) | Runnable(build_prompt)).invoke("where do refunds go")
print(prompt_text)
```

```
chain: retrieve | prompt | model | parse
To reset your password open settings and click reset link
['Refunds are issued to the original card within five days', 'Export your data from the dashboard menu as a csv file']
Answer from the context only.
Context:
- Refunds are issued to the original card within five days
- The mobile app crash was fixed in the latest update
Question: where do refunds go
```

1. A `Runnable` wraps one function. `__or__` is the special method Python calls for `a | b`; it returns a new Runnable that calls `a` and passes its result to `b`. That is the whole trick behind `prompt | model | parser`.
2. `retrieve` returns a dictionary because the next step needs both the question and the documents. In LangChain that is what the dict-of-runnables syntax produces.

3. `build_prompt` turns the dictionary into the text a model would receive. `fake_model` stands in for a real model: it returns an object with a `content` field, as chat model replies do.
4. The chain output for the password question is the password document, the refund question returns the refund sentence, and the export question returns the export sentence: the retriever put the matching document first, and the scripted model repeats the first context line.
5. The last print shows the intermediate prompt. Being able to print the prompt that was actually sent is the most useful debugging move in any LLM pipeline: most "the model is wrong" bugs are "the model was given the wrong context".

REAL JOB

Use a framework when it removes code you would otherwise write (provider switching, tracing, retries, vector store adapters). Skip it when it hides what is sent to the model. A good rule: you must be able to print the final prompt and the raw response at any step. If a framework makes that hard, wrap the SDK directly; the patterns in 21.9 are only a few dozen lines.

Evaluation: the harness that tells you if you improved

An **evaluation harness** is a script that runs your system on a fixed set of examples with known good answers and reports numbers. The ingredients are always the same: (1) a **golden set** of inputs with expected outputs, held out from any prompt tuning; (2) a **metric** that fits the task; (3) a **baseline** to beat; (4) **uncertainty**, because a difference of three points on 100 examples may be noise.

Task	Metrics to use	Watch out for
Classification / extraction	Accuracy, per-class precision and recall, exact match on fields, JSON validity rate	Class imbalance hides failures behind a high accuracy
Retrieval	recall@k, MRR, nDCG@k	Labelling only one relevant document when several are valid
Short-answer QA	Exact match, token F1	Correct answers phrased differently score zero on exact match
Free-text generation	Rubric scoring by humans or an LLM judge, groundedness (is each claim supported by the context?)	Judges favour long answers; check judge agreement with humans
Operations	Latency percentiles (p50, p95), cost per request, refusal rate	Averages hide the slow tail

First the retrieval metrics, small enough to check by hand. Suppose the relevant documents for a query are {2, 5} and the system returns [7, 2, 9, 5, 1]. **recall@3** = relevant found in the top 3 divided by relevant total = 1/2. **MRR** (mean reciprocal rank) uses 1 divided by the rank of the first relevant hit = 1/2. **nDCG@5** rewards early hits: DCG = $1/\log_2(3) + 1/\log_2(5) = 0.6309 + 0.4307 = 1.0616$; the ideal ranking would put both at ranks 1 and 2, IDCG = 1 + 0.6309 = 1.6309; nDCG = 0.651.

```
import numpy as np

def recall_at_k(ranked, relevant, k):
    return len(set(ranked[:k]) & set(relevant)) / len(relevant)

def precision_at_k(ranked, relevant, k):
    return len(set(ranked[:k]) & set(relevant)) / k

def mrr(ranked, relevant):
    for rank, item in enumerate(ranked, start=1):
        if item in relevant:
            return 1 / rank
    return 0.0

def ndcg_at_k(ranked, relevant, k):
    dcg = sum(1 / np.log2(r + 1) for r, item in enumerate(ranked[:k], start=1) if item in relevant)
    ideal = sum(1 / np.log2(r + 1) for r in range(1, min(len(relevant), k) + 1))
    return dcg / ideal

ranked, relevant = [7, 2, 9, 5, 1], {2, 5}
print("recall@3   :", recall_at_k(ranked, relevant, 3))
print("recall@5   :", recall_at_k(ranked, relevant, 5))
print("precision@3:", round(precision_at_k(ranked, relevant, 3), 4))
print("MRR        :", mrr(ranked, relevant))
print("nDCG@5     :", round(ndcg_at_k(ranked, relevant, 5), 4))

def token_f1(pred, gold):
    p, g = pred.lower().split(), gold.lower().split()
    common = sum(min(p.count(w), g.count(w)) for w in set(p))
    if common == 0:
        return 0.0
    prec, rec = common / len(p), common / len(g)
    return 2 * prec * rec / (prec + rec)
print("exact match:", "reset link" == "reset the link", "| token F1:", round(token_f1("reset link", "reset the link"), 3))
```

```
recall@3   : 0.5
recall@5   : 1.0
precision@3: 0.3333
MRR        : 0.5
nDCG@5     : 0.6509
exact match: False | token F1: 0.8
```

1. The functions mirror the hand calculations: recall@3 is 0.5 because only document 2 is among the top three; recall@5 is 1.0.
2. `precision@3` is 1/3 because one of three returned documents is relevant.
3. MRR is 0.5 (first relevant at rank 2). nDCG@5 prints 0.6509, the 0.651 computed by hand above; `ideal` assumes the relevant documents sit at the very top.
4. Token F1 gives partial credit: "reset link" versus "reset the link" has two shared words, precision 1.0, recall 2/3, F1 = 0.8, while exact match says False. That is why QA evaluation uses both.

Now a whole harness on a realistic comparison: a keyword-rule baseline versus a TF-IDF plus logistic regression classifier on a held-out set of synthetic tickets (generated with overlapping vocabulary so the task is not trivial). We report accuracy, a confusion matrix, and a **paired bootstrap** confidence interval for the accuracy difference: resample test examples with replacement many times, recompute the difference each time, and read off the middle 95 percent.

```
import random
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.metrics import confusion_matrix

rng = random.Random(0)
pools = {
    "billing": ["refund", "charged", "invoice", "payment", "card", "receipt", "subscription", "price", "billing",
"discount"],
    "bug": ["crash", "error", "broken", "freezes", "fails", "slow", "bug", "blank", "timeout", "stuck"],
    "how-to": ["how", "export", "reset", "change", "find", "invite", "connect", "setup", "where", "enable"],
}
filler = ["the", "my", "app", "page", "account", "please", "when", "on", "I", "can", "with", "after", "today", "again",
"dashboard"]
labels = list(pools)

def make(n):
    X, y = [], []
    for i in range(n):
        lab = labels[i % 3]
        words = rng.sample(pools[lab], rng.choice([1, 2]))
        if rng.random() < 0.6:                                  # confusing words from another class
            other = rng.choice([l for l in labels if l != lab])
            words += rng.sample(pools[other], rng.choice([1, 2]))
        words += rng.sample(filler, 3)
        rng.shuffle(words)
        X.append(" ".join(words)); y.append(lab)
    return X, np.array(y)

Xtr, ytr = make(300)
Xte, yte = make(150)

model = make_pipeline(TfidfVectorizer(), LogisticRegression(max_iter=1000)).fit(Xtr, ytr)
pred_model = model.predict(Xte)

def rule(text):                                                 # the baseline to beat
    w = set(text.split())
    if w & {"refund", "charged", "invoice", "payment"}: return "billing"
    if w & {"crash", "error", "broken", "freezes"}: return "bug"
    return "how-to"
pred_rule = np.array([rule(t) for t in Xte])

ok_model, ok_rule = (pred_model == yte).astype(int), (pred_rule == yte).astype(int)
print("accuracy model:", round(ok_model.mean(), 3), " rule baseline:", round(ok_rule.mean(), 3))
print("model wins / baseline wins:", int(((ok_model == 1) & (ok_rule == 0)).sum()), "/", int(((ok_model == 0) & (ok_rule
== 1)).sum()))

boot = np.random.default_rng(0)
diffs = []
for _ in range(2000):
    idx = boot.integers(0, len(yte), len(yte))
    diffs.append(ok_model[idx].mean() - ok_rule[idx].mean())
lo, hi = np.percentile(diffs, [2.5, 97.5])
print(f"accuracy gain 95% CI: [{lo:.3f}, {hi:.3f}]")
print("confusion matrix (rows true, columns predicted), order", labels)
print(confusion_matrix(yte, pred_model, labels=labels))
```

```
accuracy model: 0.68  rule baseline: 0.56
model wins / baseline wins: 37 / 19
accuracy gain 95% CI: [0.027, 0.213]
confusion matrix (rows true, columns predicted), order ['billing', 'bug', 'how-to']
[[31 13  6]
 [ 7 38  5]
 [ 9  8 33]]
```

1. The generator mixes in words from other classes on purpose. Real data is ambiguous; a test set where everything is easy tells you nothing. (An earlier version of this script scored 100 percent, which was a warning sign that the task was too easy, not a success.)
2. Training and test sets are different draws. The model never sees the test tickets, and any prompt or threshold tuning must also use a separate validation set, never the test set.
3. The model scores 0.68 and the baseline 0.56. The win/loss count is a paired view: on 37 examples only the model was right, on 19 only the baseline was right. Paired comparison is more informative than two separate accuracies.
4. The bootstrap resamples the 150 test items 2,000 times. The interval for the gain is about 0.03 to 0.21: it excludes zero but is wide, so we can say the model is probably better, but the size of the gain is uncertain. With 150 examples you cannot detect small improvements; collect more labels or accept the uncertainty.
5. The confusion matrix shows where errors go. Each row is a true class; the diagonal is correct. Read the off-diagonal cells to decide what to fix: for example whether billing tickets are mistaken for bugs.
6. Replace the two classifiers with "prompt version 1" and "prompt version 2" calling an LLM and the harness is unchanged. That is the point: keep the harness fixed and change only the system under test.

LLM-as-judge, carefully

For free text there is no single correct string, so teams ask a second model to grade replies against a rubric. It is useful but biased, so calibrate it: label 50 or more examples by hand, run the judge on the same examples, and measure agreement. Cohen's kappa corrects raw agreement for chance. Randomise the order when comparing two answers (judges often favour the first), and keep the rubric specific (for example "is every claim supported by the context?" rather than "is it good?").

```
import numpy as np
from sklearn.metrics import cohen_kappa_score

human = np.array([1, 1, 1, 0, 1, 0, 0, 1, 1, 0, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1])
judge = np.array([1, 1, 1, 0, 1, 1, 0, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 0, 1, 1])

agree = (human == judge).mean()
print("raw agreement:", agree)
print("kappa:", round(cohen_kappa_score(human, judge), 3))
print("judge says 'good' on", int(judge.sum()), "of", len(judge), "| humans say 'good' on", int(human.sum()))
fp = int(((judge == 1) & (human == 0)).sum())
print("judge approves", fp, "replies that humans rejected")

always_yes = np.ones_like(human)
print("an always-yes judge: agreement", (human == always_yes).mean(), "kappa", round(cohen_kappa_score(human, always_yes),
3))
```

```
raw agreement: 0.9
kappa: 0.765
judge says 'good' on 15 of 20 | humans say 'good' on 13
judge approves 2 replies that humans rejected
an always-yes judge: agreement 0.65 kappa 0.0
```

1. The two arrays are the grades (1 = acceptable) from a person and from a judge for the same 20 replies. They disagree on exactly two replies, so raw agreement is 0.9.
2. Kappa is lower than 0.9 because some agreement would occur by chance, especially when most replies are good. Here kappa is 0.765. A kappa above about 0.6 is usually read as substantial agreement (a rule of thumb, not a law).

3. The judge is more lenient than the humans: it approves two replies that humans rejected. A lenient judge inflates every system's score, which matters if you compare against old numbers.
4. An always-yes judge reaches 0.65 raw agreement (13 of 20 replies are good) yet kappa 0.0. This is the reason to look at kappa and not only agreement.

COMMON MISTAKE	**Wrong:** editing the prompt until the score on the test set goes up, then reporting that score. **Symptom:** an impressive offline number that collapses in production, because the prompt now fits the test examples. **Fix:** tune on a validation set, touch the test set once at the end, and keep a few fresh real examples for a final check.

21.11 Recipes: How Do I ...?

Each recipe is a small, real task. Torch snippets are marked not run here; the ones with output are runnable stand-ins you can execute.

Recipe 1: How do I freeze the pretrained body and train only a new head?

Set `requires_grad_(False)` on the body and give the optimiser only the parameters that still require gradients (not run here):

```
for p in model.body.parameters():
    p.requires_grad_(False)
trainable = [p for p in model.parameters() if p.requires_grad]
print(sum(p.numel() for p in trainable), "trainable of", sum(p.numel() for p in model.parameters()))
opt = torch.optim.AdamW(trainable, lr=1e-3)
model.body.eval()          # also stop dropout / batch-norm updates inside the frozen part
```

Recipe 2: How do I use a smaller learning rate for the pretrained body than for the new head?

Pass parameter groups (not run here). Typical: 10 to 100 times smaller for pretrained layers.

```
opt = torch.optim.AdamW([
    {"params": model.encoder.parameters(), "lr": 2e-5},
    {"params": model.classifier.parameters(), "lr": 1e-3},
], weight_decay=0.01)
```

Recipe 3: How do I resume training after a crash?

Save the model, the optimiser *and* the scheduler plus the epoch; restore all of them (not run here):

```
torch.save({"model": model.state_dict(), "opt": opt.state_dict(),
            "sched": sched.state_dict(), "epoch": epoch}, "last.pt")

ck = torch.load("last.pt", map_location=device, weights_only=True)
model.load_state_dict(ck["model"]); opt.load_state_dict(ck["opt"]); sched.load_state_dict(ck["sched"])
start_epoch = ck["epoch"] + 1
```

Recipe 4: How do I handle imbalanced classes?

Weight the loss by inverse class frequency (or oversample). The weights below follow the common rule weight = N / (K x count).

```
import numpy as np

y = np.array([0] * 90 + [1] * 8 + [2] * 2)          # 90 billing, 8 bug, 2 outage
counts = np.bincount(y)
weights = len(y) / (len(counts) * counts)
print("counts :", counts)
print("weights:", np.round(weights, 3))

def weighted_ce(logits, y, w):
    z = logits - logits.max(1, keepdims=True)
    logp = z - np.log(np.exp(z).sum(1, keepdims=True))
    per = -logp[np.arange(len(y)), y]
    return (w[y] * per).sum() / w[y].sum()          # PyTorch normalises by the sum of weights too

logits = np.zeros((len(y), 3))                      # an uninformed model
print("plain loss   :", round(float(-np.log(1 / 3)), 4))
print("weighted loss:", round(float(weighted_ce(logits, y, weights)), 4))
logits[:, 0] = 3.0                                  # a model that always says class 0
print("always-0 plain accuracy:", (logits.argmax(1) == y).mean(), "| weighted loss:", round(float(weighted_ce(logits, y,
weights)), 3))
```

```
counts : [90  8  2]
weights: [ 0.37   4.167 16.667]
plain loss   : 1.0986
weighted loss: 1.0986
always-0 plain accuracy: 0.9 | weighted loss: 2.095
```

1. Class 2 appears only twice in 100 rows, so its weight is 100 / (3 x 2) = 16.667, while class 0 gets 0.370.
2. With zero logits every class has probability 1/3, so the plain loss is log 3 = 1.0986; the weighted loss is the same because the model is uniform.
3. A model that always predicts the majority class gets 90 percent accuracy, which looks good but is useless. The weighted loss (about 2.1) exposes it because the rare examples are expensive. In PyTorch: `nn.CrossEntropyLoss(weight=torch.tensor(weights, dtype=torch.float32))`. Judge with per-class recall, not accuracy.

Recipe 5: How do I run inference on 100,000 texts without running out of memory?

Sort by length so batches have little padding, use no-grad mode, move results to the CPU as you go (not run here), and restore the original order at the end. The runnable part below shows the ordering logic.

```
import numpy as np

lengths = np.array([5, 40, 7, 38, 6, 41, 9, 37, 8, 39])
order = np.argsort(lengths)                         # short with short, long with long
batches = [order[i:i + 5] for i in range(0, len(order), 5)]
padded_sorted = sum(lengths[b].max() * len(b) for b in batches)
unsorted = [np.arange(10)[i:i + 5] for i in range(0, 10, 5)]
padded_unsorted = sum(lengths[b].max() * len(b) for b in unsorted)
print("padded cells, unsorted batches:", padded_unsorted, "| sorted batches:", padded_sorted)

preds_sorted = lengths[order] * 2                   # pretend model output, in sorted order
restored = np.empty_like(preds_sorted)
restored[order] = preds_sorted                      # undo the sort
print("original order restored:", np.array_equal(restored, lengths * 2))
```

```
padded cells, unsorted batches: 405 | sorted batches: 250
original order restored: True
```

1. Padding cost is batch size times the longest item in the batch. Unsorted, each batch of five contains a 40-ish token text, so the batches pad to 40 and 41 and cost 200 + 205 = 405 cells. Sorted, one batch pads to 9 and the other to 41: 45 + 205 = 250 cells.

2. `restored[order] = preds_sorted` is the standard inverse permutation. Skipping it returns predictions in the wrong order: a silent, serious bug.

```
@torch.inference_mode()
def predict(texts, batch_size=64):
    order = sorted(range(len(texts)), key=lambda i: len(texts[i]))
    out = [None] * len(texts)
    model.eval()
    for s in range(0, len(order), batch_size):
        idx = order[s:s + batch_size]
        enc = tok([texts[i] for i in idx], padding=True, truncation=True, max_length=256, return_tensors="pt").to(device)
        probs = torch.softmax(model(**enc).logits, dim=-1).cpu().numpy()      # move off the GPU each batch
        for j, i in enumerate(idx):
            out[i] = probs[j]
    return out
```

Recipe 6: How do I chunk long documents for retrieval?

Cut into windows of roughly fixed size with overlap so that a sentence cut at one boundary appears whole in the neighbour. Here words stand in for tokens.

```
def chunk(words, size, overlap):
    assert 0 <= overlap < size
    step = size - overlap
    return [words[i:i + size] for i in range(0, max(1, len(words) - overlap), step)]

text = "one two three four five six seven eight nine ten eleven twelve".split()
for c in chunk(text, size=5, overlap=2):
    print(c)
print("chunks for 1000 words, size 200, overlap 50:", len(chunk(list(range(1000)), 200, 50)))
```

```
['one', 'two', 'three', 'four', 'five']
['four', 'five', 'six', 'seven', 'eight']
['seven', 'eight', 'nine', 'ten', 'eleven']
['ten', 'eleven', 'twelve']
chunks for 1000 words, size 200, overlap 50: 7
```

1. The step between window starts is `size - overlap` = 3, so windows start at words 0, 3, 6 and 9, and consecutive chunks share two words.
2. For 1000 words with step 150 the starts are 0, 150, ..., 900 (7 windows; the last one is shorter). Store the start offset in the metadata so you can cite where an answer came from.
3. Chunk by tokens (or sentences) in real systems, not characters, because the embedding model limit is in tokens.

Recipe 7: How do I find near-duplicate tickets?

Embed, normalise, compute the similarity matrix, and keep pairs above a threshold. For large sets use an ANN index instead of the full matrix.

```
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer

tickets = ["charged twice this month", "I was charged twice this month", "app crashes on login",
           "the app crashes on login screen", "how do I export data"]
M = TfidfVectorizer(ngram_range=(1, 2)).fit_transform(tickets)
S = (M @ M.T).toarray()                       # TF-IDF rows are unit length, so this is cosine
np.fill_diagonal(S, 0)
i, j = np.where(np.triu(S) > 0.5)
for a, b in zip(i, j):
    print(f"{a} ~ {b}  cosine {S[a, b]:.2f}  {tickets[a]!r} / {tickets[b]!r}")
```

```
0 ~ 1  cosine 0.83  'charged twice this month' / 'I was charged twice this month'
2 ~ 3  cosine 0.73  'app crashes on login' / 'the app crashes on login screen'
```

1. `M @ M.T` gives every pairwise cosine at once. The diagonal (each text with itself) is zeroed so it does not match.
2. `np.triu` keeps each pair once. Only the two paraphrase pairs should pass the 0.5 threshold; pick the threshold by inspecting pairs near it.
3. For 1 million tickets the matrix would have 10^{12} cells, so use FAISS range search or nearest neighbours per item.

Recipe 8: How do I cache LLM answers so repeated prompts are free?

```
import hashlib, json

class CachedClient:
    def __init__(self, llm):
        self.llm, self.cache, self.hits = llm, {}, 0
    def complete(self, model, prompt, temperature=0):
        key = hashlib.sha256(json.dumps([model, prompt, temperature], sort_keys=True).encode()).hexdigest()
        if key in self.cache:
            self.hits += 1
            return self.cache[key]
        self.cache[key] = self.llm(prompt)
        return self.cache[key]

calls = []
def llm(prompt):
    calls.append(prompt)
    return prompt.upper()

c = CachedClient(llm)
for p in ["hello", "hello", "bye", "hello"]:
    c.complete("m1", p)
print("real calls:", len(calls), "| cache hits:", c.hits)
c.complete("m2", "hello")
print("different model -> new call, total real calls:", len(calls))
```

```
real calls: 2 | cache hits: 2
different model -> new call, total real calls: 3
```

1. The key hashes everything that changes the answer: model name, prompt and temperature. Leaving one out returns stale answers.
2. Four requests make two real calls and two hits; asking the same prompt of a different model is a miss.
3. Cache only when output is meant to be repeatable (temperature 0). Caching a creative sampling call makes every user see one answer. Providers also offer prompt caching on the server side for long repeated prefixes; that is separate.

Recipe 9: How do I call an LLM for 1,000 rows concurrently without flooding the API?

Use `asyncio` with a semaphore that caps in-flight calls. The fake client sleeps briefly to imitate latency.

```
import asyncio

active, peak = 0, 0

async def fake_call(i):
    global active, peak
    active += 1
    peak = max(peak, active)
    await asyncio.sleep(0.001)
    active -= 1
    return i * 2

async def run_all(n, limit):
    sem = asyncio.Semaphore(limit)
    async def guarded(i):
        async with sem:
            return await fake_call(i)
    return await asyncio.gather(*(guarded(i) for i in range(n)))

results = asyncio.run(run_all(200, limit=8))
print("results in input order:", results[:5], "... count", len(results))
print("peak concurrent calls:", peak)
```

```
results in input order: [0, 2, 4, 6, 8] ... count 200
peak concurrent calls: 8
```

1. All 200 tasks are created at once, but the semaphore lets only 8 run the call at a time; the peak counter confirms 8.
2. `asyncio.gather` returns results in the order the tasks were given, not the order they finished, so rows stay aligned with inputs.
3. With real async clients (`AsyncAnthropic`, `AsyncOpenAI`) the structure is the same; combine it with the retry helper from 21.9 inside `guarded`, and keep the limit under your rate limit.

Recipe 10: How do I make a run reproducible?

Seed every generator you use, and know the limits. In PyTorch: `random.seed`, `np.random.seed`, `torch.manual_seed` (this also seeds CUDA), a seeded `torch.Generator` for the DataLoader, and optionally `torch.use_deterministic_algorithms(True)` (slower, and some operations then raise errors). The NumPy side of the idea:

```
import numpy as np

def noisy_mean(seed):
    rng = np.random.default_rng(seed)
    return float(rng.normal(size=1000).mean())

print(noisy_mean(7) == noisy_mean(7), noisy_mean(7) == noisy_mean(8))
child1, child2 = np.random.default_rng(1), np.random.default_rng(2)
print("independent streams differ:", child1.random() != child2.random())
```

```
True False
independent streams differ: True
```

1. The same seed reproduces the same number exactly; a different seed does not.
2. Prefer one explicit `Generator` object per component over the global state, so adding a random call in one place does not shift the numbers somewhere else.
3. Exact reproducibility across different GPUs or library versions is not guaranteed; report mean and spread over several seeds when the result matters.

Recipe 11: How do I keep a chat history inside the model's token limit?

Keep the system message, then the newest turns that fit the budget. Count with the provider's token counter (`count_tokens`, or the model's tokenizer); words are used below only to keep the example runnable.

```
def fit_history(system, turns, budget, count=lambda s: len(s.split())):
    used = count(system)
    kept = []
    for role, text in reversed(turns):            # walk from the newest turn backwards
        c = count(text)
        if used + c > budget:
            break
        kept.append((role, text)); used += c
    return list(reversed(kept)), used

turns = [("user", "hello there"), ("assistant", "hi how can I help you today"),
         ("user", "my invoice is wrong"), ("assistant", "which month is it from"), ("user", "last month")]
kept, used = fit_history("You are a billing assistant", turns, budget=16)
print([t[1] for t in kept], "| used:", used)
```

```
['my invoice is wrong', 'which month is it from', 'last month'] | used: 16
```

1. The system message costs 5 words and is always kept.
2. Walking backwards keeps the three newest turns (2 + 5 + 4 = 11 more words, total 16, exactly the budget). The next older turn ("hi how can I help you today", 7 words) would exceed it, so the loop stops.
3. Dropping old turns blindly can lose facts; summarising old turns into one short message is the usual refinement.

Recipe 12: How do I turn logits into top-k labels with probabilities?

```
import numpy as np

labels = np.array(["billing", "bug", "how-to", "other"])
logits = np.array([[2.0, 0.5, -1.0, 0.0], [0.1, 0.2, 3.0, 0.0]])
z = logits - logits.max(1, keepdims=True)
probs = np.exp(z) / np.exp(z).sum(1, keepdims=True)
top2 = np.argsort(-probs, axis=1)[:, :2]                    # torch.topk(probs, 2)
for row, idx in enumerate(top2):
    print([(str(labels[i]), round(float(probs[row, i]), 3)) for i in idx])
```

```
[('billing', 0.71), ('bug', 0.158)]
[('how-to', 0.858), ('bug', 0.052)]
```

1. Softmax turns each row of scores into probabilities that sum to 1.
2. `argsort(-probs)` sorts descending; the first two columns are the top two classes per row. `torch.topk` returns values and indices together.
3. Show users the probability, but remember softmax probabilities of an uncalibrated network are often over-confident.

21.12 Performance and Pitfalls

Twelve traps, each with the wrong code, the symptom and the fix. The numbers in this section are timings measured on the machine that produced this book; they vary on yours, so measure your own.

COMMON MISTAKE 1: FORGOT TO CLEAR GRADIENTS

Wrong: a loop with `loss.backward(); opt.step()` and no `opt.zero_grad()`. **Symptom:** the loss behaves erratically or diverges, because every step uses the sum of all gradients so far (we saw accumulation in 21.2). **Fix:** call `opt.zero_grad(set_to_none=True)` once per optimiser step, before `backward()`.

COMMON MISTAKE 2: EVALUATING IN TRAIN MODE

Wrong: `preds = model(x)` after training without `model.eval()` or `no_grad`. **Symptom:** predictions change each call (dropout), batch-norm statistics drift, and memory climbs because a graph is built. **Fix:** `model.eval()` plus `with torch.inference_mode():`, and switch back with `model.train()` before training again.

COMMON MISTAKE 3: ROW-BY-ROW INSTEAD OF BATCHED

Wrong: `[model(x) for x in rows]` or `for row in df.itertuples()` around a model. **Symptom:** inference is many times slower than needed. **Fix:** stack rows into a batch and call once. The timing below shows the same effect in NumPy.

```
import time
import numpy as np

rng = np.random.default_rng(0)
X = rng.normal(size=(2000, 64)).astype(np.float32)
W = rng.normal(size=(64, 32)).astype(np.float32)

t0 = time.perf_counter(); slow = np.stack([x @ W for x in X]); t_loop = time.perf_counter() - t0
t0 = time.perf_counter(); fast = X @ W; t_batch = time.perf_counter() - t0
print("same result:", np.allclose(slow, fast, atol=1e-4))
print("batched is more than 2x faster:", t_loop / t_batch > 2)
```

```
same result: True
batched is more than 2x faster: True
```

On the machine used here the loop took about 2 ms and the batch about 0.4 ms, a ratio around 6 (it varies from run to run). With a real model on a GPU the gap is much larger, because each call has a fixed launch overhead.

COMMON MISTAKE 4: A STARVING GPU

Wrong: `DataLoader(ds, batch_size=64)` with heavy preprocessing in `__getitem__` and the default `num_workers=0`. **Symptom:** the GPU shows low utilisation and each epoch is slow even though the model is small. **Fix:** `num_workers=2` or more, `pin_memory=True`, do the tokenisation once in a `map` step instead of per item, and time data loading separately from the model step.

COMMON MISTAKE 5: LEAKAGE THROUGH PREPROCESSING

Wrong: `scaler.fit(X_all)` or learning a vocabulary on all text before splitting, or duplicates of one ticket on both sides of the split. **Symptom:** validation scores far better than production. **Fix:** split first, fit every learned transform on the training part only, deduplicate (recipe 7) before splitting, and split by customer or time when rows are related.

COMMON MISTAKE 6: LOADING UNTRUSTED PICKLES

Wrong: `torch.load("model_from_the_internet.pt")` with old defaults, or `pickle.load`. **Symptom:** none until a malicious file runs code on your machine. **Fix:** `weights_only=True`, or the safetensors format, and download models only from sources you trust.

COMMON MISTAKE 7: LEARNING RATE TOO HIGH AND NO CLIPPING	**Wrong:** `lr=0.1` with Adam on a transformer. **Symptom:** the loss jumps to `nan` within a few steps (we saw divergence in 21.3). **Fix:** start near 1e-5 to 3e-4 for Adam-type optimisers on transformers, add warm-up, clip the gradient norm to 1.0, and log the pre-clip norm to see spikes.
COMMON MISTAKE 8: WRONG DIMENSION FOR SEQUENCE LOGITS	**Wrong:** `F.cross_entropy(logits, targets)` with logits of shape (B, T, V) and targets (B, T). **Symptom:** a shape error, or (worse, if sizes line up) a loss computed over the wrong axis. **Fix:** `F.cross_entropy(logits.reshape(-1, V), targets.reshape(-1))`, or `logits.transpose(1, 2)` to (B, V, T), and set `ignore_index=-100` for padded positions.
COMMON MISTAKE 9: AN IN-PLACE OPERATION ON A TRACKED TENSOR	**Wrong:** `h = layer(x); h += 1; h.relu_()` where autograd still needs the old values. **Symptom:** `RuntimeError: one of the variables needed for gradient computation has been modified by an inplace operation`. **Fix:** use the out-of-place form (`h = h + 1`). Reserve in-place updates for code under `torch.no_grad()`, such as manual weight updates.
COMMON MISTAKE 10: SILENT PROMPT TRUNCATION	**Wrong:** stuffing all retrieved text into one prompt without counting tokens, or embedding long documents with a model that has a 256-token limit. **Symptom:** answers ignore the end of the document, or the API returns a context-length error. **Fix:** count tokens before sending, chunk (recipe 6), limit retrieved chunks, and check the embedding model's `max_seq_length`.
COMMON MISTAKE 11: RETRYING ACTIONS THAT ARE NOT SAFE TO REPEAT	**Wrong:** wrapping a tool call that sends an email or issues a refund inside the same retry loop as the model call. **Symptom:** after a timeout the action runs twice. **Fix:** retry only the read-only model request; make side-effect tools idempotent with an idempotency key, and record completed actions.
COMMON MISTAKE 12: SECRETS IN CODE	**Wrong:** `client = OpenAI(api_key="sk-...")` committed to a repository or notebook. **Symptom:** leaked key, unexpected bills, a forced key rotation. **Fix:** read keys from environment variables or a secret manager (the SDKs do this by default), keep `.env` in `.gitignore`, and never print request headers in logs.
KEY IDEA	Speed order of attack: first make it correct, then *measure*, then fix the biggest cost. In training the usual order is data loading, batch size and padding, mixed precision (bfloat16), and only then compile or kernel-level work. In LLM apps it is fewer and shorter calls, caching, concurrency within rate limits, and a smaller model for easy cases.

21.13 Common Error Messages and Their Fixes

Exact wording differs between library versions, so search for the distinctive phrase. The table lists the situation, the cause and the fix.

Message (abridged)	Cause	Fix
`RuntimeError: mat1 and mat2 shapes cannot be multiplied (32x10 and 20x5)`	Layer expects 20 inputs, got 10 features	Print `x.shape`; fix `in_features` or the flatten step
`RuntimeError: Expected all tensors to be on the same device, but found at least two`	Model on GPU, a tensor on CPU	`x = x.to(device)` for every batch field

Message (abridged)	Cause	Fix
`devices, cuda:0 and cpu`		
`torch.OutOfMemoryError: CUDA out of memory. Tried to allocate ...`	Batch, sequence or model too big for the GPU	Smaller batch, gradient accumulation, bfloat16, checkpointing, shorter sequences, LoRA (21.5)
`RuntimeError: element 0 of tensors does not require grad and does not have a grad_fn`	Loss detached from the graph (`no_grad` , `.detach()` , frozen everything, or `.item()` used in the loss)	Keep the loss as a tensor built from model outputs; check `requires_grad`
`RuntimeError: Trying to backward through the graph a second time`	Reusing a graph, often hidden state carried across batches	`.detach()` the carried state; avoid `retain_graph`
`one of the variables needed for gradient computation has been modified by an inplace operation`	In-place op on a needed tensor	Use the out-of-place version
`Can't call numpy() on Tensor that requires grad. Use tensor.detach().numpy()`	Converting a tracked tensor	`t.detach().cpu().numpy()`
`expected scalar type Long but found Float` (or the reverse)	Class ids must be integer, inputs float	`y.long()` for CrossEntropy class ids; `x.float()`
`IndexError: Target 5 is out of bounds` / `CUDA error: device-side assert triggered`	Label outside 0..K-1, or token id beyond the embedding size	Check `y.min(), y.max()` and `num_labels` ; rerun on CPU for a clear message
`view size is not compatible with input tensor's size and stride`	`.view` on non-contiguous memory	`.reshape(...)` or `.contiguous().view(...)`
`ValueError: expected sequence of length 5 at dim 1 (got 3)`	Building a tensor from ragged lists	Pad in a collate function (21.4)
`Input type (torch.cuda.FloatTensor) and weight type (torch.FloatTensor) should be the same`	Data on GPU, model on CPU	`model.to(device)`
`ValueError: Asking to pad but the tokenizer does not have a padding token`	Decoder-only tokenisers often lack one	`tok.pad_token = tok.eos_token`
`Token indices sequence length is longer than the specified maximum sequence length`	Text exceeds the model limit	`truncation=True, max_length=...` or chunk the text
`OSError: ... is not a local folder and is not a valid model identifier`	Typo in the model id, no network, or a private model without a token	Check the id; log in; pass a local path
`Error(s) in loading state_dict ... Missing key(s) / size mismatch`	Architecture differs from the checkpoint	Rebuild with the saved config; `strict=False` only to inspect
`AssertionError: ... d == self.d` (FAISS) or dtype errors	Wrong vector size or float64	`x.astype("float32")` , same dimension as the index
`openai.RateLimitError` / `anthropic.RateLimitError` (429)	Too many requests or tokens per minute	Backoff, `retry-after` , lower concurrency (21.9)
`BadRequestError` (400) mentioning context length or max tokens	Prompt plus output exceeds the window, or invalid parameters	Count and trim tokens; do not retry unchanged

Message (abridged)	Cause	Fix
`AuthenticationError` (401)	Missing or wrong key	Set the environment variable; check the organisation
`json.JSONDecodeError` on a model reply	Fence, trailing text, truncation	Structured output feature, validator, repair loop (21.9)

The NumPy equivalents of the first and last of these are real errors you can see right now:

```
import numpy as np

x = np.ones((32, 10)); W = np.ones((20, 5))
try:
    x @ W
except ValueError as e:
    print("numpy:", str(e).splitlines()[0][:70])
try:
    np.array([[1, 2, 3, 4, 5], [1, 2, 3]], dtype=float)
except ValueError as e:
    print("ragged:", str(e)[:80])
t = np.arange(12).reshape(3, 4).T
try:
    t.view().shape = (12,)
except AttributeError as e:
    print("reshape in place:", str(e)[:70])
```

```
numpy: matmul: Input operand 1 has a mismatch in its core dimension 0, with g
ragged: setting an array element with a sequence. The requested array has an inhomogeneo
reshape in place: Incompatible shape for in-place modification. Use `.reshape()` to make
```

1. The first error is NumPy's version of the PyTorch matrix shape message. Both name the offending sizes; read them as (rows, columns) pairs and check the inner numbers (10 versus 20).
2. Building an array from ragged lists fails for the same reason a tensor does: a rectangle needs equal lengths.
3. Assigning a new shape to a transposed array in place fails because the memory is not contiguous; `reshape` copies where necessary, exactly as `tensor.reshape` does.

21.14 Cross-Library Integration

Real projects pass data between libraries. Most integration bugs are one of three kinds: **dtype** (float64 versus float32, int32 versus int64), **memory sharing** (a view changes the original), and **location** (CPU versus GPU, or a tracked tensor versus plain NumPy). The table is the handoff cheat sheet.

From to	How	Watch out for
pandas to PyTorch	`torch.tensor(df[cols].to_numpy(dtype="float32"))`	Mixed or string columns give object arrays; encode first. Missing values become NaN and poison the loss
NumPy to PyTorch	`torch.from_numpy(arr)` (shares memory) or `torch.tensor(arr)` (copies)	float64 stays float64; the model's weights are float32, so cast with `.float()`
PyTorch to NumPy	`t.detach().cpu().numpy()`	The result shares memory with the tensor on the CPU
PyTorch to scikit-learn metrics	Convert predictions and labels to NumPy first	Pass class ids (argmax), not logits, to accuracy

From to	How	Watch out for
scikit-learn to PyTorch	Fit scalers and encoders on train; save them with `joblib`; apply at inference	Training-time and serving-time preprocessing must match exactly
Hugging Face dataset to pandas	`ds.to_pandas()`, back with `Dataset.from_pandas(df)`	Index columns, categorical dtypes, and the new pandas string dtype
Embeddings to FAISS	`np.ascontiguousarray(E, dtype="float32")`	Dimension fixed at index creation; normalise if you use inner product
Embeddings to Chroma	`E.tolist()` or let Chroma embed	Different embedding function at query time than at ingest time
SDK reply to pandas	Validate to a dict or dataclass, then `pd.DataFrame(records)`	Store the raw reply and the parsed fields so you can audit later
Config and secrets	`yaml.safe_load` for settings, environment variables for keys	Never log keys; record the config with every run

Problem 21.14.1 Take a small DataFrame, build float32 features and integer labels ready for a PyTorch model, check memory sharing, evaluate with scikit-learn, and put the results back into pandas. (The model is a logistic regression, standing in for the network.)

```
import numpy as np
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

df = pd.DataFrame({
    "text": ["refund my charge", "charged twice", "invoice is wrong", "refund the payment",
             "app crashes", "error on save", "page is broken", "login fails with error"],
    "label": ["billing", "billing", "billing", "billing", "bug", "bug", "bug", "bug"],
})
print("text dtype:", df["text"].dtype, "| label dtype:", df["label"].dtype)

cat = pd.Categorical(df["label"])
y = cat.codes.astype(np.int64)                        # class ids: int64 is what PyTorch's CrossEntropy wants
X = TfidfVectorizer().fit_transform(df["text"]).toarray().astype(np.float32)
X = np.ascontiguousarray(X)
print("X:", X.shape, X.dtype, "| y:", y.dtype, "| classes:", list(cat.categories))

view = X[:2]
print("slice shares memory (torch.from_numpy would too):", np.shares_memory(X, view))

clf = LogisticRegression(max_iter=200).fit(X, y)
pred = clf.predict(X)
print("accuracy:", accuracy_score(y, pred))

out = df.assign(pred=cat.categories[pred], correct=(pred == y))     # a NEW frame (pandas 3 copy-on-write)
subset = out[out["label"] == "bug"]
subset.loc[subset.index[0], "pred"] = "changed"                      # touches only the subset
print("original untouched:", out.loc[4, "pred"] != "changed", "| rows:", len(out), "columns:", list(out.columns))
```

```
text dtype: str | label dtype: str
X: (8, 20) float32 | y: int64 | classes: ['billing', 'bug']
slice shares memory (torch.from_numpy would too): True
accuracy: 1.0
original untouched: True | rows: 8 columns: ['text', 'label', 'pred', 'correct']
```

1. In pandas 3 the text column has the string dtype (printed `str`); the label column too. The model needs numbers, so labels become integer category codes and text becomes a float matrix.
2. The label ids are `int64` because PyTorch's cross-entropy needs `long` class ids. Features are cast to `float32`, the dtype of model weights: `float64` input to a `float32` layer raises a dtype error.
3. `np.ascontiguousarray` guarantees a plain C-ordered block, which `torch.from_numpy` and FAISS both prefer.
4. Slicing shares memory (`True`), as with `from_numpy`: changing the tensor changes the array. Use `.copy()` or `torch.tensor` if you need independence.
5. The scikit-learn metric takes plain integer arrays. With a PyTorch model you would first call `logits.argmax(-1).cpu().numpy()`.
6. `df.assign` returns a new frame. Under pandas 3 copy-on-write, writing into `subset` never changes `out`; the final check prints `True`. The old chained-assignment bugs ("SettingWithCopy") are gone.

Talking to a model server over HTTP with httpx

Underneath every SDK is an HTTP call. Knowing the raw form helps when you debug, use a self-hosted model server, or need to see headers. Here a local server plays a rate-limited LLM endpoint: the first request gets a 429 with `Retry-After`, the second succeeds. The client obeys the header. (Local only; no external network.)

```
import json, threading, time
import httpx
from http.server import BaseHTTPRequestHandler, HTTPServer

state = {"calls": 0}

class Handler(BaseHTTPRequestHandler):
    def do_POST(self):
        state["calls"] += 1
        body = json.loads(self.rfile.read(int(self.headers["Content-Length"])))
        if state["calls"] == 1:
            self.send_response(429); self.send_header("Retry-After", "0.05"); self.end_headers(); return
        reply = json.dumps({"text": body["prompt"].upper(), "usage": {"input_tokens": 3, "output_tokens": 3}}).encode()
        self.send_response(200); self.send_header("Content-Type", "application/json")
        self.send_header("Content-Length", str(len(reply))); self.end_headers(); self.wfile.write(reply)
    def log_message(self, *a): pass

srv = HTTPServer(("127.0.0.1", 0), Handler)
threading.Thread(target=srv.serve_forever, daemon=True).start()
url = f"http://127.0.0.1:{srv.server_address[1]}/v1/complete"

def post_with_retry(client, url, payload, max_retries=3):
    for attempt in range(max_retries + 1):
        r = client.post(url, json=payload, headers={"Authorization": "Bearer demo-key"})
        if r.status_code == 429 and attempt < max_retries:
            time.sleep(float(r.headers.get("Retry-After", 1)))
            continue
        r.raise_for_status()                    # 4xx/5xx other than a retried 429 raise here
        return r.json()

with httpx.Client(timeout=5.0) as client:
    data = post_with_retry(client, url, {"prompt": "hello llm"})
print(data["text"], "| usage:", data["usage"], "| server saw", state["calls"], "requests")
srv.shutdown()
```

```
HELLO LLM | usage: {'input_tokens': 3, 'output_tokens': 3} | server saw 2 requests
```

1. The server handler counts requests: the first answer is `429` with a `Retry-After` header, later ones are JSON. The server is started in a thread on port 0 so the operating system picks a free port.
2. `post_with_retry` sends the JSON body and an `Authorization` header. If the status is 429 and retries remain, it sleeps for the number of seconds in `Retry-After` and loops.

3. `raise_for_status()` turns any other error status into an exception, so a 400 is never mistaken for data (httpx, like requests, does not raise by itself).
4. The output shows the uppercased text, the usage numbers and that the server saw two requests: one refused, one served. This is exactly what an SDK's built-in retry does for you.

21.15 How to Solve Any Deep Learning and GenAI Libraries Problem

The decision flow

1. **Name the symptom exactly**: error text, wrong number, slow, too big, or unreliable output. Copy the last line of the error, not the whole traceback.
2. **Shrink the problem**: one batch, ten rows, one prompt, CPU only, a fixed seed. Most bugs reproduce on tiny input, and tiny input runs in seconds.
3. **Print the facts**: shapes, dtypes, devices, min/max/NaN counts, the actual prompt sent, the raw reply received.
4. **Test the pipeline in pieces**: data to batch, batch to loss, loss to gradient, gradient to update. Find the first stage whose output is wrong.
5. **Compare against a trivial baseline** (majority class, keyword rule, random). If the fancy model barely beats it, suspect the data or the labels before the architecture.
6. **Change one thing at a time** and record the result.

Symptom	First check	Usual cause	Fix
Loss does not go down	Overfit 8 examples	No step, wrong labels, lr too small, frozen layers, gradients not zeroed	Pattern 1
Loss is NaN or inf	First step it appears; input NaNs	lr too high, log of zero, bad data, float16 overflow	Pattern 2
Shape or device error	Print shapes and devices	Wrong flatten, batch-first mix-up, tensor left on CPU	21.6, 21.13
Out of memory	Do the arithmetic (21.5)	Batch, sequence length, no `no_grad`, stored graphs	Pattern 4
Search results are poor	Print top-5 for 10 queries and read them	Bad chunks, mismatched embedder, no normalisation	Pattern 5
LLM output unreliable	Print the exact prompt and raw reply	Vague instruction, no schema, no validation	Pattern 6
Too slow	Time data versus compute	Loading, padding, tiny batches, many calls	Pattern 7
Results differ between runs	Fix all seeds	Unseeded shuffle, dropout in eval, nondeterministic kernels	Recipe 10

Pattern 1: the loss does not decrease. Overfit one small batch.

A correct training pipeline can memorise 8 examples. If it cannot, the bug is in the code, not in the data size or the architecture. Below, the correct loop reaches almost zero loss, and a loop with one wrong sign (updating *up* the gradient) does not.

```
import numpy as np

def make_data():
    rng = np.random.default_rng(0)
    X = rng.normal(size=(8, 4))
    y = np.array([0, 1, 0, 1, 1, 0, 1, 0])
    return X, y, rng

def train(sign, steps=300, lr=0.5):
    X, y, rng = make_data()
    W1, b1 = rng.normal(scale=0.5, size=(4, 16)), np.zeros(16)
    W2, b2 = rng.normal(scale=0.5, size=(16, 2)), np.zeros(2)
    losses = []
    for _ in range(steps):
        h = np.maximum(X @ W1 + b1, 0)
        z = h @ W2 + b2
        z -= z.max(1, keepdims=True)
        p = np.exp(z); p /= p.sum(1, keepdims=True)
        losses.append(-np.log(p[np.arange(8), y]).mean())
        g = p.copy(); g[np.arange(8), y] -= 1; g /= 8
        gW2, gb2 = h.T @ g, g.sum(0)
        gh = (g @ W2.T) * (h > 0)
        gW1, gb1 = X.T @ gh, gh.sum(0)
        W1 -= sign * lr * gW1; b1 -= sign * lr * gb1
        W2 -= sign * lr * gW2; b2 -= sign * lr * gb2
    return losses

good, bad = train(+1), train(-1)
print("correct update: loss", round(good[0], 3), "->", f"{good[-1]:.4f}")
print("sign bug      : loss", round(bad[0], 3), "->", round(bad[-1], 3))
```

```
correct update: loss 0.715 -> 0.0029
sign bug      : loss 0.715 -> inf
```

1. Eight points with random labels cannot be fitted by a straight line, but a 16-unit hidden layer has enough capacity to memorise them.
2. The correct run starts near log 2 = 0.69 (chance for two classes) and falls close to zero. The buggy run climbs the loss surface instead: the loss grows until it overflows to `inf`.
3. This test costs seconds and splits the world in two: if the tiny batch cannot be fitted, look at the loop (step, zero_grad, labels, learning rate, frozen parameters); if it can, look at data quality, capacity, regularisation and learning-rate schedule.
4. In PyTorch the same test is: take one batch, `for _ in range(200): train_step(batch)`, and expect the loss near zero.

Pattern 2: NaN or infinite loss. Find the first bad step.

Do not stare at the final NaN; find the step where the loss first stops being finite, and the quantity that exploded just before it. Then test the two classic fixes: a lower learning rate and gradient clipping.

```
import numpy as np

def run(lr, clip=None, steps=400):
    x = 3.0                                          # minimise 50 * x^2, gradient 100 * x
    for step in range(1, steps + 1):
        g = 100 * x
        if clip is not None and abs(g) > clip:
            g = clip * np.sign(g)                    # clip the gradient
        x = x - lr * g
        if not np.isfinite(50 * x * x):
            return f"loss not finite at step {step}"
    return f"finished, x = {x:.4f}"

with np.errstate(all="ignore"):
    print("lr 0.05           :", run(0.05))
    print("lr 0.05 + clip 5  :", run(0.05, clip=5.0))
    print("lr 0.005          :", run(0.005))
```

```
lr 0.05           : loss not finite at step 254
lr 0.05 + clip 5  : finished, x = 0.0000
lr 0.005          : finished, x = 0.0000
```

1. The curvature is 100, so the stable range is lr below 2/100 = 0.02. At 0.05 each step multiplies x by 1 - 0.05 x 100 = -4, so x grows fourfold per step until the number overflows; the function reports the step.
2. With clipping the step length is capped (here to 0.05 x 5 = 0.25), so x walks in at a bounded speed and settles at zero.
3. Lowering the learning rate to 0.005 gives a contraction factor 0.5 per step: smooth convergence.
4. In real training add the same detective work with `torch.isfinite(loss)` checks, `torch.autograd.set_detect_anomaly(True)` (slow, for debugging only), and logging of the gradient norm.

Pattern 3: shape and device errors

Solved example: the matmul error `mat1 and mat2 shapes cannot be multiplied (32x10 and 20x5)`. Read it as batch 32, 10 features coming in, a layer that expects 20. The fix is either the layer (`nn.Linear(10, 5)`) or the data pipeline that should produce 20 features. Use the `check` helper of 21.6 at the top of `forward`.

Pattern 4: out of memory

Solved example: a 1.3-billion-parameter model fully fine-tuned in fp32 with Adam needs 1.3e9 x 16 = 20.8 GB before activations, so it cannot fit a 16 GB card at any batch size. The ladder of fixes, cheapest first: `inference_mode` for evaluation; halve the batch and use gradient accumulation; bfloat16 autocast; gradient checkpointing; shorter sequences; 8-bit or paged optimisers; LoRA (21.7); a smaller model.

Pattern 5: poor retrieval. Measure, do not guess.

Write 10 to 30 real queries with the topic they should hit, then compute hit rates for each variant. Below, three embedding variants are compared on the 24-ticket corpus by *precision@3*: the share of the top-3 results coming from the expected topic.

```
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.decomposition import TruncatedSVD
from sklearn.preprocessing import Normalizer
from sklearn.pipeline import make_pipeline

docs = [
 "I was charged twice for my monthly subscription", "please refund the duplicate payment on my invoice",
 "my invoice shows the wrong billing amount", "how do I update the credit card used for payment",
 "the discount code was not applied to my invoice", "I want a refund for the annual plan I never used",
 "payment failed but my card was charged", "need a receipt for last month payment",
 "the app crashes when I open the settings page", "login button does nothing on the mobile app",
 "error 500 appears when I save my profile", "the export freezes and the page never loads",
 "dark mode breaks the layout of the dashboard", "the mobile app crashes after the latest update",
 "search returns an error instead of results", "upload stops at ninety percent and shows an error",
 "how do I reset my password", "where can I find my api key in the dashboard",
 "how to export my data as a csv file", "how can I invite a teammate to my workspace",
 "how do I change the language of the app", "how do I enable two factor login",
 "how to connect the dashboard to my database", "how can I delete my account and data",
]
topic = np.array(["billing"] * 8 + ["bug"] * 8 + ["how-to"] * 8)
queries = [("my card was billed two times", "billing"), ("give me my money back", "billing"),
           ("receipt for payment", "billing"), ("the mobile app keeps crashing", "bug"),
           ("error when saving", "bug"), ("page freezes", "bug"),
           ("how can I export my data", "how-to"), ("where is the api key", "how-to"),
           ("how do I invite someone", "how-to")]

def precision_at_3(embed):
    E = embed.fit_transform(docs)
    hits = 0
    for q, t in queries:
        s = E @ embed.transform([q])[0] if not hasattr(E, "toarray") else (E @ embed.transform([q]).T).toarray().ravel()
        top = np.argsort(-s)[:3]
        hits += (topic[top] == t).sum()
    return hits / (3 * len(queries))

variants = {
    "tf-idf words only": make_pipeline(TfidfVectorizer(), Normalizer()),
    "tf-idf + SVD 2 dims": make_pipeline(TfidfVectorizer(), TruncatedSVD(2, random_state=0), Normalizer()),
    "tf-idf + SVD 8 dims": make_pipeline(TfidfVectorizer(), TruncatedSVD(8, random_state=0), Normalizer()),
}
for name, v in variants.items():
    print(f"{name:22s} precision@3 = {precision_at_3(v):.3f}")
```

```
tf-idf words only      precision@3 = 0.741
tf-idf + SVD 2 dims    precision@3 = 0.741
tf-idf + SVD 8 dims    precision@3 = 0.889
```

1. The nine queries use different words from the documents on purpose ("billed two times" versus "charged twice", "money back" versus "refund"), which is where lexical matching struggles and real embedding models help.
2. Precision@3 counts how many of the three results come from the expected topic, averaged over queries. It is crude but it turns an argument ("it feels worse") into a number.
3. The three rows compare variants under the same test. In this run plain words and the 2-dimension compression both score 0.741, while 8 dimensions reach 0.889: enough dimensions let the SVD merge words that co-occur (billed, charged, payment) without discarding too much. With only 9 queries a difference of one or two hits is noise, so treat the numbers as a demonstration of the method, not a verdict.
4. The same loop is how you will choose between embedding models, chunk sizes and values of k in a real system.

Pattern 6: unreliable LLM output

Solved example: the model sometimes replies with prose before the JSON. Fixes in order: state the exact schema and one example in the prompt; use the provider's structured output feature; set temperature low for extraction; validate every reply (21.9); retry with the error message at most twice; log failures and add them to the evaluation set. Measure the *valid-JSON rate* and *field accuracy* before and after each change.

Pattern 7: too slow. Time each stage.

```
import time
import numpy as np

class Timer:
    def __init__(self): self.t = {}
    def stage(self, name):
        timer = self
        class Ctx:
            def __enter__(s): s.t0 = time.perf_counter()
            def __exit__(s, *a): timer.t[name] = timer.t.get(name, 0) + time.perf_counter() - s.t0
        return Ctx()

texts = ["support ticket number %d about billing and refunds" % i for i in range(3000)]
W = np.random.default_rng(0).normal(size=(64, 64)).astype(np.float32)
tm = Timer()
for start in range(0, 3000, 100):
    with tm.stage("data"):                          # python-level tokenising and hashing per item
        batch = np.array([[hash((w, j)) % 97 for j, w in enumerate(t.split())] + [0] * 56 for t in texts[start:start +
100]],
                         dtype=np.float32)[:, :64]
    with tm.stage("compute"):
        out = batch @ W
slowest = max(tm.t, key=tm.t.get)
print("stages measured:", sorted(tm.t))
print("the slower stage in this run is:", slowest)
```

```
stages measured: ['compute', 'data']
the slower stage in this run is: data
```

1. A tiny `Timer` accumulates the time spent in named stages using `perf_counter`.
2. The data stage builds each batch with Python loops and the compute stage is one matrix product, so the data stage dominates here. That mirrors a common real situation: the model is fast, the pipeline around it is slow.
3. Only after you know which stage dominates should you optimise (more workers and pre-tokenised data when data dominates; mixed precision and bigger batches when compute dominates). For GPU code remember that kernels run asynchronously: call `torch.cuda.synchronize()` before reading the clock, or use the built-in profiler.

How to verify and how to debug

- **Verify numerically**: finite-difference gradient checks for custom layers, tiny known inputs with hand-computed outputs, and shape assertions.
- **Verify behaviourally**: overfit one batch; compare with a trivial baseline; check that train loss falls, validation loss behaves, and the confusion matrix makes sense.
- **Verify the data**: print ten random examples with labels and read them; check class counts, duplicates between splits, and the longest items.
- **Debug by bisection**: halve the pipeline until you find the first stage with a wrong output. Print the real prompt and reply for any LLM step.
- **Keep a log** of config, seed, library versions and metrics for every run; a result you cannot reproduce is an anecdote.

21.16 Going Deeper: Internals, Memory, Speed and What Interviewers Probe

Autograd internals

Every tensor produced by a tracked operation carries a `grad_fn` pointing to a node that knows how to compute the vector-Jacobian product for that operation, and to the nodes of its inputs. Nodes save whatever the backward formula needs (for a product, the other factor). Each tensor has a version counter; an in-place edit bumps it, and backward compares the saved version to the current one, which is where the "modified by an inplace operation" error comes from. `backward()` frees the saved buffers unless `retain_graph=True`. The reason for reverse mode is cost: for one scalar loss and millions of parameters, reverse mode computes all gradients in a few times the cost of one forward pass; forward mode would need one pass per parameter.

Where the memory goes

Activations saved for backward scale with batch x sequence length x width x depth, and attention scores add batch x heads x T x T per layer unless a fused attention kernel is used. Gradient checkpointing stores only some activations and recomputes the rest, trading roughly one extra forward pass for a large memory saving. The CUDA caching allocator keeps freed blocks for reuse, so `nvidia-smi` shows more than `memory_allocated()`; fragmentation can cause an out-of-memory error even when total free memory looks sufficient. Text generation adds the **KV cache**: keys and values of all past tokens per layer, so its size grows linearly with context length and batch size.

Where the time goes

Large matrix multiplications are efficient on GPUs; many small operations are not, because each launch has overhead. Hence larger batches, fused operations, `torch.compile`, fused attention kernels, and bfloat16 (half the memory traffic). Padding wastes compute (recipe 5). In data loading, avoid doing string work per item inside the training loop. For LLM APIs the cost is tokens: shorten prompts, cache stable prefixes, batch offline jobs, and use a smaller model for easy cases.

How generation samples a token

A language model outputs a logit for every vocabulary entry. **Temperature** divides logits (lower means sharper, higher means flatter); **top-k** keeps only the k largest; **top-p** keeps the smallest set of tokens whose probability sums to at least p. Then one token is sampled, appended, and the model runs again.

```
import numpy as np

def probs_from_logits(logits, temperature=1.0, top_k=None, top_p=None):
    z = np.asarray(logits, dtype=float) / temperature
    p = np.exp(z - z.max()); p /= p.sum()
    if top_k is not None:
        cutoff = np.sort(p)[-top_k]
        p = np.where(p >= cutoff, p, 0.0)
    if top_p is not None:
        order = np.argsort(-p)
        cum = np.cumsum(p[order])
        keep = order[: np.searchsorted(cum, top_p) + 1]      # smallest prefix reaching top_p
        mask = np.zeros_like(p); mask[keep] = 1
        p = p * mask
    return p / p.sum()

logits = [4.0, 3.0, 2.0, 1.0, 0.0]
for name, kw in [("T=1", {}), ("T=0.5 (sharper)", {"temperature": 0.5}), ("T=2 (flatter)", {"temperature": 2.0}),
                 ("top_k=2", {"top_k": 2}), ("top_p=0.9", {"top_p": 0.9})]:
    print(f"{name:16s}", np.round(probs_from_logits(logits, **kw), 3))

rng = np.random.default_rng(0)
draws = rng.choice(5, size=1000, p=probs_from_logits(logits, top_k=2))
print("top_k=2 draws only use tokens:", sorted(set(draws.tolist())))
```

```
T=1              [0.636 0.234 0.086 0.032 0.012]
T=0.5 (sharper)  [0.865 0.117 0.016 0.002 0.   ]
T=2 (flatter)    [0.429 0.26  0.158 0.096 0.058]
top_k=2          [0.731 0.269 0.    0.    0.   ]
top_p=0.9        [0.665 0.245 0.09  0.    0.   ]
top_k=2 draws only use tokens: [0, 1]
```

1. At T = 1 the probabilities are the plain softmax: the top token gets about 0.64. Halving the temperature sharpens the distribution (top token about 0.87); doubling it flattens it.
2. Top-k 2 zeroes everything but the two largest tokens and renormalises them to roughly 0.73 and 0.27.
3. Top-p 0.9 keeps the smallest prefix of tokens (sorted by probability) whose cumulative probability reaches 0.9, here the top three tokens, then renormalises.
4. Sampling 1000 tokens with top-k 2 only ever produces tokens 0 and 1. Temperature 0 (greedy) is the limit of ever lower temperatures: always the argmax. Greedy is deterministic, but it is not a guarantee of identical output across hardware or service versions.

Vector search internals

Flat search costs N x d multiplications per query. IVF reduces the number of vectors compared (21.8). HNSW builds a layered neighbour graph and walks it greedily; it is accurate and fast but memory-heavy. Product quantisation (PQ) compresses each vector into a few bytes by coding sub-vectors with small codebooks: for 1 million vectors of 768 float32 numbers the raw size is 1e6 x 768 x 4 = 3.07 GB, while with 96 one-byte codes per vector it is 96 MB. Compression loses accuracy, so measure recall.

What interviewers probe

- Explain backpropagation on a two-layer network and why gradients accumulate in `.grad`.
- What do `model.eval()` and `torch.no_grad()` each do? Why do you usually need both?
- Estimate the memory to fine-tune a 7B model with Adam and name three ways to reduce it.
- Why does cross-entropy take logits? Write the stable log-softmax.
- What is the difference between `view` and `reshape`; `detach` and `no_grad`?
- Why bfloat16 over float16? What is a GradScaler for?
- How does LoRA work and what do `r` and `alpha` control?
- Design a RAG system: chunking, embedding, index, retrieval metrics, failure modes, evaluation.
- How would you make an LLM call reliable and cheap at 1 million requests a day?
- How do you know your prompt change helped? (Golden set, baseline, paired comparison, confidence interval, leakage.)

Chapter recap

- Every deep learning program is one loop: batch, forward, loss, backward, optimiser step. If you can write it from memory (21.4), you can debug any framework built on it, including `Trainer`.
- A tensor is an array that remembers its history. `requires_grad` marks what to train, `backward()` fills `.grad`, gradients accumulate until cleared, and the graph is freed after use.
- `nn.Module` registers parameters through attributes. Use `ModuleList` or `Sequential`, never a plain list. `train()` and `eval()` switch dropout and batch-norm behaviour.
- Classification losses take logits. Compute log-softmax stably; the gradient with respect to logits is softmax minus one-hot.
- Shapes, dtypes and devices explain most errors. Write shapes in comments, print them for the first batch and assert the important ones.
- Training memory is roughly parameters x (weights + gradients + optimiser state) plus activations: 16 bytes per parameter for fp32 with Adam. Mixed precision, accumulation, checkpointing and LoRA are the standard levers.
- Hugging Face is a set of conventions: tokeniser, config, weights, `datasets` tables, `Trainer`. Always use the tokeniser that matches the model and pad dynamically.
- LoRA trains W + (alpha / r) B A with W frozen: tiny trainable share, exact no-op at the start, and mergeable at serving time.

- Vector search is nearest-neighbour lookup on embeddings. Normalise for cosine, use float32, measure recall of approximate indexes against exact search, and filter metadata before ranking.
- LLM calls fail, stream and return unchecked text: set client retries and timeouts, back off with jitter, never retry 4xx errors, validate every reply and cap tool loops.
- Evaluation is a fixed golden set, a metric that fits the task, a baseline, a paired comparison with uncertainty, and a held-out test set you touch once.

Formula / pattern	Meaning
loss.backward(); opt.step(); opt.zero_grad()	Backpropagate, update, clear: one training step
dL/dz = softmax(z) - onehot(y)	Gradient of cross-entropy with respect to logits
log softmax(z)$_y$ = z_y - max(z) - log sum exp(z - max(z))	Stable cross-entropy ingredient
Adam: w -= lr $\hat{m}$ / ($\sqrt{\hat{v}}$ + ε)	Per-parameter adaptive step with bias correction
bytes = P x (w + g + opt), fp32 + Adam = 16 P	Training memory before activations
W_{eff} = W + (alpha / r) B A	LoRA update; B starts at zero
mean of k micro-batch grads = big-batch grad	Gradient accumulation (divide loss by k)
$\|\|a - b\|\|^2$ = 2 - 2 cos(a, b) for unit vectors	L2 ranking equals cosine ranking
wait = uniform(0, min(cap, base x $2^{attempt}$))	Full-jitter exponential backoff
recall@k, MRR, nDCG@k	Retrieval quality metrics
paired bootstrap on per-item correctness	Uncertainty of a difference between two systems
overfit one batch	The first test for any training pipeline

Practice set

Graded from easy to expert. Code shown in a problem is not run by the book; answers were checked by running equivalent code.

Problem 21.P.1 (easy) What are logits, and why do `CrossEntropyLoss` and `BCEWithLogitsLoss` want them instead of probabilities?

Answer: Logits are the raw scores a model outputs before softmax or sigmoid. The loss functions fuse the squashing with the logarithm using the log-sum-exp trick, which avoids overflow and taking log of values that underflow to 0. Passing probabilities means softmax is applied twice and stability is lost.

Problem 21.P.2 (easy) How many trainable parameters does `nn.Linear(10, 4)` have?

Answer: The weight matrix has 4 x 10 = 40 numbers and the bias has 4: total 44.

Problem 21.P.3 (easy) A batch holds 32 colour images of 224 x 224 pixels. Give its PyTorch shape, and the shape after `x.flatten(1)`.

Answer: (32, 3, 224, 224), channels first. Flattening from dimension 1 keeps the batch: (32, 3 x 224 x 224) = (32, 150528).

Problem 21.P.4 (easy, what does this print?)

```
import numpy as np
a = np.arange(6)
b = a[:3]
b[0] = 9
c = a[:3].copy()
c[1] = 7
print(a[0], a[1])
```

Answer: `9 1`. The slice `b` is a view, so writing 9 changes `a[0]`. The copy `c` is independent, so `a[1]` stays 1. (Torch behaves the same way for slices and `.clone()`.)

Problem 21.P.5 (easy) What is the broadcast result shape of `(8, 1, 5)` plus `(3, 1)`?

Answer: Align from the right: (8, 1, 5) with (1, 3, 1). Each pair is equal or has a 1, giving (8, 3, 5).

Problem 21.P.6 (easy) Which of these HTTP errors should a retry loop retry: 400, 401, 429, 503, 404?

Answer: Only 429 (rate limit, honour `retry-after`) and 503 (temporary overload), together with timeouts and connection errors. 400, 401 and 404 describe a problem with the request, key or address; repeating the same request fails the same way.

Problem 21.P.7 (easy) How much memory do the weights alone need for a 1-billion-parameter model in float32, bfloat16 and 4-bit?

Answer: 1e9 x 4 bytes = 4 GB, 1e9 x 2 = 2 GB, and 1e9 x 0.5 = 0.5 GB (decimal gigabytes). Full fine-tuning in fp32 with Adam needs 16 GB for weights, gradients and optimiser state before activations.

Problem 21.P.8 (medium, find the bug) The loss wanders and sometimes explodes. What is wrong?

```
for x, y in loader:
    loss = loss_fn(model(x), y)
    loss.backward()
    opt.step()
```

Answer: `opt.zero_grad()` is missing, so gradients from all previous batches keep adding up and every step uses a growing sum. Add `opt.zero_grad()` before `loss.backward()` (or right after `opt.step()`).

Problem 21.P.9 (medium, find the bug) After training, `model.parameters()` seems to contain only the output layer. Why?

```
class Net(nn.Module):
    def __init__(self):
        super().__init__()
        self.hidden = [nn.Linear(8, 8) for _ in range(3)]
        self.out = nn.Linear(8, 2)
```

Answer: A plain Python list is not registered by `nn.Module`, so the three hidden layers are invisible to `parameters()`, `state_dict()` and `.to(device)`; the optimiser never trains them. Use `nn.ModuleList([...])`.

Problem 21.P.10 (medium, find the bug) Training accuracy plateaus low although the code runs. Find the issue.

```
logits = model(x)
probs = torch.softmax(logits, dim=-1)
loss = nn.CrossEntropyLoss()(probs, y)
```

Answer: The loss applies log-softmax internally, so the model's softmax is applied twice. The squashed probabilities flatten gradients and the loss cannot go to zero. Pass `logits` directly.

Problem 21.P.11 (medium, find the bug) Predictions on 10,000 texts come back misaligned with the input rows.

```
order = sorted(range(len(texts)), key=lambda i: len(texts[i]))
preds = []
for s in range(0, len(order), 64):
    batch = [texts[i] for i in order[s:s + 64]]
    preds.extend(predict_batch(batch))
return preds
```

Answer: The predictions are in length-sorted order but returned as if they were in input order. Write each result back to its original index: `out[order[s + j]] = pred_j`, or invert the permutation at the end as in recipe 5.

Problem 21.P.12 (medium, find the bug) A FAISS search sometimes returns the wrong document text for tiny corpora.

```
D, I = index.search(q, 5)
answers = [docs[i] for i in I[0]]
```

Answer: When fewer than 5 vectors are stored, FAISS fills the missing slots with `-1`, and `docs[-1]` in Python is the last document, which silently appears as a fake hit. Filter first: `[docs[i] for i in I[0] if i != -1]`.

Problem 21.P.13 (medium) Compute the cross-entropy loss and the gradient with respect to the logits for logits z = [1, 0, 0] and true class 0.

Answer: exp(1) = 2.718, exp(0) = 1, so the sum is 4.718 and the probabilities are [0.576, 0.212, 0.212]. Loss = -ln 0.576 = 0.551. Gradient = p - onehot = [-0.424, 0.212, 0.212]. The three gradient entries sum to 0, as they always do for softmax cross-entropy.

Problem 21.P.14 (medium) Adam, first step: gradient g = 0.5, lr = 0.1, beta1 = 0.9, beta2 = 0.999, epsilon negligible. By how much does the parameter move?

Answer: m = 0.1 x 0.5 = 0.05 and v = 0.001 x 0.25 = 0.00025. Bias correction at t = 1 divides by 0.1 and 0.001, giving $\hat{m} = 0.5$ and $\hat{v} = 0.25$, so $\sqrt{\hat{v}} = 0.5$. The step is 0.1 x 0.5 / 0.5 = 0.1. Early Adam steps have size close to lr whatever the scale of the gradient.

Problem 21.P.15 (medium) You want an effective batch of 64 but only 8 examples fit in memory. What do you change, and what must you remember about the loss?

Answer: Accumulate over 8 micro-batches: call `backward()` on each and `opt.step(); opt.zero_grad()` every 8th one. Divide each micro-batch loss by 8 (for mean losses) so the summed gradient equals the mean over 64. The scheduler should count optimiser steps, not micro-batches. Batch-norm statistics still see only 8 examples.

Problem 21.P.16 (medium) A transformer with d = 4096 adapts the query and value projections (each 4096 x 4096) in all 32 layers with LoRA rank r = 8. How many trainable parameters is that?

Answer: One adapted matrix has A (8 x 4096) and B (4096 x 8): 2 x 4096 x 8 = 65,536. Two matrices per layer give 131,072; times 32 layers gives 4,194,304 trainable parameters, about 4.2 million, versus 16.8 million per full matrix.

Problem 21.P.17 (medium) A system returns [1, 5, 2, 9] and the relevant documents are {1, 2}. Compute recall@2, MRR and nDCG@4.

Answer: Top 2 are [1, 5]: one of two relevant found, recall@2 = 0.5. The first relevant item is at rank 1, so MRR = 1. DCG = $1/\log_2 2 + 1/\log_2 4 = 1 + 0.5 = 1.5$ (relevant items at ranks 1 and 3). IDCG = $1 + 1/\log_2 3 = 1.6309$. nDCG = 1.5 / 1.6309 = 0.920.

Problem 21.P.18 (medium) With base 0.5 s, cap 8 s and full jitter, give the maximum wait for attempts 0 to 5 and their sum.

Answer: $\min(8, 0.5 \times 2^n)$: 0.5, 1, 2, 4, 8, 8 (attempt 5 would be 16, capped). The sum is 23.5 s, the worst-case total waiting time; the expected total with jitter is about half of that.

Problem 21.P.19 (medium) A job makes 2,000 LLM calls a day, each with 1,500 input and 300 output tokens, at 2 dollars per million input tokens and 10 dollars per million output tokens (invented prices). Cost per day and per 30 days?

Answer: Per call: 1,500 x 2 / 1e6 = 0.003 and 300 x 10 / 1e6 = 0.003, total 0.006 dollars. Per day: 2,000 x 0.006 = 12 dollars. Per 30 days: 360 dollars. Output tokens cost five times as much per token here, so cutting answer length is as valuable as cutting the prompt.

Problem 21.P.20 (medium, what does this print?)

```
attempts = 0
def flaky():
    global attempts
    attempts += 1
    if attempts < 3:
        raise TimeoutError
    return "ok"

for i in range(5):
    try:
        print(flaky())
        break
    except TimeoutError:
        print("retry", i)
```

Answer: `retry 0`, `retry 1`, then `ok`. The first two calls raise, the third succeeds and `break` ends the loop, so the remaining iterations never run.

Problem 21.P.21 (expert) Derive the gradient of the cross-entropy $L = -z_y + \log \sum_j \exp(z_j)$ with respect to z_k.

Answer: The first term gives -1 when k = y and 0 otherwise. The derivative of log sum $\exp(z_j)$ with respect to z_k is $\exp(z_k) / \sum_j \exp(z_j) = \text{softmax}(z)_k$. Adding: $dL/dz_k = \text{softmax}(z)_k - 1[k = y]$, that is p minus the one-hot vector. This is why backward from the logits needs no extra work, and why the gradient entries sum to 0.

Problem 21.P.22 (expert) Show that merging a LoRA adapter does not change the layer's outputs, and explain why merging helps at serving time.

Answer: With scale s = alpha / r, the adapted layer computes $y = x W^T + s (x A^T) B^T = x (W + s B A)^T$, because $(B A)^T = A^T B^T$ and matrix multiplication is associative. So replacing W by W + s B A gives identical outputs. Merging removes the two extra small matrix products per layer (less latency, simpler deployment). The cost is that the adapter can no longer be swapped without keeping the original W, and merging into a quantised base needs care for rounding.

Problem 21.P.23 (expert) Self-attention scores are materialised in float16 with shape (B, H, T, T) for B = 8, H = 16, T = 4096. How large is that tensor for one layer, and what happens when T doubles?

Answer: Elements: 8 x 16 x 4096 x 4096 = 2,147,483,648. At 2 bytes each that is 4,294,967,296 bytes, about 4.3 GB for one layer's score matrix (more for the softmax output and for backward). Doubling T quadruples it to about 17 GB. This quadratic growth is why fused attention kernels that never write the full matrix, and shorter or chunked contexts, matter.

Problem 21.P.24 (expert, design) You must show that a new prompt for a ticket classifier is better than the current one before shipping. Describe the evaluation.

Answer: (1) Build a golden set of real tickets labelled by people, split into a development set for prompt iteration and a test set touched once; deduplicate before splitting. (2) Choose metrics that match the business: per-class precision and recall (and a cost-weighted error if mistakes differ in cost), valid-output rate, latency and cost per call. (3) Run both prompts at the same settings, with temperature 0 or several samples. (4) Compare per item (paired): count items where only the new prompt is right versus only the old one, and bootstrap a confidence interval of the accuracy difference. (5) Read the errors, not only the number. (6) Ship behind a monitored rollout and keep sampling real traffic into the golden set. If the interval includes zero, say that the evidence is inconclusive and collect more labels.

Problem 21.P.25 (expert, interview) Explain the difference between `view` and `reshape`, and between `detach()` and `torch.no_grad()`.

Answer: `view` never copies and requires compatible strides (contiguous memory); it errors otherwise. `reshape` returns a view when possible and silently copies when not. `detach()` returns a tensor that shares data but is cut from the graph, for a single tensor. `torch.no_grad()` is a context in which no operation is recorded at all, so no graph is built and memory is saved; it does not change `requires_grad` of existing parameters. Use `no_grad` or `inference_mode` for evaluation and `detach` to stop gradients through one value (for example a target or a stored state).

Problem 21.P.26 (expert, what does this print?)

```
import numpy as np
x = np.arange(12.0).reshape(3, 4)
print(x.sum(0).shape, x.sum(1, keepdims=True).shape)
print((x - x.mean(1, keepdims=True)).mean(1))
```

Answer: The first line prints `(4,) (3, 1)`: summing over axis 0 removes it, while `keepdims=True` keeps a length-1 axis. The second line subtracts each row's mean from the row, so every row now has mean zero: it prints `[0. 0. 0.]` (possibly tiny floating-point values such as `-0.`). Without `keepdims=True` the mean has shape (3,), which cannot broadcast against (3, 4) and raises an error (or worse, mis-aligns when the sizes happen to match).

Prompt pack

AI prompt: copy and paste into any AI

```
You are my patient, practical professor of deep learning engineering. I am a working professional moving into data roles and I know basic Python, NumPy and pandas. Teach me PyTorch and the LLM toolchain one idea at a time: tensors and autograd, nn.Module, optimisers, DataLoader, the training loop, device and memory, Hugging Face, embeddings with vector search, and LLM API calls with retries and structured output. For each idea: (1) explain it in plain English with an analogy, (2) show a minimal example and tell me the output before I run it, (3) ask me one prediction question and wait for my answer, (4) correct me kindly and ask a harder follow-up. After five ideas, give me a 10-question quiz mixing what-does-this-print, find-the-bug and shape questions, and grade it. Do not move on until I say I understand.
```

AI prompt: copy and paste into any AI

```
Act as a senior ML engineer doing a code review. I will paste a PyTorch or LLM-API script. Review it in this order: (1) correctness: shapes, dtypes, devices, missing zero_grad, eval and no_grad, loss inputs (logits versus probabilities), data leakage; (2) memory and speed: batch size, padding, num_workers, mixed precision, storing tensors with graphs; (3) LLM calls: retries only for 429, 5xx and timeouts, timeouts set, streaming, reply validation, secrets in code, cost per call; (4) evaluation: is there a held-out set, a baseline and a confidence interval? For every problem give the exact line, why it matters, the symptom I would see, and a fixed version. End with the three most important changes. Ask me for the missing information (library versions, hardware, data size) before guessing.
```

AI prompt: copy and paste into any AI

```
Run a mock interview for a machine learning engineer role focused on deep learning and GenAI libraries. Ask me one question at a time and wait for my answer. Mix: explain autograd and why gradients accumulate; estimate the memory to fine-tune a 7B-parameter model and name ways to reduce it; explain LoRA with parameter counts; debug a shape error from an error message I will paste; design a retrieval-augmented generation system with chunking, embeddings, an index, metrics and failure modes; make an LLM call reliable and cheap at one million requests a day; show that a new prompt beats the old one. After each answer, score me from 1 to 5, point out what I missed, and give a model answer in under 120 words. After eight questions, summarise my strengths and the three topics I should revise, and propose a one-week study plan.
```

CHAPTER 22

SCALING UP: POLARS, DUCKDB, DASK AND PYSPARK

Library Handbook: when pandas is not enough and how to choose the next tool.

Priya works at an online shop. On Monday the data team drops a file called `events.csv` into a shared folder: every click, cart and purchase from the last year. It is 10 GB. Her laptop has 16 GB of memory. She types `pd.read_csv("events.csv")`, the fan spins up, the notebook freezes for four minutes, and then the kernel dies with no message. She did nothing wrong. Pandas is a superb tool, but it was designed to hold a table in memory, and a table can be bigger than memory.

This chapter is the map of what to do next. You will learn to measure how big a problem really is, shrink it with cheap pandas tricks, read it in chunks, and then move to the right bigger tool: **Parquet** files (a compact column-based file format), **Polars** (a very fast DataFrame library), **DuckDB** (a database that runs SQL directly on files), **Dask** (pandas split across many pieces) and **PySpark** (the cluster engine used in many companies). By the end you will be able to choose between them on purpose, write the same task in each, read a Spark UI, and fix the usual disasters: out-of-memory kills, skewed joins and slow Python functions.

How the code in this chapter is labelled. Polars, DuckDB, Dask, PySpark and PyArrow are **not installed** on the study machine, so their code is marked **NOT RUN HERE**. It is written to be correct and current, and every such example solves the same small task as a block that **was run** with pandas and with `sqlite3` (which ships with Python), so you can verify the expected result by eye. Everything else was run with Python 3.14, pandas 3.0 and NumPy 2.x, and the output shown is the real output. Timings are measured on one laptop and will vary on yours; trust the ratios and the shapes, not the exact seconds.

22.1 What these libraries are, and when to leave pandas

Beginner: the problem in one picture

A **DataFrame** is a table: rows are records, columns are fields. Pandas keeps the whole table in RAM (the fast working memory of your computer) and works on one CPU core for most operations. That gives two limits. **Memory**: if the table needs more RAM than you have, the program crashes or starts swapping to disk and crawls. **Speed**: if a job takes an hour on one core while your laptop has eight, seven cores sit idle.

The tools in this chapter attack those two limits with three ideas. Learn the ideas once and every library becomes easier.

- **Columnar storage.** Store each column together, not each row. A question about one column reads only that column (figure below).
- **Lazy execution.** Do not run each line immediately. Collect the whole plan, let an optimizer clean it up, then run once.
- **Partitioning.** Cut the data into pieces that can be processed independently (in parallel, or one at a time), then combine the partial answers.

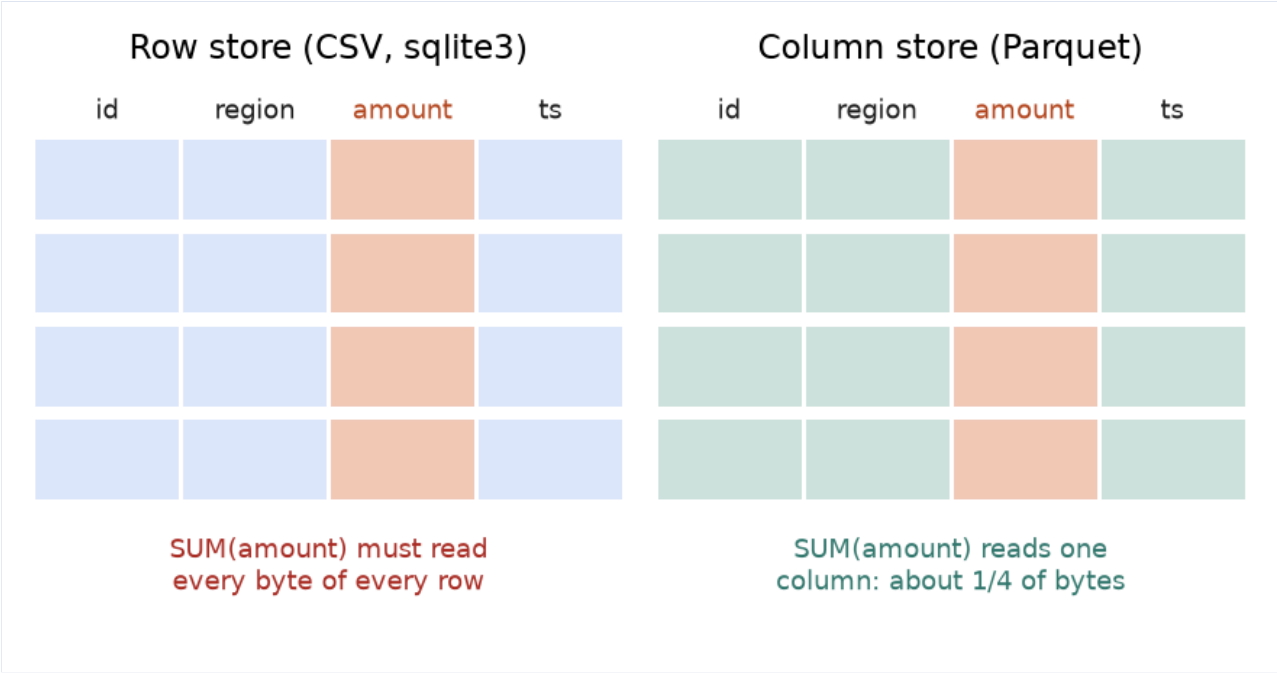


Figure 22.1. Row storage versus column storage. The highlighted column is the only one the query needs.

How to read it: each small square is one cell. In the row store (a CSV file or a `sqlite3` table) the cells of a row sit next to each other on disk, so adding up `amount` forces the reader to walk past every other cell. In the column store the cells of a column sit together, so only the orange column is read. With 4 columns that saves about three quarters of the reading; with 100 columns it saves 99 percent. This is the main reason Parquet, Polars and DuckDB are fast on analytics.

The five libraries at a glance

Library	What it is	Use it when	Do NOT use it when
pandas	In-memory DataFrame, one core mostly	Data fits in RAM comfortably (a good rule: the in-memory size is under a third of your RAM)	The file will not load, or one step takes hours
Polars	Rust DataFrame library, multi-core, eager and lazy modes	One machine, data from a few GB to larger than RAM (lazy/streaming), you like method chains	You need the exact pandas API, or a cluster
DuckDB	In-process analytical SQL database	You think in SQL, you query Parquet/CSV files directly, you want window functions and joins on big files	Many small writes per second (that is OLTP, use PostgreSQL or SQLite)
Dask	Parallel pandas: many pandas pieces plus a scheduler	You have pandas code and need more cores or more than RAM, maybe a small cluster	Data is small (the overhead is larger than the gain), or you can use Polars/DuckDB on one machine
PySpark	Python API for Apache Spark, a distributed engine	Data is many hundreds of GB or TB, or your company already runs Spark (Databricks, EMR, Dataproc, Synapse)	Data fits on one machine: Spark startup and shuffle costs will make it slower

Install, version check and import conventions

Install with pip into a virtual environment (see Chapter 9 for environments). PySpark also needs a Java runtime installed on the machine, because Spark itself runs on the JVM.

```
pip install pyarrow polars duckdb "dask[dataframe]" pyspark
java -version        # PySpark needs a Java runtime; install a JDK if this fails
```

This block checks which of the libraries exist on the current machine and prints the version of each one that does. It is the first thing to run when someone says "it works on my laptop".

```
import importlib.util
from importlib.metadata import version, PackageNotFoundError

for name in ["pandas", "numpy", "pyarrow", "polars", "duckdb", "dask", "pyspark"]:
    if importlib.util.find_spec(name) is None:
        print(f"{name:8s} not installed")
    else:
        try:
            print(f"{name:8s} {version(name)}")
        except PackageNotFoundError:
            print(f"{name:8s} present")
import sqlite3
print("sqlite3  ", sqlite3.sqlite_version)
```

```
pandas   3.0.6
numpy    2.5.3
pyarrow  not installed
polars   not installed
duckdb   not installed
dask     not installed
pyspark  not installed
sqlite3   3.50.4
```

1. `find_spec` asks Python whether an import would succeed, without importing (a slow import of Spark is avoided).
2. For installed packages, `importlib.metadata.version` returns the version string from the package metadata.
3. The last two lines show the version of the SQLite engine inside Python. It matters: window functions need SQLite 3.25 or newer.
4. On the study machine the output above shows pandas and NumPy present, and PyArrow, Polars, DuckDB, Dask and PySpark missing. That is why their code is labelled NOT RUN HERE.

The import conventions you will see everywhere:

```
# NOT RUN HERE (libraries not installed)
import pandas as pd
import polars as pl
import duckdb
import dask.dataframe as dd
import pyarrow as pa
import pyarrow.parquet as pq
from pyspark.sql import SparkSession, functions as F, Window
```

KEY IDEA	There is no best library, only the smallest tool that finishes in time. Each step up (pandas, Polars/DuckDB, Dask, Spark) buys capacity and costs simplicity: more setup, less debugging comfort, more concepts (partitions, shuffles). Cheap fixes first: read fewer columns, use smaller dtypes, read in chunks.

The running task used in this chapter

To compare libraries fairly, we solve one small task in all of them. Ten orders, six customers. **For each region, rank customers by their total revenue (rank 1 is the biggest), and attach each customer's loyalty tier.** That needs a group-by, a join and a window function (rank within a group), which are the three operations that decide whether a tool is any good. First in pandas.

Problem 22.1.1 Using the ten orders and the customer tiers below, produce one row per (region, customer) with the customer's revenue, tier and rank within the region. Compute it with pandas.

```
import io
import pandas as pd

orders = pd.read_csv(io.StringIO("""order_id,customer_id,region,category,amount
1,c1,EU,book,20
2,c2,EU,toy,35
3,c1,EU,toy,15
4,c3,US,book,50
5,c4,US,toy,10
6,c3,US,toy,25
7,c5,APAC,book,40
8,c5,APAC,book,30
9,c6,APAC,toy,5
10,c2,EU,book,45"""))
tiers = pd.DataFrame({"customer_id": ["c1", "c2", "c3", "c4", "c5", "c6"],
                      "tier": ["gold", "silver", "gold", "bronze", "silver", "bronze"]})

rev = orders.groupby(["region", "customer_id"], as_index=False)["amount"].sum()
rev = rev.rename(columns={"amount": "revenue"})
rev = rev.merge(tiers, on="customer_id", how="left")
rev["rank"] = rev.groupby("region")["revenue"].rank(method="first", ascending=False).astype(int)
print(rev.sort_values(["region", "rank"]).to_string(index=False))
```

```
region customer_id  revenue   tier  rank
  APAC          c5       70 silver     1
  APAC          c6        5 bronze     2
    EU          c2       80 silver     1
    EU          c1       35   gold     2
    US          c3       75   gold     1
    US          c4       10 bronze     2
```

1. `read_csv(io.StringIO(...))` reads CSV text from a string as if it were a file. This keeps the block standalone.
2. `groupby([...], as_index=False)["amount"].sum()` adds up the amount per (region, customer) pair. `as_index=False` keeps the keys as normal columns.
3. `rename` gives the new column an honest name, `revenue`.
4. `merge(tiers, on="customer_id", how="left")` is a left join: every revenue row is kept and the tier is attached.
5. `groupby("region")["revenue"].rank(...)` is the window function: it ranks within each region. `method="first"` breaks ties by order of appearance and `ascending=False` makes the largest revenue rank 1.
6. The last line sorts for a stable, readable print. Check by hand: EU c2 = 35 + 45 = 80, c1 = 20 + 15 = 35. US c3 = 75, c4 = 10. APAC c5 = 70, c6 = 5.

Keep these six rows in mind. Every other library in the chapter must give the same six rows, and the next block proves the SQL version does.

```
import io, sqlite3
import pandas as pd

csv = """order_id,customer_id,region,category,amount
1,c1,EU,book,20
2,c2,EU,toy,35
3,c1,EU,toy,15
4,c3,US,book,50
5,c4,US,toy,10
6,c3,US,toy,25
7,c5,APAC,book,40
8,c5,APAC,book,30
9,c6,APAC,toy,5
10,c2,EU,book,45"""
con = sqlite3.connect(":memory:")
pd.read_csv(io.StringIO(csv)).to_sql("orders", con, index=False)
con.execute("CREATE TABLE tiers(customer_id TEXT, tier TEXT)")
con.executemany("INSERT INTO tiers VALUES (?, ?)", [("c1","gold"),("c2","silver"),("c3","gold"),("c4","bronze"),
("c5","silver"),("c6","bronze")])

sql = """
WITH rev AS (
    SELECT region, customer_id, SUM(amount) AS revenue
    FROM orders GROUP BY region, customer_id
)
SELECT r.region, r.customer_id, r.revenue, t.tier,
       ROW_NUMBER() OVER (PARTITION BY r.region ORDER BY r.revenue DESC) AS rnk
FROM rev r LEFT JOIN tiers t ON t.customer_id = r.customer_id
ORDER BY r.region, rnk
"""
for row in con.execute(sql):
    print(row)
```

```
('APAC', 'c5', 70, 'silver', 1)
('APAC', 'c6', 5, 'bronze', 2)
('EU', 'c2', 80, 'silver', 1)
('EU', 'c1', 35, 'gold', 2)
('US', 'c3', 75, 'gold', 1)
('US', 'c4', 10, 'bronze', 2)
```

1. An in-memory SQLite database is created; the orders table is loaded from the same ten rows with `to_sql`.
2. A small `tiers` table is created and filled with `executemany` (placeholders `?` keep values out of the SQL text).
3. The CTE (`WITH rev AS (...)`) is the group-by step: one row per region and customer.
4. `LEFT JOIN tiers` attaches the tier, same as `merge(..., how="left")`.
5. `ROW_NUMBER() OVER (PARTITION BY region ORDER BY revenue DESC)` is the window function: number the rows inside each region, biggest revenue first.
6. The tuples printed match the pandas table cell for cell (pandas prints a table, SQLite prints tuples; the values are the same). **DuckDB accepts this exact SQL text**, which is why the DuckDB section can reuse it.

REAL JOB	When you move a pipeline to a bigger tool, never trust it on faith. Keep a tiny sample dataset with a known answer (like these ten rows) and a test that compares the new output with the old output. Most migration bugs (different tie-breaking, null handling, integer overflow) show up on ten rows.

22.2 Before you scale: measure, shrink and chunk in pandas

The cheapest speed-up is the one that needs no new library. Most "big data" problems at the 1 to 20 GB scale are really "wasteful data" problems: columns you never use, 64-bit numbers holding values below 100, and the word `"EU"` stored a million times. This section gives you the measuring tools and the three fixes: read less, store smaller, read in pieces.

Reference table: the 24 most useful pandas scaling tools

Name	What it does in plain English	Minimal example
`df.info(memory_usage="deep")`	Prints dtypes and the true memory including string contents	`df.info(memory_usage="deep")`
`df.memory_usage(deep=True)`	Bytes used per column	`df.memory_usage(deep=True).sum()`
`pd.read_csv(usecols=)`	Read only the columns you need	`pd.read_csv(p, usecols=["a","b"])`
`pd.read_csv(dtype=)`	Choose dtypes at read time, skip the guessing	`dtype={"qty": "uint8"}`
`pd.read_csv(chunksize=)`	Iterator of DataFrames of N rows each	`for ch in pd.read_csv(p, chunksize=10**5):`
`pd.read_csv(nrows=)`	Read only the first N rows (peek at a huge file)	`pd.read_csv(p, nrows=1000)`
`pd.read_csv(skiprows=)`	Skip rows (also a callable to sample at random)	`skiprows=lambda i: i > 0 and random() > .01`
`pd.read_csv(parse_dates=)`	Turn text dates into real datetimes while reading	`parse_dates=["ts"]`
`pd.to_numeric(downcast=)`	Shrink a number column to the smallest dtype that fits	`pd.to_numeric(s, downcast="unsigned")`
`astype("category")`	Store repeated strings once, plus small integer codes	`df["region"].astype("category")`
`astype("float32")`	Halve a float column (loses precision)	`df["x"].astype("float32")`
`astype("uint8")` etc.	Integer dtypes: int8/16/32/64, uint8/16/32/64	`df["qty"].astype("uint8")`
`pd.Series.nunique()`	Counts distinct values; low counts suggest category	`df["region"].nunique()`
`df.select_dtypes()`	Pick columns by dtype, for bulk conversion	`df.select_dtypes("int64").columns`
`df.convert_dtypes()`	Switch to nullable dtypes (Int64, boolean)	`df.convert_dtypes()`
`groupby(observed=True)`	Group on a category without empty combinations (the default since pandas 3.0; older versions needed it)	`df.groupby("region", observed=True)`
`Series.add(other, fill_value=0)`	Add two partial results, treating a missing key as 0	`total.add(part, fill_value=0)`
`pd.concat(list)`	Glue many small frames once (not in a loop)	`pd.concat(parts, ignore_index=True)`
`df.to_csv(mode="a", header=False)`	Append a chunk to a file	`ch.to_csv(out, mode="a", header=first)`
`pd.read_sql(chunksize=)`	Stream a database query in chunks	`pd.read_sql(q, con, chunksize=50_000)`

Name	What it does in plain English	Minimal example
`df.sample(frac=)`	Random sample to develop on a small slice	`df.sample(frac=0.01, random_state=0)`
`np.load(mmap_mode="r")`	Memory-map a .npy file: arrays on disk behave like arrays in RAM	`np.load("a.npy", mmap_mode="r")`
`tracemalloc`	Standard-library tool to measure peak Python memory	`tracemalloc.start(); ...; tracemalloc.get_traced_memory()`
`time.perf_counter()`	Precise clock for timing code	`t0 = time.perf_counter()`
`del df; gc.collect()`	Drop big objects you no longer need	`del raw`

Beginner: how big is a column really?

An integer in pandas defaults to 64 bits, which is 8 bytes, even if the value is 7. A column of 1 million integers therefore takes 8 MB. A string column is worse: each Python string object costs dozens of bytes beyond its characters. A **category** column keeps each distinct value once and stores a tiny integer code per row. The next block builds a synthetic table of 1 million orders and measures every column before and after shrinking. This synthetic table is reused in the next blocks.

Problem 22.2.1 Build a 1-million-row orders table (order_id, customer_id, region, qty, price). Measure the memory of each column, then shrink it with downcasting, `category` and `float32`, and compare.

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(42)
n = 1_000_000
df = pd.DataFrame({
    "order_id": np.arange(n),
    "customer_id": rng.integers(0, 50_000, n),
    "region": rng.choice(["EU", "US", "APAC", "LATAM", "MEA"], n),
    "qty": rng.integers(1, 20, n),
    "price": rng.normal(50, 15, n).round(2),
})
mb = lambda s: round(s.memory_usage(deep=True, index=False) / 1e6, 1)
print("BEFORE")
for c in df:
    print(f"  {c:12s} {str(df[c].dtype):6s} {mb(df[c]):6.1f} MB")
print("  total", round(df.memory_usage(deep=True).sum() / 1e6, 1), "MB")

small = df.assign(
    order_id=pd.to_numeric(df.order_id, downcast="unsigned"),
    customer_id=pd.to_numeric(df.customer_id, downcast="unsigned"),
    region=df.region.astype("category"),
    qty=pd.to_numeric(df.qty, downcast="unsigned"),
    price=df.price.astype("float32"),
)
print("AFTER")
for c in small:
    print(f"  {c:12s} {str(small[c].dtype):8s} {mb(small[c]):5.1f} MB")
print("  total", round(small.memory_usage(deep=True).sum() / 1e6, 1), "MB")
```

```
BEFORE
  order_id     int64     8.0 MB
  customer_id  int64     8.0 MB
  region       str      52.2 MB
  qty          int64     8.0 MB
  price        float64    8.0 MB
  total 84.2 MB
AFTER
  order_id     uint32     4.0 MB
  customer_id  uint16     2.0 MB
  region       category   1.0 MB
  qty          uint8      1.0 MB
  price        float32    4.0 MB
  total 12.0 MB
```

1. Lines 4 to 12 build the table with a seeded random generator, so every run gives the same numbers.
2. `mb` is a one-line helper: `deep=True` counts the real size of string contents, not just pointers.
3. BEFORE: the four numeric columns cost 8 MB each (1 million values times 8 bytes). The region column (the text `EU`, `US` and so on) costs over 50 MB, more than the other four together. In pandas 3.0 text columns default to the `str` dtype; without PyArrow installed it is still backed by Python string objects, which is why it is large. With PyArrow installed the text is stored more compactly, but a category is smaller still for few distinct values.
4. `pd.to_numeric(..., downcast="unsigned")` picks the smallest unsigned integer dtype that holds every value: 1,000,000 ids need 32 bits, customers below 50,000 need 16, quantities below 20 need 8.
5. `astype("category")` stores 5 labels and one 8-bit code per row: 1 MB.
6. `float32` halves the price column. It keeps about 7 significant digits, fine for a price of 49.99 in analytics, wrong for money you must add up exactly (see pitfalls).
7. Result: about 84 MB becomes about 12 MB, seven times smaller, with the same information. Check the range before you force a type: `to_numeric(downcast=)` picks a dtype from the actual min and max, but a blind `astype("uint8")` on a value of 300 silently wraps it to 44, and writing 300 into an existing `uint8` column raises an error.

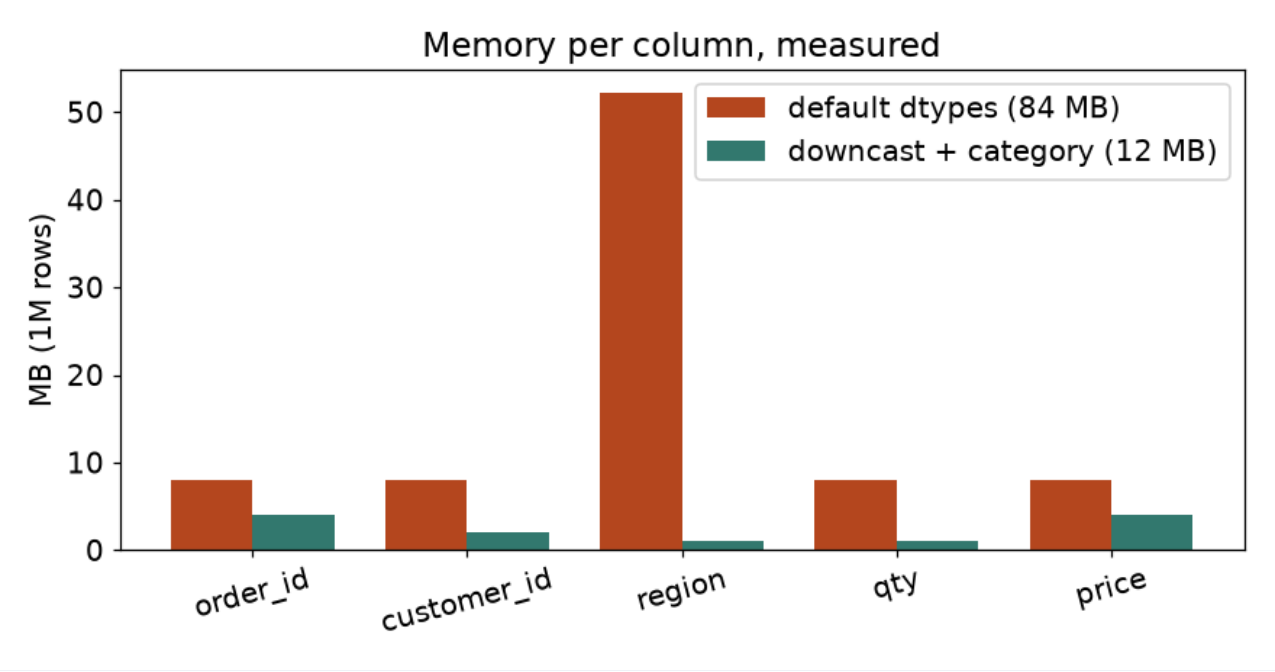


Figure 22.2. Memory per column for the 1-million-row table, measured with `memory_usage(deep=True)`.

How to read it: each pair of bars is one column. The orange bar is the default dtype, the green bar the shrunk version. The tall orange bar is the text column; the other columns shrink by half or more. The lesson: look at the biggest bar first. Shrinking an already small column barely helps.

Intermediate: chunking, the way to beat the memory wall

If you cannot fit the file, do not load the file. A **chunk** is a block of rows. You read one chunk, reduce it to a small partial answer (a sum, a count, a filtered set of rows), throw the chunk away, and move to the next. At the end you combine the partial answers. This works for any **decomposable** statistic: sums, counts, min, max, and the mean (as sum divided by count). It does not work directly for the median or the exact distinct count, which need other tricks (Section 22.9).

Problem 22.2.2 Write the 1-million-row table to a CSV file. Compute the revenue (qty times price) per region two ways: reading the whole file, and in chunks of 100,000 and 10,000 rows. Compare the peak memory and check that the answers agree.

```
import os, tempfile, tracemalloc
import numpy as np
import pandas as pd

rng = np.random.default_rng(42)
n = 1_000_000
df = pd.DataFrame({
    "order_id": np.arange(n),
    "customer_id": rng.integers(0, 50_000, n),
    "region": rng.choice(["EU", "US", "APAC", "LATAM", "MEA"], n),
    "qty": rng.integers(1, 20, n),
    "price": rng.normal(50, 15, n).round(2),
})
path = os.path.join(tempfile.mkdtemp(), "orders.csv")
df.to_csv(path, index=False)
del df
print("file size over 20 MB:", os.path.getsize(path) > 20e6)

def revenue_full():
    d = pd.read_csv(path)
    return (d.qty * d.price).groupby(d.region).sum()

def revenue_chunked(size):
    total = None
    for ch in pd.read_csv(path, chunksize=size):
        part = (ch.qty * ch.price).groupby(ch.region).sum()
        total = part if total is None else total.add(part, fill_value=0)
    return total

peaks, answers = {}, {}
for label, fn in [("full", revenue_full),
                  ("chunk 100k", lambda: revenue_chunked(100_000)),
                  ("chunk 10k", lambda: revenue_chunked(10_000))]:
    tracemalloc.start()
    answers[label] = fn().sort_index()
    peaks[label] = tracemalloc.get_traced_memory()[1]
    tracemalloc.stop()

print("full peak above 60 MB:", peaks["full"] > 60e6)
print("chunk 100k peak below 30 MB:", peaks["chunk 100k"] < 30e6)
print("chunk 10k peak below 10 MB:", peaks["chunk 10k"] < 10e6)
print("same answer:", np.allclose(answers["full"], answers["chunk 10k"]))
print(answers["chunk 10k"].round(0).astype("int64").to_string())
```

```
file size over 20 MB: True
full peak above 60 MB: True
chunk 100k peak below 30 MB: True
chunk 10k peak below 10 MB: True
same answer: True
region
APAC     100744849
EU        99851750
LATAM     99823225
MEA       99991196
US        99891792
```

1. The first half is the same data generator, written to a temporary CSV. `del df` frees the in-memory copy, so only the file remains, like Priya's situation.
2. `revenue_full` loads everything, multiplies `qty * price` row by row, and groups the products by the region column.
3. `revenue_chunked` asks for an iterator: `read_csv(chunksize=size)` yields DataFrames of at most `size` rows. Each one is reduced to a 5-row `Series` of partial sums.
4. `total.add(part, fill_value=0)` adds two Series aligned on the region label. `fill_value=0` treats a region missing from one chunk as zero instead of producing NaN.
5. `tracemalloc` measures the peak Python-level memory while the function runs. The prints use thresholds, not exact bytes, because exact numbers differ a little between machines.
6. The peak shrinks roughly in proportion to the chunk size, while the answer is identical (up to floating-point rounding, hence `allclose`). The last line prints the revenue per region as whole numbers.

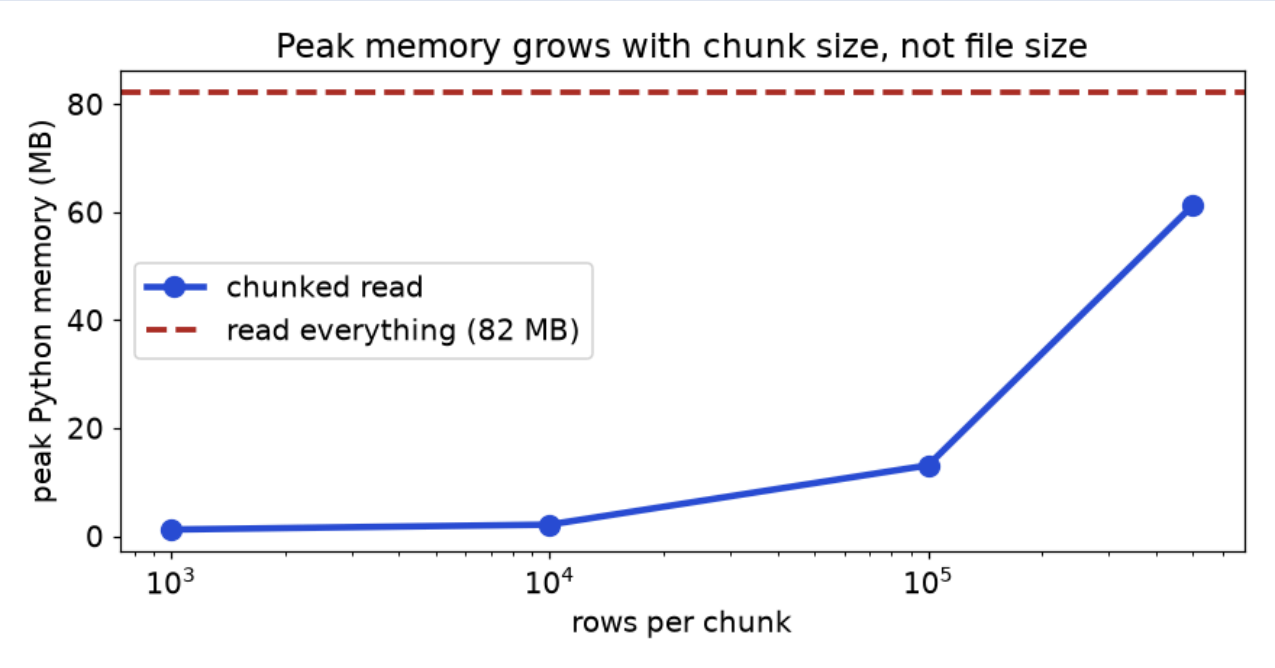


Figure 22.3. Peak memory against chunk size for the same file, measured with `tracemalloc`.

How to read it: the x-axis is the number of rows per chunk (log scale), the y-axis the peak memory. The blue line climbs with chunk size and the red dashed line is the cost of loading everything. If the file were 100 times larger, the blue points would not move at all, because the chunk size sets the memory, not the file size. That is what lets a laptop process a 10 GB file.

Expert: speed numbers, honestly measured

Smaller data is also faster data, but not every trick helps every time. The next block measures reading and group-by variants on the same million rows. Timings are not pasted as checked output because they change on every run; the table after it shows what one run on one laptop gave.

```
import os, tempfile, time
import numpy as np
import pandas as pd

rng = np.random.default_rng(42)
n = 1_000_000
df = pd.DataFrame({
    "order_id": np.arange(n),
    "customer_id": rng.integers(0, 50_000, n),
    "region": rng.choice(["EU", "US", "APAC", "LATAM", "MEA"], n),
    "qty": rng.integers(1, 20, n),
    "price": rng.normal(50, 15, n).round(2),
})
path = os.path.join(tempfile.mkdtemp(), "orders.csv")
df.to_csv(path, index=False)

def best(fn, reps=5):
    times = []
    for _ in range(reps):
        t0 = time.perf_counter(); fn(); times.append(time.perf_counter() - t0)
    return min(times)

print("--- reading the CSV (best of 5, seconds) ---")
print("read everything       ", round(best(lambda: pd.read_csv(path)), 3))
print("usecols 2 of 5        ", round(best(lambda: pd.read_csv(path, usecols=["region", "price"])), 3))
print("usecols + dtype       ", round(best(lambda: pd.read_csv(path, usecols=["region", "price"], dtype={"region":
"category", "price": "float32"})), 3))
print("chunks of 100k, sum   ", round(best(lambda: sum(c.price.sum() for c in pd.read_csv(path, chunksize=100_000))), 3))

df_cat = df.assign(region=df.region.astype("category"))
codes = df_cat.region.cat.codes.to_numpy()
price = df.price.to_numpy()
region = df.region.tolist()
def py_loop():
    tot = {}
    for r, p in zip(region, price):
        tot[r] = tot.get(r, 0.0) + p
    return tot
print("--- mean price per region (best of 5, seconds) ---")
print("python dict loop      ", round(best(py_loop), 3))
print("groupby str column    ", round(best(lambda: df.groupby("region")["price"].mean()), 3))
print("groupby category      ", round(best(lambda: df_cat.groupby("region", observed=True)["price"].mean()), 3))
print("numpy bincount codes  ", round(best(lambda: np.bincount(codes, weights=price) / np.bincount(codes)), 3))
```

Operation (1M rows)	Best of 5 (s), one laptop	Reading
read_csv, everything	0.164	baseline
read_csv, usecols (2 of 5 columns)	0.105	fewer columns, less parsing
usecols + category/float32 dtypes	0.095	a bit faster still, and far smaller in memory
chunks of 100k, summing price	0.181	slightly slower than one read: chunking buys memory, not speed
mean price per region, Python dict loop	0.087	interpreted loop, one row at a time
groupby on a str column	0.036	vectorized, about 2.4 times faster than the loop
groupby on a category column	0.007	integer codes, about 5 times faster than the str groupby
NumPy bincount on the codes	0.004	the floor: pure integer arithmetic

1. The timing helper runs each variant five times and keeps the minimum, which is the least noisy estimate of the cost of the code itself.
2. Reading fewer columns saved about a third here. On a 100-column file the saving is far larger because parsing cost is per column.
3. Chunking is not free: the same work done in 10 pieces carries a little overhead. You chunk because you must, not to go faster.
4. Group-by on a category is much faster than on strings because comparing small integers is cheaper than comparing text. This is the same reason databases dictionary-encode strings.
5. Your numbers will differ, and the ordering is what you should expect to reproduce. The ratios shown are computed from this single run.

COMMON MISTAKE

Wrong: `df["price"] = df["price"].astype("float32")` on money columns, then summing a billion of them. **Symptom:** totals differ from the finance system by a few units, and differ again if you change the row order. **Fix:** store money as integer cents (`int64` , or `int32` if the range allows) and divide by 100 only for display. Use float32 only for measurements where 7 digits are plenty.

22.3 Parquet and columnar file formats

Beginner: why CSV is the wrong format for big data

A CSV file is plain text. That is friendly (you can open it anywhere) and wasteful. The number 49.99 is stored as five characters, not one 8-byte float. Nothing says whether a column is an integer or a date, so the reader must guess. You cannot read one column without scanning every line. And a CSV cannot be split safely in the middle of a quoted multi-line field. **Parquet** is an open file format built for analytics. It stores data by column, with a schema (names and types), compression, and small statistics (minimum and maximum) for each block. The same data is typically many times smaller than the CSV, and a query that needs two columns reads two columns.

DEFINITION

Row group: a block of rows (for example 100,000 to 1,000,000) inside a Parquet file. **Column chunk**: the data of one column inside one row group. **Footer**: metadata at the end of the file with the schema and, for each column chunk, its location, row count and minimum/maximum. A reader jumps to the footer first, decides which chunks it needs, and reads only those. **Apache Arrow** is the matching in-memory columnar format that Polars, DuckDB, PyArrow, and (optionally) pandas use, so data moves between them with little copying.

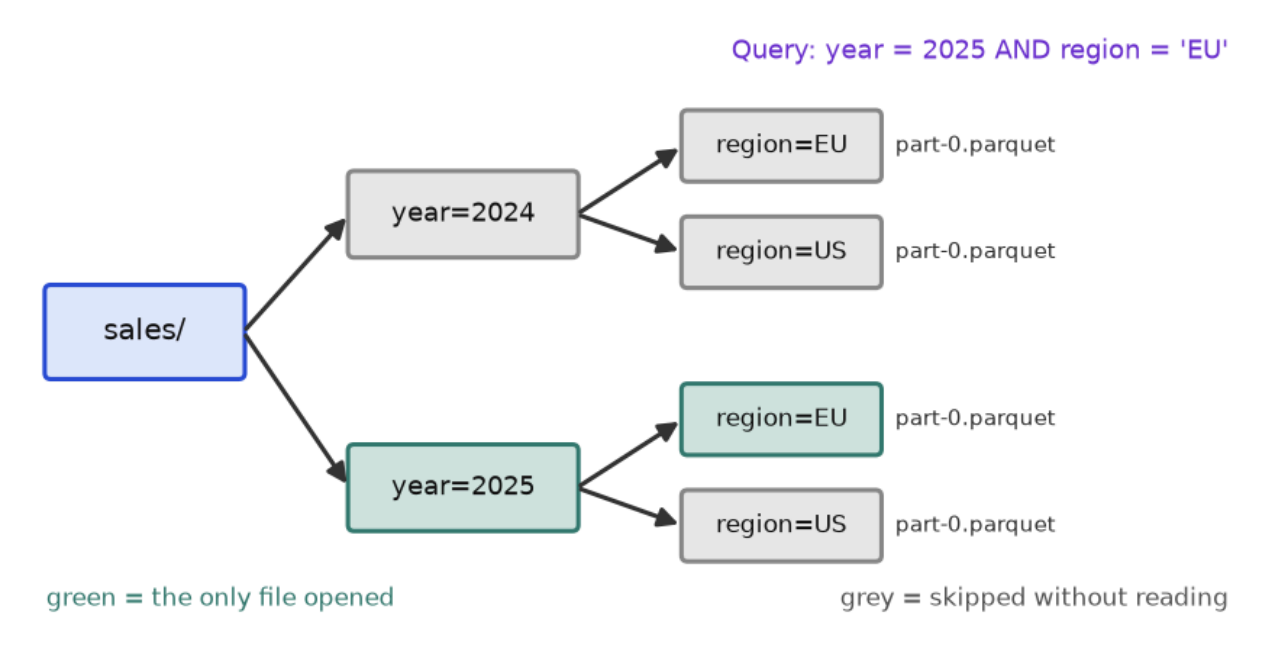


Figure 22.4. A Hive-style partitioned dataset. The folder names carry the values of the partition columns, so a filter on them skips whole folders.

How to read it: the dataset is a folder, not a file. Each level of sub-folders is named `column=value` . The query asks for year 2025 and region EU, so the engine lists the folders, discards every branch whose name fails the filter (grey) and opens one file (green). Nothing was read from the other three

files. This is called **partition pruning**. Inside the one file we additionally get **row-group skipping** from the footer statistics.

Reference table: the 24 most useful Parquet-related calls

Name	What it does in plain English	Minimal example (not run here)
`df.to_parquet(path)`	pandas: write a DataFrame (needs pyarrow or fastparquet)	`df.to_parquet("o.parquet", index=False)`
`pd.read_parquet(path, columns=)`	pandas: read, optionally only some columns	`pd.read_parquet("o.parquet", columns=["region"])`
`pd.read_parquet(filters=)`	pandas: push a filter down to the reader	`filters=[("year", "==", 2025)]`
`df.to_parquet(partition_cols=)`	pandas: write a partitioned folder	`df.to_parquet("sales/", partition_cols=["year"])`
`df.to_parquet(compression=)`	Choose snappy (fast), zstd (small and fast) or gzip (small, slow)	`compression="zstd"`
`pa.Table.from_pandas(df)`	pyarrow: convert pandas to an Arrow table	`t = pa.Table.from_pandas(df)`
`pq.write_table(t, path)`	pyarrow: write an Arrow table	`pq.write_table(t, "o.parquet", row_group_size=500_000)`
`pq.read_table(path, columns=)`	pyarrow: read to an Arrow table	`pq.read_table("o.parquet", columns=["qty"])`
`pq.ParquetFile(path).metadata`	Inspect rows, row groups and column statistics without reading data	`pq.ParquetFile("o.parquet").metadata.num_row_groups`
`pq.ParquetFile(path).schema_arrow`	Show column names and types	`print(pf.schema_arrow)`
`pq.ParquetFile(path).iter_batches()`	Stream a big file in batches (chunking for Parquet)	`for b in pf.iter_batches(batch_size=100_000):`
`pyarrow.dataset.dataset(path)`	Open a folder of files as one lazy dataset	`ds.dataset("sales/", format="parquet", partitioning="hive")`
`dataset.to_table(filter=, columns=)`	Read only what the filter and columns need	`d.to_table(filter=ds.field("year") == 2025)`
`pl.read_parquet(path)`	Polars: read eagerly	`pl.read_parquet("o.parquet")`

Name	What it does in plain English	Minimal example (not run here)
`pl.scan_parquet(path)`	Polars: lazy scan with pushdown; globs allowed	`pl.scan_parquet("sales/**/*.parquet")`
`df.write_parquet(path)`	Polars: write a DataFrame	`df.write_parquet("o.parquet")`
`lf.sink_parquet(path)`	Polars: run a lazy query and stream the result to disk without holding it	`lf.sink_parquet("out.parquet")`
`read_parquet('glob')` (DuckDB SQL)	Query files as a table	`SELECT * FROM read_parquet('sales/*/*.parquet')`
`COPY ... TO ... (FORMAT PARQUET)`	DuckDB: write a query result as Parquet	`COPY (SELECT ...) TO 'out.parquet' (FORMAT PARQUET)`
`PARTITION_BY` (DuckDB)	DuckDB: write a Hive-partitioned folder	`COPY t TO 'sales' (FORMAT PARQUET, PARTITION_BY (year))`
`hive_partitioning = true`	DuckDB: turn folder names into columns on read	`read_parquet('sales/*/*.parquet', hive_partitioning = true)`
`dd.read_parquet(path)`	Dask: lazy read, one partition per file or row-group chunk	`dd.read_parquet("sales/")`
`ddf.to_parquet(path, partition_on=)`	Dask: write partitioned output	`ddf.to_parquet("out/", partition_on=["year"])`
`spark.read.parquet(path)`	Spark: read a file or folder (schema comes from the files)	`spark.read.parquet("sales/")`
`df.write.partitionBy().parquet()`	Spark: write a partitioned folder	`df.write.partitionBy("year").mode("overwrite").parquet("out/")`

Intermediate: see the columnar idea with tools you have

We cannot write Parquet here (no PyArrow), but we can show the principle with NumPy. Store each numeric column as its own binary `.npy` file (typed, no parsing, one file per column), and compare with the CSV.

Problem 22.3.1 Write the 1-million-row table as one CSV and as one binary file per column. Then compute the total of the price column from each, and report how many bytes each approach had to touch.

```
import os, tempfile
import numpy as np
import pandas as pd

rng = np.random.default_rng(42)
n = 1_000_000
df = pd.DataFrame({
    "order_id": np.arange(n, dtype="uint32"),
    "customer_id": rng.integers(0, 50_000, n).astype("uint16"),
    "region": pd.Categorical(rng.choice(["EU", "US", "APAC", "LATAM", "MEA"], n)),
    "qty": rng.integers(1, 20, n).astype("uint8"),
    "price": rng.normal(50, 15, n).round(2),
})
d = tempfile.mkdtemp()
csv_path = os.path.join(d, "orders.csv")
df.to_csv(csv_path, index=False)
for c in ["order_id", "customer_id", "qty", "price"]:
    np.save(os.path.join(d, c + ".npy"), df[c].to_numpy())
np.save(os.path.join(d, "region_codes.npy"), df["region"].cat.codes.to_numpy())

size = lambda p: os.path.getsize(p)
col_files = [f for f in os.listdir(d) if f.endswith(".npy")]
print("csv MB          :", round(size(csv_path) / 1e6, 1))
print("all columns MB  :", round(sum(size(os.path.join(d, f)) for f in col_files) / 1e6, 1))
print("price.npy MB    :", round(size(os.path.join(d, "price.npy")) / 1e6, 1))

total_csv = pd.read_csv(csv_path)["price"].sum()
price = np.load(os.path.join(d, "price.npy"), mmap_mode="r")
total_npy = float(price.sum())
print("same total      :", np.isclose(total_csv, total_npy))
print("bytes touched, binary vs csv:", f"{size(os.path.join(d, 'price.npy')) / size(csv_path):.0%}")
```

```
csv MB          : 25.3
all columns MB  : 16.0
price.npy MB    : 8.0
same total      : True
bytes touched, binary vs csv: 32%
```

1. The table uses already-small dtypes, built with the knowledge from Section 22.2.
2. The CSV is the row-oriented, text version. Each numeric column is saved by `np.save` into its own file; the region column is saved as 8-bit category codes.
3. The CSV is about 25 MB. All column files together are about 16 MB, because 8-bit and 16-bit numbers need fewer bytes than their decimal text.
4. For the total price, the CSV approach must parse the entire file. The column approach opens only `price.npy`.
5. `mmap_mode="r"` memory-maps the file: the operating system pages bytes in on demand and the array behaves like a normal NumPy array.
6. The last line reports that the binary price column is about a third of the size of the CSV that held all five columns. Real Parquet adds compression on top (dictionary encoding, run-length encoding, a general compressor), so it usually does better still, and it adds a schema and statistics that this toy lacks.

Now the folder idea. Here we write a small dataset in the Hive layout (as CSV files, since Parquet is unavailable), then answer a filtered query by opening only the matching folder.

Problem 22.3.2 Write 100,000 orders into `year=.../region=.../part-0.csv` folders. Answer "total amount for year 2025 and region EU" by reading only one file, and prove the answer equals the full scan.

```
import os, tempfile
from pathlib import Path
import numpy as np
import pandas as pd

rng = np.random.default_rng(5)
n = 100_000
df = pd.DataFrame({
    "year": rng.choice([2024, 2025], n),
    "region": rng.choice(["EU", "US", "APAC"], n),
    "amount": rng.integers(1, 500, n),
})
root = Path(tempfile.mkdtemp()) / "sales"
for (year, region), part in df.groupby(["year", "region"]):
    folder = root / f"year={year}" / f"region={region}"
    folder.mkdir(parents=True)
    part[["amount"]].to_csv(folder / "part-0.csv", index=False)

for p in sorted(root.rglob("*.csv")):
    print(p.relative_to(root))

wanted = root / "year=2025" / "region=EU" / "part-0.csv"
fast = pd.read_csv(wanted)
print("rows read with pruning:", len(fast), "of", n)
full = df[(df.year == 2025) & (df.region == "EU")]["amount"].sum()
print("same total:", fast["amount"].sum() == full)
```

```
year=2024/region=APAC/part-0.csv
year=2024/region=EU/part-0.csv
year=2024/region=US/part-0.csv
year=2025/region=APAC/part-0.csv
year=2025/region=EU/part-0.csv
year=2025/region=US/part-0.csv
rows read with pruning: 16521 of 100000
same total: True
```

1. The generator makes three columns; `year` and `region` will become folder names.
2. `groupby(["year", "region"])` yields one group per combination. For each, we build the path `year=2025/region=EU` and write only the `amount` column, because the other two are encoded in the folder names (that is how Hive layout saves space).
3. `rglob("*.csv")` walks the tree; printing the relative paths shows the six files.
4. A reader that knows the layout builds the path for the wanted year and region and opens that single file: about one sixth of the rows.
5. The check compares with a brute-force filter over the full table. Equal totals prove that pruning changes how much you read, never the answer.

Real engines do this automatically. The same task in real libraries (not run here):

```
# NOT RUN HERE (library not installed)
import pandas as pd
import duckdb
import polars as pl
import pyarrow.dataset as ds

# df is the 100,000-row DataFrame (year, region, amount) from the block above
# 1. pandas + pyarrow: write a partitioned folder, then read with a filter
df.to_parquet("sales", partition_cols=["year", "region"], index=False)
eu_2025 = pd.read_parquet("sales", filters=[("year", "==", 2025), ("region", "==", "EU")])

# 2. DuckDB: the folder names become columns; the WHERE clause prunes folders
total = duckdb.sql("""
    SELECT SUM(amount) FROM read_parquet('sales/*/*/*.parquet', hive_partitioning = true)
    WHERE year = 2025 AND region = 'EU'
""").fetchone()[0]

# 3. Polars lazy: scan, filter, aggregate, collect
total = (pl.scan_parquet("sales/**/*.parquet", hive_partitioning=True)
           .filter((pl.col("year") == 2025) & (pl.col("region") == "EU"))
           .select(pl.col("amount").sum())
           .collect()
           .item())

# 4. pyarrow dataset
dataset = ds.dataset("sales", format="parquet", partitioning="hive")
table = dataset.to_table(filter=(ds.field("year") == 2025) & (ds.field("region") == "EU"), columns=["amount"])
```

1. All four read only the matching folder and only the `amount` column. The answer is the same number as the runnable block above prints (`fast["amount"].sum()`).
2. `partition_cols` in pandas and `PARTITION_BY` in DuckDB create the `column=value` folders for you.
3. In DuckDB, `hive_partitioning = true` turns the folder names back into columns, so the `WHERE` can use them.
4. Polars `scan_parquet` is lazy: nothing is read until `collect()` , and the optimizer pushes the filter and the column choice into the scan.
5. The PyArrow dataset API is the lower-level building block that several of the others sit on.

Expert: choosing a layout

- **Partition by a column that queries filter on and that has few values** (date, country, tenant). Do not partition by a high-cardinality column such as `customer_id` : you will create millions of tiny files, and listing them costs more than reading them.
- **Aim for files of roughly 100 MB to 1 GB** in a distributed setting. Thousands of tiny files (the "small files problem") slow every engine, since each file costs an open, a footer read and a task.
- **Sort within files by the column you filter on.** Row-group statistics then become tight (a small min-max range per group), so more groups can be skipped.
- **Compression**: snappy is quick and moderate; zstd gives smaller files at a similar speed; gzip is small but slow. Measure on your data.
- **Schema is part of the file.** Write dates as dates and money as decimals or integers, not strings. Changing a column type later means rewriting files.

KEY IDEA	Convert once, query many times. Reading a 10 GB CSV costs minutes every time you do it. Convert it to partitioned Parquet once (the recipe in Section 22.9), and every later query reads only the columns and folders it needs.

22.4 Polars: fast DataFrames with expressions and a lazy engine

Beginner: what Polars is and the mental model

Polars is a DataFrame library written in Rust. Three things make it different from pandas. It uses all your CPU cores by default. It stores data in the Arrow columnar format. And it has a **lazy mode** that builds a query plan and optimizes it before running. It also has no row index: rows are just rows, which removes a whole class of alignment surprises.

There are four objects to know. A **Series** is one column. A **DataFrame** is an eager table: every operation runs at once. A **LazyFrame** is a plan: operations are recorded and nothing runs until you call `.collect()`. An **Expression** (written `pl.col("amount") * 2`) is a recipe for computing a column; it does nothing until placed in a *context* such as `select`, `with_columns`, `filter` or `group_by().agg`.

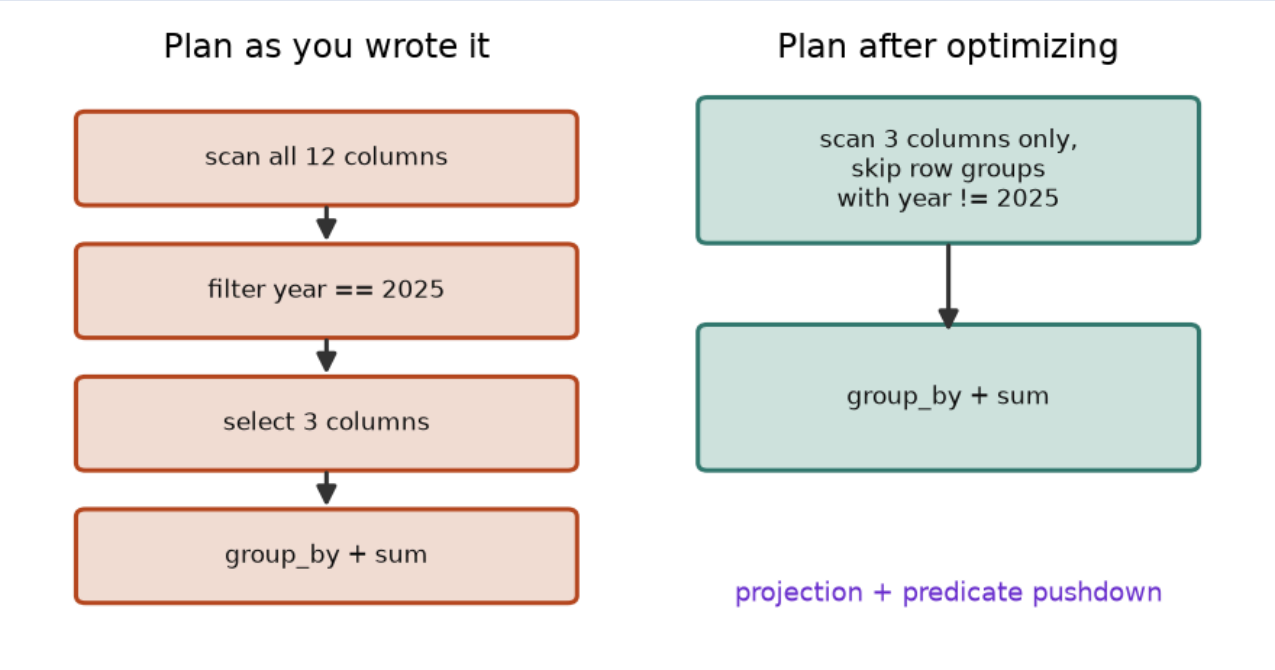


Figure 22.5. A lazy plan before and after optimization (a diagram, not a measurement).

How to read it: the left stack is the order in which you wrote the steps. The right stack is what the optimizer runs. It notices that only 3 columns are ever used (**projection pushdown**) and that the year filter can be applied inside the scan so whole row groups are skipped (**predicate pushdown**). You wrote four steps; the engine ran two cheaper ones.

Reference table: the 26 most important Polars names

Name	What it does in plain English	Minimal example
`pl.read_csv`	Read a CSV eagerly into a DataFrame	`pl.read_csv("o.csv")`
`pl.scan_csv`	Lazy CSV scan: returns a LazyFrame, reads nothing yet	`pl.scan_csv("o.csv")`
`pl.read_parquet / scan_parquet`	Eager / lazy Parquet reading	`pl.scan_parquet("sales/**/*.parquet")`
`pl.col(name)`	Refer to a column inside an expression	`pl.col("amount")`
`pl.lit(value)`	A constant inside an expression	`pl.lit(1)`
`df.select(exprs)`	Pick or compute columns (output has only these)	`df.select("region", pl.col("amount") * 2)`
`df.with_columns(exprs)`	Add or replace columns, keep the rest	`df.with_columns(vat=pl.col("amount") * 0.2)`

Name	What it does in plain English	Minimal example
`df.filter(expr)`	Keep rows where the expression is true	`df.filter(pl.col("amount") > 20)`
`df.group_by(keys).agg(exprs)`	Group, then aggregate (note the underscore)	`df.group_by("region").agg(pl.col("amount").sum())`
`expr.over(keys)`	Window function: compute within groups, keep every row	`pl.col("amount").sum().over("region")`
`expr.rank(method=, descending=)`	Rank values (methods: average, min, max, dense, ordinal)	`pl.col("r").rank("dense", descending=True)`
`df.join(other, on=, how=)`	Join two frames (inner, left, full, semi, anti, cross)	`df.join(t, on="customer_id", how="left")`
`df.sort(by, descending=)`	Sort rows	`df.sort("amount", descending=True)`
`pl.when().then().otherwise()`	If-else as an expression	`pl.when(pl.col("a") > 5).then(1).otherwise(0)`
`expr.cast(dtype)`	Change type (pl.Int32, pl.Float32, pl.Categorical...)	`pl.col("qty").cast(pl.UInt8)`
`expr.str.* / expr.dt.*`	String and date/time methods	`pl.col("ts").dt.year()`
`expr.is_in(list)`	Membership test	`pl.col("region").is_in(["EU", "US"])`
`expr.fill_null / df.drop_nulls`	Handle missing values (null, not NaN)	`pl.col("x").fill_null(0)`
`df.unique(subset=, keep=)`	Drop duplicate rows	`df.unique(subset=["order_id"], keep="first")`
`expr.n_unique() / pl.len()`	Distinct count / row count in an aggregation	`.agg(pl.len(), pl.col("c").n_unique())`
`df.pivot / df.unpivot`	Long to wide / wide to long	`df.pivot(on="region", index="year", values="amount")`
`pl.concat([a, b])`	Stack frames vertically (or horizontally)	`pl.concat([jan, feb])`
`lf.collect()`	Run a lazy plan and return a DataFrame	`lf.collect()`
`lf.explain()`	Print the optimized plan (use it to confirm pushdown)	`print(lf.explain())`
`lf.sink_parquet(path)`	Stream the result to a file with low memory	`lf.sink_parquet("out.parquet")`
`df.to_pandas() / pl.from_pandas()`	Convert to and from pandas (needs pyarrow)	`df.to_pandas()`
`df.schema / df.estimated_size("mb")`	Inspect types and memory	`print(df.schema)`

Intermediate: how pandas habits translate

Task	pandas	Polars
Filter rows	`df[df.amount > 20]`	`df.filter(pl.col("amount") > 20)`
New column	`df["v"] = df.amount * 2`	`df.with_columns(v=pl.col("amount") * 2)`
Group and sum	`df.groupby("r")["a"].sum()`	`df.group_by("r").agg(pl.col("a").sum())`
Group transform	`df.groupby("r")["a"].transform("sum")`	`pl.col("a").sum().over("r")`
Rank in group	`g["a"].rank(method="first")`	`pl.col("a").rank("ordinal").over("r")`
Merge	`a.merge(b, on="k", how="left")`	`a.join(b, on="k", how="left")`
Row-wise Python function	`df.apply(f, axis=1)`	Avoid. Use expressions; last resort `map_elements`
Missing value	`NaN` / `isna()`	`null` / `is_null()`
Index	`df.loc[...]`, index alignment	None: use `filter`, `join`

The big change in thinking: in pandas you modify columns step by step. In Polars you describe columns as expressions, and several expressions inside one `with_columns` run in parallel and all see the *original* frame. They cannot see each other's new columns; chain a second `with_columns` if you need that.

Worked example 1 (trivial): create, inspect, compute

```
# NOT RUN HERE (library not installed)
import polars as pl

df = pl.DataFrame({
    "region": ["EU", "EU", "US", "APAC"],
    "amount": [20, 35, 50, 40],
})
print(df.shape)                                  # (4, 2)
print(df.schema)                                 # region: String, amount: Int64
out = df.with_columns(
    amount_with_vat=pl.col("amount") * 1.2,
    big=pl.col("amount") > 30,
)
print(out)
print(df.select(pl.col("amount").sum()).item())   # 145
```

1. `pl.DataFrame({...})` builds a table from a dict of columns, like pandas. Polars infers `String` and `Int64`.
2. `df.shape` is rows and columns: (4, 2). `df.schema` maps column names to dtypes. It is cheap and worth printing at every step.
3. `with_columns` adds two new columns from two expressions in a single pass: the amount times 1.2 and a True/False flag.
4. `select(pl.col("amount").sum())` returns a one-cell DataFrame; `.item()` pulls out the number, 145 (20 + 35 + 50 + 40).

Expected result: `(4, 2)`, the schema, a four-row table with the two new columns (24.0, 42.0, 60.0 and 48.0 for the VAT column), and the final number 145. The same in pandas runs everywhere: `df["amount"].sum()` gives 145.

Worked example 2 (realistic): the running task in Polars

The task from Problem 22.1.1: revenue per (region, customer), tier by left join, rank within region. Here `orders.csv` holds the same ten rows.

```
# NOT RUN HERE (library not installed)
import polars as pl

orders = pl.read_csv("orders.csv")
tiers = pl.DataFrame({
    "customer_id": ["c1", "c2", "c3", "c4", "c5", "c6"],
    "tier": ["gold", "silver", "gold", "bronze", "silver", "bronze"],
})

result = (
    orders
    .group_by(["region", "customer_id"], maintain_order=True)
    .agg(pl.col("amount").sum().alias("revenue"))
    .join(tiers, on="customer_id", how="left")
    .with_columns(
        pl.col("revenue").rank("ordinal", descending=True).over("region").alias("rank")
    )
    .sort(["region", "rank"])
)
print(result)
```

1. `read_csv` loads the ten rows eagerly.
2. `group_by([...], maintain_order=True).agg(...)` sums the amount per pair. Without `maintain_order=True` Polars may return groups in any order, which matters only for tie-breaking later.
3. `.alias("revenue")` names the aggregate. Without it the column would keep the name `amount`.
4. `join(..., how="left")` keeps every revenue row and attaches the tier.
5. `rank("ordinal", descending=True).over("region")` is the window function. `over("region")` means "do this separately inside each region but keep all rows". `"ordinal"` is pandas `method="first"`: ties get distinct ranks in order of appearance.
6. `sort` makes the output order reproducible.

Expected result (verified by the pandas and SQLite blocks in Section 22.1; Polars draws it in its own bordered table with dtype labels):

```
region  customer_id  revenue  tier    rank
APAC    c5                70  silver     1
APAC    c6                 5  bronze     2
EU      c2                80  silver     1
EU      c1                35  gold       2
US      c3                75  gold       1
US      c4                10  bronze     2
```

Worked example 3 (realistic): lazy scanning of a file bigger than memory

```
# NOT RUN HERE (library not installed)
import polars as pl

lf = (
    pl.scan_csv("events.csv", try_parse_dates=True)
    .filter(pl.col("country") == "DE")
    .group_by("user_id")
    .agg(
        pl.len().alias("events"),
        pl.col("price").sum().alias("spend"),
    )
    .filter(pl.col("events") >= 5)
    .sort("spend", descending=True)
)

print(lf.explain())                          # read the optimized plan first
top = lf.head(1000).collect()                # run it, keep the top 1000 only
lf.sink_parquet("german_users.parquet")      # or stream the whole result to disk
```

1. `scan_csv` returns a LazyFrame. No data is read yet, so this line is instant even for 10 GB.
2. The next four calls only add steps to the plan. `pl.len()` counts rows per group.
3. `lf.explain()` prints the optimized plan as text. Look for the scan line listing only the needed columns (`user_id` , `country` , `price`) and a filter pushed into the scan. If you see every column, something blocked the pushdown.
4. `collect()` executes the plan and returns a DataFrame. Because only the top 1000 rows are needed, ask for them in the plan (`head`) instead of collecting everything and cutting later.
5. `sink_parquet` streams the result to disk in batches, so the final result need not fit in memory either. In recent Polars versions you can also request the streaming engine with `lf.collect(engine="streaming")` (older versions used `collect(streaming=True)` ; check the documentation for your version).

The output cannot be shown because there is no 10 GB file here; the structure to expect is a 1000-row table with columns `user_id` , `events` , `spend` sorted by spend. The idea of the plan is demonstrated by the runnable toy below.

Worked example 4 (expert, runnable): build a tiny lazy engine to see why it is faster

The toy below records filter and select steps without running them. At `collect` time, it works out which columns are really needed and reads only those from the CSV. It is a few lines, and it is the same trick as projection pushdown.

```
import io, os, tempfile
import pandas as pd

csv = """order_id,customer_id,region,category,amount
1,c1,EU,book,20
2,c2,EU,toy,35
3,c1,EU,toy,15
4,c3,US,book,50
5,c4,US,toy,10
6,c3,US,toy,25
7,c5,APAC,book,40
8,c5,APAC,book,30
9,c6,APAC,toy,5
10,c2,EU,book,45
"""
path = os.path.join(tempfile.mkdtemp(), "orders.csv")
open(path, "w").write(csv)

class Lazy:
    def __init__(self, path):
        self.path, self.steps = path, []
    def filter(self, col, test):
        self.steps.append(("filter", col, test)); return self
    def select(self, *cols):
        self.steps.append(("select", cols)); return self
    def collect(self, optimize=True):
        needed = set()
        for s in self.steps:
            needed.update([s[1]] if s[0] == "filter" else s[1])
        usecols = sorted(needed) if optimize else None
        df = pd.read_csv(self.path, usecols=usecols)
        print("columns parsed:", list(df.columns))
        out = list(df.columns)
        for s in self.steps:
            if s[0] == "filter":
                df = df[s[2](df[s[1]])]
            else:
                out = list(s[1])
        return df[out]

def plan():
    return Lazy(path).filter("region", lambda s: s == "EU").select("customer_id", "amount")

print("naive:")
a = plan().collect(optimize=False)
print("optimized:")
b = plan().collect()
print("same result:", a.equals(b))
print(b.to_string(index=False))
```

```
naive:
columns parsed: ['order_id', 'customer_id', 'region', 'category', 'amount']
optimized:
columns parsed: ['customer_id', 'region', 'amount']
same result: True
customer_id  amount
         c1      20
         c2      35
         c1      15
         c2      45
```

1. The ten-row file is written to a temporary folder, so the block is standalone.

2. The class `Lazy` stores steps in a list. `filter` and `select` run nothing, and they return `self` so calls can be chained like Polars methods.
3. In `collect`, the optimizer step is the set of columns mentioned anywhere in the plan: `region` (filter), `customer_id` and `amount` (select).
4. With `optimize=False` all five columns are parsed. With `optimize=True` only the three needed columns are parsed, the saving you saw in Figure 22.5.
5. Both plans give the same rows. Optimization changes the cost, never the answer; that is the contract of every query optimizer, in Polars, DuckDB and Spark.

COMMON MISTAKE	**Wrong:** `pl.read_csv("events.csv").filter(...).group_by(...)` on a huge file. **Symptom:** memory spike or kill at the first line; the optimizer never gets a chance to skip columns. **Fix:** `pl.scan_csv("events.csv")` and a final `.collect()` (or `sink_parquet`). Use eager `read_*` only when the file is small or you need to look at the data interactively.

Going deeper: Polars internals worth knowing

- Expressions are compiled into a plan and executed with parallelism across columns and within large columns. This is why `pl.col(...).str.to_uppercase()` beats a Python loop by a huge factor: the loop (via `map_elements`) hands one value at a time to the interpreter.
- Strings repeated often are best stored as `pl.Categorical` or `pl.Enum` (a fixed set of allowed values). It is the same idea as pandas `category`.
- Missing values are `null` everywhere, including integers, so an integer column with a missing value stays an integer column (in pandas classic dtypes it becomes float).
- Interview question: "Why can Polars be faster than pandas on the same machine?" Good answer: columnar Arrow memory, multi-threaded Rust kernels, lazy plans with pushdown, and no row index. Always add: for small data the difference is small, and the pandas ecosystem is broader.

22.5 DuckDB: SQL straight on files and DataFrames

Beginner: what DuckDB is

DuckDB is a database that lives inside your Python process, like SQLite, but built for analytics instead of transactions. There is no server to install or start. You point SQL at a CSV or Parquet file, or at a pandas DataFrame sitting in a variable, and it answers using a columnar, multi-threaded engine that can spill to disk when a query is larger than memory. If you already know SQL (and the SQL document in this library teaches it), DuckDB lets you use that knowledge on files that do not fit in pandas.

The mental model: *files are tables*. `SELECT ... FROM 'orders.csv'` or `FROM read_parquet('sales/*/*.parquet')` works with no loading step. Results come back as a DataFrame with `.df()` (pandas) or `.pl()` (Polars). The comparison with SQLite is useful: SQLite stores rows and is excellent for many small reads and writes; DuckDB stores columns and is excellent for scanning, grouping and joining many rows.

INTUITION	Think of DuckDB as "SQLite for analysis". Same lightweight feel (one import, no server), opposite design (columns, vectors, parallel scans). It is a poor fit for an app that writes one row at a time from many users; for that, use PostgreSQL or SQLite.

Reference table: the 28 most useful DuckDB names

Name	What it does in plain English	Minimal example (not run here)
`duckdb.sql(q)`	Run SQL on the default in-memory database; returns a lazy relation	`duckdb.sql("SELECT 42 AS x")`
`duckdb.connect(path)`	Open a connection; give a file name for a persistent database	`con = duckdb.connect("shop.duckdb")`
`con.execute(q, params)`	Run SQL with `?` placeholders	`con.execute("SELECT * FROM t WHERE a = ?", [5])`

Name	What it does in plain English	Minimal example (not run here)
`.fetchall() / .fetchone()`	Get Python tuples	`rel.fetchall()`
`.df() / .pl() / .arrow()`	Get a pandas / Polars / Arrow result (Arrow accessor name varies by version)	`duckdb.sql(q).df()`
`FROM 'file.csv'`	Query a CSV or Parquet by file name	`SELECT * FROM 'orders.csv' LIMIT 5`
`read_csv(path, ...)`	CSV reader with options (header, delim, types, sample size)	`read_csv('e.csv', header = true, types = {'amount': 'DOUBLE'})`
`read_parquet(glob)`	Read one or many Parquet files	`read_parquet('sales/**/*.parquet')`
`hive_partitioning = true`	Folder names `k=v` become columns (enables pruning)	`read_parquet('s/*/*.parquet', hive_partitioning = true)`
Replacement scan	A pandas/Polars/Arrow variable can be used as a table by its name	`duckdb.sql("SELECT region, SUM(amount) FROM df GROUP BY region")`
`con.register(name, df)`	Explicitly expose a DataFrame as a view	`con.register("orders", df)`
`CREATE TABLE t AS SELECT`	Materialize a result inside the database	`CREATE TABLE rev AS SELECT ...`
`COPY (q) TO file`	Write a result as Parquet, CSV or JSON	`COPY (SELECT ...) TO 'o.parquet' (FORMAT PARQUET)`
`PARTITION_BY`	Write Hive-partitioned folders	`COPY t TO 'out' (FORMAT PARQUET, PARTITION_BY (year))`
`GROUP BY ALL`	Group by every non-aggregated select column	`SELECT region, SUM(a) FROM t GROUP BY ALL`
`SELECT * EXCLUDE (c)`	All columns except some	`SELECT * EXCLUDE (note) FROM t`
`COLUMNS('regex')`	Apply an expression to all matching columns	`SELECT MAX(COLUMNS('amt_.*')) FROM t`
`QUALIFY`	Filter on a window function result without a subquery	`... QUALIFY ROW_NUMBER() OVER (...) = 1`
`OVER (PARTITION BY .. ORDER BY ..)`	Window functions: rank, running total, lag, lead	`SUM(a) OVER (PARTITION BY r ORDER BY d)`
`arg_max(x, y)`	The value of x on the row where y is largest (latest-row-per-key)	`arg_max(status, ts)`
`approx_count_distinct(c)`	Fast, memory-light distinct count (small error)	`SELECT approx_count_distinct(user_id)`
`quantile_cont(c, p)`	Percentile / median	`quantile_cont(amount, 0.5)`
`USING SAMPLE`	Random sample of a table or query	`SELECT * FROM t USING SAMPLE 1%`
`ASOF JOIN`	Join each row to the nearest earlier row (prices, sessions)	`a ASOF JOIN b ON a.k = b.k AND a.ts >= b.ts`
`PIVOT / UNPIVOT`	Reshape in SQL	`PIVOT t ON region USING SUM(amount)`
`DESCRIBE / SUMMARIZE`	Schema / per-column statistics for a table or query	`SUMMARIZE SELECT * FROM 'e.csv'`
`EXPLAIN / EXPLAIN ANALYZE`	Show the plan / the plan with measured times	`EXPLAIN ANALYZE SELECT ...`

Name	What it does in plain English	Minimal example (not run here)
`SET memory_limit / threads`	Resource limits; larger queries spill to `temp_directory`	`SET memory_limit = '8GB'`
`duckdb.__version__`	Version check	`print(duckdb.__version__)`

Worked example 1 (trivial): SQL on a file and on a DataFrame

```
# NOT RUN HERE (library not installed)
import duckdb
import pandas as pd

print(duckdb.__version__)

# 1. Query a CSV file by name. No loading step.
print(duckdb.sql("SELECT region, SUM(amount) AS revenue FROM 'orders.csv' GROUP BY region ORDER BY region").fetchall())

# 2. Query a pandas DataFrame by its variable name (a "replacement scan").
df = pd.read_csv("orders.csv")
print(duckdb.sql("SELECT region, SUM(amount) AS revenue FROM df GROUP BY region ORDER BY region").df())
```

1. `duckdb.sql(...)` parses and plans the query. The data is not read until you ask for results with `fetchall()` or `.df()`.
2. `FROM 'orders.csv'`: DuckDB sees a file name in the table position, detects the delimiter, header and column types by sampling, and scans the file.
3. In the second query `df` is not a table in any database. DuckDB looks up the Python variable named `df` in the calling code and scans it directly in memory with no copy of the data into a database.
4. `ORDER BY region` makes the order reproducible; without it SQL gives no order guarantee.
5. Expected values: APAC 75, EU 115, US 85 (hand check: APAC 40 + 30 + 5, EU 20 + 35 + 15 + 45, US 50 + 10 + 25). The runnable SQLite twin below returns exactly these.

The SQL text is standard enough to run in SQLite. This runnable block is the twin of both DuckDB queries: we load the same ten rows into SQLite from a DataFrame (the manual version of a replacement scan) and run the same SQL.

```
import io, sqlite3
import pandas as pd

csv = """order_id,customer_id,region,category,amount
1,c1,EU,book,20
2,c2,EU,toy,35
3,c1,EU,toy,15
4,c3,US,book,50
5,c4,US,toy,10
6,c3,US,toy,25
7,c5,APAC,book,40
8,c5,APAC,book,30
9,c6,APAC,toy,5
10,c2,EU,book,45"""
df = pd.read_csv(io.StringIO(csv))
con = sqlite3.connect(":memory:")
df.to_sql("df", con, index=False)
rows = con.execute("SELECT region, SUM(amount) AS revenue FROM df GROUP BY region ORDER BY region").fetchall()
print(rows)
print(pd.read_sql("SELECT category, COUNT(*) AS n, AVG(amount) AS avg_amount FROM df GROUP BY category ORDER BY category",
con))
```

```
[('APAC', 75), ('EU', 115), ('US', 85)]
  category  n  avg_amount
0     book  5        37.0
1      toy  5        18.0
```

1. The CSV text becomes a DataFrame; `to_sql("df", con)` copies it into an SQLite table named `df`, the table name the DuckDB version found by variable name.
2. The first query is character-for-character the DuckDB query. The three revenues match the hand check.
3. `pd.read_sql` runs a query and returns a DataFrame: pandas and SQL together, which DuckDB makes much faster for large inputs.
4. Category stats: books are orders 1, 4, 7, 8, 10 (20 + 50 + 40 + 30 + 45 = 185 over 5 = 37.0), toys are orders 2, 3, 5, 6, 9 (35 + 15 + 10 + 25 + 5 = 90 over 5 = 18.0).

Worked example 2 (intermediate): window functions, the part SQL does best

A **window function** computes a value for each row using a "window" of related rows, without collapsing the rows the way `GROUP BY` does. The three clauses: `PARTITION BY` (which rows belong together), `ORDER BY` (their order inside the window) and optionally a frame such as `ROWS BETWEEN 1 PRECEDING AND CURRENT ROW` (which neighbours to include). SQLite 3.25 and later, and DuckDB, share this syntax.

Problem 22.5.1 Daily sales per region: compute the running total, the change from the previous day, a 2-day moving average, and the rank of each day by sales inside its region (with ties). Run it in SQLite; the same SQL runs in DuckDB.

```
import sqlite3

con = sqlite3.connect(":memory:")
con.execute("CREATE TABLE sales(day INTEGER, region TEXT, amount INTEGER)")
con.executemany("INSERT INTO sales VALUES (?, ?, ?)", [
    (1, "EU", 10), (2, "EU", 30), (3, "EU", 30), (4, "EU", 20),
    (1, "US", 5), (2, "US", 15), (3, "US", 10),
])
q = """
SELECT region, day, amount,
  SUM(amount) OVER (PARTITION BY region ORDER BY day)                 AS running,
  amount - LAG(amount) OVER (PARTITION BY region ORDER BY day)        AS change,
  AVG(amount) OVER (PARTITION BY region ORDER BY day
                    ROWS BETWEEN 1 PRECEDING AND CURRENT ROW)         AS avg2,
  RANK()       OVER (PARTITION BY region ORDER BY amount DESC)        AS rnk,
  DENSE_RANK() OVER (PARTITION BY region ORDER BY amount DESC)        AS drnk
FROM sales ORDER BY region, day
"""
print("region day amount running change avg2 rnk drnk")
for r in con.execute(q):
    print(*r)
```

```
region day amount running change avg2 rnk drnk
EU 1 10 10 None 10.0 4 3
EU 2 30 40 20 20.0 1 1
EU 3 30 70 0 30.0 1 1
EU 4 20 90 -10 25.0 3 2
US 1 5 5 None 5.0 3 3
US 2 15 20 10 10.0 1 1
US 3 10 30 -5 12.5 2 2
```

1. The table holds four EU days (with a tie: days 2 and 3 both sold 30) and three US days.
2. `SUM(amount) OVER (PARTITION BY region ORDER BY day)` is a running total: with an `ORDER BY` and no frame, the default frame runs from the first row to the current one. EU runs 10, 40, 70, 90.
3. `LAG(amount)` reads the previous row's amount inside the partition; the first day has no previous row, so the subtraction gives `None` (SQL NULL).

4. The frame `ROWS BETWEEN 1 PRECEDING AND CURRENT ROW` averages the current and previous row: EU day 2 is (10 + 30) / 2 = 20.0. The first row has only itself, so its average is 10.0.
5. `RANK` gives tied rows the same rank and then skips: EU amounts 30, 30, 20, 10 get ranks 1, 1, 3, 4. `DENSE_RANK` does not skip: 1, 1, 2, 3.
6. Remember this about ties for the Practice set: `ROW_NUMBER` never ties, `RANK` skips after a tie, `DENSE_RANK` does not.

Worked example 3 (realistic): the running task in DuckDB, with QUALIFY

```
# NOT RUN HERE (library not installed)
import duckdb

duckdb.sql("""
    CREATE OR REPLACE TABLE rev AS
    SELECT o.region, o.customer_id, SUM(o.amount) AS revenue
    FROM 'orders.csv' o
    GROUP BY o.region, o.customer_id
""")

tiers = duckdb.sql("""
    SELECT * FROM (VALUES ('c1','gold'),('c2','silver'),('c3','gold'),
                          ('c4','bronze'),('c5','silver'),('c6','bronze')) AS t(customer_id, tier)
""")
duckdb.register("tiers", tiers)

# All ranks, same SQL text as the SQLite block in Section 22.1:
print(duckdb.sql("""
    SELECT r.region, r.customer_id, r.revenue, t.tier,
           ROW_NUMBER() OVER (PARTITION BY r.region ORDER BY r.revenue DESC) AS rnk
    FROM rev r LEFT JOIN tiers t USING (customer_id)
    ORDER BY r.region, rnk
""").fetchall())

# Only the top customer per region, using QUALIFY (DuckDB; SQLite does not have it):
print(duckdb.sql("""
    SELECT r.region, r.customer_id, r.revenue
    FROM rev r
    QUALIFY ROW_NUMBER() OVER (PARTITION BY r.region ORDER BY r.revenue DESC) = 1
    ORDER BY r.region
""").fetchall())
```

1. The first statement reads the CSV and groups. `CREATE OR REPLACE TABLE ... AS` stores the small result inside DuckDB for reuse.
2. The tiers are a literal `VALUES` list; `AS t(customer_id, tier)` names the columns.
3. The first query is the same logic as the SQLite window query: six rows, ranks 1 and 2 in each region. `USING (customer_id)` is shorthand for `ON r.customer_id = t.customer_id` that also merges the key column.
4. `QUALIFY` filters on a window function directly. Without it you need a subquery: `SELECT * FROM (SELECT ..., ROW_NUMBER() OVER (...) AS rn FROM rev) WHERE rn = 1`. The runnable block below uses that portable form.
5. Expected: the first query returns the six rows already shown in Section 22.1; the second returns `('APAC', 'c5', 70)`, `('EU', 'c2', 80)`, `('US', 'c3', 75)`.

```
import io, sqlite3
import pandas as pd

csv = """order_id,customer_id,region,category,amount
1,c1,EU,book,20
2,c2,EU,toy,35
3,c1,EU,toy,15
4,c3,US,book,50
5,c4,US,toy,10
6,c3,US,toy,25
7,c5,APAC,book,40
8,c5,APAC,book,30
9,c6,APAC,toy,5
10,c2,EU,book,45"""
con = sqlite3.connect(":memory:")
pd.read_csv(io.StringIO(csv)).to_sql("orders", con, index=False)
q = """
SELECT region, customer_id, revenue FROM (
    SELECT region, customer_id, SUM(amount) AS revenue,
           ROW_NUMBER() OVER (PARTITION BY region ORDER BY SUM(amount) DESC) AS rn
    FROM orders GROUP BY region, customer_id
) WHERE rn = 1 ORDER BY region
"""
print(con.execute(q).fetchall())
```

```
[('APAC', 'c5', 70), ('EU', 'c2', 80), ('US', 'c3', 75)]
```

1. This is the portable equivalent of QUALIFY: compute the window function in an inner query, filter it in the outer one.
2. A window function can wrap an aggregate: `ROW_NUMBER() OVER (... ORDER BY SUM(amount) DESC)` works because windows are evaluated after `GROUP BY`.
3. The result matches the DuckDB expectation above: APAC c5 70, EU c2 80, US c3 75.

Worked example 4 (expert): bigger than memory and Parquet output

```
# NOT RUN HERE (library not installed)
import duckdb

con = duckdb.connect("analytics.duckdb")          # a persistent database file
con.execute("SET memory_limit = '6GB'")           # cap RAM; bigger operators spill to disk
con.execute("SET temp_directory = '/tmp/duck_spill'")
con.execute("SET threads = 4")

# Convert a huge CSV into partitioned Parquet in one streaming query.
con.execute("""
    COPY (
        SELECT *, year(CAST(ts AS TIMESTAMP)) AS year
        FROM read_csv('events.csv', header = true)
    ) TO 'events_pq' (FORMAT PARQUET, PARTITION_BY (year), COMPRESSION zstd)
""")

# Query it back; the WHERE prunes the year folders.
top = con.execute("""
    SELECT user_id, COUNT(*) AS n
    FROM read_parquet('events_pq/*/*.parquet', hive_partitioning = true)
    WHERE year = 2025
    GROUP BY user_id
    ORDER BY n DESC
    LIMIT 10
""").df()
print(top)
con.execute("EXPLAIN ANALYZE SELECT COUNT(*) FROM read_parquet('events_pq/*/*.parquet')").fetchall()
```

1. `connect("analytics.duckdb")` stores the database in a file you can reopen later. Only one process may write to it at a time.
2. `SET memory_limit` and `temp_directory` tell DuckDB how much RAM it may use and where to spill. Sorts, big joins and large aggregates then continue from disk instead of failing.
3. The `COPY (SELECT ...) TO ... (FORMAT PARQUET, PARTITION_BY (year))` statement reads the CSV in a streaming fashion and writes one folder per year. This is the "convert once" step. The output folder must not already exist.
4. The second query reads only the `year=2025` folder; only the `user_id` column is touched.
5. `EXPLAIN ANALYZE` runs the query and prints the plan with row counts and timings per operator; use it to see where time goes.

COMMON MISTAKE

Wrong: `SELECT * FROM 'events.csv'` followed by `.df()` on a 10 GB file. **Symptom:** DuckDB streams happily, then pandas runs out of memory building the result. **Fix:** do the filtering and aggregation in SQL and convert only the small result to pandas; or `COPY ... TO` a file. The engine can handle data larger than memory, but your Python variable cannot.

Going deeper: why DuckDB is fast

- **Vectorized execution**: operators process batches of values (around 2,000) at a time instead of one row, keeping data in CPU caches and avoiding per-row interpretation costs.
- **Morsel-driven parallelism**: the input is cut into small work units that threads pick up, so all cores stay busy.
- **Pushdown into Parquet**: projection and filters are applied while reading, using the footer statistics to skip row groups.
- Interview question: "When DuckDB and when Spark?" Answer with the data size, the number of machines, the existing platform, and the fact that one big machine with DuckDB beats a small cluster surprisingly often for datasets up to hundreds of GB, while data that must be written by many jobs concurrently belongs in a warehouse or lakehouse.

22.6 Dask: pandas in pieces, with a scheduler

Beginner: the idea

Dask splits a big table into many pandas DataFrames called **partitions**, and builds a **task graph** (a diagram of small jobs and what each one needs). Calling a method such as `.sum()` only adds boxes to the graph, which is lazy execution. Calling `.compute()` hands the graph to a **scheduler** that runs the small jobs on all cores (threads or processes), or on many machines through `dask.distributed`, and returns an ordinary pandas object. The API deliberately looks like pandas, so existing code often needs few changes. It does not copy the pandas API completely: operations that need all the data in order (a global sort, a rank within groups, an arbitrary row lookup) are expensive or missing.

DEFINITION

Partition: one pandas DataFrame inside the Dask DataFrame. **Lazy**: building a plan without running it. **Persist**: run the plan now and keep the result in (distributed) memory so later steps reuse it. **Shuffle**: move rows between partitions so that rows with the same key end up together; the expensive step.

Reference table: the 24 most useful Dask names

Name	What it does in plain English	Minimal example (not run here)
`dd.read_csv(path, blocksize=)`	Lazy CSV read; one partition per block; globs allowed	`dd.read_csv("logs/*.csv", blocksize="64MB")`
`dd.read_parquet(path)`	Lazy Parquet read	`dd.read_parquet("sales/", columns=["a"])`
`dd.from_pandas(df, npartitions=)`	Turn a pandas frame into a Dask frame	`dd.from_pandas(df, npartitions=8)`
`ddf.npartitions`	How many pieces	`ddf.npartitions`
`ddf.compute()`	Run the graph, return pandas	`ddf.amount.sum().compute()`
`dask.compute(a, b)`	Run several results in one pass (shares work)	`dask.compute(x.sum(), x.mean())`
`ddf.persist()`	Compute now, keep partitions in memory	`ddf = ddf.persist()`
`ddf.head(n)`	First rows of the FIRST partition only (cheap, but not a sample)	`ddf.head(5)`
`ddf.groupby(k).agg()`	Tree-reduced group-by	`ddf.groupby("region").amount.sum()`
`ddf.merge(other, on=, how=)`	Join; a small pandas side is broadcast, two large sides are shuffled	`ddf.merge(tiers, on="customer_id")`
`ddf.map_partitions(func)`	Run any pandas function on each partition	`ddf.map_partitions(add_rank, meta=m)`
`meta=`	Empty frame describing output columns and dtypes	`meta=ddf._meta.assign(rank=0)`
`ddf.repartition(npartitions= / partition_size=)`	Change the number or size of partitions	`ddf.repartition(partition_size="100MB")`
`ddf.shuffle(on=)`	Co-locate equal keys in one partition	`ddf.shuffle(on="region")`
`ddf.set_index(col)`	Sort by a column and record partition boundaries (costly, speeds joins/lookups)	`ddf.set_index("ts")`

Name	What it does in plain English	Minimal example (not run here)
`ddf.drop_duplicates()`	Distinct rows (a shuffle-like reduction)	`ddf.drop_duplicates(subset=["order_id"])`
`ddf.astype / ddf.categorize`	Change dtypes; make categoricals with known categories	`ddf.astype({"qty": "uint8"})`
`ddf.to_parquet(path, partition_on=)`	Write output, one file per partition	`ddf.to_parquet("out/", partition_on=["year"])`
`ddf.memory_usage(deep=True)`	Lazy memory report (compute it)	`ddf.memory_usage(deep=True).compute()`
`ddf.visualize()`	Draw the task graph (needs graphviz)	`ddf.amount.sum().visualize()`
`Client() / LocalCluster()`	Start the distributed scheduler with a live dashboard	`from dask.distributed import Client; client = Client()`
`client.dashboard_link`	URL of the dashboard (task stream, memory, progress)	`print(client.dashboard_link)`
`dask.config.set(scheduler=)`	Choose "threads", "processes" or "synchronous" (debug)	`dask.config.set(scheduler="synchronous")`
`dask.delayed`	Make any Python function lazy and parallel	`dask.delayed(load)(path)`

Worked example 1 (trivial, not run): the Dask version of a pandas group-by

```
# NOT RUN HERE (library not installed)
import dask.dataframe as dd

ddf = dd.read_csv("orders.csv")
print(ddf.npartitions)                      # 1 for a tiny file; many for a big one
lazy = ddf.groupby("region").amount.sum()   # nothing has been computed yet
print(lazy.compute().sort_index())          # APAC 75, EU 115, US 85
```

1. `read_csv` splits the file into blocks (default tens of MB) and returns immediately; nothing is parsed yet.
2. `ddf.npartitions` is 1 for a ten-row file. A 10 GB file at 64 MB blocks gives about 160 partitions.
3. The group-by is a plan: sum within each partition, then combine the partial sums. `compute()` runs it and returns a pandas Series. Expected: APAC 75, EU 115, US 85, the same numbers as the pandas, SQLite and DuckDB blocks.

Worked example 2 (runnable): what Dask does inside, in plain pandas

We can reproduce Dask's strategy with pandas alone: cut the table into partitions, reduce each partition separately, combine the partial results. The block also shows the classic trap: an average of averages is wrong when partitions have different sizes.

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(1)
n = 300_000
df = pd.DataFrame({"region": rng.choice(["EU", "US", "APAC"], n), "price": rng.normal(50, 15, n)})

cuts = [0, 20_000, 120_000, n]                          # unequal partitions on purpose
parts = [df.iloc[a:b] for a, b in zip(cuts[:-1], cuts[1:])]
print("partition sizes:", [len(p) for p in parts])

partials = [p.groupby("region")["price"].agg(["sum", "count"]) for p in parts]
combined = sum(partials)                                 # element-wise add, aligned on region
mean_correct = combined["sum"] / combined["count"]

mean_of_means = pd.concat([p.groupby("region")["price"].mean() for p in parts], axis=1).mean(axis=1)
truth = df.groupby("region")["price"].mean()

print("sum/count equals pandas:", np.allclose(mean_correct.sort_index(), truth.sort_index()))
print("mean of means equals pandas:", np.allclose(mean_of_means.sort_index(), truth.sort_index()))
print("worst error of mean-of-means:", float((mean_of_means - truth).abs().max()) > 1e-6)
```

```
partition sizes: [20000, 100000, 180000]
sum/count equals pandas: True
mean of means equals pandas: False
worst error of mean-of-means: True
```

1. Three partitions of 20,000, 100,000 and 180,000 rows stand in for Dask partitions.
2. Step one (the "map"): every partition computes the sum and the count of price per region. These are small and can be done anywhere, in parallel.
3. Step two (the "reduce"): `sum(partials)` adds the small tables aligned on region. This is what Dask's tree reduction does, in a tree so it works for thousands of partitions.
4. The correct mean is total sum divided by total count. Dask's `mean()` does exactly this internally.
5. The wrong way averages the three partition means with equal weight, so the 20,000-row partition counts as much as the 180,000-row one. The result differs from pandas, so the equality check prints `False`, and the last line prints `True` because the error is bigger than 1e-6.
6. Rule: only combine *decomposable* partial results (sum, count, min, max, sum of squares). Medians and exact distinct counts need other approaches.

A rank inside a group needs all of a group's rows in one place. In Dask that means a shuffle first. The running task in Dask (not run here):

```
# NOT RUN HERE (library not installed)
import dask.dataframe as dd
import pandas as pd

orders = dd.read_csv("orders.csv")
tiers = pd.DataFrame({"customer_id": ["c1", "c2", "c3", "c4", "c5", "c6"],
                      "tier": ["gold", "silver", "gold", "bronze", "silver", "bronze"]})

rev = (orders.groupby(["region", "customer_id"]).amount.sum()
             .reset_index().rename(columns={"amount": "revenue"}))
rev = rev.merge(tiers, on="customer_id", how="left")     # small pandas side: cheap join

def add_rank(part):
    part = part.copy()
    part["rank"] = part.groupby("region")["revenue"].rank(method="first", ascending=False).astype(int)
    return part

ranked = rev.shuffle(on="region").map_partitions(add_rank, meta=rev._meta.assign(rank=0))
print(ranked.compute().sort_values(["region", "rank"]))
```

1. The group-by and `reset_index` give one row per region and customer; `rename` gives the honest column name.
2. The merge with a small pandas DataFrame is cheap: Dask sends the small table to every partition (a broadcast join) instead of shuffling the big one.
3. Dask has no distributed "rank within group". The standard pattern is: `shuffle(on="region")` so that all rows of a region are in the same partition, then run an ordinary pandas function on each partition with `map_partitions`.
4. `meta=` tells Dask the output columns and dtypes without running the function. Without it, Dask guesses by running your function on fake data, which can be slow or wrong.
5. Expected: the same six rows as Problem 22.1.1.

Worked example 3 (runnable): a shuffle join, step by step

Joining two big tables is the hardest distributed operation. Both sides are hash-partitioned by the join key, so matching keys land in the same partition number, then each pair of partitions is joined locally. The block does that with pandas and checks it against a single big merge.

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(2)
orders = pd.DataFrame({"customer_id": rng.integers(0, 5_000, 100_000), "amount": rng.integers(1, 100, 100_000)})
customers = pd.DataFrame({"customer_id": np.arange(5_000), "tier": rng.choice(["gold", "silver", "bronze"], 5_000)})
K = 4

def split(frame):
    bucket = pd.util.hash_pandas_object(frame["customer_id"], index=False) % K
    return [frame[bucket == i] for i in range(K)]

o_parts, c_parts = split(orders), split(customers)
print("order rows per partition:", [len(p) for p in o_parts])
joined = pd.concat([o.merge(c, on="customer_id", how="left") for o, c in zip(o_parts, c_parts)])
direct = orders.merge(customers, on="customer_id", how="left")
key = ["customer_id", "amount", "tier"]
print("rows:", len(joined), len(direct))
print("same result:",
joined.sort_values(key).reset_index(drop=True).equals(direct.sort_values(key).reset_index(drop=True)))
```

```
order rows per partition: [23741, 26181, 24196, 25882]
rows: 100000 100000
same result: True
```

1. Two tables: 100,000 orders and 5,000 customers, joined on `customer_id`.
2. `hash_pandas_object` gives a stable hash for each key; the remainder modulo K (4) picks a partition. A stable hash is essential: both sides must agree where each key goes. (Python's built-in `hash` of strings changes between runs, which is a real production bug.)
3. The orders table splits into four roughly equal parts, since customer ids are spread evenly.
4. Each pair of partitions is merged independently; no pair needs rows from another pair. That independence is the whole trick, and it is what lets different machines do the pairs at the same time.
5. The concatenated result equals one big merge (after sorting, because partition order is not row order). Moving rows to their partition is the **shuffle**, the expensive part on a real cluster because it crosses the network.

COMMON MISTAKE

Wrong: `ddf.head()` to peek at the data, or calling `.compute()` on a result as large as the input. **Symptom:** `head()` shows only rows from the first partition and misleads you; `compute()` of a huge frame crashes the notebook because the result becomes one pandas object. **Fix:** aggregate first and compute small results, write big results with `to_parquet`, and use `ddf.sample(frac=0.01).compute()` for a representative peek.

Going deeper: scheduling and sizing

- Aim for partitions of roughly 100 MB. Thousands of tiny partitions drown the scheduler in overhead (each task costs about a millisecond of bookkeeping); a few giant ones lose parallelism and risk memory errors.
- The threaded scheduler is good for NumPy/pandas code that releases Python's global interpreter lock. Pure-Python functions need processes (or a distributed cluster).
- With `Client()` the dashboard shows the task stream, per-worker memory and progress. A worker bar turning orange or red means it is spilling to disk or close to being killed: use smaller partitions or fewer parallel tasks.
- Recent Dask versions plan queries with an optimizer (the dask-expr layer), so column selection and filters are pushed into the read much like Polars; the API you use stays the same.

22.7 PySpark: partitions, shuffles, joins, skew, caching and the Spark UI

Beginner: the model

Apache Spark is an engine that runs a query across many machines (or many cores of one). **PySpark** is its Python API. A Spark program has one **driver** (your Python process, which plans the work) and many **executors** (processes that hold data and run tasks). The data is a **DataFrame** cut into **partitions**; each partition is processed by one **task**. Operations come in two kinds. **Transformations** (`filter` , `select` , `groupBy` , `join`) are lazy: they extend the plan. **Actions** (`count` , `show` , `collect` , `write`) trigger the run. A **job** is the work for one action; it is cut into **stages** at every shuffle; each stage is a set of parallel tasks.

The key distinction for performance: a **narrow** transformation (`filter` , `select` , `withColumn`) works on each partition independently and needs no data movement. A **wide** transformation (`groupBy` , `join` , `distinct` , `orderBy` , `repartition`) needs rows with the same key to meet, so data is **shuffled**: written to local disk, sent across the network, read by the next stage. Shuffles dominate the cost of most Spark jobs.

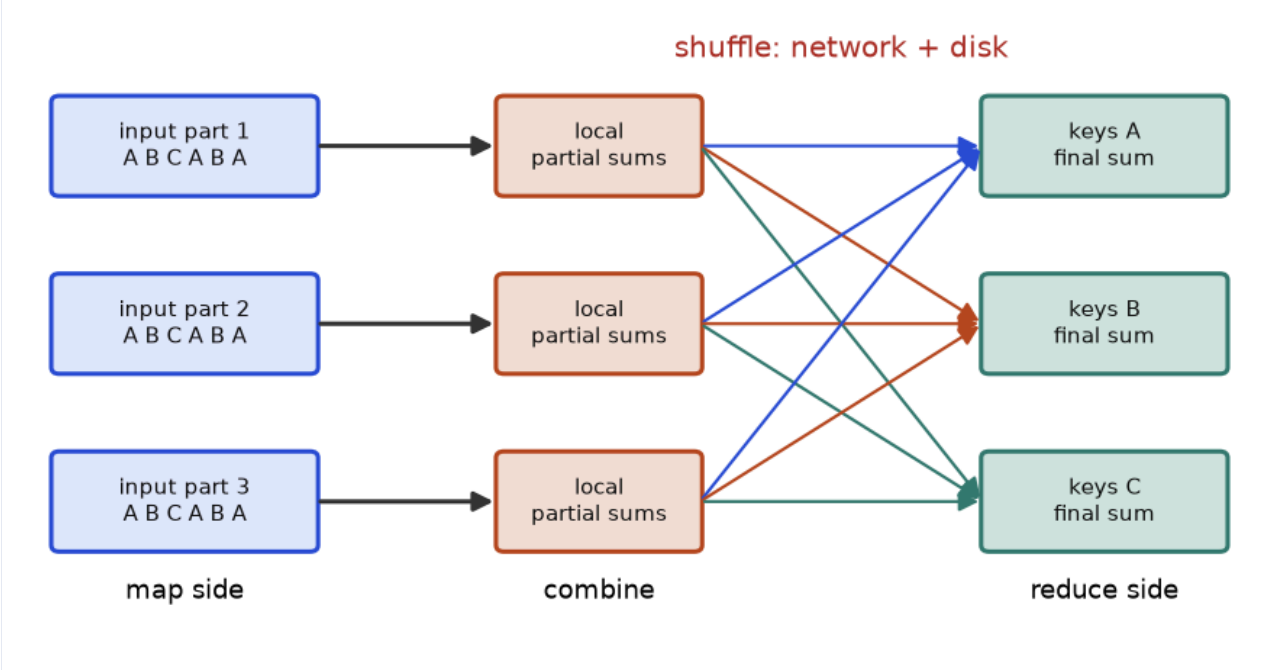


Figure 22.6. A shuffle: every map-side partition sends rows to every reduce-side partition (a diagram).

How to read it: the left boxes are input partitions holding mixed keys. Each computes partial sums locally (the middle boxes), which is why `groupBy().sum()` moves only small partial results, not raw rows. The nine arrows are the shuffle: each of three senders sends data to each of three receivers. On a cluster that is network and disk traffic, and the next stage cannot start until all of it has arrived. Fewer, smaller shuffles mean faster jobs.

Reference table: the 28 most useful PySpark names

Name	What it does in plain English	Minimal example (not run here)
`SparkSession.builder...getOrCreate()`	Start (or reuse) Spark; the entry point	`spark = SparkSession.builder.master("local[*]").getOrCreate()`
`spark.read.csv / parquet / json`	Read data into a DataFrame (lazy)	`spark.read.parquet("sales/")`
`inferSchema=True`	Guess types (costs an extra pass; prefer an explicit schema in production)	`spark.read.csv(p, header=True, inferSchema=True)`
`spark.createDataFrame(data, cols)`	Build from Python data or pandas	`spark.createDataFrame([("c1","gold")], ["id","tier"])`
`df.select / withColumn / drop`	Choose, add, remove columns	`df.withColumn("vat", F.col("a") * 0.2)`
`df.filter / where`	Keep rows matching a condition	`df.filter(F.col("amount") > 20)`
`df.groupBy(k).agg(...)`	Group and aggregate (wide)	`df.groupBy("region").agg(F.sum("amount"))`
`pyspark.sql.functions as F`	Built-in column functions: sum, count, when, concat, regexp_extract, to_date...	`F.when(F.col("a") > 5, 1).otherwise(0)`
`df.join(other, on, how)`	Join (inner, left, right, full, left_semi, left_anti, cross)	`a.join(b, "customer_id", "left")`
`F.broadcast(df)`	Hint: send the small table to all executors, skip the shuffle	`big.join(F.broadcast(small), "k")`
`Window.partitionBy().orderBy()`	Define a window for ranks, running totals, lag	`w = Window.partitionBy("region").orderBy(F.desc("revenue"))`
`F.row_number().over(w)`	Window function on that window (also rank, dense_rank, lag, sum)	`df.withColumn("rk", F.row_number().over(w))`
`df.dropDuplicates(subset)`	Remove duplicate rows (a shuffle)	`df.dropDuplicates(["order_id"])`
`df.orderBy / sort`	Global sort (wide, expensive; avoid when not needed)	`df.orderBy(F.desc("amount"))`

Name	What it does in plain English	Minimal example (not run here)
`df.repartition(n, cols)`	Reshuffle into n partitions (optionally by key)	`df.repartition(200, "customer_id")`
`df.coalesce(n)`	Reduce partitions without a full shuffle	`df.coalesce(8)`
`df.rdd.getNumPartitions()`	How many partitions now	`df.rdd.getNumPartitions()`
`df.cache() / persist() / unpersist()`	Keep a result in memory (lazy: filled by the first action)	`df = df.cache(); df.count()`
`df.explain()`	Print the physical plan (look for Exchange and join type)	`df.explain("formatted")`
`df.show / count / collect / take`	Actions: run the plan	`df.show(5)`
`df.toPandas()`	Bring the result to the driver as pandas (must be small)	`df.limit(1000).toPandas()`
`df.write.partitionBy().parquet()`	Write a partitioned folder	`df.write.mode("overwrite").partitionBy("year").parquet("out")`
`spark.sql(q) / createOrReplaceTempView`	Use SQL on DataFrames	`df.createOrReplaceTempView("t"); spark.sql("SELECT ...")`
`F.udf(f, type)`	Row-at-a-time Python function (slow; last resort)	`F.udf(lambda s: s.lower(), "string")`
`F.pandas_udf`	Vectorized Python function over pandas Series (faster than a plain UDF)	`@F.pandas_udf("double")`
`spark.conf.set(k, v)`	Tune: shuffle partitions, broadcast threshold, AQE	`spark.conf.set("spark.sql.shuffle.partitions", "64")`
`F.rand / F.explode / F.array`	Building blocks for salting a skewed key	`(F.rand() * 8).cast("int")`
`spark.stop()`	Release resources at the end of a script	`spark.stop()`

Worked example 1 (realistic, not run): the running task in PySpark

```
# NOT RUN HERE (library not installed)
from pyspark.sql import SparkSession, functions as F, Window

spark = (SparkSession.builder.appName("ch22").master("local[*]")
         .config("spark.sql.shuffle.partitions", "8")      # the default is 200, too many for tiny data
         .getOrCreate())

orders = spark.read.csv("orders.csv", header=True, inferSchema=True)
tiers = spark.createDataFrame(
    [("c1", "gold"), ("c2", "silver"), ("c3", "gold"),
     ("c4", "bronze"), ("c5", "silver"), ("c6", "bronze")],
    ["customer_id", "tier"],
)

rev = orders.groupBy("region", "customer_id").agg(F.sum("amount").alias("revenue"))
w = Window.partitionBy("region").orderBy(F.col("revenue").desc())
result = (rev.join(tiers, "customer_id", "left")
             .withColumn("rank", F.row_number().over(w))
             .select("region", "customer_id", "revenue", "tier", "rank")
             .orderBy("region", "rank"))
result.show()
result.explain()
spark.stop()
```

1. The session is the entry point. `master("local[*]")` runs Spark on all cores of this machine, ideal for learning and tests. On a cluster the platform sets the master for you.
2. `spark.sql.shuffle.partitions` is the number of partitions after any shuffle; its default is 200. For ten rows that is 200 mostly empty tasks, so we lower it. On big data you raise it so each partition is a manageable size.
3. `read.csv(..., inferSchema=True)` reads the file once extra to guess types. On big files supply a schema instead.
4. `groupBy().agg(F.sum(...))` is the wide step: partial sums per partition, shuffle by (region, customer_id), final sums.
5. The window is defined once (`Window.partitionBy("region").orderBy(...)`) and applied with `F.row_number().over(w)` . This needs all rows of a region together, so Spark shuffles by region.
6. With tables this small Spark would pick a broadcast join for `tiers` automatically. `explain()` prints the physical plan; read it from the bottom up and look for `Exchange` (a shuffle) and `BroadcastHashJoin` .
7. Expected output of `show()` : the same six rows as before, drawn in Spark's ASCII table. Spark does not guarantee row order without `orderBy` , and that is why we sort.

```
region  customer_id  revenue  tier    rank
APAC    c5                70  silver     1
APAC    c6                 5  bronze     2
EU      c2                80  silver     1
EU      c1                35  gold       2
US      c3                75  gold       1
US      c4                10  bronze     2
```

Intermediate: partitions, joins and caching

- **Partition count**: when reading files, Spark makes about one partition per 128 MB of input by default. After a shuffle you get `spark.sql.shuffle.partitions` partitions. Rule of thumb: target partitions of about 100 to 200 MB, at least 2 to 3 times as many partitions as cores. Too few: underused cores and memory errors. Too many: scheduling overhead and tiny output files.
- **Join strategies**: if one side is small (default threshold 10 MB, setting `spark.sql.autoBroadcastJoinThreshold`), Spark broadcasts it and no shuffle of the big side happens. Otherwise it does a **sort-merge join**: shuffle both sides by the key, sort, merge. Use `F.broadcast(small)` to force the cheap path when you know a table is small.

- **Join key quality**: filter and select before the join so less data crosses the network; make sure both key columns have the same type, or Spark casts and may skip optimizations; check nulls (a null key never matches).
- **Caching**: `df.cache()` marks a DataFrame to be kept in memory after its first action. Use it when the same expensive DataFrame feeds several actions (for example, a model's training loop or three reports from one cleaned table). Do not cache something used once, and call `unpersist()` when done. The Storage tab of the UI shows what is cached and how much space it uses.
- **Lazy plan, repeated work**: without cache, every action re-runs the whole plan from the source files. Two `count()` calls on the same unmaterialized DataFrame read and process the data twice.

```
# NOT RUN HERE (library not installed)
from pyspark.sql import SparkSession, functions as F

spark = SparkSession.builder.master("local[*]").getOrCreate()
events = spark.read.parquet("events_pq")          # big
users = spark.read.parquet("users_pq")            # small (a few MB)

slim = events.select("user_id", "price").filter(F.col("price") > 0)   # prune early
joined = slim.join(F.broadcast(users), "user_id", "left")               # no shuffle of events
joined.explain()                                  # expect BroadcastHashJoin, no Exchange on events

clean = joined.cache()                            # lazy: nothing happens yet
print(clean.count())                              # first action: computes and fills the cache
print(clean.groupBy("country").agg(F.sum("price")).count())   # reuses the cache
clean.unpersist()
```

1. Selecting two columns and filtering before the join cuts the rows and the bytes that every later step handles.
2. `F.broadcast(users)` promises Spark that `users` is small enough to ship to every executor. If the promise is false the executors run out of memory, so use it only for genuinely small tables.
3. In `explain()` a `BroadcastHashJoin` with a `BroadcastExchange` on the small side only, and no `Exchange hashpartitioning` on the events side, confirms the shuffle is avoided.
4. `cache()` itself does nothing. The first action (`count`) computes the join and stores the partitions; the second action reads from memory.
5. `unpersist()` frees the memory for other work.

Expert (runnable): data skew and salting

Skew means a few keys have far more rows than the others. After a shuffle by key, one partition gets the hot key's rows and one task runs far longer than its peers while the rest of the cluster sits idle. The block simulates it: 1 million rows, 1,000 keys with a Zipf-like popularity (key 0 gets about 13 percent of all rows), hashed into 8 partitions. **Salting** adds a random suffix 0 to 7 to the key so the hot key is spread over several partitions; an aggregation is done in two stages so the answer stays exact.

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(7)
n, n_keys, n_parts = 1_000_000, 1000, 8
w = 1 / np.arange(1, n_keys + 1)
keys = rng.choice(n_keys, n, p=w / w.sum())
vals = rng.integers(1, 100, n)

def part_sizes(k):
    h = pd.util.hash_array(k.astype(np.int64))
    return np.bincount(h % n_parts, minlength=n_parts)

before = part_sizes(keys)
print("hottest key share:", round((keys == 0).mean(), 3))
print("before:", before.tolist(), "max/mean =", round(before.max() / before.mean(), 2))
salt = rng.integers(0, 8, n)
after = part_sizes(keys * 8 + salt)
print("after :", after.tolist(), "max/mean =", round(after.max() / after.mean(), 2))

df = pd.DataFrame({"k": keys, "salt": salt, "v": vals})
stage1 = df.groupby(["k", "salt"])["v"].sum().reset_index()
stage2 = stage1.groupby("k")["v"].sum()
direct = df.groupby("k")["v"].sum()
print("same answer:", stage2.equals(direct))
```

```
hottest key share: 0.134
before: [247565, 101396, 117133, 71796, 152006, 160663, 60940, 88501] max/mean = 1.98
after : [138513, 128210, 121237, 106290, 161283, 138354, 91594, 114519] max/mean = 1.29
same answer: True
```

1. Key popularity is proportional to 1/rank. This heavy-tailed shape is common: a few customers, bots or null values account for most rows.
2. `part_sizes` mimics Spark's hash partitioning: hash the key, take the remainder by the number of partitions, count rows per partition.
3. Before salting, the biggest partition holds nearly twice the average (max/mean about 2). That task decides when the whole stage finishes. With 1,000 keys the ratio is mild; with a real hot key holding 50 percent of rows, it is severe.
4. After salting, the hot key's rows spread over all 8 partitions and the ratio falls. Not to exactly 1.0: other keys still hash unevenly.
5. Stage 1 sums by (key, salt), stage 2 sums the partial sums by key. Because a sum of partial sums is the total, the answer equals the direct group-by (`True`). Salting works for decomposable aggregations; for joins, the small side must be replicated across all salt values.

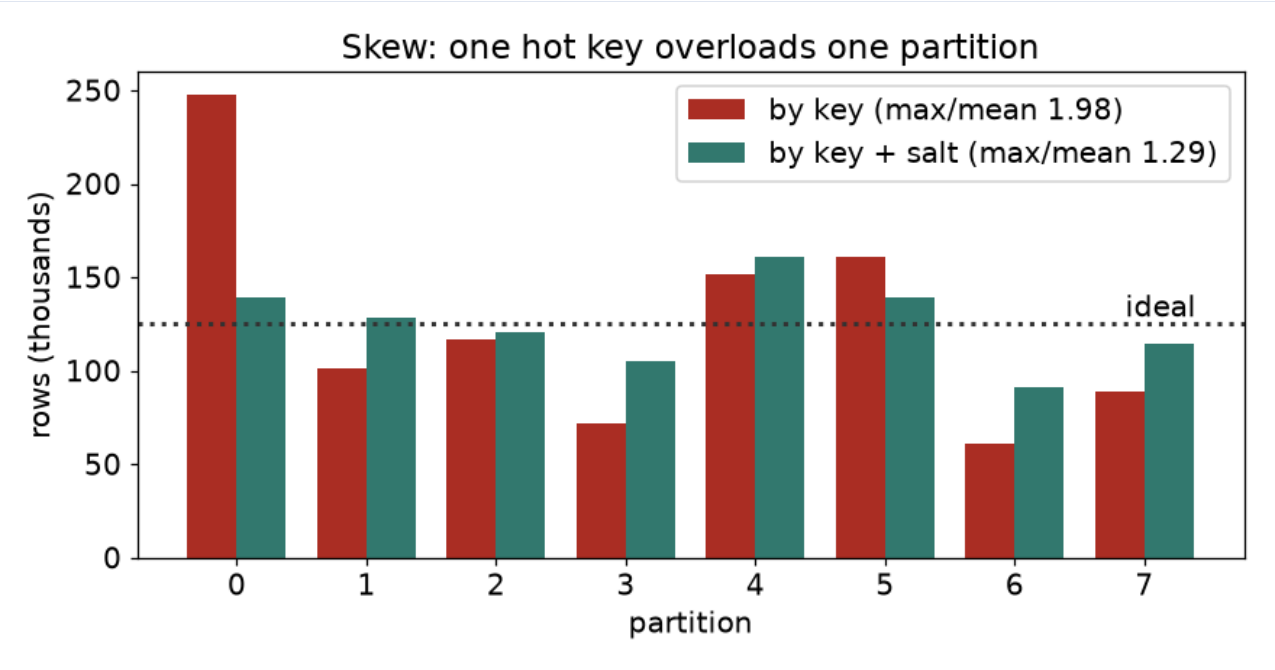


Figure 22.7. Rows per partition before and after salting, measured in the block above.

How to read it: red bars are the partitions when hashing only by key, green bars when hashing key plus salt. The dotted line is the perfect split. Partition 0 in red holds a quarter million rows because the hot key lives there. Green is closer to the line. In Spark you would first try the built-in fix, adaptive query execution, which can split skewed shuffle partitions in sort-merge joins automatically; salt by hand only when the UI still shows one monster task.

```
# NOT RUN HERE (library not installed)
from pyspark.sql import functions as F

SALT = 8
# Two-stage aggregation for a skewed group-by key
stage1 = (df.withColumn("salt", (F.rand(seed=1) * SALT).cast("int"))
            .groupBy("k", "salt").agg(F.sum("v").alias("s")))
total = stage1.groupBy("k").agg(F.sum("s").alias("v"))

# Skewed join: salt the big side, replicate the small side across all salts
big = big.withColumn("salt", (F.rand(seed=1) * SALT).cast("int"))
small = small.withColumn("salt", F.explode(F.array([F.lit(i) for i in range(SALT)])))
joined = big.join(small, ["k", "salt"])

# Often enough: let Spark do it
spark.conf.set("spark.sql.adaptive.enabled", "true")
spark.conf.set("spark.sql.adaptive.skewJoin.enabled", "true")
```

1. The first part is the Spark version of the pandas two-stage aggregate just run.
2. `F.rand(seed=1)` gives a uniform number in [0, 1); times 8 and cast to int gives a salt from 0 to 7.
3. For the join, each row of the big side gets one random salt. Each row of the small side is copied 8 times, once per salt, using `explode` on an array. The join then matches on (key, salt), so a hot key's rows split into 8 smaller joins.
4. The cost is 8 times the small side. Salt only the hot keys if the small side is not small.
5. Adaptive Query Execution (AQE) has been on by default since Spark 3.2. It coalesces tiny shuffle partitions, switches join strategy using real runtime sizes, and can split skewed partitions. Setting the flags explicitly documents your intent.

UDFs versus built-in functions

A plain Python UDF (user-defined function) is called once per row: Spark must serialize each row from the JVM to a Python process and back, and the optimizer cannot see inside it. Built-in functions (`pyspark.sql.functions`) run inside the JVM, are optimized and can be pushed down. The same

effect shows up in pandas: `df.apply(f, axis=1)` is a row-by-row Python loop, vectorized column arithmetic is the built-in. The block measures the gap on 200,000 rows (timings are not checked because they vary).

```
import time
import numpy as np
import pandas as pd

rng = np.random.default_rng(0)
n = 200_000
df = pd.DataFrame({"amount": rng.integers(1, 500, n), "region": rng.choice(["EU", "US", "APAC"], n)})
rate = {"EU": 0.92, "US": 1.0, "APAC": 0.0065}

def row_udf(row):                       # like a plain Spark UDF: called once per row
    return row["amount"] * rate[row["region"]]

def best(fn, reps=3):
    times = []
    for _ in range(reps):
        t0 = time.perf_counter(); fn(); times.append(time.perf_counter() - t0)
    return min(times)

slow = best(lambda: df.apply(row_udf, axis=1))
fast = best(lambda: df["amount"] * df["region"].map(rate))      # column-at-a-time, like F.when / F.col arithmetic
print("row-at-a-time:", round(slow, 3), "s; vectorized:", round(fast, 4), "s; ratio:", round(slow / fast))
print("same values:", np.allclose(df.apply(row_udf, axis=1), df["amount"] * df["region"].map(rate)))
```

1. `row_udf` mimics a plain UDF: a Python function receiving one row.
2. The vectorized line maps the region to its rate with a dictionary lookup (done once per distinct value internally) and multiplies whole columns.
3. On one laptop run: 0.681 s for the row-at-a-time version and 0.0063 s for the vectorized one, about 108 times faster, with identical values. In Spark the penalty is usually larger because of the JVM-to-Python serialization on top.
4. Escalation ladder in Spark: (1) a built-in function or `F.expr` SQL expression; (2) a `pandas_udf`, which receives whole pandas Series in batches via Arrow; (3) a plain UDF only when nothing else works.

```
# NOT RUN HERE (library not installed)
from pyspark.sql import functions as F
import pandas as pd

rates = {"EU": 0.92, "US": 1.0, "APAC": 0.0065}

# 1. Built-in: a map literal and column arithmetic (best)
rate_col = F.create_map([F.lit(x) for kv in rates.items() for x in kv])
df = df.withColumn("eur", F.col("amount") * rate_col[F.col("region")])

# 2. Vectorized pandas UDF: batches of rows as pandas Series
@F.pandas_udf("double")
def to_eur(amount: pd.Series, region: pd.Series) -> pd.Series:
    return amount * region.map(rates)
df = df.withColumn("eur2", to_eur("amount", "region"))

# 3. Plain UDF: one row at a time (slowest)
to_eur_slow = F.udf(lambda a, r: float(a * rates[r]), "double")
df = df.withColumn("eur3", to_eur_slow("amount", "region"))
```

Reading the Spark UI

While a job runs, the driver serves a web page, by default at `http://localhost:4040`. Learn to read five tabs: **Jobs** (one row per action), **Stages** (the units between shuffles), **Storage** (cached data), **Executors** (memory, GC, failed tasks per executor) and **SQL / DataFrame** (the query plan with row counts). Open the page of a slow stage and read its *summary metrics for completed tasks*: min, 25th percentile, median, 75th percentile and max of duration, shuffle read size, spill.

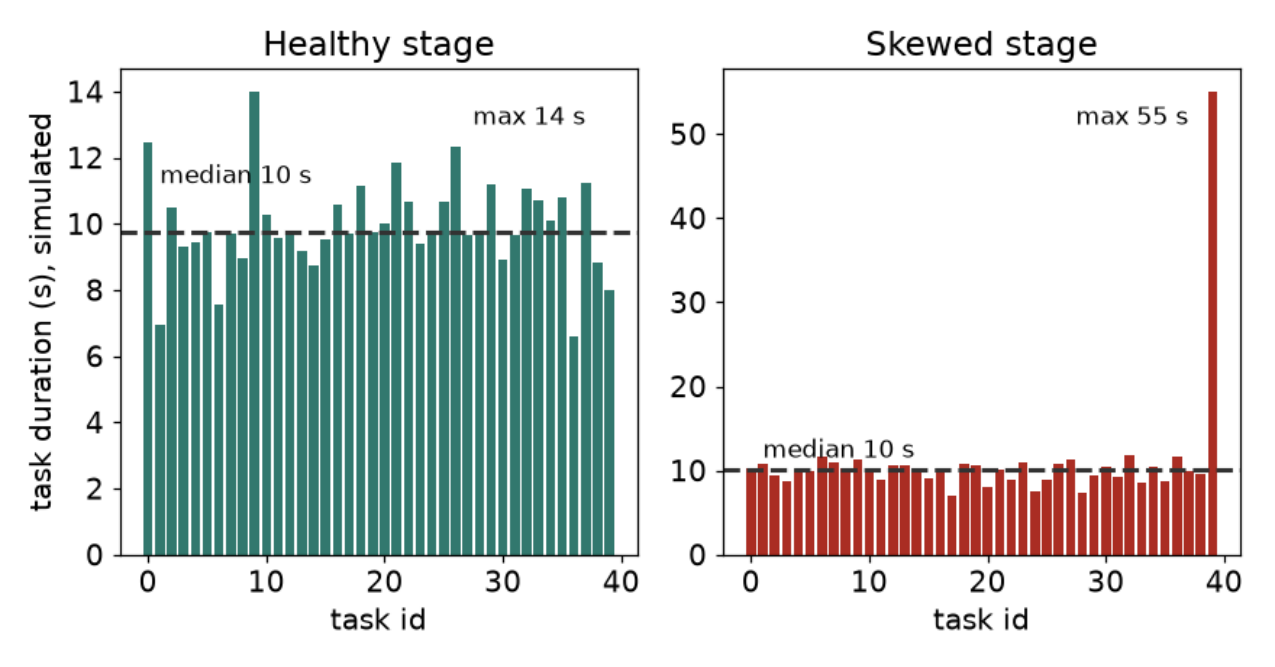


Figure 22.8. Simulated task durations in a healthy and in a skewed stage (an illustration, not a real Spark run).

How to read it: every bar is one task in the stage. The dashed line is the median. In the healthy stage all bars are about 10 seconds, so the stage ends in about 10 seconds' worth of work on each core. In the skewed stage 39 tasks finish quickly and then one task keeps running for about 55 seconds; the stage cannot end until it does. In the UI that appears as a Max duration far above the Median.

What you see in the UI	What it usually means	First fix to try
Max task duration is 5 times or more the median	Data skew, or one slow node	Check the key distribution; AQE skew join; salting
Shuffle spill (memory) and (disk) columns are non-zero	Partitions too big for execution memory	More partitions; larger executor memory
Thousands of tasks, each a few milliseconds	Too many tiny partitions	`coalesce`, lower shuffle partitions, compact small files
A few tasks, each very long, idle cores	Too few partitions	`repartition` to 2 to 3 times the core count or more
High GC time share	Memory pressure, many objects (often Python UDF data)	Fewer columns, built-ins, more memory
Huge shuffle read/write for a simple query	Unneeded wide operations	Broadcast small tables; filter earlier; drop `distinct` / `orderBy`
SQL tab: scan reads all files and all columns	No pushdown (filter after a UDF, or wrong partition column)	Filter on partition columns; select columns early
Same scan appears in two jobs	Recomputation without cache	`cache()` or write an intermediate Parquet
Executors tab shows lost executors	Out-of-memory kills	Smaller partitions; avoid `collect()`; fix skew

KEY IDEA

Debug Spark in this order: (1) `explain()` to see the plan; (2) the SQL tab to see where the rows are; (3) the slow stage's task summary for skew and spill; (4) only then change configuration. Most "tuning" is really fixing the data layout or the join order.

22.8 How to choose

Start with the size of the data and the size of your machine, then ask what you already know. The flow chart below encodes the order of questions. Follow it from the top; stop at the first tool that answers yes.

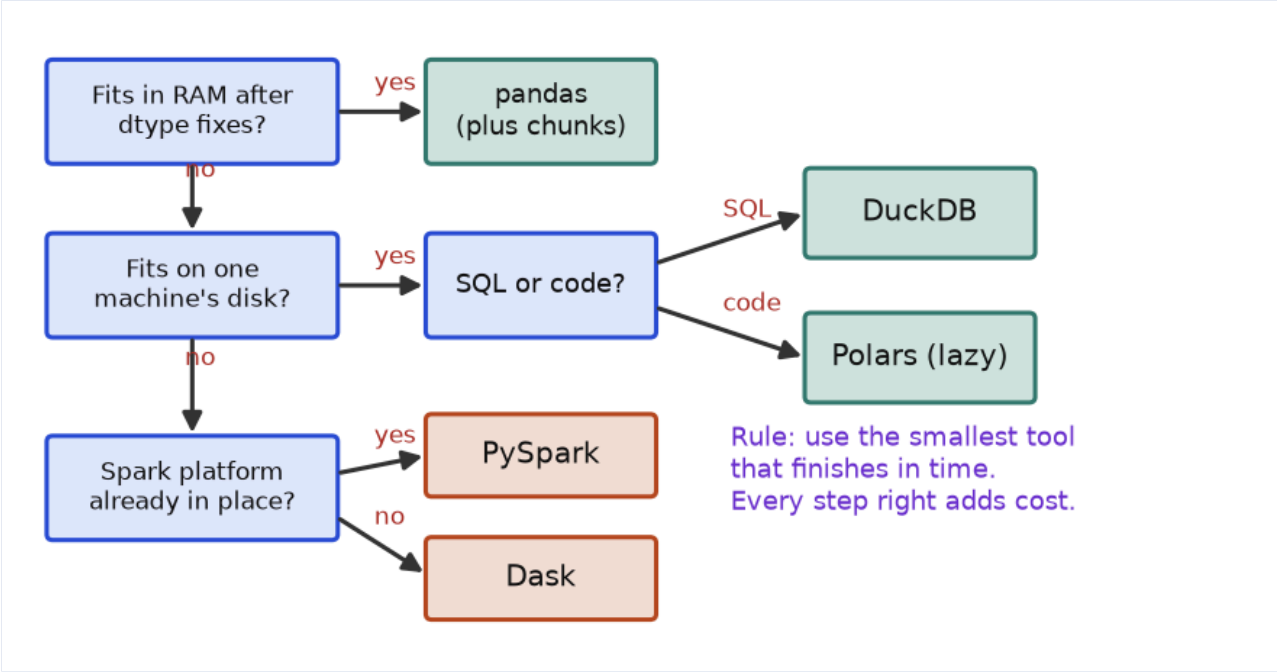


Figure 22.9. Decision flow for choosing a data library.

How to read it: blue boxes are questions, green boxes are single-machine answers, orange boxes are multi-machine answers. Notice how many paths end on one machine: a modern laptop or a single big cloud machine with DuckDB or Polars handles data far beyond what people expect, and avoids the cost of a cluster. Go right and down only when the data truly cannot be handled by one machine, or when the platform (data lake, scheduler, security) is already built on Spark.

Situation	Pick	Why
2 GB CSV, analysis in a notebook, 16 GB laptop	pandas with `usecols`, dtypes	No new tool needed
30 GB of Parquet, SQL-literate team, one workstation	DuckDB	Reads Parquet directly, spills to disk
30 GB, Python method chains, repeated transformations	Polars lazy	Optimizer, expressions, streaming
Existing 5,000 lines of pandas that must scale somewhat	Dask DataFrame	Same API, partitions
2 TB of logs in a data lake, nightly jobs, shared cluster	PySpark	Distributed shuffle, ecosystem, schedulers
Streaming ingestion with exactly-once guarantees	Spark Structured Streaming or a stream processor	Beyond this chapter

A quick way to know which row you are in: estimate the in-memory size from a sample before you load anything. The block builds a 200,000-row CSV, reads only the first 20,000 rows, and extrapolates.

```
import os, tempfile
import numpy as np
import pandas as pd

rng = np.random.default_rng(3)
n = 200_000
df = pd.DataFrame({
    "order_id": np.arange(n),
    "region": rng.choice(["EU", "US", "APAC", "LATAM", "MEA"], n),
    "qty": rng.integers(1, 20, n),
    "price": rng.normal(50, 15, n).round(2),
})
path = os.path.join(tempfile.mkdtemp(), "o.csv")
df.to_csv(path, index=False)

total_rows = sum(1 for _ in open(path)) - 1
sample = pd.read_csv(path, nrows=20_000)
per_row = sample.memory_usage(deep=True, index=False).sum() / len(sample)
estimate = per_row * total_rows
actual = pd.read_csv(path).memory_usage(deep=True, index=False).sum()
print("rows:", total_rows)
print("estimate within 5% of actual:", abs(estimate - actual) / actual < 0.05)
print("in-memory is larger than the file:", actual > os.path.getsize(path))
```

```
rows: 200000
estimate within 5% of actual: True
in-memory is larger than the file: True
```

1. Counting lines is a cheap streaming pass over the file (no parsing), giving the row count.
2. Reading the first 20,000 rows and measuring `memory_usage(deep=True)` gives bytes per row at the dtypes pandas would choose.
3. Multiplying by the row count estimates the full size. The estimate is good when the sample is representative: if the file is sorted so that long strings appear only at the end, the sample under-estimates.
4. For this table the in-memory size exceeds the CSV file size, mostly because strings are bulky as Python objects. A common surprise: a 10 GB CSV can need 20 GB or more of RAM in pandas, plus working memory for every operation.

REAL JOB

In interviews and design reviews, defend the choice with numbers: data size, growth rate, how many people run the job, how often, and the cost of the platform. "We used Spark because it is big data" is a weak answer. "Our largest table is 40 GB, DuckDB on one 64 GB machine finished in 3 minutes, so we skipped the cluster" is a strong one.

22.9 Recipes: How do I ...?

Every recipe below follows the same shape: the task, a runnable pandas or standard-library solution with a check, and the equivalent in the big libraries where it differs. All files are generated in temporary folders.

How do I process a 10 GB CSV in chunks?

Stream the file, clean each chunk, update small running totals, and never hold more than one chunk. This block makes a dirty 300,000-row file (lowercase region names, missing prices, negative quantities) and aggregates it in 50,000-row chunks.

```
import os, tempfile
import numpy as np
import pandas as pd

rng = np.random.default_rng(11)
n = 300_000
raw = pd.DataFrame({
    "region": rng.choice(["eu", "US", " apac", "EU"], n),
    "qty": rng.integers(-2, 10, n),
    "price": np.where(rng.random(n) < 0.05, np.nan, rng.normal(40, 10, n).round(2)),
})
path = os.path.join(tempfile.mkdtemp(), "dirty.csv")
raw.to_csv(path, index=False)

revenue, kept, dropped = {}, 0, 0
for chunk in pd.read_csv(path, chunksize=50_000):
    chunk["region"] = chunk["region"].str.strip().str.upper()
    ok = chunk["qty"].gt(0) & chunk["price"].notna()
    dropped += int((~ok).sum())
    good = chunk[ok]
    kept += len(good)
    for region, value in (good["qty"] * good["price"]).groupby(good["region"]).sum().items():
        revenue[region] = revenue.get(region, 0.0) + value

print("kept", kept, "dropped", dropped, "total", kept + dropped)
print({k: round(v) for k, v in sorted(revenue.items())})

whole = raw.assign(region=raw.region.str.strip().str.upper())
whole = whole[whole.qty.gt(0) & whole.price.notna()]
check = (whole.qty * whole.price).groupby(whole.region).sum()
print("matches one-shot pandas:", all(abs(check[k] - revenue[k]) < 1e-6 * check[k] for k in check.index))
```

```
kept 213790 dropped 86210 total 300000
{'APAC': 10740859, 'EU': 21399204, 'US': 10654963}
matches one-shot pandas: True
```

1. The generator plants three kinds of dirt: inconsistent region spellings, missing prices (about 5 percent) and non-positive quantities.
2. For each chunk, `str.strip().str.upper()` normalizes the region, then a boolean mask keeps only valid rows. Counting the dropped rows is part of any honest pipeline: you must report how many rows you discarded.
3. The running totals live in a plain dictionary: small, and independent of file size.
4. The last block recomputes everything in one shot on the in-memory frame and compares. Chunked and one-shot agree to floating-point accuracy.
5. In Polars the equivalent is a lazy scan with the same filters and a `group_by`, and the engine does the chunking for you: `pl.scan_csv(p).filter((pl.col("qty") > 0) & pl.col("price").is_not_null()).group_by(pl.col("region").str.strip_chars().str.to_uppercase()).agg((pl.col("qty") * pl.col("price")).sum()).collect(engine="streaming")` (not run here).

How do I dedupe at scale?

A hash set of all keys may not fit in memory. Use the shuffle idea: write rows into B bucket files by hash of the key, so every copy of the same key lands in the same bucket; then dedupe each bucket on its own, which fits in memory.

```
import os, tempfile
import numpy as np
import pandas as pd

rng = np.random.default_rng(21)
n = 200_000
df = pd.DataFrame({
    "order_id": rng.integers(0, 120_000, n),          # many repeated ids
    "ts": np.arange(n),                               # later row = newer version
    "status": rng.choice(["new", "paid", "shipped"], n),
})
d = tempfile.mkdtemp()
src = os.path.join(d, "orders.csv")
df.to_csv(src, index=False)

B = 8
for chunk in pd.read_csv(src, chunksize=40_000):                      # pass 1: scatter
    bucket = pd.util.hash_pandas_object(chunk["order_id"], index=False) % B
    for b, part in chunk.groupby(bucket):
        part.to_csv(os.path.join(d, f"bucket{b}.csv"), mode="a", header=not os.path.exists(os.path.join(d,
f"bucket{b}.csv")), index=False)

kept = []
for b in range(B):                                                     # pass 2: dedupe each bucket
    part = pd.read_csv(os.path.join(d, f"bucket{b}.csv"))
    kept.append(part.sort_values("ts").drop_duplicates("order_id", keep="last"))
result = pd.concat(kept)

truth = df.sort_values("ts").drop_duplicates("order_id", keep="last")
print("rows in:", n, "unique ids:", len(result), "expected:", len(truth))
print("same rows:",
result.sort_values("order_id").reset_index(drop=True).equals(truth.sort_values("order_id").reset_index(drop=True)))
print("largest bucket rows:", max(len(k) for k in kept) < n / 2)
```

```
rows in: 200000 unique ids: 97410 expected: 97410
same rows: True
largest bucket rows: True
```

1. The data has 200,000 rows but only about 97,000 distinct ids, each possibly repeated with different timestamps. We keep the newest row per id.
2. Pass 1 reads chunks and appends each row to `bucketK.csv`, where K is the stable hash of the id modulo 8. The header is written only when the bucket file is first created.
3. Pass 2 loads one bucket at a time. All copies of an id are in the same bucket, so sorting by `ts` and `drop_duplicates(keep="last")` gives the right answer locally.
4. The result equals a global dedupe on the in-memory table. Memory use is one bucket, which you control by raising B.
5. This is exactly what Spark's `dropDuplicates` does (shuffle by key, dedupe within partitions). For "keep the latest", use a window: `row_number() over (partition by order_id order by ts desc)` and keep 1, in Spark, DuckDB (`QUALIFY`) or Polars (`df.sort("ts").unique(subset="order_id", keep="last")`).

How do I aggregate incrementally as new files arrive?

Keep a small state (sums and counts, never means) and a list of processed files. Each run adds only the new files, and a rerun does nothing.

```
import json, os, tempfile
import numpy as np
import pandas as pd

d = tempfile.mkdtemp()
state_path = os.path.join(d, "state.json")
rng = np.random.default_rng(8)
for day in (1, 2):
    pd.DataFrame({"region": rng.choice(["EU", "US"], 1000), "amount": rng.integers(1, 100, 1000)}).to_csv(os.path.join(d,
f"day{day}.csv"), index=False)

def update(folder):
    state = json.load(open(state_path)) if os.path.exists(state_path) else {"files": [], "sum": {}, "count": {}}
    for name in sorted(f for f in os.listdir(folder) if f.startswith("day")):
        if name in state["files"]:
            continue                                        # already processed: idempotent
        part = pd.read_csv(os.path.join(folder, name)).groupby("region")["amount"].agg(["sum", "count"])
        for region, row in part.iterrows():
            state["sum"][region] = state["sum"].get(region, 0) + int(row["sum"])
            state["count"][region] = state["count"].get(region, 0) + int(row["count"])
        state["files"].append(name)
    json.dump(state, open(state_path, "w"))
    return state

s1 = update(d)
print("after run 1:", s1["files"], s1["count"])
s2 = update(d)
print("after rerun:", s2["files"], s2["count"] == s1["count"])
pd.DataFrame({"region": rng.choice(["EU", "US"], 500), "amount": rng.integers(1, 100, 500)}).to_csv(os.path.join(d,
"day3.csv"), index=False)
s3 = update(d)
print("after day 3:", s3["files"], sum(s3["count"].values()))
print("mean EU:", round(s3["sum"]["EU"] / s3["count"]["EU"], 2))
```

```
after run 1: ['day1.csv', 'day2.csv'] {'EU': 1010, 'US': 990}
after rerun: ['day1.csv', 'day2.csv'] True
after day 3: ['day1.csv', 'day2.csv', 'day3.csv'] 2500
mean EU: 49.49
```

1. The state holds, per region, the running sum and count, so the mean can be computed at any time without the raw data.
2. Processed file names are stored in the state. Each run skips any file already in the list, which makes the job **idempotent**: running it twice changes nothing. This matters when a scheduler retries a failed night.
3. Run 1 processes two files with 1,000 rows each. The rerun reads nothing new and the counts are identical (`True`).
4. When day 3 arrives, only that file is read, and the total count becomes 2,500.
5. In Spark or Delta-style systems the same pattern is called an incremental load, with a checkpoint or watermark in place of the JSON file. Write the state and the outputs atomically (write a temporary file and rename) so a crash cannot leave half an update.

How do I join a big table with a small one?

Load the small table into memory and look up values chunk by chunk. This is a **broadcast join**: the small side is copied to wherever the big side is processed, so the big side never moves. The block checks the answer with a real SQL join in `sqlite3` .

```
import io, os, sqlite3, tempfile
import numpy as np
import pandas as pd

rng = np.random.default_rng(4)
n = 200_000
big = pd.DataFrame({"customer_id": rng.integers(0, 1_000, n), "amount": rng.integers(1, 100, n)})
small = pd.DataFrame({"customer_id": np.arange(0, 900), "tier": rng.choice(["gold", "silver", "bronze"], 900)})   # 100
customers unknown
path = os.path.join(tempfile.mkdtemp(), "big.csv")
big.to_csv(path, index=False)

lookup = small.set_index("customer_id")["tier"]
totals, unmatched = {}, 0
for chunk in pd.read_csv(path, chunksize=50_000):
    tier = chunk["customer_id"].map(lookup)                   # hash lookup, no shuffle
    unmatched += int(tier.isna().sum())
    for t, v in chunk["amount"].groupby(tier.fillna("unknown")).sum().items():
        totals[t] = totals.get(t, 0) + int(v)
print(dict(sorted(totals.items())), "unmatched rows:", unmatched)

con = sqlite3.connect(":memory:")
big.to_sql("big", con, index=False); small.to_sql("small", con, index=False)
rows = con.execute("""SELECT COALESCE(s.tier, 'unknown') AS tier, SUM(b.amount)
                      FROM big b LEFT JOIN small s ON s.customer_id = b.customer_id
                      GROUP BY 1 ORDER BY 1""").fetchall()
print("sqlite3 agrees:", dict(rows) == totals)
```

```
{'bronze': 2996573, 'gold': 3034570, 'silver': 2974233, 'unknown': 1011012} unmatched rows: 20123
sqlite3 agrees: True
```

1. The big table has customers 0 to 999 but the small table knows only 0 to 899, so about a tenth of the rows will not match. This is deliberate: real joins always have unmatched keys.
2. `small.set_index(...)["tier"]` is a Series that acts as a dictionary. `chunk["customer_id"].map(lookup)` gives the tier for each row, NaN where unknown.
3. The unmatched rows are counted and grouped as `unknown`. A left join keeps them; an inner join would silently drop about 10 percent of the data. Always check the unmatched count after a join.
4. The SQL `LEFT JOIN` in sqlite3 gives identical totals per tier.
5. Spark: `big.join(F.broadcast(small), "customer_id", "left")`. Polars: `big_lf.join(small_lf, on="customer_id", how="left")` (the small side is held in memory). DuckDB: the same SQL, and its hash join builds the table on the smaller side.

How do I write a partitioned dataset from a stream of chunks?

Append each chunk's rows into the right partition folder as you go, so the whole dataset never sits in memory. The block uses CSV files because Parquet is unavailable; the layout logic is identical to Parquet.

```
import os, tempfile
from pathlib import Path
import numpy as np
import pandas as pd

rng = np.random.default_rng(6)
n = 120_000
src = os.path.join(tempfile.mkdtemp(), "events.csv")
pd.DataFrame({
    "ts": pd.to_datetime("2025-01-01") + pd.to_timedelta(rng.integers(0, 90, n), unit="D"),
    "country": rng.choice(["DE", "FR", "US"], n),
    "value": rng.integers(1, 1000, n),
}).to_csv(src, index=False)

root = Path(tempfile.mkdtemp()) / "events"
rows_written = 0
for chunk in pd.read_csv(src, chunksize=30_000, parse_dates=["ts"]):
    chunk["month"] = chunk["ts"].dt.strftime("%Y-%m")
    for (month, country), part in chunk.groupby(["month", "country"]):
        folder = root / f"month={month}" / f"country={country}"
        folder.mkdir(parents=True, exist_ok=True)
        target = folder / "part-0.csv"
        part[["ts", "value"]].to_csv(target, mode="a", header=not target.exists(), index=False)
        rows_written += len(part)

files = sorted(root.rglob("*.csv"))
print("folders:", len({p.parent for p in files}), "rows written:", rows_written == n)
print(files[0].relative_to(root))
only = pd.read_csv(root / "month=2025-02" / "country=DE" / "part-0.csv")
print("one partition rows:", len(only), "of", n)
```

```
folders: 9 rows written: True
month=2025-01/country=DE/part-0.csv
one partition rows: 12355 of 120000
```

1. Dates are spread over 90 days from 1 January, so months 2025-01, 2025-02 and 2025-03 appear; there are 3 countries, so 9 folders.
2. Each chunk is split by (month, country). The target file is opened in append mode and the header is written only when the file does not exist yet.
3. Only `ts` and `value` are written inside the files, since month and country are in the folder names.
4. The counters prove that no row was lost. Reading a single partition touches about a tenth of the rows.
5. Parquet equivalent (not run): in PyArrow use `pq.write_to_dataset(table, root, partition_cols=["month", "country"])` for each chunk, or in one statement in DuckDB: `COPY (SELECT *, strftime(ts, '%Y-%m') AS month FROM 'events.csv') TO 'events' (FORMAT PARQUET, PARTITION_BY (month, country))`. Spark: `df.write.partitionBy("month", "country").parquet("events")`.
6. Watch out for the small-files problem: appending every chunk as a new Parquet file makes thousands of files. Prefer one big conversion job, or compact files afterwards.

More "How do I ...?" one-liners (not run here)

How do I ...?	Polars	DuckDB SQL	PySpark
Convert CSV to Parquet	`pl.scan_csv(p).sink_parquet(q)`	`COPY (SELECT * FROM 'p.csv') TO 'q.parquet' (FORMAT PARQUET)`	`spark.read.csv(p, header=True).write.parquet(q)`
Read many files at once	`pl.scan_parquet("d/*.parquet")`	`FROM read_parquet('d/*.parquet')`	`spark.read.parquet("d/")`

How do I ...?	Polars	DuckDB SQL	PySpark
Top 3 per group	`df.sort("v", descending=True).group_by("g").head(3)`	`QUALIFY ROW_NUMBER() OVER (PARTITION BY g ORDER BY v DESC) <= 3`	`filter(F.row_number().over(w) <= 3)` on a withColumn
Latest record per key	`df.sort("ts").unique("k", keep="last")`	`SELECT k, arg_max(status, ts) FROM t GROUP BY k`	window `row_number` ordered by `ts` desc, keep 1
Count distinct, approximately	`pl.col("u").approx_n_unique()`	`approx_count_distinct(u)`	`F.approx_count_distinct("u")`
Take a 1 percent sample	`df.sample(fraction=0.01, seed=0)`	`SELECT * FROM t USING SAMPLE 1%`	`df.sample(fraction=0.01, seed=0)`
Median per group	`pl.col("v").median()` in `agg`	`quantile_cont(v, 0.5)`	`F.percentile_approx("v", 0.5)`
Rolling 7-day sum	`pl.col("v").rolling_sum_by("ts", window_size="7d")`	`SUM(v) OVER (ORDER BY ts RANGE BETWEEN INTERVAL 6 DAYS PRECEDING AND CURRENT ROW)`	window with `rangeBetween` on a numeric time column
Find rows with no match	`a.join(b, on="k", how="anti")`	`a ANTI JOIN b USING (k)`	`a.join(b, "k", "left_anti")`
Peek at the schema cheaply	`pl.scan_csv(p).collect_schema()`	`DESCRIBE SELECT * FROM 'p.csv'`	`spark.read.csv(p, header=True).printSchema()`

Count so far: five fully worked runnable recipes and ten more one-liners, fifteen "How do I" tasks in total.

22.10 Performance and pitfalls

These are the mistakes that cost the most hours. Each box gives the wrong code, the symptom you will see, and the fix.

COMMON MISTAKE 1: LOADING EVERYTHING FIRST

Wrong: `df = pd.read_csv("events.csv")` for a file larger than a third of RAM. **Symptom:** frozen notebook, swapping, or a killed kernel with no traceback. **Fix:** measure first (Section 22.8), then `usecols` and `dtype`, or `chunksize`, or `pl.scan_csv` / DuckDB on the file.

COMMON MISTAKE 2: ROW-BY-ROW PYTHON

Wrong: `df.apply(f, axis=1)`, `df.iterrows()`, or a Spark `F.udf` for arithmetic. **Symptom:** a step that takes minutes on 200,000 rows (we measured about 100 times slower than the vectorized form). **Fix:** column expressions, `map` with a dictionary, `np.where`, Polars expressions, Spark built-ins, then `pandas_udf`.

COMMON MISTAKE 3: PULLING BIG RESULTS TO THE DRIVER

Wrong: `sdf.toPandas()`, `sdf.collect()` or `ddf.compute()` on a result as big as the input. **Symptom:** driver out-of-memory error after the cluster finished all the work. **Fix:** aggregate or `limit` first, or write to Parquet and read the part you need.

COMMON MISTAKE 4: AVERAGING AVERAGES

Wrong: combining per-partition means with an unweighted mean. **Symptom:** totals do not match the single-machine answer (we showed it in Section 22.6). **Fix:** keep sum and count per partition and divide at the end; for medians use an approximate quantile and say so.

COMMON MISTAKE 5: A MILLION TINY FILES

Wrong: `df.write.partitionBy("customer_id").parquet(...)`, or writing a file per chunk or per minute. **Symptom:** a listing that takes minutes, thousands of 5 ms tasks, slow reads. **Fix:** partition by low-cardinality columns (date, country), then compact to files of about 100 MB to 1 GB.

COMMON MISTAKE 6: DEFAULT SHUFFLE PARTITIONS

Wrong: leaving `spark.sql.shuffle.partitions` at 200 for a 50 MB job or for a 5 TB job. **Symptom:** 200 near-empty tasks, or 200 giant tasks that spill to disk. **Fix:** aim for partitions of 100 to 200 MB after the shuffle; check the Stages page; let AQE coalesce small ones.

COMMON MISTAKE 7: IGNORING SKEW

Wrong: joining on a key where 20 percent of rows are `NULL` or one bot id. **Symptom:** 199 tasks finish in seconds, one runs for an hour, executors idle. **Fix:** filter or isolate the hot keys, enable AQE skew join, or salt (Section 22.7).

COMMON MISTAKE 8: CACHING BLINDLY, OR NEVER

Wrong: `df.cache()` on everything, or calling three actions on an expensive plan with no cache. **Symptom:** executors fill with cached data and evict useful blocks; or the same scan appears three times in the Jobs tab. **Fix:** cache only DataFrames reused by several actions, trigger them with one action, `unpersist()` after; or write a checkpoint Parquet file.

COMMON MISTAKE 9: UNSTABLE HASHING

Wrong: routing rows to buckets with Python's built-in `hash(key)` for strings. **Symptom:** the same key lands in different buckets in different processes or runs; duplicates survive dedupe, joins miss matches. **Fix:** a stable hash such as `pd.util.hash_pandas_object`, `hashlib` or the engine's own partitioner.

COMMON MISTAKE 10: SCHEMA DRIFT AND GUESSED TYPES

Wrong: `inferSchema=True` on a huge CSV, or appending a file where `id` is a string to a folder where it is an integer. **Symptom:** an extra full pass, ids turning into floats like `1.0e7`, or "cannot merge incompatible data types" at read time. **Fix:** an explicit schema, ids as strings, types written into Parquet, and a schema check at ingestion.

COMMON MISTAKE 11: NULL KEYS AND SILENT ROW LOSS

Wrong: an inner join where one side has missing keys, or `NaN` treated like `null`. **Symptom:** row counts drop without an error, or `NULL = NULL` never matches. **Fix:** count rows before and after every join, use `left` joins when you must keep all rows, report unmatched keys, and learn your library's null rules (in Polars `NaN` and `null` are different).

COMMON MISTAKE 12: A GLOBAL SORT YOU DID NOT NEED

Wrong: `df.orderBy("ts")` or `ddf.set_index("ts")` in the middle of a pipeline. **Symptom:** an extra full shuffle and a stage that dominates the run. **Fix:** sort only for final output or for window functions that need it; use `sortWithinPartitions`, or sort when writing.

COMMON MISTAKE 13: TRUSTING FLOAT SUMS ACROSS ENGINES

Wrong: asserting `spark_total == pandas_total` on floats. **Symptom:** the check fails by 1e-9, or flips between runs, because partitions add in a different order. **Fix:** compare with a tolerance (`np.isclose`) or sum integer cents.

Quick performance checklist

1. Read only needed columns; filter as early as possible; prefer Parquet over CSV.
2. Use the smallest dtypes that are safe; categories for repeated strings.
3. Prefer built-in column operations to Python functions.
4. Reduce data before a join (filter, select, pre-aggregate); broadcast small tables.
5. Count shuffles in the plan; remove the ones you can.
6. Check for skew and for tiny or giant partitions before touching configuration.
7. Time one stage at a time on a sample, then scale up and measure again; trust measurements over folklore.

22.11 Cross-library integration

You will often mix tools: aggregate in DuckDB or Spark, then finish in pandas, then plot or model. The rule is to move *small* things between libraries and leave big things on disk as Parquet.

From to	How (not run here)	Cost
pandas to Polars	`pl.from_pandas(df)`	Needs pyarrow; copies unless dtypes map directly
Polars to pandas	`lf.collect().to_pandas()`	Needs pyarrow; the whole result must fit in RAM
pandas to DuckDB	`duckdb.sql("SELECT ... FROM df")`	Reads the DataFrame in place, no copy of the data into a database
DuckDB to pandas / Polars / Arrow	`.df()`, `.pl()`, `.arrow()` (accessor name varies by version)	Cost proportional to the result size
Dask to pandas	`ddf.compute()`	The result becomes one pandas object
pandas to Dask	`dd.from_pandas(df, npartitions=8)`	Splits in memory
Spark to pandas	`sdf.toPandas()` after `spark.conf.set("spark.sql.execution.arrow.pyspark.enabled", "true")`	Collects to the driver; faster with Arrow
pandas to Spark	`spark.createDataFrame(df)`	Serializes through the driver: for small data
Anything to anything, big data	Write Parquet, read Parquet	Zero shared memory, no size limit, fully reproducible
Spark with the pandas API	`sdf.pandas_api()` (module `pyspark.pandas`)	Familiar syntax; some operations are shuffles in disguise

The scikit-learn side matters too. Most models need all training rows in memory, so you either aggregate or sample before training, or use an estimator with `partial_fit` that learns from one chunk at a time. The block trains a linear model on 200,000 rows in 20,000-row chunks and compares it with a full fit.

```
import numpy as np
from sklearn.linear_model import LinearRegression, SGDRegressor
from sklearn.preprocessing import StandardScaler

rng = np.random.default_rng(0)
n = 200_000
X = rng.normal(size=(n, 3))
y = X @ np.array([3.0, -2.0, 0.5]) + 10 + rng.normal(0, 1, n)

full = LinearRegression().fit(X, y)

scaler = StandardScaler().fit(X[:20_000])                # in practice: fit the scaler on one chunk or a sample
model = SGDRegressor(learning_rate="constant", eta0=0.001, random_state=0)
for epoch in range(3):
    for start in range(0, n, 20_000):
        sl = slice(start, start + 20_000)
        model.partial_fit(scaler.transform(X[sl]), y[sl])

pred_full = full.predict(X[:2000])
pred_sgd = model.predict(scaler.transform(X[:2000]))
print("full coefficients:", np.round(full.coef_, 2))
print("streamed predictions close to full fit:", bool(np.abs(pred_full - pred_sgd).max() < 0.3))
```

```
full coefficients: [ 3.  -2.   0.5]
streamed predictions close to full fit: True
```

1. The synthetic data has known coefficients 3, -2 and 0.5 and intercept 10, plus noise.
2. `LinearRegression().fit(X, y)` needs the whole matrix at once; this is the reference answer. It recovers the true coefficients to two decimals.
3. The scaler is fitted on the first chunk only, which is what you do when the whole data cannot be scanned twice. Scaling matters because SGD is sensitive to feature scale.
4. `partial_fit` updates the model with one chunk and remembers its state. Three passes (epochs) over the chunks let it converge.
5. The streamed model's predictions agree with the exact fit to within 0.3 on the first 2,000 rows, close but not identical: SGD is approximate. Other estimators with `partial_fit` include `MiniBatchKMeans`, `SGDClassifier`, `MultinomialNB` and `IncrementalPCA`.

- **Plotting:** never send 100 million points to matplotlib. Aggregate to bins or a few thousand points (a group-by or a histogram in the engine), then plot.
- **SQL and pandas:** `pd.read_sql(q, con, chunksize=50_000)` streams a database result; for real warehouses push the aggregation into SQL.
- **joblib and multiprocessing:** for embarrassingly parallel work on a few big files (one file per worker), a `joblib.Parallel` over files is often enough before you need Dask.

22.12 How to solve any big-data and fast-dataframe problem: the playbook

Step 0: the questions to ask in order

1. **How big, really?** Rows, columns, bytes on disk, estimated bytes in memory (Section 22.8).
2. **What is the answer's size?** A report with 50 rows from 5 billion rows is an aggregation problem; a full-size output is a transformation problem and needs a writable sink.
3. **Which operation is expensive?** Narrow steps are cheap. Joins, group-bys, distincts and sorts are the costs.
4. **Who else uses it, and how often?** A one-off analysis tolerates minutes; a nightly job needs idempotence and monitoring.
5. **How will I know it is right?** Decide the check before you scale up.

Eight recurring problem patterns

#	Pattern and symptom	Fix	Solved example
1	Does not fit in RAM; kernel dies on load	Read less, smaller dtypes, chunks, or a lazy engine	Problem 22.2.1 and 22.2.2: 84 MB to 12 MB; peak memory set by the chunk size
2	Big group-by or counts	Partial aggregates (sum, count), then combine	Section 22.6, sum/count versus mean of means; recipe "incremental aggregation"
3	Big table joined to a small one	Broadcast: look up in memory, no shuffle	Recipe "join a big table with a small one": `map` + SQL check
4	Big table joined to a big table, or dedupe at scale	Hash-partition both sides on the key, join or dedupe each bucket	Section 22.6 shuffle join; recipe "dedupe at scale"
5	One task or one partition far slower than the rest	Isolate or filter hot keys; AQE; salting	Section 22.7: max/mean 1.98 to 1.29, same sums
6	The same expensive result is recomputed	`cache` / `persist` , or write an intermediate Parquet checkpoint	Section 22.7 broadcast + cache example
7	A step is slow although the cluster is idle	Replace Python UDFs and row loops with column expressions	Section 22.7: about 100 times in the pandas analogue
8	Slow listing, thousands of tiny tasks, slow reads	Partition by low-cardinality columns; compact files; Parquet instead of CSV	Problems 22.3.1 and 22.3.2; recipe "partitioned dataset from chunks"

How to verify a result

Never ship a scaled-up pipeline without comparing it with a trusted slow version on a sample. Check row counts, key uniqueness, null counts, and totals (with a tolerance for floats). The helper below does that, and the demo shows it catching a planted bug (an inner join that silently drops rows).

```
import numpy as np
import pandas as pd

def compare(trusted, candidate, keys, value_cols):
    """Return a list of human-readable differences (empty list means they agree)."""
    problems = []
    if len(trusted) != len(candidate):
        problems.append(f"row count {len(trusted)} vs {len(candidate)}")
    if candidate.duplicated(keys).any():
        problems.append("duplicate keys in candidate")
    merged = trusted.merge(candidate, on=keys, how="outer", suffixes=("_t", "_c"), indicator=True)
    only = merged["_merge"].value_counts()
    if only.get("left_only", 0) or only.get("right_only", 0):
        problems.append(f"keys only in trusted: {only.get('left_only', 0)}, only in candidate: {only.get('right_only',
0)}")
    both = merged[merged["_merge"] == "both"]
    for c in value_cols:
        if not np.allclose(both[c + "_t"], both[c + "_c"]):
            problems.append(f"values differ in {c}")
    return problems

orders = pd.DataFrame({"cid": [1, 2, 3, 4], "amount": [10.0, 20.0, 30.0, 40.0]})
tiers = pd.DataFrame({"cid": [1, 2, 3], "tier": ["g", "s", "g"]})           # customer 4 unknown
trusted = orders.merge(tiers, on="cid", how="left").groupby("cid", as_index=False)["amount"].sum()
buggy = orders.merge(tiers, on="cid", how="inner").groupby("cid", as_index=False)["amount"].sum()
print("good:", compare(trusted, trusted.copy(), ["cid"], ["amount"]))
print("bug :", compare(trusted, buggy, ["cid"], ["amount"]))
```

```
good: []
bug : ['row count 4 vs 3', 'keys only in trusted: 1, only in candidate: 0']
```

1. `compare` checks four things in turn: row counts, duplicate keys, keys present on only one side, and values on the shared keys using `np.allclose`.
2. The outer merge with `indicator=True` adds a `_merge` column saying whether each key is left-only, right-only or in both. It is the quickest way to see which keys went missing.
3. The trusted version uses a left join and keeps customer 4. The buggy version uses an inner join and drops the unknown customer, so the totals lose 40 and a row.
4. The good comparison prints an empty list. The buggy one reports the row-count mismatch and the key that exists only in the trusted result, which tells you where to look.

How to debug when it is slow or crashing

1. **Shrink the input** (a sample, one partition, the first million rows) until the problem reproduces in seconds.
2. **Look at the plan** before the clock: Polars `explain()`, DuckDB `EXPLAIN ANALYZE`, Spark `explain("formatted")`, Dask `visualize()`.
3. **Bisect the pipeline**: run it step by step, forcing each with `collect()` or `count()`, and time each. The slowest step is usually obvious.
4. **Check data shape**: key distribution (`value_counts().head()`), null share, partition sizes.
5. **Read the right monitor**: Spark UI stage task summary, Dask dashboard memory bars, `htop` or Activity Monitor for a local engine (one busy core means no parallelism, all cores with high memory means spill risk).
6. **Change one thing at a time** and keep the measurement next to the change.

22.13 Going deeper: what is under the hood, and what interviewers probe

Memory formats

- **Apache Arrow** lays a column out as contiguous buffers: a values buffer, an optional validity bitmap (one bit per row saying null or not), and for strings an offsets buffer plus a bytes buffer. Contiguous, typed memory is what makes vectorized CPU code and zero-copy sharing between libraries possible.
- **Parquet encodings**: dictionary encoding (store distinct values once, codes per row), run-length and bit-packing for the codes, delta encoding for sorted numbers; then a general compressor per page. Low-cardinality and sorted columns compress extremely well, which is why sorting before writing helps.
- **pandas 3.0**: copy-on-write is the default, so derived objects share memory until one is modified, which removes many hidden copies and makes chained assignment safe by construction; text columns default to the `str` dtype (backed by PyArrow when it is installed). Even so, a typical pandas operation needs more than one copy of the data in the worst case, which is why you size RAM generously relative to the file.

Execution models

- **DuckDB** processes vectors of values per operator and parallelizes with small work units shared by threads; large hash tables and sorts can spill to disk.
- **Polars** turns lazy plans into optimized physical plans (projection and predicate pushdown, common subexpression elimination) and has a streaming engine for larger-than-memory work, though not every operation is streamable; unsupported ones fall back to in-memory execution.
- **Dask** builds a graph of small tasks and a scheduler runs them; overhead is per task, so partition size is the main tuning knob.
- **Spark** converts your DataFrame code into a logical plan, optimizes it (the Catalyst optimizer), chooses physical operators (broadcast or sort-merge join), and generates JVM code for pipelines of narrow operations (whole-stage code generation). Adaptive query execution revises the plan at stage boundaries using real statistics.

Questions interviewers like

Question	Strong answer, in one breath
Why is Parquet faster than CSV?	Columnar (read only needed columns), typed with a schema (no parsing guesses), compressed with encodings, with min/max statistics that let readers skip row groups.
What is a shuffle and why is it expensive?	Redistributing rows by key across partitions; it writes to disk, crosses the network and creates a stage barrier. Avoid with broadcast joins, pre-aggregation and fewer wide operations.
Narrow versus wide transformation?	Narrow: each output partition depends on one input partition. Wide: depends on many, so a shuffle.
How do you handle skew?	Detect it (max task time versus median), isolate or filter hot keys, AQE, salting with a two-stage aggregation or a replicated small side.
Lazy versus eager evaluation?	Lazy builds a plan the optimizer can rewrite (pushdown, reordering) and runs on demand; eager runs each step immediately and is easier to debug.
UDF versus built-in?	Built-ins are optimized and run in the engine; plain UDFs cross a process boundary per row. Use built-ins, then pandas UDFs, then plain UDFs.
When not to use Spark?	When the data fits one machine: DuckDB or Polars is simpler and often faster; Spark startup and shuffles are overhead.
How would you dedupe 5 billion rows?	Hash-partition by key, dedupe within partitions with a window or `dropDuplicates`; if exactness is not needed, a Bloom filter or approximate counts.

Chapter recap

- Pandas is limited by memory (the whole table in RAM, often several times the file size) and by mostly one core. Measure before you change tools.
- Cheap fixes first: read fewer columns, downcast integers, use `category` for repeated strings, and read in chunks. Our 1-million-row table went from 84 MB to 12 MB.
- Chunking sets memory by chunk size, not file size, but only works for decomposable statistics: sums, counts, min, max, and means as sum over count.
- Parquet is columnar, typed and compressed, with statistics. Partitioned folders (`year=2025/region=EU`) allow partition pruning.
- Polars: expressions, `group_by().agg()` , `over()` windows, and lazy `scan_*` + `collect()` with pushdown. Use `explain()` .
- DuckDB: SQL on files and DataFrames, window functions, `QUALIFY` , `COPY ... TO` Parquet, memory limits with disk spill.
- Dask: partitions plus a task graph; `compute()` is the action; combine partial results correctly; rank-in-group needs a shuffle.
- Spark: transformations are lazy, actions run jobs, wide operations shuffle. Broadcast small tables, fix skew, cache only reused data, prefer built-ins to UDFs, read the Stages page.
- Choose the smallest tool that finishes in time: pandas, then DuckDB or Polars, then Dask or Spark.
- Verify every migration against a trusted slow version on a sample, with tolerance for floats and counts for rows.

Formula / pattern	Meaning
mean = (sum of partial sums) / (sum of partial counts)	The only correct way to combine partition means
partition = stable_hash(key) % B	Co-locates equal keys; basis of shuffle joins, dedupe and group-by
peak memory is about the chunk size, not the file size	Why chunking works
number of partitions is about data size / 128 MB (reads), then 100 to 200 MB after shuffles	Sizing rule of thumb
skew ratio = max partition size / mean partition size	Near 1 is healthy; large values mean stragglers
salt s in 0..S-1, key becomes (key, s), then re-aggregate	Spreads a hot key over S partitions
scan(columns, filters) before join before group-by	Reduce data before the expensive steps
lazy plan, then one `collect()` , `compute()` or action	Let the optimizer see the whole query
built-in, then vectorized UDF, then plain UDF	Order of preference for custom logic
compare(trusted, candidate): counts, keys, values with tolerance	Safe migration test

Practice set

Twenty-three problems, easy first. Try each before reading the answer. Code in the problems that is marked as a block was run, and the printed results quoted in the answers are real.

Problem 22.P.1 (Easy) Name the three ideas behind Parquet, Polars, DuckDB, Dask and Spark, and give one sentence on what each buys you.

Answer: Columnar storage (a query reads only the columns it needs), lazy execution (the whole plan is optimized before it runs: pushdown, reordering) and partitioning (independent pieces are processed in parallel or one at a time, then combined).

Problem 22.P.2 (Easy) A column holds 10 million integers between 0 and 200. How many MB does it take as `int64` , and as the smallest safe unsigned type?

Answer: `int64` is 8 bytes: 10,000,000 times 8 = 80,000,000 bytes = 80 MB. Values up to 200 fit in `uint8` (0 to 255): 10,000,000 bytes = 10 MB. Eight times smaller.

Problem 22.P.3 (Easy) Classify as narrow or wide in Spark: `filter`, `groupBy().sum()`, `select`, a sort-merge `join`, `distinct`, `withColumn`.

Answer: Narrow: `filter`, `select`, `withColumn` (each partition is processed alone). Wide (shuffle): `groupBy().sum()`, the sort-merge `join` (a broadcast join of a small side avoids shuffling the big side), and `distinct`.

Problem 22.P.4 (Easy) A 10 GB CSV is read by Spark with the default of 128 MB per input partition. Roughly how many partitions, and why does the number matter?

Answer: 10 GB is about 10,240 MB, divided by 128 MB gives about 80 partitions (80 tasks that can run in parallel). Too few partitions waste cores and risk memory errors, too many cost scheduling overhead.

Problem 22.P.5 (Easy) What does this print?

```
import io
import pandas as pd

csv = "k,v\na,1\nb,2\na,3\nb,4\na,5"
tot = None
for ch in pd.read_csv(io.StringIO(csv), chunksize=2):
    s = ch.groupby("k")["v"].sum()
    tot = s if tot is None else tot.add(s, fill_value=0)
print(tot.to_dict())
```

Answer: `{'a': 9.0, 'b': 6.0}`. The chunks are rows 1 to 2, 3 to 4 and row 5. Partial sums: a=1, b=2, then a=3, b=4, then a=5. Totals: a = 1 + 3 + 5 = 9 and b = 2 + 4 = 6. They print as floats because `add(..., fill_value=0)` aligns on labels and returns floating-point values.

Problem 22.P.6 (Easy) Values 30, 30, 20 sorted descending. Give `ROW_NUMBER`, `RANK` and `DENSE_RANK` for each.

Answer: ROW_NUMBER: 1, 2, 3 (never ties; which 30 gets 1 is arbitrary unless you add a tie-breaker). RANK: 1, 1, 3 (a gap after the tie). DENSE_RANK: 1, 1, 2 (no gap). Running it in SQLite gives `(30, 1, 1, 1)`, `(30, 2, 1, 1)`, `(20, 3, 3, 2)` in that column order.

Problem 22.P.7 (Medium) Which lines of this PySpark script are actions, and which only build a plan?
`df = spark.read.parquet("e")` / `f = df.filter("price > 0")` / `g = f.groupBy("k").count()` / `g.show()` / `n = f.count()`

Answer: `g.show()` and `f.count()` are actions and each runs a job (the second re-reads and re-filters the data unless `f` was cached). The filter and the group-by are lazy transformations. The read normally touches only file metadata (to find the schema) until an action runs.

Problem 22.P.8 (Medium) What does this print?

```
import numpy as np

keys = np.array([0, 0, 0, 0, 1, 2, 3, 4, 8])
sizes = np.bincount(keys % 4, minlength=4)
print(sizes.tolist(), round(sizes.max() / sizes.mean(), 2))
```

Answer: `[6, 1, 1, 1] 2.67`. Remainders modulo 4: key 0 gives 0 (four rows), key 4 gives 0, key 8 gives 0, so partition 0 has 4 + 1 + 1 = 6 rows; keys 1, 2, 3 give one row each. The mean is 9/4 = 2.25 and 6 / 2.25 = 2.67. That is a skew ratio well above 1.

Problem 22.P.9 (Medium, find the bug) A Dask user computes the average order value like this and finds it differs from the pandas answer. Partition A has one order of 10; partition B has three orders of 20 each. Their code averages the two partition means. What do they get, what is correct, and what is the fix?

Answer: They compute (10 + 20) / 2 = 15. Correct: total = 10 + 60 = 70 over 4 orders = 17.5. The unweighted average gives partition A (one order) the same weight as B (three orders). Fix: keep (sum, count) per partition, add them (70 and 4) and divide at the end, which is what `ddf.amount.mean()` does for you.

Problem 22.P.10 (Medium, find the bug) A teammate downcasts with `s.astype("uint8")` on a column that sometimes holds 300. What does `pd.Series([100, 300]).astype("uint8").tolist()` print and what is the lesson?

Answer: `[100, 44]`. 300 does not fit in 8 bits and wraps around (300 - 256 = 44), silently. Lesson: choose dtypes from the real minimum and maximum (`pd.to_numeric(downcast="unsigned")` does) and assert the range yourself (`s.max() <= 255`) before a forced `astype`.

Problem 22.P.11 (Medium, find the bug) After enriching 3 orders with a customer table, the result has 4 rows and the revenue total is too high. The code is `orders.merge(tiers, on="k", how="left")`, with `orders.k = [1, 2, 3]` and `tiers.k = [1, 1, 2]`. Explain and fix.

Answer: The key 1 appears twice in `tiers`, so the order with k = 1 matches two rows and is duplicated: the merge returns 4 rows (this is a many-to-one join that turned into many-to-many). Orders with k = 3 get a missing tier. Fix: deduplicate the dimension first (`tiers.drop_duplicates("k")` with a rule for which row to keep), and validate with `merge(..., validate="many_to_one")`, which raises an error if the right side has duplicate keys. Also assert that the row count is unchanged after a left join.

Problem 22.P.12 (Medium) Choose pandas, DuckDB, Polars, Dask or Spark, with a one-line reason: (a) an 800 MB CSV on a 16 GB laptop; (b) 200 GB of Parquet on one 128 GB server, with SQL-fluent analysts; (c) 5 TB in object storage, and the company already runs Databricks.

Answer: (a) pandas with `usecols` and dtypes: the data fits, no new tool needed. (b) DuckDB: SQL straight on Parquet, spills to disk, no cluster; Polars lazy is a good second choice for a code-first team. (c) PySpark: the data is beyond one machine and the platform and skills exist.

Problem 22.P.13 (Medium) Write portable SQL (it should run in SQLite and DuckDB) for "the single highest order per region" on the orders table of this chapter, and give the answer.

Answer: Rank in an inner query and filter in the outer query (QUALIFY is DuckDB-only):
`SELECT region, order_id, amount FROM (SELECT region, order_id, amount, ROW_NUMBER() OVER (PARTITION BY region ORDER BY amount DESC) AS rn FROM orders) WHERE rn = 1 ORDER BY region`
Largest orders: APAC order 7 (40), EU order 10 (45), US order 4 (50). So the rows are (APAC, 7, 40), (EU, 10, 45), (US, 4, 50).

Problem 22.P.14 (Medium) Write a function that keeps an exact running mean over a stream of chunks using constant memory, and test it on the chunks [1, 2, 3] and [4, 5].

```
def update(state, chunk):
    total, count = state
    return total + sum(chunk), count + len(chunk)

state = (0, 0)
for chunk in ([1, 2, 3], [4, 5]):
    state = update(state, chunk)
print(state, state[0] / state[1])
```

Answer: It prints `(15, 5) 3.0`. The state is two numbers however long the stream is. Do not store the mean itself: a mean cannot be merged without the count behind it. The same state design extends to variance (keep the count, the sum and the sum of squares, or use Welford's update for numerical stability).

Problem 22.P.15 (Hard) The Spark UI shows, for one stage: median task duration 8 s, max 190 s, shuffle spill 0, 200 tasks. Diagnose it and list three things to try in order.

Answer: 190 / 8 is about 24 times the median, with no spill: one partition is far larger than the others, so this is data skew (or one bad node, but a node problem would not be tied to a stage's data). Try: (1) look at the key distribution of the shuffle key (`groupBy(key).count().orderBy(desc)`) and filter or isolate null or bot keys; (2) make sure AQE skew handling is on (`spark.sql.adaptive.skewJoin.enabled`); (3) salt the hot keys with a two-stage aggregation or a replicated small side. Re-check the task summary after each change.

Problem 22.P.16 (Hard) Translate to Polars: for each row add the region total and the row's share of it, then the rank of the row within its region (largest first). Pandas: `t = df.groupby("region")["amount"].transform("sum"); share = df["amount"] / t; rk = df.groupby("region")["amount"].rank(method="first", ascending=False)`.

Answer (not run here):
`df.with_columns(region_total=pl.col("amount").sum().over("region")).with_columns(share=pl.col("amount") / pl.col("region_total"), rk=pl.col("amount").rank("ordinal", descending=True).over("region"))`

The first `with_columns` creates the total; a second one is needed for the share because expressions in one call cannot see each other's new columns. `over("region")` is the transform: compute within groups, keep all rows.

Problem 22.P.17 (Hard) Design the storage layout for 2 TB of web logs per month (columns: ts, user_id, country, url, status, bytes), queried mostly by date range and country.

Answer: Parquet, compressed with zstd or snappy. Partition by date (`date=2025-06-01`), and country only if its cardinality is small and queries filter on it. Do not partition by `user_id` (millions of folders). Target files of 128 MB to 1 GB (so about 20 to 100 files per day, not 100,000). Sort within files by `country` then `ts` so row-group statistics skip well. Write dates as timestamps and ids as strings or 64-bit integers, consistently. Add a compaction job for late data.

Problem 22.P.18 (Hard) Hash-bucket dedupe of 2 billion order ids (8 bytes each) with 1 GB of memory available per bucket. How many buckets at least, and why add a safety margin?

Answer: 2,000,000,000 times 8 bytes = 16 GB of raw keys, so at least 16 buckets of 1 GB each; in practice more (32 or 64) because hashing is only roughly even, Python or pandas structures need several times the raw key bytes, and rows carry other columns too. Choose the bucket count so one bucket comfortably fits, and test the largest bucket size.

Problem 22.P.19 (Hard) A join key has one value holding 40 percent of all rows, the shuffle uses 100 partitions, and you want no partition above twice the mean. What is the least number of salts S for that key (ignore other keys)?

Answer: The mean share per partition is 1 percent. After salting, the hot key's 40 percent is divided into S pieces of 40/S percent. We need 40 / S <= 2, so S >= 20. The small side of the join is then replicated 20 times for that key (or only for hot keys, if you salt selectively).

Problem 22.P.20 (Hard) Why can a Spark filter written as `df.filter(my_udf(F.col("name")) == "x")` be much slower than `df.filter(F.col("name") == "x")` on Parquet, even when the UDF is trivial?

Answer: The built-in comparison on a raw column can be pushed into the Parquet scan, so row groups whose statistics rule out "x" are skipped and the rows never leave the JVM. The optimizer cannot see inside the UDF: the filter cannot be pushed down, every row of the column is read, serialized to a Python process, evaluated and sent back. Fix: express it with built-ins (`F.lower` , `F.regexp_extract`), or filter on the raw column first.

Problem 22.P.21 (Hard) A Spark job does `df.repartition(200, "customer_id").write.partitionBy("date").parquet(out)` and produces a huge number of small files for a 30-day dataset. Why, and what is the fix?

Answer: Each of the 200 tasks holds customers from all 30 days, so each task writes one file per date it contains: up to 200 x 30 = 6,000 files, each small. Fix: repartition by the same column you partition by, so each date is written by few tasks: `df.repartition("date").write.partitionBy("date")...` , or `repartition(n, "date", salt)` to control files per date, or coalesce before writing; compaction afterwards is the fallback.

Problem 22.P.22 (Expert) The same dataset summed in two orders gives different float totals. Show it with three numbers, then say how to design checks between a pandas and a Spark pipeline.

```
a = (1e16 + 1.0) - 1e16
b = (1e16 - 1e16) + 1.0
print(a, b)
```

Answer: It prints `0.0 1.0` . In 64-bit floating point, 1e16 + 1.0 rounds back to 1e16, so the first order loses the 1.0. Floating-point addition is not associative, and a distributed engine adds partial sums in a data-dependent order. Therefore: compare floats with a relative tolerance (`np.isclose`), compare counts and integer columns exactly, store money as integer cents so sums are exact, and compare many cheap invariants (row counts, distinct keys, null counts, min and max, a checksum of an integer column) instead of just one total.

Problem 22.P.23 (Expert, interview) "Count distinct users per day over 3 billion rows is slow in Spark. Why, and what are your options?" Give an answer you would defend.

Answer: An exact distinct count needs every distinct (day, user) pair to be brought together: it shuffles by the pair or by the day, deduplicates, then counts, which is memory- and network-heavy, and a few busy days create skew. Options: (1) `F.approx_count_distinct` (HyperLogLog sketch, mergeable across partitions so it needs a tiny partial state; the default relative standard deviation is about 5 percent and can be tightened at higher memory cost) when a small error is acceptable; (2) first deduplicate (day, user) with a narrower row (select only those two columns) and then count, to

reduce shuffle bytes; (3) pre-aggregate hourly to daily distinct sets in a table, or sketches stored in a table and merged; (4) fix skew with salting if one day dominates. State the accuracy trade-off explicitly.

Prompt pack

AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data roles, and English is my second language, so use simple English. Teach me when pandas stops being enough and how to choose between Parquet, Polars, DuckDB, Dask and PySpark. Use small Python examples that I can run with only pandas, numpy and sqlite3, show real output, and relate each idea to a story about a 10 GB CSV on a laptop. Cover: dtype downcasting, categories, chunked reading, partial aggregates (sum and count, never mean of means), columnar storage, lazy execution, partitions, shuffles and skew. After each part, ask me one check question and wait for my answer before continuing.
```

AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional learning big-data tools. Below I will paste a pandas, Polars, DuckDB SQL, Dask or PySpark script that I wrote for a large dataset. Review it like a senior data engineer: find memory and speed risks (reading everything, row-by-row functions, UDFs, collect or toPandas on big results, unneeded sorts, tiny files, skewed keys, null join keys, float sums), explain each problem in simple English with the symptom I would see, and show the fixed code. Tell me which library you would choose for my data size and why, and how I can verify the fixed pipeline gives the same result as the old one on a small sample.
```

AI prompt: copy and paste into any AI

```
You are a hiring manager for a data engineering or data science role. Run a 30-minute mock interview on scaling data processing: pandas memory limits, Parquet versus CSV, lazy versus eager evaluation, Polars expressions, DuckDB window functions, Dask partitions, Spark shuffles, broadcast joins, data skew and salting, caching, UDFs versus built-ins, and reading the Spark UI. Ask one question at a time, starting easy and getting harder, and include two find-the-bug code questions and one design question (storage layout for 2 TB of logs per month). After each of my answers, score it out of 10, say what a strong answer would include, then continue. At the end give me a summary of my weak spots and a three-day study plan.
```

CHAPTER 23

ENGINEERING TOOLKIT: PYTEST, CLI TOOLS, FASTAPI, PACKAGING AND ML TRACKING

Library Handbook: tests, command-line tools, APIs, packaging and reproducible training.

Priya is a data analyst at an online grocer. Last month she built a churn model in a notebook. It works: 81% of the customers it flags really do leave. Now her manager says: "Great. Run it every Monday on the new export, keep the numbers somewhere we can compare, and let the web team call it from the app." Priya looks at her notebook. It has a hard-coded file path, a password pasted in cell 3, no tests, cells that only work when run in a certain order, and a model that nobody can rebuild if her laptop dies.

Nothing in that notebook is a modelling problem. Every problem is an engineering problem, and the Python world has a standard answer to each: **tests** (pytest) to know a change did not break anything, **logging** to see what a job did at 3 a.m., **argparse or Typer** to turn a script into a command, **pathlib** for paths that work on every machine, **dataclasses and type hints** for configuration that cannot silently be wrong, **venv, uv and pyproject.toml** for an environment you can recreate, **FastAPI** to serve predictions, **joblib** to cache and parallelise, **MLflow and DVC** to remember which data and parameters produced which model, **Docker** to ship it, and **Airflow or Prefect** to run it on a schedule.

This chapter is part of the Library Handbook, so it is practical and point by point. By the end you will be able to write and run tests, build a command-line tool and an API, train a model in a reproducible script that records its parameters and metrics, lay out a project that another person can install in two commands, and recognise the standard fixes for the problems that appear again and again. Only the standard library, numpy, pandas, scikit-learn, matplotlib, joblib, httpx and pyyaml are installed on the study machine, so some tools are shown as code marked **not run here**. Wherever possible, a runnable equivalent built on the standard library is shown beside them, and every printed output in the chapter was produced by running the code with Python 3.14.

23.1 The Engineering Toolkit: What, When, Install, Import and the Mental Model

INTUITION

A model is a recipe. Engineering is the kitchen: labelled shelves (project layout), a thermometer (tests), a logbook (logging, tracking), a delivery van (Docker, API) and a rota (scheduler). A great recipe in a messy kitchen produces a different dish every day.

What each tool is for, and when NOT to use it

Tool	Use it for	Do NOT reach for it when
pytest / unittest	Checking that functions behave as promised, every time you change code	You only need to explore data once; a throw-away notebook does not need tests
logging	A timestamped record of what a job did, with levels	You want to show the user a result; use `print` or the return value
argparse, Typer, Click	Turning a script into a command with options and help	The script has no inputs; a constant is simpler
pathlib	Building and inspecting file paths, reading and writing small files	You need to read millions of rows; use pandas or pyarrow after pathlib finds the file
dataclasses, typing, mypy	Typed config and records, static checking of mistakes	Data arrives from outside and must be validated at run time; use pydantic for that
venv, pip, uv, pyproject.toml	A recreatable environment and an installable package	Your system needs non-Python libraries pinned too; use Docker or conda
FastAPI	A typed web API with validation and automatic docs	A nightly batch job; write a CLI and a scheduler instead

Tool	Use it for	Do NOT reach for it when
joblib	Caching slow function results on disk, parallel loops, saving sklearn models	The work is tiny (overhead dominates) or already vectorised in numpy
MLflow, DVC	Comparing many experiments; versioning big data files	One model, one dataset, one person; a JSON file of metrics is enough
Docker	Shipping the same environment everywhere	You only run on your laptop
Airflow, Prefect	Scheduled, retried, monitored multi-step pipelines	One script run by cron is enough
pre-commit	Running formatters and checks before every commit	You work alone on a one-file script

Install and version check

Not run here: these commands need network access.

```
# create and activate an environment, then install what the chapter uses
python3 -m venv .venv
source .venv/bin/activate            # Windows: .venv\Scripts\activate
python -m pip install pytest typer fastapi uvicorn mypy mlflow pre-commit
# or, with uv (a fast installer and project manager):
uv venv && uv pip install pytest typer fastapi uvicorn
```

Whatever you install, always ask the interpreter which versions it actually sees. This block works for every library, installed or not, and it is the first thing to run when something "should work".

```
import importlib.util
import importlib.metadata as md
import sys

print("python", sys.version.split()[0])
libs = {"numpy": "numpy", "pandas": "pandas", "sklearn": "scikit-learn", "joblib": "joblib",
        "yaml": "pyyaml", "httpx": "httpx", "pytest": "pytest", "fastapi": "fastapi",
        "typer": "typer", "mlflow": "mlflow"}
for module, dist in libs.items():
    if importlib.util.find_spec(module) is None:
        print(f"{module:8s} not installed")
    else:
        print(f"{module:8s} {md.version(dist)}")
```

```
python 3.14.3
numpy    2.5.3
pandas   3.0.6
sklearn  1.9.1
joblib   1.6.0
yaml     6.0.3
httpx    0.28.1
pytest   not installed
fastapi  not installed
typer    not installed
mlflow   not installed
```

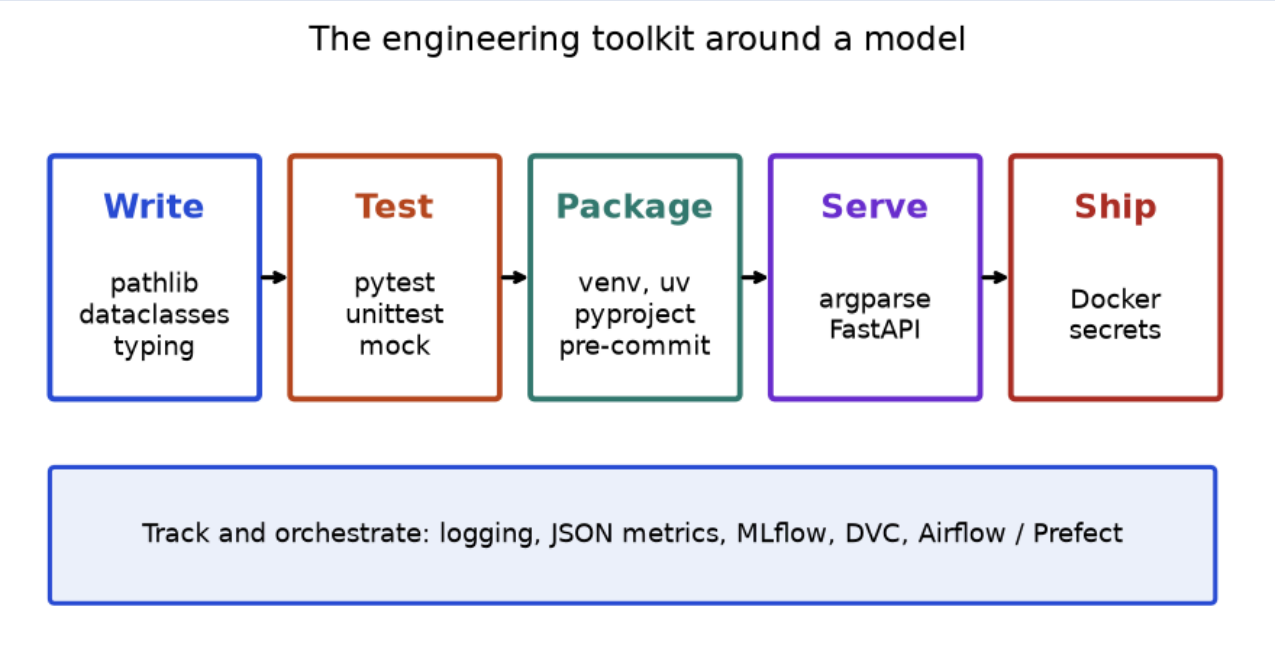
1. `find_spec` asks Python whether an import would succeed, without importing (and without crashing).
2. The dictionary maps the import name to the distribution name, because they differ: you `import sklearn` but `pip install scikit-learn`.
3. `importlib.metadata.version` reads the installed version from package metadata, the same number `pip list` shows.
4. In the output, `pytest`, `fastapi`, `typer` and `mlflow` are missing on this machine. That is why they appear as "not run here" below.

Import conventions

- Standard library first, then third party, then your own package, each group separated by a blank line (tools such as ruff and isort enforce this).
- Import the module, not every name: `from pathlib import Path` is the accepted exception for very common names.
- Conventional aliases: `import numpy as np`, `import pandas as pd`, `import typer`, `import pytest`, `import joblib`. Do not invent others.
- Never `from module import *` in real code. It hides where names come from.

Mental model

Think of a project as a conveyor belt with five stations and a logbook underneath. You write typed, tidy code; you test it; you package it so it installs the same everywhere; you expose it as a command or an API; you ship it in a container. Underneath, logging and tracking record what happened, and a scheduler decides when the belt moves.



The engineering toolkit as a conveyor belt with a logbook underneath.

How to read it: move left to right to follow the life of a change. Every box answers one question: is the code tidy (Write), is it right (Test), can someone else install it (Package), how do people call it (Serve), where does it run (Ship). The wide bottom box answers "what happened, and when does it run again?". When you debug, find the box where the problem lives first.

KEY IDEA	Each tool exists to remove one kind of surprise: wrong results (tests), unknown behaviour (logs), different machines (environments, Docker), unknown inputs (types, validation), and lost history (tracking, versioning). Pick tools by the surprise you want to remove, not by fashion.

23.2 Testing: pytest, unittest, Fixtures, Parametrize, Markers and Mocking

A test is a small function that calls your code with a known input and checks the answer. When you change code next month, the tests tell you in seconds whether you broke something. **pytest** is the standard runner: tests are plain functions and plain `assert` statements. It is **not installed** on the study machine, so for every pytest example below you see the pytest version labelled *not run here*, followed by the same checks written with **unittest** (standard library), which really ran and printed the output shown. The ideas (arrange, act, assert) are identical.

KEY IDEA	Every test has three parts: **arrange** (build the input), **act** (call the code once), **assert** (compare with the expected answer). If a test has several acts, split it into several tests, so a failure points at exactly one behaviour.

Reference table: the pytest features you will use

Name	What it does	Minimal example
`assert`	The only assertion you need; pytest rewrites it to show both sides on failure	`assert f(2) == 4`
`pytest.approx`	Compares floats with a tolerance	`assert 0.1 + 0.2 == pytest.approx(0.3)`
`pytest.raises`	Passes only if the block raises the exception	`with pytest.raises(ValueError): f("x")`
`pytest.warns`	Passes only if a warning is issued	`with pytest.warns(UserWarning): f()`
`@pytest.fixture`	A reusable setup function; tests ask for it by argument name	`@pytest.fixture` `def df(): return pd.DataFrame(...)`
fixture with `yield`	Code after `yield` is teardown	`conn = connect(); yield conn; conn.close()`
`scope=`	How often a fixture is rebuilt: function, class, module, package, session	`@pytest.fixture(scope="session")`
`autouse=True`	Applies the fixture to every test without being requested	`@pytest.fixture(autouse=True)`
`conftest.py`	File whose fixtures are shared by all tests in its folder	`tests/conftest.py`
`@pytest.mark.parametrize`	Runs one test over many input rows	`("x,y", [(1, 2), (2, 4)])`
`pytest.param`	Adds an id or marks to one parameter row	`pytest.param(0, id="zero")`
`@pytest.mark.skip` / `skipif`	Do not run (always, or on a condition)	`@pytest.mark.skipif(sys.platform == "win32", reason="posix")`
`@pytest.mark.xfail`	Expected to fail; reports if it unexpectedly passes	`@pytest.mark.xfail(reason="bug 12")`
custom marker	Tag tests (e.g. slow) and select with `-m`	`@pytest.mark.slow`
`tmp_path`	Fixture giving a fresh empty temporary folder (a `Path`)	`(tmp_path / "a.csv").write_text("x\n1")`
`monkeypatch`	Fixture to change attributes, dict items and env vars for one test, auto-undone	`monkeypatch.setenv("API_KEY", "t")`
`capsys`	Fixture capturing what the code prints	`out = capsys.readouterr().out`
`caplog`	Fixture capturing log records	`assert "done" in caplog.text`
`request`	Fixture giving a fixture access to its parameters	`request.param`
`unittest.mock.Mock` / `patch`	Stand-ins for slow or external objects	`with patch("mod.requests.get") as g: ...`
CLI: `-k`	Run tests whose name matches an expression	`pytest -k parse`
CLI: `-x`, `--lf`	Stop at first failure; rerun only last failures	`pytest -x --lf`
CLI: `-q`, `-vv`	Quieter or more verbose output	`pytest -vv tests/test_io.py::test_read`
CLI: `--cov` (plugin)	Coverage report (needs pytest-cov)	`pytest --cov=mypkg`

Example 1: the smallest test

The function under test cleans a price string such as `"$1,299.50"` into a float. First pytest (not run here), then unittest (run).

```
# test_prices.py   -- NOT RUN HERE: pytest is not installed
from prices import clean_price

def test_plain_number():
    assert clean_price("12") == 12.0

def test_dollar_and_comma():
    assert clean_price("$1,299.50") == 1299.5

def test_blank_is_none():
    assert clean_price("  ") is None
```

```
import io
import unittest

def clean_price(text):
    text = text.strip().replace("$", "").replace(",", "")
    return float(text) if text else None

class TestPrices(unittest.TestCase):
    def test_plain_number(self):
        self.assertEqual(clean_price("12"), 12.0)

    def test_dollar_and_comma(self):
        self.assertEqual(clean_price("$1,299.50"), 1299.5)

    def test_blank_is_none(self):
        self.assertIsNone(clean_price("  "))

def run(case):
    suite = unittest.defaultTestLoader.loadTestsFromTestCase(case)
    res = unittest.TextTestRunner(stream=io.StringIO(), verbosity=0).run(suite)
    print("ran", res.testsRun, "failures", len(res.failures), "errors", len(res.errors))
    return res

run(TestPrices)
```

```
ran 3 failures 0 errors 0
```

1. The pytest file imports the function and defines three functions whose names start with `test_`. pytest finds them by that prefix; no class is needed.
2. In the unittest version we define `clean_price` and the tests in one file so the block is standalone. A test class must inherit from `unittest.TestCase` and its methods must start with `test`.
3. `assertEqual(a, b)` is the unittest spelling of `assert a == b`; `assertIsNone(x)` is `assert x is None`.
4. `run` loads all tests in the class, runs them silently into a string buffer, and prints only the counts. We keep it small because the next blocks reuse the same idea.
5. Output: 3 tests ran, 0 failures and 0 errors. A *failure* is a wrong answer (an assertion); an *error* is an unexpected exception.

Example 2: parametrize, many inputs one test

Copy-pasting a test per input is noisy. `parametrize` turns one test into a table of cases. unittest's closest tool is `subTest`, which reports each failing row separately. We include one wrong expectation on purpose to see what a failure looks like.

```
# NOT RUN HERE: pytest is not installed
import pytest
from prices import clean_price

@pytest.mark.parametrize(
    "text, expected",
    [
        ("12", 12.0),
        ("$1,299.50", 1299.5),
        ("  7.5 ", 7.5),
        pytest.param("$0", 1.0, id="wrong-on-purpose"),
    ],
)
def test_clean_price(text, expected):
    assert clean_price(text) == expected
```

```
import io
import unittest

def clean_price(text):
    text = text.strip().replace("$", "").replace(",", "")
    return float(text) if text else None

class TestPrices(unittest.TestCase):
    def test_table(self):
        cases = [("12", 12.0), ("$1,299.50", 1299.5), ("  7.5 ", 7.5), ("$0", 1.0)]
        for text, expected in cases:
            with self.subTest(text=text):
                self.assertEqual(clean_price(text), expected)

suite = unittest.defaultTestLoader.loadTestsFromTestCase(TestPrices)
res = unittest.TextTestRunner(stream=io.StringIO(), verbosity=0).run(suite)
print("failures:", len(res.failures))
case, message = res.failures[0]
print(str(case))
print(message.strip().splitlines()[-1])
```

```
failures: 1
test_table (__main__.TestPrices.test_table) (text='$0')
AssertionError: 0.0 != 1.0
```

1. The pytest version lists four rows; `pytest.param(..., id=...)` gives the last row a readable name in the report. It will fail on purpose, and pytest prints that one row only; the other three pass.
2. The unittest loop does the same: `with self.subTest(text=text)` wraps each row so a failure in one row does not stop the loop.
3. After the run, `res.failures` is a list of `(test, traceback_text)` pairs. We print the test label, which includes the failing parameter, and the last line of the traceback, which holds the comparison.
4. The output shows 1 failure. Read it as: the case with `text='$0'` returned `0.0` while the test expected `1.0`. The code is right and the test row is wrong, a reminder that tests have bugs too.

Example 3: fixtures and temporary folders

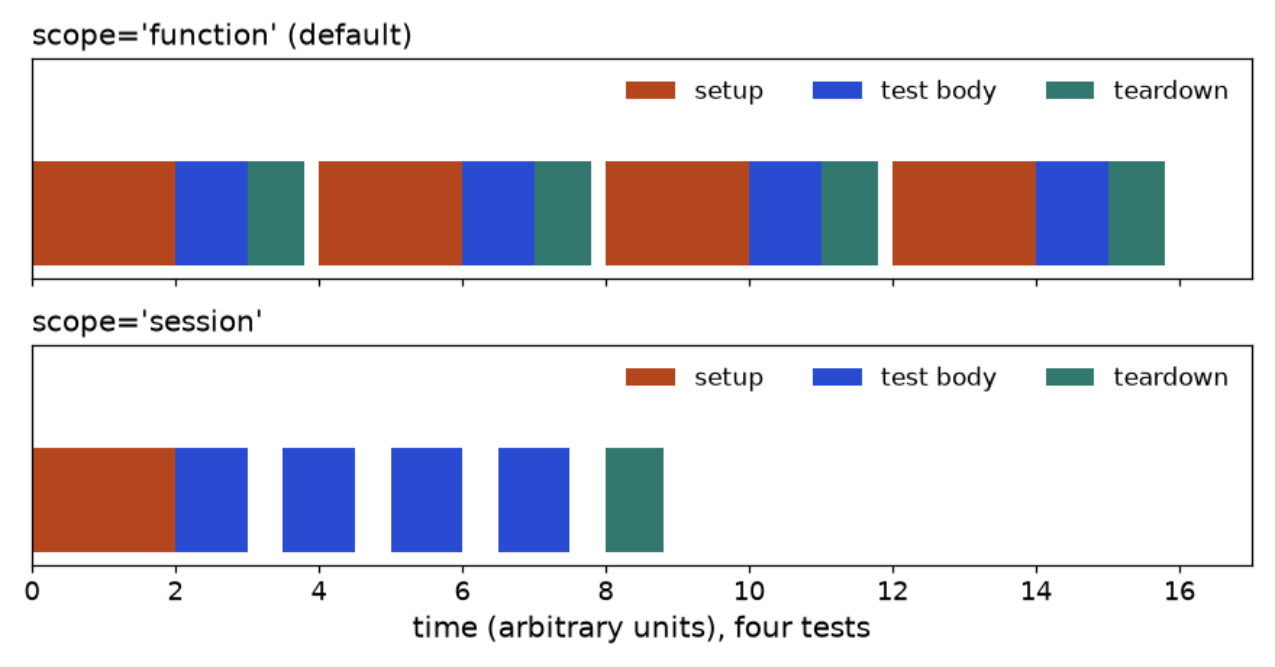
A **fixture** prepares something many tests need (a DataFrame, a file, a database connection) and cleans it up afterwards. pytest injects it by argument name. `tmp_path` is a built-in fixture that gives every test its own empty folder.

```
# NOT RUN HERE: pytest is not installed
import pandas as pd
import pytest

@pytest.fixture
def csv_file(tmp_path):
    path = tmp_path / "orders.csv"
    path.write_text("order_id,amount\n1,10.5\n2,4.5\n")
    yield path            # the test runs here
    print("teardown")     # code after yield runs afterwards

def test_total(csv_file):
    df = pd.read_csv(csv_file)
    assert df["amount"].sum() == 15.0
```

```
import io
import tempfile
import unittest
from pathlib import Path

import pandas as pd

events = []

class TestOrders(unittest.TestCase):
    def setUp(self):
        events.append("setup")
        tmp = tempfile.TemporaryDirectory()
        self.addCleanup(lambda: events.append("teardown"))
        self.addCleanup(tmp.cleanup)
        self.path = Path(tmp.name) / "orders.csv"
        self.path.write_text("order_id,amount\n1,10.5\n2,4.5\n")

    def test_total(self):
        events.append("test_total")
        df = pd.read_csv(self.path)
        self.assertEqual(df["amount"].sum(), 15.0)

    def test_rows(self):
        events.append("test_rows")
        self.assertEqual(len(pd.read_csv(self.path)), 2)

suite = unittest.defaultTestLoader.loadTestsFromTestCase(TestOrders)
res = unittest.TextTestRunner(stream=io.StringIO()).run(suite)
print(res.testsRun, "tests, ok:", res.wasSuccessful())
print(events)
```

```
2 tests, ok: True
['setup', 'test_rows', 'teardown', 'setup', 'test_total', 'teardown']
```

1. The pytest fixture `csv_file` asks for `tmp_path`, a fixture that pytest builds for you. A fixture can depend on other fixtures in the same way a test does.
2. Everything before `yield` is setup; the yielded value is what the test receives; everything after is teardown, even when the test fails.
3. In unittest, `setUp` runs before each test method. `tempfile.TemporaryDirectory` is the standard-library counterpart of `tmp_path`.
4. `addCleanup` registers a function to run after the test, in last-in first-out order. That is why the `tmp.cleanup` registered second runs before the `"teardown"` note registered first.
5. The printed event list shows the lifecycle twice: setup, test, teardown for each of the two tests. Tests run in alphabetical order (`test_rows` before `test_total`), so never depend on order.



Fixture scope controls how many times the expensive setup is paid.

How to read it: each coloured bar is a block of time. In the top row (the default `scope="function"`) the orange setup bar repeats before every test. In the bottom row (`scope="session"`) it is paid once and shared, which is what you want for loading a large model or starting a database. The price of sharing: tests can now affect each other through the shared object, so share only things tests do not modify.

Example 4: monkeypatch and mocking

Tests must not call the internet, read your real secrets or wait on a slow model. `monkeypatch` temporarily replaces an environment variable or attribute; a `Mock` replaces an object and records how it was called.

```
# NOT RUN HERE: pytest is not installed
import os
from unittest.mock import Mock

def get_key():
    return os.environ["API_KEY"]

def score_all(client, ids):
    return [client.predict(i) for i in ids]

def test_get_key(monkeypatch):
    monkeypatch.setenv("API_KEY", "test-key")      # undone after the test
    assert get_key() == "test-key"

def test_missing_key(monkeypatch):
    monkeypatch.delenv("API_KEY", raising=False)
    import pytest
    with pytest.raises(KeyError):
        get_key()

def test_score_all():
    client = Mock()
    client.predict.side_effect = [0.1, 0.9]
    assert score_all(client, [7, 8]) == [0.1, 0.9]
    client.predict.assert_any_call(8)
```

```
import os
import unittest
from unittest import mock
from unittest.mock import Mock

def get_key():
    return os.environ["API_KEY"]

def score_all(client, ids):
    return [client.predict(i) for i in ids]

# 1. replace an environment variable for the length of a with-block
with mock.patch.dict(os.environ, {"API_KEY": "test-key"}):
    print(get_key())
print("API_KEY" in os.environ)

# 2. a Mock records calls and returns scripted values
client = Mock()
client.predict.side_effect = [0.1, 0.9]
print(score_all(client, [7, 8]))
print(client.predict.call_count, client.predict.call_args_list)
client.predict.assert_any_call(8)

# 3. patch an attribute of a real module
import math
with mock.patch.object(math, "sqrt", return_value=42.0):
    print(math.sqrt(4))
print(math.sqrt(4))
```

```
test-key
False
[0.1, 0.9]
2 [call(7), call(8)]
42.0
2.0
```

1. `monkeypatch.setenv` sets the variable and pytest restores the old state when the test ends. `mock.patch.dict(os.environ, {...})` is the standard-library equivalent, scoped to a `with` block.
2. Inside the block `get_key()` sees `test-key`; afterwards the variable is gone (`False`).
3. A `Mock` accepts any call. `side_effect=[0.1, 0.9]` makes consecutive calls return those values in order.
4. `call_count` and `call_args_list` show how the code under test used the fake. `assert_any_call(8)` raises if nobody called `predict(8)`; here it passes silently.
5. `patch.object(math, "sqrt", return_value=42.0)` swaps one attribute. Inside the block `math.sqrt(4)` is 42.0; after the block it is the real 2.0. Patch the name where it is *looked up*, not where it is defined (see the pitfalls section).

Example 5: a realistic data test, markers and configuration

Testing a pandas function: compare whole frames with `pandas.testing.assert_frame_equal`, which tells you which cell differs, and use a plain exception test for bad input.

```
import io
import unittest

import pandas as pd
from pandas.testing import assert_frame_equal

def add_revenue(df):
    if "price" not in df.columns or "qty" not in df.columns:
        raise KeyError("need price and qty")
    out = df.copy()
    out["revenue"] = out["price"] * out["qty"]
    return out

class TestRevenue(unittest.TestCase):
    def test_values(self):
        df = pd.DataFrame({"price": [2.5, 4.0], "qty": [2, 3]})
        want = pd.DataFrame({"price": [2.5, 4.0], "qty": [2, 3], "revenue": [5.0, 12.0]})
        assert_frame_equal(add_revenue(df), want)

    def test_input_not_modified(self):
        df = pd.DataFrame({"price": [1.0], "qty": [1]})
        add_revenue(df)
        self.assertEqual(list(df.columns), ["price", "qty"])

    def test_missing_column(self):
        with self.assertRaises(KeyError):
            add_revenue(pd.DataFrame({"price": [1.0]}))

    def test_wrong_dtype_is_caught(self):
        df = pd.DataFrame({"price": [2.5], "qty": [2]})
        bad = pd.DataFrame({"price": [2.5], "qty": [2], "revenue": [5]})   # int, not float
        with self.assertRaises(AssertionError) as cm:
            assert_frame_equal(add_revenue(df), bad)
        print(str(cm.exception).splitlines()[0])

suite = unittest.defaultTestLoader.loadTestsFromTestCase(TestRevenue)
res = unittest.TextTestRunner(stream=io.StringIO()).run(suite)
print(res.testsRun, "tests, ok:", res.wasSuccessful())
```

```
Attributes of DataFrame.iloc[:, 2] (column name="revenue") are different
4 tests, ok: True
```

1. `add_revenue` checks its input, copies the frame (so the caller's data is untouched) and adds a column. Copy-on-write in pandas 3.0 would also protect the caller, but an explicit `copy()` states the intent.
2. `assert_frame_equal` compares values, dtypes, index and column names. On a mismatch it raises `AssertionError` naming the difference.
3. `test_input_not_modified` checks a promise ("I do not mutate your data") that is easy to break later; this is a typical regression test.
4. The last test expects the dtype check to fail because we compare a float column with an int column. The printed first line, `Attributes of DataFrame.iloc[:, 2] (column name="revenue") are different`, is what you would see in a real failure. The test run reports 4 tests, all ok.

In pytest you would also tag slow tests and register the tag so that typos are caught. Configuration lives in `pyproject.toml` (not run here):

```
[tool.pytest.ini_options]
testpaths = ["tests"]
addopts = "-q --strict-markers"
markers = [
    "slow: tests that train a model (deselect with -m 'not slow')",
]
```

```
pytest                      # everything
pytest -m "not slow"        # skip the slow ones
pytest -k clean_price -x    # name match, stop at first failure
pytest tests/test_io.py::test_read_csv -vv
```

COMMON MISTAKE	Writing a test that mirrors the code. `assert add_revenue(df)["revenue"].tolist() == (df["price"] * df["qty"]).tolist()` repeats the implementation, so if the formula is wrong, the test is wrong in the same way. Compute the expected value by hand (2.5 * 2 = 5.0) and write the number into the test.

Problem 23.2.1 A test passes alone but fails when the whole suite runs. List the likely causes in order of how often they happen.

Idea	Tests share state somewhere.
Cause 1	A module-level list, dict or DataFrame that one test mutates (fix: build it in a function-scoped fixture).
Cause 2	An environment variable or global set without monkeypatch (fix: use monkeypatch so it is undone).
Cause 3	Files written to a fixed path (fix: use tmp_path).
Cause 4	A session-scoped fixture that tests modify (fix: narrower scope or return a copy).
Check	Run `pytest -p no:randomly` or reverse the order; if the failure moves, state is shared.

23.3 Logging, argparse, Typer/Click and pathlib: Jobs That Run Without You

A job that runs at 3 a.m. has no screen. You need three things: a record of what happened (logging), a way to give it inputs (a command-line interface) and paths that do not break when the folder changes (pathlib).

Logging

The `logging` module sends **records** (message plus level plus time plus source) from **loggers** to **handlers** (screen, file, network), which format them with **formatters**. Levels are ordered: DEBUG (10), INFO (20), WARNING (30), ERROR (40), CRITICAL (50). Anything below the configured level is dropped.

Name	What it does	Minimal example
`logging.getLogger(name)`	Gets (or creates) the named logger; the same name gives the same object	`log = logging.getLogger(__name__)`
`log.debug/info/warning/error/critical`	Emit a record at that level	`log.info("rows=%d", n)`
`log.exception`	Error record with the current traceback; use inside `except`	`log.exception("load failed")`
`basicConfig`	One-call setup of the root logger (does nothing if already configured, unless `force=True`)	`logging.basicConfig(level="INFO")`

Name	What it does	Minimal example
`setLevel`	Minimum level a logger or handler lets through	`log.setLevel(logging.DEBUG)`
`StreamHandler`	Writes to a stream (stderr by default)	`logging.StreamHandler(sys.stdout)`
`FileHandler`	Appends to a file	`logging.FileHandler("job.log")`
`RotatingFileHandler`	Starts a new file when the size limit is reached (in `logging.handlers`)	`RotatingFileHandler("job.log", maxBytes=1_000_000, backupCount=3)`
`Formatter`	Layout of one line	`Formatter("%(asctime)s %(levelname)s %(message)s")`
`addHandler`	Attaches a handler to a logger	`log.addHandler(h)`
`propagate`	Whether records also go to parent loggers' handlers	`log.propagate = False`
`extra=`	Attach custom fields to a record	`log.info("done", extra={"run_id": 7})`
`dictConfig`	Whole configuration as a dictionary (or YAML/JSON)	`logging.config.dictConfig(cfg)`
`NullHandler`	What a library attaches so it never prints unasked	`logging.getLogger("mylib").addHandler(logging.NullHandler())`
`captureWarnings`	Routes `warnings.warn` into logging	`logging.captureWarnings(True)`
`LoggerAdapter`	Adds fixed context to every message	`logging.LoggerAdapter(log, {"job": "etl"})`
`logging.disable`	Silence everything up to a level (useful in tests)	`logging.disable(logging.CRITICAL)`

Example 6: the basic setup

```
import logging
import sys

logging.basicConfig(level=logging.INFO, stream=sys.stdout,
                    format="%(levelname)s %(name)s: %(message)s")
log = logging.getLogger("etl")

rows = 1250
log.debug("this is hidden, level is INFO")
log.info("loaded %d rows", rows)
log.warning("column %s has %.1f%% missing", "age", 12.34)
log.error("could not write %s", "out.csv")
```

```
INFO etl: loaded 1250 rows
WARNING etl: column age has 12.3% missing
ERROR etl: could not write out.csv
```

1. `basicConfig` configures the root logger once: minimum level INFO, output to stdout (default is stderr), and a format using placeholders like `%(levelname)s`.
2. `getLogger("etl")` returns a child logger. In real modules use `getLogger(__name__)` so the name shows which file spoke.
3. The `debug` call is dropped because DEBUG (10) is below INFO (20).
4. Arguments are passed separately (`"loaded %d rows", rows`), not pre-formatted with an f-string. Logging formats the text only if the record is actually emitted, which saves work for hidden debug messages.
5. Output: three lines, one per emitted level. In production you would add `%(asctime)s` to the format; we left it out so this output is the same every time.

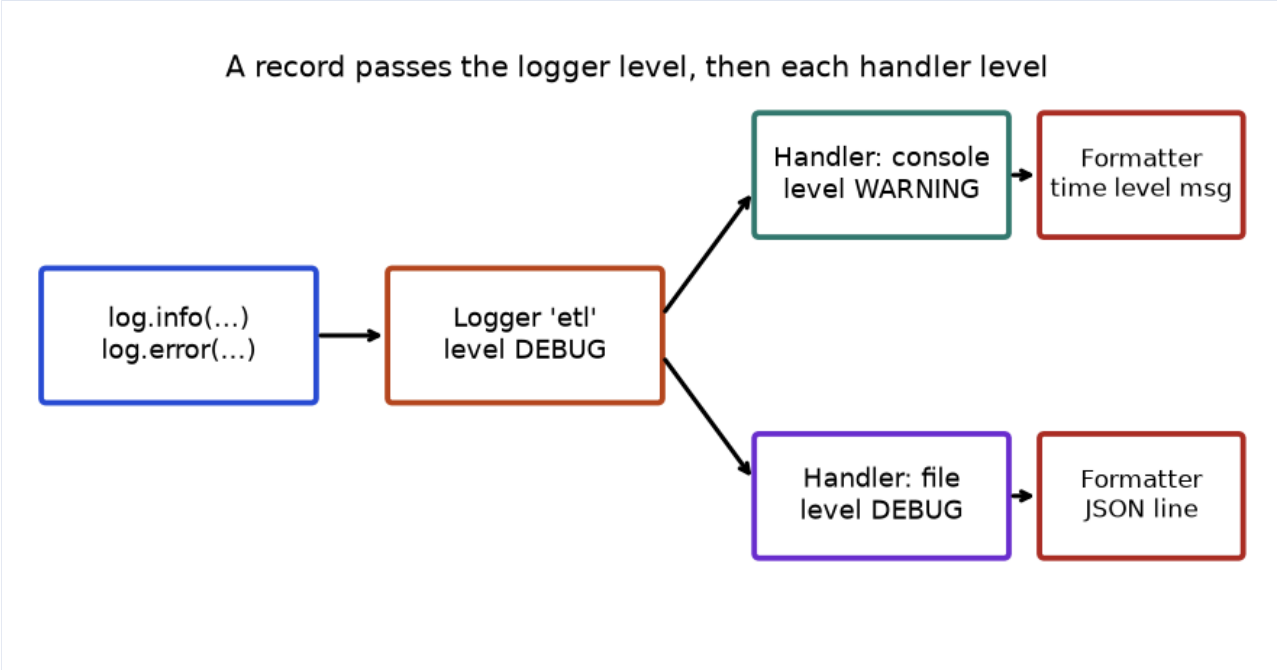
Example 7: two handlers with different levels

Typical need: the console shows only warnings, the log file keeps everything.

```
import io
import logging

console, logfile = io.StringIO(), io.StringIO()
log = logging.getLogger("train")
log.setLevel(logging.DEBUG)
log.propagate = False

h1 = logging.StreamHandler(console)
h1.setLevel(logging.WARNING)
h1.setFormatter(logging.Formatter("%(levelname)s | %(message)s"))
h2 = logging.StreamHandler(logfile)          # stands in for FileHandler
h2.setLevel(logging.DEBUG)
h2.setFormatter(logging.Formatter("%(name)s %(levelname)-7s %(message)s"))
log.addHandler(h1)
log.addHandler(h2)

log.debug("fold 1 started")
log.info("fold 1 accuracy=0.81")
log.warning("fold 2 has few positives")

print("CONSOLE:")
print(console.getvalue(), end="")
print("FILE:")
print(logfile.getvalue(), end="")
```

```
CONSOLE:
WARNING | fold 2 has few positives
FILE:
train DEBUG   fold 1 started
train INFO    fold 1 accuracy=0.81
train WARNING fold 2 has few positives
```

1. The logger level (DEBUG) is the first gate: nothing below it is created. Each handler then applies its own level as a second gate.
2. `propagate = False` stops the record from also reaching the root logger, which would print duplicates if the root has a handler (a very common surprise).
3. `h1` only accepts WARNING and above and uses a short format. `h2` accepts everything and uses a longer one. `StringIO` stands in for a file so we can print what would be in it.
4. Output: the console received one line, the file received three lines, with the level padded to 7 characters by `%(levelname)-7s`.



One record, two gates: the logger level and then each handler level.

How to read it: follow an `info` record from the left. It passes the logger (DEBUG allows it), reaches both handlers, is blocked by the console handler (INFO is below WARNING) and written by the file handler. Always ask "which gate dropped my message?": the logger level, a handler level, or `propagate`.

Example 8: dictConfig and exceptions

Larger programs keep the configuration in one dictionary (often loaded from YAML), so changing the format needs no code change. Inside `except`, use `log.exception` to record the traceback.

```
import logging
import logging.config
import sys

class ShortExc(logging.Formatter):
    """Keep only the last traceback line (full paths differ on every machine)."""
    def formatException(self, exc_info):
        return super().formatException(exc_info).splitlines()[-1]

config = {
    "version": 1,
    "disable_existing_loggers": False,
    "formatters": {"plain": {"()": ShortExc, "format": "%(levelname)s [%(name)s] %(message)s"}},
    "handlers": {"out": {"class": "logging.StreamHandler", "stream": "ext://sys.stdout",
                         "formatter": "plain", "level": "INFO"}},
    "loggers": {"pipeline": {"level": "DEBUG", "handlers": ["out"], "propagate": False}},
}
logging.config.dictConfig(config)
log = logging.getLogger("pipeline.clean")     # a child of "pipeline"

def parse_age(text):
    try:
        return int(text)
    except ValueError:
        log.exception("bad age %r", text)
        return None

print(parse_age("41"))
print(parse_age("forty"))
```

```
41
ERROR [pipeline.clean] bad age 'forty'
ValueError: invalid literal for int() with base 10: 'forty'
None
```

1. A formatter turns the traceback into text with `formatException`. We override it to keep only the last line, so the output below is stable; in production keep the full traceback. The dictionary key `"()"` means "build this callable instead of the default class".
2. The dictionary describes formatters, handlers and loggers by name; `"ext://sys.stdout"` is the syntax for "the object `sys.stdout`". `disable_existing_loggers: False` keeps loggers that other libraries created before.
3. `getLogger("pipeline.clean")` is a child of `pipeline`; dots make the hierarchy, so the child inherits the handler configured on its parent.
4. `parse_age("41")` returns 41, nothing is logged. `parse_age("forty")` raises inside `int`, the `except` block logs at ERROR level with the traceback attached, and the function returns `None`.
5. The output shows `41`, then the `ERROR` line followed by the one-line exception `ValueError: invalid literal for int() with base 10: 'forty'`, then `None`.

argparse: turning a script into a command

`argparse` (standard library) reads `sys.argv`, converts and validates the values, writes `--help` for you and exits with a clear message on bad input. In tests you pass a list to `parse_args` instead of reading the real command line.

Name	What it does	Minimal example
`ArgumentParser(description=...)`	Creates the parser	`p = argparse.ArgumentParser(prog="clean")`
positional argument	A required value identified by position	`p.add_argument("csv")`
optional flag	A named value, optional by default	`p.add_argument("--out", default="out.csv")`

Name	What it does	Minimal example
`type=`	Converts the text (int, float, Path, or your function)	`p.add_argument("--n", type=int)`
`choices=`	Restricts to listed values	`choices=["csv", "json"]`
`required=True`	Makes a flag mandatory	`p.add_argument("--key", required=True)`
`action="store_true"`	Boolean switch, False unless given	`p.add_argument("--dry-run", action="store_true")`
`action=BooleanOptionalAction`	Creates `--x` and `--no-x`	`action=argparse.BooleanOptionalAction`
`action="count"`	Counts repeats such as `-vvv`	`p.add_argument("-v", action="count", default=0)`
`nargs`	Number of values: `"+"` one or more, `"*"` zero or more, an int	`p.add_argument("files", nargs="+")`
`dest` , `metavar`	Attribute name; placeholder shown in help	`dest="out_path", metavar="FILE"`
`help=`	Text shown by `-h`	`help="output file"`
`add_subparsers`	Git-style subcommands	`sub = p.add_subparsers(dest="cmd", required=True)`
`set_defaults(func=...)`	Attach the function a subcommand runs	`s.set_defaults(func=run_train)`
`add_mutually_exclusive_group`	Only one of several flags allowed	`g = p.add_mutually_exclusive_group()`
`parse_args(list)`	Parse a given list (or `sys.argv[1:]` when omitted)	`p.parse_args(["data.csv", "--n", "5"])`
`parse_known_args`	Returns unknown arguments instead of failing	`ns, rest = p.parse_known_args()`
`p.error(msg)`	Print usage and exit with status 2	`p.error("--a needs --b")`

Example 9: arguments, defaults and types

```
import argparse

p = argparse.ArgumentParser(prog="clean", description="Clean a CSV file")
p.add_argument("csv", help="input file")
p.add_argument("--out", default="clean.csv", help="output file")
p.add_argument("--min-rows", type=int, default=1, help="fail if fewer rows")
p.add_argument("--dry-run", action="store_true")
p.add_argument("-v", "--verbose", action="count", default=0)

ns = p.parse_args(["orders.csv", "--min-rows", "100", "-vv"])
print(ns)
print(ns.min_rows + 1, ns.dry_run, ns.verbose)
```

```
Namespace(csv='orders.csv', out='clean.csv', min_rows=100, dry_run=False, verbose=2)
101 False 2
```

1. `prog` is the name shown in help. `csv` has no dashes so it is positional and required.

2. `--out` has a default; `--min-rows` has `type=int`, so the text `"100"` becomes the integer 100. Dashes in the flag become underscores in the attribute (`min_rows`).
3. `--dry-run` is a switch: absent means `False`. `-v` with `action="count"` counts how many times it appears; `-vv` gives 2.
4. We pass a list to `parse_args`, exactly what a test would do. The result is a `Namespace`, a simple object with one attribute per argument. The line `101 False 2` proves `min_rows` is a real int.

Example 10: errors and subcommands

```
import argparse
import contextlib
import io

def make_parser():
    p = argparse.ArgumentParser(prog="mlcli")
    sub = p.add_subparsers(dest="cmd", required=True)
    t = sub.add_parser("train", help="train a model")
    t.add_argument("--C", type=float, default=1.0)
    t.add_argument("--model", choices=["logreg", "tree"], default="logreg")
    t.set_defaults(func=lambda a: f"training {a.model} with C={a.C}")
    e = sub.add_parser("eval", help="evaluate a model")
    e.add_argument("run_id")
    e.set_defaults(func=lambda a: f"evaluating run {a.run_id}")
    return p

p = make_parser()
for argv in (["train", "--C", "0.5"], ["eval", "r42"]):
    a = p.parse_args(argv)
    print(a.func(a))

err = io.StringIO()
with contextlib.redirect_stderr(err):
    try:
        p.parse_args(["train", "--model", "svm"])
    except SystemExit as exc:
        print("exit status", exc.code)
print(err.getvalue().strip().splitlines()[-1])
```

```
training logreg with C=0.5
evaluating run r42
exit status 2
mlcli train: error: argument --model: invalid choice: 'svm' (choose from logreg, tree)
```

1. `add_subparsers` creates the `train` and `eval` sub-commands, each with its own flags; `dest="cmd"` stores which was chosen and `required=True` forces a choice.
2. `set_defaults(func=...)` stores the function on the namespace, so the main program is just `args.func(args)`, with no long `if/elif` chain.
3. The first loop prints `training logreg with C=0.5` (default model) and `evaluating run r42`.
4. Invalid input does not raise a normal exception: argparse prints usage plus a message to stderr and calls `sys.exit(2)`. We catch `SystemExit` and capture stderr to show it: status 2, and a final line saying that `'svm'` is an invalid choice and listing the allowed ones.

Typer and Click (not run here)

Click builds commands from decorators. **Typer** is built on Click and uses ordinary type hints, so the function signature is the interface. Neither is installed here; the code is the standard form. Use argparse for a small script with no dependencies, Typer for a larger tool where help text, completion and tests matter.

Typer / Click item	What it does	Minimal example
`typer.Typer()`	The application; holds commands	`app = typer.Typer()`
`@app.command()`	Registers a function as a command	`@app.command()` `def train(...): ...`
parameter without default	Becomes a required argument	`def run(csv: Path)`
parameter with default	Becomes an option `--n`	`def run(n: int = 5)`
`bool` parameter	Becomes `--flag/--no-flag`	`dry_run: bool = False`
`typer.Option(...)`, `Argument(...)`	Customise help text, names, validation	`Annotated[int, typer.Option(min=1)]`
`typer.Exit(code)`	Stop with an exit status	`raise typer.Exit(code=1)`
`typer.echo`	Print (handles encodings)	`typer.echo("done")`
`typer.testing.CliRunner`	Invoke the app inside a test	`CliRunner().invoke(app, ["train"])`
`click.command / option / argument`	The Click equivalents	`@click.option("--n", default=5)`
`click.group()`	A group of subcommands	`@click.group()`

```
# NOT RUN HERE: typer is not installed
from pathlib import Path
from typing import Annotated

import typer

app = typer.Typer(help="Tools for order files")

@app.command()
def clean(
    csv: Annotated[Path, typer.Argument(exists=True, help="input CSV")],
    out: Annotated[Path, typer.Option(help="output file")] = Path("clean.csv"),
    min_rows: Annotated[int, typer.Option(min=1)] = 1,
    dry_run: bool = False,
):
    """Remove duplicate rows."""
    typer.echo(f"cleaning {csv} -> {out} (dry_run={dry_run})")

@app.command()
def version():
    typer.echo("1.0")

if __name__ == "__main__":
    app()
```

```
# NOT RUN HERE: click is not installed
import click

@click.command()
@click.argument("csv", type=click.Path(exists=True))
@click.option("--out", default="clean.csv", show_default=True)
@click.option("--min-rows", type=int, default=1)
@click.option("--dry-run", is_flag=True)
def clean(csv, out, min_rows, dry_run):
    """Remove duplicate rows."""
    click.echo(f"cleaning {csv} -> {out}")

if __name__ == "__main__":
    clean()
```

The Typer function reads like normal Python: `Path` makes Typer check the type, `exists=True` makes it verify the file, `min=1` rejects zero, and `bool` becomes `--dry-run / --no-dry-run`. Docstrings become help text. Test with `CliRunner().invoke(app, ["clean", "x.csv"])` and check `result.exit_code` and `result.stdout`. In a package, register the entry point in `pyproject.toml` under `[project.scripts]` (shown in section 23.4).

pathlib: paths as objects

Strings like `"data/" + name + ".csv"` break on Windows and when a slash is missing. `pathlib.Path` wraps a path in an object with methods; the `/` operator joins parts correctly on every operating system.

Name	What it does	Minimal example
`Path("a/b.csv")`	Creates a path (nothing is touched on disk)	`p = Path("data") / "raw" / "a.csv"`
`Path.cwd()`, `Path.home()`	Current folder, home folder	`Path.home() / ".config"`
`.name`, `.stem`, `.suffix`	"a.tar.gz" splits to name, "a.tar", ".gz"	`Path("x.csv").suffix`
`.suffixes`	All suffixes as a list	`[".tar", ".gz"]`
`.parent`, `.parents`	The folder above; the sequence of all ancestors	`p.parent.name`
`.with_suffix`, `.with_name`, `.with_stem`	Returns a modified copy	`p.with_suffix(".parquet")`
`.exists()`, `.is_file()`, `.is_dir()`	Tests on disk	`if p.is_file(): ...`
`.mkdir(parents=True, exist_ok=True)`	Create a folder, with parents, no error if present	`out.parent.mkdir(parents=True, exist_ok=True)`
`.iterdir()`	Items directly inside a folder	`list(folder.iterdir())`
`.glob(pattern)`, `.rglob`	Match files; `rglob` searches all subfolders	`folder.rglob("*.csv")`
`.read_text()`, `.write_text()`	Whole text file in or out	`p.write_text("hi", encoding="utf-8")`
`.read_bytes()`, `.write_bytes()`	Binary equivalents	`p.read_bytes()`
`.open()`	Same as built-in `open`	`with p.open() as f: ...`
`.stat().st_size`	Size in bytes (and times)	`p.stat().st_size`
`.resolve()`	Absolute path with `..` and symlinks resolved	`Path("../x").resolve()`
`.relative_to(base)`	Path relative to a folder	`p.relative_to(root)`

Name	What it does	Minimal example
`.rename()`, `.replace()`	Move; `replace` overwrites an existing target	`tmp.replace(final)`
`.unlink(missing_ok=True)`	Delete a file	`p.unlink(missing_ok=True)`
`.touch()`	Create an empty file or update its time	`(d / "_SUCCESS").touch()`
`.expanduser()`	Replace `~` by the home folder	`Path("~/data").expanduser()`
`__file__` idiom	Folder of the current script, independent of the working directory	`HERE = Path(__file__).resolve().parent`

Example 11: taking a path apart and building one

```
from pathlib import Path, PurePosixPath

p = PurePosixPath("/data/raw/orders_2026.csv.gz")
print(p.name)
print(p.stem)
print(p.suffix, p.suffixes)
print(p.parent)
print(p.with_suffix(""))
print(p.parents[1])

root = PurePosixPath("/data")
print(p.relative_to(root))
print(root / "clean" / "orders.parquet")
```

```
orders_2026.csv.gz
orders_2026.csv
.gz ['.csv', '.gz']
/data/raw
/data/raw/orders_2026.csv
/data
raw/orders_2026.csv.gz
/data/clean/orders.parquet
```

1. `PurePosixPath` is the "paths only, never touch the disk" cousin of `Path`. We use it so the output is identical on every operating system; `Path` has the same methods.
2. `name` is the final part, `orders_2026.csv.gz`. `stem` removes only the last suffix, giving `orders_2026.csv`. `suffix` is `.gz`, while `suffixes` lists `['.csv', '.gz']`.
3. `parent` is `/data/raw`; `parents[1]` is two levels up, `/data`. `with_suffix("")` removes the last suffix.
4. `relative_to(root)` gives `raw/orders_2026.csv.gz`. The `/` operator joins parts without any slash handling by you.

Example 12: files on disk, glob, and a safe write

```
import tempfile
from pathlib import Path

with tempfile.TemporaryDirectory() as tmp:
    root = Path(tmp)
    (root / "raw" / "2026").mkdir(parents=True)
    (root / "raw" / "a.csv").write_text("x\n1\n")
    (root / "raw" / "2026" / "b.csv").write_text("x\n1\n2\n")
    (root / "raw" / "notes.txt").write_text("hello")

    print(sorted(p.name for p in (root / "raw").iterdir()))
    print(sorted(str(p.relative_to(root)) for p in root.rglob("*.csv")))
    print([p.stat().st_size for p in sorted(root.rglob("*.csv"))])

    # safe write: write to a temporary name, then atomically replace
    target = root / "report.json"
    tmp_file = target.with_suffix(".json.tmp")
    tmp_file.write_text('{"ok": true}')
    tmp_file.replace(target)
    print(target.read_text(), tmp_file.exists())
```

```
['2026', 'a.csv', 'notes.txt']
['raw/2026/b.csv', 'raw/a.csv']
[6, 4]
{"ok": true} False
```

1. `TemporaryDirectory` gives a throw-away folder removed at the end of the block. All paths are built from `root` with `/`.
2. `mkdir(parents=True)` creates `raw` and `raw/2026` in one call. Without `parents=True` a missing parent raises `FileNotFoundError`.
3. `iterdir()` lists direct children in arbitrary order, so we sort. The result is two files and one folder.
4. `rglob("*.csv")` finds CSV files in all subfolders. The sizes are 6 and 4 bytes, because the sorted paths put `b.csv` first: `x\n1\n2\n` is six characters, `x\n1\n` is four.
5. The safe-write pattern writes a temporary file and then `replace`s the target. Readers never see a half-written file, because the rename is a single step on the same filesystem. The output ends with the JSON text and `False`: the temporary name no longer exists.

23.4 dataclasses, typing, mypy, Environments, pre-commit and Secrets

dataclasses: records and configuration without boilerplate

A `@dataclass` writes `__init__`, `__repr__` and `__eq__` for you from annotated fields. It is the standard way to hold configuration (learning rate, file paths) so that a typo becomes an error instead of a silent default.

Name	What it does	Minimal example
`@dataclass`	Generates init, repr, eq from annotations	`@dataclass` `class Cfg: lr: float = 0.1`
field with default	A default makes the argument optional (non-default fields must come first)	`seed: int = 0`
`field(default_factory=...)`	A fresh mutable default per object	`tags: list[str] = field(default_factory=list)`
`frozen=True`	Instances are read-only and hashable	`@dataclass(frozen=True)`

Name	What it does	Minimal example
`slots=True`	Less memory, no stray attributes	`@dataclass(slots=True)`
`kw_only=True`	All fields must be passed by name	`@dataclass(kw_only=True)`
`order=True`	Adds `<`, `>` comparing fields in order	`@dataclass(order=True)`
`__post_init__`	Runs after init: validate or derive fields	`if self.lr <= 0: raise ValueError`
`field(repr=False)`	Hide a field from `repr` (secrets)	`token: str = field(repr=False)`
`field(init=False)`	Not an init argument; set it yourself	`total: float = field(init=False)`
`asdict(obj)`	Recursively convert to a dict	`asdict(cfg)`
`astuple(obj)`	Convert to a tuple	`astuple(cfg)`
`replace(obj, **changes)`	Copy with some fields changed (the way to "edit" a frozen one)	`replace(cfg, lr=0.01)`
`fields(cls)`	Introspect field names and types	`[f.name for f in fields(Cfg)]`
`ClassVar`, `InitVar`	Class-level constant; init-only value	`x: ClassVar[int] = 3`

Example 13: a validated, frozen training configuration

```
from dataclasses import dataclass, field, asdict, replace

@dataclass(frozen=True, kw_only=True)
class TrainConfig:
    csv: str
    lr: float = 0.1
    epochs: int = 20
    seed: int = 0
    features: tuple[str, ...] = field(default_factory=tuple)

    def __post_init__(self):
        if self.lr <= 0:
            raise ValueError(f"lr must be positive, got {self.lr}")

cfg = TrainConfig(csv="train.csv", features=("age", "tenure"))
print(cfg)
print(asdict(cfg)["epochs"])
cfg2 = replace(cfg, lr=0.01)
print(cfg2.lr, cfg.lr, cfg == cfg2)

try:
    cfg.lr = 5
except Exception as exc:
    print(type(exc).__name__)
try:
    TrainConfig(csv="x.csv", lr=-1)
except ValueError as exc:
    print(exc)
try:
    TrainConfig(csv="x.csv", learning_rate=0.5)
except TypeError as exc:
    print("TypeError")
```

```
TrainConfig(csv='train.csv', lr=0.1, epochs=20, seed=0, features=('age', 'tenure'))
20
0.01 0.1 False
FrozenInstanceError
lr must be positive, got -1
TypeError
```

1. `frozen=True` makes instances immutable (hashable too); `kw_only=True` forces `TrainConfig(csv=...)` by name so that argument order never matters.
2. Defaults are plain values. `field(default_factory=tuple)` builds a fresh empty tuple (writing `()` would also work for a tuple). For a list or dict you must use `default_factory`: a bare `[]` default is rejected by dataclasses with a `ValueError`, so the classic shared-list bug cannot happen.
3. `__post_init__` runs right after the generated init and raises on invalid values. The error is raised at the moment of creation, not an hour into training.
4. The first output line is the generated `repr`. `asdict` gives a plain dict (the epochs value, 20) which you can dump to JSON.
5. `replace` returns a new object, so `cfg.lr` is still 0.1 while `cfg2.lr` is 0.01 and the two compare unequal (`False`).
6. Assigning to a frozen field raises `FrozenInstanceError`; the negative learning rate prints our message; the misspelt keyword raises `TypeError`. Three kinds of mistakes, three immediate, specific errors.

typing and mypy

Type hints are labels on function signatures and variables. Python does **not** enforce them when the program runs. A separate tool, **mypy** (also pyright), reads the labels and reports inconsistencies without running the code. They catch mistakes such as passing a string where a list is expected, or forgetting that a value may be `None`.

Hint	Meaning	Minimal example
`int`, `float`, `str`, `bool`	Basic types (`int` is accepted where `float` is expected)	`def f(x: float) -> float`
`list[int]`, `dict[str, float]`, `tuple[int, str]`	Containers with element types (built-in generics, Python 3.9+)	`scores: dict[str, float] = {}`
`X | None`	Value or `None` (replaces `Optional[X]`)	`def f(p: Path | None = None)`
`X | Y`	Either type (replaces `Union`)	`id: int | str`
`Literal["a", "b"]`	Only these exact values	`mode: Literal["train", "eval"]`
`TypedDict`	Dict with known keys and value types	`class Row(TypedDict): id: int`
`NamedTuple`	Typed immutable tuple with names	`class P(NamedTuple): x: int`
`Callable[[int], str]`	A function taking an int returning a str	`fn: Callable[[float], float]`
`Protocol`	"Anything with these methods" (structural typing)	`class Model(Protocol): def predict(self, X): ...`
generics (`TypeVar`, `def f[T](x: T) -> T`)	Same type in and out	`def first[T](xs: list[T]) -> T`
`Any`	Switch checking off for this value	`x: Any`
`Final`	Constant that must not be reassigned	`SEED: Final = 42`
`Annotated[T, meta]`	Type plus metadata (used by Typer, FastAPI, pydantic)	`Annotated[int, "units: seconds"]`

Hint	Meaning	Minimal example
`cast(T, x)`	Tell the checker a type without any runtime effect	`cast(list[str], data)`
`TYPE_CHECKING`	Imports only for the checker, avoiding circular imports	`if TYPE_CHECKING: import pandas as pd`
`reveal_type(x)`	mypy prints the inferred type	`reveal_type(df)`
`# type: ignore[code]`	Silence one known error on one line	`x = f() # type: ignore[arg-type]`

Example 14: hints are not enforced, but they are readable at run time

```
import typing

def mean(values: list[float]) -> float:
    return sum(values) / len(values)

print(mean([1.0, 2.0, 6.0]))
print(mean((1, 2, 6)))            # a tuple of ints: works, hints are not enforced
print(typing.get_type_hints(mean))

def find(name: str, table: dict[str, int]) -> int | None:
    return table.get(name)

result = find("b", {"a": 1})
print(result)
if result is not None:
    print(result + 1)
```

```
3.0
3.0
{'values': list[float], 'return': <class 'float'>}
None
```

1. The first call passes a list of floats and returns 3.0 (the sum 9.0 divided by 3).
2. The second call passes a tuple of ints, which violates the hint `list[float]`, yet Python happily returns 3.0. This is the key fact: **hints do not change run-time behaviour**. A checker like mypy would flag this call.
3. `get_type_hints` shows that the hints are stored on the function and can be read by tools. FastAPI, Typer and pydantic do exactly this: they read the hints and build validation and documentation from them.
4. `find` returns `int | None`. The name `"b"` is not in the table, so the result is `None`. The `if result is not None` guard is what a type checker demands before arithmetic, because `None + 1` would crash.

Running mypy (not run here, mypy is not installed). Typical command and the kind of message it prints:

```
python -m pip install mypy
mypy src/                 # check a folder
mypy --strict src/        # strictest mode, good for new code
# example of a report line:
# src/stats.py:12: error: Argument 1 to "mean" has incompatible type "str"; expected "list[float]"  [arg-type]
```

```
[tool.mypy]
python_version = "3.12"
warn_unused_ignores = true
disallow_untyped_defs = true
ignore_missing_imports = true      # for libraries without type information
```

REAL JOB

Do not annotate everything at once. Start with the public functions of the code other people call (parsing, feature building, the predict function), turn on `disallow_untyped_defs` for new modules only, and let mypy run in the pre-commit hook and in continuous integration. Many teams treat typed function signatures as the cheapest documentation they have.

Environments: venv, pip, uv and pyproject.toml

An **environment** is a private folder of installed packages, so project A can use pandas 2 while project B uses pandas 3. `pyproject.toml` is the single file describing the project, its dependencies and the settings of tools. **uv** is a fast, newer installer and project manager from Astral that reads the same file; it is not installed here, so its commands are shown but not run.

Command / section	What it does
`python -m venv .venv`	Creates an environment in `.venv`
`source .venv/bin/activate`	Puts that environment's `python` and `pip` first on the PATH (Windows: `.venv\Scripts\activate`)
`python -m pip install pkg==1.2.3`	Installs an exact version; `python -m pip` guarantees the pip of that python
`pip install -r requirements.txt`	Installs a list of pinned requirements
`pip freeze > requirements.txt`	Records the exact versions installed now
`pip install -e .`	Editable install of your own package (changes apply immediately)
`pip list --outdated`, `pip check`	Show old packages; find broken dependency combinations
`uv venv`, `uv pip install pkg`	uv versions of the above
`uv init`, `uv add pkg`	Create a project; add a dependency to `pyproject.toml` and the lock file
`uv lock`, `uv sync`	Resolve exact versions into `uv.lock`; install exactly those
`uv run pytest`	Run a command inside the project environment
`[project]`	Name, version, `requires-python`, `dependencies`
`[project.optional-dependencies]`	Extras such as `dev = ["pytest"]`, installed with `pip install -e ".[dev]"`
`[project.scripts]`	Command names that map to Python functions
`[build-system]`	Which tool builds the package (setuptools, hatchling, ...)
`[tool.xxx]`	Settings for pytest, mypy, ruff and others

Example 15: a pyproject.toml, read back with the standard library

Python 3.11+ includes `tomllib`, which reads TOML. This is useful for checking in a script that the installed versions satisfy what the project file says.

```
import tomllib

text = """
[project]
name = "churn-tools"
version = "0.1.0"
requires-python = ">=3.11"
dependencies = ["pandas>=2.2", "scikit-learn>=1.5", "typer>=0.12"]

[project.optional-dependencies]
dev = ["pytest>=8", "mypy"]

[project.scripts]
churn = "churn_tools.cli:app"

[tool.pytest.ini_options]
testpaths = ["tests"]
"""
data = tomllib.loads(text)
proj = data["project"]
print(proj["name"], proj["version"])
print(proj["dependencies"])
print(proj["optional-dependencies"]["dev"])
print(proj["scripts"])
print(data["tool"]["pytest"]["ini_options"]["testpaths"])
```

```
churn-tools 0.1.0
['pandas>=2.2', 'scikit-learn>=1.5', 'typer>=0.12']
['pytest>=8', 'mypy']
{'churn': 'churn_tools.cli:app'}
['tests']
```

1. The text is a TOML document: `[tables]` hold `key = value` lines; lists use square brackets; strings use quotes.
2. `tomllib.loads` parses it to nested dictionaries. Note that tables like `[project.scripts]` become `proj["scripts"]`.
3. `dependencies` uses *ranges* (`pandas>=2.2`), because a library should allow compatible versions. Applications (services, training jobs) additionally keep a lock file of exact versions.
4. `scripts` maps the command `churn` to the function `app` in the module `churn_tools.cli` ; after `pip install -e .` , typing `churn` in a terminal runs that function.

```
[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"
```

Example 16: create a real environment and use it

```
import subprocess
import sys
import tempfile
import venv
from pathlib import Path

with tempfile.TemporaryDirectory() as tmp:
    env_dir = Path(tmp) / ".venv"
    venv.EnvBuilder(with_pip=False).create(env_dir)       # with_pip=False keeps it offline and quick
    print((env_dir / "pyvenv.cfg").exists())
    py = env_dir / "bin" / "python"                         # Windows: Scripts\\python.exe
    code = "import sys; print(sys.prefix != sys.base_prefix)"
    out = subprocess.run([str(py), "-c", code], capture_output=True, text=True)
    print(out.stdout.strip())
    out = subprocess.run([str(py), "-c", "import pandas"], capture_output=True, text=True)
    print(out.returncode, out.stderr.strip().splitlines()[-1])
```

```
True
True
1 ModuleNotFoundError: No module named 'pandas'
```

1. `venv.EnvBuilder(...).create(path)` is what `python -m venv` calls. We skip installing pip (`with_pip=False`) to stay offline and fast.
2. The file `pyvenv.cfg` marks a folder as an environment; its presence prints `True`.
3. Inside the environment's own interpreter, `sys.prefix` differs from `sys.base_prefix`, which is the standard test for "am I in a venv?" (`True`).
4. The new environment is empty: importing `pandas` fails with return code 1 and `ModuleNotFoundError`. This is the isolation promise: nothing from your global Python leaks in. It is also why "works on my machine" bugs happen when a package was installed globally and forgotten from `requirements.txt`.

pre-commit

pre-commit runs checks (formatter, linter, type checker, secret scanner) on the files you are about to commit, and blocks the commit if one fails. Install once per clone with `pre-commit install`; run on everything with `pre-commit run --all-files`. Not run here (not installed). Pin each `rev` to a real released tag of the hook repository; the placeholder below must be replaced.

```
# .pre-commit-config.yaml
repos:
  - repo: https://github.com/astral-sh/ruff-pre-commit
    rev: vX.Y.Z            # replace with a real tag
    hooks:
      - id: ruff           # linter
      - id: ruff-format    # formatter
  - repo: https://github.com/pre-commit/pre-commit-hooks
    rev: vX.Y.Z            # replace with a real tag
    hooks:
      - id: end-of-file-fixer
      - id: check-yaml
      - id: check-added-large-files
      - id: detect-private-key
```

Secrets

A secret is a password, token or key. Rules: never in code, never in git (history remembers; if one leaks, **rotate it**, deleting the file is not enough), never in logs, and never in an image layer. Read secrets from environment variables (set by the scheduler, the container platform or a secrets manager) and fail early when one is missing. A local `.env` file is fine for development if it is listed in `.gitignore`; the `python-dotenv` package loads it (not installed here).

Example 17: loading secrets safely

```
import os
from dataclasses import dataclass, field
from unittest import mock

@dataclass(frozen=True)
class Settings:
    db_url: str
    api_token: str = field(repr=False)          # never shown in repr, logs or tracebacks

    @classmethod
    def from_env(cls):
        missing = [k for k in ("DB_URL", "API_TOKEN") if not os.environ.get(k)]
        if missing:
            raise RuntimeError("missing environment variables: " + ", ".join(missing))
        return cls(os.environ["DB_URL"], os.environ["API_TOKEN"])

def mask(secret, keep=2):
    return secret[:keep] + "*" * (len(secret) - keep)

with mock.patch.dict(os.environ, {"DB_URL": "sqlite:///demo.db", "API_TOKEN": "s3cr3t-token"}, clear=True):
    s = Settings.from_env()
print(s)
print(mask(s.api_token))

with mock.patch.dict(os.environ, {"DB_URL": "sqlite:///demo.db"}, clear=True):
    try:
        Settings.from_env()
    except RuntimeError as exc:
        print(exc)
```

```
Settings(db_url='sqlite:///demo.db')
s3**********
missing environment variables: API_TOKEN
```

1. `field(repr=False)` keeps the token out of `print(settings)`, which is how secrets usually leak into logs.
2. `from_env` checks every required variable and reports *all* missing names at once. Failing at start-up is better than a cryptic 401 error later.
3. `mock.patch.dict(..., clear=True)` gives the block a clean fake environment so the example does not depend on your machine; real code just uses `os.environ`.
4. The first printed line shows the settings object without the token. `mask` keeps two characters for debugging (`s3**********`) and hides the rest.
5. The second environment has no token, so the code prints `missing environment variables: API_TOKEN`.

```
# .gitignore (minimum for a data project)
.venv/
.env
__pycache__/
*.pkl
mlruns/
data/raw/
```

COMMON MISTAKE

Committing `.env` or a notebook cell containing a key, then "fixing" it by deleting the line in a later commit. Every earlier commit still holds the secret, and bots scan public repositories within minutes. The fix is: revoke and replace the key first, then clean history if needed, then add a secret scanner (`detect-private-key` above, or a dedicated scanner) to pre-commit.

23.5 Serving and Speed: FastAPI, a Standard-Library API, and joblib

FastAPI

FastAPI turns type-hinted Python functions into a web API. You declare the shape of the input with a **pydantic** model; FastAPI validates every request against it, answers `422` with a precise error list when it is wrong, converts the output to JSON and writes interactive documentation at `/docs`. It runs on an ASGI server such as **uvicorn**. Neither FastAPI, pydantic nor uvicorn is installed here, so the FastAPI code is **not run here**. Right after it, a runnable stdlib server shows the same request and response flow. Use FastAPI when other programs need on-demand predictions; for a nightly batch use a CLI.

Name	What it does	Minimal example
`FastAPI(title=...)`	Creates the application	`app = FastAPI()`
`@app.get / post / put / delete`	Maps an HTTP method and path to a function	`@app.get("/health")`
path parameter	A part of the URL, converted by its hint	`@app.get("/items/{item_id}")` `def f(item_id: int)`
query parameter	A function argument not in the path	`def f(limit: int = 10)` for `?limit=5`
`pydantic.BaseModel`	Typed body schema with validation	`class Item(BaseModel): name: str`
`Field(ge=, le=, gt=, min_length=)`	Constraints on a value	`age: int = Field(ge=18, le=100)`
`response_model=`	Shape of the reply; filters extra fields	`@app.post("/p", response_model=Out)`
`status_code=`	Success status	`@app.post("/items", status_code=201)`
`HTTPException`	Return an error status	`raise HTTPException(404, "not found")`
`Depends`	Dependency injection: shared logic, auth, database sessions	`def f(db=Depends(get_db))`
`lifespan`	Code at start-up and shut-down; load the model once	`FastAPI(lifespan=lifespan)`
`APIRouter`	Group routes in separate files	`router = APIRouter(prefix="/v1")`
`def` vs `async def`	`def` runs in a thread pool; `async def` must never block	use `def` for CPU-bound model calls
`BackgroundTasks`	Do work after the response is sent	`tasks.add_task(log_it, row)`
`Header`, `Cookie`, `Query`	Read or constrain other inputs	`x_key: str = Header()`
`field_validator`	Custom check on one field	`@field_validator("email")`
`model_dump()`	Pydantic model to dict	`item.model_dump()`
`TestClient`	Calls the app in-process, no server needed (uses httpx)	`client = TestClient(app)`
`app.dependency_overrides`	Replace a dependency in tests	`app.dependency_overrides[get_db] = fake`
`uvicorn app:app --reload`	Run the server (`--reload` only for development)	`uvicorn app:app --port 8000`

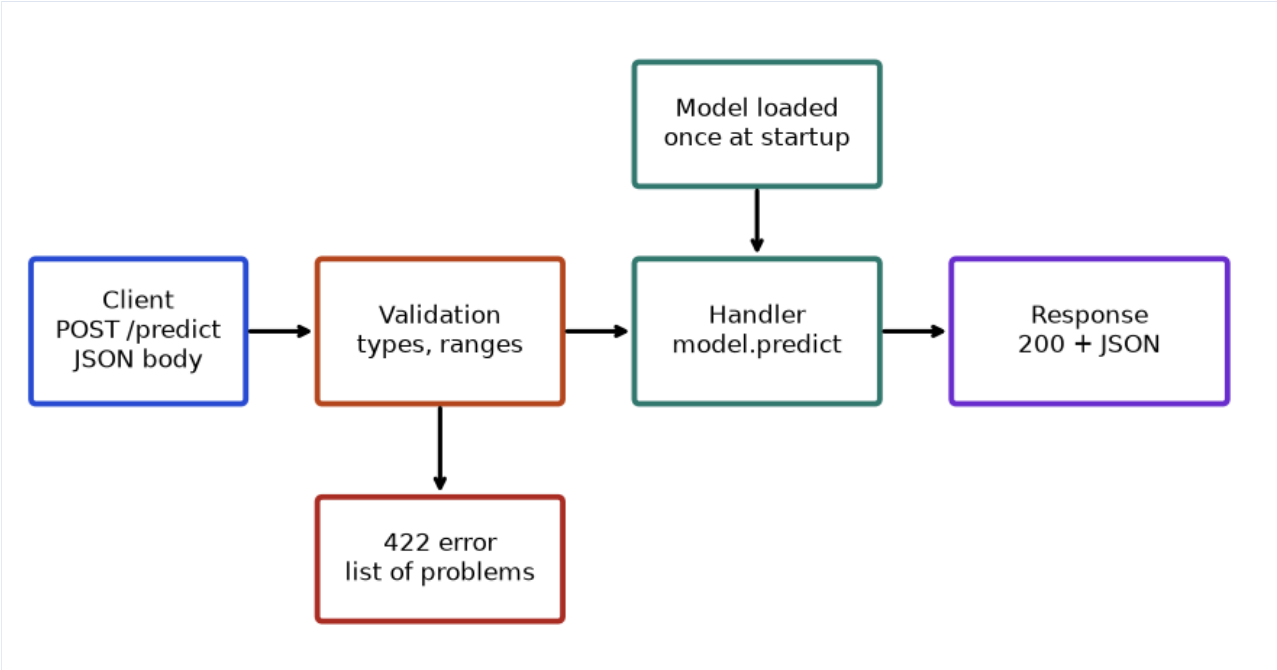
Name	What it does	Minimal example
`/docs`, `/openapi.json`	Automatic interactive docs and machine-readable schema	open in a browser

Example 18: a prediction API with validation (not run here)

```
# app.py  -- NOT RUN HERE: fastapi, pydantic and uvicorn are not installed
from contextlib import asynccontextmanager
from pathlib import Path

import joblib
from fastapi import FastAPI
from pydantic import BaseModel, Field

MODEL_PATH = Path("model.joblib")
state = {}

@asynccontextmanager
async def lifespan(app: FastAPI):
    state["model"] = joblib.load(MODEL_PATH)      # load once, not per request
    yield
    state.clear()

app = FastAPI(title="Churn API", lifespan=lifespan)

class Customer(BaseModel):
    age: int = Field(ge=18, le=100)
    tenure_months: float = Field(ge=0)
    monthly_spend: float = Field(gt=0)

class Prediction(BaseModel):
    churn_probability: float
    label: int

@app.get("/health")
def health():
    return {"status": "ok"}

@app.post("/predict", response_model=Prediction)
def predict(c: Customer):
    x = [[c.age, c.tenure_months, c.monthly_spend]]
    p = float(state["model"].predict_proba(x)[0, 1])
    return Prediction(churn_probability=round(p, 4), label=int(p >= 0.5))
```

```python
# test_app.py  -- NOT RUN HERE
from fastapi.testclient import TestClient
from app import app

def test_health():
    with TestClient(app) as client:               # the with-block runs lifespan
        r = client.get("/health")
        assert r.status_code == 200
        assert r.json() == {"status": "ok"}

def test_predict_ok():
    with TestClient(app) as client:
        r = client.post("/predict", json={"age": 40, "tenure_months": 12, "monthly_spend": 30.5})
        assert r.status_code == 200
        body = r.json()
        assert 0.0 <= body["churn_probability"] <= 1.0
        assert body["label"] in (0, 1)

def test_predict_rejects_young_customer():
    with TestClient(app) as client:
        r = client.post("/predict", json={"age": 17, "tenure_months": 1, "monthly_spend": 5})
        assert r.status_code == 422
        assert r.json()["detail"][0]["loc"][-1] == "age"

def test_predict_missing_field():
    with TestClient(app) as client:
        r = client.post("/predict", json={"age": 30})
        assert r.status_code == 422
```

1. `lifespan` loads the trained model from disk once when the server starts. Loading inside `predict` would re-read the file on every request, a very common slowdown.
2. `Customer` is the contract. `Field(ge=18, le=100)` means "greater or equal 18, less or equal 100"; `gt=0` is strictly greater. A JSON body with the wrong type or range never reaches your function.
3. `response_model=Prediction` fixes the shape of the answer and appears in the generated documentation.
4. `predict` is a plain `def` because `predict_proba` is CPU work; FastAPI runs it in a worker thread so it does not freeze the event loop. Use `async def` only for code that awaits I/O.
5. The tests use `TestClient`, which calls the app in-process through httpx. The `with` form is needed so the lifespan code runs. They check the happy path, a business-rule rejection (age 17 gives status 422 and the error location points at `age`), and a missing field.
6. Run it with `uvicorn app:app --port 8000` and open `http://127.0.0.1:8000/docs` to try the endpoints by hand.



The life of one prediction request.

How to read it: the horizontal path is the happy case. The downward arrow is the rejection path: invalid data never touches the model. The box above the handler is a reminder that the expensive model object is created once, outside the request path.

Example 19: the same flow with the standard library (run)

Python's `http.server` is not for production, but it is perfect for understanding what a framework does for you. This server trains a scikit-learn model, validates the JSON by hand and answers with the same status codes (200 and 422). The client is httpx, which is installed. The server runs in a background thread on a free port (`port 0` means "any free port").

```
import json
import threading
from http.server import BaseHTTPRequestHandler, HTTPServer

import httpx
import numpy as np
from sklearn.linear_model import LogisticRegression

rng = np.random.default_rng(0)
X = np.column_stack([rng.integers(18, 80, 400), rng.uniform(0, 60, 400), rng.uniform(5, 100, 400)])
y = ((X[:, 1] < 12) & (X[:, 2] > 40)).astype(int)
model = LogisticRegression(max_iter=2000).fit(X / [100, 60, 100], y)

RULES = {"age": (18, 100), "tenure_months": (0, 1000), "monthly_spend": (0.01, 10_000)}

def validate(body):
    problems = []
    for name, (lo, hi) in RULES.items():
        v = body.get(name)
        if not isinstance(v, (int, float)) or isinstance(v, bool):
            problems.append({"loc": name, "msg": "field required or not a number"})
        elif not lo <= v <= hi:
            problems.append({"loc": name, "msg": f"must be between {lo} and {hi}"})
    return problems

class Handler(BaseHTTPRequestHandler):
    def _send(self, code, payload):
        data = json.dumps(payload).encode()
        self.send_response(code)
        self.send_header("Content-Type", "application/json")
        self.send_header("Content-Length", str(len(data)))
        self.end_headers()
        self.wfile.write(data)

    def do_GET(self):
        if self.path == "/health":
            self._send(200, {"status": "ok"})
        else:
            self._send(404, {"detail": "not found"})

    def do_POST(self):
        if self.path != "/predict":
            return self._send(404, {"detail": "not found"})
        raw = self.rfile.read(int(self.headers.get("Content-Length", 0)))
        try:
            body = json.loads(raw)
        except json.JSONDecodeError:
            return self._send(400, {"detail": "invalid JSON"})
        problems = validate(body)
        if problems:
            return self._send(422, {"detail": problems})
        x = np.array([[body["age"], body["tenure_months"], body["monthly_spend"]]]) / [100, 60, 100]
        p = float(model.predict_proba(x)[0, 1])
        self._send(200, {"churn_probability": round(p, 4), "label": int(p >= 0.5)})

    def log_message(self, *args):          # keep the console quiet
        pass

server = HTTPServer(("127.0.0.1", 0), Handler)
threading.Thread(target=server.serve_forever, daemon=True).start()
base = f"http://127.0.0.1:{server.server_port}"

with httpx.Client(base_url=base, timeout=30, trust_env=False) as client:
    r = client.get("/health")
```

```
    print(r.status_code, r.json())
    r = client.post("/predict", json={"age": 40, "tenure_months": 3, "monthly_spend": 80})
    print(r.status_code, r.json())
    r = client.post("/predict", json={"age": 40, "tenure_months": 50, "monthly_spend": 10})
    print(r.status_code, r.json())
    r = client.post("/predict", json={"age": 17, "tenure_months": 1, "monthly_spend": 5})
    print(r.status_code, r.json())
    r = client.post("/predict", content=b"not json")
    print(r.status_code, r.json())
    r = client.get("/nothing")
    print(r.status_code)
server.shutdown()
```

```
200 {'status': 'ok'}
200 {'churn_probability': 0.6531, 'label': 1}
200 {'churn_probability': 0.0034, 'label': 0}
422 {'detail': [{'loc': 'age', 'msg': 'must be between 18 and 100'}]}
400 {'detail': 'invalid JSON'}
404
```

1. We generate 400 fake customers and define churn as "short tenure and high spend", then fit a logistic regression on scaled features. The scaling vector `[100, 60, 100]` must be applied again at prediction time; forgetting it is a classic training-serving mismatch.
2. `validate` plays the role of the pydantic model: it checks presence, type and range and returns a list of problems, shaped like FastAPI's `detail` list.
3. The handler class maps HTTP methods to `do_GET` and `do_POST`. `_send` writes the status line, headers (`Content-Length` is needed so the client knows when the body ends) and the JSON bytes.
4. `do_POST` reads exactly `Content-Length` bytes, parses them, returns 400 for broken JSON, 422 for invalid fields and otherwise predicts. This is the same decision flow as the figure above.
5. `HTTPServer(("127.0.0.1", 0), Handler)` binds to a free port; `serve_forever` in a daemon thread lets the same script act as client. The client sets a generous `timeout` and `trust_env=False` so that proxy environment variables on your machine cannot intercept the local call.
6. The outputs, in order: health returns 200 with `ok`; a short-tenure high-spend customer gets probability 0.6531 and label 1; a long-tenure low-spend customer gets 0.0034 and label 0; age 17 returns 422 with the problem list; invalid JSON returns 400; an unknown path returns 404. The probabilities depend on the seeded data and the solver, and are exact for this setup.

INTUITION	A framework adds no magic: it parses the request, validates it, calls your function, converts the answer to JSON and picks the status code. What FastAPI adds is that your type hints do the validation, the documentation and the conversion in one declaration, which is far less code to get wrong.

joblib: cache, parallelise, save models

joblib is a small library used throughout scikit-learn. Three jobs: `dump/load` to save big numpy-heavy objects (like fitted models), `Memory` to cache results of slow functions on disk, and `Parallel/delayed` to run independent calls on several CPU cores.

Name	What it does	Minimal example
`joblib.dump(obj, path)`	Saves an object to disk	`joblib.dump(model, "m.joblib")`
`joblib.load(path)`	Loads it back (only from sources you trust)	`model = joblib.load("m.joblib")`
`compress=`	Smaller files, slower save (0 to 9, or a name)	`joblib.dump(m, p, compress=3)`
`mmap_mode="r"`	Load big arrays lazily from disk	`joblib.load(p, mmap_mode="r")`
`Memory(location, verbose=0)`	Creates a disk cache folder	`mem = Memory("cache_dir")`

Name	What it does	Minimal example
`mem.cache(func)`	Wraps a function so repeated calls with the same arguments read the stored result	`slow = mem.cache(slow)`
`func.clear()`, `mem.clear()`	Delete cached results	`mem.clear(warn=False)`
`Parallel(n_jobs=)`	Runs tasks on several worker processes; `-1` means all cores	`Parallel(n_jobs=4)(...)`
`delayed(f)(args)`	Describes a call without running it	`delayed(f)(3)`
`cpu_count()`	Number of cores	`joblib.cpu_count()`
`parallel_config`	Choose backend and worker count for a block	`with parallel_config(backend="threading", n_jobs=4):`
`joblib.hash(obj)`	Stable fingerprint of any object	`joblib.hash(df)`

Example 20: save and load a model

```
import tempfile
from pathlib import Path

import joblib
import numpy as np
from sklearn.ensemble import RandomForestClassifier

rng = np.random.default_rng(1)
X = rng.normal(size=(300, 5))
y = (X[:, 0] + X[:, 1] > 0).astype(int)
model = RandomForestClassifier(n_estimators=50, random_state=0).fit(X, y)

with tempfile.TemporaryDirectory() as tmp:
    path = Path(tmp) / "model.joblib"
    joblib.dump(model, path)
    small = Path(tmp) / "model_c.joblib"
    joblib.dump(model, small, compress=3)
    print(path.stat().st_size > small.stat().st_size)
    loaded = joblib.load(path)

print(np.array_equal(model.predict(X), loaded.predict(X)))
print(type(loaded).__name__, loaded.n_estimators)
```

```
True
True
RandomForestClassifier 50
```

1. We fit a small random forest on synthetic data whose label depends on the first two columns.
2. `dump` writes the fitted object. `compress=3` trades a little speed for a smaller file; the first printed value (`True`) confirms the compressed file is smaller.
3. `load` rebuilds an equivalent object. The predictions of original and loaded model are identical (`True`).
4. The loaded object is a `RandomForestClassifier` with 50 trees. Remember: joblib files are Python pickles; **loading one can run arbitrary code**, so never load a file from an untrusted source, and record the scikit-learn version used to create it (a model saved with one version may not load in another).

Example 21: cache a slow function

```
import tempfile

from joblib import Memory

calls = []

def slow_square(x):
    calls.append(x)               # a side effect, so we can count real executions
    return x * x

with tempfile.TemporaryDirectory() as tmp:
    memory = Memory(tmp, verbose=0)
    cached = memory.cache(slow_square)
    print(cached(12), cached(12), cached(13))
    print("real calls:", calls)
    cached.clear(warn=False)
    cached(12)
    print("after clear:", calls)
```

```
144 144 169
real calls: [12, 13]
after clear: [12, 13, 12]
```

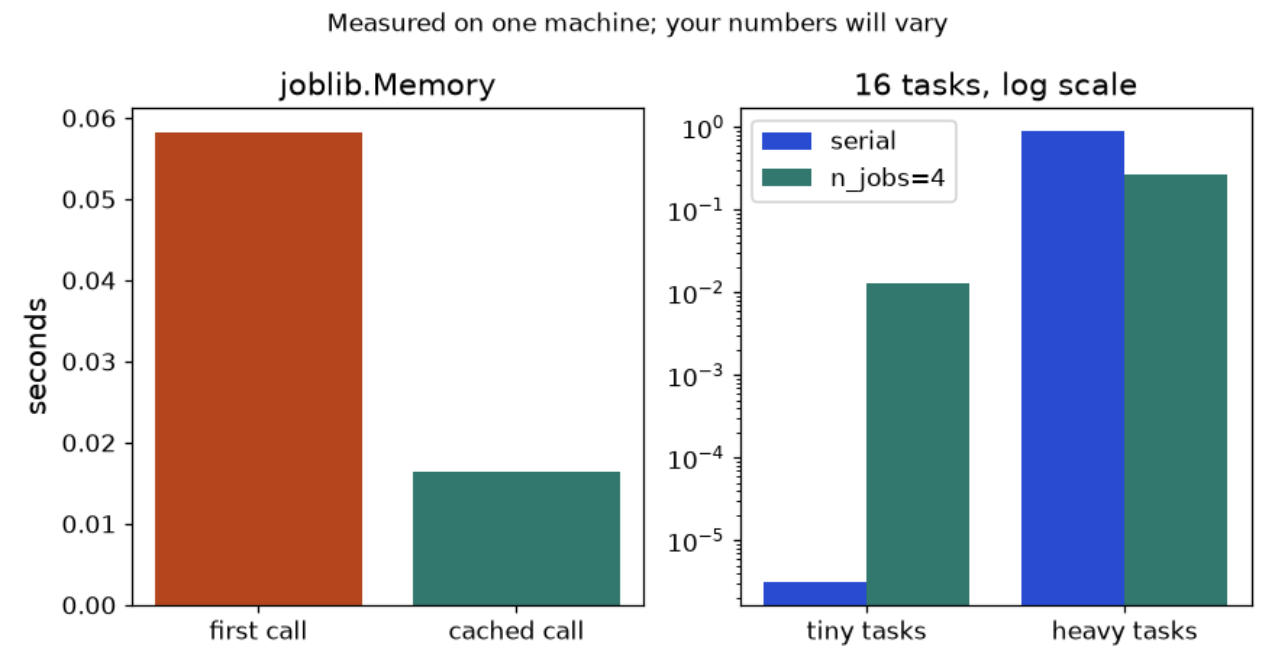
1. `Memory(tmp)` chooses a folder for the cache. In a project use a folder such as `.cache/` that is in `.gitignore`.
2. `memory.cache(slow_square)` returns a wrapped function. joblib computes a hash of the function's source code and arguments to find stored results.
3. The first `cached(12)` executes the function and stores the result; the second finds it on disk. `cached(13)` has a new argument and executes. The print shows `144 144 169` and the real calls are `[12, 13]`: only two executions for three calls.
4. After `clear`, the next call recomputes, so 12 is appended again: `[12, 13, 12]`.

Example 22: Parallel and when it does not help

```
import time
from joblib import Parallel, delayed

def tiny(i):
    return i * i

def heavy(n):
    s = 0
    for k in range(n):
        s += k % 7
    return s

def clock(f):
    t = time.perf_counter()
    f()
    return time.perf_counter() - t

print(Parallel(n_jobs=2)(delayed(tiny)(i) for i in range(6)))

clock(lambda: Parallel(n_jobs=4)(delayed(tiny)(i) for i in range(4)))      # warm up the workers
for name, fn, arg in (("tiny", tiny, 5), ("heavy", heavy, 2_000_000)):
    serial = clock(lambda: [fn(arg) for _ in range(16)])
    par = clock(lambda: Parallel(n_jobs=4)(delayed(fn)(arg) for _ in range(16)))
    print(name, "parallel faster:", par < serial)
```

```
[0, 1, 4, 9, 16, 25]
tiny parallel faster: False
heavy parallel faster: True
```

1. `Parallel(n_jobs=2)(generator)` runs the delayed calls on two worker processes and returns the results **in the order submitted**: `[0, 1, 4, 9, 16, 25]`.
2. `clock` measures wall time with `perf_counter`. The first parallel call only starts the worker processes, so it is excluded from the comparison.
3. For `tiny` tasks (microseconds) sending arguments to another process costs far more than the work, so parallel is slower (`False`). For `heavy` tasks (tens of milliseconds each) the work dominates and four workers win (`True`), on a machine with several cores.
4. We print a boolean rather than a time, because exact timings vary from run to run and machine to machine. The measured bars in the figure below came from one machine and also vary.



Measured on one machine, your numbers will differ. Caching pays on the second call; parallelism pays only when each task is heavy.

How to read it: the left panel compares a first call (computes and stores) with a repeated call (loads the result from disk). The right panel has a logarithmic axis, so each gridline is ten times larger than the one below. For tiny tasks the parallel bar is thousands of times taller, which is overhead; for heavy tasks it is lower than the serial bar, which is the benefit.

23.6 Tracking, Versioning, Shipping and Scheduling: MLflow, DVC, Docker, Airflow and Prefect

This section is about **awareness**: what each tool does, the five lines you need to start, and a runnable stand-in built from the standard library so you understand the idea even without installing the tool. MLflow, DVC, Docker, Airflow and Prefect are all **not run here**.

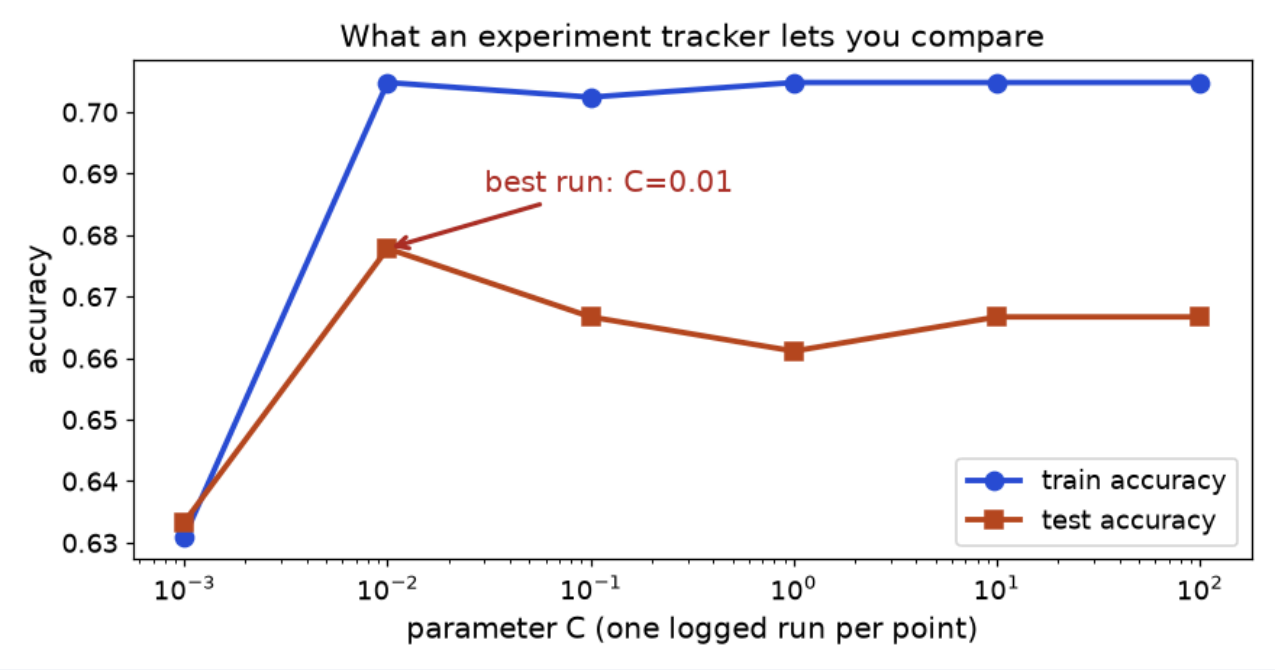
Experiment tracking: the idea, runnable first

Every training run has inputs (parameters, data version, code version) and outputs (metrics, model file). A tracker stores both side by side so that in three weeks you can answer "which settings gave the 0.84?". The smallest possible tracker is a context manager that writes one JSON file per run. MLflow is this idea with a database, a web UI and a model registry.

```
import json
import tempfile
import time
import uuid
from contextlib import contextmanager
from pathlib import Path

import pandas as pd
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

@contextmanager
def start_run(folder, name):
    record = {"run_id": uuid.uuid4().hex[:8], "name": name, "params": {}, "metrics": {}}
    started = time.time()
    try:
        yield record
        record["status"] = "FINISHED"
    except Exception:
        record["status"] = "FAILED"
        raise
    finally:
        record["seconds"] = round(time.time() - started, 3)
        (Path(folder) / f"{record['run_id']}.json").write_text(json.dumps(record))

X, y = make_classification(n_samples=600, n_features=10, n_informative=4, flip_y=0.1, random_state=0)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=0)

with tempfile.TemporaryDirectory() as tmp:
    for C in (0.001, 0.01, 0.1, 1, 10):
        with start_run(tmp, f"logreg-C{C}") as run:
            run["params"]["C"] = C
            m = LogisticRegression(C=C, max_iter=1000).fit(Xtr, ytr)
            run["metrics"]["train_acc"] = round(m.score(Xtr, ytr), 4)
            run["metrics"]["test_acc"] = round(m.score(Xte, yte), 4)
    runs = [json.loads(p.read_text()) for p in Path(tmp).glob("*.json")]

df = pd.json_normalize(runs).sort_values("params.C")
print(df[["name", "params.C", "metrics.train_acc", "metrics.test_acc", "status"]].to_string(index=False))
best = df.loc[df["metrics.test_acc"].idxmax()]
print("best:", best["name"])
```

```
         name  params.C  metrics.train_acc  metrics.test_acc   status
logreg-C0.001     0.001             0.6310            0.6333 FINISHED
 logreg-C0.01     0.010             0.7048            0.6778 FINISHED
  logreg-C0.1     0.100             0.7024            0.6667 FINISHED
    logreg-C1     1.000             0.7048            0.6611 FINISHED
   logreg-C10    10.000             0.7048            0.6667 FINISHED
best: logreg-C0.01
```

1. `start_run` is a context manager: code before `yield` prepares an empty record, the `with` body fills it, and `finally` always saves the record, even when training crashes (status `FAILED`). MLflow's `with mlflow.start_run():` works the same way.
2. The data come from `make_classification` with fixed seeds, so every run sees identical train and test sets. Only the parameter `C` (inverse regularisation strength) changes.
3. Inside the loop, `run["params"]` receives inputs and `run["metrics"]` receives results, like `log_param` and `log_metric`.
4. `pd.json_normalize` flattens nested dictionaries into columns named `params.C` and `metrics.test_acc`; this is also what `mlflow.search_runs()` returns.
5. The table lists five runs sorted by `C`; `idxmax` finds the run with the best test accuracy, printed last (here `logreg-C0.01`). The run ids are random and are not printed.



Six logged runs plotted: the picture an experiment tracker builds for you.

How to read it: each point is one logged run. Blue is accuracy on the data the model trained on, orange is accuracy on unseen data. With very small C the model is over-restricted and both are low. The red arrow marks the best test score; this is the run you would promote. The gap between the lines is a quick overfitting signal.

MLflow

Name	What it does	Minimal example
`mlflow.set_tracking_uri`	Where runs are stored (folder, database or server)	`mlflow.set_tracking_uri("http://localhost:5000")`
`mlflow.set_experiment`	Groups runs under a name	`mlflow.set_experiment("churn")`
`mlflow.start_run`	Opens a run (use as a context manager)	`with mlflow.start_run(run_name="C=0.1"):`
`log_param / log_params`	Record inputs (once per run)	`mlflow.log_param("C", 0.1)`
`log_metric / log_metrics`	Record results; `step=` gives a curve	`mlflow.log_metric("loss", 0.4, step=3)`
`set_tag`	Free labels such as git commit or dataset version	`mlflow.set_tag("data_version", "v3")`
`log_artifact`	Attach a file (plot, report)	`mlflow.log_artifact("confusion.png")`
`mlflow.sklearn.log_model`	Save the fitted model with its environment	`mlflow.sklearn.log_model(model, name="model")`
`mlflow.autolog()`	Automatic logging for supported libraries	`mlflow.sklearn.autolog()`
`mlflow.search_runs`	Query runs into a DataFrame	`mlflow.search_runs(order_by=["metrics.test_acc DESC"])`
`mlflow.pyfunc.load_model`	Load a logged model by URI	`mlflow.pyfunc.load_model("runs:/<run_id>/model")`
`mlflow ui`	Browse runs in a browser	shell command

```
# NOT RUN HERE: mlflow is not installed. Check the docs for your version
# (older releases used log_model(model, artifact_path="model")).
import mlflow
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

X, y = make_classification(n_samples=600, n_features=10, random_state=0)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=0)

mlflow.set_experiment("churn")
for C in (0.01, 0.1, 1):
    with mlflow.start_run(run_name=f"logreg-C{C}"):
        mlflow.log_params({"model": "logreg", "C": C, "seed": 0})
        model = LogisticRegression(C=C, max_iter=1000).fit(Xtr, ytr)
        mlflow.log_metric("test_acc", model.score(Xte, yte))
        mlflow.sklearn.log_model(model, name="model")
```

The structure is identical to the stand-in above: one `with` block per run, parameters first, metrics after training. Launch the browser UI with `mlflow ui` and compare the runs side by side. When there is only one person and a handful of runs, the JSON approach is enough; adopt MLflow when you need a shared server, a model registry or many comparable runs.

DVC: version control for data and pipelines

Git is poor at large files. **DVC** (Data Version Control) stores a small text pointer (`data.csv.dvc`) in git, and the real file in a cache or remote storage (S3, a shared drive). It can also describe the pipeline as stages in `dvc.yaml` , so `dvc repro` re-runs only the stages whose inputs changed.

Command / file	What it does
`dvc init`	Sets up DVC inside a git repository
`dvc add data/raw.csv`	Tracks a file: moves it to the cache, writes `raw.csv.dvc` , adds it to `.gitignore`
`dvc remote add -d store s3://bucket/path`	Defines the default shared storage
`dvc push` / `dvc pull`	Upload / download data for the current git commit
`dvc.yaml`	Pipeline stages: `cmd` , `deps` , `params` , `outs` , `metrics`
`params.yaml`	Default parameter file read by stages
`dvc repro`	Runs stages whose dependencies changed
`dvc.lock`	Exact hashes of the last run (commit it)
`dvc status` , `dvc dag`	What is out of date; draw the stage graph
`dvc metrics show` , `dvc metrics diff`	Print or compare metrics files across commits
`dvc exp run` , `dvc exp show`	Try experiments with changed params without committing each

```
# dvc.yaml  -- NOT RUN HERE: dvc is not installed
stages:
  prepare:
    cmd: python src/prepare.py data/raw.csv data/clean.csv
    deps:
      - src/prepare.py
      - data/raw.csv
    outs:
      - data/clean.csv
  train:
    cmd: python src/train.py data/clean.csv models/model.joblib metrics.json
    deps:
      - src/train.py
      - data/clean.csv
    params:
      - train.C
      - train.seed
    outs:
      - models/model.joblib
    metrics:
      - metrics.json:
          cache: false
```

```
# params.yaml
train:
  C: 0.1
  seed: 0
```

Here `train` depends on the clean data, the script and two parameters. Change `train.C` in `params.yaml` and `dvc repro` re-runs only `train`; change the raw data and it re-runs both. Because `metrics.json` is a metrics file, `dvc metrics diff` compares it between git commits.

Docker for a Python service

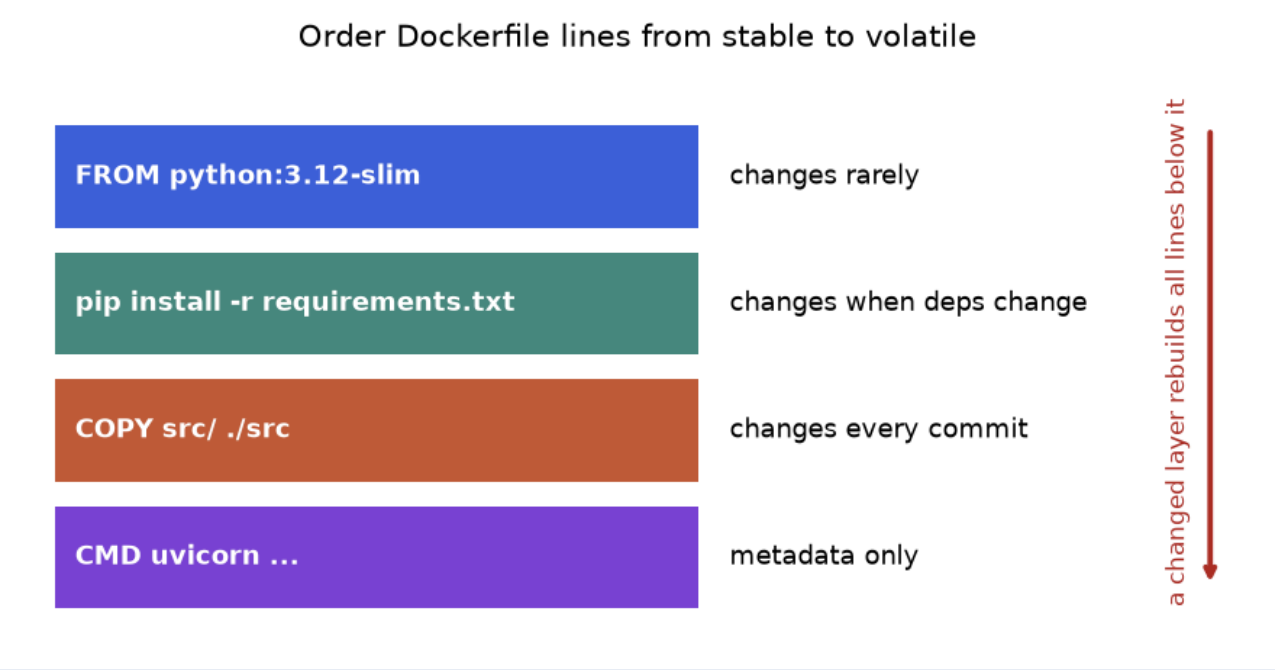
A **Docker image** is a frozen filesystem with your code, Python and libraries; a **container** is a running copy. The same image runs on your laptop, a server and a cloud service. Docker is not installed here, so the files and commands are **not run here**.

Instruction / command	What it does
`FROM python:3.12-slim`	Starting image (pick a tag you chose on purpose, not `latest`)
`WORKDIR /app`	Folder for following instructions
`COPY requirements.txt .`	Copy files from your project into the image
`RUN pip install --no-cache-dir -r requirements.txt`	Run a command at build time; makes a layer
`ENV PYTHONUNBUFFERED=1`	Set an environment variable (logs appear immediately)
`EXPOSE 8000`	Documents the port (does not publish it)
`USER app`	Do not run as root
`CMD [...]` vs `ENTRYPOINT [...]`	Default command; fixed program that `CMD` passes arguments to
`.dockerignore`	Files that must never enter the build (`.venv`, `.git`, `.env`, data)
`docker build -t churn-api:1.0 .`	Build an image from the Dockerfile in this folder
`docker run -p 8000:8000 --env-file .env churn-api:1.0`	Start a container, publish the port, pass secrets at run time

Instruction / command	What it does
`docker logs` , `docker exec -it ... sh` , `docker ps`	Inspect running containers

```
# Dockerfile  -- NOT RUN HERE: docker is not installed
FROM python:3.12-slim

ENV PYTHONUNBUFFERED=1 PIP_NO_CACHE_DIR=1
WORKDIR /app

# 1. dependencies first: this layer is rebuilt only when requirements change
COPY requirements.txt .
RUN pip install -r requirements.txt

# 2. code and model next: these change often
COPY app.py .
COPY model.joblib .

RUN useradd --create-home app
USER app
EXPOSE 8000
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

```
# .dockerignore
.venv
.git
.env
__pycache__
data/
mlruns/
```

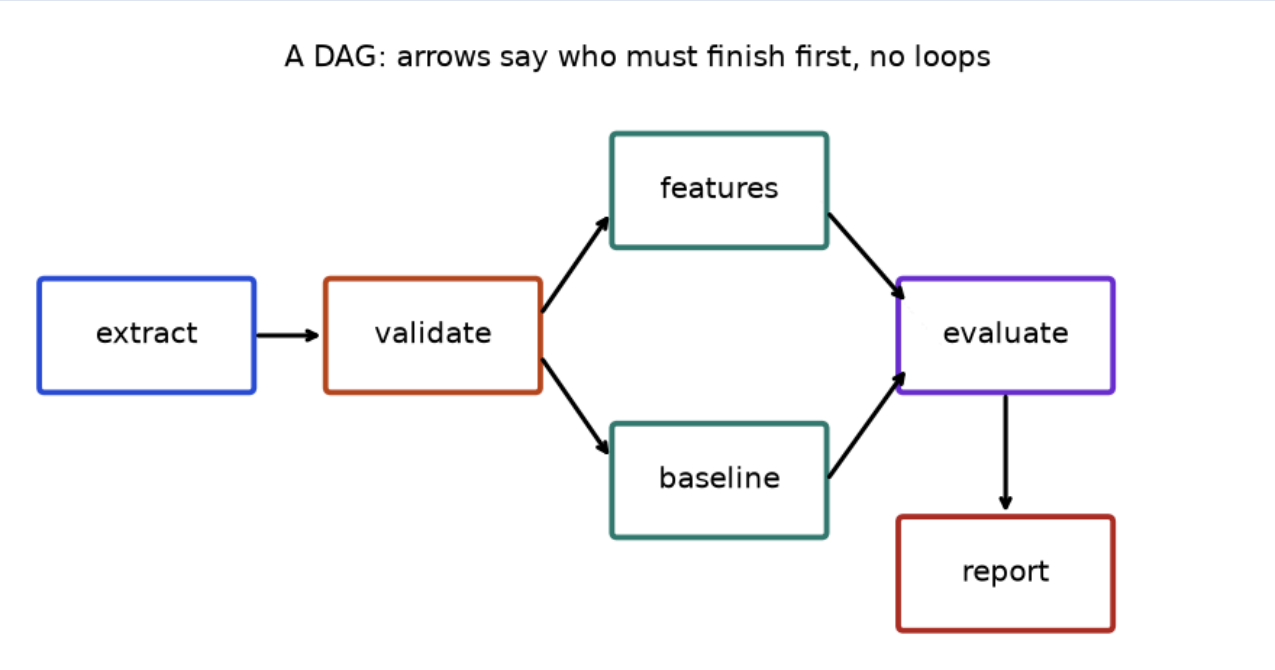


Docker caches each instruction as a layer. Put what changes least at the top.

How to read it: each bar is one Dockerfile instruction. Docker reuses a layer if its instruction and inputs are unchanged, but once one layer changes, every layer after it is rebuilt. Because you edit source code all day and change dependencies rarely, copying `requirements.txt` and installing before copying the code makes most rebuilds take seconds instead of minutes. Binding to `0.0.0.0` is required inside a container, otherwise the published port cannot reach the server.

Orchestration: Airflow and Prefect

An **orchestrator** runs a pipeline as a **DAG** (directed acyclic graph): tasks plus "B after A" arrows, with no loops. It schedules runs, retries failures, keeps history and sends alerts. **Airflow** is the long-established choice (dags defined in Python files, a scheduler and web UI). **Prefect** lets you turn ordinary functions into tasks with decorators. Neither is installed here.



A DAG: tasks and the order constraints between them.

How to read it: an arrow from A to B means B may start only after A succeeded. `features` and `baseline` have no arrow between them, so the orchestrator may run them at the same time. There are no loops, which is what lets the scheduler always find a valid order.

```
# NOT RUN HERE: apache-airflow is not installed. Airflow 3 style; in Airflow 2 import from airflow.decorators.
from datetime import datetime
from airflow.sdk import dag, task

@dag(schedule="0 6 * * 1", start_date=datetime(2026, 1, 1), catchup=False, tags=["churn"])
def weekly_churn():
    @task(retries=2)
    def extract() -> str:
        return "data/raw.csv"

    @task
    def validate(path: str) -> str:
        return path

    @task
    def train(path: str) -> float:
        return 0.81

    @task
    def report(score: float):
        print(f"score={score}")

    report(train(validate(extract())))

weekly_churn()
```

```
# NOT RUN HERE: prefect is not installed (Prefect 3 style)
from prefect import flow, task

@task(retries=3, retry_delay_seconds=60)
def extract() -> str:
    return "data/raw.csv"

@task
def train(path: str) -> float:
    return 0.81

@flow(log_prints=True)
def weekly_churn():
    path = extract()
    score = train(path)
    print(f"score={score}")

if __name__ == "__main__":
    weekly_churn()                                  # run once now
    # weekly_churn.serve(name="weekly", cron="0 6 * * 1")   # or run on a schedule
```

In both, the function calls define the arrows: `train(validate(extract()))` means train waits for validate which waits for extract. `catchup=False` in Airflow stops it from running every missed week since the start date when you first switch the DAG on. The cron string `0 6 * * 1` means 06:00 every Monday. Keep tasks **idempotent**: running one twice with the same input must give the same result and not duplicate rows.

The idea behind the scheduler, runnable with the standard library: `graphlib.TopologicalSorter` orders tasks by their dependencies, and a loop adds retries.

```
from graphlib import TopologicalSorter

deps = {                       # task: the tasks it depends on
    "extract": set(),
    "validate": {"extract"},
    "features": {"validate"},
    "baseline": {"validate"},
    "evaluate": {"features", "baseline"},
    "report": {"evaluate"},
}
attempts = {"baseline": 0}

def run_task(name):
    if name == "baseline":
        attempts["baseline"] += 1
        if attempts["baseline"] < 3:
            raise RuntimeError("flaky source")
    return f"{name} ok"

ts = TopologicalSorter(deps)
ts.prepare()
wave = 0
while ts.is_active():
    ready = list(ts.get_ready())
    print(f"wave {wave}: {sorted(ready)}")
    for name in sorted(ready):
        for attempt in range(1, 4):
            try:
                run_task(name)
                break
            except RuntimeError:
                print(f"  {name} failed on attempt {attempt}")
        ts.done(name)
    wave += 1

try:
    TopologicalSorter({"a": {"b"}, "b": {"a"}}).prepare()
except Exception as exc:
    print(type(exc).__name__)
```

```
wave 0: ['extract']
wave 1: ['validate']
wave 2: ['baseline', 'features']
  baseline failed on attempt 1
  baseline failed on attempt 2
wave 3: ['evaluate']
wave 4: ['report']
CycleError
```

1. `deps` maps each task to the set of tasks that must finish first. This is the dictionary form of the figure.
2. `prepare()` freezes the graph (and detects cycles); `get_ready()` returns all tasks whose dependencies are done; `done(name)` marks one finished, which may unlock others.
3. Each loop iteration is a "wave" of tasks that could run in parallel: extract, then validate, then features and baseline together, then evaluate, then report. The output prints the five waves.
4. `baseline` raises twice (a flaky source), so the retry loop prints two failed attempts and the third succeeds. Real orchestrators add a delay between attempts.
5. A graph with a loop (a needs b, b needs a) can never be ordered; `prepare` raises `CycleError`, which is why the graphs are called acyclic.

23.7 Four Complete Recipes

Now the pieces work together. Each recipe is a complete, runnable solution to a task you will really meet.

Recipe A: a CSV-processing command-line tool, with tests

Task: "Give me a command that reads a sales CSV, groups by a column, sums another column and prints the top N groups. It must fail politely on bad input." The design rule that makes it testable: `main(argv)` takes the argument list and returns an exit code, so tests call it directly instead of starting a process.

```
import argparse
import io
import sys
import tempfile
import unittest
from contextlib import redirect_stderr, redirect_stdout
from pathlib import Path

import pandas as pd

def build_parser():
    p = argparse.ArgumentParser(prog="csvtool", description="Sum a value column by group")
    p.add_argument("csv", type=Path)
    p.add_argument("--group", required=True)
    p.add_argument("--value", required=True)
    p.add_argument("--top", type=int, default=3)
    return p

def top_groups(df, group, value, top):
    for col in (group, value):
        if col not in df.columns:
            raise KeyError(f"column {col!r} not found; have {list(df.columns)}")
    return df.groupby(group)[value].sum().sort_values(ascending=False).head(top)

def main(argv=None):
    args = build_parser().parse_args(argv)
    if not args.csv.is_file():
        print(f"error: file not found: {args.csv}", file=sys.stderr)
        return 2
    try:
        result = top_groups(pd.read_csv(args.csv), args.group, args.value, args.top)
    except KeyError as exc:
        print(f"error: {exc.args[0]}", file=sys.stderr)
        return 2
    print(result.to_string())
    return 0

class TestCsvTool(unittest.TestCase):
    def setUp(self):
        tmp = tempfile.TemporaryDirectory()
        self.addCleanup(tmp.cleanup)
        self.csv = Path(tmp.name) / "sales.csv"
        self.csv.write_text("region,sales\nnorth,10\nsouth,30\nnorth,25\neast,5\nsouth,1\n")

    def run_main(self, *argv):
        out, err = io.StringIO(), io.StringIO()
        with redirect_stdout(out), redirect_stderr(err):
            code = main([str(a) for a in argv])
        return code, out.getvalue(), err.getvalue()

    def test_happy_path(self):
        code, out, _ = self.run_main(self.csv, "--group", "region", "--value", "sales", "--top", 2)
        self.assertEqual(code, 0)
        self.assertEqual(out.split(), ["region", "north", "35", "south", "31"])

    def test_unknown_column(self):
        code, _, err = self.run_main(self.csv, "--group", "city", "--value", "sales")
        self.assertEqual(code, 2)
        self.assertIn("column 'city' not found", err)

    def test_missing_file(self):
        code, _, err = self.run_main("nope.csv", "--group", "a", "--value", "b")
        self.assertEqual(code, 2)
        self.assertIn("file not found", err)
```

```
    def test_pure_function(self):
        df = pd.DataFrame({"g": ["a", "b", "a"], "v": [1, 5, 2]})
        self.assertEqual(top_groups(df, "g", "v", 1).to_dict(), {"b": 5})

suite = unittest.defaultTestLoader.loadTestsFromTestCase(TestCsvTool)
res = unittest.TextTestRunner(stream=io.StringIO()).run(suite)
print(res.testsRun, "tests, ok:", res.wasSuccessful())

with tempfile.TemporaryDirectory() as tmp:
    f = Path(tmp) / "sales.csv"
    f.write_text("region,sales\nnorth,10\nsouth,30\nnorth,25\neast,5\nsouth,1\n")
    print("exit code:", main([str(f), "--group", "region", "--value", "sales", "--top", "2"]))
```

```
4 tests, ok: True
region
north    35
south    31
exit code: 0
```

1. `build_parser` declares the interface. `type=Path` turns the text into a `Path`, so the code can call `is_file()`.
2. `top_groups` holds the logic and knows nothing about the command line, so it can be tested with a tiny DataFrame. It raises `KeyError` with a helpful message listing the available columns.
3. `main` converts problems into messages on stderr and exit code 2 (the convention: 0 success, non-zero failure). It returns the code instead of calling `sys.exit`; the real entry point is `sys.exit(main())`.
4. The tests build a fresh CSV in `setUp`, call `main` with an argument list and capture stdout and stderr with `redirect_stdout` and `redirect_stderr`. They cover the happy path, a bad column, a missing file and the pure function. In pytest you would use the `tmp_path` and `capsys` fixtures instead.
5. The first output line says 4 tests ran and all passed. The final lines are the real tool output for `--top 2`: north (10 + 25 = 35) and south (30 + 1 = 31), then `exit code: 0`.

```
# test_csvtool.py -- NOT RUN HERE: the same tests in pytest style
from csvtool import main

def test_happy_path(tmp_path, capsys):
    f = tmp_path / "sales.csv"
    f.write_text("region,sales\nnorth,10\nsouth,30\nnorth,25\n")
    assert main([str(f), "--group", "region", "--value", "sales", "--top", "1"]) == 0
    assert "north" in capsys.readouterr().out

def test_unknown_column(tmp_path, capsys):
    f = tmp_path / "s.csv"
    f.write_text("a,b\n1,2\n")
    assert main([str(f), "--group", "zzz", "--value", "b"]) == 2
    assert "not found" in capsys.readouterr().err
```

Recipe B: serving a scikit-learn model safely

Task: "Save the model so a different process can serve it, and make it impossible to use it with the wrong features or the wrong library version." A bare `.joblib` file has no memory of which columns, in which order, or which version made it. Save a small *bundle* with the model plus a metadata file, and check on load.

```
import json
import tempfile
from pathlib import Path

import joblib
import numpy as np
import pandas as pd
import sklearn
from sklearn.linear_model import LogisticRegression

FEATURES = ["age", "tenure_months", "monthly_spend"]

def save_bundle(model, folder, metrics):
    folder = Path(folder)
    folder.mkdir(parents=True, exist_ok=True)
    joblib.dump(model, folder / "model.joblib")
    meta = {"features": FEATURES, "sklearn": sklearn.__version__, "metrics": metrics}
    (folder / "meta.json").write_text(json.dumps(meta, indent=2))

def load_bundle(folder):
    folder = Path(folder)
    meta = json.loads((folder / "meta.json").read_text())
    if meta["sklearn"] != sklearn.__version__:
        raise RuntimeError(f"model built with scikit-learn {meta['sklearn']}, running {sklearn.__version__}")
    return joblib.load(folder / "model.joblib"), meta

def predict(model, meta, rows):
    df = pd.DataFrame(rows)
    missing = set(meta["features"]) - set(df.columns)
    if missing:
        raise ValueError(f"missing features: {sorted(missing)}")
    return model.predict_proba(df[meta["features"]].to_numpy())[:, 1]     # enforce order

rng = np.random.default_rng(0)
X = pd.DataFrame({"age": rng.integers(18, 80, 300), "tenure_months": rng.uniform(0, 60, 300),
                  "monthly_spend": rng.uniform(5, 100, 300)})
y = ((X["tenure_months"] < 12) & (X["monthly_spend"] > 40)).astype(int)
model = LogisticRegression(max_iter=2000).fit(X[FEATURES].to_numpy() / [100, 60, 100], y)

with tempfile.TemporaryDirectory() as tmp:
    save_bundle(model, Path(tmp) / "v1", {"train_acc": round(model.score(X[FEATURES].to_numpy() / [100, 60, 100], y), 3)})
    loaded, meta = load_bundle(Path(tmp) / "v1")
    print(sorted(p.name for p in (Path(tmp) / "v1").iterdir()))
    print(meta["features"], meta["metrics"])
    meta_bad = json.loads((Path(tmp) / "v1" / "meta.json").read_text())
    meta_bad["sklearn"] = "0.0.1"
    (Path(tmp) / "v1" / "meta.json").write_text(json.dumps(meta_bad))
    try:
        load_bundle(Path(tmp) / "v1")
    except RuntimeError as exc:
        print("blocked:", str(exc)[:42])

try:
    predict(loaded, meta, [{"age": 30, "tenure_months": 2}])
except ValueError as exc:
    print(exc)
```

```
['meta.json', 'model.joblib']
['age', 'tenure_months', 'monthly_spend'] {'train_acc': 0.897}
blocked: model built with scikit-learn 0.0.1, runni
missing features: ['monthly_spend']
```

1. The bundle is a folder: `model.joblib` (the fitted object) and `meta.json` (feature names in training order, the scikit-learn version, the headline metric). Both are small and diffable in review.
2. `load_bundle` refuses to load a model built with a different scikit-learn version. Pickled models are tied to the library version, so a silent mismatch can give wrong answers or crash later.
3. `predict` builds a DataFrame from request rows, checks every required feature is present, then selects columns *in the stored order*. This makes input column order irrelevant to the caller, which removes a very common serving bug.
4. The scaling by `[100, 60, 100]` is part of how the model was trained; a more robust design puts the scaler and model in one `sklearn.pipeline.Pipeline` so they are saved together (see the pitfalls).
5. Output: the folder holds the two files; the metadata shows the feature list and a training accuracy; after we tamper with the stored version, loading is blocked with a message starting `model built with scikit-learn 0.0.1, runni`; and a request missing `monthly_spend` raises `missing features: ['monthly_spend']`.

Put `load_bundle` inside the FastAPI `lifespan` from Example 18 and call `predict` in the endpoint; the validation, serving and loading each live in one place.

Recipe C: a reproducible training script that logs parameters and metrics

Task: "Anyone should be able to run `python train.py --C 0.1 --seed 3` and get the same numbers, and every run must append a record of what it did." Reproducibility needs four things pinned: random seeds, the data (fingerprint it), the code and library versions (record them), and the parameters (log them).

```
import argparse
import hashlib
import json
import platform
import tempfile
from dataclasses import asdict, dataclass
from pathlib import Path

import numpy as np
import sklearn
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, roc_auc_score
from sklearn.model_selection import train_test_split

@dataclass(frozen=True)
class Config:
    C: float = 1.0
    seed: int = 0
    n_samples: int = 800

def fingerprint(*arrays):
    h = hashlib.sha256()
    for a in arrays:
        h.update(np.ascontiguousarray(a).tobytes())
    return h.hexdigest()[:12]

def train(cfg):
    X, y = make_classification(n_samples=cfg.n_samples, n_features=12, n_informative=5,
                               flip_y=0.05, random_state=cfg.seed)
    Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, random_state=cfg.seed, stratify=y)
    model = LogisticRegression(C=cfg.C, max_iter=1000, random_state=cfg.seed).fit(Xtr, ytr)
    proba = model.predict_proba(Xte)[:, 1]
    return {
        "params": asdict(cfg),
        "data_hash": fingerprint(X, y),
        "metrics": {"accuracy": round(accuracy_score(yte, proba > 0.5), 4),
                    "roc_auc": round(roc_auc_score(yte, proba), 4)},
        "env": {"python": platform.python_version(), "sklearn": sklearn.__version__, "numpy": np.__version__},
    }

def main(argv=None, out_dir="."):
    p = argparse.ArgumentParser()
    p.add_argument("--C", type=float, default=1.0)
    p.add_argument("--seed", type=int, default=0)
    a = p.parse_args(argv)
    record = train(Config(C=a.C, seed=a.seed))
    path = Path(out_dir) / "runs.jsonl"
    with path.open("a") as f:
        f.write(json.dumps(record, sort_keys=True) + "\n")
    return record

with tempfile.TemporaryDirectory() as tmp:
    r1 = main(["--C", "0.1", "--seed", "3"], tmp)
    r2 = main(["--C", "0.1", "--seed", "3"], tmp)
    r3 = main(["--C", "0.1", "--seed", "4"], tmp)
    lines = (Path(tmp) / "runs.jsonl").read_text().splitlines()

print(len(lines), "records appended")
print(r1["metrics"], r1["data_hash"])
print("same seed identical:", r1 == r2)
print("other seed differs:", r1["metrics"] != r3["metrics"], r1["data_hash"] != r3["data_hash"])
print(sorted(json.loads(lines[0])))
```

```
3 records appended
{'accuracy': 0.715, 'roc_auc': 0.7759} 4ff7770f41e0
same seed identical: True
other seed differs: True True
['data_hash', 'env', 'metrics', 'params']
```

1. `Config` is a frozen dataclass: the complete list of knobs of one run. Everything random derives from `seed`: the dataset generator, the train/test split and the model.
2. `fingerprint` hashes the raw bytes of the arrays with SHA-256 and keeps 12 hex characters. If someone changes the data, the hash changes; two records with the same hash saw the same data.
3. `train` returns one dictionary: the parameters, the data hash, the metrics (rounded) and the environment (Python, scikit-learn and numpy versions). Comparing metrics from different library versions is a common source of "irreproducible" results, so they are recorded.
4. `main` parses the command line and appends one JSON line to `runs.jsonl`. Appending JSON lines never corrupts earlier records and loads straight into pandas with `pd.read_json(path, lines=True)`. The timestamp is deliberately omitted so records can be compared; add one in real use.
5. Output: three records were appended; the metrics dictionary and data hash of the first run; running the same seed twice gives identical records (`True`); a different seed changes both the metrics and the data hash (`True True`); and the stored keys are `['data_hash', 'env', 'metrics', 'params']`.

KEY IDEA	Reproducible means "same inputs give the same outputs", so write down all the inputs: parameters, seed, data fingerprint and library versions. The code version (the git commit hash) belongs in the record too, for example by reading `git rev-parse HEAD` with `subprocess`. Equal results across different machines or different scikit-learn versions are not guaranteed, only on the same environment.

Recipe D: a project skeleton

Task: "Create a layout another person can clone, install and test in two commands." The accepted modern layout puts the package under `src/`, so tests import the installed package, not the folder you happen to be standing in. The script below builds the skeleton in a temporary folder, prints the tree and runs the skeleton's own test.

```
import os
import subprocess
import sys
import tempfile
from pathlib import Path

FILES = {
    "pyproject.toml": '[project]\nname = "churn-tools"\nversion = "0.1.0"\nrequires-python = ">=3.11"\n'
                      'dependencies = ["pandas"]\n\n[project.scripts]\nchurn = "churn_tools.cli:main"\n',
    "README.md": "# churn-tools\n\nInstall: `uv sync` or `pip install -e .[dev]`. Test: `pytest`.\n",
    ".gitignore": ".venv/\n.env\n__pycache__/\n*.pkl\nmlruns/\ndata/raw/\n",
    "src/churn_tools/__init__.py": '__version__ = "0.1.0"\n',
    "src/churn_tools/cli.py": 'def add(a, b):\n    return a + b\n\ndef main():\n    print(add(2, 3))\n',
    "tests/test_cli.py": "import unittest\nfrom churn_tools.cli import add\n\nclass T(unittest.TestCase):\n"
                         "    def test_add(self):\n        self.assertEqual(add(2, 3), 5)\n",
    "data/.gitkeep": "",
}

with tempfile.TemporaryDirectory() as tmp:
    root = Path(tmp) / "churn-tools"
    for rel, text in FILES.items():
        path = root / rel
        path.parent.mkdir(parents=True, exist_ok=True)
        path.write_text(text)
    for p in sorted(root.rglob("*")):
        depth = len(p.relative_to(root).parts) - 1
        print("  " * depth + p.name + ("/" if p.is_dir() else ""))
    env = {**os.environ, "PYTHONPATH": str(root / "src")}
    r = subprocess.run([sys.executable, "-m", "unittest", "discover", "-s", "tests"],
                       cwd=root, env=env, capture_output=True, text=True)
    print(r.returncode, r.stderr.strip().splitlines()[-1])
```

```
.gitignore
README.md
data/
  .gitkeep
pyproject.toml
src/
  churn_tools/
    __init__.py
    cli.py
tests/
  test_cli.py
0 OK
```

1. `FILES` maps each relative path to its content: a `pyproject.toml` with a script entry point, a README that states the two commands, a `.gitignore`, the package under `src/churn_tools/`, a `tests/` folder, and an empty `data/.gitkeep` so git keeps the otherwise empty folder.
2. For each path, `parent.mkdir(parents=True, exist_ok=True)` then `write_text` creates the folder tree and the file.
3. The printing loop uses `rglob("*")`, sorted, and indents by depth; a trailing slash marks folders. The tree is the layout you should aim for.
4. The test run uses a subprocess with `PYTHONPATH` pointing at `src`; with a real install (`pip install -e .`) you would not need that. The result `0 OK` means the process exited with status 0 and unittest printed `OK`.
5. Add `[tool.pytest.ini_options]`, `[tool.mypy]` and the pre-commit file from section 23.4, a `Dockerfile` from 23.6 and you have the whole toolkit in one repository.

23.8 Recipes: "How Do I ...?"

Short, solved, runnable answers to questions that come up every week. Each is standalone.

How do I load a YAML config into a typed object?

```
from dataclasses import dataclass, fields
import yaml

@dataclass(frozen=True)
class Config:
    csv: str
    C: float = 1.0
    seed: int = 0

text = "csv: data/train.csv\nC: 0.5\n"
raw = yaml.safe_load(text)
cfg = Config(**raw)
print(cfg)
try:
    Config(**yaml.safe_load("csv: a.csv\nlearning_rate: 0.1\n"))
except TypeError as exc:
    print("unknown key rejected:", "learning_rate" in str(exc))
print([f.name for f in fields(Config)])
```

```
Config(csv='data/train.csv', C=0.5, seed=0)
unknown key rejected: True
['csv', 'C', 'seed']
```

1. `yaml.safe_load` parses text to a dict. Always use `safe_load`, never `yaml.load` on untrusted text, because the unsafe loader can build arbitrary Python objects.
2. `Config(**raw)` unpacks the dict into keyword arguments, so missing defaults apply (`seed=0`) and unknown keys raise `TypeError`, which catches typos in the YAML.
3. Note: YAML turns `0.5` into a float and `yes` into `True`; check types in `__post_init__` if it matters.

How do I retry a flaky call and log each attempt?

```
import functools
import logging
import sys

logging.basicConfig(level=logging.INFO, stream=sys.stdout, format="%(levelname)s %(message)s")
log = logging.getLogger("retry")

def retry(times=3, exceptions=(ConnectionError,)):
    def deco(fn):
        @functools.wraps(fn)
        def wrapper(*args, **kwargs):
            for attempt in range(1, times + 1):
                try:
                    return fn(*args, **kwargs)
                except exceptions as exc:
                    log.warning("%s failed (attempt %d/%d): %s", fn.__name__, attempt, times, exc)
                    if attempt == times:
                        raise
        return wrapper
    return deco

state = {"n": 0}

@retry(times=3)
def fetch():
    state["n"] += 1
    if state["n"] < 3:
        raise ConnectionError("timeout")
    return "data"

print(fetch())
```

```
WARNING fetch failed (attempt 1/3): timeout
WARNING fetch failed (attempt 2/3): timeout
data
```

1. A decorator with arguments needs three levels: `retry(...)` returns `deco`, which takes the function and returns `wrapper`. `functools.wraps` keeps the original name and docstring.
2. Only the listed exceptions are retried; a `ValueError` from a bug would propagate at once, as it should.
3. Two warnings are logged, then the third attempt returns `data`. Real code adds a growing delay (backoff) between attempts, and a limit so a dead service does not hang forever.

How do I process a big CSV in chunks?

```
import tempfile
from pathlib import Path

import pandas as pd

with tempfile.TemporaryDirectory() as tmp:
    path = Path(tmp) / "big.csv"
    pd.DataFrame({"user": range(10), "amount": [5.0] * 10}).to_csv(path, index=False)
    total, rows = 0.0, 0
    for i, chunk in enumerate(pd.read_csv(path, chunksize=4)):
        total += chunk["amount"].sum()
        rows += len(chunk)
        print(f"chunk {i}: {len(chunk)} rows")
print(rows, total)
```

```
chunk 0: 4 rows
chunk 1: 4 rows
chunk 2: 2 rows
10 50.0
```

1. `chunksize=4` makes `read_csv` return an iterator of DataFrames of at most 4 rows (use 100,000 or more in real life). Only one chunk is in memory at a time.
2. Keep running totals outside the loop; anything that needs all rows at once (a median, a sort) needs a different plan, such as a database or Parquet.
3. The output shows chunks of 4, 4 and 2 rows and the final totals `10 50.0`.

How do I fingerprint a dataset so I know it changed?

```
import hashlib
import pandas as pd

def df_hash(df):
    row_hashes = pd.util.hash_pandas_object(df, index=True).to_numpy()
    return hashlib.sha256(row_hashes.tobytes()).hexdigest()[:10]

a = pd.DataFrame({"x": [1, 2, 3], "y": ["a", "b", "c"]})
b = a.copy()
c = a.copy()
c.loc[1, "x"] = 99
print(df_hash(a) == df_hash(b), df_hash(a) == df_hash(c))
print(len(df_hash(a)))
```

```
True False
10
```

1. `hash_pandas_object` returns one 64-bit hash per row (including the index), which we combine with SHA-256 into one short string.
2. An unchanged copy has the same fingerprint (`True`); changing one cell changes it (`False`).
3. For a file on disk, hash the file bytes in blocks with `hashlib` instead; that is what DVC does.

How do I test code that calls an HTTP API without the network?

```
import httpx

def get_user_name(client, user_id):
    r = client.get(f"/users/{user_id}")
    r.raise_for_status()
    return r.json()["name"]

def handler(request):
    if request.url.path == "/users/1":
        return httpx.Response(200, json={"name": "Ada"})
    return httpx.Response(404, json={"detail": "not found"})

client = httpx.Client(transport=httpx.MockTransport(handler), base_url="http://api.test")
print(get_user_name(client, 1))
try:
    get_user_name(client, 2)
except httpx.HTTPStatusError as exc:
    print(exc.response.status_code)
```

```
Ada
404
```

1. `httpx.MockTransport` replaces the network with a Python function that receives the request and returns a response. No server and no patching.
2. The code under test receives the client as an argument (dependency injection), which is why it is easy to test. If it built its own client inside, you would have to patch.
3. User 1 gives `Ada`; user 2 gives a 404, which `raise_for_status` turns into `HTTPStatusError`, printing `404`.

How do I find the newest file matching a pattern?

```
import os
import tempfile
from pathlib import Path

with tempfile.TemporaryDirectory() as tmp:
    root = Path(tmp)
    for i, name in enumerate(["export_01.csv", "export_02.csv", "notes.txt", "export_03.csv"]):
        f = root / name
        f.write_text("x")
        os.utime(f, (1_700_000_000 + i * 100, 1_700_000_000 + i * 100))   # set modified times
    newest = max(root.glob("export_*.csv"), key=lambda p: p.stat().st_mtime)
    print(newest.name)
    print(sorted(p.name for p in root.glob("export_*.csv"))[-1])
```

```
export_03.csv
export_03.csv
```

1. We set the modification times with `os.utime` so the example is deterministic.
2. `max(..., key=...)` picks the file with the largest modification time, `st_mtime`. Both lines print `export_03.csv`.
3. The second line shows a common alternative: if file names contain sortable dates, sorting names is simpler and does not depend on file times (which copying can change).

How do I cache an expensive feature computation across runs?

```
import tempfile

import numpy as np
import pandas as pd
from joblib import Memory

counter = {"n": 0}

def build_features(n_rows, seed):
    counter["n"] += 1
    rng = np.random.default_rng(seed)
    return pd.DataFrame({"a": rng.normal(size=n_rows)}).assign(b=lambda d: d["a"] ** 2)

with tempfile.TemporaryDirectory() as tmp:
    cached = Memory(tmp, verbose=0).cache(build_features)
    f1 = cached(1000, 7)
    f2 = cached(1000, 7)
    f3 = cached(1000, 8)
print(counter["n"], f1.equals(f2), f1.equals(f3), f1.shape)
```

```
2 True False (1000, 2)
```

1. The cache key is made from the arguments (and the function's code), so a different seed gives a separate entry, and changing the function body invalidates old entries.
2. Three calls, but the function body ran only twice (`counter` is 2); the repeat returned the stored frame, equal to the first.
3. Cache only pure functions (same inputs, same output, no hidden dependence on files or time). A function that reads a file whose content changes would serve stale results; pass a file fingerprint as an argument.

How do I read my run log (JSON lines) into pandas?

```
import io
import pandas as pd

log = io.StringIO(
    '{"run": "a", "C": 0.1, "acc": 0.81}\n'
    '{"run": "b", "C": 1.0, "acc": 0.84}\n'
    '{"run": "c", "C": 10.0, "acc": 0.83}\n'
)
df = pd.read_json(log, lines=True)
print({c: str(t) for c, t in df.dtypes.items()})
print(df.sort_values("acc", ascending=False).head(1).to_string(index=False))
```

```
{'run': 'str', 'C': 'float64', 'acc': 'float64'}
run   C  acc
  b 1.0 0.84
```

1. `lines=True` tells pandas that each line is one JSON record.
2. Types are inferred: `run` is text (a string column in pandas 3), `C` and `acc` are floats.
3. Sorting by accuracy puts run `b` first. Append-only JSON lines make experiment history trivial to analyse.

How do I save a dataclass with Path and datetime fields as JSON?

```
import json
from dataclasses import asdict, dataclass
from datetime import datetime
from pathlib import Path

@dataclass
class RunInfo:
    data: Path
    started: datetime
    C: float

def encode(obj):
    if isinstance(obj, Path):
        return obj.as_posix()
    if isinstance(obj, datetime):
        return obj.isoformat()
    raise TypeError(f"cannot serialise {type(obj).__name__}")

info = RunInfo(Path("data/train.csv"), datetime(2026, 11, 9, 6, 0), 0.5)
text = json.dumps(asdict(info), default=encode)
print(text)
print(json.loads(text)["started"][:10])
```

```
{"data": "data/train.csv", "started": "2026-11-09T06:00:00", "C": 0.5}
2026-11-09
```

1. `json.dumps` does not know `Path` or `datetime`. The `default` function is called for every object it cannot handle and must return something serialisable.
2. Paths become POSIX strings (forward slashes) so the file reads the same everywhere; datetimes become ISO 8601 text, which sorts correctly as text.
3. Raise `TypeError` for anything unknown rather than guessing; silent `str()` conversions hide bugs.

How do I check a DataFrame's schema before training?

```
import pandas as pd

EXPECTED = {"customer_id": "int64", "plan": "str", "monthly_spend": "float64"}

def check_schema(df):
    problems = []
    for col, dtype in EXPECTED.items():
        if col not in df.columns:
            problems.append(f"missing column {col}")
        elif str(df[col].dtype) != dtype:
            problems.append(f"{col}: expected {dtype}, got {df[col].dtype}")
    if "monthly_spend" in df and (df["monthly_spend"] < 0).any():
        problems.append("monthly_spend has negative values")
    return problems

good = pd.DataFrame({"customer_id": [1, 2], "plan": ["a", "b"], "monthly_spend": [9.5, 20.0]})
bad = pd.DataFrame({"customer_id": ["1", "2"], "monthly_spend": [-1.0, 3.0]})
print(check_schema(good))
for p in check_schema(bad):
    print(p)
```

```
[]
customer_id: expected int64, got str
missing column plan
monthly_spend has negative values
```

1. In pandas 3.0 text columns have the dedicated `str` dtype by default (before 3.0 they were `object`), so the expected name is `"str"`. Check your pandas version if this fails.
2. The function collects all problems instead of stopping at the first, so one run reports everything wrong with a file.
3. The good frame returns an empty list. The bad frame reports a wrong type (`customer_id` is text), a missing column and negative values. For larger projects use pandera or pydantic models for the same idea.

How do I read a .env file without extra packages?

```
import os

def parse_env(text):
    values = {}
    for line in text.splitlines():
        line = line.strip()
        if not line or line.startswith("#") or "=" not in line:
            continue
        key, _, value = line.partition("=")
        values[key.strip()] = value.strip().strip('"').strip("'")
    return values

sample = '# demo\nDB_URL="sqlite:///demo.db"\nDEBUG=1\n\nTOKEN = abc=def\n'
env = parse_env(sample)
print(env)
for k, v in env.items():
    os.environ.setdefault(k, v)           # real environment variables win
print(os.environ["DEBUG"])
```

```
{'DB_URL': 'sqlite:///demo.db', 'DEBUG': '1', 'TOKEN': 'abc=def'}
1
```

1. Blank lines and comments are skipped. `partition("=")` splits at the *first* equals sign, so a value containing `=` (like `abc=def`) survives.
2. Surrounding quotes are removed. The printed dictionary has three keys.
3. `setdefault` only sets a variable that is not already set, so the real environment (from the server) overrides the file. In practice use the `python-dotenv` package, which handles more edge cases.

How do I run an external command safely?

```
import subprocess
import sys

result = subprocess.run([sys.executable, "-c", "print('hello from child')"],
                        capture_output=True, text=True, check=True, timeout=30)
print(result.stdout.strip(), result.returncode)

try:
    subprocess.run([sys.executable, "-c", "import sys; sys.exit(3)"], check=True)
except subprocess.CalledProcessError as exc:
    print("failed with", exc.returncode)
```

```
hello from child 0
failed with 3
```

1. Pass the command as a **list** of arguments, not one string with `shell=True`, so spaces and special characters in values cannot change the command (shell injection).
2. `capture_output=True, text=True` collects stdout and stderr as strings. `timeout` stops a hung child. `sys.executable` is the current interpreter, so the child uses the same environment.
3. `check=True` turns a non-zero exit status into `CalledProcessError`. The output is `hello from child 0` and then `failed with 3`.

How do I run a job every Monday morning?

Not run here. The simplest scheduler is cron on a server (fields: minute, hour, day of month, month, day of week). Make the job safe to run twice (idempotent) and give it a lock or a unique output name.

```
# crontab -e
# minute hour day-of-month month day-of-week  command
0 6 * * 1 cd /srv/churn && /srv/churn/.venv/bin/python -m churn_tools.cli >> /var/log/churn.log 2>&1
```

Use full paths (cron has a minimal environment), redirect output to a log, and move to Airflow or Prefect as soon as you need retries, dependencies between jobs or alerts.

23.9 Performance and Pitfalls

Performance in one list

- **Measure first.** Use `time.perf_counter` or `cProfile`; never guess. Timings in this chapter vary by machine.
- **Load once.** Load models and read config at start-up (FastAPI `lifespan`), not in every request.
- **Cache pure, slow functions** with `joblib.Memory`; do not cache anything that depends on hidden state.
- **Parallelise heavy tasks only.** Process start-up and data transfer cost milliseconds; a task must take clearly longer. Many scikit-learn estimators already accept `n_jobs`.
- **Order Dockerfile lines** from stable to volatile, so rebuilds reuse layers.
- **Log lazily.** `log.debug("x=%s", x)` does no formatting when DEBUG is off. Do not log inside a loop of a million rows.
- **Keep tests fast** (seconds). Tag slow ones with a marker; small synthetic data in unit tests, real data only in a separate suite.

Fourteen mistakes that cost real time

COMMON MISTAKE	**1. Mutable default values.** A list or dict default is created once and shared by all calls (functions) or rejected (dataclasses).

```
def add_tag_wrong(tag, tags=[]):
    tags.append(tag)
    return tags

def add_tag(tag, tags=None):
    tags = [] if tags is None else tags
    return [*tags, tag]

print(add_tag_wrong("a"), add_tag_wrong("b"))      # symptom: "b" call remembers "a"
print(add_tag("a"), add_tag("b"))
```

```
['a', 'b'] ['a', 'b']
['a'] ['b']
```

1. The wrong version prints `['a', 'b'] ['a', 'b']`: both results are the same list object, because the default list was created once when the function was defined.
2. The fix uses `None` as the default and creates a new list inside. In dataclasses use `field(default_factory=list)`.

COMMON MISTAKE

2. Patching the wrong name. You must patch the name where it is *used*, not where it was defined. After `from math import sqrt` in module `m`, the name `m.sqrt` is a separate reference.

```
import math
import types
from unittest import mock

m = types.ModuleType("m")                  # a stand-in module that did "from math import sqrt"
m.sqrt = math.sqrt
m.root = lambda x: m.sqrt(x)

with mock.patch("math.sqrt", return_value=-1):        # wrong: patches the original only
    print(m.root(9))
with mock.patch.object(m, "sqrt", return_value=-1):   # right: patch the name the code looks up
    print(m.root(9))
```

```
3.0
-1
```

1. The first patch replaces `math.sqrt`, but `m.sqrt` still points to the real function, so the symptom is that the mock "does nothing" and the output is `3.0`.
2. Patching `m.sqrt` (the attribute the code reads) works: output `-1`. With strings, write `patch("mymodule.sqrt")`, where `mymodule` is the module that *imports* the name.

COMMON MISTAKE

3. Duplicate log lines. Adding a handler every time a function runs, or leaving `propagate` on while the root logger also has a handler, prints each message several times.

```
import io
import logging

buf = io.StringIO()
logging.basicConfig(stream=buf, level=logging.INFO, format="%(message)s")   # root handler

def get_logger_wrong():
    lg = logging.getLogger("job")
    lg.setLevel(logging.INFO)
    lg.addHandler(logging.StreamHandler(buf))      # a new handler on every call
    return lg

get_logger_wrong().info("start")
get_logger_wrong().info("start")
print(buf.getvalue().count("start"))

buf.truncate(0); buf.seek(0)
lg = logging.getLogger("job2")                     # fix: no handler here, rely on the root handler
lg.setLevel(logging.INFO)
lg.info("start")
lg.info("start")
print(buf.getvalue().count("start"))
```

```
5
2
```

1. The wrong version: each call adds one more handler to `job`, and the record also propagates to the root handler. First call: 2 copies; second: 3 copies; total 5 lines printed for two messages.
2. The fix: configure handlers once at program start (`basicConfig` or `dictConfig`), and in modules only call `getLogger(__name__)`. Then each message appears once per call (2 for two messages).

COMMON MISTAKE

4. Paths relative to the working directory. `open("data/train.csv")` works when you run from the project root and fails from cron, a notebook folder or a container.

```
import os
import tempfile
from pathlib import Path

with tempfile.TemporaryDirectory() as tmp:
    root = Path(tmp).resolve()
    (root / "data").mkdir()
    (root / "data" / "train.csv").write_text("x\n1\n")
    script = root / "job.py"
    script.write_text("")                   # imagine this is the file containing the code below

    HERE = script.resolve().parent          # fix: anchor to the file, not to the cwd
    os.chdir(tempfile.gettempdir())         # simulate running from somewhere else
    print(Path("data/train.csv").exists())
    print((HERE / "data" / "train.csv").exists())
```

```
False
True
```

1. The relative path is looked up from the current directory, which we changed: `False`. In real code the symptom is `FileNotFoundError` only on the server.
2. The fix anchors paths to the code location with `Path(__file__).resolve().parent` (here simulated with the script path), or takes the data folder as a command-line option: `True`.

COMMON MISTAKE

5. Saving the model but not the preprocessing. Scaling inside a notebook cell and saving only the classifier means the service feeds unscaled numbers into a model trained on scaled ones. The symptom is plausible-looking but wrong predictions, with no error.

```
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

rng = np.random.default_rng(0)
X = np.column_stack([rng.normal(50, 10, 400), rng.normal(0, 1, 400)])
y = (X[:, 0] > 50).astype(int)

scaler = StandardScaler().fit(X)
lonely = LogisticRegression().fit(scaler.transform(X), y)       # wrong to ship this alone
pipe = make_pipeline(StandardScaler(), LogisticRegression()).fit(X, y)

row = np.array([[45.0, 0.0]])                                   # clearly class 0 (45 is below 50)
print(lonely.predict(row), pipe.predict(row))
print(round(lonely.predict_proba(row)[0, 1], 3), round(pipe.predict_proba(row)[0, 1], 3))
```

```
[1] [0]
1.0 0.038
```

1. The lone classifier expects standardised input (roughly -3 to 3). We give it the raw value 45, which it reads as "45 standard deviations above the mean", so it predicts class 1 with probability 1.0. The pipeline scales first, sees a value half a standard deviation below the mean, and correctly predicts class 0 with a low probability. Output: `[1] [0]`, then the two probabilities.
2. The fix is to put preprocessing and model in one `Pipeline` and save the pipeline. One object, one `predict`, no way to forget a step.

COMMON MISTAKE

6. Feature order and names drift. A model fitted on a NumPy array depends only on column *position*. If the service builds features in a different order, nothing fails; the answers are just wrong.

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegression

rng = np.random.default_rng(1)
train = pd.DataFrame({"age": rng.normal(0, 1, 300), "spend": rng.normal(0, 1, 300)})
y = (train["spend"] > 0).astype(int)                          # only "spend" matters
model = LogisticRegression().fit(train.to_numpy(), y)         # names are lost here

request = {"spend": 2.0, "age": -1.0}                         # dictionary in a different order
wrong = model.predict(pd.DataFrame([request]).to_numpy())     # columns: spend, age (swapped)
right = model.predict(pd.DataFrame([request])[["age", "spend"]].to_numpy())
print(wrong, right)

named = LogisticRegression().fit(train, y)                    # fitted on a DataFrame: remembers names
print(list(named.feature_names_in_))
```

```
[0] [1]
['age', 'spend']
```

1. The wrong call hands `spend=2.0` to the position where `age` was trained. The prediction flips to class 0 (`[0] [1]`) with no warning, while the correctly ordered row gives class 1.
2. Fixes: select columns by a stored list (Recipe B), or fit on a DataFrame so scikit-learn records `feature_names_in_` and warns when names do not match at predict time.

COMMON MISTAKE

7. `type=bool` in argparse. `bool("False")` is `True`, because any non-empty string is true.

```
import argparse

p = argparse.ArgumentParser()
p.add_argument("--shuffle", type=bool, default=True)        # wrong
print(p.parse_args(["--shuffle", "False"]).shuffle)

q = argparse.ArgumentParser()
q.add_argument("--shuffle", action=argparse.BooleanOptionalAction, default=True)
print(q.parse_args(["--no-shuffle"]).shuffle, q.parse_args([]).shuffle)
```

```
True
False True
```

1. The wrong version prints `True` although the user asked for False. The symptom is "my flag is ignored".
2. `BooleanOptionalAction` creates `--shuffle` and `--no-shuffle`; the output is `False True`.

COMMON MISTAKE

8. Comparing floats with equality in tests. The test passes on one machine and fails on another, or fails for the sum `0.1 + 0.2`.

```
import math
import numpy as np

print(0.1 + 0.2 == 0.3, 0.1 + 0.2)
print(math.isclose(0.1 + 0.2, 0.3, rel_tol=1e-9))
print(np.allclose([0.1 + 0.2], [0.3]))
```

```
False 0.30000000000000004
True
True
```

1. Binary floating point cannot store 0.1 exactly, so the sum is `0.30000000000000004` and the equality is `False`.
2. In tests use a tolerance: `pytest.approx`, `math.isclose` or `np.allclose` (`True` both). Choose a tolerance that matches the precision you really need.

COMMON MISTAKE

9. Tests that read the clock or random state. A test calling `datetime.now()` works today and fails after midnight; unseeded randomness fails once in a hundred runs.

```
from datetime import datetime, timedelta

def is_stale(updated, now=None):
    now = now or datetime.now()              # real code uses the clock; tests inject a fixed one
    return now - updated > timedelta(days=7)

fixed_now = datetime(2026, 11, 9, 12, 0)
print(is_stale(datetime(2026, 11, 1), now=fixed_now))
print(is_stale(datetime(2026, 11, 5), now=fixed_now))
```

```
True
False
```

1. The function takes `now` as an optional argument. Production passes nothing and gets the real time; tests pass a fixed value.
2. Nov 1 to Nov 9 12:00 is 8.5 days, so `True`; Nov 5 is 4.5 days, so `False`. The result no longer depends on the day you run the test. The same idea (inject the dependency) applies to random generators: pass in a seeded `numpy.random.Generator`.

COMMON MISTAKE

10. Stale cache from hidden inputs. `joblib.Memory` hashes the arguments and the function source. A global variable or a file the function reads is invisible to it.

```
import tempfile
from joblib import Memory

SETTINGS = {"rate": 2}

def scaled(x):                                  # wrong: depends on a global
    return x * SETTINGS["rate"]

def scaled_ok(x, rate):                         # right: every input is an argument
    return x * rate

with tempfile.TemporaryDirectory() as tmp:
    mem = Memory(tmp, verbose=0)
    a, b = mem.cache(scaled), mem.cache(scaled_ok)
    print(a(10))
    SETTINGS["rate"] = 3
    print(a(10))                                # symptom: still 20
    print(b(10, 2), b(10, 3))
```

```
20
20
20 30
```

1. After the global changes to 3, `a(10)` should be 30, but the cache sees identical arguments and source, and returns the stored `20`.
2. Passing `rate` as an argument makes it part of the key, so the cached function correctly gives `20 30`.

COMMON MISTAKE

11. No seed on the split. `train_test_split` without `random_state` gives a different split on every run, so your metric moves and you cannot tell a real improvement from noise.

```
import numpy as np
from sklearn.model_selection import train_test_split

X = np.arange(20).reshape(10, 2)
y = np.arange(10)
a = train_test_split(y, test_size=0.3)[1]
b = train_test_split(y, test_size=0.3)[1]
c = train_test_split(y, test_size=0.3, random_state=42)[1]
d = train_test_split(y, test_size=0.3, random_state=42)[1]
print(np.array_equal(c, d))
print(len(a), len(b))
```

```
True
3 3
```

1. With `random_state=42` both calls return the same test rows: `True`.
2. Without a seed, `a` and `b` have the same size (3) but usually different members; we print only the size to keep this output stable. Seed every source of randomness you control (splitters, models, `numpy.random.default_rng(seed)`).

COMMON MISTAKE

12. Blocking the event loop in `async def`. Not run here (FastAPI is not installed). Inside `async def`, a slow synchronous call such as `model.predict` or `requests.get` freezes every other request while it runs. The symptom: latency grows with concurrency even though CPU is idle on other cores.

```
# NOT RUN HERE
@app.post("/predict")
async def predict(c: Customer):           # wrong: async, but the body is blocking CPU work
    return model.predict_proba(to_row(c))

@app.post("/predict")
def predict(c: Customer):                  # right: plain def runs in a worker thread
    return model.predict_proba(to_row(c))
```

1. The first handler runs on the single event-loop thread and holds it for the whole prediction.
2. Use plain `def` for blocking work. Use `async def` only when every slow call inside is awaited (an async database or HTTP client).

COMMON MISTAKE

13. A Dockerfile that copies everything first. Not run here. `COPY . .` before `pip install` means every code edit invalidates the install layer. The symptom is a three-minute rebuild after changing one line, and an image that contains `.git`, `.env` and data.

```
# wrong
FROM python:3.12-slim
COPY . /app
RUN pip install -r /app/requirements.txt

# right
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY src/ ./src/              # plus a .dockerignore listing .git, .env, data/
```

1. In the wrong version the single `COPY . /app` layer changes whenever any file changes, so the install runs every time.
2. In the right version the dependency layer depends only on `requirements.txt`, and the secrets and data never enter because of `.dockerignore` and the narrow `COPY`. Also pin the base image tag and package versions so a rebuild next month gives the same environment.

COMMON MISTAKE

14. Tests that depend on each other. A module-level list that one test appends to makes a second test pass or fail depending on order.

```
import io
import unittest

SHARED = []

class Bad(unittest.TestCase):
    def test_a_adds(self):
        SHARED.append(1)
        self.assertEqual(len(SHARED), 1)

    def test_b_adds(self):
        SHARED.append(2)
        self.assertEqual(len(SHARED), 1)       # passes only if it runs first

class Good(unittest.TestCase):
    def setUp(self):
        self.items = []                         # fresh state for every test

    def test_a_adds(self):
        self.items.append(1)
        self.assertEqual(len(self.items), 1)

    def test_b_adds(self):
        self.items.append(2)
        self.assertEqual(len(self.items), 1)

for case in (Bad, Good):
    suite = unittest.defaultTestLoader.loadTestsFromTestCase(case)
    res = unittest.TextTestRunner(stream=io.StringIO()).run(suite)
    print(case.__name__, "failures:", len(res.failures))
```

```
Bad failures: 1
Good failures: 0
```

1. Tests run alphabetically: `test_a_adds` makes the list length 1 and passes; then `test_b_adds` appends again, the length is 2, and it fails (`Bad failures: 1`). Run `test_b_adds` alone and it would pass, the classic "passes alone, fails in the suite".
2. Creating state in `setUp` (or a function-scoped fixture) gives every test a fresh start: `Good failures: 0`.

23.10 Cross-Library Integration

The tools in this chapter are designed to snap together. The table shows the seams where most real work happens.

From	To	How they connect
argparse / Typer	dataclass	Parse flags, then build one frozen `Config`; pass the config, not the namespace, to the code
YAML file	dataclass	`Config(**yaml.safe_load(text))`; DVC's `params.yaml` is the same file
pandas	pytest	Small DataFrames in fixtures; compare with `pandas.testing.assert_frame_equal`
scikit-learn	joblib	Save a fitted `Pipeline` with `joblib.dump`; speed up search with `n_jobs`
joblib bundle	FastAPI	Load once in `lifespan`; validate the request with a pydantic model
logging	tests	`caplog` (pytest) or `assertLogs` (unittest) check that important events are logged
logging	MLflow / JSON metrics	Logs are for humans and time ordering; metrics and parameters go to the tracker
CLI	Airflow / Prefect / cron	Make the job a command with exit codes; the scheduler only runs the command

From	To	How they connect
pyproject.toml	everything	One file holds dependencies, the script entry point, pytest and mypy settings
environment variables	Docker, schedulers	Secrets and paths enter from outside; the code reads `os.environ` once into a settings object
Dockerfile	requirements / uv.lock	The image installs exactly the locked versions
DVC	git	Git holds code and small pointer files; DVC holds the big data; both are tied to the same commit

Here is the common shape: a batch scoring job that uses a dataclass for configuration, pathlib for paths, logging for the story, joblib for the model, pandas for the data, an atomic write for the result and an exit code for the scheduler. The model is trained in the first lines only so that the block is standalone.

```
import io
import logging
import tempfile
from dataclasses import dataclass
from pathlib import Path

import joblib
import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

stream = io.StringIO()
logging.basicConfig(stream=stream, level=logging.INFO, format="%(levelname)s %(message)s", force=True)
log = logging.getLogger("score_job")

@dataclass(frozen=True)
class JobConfig:
    model_path: Path
    input_csv: Path
    output_csv: Path
    threshold: float = 0.5

def run(cfg):
    model = joblib.load(cfg.model_path)
    df = pd.read_csv(cfg.input_csv)
    log.info("read %d rows from %s", len(df), cfg.input_csv.name)
    if df.isna().any().any():
        log.error("input has missing values")
        return 1
    df["score"] = model.predict_proba(df[["age", "spend"]].to_numpy())[:, 1].round(3)
    df["flag"] = (df["score"] >= cfg.threshold).astype(int)
    tmp = cfg.output_csv.with_suffix(".tmp")
    df.to_csv(tmp, index=False)
    tmp.replace(cfg.output_csv)
    log.info("wrote %d rows, %d flagged", len(df), df["flag"].sum())
    return 0

with tempfile.TemporaryDirectory() as d:
    d = Path(d)
    rng = np.random.default_rng(0)
    Xt = np.column_stack([rng.uniform(18, 80, 300), rng.uniform(5, 100, 300)])
    yt = (Xt[:, 1] > 60).astype(int)
    joblib.dump(make_pipeline(StandardScaler(), LogisticRegression()).fit(Xt, yt), d / "m.joblib")

    pd.DataFrame({"age": [30, 45, 52, 61], "spend": [10.0, 90.0, 70.0, 20.0]}).to_csv(d / "in.csv", index=False)
    cfg = JobConfig(d / "m.joblib", d / "in.csv", d / "out.csv")
    print("exit", run(cfg))
    print(pd.read_csv(d / "out.csv")[["age", "spend", "flag"]].to_string(index=False))
    print(sorted(p.name for p in d.iterdir()))

    pd.DataFrame({"age": [30, None], "spend": [10.0, 90.0]}).to_csv(d / "bad.csv", index=False)
    print("exit", run(JobConfig(d / "m.joblib", d / "bad.csv", d / "out2.csv")))
print(stream.getvalue(), end="")
```

```
exit 0
 age  spend  flag
  30   10.0     0
  45   90.0     1
  52   70.0     1
  61   20.0     0
['in.csv', 'm.joblib', 'out.csv']
exit 1
INFO read 4 rows from in.csv
INFO wrote 4 rows, 2 flagged
INFO read 2 rows from bad.csv
ERROR input has missing values
```

1. The model is a `Pipeline` (scaler plus classifier) saved as one object, avoiding pitfall 5.
2. `JobConfig` collects all paths and the threshold in one frozen object; `run` never reads global state.
3. Logging goes into a `StringIO` so we can print it at the end; a real job would log to a file or stdout. `force=True` replaces any earlier logging setup.
4. The job validates input before scoring, logs an error and returns exit code 1 instead of crashing with a vague pandas error. The scheduler sees the non-zero code and can alert or retry.
5. Output is written to a `.tmp` name and moved with `replace` (atomic). The directory listing shows no leftover temporary file.
6. The printed results: exit 0, the four scored rows with flags (only high-spend customers are flagged, matching how we generated labels), the directory listing, exit 1 for the bad file, and the log lines in the order they were produced.

23.11 How to Solve Any Engineering and MLOps Problem: A Playbook

The decision flow

1. **Name the surprise.** Wrong result, crash, slow, different on another machine, or "I cannot reproduce it"? Each has a different first move.
2. **Reproduce it small.** Make a failing test or a ten-line script. If you cannot reproduce it, you cannot know you fixed it.
3. **Find the layer.** Use the toolkit figure: code (Write), behaviour (Test), install (Package), interface (Serve), runtime (Ship), history (Track).
4. **Change one thing, observe, repeat.** Bisect: halve the possibilities (comment out half the pipeline, `git bisect`, try an older version).
5. **Fix, then lock it in** with a regression test, a log line or a validation check so the same surprise cannot return unseen.

Symptom	First move	Likely tools
Works here, fails there	Print versions and paths on both	`sys.executable`, `importlib.metadata`, `pip list`, Docker
Passes alone, fails in suite (or randomly)	Shuffle order, look for shared state, time, randomness	fixtures, seeds, injected clock
Wrong or empty output, no error	Log row counts at each stage; add assertions	logging, validation function
Slow	Profile, then fix the biggest cost	`cProfile`, `timeit`, joblib, caching
Fails only in production	Capture the failing input and replay it in a test	logs, TestClient, regression test
Cannot tell which run produced which model	Record params, data hash, versions with every run	JSON lines, MLflow, DVC
Secrets or settings scattered	One settings object read from the environment	dataclass, env vars
Hard for others to run	Make install and test two commands	pyproject.toml, uv, README

Pattern 1: "It works on my machine"

The cause is almost always a different interpreter, a different version, or an unpinned dependency. Compare what is installed against what the project requires.

```
import importlib.metadata as md

REQUIRED = {"numpy": (2, 0), "pandas": (3, 0), "scikit-learn": (1, 5), "not-a-real-package": (1, 0)}

def parse(version):
    return tuple(int(p) for p in version.split(".")[:2])

for dist, minimum in REQUIRED.items():
    try:
        have = md.version(dist)
    except md.PackageNotFoundError:
        print(f"{dist}: MISSING")
        continue
    status = "ok" if parse(have) >= minimum else f"TOO OLD (need {minimum[0]}.{minimum[1]})"
    print(f"{dist}: {have} {status}")
```

```
numpy: 2.5.3 ok
pandas: 3.0.6 ok
scikit-learn: 1.9.1 ok
not-a-real-package: MISSING
```

1. `REQUIRED` lists minimum versions as tuples, which compare correctly (`(1, 10) > (1, 9)` , unlike the strings `"1.10"` and `"1.9"`).
2. `md.version` reads the installed version or raises `PackageNotFoundError` , which we report as MISSING instead of crashing.
3. The output shows each real library as ok, plus the invented package as MISSING. Run such a check at the start of a job and print the result into the log; it is the first thing to ask a colleague for.

Pattern 2: the test that fails only sometimes

Expose order dependence on purpose by running the tests in several shuffled orders. The pytest plugin `pytest-randomly` does this automatically; here is the idea in plain Python.

```
import io
import random
import unittest

STATE = {"count": 0}

class T(unittest.TestCase):
    def test_one(self):
        STATE["count"] += 1
        self.assertEqual(STATE["count"], 1)         # hidden dependence on order

    def test_two(self):
        STATE["count"] += 1
        self.assertGreaterEqual(STATE["count"], 1)

tests = [T("test_one"), T("test_two")]
outcomes = []
for seed in range(4):
    STATE["count"] = 0
    order = tests[:]
    random.Random(seed).shuffle(order)
    res = unittest.TextTestRunner(stream=io.StringIO()).run(unittest.TestSuite(order))
    outcomes.append((tuple(t._testMethodName for t in order), len(res.failures)))
for order, failures in outcomes:
    print(order, "failures:", failures)
print("order dependent:", len({f for _, f in outcomes}) > 1)
```

```
('test_one', 'test_two') failures: 0
('test_two', 'test_one') failures: 1
('test_two', 'test_one') failures: 1
('test_two', 'test_one') failures: 1
order dependent: True
```

1. Both tests touch the shared dictionary; `test_one` only passes if it runs first.
2. We run the same two tests in four seeded orders (the seeds make the experiment repeatable) and record the order and number of failures.
3. Whenever `test_two` runs first, `test_one` fails. The last line shows whether the failure count changed with the order, which is proof of shared state. The fix is a fresh state per test (pitfall 14).

Pattern 3: the job "succeeds" but the output is wrong

Silent wrongness (a join that drops rows, a filter that removes everything) is the most dangerous failure. Add cheap checks between stages that fail loudly.

```
import pandas as pd

def check_stage(name, df, min_rows, key=None):
    if len(df) < min_rows:
        raise ValueError(f"{name}: only {len(df)} rows, expected at least {min_rows}")
    if key is not None and df[key].duplicated().any():
        raise ValueError(f"{name}: duplicate values in {key}")
    print(f"{name}: {len(df)} rows ok")

orders = pd.DataFrame({"order_id": [1, 2, 3, 4], "customer": ["a", "b", "z", "a"]})
customers = pd.DataFrame({"customer": ["a", "b"], "segment": ["gold", "basic"]})

check_stage("orders", orders, 4, key="order_id")
joined = orders.merge(customers, on="customer", how="inner")       # silently drops customer "z"
try:
    check_stage("joined", joined, min_rows=len(orders))
except ValueError as exc:
    print("caught:", exc)
```

```
orders: 4 rows ok
caught: joined: only 3 rows, expected at least 4
```

1. `check_stage` encodes two expectations: a minimum row count and a unique key. Raising an exception stops the pipeline at the stage that broke.
2. The inner join keeps only customers present in both tables. Order 3 belongs to customer `z`, who is not in the customers table, so one row disappears without any error from pandas.
3. The orders check passes (`orders: 4 rows ok`); the joined check catches the loss: `joined: only 3 rows, expected at least 4`. In a real job also log the counts at every stage so you can see where rows vanish.

Pattern 4: it is slow

Profile before optimising. `cProfile` counts every function call and the time inside it. We print call counts (stable) rather than times (which vary).

```
import cProfile
import pstats

def slow_unique(items):
    out = []
    for x in items:
        if x not in out:                # list membership is linear: O(n) each time
            out.append(x)
    return out

def fast_unique(items):
    return list(dict.fromkeys(items))     # hash lookup: O(1) each time, keeps order

def pipeline():
    data = [i % 500 for i in range(5000)]
    return slow_unique(data), fast_unique(data)

prof = cProfile.Profile()
prof.runcall(pipeline)
stats = pstats.Stats(prof)
rows = {k[2]: v for k, v in stats.stats.items() if k[2] in ("slow_unique", "fast_unique", "pipeline")}
print({name: rows[name][1] for name in sorted(rows)})
print(slow_unique([3, 1, 3, 2]) == fast_unique([3, 1, 3, 2]))
```

```
{'fast_unique': 1, 'pipeline': 1, 'slow_unique': 1}
True
```

1. Two functions give the same result; the first does a linear scan of a list for every item, so its cost grows with the square of the input size.
2. `runcall` profiles a single call; `pstats.Stats` holds the numbers. In day-to-day use call `stats.sort_stats("cumulative").print_stats(10)` to see which functions take the most time, and read the top rows.
3. We print the number of calls for our three functions (each was called once, so every value is 1). The check on the last line confirms both give identical results, which is the proof you demand before swapping one for the other.
4. The fix pattern: replace repeated linear searches with a set or dict, use vectorised pandas or numpy operations instead of Python loops, cache repeated work, and only then reach for parallelism.

Pattern 5: failure in production, replay it as a test

The cheapest way to fix a production bug permanently is to capture the exact input, write a failing test, then fix.

```
import io
import unittest

def parse_amount(text):
    text = text.replace("$", "").replace(",", "").strip()
    if text.startswith("(") and text.endswith(")"):        # the fix: accounting negatives "(12.50)"
        return -float(text[1:-1])
    return float(text)

class TestRegression(unittest.TestCase):
    def test_accounting_negative_from_incident_2291(self):
        self.assertEqual(parse_amount("($12.50)"), -12.5)

    def test_old_behaviour_still_works(self):
        self.assertEqual(parse_amount("$1,200.00"), 1200.0)

res = unittest.TextTestRunner(stream=io.StringIO()).run(unittest.defaultTestLoader.loadTestsFromTestCase(TestRegression))
print(res.testsRun, res.wasSuccessful())
```

```
2 True
```

1. Imagine production crashed with `ValueError` on the input `"($12.50)"` found in the log. Step one is the test named after the incident: it fails before the fix.
2. The fix handles brackets as negative amounts. The second test pins the old behaviour so the fix cannot break it.
3. Both pass: `2 True`. The incident number in the test name is a cheap way to leave a trail for the next engineer.

Pattern 6: which run produced this model?

Use Recipe C: every run writes parameters, a data fingerprint, library versions and metrics. To answer "which run?" later, read `runs.jsonl` with `pd.read_json(path, lines=True)` and filter. If the answer is "I do not know", the fix is to start logging today; history cannot be reconstructed afterwards.

Pattern 7: settings and secrets scattered

Use Example 17: one frozen settings dataclass read once from `os.environ`, with secrets hidden in `repr`, validated at start-up. Pass the object down; never read `os.environ` deep inside functions, because it makes tests and reasoning hard.

Pattern 8: hard for others to run

Checklist (verify each from a clean clone in a new folder): the README has two commands (install, test); `pyproject.toml` lists every dependency; tests pass with no network and no private files; the CLI prints `--help`; a `.gitignore` keeps data, models and secrets out; the Dockerfile builds. If any step needs a message to a colleague, document or automate it.

How to verify any fix

- A test that failed before and passes now (regression test).
- The same command run twice gives the same result (idempotent, deterministic).
- Run in a fresh environment (new venv or container), not only in your tired one.
- Compare before and after on the same inputs: row counts, metrics, a data hash.

How to debug

- **Read the traceback from the bottom.** The last line is the error; the nearest frame in your code is where to look.
- `breakpoint()` drops into the debugger at that line (`n` next, `s` step in, `p x` print, `c` continue); with pytest use `pytest --pdb` to stop at the first failure.
- Print the *type, shape and first rows* of the objects at the boundary between two steps: `type(x)`, `df.shape`, `df.head(3)`.
- Turn on DEBUG logging for one module: `logging.getLogger("mypkg.io").setLevel("DEBUG")`.
- Run `python -X dev` to enable extra runtime warnings, and `python -W error` to turn warnings into exceptions while you hunt a deprecation.
- Minimise: delete code until the bug disappears; the last thing you deleted is related.

23.12 Going Deeper: Internals, Memory, Speed and What Interviewers Probe

This part is for when you want to know why things behave as they do. You can skip it on a first read and return later.

How pytest finds fixtures: a 15-line model

pytest looks at the parameter names of a test function and builds each requested fixture, recursively building fixtures that the fixture itself requests, and caching them for the scope. The model below reproduces that idea with `inspect.signature`.

```
import inspect

FIXTURES, builds = {}, []

def fixture(fn):
    FIXTURES[fn.__name__] = fn
    return fn

@fixture
def base():
    builds.append("base")
    return 10

@fixture
def doubled(base):
    builds.append("doubled")
    return base * 2

def resolve(name, cache):
    if name not in cache:
        fn = FIXTURES[name]
        cache[name] = fn(*[resolve(p, cache) for p in inspect.signature(fn).parameters])
    return cache[name]

def run_test(test):
    cache = {}                                  # one cache per test = function scope
    test(*[resolve(p, cache) for p in inspect.signature(test).parameters])

def test_values(doubled, base):
    assert doubled == 2 * base

run_test(test_values)
print("passed; built:", builds)
```

```
passed; built: ['base', 'doubled']
```

1. `fixture` registers a function under its name. The decorator in real pytest also records scope and parameters.
2. `resolve` builds a fixture by first building every fixture named in *its* signature. `doubled` needs `base`, so `base` is built first.
3. The cache makes sure `base` is built once even though both the test and `doubled` ask for it. A wider scope means a longer-lived cache.
4. Output: `passed; built: ['base', 'doubled']`. This is why fixture names must match argument names exactly, and why a typo gives "fixture not found" instead of a silent default.

Two more pytest internals worth knowing: pytest rewrites the `assert` statements of test modules when it imports them (an import hook edits the syntax tree), which is how a bare `assert a == b` can print both values; and it collects tests by name patterns (`test_*.py`, functions named `test_*`, classes named `Test*` without an `__init__`).

Logging internals: lazy work and the hierarchy

```
import logging

class Loud:
    calls = 0
    def __str__(self):
        Loud.calls += 1
        return "value"

log = logging.getLogger("lazy")
log.setLevel(logging.INFO)
log.addHandler(logging.NullHandler())
log.propagate = False

log.debug("a %s", Loud())           # DEBUG is below INFO: the record is never formatted
print(Loud.calls)
log.debug(f"a {Loud()}")            # the f-string is evaluated BEFORE the call
print(Loud.calls)
log.info("a %s", Loud())            # emitted, so the argument is formatted once
print(Loud.calls)
print(logging.getLogger("a.b").parent.name, logging.getLogger("a.b.c").parent.name)
```

```
0
1
1
root a.b
```

1. `Loud.__str__` counts how many times Python converts the object to text, which is the work we want to avoid.
2. With %-style arguments a disabled call formats nothing: the counter stays at 0. With an f-string the text is built before `log.debug` is even called, so the counter becomes 1 for a message nobody sees. The object still has to be created in both cases, only the formatting is deferred.
3. An emitted message is formatted once, so the counter reaches 2. Handlers do not format again.
4. The last line shows the hierarchy: the parent of `a.b` is the root logger (name `root`) until `a` exists, and once `a.b` exists the parent of `a.b.c` is `a.b`. Dotted names are how configuration for `"pipeline"` reaches `"pipeline.clean"`.

Memory: dataclasses with slots

```
import tracemalloc
from dataclasses import dataclass

@dataclass
class Plain:
    x: int
    y: int

@dataclass(slots=True)
class Slotted:
    x: int
    y: int

def traced_bytes(cls, n=20000):
    tracemalloc.start()
    objs = [cls(1, 2) for _ in range(n)]
    current, _ = tracemalloc.get_traced_memory()
    tracemalloc.stop()
    return current, len(objs)

plain, _ = traced_bytes(Plain)
slotted, _ = traced_bytes(Slotted)
print(slotted < plain)
print(hasattr(Plain(1, 2), "__dict__"), hasattr(Slotted(1, 2), "__dict__"))
```

```
True
True False
```

1. A normal instance stores its attributes in a per-object dictionary (`__dict__`); `slots=True` stores them in fixed slots, with no dictionary.
2. `tracemalloc` counts bytes allocated by Python code. We create 20,000 objects of each kind and compare. We print a boolean because the byte counts depend on the Python build.
3. Slotted instances use less memory (`True`), and have no `__dict__` : the second line is `True False` . The trade-off: you cannot add attributes at will, and some tools that expect `__dict__` behave differently. It matters when you hold millions of small records, not for a config object.

Speed and concurrency, in plain terms

- **The GIL.** In the standard CPython build, only one thread runs Python bytecode at a time. Threads help when work waits (network, disk) or when a C library releases the lock (much of numpy and scikit-learn). For pure-Python CPU work use processes. Python 3.13 and later include an optional free-threaded build, but the default build you will meet in production still has the GIL.
- **joblib backends.** The default `loky` backend uses separate worker processes, so arguments and results are pickled and copied. Large numpy arrays are memory-mapped instead of copied above a size threshold, which is why passing a big array to workers is cheaper than it sounds. The `threading` backend avoids copying but only helps when the task releases the GIL.
- **FastAPI threads.** A plain `def` endpoint runs in a thread pool of limited size (the library sets a default; check your version). If each request does heavy CPU work in pure Python, add worker processes (`uvicorn --workers N` or a process manager), not more threads. Each worker process loads its own copy of the model, so memory use multiplies.
- **pydantic v2** performs validation in a compiled core written in Rust, which is much faster than hand-written Python checks; the validation cost is small compared with model inference.
- **Docker layers.** Each instruction creates a read-only filesystem layer, identified by a content hash. The build cache reuses a layer if its instruction and inputs are unchanged. Multi-stage builds compile or download in one stage and copy only the result into a small final stage, which shrinks images and attack surface.
- **Determinism limits.** Floating-point addition is not associative, so summing in a different order (as multithreaded libraries may do) can change the last digits. Seeds fix random choices, not these effects. Compare metrics with tolerances, and fix thread counts if you need bit-for-bit repeatability.

- **Pickle safety.** Unpickling executes code. Only load joblib or pickle files you created. For exchange between teams prefer formats such as ONNX, JSON for parameters, or skops for scikit-learn models, and always verify checksums.

INTERVIEW

What interviewers probe in this area. (1) "How do you make an ML project reproducible?" Seeds, pinned and locked dependencies, data versioning, parameter and metric logging, container. (2) "How do you test a model?" Unit-test data code, test shapes and ranges of outputs, invariants (monotonicity, no NaN), a small golden dataset with a metric floor; do not assert exact accuracy on random data. (3) "Training-serving skew: what is it and how do you prevent it?" Same preprocessing code in both places, one saved Pipeline, schema validation, shadow traffic. (4) "Fixture scope trade-offs?" Speed versus isolation. (5) "What is idempotency and why do schedulers need it?" Re-running a task must be safe, because retries happen. (6) "async def versus def in FastAPI?" Blocking work must not run on the event loop. (7) "Why not `pip freeze` as your only dependency management?" It mixes direct and indirect dependencies and encodes your platform; use a lock file from a resolver. (8) "Where do secrets live?" Outside the image and the repository, in the runtime environment or a secrets manager, never in logs.

Chapter recap

- Tests: arrange, act, assert. pytest uses plain `assert`, fixtures (by argument name, scoped), `parametrize`, markers, `tmp_path`, `monkeypatch`. unittest and `unittest.mock` in the standard library express the same ideas.
- Mock where the name is *used*; inject dependencies (clock, client, random generator) so code is easy to test.
- `logging`: loggers produce records, handlers decide where, formatters decide how; two gates (logger level, handler level); configure once at start-up; pass arguments lazily.
- CLIs: `main(argv)` returns an exit code; argparse for small tools, Typer or Click for larger ones; `BooleanOptionalAction` for flags, never `type=bool`.
- `pathlib.Path` for every path; anchor to `__file__` or an option, never to the current directory; write to a temporary name then `replace`.
- Dataclasses (frozen, kw_only, `default_factory`, `__post_init__`) hold configuration; type hints document and are checked by mypy, not enforced at run time.
- Environments: venv or uv per project, `pyproject.toml` as the single description, a lock file for applications, pre-commit for automatic checks.
- Secrets come from the environment, are hidden in `repr`, and never enter git, logs or image layers; leaked means rotate.
- Serving: validate at the edge, load the model once, plain `def` for blocking work, save preprocessing and model together as a Pipeline plus a metadata file.
- joblib: `dump/load` for models (trusted sources only), `Memory` for pure slow functions, `Parallel` only for heavy tasks.
- Reproducibility means recording seeds, data fingerprint, parameters, library versions and metrics for every run; MLflow and DVC scale that idea up. Docker ships the environment; Airflow and Prefect schedule idempotent tasks as a DAG.

Formula / pattern	Meaning
`def test_x(fixture_a): assert f(a) == expected`	pytest test; fixtures injected by name
`@pytest.mark.parametrize("a,b", [...])`	One test, many input rows
`with patch.object(mod, "name", return_value=v):`	Replace what the code under test looks up
`log = logging.getLogger(__name__)`	Module logger; configure handlers once elsewhere
`def main(argv=None) -> int: ...; sys.exit(main())`	Testable CLI with an exit code
`HERE = Path(__file__).resolve().parent`	Paths independent of the working directory
`tmp.write_text(x); tmp.replace(final)`	Atomic file write
`@dataclass(frozen=True, kw_only=True)`	Immutable, named-argument configuration
`Config(**yaml.safe_load(text))`	YAML to typed config; unknown keys fail

Formula / pattern	Meaning
`python -m venv .venv` / `uv sync`	Recreatable environment
`Customer(BaseModel)` with `Field(ge=..)`	Request validation, status 422 on failure
`Memory(dir).cache(f)`	Disk cache keyed on arguments and source
`Parallel(n_jobs=k)(delayed(f)(x) for x in xs)`	Parallel map; results in input order
`record = {params, data_hash, metrics, env}`	Minimum run record for reproducibility
`TopologicalSorter(deps)`	Order tasks by dependencies (a DAG)

Practice set

Twenty-four problems, easiest first. Try each before reading the answer. Code in the problems was run to produce the answers.

Problem 23.13.1 Name the five pytest fixture scopes from narrowest to widest, and say which is the default.

Answer: `function`, `class`, `module`, `package`, `session`. The default is `function`: the fixture is rebuilt for every test, which gives the best isolation at the highest setup cost.

Problem 23.13.2 For `p = PurePosixPath("/data/a.tar.gz")`, what are `p.name`, `p.stem`, `p.suffix` and `p.suffixes`?

Answer: `name` is `a.tar.gz`; `stem` is `a.tar` (only the last suffix is removed); `suffix` is `.gz`; `suffixes` is `['.tar', '.gz']`.

Problem 23.13.3 The root logger is configured with `level=logging.WARNING`. Which of `debug`, `info`, `warning`, `error` appear?

Answer: Only `warning` and `error`. A message is shown when its level number (WARNING 30, ERROR 40) is at least the threshold (30); DEBUG 10 and INFO 20 are dropped.

Problem 23.13.4 What does this print?

```
import argparse

p = argparse.ArgumentParser()
p.add_argument("--n", type=int, default="5")
ns = p.parse_args([])
print(ns.n + 1, type(ns.n).__name__)
```

Answer: `6 int`. A default given as a *string* is passed through `type` by argparse, so `"5"` becomes the integer 5. A non-string default (such as `5`) is used as is.

Problem 23.13.5 Which line in `pyproject.toml` creates a terminal command `mytool` that runs the function `main` in `mytool/cli.py`? Which table does it go in?

Answer: In the `[project.scripts]` table: `mytool = "mytool.cli:main"`. After `pip install -e .` (or `uv sync`) the command exists in the environment's `bin` folder. The part before the colon is the module path; the part after is the callable.

Problem 23.13.6 What does this print?

```
from dataclasses import dataclass

@dataclass(frozen=True)
class P:
    x: int

a, b = P(1), P(1)
print(a == b, a is b, hash(a) == hash(b))
```

Answer: `True False True`. The generated `__eq__` compares field values, so the two objects are equal; they are two different objects, so `is` is False; a frozen dataclass with `eq=True` gets a hash computed from its fields, so equal objects have equal hashes and can be dictionary keys.

Problem 23.13.7 What does this print, and what property of `Parallel` does it show?

```
from joblib import Parallel, delayed

print(Parallel(n_jobs=2)(delayed(pow)(i, 2) for i in range(5)))
```

Answer: `[0, 1, 4, 9, 16]`. Results are returned in the order the tasks were submitted, regardless of which worker finished first.

Problem 23.13.8 Find the bug: the programmer wants DEBUG messages on stdout, but nothing appears there. Why, and how do you fix it?

```
import logging
import sys

logging.warning("starting")
logging.basicConfig(level=logging.DEBUG, stream=sys.stdout, format="%(message)s")
logging.debug("hello")
```

Answer: Nothing is printed to stdout. The first call, `logging.warning`, silently configures the root logger with a default stderr handler because none existed. `basicConfig` does nothing when the root logger already has handlers, so the later call is ignored (the message "starting" went to stderr). Fixes: call `basicConfig` before any logging call, or pass `force=True` to replace existing handlers.

Problem 23.13.9 What happens when this class is defined? Give the fix.

```
from dataclasses import dataclass

try:
    @dataclass
    class Run:
        name: str
        tags: list = []
except ValueError as exc:
    print(exc)
```

Answer: Defining the class raises `ValueError`, and the code prints its message: `mutable default <class 'list'> for field tags is not allowed: use default_factory`. Dataclasses reject `list`, `dict` and `set` defaults so that all instances do not share one list. Fix: `tags: list[str] = field(default_factory=list)`.

Problem 23.13.10 Write a pytest test (not run here) that checks `is_even` returns True for 0, 2, -4 and False for 1, -3, using one parametrized function.

Answer:

```
import pytest
from numbers_utils import is_even

@pytest.mark.parametrize("n, expected", [(0, True), (2, True), (-4, True), (1, False), (-3, False)])
def test_is_even(n, expected):
    assert is_even(n) is expected
```

One function, five reported test cases. Negative numbers and zero are included because they are classic edge cases for the modulo operator.

Problem 23.13.11 What does this print? (a trap with the `/` operator)

```
from pathlib import PurePosixPath

print(PurePosixPath("a") / "b" / "../c")
print(PurePosixPath("/data") / "/etc/passwd")
print(PurePosixPath("a/b/c.txt").parent.parent)
```

Answer: `a/b/../c` (pure paths do not simplify `..`; use `resolve()` on a real `Path` to do so), then `/etc/passwd` (joining an absolute path *replaces* everything before it, a security trap when the second part comes from a user), then `a`.

Problem 23.13.12 Find the problem in this FastAPI endpoint (not run here) and fix it.

```
@app.post("/predict")
def predict(c: Customer):
    model = joblib.load("model.joblib")          # inside the handler
    return {"p": float(model.predict_proba([[c.age, c.spend]])[0, 1])}
```

Answer: The model file is read from disk on every request, so latency is dominated by disk and unpickling, and concurrent requests multiply memory use. Load it once at start-up (in the `lifespan` function, stored in a module-level dict or `app.state`) and use it inside the handler. The plain `def` is correct because the prediction is blocking CPU work.

Problem 23.13.13 Name four problems in this Dockerfile and fix them.

```
FROM python:latest
COPY . /app
ENV API_KEY=sk-12345
RUN pip install -r /app/requirements.txt
CMD python /app/serve.py
```

Answer: (1) `latest` is a moving target, so rebuilds differ; pin a tag such as `python:3.12-slim`. (2) `COPY .` before the install invalidates the dependency layer on every edit and copies `.git`, `.env` and data into the image; copy `requirements.txt`, install, then copy only the code, and add a `.dockerignore`. (3) The secret in `ENV` is stored permanently in the image layers and visible with `docker history`; pass it at run time with `--env-file` or the platform's secret store. (4) Shell-form `CMD` runs under a shell, so signals such as stop do not reach Python cleanly; use exec form `CMD ["python", "serve.py"]`. Also: run as a non-root user.

Problem 23.13.14 What does this print, and why is the answer a trap for beginners?

```
from joblib import Parallel, delayed

seen = []

def record(i):
    seen.append(i)
    return i

out = Parallel(n_jobs=2)(delayed(record)(i) for i in range(4))
print(out, len(seen))
```

Answer: `[0, 1, 2, 3] 0`. With the default process-based backend each worker has its own copy of `seen`; appends happen there and are lost. The parent list stays empty. Workers communicate only through return values (or shared files or databases), so always *return* what you need.

Problem 23.13.15 Your CLI should read from standard input when the file argument is `-`. Write the part of `main` that opens the input and test it without a real terminal.

Answer:

```
import io
import sys
from contextlib import contextmanager
from pathlib import Path

@contextmanager
def open_input(arg):
    if arg == "-":
        yield sys.stdin
    else:
        with Path(arg).open() as f:
            yield f

sys.stdin = io.StringIO("a,b\n1,2\n")          # a test replaces stdin by a string buffer
with open_input("-") as f:
    print(f.read().splitlines())
```

Printed: `['a,b', '1,2']`. The context manager hides the difference between stdin and a file for the rest of the code. In pytest you would use `monkeypatch.setattr("sys.stdin", io.StringIO(...))`. `pandas.read_csv` also accepts the file object directly.

Problem 23.13.16 Dockerfile A copies the whole project and then runs `pip install`; Dockerfile B copies only `requirements.txt`, installs, then copies the code. You edit one line of `app.py` and rebuild. Which layers are rebuilt in each?

Answer: A: the `COPY .` layer changed, so it and every layer after it are rebuilt, including the slow `pip install`. B: the base image, requirements copy and install layers are unchanged and reused from cache; only the final `COPY` of the code (and anything after it) is rebuilt, which takes seconds.

Problem 23.13.17 Write a context manager `timed(name)` that logs `"name took X s"` at INFO level when the block ends, even if the block raises.

Answer:

```
import io
import logging
import time
from contextlib import contextmanager

buf = io.StringIO()
logging.basicConfig(stream=buf, level=logging.INFO, format="%(message)s", force=True)
log = logging.getLogger("timer")

@contextmanager
def timed(name):
    start = time.perf_counter()
    try:
        yield
    finally:
        log.info("%s took %.3f s", name, time.perf_counter() - start)

with timed("sleep"):
    time.sleep(0.05)
try:
    with timed("boom"):
        raise RuntimeError("fail")
except RuntimeError:
    pass
lines = buf.getvalue().splitlines()
print([line.split(" took")[0] for line in lines])
print(float(lines[0].split()[2]) >= 0.05)
```

Printed: `['sleep', 'boom']` then `True`. The `try/finally` around `yield` guarantees the log line even when the block raises, and the exception still propagates to the caller. `perf_counter` is the right clock for durations because it never goes backwards. We print the names and one check rather than the times, which vary.

Problem 23.13.18 Configuration precedence: command line beats environment variable beats file beats default. Write `merge(*layers)` that implements this and test it.

Answer:

```
def merge(*layers):
    """Later layers win; None means 'not set' and is ignored."""
    out = {}
    for layer in layers:
        out.update({k: v for k, v in layer.items() if v is not None})
    return out

defaults = {"C": 1.0, "seed": 0, "out": "runs.jsonl"}
file_cfg = {"C": 0.5}
env_cfg = {"seed": 7, "C": None}
cli_cfg = {"out": "tmp.jsonl", "seed": None}
print(merge(defaults, file_cfg, env_cfg, cli_cfg))
```

Printed: `{'C': 0.5, 'seed': 7, 'out': 'tmp.jsonl'}`. Layers are applied from weakest to strongest. Treating `None` as "not provided" matters because argparse gives `None` for options the user did not pass; without that filter an unset flag would erase a value from a weaker layer. Note that `0` and `False` are valid values and must not be filtered (the test is `is not None`, not truthiness).

Problem 23.13.19 A colleague saved a model with joblib yesterday. Today `joblib.load` fails or gives odd predictions on another machine. List five causes and one preventive habit.

Answer: (1) Different scikit-learn version (the class layout changed). (2) Different numpy or Python version. (3) The model refers to your own module (a custom transformer class) which is not importable on the other machine. (4) The file was saved on a different operating system or architecture with an incompatible dependency. (5) The file was truncated or corrupted during copying (check a checksum). Habit: save a metadata file with versions and feature names beside the model (Recipe B), check them on load, pin versions in the container, and keep custom code in an installed package.

Problem 23.13.20 Find the reproducibility bugs in this training script (list at least four).

```
import numpy as np
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split

np.random.seed(0)
X, y = load_data("latest.csv")
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.2)
model = RandomForestClassifier(n_estimators=200).fit(Xtr, ytr)
print(model.score(Xte, yte))
```

Answer: (1) `train_test_split` has no `random_state`; the global `np.random.seed` does not control it in the way people expect. (2) `RandomForestClassifier` has no `random_state`. (3) `"latest.csv"` is a moving target and its content is not fingerprinted, so you cannot tell later which data was used. (4) The result is only printed; parameters, metric, library versions and a code version are not recorded anywhere. (5) No stratification for a classification split (`stratify=y`) so class balance in the test set varies. Fix: a `Config` with a seed passed to every component, a data hash, a JSON-lines record per run (Recipe C).

Problem 23.13.21 A scheduler may run your task twice. Make `save_results(rows, day)` idempotent: running it twice for the same day must not duplicate rows.

Answer:

```
import json
import tempfile
from pathlib import Path

def save_results(rows, day, folder):
    target = Path(folder) / f"results_{day}.json"          # the name is derived from the day
    tmp = target.with_suffix(".tmp")
    tmp.write_text(json.dumps(rows))
    tmp.replace(target)                                    # overwrite, never append
    return target

with tempfile.TemporaryDirectory() as d:
    save_results([1, 2, 3], "2026-11-09", d)
    save_results([1, 2, 3], "2026-11-09", d)               # a retry
    print(sorted(p.name for p in Path(d).iterdir()))
    print(json.loads((Path(d) / "results_2026-11-09.json").read_text()))
```

Printed: `['results_2026-11-09.json']` and `[1, 2, 3]`. The key idea: the output location depends only on the logical input (the day), and each run *replaces* that output. With a database use "delete the day's rows and insert" in one transaction, or an upsert on a unique key. Appending to a shared file would double the data on a retry.

Problem 23.13.22 What does this print? (pandas 3 copy-on-write meets test fixtures)

```
import pandas as pd

fixture = pd.DataFrame({"a": [1, 2]})
alias = fixture
alias["b"] = 3
print(list(fixture.columns))

subset = fixture[["a"]]
subset.loc[0, "a"] = 99
print(fixture.loc[0, "a"], subset.loc[0, "a"])
```

Answer: `['a', 'b']` then `1 99`. `alias = fixture` only gives the same object a second name, so adding a column through `alias` changes it (copy-on-write protects derived objects, not aliases). `fixture[["a"]]` creates a derived object, and under pandas 3 copy-on-write modifying it never

changes the original. In a test suite this means: return a fresh frame from a function-scoped fixture, or call `.copy()` when a test needs to mutate a shared one.

Problem 23.13.23 Interview: you must add tests to a model service that has none. What do you test, in what order, and what do you refuse to test?

Answer: Order by value and cost. (1) Pure functions of the data code (cleaning, feature building): small hand-computed examples, edge cases (empty input, missing values, unseen category). (2) The validation layer: a valid request, each rule violated once, a missing field (status 422). (3) The prediction path with a tiny model trained on synthetic data in a fixture: output shape, range 0 to 1, deterministic with a seed, same result for the same row in a batch of one and a batch of many. (4) A saved-model round trip (dump, load, equal predictions). (5) One smoke test of the whole API with `TestClient`. (6) A separate, slower suite with a real golden dataset and a metric floor (for example AUC above a threshold). Refuse to test: the exact accuracy of a random model, third-party library internals, and anything that needs the network or production data in the fast suite.

Problem 23.13.24 Interview: explain, with one sentence each, `-k`, `-m` and `-x` in pytest, and write a command that runs only fast tests whose names contain `csv` and stops at the first failure.

Answer: `-k` selects tests by an expression matched against test names; `-m` selects by marker; `-x` stops after the first failure. Command: `pytest -m "not slow" -k csv -x` (not run here, pytest is not installed). Add `--lf` to rerun only the tests that failed last time, and `-vv` for full diffs of failed comparisons.

Prompt pack

AI prompt: copy and paste into any AI

```
You are my patient, practical engineering mentor. I am a working professional moving into data roles. Teach me the Python engineering toolkit around a model: pytest (fixtures, parametrize, markers, tmp_path, monkeypatch, mocking), logging, argparse and Typer, pathlib, dataclasses, type hints and mypy, venv/uv/pyproject.toml, FastAPI, joblib, MLflow/DVC, Docker, Airflow/Prefect, pre-commit and secrets handling. Work in this order: explain one idea in plain words with a small analogy, show a minimal runnable example, then ask me ONE question and wait for my answer before continuing. After each answer tell me exactly what was right, what was missing, and give a harder follow-up. After 10 questions give me a score out of 10, my weakest 3 topics and a 3-day practice plan. Use only the standard library plus numpy, pandas and scikit-learn in examples and label anything that needs another package as "not run".
```

AI prompt: copy and paste into any AI

```
You are a strict but kind senior engineer doing a code review. I will paste a Python data project (a training script, a CLI, a small API or a Dockerfile). Review it in this order: (1) correctness bugs, (2) reproducibility (seeds, data fingerprint, pinned versions, logged parameters and metrics), (3) tests: what is missing and write the three most valuable pytest tests, (4) configuration, secrets and logging, (5) project layout and packaging (pyproject.toml, src layout, entry points), (6) performance and deployment (model loaded once, blocking calls, Docker layer order). For each problem give the line, why it matters, the wrong code, and the fixed code. End with the single change that would improve the project most and a checklist I can reuse. Do not rewrite the whole project unless I ask.
```

AI prompt: copy and paste into any AI

```
Act as an interviewer for a machine learning engineer role. Ask me one question at a time about the engineering side of ML: testing a model service, training-serving skew, reproducibility, experiment tracking, packaging, Docker, schedulers and idempotency, secrets, FastAPI versus batch jobs, and debugging a pipeline that "succeeds" with wrong output. Start easy and get harder when I answer well. After each answer, score it from 1 to 5, point out what a strong answer would add (a concrete example, a trade-off, a failure mode), and then ask a follow-up that probes deeper. After 8 questions, summarise my strengths, my gaps, and give me three mini projects to build, each with a clear definition of done.
```

APPENDIX

CHEAT SHEET

One page per idea: the patterns from every chapter, in the order you learned them.

Run each pattern in the REPL once. A cheat sheet only helps if you have already built the thing at least one time.

Chapter 1: Setup and thinking

Need	Pattern
Run a script / one line	`python3 file.py` / `python3 -c "print(1)"`
Types	`int float complex bool str bytes None`
Float compare	`math.isclose(a, b)` (never `==`)
Text to number	`int("480")`, `float("2.5")`
Text to bytes	`s.encode("utf-8")`, `b.decode("utf-8")`
Same value / same object	`a == b` / `a is b` (use `is` only for `None`)
Copy	`x.copy()` (shallow), `copy.deepcopy(x)` (deep)
Unpack	`a, b = b, a`, `first, *rest = items`

Chapter 2: Control flow, functions, scope, recursion

Need	Pattern
Branch	`if / elif / else`, ternary `A if cond else B`
Loops	`for i, x in enumerate(xs, start=1)`, `zip(a, b)`, `while`, `break`, `continue`, loop `else`
Shape match	`match v: case [a, b]: ... case _: ...`
Safe default	`def f(x, bucket=None): if bucket is None: bucket = []`
Flexible args	`*args` (tuple), `**kwargs` (dict), `/` positional-only, `*` keyword-only
Scope order	Local, Enclosing, Global, Built-in; `nonlocal`, `global`
Fix an argument	`functools.partial(f, exp=2)`
Recursion	base case + smaller problem; `@lru_cache` for repeats

Chapter 3: Text and regex

Need	Pattern
Slice	`s[2:5]`, `s[-3:]`, `s[::-1]`

Need	Pattern
Clean	`s.strip().lower()` , `" ".join(s.split())`
Format	`f"{x:,.2f}"` , `f"{p:.1%}"` , `f"{n:05d}"` , `f"{x=}"`
Regex find	`re.search` , `re.findall` , `re.fullmatch` , `re.sub` , `re.split`
Regex pieces	`\d \w \s . [a-z] + * ? {n} ( ) (?P<name>...) ^ $ \b`
Lazy / greedy	`.+?` / `.+`
Always	raw strings `r"..."` , `re.compile(..., re.X)`

Chapter 4: Collections and toolbox

Need	Pattern
Sort	`sorted(x, key=..., reverse=True)` (new list), `x.sort()` (in place, returns `None`)
Unique + fast `in`	`set(x)` , `x in s`
Dict read / merge	`d.get(k, 0)` , `a \
Group / count	`defaultdict(list)` , `Counter(x).most_common(n)`
Window / queue	`deque(maxlen=n)`
Top-N	`heapq.nlargest(n, x)`
Comprehension	`[f(x) for x in data if test(x)]` , `{k: v ...}` , generator `(...)`
Statistics	`statistics.mean` , `median` , `quantiles(x, n=20)[18]`

Chapter 5: Files, data formats, time

Need	Pattern
Path	`Path("a") / "b.csv"` , `p.read_text(encoding="utf-8")` , `root.rglob("*.log")`
Big file	`with open(p, encoding="utf-8") as f: for line in f:`
JSON	`json.dumps(obj)` , `json.loads(text)`
CSV	`csv.DictReader(f)` , then convert with `float(...)`
Time zones	`datetime(..., tzinfo=ZoneInfo("Asia/Kolkata"))` , `.astimezone(ZoneInfo(...))`
Config	`os.environ.get("NAME", "default")`

Chapter 6: Errors, logging, debugging

Need	Pattern
Handle	`try: ... except ValueError as e: ... else: ... finally: ...`

Need	Pattern
Raise / chain	`raise AppError("...") from e`
Log	`logging.basicConfig(level=logging.INFO)`, `log.exception("what failed")`
Debug	`breakpoint()`, then `n s c p w q`

Chapter 7: Iterators, generators, decorators, context managers

Need	Pattern
Generator	`def g(): yield x`; `yield from other`
First N	`list(itertools.islice(gen, N))`
Decorator	`def d(f): @wraps(f) def w(*a, **k): return f(*a, **k); return w`
Context manager	`@contextmanager def cm(): setup; try: yield; finally: cleanup`

Chapter 8: OOP

Need	Pattern
Record	`@dataclass(frozen=True, order=True)`, `field(default_factory=list)`
Computed attribute	`@property`; alternative constructor `@classmethod`
Interface	`ABC` + `@abstractmethod`, or `Protocol`
Inheritance order	`Class.__mro__`
SOLID	Single responsibility, Open/closed, Liskov, Interface segregation, Dependency inversion

Chapter 9: Modules, environments, types

Need	Pattern
Entry guard	`if __name__ == "__main__":`
Virtual env	`python3 -m venv .venv`, `source .venv/bin/activate`
Save / restore	`pip freeze > requirements.txt`, `pip install -r requirements.txt`
Type hints	`list[int]`, `int \`

Chapter 10: Testing and quality

Need	Pattern
Run tests	`python3 -m unittest discover -v`, `pytest -x -q`

Need	Pattern
Many cases	`@pytest.mark.parametrize`
Replace a name	`mock.patch("module_where_used.name")`
Quality tools	`ruff format .`, `ruff check .`, `mypy src`, `pre-commit install`

Chapter 11: Concurrency and performance

Need	Pattern
Measure	`timeit`, `python3 -m cProfile -s cumulative script.py`, `tracemalloc`
Waiting (I/O)	threads, `asyncio`
Calculating (CPU)	processes (`ProcessPoolExecutor`)
Little's law	`L = λ × W` (concurrency = throughput x time in system)

Chapter 12: APIs, web, CLI

Need	Pattern
Call an API	`client.get(url, timeout=5)`, `raise_for_status()`
Retry	wait `base * 2**attempt`, only for temporary errors
CLI	`argparse`, `main(argv=None)`, exit code 0 or non-zero
Run a program	`subprocess.run([...], capture_output=True, text=True, check=True)`

Chapter 13: Professional practice

Need	Pattern
Safe SQL	`execute("... WHERE x = ?", (value,))`, never an f-string
Passwords	`hashlib.scrypt` with a random salt; compare with `hmac.compare_digest`
Secrets	in env vars, `.env` in `.gitignore`, commit `.env.example`, rotate leaked keys
CI order	lint, types, unit tests, integration tests, build, deploy
Package	`pyproject.toml`, `python -m build`

Chapter 14: Capstones and interviews

Need	Pattern
Percentile (nearest rank)	`xs[max(0, ceil(n*p/100) - 1)]` after sorting

Need	Pattern
Two-sum	dict `seen[value] = index`
Shortest path (unweighted)	BFS with `deque` and a `seen` set
Explain-X answer	define, consequence, fix, judgement

Glossary of Key Terms

Quick definitions of terms used across the chapters, with the chapter where each is taught.

Term	Meaning	Chapter
Mutable	An object that can change in place (list, dict, set). Immutable ones (int, str, tuple) cannot.	1
Reference	A name pointing at an object. Assignment copies the pointer, not the object.	1
Scope (LEGB)	Where a name is looked up: Local, Enclosing, Global, Built-in.	2
Closure	A function that remembers variables from the function that created it.	2
Regular expression	A pattern language for matching text.	3
Hashable	Has a value that never changes and a hash, so it can be a dict key or set member.	4
Comprehension	A one-expression way to build a list, dict or set from an iterable.	4
Serialisation	Turning objects into bytes or text (JSON, CSV) and back.	5
Exception	An object signalling an error that unwinds the call stack until something catches it.	6
Iterator	An object that produces one value per `next()` call until it raises `StopIteration`.	7
Generator	A function with `yield` that produces values lazily.	7
Decorator	A function that takes a function and returns a wrapped one.	7
Context manager	An object used with `with` that sets up and cleans up (files, locks, transactions).	7
MRO	Method resolution order: the sequence of classes searched for an attribute.	8
Virtual environment	An isolated folder of installed packages for one project.	9
Type hint	An annotation describing expected types, checked by tools like mypy, not at run time.	9
Fixture	Reusable test setup supplied to tests by name.	10
GIL	The global interpreter lock: only one thread runs Python bytecode at a time in the standard build.	11
Coroutine	A function defined with `async def` that can pause at `await` and let others run.	11
Idempotent	Doing it twice has the same effect as doing it once (PUT, DELETE).	12
Transaction	A group of database changes that all happen or none do.	12
N+1 queries	One query for a list plus one more per item; fix with a join.	12
Correlation id	An identifier attached to every log line of one request.	13

Term	Meaning	Chapter
SQL injection	Attacker text altering a query because it was pasted into the SQL string; use parameters.	13
Test-driven development	Writing a failing test first, then the code that passes it.	15

An Honest Closing Note

If you worked through these chapters with your hands on the keyboard, you can now read and write real Python: parse a log, build a clean data pipeline in plain Python, design classes, write tests, protect a database query from injection, call and build a small API, and explain generators, decorators, the GIL and MRO in your own words. That is the core a junior data or ML engineer needs from Python.

This document does not make you an expert on its own. It does not cover the data libraries (NumPy, pandas, scikit-learn), Docker, cloud services, or most of the deep topics in the missing-topics list. Several tools (pytest, FastAPI, pydantic, mypy, ruff, httpx) were shown as standard usage that was not run in the environment where this text was prepared, so run them yourself and compare.

Practise next: finish every chapter's practice set again without notes, build the four capstones in Chapter 14 and put them on GitHub with a clear README, and solve one coding problem a day. Then move to the pandas, SQL and statistics documents, where Python becomes a daily tool. Be patient with yourself. Clear understanding, built slowly, is what employers pay for.